Traffic Alert & Collision Avoidance System (TCAS)
Program Office

Algorithm Design Description of the Airborne Collision Avoidance System sXu

ACAS_ADS_21_001_V4R0, V4R0

August 31, 2021

Prepared By: _____ Date: 08/31/2021
Jared Wikle, sXu STM Lead

Prepared By: Luis Alvarez Date: 08/31/2021
Luis Alvarez, sXu TRM Lead

Approved By: NEAL A SUCHY Digitally signed by NEAL A SUCHY Date: 2021.09.01 10:08:09 -04'00' Date: _____
Neal Suchy, Program Manager, AJM-42
FAA TCAS Program Office

SXU REVISION HISTORY

Vers.	Rev.	Sec.	Description of Change	STM / TRM / CM	Release Date
0	0	N/A	sXu Run0 Initial Release: Ownship correlation algorithms and interface for receiving ground-based surveillance reports.	R.Moss	1-June-2018
1	0	N/A	sXu Run1 Release: Broadcast Remote ID interface for receiving V2V surveillance reports and scaled alerting logic.	I.Jessen/L.Alvarez/R.Moss	31-March-2019
2	0	N/A	sXu Run2 Release: Point Obstacle interface, active surveillance/coordination for research, and changed radio altitude to height AGL.	J.Wikle/L.Alvarez/K.Breeden	10-December-2019
3	0	N/A	sXu Run3 Release: Removal of active surveillance and ADS-R surveillance, change active coordinate to V2V coordination (UF16UDS30 to V2VCoordination, transponder to ownship-capability-data, capability-report to V2V-capability-report), interface name changes (Broadcast Remote ID to V2V, XuATAR to ORNCT, Ground-Surveillance to AGT, airspeed to true-airspeed), track LLA based tracks in absolute space instead of relative space, added externally validated flags for ADS-B and V2V tracks, include pressure or HAE altitudes in STM report, automatic switching to wind-relative horizontal logic, addition of ground-point-obstacle-avoidance based on ownship-AGL, removal of LARPD online cost as it was not appropriate for sUAS operations, robust TRM alerting for degraded ownship altitude and velocity.	J.Wikle/L.Alvarez/K.Breeden	30-Novemeber-2020
3	1	N/A	sXu Run3 Revision 1 Release for Review Only: Updated STM vertical and horizontal tracker parameters, vertical trackers use new altimetry error assumptions, horizontal trackers more closely reflect ADS-B/V2V measurement error characteristics, introduction of separate scale factors for rho and drho scaling of vertical entry tables, bug fix for tau calculation of GPOA to use vertical tau, overall scale factor tuning, removal of low altitude climb incentive that counteracted LAPRD cost that is no longer used, check added for offline global action being contained as an optimal action of at least one individual belief action.	J.Wikle/L.Alvarez/K.Breeden	13-May-2021
4	0	N/A	sXu V4R0 Release: Populate STM reports for display only even if TRM feasiblility checks fail, Update aircraft address variable names, Proper handling of multiple ID spaces, Output all received IDs for a track in STM report, Implement new coordination flowchart, Overhaul correlation based on new ID spaces, Properly maintain correlation histories, Proper handling of invalid intruder altitudes, Updated Track Source Selection logic and added selection lock, Classification added to relavant interfaces, Changed automatic operating mode logic to manual user inputs, Restore check for proper ADS-B velocity track initialization, Use real time HAE accuracy and integrity in altitude selection logic and covariance inflation, Undo ADS-B horizontal tracker changes made in V3R1, Integration of intruder specific cost values for online costs applicable to multiple intruder types, Re-tuning of sensitivity tables, vertical cost table, HTRM maintain online costs, HTRM weakening cost, VTRM parameters changed for unequipped MTLO, sandwhich prevention, force alert, prevent early coc (airborne and point obstacle). Removed several unused parameters; moved inhibit altitudes from parameter file to receive interface.	J.Wikle/L.Alvarez/K.Breeden	31-August-2021

Federal Aviation Administration
800 Independence Avenue, SW
Washington, DC 20591

ACKNOWLEDGMENTS

Many people have contributed to this document by providing content or participating in analysis, review, and editing. Thanks goes out to those individuals from MIT Lincoln Laboratory, JHU Applied Physics Laboratory, the FAA Technical Center, Aurora Sciences, and Regulus Group that have provided input. Special recognition is given to the following individuals that have provided significant contributions.

Last Name	First Name	Organization
Alvarez	Luis	MIT Lincoln Laboratory
Asmar	Dylan	MIT Lincoln Laboratory
Bender	Walter	JHU Applied Physics Laboratory
Bezanson	Jeffrey	Julia Computing Inc.
Billingsley	Thomas	MIT Lincoln Laboratory
Breeden	Kara	MIT Lincoln Laboratory
Brush	Jeff	JHU Applied Physics Laboratory
Bryan	Austin	MIT Lincoln Laboratory
Capuder	Lawrence	MIT Lincoln Laboratory
Chryssanthacopoulos	James	MIT Lincoln Laboratory
Chu	Arthur	MIT Lincoln Laboratory
Cowen	Emilie	MIT Lincoln Laboratory
Das	Anshuman	MIT Lincoln Laboratory
Deal	Forest	JHU Applied Physics Laboratory
Dean	Christopher	MIT Lincoln Laboratory
Drumm	Ann	MIT Lincoln Laboratory
Edwards	Christopher	MIT Lincoln Laboratory
Elder	Tomas	MIT Lincoln Laboratory
Evans	Jenifer	MIT Lincoln Laboratory
Eyler	Alexander	MIT Lincoln Laboratory
Gardner	Ryan	JHU Applied Physics Laboratory
Genin	Daniel	JHU Applied Physics Laboratory
Gjersvik	Adam	MIT Lincoln Laboratory
Greaves	Jesse	MIT Lincoln Laboratory
Guendel	Randal	MIT Lincoln Laboratory
Havener	Robin	MIT Lincoln Laboratory
Jessen	Ian	MIT Lincoln Laboratory
Johnson	David	MIT Lincoln Laboratory
Kerley	Cedric	MIT Lincoln Laboratory
Klaus	Robert	MIT Lincoln Laboratory
Kobzik-Juul	Barbara	JHU Applied Physics Laboratory
Kochenderfer	Mykel	MIT Lincoln Laboratory
Leahy	Kevin	MIT Lincoln Laboratory
Leeper	Charles	JHU Applied Physics Laboratory

Last Name	First Name	Organization
Lempka	Matthew	JHU Applied Physics Laboratory
Londner	Edward	MIT Lincoln Laboratory
Lopez	Jessica	JHU Applied Physics Laboratory
Lopez-Zayas	Carlos	JHU Applied Physics Laboratory
Lorenzo	Ed	JHU Applied Physics Laboratory
Mackay	Justin	MIT Lincoln Laboratory
Madali	Christian	MIT Lincoln Laboratory
Martin	Sean	JHU Applied Physics Laboratory
McLain	Cynthia	MIT Lincoln Laboratory
Minor	Christian	MIT Lincoln Laboratory
Monath	Nicholas	MIT Lincoln Laboratory
Moss	Robert	MIT Lincoln Laboratory
Owen	Michael	MIT Lincoln Laboratory
Pagano	Tom	FAA Technical Center
Panken	Adam	MIT Lincoln Laboratory
Perez	Christian	MIT Lincoln Laboratory
Pierce	Mark	MIT Lincoln Laboratory
Puntin	Brendon	MIT Lincoln Laboratory
Schmidt	Aurora	JHU Applied Physics Laboratory
Selby	Christina	JHU Applied Physics Laboratory
Serres	Christine	MIT Lincoln Laboratory
Shao	Peijun	MIT Lincoln Laboratory
Silbermann	Joshua	JHU Applied Physics Laboratory
Thober	Mark	JHU Applied Physics Laboratory
Treleaven	Kyle	MIT Lincoln Laboratory
Walters	Ethan	Aurora Sciences
Wikle	Jared	MIT Lincoln Laboratory
Williams	Richard	MIT Lincoln Laboratory
Wood	Paul	JHU Applied Physics Laboratory
Wu	Katherine	JHU Applied Physics Laboratory
Wu	Samuel	MIT Lincoln Laboratory
Young	Tyler	JHU Applied Physics Laboratory
Zanlongo	Sebastian	JHU Applied Physics Laboratory
Zintak	Benjamin	JHU Applied Physics Laboratory

This document is a product of MIT Lincoln Laboratory, prepared on behalf of the United States, Federal Aviation Administration, Air Traffic Systems, Collision Avoidance Program Office (AJM-42).

EXECUTIVE SUMMARY

ACAS X is an airborne collision avoidance system designed to reduce the risk of mid-air collision while minimizing unnecessary alerts. The system described in this document is proposed as an initial scaled DAA alerting capability specifically designed for smaller unmanned aircraft. Like the ACAS Xa system for manned aircraft and the ACAS Xu for unmanned aircraft, the ACAS sXu design consists of two main modules: (1) the Surveillance and Tracking Module (STM), which detects aircraft in the vicinity and tracks their position, and (2) the Threat Resolution Module (TRM), which identifies threats and provides resolution guidance. The ACAS sXu TRM provides horizontal resolution guidance in addition to vertical resolution guidance, as provided by the ACAS Xa TRM. This document provides the Algorithm Design Description (ADD) for both the STM and TRM.

This document builds upon capabilities described in the ACAS Xa and Xu ADD. It supports additional surveillance and scaled alerting capabilities specifically designed for smaller unmanned aircraft operations. This design is a result of system performance evaluations from both model-based fast-time simulation, flight tests conducted in 2019, and technology demonstrations from government and industry partners such as UTM Pilot Program (UPP) phase 1 and 2, NASA TCL4, and state DOT funded corridor operations.

The algorithm descriptions in this document are prototypes of the final system capability and will change as ACAS sXu development continues.

TABLE OF CONTENTS

Page

1. Introduction 1
   - 1.1 Purpose 1
   - 1.2 Scope 1
   - 1.3 Background 1
   - 1.4 Context 2
   - 1.5 Prescription 2
   - 1.6 Specification Language: Julia 3
     - 1.6.1 Data Types 3
     - 1.6.2 Built-In Values and Functions 4
   - 1.7 Notation 5
     - 1.7.1 Variables 5
     - 1.7.2 Physical Quantities 6
     - 1.7.3 Statistical Quantities 7
     - 1.7.4 Functions 8
     - 1.7.5 Diagrams 8

2. Surveillance and Tracking Module Description 9
   - 2.1 System Overview 9
   - 2.2 Passive Surveillance 10
     - 2.2.1 Input Processing 11
       - 2.2.1.1 Automatic Dependent Surveillance Broadcast (ADS-B) 11
         - 2.2.1.1.1 State Vector Position Report 11
         - 2.2.1.1.2 State Vector Velocity Report 21
         - 2.2.1.1.3 Universal Access Transceiver Report 25
         - 2.2.1.1.4 Mode Status Report 28
         - 2.2.1.1.5 Externally Validated ADSB Report 29
       - 2.2.1.2 Vehicle To Vehicle (V2V) 30
         - 2.2.1.2.1 V2V Report 30
         - 2.2.1.2.2 Externally Validated V2V Report 36
     - 2.2.2 Intruder Horizontal Estimation 36
     - 2.2.3 Intruder Vertical Estimation 50
   - 2.3 Ownship Relative Non Cooperative Track Surveillance 56
     - 2.3.1 Input Processing 57
     - 2.3.2 Estimation 64

2.4 Absolute Geodetic Track Surveillance 65
2.4.1 Input Processing 65
2.4.2 Estimation 74
2.5 Point Obstacle Surveillance 77
2.5.1 Input Processing 77
2.6 Ownship Tracking 79
2.6.1 Input Processing 79
2.6.2 Ownship Discretes 79
2.6.3 Pressure Altitude Estimation 80
2.6.4 Height Above Ground Level Estimation 85
2.6.5 True Airspeed Estimation 85
2.6.6 Heading Estimation 86
2.6.7 WGS84 State Estimation 90
2.6.7.1 HAE Altitude Estimation 95
2.7 Coordination Processing 96
2.7.1 Input Processing 96
2.7.1.1 V2V Coordination 96
2.7.1.2 V2V Operational Status Message 103
2.7.1.3 ADS-B Operational Status Message 105
2.8 Track Management 105
2.8.1 Mode Management 106
2.8.2 Ownship V2V Operational Status Message 110
2.8.3 Surveillance Source Correlation 110
2.8.4 STM Report Generation 111
2.8.4.1 ADS-B Track File 133
2.8.4.2 V2V Track File 148
2.8.4.3 ORNC Track File 151
2.8.4.4 AGT Track File 153
2.8.4.5 Point Obstacle Track File 156
2.8.5 Track Deletion 157
2.8.6 Validation 161

3. Threat Resolution Module Description 163
3.1 sXu TRM Overview 163
3.1.1 Inputs and Outputs to the sXu TRM 164
3.1.2 sXu TRM System Architecture 165
3.1.3 Intruder Designation and Protection Modes 165
3.2 Horizontal Threat Resolution Module 166
3.2.1 Horizontal TRM Overview 166

3.2.2 Horizontal Logic State Estimation 174
3.2.3 Offline Horizontal Cost Estimation 182
3.2.4 Online Horizontal Cost Estimation 193
3.2.4.1 State Dependent Online Costs 193
3.2.4.2 State Independent Online Costs 198
3.2.5 Horizontal Action Selection 209
3.2.6 Horizontal Coordination Selection 220
3.2.7 Horizontal Track Threat Assessment 228
3.2.7.1 Determine Horizontal Code 229
3.2.7.2 Determine Horizontal Score 229
3.2.8 Horizontal Display Logic 230
3.2.8.1 Determine Target Track Angle and Turn Rate 231
3.2.8.2 Determine Label 271 Value 232
3.2.8.3 Determine Horizontal Desensitivity Mode 233
3.2.9 Generate Horizontal TRM Output 233
3.3 Vertical Threat Resolution Module 238
3.3.1 Vertical TRM Overview 238
3.3.2 Vertical Logic State and Cost Estimation 248
3.3.2.1 Intruder Prep 252
3.3.2.2 Vertical Logic State Estimation 254
3.3.2.2.1 Tau Estimation 261
3.3.2.2.2 Ground Point Obstacle Tau Estimation 266
3.3.2.2.3 Point Obstacle Tau Estimation 268
3.3.2.2.4 Combine Vertical Beliefs 269
3.3.2.2.5 Set Belief-Based Online Cost 270
3.3.2.3 Offline Vertical Cost Estimation 275
3.3.2.4 Online Vertical Cost Estimation Independent of Coordination 282
3.3.2.4.1 Advisory Restart Cost 287
3.3.2.4.2 Altitude Dependent Clear of Conflict Cost 289
3.3.2.4.3 Altitude Inhibit Cost 289
3.3.2.4.4 Bad Transition Cost 290
3.3.2.4.5 Critical Interval Protection Cost 293
3.3.2.4.6 Initialization Cost 298
3.3.2.4.7 Prevent Early COC Cost 299
3.3.2.4.8 Safe Crossing Resolution Advisory Deferral Cost 302
3.3.2.4.9 Vertical Rate Limit Dependent Strengthen Cost 304
3.3.2.4.10 Point Obstacle Descend Cost 304
3.3.2.4.11 Point Obstacle Early COC Cost 305
3.3.2.5 Update Intruder Inputs Mid-Cycle 306

3.3.2.6 Online Vertical Cost Estimation Dependent on Coordination 307
3.3.2.6.1 Response Estimation 312
3.3.2.6.2 Compatibility Cost 316
3.3.2.6.3 Coordinated Resolution Advisory Deferral Cost 320
3.3.2.6.4 Coordination Delay Cost 322
3.3.2.6.5 Crossing No Alert Cost 323
3.3.2.6.6 Max Reversal Cost 325
3.3.2.6.7 Prevent Early Weakening Cost 327
3.3.2.6.8 Restrict Clear of Conflict Due To Reversal 328
3.3.2.6.9 SA01 Heuristic 329
3.3.2.6.10 Time-Based Non-Compliance Cost 333
3.3.2.7 Individual Cost Estimation 337
3.3.3 Vertical Action Selection 340
3.3.3.1 Single Intruder Rate Selection 342
3.3.3.2 Multiple Intruder Rate Selection 343
3.3.3.2.1 Sandwich Prevention Cost for Unequipped Intruders 347
3.3.3.2.2 Cost Fusion for Unequipped Intruders 348
3.3.3.2.3 Multi Threat Level Off (MTLO) Determination 349
3.3.3.2.4 Balancing Intruder Costs 353
3.3.3.2.5 Action Arbitration 355
3.3.4 Vertical Coordination Selection 359
3.3.5 Vertical Display Logic 364
3.3.5.1 Determine Crossing 366
3.3.5.2 Determine Display Data 367
3.3.6 Generate Vertical TRM Output 373
3.3.6.1 RA Mode Processing 376
3.3.6.2 Surveillance Only Mode Processing 381
3.3.6.3 Standby Mode Processing 383
3.3.6.4 Resolution Advisory Output Messages 383
3.3.6.4.1 Vertical RA Report 386
3.3.6.4.2 Broadcast Interrogation Message 390
3.3.6.5 Intruder Management 391
3.4 Generate Blended sXu TRM Output 395
3.5 Standard States and Conversions 409

A. Appendix STM Housekeeping 437

B. Appendix Correlation Processing Implementation 439
B.1 Track Extrapolation 439
B.2 Ownship Track Decorrelation 449
B.3 Ownship Track Correlation 452
B.4 Track Decorrelation 455
B.5 Track Correlation 460

C. Appendix Constant Variables 473
C.1 Geometric Constants 473
C.2 Global Constants 473

D. Appendix Data Structure Definitions 480
D.1 STM Output Data Structures 480
D.2 STM Internal Data Structures 483
D.3 STM Output-TRM Input Data Structures 496
D.4 TRM Output-STM Input Data Structures 501
D.5 TRM Internal Data Structures 509
D.6 TRM Context-Specific State Data Structures 525

E. Appendix Parameter File Specification 533
E.1 ACAS X Parameter Data Item File (PDIF) Checksums 533
E.2 STM Parameters 535
E.3 TRM Parameters 545

F. Appendix Data Table Format Specification 573
F.1 Block Dictionary Table Format 573
F.2 Horizontal Policy Table Format 574

G. Appendix Offline-Table Scaling and Online Cost Index Selection 576
G.1 Horizontal Offline-Table Scaling 576
G.2 Vertical Offline-Table Scaling 578

H. Appendix Math Utilities 580

I. Appendix Data Management Utilities 586

J. Appendix Julia Standard Library Functions 591

K. Appendix Design Notes 595
K.1 Overview 595
K.2 Interfacing with ACAS X 595
K.2.1 General 595
K.2.2 Input Message Entry Points 595
K.2.3 Report Generation 597
K.2.3.1 Module Memory Management 597
K.3 Specific Implementation Details 597
K.3.1 Julia Pseudocode 597
K.3.1.1 Julia Math and Array Notation 598
K.3.1.2 Automatic Data Type Conversions 599
K.3.1.3 Handling Symbols 600
K.3.2 Single versus Multi-Threaded Execution 600
K.3.3 32-Bit versus 64-Bit Applications 600
K.3.4 Floating Point Precision 600
K.3.5 Subsystem Modularity 601
K.3.6 Running Julia as a Development Tool 601
K.3.7 Static Allocation 602
K.3.8 Target IDs 604
K.4 Error Checking 604
K.4.1 Divide-by-Zero Errors 604
K.4.2 Invalid Input 606
K.4.3 Matrix Inversion 607

L. Appendix Provisions for Potential Modifications 608
L.1 Descent Inhibit Thresholds 608
L.2 Target Designation 608

References 609

LIST OF ALGORITHMS

Algorithm No.		Page
1	ReceiveStateVectorPositionReport	12
2	ConvertWGS84ToECEF	13
3	ConvertECEFToWGS84	14
4	ConvertOrthometricToGeodeticHeight	15
5	ConvertGeodeticToOrthometricHeight	15
6	GetEarthRadiusAtLatitude	15
7	GetNormalGravityAtLatitude	16
8	AssociateADSBReportToTarget	16
9	InitializePassiveTrackFile	17
10	AddADSBTrackToDB	18
11	AssociateICAOtoTarget	18
12	TrackExists	19
13	AddToDB	19
14	RetrieveWithID	19
15	UpdateAltInHorGeoPos	20
16	GetAltForHorTrkObservation	20
17	ReceiveStateVectorVelocityReport	22
18	PropagateGeodeticToToa	23
19	RotateECEFToENU	23
20	RotateENUToECEF	24
21	ENUToECEFRotation	24
22	ECEFToENURotation	24
23	RefineECEFAndLLA	25
24	PolarAlignmentHorizontalRotation	25
25	ReceiveStateVectorUatReport	27
26	ReceiveModeStatusReport	29
27	UpdatePassiveQualityHistory	29
28	ReceiveExternallyValidatedADSB	30
29	ReceiveStateVectorV2VReport	33
30	GetPresAEMSigma	34
31	ConvertVFOMToSigmaVEPU	35
32	AssociateV2VToTarget	35
33	AddV2VTrackToDB	35
34	ReceiveExternallyValidatedV2V	36
35	InitializePassiveTrackerPosition	37
36	ConvertNACvToSigmaHVA	38
37	AdvancePassiveTrackPosition	39
38	InflatePassiveHorSigmaUL	40
39	IsOutlier	41
40	ReCenterHorizontalTrackLocation	42
41	RedefineEstimateInRotatedFrame	42
42	AdvancePassiveTrackVelocity	43
43	AdvancePassiveTrackState	45
44	PredictPassiveTracker	46

45	Predict1DKalmanFilter	47
46	Predict2DKalmanFilter	47
47	PredictKalmanFilter	47
48	UpdatePassiveTrackerPosition	48
49	UpdateKalmanFilter	49
50	UpdatePassiveTrackerVelocity	49
51	UpdatePassiveTrackerState	50
52	InitializeVerticalTracker	51
53	AdvanceVerticalTrack	53
54	PredictVerticalTracker	54
55	UpdateVerticalTracker	55
56	PredictedAltitude	56
57	ReceiveOwnRelNonCoopTrack	59
58	CheckORNCTUncertainty	60
59	ValidCovarianceMatrix	60
60	ParseORNCTStatus	61
61	AssociateORNCTIdToTarget	61
62	DetermineORNCTDeletion	62
63	RotateHorizontalFrame	63
64	MatrixRotation	63
65	AddORNCTTrackToDB	64
66	PredictORNCTTrack	64
67	UpdateORNCTTrack	65
68	ReceiveAbsoluteGeodeticTrack	68
69	AssociateAGTToTarget	69
70	AssociateAGTToOwnship	69
71	CheckTrackStatus	70
72	AssembleAGTTrack	72
73	AddAGTTrackToDB	72
74	InitializeAGTTrack	73
75	DeleteAGTTrack	74
76	UpdateAGTTrack	76
77	ReceivePointObstacleReport	78
78	AssociatePointObstacleReportToTarget	78
79	AddPointObstacleTrackToDB	79
80	ReceiveDiscretes	80
81	ReceivePresAltObservation	82
82	InitializeOwnVerticalTracker	83
83	UpdateHistory	83
84	PresAltAtToa	84
85	SurroundingPoints	85
86	ReceiveHeightAglObservation	85
87	ReceiveTrueAirspeedObservation	86
88	ReceiveHeadingObservation	88
89	InitializeHeadingTracker	89
90	UpdateHeadingTracker	89
91	HeadingAtToa	90
92	ReceiveWgs84Observation	92

93	OffsetTracksByOwnshipTurn	93
94	CalculateOwnshipVelocityOffsets	94
95	ConvertNACpToSigmaHEPU	95
96	HAEAltAtToa	96
97	ReceiveV2VCoordination	99
98	AssociateV2VToV2VAndAGTTargets	100
99	DeleteIntent	100
100	UpdateIntruderVRC	101
101	UpdateIntruderHRC	102
102	CoordinationTimeoutCheck	102
103	ReceiveV2VOperationalStatusMessage	103
104	SetCoordination	104
105	ReceiveADSBOperationalStatusMessage	105
106	UpdateAdvisoryMode	106
107	RefineOwnshipAGL	107
108	AlignAltitudeTypesUsedInGeodeticPositions	109
109	GetGroundSpeed	110
110	SetOwnV2VOperationalStatusMessage	110
111	CorrelationProcessing	111
112	GenerateStmReport	112
113	OwnWgs84Timeout	112
114	PropagateOwnshipToToa	114
115	SetOwnshipData	116
116	SetOwnshipVelocity	117
117	UpdateHorizontalVelocityMode	118
118	AddTracksToReport	119
119	TrackSourceSelection	121
120	OptimalPreTrackedTracks	122
121	AssessVerticalValidity	123
122	ConvertGVAToSigmaVEPU	124
123	OptimalQualityTrack	125
124	PredictPreTrackedTrack	126
125	PassiveQualityCheck	127
126	OptimalTrackHistoryUpdate	128
127	SelectFinalTrack	129
128	GetPreviouslySelectedTrack	130
129	FilterTracksForTRM	132
130	CompareIntruderRange	133
131	SlantRange	133
132	AddADSBTrackToReport	135
133	SetClassification	136
134	SetIntruderIDDirectory	137
135	SetIntruderDegradedFlags	138
136	GetRelativeAltitude	139
137	AddHorBias	140
138	AbsoluteGeodeticToOwnshipRelative	141
139	AddAltBiasAndSample	143
140	SigmaPointSample	144

141 SetIntruderBeliefStates 144
142 SetDisplayDataPassive 146
143 SetAvailableSources 147
144 VerticalRateArrowUpdate 148
145 AddV2VTrackToReport 150
146 AddORNCTTrackToReport 152
147 SetDisplayDataORNCT 153
148 AddAGTTrackToReport 155
149 AddPointObstacleTrackToReport 157
150 RemoveStaleTracks 159
151 RemoveStaleADSBTrack 160
152 RemoveStaleORNCTTrack 160
153 RemoveStaleV2VTrack 161
154 TargetIsEmpty 161
155 ValidationCheck 162
156 sXuTRMUpdate 164
157 HorizontalTRMUpdate 168
158 SetHorizontalTRMSurvOnlyMode 169
159 PrioritizeAndFilterIntruders 171
160 IsIntruderInScope 172
161 UpdateHTRMIntruderInputs 173
162 ResetHTRMState 174
163 HorizontalStateEstimation 175
164 ConvertOwnInputsToEnuBeliefs 176
165 ConvertOwnInputsToEnuBeliefsGroundAndCompassMix 177
166 ConvertOwnInputsToEnuBeliefsWindRelative 178
167 ConvertIntruderInputsToEnuBeliefs 179
168 PointObstacleEnuBeliefEstimation 180
169 EnuBeliefEstimation 181
170 FindMaximumWeightAndIndex 182
171 GetHorizontalPolicyCosts 184
172 GetMaxApplicabilityRange 184
173 SelectScaledPolicyBins 185
174 GetCutpointIndexVector 185
175 CreatePolicyStateBeliefs 186
176 CalculatePolicyState 187
177 SelectAndModifyPolicyBins 188
178 ModifyPolicySpeedBinsIndividual 188
179 GetPolicyTableCosts 190
180 UpdatePolicySpeedBins 191
181 UpdatePolicySpeedBinIndividual 192
182 DiscretizePolicyState 192
183 ApplyStateDependentOnlineCosts 193
184 StateDependentCrossingCostFactors 195
185 CalculateCrossingCostPsiFactor 196
186 CalculateCrossingCostSpeedFactor 196
187 CalculateCrossingCostThetaFactor 197
188 CalculateCrossingCostVerticalTauFactor 198

189 DetermineStateDependentSpeedFactor 198
190 ApplyHorizontalOnlineCosts 200
191 GetPerformanceBasedParams 201
192 HorizontalReversalCost 202
193 HorizontalWeakeningCostFactor 202
194 HorizontalInitialCACost 202
195 HorizontalDesensitivityCost 203
196 HorizontalTurnRateSensitivityCostFactor 203
197 HorizontalCoordinationCost 204
198 HorizontalMultiThreatReversalCost 205
199 HorizontalCoordinatedReversalCost 206
200 FindHighestThreatIndex 206
201 HorizontalInitialMaintainCost 207
202 HorizontalMaintainHeadingIncentive 207
203 HorizontalTurnFromMaintainIncentive 207
204 HorizontalMultiThreatMaintainHeadingCost 208
205 HorizontalMultiThreatTurnFromMaintainCost 209
206 SelectHorizontalAdvisory 210
207 SelectHorizontalAdvisoryWindRelative 211
208 HorizontalAlertLookahead 214
209 SelectHorizontalAlertLookaheadAction 216
210 HorizontalCostFusion 217
211 UpdateTargetTrackAngle 219
212 UpdateNumHorizontalReversals 220
213 HorizontalCoordinationSelection 222
214 GetHorizontalCoordinationPolicy 223
215 GetHorizontalCoordinationOffsetFactor 224
216 ApplyHorizontalCoordinationHysteresis 225
217 EncodeHRC 226
218 EncodeCHC 227
219 EncodeHSB 227
220 HorizontalTrackThreatAssessment 228
221 DetermineHorizontalCode 229
222 DetermineHorizontalScore 230
223 HorizontalDisplayLogicDetermination 231
224 DetermineDisplayTrackAngleAndTurnRate 232
225 DetermineLabel271 232
226 Consistent271 233
227 DetermineHorizontalDesensitivity 233
228 GenerateHTRMOutput 234
229 SetHorizontalIntruderOutput 235
230 SetHorizontalDroppedIntruderOutput 236
231 SetAHRAReportData 237
232 VerticalTRMUpdate 241
233 CreateGroundIntruder 242
234 GetVTRMOwnData 243
235 GetOwnWeightedAverages 243
236 UpdateAltitudeInhibitCState 244

237 DetermineLDI 245
238 SetVerticalTRMSurvOnlyMode 247
239 DetermineGroundAlert 248
240 StateAndCostEstimation 251
241 IntruderPrep 254
242 StateEstimation 257
243 ConvertHorizontal 258
244 HorizontalWorstCase 259
245 DetermineHorizontalWorstCaseNearMeanFactor 260
246 DetermineHorizontalWorstCasePhiSpreadFactor 261
247 UpdateInitializationCState 261
248 TauEstimation 263
249 GetEntryDistributionTables 264
250 DetermineTauVerticalDistribution 265
251 LookupEntryDistribution 266
252 InterpolateBlocks 266
253 GroundPointObstacleTauEstimation 267
254 PointObstacleTauEstimation 269
255 CombineVerticalBeliefs 270
256 SetBeliefBasedOnlineCostState 271
257 DetermineAltitudeDependentCOCFactor 273
258 DetermineVerticalRateReductionFactor 274
259 DetermineVerticalProximateFactor 275
260 OfflineCostEstimation 277
261 GetOfflineCostTauBeliefs 278
262 OfflineStatesScaleFactor 279
263 GetEquivClassTableMaxCutValues 279
264 GetTableMaxCutValue 280
265 LookupOfflineCost 281
266 OnlineUncoordinatedCostEstimation 284
267 GetModifiedGlobalRates 285
268 OnlineUncoordinatedActionCostEstimation 287
269 AdvisoryRestartCost 288
270 UpdateAdvisoryRestartCState 288
271 AltitudeDependentCOCCost 289
272 AltitudeInhibitCost 290
273 BadTransitionCost 291
274 UpdateBadTransitionCState 292
275 BadMaintainTransitionCost 293
276 CriticalIntervalProtectionCost 294
277 UpdateCriticalIntervalProtectionCState 296
278 IsOutsideCriticalInterval 297
279 CriticalIntervalRequiresVerticalDivergence 298
280 InitializationCost 299
281 PreventEarlyCOCCost 299
282 UpdatePreventEarlyCOCCState 301
283 SafeCrossingRADeferralCost 302
284 UpdateSafeCrossingRADeferralCState 303

285 VerticalRateLimitDependentStrengthenCost 304
286 PointObsDescendCost 305
287 PointObsPreventEarlyCOCCost 306
288 UpdateIntruderInputs 307
289 OnlineCostEstimation 310
290 OnlineCoordinatedActionCostEstimation 312
291 IntruderResponseEstimation 314
292 OwnResponseEstimation 315
293 DetermineResponse 316
294 CompatibilityCost 319
295 UpdateCompatibilityCState 320
296 CoordinatedRADeferralCost 321
297 UpdateCoordinatedRADeferralCState 322
298 CoordinationDelayCost 323
299 UpdateCoordinationDelayCState 323
300 CrossingNoAlertCost 324
301 UpdateCrossingNoAlertCState 325
302 MaxReversalCost 326
303 UpdateMaxReversalCState 327
304 PreventEarlyWeakeningCost 328
305 RestrictCOCDueToReversal 329
306 UpdateRestrictCOCDueToReversalCState 329
307 SA01Heuristic 330
308 UpdateSA01HeuristicCState 331
309 ShouldReverse 332
310 TimeBasedNonComplianceCost 333
311 UpdateTimeBasedNonComplianceCState 335
312 FindCompliantRateDelta 336
313 DetermineOwnNonComplianceFactor 337
314 IndividualCostEstimation 339
315 ActionSelection 341
316 NoIntrudersAction 342
317 DetermineMinimumCostAction 343
318 DetermineMultiIntruderAction 345
319 IndividualSelectionCostEstimation 346
320 SandwichPreventionCost 348
321 UnequippedCostFusion 349
322 MTLODetermination 350
323 DetermineSenses 351
324 DetermineConflictingSenses 351
325 AllowUnequippedMTLO 352
326 MultithreatCostBalancing 354
327 ActionArbitration 355
328 ArbitrateConflictingSenses 357
329 ArbitrateMatchingSenses 358
330 CoordinationSelection 361
331 Crosslink 362
332 EncodeCVC 363

333 EncodeVRC 364
334 EncodeVSB 364
335 DisplayLogicDetermination 366
336 DetermineCrossing 367
337 DetermineDisplayData 368
338 GetMatchingLabel270Rule 370
339 Consistent 371
340 DetermineCode 372
341 SetIntruderVerticalScores 372
342 DetermineVerticalScore 373
343 GenerateVTRMOutput 376
344 GenerateRAModeOutput 378
345 AdjustRAModeIntruderOutput 379
346 UpdateIndivAdjustThreatInfo 380
347 UpdateIndivAdjustCounts 381
348 GenerateSurvOnlyModeOutput 382
349 GenerateStandbyModeOutput 383
350 DetermineVerticalRAData 385
351 SetVerticalRAMessageOutput 386
352 DetermineTTIAndTID 387
353 EncodeTIDAltitude 388
354 EncodeTIDRange 389
355 EncodeTIDBearing 390
356 SetRABroadcastData 391
357 DroppedIntrudersAdjustment 393
358 SetDroppedIntruder 394
359 SetInvalidIntruder 395
360 GenerateBlendedsXuTRMOutput 396
361 sXuTRMBlendedReportPrep 397
362 UpdateRaPubState 398
363 SetBlendedIntruderThreatData 400
364 SetThreatIdentityData 401
365 SetRaPublishedState 402
366 SetReportDataOnRATerm 403
367 SetReportRATerminationBits 404
368 SetPrevVerticalTerminationData 405
369 SetPrevHorizontalTerminationData 406
370 DetermineDimensionalRAState 407
371 DetermineMultidimensionalMultithreat 408
372 SetBlendedCoordinationData 409
373 ActionToRates 410
374 CalculateThresholdRampDownFactor 410
375 CalculateThresholdRampUpFactor 411
376 CoastEnuBeliefBasedOnIndividualSamples 411
377 CoastEnuPositionVelocity 411
378 ExpectedTau 412
379 GetOwnHeight 412
380 HorizontalActionToRate 413

381 HorizontalRateToAction 413
382 HorizontalRateToSense 414
383 HorizontalRateToAdvisory 414
384 HRCToAdvisory 415
385 Interpolants 416
386 InterpolateOneDimension 418
387 IsCOC 419
388 IsCorrective 419
389 IsCrossing 420
390 IsDiverging 420
391 IsDNC 421
392 IsDND 421
393 IsHorizontalCOC 422
394 IsHorizontalReversal 422
395 IsHorizontalWeakening 423
396 IsIntruderEquipped 423
397 IsIntruderMaster 424
398 IsMaintain 424
399 IsMasterForcingReversal 425
400 IsMTLO 425
401 IsPreventive 426
402 IsProjectedCrossing 426
403 IsReversal 427
404 IsSlowClosure 427
405 IsStrengthening 428
406 IsVerticallyConverging 428
407 IsWithinRateBounds 429
408 MaintainRates 429
409 MinCostIndex 430
410 MTLOAction 430
411 PersistMTLO 431
412 RatesToAction 432
413 RatesToSense 432
414 TRMIntruderStateUpdate 434
415 TurnEnuBeliefBasedOnIndividualSamples 434
416 TurnEnuPositionVelocity 435
417 UpdateHTRMIntruderState 435
418 UpdateHTRMIntruderStates 436
419 VRCToSense 436
420 StmHousekeeping 437
421 StmHousekeepingPrioritization 438
422 PredictTrackSummary 439
423 GetTracksToExtrapolate 440
424 GetAircraftTracks 440
425 ExtrapolateTrack 440
426 ExtrapolateOwnshipTrack 442
427 ExtrapolateADSBTrack 443
428 ExtrapolateV2VTrack 444

429 ExtrapolateORNCTTrack 445
430 LinearTransform 446
431 ExtrapolateAGTTrack 447
432 ConvertCartesianToPolar2D 448
433 WeightedMeanAndCovariance 448
434 CorrelationTrackType 449
435 DecorrelateTargetsFromOwnship 451
436 RemoveDecorrelatedTrackFromOwnship 451
437 CorrelateTargetsToOwnship 452
438 AGTOnlyTarget 453
439 CorrelateIndividualTracksToOwnship 454
440 CorrelateToOwnshipByType 455
441 DecorrelateTargets 457
442 GetTracksToDecorrelate 458
443 CorrelationHistoryUpdate 458
444 CorrelationHistoryCheck 458
445 AddDecorrelatedTrackToTarget 459
446 RemoveDecorrelatedTrackFromTarget 459
447 GenerateLeadTrackList 460
448 CorrelateTargets 461
449 SortByRange 461
450 CorrelateIndividualTracks 462
451 MergeTargets 464
452 CorrelateByType 466
453 CorrelateID 468
454 GetCorrelationMechanism 469
455 CorrelatePosition 471
456 constants 474
457 GetIndexFromClassification 576
458 CreateHorizontalScaledCutsSets 577
459 ScaleHorizontalCuts 577
460 CreateVerticalScaledCutsSets 578
461 ScaleVerticalCuts 579
462 AngleDifference 580
463 WrapAngleDiffToPi 580
464 block_diag 580
465 mahal 581
466 uchol 582
467 eye 582
468 Normalize 582
469 WrapTo2Pi 583
470 UnitVectorNormalize 583
471 WrapToPi 583
472 nanmin 584
473 nanmax 584
474 nanminimum 584
475 nanmaximum 585
476 VerticalTRMUpdatePrep 587

477 HorizontalTRMUpdatePrep 589
478 ResetHorizontalTRMOwnshipState 590
479 ReceiveDescentInhibitThresholds 608
480 ReceiveTargetDesignation 608
481 StmHousekeepingTargetDesignation 609

LIST OF TYPES

Type No.		Page
1	GeoUtils	473
2	StmReport	480
3	StmDisplayStruct	481
4	IntruderIDDirectory	482
5	IntruderIDEntry	482
6	TrackFile	483
7	ORNCTTrackFile	484
8	ADSBTrackFile	485
9	AGTTrackFile	486
10	PointObstacleTrackFile	487
11	V2VTrackFile	488
12	CorrelationHistory	489
13	DesignationState	489
14	OwnDiscreteData	490
15	OwnShipData	491
16	OwnHistory	492
17	GeodeticStates	492
18	ReceivedCoordinationData	492
19	Target	493
20	Database	494
21	PriorityCodes	494
22	TrackMap	494
23	TrackSummary	495
24	ValueTime	495
25	ADSBOperationalStatusMessage	496
26	V2VOperationalStatusMessage	496
27	TRMInput	497
28	TRMOwnInput	498
29	TRMIntruderInput	499
30	IntruderHorizontalBelief	500
31	IntruderVerticalBelief	500
32	OwnVerticalBelief	501
33	sXuTRMReport	501
34	VTRMDisplayData	502
35	TRMIntruderDisplayData	503
36	HTRMDisplayData	503
37	HTRMIntruderDisplayData	504
38	TRMCoordinationInterrogationData	504
39	TRMDesignationData	505
40	TRMIntruderDesignationData	505
41	TRMLogicModeData	506
42	TRMAlertingData	506
43	sXuTRMBroadcastData	507
44	VTRMRAData	508

45	TID	508
46	TIDAddress	508
47	TIDAltRngBrg	509
48	CCCB	509
49	sXuTRMState	510
50	TRMState	510
51	TRMOwnState	511
52	TRMIntruderState	512
53	HTRMState	513
54	HTRMOwnState	513
55	HTRMIntruderState	514
56	AdvisoryBeliefState	515
57	CombinedVerticalBelief	515
58	DisplayLogic	516
59	EnuBelief	516
60	EnuBeliefSet	517
61	EnuPositionVelocity	517
62	GlobalAdvisory	517
63	HorizontalAdvisory	518
64	HorizontalDisplayLogic	518
65	HTRMReport	519
66	IndividualAdvisory	519
67	OnlineCostState	520
68	PolicyStateBelief	521
69	PolicyUInt8DataTable	521
70	RDataTable	521
71	RDataTableScaled	522
72	TauBelief	522
73	TRMIndivAdjustState	523
74	TRMIntruderData	523
75	VTRMReport	524
76	VerticalRAOutputState	524
77	TRMOfflineTable	525
78	IntruderPublishedRaState	525
79	ActionArbitrationCState	526
80	ActionArbitrationGlobalCState	526
81	AdvisoryRestartCState	526
82	AltitudeDependentCOCCState	527
83	AltitudeInhibitCState	527
84	BadTransitionCState	527
85	CompatibilityCState	528
86	CoordinationDelayCState	528
87	CoordinatedRADeferralCState	528
88	CriticalIntervalProtectionCState	529
89	CrossingNoAlertCState	529
90	InitializationCState	529
91	MaxReversalCState	530
92	MultithreatCostBalancingCState	530

93	PreventEarlyCOCCState	530
94	ResponseEstimationCState	531
95	RestrictCOCCState	531
96	SA01HeuristicCState	531
97	SafeCrossingRADeferralCState	532
98	TimeBasedNonComplianceCState	532

1 Introduction

1.1 Purpose

This document provides the Algorithm Design Description (ADD) for the Surveillance and Tracking Module (STM) and the Threat Resolution Module (TRM) of the next generation Airborne Collision Avoidance System (ACAS X). The algorithms are described at a sufficiently high level to allow for implementation in a variety of software languages and hardware platforms, thereby providing maximum freedom to manufacturers while ensuring the intended output from the system. This document provides descriptions of the algorithm's operations. Every effort has been made to ensure the accuracy of this text, however, in case of a conflict between the description and the function of the algorithm, the algorithm function is correct.

1.2 Scope

This document assumes the reader is familiar with ACAS sXu including planned variants of the system software and its modular breakdown. If not, the reader is encouraged to review the ACAS sXu Concept of Operations (ACAS sXu CONOPS) [1].

The ACAS X algorithm design consists of two separate modules: (1) the Surveillance and Tracking Module (STM), which detects aircraft in the vicinity and tracks their position, and (2) the Threat Resolution Module (TRM), which identifies threats and provides resolution guidance. The TRM consists of two modules: (1) the vertical TRM, which produces climb and descend advisories, (2) the horizontal TRM, which produces right and left advisories. The algorithms in this ADD are limited to ACAS sXu, the variants of ACAS X intended for installation on smaller unmanned aircraft. This document contains modifications informed by operational suitability and performance evaluations conducted via modeling and simulation as well as analysis of flight tests.

Although this document provides justification for certain aspects of algorithms, it is not intended to overview the analyses that contributed to the various design decisions. This document does not discuss the process of optimizing the lookup tables. For such details, see [2].

The algorithm descriptions in this document are prototypes that have not been fully validated. Several of the algorithms will be updated in future versions of this document.

1.3 Background

The Traffic-Alert and Collision Avoidance System (TCAS), currently mandated worldwide on all large transport and cargo aircraft, has been shown to significantly reduce the risk of mid-air collision. TCAS uses an on-board surveillance system to monitor nearby air traffic. The surveillance information is then fed into the threat resolution logic to determine whether to alert the pilots of a potential collision. If deemed necessary to prevent collision, TCAS will issue a resolution advisory to the pilots to climb or descend at a particular rate.

Developing robust collision avoidance logic that reliably prevents collision without excessive alerting is challenging due to sensor error and uncertainty in pilot response and, consequently, the future paths of the aircraft. The current TCAS logic was the result of many years of development and involved the careful engineering of many heuristic rules. Unfortunately, due to the

complexity of the logic, it is difficult to revise it to accommodate the evolution of the airspace and the introduction of new surveillance technologies and procedures.

Since the mid-2000s, research has focused on the use of a computational method known as dynamic programming for producing optimized decision logic for airborne collision avoidance. This research has resulted in the establishment of the Airborne Collision Avoidance System X (ACAS X) program that is planned to become the next international standard for collision avoidance for both manned and unmanned aircraft.

1.4 Context

The ACAS sXu design is a result of system performance evaluations from both model-based fast-time simulation and ACAS sXu flight test campaigns in 2018. It incorporates changes informed by operational suitability and performance evaluations of ACAS Xu flight tests in 2014 and 2017 and ACAS Xa flight tests in 2013 and 2015 and from the ACAS Xa Operational Evaluation in 2017.

1.5 Prescription

Each algorithm in this document falls into one of three categories: prescriptive, suggested, or utility.

Prescriptive algorithms specify required functionality which must be included in any ACAS sXu system in a manner consistent with the Julia code of this document. The prescriptive algorithms therefore fulfill a similar role to "shall" statements within the ACAS sXu MOPS Volume I [3]. Optimization, refactoring, and translation of the Julia algorithms when developing a final system are acceptable, so long as verification ensures that outputs and side-effects (i.e., database state) for permissible inputs match those of the Julia code contained in this document.

Suggested algorithms are provided as a reference for the implementer when attempting to achieve particular MOPS functional and performance requirements. The suggested algorithms are not the only acceptable approach and may require adaptations due to specific hardware or other system characteristics (i.e., the suggested algorithms are not necessary and may not be sufficient for all systems). There are limited interactions between suggested algorithms and prescriptive algorithms or data structures. An implementation not leveraging the provided suggested algorithms may safely omit calls to suggested algorithms or data structure elements set solely by the suggested algorithms without affecting prescriptive functionality.

Utility algorithms consist of math functions, data structure modifiers, and other routines that are often found in standard libraries. Like prescriptive algorithms, any implementation must match the Julia code within this document with respect to outputs and side-effects. Unlike the prescriptive algorithms, the utility algorithms may not be specific to ACAS X.

Provisions for Potential Modifications algorithms consist of modifications to the system that require additional approval, therefore deviating from the standard defined in the prescriptive algorithms of this document. Further discussion can be found in Appendix L.

Table 1 details the categorization of all algorithms, listed by section, within this document. The listed categorization applies to the entirety of each listed section.

Table 1. Algorithm Prescription Categorization

Algorithms	Categorization
Section 2 (STM)	Prescriptive
Section 3 (TRM)	Prescriptive
Appendix A (STM Housekeeping)	Prescriptive
Appendix B (Correlation)	Suggested
Appendix C (Constants)	Prescriptive
Appendix D (Data Structures)	Prescriptive
Appendix G (Offline-Table Scaling)	Prescriptive
Appendix H (Math Utilities)	Utility
Appendix I (Data Management)	Prescriptive
Appendix J (Standard Library)	Utility
Appendix L (Provisions for Potential Modifications)	Provisions for Potential Modifications

1.6 Specification Language: Julia

In addition to their English descriptions, the algorithms of the ACAS sXu are provided in the Julia programming language [4] (`http://julialang.org`), an open source, high-level programming language with a natural and familiar syntax that is being developed at MIT. At the time of this release, the supported Julia version is 1.0.3.

The Julia code blocks in this document will be of *named* algorithm components, and will consist either of *types* or *algorithms* (algorithms also are called *functions*).

1.6.1 Data Types

The data types used in the algorithms are summarized in Table 2. The basic types of Julia are Float64, Int64, Bool, Symbol, and Array. Array indexing starts at 1. The aliases R and Z are defined for the basic Julia types Float64 and Int64, respectively.

The type of a variable in Julia can be specified after a character sequence '::' asserting that the next name is the type of the variable. For example, *dz_min::R* declares *dz_min* to be a variable of type *R* (Float64). The character sequence '<:' indicates a parent type during type declaration or indicates if the left-hand side of the expression is a subtype of the right-hand side. For example, *ModeSTrackFile <: TrackFile* means *ModeSTrackFile* "is a subtype of" *TrackFile*.

This document also uses *composite types*, which organize other types into collections. Composite types are generally defined when several simpler types may have limited meaning individually but describe a clear conceptual object together. For example, TRMINPUT is a composite type consisting of "fields" own, and $intruder$. The "fields" of a composite type are made accessible in Julia code by way of the '.' symbol. For example, if trm_input is a variable of composite type TRMINPUT, *trm_input.own* gains access to the input data for the own aircraft.

All of the composite types used in the ACAS sXu are described within the body of this document and in the appendices. See the "List of Types" for a complete list, which is in the front matter of this volume.

Table 2. Data Types

Data Type	Source	Julia Type	Description
Array{T,N}	Julia	Array{T,N}	array of data type *T* with *N* dimensions e.g., Array{Symbol,1} is a 1-dimensional array of type Symbol e.g., Array{Float64,2} is 2-dimensional array of type Float64
BitArray{N}	Julia	BitArray{N}	array of bits (1s and 0s) with *N* dimensions
Bool	Julia	Bool	boolean, contains value of TRUE or FALSE
Dict{K,V}	Julia	Dict{K,V}	dictionary, table with keys of type *K* and values of type *V*
Float16	Julia	Float16	16-bit floating point (real) value (e.g., *1.0*)
Float64	Julia	Float64	64-bit floating point (real) value (e.g., *1.0*)
Int64	Julia	Int64	64-bit integer (e.g., *1*)
Matrix{T}	Julia	Array{T,2}	alias for 2-dimensional array of data type *T*
String	Julia	String	character string (e.g., *"this is a string"*)
Symbol	Julia	Symbol	symbol (e.g., *:None*)
UInt8	Julia	UInt8	8-bit unsigned integer, minimal size for integer values
UInt16	Julia	UInt16	16-bit unsigned integer
UInt32	Julia	UInt32	32-bit unsigned integer
Union{T1,T2,...}	Julia	Union{T1,T2,...}	any one of the specified data types (*T1*, *T2*, etc.)
Vector{T}	Julia	Array{T,1}	alias for 1-dimensional array of data type *T*
R	Alias	Float64	alias for 64-bit floating point (real)
Z	Alias	Int64	alias for 64-bit integer

1.6.2 Built-In Values and Functions

In addition to data types, Julia provides several special key words and functions that are used in this document. Special keywords are summarized in Table 3. This document relies heavily on the names (defined in Julia's standard library) "Inf", "-Inf", and "NaN", which represent positive infinity, negative infinity, and not a number. It also relies heavily on the Julia key words for boolean values: "true" and "false".

Table 3. Julia Keywords

Keyword	Description
false	boolean value for FALSE
-Inf	negative infinity stored as Float64
Inf	positive infinity stored as Float64
NaN	not a number, used to represent no value, stored as Float64
nothing	a value containing no data, used when no particular value is needed
pi	value of pi (3.141592...)
true	boolean value for TRUE
undef	token passed to an array constructor to request an uninitialized array

1.7 Notation

The notational convention used in this document was aimed at making the algorithms as compact as possible while ensuring readability and preventing ambiguity.

Table 4 summarizes the units used in this document.

Table 4. Fundamental Units

Dimension	Unit	Abbreviation
time	seconds	s
distance	feet	ft
angle	radians	rad
velocity	feet per second	ft/s
acceleration	feet per second per second	ft/s^2

1.7.1 Variables

Many of the important variables have both a symbolic notation as well as a textual notation. For example, time would be represented as t symbolically and t textually. The symbolic notation is used in the reference material and the textual notation is used in the pseudocode without the use of subscripts, superscripts, Greek, or any special mathematical symbols. Associated with each scalar variable is a unit of measurement. The convention described in this section is generally used within the TRM and applies to logic focused variables.

Many of the variables used by the logic are defined in terms of the own, intruder, and relative perspectives. To disambiguate what perspective is used, a text subscript will be used. For example, variable x from the perspective of the own, intruder, and relative perspectives is written x_own, x_int, and x_rel, respectively. In general, $x_rel \equiv x_int - x_own$.

The standard mathematical notation for the first derivative of any variable x with respect to time is written $\dot{x}$. If x is in ft, then $\dot{x}$ is in ft/s. The second derivative of any variable x with respect to time is written $\ddot{x}$. If x is in ft, then $\ddot{x}$ is in ft/s^2. In textual form, $\dot{x}$ is written dx and $\ddot{x}$ is written ddx.

The functions within this document create variables that are read-in from the data elements in the parameter file specification. These variables are created at the top of functions unless they need to be set once within a conditional. Variables that are read-in from the parameter file utilize an uppercase prefix. For example, VERTICALTRMUPDATE (Algorithm 232) reads in *params*().*actions.num_actions* from the parameter file and sets the variable to *N_actions* at the top of the algorithm. The parameters are further described in Appendix E.

Commonly used variable prefixes and their meanings are listed in Table 5.

Table 5. Key to Common Variable Prefixes

Prefix	Description
a_	advisory
b_	belief
C_	cost parameter
D_	distance parameter
ddz_	vertical acceleration
dz_	vertical rate
H_	altitude threshold parameter
h_	altitude above ground
input_	input data structure
is_	boolean result
L_	display logic parameter
N_	number parameter
R_	rate parameter
r_	range
ra_	Resolution Advisory
S_	state parameter (could be name or number)
s_c	a context-specific state data structure (refer to Appendix D.6)
st_	state
T_	time parameter
t_	time
w_	weight
X_	coordination parameter
x_	lateral east-component
y_	lateral north-component
z_	pressure/HAE altitude

1.7.2 Physical Quantities

Table 6 specifies the fundamental physical quantities used by the logic. Time is relative to some fixed instant in time. The variable dt is used to denote the duration of a time step. In the TRM the value of dt is kept fixed at 1 s, the ACAS X cycle time. In the STM the value of dt is relative to the time of applicability of the input data. The lateral and vertical components are with respect to a fixed global coordinate system. Headings are with respect to true north, increasing clockwise.

Table 6. Fundamental Physical Quantities

Symbol	Text	Description	Units
t	t	time	s
x	x	lateral east-component	ft
y	y	lateral north-component	ft
z	z	pressure/HAE altitude	ft
h	h	altitude above ground	ft
ψ	psi	heading	rad
q	quant	altitude quantization	ft

There are a number of physical quantities that are derived from the fundamental quantities. These derived quantities are used frequently in the pseudocode and merit their own symbols (Table 7). Both chi_abs and chi_rel increase clockwise.

Table 7. Derived Physical Quantities

Symbol	Text	Description	Units	Definition
r_{slant}	r_slant	slant range	ft	$\sqrt{x_{\text{rel}}^2 + y_{\text{rel}}^2 + z_{\text{rel}}^2}$
r_{ground}	r_ground	ground (or lateral) range	ft	$\sqrt{x_{\text{rel}}^2 + y_{\text{rel}}^2}$
s_{ground}	s_ground	magnitude of relative velocity vector	ft/s	$\sqrt{\dot{x}_{\text{rel}}^2 + \dot{y}_{\text{rel}}^2}$
χ_{abs}	chi_abs	bearing of intruder relative to north	rad	$\text{atan2}(x_{\text{rel}}, y_{\text{rel}})$
χ_{rel}	chi_rel	bearing of intruder relative to own heading angle	rad	$\textsc{WrapToPi}(\text{atan2}(x_{\text{rel}}, y_{\text{rel}}) - \psi_{\text{own}})$
ϕ_{rel}	phi_rel	absolute value of difference in the direction of the relative horizontal velocity and the bearing of the intruder	rad	$\lvert\textsc{WrapToPi}(\text{atan2}(\dot{x}_{\text{rel}}, \dot{y}_{\text{rel}}) - \text{atan2}(x_{\text{rel}}, y_{\text{rel}}))\rvert$

1.7.3 Statistical Quantities

Central to many of the concepts in this document is the normal (or Gaussian) distribution. Univariate normal distributions are defined by a scalar mean μ and scalar standard deviation σ. Multivariate normal distributions are defined by a vector μ and a covariance matrix Σ. In textual form, μ is represented mu, σ is represented sigma, σ^2 is represented var, and Σ is represented Sigma. To associate one of these parameters with a distribution over a particular variable, the variable can be used as a subscript. For example, the standard deviation of dx_rel may be written sigma_dx_rel.

Many of the algorithms involve weighted samples. A set of m samples, with each sample being represented using an n element vector, is represented by an $n \times m$ matrix, where the columns correspond to the different samples. The set of corresponding weights are represented using an m-element vector. Weights are generally represented using w or textually as weights. Each weight is a real number between 0 and 1, inclusive.

1.7.4 Functions

This document describes the algorithms that implement the ACAS sXu system. These algorithms are defined as functions in the Julia language. A function takes values as input, applies an algorithm, and outputs values.

- **Input.** The inputs to functions are passed by reference, and only passed by value if they are scalars.
- **Output.** The outputs of a function are initialized in the body of the function.
- **Constant.** Constant variables keep the same values. The assignment of a constant is specified where it is declared.

In some functions, the same variable name is used in the input and output. The output value is provided to the caller via the return statement. In functions that use state variables, the value of a state variable may be modified by the algorithm but not provided as an output of the function.

1.7.5 Diagrams

Diagrams are used in this document to show data processing flow. These diagrams are composed of data blocks and process blocks. Arrows indicate the flow of data into and out of the processing blocks. Data blocks containing scalar variables are represented with a single border. Blocks representing arrays are indicated using a double border. Figure 1 shows an example data processing flow involving arrays a and b as input and scalars x and y as output.

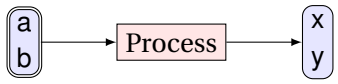

Figure 1. Example data processing flow.

The following helper functions are excluded from figures for clarity:

- ActionToRates (Algorithm 373)
- GetOwnHeight (Algorithm 379)
- HorizontalRateToSense (Algorithm 382)
- HorizontalTRMUpdatePrep (Algorithm 477)
- IsIntruderMaster (Algorithm 397)
- IsPreventive (Algorithm 401)
- NoIntrudersAction (Algorithm 316)
- RatesToAction (Algorithm 412)
- UpdateIntruderVRC (Algorithm 100)
- VerticalTRMUpdatePrep (Algorithm 476)
- WrapToPi (Algorithm 471)

2 Surveillance and Tracking Module Description

2.1 System Overview

There are multiple entry points to the STM, each associated with a unique type of sensor measurement. A summary of these entry points is given by Table 8. The algorithms in this document have been developed to allow the entry points to be accessed in an asynchronous fashion. This design was chosen to allow the STM to accommodate different update rates for the various sensor inputs.

Table 8. Summary of Entry Points to the STM

Algorithm	Input Variables
RECEIVESTATEVECTORPOSITIONREPORT (Algorithm 1)	Table 9
RECEIVESTATEVECTORVELOCITYREPORT (Algorithm 17)	Table 10
RECEIVEMODESTATUSREPORT (Algorithm 26)	Table 12
RECEIVEEXTERNALLYVALIDATEDADSB (Algorithm 28)	Table 13
RECEIVESTATEVECTORUATREPORT (Algorithm 25)	Table 11
RECEIVEADSBOPERATIONALSTATUSMESSAGE (Algorithm 105)	Table 28
RECEIVEOWNRELNONCOOPTRACK (Algorithm 57)	Table 17
RECEIVEABSOLUTEGEODETICTRACK (Algorithm 68)	Table 18
RECEIVESTATEVECTORV2VREPORT (Algorithm 29)	Table 14
RECEIVEEXTERNALLYVALIDATEDV2V (Algorithm 34)	Table 15
RECEIVEPOINTOBSTACLEREPORT (Algorithm 77)	Table 19
RECEIVEDISCRETES (Algorithm 80)	Table 20
RECEIVEPRESALTOBSERVATION (Algorithm 81)	Table 21
RECEIVEHEIGHTAGLOBSERVATION (Algorithm 86)	Table 22
RECEIVETRUEAIRSPEEDOBSERVATION (Algorithm 87)	Table 23
RECEIVEHEADINGOBSERVATION (Algorithm 88)	Table 24
RECEIVEWGS84OBSERVATION (Algorithm 92)	Table 25
RECEIVEV2VCOORDINATION (Algorithm 97)	Table 26
RECEIVEV2VOPERATIONALSTATUSMESSAGE (Algorithm 103)	Table 27
GENERATESTMREPORT (Algorithm 112)	N/A

When a sensor measurement is received by the STM, the measurement data is processed by the appropriate algorithms and target or ownship state information is stored in memory. Each measurement is accompanied by a Time of Applicability (TOA) field, thus allowing the STM to receive sensor measurements asynchronously.

The STM uses global data structures to keep track of internal state memory between cycles (more information can be found in Section K.2.3.1). All information for a single intruding aircraft is stored in a data structure known as a TARGET. A TARGET can contain any number of ADSBTRACKFILES ORNCTTRACKFILES, AGTTRACKFILES, V2VTRACKFILES, or POINTOBSTACLETRACKFILES. Each Track File contains the estimated relative position and velocity of the intruding aircraft as well as any additional information (e.g., reported accuracy or integrity) that has been derived from the corresponding surveillance source. A TARGET can also contain information such as the designation state of an aircraft based on input from the flight crew.

All information related to ownship is stored in a dedicated data structure named *own* of type OwnShipData. This information includes tracks on ownship heading and pressure altitude as well as the most recent height above-ground-level and WGS84 position and velocity updates. Ownship state information is regularly accessed by other functions in the STM to transform intruder measurements from their native coordinates to a common, relative coordinate frame.

Upon request, the STM will produce a single data structure that contains all elements needed by the TRM to perform threat evaluation. This is done through a call to GenerateStmReport. It is expected that such requests will be made on a 1 Hz cycle, however, the algorithms in this document do not preclude requests from being made at other rates. The data structure, known as the StmReport, also contains fields that are specifically intended for other external modules such as the pilot display and the aircraft's V2V capability data transmission device. The process of report generation iterates through the stored Targets and determines which, if any, Track File for a given Target is most appropriate to append to the StmReport. If necessary, the target's relative position will be extrapolated to the time of the request.

Several reference frames are used throughout the STM. Intruder reports produced by the STM include separate horizontal (relative to ownship) and vertical estimates of intruder position, each in their own reference frame. Aircraft flying in indicated level flight traverse horizontal planes of constant pressure (i.e., constant pressure altitude, also referred to in this document as barometric altitude), therefore the vertical component of the STM intruder report references the intruder's relative pressure altitude displacement from ownship. The horizontal component of STM intruder reports is delivered in a two-dimension (True North-East) Cartesian reference frame centered about the ownship.

Other reference frames are employed within the tracking portions of the STM specific to each surveillance source. Passive surveillance (ADS-B) provides intruder observations in geodetic coordinates. Intruder geodetic postion and velocity observations are converted to the Cartesian coordinate frame via ownship WGS84 state information prior to tracking. The geodetic-to-Cartesian conversion employed within the STM makes accommodation for the relative rotation of the north-aligned Cartesian reference frame when maneuvering in close proximity to a true pole.

Notice that the Cartesian plane is tangential to the geoid at the ownship position and describes a flat plane in an earth-centered earth-fixed (ECEF) reference frame. The STM horizontal trackers assume that intruder velocities are vectors along this plane. This contrasts with actual intruder trajectories, which typically follow curved surfaces within the ECEF reference frame for level flight maintaining constant orthometric height relative to the geoid. The magnitude of this discrepancy decreases with decreasing intruder range, and is minimal in comparison with other sources of error.

2.2 Passive Surveillance

This section defines the algorithms to process surveillance data that has been passively received from an ADS-B Report Generator [5] or other V2V cooperative surveillance link. Several major functions performed by these algorithms include:

1. The association of passive cooperative surveillance reports to an existing Track File.
2. The initialization of new Track Files and Targets.

3. The update of an existing Track File.
4. The correlation of a Track File to an existing TARGET.

It is important to note that ADS-R Track Files are treated as ADS-B Track Files with the *rebroadcast* flag set to TRUE. All ADS-R tracks sent to the STM will be ignored, as sXu will have a dual 1090-ES and 978-UAT link. The algorithms in this section do not include the functions that select, prepare and form the intruder information that is provided to the TRM and other external modules. That process is defined in Section 2.8, specifically ADDADSBTRACKTOREPORT (Algorithm 132).

2.2.1 Input Processing

2.2.1.1 Automatic Dependent Surveillance Broadcast (ADS-B)

There are seven entry points to the STM for passive surveillance. Each entry point is associated with the reception of a specific ADS-B or V2V Report type as generated by an ADS-B or V2V Report Generator. As such, each of the entry points require a different set of input parameters as indicated in Table 9, Table 10, Table 11, Table 12, and Table 13 for ADS-B surveillance; and Table 14, and Table 15 for V2V surveillance.

2.2.1.1.1 State Vector Position Report

RECEIVESTATEVECTORPOSITIONREPORT (Algorithm 1) requires the inputs defined in Table 9. Upon the reception of an ADS-B State Vector Position Report, the STM first checks if the reported latitude and longitude are valid. Additionally, reports that are rebroadcasted are not processed. The position information contained within the report (*lat*, *lon* and *alt*) is combined into a 3D position vector *obs_lla*, where the angles are converted to radians. Next, the reported Mode S address (*mode_s*), and the *non_icao* flag are used to associate the report to an existing Target (ASSOCIATEADSBREPORTTOTARGET (Algorithm 8)).

If the State Vector Position Report is not associated with an existing Target, a new ADSBTRACKFILE is created and initialized through a call to INITIALIZEPASSIVETRACKFILE (Algorithm 9). Next, the new ADSBTRACKFILE is stored in memory through ADDADSBTRACKTODB (Algorithm 10). These algorithms attempt to correlate the new ADSBTRACKFILE to an existing Target (ASSOCIATEICAOTOTARGET (Algorithm 11)). If correlation is successful, the ADSBTRACKFILE is added to the existing Target. Otherwise, the ADSBTRACKFILE is added to a new Target. The horizontal track is defined relative to the geodetic location of the intruder, and this absolute geodetic location is stored as additional data elements in the ADSBTRACKFILE.

If the call to ASSOCIATEADSBREPORTTOTARGET successfully associates to a Target, the corresponding ADSBTRACKFILE is retrieved from memory. The track will only be updated if the existing track was not based on UAT reports as indicated by *uat*. If a minimum allowed time (*min_obs_toa_step*) has not passed since the last track update, the ADSBTRACKFILE is not updated with the State Vector Position Report. If the minimum time threshold has passed, the *last_valid_vert* and *last_alt_src_hae* variables are saved off prior to calling ADVANCEVERTICALTRACK (Algorithm 53). If the *valid_vert* Boolean has changed or the altitude type saved in the vertical tracker has changed, the altitude within the absolute geodetic LLA and ECEF positions of the horizontal tracker are updated with a call to UPDATEALTINHORGEOPOS (Algorithm 15). Next, since the original *obs_lla* vector used in INITIALIZEPASSIVETRACKFILE (Algorithm 9) may contain an invalid altitude, this altitude observation is modified to use the altitude returned by

GETALTFORHORTRKOBSERVATION (Algorithm 16). Then *obs_ecef* is calculated by a call to CONVERTWGS84TOECEF. The horizontal tracker is then advanced by a call to ADVANCEPASSIVETRACKPOSITION (Algorithm 37).

This algorithm updates *trk*.

Table 9. STM Input Variables - State Vector Position Report

Variable	Units	Type	Description
lat	degrees	real	Intruder latitude
lon	degrees	real	Intruder longitude
alt	feet	real	Intruder altitude (nominally pressure altitude)
is_alt_geo_hae	N/A	bool	Intruder reporting geodetic height-above-ellipsoid (HAE) altitude
mode_s	N/A	uint32	Intruder 24-bit aircraft address
nic	N/A	uint32	Intruder Navigation Integrity Category (NIC)
q_int	feet	uint32	Intruder altitude quantization (25 or 100)
rebroadcast	N/A	bool	Flag to indicate message received from ADS-R
non_icao	N/A	bool	Flag to indicate that the address is a non-ICAO 24-bit aircraft address (anonymous address)
toa	seconds	real	Time of applicability

Algorithm 1 ReceiveStateVectorPositionReport

```
function ReceiveStateVectorPositionReport(lat::R, lon::R, alt::R, is_alt_geo_hae::Bool, mode_s::UInt32, nic::
     UInt32, q_int::UInt32, rebroadcast::Bool, non_icao::Bool, toa::R)
    min_obs_toa_step::R = params().surveillance.min_obs_toa_step
    if !isnan(lat) && !isnan(lon) && !rebroadcast
        obs_lla = [deg2rad(lat), deg2rad(lon), alt]
        id = AssociateADSBReportToTarget(p. 16)(mode_s, non_icao)
        if (id == NO_TARGET_FOUND)
            trk = ADSBTrackFile(p. 485)(mode_s, non_icao, false)
            InitializePassiveTrackFile(p. 17)(trk, obs_lla, is_alt_geo_hae, nic, toa)
            AddADSBTrackToDB(p. 18)(trk)
        else
            tgt = RetrieveWithID(p. 19)(target_db, id)
            trk = tgt.adsb_track
            if (!trk.uat)
                dt = toa - trk.toa
                if (dt >= min_obs_toa_step)
                    last_valid_vert::Bool = trk.valid_vert
                    last_alt_src_hae::Bool = trk.alt_src_hae
                    AdvanceVerticalTrack(p. 53)(trk, alt, q_int, is_alt_geo_hae, true, toa)
                    if (last_valid_vert != trk.valid_vert) || (last_alt_src_hae != trk.alt_src_hae)
                        UpdateAltInHorGeoPos(p. 20)(trk)
                    end
                    obs_lla[3] = GetAltForHorTrkObservation(p. 20)(trk, toa)
                    obs_ecef = ConvertWGS84ToECEF(p. 13)(obs_lla, trk.alt_src_hae)
                    AdvancePassiveTrackPosition(p. 39)(trk, obs_lla, obs_ecef, nic, toa)
                end
            end
        end
    end
end
```

CONVERTWGS84TOECEF (Algorithm 2) converts a WGS84 position into ECEF coordinates. The calibrated height above mean sea level (MSL) from a barometric sensor (i.e. orthometric height above MSL) is first converted to WGS84 height above ellipsoid (HAE) through a call to CONVERTORTHOMETRICTOGEODETICHEIGHT (Algorithm 4) to avoid skewing calculations for X/Y positions.

This algorithm takes as input *lla*. This algorithm returns a vector *ecef*.

Algorithm 2 ConvertWGS84ToECEF

```
function ConvertWGS84ToECEF(lla::Vector{R}, is_alt_geo_hae::Bool)
    a::R     = geoutils.earth_semimajor_axis
    e1_sq::R = geoutils.earth_first_eccentricity_sq
    m2ft::R  = geoutils.meters_to_feet
    (lat::R, lon::R, h::R) = lla
    h /= m2ft
    if !is_alt_geo_hae
        h = ConvertOrthometricToGeodeticHeight(p. 15)(h, lat)
    end
    N = a / sqrt(1 - (e1_sq*(sin(lat)^2)))
    X = (N + h) * cos(lat) * cos(lon)
    Y = (N + h) * cos(lat) * sin(lon)
    Z = (N*(1-e1_sq) + h) * sin(lat)
    ecef::Vector{R} = [X, Y, Z]
    return ecef::Vector{R}
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), UpdateAltInHorGeoPos(p. 20), RefineECEFAndLLA(p. 25), PropagateGeodeticToToa(p. 23), ReceiveStateVectorV2VReport(p. 33), AlignAltitudeTypesUsedInGeodeticPositions(p. 109), PropagateOwnshipToToa(p. 114), UpdateAGTTrack(p. 76), InitializeAGTTrack(p. 73), ReceivePointObstacleReport(p. 78), InitializePassiveTrackFile(p. 17), ReceiveStateVectorUatReport(p. 27)

CONVERTECEFTOWGS84 (Algorithm 3) converts an ECEF position into WGS84 coordinates. At extreme polar regions of the earth, the variable *inner_square_root* may become less than zero. If the variable *inner_square_root* is greater than or equal to zero, the WGS84 coordinates are calculated as described in DO-317B [6]. If the variable *inner_square_root* is less than zero, a simplified calculation of the latitude and altitude is performed. In the simplified calculation, if the ECEF position has some offset from the axis of rotation, then the longitude is found in the usual way; otherwise, the longitude is set to zero. The altitude was originally reported as height above mean sea level (MSL) from a barometric sensor (i.e. orthometric height above MSL), therefore the WGS84 height above ellipsoid (HAE) is transformed back to this coordinate system using CONVERTGEODETICTOORTHOMETRICHEIGHT (Algorithm 5).

This algorithm takes as input *ecef*. This algorithm returns *lla*.

Algorithm 3 ConvertECEFToWGS84

```
function ConvertECEFToWGS84(ecef::Vector{R}, is_alt_geo_hae::Bool)
    e1_sq::R = geoutils.earth_first_eccentricity_sq
    e2_sq::R = geoutils.earth_second_eccentricity_sq
    m2ft::R = geoutils.meters_to_feet
    a::R = geoutils.earth_semimajor_axis
    b::R = geoutils.earth_semiminor_axis
    (Xa::R, Ya::R, Za::R) = ecef
    r_ecef = hypot(Xa, Ya)
    E_sq = a*a - b*b
    F = 54*(b*b)*(Za*Za)
    G = r_ecef*r_ecef + (1-e1_sq)*(Za*Za) - e1_sq*E_sq
    C = (e1_sq*e1_sq)*F*(r_ecef*r_ecef)/(G*G*G)
    S = (1 + C + sqrt(C*C+2*C))^(1.0/3.0)
    P = F/(3*((S+(1/S)+1)^2)*(G*G))
    Q = sqrt(1+2*(e1_sq*e1_sq)*P)
    inner_square_root = ((a*a)/2)*(1+(1/Q)) - (P*(1-e1_sq)*(Za*Za))/(Q*(1+Q)) - P*(r_ecef*r_ecef)/2
    if (inner_square_root >= 0)
        r_0 = -(P*e1_sq*r_ecef)/(1+Q) + sqrt(inner_square_root)
        U = hypot(r_ecef - e1_sq*r_0, Za)
        V = sqrt((r_ecef-e1_sq*r_0)^2 + (1-e1_sq)*(Za*Za))
        Z_0 = ((b*b)*Za)/(a*V)
        lat = atan(Za+e2_sq*Z_0, r_ecef)
        lon = atan(Ya, Xa)
        h = U*(1 - (b*b)/(a*V))
    else
        lat = atan(Za,r_ecef)
        if(r_ecef > 0)
            lon = atan(Ya, Xa)
        else
            lon = 0.0
        end
        h = abs(Za) - b
    end
    if !is_alt_geo_hae
        h = ConvertGeodeticToOrthometricHeight(p. 15)(h, lat)
    end
    h *= m2ft
    lla::Vector{R} = [lat, lon, h]
    return lla::Vector{R}
end
```

Referenced In: RefineECEFAndLLA(p. 25)

CONVERTORTHOMETRICTOGEODETICHEIGHT (Algorithm 4) converts an orthometric altitude to a geodetic height based on the provided latitude.

The algorithm takes as inputs the orthometric altitude observation of an aircraft (h_o), and the latitude coordinate of an aircraft (phi). The algorithm outputs the geodetic height of an aircraft (geo_h).

Algorithm 4 ConvertOrthometricToGeodeticHeight

```
function ConvertOrthometricToGeodeticHeight(h_o::R, phi::R)
    gamma_s_45::R = geoutils.earth_gravity_lat45
    R_phi::R       = GetEarthRadiusAtLatitude(p. 15)(phi)
    gamma_s_phi::R = GetNormalGravityAtLatitude(p. 16)(phi)
    geo_h::R = R_phi * h_o / ((R_phi * gamma_s_phi / gamma_s_45) - h_o)
    return geo_h::R
end
```

Referenced In: ConvertWGS84ToECEF(p. 13)

CONVERTGEODETICTOORTHOMETRICHEIGHT (Algorithm 5) converts a geodetic height to an orthometric altitude based on the provided latitude.

The algorithm takes as inputs the geodetic height of an aircraft (*h_g*), and the latitude coordinate of an aircraft (*phi*). The algorithm outputs the orthometric altitude of an aircraft (*ortho_h*).

Algorithm 5 ConvertGeodeticToOrthometricHeight

```
function ConvertGeodeticToOrthometricHeight(h_g::R, phi::R)
    gamma_s_45::R = geoutils.earth_gravity_lat45
    R_phi::R       = GetEarthRadiusAtLatitude(p. 15)(phi)
    gamma_s_phi::R = GetNormalGravityAtLatitude(p. 16)(phi)
    ortho_h::R = R_phi * (gamma_s_phi/gamma_s_45) / (R_phi/h_g + 1)
    return ortho_h::R
end
```

Referenced In: ConvertECEFToWGS84(p. 14)

GETEARTHRADIUSATLATITUDE (Algorithm 6) uses the provided latitude to calculate and return the radius of the earth at a given latitude, considering the oblate spheroid nature of the earth.

The algorithm takes as input the latitude of an aircraft (*phi*). This algorithm outputs the radius of the earth at the provided latitude (*r*).

Algorithm 6 GetEarthRadiusAtLatitude

```
function GetEarthRadiusAtLatitude(phi::R)
    a::R   = geoutils.earth_semimajor_axis
    f::R   = geoutils.earth_flattening
    m_r::R = geoutils.gravity_ratio
    r = a / (1 + f + m_r - (2*f*(sin(phi)^2)))
    return r::R
end
```

Referenced In: ConvertOrthometricToGeodeticHeight(p. 15), ConvertGeodeticToOrthometricHeight(p. 15)

Gravity increases as an object moves away from the equator towards a pole. GETNORMALGRAVITYATLATITUDE (Algorithm 7) calculates the force of gravity at the provided latitude and returns the value.

The algorithm takes as input the latitude of an aircraft (*phi*). This algorithm outputs the value of gravity at that given latitude (*g*).

Algorithm 7 GetNormalGravityAtLatitude

```
function GetNormalGravityAtLatitude(phi::R)
    e1_sq::R   = geoutils.earth_first_eccentricity_sq
    k_s::R     = geoutils.somiglianas_constant
    gamma_e::R = geoutils.earth_equatorial_gravity
    g = gamma_e * (1+(k_s*(sin(phi)^2))) / sqrt(1 - (e1_sq*(sin(phi)^2)))
    return g::R
end
```

Referenced In: ConvertOrthometricToGeodeticHeight(p. 15), ConvertGeodeticToOrthometricHeight(p. 15)

ASSOCIATEADSBREPORTTOTARGET (Algorithm 8) attempts to match a provided Mode S address to an existing ADS-B or ADS-R Track File. If a matching Track File is found, the Target ID is returned; otherwise, *NO_TARGET_FOUND* is returned.

The algorithm takes as inputs a 24-bit address (*mode_s*), a flag that states whether the source message is an ADS-R rebroadcast (*rebroadcast*), and a flag that states whether the address is an anonymous non-ICAO address (*non_icao*). The algorithm outputs the ID of the Target with a matching address.

Algorithm 8 AssociateADSBReportToTarget

```
function AssociateADSBReportToTarget(mode_s::UInt32, non_icao::Bool)
    for id in keys(target_db)
        if (TrackExists(p. 19)(target_db[id].adsb_track))
            if (target_db[id].adsb_track.mode_s == mode_s) && (target_db[id].adsb_track.non_icao == non_icao
                )
                return id::UInt32
            end
        end
    end
    return NO_TARGET_FOUND::UInt32
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), ReceiveStateVectorVelocityReport(p. 22), ReceiveModeStatusReport(p. 29), ReceiveADSBOperationalStatusMessage(p. 105), ReceiveExternallyValidatedADSB(p. 30), ReceiveStateVectorUatReport(p. 27)

INITIALIZEPASSIVETRACKFILE (Algorithm 9) ensures that the vertical and horizontal tracks are initialized with INITIALIZEVERTICALTRACKER (Algorithm 52) and INITIALIZEPASSIVETRACKERPOSITION (Algorithm 35) respectively. If *trk.valid_vert* is false, the altitude component of *trk.lla_at_hor_toa* is set to zero. Since *trk.lla_at_hor_toa* may change here, the corresponding ECEF position must then be set with a call to CONVERTWGS84TOECEF. Additional data elements (some of which are not available with an ADS-B Position Report and must be initialized from the parameters file) are stored in the ADSBTRACKFILE either directly or when passed to the initialization functions. These elements include the accuracy and integrity of the reported position and velocity (*init_NACp*, *init_NACv*, and *init_SIL*), and the time of applicability for the horizontal and vertical tracks, as well as the time of last position report reception (*toa*).

This algorithm takes as input *trk*, *obs_lla*, *is_alt_geo_hae*, *nic* and *toa*. This algorithm updates *trk*.

Algorithm 9 InitializePassiveTrackFile

```
function InitializePassiveTrackFile(trk::Union{ADSBTrackFile(p. 485), V2VTrackFile(p. 488)}, obs_lla::Vector{R},
    is_alt_geo_hae::Bool, nic::UInt32, toa::R)
    is_v2v = (typeof(trk) == V2VTrackFile(p. 488))
    if is_v2v
        mhod::Z = params().surveillance.horizontal_v2v.max_outlier_detections
        if is_alt_geo_hae
            mvod::Z = params().surveillance.vertical.hae_small_intruder.max_outlier_detections
        else
            mvod = params().surveillance.vertical.pres_small_intruder.max_outlier_detections
        end
    else
        mhod = params().surveillance.horizontal_adsb.max_outlier_detections
        if is_alt_geo_hae
            mvod = params().surveillance.vertical.hae_large_intruder.max_outlier_detections
        else
            mvod = params().surveillance.vertical.pres_large_intruder.max_outlier_detections
        end
    end
    is_large::Bool = true
    if is_v2v
        is_large = (trk.classification != CLASSIFICATION_SMALL_UNMANNED)
    end
    InitializeVerticalTracker(p. 51)(trk, obs_lla[3], is_alt_geo_hae, is_large)
    trk.odc_vert = mvod
    trk.valid_vert = (trk.updates_vert > 0)
    (trk.mu_hor, trk.Sigma_hor) = InitializePassiveTrackerPosition(p. 37)(is_v2v)
    trk.odc_hor = mhod
    trk.updates_pos = 1
    trk.lla_at_hor_toa = obs_lla
    if !trk.valid_vert
        trk.lla_at_hor_toa[3] = 0.0
    end
    trk.ecef_at_hor_toa = ConvertWGS84ToECEF(p. 13)(trk.lla_at_hor_toa, trk.alt_src_hae)
    trk.nic = nic
    trk.toa = trk.toa_hor = trk.toa_vert = trk.toa_pos_update = toa
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), ReceiveStateVectorV2VReport(p. 33), ReceiveStateVectorUatReport(p. 27)

ADDADSBTRACKTODB (Algorithm 10) takes an intruder's ADS-B track and adds it to the intruder database. Since the database can only have one intruder per id, the algorithm first checks to see if the intruder exists within the database before adding the track. There are no outputs of the algorithm.

Algorithm 10 AddADSBTrackToDB

```
function AddADSBTrackToDB(trk::ADSBTrackFile(p. 485))
    id = AssociateICAOtoTarget(p. 18)(trk.mode_s, trk.non_icao)
    if (id == NO_TARGET_FOUND)
        tgt = Target(p. 493)()
        tgt.init_time = trk.toa
        tgt.adsb_track = trk
        AddToDB(p. 19)(target_db, tgt)
    else
        target_db[id].adsb_track = trk
    end
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), ReceiveStateVectorUatReport(p. 27)

ASSOCIATEICAOTOTARGET (Algorithm 11) searches in the target database for any target containing a track file associated with the given address *icao_address* and whether the address is an ICAO or anonymous address *non_icao*. If a matching track file is found, the algorithm returns the target ID. If no target is found, the algorithm returns *NO_TARGET_FOUND*.

This algorithm takes as input *icao_address* and *non_icao*. This algorithm returns *id* or *NO_TARGET_FOUND*.

Algorithm 11 AssociateICAOtoTarget

```
function AssociateICAOtoTarget(icao_address::UInt32, non_icao::Bool)
    for id in keys(target_db)
        if (TrackExists(p. 19)(target_db[id].adsb_track))
            if (target_db[id].adsb_track.mode_s == icao_address) && (target_db[id].adsb_track.non_icao ==
                non_icao)
                return id::UInt32
            end
        end
    end
    return NO_TARGET_FOUND::UInt32
end
```

Referenced In: AddADSBTrackToDB(p. 18)

A TARGET can store multiple types of Track Files given different surveillance sources. Multiple Ground Surveillance Track Files can be associated to a TARGET, but only a single Mode S Track File, ADS-B Track File, ADS-R Track File, V2V Track File, ORNCT Track File, AGT Track File, and Point Obstacle Track File can be stored. Because of this, the TRACKEXISTS (Algorithm 12) can handle different inputs in order to properly check if a given Track exists or not. TRACKEXISTS takes in as input a track, which can either be a vector, a TRACKFILE, or *Nothing*. If the input track is a vector, then a boolean value of TRUE is returned only if the vector of tracks is not empty. Otherwise, a boolean value of TRUE is returned only if the track does not evaluate to *nothing*.

This algorithm takes as input *trk*. This algorithm returns either TRUE or FALSE.

Algorithm 12 TrackExists

```
function TrackExists(trk::Union{Vector, TrackFile(p. 483), Nothing})
    if typeof(trk) <: Vector
        return !isempty(trk)
    else
        return (trk != nothing)
    end
end
```

Referenced In: RemoveDecorrelatedTrackFromTarget(p. 459), SetClassification(p. 136), MergeTargets(p. 464), TrackSourceSelection(p. 121), AssociateADSBReportToTarget(p. 16), RefineOwnshipAGL(p. 107), AssociateV2VToTarget(p. 35), GetAircraftTracks(p. 440), PassiveQualityCheck(p. 127), AssociateV2VToV2VAndAGTTargets(p. 100), ReceiveWgs84Observation(p. 92), AddTracksToReport(p. 119), AssociateORNCTIdToTarget(p. 61), AssociatePointObstacleReportToTarget(p. 78), AssociateICAOtoTarget(p. 18), SetIntruderIDDirectory(p. 137), SetAvailableSources(p. 147), GetPreviouslySelectedTrack(p. 130), OptimalPreTrackedTracks(p. 122), FilterTracksForTRM(p. 132), TargetIsEmpty(p. 161), RemoveStaleTracks(p. 159), AGTOnlyTarget(p. 453), GenerateLeadTrackList(p. 460)

The ADDTODB (Algorithm 13) takes a database *db* and a value *v* as input. The database is modified by the addition of the value and incrementing the value counter. The modified database is returned.

Algorithm 13 AddToDB

```
function AddToDB(db::Database(p. 494){K,V}, v::V) where {K,V}
    db.increment += 1
    while haskey(db, db.increment)
        db.increment += 1
    end
    db[db.increment] = v
    return db::Database(p. 494){K,V}
end
```

Referenced In: AddPointObstacleTrackToDB(p. 79), AddORNCTTrackToDB(p. 64), DecorrelateTargetsFromOwnship(p. 451), AddV2VTrackToDB(p. 35), AddAGTTrackToDB(p. 72), AddADSBTrackToDB(p. 18), DecorrelateTargets(p. 457)

The RETRIEVEWITHID (Algorithm 14) searches a given database (input *a*) for an key-value pair with key matching the input *k*. If a key-value pair is identified, then the value is returned (*V*), otherwise, *nothing* is returned.

Algorithm 14 RetrieveWithID

```
function RetrieveWithID(a::Database(p. 494){K,V}, k::K) where {K,V}
    v = get(a, k, nothing)
    return v::Target(p. 493)
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), SetClassification(p. 136), SetCoordination(p. 104), ReceiveStateVectorV2VReport(p. 33), ReceiveExternallyValidatedV2V(p. 36), StmHousekeepingPrioritization(p. 438), DeleteIntent(p. 100), ReceiveAbsoluteGeodeticTrack(p. 68), ReceiveV2VOperationalStatusMessage(p. 103), ReceiveStateVectorVelocityReport(p. 22), ReceiveModeStatusReport(p. 29), ReceiveADSBOperationalStatusMessage(p. 105), DeleteAGTTrack(p. 74), ReceivePointObstacleReport(p. 78), ReceiveExternallyValidatedADSB(p. 30), ReceiveV2VCoordination(p. 99), ReceiveStateVectorUatReport(p. 27)

UPDATEALTINHORGEOPOS (Algorithm 15) is used to update the altitude component of the horizontal tracker's global geodetic position whenever the vertical tracker has undergone a major change (e.g. changing to valid or invalid, or changing altitude to HAE or pressure altitude). If the track's vertical tracker is valid, indicated by *trk.valid_vert*, then the updated altitude is generated with a constant velocity prediction equation based on the vertical tracker's mean states, otherwise, the updated altitude is set equal to zero. After the LLA is updated with the new altitude, the corresponding ECEF is updated with a call to CONVERTWGS84TOECEF.

This algorithm takes as input *trk*. This algorithms updates *trk.lla_at_hor_toa* and *trk.ecef_at_hor_toa*.

Algorithm 15 UpdateAltInHorGeoPos

```
function UpdateAltInHorGeoPos(trk::TrackFile(p. 483))
    if trk.valid_vert
        dt_hor_toa = trk.toa_hor - trk.toa_vert
        trk.lla_at_hor_toa[3] = trk.mu_vert[1] + trk.mu_vert[2] * dt_hor_toa
    else
        trk.lla_at_hor_toa[3] = 0.0
    end
    trk.ecef_at_hor_toa = ConvertWGS84ToECEF(p. 13)(trk.lla_at_hor_toa, trk.alt_src_hae)
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), ReceiveStateVectorV2VReport(p. 33), ReceiveStateVectorUatReport(p. 27)

GETALTFORHORTRKOBSERVATION (Algorithm 16) is used to generate a temporary altitude value to be used as a pseudo altitude observation in combination with the observed latitude and longitude used to update the passive horizontal trackers. This is necessary to ensure the horizontal tracker always uses a valid altitude and consistent altitude type during outlier detection of the horizontal tracker and whenever the horizontal tracker is re-initialzed. If the track's vertical tracker is valid, indicated by *trk.valid_vert*, then the pseudo altitude observation is generated with a constant velocity prediction equation based on the vertical tracker's mean states, otherwise, the pseudo altitude observation is set equal to the previous altitude stored in *trk.lla_at_hor_toa*.

This algorithm takes as input *trk*, and *toa_obs*. This algorithms returns *alt_ft_toa*.

Algorithm 16 GetAltForHorTrkObservation

```
function GetAltForHorTrkObservation(trk::Union{ADSBTrackFile(p. 485), V2VTrackFile(p. 488), AGTTrackFile(p. 486)}, toa_obs
    ::R)
    if trk.valid_vert
        dt_vert = toa_obs - trk.toa_vert
        alt_ft_toa = trk.mu_vert[1] + trk.mu_vert[2] * dt_vert
    else
        alt_ft_toa = trk.lla_at_hor_toa[3]
    end
    return alt_ft_toa::R
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), ReceiveStateVectorV2VReport(p. 33), ReceiveStateVectorVelocityReport(p. 22), UpdateAGTTrack(p. 76), ReceiveStateVectorUatReport(p. 27)

2.2.1.1.2 State Vector Velocity Report

Another entry point to the STM for passive surveillance is defined by RECEIVESTATEVECTORVELOCITYREPORT (Algorithm 17) with inputs defined in Table 10. The velocity information included in the report is ground velocity of the broadcasting aircraft (in North/South and East/West components). When a State Vector Velocity Report is received by the STM, the algorithm first ensures that the reported velocity components are valid (i.e, not a *NaN*). Additionally, reports that are rebroadcasted are not processed. Next, the reported Mode S address (*mode_s*), and the *non_icao* flag are used to associate the report to an existing Target (ASSOCIATEADSBREPORTTOTARGET (Algorithm 8)). If the association attempt is not successful, the algorithm ends. This implies that the STM will only initialize a new ADSBTRACKFILE using a State Vector Position Report.

If the association attempt is successful, the corresponding ADSBTRACKFILE is retrieved from the appropriate memory location. The track will only be updated if the existing track was not based on UAT reports as indicated by *uat*.

The intruder's altitude at *toa* (*alt_ft_toa*) is first generated with a call to GETALTFORHORTRKOBSERVATION (Algorithm 16) and then passed to PROPAGATEGEODETICTOTOA (Algorithm 18) which will propagate the geodetic LLA and ECEF positions to the desired TOA. The predicted geodetic states are then used in POLARALIGNMENTHORIZONTALROTATION (Algorithm 24) along with the previous LLA and ECEF location that the horizontal track was defined relative to, to create a horizontal rotation matrix. This rotation matrix is then used to rotate the reported ENU velocities from the observation frame (LLA/ECEF of intruder at observation TOA) to the horizontal track frame (LLA/ECEF of intruder at the previous horizontal track TOA). The ADS-B Track File, *trk*, is then advanced forward using ADVANCEPASSIVETRACKVELOCITY (Algorithm 42).

This algorithm updates *trk*.

Table 10. STM Input Variables - State Vector Velocity Report

Variable	Units	Type	Description
vel_ew	knots	real	Intruder velocity in east-west direction, true east is positive
vel_ns	knots	real	Intruder velocity in north-south direction, true north is positive
mode_s	N/A	uint32	Intruder 24-bit aircraft address
nic	N/A	uint32	Intruder Navigation Integrity Category (NIC)
rebroadcast	N/A	bool	Flag to indicate message received from ADS-R (not an allowed source for sXu)
non_icao	N/A	bool	Flag to indicate that the address is a non-ICAO 24-bit aircraft address (anonymous address)
toa	seconds	real	Time of applicability

Algorithm 17 ReceiveStateVectorVelocityReport

```
function ReceiveStateVectorVelocityReport(vel_ew::R, vel_ns::R, mode_s::UInt32, nic::UInt32, rebroadcast::Bool,
    non_icao::Bool, toa::R)
    kts2mps::R = geoutils.kts_to_mps
    if !isnan(vel_ew) && !isnan(vel_ns) && !rebroadcast
        id = AssociateADSBReportToTarget(p. 16)(mode_s, non_icao)
        if (id != NO_TARGET_FOUND)
            tgt = RetrieveWithID(p. 19)(target_db, id)
            trk = tgt.adsb_track
            if (!trk.uat)
                alt_ft_toa = GetAltForHorTrkObservation(p. 20)(trk, toa)
                dt_hor = toa - trk.toa_hor
                geodetic_states_pred = PropagateGeodeticToToa(p. 23)(trk.lla_at_hor_toa, alt_ft_toa, trk.
                    alt_src_hae, trk.mu_hor[2], trk.mu_hor[4], trk.mu_vert[2], dt_hor)
                obs_lla_pred = geodetic_states_pred.lla_rad_ft
                obs_ecef_pred = geodetic_states_pred.ecef_m
                R_obs_to_trk = PolarAlignmentHorizontalRotation(p. 25)(obs_lla_pred, obs_ecef_pred, trk.
                    lla_at_hor_toa, trk.ecef_at_hor_toa)
                (obs_dx, obs_dy) = R_obs_to_trk * [vel_ew, vel_ns] * kts2mps
                AdvancePassiveTrackVelocity(p. 43)(trk, obs_dx, obs_dy, nic, toa, obs_lla_pred[1])
            end
        end
    end
end
```

PROPAGATEGEODETICTOTOA (Algorithm 18) receives a set of absolute geodetic positions (LLA/ECEF), along with a set of ENU velocities and propagates these states forward in time indicated by *dt* and returns the new LLA/ECEF positions and the ENU velocities defined in a new local ENU coordinate frame.

If the *dt* is too small or is a NaN, the outputted LLA is set equal to the inputted LLA and the ECEF position is either set by a call to CONVERTWGS84TOECEF or set to NaN's. The horizontal velocities are also set equal to the inputted velocities.

Otherwise, if *dt* is a valid number, the LLA is first converted to ECEF. Then a pseudo altitude rate is calculated using the new and previous altitudes and the time step *dt*. Then the three dimensional ENU velocities are converted to ECEF velocities by a call to ROTATEENUTOECEF. Next, the ECEF velocity is used to propagate the ECEF position forward by the specified time step *dt*. A refinement of the new ECEF position is then performed along with the generation of the corresponding LLA position by a call to REFINEECEFANDLLA. A horizontal rotation matrix is then created by a call to POLARALIGNMENTHORIZONTALROTATION, which will rotate between two sets of LLA/ECEF coordinates. This rotation matrix is then used to rotate the horizontal ENU velocities into the new local ENU coordinate frame defined at the new LLA/ECEF position.

The propagated states are then all added to an empty GEODETICSTATES.

This algorithm takes as input *lla_rad_ft*, *alt_ft_toa*, *is_alt_geo_hae*, *vel_ew_mps*, *vel_ns_mps*, *alt_rate_fps*, and *dt*. This algorithm returns *geodetic_states_at_toa*.

Algorithm 18 PropagateGeodeticToToa

```
function PropagateGeodeticToToa(lla_rad_ft::Vector{R}, alt_ft_toa::R, is_alt_geo_hae::Bool, vel_ew_mps::R,
     vel_ns_mps::R, alt_rate_fps::R, dt::R)
    min_extrap_toa_step::R = params().surveillance.min_extrap_toa_step
    m2ft::R = geoutils.meters_to_feet
    if (dt < min_extrap_toa_step) || isnan(dt)
        lla_rad_ft_toa = lla_rad_ft
        lla_rad_ft_toa[3] = alt_ft_toa
        if !isnan(dt)
            ecef_m_toa = ConvertWGS84ToECEF(p. 13)(lla_rad_ft_toa, is_alt_geo_hae)
        else
            ecef_m_toa = [NaN, NaN, NaN]
        end
        vel_ew_mps_toa = vel_ew_mps
        vel_ns_mps_toa = vel_ns_mps
    else
        ecef_m = ConvertWGS84ToECEF(p. 13)(lla_rad_ft, is_alt_geo_hae)
        temp_alt_rate_fps = (alt_ft_toa - lla_rad_ft[3]) / dt
        enu_vel_mps = [vel_ew_mps, vel_ns_mps, temp_alt_rate_fps / m2ft]
        ecef_vel_mps = RotateENUToECEF(p. 24)(enu_vel_mps, lla_rad_ft)
        ecef_m_toa = ecef_m + ecef_vel_mps*dt
        (ecef_m_toa, lla_rad_ft_toa) = RefineECEFAndLLA(p. 25)(ecef_m_toa, alt_ft_toa, is_alt_geo_hae)
        R_0_to_1 = PolarAlignmentHorizontalRotation(p. 25)(lla_rad_ft, ecef_m, lla_rad_ft_toa, ecef_m_toa)
        (vel_ew_mps_toa, vel_ns_mps_toa) = R_0_to_1 * [vel_ew_mps, vel_ns_mps]
    end
    geodetic_states_at_toa = GeodeticStates(p. 492)()
    geodetic_states_at_toa.lla_rad_ft = lla_rad_ft_toa
    geodetic_states_at_toa.ecef_m = ecef_m_toa
    geodetic_states_at_toa.vel_ew_mps = vel_ew_mps_toa
    geodetic_states_at_toa.vel_ns_mps = vel_ns_mps_toa
    geodetic_states_at_toa.alt_rate_fps = alt_rate_fps
    geodetic_states_at_toa.is_alt_geo_hae = is_alt_geo_hae
    return geodetic_states_at_toa::GeodeticStates(p. 492)
end
```

Referenced In: PropagateOwnshipToToa(p. 114), ReceiveStateVectorVelocityReport(p. 22)

ROTATEECEFTOENU (Algorithm 19) redefines a position from ECEF to ENU by applying a three dimensional rotation *R_0* by calling ECEFTOENUROTATION (Algorithm 22) with the provided latitude (*phi*) and longitude (*lambda*).

This algorithm takes as input *ecef* (defined as [*X*, *Y*, *Z*]), and *lla* (defined as [*phi*, *lambda*, *altitude*]). This algorithm returns *enu* (defined as [*ew*, *ns*, *ud*]).

Algorithm 19 RotateECEFToENU

```
function RotateECEFToENU(ecef::Vector{R}, lla::Vector{R})
    (phi::R, lambda::R, nothing) = lla
    R_0 = ECEFToENURotation(p. 24)(phi, lambda)
    enu::Vector{R} = R_0 * ecef
    return (enu::Vector{R})
end
```

Referenced In: AdvancePassiveTrackState(p. 45), RefineOwnshipAGL(p. 107), AbsoluteGeodeticToOwnshipRelative(p. 141), AddPointObstacleTrackToReport(p. 157), AdvancePassiveTrackPosition(p. 39), UpdateAGTTrack(p. 76), CalculateOwnshipVelocityOffsets(p. 94), PolarAlignmentHorizontalRotation(p. 25)

ROTATEENUTOECEF (Algorithm 20) redefines a position from ENU to ECEF by applying a three dimensional rotation *R_0* by calling ENUTOECEFROTATION (Algorithm 21) with the provided latitude (*phi*) and longitude (*lambda*) from the *lla* vector.

This algorithm takes as input *enu* and *lla* vectors. This algorithm returns the *ecef* vector.

Algorithm 20 RotateENUToECEF

```
function RotateENUToECEF(enu::Vector{R}, lla::Vector{R})
    (phi::R, lambda::R, nothing) = lla
    R_0 = ENUToECEFRotation(p. 24)(phi, lambda)
    ecef::Vector{R} = R_0 * enu
    return (ecef::Vector{R})
end
```

Referenced In: ReCenterHorizontalTrackLocation(p. 42), PropagateGeodeticToToa(p. 23), CalculateOwnshipVelocity-Offsets(p. 94)

ENUTOECEFROTATION (Algorithm 21) takes as input the latitude (*phi*) and longitude (*lambda*) and returns a rotation matrix *R0* that redefines the ENU position to ECEF.

Algorithm 21 ENUToECEFRotation

```
function ENUToECEFRotation(phi::R, lambda::R)
    s_phi = sin(phi)
    c_phi = cos(phi)
    s_lambda = sin(lambda)
    c_lambda = cos(lambda)
    R0::Matrix{R} = [-s_lambda -s_phi*c_lambda c_phi*c_lambda;
                      c_lambda -s_phi*s_lambda c_phi*s_lambda;
                      0         c_phi          s_phi]
    return R0::Matrix{R}
end
```

Referenced In: ECEFToENURotation(p. 24), RotateENUToECEF(p. 24)

ECEFTOENUROTATION (Algorithm 22) utilizes ENUTOECEFROTATION to redefine the ECEF position to ENU using the transpose of the resulting *R0* rotation matrix.

This algorithm takes as input *phi* and *lambda*. This algorithm returns *R0*.

Algorithm 22 ECEFToENURotation

```
function ECEFToENURotation(phi::R, lambda::R)
    R0::Matrix{R} = ENUToECEFRotation(p. 24)(phi,lambda)
    R0 = R0'
    return (R0::Matrix{R})
end
```

Referenced In: RotateECEFToENU(p. 23)

REFINEECEFANDLLA (Algorithm 23) receives an ECEF position that needs to be refined along

with the correct altitude. The ECEF position is first converted to LLA with a call to CONVERTECEFTOWGS84, then the altitude is replaced with the inputted altitude. Then the new LLA is converted back to ECEF with a call to CONVERTWGS84TOECEF.

This algorithm takes as input *ecef_m*, *alt_ft_toa*, and *is_alt_geo_hae*. This algorithm returns *ecef_m*, and *lla_rad_ft*.

Algorithm 23 RefineECEFAndLLA

```
function RefineECEFAndLLA(ecef_m::Vector{R}, alt_ft_toa::R, is_alt_geo_hae::Bool)
    lla_rad_ft = ConvertECEFToWGS84(p. 14)(ecef_m, is_alt_geo_hae)
    lla_rad_ft[3] = alt_ft_toa
    ecef_m = ConvertWGS84ToECEF(p. 13)(lla_rad_ft, is_alt_geo_hae)
    return (ecef_m::Vector{R}, lla_rad_ft::Vector{R})
end
```

Referenced In: ReCenterHorizontalTrackLocation(p. 42), PropagateGeodeticToToa(p. 23)

POLARALIGNMENTHORIZONTALROTATION (Algorithm 24) will create a rotation matrix, which will rotate between two sets of LLA/ECEF coordinates.

First, the difference between both ECEF positions is calculated. Then *delta_ecef* is converted to a delta ENU defined at both LLA positions. Then both sets of delta ENU positions are used to calculate two sets of track angles. The difference between these two track angles is then calculated and passed to WRAPTOPI. This *rotation_angle* is then used to create the final rotation matrix.

This algorithm takes as input *lla_0*, *ecef_0*, *lla_1*, and *ecef_1*. This algorithm returns *R_0_to_1*.

Algorithm 24 PolarAlignmentHorizontalRotation

```
function PolarAlignmentHorizontalRotation(lla_0::Vector{R}, ecef_0::Vector{R}, lla_1::Vector{R}, ecef_1::Vector{
    R})
    delta_ecef = ecef_1 - ecef_0
    delta_enu_0 = RotateECEFToENU(p. 23)(delta_ecef, lla_0)
    delta_enu_1 = RotateECEFToENU(p. 23)(delta_ecef, lla_1)
    track_angle_0 = atan(delta_enu_0[1], delta_enu_0[2])
    track_angle_1 = atan(delta_enu_1[1], delta_enu_1[2])
    rotation_angle = WrapToPi(p. 583)(track_angle_1 - track_angle_0)
    s_rotation_angle = sin(rotation_angle)
    c_rotation_angle = cos(rotation_angle)
    R_0_to_1::Matrix{R} = [c_rotation_angle -s_rotation_angle;
                           s_rotation_angle  c_rotation_angle]
    return R_0_to_1::Matrix{R}
end
```

Referenced In: RedefineEstimateInRotatedFrame(p. 42), PropagateGeodeticToToa(p. 23), ReceiveStateVectorV2VReport(p. 33), ReceiveStateVectorVelocityReport(p. 22), UpdateAGTTrack(p. 76), ReceiveStateVectorUatReport(p. 27)

2.2.1.1.3 Universal Access Transceiver Report

ADS-B state vector information transmitted through a Universal Access Transceiver (UAT) system is processed through RECEIVESTATEVECTORUATREPORT (Algorithm 25). The required inputs

are defined in Table 11. The state vector report as transmitted by a UAT equipped aircraft contains all the relevant information found separately in the ADS-B state vector position and velocity reports transmitted by aircraft not equipped with an UAT system. The processing is therefore a consolidation of that done for the state vector position and velocity reports, but with some simplifications afforded by receiving the position and velocity information concurrently.

Upon the reception of an ADS-B State Vector UAT Report, the STM first checks if the reported latitude, longitude, and velocities are valid (i.e, not a *NaN*). The reported Mode S address (*mode_s*) and *non_icao* flag are used to associate the report to an existing Target (AssociateADSBReportToTarget (Algorithm 8)).

The position information contained within the report (*lat*, *lon* and *alt*) is combined into a 3D position vector *obs_lla*, where the angles are converted to radians.

If the State Vector UAT Report is not associated with an existing Target, a new ADSBTrackFile is created and its horizontal and vertical tracks are initialized with InitializePassiveTrackFile (Algorithm 9). The velocities *dx_rel* and *dy_rel* received in the UAT report are used to immediately initialize the track velocities. Next, the new ADSBTrackFile is stored in memory through AddADSBTrackToDB (Algorithm 10). This algorithm attempts to correlate the new ADSBTrackFile to an existing Target (AssociateICAOtoTarget (Algorithm 11)). If correlation is successful, the ADSBTrackFile is added to the existing Target. Otherwise, the ADSBTrackFile is added to a new Target.

If the call to AssociateADSBReportToTarget successfully associates the report to a Target, the corresponding ADSBTrackFile is retrieved from memory. The track will only be updated if the existing track was based on UAT reports as indicated by *uat*. If a minimum allowed time (*min_obs_toa_step*) has not passed since the last track update, the ADSBTrackFile is not updated with the State Vector UAT Report. If the minimum time threshold has passed, the *last_valid_vert* and *last_alt_src_hae* variables are saved off prior to calling AdvanceVerticalTrack (Algorithm 53). If the *valid_vert* Boolean has changed or the altitude type saved in the vertical tracker has changed, the altitude within the absolute geodetic LLA and ECEF positions of the horizontal tracker are updated with a call to UpdateAltInHorGeoPos (Algorithm 15). Next, since the original *obs_lla* vector used in InitializePassiveTrackFile (Algorithm 9) may contain an invalid altitude, this altitude observation is modified to use the altitude returned by GetAltForHorTrkObservation (Algorithm 16). Then *obs_ecef* is calculated by a call to ConvertWGS84ToECEF.

The observed geodetic states are then used in PolarAlignmentHorizontalRotation (Algorithm 24) along with the previous LLA and ECEF location that the horizontal track was defined relative to, to create a horizontal rotation matrix. This rotation matrix is then used to rotate the reported ENU velocities from the observation frame (LLA/ECEF of intruder at observation TOA) to the horizontal track frame (LLA/ECEF of intruder at the previous horizontal track TOA).

The horizontal tracker is then advanced by a call to AdvancePassiveTrackState (Algorithm 43).

This algorithm takes as inputs the variables in Table 11. This algorithm updates *trk*.

Table 11. STM Input Variables - State Vector UAT Report

Variable	Units	Type	Description
lat	degrees	real	Intruder latitude
lon	degrees	real	Intruder longitude
alt	feet	real	Intruder altitude (nominally pressure altitude)
is_alt_geo_hae	N/A	bool	Intruder reporting geodetic height-above-ellipsoid (HAE) altitude
vel_ew	knots	real	Intruder velocity in east-west direction, true east is positive
vel_ns	knots	real	Intruder velocity in north-south direction, true north is positive
mode_s	N/A	uint32	Intruder 24-bit aircraft address
nic	N/A	uint32	Intruder Navigation Integrity Category (NIC)
q_int	feet	uint32	Intruder altitude quantization (25 or 100)
non_icao	N/A	bool	Flag to indicate that the address is a non-ICAO 24-bit aircraft address (anonymous address)
toa	seconds	real	Time of applicability

Algorithm 25 ReceiveStateVectorUatReport

```
function ReceiveStateVectorUatReport(lat::R, lon::R, alt::R, is_alt_geo_hae::Bool, vel_ew::R, vel_ns::R, mode_s
    ::UInt32, nic::UInt32, q_int::UInt32, non_icao::Bool, toa::R)
    kts2mps::R = geoutils.kts_to_mps
    min_obs_toa_step::R = params().surveillance.min_obs_toa_step
    if isnan(lat) || isnan(lon) || isnan(vel_ew) || isnan(vel_ns)
        return
    end
    id = AssociateADSBReportToTarget(p. 16)(mode_s, non_icao)
    obs_lla = [deg2rad(lat), deg2rad(lon), alt]
    if (id == NO_TARGET_FOUND)
        trk = ADSBTrackFile(p. 485)(mode_s, non_icao, true)
        InitializePassiveTrackFile(p. 17)(trk, obs_lla, is_alt_geo_hae, nic, toa)
        trk.mu_hor[2] = vel_ew * kts2mps
        trk.mu_hor[4] = vel_ns * kts2mps
        trk.init_velocity = true
        AddADSBTrackToDB(p. 18)(trk)
    else
        tgt = RetrieveWithID(p. 19)(target_db, id)
        trk = tgt.adsb_track
        if (trk.uat)
            dt = toa - trk.toa
            if (dt >= min_obs_toa_step)
                last_valid_vert::Bool = trk.valid_vert
                last_alt_src_hae::Bool = trk.alt_src_hae
                AdvanceVerticalTrack(p. 53)(trk, alt, q_int, is_alt_geo_hae, true, toa)
                if (last_valid_vert != trk.valid_vert) || (last_alt_src_hae != trk.alt_src_hae)
                    UpdateAltInHorGeoPos(p. 20)(trk)
                end
                obs_lla[3] = GetAltForHorTrkObservation(p. 20)(trk, toa)
                obs_ecef = ConvertWGS84ToECEF(p. 13)(obs_lla, trk.alt_src_hae)
                R_obs_to_trk = PolarAlignmentHorizontalRotation(p. 25)(obs_lla, obs_ecef, trk.lla_at_hor_toa, trk.
                    ecef_at_hor_toa)
                (obs_dx, obs_dy) = R_obs_to_trk * [vel_ew, vel_ns] * kts2mps
                AdvancePassiveTrackState(p. 45)(trk, obs_lla, obs_ecef, obs_dx, obs_dy, nic, toa)
            end
        end
    end
end
```

2.2.1.1.4 Mode Status Report

The fourth entry point to the STM for passive surveillance is given by RECEIVEMODESTATUSREPORT (Algorithm 26) with inputs defined in Table 12. The Mode S address and the *non_icao* flag are used to associate the received report with an existing Target. Reports that are rebroadcasted are not processed. If the call to ASSOCIATEADSBREPORTTOTARGET (Algorithm 8) is unsuccessful, the algorithm will not create a new ADSBTRACKFILE, the report is discarded, and the algorithm ends. If association is successful, the corresponding ADSBTRACKFILE is retrieved from the appropriate memory location.

Before updating any of the target or track information, one more check is performed; the *uat* flag is compared to the received *is_uat* flag.

If these two flags match, then the ADSBTRACKFILE is updated with the relevant elements of the Mode Status Report including the ADS-B version (*adsb_version*), the accuracy of the reported horizontal position, horizontal velocity and vertical position data (*nacp*, *nacv*, *gva*), a measure of data integrity (*sil*), and the system design assurance (*sda*).

In the final step of RECEIVEMODESTATUSREPORT, the ADS-B data quality history of the track is updated through a call to UPDATEPASSIVEQUALITYHISTORY.

UPDATEPASSIVEQUALITYHISTORY (Algorithm 27) keeps an updated record of the data quality reported by an intruder so that an M of N check can be performed when the STMREPORT is generated. The current reported quality values are first compared against the minimum requirement values retrieved from the parameter file, and proper initialization of the intruder vertical track is checked. If the length of the history exceeds the maximum length (N), the oldest element is removed from the history. The result of the quality check is then prepended to history data structure. Preserving the most recent N quality values supports a later confirmation in PASSIVEQUALITYCHECK (Algorithm 125) that at least M of those saved values meet the minimum required quality. This is referred to as a "M of N" check throughout this document.

This algorithm updates *trk*.

UPDATEPASSIVEQUALITYHISTORY takes as input *trk*. This algorithm updates *update_quality*.

Table 12. STM Input Variables - Mode Status Report

Variable	Units	Type	Description
adsb_version	N/A	uint32	Intruder ADS-B version
nacp	N/A	uint32	Intruder Navigation Accuracy Category for Position (NACp) (0-11)
nacv	N/A	uint32	Intruder Navigation Accuracy Category for Velocity (NACv) (0-4)
gva	N/A	uint32	Intruder Geometric Vertical Accuracy (0-3)
sil	N/A	uint32	Intruder Source Integrity Level (SIL) (0-3)
sda	N/A	uint32	Intruder System Design Assurance (SDA) (0-3)
mode_s	N/A	uint32	Intruder 24-bit aircraft address
rebroadcast	N/A	bool	Flag to indicate message received from ADS-R
is_uat	N/A	bool	Flag to indicate message received on UAT 978 MHz frequency
non_icao	N/A	bool	Flag to indicate that the address is a non-ICAO 24-bit aircraft address (anonymous address)

Algorithm 26 ReceiveModeStatusReport

```
function ReceiveModeStatusReport(adsb_version::UInt32, nacp::UInt32, nacv::UInt32, gva::UInt32, sil::UInt32,
     sda::UInt32, mode_s::UInt32, rebroadcast::Bool, is_uat::Bool, non_icao::Bool)
    if !rebroadcast
        id = AssociateADSBReportToTarget(p. 16)(mode_s, non_icao)
        if (id != NO_TARGET_FOUND)
            tgt = RetrieveWithID(p. 19)(target_db, id)
            trk = tgt.adsb_track
            if (trk.uat == is_uat)
                trk.adsb_version = adsb_version
                trk.nacp = nacp
                trk.nacv = nacv
                trk.gva = gva
                trk.sil = sil
                trk.sda = sda
                UpdatePassiveQualityHistory(p. 29)(trk)
            end
        end
    end
end
```

Algorithm 27 UpdatePassiveQualityHistory

```
function UpdatePassiveQualityHistory(trk::TrackFile(p. 483))
    if (isa(trk, ADSBTrackFile(p. 485)))
        min_nic = params().surveillance.report_generation.min_adsb_quality.nic
        min_nacp = params().surveillance.report_generation.min_adsb_quality.nacp
        min_nacv = params().surveillance.report_generation.min_adsb_quality.nacv
        min_sil = params().surveillance.report_generation.min_adsb_quality.sil
        min_sda::UInt32 = params().surveillance.report_generation.min_adsb_quality.sda
        min_adsb_version = params().surveillance.report_generation.min_adsb_quality.adsb_version
        N = params().surveillance.report_generation.min_adsb_quality.N
        update_quality = ((trk.nic >= min_nic) && (trk.nacp >= min_nacp) && (trk.nacv >= min_nacv) && (trk.
             sil >= min_sil) && (trk.adsb_version >= min_adsb_version) && (trk.sda >= min_sda))
    elseif (isa(trk, V2VTrackFile(p. 488)))
        min_nic = params().surveillance.report_generation.min_v2v_quality.nic
        min_nacp = params().surveillance.report_generation.min_v2v_quality.nacp
        min_nacv = params().surveillance.report_generation.min_v2v_quality.nacv
        min_sil = params().surveillance.report_generation.min_v2v_quality.sil
        min_sda = params().surveillance.report_generation.min_v2v_quality.sda
        N = params().surveillance.report_generation.min_v2v_quality.N
        update_quality = ((trk.nic >= min_nic) && (trk.nacp >= min_nacp) && (trk.nacv >= min_nacv) && (trk.
             sil >= min_sil) && (trk.sda >= min_sda))
    end
    if (length(trk.passive_qual_history) >= N)
        pop!(trk.passive_qual_history)
    end
    pushfirst!(trk.passive_qual_history,update_quality)
end
```

Referenced In: ReceiveStateVectorV2VReport(p. 33), ReceiveModeStatusReport(p. 29)

2.2.1.1.5 Externally Validated ADSB Report

Another entry point to the STM passive surveillance is given by RECEIVEEXTERNALLYVALIDATEDADSB (Algorithm 28) with inputs defined in Table 13. Raw ADS-B signals may be maliciously or unintentionally "spoofed". This may degrade system safety or cause an operationally unacceptable

amount of alerting. To address this concern, ACAS sXu will at minimum accept an optional external flag for each intruder indicating external ADS-B validation status. This flag indicates whether an upstream source (on- or off-board) has validated the ADS-B data.

RECEIVEEXTERNALLYVALIDATEDADSB first associates the incoming message to a target based on the incoming *mode_s* address. If no target exists with a ADSBTRACKFILE using that *mode_s* address, nothing happens. If a target is found, that target's ADSBTRACKFILE is extracted and *trk.externally_validated* is set based on the incoming *externally_validated* flag.

This algorithm updates *trk*.

Table 13. STM Input Variables - Externally Validated ADSB Report

Variable	Units	Type	Description
externally_validated	N/A	bool	Flag to indicate ADSB intruder has been validated by an external source
mode_s	N/A	uint32	Intruder 24-bit aircraft address
non_icao	N/A	bool	Flag to indicate that the address is a non-ICAO 24-bit aircraft address (anonymous address)

Algorithm 28 ReceiveExternallyValidatedADSB

```
function ReceiveExternallyValidatedADSB(externally_validated::Bool, mode_s::UInt32, non_icao::Bool)
    id = AssociateADSBReportToTarget(p. 16)(mode_s, non_icao)
    if (id != NO_TARGET_FOUND)
        tgt = RetrieveWithID(p. 19)(target_db, id)
        trk = tgt.adsb_track
        trk.externally_validated = externally_validated
    end
end
```

2.2.1.2 Vehicle To Vehicle (V2V)

2.2.1.2.1 V2V Report

ACAS sXu implements an interface supporting a passive, cooperative vehicle-to-vehicle (V2V) surveillance link between small UAS. This interface is expected to be adapted as further development within standards bodies progresses.

State vector information transmitted through a V2V surveillance link is processed through RECEIVESTATEVECTORV2VREPORT (Algorithm 29). The required inputs are defined in Table 14. The state vector report as transmitted by a V2V-equipped intruder contains all the relevant information found separately in the ADS-B State Vector and Mode Status messages. The processing is therefore a consolidation of that done for ADS-B surveillance input and leverages the existing horizontal tracking algorithms for ADS-B.

Upon the reception of a V2V State Vector Report, the STM checks if the reported latitude, longitude, and velocities are valid (i.e, not a *NaN*). Next, the algorithm selects its altitude observation

alt from one of the two sources, namely the pressure altitude (PA), *alt_pres_ft*, or the geometric height above ellipsoid (HAE), *alt_hae_ft*.

The observations from both sources are first examined by a set of validity criteria to ascertain their respective validity (*valid_alt_pres*, *valid_alt_hae*). These criteria are:

- The altitude observation cannot be a *NaN*;
- The altitude observation has to fall between *NARS_THRESHOLD_MIN* and *NARS_THRESHOLD_MAX*; and
- For the HAE to be valid, the reported Vertical Figure of Merit (VFOM), *vfom_m*, and the reported Vertical Protection Level (VPL), *vpl_m*, cannot exceed their respective, pre-determined thresholds (*max_vfom_m*, *max_vpl_m*).

The following criteria are then applied to select one of the observation sources:

1. If only one of the two sources are valid, the valid source is selected;
2. If both sources are invalid, the PA values are selected; and
3. If both sources are valid, the source with a lower degree of altitude uncertainty is selected. For PA, in the absence of a reported uncertainty, the expected uncertainty is derived through a call to GETPRESAEMSIGMA (Algorithm 30). For HAE, this expected uncertainty is derived from VFOM through a call to CONVERTVFOMTOSIGMAVEPU (Algorithm 31).

Once the selection has been made, the Boolean flag *alt_src_hae* is set accordingly.

Next, the reported V2V unique identifier (*v2v_uid*) is used to associate the report to an existing Target (ASSOCIATEV2VTOTARGET (Algorithm 32)).

The position information contained within the report (*lat*, *lon* and *alt*) is combined into a 3D position vector *obs_lla*, where the angles are converted to radians.

If the State Vector V2V Report is not associated with an existing Target, a new V2VTRACKFILE is created and its horizontal and vertical tracks are initialized with INITIALIZEPASSIVETRACKFILE (Algorithm 9). The velocities *vel_ew* and *vel_ns* received in the V2V report are used to immediately initialize the track velocities. Next, the new V2VTRACKFILE is stored in memory through ADDV2VTRACKTODB (Algorithm 33).

If the call to ASSOCIATEV2VTOTARGET successfully associates the report to a Target, the corresponding V2VTRACKFILE is retrieved from memory. If a minimum allowed time (*min_obs_toa_step*) has not passed since the last track update, the V2VTRACKFILE is not updated with the State Vector V2V Report. If the minimum time threshold has passed, the *last_valid_vert* and *last_alt_src_hae* variables are saved off prior to calling ADVANCEVERTICALTRACK (Algorithm 53). If the *valid_vert* Boolean has changed or the altitude type saved in the vertical tracker has changed, the altitude within the absolute geodetic LLA and ECEF positions of the horizontal tracker are updated with a call to UPDATEALTINHORGEOPOS (Algorithm 15). Next, since the original *obs_lla* vector used in INITIALIZEPASSIVETRACKFILE (Algorithm 9) may contain an invalid altitude, this altitude observation is modified to use the altitude returned by GETALTFORHORTRKOBSERVATION (Algorithm 16). Then *obs_ecef* is calculated by a call to CONVERTWGS84TOECEF.

The observed geodetic states are then used in POLARALIGNMENTHORIZONTALROTATION (Algorithm 24) along with the previous LLA and ECEF location that the horizontal track was defined

relative to, to create a horizontal rotation matrix. This rotation matrix is then used to rotate the reported ENU velocities from the observation frame (LLA/ECEF of intruder at observation TOA) to the horizontal track frame (LLA/ECEF of intruder at the previous horizontal track TOA).

The horizontal tracker is then advanced by a call to ADVANCEPASSIVETRACKSTATE (Algorithm 43).

Additional data elements available in the report are stored in the V2VTRACKFILE.

In the final step of RECEIVESTATEVECTORV2VREPORT, the track's V2V data quality history is updated through a call to UPDATEPASSIVEQUALITYHISTORY (Algorithm 27).

This algorithm takes as inputs the variables in Table 14. This algorithm updates *trk*.

Table 14. STM Input Variables - V2V Report

Variable	Units	Type	Description
lat	degrees	real	Intruder latitude
lon	degrees	real	Intruder longitude
alt_pres_ft	feet	real	Intruder pressure altitude
alt_hae_ft	feet	real	Intruder geodetic height-above-ellipsoid (HAE) altitude
vel_ew	knots	real	Intruder velocity in east-west direction, true east is positive
vel_ns	knots	real	Intruder velocity in north-south direction, true north is positive
nic	N/A	uint32	Intruder Navigation Integrity Category (NIC)
nacp	N/A	uint32	Intruder Navigation Accuracy Category for Position (NACp) (0-11)
nacv	N/A	uint32	Intruder Navigation Accuracy Category for Velocity (NACv) (0-4)
vfom_m	meters	real	Intruder Vertical Figure of Merit (VFOM)
vpl_m	meters	real	Intruder Vertical Protection Level
sil	N/A	uint32	Intruder Geometric Vertical Accuracy (GVA) (0-3)
sda	N/A	uint32	Intruder Source Integrity Level (SIL) (0-3)
v2v_uid	N/A	uint32	Intruder System Design Assurance (SDA) (0-3)
mode_s	N/A	uint32	Intruder V2V unique identifier
mode_s_non_icao	N/A	bool	Intruder 24-bit address
mode_s_valid	N/A	bool	Flag to indicate that the 24-bit address is a non-ICAO address (anonymous address)
classification	N/A	uint8	Flag to indicate that the 24-bit address is valid (populated)
q_int	N/A	uint32	Intruder classification
toa	seconds	real	Intruder altitude quantization

Algorithm 29 ReceiveStateVectorV2VReport

```
function ReceiveStateVectorV2VReport(lat::R, lon::R, alt_pres_ft::R, alt_hae_ft::R, vel_ew::R, vel_ns::R, nic::
    UInt32, nacp::UInt32, nacv::UInt32, vfom_m::R, vpl_m::R, sil::UInt32, sda::UInt32, v2v_uid::UInt32,
    mode_s::UInt32, mode_s_non_icao::Bool, mode_s_valid::Bool, classification::UInt8, q_int::UInt32, toa::R)
    kts2mps::R = geoutils.kts_to_mps
    min_obs_toa_step::R = params().surveillance.min_obs_toa_step
    if isnan(lat) || isnan(lon) || isnan(vel_ew) || isnan(vel_ns)
        return
    end
    if (classification == CLASSIFICATION_SMALL_UNMANNED)
        max_vfom_m::R = params().surveillance.vertical.hae_small_intruder.max_vfom
        max_vpl_m::R = params().surveillance.vertical.hae_small_intruder.max_vpl
        pres_aem_sigma::Vector{R} = params().surveillance.vertical.pres_small_intruder.aem_sigma
    else
        max_vfom_m = params().surveillance.vertical.hae_large_intruder.max_vfom
        max_vpl_m = params().surveillance.vertical.hae_large_intruder.max_vpl
        pres_aem_sigma = params().surveillance.vertical.pres_large_intruder.aem_sigma
    end
    valid_alt_pres::Bool = !isnan(alt_pres_ft) && (alt_pres_ft <= NARS_THRESHOLD_MAX) && (alt_pres_ft >=
        NARS_THRESHOLD_MIN)
    valid_alt_hae::Bool  = !isnan(alt_hae_ft)  && (alt_hae_ft  <= NARS_THRESHOLD_MAX) && (alt_hae_ft  >=
        NARS_THRESHOLD_MIN) && (vfom_m <= max_vfom_m) && (vpl_m <= max_vpl_m)
    pres_alt_bias_sigma::R = GetPresAEMSigma(p. 34)(alt_pres_ft, pres_aem_sigma)
    sigma_vepu_ft::R = ConvertVFOMToSigmaVEPU(p. 35)(vfom_m)
    alt_src_hae::Bool = valid_alt_hae && (!valid_alt_pres || (sigma_vepu_ft < pres_alt_bias_sigma))
    if alt_src_hae
        alt = alt_hae_ft
        mvod::Z = params().surveillance.vertical.hae_small_intruder.max_outlier_detections
    else
        alt = alt_pres_ft
        mvod = params().surveillance.vertical.pres_small_intruder.max_outlier_detections
    end
    id = AssociateV2VToTarget(p. 35)(v2v_uid)
    obs_lla = [deg2rad(lat), deg2rad(lon), alt]
    if (id == NO_TARGET_FOUND)
        trk = V2VTrackFile(p. 488)(v2v_uid, mode_s, mode_s_non_icao, mode_s_valid, classification)
        InitializePassiveTrackFile(p. 17)(trk, obs_lla, alt_src_hae, nic, toa)
        trk.mu_hor[2] = vel_ew * kts2mps
        trk.mu_hor[4] = vel_ns * kts2mps
        trk.init_velocity = true
        AddV2VTrackToDB(p. 35)(trk)
    else
        tgt = RetrieveWithID(p. 19)(target_db, id)
        trk = tgt.v2v_track
        dt = toa - trk.toa
        if (dt >= min_obs_toa_step)
            last_valid_vert::Bool = trk.valid_vert
            last_alt_src_hae::Bool = trk.alt_src_hae
            is_large::Bool = (trk.classification != CLASSIFICATION_SMALL_UNMANNED)
            AdvanceVerticalTrack(p. 53)(trk, alt, q_int, alt_src_hae, is_large, toa)
            if (last_valid_vert != trk.valid_vert) || (last_alt_src_hae != trk.alt_src_hae)
                UpdateAltInHorGeoPos(p. 20)(trk)
            end
            obs_lla[3] = GetAltForHorTrkObservation(p. 20)(trk, toa)
            obs_ecef = ConvertWGS84ToECEF(p. 13)(obs_lla, trk.alt_src_hae)
            R_obs_to_trk = PolarAlignmentHorizontalRotation(p. 25)(obs_lla, obs_ecef, trk.lla_at_hor_toa, trk.
                ecef_at_hor_toa)
            (obs_dx, obs_dy) = R_obs_to_trk * [vel_ew, vel_ns] * kts2mps
            AdvancePassiveTrackState(p. 45)(trk, obs_lla, obs_ecef, obs_dx, obs_dy, nic, toa)
        end
    end
    trk.sigma_vepu = sigma_vepu_ft
    trk.classification = classification
    trk.nacp = nacp
    trk.nacv = nacv
    trk.sil = sil
    trk.sda = sda
    trk.mode_s = mode_s
    trk.mode_s_non_icao = mode_s_non_icao
```

```
    trk.mode_s_valid = mode_s_valid
    UpdatePassiveQualityHistory(p. 29)(trk)
end
```

GETPRESAEMSIGMA (Algorithm 30) extracts the appropriate 1-σ value for the specified pressure altitude, *alt_pres_ft*, based on the *pres_aem_sigma* vector, which characterizes the behavior of a given altimetry error model (AEM).

Each element of the *pres_aem_sigma* represents the 1-σ of the AEM between the corresponding element in the *aem_max_pres_alt* vector and the immediately preceding element. For instance, the second element in *pres_aem_sigmas* is the 1-σ for pressure altitudes between the first and the second elements of *aem_max_pres_alt*. Note that the first element in *pres_aem_sigmas* applies to all pressure altitudes lower than the first element of *aem_max_pres_alt*. The elements of *aem_max_pres_alt* are selected based in accordance with [7, 8].

The algorithm takes as input *alt_pres_ft* and *pres_aem_sigmas*. The algorithm returns *pres_alt_bias_sigma*.

Algorithm 30 GetPresAEMSigma

```
function GetPresAEMSigma(alt_pres_ft::R, pres_aem_sigmas::Vector{R})
    aem_max_pres_alt::Vector{Z} = params().surveillance.vertical.aem_max_pres_alt
    pres_alt_bias_sigma::R = 0
    for i in 1:length(aem_max_pres_alt)
        if (alt_pres_ft < aem_max_pres_alt[i])
            pres_alt_bias_sigma = pres_aem_sigmas[i]
            break
        end
    end
    return pres_alt_bias_sigma::R
end
```

Referenced In: AddAltBiasAndSample(p. 143), ReceiveStateVectorV2VReport(p. 33), AssembleAGTTrack(p. 72), SetOwnshipData(p. 116)

CONVERTVFOMTOSIGMAVEPU (Algorithm 31) converts a Vertical Figure of Merit (VFOM), in meters, into the 1-σ value of the Vertical Estimated Position Uncertainty (VEPU), in feet, as per the procedure established in [5].

The algorithm takes as input *vfom_m*. The algorithm returns the 1-σ value of VEPU, *sigma_vepu_ft*.

Algorithm 31 ConvertVFOMToSigmaVEPU

```
function ConvertVFOMToSigmaVEPU(vfom_m::R)
    m2ft::R  = geoutils.meters_to_feet
    sigma_vepu_ft = vfom_m * m2ft / 1.96
    return sigma_vepu_ft::R
end
```

Referenced In: ReceiveWgs84Observation(p. 92), ReceiveStateVectorV2VReport(p. 33), ConvertGVAToSigmaVEPU(p. 124)

ASSOCIATEV2VTOTARGET (Algorithm 32) iterates through all TARGETs in the *target_db* to identify the V2VTRACKFILE that corresponds to the incoming track, if any, based on the *v2v_uid* field. If a corresponding track is located (i.e. the incoming track associates to the Target thereof), the Target ID (*id*) is returned. Otherwise, *id* will return *NO_TARGET_FOUND*.

Algorithm 32 AssociateV2VToTarget

```
function AssociateV2VToTarget(v2v_uid::UInt32)
    for id in keys(target_db)
        tgt = target_db[id]
        if(TrackExists(p. 19)(tgt.v2v_track))
            trk::V2VTrackFile(p. 488) = tgt.v2v_track
            if (trk.v2v_uid == v2v_uid)
                return (id::UInt32)
            end
        end
    end
    return NO_TARGET_FOUND::UInt32
end
```

Referenced In: ReceiveStateVectorV2VReport(p. 33), ReceiveExternallyValidatedV2V(p. 36)

ADDV2VTRACKTODB (Algorithm 33) adds the input V2VTRACKFILE to a newly created TARGET object, sets the initialization time (*init_time*) based on the time of applicability (*toa*) of the track, and adds the TARGET to the Target database (*target_db*).

This algorithm takes as input *trk*. This algorithm modifies the *target_db* and has no return value.

Algorithm 33 AddV2VTrackToDB

```
function AddV2VTrackToDB(trk::V2VTrackFile(p. 488))
    tgt = Target(p. 493)()
    tgt.init_time = trk.toa
    tgt.v2v_track = trk
    AddToDB(p. 19)(target_db, tgt)
end
```

Referenced In: ReceiveStateVectorV2VReport(p. 33)

2.2.1.2.2 Externally Validated V2V Report

Another entry point to the STM passive surveillance is given by RECEIVEEXTERNALLYVALIDATEDV2V (Algorithm 34) with inputs defined in Table 15. Raw V2V signals may be maliciously or unintentionally "spoofed". This may degrade system safety or cause an operationally unacceptable amount of alerting. To address this concern, ACAS sXu will at minimum accept an optional external flag for each intruder indicating external V2V validation status. This flag indicates whether an upstream source has validated the V2V data.

RECEIVEEXTERNALLYVALIDATEDV2V first associates the incoming message to a target based on the incoming *v2v_uid*. If no target exists with a V2VTRACKFILE using that *v2v_uid*, nothing happens. If a target is found, that target's V2VTRACKFILE is extracted and *trk.externally_validated* is set based on the incoming *externally_validated* flag.

This algorithm updates *trk*.

Table 15. STM Input Variables - Externally Validated V2V Report

Variable	Units	Type	Description
externally_validated	N/A	bool	Flag to indicate V2V intruder has been validated by an external source
v2v_uid	N/A	uint32	Intruder V2V unique identifier

Algorithm 34 ReceiveExternallyValidatedV2V

```
function ReceiveExternallyValidatedV2V(externally_validated::Bool, v2v_uid::UInt32)
    id = AssociateV2VToTarget(p. 35)(v2v_uid)
    if (id != NO_TARGET_FOUND)
        tgt = RetrieveWithID(p. 19)(target_db, id)
        trk = tgt.v2v_track
        trk.externally_validated = externally_validated
    end
end
```

2.2.2 Intruder Horizontal Estimation

The estimation of a target's horizontal position and velocity using ADS-B and V2V reports is performed with a Kalman filter based on the tracking algorithms described in Appendix C of DO-317B [6]. Modifications have been made to this algorithm to make it more suitable for ACAS X. For the purposes of this document, this modified version of the DO-317 algorithm is referred to as the Passive Tracker.

In one notable departure from the DO-317 tracking algorithms, the Passive Tracker in this document does not estimate the vertical position and velocity of a target aircraft as prescribed in DO-317. The estimation of a target's vertical state for use by the TRM is done with a modified Kalman filter. The vertical state estimate from the modified Kalman filter is used in conjunction with the latitude and longitude data in a State Vector Position Report to convert observations to

ENU coordinates.

The Passive Tracker is initialized with INITIALIZEPASSIVETRACKERPOSITION (Algorithm 35). The algorithm takes as input an indication whether the track is ADS-B or V2V (*is_v2v*)). The algorithm sets the initial position distribution to a zero mean Gaussian with initial standard deviation (*init_sigma_pos*) read from the ADS-B or V2V portions of the parameters file. Remembering that the passive horizontal tracker is an absolute tracker, the mean position is defined as the relative position of the intruder relative to its own absolute geodetic position specified by *trk.lla_at_hor_toa*/ *trk.ecef_at_hor_toa*. This relative position will usually always be zero. The only time this position will be non-zero is temporarily during the prediction and update steps of the Kalman filter, however, the mean position is immediately recentered about the new absolute geodetic position which brings the mean position back to zero. The initial velocity distribution is considered zero mean Gaussian with a standard deviation derived from the parameter *init_NACv* (using CONVERTNACVTOSIGMAHVA). An STM report will never be generated with these initial velocity states, and instead will only be generated after the velocity states have been seeded with either the velocities of a subsequent ADS-B velocity report in ADVANCEPASSIVETRACKVELOCITY or with velocities derived from a subsequent ADS-B position report in ADVANCEPASSIVETRACKPOSITION for ADS-B 1090ES surveillance, or with the velocities of a current ADS-B UAT report or V2V report. A small offset value (*kappa*) is added to the diagonal terms of the covariance matrix *Sigma* to ensure a positive-semidefinite matrix during later Cholesky factorization. The initial mean (*mu*) and covariance (*Sigma*) are returned by the algorithm.

The algorithm takes as input *is_v2v*. This algorithm returns *mu* and *Sigma*.

Algorithm 35 InitializePassiveTrackerPosition

```
function InitializePassiveTrackerPosition(is_v2v::Bool)
    if is_v2v
        init_sigma_pos::R = params().surveillance.horizontal_v2v.init_sigma_pos
        init_NACv::UInt32 = params().surveillance.horizontal_v2v.init_nacv
        kappa::R = params().surveillance.horizontal_v2v.kappa
    else
        init_sigma_pos = params().surveillance.horizontal_adsb.init_sigma_pos
        init_NACv = params().surveillance.horizontal_adsb.init_nacv
        kappa = params().surveillance.horizontal_adsb.kappa
    end
    mu = zeros(4)
    Sigma_tmp = zeros(4,2)
    Sigma_tmp[1,1] = Sigma_tmp[3,2] = init_sigma_pos
    Sigma_tmp[2,1] = Sigma_tmp[4,2] = ConvertNACvToSigmaHVA(p. 38)(init_NACv)
    Sigma = Sigma_tmp * Sigma_tmp'
    Sigma = Sigma + kappa*eye(p. 582)(4)
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: AdvancePassiveTrackState(p. 45), AdvancePassiveTrackPosition(p. 39), InitializePassiveTrackFile(p. 17)

CONVERTNACVTOSIGMAHVA (Algorithm 36) converts an input *NACv* to a velocity uncertainty in a single horizontal dimension. *horiz_vel_accuracy_mps* is assigned to the 95 percent radial uncertainty as defined in DO-242A [9]. In order to convert this value from a standard deviation in two dimensions into a single standard deviation in one dimension, this value is then divided by 2.448 in accordance with a Chi-squared distribution. The default value for *horiz_vel_accuracy_mps* corresponds to NACv of 1.

This algorithm takes as input *NACv*. This algorithm returns *sigma_hva*.

Algorithm 36 ConvertNACvToSigmaHVA

```
function ConvertNACvToSigmaHVA(NACv::UInt32)
    horiz_vel_accuracy_mps::R = 10
    if (NACv == 2)
        horiz_vel_accuracy_mps = 3
    elseif (NACv == 3)
        horiz_vel_accuracy_mps = 1
    elseif (NACv == 4)
        horiz_vel_accuracy_mps = 0.3
    end
    sigma_hva::R = horiz_vel_accuracy_mps/2.448
    return sigma_hva::R
end
```

Referenced In: InitializePassiveTrackerPosition(p. 37), AddHorBias(p. 140), UpdatePassiveTrackerVelocity(p. 49), ReceiveWgs84Observation(p. 92), UpdatePassiveTrackerState(p. 50)

Following initialization, the Passive Tracker is advanced based on the ADS-B report type through a call to ADVANCEPASSIVETRACKPOSITION (Algorithm 37). This algorithm checks the validity of the observation data and performs outlier detection to determine how the Passive Track should be updated. In ADVANCEPASSIVETRACKPOSITION, additional data elements available in the report (*NIC* and *toa*) are stored in the ADSBTRACKFILE.

The first step is to convert the observed LLA/ECEF geodetic position into a local ENU observation. This is done by taking the difference between the observed ECEF position, *obs_ecef*, and the ECEF position at the horizontal trackers TOA, *ecef_at_hor_toa*. This *obs_delta_ecef* is then rotated into local ENU with a call to ROTATEECEFTOENU, and then the east/west and north/south components are extracted from the first two elements of the vector.

If any of the derived quantities are invalid (indicated by evaluating to *NaN*), the Passive Track is not updated and will be coasted. Otherwise, the existing Passive Track is extrapolated to the time of the observation through PREDICTPASSIVETRACKER (Algorithm 44). If the observation is identified as an outlier (through a call to ISOUTLIER (Algorithm 39)), the track is not updated (i.e., the track is either coasted or reinitialized). Indices specific to the Passive horizontal track are used to select the X and Y position (*I_xy*) from the *mu_s* and *Sigma_s* before calling outlier detection. ADVANCEPASSIVETRACKPOSITION can decrement the outlier detection count (*trk.odc_hor*) and a coast occurs when the outlier detection count permits such an action (i.e., this decrementing counter is greater than zero). If the outlier count (*odc_hor*) reaches zero, ADVANCEPASSIVETRACKPOSITION will reinitialize the track using INITIALIZEPASSIVETRACKERPOSITION (Algorithm 35), and reset track data fields including update status and associated timestamps.

Finally, if the observation is valid and not an outlier, ADVANCEPASSIVETRACKPOSITION checks whether a velocity report has not been received for this track (indicated by *trk.init_velocity*) and that the incoming update is the second measurement received since the last time the track was initialized (indicated by *trk.updates_pos*). If both of these conditions are true, then the track velocity is replaced by taking the difference in position over the difference in time of the current measurement and the previous track position after which the track is predicted to the current time using that new velocity. The track is updated with a call to the update step of the Passive Tracker (UPDATEPASSIVETRACKERPOSITION) and track data fields including update status

and associated timestamps are set. Lastly, since the horizontal track is maintained in absolute geodetic coordinates, the new LLA/ECEF positions of the intruder are calculated (and stored as additional data elements in the ADSBTRACKFILE) and the horizontal mean and covariance are re-centered about this new location by a call to RECENTERHORIZONTALTRACKLOCATION.

The algorithm takes as input *trk*, *obs_lla*, *obs_ecef*, *nic*, and *toa*. This algorithm updates *trk*.

Algorithm 37 AdvancePassiveTrackPosition

```
function AdvancePassiveTrackPosition(trk::ADSBTrackFile(p. 485), obs_lla::Vector{R}, obs_ecef::Vector{R}, nic::
    UInt32, toa::R)
    max_outliers::Z = params().surveillance.horizontal_adsb.max_outlier_detections
    outlier_thresh::R = params().surveillance.horizontal_adsb.outlier_threshold
    trk.nic = nic
    trk.toa = toa
    I_xy::Vector{Z} = [1,3]
    obs_delta_ecef = obs_ecef - trk.ecef_at_hor_toa
    obs_delta_enu = RotateECEFToENU(p. 23)(obs_delta_ecef, trk.lla_at_hor_toa)
    (obs_delta_x, obs_delta_y, nothing) = obs_delta_enu
    if (!isnan(obs_delta_x)) && (!isnan(obs_delta_y))
        dt = toa - trk.toa_hor
        (mu_s, Sigma_s) = PredictPassiveTracker(p. 46)(trk.mu_hor, trk.Sigma_hor, dt, false)
        if (IsOutlier(p. 41)([obs_delta_x,obs_delta_y],mu_s[I_xy],Sigma_s[I_xy,I_xy],outlier_thresh))
            if (trk.odc_hor > 0)
                trk.odc_hor -= 1
            else
                (trk.mu_hor,trk.Sigma_hor) = InitializePassiveTrackerPosition(p. 37)(false)
                trk.lla_at_hor_toa = obs_lla
                trk.ecef_at_hor_toa = obs_ecef
                trk.odc_hor = max_outliers
                trk.init_velocity = false
                trk.toa_hor = toa
                trk.toa_pos_update = toa
                trk.updates_pos = 1
            end
        else
            if (!trk.init_velocity) && (trk.updates_pos == 1)
                trk.mu_hor[2] = obs_delta_x / dt
                trk.mu_hor[4] = obs_delta_y / dt
                (mu_s,Sigma_s) = PredictPassiveTracker(p. 46)(trk.mu_hor, trk.Sigma_hor, dt, false)
            end
            (trk.mu_hor,trk.Sigma_hor) = UpdatePassiveTrackerPosition(p. 48)(obs_delta_x,obs_delta_y,mu_s,Sigma_s)
            ReCenterHorizontalTrackLocation(p. 42)(trk, toa)
            trk.odc_hor = max_outliers
            trk.updates_pos += 1
            trk.toa_hor = toa
            trk.toa_pos_update = toa
        end
    end
    return
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12)

INFLATEPASSIVEHORSIGMAUL (Algorithm 38) is used to inflate horizontal tracker uncertainty due to uncompensated latency. Uncompensated latency impacts the overall uncertainty in two ways.

First, the average uncompensated latency delay will cause the position estimate to accumulate a bias that is proportional to the velocity of the aircraft. In this algorithm, the velocity of the in-

truder aircraft is set equal to the absolute value of the East/West and North/South velocities plus extra velocity (*vel_min_ul*), which helps account for velocity estimation errors that may result from tracker lag during initialization or dynamic maneuvers of the aircraft. This first source of uncompensated latency is used to inflate a copy of the covariance matrix (*inflated_Sigma*). The variance of the diagonal position terms are inflated by the velocity times the average uncompensated latency delay (*mean_delay_ul*) squared.

Second, uncompensated latency impacts both the position and velocity uncertainty due to the accelerations of the aircraft. Since the passive trackers of the STM are modeled as constant velocity models, the accelerations of the aircraft are unknown. A tunable process noise acceleration value (*Q*) is used instead. This process noise effects various terms of the covariance matrix according to the process noise coupling matrix (*B*), which is a function of (*mean_delay_ul*. This second source of uncompensated latency error is also used to inflate *inflated_Sigma*.

This algorithm takes as input *mu*, *Sigma*, and *Q*. This algorithm returns *inflated_Sigma*.

Algorithm 38 InflatePassiveHorSigmaUL

```
function InflatePassiveHorSigmaUL(mu::Vector{R}, Sigma::Matrix{R}, Q::R)
    vel_min_ul::R = params().surveillance.horizontal_v2v.covariance_inflation.uncompensated_latency.vel_-
        min_ul
    mean_delay_ul::R = params().surveillance.horizontal_v2v.covariance_inflation.uncompensated_-
        latency.mean_delay_ul
    v_x::R = abs(mu[2]) + vel_min_ul
    v_y::R = abs(mu[4]) + vel_min_ul
    inflated_Sigma = copy(Sigma)
    inflated_Sigma[1,1] = inflated_Sigma[1,1] + v_x * v_x * mean_delay_ul * mean_delay_ul
    inflated_Sigma[3,3] = inflated_Sigma[3,3] + v_y * v_y * mean_delay_ul * mean_delay_ul
    B::Matrix{R} = zeros(4,2)
    B[1, 1] = B[3, 2] = 0.5 * mean_delay_ul * mean_delay_ul
    B[2, 1] = B[4, 2] = mean_delay_ul
    inflated_Sigma = inflated_Sigma + (B * Q * B')
    return inflated_Sigma::Matrix{R}
end
```

Referenced In: AdvancePassiveTrackState(p. 45), AddHorBias(p. 140)

Within the STM, outliers are determined in IsOutlier (Algorithm 39) by comparing the square of the Mahalanobis distance between an estimate (*mu*, *Sigma*) and an observation (*o*) to a threshold (*thresh*). The outlier threshold for each of the tracks is defined in the parameter file.

This algorithm takes as input *o*, *mu*, *Sigma*, and *thresh*. This algorithm returns TRUE or FALSE.

Algorithm 39 IsOutlier

```
function IsOutlier(o::Union{R, Vector{R}}, mu::Union{R, Vector{R}}, Sigma::Union{R, Matrix{R}}, thresh::R)
    if (thresh > 0) && (!any(isnan, o)) && (!any(isnan, mu)) && (!any(isnan, Sigma))
        D = mahal(p. 581)(o, Sigma, mu, 0)^2
        return (D >= thresh)::Bool
    else
        return false
    end
end
```

Referenced In: ReceiveHeadingObservation(p. 88), AdvancePassiveTrackState(p. 45), ReceivePresAltObservation(p. 82), ReceiveWgs84Observation(p. 92), AdvancePassiveTrackPosition(p. 39), AdvanceVerticalTrack(p. 53), AdvancePassiveTrackVelocity(p. 43)

RECENTERHORIZONTALTRACKLOCATION (Algorithm 40) is called after the horizontal track has been either predicted forward in time and/or updated with an observation, and the horizontal mean (*trk.mu_hor*) will therefore contain non-zero position information. Since the horizontal track is maintained in absolute geodetic coordinates, the new LLA/ECEF positions of the intruder are calculated and the horizontal mean and covariance are re-centered about this new location. This results in the horizontal mean containing position elements equal to zero.

This algorithm first calculates the predicted altitude of the intruder at the TOA of the horizontal track. If the track's vertical tracker is valid, indicated by *trk.valid_vert*, then this predicted altitude is generated with a constant velocity prediction equation based on the vertical tracker's mean states, otherwise, the predicted altitude is set equal to the existing altitude in *trk.lla_at_hor_toa*. Then *delta_enu* is generated and converted to *delta_ecef* by a call to ROTATEENUTOECEF. The new absolute ECEF position is then formed by adding *delta_ecef* to the ECEF of the track at the prior TOA. A refinement of the new ECEF position is then performed along with the generation of the corresponding LLA position by a call to REFINEECEFANDLLA. Next, the mean positions are manually set to zero. The new LLA and ECEF positions are then fed into REDEFINEESTIMATEINROTATEDFRAME along with the mean and covariance of the intruder's horizontal tracker, which will rotate the tracker mean and covariance to properly align with the poles of the earth when centered about the new LLA/ECEF location instead of the previous LLA/ECEF location of the tracker. Finally, the new LLA/ECEF positions are stored as additional elements in the TRACKFILE.

This algorithm takes as input *trk* and *toa*. This algorithm updates *trk*.

Algorithm 40 ReCenterHorizontalTrackLocation

```
function ReCenterHorizontalTrackLocation(trk::TrackFile(p. 483), toa::R)
    I_xy = [1, 3]
    if trk.valid_vert
        dt = toa - trk.toa_vert
        alt_ft_toa = trk.mu_vert[1] + trk.mu_vert[2] * dt
    else
        alt_ft_toa = trk.lla_at_hor_toa[3]
    end
    delta_up = alt_ft_toa - trk.lla_at_hor_toa[3]
    delta_enu = [trk.mu_hor[I_xy]; delta_up]
    delta_ecef = RotateENUToECEF(p. 24)(delta_enu, trk.lla_at_hor_toa)
    ecef = trk.ecef_at_hor_toa + delta_ecef
    (ecef, lla) = RefineECEFAndLLA(p. 25)(ecef, alt_ft_toa, trk.alt_src_hae)
    trk.mu_hor[I_xy] = zeros(2)
    (trk.mu_hor, trk.Sigma_hor) = RedefineEstimateInRotatedFrame(p. 42)(trk.mu_hor, trk.Sigma_hor, trk.
        lla_at_hor_toa, trk.ecef_at_hor_toa, lla, ecef)
    trk.lla_at_hor_toa = lla
    trk.ecef_at_hor_toa = ecef
end
```

Referenced In: ExtrapolateADSBTrack(p. 443), AdvancePassiveTrackState(p. 45), ExtrapolateAGTTrack(p. 447), AddADSBTrackToReport(p. 135), AdvancePassiveTrackPosition(p. 39), AddAGTTrackToReport(p. 155), ExtrapolateV2VTrack(p. 444), AdvancePassiveTrackVelocity(p. 43), AddV2VTrackToReport(p. 150)

REDEFINEESTIMATEINROTATEDFRAME (Algorithm 41) redefines the horizontal track's mean and covariance in a rotated frame based on two sets of LLA/ECEF coordinates. During redefinition, variables that are defined in association with the new coordinate frame are indicated by *_1*, while variables that are defined in association with the previous coordinate frame are indicated by *_0*.

The first step is to create a horizontal rotation matrix using both sets of LLA/ECEF coordinates in a call to POLARALIGNMENTHORIZONTALROTATION. This rotation matrix is needed to rotate the tracker's horizontal mean and covariance from the previous frame (LLA/ECEF of intruder at previous horizontal track TOA) to the new frame (LLA/ECEF of intruder at the new horizontal track TOA). The rotation of the mean and covariance is accomplished by a call to MATRIXROTATION.

This algorithm takes as input *mu_hor_0*, *Sigma_hor_0*, *lla_0*, *ecef_0*, *lla_1*, and *ecef_l*. It returns *mu_hor_l* and *Sigma_hor_l*.

Algorithm 41 RedefineEstimateInRotatedFrame

```
function RedefineEstimateInRotatedFrame(mu_hor_0::Vector{R}, Sigma_hor_0::Matrix{R}, lla_0::Vector{R}, ecef_0::
    Vector{R}, lla_1::Vector{R}, ecef_1::Vector{R})
    I_pos::Vector{Z} = [1, 3]
    I_vel::Vector{Z} = [2, 4]
    I_output::Vector{Z} = [1, 3, 2, 4]
    R_0_to_1 = PolarAlignmentHorizontalRotation(p. 25)(lla_0, ecef_0, lla_1, ecef_1)
    (mu_hor_1::Vector{R}, Sigma_hor_1::Matrix{R}) =
        MatrixRotation(p. 63)(mu_hor_0, Sigma_hor_0, I_pos, I_vel, I_output, R_0_to_1)
    return (mu_hor_1::Vector{R}, Sigma_hor_1::Matrix{R})
end
```

Referenced In: AbsoluteGeodeticToOwnshipRelative(p. 141), ReCenterHorizontalTrackLocation(p. 42)

ADVANCEPASSIVETRACKVELOCITY (Algorithm 42) determines whether to update the ADSBTRACK-

FILE based on the incoming intruder horizontal observation. If no velocity estimates have previously been developed for this track and the ownship is at a latitude in nonpolar regions of the earth, the velocity estimates in the ADS-B track are initialized to those derived from the report and the *init_velocity* flag is set to true (note that the indices 2 and 4 correspond to the velocity components of the passive horizontal state estimate). Else if the ownship is at a latitude in the polar regions of the earth and the velocity estimates in the ADS-B track have been seeded by two position measurements, the *init_velocity* flag is set to true. Otherwise, the *init_velocity* flag is left as false. The ADS-B track is predicted forward by calling PREDICTPASSIVETRACKER and checked against the *outlier_thresh*. If the *init_velocity* flag is true, the velocity values are non-NaN, and the observation is not an outlier, then the velocity portion of the ADS-B track can be updated through UPDATEPASSIVETRACKERVELOCITY. If the track's NACv is 0 (indicating that it is uninitialized), the track is updated with a NACv of *init_NACv*. Since the State Vector Velocity Report does not include an altitude, the vertical track cannot be updated. The additional data elements available in the report (*nic* and *toa*) are stored in the ADSBTRACKFILE. Lastly, since the horizontal track is maintained in absolute geodetic coordinates, the new LLA/ECEF positions of the intruder are calculated (and stored as additional data elements in the ADSBTRACKFILE) and the horizontal mean and covariance are re-centered about this new location by a call to RECENTERHORIZONTALTRACKLOCATION.

The algorithm takes as input *trk*, *obs_dx*, *obs_dy*, *nic*, *toa*, and *lat_pred*. This algorithm potentially updates *trk*.

Algorithm 42 AdvancePassiveTrackVelocity

```
function AdvancePassiveTrackVelocity(trk::ADSBTrackFile(p. 485), obs_dx::R, obs_dy::R, nic::UInt32, toa::R, lat_pred
    ::R)
    outlier_thresh::R = params().surveillance.horizontal_adsb.outlier_threshold
    vel_init_lat_thresh::R = params().surveillance.horizontal_adsb.vel_init_lat_thresh
    init_NACv::UInt32 = params().surveillance.horizontal_adsb.init_nacv
    I_dxdy::Vector{Z} = [2,4]
    dt = toa - trk.toa_hor
    (trk.nic, trk.toa) = (nic, toa)
    mu_hor_with_vel = copy(trk.mu_hor)
    if (!trk.init_velocity)
        vel_init_enabled::Bool = (abs(lat_pred) < deg2rad(vel_init_lat_thresh))
        vel_init_seeded::Bool = (trk.updates_pos > 1)
        if (vel_init_enabled)
            mu_hor_with_vel[I_dxdy] = [obs_dx, obs_dy]
            trk.init_velocity = true
        elseif (vel_init_seeded)
            trk.init_velocity = true
        end
    end
    (mu_s, Sigma_s) = PredictPassiveTracker(p. 46)(mu_hor_with_vel, trk.Sigma_hor, dt, false)
    outlier::Bool = IsOutlier(p. 41)([obs_dx,obs_dy],mu_s[I_dxdy],Sigma_s[I_dxdy,I_dxdy],outlier_thresh)
    if (trk.init_velocity) && (!isnan(obs_dx)) && (!isnan(obs_dy)) && (!outlier)
        nacv = trk.nacv
        if (trk.nacv == 0)
            nacv = init_NACv
        end
        (trk.mu_hor, trk.Sigma_hor) = UpdatePassiveTrackerVelocity(p. 49)(obs_dx, obs_dy, nacv, mu_s, Sigma_s)
        ReCenterHorizontalTrackLocation(p. 42)(trk, toa)
        trk.toa_hor = toa
    end
    return
end
```

Referenced In: ReceiveStateVectorVelocityReport(p. 22)

Similar to ADVANCEPASSIVETRACKPOSITION and ADVANCEPASSIVETRACKVELOCITY, ADVANCEPASSIVETRACKSTATE (Algorithm 43) combines these two steps into one algorithm to advance the position and velocity tracks of the incoming UAT or V2V observation.

This algorithm first checks whether the input track file is V2V or ADS-B. This Boolean is used to determine which tracker parameters to use. In ADVANCEPASSIVETRACKSTATE, additional data elements available in the report (*NIC* and *toa*) are stored in the track file.

The next step is to convert the observed LLA/ECEF geodetic position into a local ENU observation. This is done by taking the difference between the observed ECEF position, *obs_ecef*, and the ECEF position at the horizontal trackers TOA, *ecef_at_hor_toa*. This *obs_delta_ecef* is then rotated into local ENU with a call to ROTATEECEFTOENU, and then the east/west and north/south components are extracted from the first two elements of the vector.

If any of the measured quantities are invalid (indicated by evaluating to *NaN*), the track is not updated. Otherwise, the existing Passive Track is extrapolated to the time of the observation through PREDICTPASSIVETRACKER (Algorithm 44). If the input track file is V2V, the tracker uncertainty used in outlier detection is inflated using INFLATEPASSIVEHORSIGMAUL. If the observation is identified as an outlier (through a call to ISOUTLIER), the track is not updated (i.e., the track is either coasted or reinitialized). The algorithm can decrement the outlier detection count (*trk.odc_hor*) and a coast occurs when the outlier detection count permits such an action (i.e., this decrementing counter is greater than zero). If the outlier count (*odc_hor*) reaches zero, the track will reinitialize the track using INITIALIZEPASSIVETRACKERPOSITION (Algorithm 35), and reset track data fields including update status and associated timestamps. The velocities *obs_dx* and *obs_dy* received in the report are used to immediately initialize the track velocities.

Finally, if the observation is valid and not an outlier, the NACv to use during the update is evaluated. If the track's NACv is 0 (indicating that it is uninitialized), the track is updated with a NACv of *init_NACv*. The track is then updated with a call to the update step of the ADS-B Tracker (UPDATEPASSIVETRACKERSTATE and track data fields including update status and associated timestamps are set. Lastly, since the horizontal track is maintained in absolute geodetic coordinates, the new LLA/ECEF positions of the intruder are calculated (and stored as additional data elements in the ADSBTRACKFILE) and the horizontal mean and covariance are re-centered about this new location by a call to RECENTERHORIZONTALTRACKLOCATION.

This algorithm takes as input *trk*, *obs_lla*, *obs_ecef*, *obs_dx*, *obs_dy*, *nic*, and *toa*. This algorithm can potentially update *trk*.

Algorithm 43 AdvancePassiveTrackState

```
function AdvancePassiveTrackState(trk::Union{ADSBTrackFile(p.485), V2VTrackFile(p.488)}, obs_lla::Vector{R}, obs_ecef
    ::Vector{R}, obs_dx::R, obs_dy::R, nic::UInt32, toa::R)
    is_v2v::Bool = (typeof(trk) == V2VTrackFile(p.488))
    if is_v2v
        max_outliers::Z = params().surveillance.horizontal_v2v.max_outlier_detections
        outlier_thresh::R = params().surveillance.horizontal_v2v.outlier_threshold
        Q_outlier_ul::R = params().surveillance.horizontal_v2v.covariance_inflation.uncompensated_-
            latency.Q_outlier_ul
        init_NACv::UInt32 = params().surveillance.horizontal_v2v.init_nacv
    else
        max_outliers = params().surveillance.horizontal_adsb.max_outlier_detections
        outlier_thresh = params().surveillance.horizontal_adsb.uat_outlier_threshold
        init_NACv = params().surveillance.horizontal_adsb.init_nacv
    end
    (trk.toa, trk.nic) = (toa, nic)
    obs_delta_ecef = obs_ecef - trk.ecef_at_hor_toa
    obs_delta_enu = RotateECEFToENU(p.23)(obs_delta_ecef, trk.lla_at_hor_toa)
    (obs_delta_x, obs_delta_y, nothing) = obs_delta_enu
    if (!isnan(obs_delta_x)) && (!isnan(obs_dx)) && (!isnan(obs_delta_y)) && (!isnan(obs_dy))
        dt = toa - trk.toa_hor
        (mu_s, Sigma_s) = PredictPassiveTracker(p.46)(trk.mu_hor, trk.Sigma_hor, dt, is_v2v)
        Sigma_tmp = copy(Sigma_s)
        if is_v2v
            Sigma_tmp = InflatePassiveHorSigmaUL(p.40)(mu_s, Sigma_s, Q_outlier_ul)
        end
        if (IsOutlier(p.41)([obs_delta_x,obs_dx,obs_delta_y,obs_dy],mu_s,Sigma_tmp,outlier_thresh))
            if (trk.odc_hor > 0)
                trk.odc_hor -= 1
            else
                (trk.mu_hor, trk.Sigma_hor) = InitializePassiveTrackerPosition(p.37)(is_v2v)
                trk.mu_hor[2] = obs_dx
                trk.mu_hor[4] = obs_dy
                trk.lla_at_hor_toa = obs_lla
                trk.ecef_at_hor_toa = obs_ecef
                trk.odc_hor = max_outliers
                trk.init_velocity = true
                trk.toa_hor = toa
                trk.toa_pos_update = toa
                trk.updates_pos = 1
            end
        else
            nacv = trk.nacv
            if (trk.nacv == 0)
                nacv = init_NACv
            end
            (trk.mu_hor, trk.Sigma_hor) = UpdatePassiveTrackerState(p.50)(obs_delta_x, obs_dx, obs_delta_y,
                obs_dy, nacv, mu_s,Sigma_s, is_v2v)
            ReCenterHorizontalTrackLocation(p.42)(trk, toa)
            trk.odc_hor = max_outliers
            trk.updates_pos += 1
            trk.toa_hor = toa
            trk.toa_pos_update = toa
        end
    end
    return
end
```

Referenced In: ReceiveStateVectorV2VReport(p.33), ReceiveStateVectorUatReport(p.27)

The Passive Tracker utilizes the standard Kalman filter prediction step in PREDICTPASSIVETRACKER (Algorithm 44). This algorithm first reads the process noise matrix (Q) from the parameter file. Depending on whether the input *is_v2v* indicates that the track being predicted is a V2V or ADS-B

track, the V2V or ADS-B Q is chosen. Then algorithm will call PREDICT2DKALMANFILTER which performs the standard Kalman prediction step for two dimensions, and returns the predicted state and covariance.

The algorithm takes as input *mu*, *Sigma*, *dt*, and *is_v2v*. This algorithm returns the output of PREDICT2DKALMANFILTER, namely *mu_pred* and *Sigma_pred*.

Algorithm 44 PredictPassiveTracker

```
function PredictPassiveTracker(mu::Vector{R}, Sigma::Matrix{R}, dt::R, is_v2v::Bool)
    if is_v2v
        Q::R = params().surveillance.horizontal_v2v.Q
    else
        Q = params().surveillance.horizontal_adsb.Q
    end
    return Predict2DKalmanFilter(p. 47)(mu, Sigma, Q, dt)
end
```

Referenced In: ExtrapolateADSBTrack(p. 443), AdvancePassiveTrackState(p. 45), ExtrapolateAGTTrack(p. 447), AddADSBTrackToReport(p. 135), PredictPreTrackedTrack(p. 126), AdvancePassiveTrackPosition(p. 39), UpdateAGTTrack(p. 76), AddAGTTrackToReport(p. 155), ExtrapolateV2VTrack(p. 444), AdvancePassiveTrackVelocity(p. 43), AddV2VTrackToReport(p. 150)

The Kalman filter prediction steps are performed in PREDICT1DKALMANFILTER (Algorithm 45) to predict the mean and covariance of a linear system using a constant-velocity dynamic model.

This prediction is not performed if the difference between the toa and the time of the last track update is less than a defined threshold (*min_extrap_toa_step*); instead the most recent mean and covariance *mu* and *Sigma* are returned. Otherwise, the duration of time that has passed since the last successful update to the track (*dt*) is used to formulate the state transition matrix (F) and the process noise coupling matrix (G). If the process noise Q is a scalar value rather than a matrix, it is changed into a 2 by 2 matrix. These matrices are then used in a call to PREDICTKALMANFILTER to produce the predicted mean and covariance.

This algorithm takes as input *mu*, *Sigma*, Q, and *dt*. This algorithm returns predicted mean and covariance *mu_pred* and *Sigma_pred*.

PREDICT2DKALMANFILTER (Algorithm 46) is nearly identical to PREDICT1DKALMANFILTER, except that it predicts the mean and covariance of a linear system using a constant-velocity dynamic model for a two-dimensional state space.

PREDICTKALMANFILTER (Algorithm 47) includes the Kalman filter prediction equations for predicting the mean and covariance of a linear system.

This algorithm takes as input *mu*, *Sigma*, F, G, and Q. This algorithm returns the predicted mean and covariance *mu_pred* and *Sigma_pred*.

Algorithm 45 Predict1DKalmanFilter

```
function Predict1DKalmanFilter(mu::Vector{R}, Sigma::Matrix{R}, Q::Union{R, Matrix{R}}, dt::R)
    min_extrap_toa_step::R = params().surveillance.min_extrap_toa_step
    if (dt < min_extrap_toa_step)
        (mu_pred, Sigma_pred) = (mu, Sigma)
    else
        F = [1.0 dt ;
             0.0 1.0]
        Gdt = 0.5*dt^2
        G = [dt  Gdt;
             0   dt ]
        if (size(Q, 1) == 1)
               Q = [0  0;
                 0  Q]
        end
        (mu_pred, Sigma_pred) = PredictKalmanFilter(p. 47)(mu, Sigma, F, G, Q)
    end
    return (mu_pred::Vector{R}, Sigma_pred::Matrix{R})
end
```

Referenced In: ReceiveHeadingObservation(p. 88), PredictVerticalTracker(p. 54), HeadingAtToa(p. 90), PredictORNCTTrack(p. 64)

Algorithm 46 Predict2DKalmanFilter

```
function Predict2DKalmanFilter(mu::Vector{R}, Sigma::Matrix{R}, Q::R, dt::R)
    min_extrap_toa_step::R = params().surveillance.min_extrap_toa_step
    if (dt < min_extrap_toa_step)
        (mu_pred, Sigma_pred) = (mu, Sigma)
    else
        F = [1   dt  0   0  ;
             0   1   0   0  ;
             0   0   1   dt ;
             0   0   0   1  ]
        Gdt = 0.5*dt^2
        G = [Gdt 0  ;
             dt  0  ;
             0   Gdt;
             0   dt ]
        (mu_pred, Sigma_pred) = PredictKalmanFilter(p. 47)(mu, Sigma, F, G, Q)
    end
    return (mu_pred::Vector{R}, Sigma_pred::Matrix{R})
end
```

Referenced In: PredictPassiveTracker(p. 46), PredictORNCTTrack(p. 64)

Algorithm 47 PredictKalmanFilter

```
function PredictKalmanFilter(mu::Vector{R}, Sigma::Matrix{R}, F::Matrix{R}, G::Matrix{R}, Q::Union{R, Matrix{R
    }})
    mu_s = F*mu
    Sigma_s = (F*Sigma*F') + (G*Q*G')
    return (mu_s::Vector{R}, Sigma_s::Matrix{R})
end
```

Referenced In: Predict2DKalmanFilter(p. 47), Predict1DKalmanFilter(p. 47)

The passive trackers also utilize the standard Kalman filter update step. However three update algorithms are needed to reflect fundamental differences in State Vector Position Reports (UPDATEPASSIVETRACKERPOSITION (Algorithm 48)), State Vector Velocity Reports (UPDATEPASSIVETRACKERVELOCITY (Algorithm 50)), and State Vector UAT/V2V Reports (UPDATEPASSIVETRACKERSTATE (Algorithm 51)). These three update algorithms each use UPDATEKALMANFILTER but require a different observation coupling matrix (*H*) and observation noise matrix (*R_q* or *R_k*). For ADS-B position, a NACp of 7 is used to calculate the *update_sigma_pos* parameter that fills *R_q*. NACp of 7 is used even when a higher value is reported because the error model does not account for uncompensated latency, which dominates the position error for higher NACps. Treating all position measurements at the same NACp is a way to inflate the uncertainty for higher NACps to help account for uncompensated latency. For V2V reports, *update_sigma_pos* is set based on the dynamic noise of the position measurements, and uncompensated latency is explicitly accounted for in a separate algorithm INFLATEPASSIVEHORSIGMAUL (Algorithm 38). For ADS-B velocity and V2V reports, a call to CONVERTNACVTOSIGMAHVA given the track's stored NACv is used when initializing *R_k* or *R_q*. In UPDATEPASSIVETRACKERSTATE, the input *is_v2v* flag determines whether to use the STM parameters for the ADS-B or V2V trackers. In each case, the call to UPDATEKALMANFILTER returns the updated state and covariance. In a final step, the updated state covariance (*Sigma*) is scaled by a constant factor (*cov_inflation_factor*) to increase the overall uncertainty. The *cov_inflation_factor* is another way to add uncertainty to help account for uncompensated latency in ADS-B tracks, which the error model does not account for.

The UPDATEPASSIVETRACKERPOSITION algorithm takes as input *x_rel*, *y_rel*, *mu_s*, and *Sigma_s*.

The UPDATEPASSIVETRACKERVELOCITY algorithm takes as input *dx_rel*, *dy_rel*, *nacv*, *mu_s*, and *Sigma_s*.

The UPDATEPASSIVETRACKERSTATE algorithm takes as input *x_rel*, *dx_rel*, *y_rel*, *dy_rel*, *nacv*, *mu_s*, *Sigma_s*, and *is_v2v*.

Each algorithm returns the updated *mu* and *Sigma*.

Algorithm 48 UpdatePassiveTrackerPosition

```
function UpdatePassiveTrackerPosition(x_rel::R, y_rel::R, mu_s::Vector{R}, Sigma_s::Matrix{R})
    cov_inflation_factor::R = params().surveillance.horizontal_adsb.cov_inflation_factor
    update_sigma_pos::R = params().surveillance.horizontal_adsb.update_sigma_pos
    H = zeros(2,4)
    H[1,1] = H[2,3] = 1
    R_q = diagm(0 => [update_sigma_pos*update_sigma_pos, update_sigma_pos*update_sigma_pos])
    (mu, Sigma) = UpdateKalmanFilter(p. 49)([x_rel, y_rel], mu_s, Sigma_s, H, R_q)
    Sigma = cov_inflation_factor*Sigma
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: AdvancePassiveTrackPosition(p. 39)

The standard Kalman filter update step is outlined in UPDATEKALMANFILTER, which is called with *o* as the measured ground range and *R_k* as the measurement covariance, as well as the predicted state (*mu_s*), the state prediction covariance (*Sigma_s*), and the observation coupling matrix (*H*). The Kalman filter matrices are identified in the following table:

This algorithm takes as input *o*, *mu_s*, *Sigma_s*, *H*, and *R_k*. This algorithm returns *mu* and

Table 16. Kalman F, G, H, Q, and R Matrix Definitions

Matrix	Definition
F	State Transition Matrix
G	Process Noise Coupling Matrix
H	Observation Coupling Matrix
Q	Process Noise Matrix
R	Measurement Noise Matrix

Sigma.

Algorithm 49 UpdateKalmanFilter

```
function UpdateKalmanFilter(o::Union{R, Vector{R}}, mu_s::Vector{R}, Sigma_s::Matrix{R}, H::Matrix{R}, R_k::
    Union{R, Matrix{R}})
    S = H*Sigma_s*H' .+ R_k
    K = Sigma_s * H' * inv(S)
    mu = mu_s + K*(o .- H*mu_s)
    I = eye(p. 582)(size(Sigma_s,1))
    Sigma = (I - K*H)*Sigma_s*(I - K*H)' + K*R_k*K'
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: UpdateVerticalTracker(p. 55), UpdatePassiveTrackerVelocity(p. 49), UpdatePassiveTrackerPosition(p. 48), UpdatePassiveTrackerState(p. 50)

Algorithm 50 UpdatePassiveTrackerVelocity

```
function UpdatePassiveTrackerVelocity(dx_rel::R, dy_rel::R, nacv::UInt32, mu_s::Vector{R}, Sigma_s::Matrix{R})
    cov_inflation_factor::R = params().surveillance.horizontal_adsb.cov_inflation_factor
    H = zeros(2,4)
    H[1,2] = H[2,4] = 1
    sigma_hva = ConvertNACvToSigmaHVA(p. 38)(nacv)
    R_k = diagm(0 => [sigma_hva*sigma_hva, sigma_hva*sigma_hva])
    (mu, Sigma) = UpdateKalmanFilter(p. 49)([dx_rel, dy_rel], mu_s, Sigma_s, H, R_k)
    Sigma = cov_inflation_factor*Sigma
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: AdvancePassiveTrackVelocity(p. 43)

Algorithm 51 UpdatePassiveTrackerState

```
function UpdatePassiveTrackerState(x_rel::R, dx_rel::R, y_rel::R, dy_rel::R, nacv::UInt32, mu_s::Vector{R},
    Sigma_s::Matrix{R}, is_v2v::Bool)
    if is_v2v
        cov_inflation_factor::R = params().surveillance.horizontal_v2v.cov_inflation_factor
        update_sigma_pos::R = params().surveillance.horizontal_v2v.update_sigma_pos
    else
        cov_inflation_factor = params().surveillance.horizontal_adsb.cov_inflation_factor
        update_sigma_pos = params().surveillance.horizontal_adsb.update_sigma_pos
    end
    H = eye(p. 582)(4)
    sigma_hva = ConvertNACvToSigmaHVA(p. 38)(nacv)
    R_q = diagm(0 => [update_sigma_pos*update_sigma_pos, sigma_hva*sigma_hva, update_sigma_pos*
        update_sigma_pos, sigma_hva*sigma_hva])
    (mu, Sigma) = UpdateKalmanFilter(p. 49)([x_rel, dx_rel, y_rel, dy_rel], mu_s, Sigma_s, H, R_q)
    Sigma = cov_inflation_factor*Sigma
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: AdvancePassiveTrackState(p. 45)

2.2.3 Intruder Vertical Estimation

The estimation of a target's vertical position and velocity with ADS-B or V2V position messages is performed using a linear Kalman filter with some modifications to account for quantized measurements [10]. This tracker is initialized with INITIALIZEVERTICALTRACKER (Algorithm 52). This algorithm produces a NAR track if the input altitude measurement is invalid (i.e., evaluates to a *NaN*) or is outside the NAR threshold. This special condition is represented by setting all elements of the tracked state (*trk.mu_vert*) and covariance (*trk.Sigma_vert*) to *NaN*, resetting the number of track updates (*trk.updates_vert*) to zero, and setting the validity flag of the vertical track (*trk.valid_vert*) to FALSE.

If the intruder altitude measurement is valid, the initial altitude distribution is Gaussian centered on the input altitude measurement (*z_int*) while the initial altitude rate distribution is considered to be zero mean Gaussian. The initial variance for these distributions (*var_zint* and *var_dzint* respectively) are read in from the parameter file as they cannot be determined on a single measurement alone. The parameter file has separate parameters for each type of aircraft and altitude source. These values are independent of quantization, and are selected to allow any track to quickly converge to a steady state covariance estimate. The vertical rate arrow for the display is initialized as level.

This algorithm takes as input *trk*, *z_int*, *is_alt_geo_hae*, and *is_large*. This algorithm updates *trk*.

Algorithm 52 InitializeVerticalTracker

```
function InitializeVerticalTracker(trk::Union{ADSBTrackFile(p. 485), V2VTrackFile(p. 488)}, z_int::R, is_alt_geo_hae::Bool
    , is_large::Bool)
    if is_alt_geo_hae
        if is_large
            var_zint::R = params().surveillance.vertical.hae_large_intruder.var_zint
            var_dzint::R = params().surveillance.vertical.hae_large_intruder.var_dzint
        else
            var_zint = params().surveillance.vertical.hae_small_intruder.var_zint
            var_dzint = params().surveillance.vertical.hae_small_intruder.var_dzint
        end
    else
        if is_large
            var_zint = params().surveillance.vertical.pres_large_intruder.var_zint
            var_dzint = params().surveillance.vertical.pres_large_intruder.var_dzint
        else
            var_zint = params().surveillance.vertical.pres_small_intruder.var_zint
            var_dzint = params().surveillance.vertical.pres_small_intruder.var_dzint
        end
    end
    if (isnan(z_int)) || (z_int > NARS_THRESHOLD_MAX) || (z_int < NARS_THRESHOLD_MIN)
        trk.mu_vert = fill(NaN,2)
        trk.Sigma_vert = fill(NaN,2,2)
        trk.alt_src_hae = is_alt_geo_hae
        trk.updates_vert = 0
        trk.valid_vert = false
    else
        trk.mu_vert = [z_int, 0]
        trk.Sigma_vert = diagm(0 => [var_zint, var_dzint])
        trk.alt_src_hae = is_alt_geo_hae
        trk.updates_vert = 1
        trk.display_arrow_current = DISPLAY_ARROW_LEVEL
        trk.vert_arrow_history = Z[]
        pushfirst!(trk.vert_arrow_history, DISPLAY_ARROW_LEVEL)
    end
end
```

Referenced In: AdvanceVerticalTrack(p. 53), InitializePassiveTrackFile(p. 17)

Following initialization, the vertical tracker is advanced through ADVANCEVERTICALTRACK (Algorithm 53). This algorithm checks the validity of the observation data and performs outlier detection to determine how the vertical track should be updated. The maximum sequential outlier count (*max_outlier*) is configured based on the classification of the intruder aircraft and the altitude source of the track. The existing vertical track is extrapolated to the time of the observation through PREDICTVERTICALTRACKER. If this is the second update since initialization, the vertical rate observation is compared to a static window specified in the parameter file (*window*). This allows for track startup on high vertical rate intruders without requiring an artificially high initial vertical rate uncertainty. All other updates are subject to the typical Mahalanobis distance test for outlier detection. The effects of altitude quantization on track uncertainty are explicitly addressed in UPDATEVERTICALTRACKER, but these effects must be anticipated in the covariance provided for outlier detection prior to the update, therefore an *altitude_inflation* derived from Sheppard's correction is added to the *Sigma_s* used for Mahalanobis distance based outlier detection. A vertical track will not be advanced if it has only been updated once and the change in time is too small, indicated by *dt_too_small_for_velocity_seed*. This is to prevent an inadvertently large vertical rate seed *dz* that can arise from a small *dt* and the straddling of quantization bins by the first two altitude measurements. Note that the threshold *dt_small* was reduced by a factor

of 4 during its initialization if the quantization of the track was 25 feet as opposed to 100 feet.

The track is coasted or reinitialized if the altitude measurement is invalid (indicated by evaluating to *NaN*), if the measurement is identified as an outlier (through a call to IsOutlier or the static window described above), if the altitude measurement is outside of the allowed bounds (specified by *NARS_THRESHOLD_MAX* and *NARS_THRESHOLD_MIN*), if there have been no previous updates to the vertical track (*trk.updates_vert*), or if the altitude source has changed since the last measurement (*alt_source_hae*). A coast occurs when the outlier detection count (*trk.odc_vert*) permits such an action (i.e., this decrementing counter is greater than zero), there has been more than one update to the vertical track, and the vertical track is in a valid state. If any of these conditions are false, the track is reinitialized.

If the track is not advanced through either a coast or reinitialization, the track is updated with a call to the update step of the vertical tracker (UpdateVerticalTracker (Algorithm 55)). If this is the second update since track initialization, the vertical rate is seeded based on the observation prior to the call to UpdateVerticalTracker. The outlier detection count is then reset to its maximum value *max_outliers*, and the update counter for the track is incremented. If the update counter exceeds a specified threshold (*detections_to_recover*), the validity flag for the vertical track is set to TRUE, thus causing a NAR Track File to be marked as having a valid altitude estimate available. The vertical rate arrow for the pilot's display is then set using VerticalRateArrowUpdate (Algorithm 144).

This algorithm takes as input *trk*, *z*, *q_int*, *alt_source_hae*, and *toa*. This algorithm updates *trk*.

Algorithm 53 AdvanceVerticalTrack

```
function AdvanceVerticalTrack(trk::Union{ADSBTrackFile(p.485), V2VTrackFile(p.488)}, z::R, q_int::UInt32,
    is_alt_geo_hae::Bool, is_large::Bool, toa::R)
    vertical = params().surveillance.vertical
    detections_to_recover::Z = vertical.detections_to_recover
    window::Z = vertical.outlier_window
    dt_small::R = vertical.dt_too_small_for_velocity
    dt_small = dt_small * q_int / MODE_C_QUANT
    if trk.alt_src_hae
        if is_large
            outlier_thresh::R = vertical.hae_large_intruder.outlier_threshold
            max_outliers_normal::Z = vertical.hae_large_intruder.max_outlier_detections
        else
            outlier_thresh = vertical.hae_small_intruder.outlier_threshold
            max_outliers_normal = vertical.hae_small_intruder.max_outlier_detections
        end
    else
        if is_large
            outlier_thresh = vertical.pres_large_intruder.outlier_threshold
            max_outliers_normal = vertical.pres_large_intruder.max_outlier_detections
        else
            outlier_thresh = vertical.pres_small_intruder.outlier_threshold
            max_outliers_normal = vertical.pres_small_intruder.max_outlier_detections
        end
    end
    max_outliers::Z = max_outliers_normal
    dt = toa - trk.toa_vert
    trk.quant = q_int
    (mu_s, Sigma_s) = PredictVerticalTracker(p.54)(trk.mu_vert, trk.Sigma_vert, dt, trk.alt_src_hae, false,
        is_large)
    dz::R = z - trk.mu_vert[1]
    if (isnan(z))
        outlier = true
    elseif (trk.updates_vert == 1)
        outlier = (abs(dz/dt) > window)
    else
        altitude_inflation::R = (q_int^2) / 12.0
        outlier = IsOutlier(p.41)(z,mu_s[1],Sigma_s[1,1]+altitude_inflation,outlier_thresh)
    end
    dt_too_small_for_vel_seed::Bool = (trk.updates_vert==1) && (dz!=0) && (dt < dt_small)
    if !(dt_too_small_for_vel_seed)
        invalid_alt::Bool = (NARS_THRESHOLD_MAX < z) || (NARS_THRESHOLD_MIN > z)
        if (outlier) || (invalid_alt) || (0 == trk.updates_vert) || (trk.alt_src_hae != is_alt_geo_hae)
            if (0 < trk.odc_vert) && (1 < trk.updates_vert) && (trk.valid_vert)
                trk.odc_vert -= 1
            else
                InitializeVerticalTracker(p.51)(trk,z,is_alt_geo_hae,is_large)
                trk.odc_vert = max_outliers
                trk.toa_vert = toa
            end
        else
            if (trk.updates_vert == 1)
                trk.mu_vert[2] = dz/dt
                (mu_s, Sigma_s) = PredictVerticalTracker(p.54)(trk.mu_vert, trk.Sigma_vert, dt, trk.alt_src_hae,
                    false, is_large)
            end
            (trk.mu_vert,trk.Sigma_vert) = UpdateVerticalTracker(p.55)(z,q_int,mu_s,Sigma_s, trk.alt_src_hae,
                false, is_large)
            trk.updates_vert += 1
            trk.odc_vert = max_outliers
            if (trk.updates_vert >= detections_to_recover)
                trk.valid_vert = true
            end
            trk.toa_vert = toa
            VerticalRateArrowUpdate(p.148)(trk, false)
        end
    end
```

```
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), ReceiveStateVectorV2VReport(p. 33), ReceiveStateVectorUatReport(p. 27)

The vertical tracker uses the standard Kalman filter prediction step (PREDICTVERTICALTRACKER (Algorithm 54)). This algorithm first reads the process noise matrix (Q) from the parameter file. Then algorithm will call PREDICT1DKALMANFILTER which performs the standard Kalman prediction step, and returns the predicted state and covariance.

This algorithm takes as input *mu*, *Sigma*, *dt*, *is_alt_geo_hae*, *is_ownship*, and *is_large*. This algorithm returns the results of the call to PREDICT1DKALMANFILTER, namely *mu_pred* and *Sigma_pred*.

Algorithm 54 PredictVerticalTracker

```
function PredictVerticalTracker(mu::Vector{R}, Sigma::Matrix{R}, dt::R, is_alt_geo_hae::Bool, is_ownship::Bool,
    is_large::Bool)
    if is_alt_geo_hae
        if is_ownship
            Q::R = params().surveillance.vertical.hae_small_ownship.Q
        else
            if is_large
                Q = params().surveillance.vertical.hae_large_intruder.Q
            else
                Q = params().surveillance.vertical.hae_small_intruder.Q
            end
        end
    else
        if is_ownship
            Q = params().surveillance.vertical.pres_small_ownship.Q
        else
            if is_large
                Q = params().surveillance.vertical.pres_large_intruder.Q
            else
                Q = params().surveillance.vertical.pres_small_intruder.Q
            end
        end
    end
    return Predict1DKalmanFilter(p. 47)(mu, Sigma, Q, dt)
end
```

Referenced In: ExtrapolateADSBTrack(p. 443), ReceivePresAltObservation(p. 82), ExtrapolateAGTTrack(p. 447), AddADSBTrackToReport(p. 135), PredictPreTrackedTrack(p. 126), ReceiveWgs84Observation(p. 92), HAEAltAtToa(p. 96), UpdateAGTTrack(p. 76), AdvanceVerticalTrack(p. 53), AddAGTTrackToReport(p. 155), ExtrapolateV2VTrack(p. 444), SetOwnshipData(p. 116), ExtrapolateOwnshipTrack(p. 442), PresAltAtToa(p. 84), AddV2VTrackToReport(p. 150)

The vertical tracker may depart from the standard Kalman filter paradigm in its update step, UPDATEVERTICALTRACKER (Algorithm 55). Based on the dimension of the observation o, i.e. whether the observation consists of both the altitude and the vertical speed or the altitude alone, the observation matrix H is populated. In addition, if *q_int* is larger than *partition_thresh* obtained from the parameter file and the observation only contains the altitude, as indicated by *use_partition_estimator*, then instead of using the standard linearly calculated predicted obser-

vation and its covariance, the vertical tracker utilizes PREDICTEDALTITUDE (Algorithm 56) to obtain these quantities to calculate the updated state (*mu*) and covariance (*Sigma*). The vertical tracker is then updated by a call to UPDATEKALMANFILTER (Algorithm 49). Finally, if *use_partition_estimator* is TRUE, then the covariance matrix is inflated using the covariance returned from PREDICTEDALTITUDE (Algorithm 56) and Kalman gain (*K*).

This algorithm takes as input *o*, *q_int*, *mu_s*, *Sigma_s*, *is_alt_geo_hae*, *is_ownship*, and *is_large*. This algorithm returns *mu*, *Sigma*.

Algorithm 55 UpdateVerticalTracker

```
function UpdateVerticalTracker(o::Union{R, Vector{R}}, q_int::UInt32, mu_s::Vector{R}, Sigma_s::Matrix{R},
     is_alt_geo_hae::Bool, is_ownship::Bool, is_large::Bool)
    partition_thresh::R = params().surveillance.vertical.partition_estimator_threshold
    if is_alt_geo_hae
        if is_ownship
            R_k = params().surveillance.vertical.hae_small_ownship.R
        else
            if is_large
                R_k = params().surveillance.vertical.hae_large_intruder.R
            else
                R_k = params().surveillance.vertical.hae_small_intruder.R
            end
        end
    else
        if is_ownship
            R_k = params().surveillance.vertical.pres_small_ownship.R
        else
            if is_large
                R_k = params().surveillance.vertical.pres_large_intruder.R
            else
                R_k = params().surveillance.vertical.pres_small_intruder.R
            end
        end
    end
    m::Z = length(o)
    if (m == 1)
        H::Matrix{R} = [1.0 0.0]
    else
        H = [1.0 0.0; 0.0 1.0]
    end
    use_partition_estimator::Bool = (q_int > partition_thresh) && (m == 1)
    (mu_o, Sigma_o) = (o, zeros(m,m))
    if use_partition_estimator
        S = H*Sigma_s*H' .+ R_k
        (mu_o, Sigma_o) = PredictedAltitude(p. 56)(o, mu_s, S[1,1], q_int)
    end
    (mu, Sigma) = UpdateKalmanFilter(p. 49)(mu_o, mu_s, Sigma_s, H, R_k)
    if use_partition_estimator
        K = Sigma_s*H'*inv(S)
        Sigma += K*Sigma_o*K'
    end
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: ReceivePresAltObservation(p. 82), ReceiveWgs84Observation(p. 92), AdvanceVerticalTrack(p. 53)

PREDICTEDALTITUDE (Algorithm 56) is the observation prediction step for the intruder vertical tracker. It diverges from the normal Kalman filter framework to handle the quantized nature of reported altitude measurements. The algorithm first intializes a set of partitions (*g*) of length (*L*).

Next, the denominator for a subsequent step is calculated (d). If the value of d is not a very small value (i.e., nearly zero defined by *stability*), the probability that the observation exists for each of the partition is computed (vector p). This probability density is then used to determine the observation prediction (*mu*) and the observation prediction covariance (*Sigma*). Otherwise, if d is close to zero and division by this factor is numerically unstable, the observation prediction (*mu*) is assigned a value that is plus or minus one half of a quantization level from the predicted state. In this case, the observation prediction covariance (*Sigma*) is set to zero. For more information, please see [11].

This algorithm takes as input *z*, *mu_s*, *S*, and *q*. This algorithm returns *mu*, *Sigma*.

Algorithm 56 PredictedAltitude

```
function PredictedAltitude(z::R, mu_s::Vector{R}, S::R, q::UInt32)
    stability_exponent::Z = params().surveillance.vertical.stability_exponent
    L::Z = params().surveillance.vertical.L
    stability::R = 10.0^stability_exponent
    g = zeros(L)
    for i in 0:L-1
        g[i+1] = (1/(2*L))*(((2*i+1)*(z+q/2))+((2*L-2*i-1)*(z-q/2)))
    end
    d::R = 0.0
    for k in 1:L
        d += exp(-0.5 * (g[k] - mu_s[1]) * inv(S) * (g[k] - mu_s[1]))
    end
    p = zeros(L)
    if (d > stability)
        for i = 1:L
            p[i] = (1/d)*exp(-0.5*(g[i]-mu_s[1])*inv(S)*(g[i]-mu_s[1]))
        end
        mu::R = 0.0
        for k in 1:L
            mu += (p[k] * (g[k] - mu_s[1]))
        end
        Sigma::R = 0.0
        for k in 1:L
            Sigma += (p[k] * (g[k] - mu_s[1])^2)
        end
        Sigma = Sigma - mu^2
        mu = mu + mu_s[1]
    else
        if (abs(q*round(mu_s[1]/q) - (z-q/2)) < abs(q*round(mu_s[1]/q) - (z+q/2)))
            mu = z - q/2
        else
            mu = z + q/2
        end
        Sigma = 0.0
    end
    return (mu::R, Sigma::R)
end
```

Referenced In: UpdateVerticalTracker(p. 55)

2.3 Ownship Relative Non Cooperative Track Surveillance

This section defines the algorithms to process tracked surveillance data that has been derived from a DO-366A compliant ownship relative Non-cooperative Surveillance (ORNCT). Several major functions performed by these algorithms include:

1. The association of a track report to an existing Track File.
2. The initialization of a new Track File.
3. The update of an existing Track File.

The algorithms in this section do not include the functions that select, prepare and form the intruder information that is provided to the TRM and other external modules. That process is defined in Section 2.8, specifically ADDORNCTTRACKTOREPORT (Algorithm 146).

2.3.1 Input Processing

The entry point to the STM for Ownship-relative Non-cooperative Track (ORNCT) surveillance is the algorithm RECEIVEOWNRELNONCOOPTRACK (Algorithm 57).

This algorithm takes as input the variables described in Table 17, which are the required tracked states and data fields for a track report generated by a non-cooperative surveillance system, e.g. an Air-to-Air Radar (ATAR) or an Electro-Optical-Infrared (EOIR) sensor.

The inputs received at this interface are defined in a local cylindrical coordinate frame that is, to comply with aeronautical conventions, *left-handed*. Specifically, the azimuth of an intruder, *azimuth_rad* is 0 if it is to the (true) North of ownship and $\frac{\pi}{2}$ radians if it is to the (true) East thereof. Similarly, if the intruder is flying at an altitude above that of ownship, a positive relative altitude, *z_rel_ft*, will be reported.

Prior to further processing, CHECKORNCTUNCERTAINTY (Algorithm 58) evaluates the uncertainty metrics of the received track report to determine whether STM processing is warranted based on specified acceptability criteria. The outcome of the said evaluation is expressed as the Boolean flag *pass_ornct_to_acas*. Following this check, the bit vector *trackStatus* is parsed by PARSEORNCTSTATUS (Algorithm 60) into the following Boolean flags:

- *final_update* flag indicates that the ORNCT surveillance front-end will discontinue further updates to the track;
- *track_negation* flag indicates that the track should be negated due to an error in the front-end;
- *is_coasted* flag indicates that the track is undergoing coasting due to the absence of a recent valid update; and
- *high_priority* flag indicates that the track has been designated as a high priority track, thus rendering the trac eligible for field-of-view (FOV) coasting.

Next, ASSOCIATEORNCTIDTOTARGET (Algorithm 61) attempts to associate the received report to an existing track in the Target database (*target_db*) by locating an ORNCTTRACKFILE with a matching *trackId*.

Two additional checks, in order, then take place to determine if the incoming report will be processed by STM:

1. If either the *final_update* or the *track_negation* status flag has been received, DETERMINE-ORNCTDELETION (Algorithm 62) determines whether the track should be removed from the target database, and performs such removal when necessary.
2. If the flag *pass_ornct_to_acas* is set to FALSE), the algorithm will return without further processing.

If the report passes the previous two checks, it is divided into three observation states:

- A horizontal state vector and covariance matrix (*mu_obs_cart*, *Sigma_obs_cart*) that consist of the easterly and northery displacements of the intruder from ownship and the rates of change thereof, derived through a call to ROTATEHORIZONTALFRAME (Algorithm 63);
- A vertical state vector and covariance matrix (*mu_obs_vert*, *Sigma_obs_vert*) that consist of the altitude difference between the intruder and ownship and the rates of change thereof; and
- A range state vector and covariance matrix (*mu_obs_rng*, *Sigma_obs_rng*) that consists of the ground range between the intruder and ownship and the rates of change thereof.

If no existing track has been identified as associated with the incoming report earlier during the call to ASSOCIATEORNCTIDTOTARGET, a new ORNCTTRACKFILE is created and populated with the reported states and data fields. Otherwise, the existing track is retrieved and the algorithm performs outlier detection on the incoming report.

If it is determined that the report is not an outlier or that the reset criteria have been met, UPDATEORNCTTRACK (Algorithm 67) is called to update the existing track with the received report, as well as the track status and timestamps. Otherwise, the outlier counter *odc* is decremented by one. The reset criteria require the current report to be an outlier and that:

- The outlier counter is equal to or less than *0*; or
- The ORNCT is undergoing FOV coast.

Table 17. STM Input Variables - Ownship Relelative Non-Cooperative Track

Variable	Units	Type	Description
trackId	N/A	uint32	Track number assigned by ORNCT system
trackStatus	N/A	uint8	Track status bit vector (see ORNCT_STATUS constants)
classification	N/A	uint8	Intruder classification
toa	seconds	real	Time of applicability
range_ft	feet	real	Intruder horizontal range
azimuth_rad	radians	real	Frame rotation angle from North (True North is 0, True East is $\pi/2$)
dgr_fps	ft/s	real	Intruder horizontal range rate (diverging is positive)
dxgr_fps	ft/s	real	Intruder cross horizontal range rate (perpendicular to dgr_fps, positive for clockwise movement of intru
rel_xgrgr_Sigma	feet, ft/s	real	Horizontal Covariance (4x4 matrix ordered [xgr gr dxgr dgr])
rel_z_ft	feet	real	Intruder relative altitude (intruder above is positive)
rel_dz_fps	ft/s	real	Intruder relative altitude rate (intruder above is positive)
rel_zdz_Sigma	feet, ft/s	real	Vertical Covariance (2x2 matrix ordered [z dz])

Algorithm 57 ReceiveOwnRelNonCoopTrack

```
function ReceiveOwnRelNonCoopTrack( trackId::UInt32, trackStatus::UInt8, classification::UInt8, toa::R,
     range_ft::R, azimuth_rad::R, dgr_fps::R, dxgr_fps::R, rel_xgrgr_Sigma::Matrix{R}, rel_z_ft::R,
     rel_dz_fps::R, rel_zdz_Sigma::Matrix{R} )
    outlier_threshold::R = params().surveillance.ornct.outlier_threshold
    lag_uncertainty::Vector{R} = params().surveillance.ornct.lag_uncertainty_minimum
    I_xdxydy::Vector{Z} = [1, 3, 2, 4]
    I_rdr::Vector{Z} = [2, 4]
    pass_ornct_to_acas::Bool = CheckORNCTUncertainty(p. 60)(range_ft, rel_xgrgr_Sigma, rel_z_ft, rel_zdz_Sigma)
    (final_update::Bool, track_negation::Bool, is_coasted::Bool, high_priority::Bool) =
         ParseORNCTStatus(p. 61)(trackStatus)
    targetId::UInt32 = AssociateORNCTIdToTarget(p. 61)(trackId)
    if (final_update || track_negation)
        DetermineORNCTDeletion(p. 62)(targetId, pass_ornct_to_acas, track_negation)
    elseif pass_ornct_to_acas
        mu_obs_cart = mu_obs_hor = [ 0.0, range_ft, dxgr_fps, dgr_fps ]
        Sigma_obs_cart = Sigma_obs_hor = copy(rel_xgrgr_Sigma)
        mu_obs_vert = [rel_z_ft, rel_dz_fps]
        Sigma_obs_vert = copy(rel_zdz_Sigma)
        mu_obs_rng = mu_obs_hor[I_rdr]
        Sigma_obs_rng = Sigma_obs_hor[I_rdr, I_rdr]
        (mu_obs_cart[I_xdxydy], Sigma_obs_cart[I_xdxydy,I_xdxydy]) = RotateHorizontalFrame(p. 63)(mu_obs_hor,
             Sigma_obs_hor, -azimuth_rad)
        if (targetId != NO_TARGET_FOUND)
            tgt = target_db[targetId]
            trk = tgt.ornct_track
            dt::R = toa - trk.toa
            (mu_vert_s::Vector{R}, Sigma_vert_s::Matrix{R}, _, _, mu_cart_s::Vector{R}, Sigma_cart_s::Matrix
                 {R}) = PredictORNCTTrack(p. 64)(trk, dt)
            mu_s = [mu_cart_s; mu_vert_s]
            Sigma_s = block_diag(p. 580)(Sigma_cart_s, Sigma_vert_s)
            mu_obs = [mu_obs_cart; mu_obs_vert]
            Sigma_obs = block_diag(p. 580)(Sigma_obs_cart, Sigma_obs_vert)
            Sigma_obs[1,1] = max(Sigma_obs[1,1], lag_uncertainty[1]^2)
            Sigma_obs[3,3] = max(Sigma_obs[3,3], lag_uncertainty[1]^2)
            outlier::Bool = mahal(p. 581)(mu_s, Sigma_s, mu_obs, Sigma_obs) > outlier_threshold
            reset_estimate::Bool = (outlier) && ((trk.odc <= 0) || trk.is_FOV_coast)
            if (outlier && !reset_estimate)
                trk.odc -= 1
            else
                UpdateORNCTTrack(p. 65)(trk, mu_obs_cart, Sigma_obs_cart, mu_obs_vert, Sigma_obs_vert,
                     mu_obs_rng, Sigma_obs_rng, high_priority, false, is_coasted, reset_estimate,
                     classification, toa)
            end
        else
            trk = ORNCTTrackFile(p. 484)(trackId)
            UpdateORNCTTrack(p. 65)(trk, mu_obs_cart, Sigma_obs_cart, mu_obs_vert, Sigma_obs_vert, mu_obs_rng,
                 Sigma_obs_rng, high_priority, true, is_coasted, false, classification, toa)
            AddORNCTTrackToDB(p. 64)(trk)
        end
    end
end
```

CHECKORNCTUNCERTAINTY (Algorithm 58) determines whether an input ORNCT report meets the ACAS sXu ORNCT surveillance performance requirements based on the size of the reported uncertainty.

The acceptability bound in each of the six input dimensions was derived from a simplified implementation of the ORNCT requirement as published in [3]. The variance bound calculation contains both static component values (*bound_static*) as well as multipliers for the range dependent-component of the requirement (*bound_range_dependent*) while incorporating the effects of Ele-

vation angle (*el*). Within a tracked range of *range_floor*, the range dependent component of the threshold is held to a static value.

The variance bound is scaled using the *scalar* parameter to form the uncertainty acceptability threshold. If any dimension of the reported uncertainty exceeds the corresponding threshold, the uncertainty is marked as unacceptable.

This algorithm takes as input *range_ft*, *rel_xgrgr_Sigma*, *rel_z_ft*, and *rel_zdzSigma*. This algorithm returns a true or false value for *acceptable_uncertainty*.

Algorithm 58 CheckORNCTUncertainty

```
function CheckORNCTUncertainty(range_ft::R, rel_xgrgr_Sigma::Matrix{R}, rel_z_ft::R, rel_zdz_Sigma::Matrix{R
    })
    scalar::R = params().surveillance.ornct.variance_check_scalar
    range_floor::R = params().surveillance.ornct.variance_check_range_floor
    bound_range_dependent::Vector{R} = copy(params().surveillance.ornct.variance_bound_range_dependent)
    bound_static::Vector{R} = params().surveillance.ornct.variance_bound_static
    el = abs(atan(rel_z_ft, range_ft))
    range = max(range_ft,range_floor)
    bound_range_dependent[4] = bound_range_dependent[4] * tan(el)
    variance_bound = ( range * bound_range_dependent ).^2 + bound_static.^2
    variance_bound =  variance_bound * ( scalar * cos(el) ).^2
    report_sigmas =  [diag(rel_xgrgr_Sigma); diag(rel_zdz_Sigma)]
    report_sigmas_too_large::Bool = any((report_sigmas) .> variance_bound)
    valid_hor::Bool = ValidCovarianceMatrix(p. 60)(rel_xgrgr_Sigma)
    valid_vert::Bool = ValidCovarianceMatrix(p. 60)(rel_zdz_Sigma)
    acceptable_uncertainty::Bool = (!report_sigmas_too_large && valid_hor && valid_vert)
    return acceptable_uncertainty::Bool
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59)

VALIDCOVARIANCEMATRIX (Algorithm 59) checks that a given covariance matrix is positive-definite, finite, non-NAN, and non-Infinite.

This algorithm takes as input *Sigma*. This algorithm returns a true or false value for *valid*.

Algorithm 59 ValidCovarianceMatrix

```
function ValidCovarianceMatrix(Sigma::Matrix{R})
    if any(isnan, Sigma) || any(isinf, Sigma)
        valid::Bool = false
    else
        eigenvalues = eigvals(Sigma)
        valid = (typeof(eigenvalues) != Array{Complex{R},1}) && all(eigenvalues .> 0) && !any(eigenvalues
            .== Inf)
    end
    return valid::Bool
end
```

Referenced In: CheckORNCTUncertainty(p. 60), AssembleAGTTrack(p. 72)

PARSEORNCTSTATUS (Algorithm 60) processes the *trackStatus* bit vector field provided by received ORNCT track reports. Individual flags for final track update, track negation, coasting, and

priority status are extracted using bit masks.

This algorithm takes as input *trackStatus*. This algorithm returns *final_update*, *track_negation*, *is_coasted*, and *high_priority*.

Algorithm 60 ParseORNCTStatus

```
function ParseORNCTStatus(trackStatus::UInt8)
    track_negation::Bool = (trackStatus & ORNCT_STATUS_TRACK_NEGATION) > 0
    final_update::Bool  = (trackStatus & ORNCT_STATUS_FINAL_UPDATE) > 0
    is_coasted::Bool    = (trackStatus & ORNCT_STATUS_MISSED_IN_LATEST_UPDATE) > 0
    high_priority::Bool = (trackStatus & ORNCT_STATUS_HIGH_PRIORITY_TRACK) > 0
    return (final_update::Bool, track_negation::Bool, is_coasted::Bool, high_priority::Bool)
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59)

ASSOCIATEORNCTIDTOTARGET (Algorithm 61) iterates through the target database searching for any target containing a matching ORNCT track file associated with the input track ID. If a matching track file is found the algorithm returns the target ID, otherwise, the algorithm indicates to the calling function that no target was identified.

This algorithm takes as input *trackId*. This algorithm returns *targetId*.

Algorithm 61 AssociateORNCTIdToTarget

```
function AssociateORNCTIdToTarget(trackId::UInt32)
    targetId::UInt32 = NO_TARGET_FOUND
    for id in keys(target_db)
        if (TrackExists(p. 19)(target_db[id].ornct_track)) && (target_db[id].ornct_track.ornct_id == trackId)
            targetId = id
            break
        end
    end
    return targetId::UInt32
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59)

DETERMINEORNCTDELETION (Algorithm 62) is called when an existing ORNCT has been marked for deletion by the surveillance system. If a valid *targetId* is provided, the target is retrieved from the target database and evaluated for special processing. Established ORNCTTRACKFILES (those that have received greater than *min_FOV_coast_age* report updates) may be eligible for extended coasting if they are believed to be threatening or proximate to ownship while outside the ORNCT field of view. Specifically, the track file is maintained and the *is_FOV_coast* status flag is set within the track file if and only if:

1. The track is passed to the STM for processing, as previously determined by CHECKORNCTUNCERTAINTY and indicated by the Boolean flag *pass_ornct_to_acas*;
2. The track is established (i.e. the track has received more than *min_FOV_coast_age* report updates);

3. The track is *FOV_spatially_eligible* (i.e. the track status indicates high prioirity, or the intruder is within close range);
4. The track meets *FOV_threat* criterion (i.e. an RA has been issued against the intruder in the past); and
5. The track is not negated.

Otherwise, the track is removed from the target database.

This algorithm takes as input *targetId*, *pass_ornct_to_acas*, and *track_negation*. This algorithm updates *tgt* and may update the *target_db* as well.

Algorithm 62 DetermineORNCTDeletion

```
function DetermineORNCTDeletion(targetId::UInt32, pass_ornct_to_acas::Bool, track_negation::Bool)
    close_rng_ft::R = params().surveillance.ornct.close_rng_ft
    min_FOV_coast_age::Z = params().surveillance.ornct.min_FOV_coast_age
    if (targetId != NO_TARGET_FOUND)
        tgt::Target(p. 493) = target_db[targetId]
        is_established_trk::Bool = tgt.ornct_track.update_count >= min_FOV_coast_age
        FOV_spatially_eligible::Bool = (tgt.ornct_track.high_priority) || (tgt.ornct_track.mu_rng[1] <
            close_rng_ft)
        FOV_threat::Bool = tgt.priority_codes.had_code
        if (pass_ornct_to_acas) && (is_established_trk) && (FOV_spatially_eligible) && (FOV_threat) && (!
            track_negation)
            tgt.ornct_track.is_FOV_coast = true
        else
            tgt.ornct_track = nothing
            if (TargetIsEmpty(p. 161)(tgt))
                delete!(target_db,targetId)
            end
        end
    end
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59)

ROTATEHORIZONTALFRAME (Algorithm 63) rotates a horizontal state estimate *mu_h* and covariance matrix *Sigma_h* by a specified *angle* in the horizontal dimension. As this rotation is only applied in the horizontal dimension, a 2-dimensional rotation matrix *R0* is created. The ordering of states in the input and output are expected to be $[x, y, dx, dy]$. The actual rotation is executed by a call to MATRIXROTATION (Algorithm 64).

This algorithm takes as input *mu_h*, *Sigma_h*, and *angle*. It returns *mu_cart* and *Sigma_cart*.

Algorithm 63 RotateHorizontalFrame

```
function RotateHorizontalFrame(mu_h::Vector{R}, Sigma_h::Matrix{R}, angle::R)
    R0 = [ cos(angle)  -sin(angle);
           sin(angle)   cos(angle); ]
    I_pos::Vector{Z} = [1, 2]
    I_vel::Vector{Z} = [3, 4]
    I_output::Vector{Z} = [I_pos; I_vel]
    (mu_cart, Sigma_cart) = MatrixRotation(p. 63)(mu_h, Sigma_h, I_pos, I_vel, I_output, R0)
    return (mu_cart::Vector{R}, Sigma_cart::Matrix{R})
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59), PredictORNCTTrack(p. 64), SetDisplayDataPassive(p. 146)

MATRIXROTATION (Algorithm 64) is a generalized function that can perform either 2-D or 3-D rotations of a tracked mean and covariance. The position and velocity subvectors are retrieved from *mu* using the *I_pos* and *I_vel* indices, then *mu* is rotated by *R0*. The resulting rotated position and velocities are reassembled into an output state vector *mu_R*.

The *Sigma* matrix is split up into the four submatrices determined from the position-position, position-velocity, velocity-position, and velocity-velocity terms. The rotation matrix and its transpose are pre and post multiplied, respectively, to the four submatrices. The horizontal components from the resulting submatrices are combined into a new covariance matrix (*Sigma_R*), after reordering the output based on the *I_output* indices.

This algorithm takes as input a *mu* and *Sigma*, the corresponding ordering indices for position *I_pos*, velocity *I_vel*, output ordering *I_output*, and the rotation matrix *R0*. This algorithm returns the rotated *mu_R* and *Sigma_R*.

Algorithm 64 MatrixRotation

```
function MatrixRotation( mu::Vector{R} , Sigma::Matrix{R}, I_pos::Vector{Z}, I_vel::Vector{Z},
                         I_output::Vector{Z}, R0::Matrix{R} )
    mu_p = mu[I_pos]
    mu_v = mu[I_vel]
    mu_p_R = R0 * mu_p
    mu_v_R = R0 * mu_v
    mu_pv_R = [ mu_p_R; mu_v_R]
    mu_R = mu_pv_R[I_output]
    Sigma_PP::Matrix{R} = Sigma[I_pos, I_pos]
    Sigma_PV::Matrix{R} = Sigma[I_pos, I_vel]
    Sigma_VP::Matrix{R} = Sigma[I_vel, I_pos]
    Sigma_VV::Matrix{R} = Sigma[I_vel, I_vel]
    Sigma_PP_R::Matrix{R} = R0 * Sigma_PP * R0'
    Sigma_PV_R::Matrix{R} = R0 * Sigma_PV * R0'
    Sigma_VP_R::Matrix{R} = R0 * Sigma_VP * R0'
    Sigma_VV_R::Matrix{R} = R0 * Sigma_VV * R0'
    Sigma_PPVV_R::Matrix{R} = [Sigma_PP_R Sigma_PV_R; Sigma_VP_R Sigma_VV_R]
    Sigma_R = Sigma_PPVV_R[I_output, I_output]
    Sigma_R = (Sigma_R + Sigma_R') / 2.0
    return (mu_R::Vector{R}, Sigma_R::Matrix{R})
end
```

Referenced In: RedefineEstimateInRotatedFrame(p. 42), RotateHorizontalFrame(p. 63)

Algorithm 65 AddORNCTTrackToDB

```
function AddORNCTTrackToDB(trk::ORNCTTrackFile(p. 484))
    tgt = Target(p. 493)()
    tgt.init_time = trk.toa
    tgt.ornct_track = trk
    AddToDB(p. 19)(target_db, tgt)
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59)

2.3.2 Estimation

For ORNCT tracks, the STM assumes a linear, constant-velocity dynamic model, therefore the standard Kalman Filter prediction steps are used to predict the vertical and Cartesian state estimates via calls to PREDICT1DKALMANFILTER (Algorithm 45) and PREDICT2DKALMANFILTER (Algorithm 46) respectively. ROTATEHORIZONTALFRAME (Algorithm 63) forms a predicted range state estimate by rotating the Cartesian prediction by *azimuth_rad* to produce a horizontal prediction aligned with the range axis.

This algorithm takes as input *mu_vert*, *Sigma_vert*, *mu_cart*, *Sigma_cart*, and *dt*. This algorithm returns *mu_vert_pred*, *Sigma_vert_pred*, *mu_rng_pred*, *Sigma_rng_pred*, *mu_cart_pred*, and *Sigma_cart_pred*.

Algorithm 66 PredictORNCTTrack

```
function PredictORNCTTrack(trk::ORNCTTrackFile(p. 484), dt::R)
    I_xydxdy::Vector{Z} = [1, 3, 2, 4]
    I_rdr::Vector{Z} = [2, 4]
    mu_vert::Vector{R} = trk.mu_vert
    Sigma_vert::Matrix{R} = trk.Sigma_vert
    mu_cart::Vector{R} = trk.mu_cart
    Sigma_cart::Matrix{R} = trk.Sigma_cart
    (mu_vert_pred, Sigma_vert_pred) = Predict1DKalmanFilter(p. 47)(mu_vert, Sigma_vert, 0.0, dt)
    (mu_cart_pred, Sigma_cart_pred) = Predict2DKalmanFilter(p. 47)(mu_cart, Sigma_cart, 0.0, dt)
    azimuth_rad = atan(mu_cart_pred[1],mu_cart_pred[3])
    (mu_hor_pred, Sigma_hor_pred) = RotateHorizontalFrame(p. 63)(mu_cart_pred[I_xydxdy], Sigma_cart_pred[I_xydxdy
        ,I_xydxdy], azimuth_rad)
    mu_rng_pred = mu_hor_pred[I_rdr]
    Sigma_rng_pred = Sigma_hor_pred[I_rdr, I_rdr]
    return ( mu_vert_pred::Vector{R}, Sigma_vert_pred::Matrix{R}, mu_rng_pred::Vector{R}, Sigma_rng_pred::
        Matrix{R}, mu_cart_pred::Vector{R}, Sigma_cart_pred::Matrix{R})
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59), PredictPreTrackedTrack(p. 126), ExtrapolateORNCTTrack(p. 445), AddORNCTTrackToReport(p. 152), OffsetTracksByOwnshipTurn(p. 93)

UPDATEORNCTTRACK (Algorithm 67) is used to initialize or update ORNCT track files based on updated ORNCT track reports provided through the STM entry point. The input track state estimates and uncertainties in the local ENU coordinate frame are stored in the ORNCT track file along with updated track status flags. The *is_FOV_coasted* track status flag indicates that the track is outside the FOV of the radar but is still being coasted by the STM, the *high_priority* flag indicates a high priority track as determined by the radar front-end, and *odc* is the decrementing outlier detection count. If *first_update* indicates a new track file is being initialized, the track

update_count and *toa* are initialized. Otherwise the *toa_update* and *update_count* are updated if either the *is_coasted* flag indicates the new track report is not a coasted estimate or the *reset_estimate* flag indicates the track coasted out and must be reset with the most recent report. If the track has been reset, the *toa_update* is decremented to prevent decorrelation by qualifying as a coasted update. Then a call to VERTICALRATEARROWUPDATE (Algorithm 144) is made to update the vertical rate arrow information for the display.

This algorithm takes as input *trk*, *mu_cart*, *Sigma_cart*, *mu_vert*, *Sigma_vert*, *mu_rng*, *Sigma_rng*, *high_priority*, *first_update*, *is_coasted*, *reset_estimate*, *classification*, and *toa*. This algorithm updates *trk*.

ADDORNCTTRACKTODB (Algorithm 65) takes as input *trk*. This algorithm adds the track to the database.

Algorithm 67 UpdateORNCTTrack

```
function UpdateORNCTTrack(trk::ORNCTTrackFile(p. 484),
                          mu_cart::Vector{R}, Sigma_cart::Matrix{R},
                          mu_vert::Vector{R}, Sigma_vert::Matrix{R},
                          mu_rng::Vector{R}, Sigma_rng::Matrix{R},
                          high_priority::Bool, first_update::Bool, is_coasted::Bool,
                          reset_estimate::Bool, classification::UInt8, toa::R)
    max_outliers::R = params().surveillance.ornct.max_outlier_detections
    decorrelation_offset::R = params().surveillance.decorrelation.max_coast_time
    trk.toa        = toa
    trk.mu_cart    = mu_cart
    trk.Sigma_cart = Sigma_cart
    trk.mu_vert    = mu_vert
    trk.Sigma_vert = Sigma_vert
    trk.mu_rng     = mu_rng
    trk.Sigma_rng  = Sigma_rng
    trk.is_FOV_coast = false
    trk.high_priority = high_priority
    trk.odc = max_outliers
    trk.classification = classification
    if (first_update)
        trk.toa_update = toa
        trk.update_count = 1
    else
        if (reset_estimate || !is_coasted)
            trk.toa_update = toa
            if reset_estimate
                trk.toa_update = toa - decorrelation_offset
            end
            trk.update_count = trk.update_count + 1
        end
        VerticalRateArrowUpdate(p. 148)(trk, true)
    end
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59)

2.4 Absolute Geodetic Track Surveillance

2.4.1 Input Processing

The entry point to the STM for absolute geodetic tracks is given by RECEIVEABSOLUTEGEODETICTRACK (Algorithm 68). This algorithm takes as input the variables described in Table 18, which

are the required data fields for the surveillance front-end.

Upon the reception of an absolute geodetic track, the STM first checks the track status bit vector of the reported track, *track_status*, to determine front-end indication of various track properties through a call to CHECKTRACKSTATUS (Algorithm 71). Then the validity of the WGS84 states are checked (i.e., latitude, longitude, and the easterly and northerly velocity components). If any of the states are *NaN* and the *to_be_deleted* flag is FALSE, the interface immediately returns.

Next, the reported *agt_id* field is used to associate the track either to a TARGET using ASSOCIATEAGTTOTARGET (Algorithm 69), or to OWNSHIPDATA using ASSOCIATEAGTTOOWNSHIP (Algorithm 70).

A successful association with a Target is indicated by the returning of a Target ID *id* that is not *NO_TARGET_FOUND*. The TARGET is retrieved through a call to RETRIEVEWITHID and the AGTTRACKFILE is retrieved using the track index, *idx*, which is guaranteed to not be *NaN*.

In the event that an incoming track fails to associate with any Target but successfully associates with ownship, as indicated by the following conditions:

1. The Target ID, *id*, is *NO_TARGET_FOUND*, which also guarantees that the Target track index *idx*, is *NaN*; and
2. ASSOCIATEAGTTOOWNSHIP returns a track index *idx_own* that is not *NaN*.

The track index *idx* is then overwritten with the value of *idx_own*, and the AGTTRACKFILE is retrieved therewith from OWNSHIPDATA.

At any rate, a non-*NaN* track index *idx* definitively indicates the existence of an associated track in the STM. A Boolean flag, *track_exists*, is hence set to that effect.

If a track deletion is not requested, the incoming reported measurements are converted from the global latitude-longitude-altitude (LLA) frame to the local East-North-Up (ENU) frame and assembled into vertical and horizontal means and covariances through a call to ASSEMBLEAGTTRACK (Algorithm 72). If *track_exists* indicates the presence of an associated track in the STM, the existing AGTTRACKFILE is updated using UPDATEAGTTRACK (Algorithm 76). Otherwise, a new AGTTRACKFILE is created using INITIALIZEAGTTRACK (Algorithm 74). On the other hand, if the track exists but is indicated to be deleted, DELETEAGTTRACK (Algorithm 75) will delete the track from the TARGET or from OWNSHIPDATA, whichever applies.

Table 18. STM Input Variables - Absolute Geodetic Track

Variable	Units	Type	Description
agt_id	N/A	uint32	Track identifier assigned by the external surveillance system
mode_s	N/A	uint32	Intruder 24-bit address
v2v_uid	N/A	uint32	Intruder V2V unique identifier
track_status	N/A	uint16	Status of incoming track report
externally_validated	N/A	bool	Indication that the surveillance source of the intruder is externally validated
classification	N/A	uint8	Intruder classification
lat_deg	degrees	real	Intruder latitude
lon_deg	degrees	real	Intruder longitude
vel_ew	knots	real	Intruder velocity in east-west direction, true east is positive
vel_ns	knots	real	Intruder velocity in north-south direction, true north is positive
covariance_horiz_ft_fps	feet, ft/s	real	4x4 horizontal covariance matrix represented as 16-element vector [ew dew ns dns]
alt_pres_ft	feet	real	Intruder pressure altitude
alt_rate_pres_fps	ft/s	real	Intruder pressure altitude rate
covariance_pres_ft_fps	feet, ft/s	real	2x2 vertical covariance matrix for pressure altitude [h dh]
alt_hae_ft	feet	real	Intruder geodetic height-above-ellipsoid (HAE) altitude
alt_rate_hae_fps	ft/s	real	Intruder geodetic height-above-ellipsoid (HAE) altitude rate
covariance_hae_ft_fps	feet, ft/s	real	2x2 vertical covariance matrix for HAE altitude [h dh]
toa	seconds	real	Time of applicability

Algorithm 68 ReceiveAbsoluteGeodeticTrack

```
function ReceiveAbsoluteGeodeticTrack( agt_id::UInt32, mode_s::UInt32, v2v_uid::UInt32, track_status::UInt16,
    externally_validated::Bool, classification::UInt8, lat_deg::R, lon_deg::R, vel_ew::R, vel_ns::R,
    covariance_horiz_ft_fps::Matrix{R}, alt_pres_ft::R, alt_rate_pres_fps::R, covariance_pres_ft_fps::Matrix
    {R}, alt_hae_ft::R, alt_rate_hae_fps::R, covariance_hae_ft_fps::Matrix{R}, toa::R )
    (to_be_deleted::Bool, is_coasted::Bool, high_priority::Bool, mode_s_valid::Bool, mode_s_non_icao::Bool,
        v2v_uid_valid::Bool) = CheckTrackStatus(p.70)(track_status)
    if (isnan(lat_deg) || isnan(lon_deg) || isnan(vel_ew) || isnan(vel_ns))  && !to_be_deleted
        return
    end
    (id, idx) = AssociateAGTToTarget(p.69)(agt_id)
    idx_own = AssociateAGTToOwnship(p.69)(agt_id)
    if (id != NO_TARGET_FOUND)
        tgt = RetrieveWithID(p.19)(target_db, id)
        trk = tgt.agt_tracks[idx]
    elseif (idx_own != NO_TRACK_FOUND)
        idx = idx_own
        trk = own.agt_tracks[idx]
    end
    track_exists::Bool = (idx != NO_TRACK_FOUND)
    if !to_be_deleted
        (mu_hor, Sigma_hor, mu_vert, Sigma_vert, alt_src_hae, valid_hor, valid_vert, obs_lla) =
            AssembleAGTTrack(p.72)( lat_deg, lon_deg, vel_ew, vel_ns, covariance_horiz_ft_fps, alt_pres_ft,
            alt_rate_pres_fps, covariance_pres_ft_fps, alt_hae_ft, alt_rate_hae_fps, covariance_hae_ft_fps,
            classification )
        if track_exists
            UpdateAGTTrack(p.76)( trk, id, idx, mu_hor, Sigma_hor, mu_vert, Sigma_vert, alt_src_hae, valid_hor
                , valid_vert, obs_lla, high_priority, is_coasted, mode_s, mode_s_non_icao, mode_s_valid,
                v2v_uid, v2v_uid_valid, externally_validated, classification, toa )
        elseif valid_hor
            InitializeAGTTrack(p.73)( agt_id, mu_hor, Sigma_hor, mu_vert, Sigma_vert, alt_src_hae, valid_vert,
                obs_lla, high_priority, mode_s, mode_s_non_icao, mode_s_valid, v2v_uid, v2v_uid_valid,
                externally_validated, classification, toa )
        end
    elseif track_exists
        DeleteAGTTrack(p.74)(id, idx)
    end
end
```

ASSOCIATEAGTTOTARGET (Algorithm 69) iterates through all TARGETS in the *target_db* to identify the AGTTRACKFILE that corresponds to the incoming track, if any, based on the *agt_id* field. If a corresponding track is located (i.e. the incoming track associates to the TARGET thereof), the Target ID (*id*) and AGTTRACKFILE index (*idx*) are returned. Otherwise, *id* and *idx* will return *NO_TARGET_FOUND* and *NaN*, respectively.

Algorithm 69 AssociateAGTToTarget

```
function AssociateAGTToTarget(agt_id::UInt32)
    for id in keys(target_db)
        tgt = target_db[id]
        for idx in 1:length(tgt.agt_tracks)
            trk::AGTTrackFile(p. 486) = tgt.agt_tracks[idx]
            if (trk.agt_id == agt_id)
                return (id::UInt32, idx::Z)
            end
        end
    end
    return (NO_TARGET_FOUND::UInt32, NO_TRACK_FOUND::Z)
end
```

Referenced In: ReceiveAbsoluteGeodeticTrack(p. 68)

ASSOCIATEAGTTOOWNSHIP (Algorithm 70) iterates through all AGTTRACKFILES previously associated to ownship and determines if the incoming track is associated to ownship as well. The track can only be associated to ownship if the AGT identifier (*agt_id*) matches the AGT identifier (*trk.agt_id*) of ownship. If the association is established, the AGTTRACKFILE index (*idx*) are returned. Otherwise, the absence of a corresponding track is indicated by returning *NaN*.

Algorithm 70 AssociateAGTToOwnship

```
function AssociateAGTToOwnship(agt_id::UInt32)
    for idx in 1:length(own.agt_tracks)
        trk::AGTTrackFile(p. 486) = own.agt_tracks[idx]
        if (trk.agt_id == agt_id)
            return idx::Z
        end
    end
    return NO_TRACK_FOUND::Z
end
```

Referenced In: ReceiveAbsoluteGeodeticTrack(p. 68)

CHECKTRACKSTATUS (Algorithm 71) checks the track status bit vector of the incoming track, *track_status*, to determine front-end indications for track deletion, coasting, and priority, as well as the validity and anonymity states of the Mode S address, the validity of the V2V unique identifier, whether the intruder is non-cooperative, and whether the intruder is under passive surveillance. This algorithm returns Boolean flags *to_be_deleted*, *is_coasted*, *high_priority*, *mode_s_valid*, *mode_s_non_icao*, *v2v_uid_valid*, *non_coop* and *is_passive*.

Algorithm 71 CheckTrackStatus

```
function CheckTrackStatus( track_status::UInt16 )
    to_be_deleted::Bool     = (track_status & AGT_STATUS_TO_BE_DELETED) > 0
    is_coasted::Bool        = (track_status & AGT_STATUS_MISSED_IN_LATEST_UPDATE) > 0
    high_priority::Bool     = (track_status & AGT_STATUS_HIGH_PRIORITY_TRACK) > 0
    mode_s_valid::Bool      = (track_status & AGT_STATUS_MODE_S_VALID) > 0
    mode_s_non_icao::Bool   = (track_status & AGT_STATUS_MODE_S_NON_ICAO) > 0
    v2v_uid_valid::Bool     = (track_status & AGT_STATUS_V2V_UID_VALID) > 0
    return (to_be_deleted::Bool, is_coasted::Bool, high_priority::Bool, mode_s_valid::Bool, mode_s_non_icao
        ::Bool, v2v_uid_valid::Bool)
end
```

Referenced In: ReceiveAbsoluteGeodeticTrack(p. 68)

ASSEMBLEAGTTRACK (Algorithm 72) takes the observations of the incoming absolute geodetic track from a surveillance front-end and assembles horizontal and vertical state vectors and covariance matrices.

The algorithm selects its vertical observation, namely the intruder altitude *alt*, the altitude rate *alt_rate* and the vertical covariances *Sigma_vert*, from one of the two sources:

- Pressure altitude (PA) observations: *alt_pres_ft*, *alt_rate_pres_fps* and *covariance_pres_ft_fps*; or
- Geodetic height above ellipsoid (HAE) observations: *alt_hae_ft*, *alt_rate_hae_fps* and *covariance_hae_ft_fps*).

The observations from both sources are first examined by a set of validity criteria to ascertain their respective validity (*valid_alt_pres*, *valid_alt_hae*). These criteria are:

- The altitude observation cannot be a *NaN*;
- The altitude rate observation cannot be a *NaN*;
- The covariance matrix has to be valid per VALIDCOVARIANCEMATRIX (Algorithm 59); and
- The altitude observation has to fall between *NARS_THRESHOLD_MIN* and *NARS_THRESHOLD_MAX*.

The following criteria are then applied to select one of the observation sources:

1. If only one of the two sources are valid, the valid source is selected;
2. If both sources are invalid, the PA values are selected; and
3. If both sources are valid, the source with a lower degree of altitude uncertainty is selected. For PA, in the absence of a reported uncertainty, the expected uncertainty is derived through a call to GETPRESAEMSIGMA (Algorithm 30). For HAE, this expected uncertainty is equivalent to the square root of the top-left element of the HAE covariance matrix.

Once the selection has been made, the Boolean flag *alt_src_hae* is set accordingly.

Following the altitude source selection, the observations are then expressed as vectors in two coordinate frames:

- The latitude-longitude-altitude (LLA) frame (*obs_lla*), where the latitude and the longitude are expressed in radians; and
- The East-North-Up (ENU) frame: the horizontal state vector and covariance matrix (*mu_hor*, *Sigma_hor*), and the vertical state vector and covariance matrix (*mu_vert*, *Sigma_vert*).

It is worth noting that in the case of the ENU coordinates, the horizontal components are expressed in meters and meters per second and are centered on the intruder; the x and y (easterly and northerly displacements) are therefore always zeroes. In contrast, the vertical components are expressed in feet and feet per second and are defined with respect to a global datum, whether it is the sea-level pressure (if PA observations were selected) or the WGS84 ellipsoid (if HAE observations were selected). This is to maintain consistency with the data formats of other sensor interfaces, namely the ADS-B and the V2V interface, so that the same tracker can be utilized by all three with minimal modifications.

Lastly, the validities of the horizontal and the vertical observations are ascertained and respectively stored in the Boolean flags *valid_hor* and *valid_vert*. *valid_hor* is obtained through examining the validity of the horizontal covariance matrix *covariance_horiz_ft_fps* through VALIDCOVARIANCEMATRIX. *valid_vert* is set to true if both altitude (*alt*) and altitude rate (*alt_rate*) are valid (i.e., not *NaN*), VALIDCOVARIANCEMATRIX indicates a valid vertical covariance matrix *covariance_vert_ft_fps*, and the altitude is within the NARS thresholds.

This algorithm returns *mu_hor*, *Sigma_hor*, *mu_vert*, *Sigma_vert*, *alt_src_hae*, *valid_hor*, *valid_vert*, and *obs_lla*.

Algorithm 72 AssembleAGTTrack

```
function AssembleAGTTrack( lat_deg::R, lon_deg::R, vel_ew::R, vel_ns::R, covariance_horiz_ft_fps::Matrix{R},
    alt_pres_ft::R, alt_rate_pres_fps::R, covariance_pres_ft_fps::Matrix{R}, alt_hae_ft::R, alt_rate_hae_fps
    ::R, covariance_hae_ft_fps::Matrix{R}, classification::UInt8 )
    kts2mps::R = geoutils.kts_to_mps
    m2ft::R = geoutils.meters_to_feet
    if (classification == CLASSIFICATION_SMALL_UNMANNED)
        pres_aem_sigma::Vector{R} = params().surveillance.vertical.pres_small_intruder.aem_sigma
    else
        pres_aem_sigma = params().surveillance.vertical.pres_large_intruder.aem_sigma
    end
    valid_alt_pres::Bool = !isnan(alt_pres_ft) && !isnan(alt_rate_pres_fps) && ValidCovarianceMatrix(p. 60)(
        covariance_pres_ft_fps) && (alt_pres_ft <= NARS_THRESHOLD_MAX) && (alt_pres_ft >= NARS_THRESHOLD_MIN
        )
    valid_alt_hae::Bool  = !isnan(alt_hae_ft)  && !isnan(alt_rate_hae_fps)  && ValidCovarianceMatrix(p. 60)(
        covariance_hae_ft_fps)  && (alt_hae_ft  <= NARS_THRESHOLD_MAX) && (alt_hae_ft  >= NARS_THRESHOLD_MIN
        )
    pres_alt_bias_sigma::R = GetPresAEMSigma(p. 34)(alt_pres_ft, pres_aem_sigma)
    sigma_vepu_ft::R = sqrt(covariance_hae_ft_fps[1, 1])
    alt_src_hae::Bool = valid_alt_hae && (!valid_alt_pres || (sigma_vepu_ft < pres_alt_bias_sigma))
    alt::R = alt_pres_ft
    alt_rate::R = alt_rate_pres_fps
    Sigma_vert = covariance_pres_ft_fps
    if alt_src_hae
        alt = alt_hae_ft
        alt_rate = alt_rate_hae_fps
        Sigma_vert = covariance_hae_ft_fps
    end
    obs_lla = [deg2rad(lat_deg), deg2rad(lon_deg), alt]
    mu_hor = [0.0, vel_ew, 0.0, vel_ns] * kts2mps
    Sigma_hor = covariance_horiz_ft_fps / (m2ft * m2ft)
    mu_vert::Vector{R} = [alt, alt_rate]
    valid_hor::Bool = ValidCovarianceMatrix(p. 60)(covariance_horiz_ft_fps)
    valid_vert::Bool = (alt_src_hae && valid_alt_hae) || (!alt_src_hae && valid_alt_pres)
    return (mu_hor::Vector{R}, Sigma_hor::Matrix{R}, mu_vert::Vector{R}, Sigma_vert::Matrix{R}, alt_src_hae
        ::Bool, valid_hor::Bool, valid_vert::Bool, obs_lla::Vector{R})
end
```

Referenced In: ReceiveAbsoluteGeodeticTrack(p. 68)

ADDAGTTRACKTODB (Algorithm 73) adds the input AGTTRACKFILE to a newly created TARGET object, sets the initialization time (*init_time*) of both based on the time of applicability (*toa*) of the track, and adds the TARGET to the Target database (*target_db*).

This algorithm takes as input *trk*. This algorithm modifies the *target_db* and has no return value.

Algorithm 73 AddAGTTrackToDB

```
function AddAGTTrackToDB(trk::AGTTrackFile(p. 486))
    tgt = Target(p. 493)()
    tgt.init_time = trk.toa
    trk.init_time = trk.toa
    push!(tgt.agt_tracks,trk)
    AddToDB(p. 19)(target_db, tgt)
end
```

Referenced In: InitializeAGTTrack(p. 73)

INITIALIZEAGTTRACK (Algorithm 74) is used to initialize the AGTTRACKFILE. After the track (*trk*) is created, the fields are set based on corresponding inputs. If *trk.valid_vert* is false, the altitude component of *trk.lla_at_hor_toa* is set to zero. Since *trk.lla_at_hor_toa* may change here, the corresponding ECEF position must be set with a call to CONVERTWGS84TOECEF. The track will then be added to either the Target database using ADDAGTTRACKTODB or, alternatively, to OWNSHIPDATA if the *v2v_uid* of the track matches that of ownship and the Mode S address of the track is invalid.

This algorithm takes as input *agt_id*, *mu_hor*, *Sigma_hor*, *mu_vert*, *Sigma_vert*, *alt_src_hae*, *valid_vert*, *obs_lla*, *high_priority*, *mode_s*, *mode_s_non_icao*, *mode_s_valid*, *v2v_uid*, *v2v_uid_valid*, *externally_validated*, *classification*, and *toa*. This algorithm generates no output.

Algorithm 74 InitializeAGTTrack

```
function InitializeAGTTrack( agt_id::UInt32, mu_hor::Vector{R}, Sigma_hor::Matrix{R}, mu_vert::Vector{R},
     Sigma_vert::Matrix{R}, alt_src_hae::Bool, valid_vert::Bool, obs_lla::Vector{R}, high_priority::Bool,
     mode_s::UInt32, mode_s_non_icao::Bool, mode_s_valid::Bool, v2v_uid::UInt32, v2v_uid_valid::Bool,
     externally_validated::Bool, classification::UInt8, toa::R )
    max_outliers::Z = params().surveillance.agt.max_outlier_detections
    trk = AGTTrackFile(p. 486)(agt_id)
    trk.toa         = toa
    trk.mu_hor      = mu_hor
    trk.Sigma_hor   = Sigma_hor
    trk.toa_hor     = toa
    trk.mu_vert     = mu_vert
    trk.Sigma_vert  = Sigma_vert
    trk.toa_vert    = toa
    trk.alt_src_hae = alt_src_hae
    trk.valid_vert  = valid_vert
    trk.toa_update  = toa
    trk.high_priority = high_priority
    trk.odc = max_outliers
    trk.mode_s = mode_s
    trk.mode_s_non_icao = mode_s_non_icao
    trk.mode_s_valid = mode_s_valid
    trk.v2v_uid = v2v_uid
    trk.v2v_uid_valid = v2v_uid_valid
    trk.lla_at_hor_toa = obs_lla
    if !trk.valid_vert
        trk.lla_at_hor_toa[3] = 0.0
    end
    trk.ecef_at_hor_toa = ConvertWGS84ToECEF(p. 13)(trk.lla_at_hor_toa, trk.alt_src_hae)
    trk.externally_validated = externally_validated
    trk.classification = classification
    if v2v_uid_valid && !mode_s_valid && (v2v_uid == own.v2v_uid)
        push!(own.agt_tracks, trk)
    else
        AddAGTTrackToDB(p. 72)(trk)
    end
end
```

Referenced In: ReceiveAbsoluteGeodeticTrack(p. 68)

DELETEAGTTRACK (Algorithm 75) will remove a AGTTRACKFILE from OWNSHIPDATA or TARGET based on the specified Target ID *id* and track index *idx*. If the Target ID is *NO_TARGET_FOUND*, the algorithm proceeds to delete the track of the specified index from OWNSHIPDATA and subsequently returns. Otherwise, the track of the specified index is deleted from the TARGET of the specified ID. If the TARGET is determined to be empty by TARGETISEMPTY, the TARGET will be

removed from the database. This algorithm takes *id* and *idx* as input, and generates no output.

Algorithm 75 DeleteAGTTrack

```
function DeleteAGTTrack( id::UInt32, idx::Z )
    if (id == NO_TARGET_FOUND)
        deleteat!(own.agt_tracks, idx)
    else
        tgt = RetrieveWithID(p. 19)(target_db, id)
        deleteat!(tgt.agt_tracks, idx)
        if (TargetIsEmpty(p. 161)(tgt))
            delete!(target_db, id)
        end
    end
end
```

Referenced In: ReceiveAbsoluteGeodeticTrack(p. 68), UpdateAGTTrack(p. 76)

2.4.2 Estimation

UPDATEAGTTRACK (Algorithm 76) is used to update an AGTTRACKFILE using the assembled horizontal and vertical states.

The algorithm begins with a series of examinations that culminates in the declaration of whether the current observation has been an outlier to the existing AGTTRACKFILE.

Firstly, if the horizontal measurements have not been valid, as indicated by a FALSE value in the *valid_hor* flag, the observation is considered an outlier. If the horizontal measurements are indeed valid, the algorithm then computes the Mahalanobis distance between the state of the existing track, time propagated to the *toa* (*mu_s*, *Sigma_s*), and the state of the observation (*mu_obs*, *Sigma_obs*). These states are assembled as follows:

1. First, a duplicate, *obs_lla_temp*, is created for the LLA observation vector, *obs_lla*. If the vertical component of the current observation has not been valid, as indicated by a FALSE value in the *valid_vert* flag, or that the altitude source of the current observation, *alt_src_hae* differs from that of the existing track, *trk.alt_src_hae*, the Boolean flag *use_obs_alt* is set to FALSE, and the altitude component of the *obs_lla_temp* vector is replaced with the value obtained through a call to GETALTFORHORTRKOBSERVATION (Algorithm 16).
2. An ECEF observation vector, *obs_ecef*, corresponding to *obs_lla_temp*, is computed through a call to CONVERTWGS84TOECEF (Algorithm 2).
3. The algorithm then computes a 2×2 rotation matrix *R_obs_to_trk* through POLARALIGNMENTHORIZONTALROTATION (Algorithm 24). This matrix is used to rotate the current measurements of horizontal velocity components, *mu_hor[I_dxdy]*, into the *track frame*, an East-North (EN) Cartesian frame in which the outlier detection is performed. The origin of the track frame is positioned at *trk.lla_at_hor_toa*, i.e., the coordinates of the track at the time *trk.hor_toa*.

   The redefined easterly and northerly velocity components are then respectively stored in the variables *obs_dx* and *obs_dy*.
4. The horizontal displacements then undergo a similar process. The total displacements in the ECEF frame, *obs_delta_ecef*, are computed, and then rotated into the track frame

through a call to ROTATEECEFTOENU (Algorithm 19), with the easterly (x) and northerly (y) displacement components extracted as *obs_delta_x* and *obs_delta_y*.

5. The horizontal components of the existing *trk* are then time-propagated by PREDICTPASSIVETRACKER and stored in *mu_hor_s* and *Sigma_hor_s*.
6. If the vertical components of *trk* are also valid (*trk.valid_vert*) and that the altitude observation is usable (*use_obs_alt*, the vertical components are similarly time-propagated through a call to PREDICTVERTICALTRACKER with the results stored in *mu_vert_s* and *Sigma_vert_s*. These are then combined with the their horizontal counterparts to form the predicted track state (*mu_s*, *Sigma_s*). The observation state in the track frame (*mu_obs*, *Sigma_obs*), are similarly constructed by combining the horizontal and the vertical components of the observation.

   In the event that the vertical components of the track are not valid or that the altitude observation is unusable, both the predicted track state and the observation state will only consist of their respective horizontal components.
7. Lastly, a call to MAHAL (Algorithm 465) computes the Mahalanobis distance between the predicted track state and the observation state. The distance is compared against the threshold *outlier_threshold* to determine if the latest observation has been an outlier.

The handling of an observation outlier, should one exist, subsequently occurs. If the observation has been an outlier, and the outlier detection count (*trk.odc*) is greater than zero, the outlier detection count is decremented by one, and no update to the track will take place (*i.e.* the track coasts). Alternatively, if the outlier detection count has been depleted, and the horizontal measurements are invalid, the track will be deleted with a call to DELETEAGTTRACK (Algorithm 75), and no update will take place. Otherwise, if the outlier detection count has been depleted, but the horizontal measurements are valid, the *perform_track_update* flag will be set to TRUE, thus allowing the track to re-initialize.

If the track has not been declared an outlier or has been marked for reinitialization, the track update will be performed based on a TRUE indication from the *perform_track_update* flag. Each field within the AGTTRACKFILE will be set according to its corresponding input value, and the track outlier detection count will be reset to *max_outliers*. If the vertical observation is invalid, indicated by *valid_vert* being FALSE, the altitude component of *trk.lla_at_hor_toa* is set to zero. This LLA is then converted to ECEF with a call to CONVERTWGS84TOECEF. Additionally, if incomming vertical observation is valid, indicated by *valid_vert*, the vertical rate arrow for the display will be updated using VERTICALRATEARROWUPDATE (Algorithm 144).

This algorithm takes as input *trk*, *id*, *idx*, *mu_hor*, *Sigma_hor*, *mu_vert*, *Sigma_vert*, *alt_src_hae*, *valid_hor*, *valid_vert*, *obs_lla*, *high_priority*, *is_coasted*, *mode_s*, *mode_s_non_icao*, *mode_s_valid*, *v2v_uid*, *v2v_uid_valid*, *externally_validated*, *classification* and *toa*. This algorithm modifies the input *trk* but generates no output.

Algorithm 76 UpdateAGTTrack

```
function UpdateAGTTrack( trk::AGTTrackFile(p.486), id::UInt32, idx::Z, mu_hor::Vector{R}, Sigma_hor::Matrix{R},
    mu_vert::Vector{R}, Sigma_vert::Matrix{R}, alt_src_hae::Bool, valid_hor::Bool, valid_vert::Bool,
    obs_lla::Vector{R}, high_priority::Bool, is_coasted::Bool, mode_s::UInt32, mode_s_non_icao::Bool,
    mode_s_valid::Bool, v2v_uid::UInt32, v2v_uid_valid::Bool, externally_validated::Bool, classification::
    UInt8, toa::R )
    outlier_threshold::R = params().surveillance.agt.outlier_threshold
    max_outliers::R = params().surveillance.agt.max_outlier_detections
    I_dxdy = [2, 4]
    is_outlier::Bool = true
    if valid_hor
        obs_lla_temp = copy(obs_lla)
        use_obs_alt::Bool = valid_vert && (trk.alt_src_hae == alt_src_hae)
        if !use_obs_alt
            obs_lla_temp[3] = GetAltForHorTrkObservation(p.20)(trk, toa)
        end
        obs_ecef = ConvertWGS84ToECEF(p.13)(obs_lla_temp, trk.alt_src_hae)
        R_obs_to_trk = PolarAlignmentHorizontalRotation(p.25)(obs_lla_temp, obs_ecef, trk.lla_at_hor_toa, trk.
            ecef_at_hor_toa)
        (obs_dx, obs_dy) = R_obs_to_trk * mu_hor[I_dxdy]
        obs_delta_ecef = obs_ecef - trk.ecef_at_hor_toa
        obs_delta_enu = RotateECEFToENU(p.23)(obs_delta_ecef, trk.lla_at_hor_toa)
        (obs_delta_x, obs_delta_y, nothing) = obs_delta_enu
        dt::R = toa - trk.toa
        (mu_hor_s::Vector{R}, Sigma_hor_s::Matrix{R}) = PredictPassiveTracker(p.46)(trk.mu_hor, trk.Sigma_hor, dt
            , false)
        if trk.valid_vert && use_obs_alt
            is_large::Bool = (trk.classification != CLASSIFICATION_SMALL_UNMANNED)
            (mu_vert_s::Vector{R}, Sigma_vert_s::Matrix{R}) = PredictVerticalTracker(p.54)(trk.mu_vert, trk.
                Sigma_vert, dt, trk.alt_src_hae, false, is_large)
            mu_s::Vector{R} = [mu_hor_s; mu_vert_s]
            Sigma_s::Matrix{R} = block_diag(p.580)(Sigma_hor_s, Sigma_vert_s)
            mu_obs::Vector{R} = [obs_delta_x; obs_dx; obs_delta_y; obs_dy; mu_vert]
            Sigma_obs::Matrix{R} = block_diag(p.580)(Sigma_hor, Sigma_vert)
        else
            mu_s = mu_hor_s
            Sigma_s = Sigma_hor_s
            mu_obs = [obs_delta_x; obs_dx; obs_delta_y; obs_dy]
            Sigma_obs = Sigma_hor
        end
        is_outlier = (mahal(p.581)(mu_s, Sigma_s, mu_obs, Sigma_obs) > outlier_threshold)
    end
    perform_track_update::Bool = true
    if is_outlier
        if (trk.odc > 0)
            trk.odc -= 1
            perform_track_update = false
        elseif !valid_hor
            DeleteAGTTrack(p.74)(id, idx)
            perform_track_update = false
        end
    end
    if perform_track_update
        trk.toa         = toa
        trk.mu_hor      = mu_hor
        trk.Sigma_hor   = Sigma_hor
        trk.toa_hor     = toa
        trk.mu_vert     = mu_vert
        trk.Sigma_vert  = Sigma_vert
        trk.toa_vert    = toa
        trk.alt_src_hae = alt_src_hae
        trk.valid_vert  = valid_vert
        if !is_coasted
            trk.toa_update = toa
        end
        trk.high_priority = high_priority
        trk.odc = max_outliers
        trk.mode_s = mode_s
        trk.mode_s_non_icao = mode_s_non_icao
        trk.mode_s_valid = mode_s_valid
```

```
        trk.v2v_uid = v2v_uid
        trk.v2v_uid_valid = v2v_uid_valid
        trk.lla_at_hor_toa = obs_lla
        if !trk.valid_vert
            trk.lla_at_hor_toa[3] = 0.0
        end
        trk.ecef_at_hor_toa = ConvertWGS84ToECEF(p. 13)(trk.lla_at_hor_toa, trk.alt_src_hae)
        trk.externally_validated = externally_validated
        trk.classification = classification
        if valid_vert
            VerticalRateArrowUpdate(p. 148)(trk, false)
        end
    end
end
```

Referenced In: ReceiveAbsoluteGeodeticTrack(p. 68)

2.5 Point Obstacle Surveillance

2.5.1 Input Processing

The entry point to the STM for point-obstacle surveillance reports is given by RECEIVEPOINTOBSTACLEREPORT (Algorithm 77). This algorithm takes as input the variables described in Table 19, which are the required reported data fields associated with the reception of data from a point-obstacle surveillance system.

Upon the reception of a point-obstacle surveillance report, the STM first checks if the reported latitude, longitude, and altitude are valid (i.e, not a *NaN*), or that the flag *to_be_deleted* is *true*. Next the reported *po_id* is used to associate the report to a Target using ASSOCIATEPOINTOBSTACLEREPORTTOTARGET (Algorithm 78).

If the incoming report is not associated with an existing Target, a new POINTOBSTACLETRACKFILE is created. Its fields are initialized with information from the incoming report; all incoming WGS84 coordinates are converted to ECEF form. The new POINTOBSTACLETRACKFILE is then stored in memory through ADDPOINTOBSTACLETRACKTODB (Algorithm 79).

If the call to ASSOCIATEPOINTOBSTACLEREPORTTOTARGET (Algorithm 78) successfully associates to a Target, the corresponding POINTOBSTACLETRACKFILE is updated with information from the incoming report. All incoming WGS84 coordinates are converted to ECEF form.

Table 19. STM Input Variables - Point Obstacle Report

Variable	Units	Type	Description
po_id	N/A	uint32	Point-obstacle identifier assigned by the external surveillance system
lat_deg	degrees	real	Point-obstacle latitude
lon_deg	degrees	real	Point-obstacle longitude
alt_hae_ft	feet	real	Point-obstacle geodetic height-above-ellipsoid (HAE) altitude
to_be_deleted	N/A	bool	Flag to indicate that the specified point-obstacle is to be deleted

Algorithm 77 ReceivePointObstacleReport

```
function ReceivePointObstacleReport(po_id::UInt32, lat_deg::R, lon_deg::R, alt_hae_ft::R, to_be_deleted::Bool)
    if ( !isnan(lat_deg) && !isnan(lon_deg) && !isnan(alt_hae_ft) ) || to_be_deleted
        id = AssociatePointObstacleReportToTarget(p. 78)(po_id)
        if (id == NO_TARGET_FOUND)
            trk = PointObstacleTrackFile(p. 487)()
            trk.po_id = po_id
            trk.lla_hae_rad_ft = [deg2rad(lat_deg), deg2rad(lon_deg), alt_hae_ft]
            trk.ecef_hae_m = ConvertWGS84ToECEF(p. 13)(trk.lla_hae_rad_ft, true)
            trk.to_be_deleted = to_be_deleted
            AddPointObstacleTrackToDB(p. 79)(trk)
        else
            tgt = RetrieveWithID(p. 19)(target_db, id)
            trk = tgt.point_obstacle_track
            trk.po_id = po_id
            trk.lla_hae_rad_ft = [deg2rad(lat_deg), deg2rad(lon_deg), alt_hae_ft]
            trk.ecef_hae_m = ConvertWGS84ToECEF(p. 13)(trk.lla_hae_rad_ft, true)
            trk.to_be_deleted = to_be_deleted
        end
    end
end
```

ASSOCIATEPOINTOBSTACLEREPORTTOTARGET (Algorithm 78) attempts to match a provided *po_id* to an existing point-obstacle Track File. If a matching Track File is found, the Target ID is returned; otherwise, a *NaN* is returned. The algorithm takes as inputs a point obstacle ID, *po_id*. The algorithm outputs the ID of the Target with a matching address.

Algorithm 78 AssociatePointObstacleReportToTarget

```
function AssociatePointObstacleReportToTarget(po_id::UInt32)
    for id in keys(target_db)
        tgt = target_db[id]
        if (TrackExists(p. 19)(tgt.point_obstacle_track))
            trk::PointObstacleTrackFile(p. 487) = tgt.point_obstacle_track
            if (trk.po_id == po_id)
                return (id::UInt32)
            end
        end
    end
    return NO_TARGET_FOUND::UInt32
end
```

Referenced In: ReceivePointObstacleReport(p. 78)

ADDPOINTOBSTACLETRACKTODB (Algorithm 79) adds the POINTOBSTACLETRACKFILE to the Target database (*target_db*).

This algorithm takes as input *trk*. This algorithm will modify the *target_db* but has no explicit output.

Algorithm 79 AddPointObstacleTrackToDB

```
function AddPointObstacleTrackToDB(trk::PointObstacleTrackFile(p. 487))
    tgt = Target(p. 493)()
    tgt.point_obstacle_track = trk
    AddToDB(p. 19)(target_db, tgt)
end
```

Referenced In: ReceivePointObstacleReport(p. 78)

2.6 Ownship Tracking

This section defines the algorithms to process data from sensors that measure the state of ownship. The STM is required to estimate several ownship parameters for two primary purposes:

1. To provide the TRM with an estimate of ownship state.
2. To perform the transformation of intruder surveillance data to the coordinate system used for state estimation.

The algorithms in this section do not include the functions that prepare and form the ownship state information that is provided to the TRM and other external modules. That process is defined in Section 2.8.

2.6.1 Input Processing

There are four unique entry points to the STM related to ownship. Three of the entry points handle the inputs from unique ownship sensors while the other entry point provides a means to input discrete values related to ownship. Each entry point requires a different set of input parameters as indicated in Table 20, Table 21, Table 22, Table 24, and Table 25. The following subsections define the algorithms that process the data provided by these various entry points.

2.6.2 Ownship Discretes

The STM requires knowledge of certain ownship parameters that change infrequently or only upon initialization. Such parameters are indicated in Table 20. The algorithm that handles the assignment of these ownship discrete values is RECEIVEDISCRETES (Algorithm 80).

This algorithm takes as input the items in Table 20. This algorithm updates the corresponding *own.discrete* fields using these inputs.

Table 20. STM Input Variables - Ownship Discretes

Variable	Units	Type	Description
v2v_uid	N/A	uint32	Ownship V2V unique identifier
opflg	N/A	bool	System operational flag
requested_opmode	N/A	uint8	Requested system operating mode
turn_rate_limit	rad/s	real	Ownship turn rate limit
vert_rate_limit	ft/s	real	Ownship vertical rate limit
prefer_wind_relative	N/A	bool	Flag to indicate the user preference to operate in wind-relative-coordinates
perform_poa	N/A	bool	Flag to perform point-obstacle-avoidance
disable_gpoa	N/A	bool	Flag to disable ground-point-obstacle-avoidance

Algorithm 80 ReceiveDiscretes

```
function ReceiveDiscretes(v2v_uid::UInt32, opflag::Bool, requested_opmode::UInt8, turn_rate_limit::R,
    vert_rate_limit::R, prefer_wind_relative::Bool, perform_poa::Bool, disable_gpoa::Bool)
    own.v2v_uid = v2v_uid
    own.discrete.v2v_uid = v2v_uid
    own.discrete.opflag = opflag
    own.discrete.requested_opmode = requested_opmode
    own.discrete.turn_rate_limit = turn_rate_limit
    own.discrete.vert_rate_limit = vert_rate_limit
    own.discrete.prefer_wind_relative = prefer_wind_relative
    own.discrete.perform_poa  = perform_poa
    own.discrete.disable_gpoa = disable_gpoa
end
```

2.6.3 Pressure Altitude Estimation

The estimation of ownship vertical state is performed with a Kalman filter. The following algorithms yield an estimate of ownship altitude as well as ownship vertical rate.

The entry point to the STM that handles sensor measurements from the onboard pressure altimeter is defined by RECEIVEPRESALTOBSERVATION (Algorithm 81) with input parameters given by Table 21.

Upon the reception of a sensor measurement, the algorithm first checks whether both inputs are *NaN*. This unique combination will lead to a reset of the ownship vertical track. If a valid *toa* is received, a check is made to determine if the measurement is appropriate to initialize the vertical rate state estimate, as indicated by *dt_too_small_for_vel_seed*. If so, the vertical track is predicted forward to the time of the sensor measurement through a call to PREDICTVERTICALTRACKER (Algorithm 54)). A subsequent check is performed to determine if the measurement is an outlier using either a vertical rate window or a call to ISOUTLIER (Algorithm 39). If the measurement is valid and determined to be an outlier, the outlier detection count is decremented. If the outlier detection count reaches zero and the measurement is valid, then the track is reset through a call to INITIALIZEOWNVERTICALTRACKER (Algorithm 82) and the track history is updated through a call to UPDATEHISTORY (Algorithm 83). Otherwise, only the vertical track TOA and update count are reset. If the measurement is valid and not an outlier, then the vertical track

is predicted forward to the time of sensor measurement and updated through a call to UPDATEVERTICALTRACKER (Algorithm 55), finishing with an update to the vertical track history.

This algorithm takes as input the variables in Table 21. This algorithm updates the ownship pressure altitude history and the track.

Table 21. STM Input Variables - Pressure Altitude

Variable	Units	Type	Description
alt_pres_ft	feet	real	Ownship pressure altitude
toa	seconds	real	Time of applicability

Algorithm 81 ReceivePresAltObservation

```
function ReceivePresAltObservation(alt_pres_ft::R, toa::R)
    own_history_window::Z = params().surveillance.own_history_window
    outlier_thresh::R = params().surveillance.vertical.pres_small_ownship.outlier_threshold
    window::Z = params().surveillance.vertical.outlier_window
    max_outliers::Z = params().surveillance.vertical.pres_small_ownship.max_outlier_detections
    dt_small::R = params().surveillance.vertical.dt_too_small_for_velocity
    if isnan(toa)
        if isnan(alt_pres_ft)
            own.toa_vert = NaN
            own.updates_vert = 0
            own.history.pres_alt = ValueTime(p. 495)()
        end
        return
    end
    dt = toa - own.toa_vert
    dz = alt_pres_ft - own.mu_vert[1]
    quant = UInt32(1)
    dt_small = dt_small * quant / MODE_C_QUANT
    dt_too_small_for_vel_seed::Bool = (own.updates_vert==1) && (dz!=0) && (dt < dt_small)
    if !(dt_too_small_for_vel_seed)
        (mu_s, Sigma_s) = PredictVerticalTracker(p. 54)(own.mu_vert, own.Sigma_vert, dt, false, true, false)
        if isnan(alt_pres_ft)
            outlier = true
        elseif (1 == own.updates_vert)
            outlier = (abs(dz/dt) > window)
        else
            altitude_inflation::R = (quant^2) / 12.0
            outlier = IsOutlier(p. 41)(alt_pres_ft,mu_s[1],Sigma_s[1,1]+altitude_inflation,outlier_thresh)
        end
        invalid_alt::Bool = (NARS_THRESHOLD_MAX < alt_pres_ft) || (NARS_THRESHOLD_MIN > alt_pres_ft)
        if outlier ||invalid_alt || (0 == own.updates_vert)
            if (0 < own.odc_vert) && (1 < own.updates_vert)
                own.odc_vert -= 1
            elseif !isnan(alt_pres_ft)
                (own.mu_vert, own.Sigma_vert) = InitializeOwnVerticalTracker(p. 83)(alt_pres_ft, 0.0, false)
                own.odc_vert = max_outliers
                own.toa_vert = toa
                UpdateHistory(p. 83)(own.history.pres_alt, own.mu_vert[1], toa, own_history_window)
            else
                own.toa_vert = NaN
                own.updates_vert = 0
            end
        else
            if (own.updates_vert == 1)
                own.mu_vert[2] = dz/dt
                (mu_s, Sigma_s) = PredictVerticalTracker(p. 54)(own.mu_vert, own.Sigma_vert, dt, false, true, false
                    )
            end
            (own.mu_vert, own.Sigma_vert) = UpdateVerticalTracker(p. 55)(alt_pres_ft, quant, mu_s, Sigma_s, false,
                  true, false)
            own.odc_vert = max_outliers
            own.updates_vert += 1
            own.toa_vert = toa
            UpdateHistory(p. 83)(own.history.pres_alt, own.mu_vert[1], toa, own_history_window)
        end
    end
end
```

Initialization of the ownship vertical state is performed with INITIALIZEOWNVERTICALTRACKER (Algorithm 82).

The ownship vertical tracker uses the standard Kalman filter prediction step, which is also used

by the intruder vertical tracker. Refer to section Section 2.2.3 for a description of this algorithm.

The ownship vertical tracker utilizes UPDATEVERTICALTRACKER when ownship altitude quantization is taken into consideration. A description of this algorithm is given in Section 2.2.3. When ownship quantization is not considered, the ownship vertical tracker uses the standard Kalman filter update step as defined in UPDATEKALMANFILTER.

This algorithm takes as input the measured pressure or geodetic altitude (*z_int*) of the ownship and the measured altitude rate (*dz_int*), and a flag indicating the altitude source (*is_alt_geo_-hae*). The output of the algorithm is initialized mean (*mu*) and covariance (*Sigma*) of the ownship vertical state estimate.

Algorithm 82 InitializeOwnVerticalTracker

```
function InitializeOwnVerticalTracker(z_int::R, dz_int::R, is_alt_geo_hae::Bool)
    if is_alt_geo_hae
        var_zint::R = params().surveillance.vertical.hae_small_ownship.var_zint
        var_dzint::R = params().surveillance.vertical.hae_small_ownship.var_dzint
    else
        var_zint = params().surveillance.vertical.pres_small_ownship.var_zint
        var_dzint = params().surveillance.vertical.pres_small_ownship.var_dzint
    end
    mu = [z_int, dz_int]
    Sigma = diagm(0 => [var_zint, var_dzint])
    if is_alt_geo_hae
        own.wgs84_updates_vert = 1
    else
        own.updates_vert = 1
    end
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: ReceivePresAltObservation(p. 82), ReceiveWgs84Observation(p. 92)

UPDATEHISTORY (Algorithm 83) takes a time-value pair (VALUETIME) that should be stored in the history. A specified time window, *window*, is used to remove any VALUETIME pairs that are outside this window.

This algorithm takes as input *vt*, *value*, *time*, and *window*. This algorithm updates *vt*.

Algorithm 83 UpdateHistory

```
function UpdateHistory(vt::ValueTime(p. 495), value::R, time::R, window::Z)
    push!(vt.time,time)
    push!(vt.value,value)
    for i in reverse(1:length(vt.time))
        if (vt.time[i] < time - window)
            popfirst!(vt.time)
            popfirst!(vt.value)
        end
    end
end
```

Referenced In: ReceiveHeadingObservation(p. 88), ReceivePresAltObservation(p. 82), ReceiveWgs84Observation(p. 92)

PRESALTATTOA (Algorithm 84) provides the estimated ownship pressure altitude for a specified time of applicability (*t*). This algorithm is used throughout the STM where ownship state is needed to translate intruder measurements into a local coordinate frame. PRESALTATTOA can also handle altitude predictions given time *t* in the past. If the time difference (*dt*) between the requested prediction time (*t*) and the time of the last ownship altitude update (*own.toa_h*) is less than zero and there are at least two entries of ownship altitude history (*own.history.pres_-alt*), then the returned prediction will use SURROUNDINGPOINTS to get two altitudes from the history that surround *t* and interpolate between those points (*p1* and *p2* given *time1* and *time2*). The derived *rate* is then used to calculate and return the predicted altitude value from the past. Otherwise, the ownship altitude will be extrapolated into the future by a call to PREDICTVERTICALTRACKER.

This algorithm takes as input *t*. This algorithm returns *inter_h* or the first element of *mu_s*.

Algorithm 84 PresAltAtToa

```
function PresAltAtToa(t::R)
    dt = t - own.toa_vert
    if (dt < 0) && (length(own.history.pres_alt.time) > 1)
        (time1, time2, p1, p2) = SurroundingPoints(p. 85)(own.history.pres_alt.time, own.history.pres_alt.value,
              t)
        h_rate::R = (p2 - p1) / (time2 - time1)
        inter_h::R = p1 + (h_rate * (t - time1))
        return inter_h::R
    else
        (mu_s, _) = PredictVerticalTracker(p. 54)(own.mu_vert, own.Sigma_vert, dt, false, true, false)
        return mu_s[1]::R
    end
end
```

Referenced In: AddAltBiasAndSample(p. 143), PropagateOwnshipToToa(p. 114)

SURROUNDINGPOINTS (Algorithm 85) returns two points with corresponding times to be interpolated between, given a time of applicability *toa*. If the *toa* is less than the first time stored in *times*, then the surrounding points and times are the first two entries. Otherwise, the array *times* will be searched to find two points that surround *toa*.

This algorithm takes as input *times*, *points*, and *toa*. This algorithm returns *t1*, *t2*, *p1*, and *p2*.

Algorithm 85 SurroundingPoints

```
function SurroundingPoints(times::Array{R}, points::Array{R}, toa::R)
    if (toa < times[1])
        p1 = points[1]
        p2 = points[2]
        t1 = times[1]
        t2 = times[2]
    else
        for i in 1:(length(times) - 1)
            if (times[i] <= toa) && (times[i+1] > toa)
                p1 = points[i]
                p2 = points[i+1]
                t1 = times[i]
                t2 = times[i+1]
            end
        end
    end
    return (t1::R, t2::R, p1::R, p2::R)
end
```

Referenced In: HeadingAtToa(p. 90), HAEAltAtToa(p. 96), PresAltAtToa(p. 84)

2.6.4 Height Above Ground Level Estimation

The entry point to the STM that handles sensor measurements from an onboard height above-ground-level (AGL) sensor is defined by RECEIVEHEIGHTAGLOBSERVATION (Algorithm 86) with input parameters given by Table 22. The algorithm performs no processing on the input height AGL measurement as this value is stored directly in memory. It is assumed that all credibility checking on the height AGL is performed external to the STM.

This algorithm takes as input the values in Table 22. This algorithm updates *own.h_agl*.

Table 22. STM Input Variables - Height Above Ground Level Observation

Variable	Units	Type	Description
h	feet	real	Ownship height above-ground-level (AGL) altitude

Algorithm 86 ReceiveHeightAglObservation

```
function ReceiveHeightAglObservation(h::R)
    own.h_agl = h
end
```

2.6.5 True Airspeed Estimation

The optional entry point to the STM that handles sensor measurements from the onboard air-speed indicator is defined by RECEIVETRUEAIRSPEEDOBSERVATION (Algorithm 87) with input pa-

rameters given by Table 23. The algorithm performs no processing on the input airspeed measurement as this value is stored directly in memory. It is assumed that all credibility checking on the airspeed measurement is performed external to the STM. The airspeed measurement is necessary if an implementation operates in a wind-relative coordinate system. This airspeed interface is intended to represent the component of the airspeed in the horizontal direction only (ignoring any airspeed in the vertical dimension), as this airspeed is used for the horizontal logic of the TRM. The STM algorithm SETOWNSHIPVELOCITY (Algorithm 116) passes the ownship *airspeed* set by this algorithm to the *StmReport*.

This algorithm takes as input the items in Table 23. This algorithm updates *own.airspeed*.

Table 23. STM Input Variables - True Airspeed Observation

Variable	Units	Type	Description
airspeed_kts	knots	real	Ownship true airspeed observation

Algorithm 87 ReceiveTrueAirspeedObservation

```
function ReceiveTrueAirspeedObservation(airspeed_kts::R)
    own.airspeed = airspeed_kts
end
```

2.6.6 Heading Estimation

The estimation of ownship heading is performed with a Kalman filter that is slightly modified to properly handle the subtraction of angles. The following algorithms estimate both ownship heading (used for coordinate conversion with active bearing measurements) and heading rate (to extrapolate the heading estimate). The analysis performed to properly tune the observation noise matrix (*R_k*) and the process noise matrix (*Q*) is provided by [12].

The entry point to the STM that handles measurements from the on-board heading sensor is defined by RECEIVEHEADINGOBSERVATION (Algorithm 88) with input parameters given by Table 24. When the algorithm receives a measurement (*Psi*), WRAPTOPI is called to ensure the heading angle is within the acceptable range of -π to π.

An initialized and valid track will only be updated if five conditions are all met:

1. The change in time since the last track update is greater than *min_obs_toa_step*
2. The change in time since the last track update is less than the coast limit duration (*max_heading_coasts*)
3. The degraded mode flag of the incoming measurement (*heading_degraded*) agrees with the current degraded mode state of the heading track i.e. there has not been a change in degraded mode state
4. The measurement is valid

5. The measurement is not an outlier

The outlier count (*heading_odc*) is decremented only when a measurement is identified as an outlier and all other conditions for updating the track are met. The track will be reset if the outlier count or coast limit has been exceeded.

The heading tracker uses the standard Kalman filter prediction step defined by PREDICT1DKALMANFILTER (Algorithm 45), taking as input the current estimated state (*own.mu_-heading*) and covariance (*own.Sigma_heading*).

The ownship heading track will be initialized if the track is marked as invalid via the *own.heading_-state* flag, or if the track needs to be reset because the outlier count or coast limit has been exceeded. The initialization procedures differ depending on whether the measurement is a *NaN* or not. In either case, the outlier count is reset to the maximum and the variable *own.heading_intialized* is set to TRUE; *own.heading_initialized* remains TRUE for the remaining duration of the system's operation.

The incoming heading measurement is stored in memory via UPDATEHISTORY if the measurement was not *NaN* and was used to either update or reinitialize the track.

If the ownship heading track initializes on a valid measurement (not a *NaN*), then the *own.heading_-state* is marked as normal or degraded according to the input *heading_degraded* mode and the tracker TOA is set with the TOA of the measurement. Alternatively, if the ownship heading track initializes on a *NaN*, the *own.heading_state* is marked as invalid and the tracker TOA is recorded as *NaN*.

This algorithm takes as input the variables in Table 24. This algorithm updates *own.heading*, along with other fields related to the ownship heading state.

Table 24. STM Input Variables - Heading Observation

Variable	Units	Type	Description
Psi	radians	real	Ownship heading (rad), clockwise from 0 to 2π (true north = 0, true east = $\pi/2$)
toa	seconds	real	Time of applicability
heading_degraded	N/A	bool	Degraded heading mode (If false, it indicates that input is valid true heading. If true, it indicates that input is track angle.)

Algorithm 88 ReceiveHeadingObservation

```
function ReceiveHeadingObservation(Psi::R, toa::R, heading_degraded::Bool)
    max_outliers::Z = params().surveillance.ownship_heading.max_outlier_detections
    outlier_thresh::R = params().surveillance.ownship_heading.outlier_threshold
    Q::R = params().surveillance.ownship_heading.Q
    min_obs_toa_step::R = params().surveillance.min_obs_toa_step
    max_heading_coasts::Z = params().surveillance.max_heading_coasts
    own_history_window::Z = params().surveillance.own_history_window
    Psi = WrapToPi(p. 583)(Psi)
    reset_track::Bool = false
    if (own.heading_state != OWN_HEADING_INVALID)
        dt = toa - own.toa_heading
        if (dt >= min_obs_toa_step) && (dt <= max_heading_coasts)
            degraded_mode_change::Bool = false
            if (heading_degraded) && (own.heading_state != OWN_HEADING_DEGRADED)
                degraded_mode_change = true
            elseif (!heading_degraded) && (own.heading_state == OWN_HEADING_DEGRADED)
                degraded_mode_change = true
            end
            if (!degraded_mode_change)
                if (!isnan(Psi))
                    (mu_s, Sigma_s) = Predict1DKalmanFilter(p. 47)(own.mu_heading, own.Sigma_heading, Q, dt)
                    if (IsOutlier(p. 41)(AngleDifference(p. 580)(mu_s[1],Psi), 0.0, Sigma_s[1,1], outlier_thresh))
                        own.heading_odc -= 1
                    else
                        (own.mu_heading, own.Sigma_heading) = UpdateHeadingTracker(p. 89)(Psi, mu_s, Sigma_s)
                        own.heading_odc = max_outliers
                        own.toa_heading = toa
                        UpdateHistory(p. 83)(own.history.heading, own.mu_heading[1], toa, own_history_window)
                    end
                end
            end
        end
        if (own.heading_odc < 0) || (dt > max_heading_coasts)
            reset_track = true
        end
    end
    if (own.heading_state == OWN_HEADING_INVALID) || (reset_track)
        (own.mu_heading, own.Sigma_heading) = InitializeHeadingTracker(p. 89)(Psi)
        own.heading_initialized = true
        own.heading_odc = max_outliers
        if (!isnan(Psi))
            if heading_degraded
                own.heading_state = OWN_HEADING_DEGRADED
            else
                own.heading_state = OWN_HEADING_NOMINAL
            end
            own.toa_heading = toa
            UpdateHistory(p. 83)(own.history.heading, own.mu_heading[1], toa, own_history_window)
        else
            own.heading_state = OWN_HEADING_INVALID
            own.toa_heading = NaN
        end
    end
end
```

Initialization of the heading tracker is performed with INITIALIZEHEADINGTRACKER (Algorithm 89). This algorithm sets the initial heading distribution to a Gaussian centered on a valid measurement (*Psi*) with the variance given by the observation noise (*R_k*). If the measurement is invalid (*NaN*) then the distribution is centered on zero. The initial velocity distribution is set to a zero-mean Gaussian with variance (*var_dhint*) read in from the parameters file as it cannot be

determined from a single measurement.

This algorithm takes as input *Psi*. This algorithm returns *mu* and *Sigma*.

Algorithm 89 InitializeHeadingTracker

```
function InitializeHeadingTracker(Psi::R)
    var_dhint::R = params().surveillance.ownship_heading.var_dhint
    R_k::R = params().surveillance.ownship_heading.R
    if (!isnan(Psi))
        mu = [Psi, 0.0]
    else
        mu = [0.0, 0.0]
    end
    Sigma = diagm(0 => [R_k, var_dhint])
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: ReceiveHeadingObservation(p. 88)

UPDATEHEADINGTRACKER (Algorithm 90) updates the heading tracker using a modified version of the standard Kalman filter. The algorithm departs from standard Kalman filter paradigm as the observation residual is determined through a call to WRAPANGLEDIFFTOPI (Algorithm 463), which ensures that the distance between the measurement and observation prediction is calculated properly. Otherwise, this algorithm calculates the observation covariance matrix (*S*), the Kalman gain matrix (*K*) and the updated state (*mu*) and covariance (*Sigma*) using the standard Kalman filter formulas.

This algorithm takes as input *Psi*, *mu_s*, and *Sigma_s*. This algorithm returns *mu* and *Sigma*.

Algorithm 90 UpdateHeadingTracker

```
function UpdateHeadingTracker(Psi::R, mu_s::Vector{R}, Sigma_s::Matrix{R})
    R_k::R = params().surveillance.ownship_heading.R
    H::Matrix{R} = [1.0 0.0]
    S = H*Sigma_s*H' .+ R_k
    K = Sigma_s* H'*inv(S)
    mu = mu_s + K*[WrapAngleDiffToPi(p. 580)(mu_s[1],Psi)]
    mu[1] = WrapToPi(p. 583)(mu[1])
    Sigma = Sigma_s - K*S*K'
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: ReceiveHeadingObservation(p. 88)

HEADINGATTOA (Algorithm 91) provides the estimated ownship heading for a specified time of applicability (*t*). This algorithm is used throughout the STM where ownship state is needed to translate intruder measurements into a local coordinate frame. It will return *NaN* if the heading track is uninitialized (as denoted in *own.heading_initialized*), meaning that there is no heading track to use in the prediction. This algorithm can handle heading predictions given time *t* in the past. If there are at least two entries of ownship heading history and the requested prediction time is less than the time of the last ownship heading update, then the calculation will use SURROUNDINGPOINTS (Algorithm 85) to get two headings from the history that surround *t* and

interpolate between those points (*h1* and *h2* given *time1* and *time2*). As part of the interpolation process, the derived *rate* is used to calculate the predicted heading value (constraining the value between $-\pi$ and π using WRAPTOPI (Algorithm 471)). The ownship heading will be extrapolated into the future by a call to PREDICT1DKALMANFILTER (Algorithm 45). The result that is returned is constrained between $-\pi$ and π due to WRAPTOPI. If the heading track is initialized but invalid (as denoted in *own.heading_state*), HEADINGATTOA will return zero.

This algorithm takes as input *t*. This algorithm returns *heading*.

Algorithm 91 HeadingAtToa

```
function HeadingAtToa(t::R)
    Q::R = params().surveillance.ownship_heading.Q
    heading::R = NaN
    if (own.heading_initialized)
        if (length(own.history.heading.time) > 1) && (t < maximum(own.history.heading.time))
            (time1, time2, h1, h2) = SurroundingPoints(p. 85)(own.history.heading.time, own.history.heading.
                value, t)
            rate::R = AngleDifference(p. 580)(h1, h2) / (time2 - time1)
            heading = WrapToPi(p. 583)(h1 + rate * (t - time1))
        elseif (own.heading_state != OWN_HEADING_INVALID)
            dt = t - own.toa_heading
            (mu_s, Sigma_s) = Predict1DKalmanFilter(p. 47)(own.mu_heading, own.Sigma_heading, Q, dt)
            heading = WrapToPi(p. 583)(mu_s[1])
        else
            heading = 0.0
        end
    end
    return heading::R
end
```

Referenced In: SetDisplayDataORNCT(p. 153), SetDisplayDataPassive(p. 146), SetOwnshipVelocity(p. 117)

2.6.7 WGS84 State Estimation

Table 25. STM Input Variables - WGS84 Observation

Variable	Units	Type	Description
lat_deg	degrees	real	Ownship latitude
lon_deg	degrees	real	Ownship longitude
vel_ew_kts	knots	real	Ownship velocity in east-west direction, true east is positive
vel_ns_kts	knots	real	Ownship velocity in north-south direction, true north is positive
alt_hae_ft	feet	real	Ownship geodetic height-above-ellipsoid (HAE) altitude
alt_rate_hae_fps	ft/s	real	Ownship geodetic height-above-ellipsoid (HAE) altitude rate
nacp	N/A	uint32	Ownship Navigation Accuracy Category for Position (NACp) (0-11)
nacv	N/A	uint32	Ownship Navigation Accuracy Category for Velocity (NACv) (0-4)
vfom_m	meters	real	Ownship Vertical Figure of Merit (VFOM)
toa	seconds	real	Time of applicability

The entry point to the STM that handles sensor measurements from an onboard WGS84 sensor (i.e., onboard GPS system) is defined by RECEIVEWGS84OBSERVATION (Algorithm 92) with input parameters given by Table 25. A report is considered valid when the reported time (*toa*) and horizontal observations (*lat_deg*, *lon_deg*, *vel_ew_kts*, *vel_ns_kts*) are all not *NaN*, and the NACp and NACv qualities are greater than the minimum threshold. If a received report is valid, the boolean *wgs84_previously_initialized* is first calculated to determine if there has been at least one valid report received prior to the current time step.

Next, a for loop iterates through all targets saved in the database. If *wgs84_previously_initialized* is true, all ORNCT tracks that are coasting due to their limited FOV are corrected based on the changes in ownship velocity states with a call to OFFSETTRACKSBYOWNSHIPTURN (Algorithm 93).

The current version of this document performs no further processing on the horizontal input WGS84 data as these values are stored directly in memory. Future versions of ACAS sXu may revisit this design with the inclusion of ownship state estimation and/or outlier detections to prevent erroneous state information from entering the TRM.

The estimation of ownship geodetic vertical state is performed with a Kalman filter. The following algorithms yield an estimate of ownship altitude as well as ownship vertical rate.

If the TOA associated with the ownship vertical track (*own.wgs84_toa_vert*) is *NaN* and the current altitude and altitude rate measurementes are not *NaN*, the track is initialized with INITIALIZEOWNVERTICALTRACKER (Algorithm 82) and the TOA is updated. This algorithm sets the initial altitude and altitude-rate distributions to Gaussians centered on the measured geodetic altitude (*alt_hae_ft*) and geodetic altitude-rate (*alt_rate_hae_fps*).

A call to UPDATEHISTORY (Algorithm 83) will then update the ownship geodetic altitude history to support receiving past surveillance inputs. If a track is already initialized, the track is predicted forward to the time of the sensor measurement (PREDICTVERTICALTRACKER (Algorithm 54)). If the observation is determined to be an outlier (by ISOUTLIER (Algorithm 39)), no update is performed. If the measurement is valid and determined to be an outlier, the outlier detection count is decremented. If the outlier detection count reaches zero and the measurements are valid, then the track is reset through a call to INITIALIZEOWNVERTICALTRACKER (Algorithm 82) and the track history is updated through a call to UPDATEHISTORY (Algorithm 83). Otherwise, only the vertical track TOA and update count are reset. If the measurement is valid and not an outlier, then the vertical track is updated with UPDATEVERTICALTRACKER (Algorithm 55). A call to UPDATEHISTORY will then update the ownship geodetic altitude history with the current ownship altitude estimate.

This algorithm takes as input the items in Table 25, and stores those values in memory. It also updates the ownship geodetic altitude history and the geodetic altitude track.

Algorithm 92 ReceiveWgs84Observation

```
function ReceiveWgs84Observation(lat_deg::R, lon_deg::R, vel_ew_kts::R, vel_ns_kts::R, alt_hae_ft::R,
    alt_rate_hae_fps::R, nacp::UInt32, nacv::UInt32, vfom_m::R, toa::R)
    kappa::R = params().surveillance.horizontal_adsb.kappa
    min_FOV_coast_age::R = params().surveillance.ornct.min_FOV_coast_age
    own_history_window::Z = params().surveillance.own_history_window
    outlier_thresh::R = params().surveillance.vertical.hae_small_ownship.outlier_threshold
    max_outliers::Z = params().surveillance.vertical.hae_small_ownship.max_outlier_detections
    min_nacp::UInt32 = params().surveillance.ownship_wgs84.min_nacp
    min_nacv::UInt32 = params().surveillance.ownship_wgs84.min_nacv
    max_vfom_m::R = params().surveillance.vertical.hae_small_ownship.max_vfom
    if !isnan(toa) && !isnan(lat_deg) && !isnan(lon_deg) && !isnan(vel_ew_kts) && !isnan(vel_ns_kts) && (
        nacp >= min_nacp) && (nacv >= min_nacv)
        wgs84_previously_initialized::Bool = (own.wgs84_state != OWN_WGS84_INVALID)
        own.wgs84_state = OWN_WGS84_VALID
        for id in keys(target_db)
            if (wgs84_previously_initialized)
                if (TrackExists(p. 19)(target_db[id].ornct_track))
                    trk = target_db[id].ornct_track
                    if ( (toa - trk.toa_update) > min_FOV_coast_age)
                        OffsetTracksByOwnshipTurn(p. 93)(trk, lat_deg, lon_deg, vel_ew_kts, vel_ns_kts, toa)
                    end
                end
            end
        end
        own.wgs84_lat_deg = lat_deg
        own.wgs84_lon_deg = lon_deg
        own.wgs84_vel_ew_kts = vel_ew_kts
        own.wgs84_vel_ns_kts = vel_ns_kts
        own.wgs84_alt_hae_ft = alt_hae_ft
        own.wgs84_alt_rate_hae_fps = alt_rate_hae_fps
        own.wgs84_toa = toa
        h_sigma_tmp = zeros(4,2)
        h_sigma_tmp[1,1] = h_sigma_tmp[3,2] = ConvertNACpToSigmaHEPU(p. 95)(nacp)
        h_sigma_tmp[2,1] = h_sigma_tmp[4,2] = ConvertNACvToSigmaHVA(p. 38)(nacv)
        own.wgs84_Sigma_hor = h_sigma_tmp * h_sigma_tmp'
        own.wgs84_Sigma_hor = own.wgs84_Sigma_hor + kappa*eye(p. 582)(4)
        valid_alt_inputs::Bool = !isnan(alt_hae_ft) && !isnan(alt_rate_hae_fps) && (vfom_m <= max_vfom_m)
        sigma_vepu_ft = ConvertVFOMToSigmaVEPU(p. 35)(vfom_m)
        if isnan(own.wgs84_toa_vert) && valid_alt_inputs
            (own.wgs84_mu_vert, own.wgs84_Sigma_vert) = InitializeOwnVerticalTracker(p. 83)(alt_hae_ft,
                alt_rate_hae_fps, true)
            own.wgs84_odc_vert = max_outliers
            own.wgs84_toa_vert = toa
            own.wgs84_sigma_vepu = sigma_vepu_ft
            UpdateHistory(p. 83)(own.history.hae_alt, own.wgs84_mu_vert[1], toa, own_history_window)
        else
            dt = toa - own.wgs84_toa_vert
            (mu_s, Sigma_s) = PredictVerticalTracker(p. 54)(own.wgs84_mu_vert, own.wgs84_Sigma_vert, dt, true, true
                , false)
            if !valid_alt_inputs || IsOutlier(p. 41)(alt_hae_ft,mu_s[1],Sigma_s[1,1],outlier_thresh) || (0 ==
                own.wgs84_updates_vert)
                if (0 < own.wgs84_odc_vert) && (1 < own.wgs84_updates_vert)
                    own.wgs84_odc_vert -= 1
                elseif valid_alt_inputs
                    (own.wgs84_mu_vert, own.wgs84_Sigma_vert) = InitializeOwnVerticalTracker(p. 83)(alt_hae_ft,
                        alt_rate_hae_fps, true)
                    own.wgs84_odc_vert = max_outliers
                    own.wgs84_toa_vert = toa
                    own.wgs84_sigma_vepu = sigma_vepu_ft
                    UpdateHistory(p. 83)(own.history.hae_alt, own.wgs84_mu_vert[1], toa, own_history_window)
                else
                    own.wgs84_toa_vert = NaN
                    own.wgs84_sigma_vepu = NaN
                    own.wgs84_updates_vert = 0
                end
            else
                obs = [alt_hae_ft, alt_rate_hae_fps]
```

```
                (own.wgs84_mu_vert, own.wgs84_Sigma_vert) = UpdateVerticalTracker(p. 55)(obs, UInt32(0), mu_s,
                    Sigma_s, true, true, false)
                own.wgs84_odc_vert = max_outliers
                own.wgs84_updates_vert += 1
                own.wgs84_toa_vert = toa
                own.wgs84_sigma_vepu = sigma_vepu_ft
                UpdateHistory(p. 83)(own.history.hae_alt, own.wgs84_mu_vert[1], toa, own_history_window)
            end
        end
    end
end
```

OFFSETTRACKSBYOWNSHIPTURN (Algorithm 93) is used to offset horizontal ORNCT tracks based on changes in the ownship's WGS84 velocity observations, and therefore can only be called by RECEIVEWGS84OBSERVATION if more than one WGS84 observation has been received since initialization of the horizontal track. First, a set of WGS84 velocity offsets are calculated with a call to CALCULATEOWNSHIPVELOCITYOFFSETS (Algorithm 94). These offsets are used to modify the relative cartesian velocities of the ORNCT track. Additionally, the ORNCT track is predicted forward to the time of the current WGS84 observation through a call to PREDICTORNCTTRACK (Algorithm 66) and the TOA of the ORNCT track is updated.

This algorithm takes as input *trk*, *lat*, *lon*, *vel_ew*, *vel_ns*, and *toa*. This algorithm updates *trk*.

Algorithm 93 OffsetTracksByOwnshipTurn

```
function OffsetTracksByOwnshipTurn( trk::ORNCTTrackFile(p. 484), lat::R, lon::R, vel_ew::R, vel_ns::R, toa::R )
    idx_vel::Vector{Z} = [2, 4]
    (offset_dx, offset_dy) = CalculateOwnshipVelocityOffsets(p. 94)(lat, lon, vel_ew, vel_ns)
    trk.mu_cart[idx_vel] = trk.mu_cart[idx_vel] - [offset_dx, offset_dy]
    dt = toa - trk.toa
    (trk.mu_vert, trk.Sigma_vert, trk.mu_rng, trk.Sigma_rng, trk.mu_cart, trk.Sigma_cart) =
        PredictORNCTTrack(p. 64)(trk, dt)
    trk.toa = toa
end
```

Referenced In: ReceiveWgs84Observation(p. 92)

CALCULATEOWNSHIPVELOCITYOFFSETS (Algorithm 94) uses previously saved ownship WGS84 data along with new WGS84 observations to calculate changes in ownship velocity in the east-/west and north/south directions. To ensure this calculation remains valid in polar regions of the earth, the difference in velocities are computed in ECEF coordinates. First, the previously saved east/west and north/south velocities, *own.wgs84_vel_ew_kts* and *own.wgs84_vel_ns_kts*, are converted into ECEF coordinates via a call to ROTATEENUTOECEF (Algorithm 20). This procedure requires the latitude-longitude-altitude (LLA) of the ownship when the previous velocities were saved (*own_lla_prev*), as well as the previous vertical rate estimate, *vel_ud_prev*. If the HAE tracker estimate is valid, it will be selected as the previous vertical rate estimate; otherwise, the vertical rate estimate from the pressure altitude tracker may be selected as an off-nominal condition, or *vel_ud_prev* will be set to zero if neither tracker estimates are valid. The same coordinate rotation procedure is then repeated for the new WGS84 velocities, *vel_ew* and *vel_ns*, and uses the current LLA of the ownship, *own_lla_curr*. The difference in ownship velocity from the

previous observation is calculated and stored in *delta_ecef_vel*.

The final step is to convert the change in velocity from ECEF back into ENU coordinates through a call to ROTATEECEFTOENU (Algorithm 19). When performing this conversion, the LLA of the ownship that was used when defining the horizontal track, *trk_own_lla*, is needed. For all tracks, this LLA is equal to the LLA of the previously saved WGS84 observation, *own_lla_prev*. The first two elements of the resulting vector contain the desired east/west and north/south velocity offsets which are returned by the algorithm.

This algorithm takes as input *lat*, *lon*, *vel_ew*, and *vel_ns*. This algorithm returns *offset_dx* and *offset_dy*.

Algorithm 94 CalculateOwnshipVelocityOffsets

```
function CalculateOwnshipVelocityOffsets( lat::R, lon::R, vel_ew::R, vel_ns::R)
    kts2fps::R = geoutils.kts_to_fps
    own_lla_prev::Vector{R} =
        [deg2rad(own.wgs84_lat_deg), deg2rad(own.wgs84_lon_deg), NaN]
    vel_ew_prev::R = own.wgs84_vel_ew_kts * kts2fps
    vel_ns_prev::R = own.wgs84_vel_ns_kts * kts2fps
    valid_alt_pres::Bool = !isnan(own.toa_vert)
    valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
    if valid_alt_hae
        vel_ud_prev::R = own.wgs84_mu_vert[2]
    elseif valid_alt_pres
        vel_ud_prev = own.mu_vert[2]
    else
        vel_ud_prev = 0.0
    end
    enu_vel_prev::Vector{R} = [vel_ew_prev, vel_ns_prev, vel_ud_prev]
    ecef_vel_prev::Vector{R} = RotateENUToECEF(p. 24)(enu_vel_prev, own_lla_prev)
    own_lla_curr::Vector{R} = [deg2rad(lat), deg2rad(lon), NaN]
    vel_ew_curr::R = vel_ew * kts2fps
    vel_ns_curr::R = vel_ns * kts2fps
    enu_vel_curr::Vector{R} = [vel_ew_curr, vel_ns_curr, vel_ud_prev]
    ecef_vel_curr::Vector{R} = RotateENUToECEF(p. 24)(enu_vel_curr, own_lla_curr)
    delta_ecef_vel::Vector{R} = ecef_vel_curr - ecef_vel_prev
    trk_own_lla::Vector{R} = own_lla_prev
    (offset_dx, offset_dy, _) =  RotateECEFToENU(p. 23)(delta_ecef_vel, trk_own_lla)
    return (offset_dx::R, offset_dy::R)
end
```

Referenced In: OffsetTracksByOwnshipTurn(p. 93)

CONVERTNACPTOSIGMAHEPU (Algorithm 95) performs the conversion from NACp values to their corresponding horizontal estimated position uncertainty (HEPU) values [5] to be used in the vertical tracker.

This algorithm takes as input *NACp*. This algorithm returns *sigma_hepu*.

Algorithm 95 ConvertNACpToSigmaHEPU

```
function ConvertNACpToSigmaHEPU(NACp::UInt32)
    epu::R = 0.0
    if (NACp == 0) || (NACp == 1)
        epu = 18520
    elseif (NACp == 2)
        epu = 7408
    elseif (NACp == 3)
        epu = 3704
    elseif (NACp == 4)
        epu = 1852
    elseif (NACp == 5)
        epu = 926
    elseif (NACp == 6)
        epu = 555.6
    elseif (NACp == 7)
        epu = 185.2
    elseif (NACp == 8)
        epu = 92.6
    elseif (NACp == 9)
        epu = 30
    elseif (NACp == 10)
        epu = 10
    elseif (NACp == 11)
        epu = 3
    end
    sigma_hepu = epu/2.448
    return sigma_hepu::R
end
```

Referenced In: AddHorBias(p. 140), ReceiveWgs84Observation(p. 92)

2.6.7.1 HAE Altitude Estimation

HAEAltAtToa (Algorithm 96) is used to produce a geodetic altitude at a particular TOA. If the ownship WGS84 state information is invalid the *hae_alt* is set to *NaN*. Otherwise, the altitude is derived in one of three ways:

- If the TOA is in the past and there are at least two saved points, then a call to SurroundingPoints (Algorithm 85) is used to get the two timestamps and two altitudes. The final geodetic altitude is then found by interpolating between these points;
- If *dt* is smaller than *min_extrap_toa_step*, then the geodetic altitude is set equal to the mean altitude from the track; or
- If a call to PredictVerticalTracker (Algorithm 54) is used to extrapolate the vertical track to the desired TOA.

This algorithm takes as input *t*, and returns the geodetic altitude *hae_alt*.

Algorithm 96 HAEAltAtToa

```
function HAEAltAtToa(t::R)
    min_extrap_toa_step::R = params().surveillance.min_extrap_toa_step
    if (own.wgs84_state != OWN_WGS84_INVALID)
        dt = t - own.wgs84_toa_vert
        if (dt < 0) && (length(own.history.hae_alt.time) > 1)
            (time1, time2, h1, h2) = SurroundingPoints(p. 85)(own.history.hae_alt.time, own.history.hae_alt.
                value, t)
            h_rate::R = (h2 - h1) / (time2 - time1)
            hae_alt::R = h1 + (h_rate * (t - time1))
        elseif (dt < min_extrap_toa_step)
            hae_alt = own.wgs84_mu_vert[1]::R
        else
            (mu_s, Sigma_s) = PredictVerticalTracker(p. 54)(own.wgs84_mu_vert, own.wgs84_Sigma_vert, dt, true, true
                , false)
            hae_alt = mu_s[1]::R
        end
    else
        hae_alt = NaN
    end
    return hae_alt::R
end
```

Referenced In: PropagateOwnshipToToa(p. 114)

2.7 Coordination Processing

2.7.1 Input Processing

This section defines the algorithms to process coordination data that has been received from a target aircraft. The interface utilizes V2V coordination messages. The algorithms in this section do not include the functions that provide the received coordination data to the TRM. That process is defined in Section 2.8.

2.7.1.1 V2V Coordination

The entry point to the STM for coordination data that has been received from a target aircraft is RECEIVEV2VCOORDINATION (Algorithm 97). This algorithm takes as input the variables described in Table 26, which represent the fields from a V2V coordination message that was transmitted by another aircraft.

RECEIVEV2VCOORDINATION attempts to associate the input V2V unique identifier (*v2v_uid*) to an existing Target through ASSOCIATEV2VTOTARGET. If association is unsuccessful, the coordination data is ignored. If association is successful, the corresponding target is retrieved from memory using RETRIEVEWITHID.

The RECEIVEV2VCOORDINATION algorithm runs parity bit checking on the incoming vertical sense bits (*vsb*). The Cancel Vertical Resolution Advisory Complement (*cvc*) and Vertical Resolution Advisory Complement (*vrc*) in the coordination message are used as an index into the parity table. This is achieved by using the bitwise OR between the *cvc* bit-shifted to the left by 2, and the *vrc* (all offset by 1 to adhere to Julia's 1-based indexing because the *parity_table* was defined to be 0-based indexed per DO-185B). If the intruder is not currently coordinating or canceling a previous coordination (*vrc*, *cvc*, and *vsb* are all 0), no further processing is done with the vertical coordination information. If the parity value is not equal to the *vsb* subfield of the coor-

dination message, then the data received was invalid. Invalid data is not processed further, but a flag is set within the Target to indicate the bad reception. An additional check tests whether the *cvc* or *vrc* is 3, or if the *cvc* equals the *vrc* indicating that the data received is invalid (cvc=3 and vrc=3 are defined as 'Not assigned'. See ACAS sXu MOPS Volume I [3] definition of TCAS Resolution Message.). If either *cvc* or *vrc* is 3, then the other aircraft has said that it is increasing and decreasing altitude simultaneously, so the data is nonsensical. If *cvc* and *vrc* are equal, then the other aircraft has said that it is sending and canceling a complement simultaneously, which is also nonsensical.

Messages that are validly formed are processed. First, the received coordination data time stamp (*toa*) is saved. Next, a check is performed to see if the received message contains the vertical complement to cancel (i.e., *cvc* does not equal zero), and that the cancel Vertical Resolution Advisory Complement (*vert_intent[cvc]*) matches the previously saved *vrc*. The map (*vert_intent*) is used to map codes in the coordination message into the vertical intent array given by *own.received_vrcs*. This translates the bit enunciation in the message to what is used internally in the array. The recorded *vrc* (*tgt.coord_data.vrc*) is reset and a call to DELETEINTENT (Algorithm 99) will delete the value in the vertical intent array (*own.received_vrcs*), as long as this *vrc* is not currently being received from another threat. The second argument to DELETEINTENT is a Boolean indicating that the vertical intent is being deleted rather than the horizontal intent (*own.received_vrcs* versus *own.received_hrcs*). Finally, if the received message contains the vertical complement to add (*vrc*), and the previously saved *vrc* doesn't already indicate desired value, the complement value is saved (*tgt.coord_data*), the indicated complement in the intent array is set (*own.received_vrcs*), and a call to DELETEINTENT (Algorithm 99) will delete the previous *vrc* value in the vertical intent array.

The same process is then done using the horizontal values (*hrc* and *chc*) with a couple of notable differences. First, when evaluating the index into the parity table, the *chc* is bit-shifted to the left by 3 instead of 2. Second, the horizontal coordination message is additionally invalid if *hrc* is 4 or 7. Horizontal coordination requires more information from the intruder than vertical coordination and therefore the *hrc* is 3 bits rather than 2 bits for the *vrc*. The *hrc* values of 4 and 7 are unused and therefore invalid.

This algorithm takes as input the variables in the following table. This algorithm updates *tgt.coord_data*, *own.received_vrcs* and *own.received_hrcs*.

DELETEINTENT takes as input *intent_index* and *is_vertical*. *is_vertical* determines whether the intent being deleted is vertical or horizontal. *intent_index* is an index into *own.received_vrcs* and *own.received_hrcs*, which store whether a Do Not Climb/Descend vrc or Do Not Turn Left/Right hrc is active for any intruder. The *intent_index* input indicates that an intruder is no longer issuing that intent. Assuming that the incoming intent is not 0, every target is looped through and checked for the input intent. If the input intent is not found in any target, the intent is removed from *own.received_vrcs* or *own.received_hrcs*.

Table 26. STM Input Variables - V2V Coordination Message

Variable	Units	Type	Description
v2v_uid	N/A	uint32	Intruder V2V unique identifier
cvc	N/A	uint32	Cancel Vertical RA Complement
vrc	N/A	uint32	Vertical RA Complement
vsb	N/A	uint32	Vertical Sense Bits
chc	N/A	uint32	Cancel Horizontal RA Complement
hrc	N/A	uint32	Horizontal RA Complement
hsb	N/A	uint32	Horizontal Sense Bits
toa	seconds	real	Time of Applicability

Algorithm 97 ReceiveV2VCoordination

```
function ReceiveV2VCoordination(v2v_uid::UInt32, cvc::UInt32, vrc::UInt32, vsb::UInt32, chc::UInt32, hrc::
    UInt32, hsb::UInt32, toa::R)
    parity_table_vert::Vector{Z} = params().coordination.parity_table_vert
    parity_table_hor::Vector{Z}  = params().coordination.parity_table_hor
    vert_intent::Vector{Z} = params().coordination.vert_intent
    hor_intent::Vector{Z} = params().coordination.hor_intent
    associated_ids::Vector{UInt32} = AssociateV2VToV2VAndAGTTargets(p. 100)(v2v_uid)
    for id in associated_ids
        if (id != NO_TARGET_FOUND)
            tgt = RetrieveWithID(p. 19)(target_db, id)
            parity_index::Z = (((cvc << 2) | vrc) + 1)
            parity::Z = parity_table_vert[parity_index]
            tgt.bad_v2vcoordination_vert = false
            vertically_coordinating::Bool = !((cvc == 0) && (vrc == 0) && (vsb == 0))
            if vertically_coordinating
                if (parity != vsb)
                    tgt.bad_v2vcoordination_vert = true
                elseif (cvc == 3) || (vrc == 3) || (cvc == vrc)
                    tgt.bad_v2vcoordination_vert = true
                else
                    tgt.coord_data.toa_vert = toa
                    if (cvc != 0) && (tgt.coord_data.vrc != 0) && (vert_intent[cvc] == vert_intent[tgt.
                        coord_data.vrc])
                        tgt.coord_data.vrc = 0
                        DeleteIntent(p. 100)(vert_intent[cvc], true)
                    end
                    if (vrc != 0) && (tgt.coord_data.vrc != vrc)
                        inferred_cvc = tgt.coord_data.vrc
                        tgt.coord_data.vrc = vrc
                        own.received_vrcs[vert_intent[vrc]] = true
                        if inferred_cvc != 0
                            DeleteIntent(p. 100)(vert_intent[inferred_cvc], true)
                        end
                    end
                end
            end
            tgt.bad_v2vcoordination_horz = false
            horizontally_coordinating::Bool = !((chc == 0) && (hrc == 0) && (hsb== 0))
            if horizontally_coordinating
                parity_index_hor::Z = (chc << 3 | hrc) + 1
                parity_hor::Z = parity_table_hor[parity_index_hor]
                parity_hor_valid::Bool  = (parity_hor  == hsb)
                valid_hor  = (chc != hrc) && (chc < 3) && (hrc != 3) && (hrc != 4) && (hrc != 7)
                tgt.bad_v2vcoordination_horz = (parity_hor_valid == false || valid_hor == false)
                if (!tgt.bad_v2vcoordination_horz)
                    tgt.coord_data.toa_hor = toa
                    if (chc != 0) && (tgt.coord_data.hrc != 0) && (hor_intent[chc] == hor_intent[tgt.
                        coord_data.hrc])
                        tgt.coord_data.hrc = 0
                        DeleteIntent(p. 100)(hor_intent[chc], false)
                    end
                    if (hrc != 0) && (tgt.coord_data.hrc != hrc)
                        inferred_chc = tgt.coord_data.hrc
                        tgt.coord_data.hrc = hrc
                        own.received_hrcs[hor_intent[hrc]] = true
                        if inferred_chc != 0
                            DeleteIntent(p. 100)(hor_intent[inferred_chc], false)
                        end
                    end
                end
            end
        end
    end
end
```

Algorithm 98 AssociateV2VToV2VAndAGTTargets

```
function AssociateV2VToV2VAndAGTTargets(v2v_uid::UInt32)
    associated_ids::Vector{UInt32} = UInt32[]
    for id in keys(target_db)
        tgt = target_db[id]
        if (TrackExists(p. 19)(tgt.v2v_track))
            v2v_trk::V2VTrackFile(p. 488) = tgt.v2v_track
            if (v2v_trk.v2v_uid == v2v_uid)
                push!(associated_ids, id)
            end
        end
        for idx in 1:length(tgt.agt_tracks)
            agt_trk::AGTTrackFile(p. 486) = tgt.agt_tracks[idx]
            if agt_trk.v2v_uid_valid && (agt_trk.v2v_uid == v2v_uid)
                push!(associated_ids, id)
            end
        end
    end
    return associated_ids::Vector{UInt32}
end
```

Referenced In: ReceiveV2VOperationalStatusMessage(p. 103), ReceiveV2VCoordination(p. 99)

Algorithm 99 DeleteIntent

```
function DeleteIntent(intent_index::Z, is_vertical::Bool)
    vert_intent::Vector{Z} = params().coordination.vert_intent
    hor_intent::Vector{Z} = params().coordination.hor_intent
    intent_found::Bool = false
    if (intent_index != 0)
        for id in keys(target_db)
            tgt = RetrieveWithID(p. 19)(target_db, id)
            vert_intent_found::Bool = (is_vertical) && (tgt.coord_data.vrc != 0) && (intent_index ==
                 vert_intent[tgt.coord_data.vrc])
            hor_intent_found::Bool = (!is_vertical) && (tgt.coord_data.hrc != 0) && (intent_index ==
                 hor_intent[tgt.coord_data.hrc])
            if (vert_intent_found) || (hor_intent_found)
                intent_found = true
            end
        end
        if (!intent_found)
            if (is_vertical)
                own.received_vrcs[intent_index] = false
            else
                own.received_hrcs[intent_index] = false
            end
        end
    end
    return
end
```

Referenced In: ReceiveV2VCoordination(p. 99), CoordinationTimeoutCheck(p. 102)

The UPDATEINTRUDERVRC (Algorithm 100) algorithm takes in a list of all valid TRM *intruders* to be updated with the most recent VRC. The algorithm loops over the *target_db* and all the *intruders* for which the TRM has requested an update to find the corresponding target in the STM database. If the IDs match, the algorithm first checks for the case where the intruder is equipped with *EQUIPAGE_LARGE_CAS* and a non-zero VRC or non-zero HRC was received from

the intruder. This is a case where a ACAS Xr senior has sent coordination message to the ownship between STM processing and this reachback algorithm, and the *intruder.equipage* and *intruder.own_coordination_policy* are updated appropriately. If the IDs match and the intruder is equipped and capable of coordinating with the ownship, the intruder VRC will be updated using the target in the STM database to ensure that the most recent VRC is used by the TRM. The combination of all VRCs received from existing intruders are returned in *received_vrcs* and *received_hrcs* to form the Resolution Advisory Complements (RAC).

This algorithm takes as input *intruders*. This algorithm returns *received_vrcs*.

Algorithm 100 UpdateIntruderVRC

```
function UpdateIntruderVRC(intruders::Vector{TRMIntruderInput(p. 499)})
    for id in keys(target_db)
        for intruder::TRMIntruderInput(p. 499) in intruders
            if (intruder.id == id)
                if (intruder.equipage == EQUIPAGE_LARGE_CAS) && ((target_db[id].coord_data.vrc != 0) || (
                     target_db[id].coord_data.hrc != 0))
                    intruder.equipage = EQUIPAGE_XR_V2V
                    intruder.own_coordination_policy = OWN_COORDINATION_POLICY_JUNIOR
                end
                if (intruder.equipage != EQUIPAGE_NONE) && (intruder.equipage != EQUIPAGE_TCAS) && (intruder
                     .equipage != EQUIPAGE_LARGE_CAS) && (intruder.equipage != EQUIPAGE_CASTA)
                    intruder.vrc = target_db[id].coord_data.vrc
                end
            end
        end
    end
    received_vrcs::Vector{Bool} = copy( own.received_vrcs )
    return received_vrcs::Vector{Bool}
end
```

Referenced In: UpdateIntruderInputs(p. 307), VerticalTRMUpdate(p. 241), SetVerticalTRMSurvOnlyMode(p. 247)

Similar to UPDATEINTRUDERVRC, the UPDATEINTRUDERHRC (Algorithm 101) algorithm takes in a list of all valid TRM intruders TRMINTRUDERINPUT (*intruders*) to be updated with the most recent HRC. The algorithm loops over the *target_db* and all the *intruders* for which the TRM has requested an update to find the corresponding target in the STM database. If the IDs match, the algorithm first checks for the case where the intruder is equipped with *EQUIPAGE_LARGE_CAS* and a non-zero VRC or non-zero HRC was received from the intruder. This is a case where a ACAS Xr senior has sent a coordination message to the ownship between STM processing and this reachback algorithm, and the *intruder.equipage* and *intruder.own_coordination_policy* are updated appropriately. If the IDs match and the intruder is equipped and capable of coordinating with the ownship, the intruder HRC will be updated using the target in the STM database to ensure that the most recent HRC is used by the TRM. The combination of all HRCs received from existing intruders is returned in *received_hrcs* to form the Resolution Advisory Complements (RAC).

This algorithm takes as input *intruders*. This algorithm returns *received_hrcs*.

Algorithm 101 UpdateIntruderHRC

```
function UpdateIntruderHRC(intruders::Vector{TRMIntruderInput(p. 499)})
    for id in keys(target_db)
        for intruder::TRMIntruderInput(p. 499) in intruders
            if ( intruder.id == id )
                if (intruder.equipage == EQUIPAGE_LARGE_CAS) && ((target_db[id].coord_data.vrc != 0) || (
                     target_db[id].coord_data.hrc != 0))
                    intruder.equipage = EQUIPAGE_XR_V2V
                    intruder.own_coordination_policy = OWN_COORDINATION_POLICY_JUNIOR
                end
                if (intruder.equipage != EQUIPAGE_NONE) && (intruder.equipage != EQUIPAGE_TCAS) && (intruder
                     .equipage != EQUIPAGE_LARGE_CAS) && (intruder.equipage != EQUIPAGE_CASTA)
                    intruder.hrc = target_db[id].coord_data.hrc
                end
            end
        end
    end
    received_hrcs::Vector{Bool} = copy( own.received_hrcs )
    return received_hrcs::Vector{Bool}
end
```

Referenced In: HorizontalTRMUpdate(p. 168), UpdateHTRMIntruderInputs(p. 173)

COORDINATIONTIMEOUTCHECK (Algorithm 102) takes as input a target retrieved from memory along with the time of the generated STM report. If the difference in the *report_time* and the time of the last known coordination update is greater than or equal to the *timeout*, the coordination data of that target is reset and the vertical or horizontal intent is removed by calling DELETEINTENT.

This algorithm takes as input *tgt* and *report_time*. This algorithm updates *tgt.coord_data*, *own.received_vrcs*, and *own.received_hrcs* (inside DELETEINTENT), if necessary.

Algorithm 102 CoordinationTimeoutCheck

```
function CoordinationTimeoutCheck(tgt::Target(p. 493), report_time::R)
    timeout::Z = params().coordination.timeout
    vert_intent::Vector{Z} = params().coordination.vert_intent
    hor_intent::Vector{Z} = params().coordination.hor_intent
    cancel_index_vert::Z = 0
    cancel_index_hor::Z = 0
    if (tgt.coord_data.toa_vert > 0) && ((report_time - tgt.coord_data.toa_vert) >= timeout)
        if (tgt.coord_data.vrc > 0)
            cancel_index_vert = vert_intent[tgt.coord_data.vrc]
        end
        tgt.coord_data.toa_vert = 0.0
        tgt.coord_data.vrc = 0
        DeleteIntent(p. 100)(cancel_index_vert, true)
    end
    if (tgt.coord_data.toa_hor > 0) && ((report_time - tgt.coord_data.toa_hor) >= timeout)
        if (tgt.coord_data.hrc > 0)
            cancel_index_hor = hor_intent[tgt.coord_data.hrc]
        end
        tgt.coord_data.toa_hor = 0.0
        tgt.coord_data.hrc = 0
        DeleteIntent(p. 100)(cancel_index_hor, false)
    end
end
```

Referenced In: RemoveStaleTracks(p. 159)

2.7.1.2 V2V Operational Status Message

The STM includes an entry point for V2V operational status message in RECEIVEV2VOPERATIONALSTATUSMESSAG (Algorithm 103) with inputs defined in Table 27. The algorithm stores information received via the V2V operational status message. The intruder's *v2v_uid* is used to associate the received data with an existing Target. If an existing Target is found, the bits are stored in the Target structure to determine equipage and coordination protocol during STM report generation.

This algorithm takes as input the variables in the following table. This algorithm updates *tgt*.

Table 27. STM Input Variables - V2V Operational Status Message

Variable	Units	Type	Description
ca_status	N/A	uint8	Indicates the operational status of collision avoidance system
sense	N/A	uint8	Vertical and/or horizontal sense bit(s) (reserved)
type_capability	N/A	uint8	CAS and type capability
priority	N/A	uint8	Priority flag
equipment	N/A	uint16	Intruder equipment
v2v_uid	N/A	uint32	Intruder V2V unique identifier

Algorithm 103 ReceiveV2VOperationalStatusMessage

```
function ReceiveV2VOperationalStatusMessage(ca_status::UInt8, sense::UInt8, type_capability::UInt8, priority
    ::UInt8, equipment::UInt16, v2v_uid::UInt32)
    associated_ids::Vector{UInt32} = AssociateV2VToV2VAndAGTTargets(p. 100)(v2v_uid)
    for id in associated_ids
        if (id != NO_TARGET_FOUND)
            tgt = RetrieveWithID(p. 19)(target_db, id)
            tgt.v2v_osm.ca_status = ca_status
            tgt.v2v_osm.v2v_cccb.sense = sense
            tgt.v2v_osm.v2v_cccb.type_capability = type_capability
            tgt.v2v_osm.v2v_cccb.priority = priority
            tgt.v2v_osm.equipment = equipment
        end
    end
end
```

During STM report generation, SETCOORDINATION (Algorithm 104) is called to further update the intruder equipage and coordination protocol flags reported for the intruder. Based on the V2V operational status message and ADS-B operational status message received from the intruder, the algorithm updates the fields of the *intruder* structure accordingly. These fields will specify if the ownship and intruder are able to coordinate, and if the coordination type is passive or responsive. Additionally, these fields will be used downstream in TRM calculations (for example, when checking if an equipped intruder able to coordinate with the ownship or if the intruder is the master or senior).

This algorithm takes as input *intruder*. This algorithm updates *intruder*.

Algorithm 104 SetCoordination

```
function SetCoordination(intruder::TRMIntruderInput(p. 499))
    intruder.own_coordination_policy = OWN_COORDINATION_POLICY_NONE
    intruder.equipage = EQUIPAGE_NONE
    intruder.coordination_msg = COORDINATION_NONE
    tgt = RetrieveWithID(p. 19)(target_db, intruder.id)
    if (tgt.v2v_osm.ca_status == CA_STATUS_RA)
        if (tgt.v2v_osm.v2v_cccb.type_capability == CAS_V2V_SXU) || ((tgt.v2v_osm.v2v_cccb.type_capability
             == CAS_V2V_XR) && (own.v2v_osm.equipment == tgt.v2v_osm.equipment))
            if (own.v2v_osm.v2v_cccb.priority == tgt.v2v_osm.v2v_cccb.priority)
                intruder.own_coordination_policy = OWN_COORDINATION_POLICY_PEER
                intruder.equipage = EQUIPAGE_CASRA
                intruder.coordination_msg = COORDINATION_V2V_OCM
            elseif (own.v2v_osm.v2v_cccb.priority > tgt.v2v_osm.v2v_cccb.priority)
                intruder.own_coordination_policy = OWN_COORDINATION_POLICY_SENIOR
                intruder.equipage = EQUIPAGE_CASRESP
                intruder.coordination_msg = COORDINATION_V2V_OCM
            elseif (own.v2v_osm.v2v_cccb.priority < tgt.v2v_osm.v2v_cccb.priority)
                intruder.own_coordination_policy = OWN_COORDINATION_POLICY_JUNIOR
                intruder.equipage = EQUIPAGE_SXU_SENIOR
                intruder.coordination_msg = COORDINATION_NONE
            end
        elseif (tgt.v2v_osm.v2v_cccb.type_capability == CAS_V2V_XR) && (own.v2v_osm.equipment != tgt.v2v_osm
             .equipment)
            intruder.own_coordination_policy = OWN_COORDINATION_POLICY_JUNIOR
            intruder.equipage = EQUIPAGE_XR_V2V
            intruder.coordination_msg = COORDINATION_NONE
        end
    elseif (tgt.v2v_osm.ca_status == CA_STATUS_RA_INHIBITED)
        intruder.own_coordination_policy = OWN_COORDINATION_POLICY_NONE
        intruder.equipage = EQUIPAGE_CASTA
        intruder.coordination_msg = COORDINATION_NONE
    else
        if (tgt.adsb_osm.adsb_cccb.type_capability != CAS_ACTIVE_TCAS)
            if (tgt.coord_data.vrc != 0) || (tgt.coord_data.hrc != 0)
                intruder.own_coordination_policy = OWN_COORDINATION_POLICY_JUNIOR
                intruder.equipage = EQUIPAGE_XR_V2V
                intruder.coordination_msg = COORDINATION_NONE
            else
                intruder.own_coordination_policy = OWN_COORDINATION_POLICY_NONE
                intruder.equipage = EQUIPAGE_LARGE_CAS
                intruder.coordination_msg = COORDINATION_NONE
            end
        else
            if (tgt.adsb_osm.ca_operational == 1)
                intruder.own_coordination_policy = OWN_COORDINATION_POLICY_NONE
                intruder.equipage = EQUIPAGE_TCAS
                intruder.coordination_msg = COORDINATION_NONE
            elseif (tgt.adsb_osm.ca_operational == 0)
                intruder.own_coordination_policy = OWN_COORDINATION_POLICY_NONE
                intruder.equipage = EQUIPAGE_NONE
                intruder.coordination_msg = COORDINATION_NONE
            end
        end
    end
end
```

Referenced In: AddADSBTrackToReport(p. 135), AddPointObstacleTrackToReport(p. 157), AddORNCTTrackToReport(p. 152), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

2.7.1.3 ADS-B Operational Status Message

The STM includes an entry point for ADS-B operational status message in RECEIVEADSBOPERATIONALSTATUSMESSAGE (Algorithm 105) with inputs defined in Table 28. The algorithm stores information received from the Collision Avoidance Coordination Capability Bits (CCCB), the CA Operational field, and DAA Subfields received via the ADS-B operational status message. CCCB only refers to *sense*, *type_capability*, and *priority*. The DAA bits (*daa*) and CA Operational (*ca_operational*) subfields are also received. The intruder's *mode_s* address and *non_icao* flag are used to associate the received data with an existing Target. If an existing Target is found, the bits are stored in the Target structure to determine equipage and coordination protocol during STM report generation.

This algorithm takes as input the variables in the following table. This algorithm updates *tgt*.

Table 28. STM Input Variables - ADS-B Operational Status Message

Variable	Units	Type	Description
ca_operational	N/A	uint8	Indicates that a collision avoidance system is operational
sense	N/A	uint8	Vertical and/or horizontal sense bit(s) (reserved)
type_capability	N/A	uint8	CAS and type capability
priority	N/A	uint8	Priority
daa	N/A	uint8	Detect and Avoid subfield that indicates the coordination type to be provided
mode_s	N/A	uint32	Intruder 24-bit aircraft address
non_icao	N/A	bool	Flag to indicate that the address is a non-ICAO 24-bit aircraft address (anonymous address)

Algorithm 105 ReceiveADSBOperationalStatusMessage

```
function ReceiveADSBOperationalStatusMessage(ca_operational::UInt8, sense::UInt8, type_capability::UInt8,
     priority::UInt8, daa::UInt8, mode_s::UInt32, non_icao::Bool)
    id = AssociateADSBReportToTarget(p. 16)(mode_s, non_icao)
    if (id != NO_TARGET_FOUND)
        tgt = RetrieveWithID(p. 19)(target_db, id)
        tgt.adsb_osm.ca_operational = ca_operational
        tgt.adsb_osm.adsb_cccb.sense = sense
        tgt.adsb_osm.adsb_cccb.type_capability = type_capability
        tgt.adsb_osm.adsb_cccb.priority = priority
        tgt.adsb_osm.daa = daa
    end
end
```

2.8 Track Management

This section defines the algorithms that perform maintenance on existing Targets stored in memory and produces reports that are used by other modules in the ACAS X system. The major functions encompassed by these algorithms include:

1. The update of the ownship advisory mode using UPDATEADVISORYMODE.
2. The determination and packaging of V2V operational status message data to send using SETOWNV2VOPERATIONALSTATUSMESSAGE.
3. The extrapolation, decorrelation, and correlation of individual tracks using CORRELATION-PROCESSING.
4. The selection of the surveillance source that is used to generate the STMREPORT.
5. The generation of the STMREPORT for consumption by the TRM and other external modules.
6. The deletion of deprecated Track Files.
7. The algorithms for validation of passive tracks.

2.8.1 Mode Management

UPDATEADVISORYMODE (Algorithm 106) sets the operating mode (*own.opmode*) based on the requested operating mode (*own.requested_opmode*) that is set in RECEIVEDISCRETES. If *own.discrete.opflag* is false or *own.requested_opmode* is *OPMODE_STANDBY*, then the operating mode is set to standby mode. If *own.requested_opmode* is *OPMODE_SURV_ONLY*, then the operating mode is set to surveillance only mode. Otherwise, the requested operating mode will be equal to *(*OPMODE_RA), and the operating mode is therefore set to RA mode.

This algorithm takes no input. This algorithm updates *own.opmode*.

Algorithm 106 UpdateAdvisoryMode

```
function UpdateAdvisoryMode()
    if (own.discrete.opflag == false) || (own.discrete.requested_opmode == OPMODE_STANDBY)
        own.opmode = OPMODE_STANDBY
    elseif (own.discrete.requested_opmode == OPMODE_SURV_ONLY)
        own.opmode = OPMODE_SURV_ONLY
    else
        own.opmode = OPMODE_RA
    end
    return
end
```

Referenced In: GenerateStmReport(p. 112)

REFINEOWNSHIPAGL (Algorithm 107) takes the ownship's height AGL input and attempts to determine if the provided AGL is actually height above the top of a building or other object represented as a point-obstacle instead of height above-ground-level. If it is determined that the provided AGL is height above the top of the point-obstacle, then this algorithm attempts to generate a refined value that represents the true height above-ground-level.

The first step is to iterate through all the point-obstacles in the target database to find which one has the closest ground-range to the ownship. Since the relative position between each point-obstacle and the ownship is calculated in global geodetic coordinates, the altitude type used to represent the point-obstacle and ownship must first be aligned with a call to ALIGNALTITUDE-TYPESUSEDINGEODETICPOSITIONS. Once the altitude types are properly aligned, the relative ECEF position is calculated and then rotated into relative ENU coordinates centered about the ownship by a call to ROTATEECEFTOENU. This relative ENU position is then used to calculate

the ground range. If the current point-obstacles ground range is less than the minimum ground range, the minimum ground range is updated and the index of the current point-obstacle is saved off as *id_min*.

The second part of this algorithm first checks to make sure there was at least one point-obstacle in the target database. If at least one exists, the point-obstacle with the minimum ground range to the ownship is extracted from the target database. The relative altitude to this point obstacle is then calculated. Next, if the ground range is less than *po_range_thresh* and the difference between the ownship's inputted AGL and the relative altitude is less than *po_height_thresh*, then the ownship's inputted AGL altitude actually represents height above the top of the point-obstacle and is refined by adding the AGL of the point-obstacle.

This algorithm takes as input *own_agl_input*. This algorithm returns *own_agl_refined*.

Algorithm 107 RefineOwnshipAGL

```
function RefineOwnshipAGL(own_agl_input::R)
    po_range_thresh::R = params().surveillance.ownship_agl.po_range_thresh
    po_height_thresh::R = params().surveillance.ownship_agl.po_height_thresh
    m2ft::R = geoutils.meters_to_feet
    own_agl_refined = copy(own_agl_input)
    if !isnan(own.wgs84_toa_vert)
        r_ground_min = Inf
        id_min = NaN
        for id in keys(target_db)
            tgt = target_db[id]
            if (TrackExists(p. 19)(tgt.point_obstacle_track))
                trk = tgt.point_obstacle_track
                (_, ecef, own_lla, own_ecef) = AlignAltitudeTypesUsedInGeodeticPositions(p. 109)(trk.lla_hae_rad_ft,
                     trk.ecef_hae_m, true, true)
                delta_ecef = ecef - own_ecef
                (delta_ew, delta_ns, _) = RotateECEFToENU(p. 23)(delta_ecef, own_lla)
                r_ground = hypot(delta_ew, delta_ns) * m2ft
                if (r_ground_min > r_ground)
                    r_ground_min = r_ground
                    id_min = id
                end
            end
        end
        if !isnan(id_min)
            trk_po = target_db[id_min].point_obstacle_track
            hae_po = trk_po.lla_hae_rad_ft[3]
            hae_own = own.geo_states_hae_alt.lla_rad_ft[3]
            own_geodetic_height_above_po = hae_own - hae_po
            if (r_ground_min < po_range_thresh) && (abs(own_agl_input - own_geodetic_height_above_po) <
                 po_height_thresh)
                own_agl_refined = own_agl_input + trk_po.alt_agl_ft
            end
        end
    end
    return own_agl_refined::R
end
```

AlignAltitudeTypesUsedInGeodeticPositions (Algorithm 108) is used to align the altitude coordinate frames of two aircraft (the ownship and one intruder) whenever ECEF or LLA geodetic positions of the two aircraft are used to generate relative separation distances. The algorithm takes in the intruder aircraft's LLA and ECEF positions along with two flags which indicate if the altitude used in these three-dimensional positions is HAE or pressure altitude, and if it is valid or

not. The ownship's LLA and ECEF information is pulled from the global ownship data structure which stores the ownship's propagated geodetic states.

If the inputted intruder's altitude type is HAE, indicated by *is_alt_geo_hae*, the first set of conditionals are checked. If the ownship's geodetic HAE altitude is valid, then the ownship's LLA and ECEF can be pulled directly from *own.geo_states_hae_alt*. Additionally, if the intruder's altitude is invalid, indicated by *valid_alt_vert_trk*, the intruder is assumed to be co-altitude with the ownship. Else if, the ownship's pressure altitude is valid, then the ownship's LLA and ECEF are pulled from *own.geo_states_pres_alt*. In this case the intruder's altitude needs to be converted to pressure altitude, however, the pressure altitude is not available so the intruder is assumed to be co-altitude with the ownship. The intruder's LLA position is adjusted by using the ownship's pressure altitude and the ECEF is then regenerated by a call to ConvertWGS84ToECEF. Otherwise, the ownship's LLA and ECEF are pulled from *own.geo_states_hae_alt* which will actually contain an altitude of zero feet. Since no ownship altitude information is available, the intruder's altitude is also set to zero by using the ownship's altitude and the ECEF is regenerated.

If the inputted intruder's altitude type is not HAE (i.e. pressure altitude), the second set of conditionals are checked. If the ownship's pressure altitude is valid, then the ownship's LLA and ECEF can be pulled directly from *own.geo_states_pres_alt*. Additionally, if the intruder's altitude is invalid, indicated by *valid_alt_vert_trk*, the intruder is assumed to be co-altitude with the ownship. Else if, the ownship's HAE altitude is valid, then the ownship's LLA and ECEF are pulled from *own.geo_states_hae_alt*. In this case the intruder's altitude needs to be converted to HAE altitude, however, the HAE altitude is not available so the intruder is assumed to be co-altitude with the ownship. The intruder's LLA position is adjusted by using the ownship's HAE altitude and the ECEF is then regenerated. Otherwise, the ownship's LLA and ECEF are pulled from *own.geo_states_pres_alt* which will actually contain an altitude of zero feet. Since no ownship altitude information is available, the intruder's altitude is also set to zero by using the ownship's altitude and the ECEF is regenerated.

This algorithm takes as input *lla_in*, *ecef_in*, *is_alt_geo_hae*, and *valid_alt_vert_trk*. This algorithm returns the aligned LLA and ECEF positions of both the intruder and ownship captured in *lla*, *ecef*, *own_lla*, and *own_ecef*, respectively.

Algorithm 108 AlignAltitudeTypesUsedInGeodeticPositions

```
function AlignAltitudeTypesUsedInGeodeticPositions(lla_in::Vector{R}, ecef_in::Vector{R}, is_alt_geo_hae::Bool,
    valid_alt_vert_trk::Bool)
    lla = copy(lla_in)
    ecef = copy(ecef_in)
    own_valid_alt_pres::Bool = !isnan(own.toa_vert)
    own_valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
    if is_alt_geo_hae
        if own_valid_alt_hae
            own_lla = own.geo_states_hae_alt.lla_rad_ft
            own_ecef = own.geo_states_hae_alt.ecef_m
            if !valid_alt_vert_trk
                lla[3] = own_lla[3]
                ecef = ConvertWGS84ToECEF(p. 13)(lla, true)
            end
        elseif own_valid_alt_pres
            own_lla = own.geo_states_pres_alt.lla_rad_ft
            own_ecef = own.geo_states_pres_alt.ecef_m
            lla[3] = own_lla[3]
            ecef = ConvertWGS84ToECEF(p. 13)(lla, false)
        else
            own_lla = own.geo_states_hae_alt.lla_rad_ft
            own_ecef = own.geo_states_hae_alt.ecef_m
            lla[3] = own_lla[3]
            ecef = ConvertWGS84ToECEF(p. 13)(lla, true)
        end
    else
        if own_valid_alt_pres
            own_lla = own.geo_states_pres_alt.lla_rad_ft
            own_ecef = own.geo_states_pres_alt.ecef_m
            if !valid_alt_vert_trk
                lla[3] = own_lla[3]
                ecef = ConvertWGS84ToECEF(p. 13)(lla, false)
            end
        elseif own_valid_alt_hae
            own_lla = own.geo_states_hae_alt.lla_rad_ft
            own_ecef = own.geo_states_hae_alt.ecef_m
            lla[3] = own_lla[3]
            ecef = ConvertWGS84ToECEF(p. 13)(lla, true)
        else
            own_lla = own.geo_states_pres_alt.lla_rad_ft
            own_ecef = own.geo_states_pres_alt.ecef_m
            lla[3] = own_lla[3]
            ecef = ConvertWGS84ToECEF(p. 13)(lla, false)
        end
    end
    return (lla::Vector{R}, ecef::Vector{R}, own_lla::Vector{R}, own_ecef::Vector{R})
end
```

Referenced In: RefineOwnshipAGL(p. 107), AbsoluteGeodeticToOwnshipRelative(p. 141), AddPointObstacleTrackToReport(p. 157)

GETGROUNDSPEED (Algorithm 109) attempts to generate a ground-speed in ft/s from ownship WGS84 information. If the ownship WGS84 state is not invalid, the ground-speed is generate by taking the hypotenuse of the east-west and north-south velocities and converting the resulting speed from m/s to ft/s. If the ownship WGS84 state is invalid, the ground-speed is set to NaN.

This algorithm takes no input. This algorithm returns the ground-speed, v.

Algorithm 109 GetGroundSpeed

```
function GetGroundSpeed()
    m2ft::R = geoutils.meters_to_feet
    if (own.wgs84_state != OWN_WGS84_INVALID)
        vel_ew = own.geo_states_hae_alt.vel_ew_mps
        vel_ns = own.geo_states_hae_alt.vel_ns_mps
        v = hypot(vel_ew, vel_ns) * m2ft
    else
        v = NaN
    end
    return v::R
end
```

Referenced In: UpdateHorizontalVelocityMode(p. 118)

2.8.2 Ownship V2V Operational Status Message

The SETOWNV2VOPERATIONALSTATUSMESSAGE (Algorithm 110) algorithm compiles all data needed to generate a V2V operational status message (sent via the V2V transmission system) using the *own.v2v_osm* structure, of type V2VOPERATIONALSTATUSMESSAGE.

This algorithm first sets the ownship CA status (*own.v2v_osm.ca_status*) based on ownship operational mode (*own.opmode*). Next, the *sense*, *type_capability*, *priority*, and *equipment* fields of the outgoing V2V operational status message are set accordingly for an ACAS sXu system.

This algorithm takes no input. The algorithm updates the global *own.v2v_osm*.

Algorithm 110 SetOwnV2VOperationalStatusMessage

```
function SetOwnV2VOperationalStatusMessage()
    if (own.opmode == OPMODE_RA)
        own.v2v_osm.ca_status = CA_STATUS_RA
    elseif (own.opmode == OPMODE_SURV_ONLY)
        own.v2v_osm.ca_status = CA_STATUS_RA_INHIBITED
    elseif (own.opmode == OPMODE_STANDBY)
        own.v2v_osm.ca_status = CA_STATUS_NONE
    end
    own.v2v_osm.v2v_cccb.sense = CAS_SENSE_BLENDED
    own.v2v_osm.v2v_cccb.type_capability = CAS_V2V_SXU
    own.v2v_osm.v2v_cccb.priority = SXU_V2V_CCCB_PRIORITY_MIDDLE
    own.v2v_osm.equipment = (EQUIPMENT_ADSB_1090ES_IN + EQUIPMENT_ADSB_UAT_IN + EQUIPMENT_V2V_IN +
        EQUIPMENT_V2V_OUT)
    return
end
```

Referenced In: GenerateStmReport(p. 112)

2.8.3 Surveillance Source Correlation

Prior to generating the STMREPORT, all targets within the STM must be evaluated in CORRELATIONPROCESSING (Algorithm 111) to determine if any two targets represent a single intruder aircraft or if two intruder aircraft are incorrectly represented by a single target. Additionally, AGT targets are evaluated to determine if any of them represent the ownship aircraft or if any AGT targets are incorrectly represented as the ownship aircraft. First, a call to PREDICTTRACKSUMMARY

extrapolates all tracks within each target and all AGT tracks within the ownship structure to the report generation time to allow for spatial correlation tests. AGT tracks that no longer correlate to the ownship are moved to new targets in DECORRELATETARGETSFROMOWNSHIP. Next, CORRELATETARGETSTOOWNSHIP iterates through all existing targets and merges any AGT tracks that correlate with ownship into the ownship structure. Next, tracks that no longer correlate to their assigned target are moved to new targets in DECORRELATETARGETS. Finally, CORRELATETARGETS iterates through all existing targets and merges track files from any two correlating targets into a single target.

Execution of CORRELATIONPROCESSING modifies the target data base so that each track is assigned to a target identified either by address or spatial matching, as described above. The performance requirements applicable to correlation are specified in §2.2.3.4 of the ACAS sXu MOPS Volume I [3]. CORRELATIONPROCESSING is a prescriptive algorithm, though the algorithm consists entirely of calls to suggested algorithms provided as guidance in Appendix B. Whether an equipment manufacturer chooses to implement the algorithms of Appendix B, a modification thereof, or provides their own design, the complete system must be shown to meet the requirements ACAS sXu MOPS Volume I [3] §2.2.3.4. The algorithms of Appendix B have been demonstrated to meet these requirements given specific data sets provided by at least one manufacturer.

This algorithm takes as input *T*. This algorithm produces no output, but calls other algorithms.

Algorithm 111 CorrelationProcessing

```
function CorrelationProcessing(T::R)
    PredictTrackSummary(p. 439)(T)
    DecorrelateTargetsFromOwnship(p. 451)(T)
    CorrelateTargetsToOwnship(p. 452)(T)
    DecorrelateTargets(p. 457)(T)
    CorrelateTargets(p. 461)(T)
end
```

Referenced In: GenerateStmReport(p. 112)

2.8.4 STM Report Generation

Upon request, the STM can produce a data structure called the STMREPORT that contains all elements required by the TRM and other external modules. This process, defined in GENERATESTMREPORT (Algorithm 112), takes a single input (t), which is the requested time of applicability for the report. If necessary, tracks are extrapolated by the algorithm so that all information contained in the report is referenced at a common time.

A call to GENERATESTMREPORT begins with a call to OWNWGS84TIMEOUT which determines whether ownship WGS84 state has been received recently, and is then followed by a call to propagate the ownship state information to the current toa PROPAGATEOWNSHIPTOTOA. Next, a call to UPDATEADVISORYMODE is made which includes calculation of ownship operating mode. Next, ownship V2V capability data is set by a call to SETOWNV2VOPERATIONALSTATUSMESSAGE. Then a call is made to CORRELATIONPROCESSING for target correlation.

The algorithm then initializes an empty STMREPORT, which is then populated with ownship state and status information by calling SETOWNSHIPDATA.

Next, intruder tracks are added to the report through a call to ADDTRACKSTOREPORT.

This algorithm takes as input *t*, updates *own.prev_rpt_toa*, and returns *report*.

Algorithm 112 GenerateStmReport

```
function GenerateStmReport(t::R)
    OwnWgs84Timeout(p. 112)(t)
    (own.geo_states_hae_alt, own.geo_states_pres_alt) = PropagateOwnshipToToa(p. 114)(t)
    UpdateAdvisoryMode(p. 106)()
    SetOwnV2VOperationalStatusMessage(p. 110)()
    CorrelationProcessing(p. 111)(t)
    report = StmReport(p. 480)()
    SetOwnshipData(p. 116)(report, t)
    AddTracksToReport(p. 119)(report, t)
    own.prev_rpt_toa = t
    return report::StmReport(p. 480)
end
```

OWNWGS84TIMEOUT (Algorithm 113) is executed to ensure the ownship WGS84 information is up to date by comparing the difference between the WGS84 TOA *wgs84_toa* and the report time *t* against a threshold *wgs84_timeout*. If the time difference is too large, then the ownship WGS84 information is old information and is considered timed out. When this happens, the ownship *wgs84_state* is first set to invalid. Finally, each of the ownship WGS84 observations are set to *NaN*.

This algorithm takes as input *t*. This algorithm updates WGS84 information inside of *own*.

Algorithm 113 OwnWgs84Timeout

```
function OwnWgs84Timeout(t::R)
    wgs84_timeout::R = params().surveillance.ownship_wgs84.timeout
    if(own.wgs84_state == OWN_WGS84_VALID) && (t - own.wgs84_toa > wgs84_timeout)
        own.wgs84_state = OWN_WGS84_INVALID
        own.wgs84_lat_deg = NaN
        own.wgs84_lon_deg = NaN
        own.wgs84_vel_ew_kts = NaN
        own.wgs84_vel_ns_kts = NaN
        own.wgs84_alt_hae_ft = NaN
        own.wgs84_alt_rate_hae_fps = NaN
        own.wgs84_toa = NaN
        own.wgs84_toa_vert = NaN
        own.history.hae_alt = ValueTime(p. 495)()
    end
end
```

Referenced In: GenerateStmReport(p. 112)

PROPAGATEOWNSHIPTOTOA (Algorithm 114) estimates the predicted location of the ownship at a specific *toa* to account for ownship motion since the most recent ownship WGS84 state input.

The result of this algorithm is two sets of geodetic states, one set that represents the vertical dimension in HAE and the other set that represents the vertical dimension as a pressure altitude.

First, the ownship pressure altitude states are calculated, where PRESALTATTOA is used to gen-

erate pressure altitudes at a specific toa. If the ownship pressure altitude tracker is not available, the pressure altitude states are set to zero.

Next, if the ownship WGS84 state is valid, then the latitude and longitude are first converted to radians. Then the ownship HAE altitude state are calculated, where HAEAltAtToa is used to generate HAE altitudes at a specific toa. If the ownship HAE altitude tracker is not available, the HAE altitude states are set equal to the pressure altitude states. Then the horizontal velocities are converted to m/s and the time-step, *dt*, is generated.

If the ownship WGS84 state is not valid, then the HAE altitude states are set equal to the pressure altitude states and all other variables are set equal to *NaN*.

With the necessary initial state information and time-step specified, the ownship states and ownship HAE altitudes are then propagated to the desired TOA by a call to PropagateGeodeticToToa and returned to the first set of geodetic states (*geo_states_hae_alt*) that represent the vertical dimension in HAE.

The second set of geodetic states, which needs to represent the vertical dimension as a pressure altitude (*geo_states_pres_alt*), is first created by making a deep copy of *geo_states_hae_alt*. Next, the altitude is replaced by the pressure altitude of the ownship at the desired TOA. Then the LLA of the ownship is converted to an ECEF position using the pressure altitude of the ownship by a call to ConvertWGS84ToECEF. Next, the altitude rate is replaced by the pressure altitude rate of the ownship. Finally, the flag *is_alt_geo_hae* is changed to false.

This algorithm takes as input *toa*. This algorithm returns *geo_states_hae_alt* and *geo_states_pres_alt*.

Algorithm 114 PropagateOwnshipToToa

```
function PropagateOwnshipToToa(toa::R)
    kts2mps::R = geoutils.kts_to_mps
    m2ft::R = geoutils.meters_to_feet
    if !isnan(own.toa_vert)
        if (own.wgs84_state == OWN_WGS84_VALID)
            wgs84_toa = own.wgs84_toa
        else
            wgs84_toa = toa
        end
        alt_pres_ft = PresAltAtToa(p. 84)(wgs84_toa)
        alt_pres_ft_toa = PresAltAtToa(p. 84)(toa)
        alt_rate_pres_fps = own.mu_vert[2]
    else
        alt_pres_ft = 0.0
        alt_pres_ft_toa = 0.0
        alt_rate_pres_fps = 0.0
    end
    if (own.wgs84_state == OWN_WGS84_VALID)
        lat_rad = deg2rad(own.wgs84_lat_deg)
        lon_rad = deg2rad(own.wgs84_lon_deg)
        if !isnan(own.wgs84_toa_vert)
            alt_hae_ft = HAEAltAtToa(p. 96)(own.wgs84_toa)
            alt_hae_ft_toa = HAEAltAtToa(p. 96)(toa)
            alt_rate_hae_fps = own.wgs84_mu_vert[2]
        else
            alt_hae_ft = alt_pres_ft
            alt_hae_ft_toa = alt_pres_ft_toa
            alt_rate_hae_fps = alt_rate_pres_fps
        end
        vel_ew_mps = own.wgs84_vel_ew_kts * kts2mps
        vel_ns_mps = own.wgs84_vel_ns_kts * kts2mps
        dt = toa - own.wgs84_toa
    else
        lat_rad = NaN
        lon_rad = NaN
        alt_hae_ft = alt_pres_ft
        alt_hae_ft_toa = alt_pres_ft_toa
        alt_rate_hae_fps = alt_rate_pres_fps
        vel_ew_mps = NaN
        vel_ns_mps = NaN
        dt = NaN
    end
    lla_hae_rad_ft = [lat_rad, lon_rad, alt_hae_ft]
    geo_states_hae_alt = PropagateGeodeticToToa(p. 23)(lla_hae_rad_ft, alt_hae_ft_toa, true, vel_ew_mps,
         vel_ns_mps, alt_rate_hae_fps, dt)
    geo_states_pres_alt = deepcopy(geo_states_hae_alt)
    geo_states_pres_alt.lla_rad_ft[3] = alt_pres_ft_toa
    geo_states_pres_alt.ecef_m = ConvertWGS84ToECEF(p. 13)(geo_states_pres_alt.lla_rad_ft, false)
    geo_states_pres_alt.alt_rate_fps = alt_rate_pres_fps
    geo_states_pres_alt.is_alt_geo_hae = false
    return (geo_states_hae_alt::GeodeticStates(p. 492), geo_states_pres_alt::GeodeticStates(p. 492))
end
```

Referenced In: GenerateStmReport(p. 112)

In SETOWNSHIPDATA (Algorithm 115), the STMREPORT is populated through a series of steps with ownship state and status information required by the TRM and other external modules.

- The first data element is ownship height AGL, which is retrieved from memory and added to the report.

- Then, the ownship V2V unique identifier (*v2v_uid*) and the AGT-IDs of all AGTTRACKFILES that have been correlated with ownship are retrieved from memory and added to the report.
- TRM alerting capability (*opmode*) state is added to the report.
- Ownship velocity information is set through a call to SETOWNSHIPVELOCITY.
- Horizontal velocity mode information is calculated through a call to UPDATEHORIZONTALVELOCITYMODE, and then added to the report.
- Then, ownship V2V capability configuration information, and TRM rate limits are retrieved from memory and added to the report.
- The degraded surveillance state (*own.degraded_own_surveillance*) is evaluated and stored for degraded pressure altitude, WGS84 states, and heading angle.
- Ownship pressure altitude (PA), if valid, is extrapolated to *t*, the time of STMREPORT generation, and stored as *mu_pres*. Similarly, ownship geometric height above ellipsoid (HAE), if valid, is extrapolated to *t* and stored as *mu_hae*.

  The following criteria are then applied to select either ownship PA or HAE for reporting:

  1. If only one of the two sources are valid, the valid source is selected;
  2. If both sources are invalid, ownship is reported as NAR.
  3. If both sources are valid, the source with a lower degree of altitude uncertainty is selected. For PA, in the absence of a reported uncertainty, the expected uncertainty is derived through a call to GETPRESAEMSIGMA (Algorithm 30). For HAE, this expected uncertainty is the *own.wgs84_sigma_vepu* derived earlier in RECEIVEWGS84OBSERVATION.

  Once the selection has been made, the state vector *mu* and TRM altitude mode selection *trm_altmode* are set accordingly. Note that when ownship is reported as NAR, *mu* is populated with *NaN*s and *trm_altmode* is set to *TRM_ALTMODE_NONE*. The vertical logic will not produce RAs, when *trm_altmode* is set to *TRM_ALTMODE_NONE*.
- Next, the flag *disable_gpoa* is retrieved from memory and *perform_gpoa* is added to the report.
- The ownship vertical belief state is populated with the ownship vertical estimate. Since the TRM expects a single sample to represent the state of ownship altitude, the mean (*mu*) of the ownship vertical track is added to the report with a weight of one.

This algorithm takes as input *report* and *t*. This algorithm updates *report*.

Algorithm 115 SetOwnshipData

```
function SetOwnshipData(report::StmReport(p. 480), t::R)
    pres_aem_sigma::Vector{R} = params().surveillance.vertical.pres_small_ownship.aem_sigma
    report.trm_input.own.h = own.h_agl
    report.trm_input.own.v2v_uid = own.v2v_uid
    for trk in own.agt_tracks
        push!(report.trm_input.own.agt_ids, trk.agt_id)
    end
    report.trm_input.own.opmode = own.opmode
    report.trm_input.own.h_opmode = own.opmode
    SetOwnshipVelocity(p. 117)(report, t)
    UpdateHorizontalVelocityMode(p. 118)()
    report.trm_input.own.trm_velmode = own.trm_velmode
    report.own_v2v_osm = own.v2v_osm
    report.trm_input.own.turn_rate_limit = own.discrete.turn_rate_limit
    report.trm_input.own.vert_rate_limit = own.discrete.vert_rate_limit
    valid_alt_pres::Bool = !isnan(own.toa_vert)
    valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
    if !valid_alt_pres
        report.trm_input.own.degraded_own_surveillance += DEGRADED_OWN_ALTPRES_INVALID
    elseif (own.toa_vert < own.prev_rpt_toa)
        report.trm_input.own.degraded_own_surveillance += DEGRADED_OWN_ALTPRES_COAST
    end
    if (own.wgs84_state == OWN_WGS84_INVALID)
        report.trm_input.own.degraded_own_surveillance += DEGRADED_OWN_WGS84_INVALID
    elseif (own.wgs84_toa < own.prev_rpt_toa)
        report.trm_input.own.degraded_own_surveillance += DEGRADED_OWN_WGS84_COAST
    end
    if (own.heading_state == OWN_HEADING_INVALID)
        report.trm_input.own.degraded_own_surveillance += DEGRADED_OWN_HDG_INVALID
    elseif (own.toa_heading < own.prev_rpt_toa)
        report.trm_input.own.degraded_own_surveillance += DEGRADED_OWN_HDG_COAST
    end
    if valid_alt_pres
        dt = t - own.toa_vert
        (mu_pres, _) = PredictVerticalTracker(p. 54)(own.mu_vert, own.Sigma_vert, dt, false, true, false)
        pres_alt_bias_sigma::R = GetPresAEMSigma(p. 34)(mu_pres[1], pres_aem_sigma)
    end
    if valid_alt_hae
        dt = t - own.wgs84_toa_vert
        (mu_hae, _) = PredictVerticalTracker(p. 54)(own.wgs84_mu_vert, own.wgs84_Sigma_vert, dt, true, true, false)
    end
    if valid_alt_hae && (!valid_alt_pres || (own.wgs84_sigma_vepu < pres_alt_bias_sigma))
        mu = mu_hae
        report.trm_input.own.trm_altmode = TRM_ALTMODE_HAE
    elseif valid_alt_pres
        mu = mu_pres
        report.trm_input.own.trm_altmode = TRM_ALTMODE_PRESSURE
    else
        mu = [NaN, NaN]
        report.trm_input.own.trm_altmode = TRM_ALTMODE_NONE
    end
    report.trm_input.own.perform_gpoa = !own.discrete.disable_gpoa
    report.trm_input.own.h_lo_ft = own.h_lo_ft
    report.trm_input.own.h_hi_ft = own.h_hi_ft
    resize!(report.trm_input.own.belief_vert,1)
    report.trm_input.own.belief_vert[1] = OwnVerticalBelief(p. 501)()
    report.trm_input.own.belief_vert[1].z = mu[1]
    report.trm_input.own.belief_vert[1].dz = mu[2]
    report.trm_input.own.belief_vert[1].weight = 1.0
    return
end
```

Referenced In: GenerateStmReport(p. 112)

SETOWNSHIPVELOCITY (Algorithm 116) sets the ownship airspeed, ground-speed, heading, and track-angle that are passed to the TRM as input. The airspeed is passed straight through the STM. The ownship heading is added to the report through a call to HEADINGATTOA, which extrapolates ownship heading to the time of the report. If the ownship WGS84 state is not invalid, then the *ground_speed* and *track_angle* are generated from the east-west and north-south WGS84 velocities. The ground-speed is also converted from m/s to ft/s. Otherwise, if the ownship WGS84 state is invalid, the *ground_speed* and *track_angle* are set to *NaN*.

This algorithm takes as input *report*, and *t*. This algorithm updates *report*.

Algorithm 116 SetOwnshipVelocity

```
function SetOwnshipVelocity(report::StmReport(p. 480), t::R)
    m2ft::R = geoutils.meters_to_feet
    report.trm_input.own.airspeed = own.airspeed
    report.trm_input.own.psi = HeadingAtToa(p. 90)(t)
    if (own.wgs84_state != OWN_WGS84_INVALID)
        vel_ew = own.geo_states_hae_alt.vel_ew_mps
        vel_ns = own.geo_states_hae_alt.vel_ns_mps
        v = hypot(vel_ew, vel_ns) * m2ft
        chi = atan(vel_ew, vel_ns)
    else
        (v, chi) = (NaN, NaN)
    end
    report.trm_input.own.ground_speed = v
    report.trm_input.own.track_angle = chi
end
```

Referenced In: SetOwnshipData(p. 116)

UPDATEHORIZONTALVELOCITYMODE (Algorithm 117) is used to tell the TRM what ownship velocity information should be used for the horizontal avoidance logic, particularly distinguishing between ground-relative and wind-relative velocity states.

The default operating mode for the horizontal logic is ground-relative mode, therefore, this algorithm first checks if ownship WGS84 velocity information is valid and then performs the following checks:

- If the user desires to bypass this default behavior and operate in wind-relative coordinates instead, indicated by *prefer_wind_relative* being true, and wind-relative velocity information is available (*wind_relative_feasible*), then the velocity mode is set to *TRM_VELMODE_AIRSPEED_HEADING*.
- Else, if the ownship is hovering or close to hovering, the track-angle is unobservable so wind-relative heading angle is preferred. If the ownship ground-speed (v) is less than v_lo, then the ownship is assumed to be close to hovering. If the ownship is assumed to be close to hovering and the heading information is valid then the velocity mode is set to *TRM_VELMODE_AIRSPEED_HEADING* if *valid_airspeed* is true, or *TRM_VELMODE_GROUNDSPEED_HEADING* if *valid_airspeed* is false.
- Else, if the ownship is not close to hovering, indicated by the ownship ground-speed (v) being greater than v_hi, or the heading is not valid, then the velocity mode is set to *TRM_VELMODE_GROUNDSPEED_TRACKANGLE*. Notice that there is a high and low ground-speed threshold which results in hysteresis to prevent the velocity mode from bouncing

back and fourth between two operational modes.

- Else, if the previous velocity mode was *TRM_VELMODE_NONE*, then the new velocity mode is set to *TRM_VELMODE_GROUNDSPEED_TRACKANGLE*.
- Otherwise, the velocity mode retains it previous value. This is necessary to enable the ground-speed hysteresis to work properly as can be seen by the high and low ground-speed threshold parameters. This hysteresis prevents the velocity mode from bouncing back and fourth between two operational modes when the ground-speed is close to the hovering threshold speed.

If the ownship WGS84 velocity information is not valid and ownship wind-relative velocities are valid, indicated by *wind_relative_feasible*, then the velocity mode is set to *TRM_VELMODE_-AIRSPEED_HEADING*.

Otherwise, if neither the ownship's WGS84 or wind-relative velocity information is available, then the velocity mode is set to *TRM_VELMODE_NONE*. When the velocity mode is set to *TRM_-VELMODE_NONE*, the horizontal logic will not produce RAs.

This algorithm takes no inputs. This algorithm updates *own.trm_velmode*.

Algorithm 117 UpdateHorizontalVelocityMode

```
function UpdateHorizontalVelocityMode()
    v_lo::R = params().surveillance.report_generation.hor_trm_velmode.v_lo
    v_hi::R = params().surveillance.report_generation.hor_trm_velmode.v_hi
    valid_WGS84 = (own.wgs84_state != OWN_WGS84_INVALID)
    valid_airspeed = !isnan(own.airspeed)
    valid_heading = (own.heading_state == OWN_HEADING_NOMINAL)
    wind_relative_feasible = (valid_airspeed && valid_heading)
    if valid_WGS84
        v::R = GetGroundSpeed(p. 110)()
        if own.discrete.prefer_wind_relative && wind_relative_feasible
            own.trm_velmode = TRM_VELMODE_AIRSPEED_HEADING
        elseif (v < v_lo) && valid_heading
            if valid_airspeed
                own.trm_velmode = TRM_VELMODE_AIRSPEED_HEADING
            else
                own.trm_velmode = TRM_VELMODE_GROUNDSPEED_HEADING
            end
        elseif (v > v_hi) || !valid_heading
            own.trm_velmode = TRM_VELMODE_GROUNDSPEED_TRACKANGLE
        elseif (own.trm_velmode == TRM_VELMODE_NONE)
            own.trm_velmode = TRM_VELMODE_GROUNDSPEED_TRACKANGLE
        end
    elseif wind_relative_feasible
        own.trm_velmode = TRM_VELMODE_AIRSPEED_HEADING
    else
        own.trm_velmode = TRM_VELMODE_NONE
    end
end
```

Referenced In: SetOwnshipData(p. 116)

ADDTRACKSTOREPORT (Algorithm 118) determines which, if any, Track Files for a given Target should be appended to the STMREPORT. Track Files are prioritized per Target based on their underlying surveillance source, validation state, vertical validity, and track quality. Component tracks within each Track File may have discrepant validities. If a Track File is selected with an

invalid component track, an appropriate notation will be set in the degraded surveillance flags for that Target (see SETINTRUDERDEGRADEDFLAGS). For example, an invalid vertical track will result in a NAR indication. Each Target is processed through a call to REMOVESTALETRACKS, which deletes any Track Files in the Target that are considered deprecated (i.e., coasted out and not designated). If all Track Files are removed from a Target, no more processing is performed on that Target.

The algorithm first calls TRACKSOURCESELECTION (Algorithm 119) to select a Track File, *aircraft_track*, for the Target based on the available sources and track characteristics. If there are no eligible tracks available, *aircraft_track* will be set to nothing. The flags*aircraft_trk_exists*, and *point_obstacle_trk_exists* are generated by calls to TRACKEXISTS. The *point_obstacle_acceptable* flag is set if the ownship WGS84 state is valid and the ownship HAE altitude is valid.

If the *aircraft_trk_exists* flag indicates that an acceptable Track File is available, the relevant data elements from *aircraft_trk* are added to the STMREPORT by calling the proper function based on track type. Otherwise, if the *point_obstacle_trk_exists* and *point_obstacle_acceptable* flags are true and the ownship discrete values indicate that the point obstacle avoidance is active, a call to ADDPOINTOBSTACLETRACKTOREPORT adds the point obstacle data elements to the STMREPORT.

Before completing, a call to FILTERTRACKSFORTRM (Algorithm 129) limits the number of intruders sent to the TRM and the display.

This algorithm takes as input time *t* and an STMREPORT *report* and appends any track files to the input report.

Algorithm 118 AddTracksToReport

```
function AddTracksToReport(report::StmReport(p. 480), t::R)
    for id in keys(target_db)
        if (RemoveStaleTracks(p. 159)(id,t))
            tgt = target_db[id]
            aircraft_trk = TrackSourceSelection(p. 121)(id, t)
            aircraft_trk_exists::Bool = TrackExists(p. 19)(aircraft_trk)
            point_obstacle_trk_exists::Bool = TrackExists(p. 19)(tgt.point_obstacle_track)
            point_obstacle_acceptable::Bool = (own.wgs84_state == OWN_WGS84_VALID) && !isnan(own.
                 wgs84_toa_vert)
            if aircraft_trk_exists
                if (typeof(aircraft_trk) == ADSBTrackFile(p. 485))
                    AddADSBTrackToReport(p. 135)(report, id, t, !ValidationCheck(p. 162)(aircraft_trk))
                elseif (typeof(aircraft_trk) == V2VTrackFile(p. 488))
                    AddV2VTrackToReport(p. 150)(report, id, t, !ValidationCheck(p. 162)(aircraft_trk))
                elseif (typeof(aircraft_trk) == ORNCTTrackFile(p. 484))
                    AddORNCTTrackToReport(p. 152)(report, id, t)
                elseif (typeof(aircraft_trk) == AGTTrackFile(p. 486))
                    AddAGTTrackToReport(p. 155)(report, id, aircraft_trk, t, !ValidationCheck(p. 162)(aircraft_trk))
                end
            elseif point_obstacle_trk_exists && point_obstacle_acceptable && own.discrete.perform_poa
                AddPointObstacleTrackToReport(p. 157)(report, id, t)
            end
        end
    end
    FilterTracksForTRM(p. 132)(report)
end
```

Referenced In: GenerateStmReport(p. 112)

TRACKSOURCESELECTION (Algorithm 119) selects the track passed to the TRM and display for a given target based on the available track sources and their characteristics. First, the *tgt* is retrieved from the target database. A call to OPTIMALPRETRACKEDTRACKS (Algorithm 120) determines the optimal track for each of four tiers of available tracks: validated pre-tracked tracks with valid vertical information, validated pre-tracked NAR tracks, unvalidated pre-tracked tracks with valid vertical information, and unvalidated pre-tracked NAR tracks. If no track exists in a given tier, the optimal track for that list is set to nothing. Next, the algorithm determines eligibility flags for use in track selection. The flags *adsb_trk_exists*, *v2v_trk_exists*, *opt_pretrk_trk_exists*, *opt_pretrk_NAR_trk_exists*, *opt_pretrk_unvalidated_trk_exists*, and *opt_pretrk_unvalidated_NAR_trk_exists* are generated by calls to TRACKEXISTS (Algorithm 12). The *adsb_acceptable* and *v2v_acceptable* flags are set if the ownship WGS84 state is valid and the corresponding tracks pass a minimum data quality check (via a call to PASSIVEQUALITYCHECK (Algorithm 125)). The large conditional statement updates *selected_trk* based on the following hierarchy:

- If the *adsb_trk_exists* and *adsb_acceptable* flags are set, the VALIDATIONCHECK (Algorithm 155) passes, and the vertical track information is valid (determined through a call to ASSESSVERTICALVALIDITY (Algorithm 121)), the ADSB track is selected.
- If the *v2v_trk_exists* and *v2v_acceptable* flags are set, the VALIDATIONCHECK passes, and the vertical track information is valid (determined through a call to ASSESSVERTICALVALIDITY (Algorithm 121)), the V2V track is selected.
- If the *opt_pretrk_trk_exists* flag is set, the *opt_pretrk_trk* is selected, which will be a validated AGT or ORNCT track with valid vertical information.
- If the *adsb_trk_exists* and *adsb_acceptable* flags are set, the VALIDATIONCHECK passes, the NAR ADSB track is selected.
- If the *v2v_trk_exists* and *v2v_acceptable* flags are set and the VALIDATIONCHECK passes, the NAR V2V track is selected.
- If the *opt_pretrk_NAR_trk_exists* flag is set, the *opt_pretrk_NAR_trk* is selected, which will be a validated AGT or ORNCT track with invalid vertical information.
- If the *adsb_trk_exists* and *adsb_acceptable* flags are set, the VALIDATIONCHECK fails, and the vertical track information is valid (determined through a call to ASSESSVERTICALVALIDITY (Algorithm 121)), the not validated ADSB track is selected.
- If the *v2v_trk_exists* and *v2v_acceptable* flags are set, the VALIDATIONCHECK fails, and the vertical track information is valid (determined through a call to ASSESSVERTICALVALIDITY (Algorithm 121)), the not validated V2V track is selected.
- If the *opt_pretrk_unvalidated_trk_exists* flag is set, the *opt_pretrk_unvalidated_trk* is selected, which will be a not validated AGT or ORNCT track with valid vertical information.
- If the *adsb_trk_exists* and *adsb_acceptable* flags are set, the VALIDATIONCHECK fails, the not validated NAR ADSB track is selected.
- If the *v2v_trk_exists* and *v2v_acceptable* flags are set and the VALIDATIONCHECK fails, the not validated NAR V2V track is selected.
- If the *opt_pretrk_unvalidated_NAR_trk_exists* flag is set, the *opt_pretrk_unvalidated_NAR_trk* is selected, which will be a not validated AGT or ORNCT track with invalid vertical information.

The *selected_trk* is used to update the target's optimal track history with a call to OPTIMALTRACKHISTORYUPDATE (Algorithm 126). Then, SELECTFINALTRACK (Algorithm 127) determines the fi-

nal track to be appended to the STMREPORT. Finally, *tgt.previously_selected_trk_uid* is updated with the selected track's unique id.

This algorithm takes as input *target_id* and *T*, updates *tgt.previously_selected_trk_uid*, and returns *selected_trk*.

Algorithm 119 TrackSourceSelection

```
function TrackSourceSelection( target_id::UInt32, T::R )
    tgt = target_db[target_id]
    (opt_pretrk_trk, opt_pretrk_NAR_trk, opt_pretrk_unvalidated_trk, opt_pretrk_unvalidated_NAR_trk) =
        OptimalPreTrackedTracks(p. 122)(target_id, T)
    adsb_trk_exists = TrackExists(p. 19)(tgt.adsb_track)
    v2v_trk_exists = TrackExists(p. 19)(tgt.v2v_track)
    opt_pretrk_trk_exists = TrackExists(p. 19)(opt_pretrk_trk)
    opt_pretrk_NAR_trk_exists = TrackExists(p. 19)(opt_pretrk_NAR_trk)
    opt_pretrk_unvalidated_trk_exists = TrackExists(p. 19)(opt_pretrk_unvalidated_trk)
    opt_pretrk_unvalidated_NAR_trk_exists = TrackExists(p. 19)(opt_pretrk_unvalidated_NAR_trk)
    adsb_acceptable::Bool = (PassiveQualityCheck(p. 127)(tgt.adsb_track)) && (own.wgs84_state == OWN_WGS84_VALID)
    v2v_acceptable::Bool = (PassiveQualityCheck(p. 127)(tgt.v2v_track)) && (own.wgs84_state == OWN_WGS84_VALID)
    selected_trk = nothing
    if adsb_trk_exists && adsb_acceptable && ValidationCheck(p. 162)(tgt.adsb_track) && AssessVerticalValidity(p. 123)(
        tgt.adsb_track)
        selected_trk = tgt.adsb_track
    elseif v2v_trk_exists && v2v_acceptable && ValidationCheck(p. 162)(tgt.v2v_track) && AssessVerticalValidity(p. 123)(
        tgt.v2v_track)
        selected_trk = tgt.v2v_track
    elseif opt_pretrk_trk_exists
        selected_trk = opt_pretrk_trk
    elseif adsb_trk_exists && adsb_acceptable && ValidationCheck(p. 162)(tgt.adsb_track)
        selected_trk = tgt.adsb_track
    elseif v2v_trk_exists && v2v_acceptable && ValidationCheck(p. 162)(tgt.v2v_track)
        selected_trk = tgt.v2v_track
    elseif opt_pretrk_NAR_trk_exists
        selected_trk = opt_pretrk_NAR_trk
    elseif adsb_trk_exists && adsb_acceptable && AssessVerticalValidity(p. 123)(tgt.adsb_track)
        selected_trk = tgt.adsb_track
    elseif v2v_trk_exists && v2v_acceptable && AssessVerticalValidity(p. 123)(tgt.v2v_track)
        selected_trk = tgt.v2v_track
    elseif opt_pretrk_unvalidated_trk_exists
        selected_trk = opt_pretrk_unvalidated_trk
    elseif adsb_trk_exists && adsb_acceptable
        selected_trk = tgt.adsb_track
    elseif v2v_trk_exists && v2v_acceptable
        selected_trk = tgt.v2v_track
    elseif opt_pretrk_unvalidated_NAR_trk_exists
        selected_trk = opt_pretrk_unvalidated_NAR_trk
    end
    OptimalTrackHistoryUpdate(p. 128)(target_id, selected_trk)
    (selected_trk, selected_trk_uid) = SelectFinalTrack(p. 129)(target_id, selected_trk)
    tgt.previously_selected_trk_uid = selected_trk_uid
    return selected_trk::Union{TrackFile(p. 483), Nothing}
end
```

Referenced In: AddTracksToReport(p. 119)

OPTIMALPRETRACKEDTRACKS (Algorithm 120) determines the optimal tracks for four different sets of pre-tracked tracks. The four track sets represent groupings of pre-tracked STM inputs (ORNCT and AGT tracks) based on validation state and vertical validity. First, the target *tgt* is retrieved from the target database and the track lists are initialized. A call to TRACKEXISTS (Algorithm 12) determines if an ORNCT track exists on the target (*ornct_trk_exists*). The *agt_ac-*

ceptable flag is set if the ownship WGS84 state is valid. Then, the track lists are compiled. If an ORNCT track exists, it is always added to the *pretrk_trks* list because it is self-validating and will always have valid vertical information. If the ownship WGS84 state is valid, indicating AGT tracks can be used, the list of AGT tracks is iterated through. The vertical validity of each AGT track is assessed with a call to ASSESSVERTICALVALIDITY (Algorithm 121). If the AGT track is validated, the vertical validity is checked and the track is added to the appropriate list (*pretrk_trks* or *pretrk_NAR_trks*). If the AGT track is not validated, the vertical validity is checked and the track is added to the appropriate list (*pretrk_unvalidated_trks* or *pretrk_unvalidated_NAR_trks*). Finally, a call to OPTIMALQUALITYTRACK (Algorithm 123) is made to determine the optimal track for each list based on a track quality comparison. This algorithm takes as input *target_id* and report time *T* and returns *opt_pretrk_trk*, *opt_pretrk_NAR_trk*, *opt_pretrk_unvalidated_trk*, and *opt_pretrk_unvalidated_NAR_trk*.

Algorithm 120 OptimalPreTrackedTracks

```
function OptimalPreTrackedTracks( target_id::UInt32, T::R )
    tgt = target_db[target_id]
    (pretrk_trks, pretrk_NAR_trks, pretrk_unvalidated_trks, pretrk_unvalidated_NAR_trks) = (TrackFile(p. 483)[],
          TrackFile(p. 483)[], TrackFile(p. 483)[], TrackFile(p. 483)[])
    ornct_trk_exists::Bool = TrackExists(p. 19)(tgt.ornct_track)
    agt_acceptable::Bool = (own.wgs84_state == OWN_WGS84_VALID)
    if ornct_trk_exists
        push!(pretrk_trks, tgt.ornct_track)
    end
    if agt_acceptable
        for agt_track in tgt.agt_tracks
            agt_track_valid_vert = AssessVerticalValidity(p. 123)(agt_track)
            if agt_track.externally_validated
                if agt_track_valid_vert
                    push!(pretrk_trks, agt_track)
                else
                    push!(pretrk_NAR_trks, agt_track)
                end
            else
                if agt_track_valid_vert
                    push!(pretrk_unvalidated_trks, agt_track)
                else
                    push!(pretrk_unvalidated_NAR_trks, agt_track)
                end
            end
        end
    end
    opt_pretrk_trk = OptimalQualityTrack(p. 125)(pretrk_trks, true, T)
    opt_pretrk_NAR_trk = OptimalQualityTrack(p. 125)(pretrk_NAR_trks, false, T)
    opt_pretrk_unvalidated_trk = OptimalQualityTrack(p. 125)(pretrk_unvalidated_trks, true, T)
    opt_pretrk_unvalidated_NAR_trk = OptimalQualityTrack(p. 125)(pretrk_unvalidated_NAR_trks, false, T)
    return (opt_pretrk_trk::Union{TrackFile(p. 483), Nothing}, opt_pretrk_NAR_trk::Union{TrackFile(p. 483), Nothing},
          opt_pretrk_unvalidated_trk::Union{TrackFile(p. 483), Nothing}, opt_pretrk_unvalidated_NAR_trk::Union{
          TrackFile(p. 483), Nothing})
end
```

Referenced In: TrackSourceSelection(p. 121)

ASSESSVERTICALVALIDITY (Algorithm 121) determines if the vertical state of *trk* is valid. First, the ownship pressure and geodetic altitude validities are determined. Next, *compatible_alt_types* is set by comparing the ownship vertical validities (*own_valid_alt_pres* and *own_valid_alt_hae*) to the altitude source of the intruder track. The altitude types are considered compatible if the

ownship altitude source that is matching the altitude source of the intruder track is valid. In addition, the *invalid_gva* flag is set if

1. the intruder track is an ADSBTRACKFILE;
2. the intruder track is reporting geometric heights above ellipsoid (HAE); and
3. the geometric vertical accuracy, *trk.gva*, field is less than the minimum acceptable value *min_gva*, or is invalid (*NaN*).

Finally, *valid_vert* is set to true if the ownship and intruder altitude types are compatible, the intruder vertical track states are valid, and the *invalid_gva* flag has not been set.

This algorithm takes as input track *trk* and returns *valid_vert*.

Algorithm 121 AssessVerticalValidity

```
function AssessVerticalValidity(trk::Union{ADSBTrackFile(p. 485), V2VTrackFile(p. 488), AGTTrackFile(p. 486)})
    min_gva::UInt32 = params().surveillance.report_generation.min_adsb_quality.gva
    own_valid_alt_pres::Bool = !isnan(own.toa_vert)
    own_valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
    compatible_alt_types::Bool = (trk.alt_src_hae && own_valid_alt_hae) || (!trk.alt_src_hae &&
        own_valid_alt_pres)
    invalid_gva::Bool = isa(trk, ADSBTrackFile(p. 485)) && trk.alt_src_hae && ((trk.gva < min_gva) || isnan(
        ConvertGVAToSigmaVEPU(p. 124)(trk)))
    valid_vert::Bool = trk.valid_vert && compatible_alt_types && !invalid_gva
    return valid_vert::Bool
end
```

Referenced In: TrackSourceSelection(p. 121), OptimalPreTrackedTracks(p. 122)

CONVERTGVATOSIGMAVEPU (Algorithm 122) converts the Geometric Vertical Accuracy (GVA) value of an ADSBTRACKFILE into the 1-σ value of the Vertical Estimated Position Uncertainty (VEPU), as per the procedure established in [5].

The algorithm first converts the GVA to a Vertical Figure of Merit (VFOM), and then invokes CONVERTVFOMTOSIGMAVEPU (Algorithm 31) to generate the return value. It should be noted that currently only 1090ES version 3 supports the reporting of the GVA value 3. Therefore, if GVA 3 has been received from an ADS-B OUT source that is not 1090ES version 3, the GVA value will be considered invalid and subsequently discarded.

The algorithm takes as input *trk*. The algorithm returns the 1-σ value of VEPU, *sigma_vepu*, or *NaN* should the ADSBTRACKFILE contains an invalid GVA value.

Algorithm 122 ConvertGVAToSigmaVEPU

```
function ConvertGVAToSigmaVEPU(trk::ADSBTrackFile(p. 485))
    vfom_m::R = NaN
    if (trk.gva == 1)
        vfom_m = 150
    elseif (trk.gva == 2)
        vfom_m = 45
    elseif (trk.adsb_version >= 3) && !trk.uat && (trk.gva == 3)
        vfom_m = 10
    end
    return ConvertVFOMToSigmaVEPU(p. 35)(vfom_m)::R
end
```

Referenced In: AddAltBiasAndSample(p. 143), SetIntruderDegradedFlags(p. 138), AssessVerticalValidity(p. 123)

OPTIMALQUALITYTRACK (Algorithm 123) finds the optimal quality track for a given list of tracks based on a quality metric. The optimal track (*opt_trk*) is first initialized as nothing. If at least one track exists in the list, then the optimal track is updated. First, the *trk_error* vector is initialized as all zeros of length equal to the number of tracks. For each target track in the list, the mean and covariance are extrapolated to the report time T with a call to PREDICTPRETRACKED-TRACK (Algorithm 124) if the track's time of applicability is older than the previous report time. Next, the mean and covariance are converted to polar coordinates via a call to CONVERTCARTESIANTOPOLAR2D (Algorithm 432). If the track list contains tracks with valid vertical information, as indicated by *valid_vert*, the quality metric is computed as a weighted norm of the horizontal (ground range rate and cross ground range rate) and vertical velocity track uncertainties. Otherwise, the metric is the norm of the ground range rate and cross ground range rate uncertainties only. Next, the indices of the ascending *trk_error* vector are specified in *sort_idx*. The first track in the sorted list is used to update *opt_trk*, selecting the track with the minimum metric value. This algorithm takes as inputs *tracks*, *valid_vert*, and T. This algorithm returns *opt_trk*.

Algorithm 123 OptimalQualityTrack

```
function OptimalQualityTrack(tracks::Vector{TrackFile(p. 483)}, valid_vert::Bool, T::R)
    hor_weight = params().surveillance.report_generation.optimal_quality_track.horizontal_weight
    vert_weight = params().surveillance.report_generation.optimal_quality_track.vertical_weight
    I_dgrdxgr = [3,4]
    I_dz = [2]
    opt_trk = nothing
    trk_error = zeros(length(tracks))
    for k in 1:length(tracks)
        trk = tracks[k]
        (mu_hor, Sigma_hor, _, Sigma_vert) = PredictPreTrackedTrack(p. 126)(trk, T)
       (_, Sigma_ra) = ConvertCartesianToPolar2D(p. 448)(mu_hor, Sigma_hor)
       if valid_vert
            trk_error[k] = hor_weight * norm(diag(Sigma_ra)[I_dgrdxgr]) + vert_weight * norm(diag(Sigma_vert
                )[I_dz])
       else
            trk_error[k] = norm(diag(Sigma_ra)[I_dgrdxgr])
       end
    end
    if length(tracks)>0
        sort_idx = sortperm(trk_error)
        opt_trk = tracks[sort_idx[1]]
    end
    return (opt_trk::Union{AGTTrackFile(p. 486), ORNCTTrackFile(p. 484), Nothing})
end
```

Referenced In: OptimalPreTrackedTracks(p. 122)

PREDICTPRETRACKEDTRACK (Algorithm 124) propagates the mean and covariance of an AGT or ORNCT track forward in time if the track time of applicability is older than *T*.

- If the track is an AGTTRACKFILE, *dt_vert* and *dt_hor* are set to the difference between *T* and the track TOAs for the vertical and horizontal tracks respectively. Next, a call to PREDICTPASSIVETRACKER (Algorithm 44) propagates the horizontal track states forward by *dt_hor* seconds and updates *mu_hor_pred* and *Sigma_hor_pred*. The propagated mean and covariances values are then converted from meters to feet. Similarly, PREDICTVERTICALTRACKER (Algorithm 54) updates *mu_vert_pred* and *Sigma_vert_pred* by *dt_vert* seconds.
- If the track is an ORNCTTRACKFILE, the horizontal and vertical track components share a common TOA and a call to *predict_ornct_track* is made to perform the track prediction for the time step *dt*. The Cartesian mean and covariance returned by the algorithm are used to update *mu_hor_pred* and *Sigma_hor_pred*.

This algorithm takes as input *trk* and *T* and returns *mu_hor_pred*, *Sigma_hor_pred*, *mu_vert_pred*, and *Sigma_vert_pred*.

Algorithm 124 PredictPreTrackedTrack

```
function PredictPreTrackedTrack(trk::Union{AGTTrackFile(p. 486), ORNCTTrackFile(p. 484)}, T::R)
    m2ft::R = geoutils.meters_to_feet
    if (typeof(trk) == AGTTrackFile(p. 486))
        dt_vert = T - trk.toa_vert
        dt_hor = T - trk.toa_hor
        (mu_hor_pred, Sigma_hor_pred) = PredictPassiveTracker(p. 46)(trk.mu_hor, trk.Sigma_hor, dt_hor, false)
        mu_hor_pred *= m2ft
        Sigma_hor_pred *= m2ft^2
        is_large::Bool = (trk.classification != CLASSIFICATION_SMALL_UNMANNED)
        (mu_vert_pred, Sigma_vert_pred) = PredictVerticalTracker(p. 54)(trk.mu_vert, trk.Sigma_vert, dt_vert, trk.
            alt_src_hae, false, is_large)
    else
        dt = T - trk.toa
        (mu_vert_pred, Sigma_vert_pred, _, _, mu_cart_pred, Sigma_cart_pred) = PredictORNCTTrack(p. 64)(trk, dt
            )
        mu_hor_pred = mu_cart_pred
        Sigma_hor_pred = Sigma_cart_pred
    end
    return (mu_hor_pred::Vector{R}, Sigma_hor_pred::Matrix{R}, mu_vert_pred::Vector{R}, Sigma_vert_pred::
        Matrix{R})
end
```

Referenced In: OptimalQualityTrack(p. 125)

PASSIVEQUALITYCHECK (Algorithm 125) performs an M-of-N check on the quality history of passive surveillance sources. First, the algorithm checks if the input *trk* is a V2V Track or an ADS-B track. If it is an ADS-B track, the passive quality flag will only be updated if the vertical track has been properly initialized, as indicated by *trk.init_velocity*. In both cases, if *M* of these tests passed in the history, PASSIVEQUALITYCHECK (Algorithm 125) returns true. Additionally, if *init_M* of *init_N* reports pass, then PASSIVEQUALITYCHECK (Algorithm 125) returns true. Otherwise, the algorithm returns false. When evaluating quality state transitions for ADS-B, if the M-of-N check passed (*passive_quality_good*) and the *trk.qual_state* has not been validated yet (*QUAL_STATE_NOT_VALIDATED*), the track will transition to a valid state as indicated by *QUAL_STATE_VALID*. If the M-of-N check failed, *trk.passive_qual_history* is full (has *N* reports), and *trk.qual_state* is valid, the track will transition to an invalid state as indicated by *QUAL_STATE_INVALID*. If *M_revalidate* of the tests have passed, the track will transition to a valid state regardless of what the state was before.

This algorithm takes as input *trk*. This algorithm updates *trk.qual_state* and returns *quality_state_valid*.

Algorithm 125 PassiveQualityCheck

```
function PassiveQualityCheck(trk::Union{TrackFile(p. 483), Nothing})
    passive_quality_good::Bool = false
    quality_state_valid::Bool = false
    if TrackExists(p. 19)(trk)
        if (typeof(trk) == ADSBTrackFile(p. 485))
            M::UInt32 = params().surveillance.report_generation.min_adsb_quality.M
            N::UInt32 = params().surveillance.report_generation.min_adsb_quality.N
            init_N::Z = params().surveillance.report_generation.min_adsb_quality.init_N
            init_M::Z = params().surveillance.report_generation.min_adsb_quality.init_M
            M_revalidate::Z = params().surveillance.report_generation.min_adsb_quality.M_revalidate
        elseif (typeof(trk) == V2VTrackFile(p. 488))
            M = params().surveillance.report_generation.min_v2v_quality.M
            N = params().surveillance.report_generation.min_v2v_quality.N
            init_N = params().surveillance.report_generation.min_v2v_quality.init_N
            init_M = params().surveillance.report_generation.min_v2v_quality.init_M
            M_revalidate = params().surveillance.report_generation.min_v2v_quality.M_revalidate
        end
        vert_initialized::Bool = (trk.updates_vert > 1)
        if (sum(trk.passive_qual_history) >= M)
            passive_quality_good = true
        elseif (init_N == length(trk.passive_qual_history)) && (init_M <= sum(trk.passive_qual_history)) &&
             vert_initialized
            passive_quality_good = true
        end
        if (passive_quality_good) && (trk.init_velocity) && (QUAL_STATE_NOT_VALIDATED == trk.qual_state)
            trk.qual_state = QUAL_STATE_VALID
        elseif ((!passive_quality_good && (N == length(trk.passive_qual_history))) || !trk.init_velocity) &&
              (QUAL_STATE_VALID == trk.qual_state)
            trk.qual_state = QUAL_STATE_INVALID
        elseif (M_revalidate <= sum(trk.passive_qual_history)) && (trk.init_velocity)
            trk.qual_state = QUAL_STATE_VALID
        end
        quality_state_valid = (QUAL_STATE_VALID == trk.qual_state)
    end
    return quality_state_valid::Bool
end
```

Referenced In: TrackSourceSelection(p. 121)

OPTIMALTRACKHISTORYUPDATE (Algorithm 126) updates the optimal track history of a given target. First, *tgt* is retrieved from the target database. If the optimal track is not nothing, the length of the optimal track history is checked. If the list is already of length *N*, the oldest element is deleted from the end of the list. Then the unique track id *opt_trk.track_uid* of the optimal track is added as the first element in the list. Otherwise, if *opt_trk* is nothing, indicating that the target only does not have any available and acceptable tracks, the optimal track history is reset. This algorithm takes as input the target *id* and *opt_trk*. This algorithm updates *tgt.optimal_trk_history*.

Algorithm 126 OptimalTrackHistoryUpdate

```
function OptimalTrackHistoryUpdate(id::UInt32, opt_trk::Union{TrackFile(p. 483), Nothing})
    N::Z = params().surveillance.report_generation.optimal_track_history.N
    tgt = target_db[id]
    if (opt_trk != nothing)
        if (length(tgt.optimal_trk_history) >= N)
            pop!(tgt.optimal_trk_history)
        end
        pushfirst!(tgt.optimal_trk_history, opt_trk.track_uid)
    else
        tgt.optimal_trk_history = String[]
    end
end
```

Referenced In: TrackSourceSelection(p. 121)

SELECTFINALTRACK (Algorithm 127) finalizes the track source selection for a given target. First, *tgt* is retrieved from the target database using *target_id*. Two counting values, *count_opt* and *count_prev_sel_trk*, are initialized. The target's optimal track history is iterated through and two checks are made:

- If the optimal track history entry matches the optimal track, *count_opt* is incremented.
- If the optimal track history entry matches the previously selected track, *count_prev_sel_trk* is incremented.

Next, the flag *has_code* is set to true if the target has an active vertical or horizontal RA. The previously selected track is obtained through a call to GETPREVIOUSLYSELECTEDTRACK (Algorithm 128) and a flag is set to true if the track is still in the target. The *ra_lock* flag is activated if the target has an active RA and the previously selected track is still in the target. The *prev_sel_trk_-is_acceptable* is set to true if the previously selected track is in the target in the optimal track history. The *opt_trk_better* flag is set to true if the M of N criteria has been met and *opt_trk* is not the previously selected track. Finally, the *selection_lock* is invoked if there is an active RA on the target or if the previously selected track is still acceptable and the optimal track is not better. The outputs *final_trk* and *final_trk_uid* are set to the previously selected track if the selection lock is active. Otherwise, they are updated to reflect the optimal track and optimal track id.

This algorithm takes as input *target_id* and *opt_trk*. This algorithm returns *final_trk* and *final_-trk_uid*.

Algorithm 127 SelectFinalTrack

```
function SelectFinalTrack(target_id::UInt32, opt_trk::Union{TrackFile(p. 483), Nothing})
    M::Z = params().surveillance.report_generation.optimal_track_history.M
    tgt = target_db[target_id]
    if (opt_trk != nothing)
        opt_trk_uid = opt_trk.track_uid
        count_opt::Z = 0
        count_prev_sel_trk::Z = 0
        for i in 1:length(tgt.optimal_trk_history)
            if (tgt.optimal_trk_history[i] == opt_trk_uid)
                count_opt += 1
            end
            if (tgt.optimal_trk_history[i] == tgt.previously_selected_trk_uid)
                count_prev_sel_trk += 1
            end
        end
        has_code::Bool = (tgt.priority_codes.vert > SXUCODE_CLEAR) || (tgt.priority_codes.horiz >
             SXUCODE_CLEAR)
        prev_trk = GetPreviouslySelectedTrack(p. 130)(tgt)
        prev_sel_trk_in_tgt::Bool = (prev_trk != nothing)
        ra_lock::Bool = has_code && prev_sel_trk_in_tgt
        prev_sel_trk_is_acceptable::Bool = prev_sel_trk_in_tgt && (count_prev_sel_trk > 0)
        opt_trk_better::Bool = (count_opt >= M) && (opt_trk_uid != tgt.previously_selected_trk_uid)
        selection_lock::Bool = ra_lock || (prev_sel_trk_is_acceptable && !opt_trk_better)
        if selection_lock
            final_trk = prev_trk
            final_trk_uid = tgt.previously_selected_trk_uid
        else
            final_trk = opt_trk
            final_trk_uid = opt_trk_uid
        end
    else
        final_trk = nothing
        final_trk_uid = nothing
    end
    return (final_trk::Union{TrackFile(p. 483), Nothing}, final_trk_uid::Union{String, Nothing})
end
```

Referenced In: TrackSourceSelection(p. 121)

GETPREVIOUSLYSELECTEDTRACK (Algorithm 128) retrieves the previously selected track for a given target. First, *prev_trk_uid* is set to the unique track identifier of the target's previously selected track. If previously selected track is not nothing, a check is made with TRACKEXISTS (Algorithm 12) to see if the appropriate track still exists for the target and that the track id matches. If this check passes, *trk* is updated. This algorithm takes as input *tgt* and returns the previously selected track, *trk*.

Algorithm 128 GetPreviouslySelectedTrack

```
function GetPreviouslySelectedTrack(tgt::Target(p. 493))
    trk = nothing
    prev_trk_uid = tgt.previously_selected_trk_uid
    if (prev_trk_uid != nothing)
        if occursin("ADSB", prev_trk_uid)
            if TrackExists(p. 19)(tgt.adsb_track) && (tgt.adsb_track.track_uid == prev_trk_uid)
                trk = tgt.adsb_track
            end
        elseif occursin("V2V", prev_trk_uid)
            if TrackExists(p. 19)(tgt.v2v_track) && (tgt.v2v_track.track_uid == prev_trk_uid)
                trk = tgt.v2v_track
            end
        elseif occursin("ORNCT", prev_trk_uid)
            if TrackExists(p. 19)(tgt.ornct_track) && (tgt.ornct_track.track_uid == prev_trk_uid)
                trk = tgt.ornct_track
            end
        elseif occursin("AGT", prev_trk_uid)
            for agt_track in tgt.agt_tracks
                if (agt_track.track_uid == prev_trk_uid)
                    trk = agt_track
                    break
                end
            end
        end
    end
    return trk::Union{TrackFile(p. 483), Nothing}
end
```

Referenced In: SelectFinalTrack(p. 129)

The ACAS X system limits the number of intruders and point-obstacles for processing by the Threat Resolution Module (TRM) to *max_intruders* and *max_point_obstacles*, respectively. In the case where there are more intruders or point-obstacles than that, some intruders or point-obstacles need to be filtered from the STM output.

FILTERTRACKSFORTRM (Algorithm 129) prioritizes intruders for output to the TRM and filters out the lowest priority intruders. Intruders are prioritized based on relative importance with respect to TRM processing.

The intruders are first sorted by slant range via nested calls to COMPAREINTRUDERRANGE (Algorithm 130) and SLANTRANGE (Algorithm 131). The range-sorted intruders are then categorized by the following priorities:

1. There is an active resolution advisory on the intruder, as indicated by an sxucode of *SXUCODE_RA*
2. A non-zero VRC or HRC coordination message was received from the intruder

The category related to alerts in the above priority list comes from the *SXUCODE_* constants output by the TRM. The intruder codes are stored in the *priority_codes* field of the TARGET during STMHOUSEKEEPING. The intruder codes rank the severity of the threat of the intruder with higher values indicating a higher threat.

To accomplish the sorting of intruders by the priorities listed above, the logic creates a vector of temporary STMDISPLAYSTRUCT and TRMINTRUDERINPUT vectors, one for each priority plus one more for intruders with no priority *MAX_IDX_PRIORITY*. The full list of intruders originally

in the report will be distributed between the *MAX_IDX_PRIORITY* number of arrays.

The intruders are then placed in range order into each of the *MAX_IDX_PRIORITY* vectors in the middle prioritization loop. The report display and *trm_input.intruder* vectors are cleared. The final loop indexes through *MAX_IDX_PRIORITY* count of the temporary display and *trm_-input.intruder*. Each temporary vector is appended to the report display and *trm_input.intruder* vectors until *max_intruders* intruders have been added. If the temporary vector is long enough to add more than *max_intruders* to the report, only the remaining intruders are added from that vector. If the report already has *max_intruders* intruders, none of the temporary vector is added. This results in a report that is priority and range sorted.

The first *max_intruders* are chosen for TRM processing from the resulting prioritized and range-ordered lists. The TRM will receive the intruders that are most likely to pose a threat. Because *report.display* and *report.trm_input.intruder* contain information about the same intruders, this filtering also impacts the targets available to the on-board pilot display. Note that while both the *report.display* and *report.trm_input.intruder* vectors are expected to be the same length, defensive programming in this algorithm will handle the alternate case.

Similar the first *max_point_obstacles* are chosen for TRM processing from the resulting prioritized and range-ordered lists.

This algorithm takes as input *report* and updates *report.display* and *report.trm_input.intruder*.

Algorithm 129 FilterTracksForTRM

```
function FilterTracksForTRM( report::StmReport(p. 480) )
    max_intruders::Z = params().surveillance.max_intruders
    max_point_obstacles::Z = params().surveillance.max_point_obstacles
    MAX_IDX_PRIORITY::Z = 6
    display_tmp::Vector{Vector{StmDisplayStruct(p. 481)}}  = Vector{Vector{StmDisplayStruct(p. 481)}}(undef,
         MAX_IDX_PRIORITY * 2)
    intruder_tmp::Vector{Vector{TRMIntruderInput(p. 499)}} = Vector{Vector{TRMIntruderInput(p. 499)}}(undef,
         MAX_IDX_PRIORITY * 2)
    for idx_priority in 1:length(display_tmp)
        display_tmp[idx_priority] = StmDisplayStruct(p. 481)[]
        intruder_tmp[idx_priority] = TRMIntruderInput(p. 499)[]
    end
    sorted_idx = sortperm( report.display, lt = CompareIntruderRange(p. 133) )
    for idx_intruder in sorted_idx
        if (idx_intruder <= length(report.trm_input.intruder))
            target = target_db[report.trm_input.intruder[idx_intruder].id]
            xu_code::Z = max( target.priority_codes.vert, target.priority_codes.horiz )
            idx_priority::Z = MAX_IDX_PRIORITY
            if (xu_code == SXUCODE_RA)
                idx_priority = 1
            elseif (0 != report.trm_input.intruder[idx_intruder].vrc) || (0 != report.trm_input.intruder[
                 idx_intruder].hrc)
                idx_priority = 2
            end
            idx_priority_offset::Z = 0
            if (TrackExists(p. 19)(target.point_obstacle_track))
                idx_priority_offset = MAX_IDX_PRIORITY
            end
            push!( display_tmp[idx_priority + idx_priority_offset],  report.display[idx_intruder] )
            push!( intruder_tmp[idx_priority + idx_priority_offset], report.trm_input.intruder[idx_intruder]
                 )
        end
    end
    report.display = StmDisplayStruct(p. 481)[]
    report.trm_input.intruder = TRMIntruderInput(p. 499)[]
    remaining::Z = max_intruders
    for idx_priority in 1:MAX_IDX_PRIORITY
        limit::Z = min( length(display_tmp[idx_priority]), remaining )
        if (0 < limit)
            append!( report.display, display_tmp[idx_priority][1:limit] )
            append!( report.trm_input.intruder, intruder_tmp[idx_priority][1:limit] )
            remaining = remaining - limit
        end
    end
    remaining = max_point_obstacles
    for idx_priority in 1:MAX_IDX_PRIORITY
        limit = min( length(display_tmp[idx_priority+MAX_IDX_PRIORITY]), remaining )
        if (0 < limit)
            append!( report.display, display_tmp[idx_priority+MAX_IDX_PRIORITY][1:limit] )
            append!( report.trm_input.intruder, intruder_tmp[idx_priority+MAX_IDX_PRIORITY][1:limit] )
            remaining = remaining - limit
        end
    end
    return
end
```

Referenced In: AddTracksToReport(p. 119)

COMPAREINTRUDERRANGE takes as input *intr_a* and *intr_b*. COMPAREINTRUDERRANGE outputs a boolean that indicates whether the SLANTRANGE of *intr_a* is less than the SLANTRANGE of *intr_b*.

Algorithm 130 CompareIntruderRange

```
function CompareIntruderRange(intr_a::StmDisplayStruct(p. 481), intr_b::StmDisplayStruct(p. 481))
    return SlantRange(p. 133)(intr_a.r_ground_ft, intr_a.z_rel_ft) < SlantRange(p. 133)(intr_b.r_ground_ft, intr_b.
        z_rel_ft)
end
```

Referenced In: FilterTracksForTRM(p. 132)

SLANTRANGE takes as input *r_ground* and *z_rel*. SLANTRANGE returns *r_ground* if *z_rel* is *NaN*, and returns the hypotenuse of *r_ground* and *z_rel* if *z_rel* is a number.

Algorithm 131 SlantRange

```
function SlantRange(r_ground::R, z_rel::R)
    if (isnan(z_rel))
        return r_ground
    else
        return hypot(r_ground, z_rel)
    end
end
```

Referenced In: ExtrapolateAGTTrack(p. 447), CompareIntruderRange(p. 133)

2.8.4.1 ADS-B Track File

ADSBTRACKFILEs are added to an STMREPORT through ADDADSBTRACKTOREPORT (Algorithm 132). First, the corresponding Track File is retrieved from memory based on the *adsb_track.uat* flag. Then a new TRMINTRUDERINPUT (i.e., the data structure in the STMREPORT that represents an intruding aircraft) is initialized. Several of the data elements in the TRMINTRUDERINPUT are then populated with the associated data in the ADSBTRACKFILE. This includes the track ID (*id*), the source of the intruder state information (*source*), and the validity of the track data (*valid*) for RA processing by the TRM. The addresses assigned by an external surveillance system, the Mode S address in the case of an ADSBTRACKFILE, are set through a call to SETINTRUDERIDDIRECTORY (Algorithm 134). Intruder classification is set with a call to SETCLASSIFICATION (Algorithm 133). SETCOORDINATION (Algorithm 104) is called to set equipage information as well as several coordination related fields. Targets are marked as invalid for RA processing when ownship is in standby mode, as indicated by *own.opmode*. Degraded surveillance flags are set in a call to SETINTRUDERDEGRADEDFLAGS.

If necessary, the two tracks contained in the ADSBTRACKFILE are extrapolated to the time of the report by calling the corresponding prediction algorithms. If necessary, the two tracks contained in the V2VTRACKFILE are extrapolated to the time of the report by calling the corresponding prediction algorithms. If *trm_altmode* is not equal to *TRM_ALTMODE_NONE*, the altitude of the intruder undergoes a conversion to ensure it is defined with respect to the same vertical datum as ownship. This is done by first generating the relative altitude by a call to GETRELATIVEALTITUDE, and then converting back into an absolute altitude by adding the reported altitude of ownship to the relative altitude. The horizontal extrapolation is not performed if the time difference between the last track update and the TOA of the report is less than a defined minimum threshold (*min_extrap_toa_step*). After predicting the absolute horizontal tracker, the mean position

is re-centered about the new geodetic latitude-longitude-altitude (LLA) position by a call to RECENTERHORIZONTALTRACKLOCATION. Finally, the absolute geodetic ADSB track is converted to ownship relative Cartesian states with a call to ABSOLUTEGEODETICTOOWNSHIPRELATIVE.

The representation of the target aircraft's vertical state is provided as a set of weighted samples given by *vsamples* and *vweights*. These samples are produced using the mean (*mu_vert*) and covariance (*Sigma_vert*) of the vertical track through a call to ADDALTBIASANDSAMPLE. The representation of the target aircraft's relative horizontal state is also provided as a set of weighted samples given by *hsamples* and *hweights*. These samples are produced using the mean (*mu_hor*) and covariance (*Sigma_hor*) of the horizontal track that has been converted from metric to English units. The vertical and horizontal samples are provided to SETINTRUDERBELIEFSTATES for assignment within the TRMINTRUDERINPUT.

The *passive_only* flag indicates whether or not the ADS-B track file has been validated. In the last steps, the STMDISPLAYSTRUCT vector gets filled by SETDISPLAYDATAPASSIVE and the TRMINTRUDERINPUT is appended to STMREPORT.

ADDADSBTRACKTOREPORT takes as input *report*, *id*, *T*, and *passive_only*. ADDADSBTRACKTOREPORT updates *report.trm_input.intruder*.

Algorithm 132 AddADSBTrackToReport

```
function AddADSBTrackToReport(report::StmReport(p. 480), id::UInt32, T::R, passive_only::Bool)
    min_extrap_toa_step::R = params().surveillance.min_extrap_toa_step
    m2ft::R = geoutils.meters_to_feet
    I_xydxdy::Vector{Z} = [1, 3, 2, 4]
    tgt = target_db[id]
    if tgt.adsb_track.uat
        (trk, source) = (tgt.adsb_track, SOURCE_978_UAT)
    else
        (trk, source) = (tgt.adsb_track, SOURCE_1090ES_ADSB)
    end
    intruder = TRMIntruderInput(p. 499)()
    intruder.id = id
    intruder.source = source
    SetClassification(p. 136)(intruder)
    SetIntruderIDDirectory(p. 137)(tgt, intruder.id_directory)
    SetCoordination(p. 104)(intruder)
    SetIntruderDegradedFlags(p. 138)(intruder, tgt, trk, passive_only)
    valid_vert = (0 == (intruder.degraded_surveillance & DEGRADED_SURVEILLANCE_NAR))
    h_own = report.trm_input.own.belief_vert[1].z
    intruder.valid = true
    if (own.opmode == OPMODE_STANDBY)
        intruder.valid = false
    end
    dt_vert = T - trk.toa_vert
    dt_hor = T - trk.toa_hor
    (mu_vert, Sigma_vert) = PredictVerticalTracker(p. 54)(trk.mu_vert, trk.Sigma_vert, dt_vert, trk.alt_src_hae,
        false, true)
    if (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        z_rel = GetRelativeAltitude(p. 139)(mu_vert[1], trk.alt_src_hae)
        mu_vert[1] = z_rel + h_own
    end
    trk_temp = deepcopy(trk)
    if (dt_hor >= min_extrap_toa_step)
        (trk_temp.mu_hor, trk_temp.Sigma_hor) = PredictPassiveTracker(p. 46)(trk_temp.mu_hor, trk_temp.Sigma_hor,
            dt_hor, false)
        ReCenterHorizontalTrackLocation(p. 42)(trk_temp, T)
    end
    (mu_hor, Sigma_hor) = AbsoluteGeodeticToOwnshipRelative(p. 141)(trk_temp)
    (vsamples,vweights) = AddAltBiasAndSample(p. 143)(trk, mu_vert, Sigma_vert, valid_vert, trk.alt_src_hae,
        true, T)
    (mu_xydxdy, Sigma_xydxdy) = (mu_hor[I_xydxdy] * m2ft, Sigma_hor[I_xydxdy, I_xydxdy] * m2ft^2)
    (hsamples, hweights) = SigmaPointSample(p. 144)(mu_xydxdy, Sigma_xydxdy)
    SetIntruderBeliefStates(p. 144)(intruder, vsamples, vweights, hsamples, hweights)
    SetDisplayDataPassive(p. 146)(report, intruder, mu_vert, mu_hor * m2ft, valid_vert, T, trk.
        display_arrow_current, !passive_only)
    intruder.adsb_cccb = deepcopy(tgt.adsb_osm.adsb_cccb)
    intruder.v2v_cccb = deepcopy(tgt.v2v_osm.v2v_cccb)
    push!(report.trm_input.intruder,intruder)
end
```

Referenced In: AddTracksToReport(p. 119)

SETCLASSIFICATION (Algorithm 133) first creates a clasification vector that contains the classifications identified by each track type in the target. Next, the final classification is selected from the classification vector based on a hierarchy that first prefers manned, then unmanned, then none, then small unmanned.

This algorithm takes as input *intruder*. This algorithm updates *intruder.classification*

Algorithm 133 SetClassification

```
function SetClassification(intruder::TRMIntruderInput(p. 499))
    tgt = RetrieveWithID(p. 19)(target_db, intruder.id)
    classification_vec::Vector{UInt8} = UInt8[]
    if TrackExists(p. 19)(tgt.adsb_track)
        push!(classification_vec, CLASSIFICATION_NONE)
    end
    if TrackExists(p. 19)(tgt.v2v_track)
        push!(classification_vec, tgt.v2v_track.classification)
    end
    if TrackExists(p. 19)(tgt.ornct_track)
        push!(classification_vec, tgt.ornct_track.classification)
    end
    if TrackExists(p. 19)(tgt.agt_tracks)
        for trk in tgt.agt_tracks
            push!(classification_vec, trk.classification)
        end
    end
    classification = CLASSIFICATION_NONE
    if (CLASSIFICATION_MANNED in classification_vec)
        classification = CLASSIFICATION_MANNED
    elseif (CLASSIFICATION_UNMANNED in classification_vec)
        classification = CLASSIFICATION_UNMANNED
    elseif (CLASSIFICATION_NONE in classification_vec)
        classification = CLASSIFICATION_NONE
    elseif (CLASSIFICATION_SMALL_UNMANNED in classification_vec)
        classification = CLASSIFICATION_SMALL_UNMANNED
    end
    intruder.classification = classification
end
```

Referenced In: AddADSBTrackToReport(p. 135), AddORNCTTrackToReport(p. 152), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

SETINTRUDERIDDIRECTORY (Algorithm 134) populates *id_directory*, an INTRUDERIDDIRECTORY object that records all valid identifiers that have been assigned to various TRACKFILEs in a TARGET by external surveillance systems.

The algorithm first checks if *tgt* is in possession of an ADSBTRACKFILE. If it is, the *icao24* or the *anon24* field is set according to the type (ICAO or anonymous) of Mode S address of the track.

The *v2v_uid* field is then set with the V2V unique identifier of the V2VTRACKFILE should one exist in *tgt*. In addition, if the V2VTRACKFILE is in possession of a valid Mode S address, the corresponding field, *icao24* or *anon24*, in *id_directory* will be set based on the type of the address, provided that the field has not previously been set during the processing of the ADSBTRACKFILE.

The *ornct_id* field is set based on the ORNCT-ID of the ORNCTTRACKFILE should one exist in *tgt*.

The algorithm then iterates over the AGTTRACKFILEs that have been associated to *tgt* and adds their respective AGT-IDs to the vector *agt_ids*. In addition, if an AGTTRACKFILE is in possession of a valid Mode S address or V2V-UID and the corresponding field in *id_directory* has not previously been set, those fields will be populated by the values held by the AGTTRACKFILE.

Lastly, the *po_id* field is set based on the PO-ID of the POINTOBSTACLETRACKFILE should one exist in *tgt*.

This algorithm takes as input *tgt* and *id_directory*. This algorithm updates *id_directory*.

Algorithm 134 SetIntruderIDDirectory

```
function SetIntruderIDDirectory(tgt::Target(p. 493), id_directory::IntruderIDDirectory(p. 482))
    if TrackExists(p. 19)(tgt.adsb_track)
        if !tgt.adsb_track.non_icao
            id_directory.icao24 = IntruderIDEntry(p. 482)(tgt.adsb_track.mode_s)
        else
            id_directory.anon24 = IntruderIDEntry(p. 482)(tgt.adsb_track.mode_s)
        end
    end
    if TrackExists(p. 19)(tgt.v2v_track)
        id_directory.v2v_uid = IntruderIDEntry(p. 482)(tgt.v2v_track.v2v_uid)
        if tgt.v2v_track.mode_s_valid
            if !tgt.v2v_track.mode_s_non_icao && !id_directory.icao24.valid
                id_directory.icao24 = IntruderIDEntry(p. 482)(tgt.v2v_track.mode_s)
            elseif tgt.v2v_track.mode_s_non_icao && !id_directory.anon24.valid
                id_directory.anon24 = IntruderIDEntry(p. 482)(tgt.v2v_track.mode_s)
            end
        end
    end
    if TrackExists(p. 19)(tgt.ornct_track)
        id_directory.ornct_id = IntruderIDEntry(p. 482)(tgt.ornct_track.ornct_id)
    end
    for trk in tgt.agt_tracks
        push!(id_directory.agt_ids, trk.agt_id)
        if trk.v2v_uid_valid && !id_directory.v2v_uid.valid
            id_directory.v2v_uid = IntruderIDEntry(p. 482)(trk.v2v_uid)
        end
        if trk.mode_s_valid
            if !trk.mode_s_non_icao && !id_directory.icao24.valid
                id_directory.icao24 = IntruderIDEntry(p. 482)(trk.mode_s)
            elseif trk.mode_s_non_icao && !id_directory.anon24.valid
                id_directory.anon24 = IntruderIDEntry(p. 482)(trk.mode_s)
            end
        end
    end
    if TrackExists(p. 19)(tgt.point_obstacle_track)
        id_directory.po_id = IntruderIDEntry(p. 482)(tgt.point_obstacle_track.po_id)
    end
    return
end
```

Referenced In: AddADSBTrackToReport(p. 135), AddPointObstacleTrackToReport(p. 157), AddORNCTTrackToReport(p. 152), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

SETINTRUDERDEGRADEDFLAGS (Algorithm 135) sets the various degraded surveillance flags for each intruder. The *intruder.degraded_surveillance* variable is an unsigned 16-bit integer and it functions to store flags for the various degraded surveillance modes. This algorithm performs a series of checks for degraded surveillance conditions, and a corresponding bit is set in *intruder.degraded_surveillance* if any are met. This accumulation of degraded surveillance bits occurs via the updating plus operator, and degraded modes are checked in other algorithms via the bitwise *AND* operation. First, this algorithm checks to see if the input *trk* is an ORNCT track. If so, the TOA of the ORNCT track update is first checked to determine if the track has been coasted, indicated by *DEGRADED_SURVEILLANCE_ORNCT_COAST*. A further check is performed to determine if the track was coasted because an intruder left the Field-of-View (FOV), indicated by *DEGRADED_SURVEILLANCE_ORNCT_FOV_COAST*; and finally the validity of the ownship altitude tracks are checked to determine if the ORNCT track is Non-Altitude-Reporting (NAR), indicated by *DEGRADED_SURVEILLANCE_NAR*. If the track is not an ORNCT track, degraded surveillance processing begins with an encompassing check of ownship and intruder altitude valid-

ity, quality, and compatibility in order to set the corresponding *DEGRADED_SURVEILLANCE_-NAR* bit. The algorithm proceeds to check if the vertical track has been coasted, indicated by *DEGRADED_SURVEILLANCE_VERT_COAST*; if the track has been externally validated, indicated by *DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED*; and if the horizontal track has been coasted, indicated by *DEGRADED_SURVEILLANCE_HORIZ_COAST*. Regardless of the input surveillance type, a check of bad V2V coordination in the vertical and horizontal dimensions is performed, and degraded bits are set accordingly.

This algorithm takes as input *intruder*, *tgt*, *trk*, and *passive_only*. This algorithm updates *intruder.degraded_surveillance*.

Algorithm 135 SetIntruderDegradedFlags

```
function SetIntruderDegradedFlags(intruder::TRMIntruderInput(p. 499), tgt::Target(p. 493), trk::TrackFile(p. 483),
    passive_only::Bool)
    min_gva::UInt32 = params().surveillance.report_generation.min_adsb_quality.gva
    is_ornct::Bool = (typeof(trk) == ORNCTTrackFile(p. 484))
    own_valid_alt_pres::Bool = !isnan(own.toa_vert)
    own_valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
    if is_ornct
        if (trk.toa_update <= own.prev_rpt_toa)
            intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_ORNCT_COAST
        end
        if trk.is_FOV_coast
            intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_ORNCT_FOV_COAST
        end
        if !own_valid_alt_pres && !own_valid_alt_hae
            intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_NAR
        end
    else
        incompatible_alt_types = ((trk.alt_src_hae && !own_valid_alt_hae) || (!trk.alt_src_hae && !
            own_valid_alt_pres))
        invalid_gva::Bool = isa(trk, ADSBTrackFile(p. 485)) && trk.alt_src_hae && ((trk.gva < min_gva) || isnan(
            ConvertGVAToSigmaVEPU(p. 124)(trk)))
        if !trk.valid_vert || incompatible_alt_types || invalid_gva
            intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_NAR
        end
        if (trk.toa_vert < own.prev_rpt_toa)
            intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_VERT_COAST
        end
        if passive_only
            intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED
        end
        if (trk.toa_hor < own.prev_rpt_toa)
            intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_HORIZ_COAST
        end
    end
    if (tgt.bad_v2vcoordination_vert)
        intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_INVALID_V2VCOORDINATION_VERT
        tgt.bad_v2vcoordination_vert = false
    end
    if (tgt.bad_v2vcoordination_horz)
        intruder.degraded_surveillance += DEGRADED_SURVEILLANCE_INVALID_V2VCOORDINATION_HORZ
        tgt.bad_v2vcoordination_horz = false
    end
end
```

Referenced In: AddADSBTrackToReport(p. 135), AddORNCTTrackToReport(p. 152), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

GETRELATIVEALTITUDE (Algorithm 136) will take in an altitude along with a flag indicating if it is geodetic height-above-ellipsoid (HAE) or pressure altitude, and then generate a relative altitude centered about the ownship's altitude of the same altitude type. If the intruder's altitude type is pressure and the ownship's pressure altitude is valid, then a relative pressure altitude is generated. If the intruder's altitude type is geodetic HAE and the ownship's HAE altitude is valid, then a relative HAE altitude is generated. Otherwise, the ownship does not have the same altitude type as the intruder available and so the two aircraft are assumed to be co-altitude and the relative altitude is set to zero.

This algorithm takes as input *alt* and *is_alt_geo_hae*. This algorithm returns the relative altitude *z_rel*.

Algorithm 136 GetRelativeAltitude

```
function GetRelativeAltitude(alt::R, is_alt_geo_hae::Bool)
    own_valid_alt_pres::Bool = !isnan(own.toa_vert)
    own_valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
    if !is_alt_geo_hae && own_valid_alt_pres
        z_rel = alt - own.geo_states_pres_alt.lla_rad_ft[3]
    elseif is_alt_geo_hae && own_valid_alt_hae
        z_rel = alt - own.geo_states_hae_alt.lla_rad_ft[3]
    else
        z_rel = 0.0
    end
    return z_rel::R
end
```

Referenced In: ExtrapolateAGTTrack(p. 447), AddADSBTrackToReport(p. 135), AddPointObstacleTrackToReport(p. 157), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

ADDHORBIAS (Algorithm 137) is used to inflate horizontal tracker uncertainty due to uncompensated latency and NACp/NACv of both the intruder and ownship. The uncertainty is inflated to account for uncompensated latency through a call to INFLATEPASSIVEHORSIGMAUL. Based on the intruder's NACp and NACv, the uncertainty is then inflated to account for passive bias. The added uncertainty is determined through calls to CONVERTNACPTOSIGMAHEPU and CONVERTNACVTOSIGMAHVA. Finally, the uncertainty is inflated to add own horizontal uncertainty.

Algorithm 137 AddHorBias

```
function AddHorBias(trk::V2VTrackFile(p. 488))
    Q_ul::R = params().surveillance.horizontal_v2v.covariance_inflation.uncompensated_latency.Q_ul
    init_NACp_bias::UInt32 = params().surveillance.horizontal_v2v.covariance_inflation.init_NACp_bias
    init_NACv_bias::UInt32 = params().surveillance.horizontal_v2v.covariance_inflation.init_NACv_bias
    nacp_factor::R = params().surveillance.horizontal_v2v.covariance_inflation.nacp_factor
    nacv_factor::R = params().surveillance.horizontal_v2v.covariance_inflation.nacv_factor
    trk.Sigma_hor = InflatePassiveHorSigmaUL(p. 40)(trk.mu_hor, trk.Sigma_hor, Q_ul)
    if (trk.nacp == 0)
        NACp = init_NACp_bias
        NACv = init_NACv_bias
    else
        NACp = trk.nacp
        NACv = trk.nacv
    end
    NPInflate::R = (ConvertNACpToSigmaHEPU(p. 95)(NACp) * nacp_factor)^2
    NVInflate::R = (ConvertNACvToSigmaHVA(p. 38)(NACv) * nacv_factor)^2
    trk.Sigma_hor += diagm(0 => [NPInflate, NVInflate, NPInflate, NVInflate])
    trk.Sigma_hor += own.wgs84_Sigma_hor .* eye(p. 582)(4)
    return
end
```

Referenced In: AddV2VTrackToReport(p. 150)

ABSOLUTEGEODETICTOOWNSHIPRELATIVE (Algorithm 138) is used to convert a tracker specified in absolute geodetic coordinates into relative coordinates centered about the ownship's geodetic position.

The first step is to align the altitude types used to represent the absolute geodetic LLA and ECEF positions of the intruder's horizontal tracker and the ownship's WGS84 location through a call to ALIGNALTITUDETYPESUSEDINGEODETICPOSITIONS. These aligned latitude-longitude-altitude (LLA) and earth-centered earth-fixed (ECEF) positions are then fed into REDEFINEESTIMATEINROTATEDFRAME along with the mean and covariance of the intruder's horizontal tracker, which will rotate the tracker states and covariance to properly align with the poles of the earth when centered about the ownship's geodetic position instead of the intruder's geodetic position. After rotating the tracker to be aligned with the ownship WGS84 states, the relative positions and velocities calculated. The relative position is first calculated in ECEF coordinates and then rotated in ENU by a call to ROTATEECEFTOENU. The east/west and north/south relative positions are then extracted and used to update *mu_hor*. The relative velocities are calculated by subtracting the ownship's velocities from the intruder's velocities.

This algorithm takes as input *trk*. This algorithm returns *mu_hor* and *Sigma_hor*.

Algorithm 138 AbsoluteGeodeticToOwnshipRelative

```
function AbsoluteGeodeticToOwnshipRelative(trk::TrackFile(p. 483))
    I_xy = [1, 3]
    (lla, ecef, own_lla, own_ecef) = AlignAltitudeTypesUsedInGeodeticPositions(p. 109)(trk.lla_at_hor_toa, trk.
        ecef_at_hor_toa, trk.alt_src_hae, trk.valid_vert)
    (mu_hor, Sigma_hor) = RedefineEstimateInRotatedFrame(p. 42)(trk.mu_hor, trk.Sigma_hor, lla, ecef, own_lla,
        own_ecef)
    delta_ecef = ecef - own_ecef
    delta_enu = RotateECEFToENU(p. 23)(delta_ecef, own_lla)
    mu_hor[I_xy] = delta_enu[1:2]
    mu_hor[2] = mu_hor[2] - own.geo_states_hae_alt.vel_ew_mps
    mu_hor[4] = mu_hor[4] - own.geo_states_hae_alt.vel_ns_mps
    return (mu_hor::Vector{R}, Sigma_hor::Matrix{R})
end
```

Referenced In: ExtrapolateADSBTrack(p. 443), ExtrapolateAGTTrack(p. 447), AddADSBTrackToReport(p. 135), AddAGTTrackToReport(p. 155), ExtrapolateV2VTrack(p. 444), AddV2VTrackToReport(p. 150)

The purpose of ADDALTBIASANDSAMPLE (Algorithm 139) is to add uncertainty to the intruder vertical state distribution to account for possible altimetry errors. This process occurs only if the intruder vertical track is valid. If the track is invalid, the algorithm returns a set of samples and weights with all values set to *NaN*, indicating a NAR intruder. If the vertical track is valid, the reported covariance matrix is inflated with the estimated uncertainties of the intruder and ownship, based on their respective tracker states and altimetry error models (AEM), if applicable.

For large intruders reporting pressure altitudes, the magnitude of this uncertainty inflation is based on the Standard Altimetry Error Model specified in [7] and further adjusted in accordance with [8]. For smaller aircraft reporting pressure altitudes, the magnitude of altimetry uncertainty is still an active area of research. An AEM derived from [13] is currently utilized, based on the expectation that both small intruders and ownship are to be equipped with pressure altimeters supplying comparable or better performances, as elaborated in [3].

For large and small intruders as well as ownship reporting the geometric height above ellipsoid (HAE), the inflation magnitude is equivalent to the self-reported uncertainties. Table 29 summarizes the means through which the magnitudes of inflation are derived in this algorithm. Note that for AGTTRACKFILES, only the ownship component will be applied during the inflation.

Table 29. Deriving Magnitude of Inflation in ADDALTBIASANDSAMPLE

Surveillance	Aircraft	Pressure Altitude	HAE
Ownship WGS84	Small	AEM [13]	*own.wgs84_sigma_vepu* derived in RECEIVEWGS84OBSERVATION
ADS-B or V2V	Large	AEM [7, 8]	Derived here from *trk.gva* by calling CONVERTGVATOSIGMAVEPU
V2V	Small	AEM [13]	*trk.sigma_vepu* derived in RECEIVESTATEVECTORV2VREPORT
AGT	All	No inflation	No inflation

The altitude bias (*alt_bias_sigma*) is then chosen. If the input track is an AGT track, the bias is set to zero. Otherwise, if the track is geodedic, the alt bias is set to *aem_sigma*. Otherwise, the altitude bias is set based on the GETPRESAEMSIGMA algorithm. *Sigma* is then copied into

inflated_Sigma and the altitude bias is added to it.

Next, vertical uncertainty from ownship is evaluated. This is only done if ownship has valid altitude for the type of altitude the intruder is reporting (based on *is_alt_geo_hae*). First, the ownship altitude bias is evaluated. Then the own vertical track's uncertainty is added based on the altitude type (*own.wgs84_Sigma_vert* if using geodetic or *own.Sigma_vert* if using barometric). Then the ownship altitude bias is added to *inflated_Sigma*.

This inflated distribution is then converted into five samples through a call to SIGMAPOINTSAMPLE.

This algorithm takes as input *mu*, *Sigma*, *valid*, *is_geodetic*, *is_large*, and *T*. This algorithm returns values from SIGMAPOINTSAMPLE or returns NaN values for invalid tracks.

Algorithm 139 AddAltBiasAndSample

```
function AddAltBiasAndSample(trk::Union{ADSBTrackFile(p. 485),V2VTrackFile(p. 488),AGTTrackFile(p. 486)}, mu::Vector{R},
    Sigma::Matrix{R}, valid::Bool, is_alt_geo_hae::Bool, is_large::Bool, T::R)
    if valid
        if is_alt_geo_hae
            aem_sigma_own::Union{Vector{R}, R} = own.wgs84_sigma_vepu
            if isa(trk, ADSBTrackFile(p. 485))
                aem_sigma::Union{Vector{R}, R} = ConvertGVAToSigmaVEPU(p. 124)(trk)
            elseif isa(trk, V2VTrackFile(p. 488))
                aem_sigma = trk.sigma_vepu
            end
        else
            aem_sigma_own = params().surveillance.vertical.pres_small_ownship.aem_sigma
            if is_large
                aem_sigma = params().surveillance.vertical.pres_large_intruder.aem_sigma
            else
                aem_sigma = params().surveillance.vertical.pres_small_intruder.aem_sigma
            end
        end
        alt_bias_sigma = 0
        alt_bias_sigma_own = 0
        if isa(trk, AGTTrackFile(p. 486))
            alt_bias_sigma = 0.0
        elseif is_alt_geo_hae
            alt_bias_sigma = aem_sigma
        else
            alt_bias_sigma = GetPresAEMSigma(p. 34)(mu[1], aem_sigma)
        end
        inflated_Sigma = copy(Sigma)
        inflated_Sigma[1,1] += alt_bias_sigma^2
        own_valid_alt_pres::Bool = !isnan(own.toa_vert)
        own_valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
        if (!is_alt_geo_hae && own_valid_alt_pres) || (is_alt_geo_hae && own_valid_alt_hae)
            if is_alt_geo_hae
                alt_bias_sigma_own = aem_sigma_own
            else
                alt_own::R = PresAltAtToa(p. 84)(T)
                alt_bias_sigma_own = GetPresAEMSigma(p. 34)(alt_own, aem_sigma_own)
            end
            if is_alt_geo_hae
                inflated_Sigma += own.wgs84_Sigma_vert
            else
                inflated_Sigma += own.Sigma_vert
            end
            inflated_Sigma[1,1] += alt_bias_sigma_own^2
        end
        return SigmaPointSample(p. 144)(mu,inflated_Sigma)
    else
        return (fill(NaN,2,5), fill(NaN,5))
    end
end
```

Referenced In: AddADSBTrackToReport(p. 135), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

SIGMAPOINTSAMPLE (Algorithm 140) is a method of generating a weighted set of states that attempts to maintain the first and second moments (mean and covariance) of the track estimate. The variable *kappa* is utilized to ensure the weights maintain positive values. The value of kappa is set equal to the length of x divided by 2. The first sigma point is set equal to the mean, while the remaining sigma points use scaled versions of the square root of the covariance matrix that come from a call to UCHOL (Algorithm 466).

This algorithm takes as input the mean *x* and covariance *Sigma*. This algorithm returns the sigma point samples *S* and a weights *w*.

Algorithm 140 SigmaPointSample

```
function SigmaPointSample(x::Vector{R}, Sigma::Matrix{R})
    m = length(x)
    kappa = m/2.0
    S = zeros(m,2*m+1)
    w = zeros(2*m+1)
    S_u = uchol(p. 582)(Sigma)
    S[:,1] = x
    w[1] = kappa/(m+kappa)
    for i in 2:m+1
        S[:,i] = x+sqrt(m+kappa)*S_u[:,i-1]
        w[i] = 1/(2*(m+kappa))
        S[:,i+m] = x-sqrt(m+kappa)*S_u[:,i-1]
        w[i+m] = 1/(2*(m+kappa))
    end
    return (S::Matrix{R}, w::Vector{R})
end
```

Referenced In: ConvertCartesianToPolar2D(p. 448), AddADSBTrackToReport(p. 135), AddAltBiasAndSample(p. 143), AddORNCTTrackToReport(p. 152), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

SETINTRUDERBELIEFSTATES takes as input *intruder*, *vsamples_zdz*, *vweights*, *hsamples_xydxdy*, and *hweights*. SETINTRUDERBELIEFSTATES updates *intruder.belief_vert* and *intruder.belief_horiz*.

Algorithm 141 SetIntruderBeliefStates

```
function SetIntruderBeliefStates(intruder::TRMIntruderInput(p. 499), vsamples_zdz::Matrix{R}, vweights::Vector{R},
     hsamples_xydxdy::Matrix{R}, hweights::Vector{R})
    resize!(intruder.belief_vert, length(vweights))
    for j in 1:length(vweights)
        intruder.belief_vert[j] = IntruderVerticalBelief(p. 500)()
        intruder.belief_vert[j].z = vsamples_zdz[1,j]
        intruder.belief_vert[j].dz = vsamples_zdz[2,j]
        intruder.belief_vert[j].weight = vweights[j]
    end
    resize!(intruder.belief_horiz,length(hweights))
    for j in 1:length(hweights)
        intruder.belief_horiz[j] = IntruderHorizontalBelief(p. 500)()
        intruder.belief_horiz[j].x_rel = hsamples_xydxdy[1,j]
        intruder.belief_horiz[j].y_rel = hsamples_xydxdy[2,j]
        intruder.belief_horiz[j].dx_rel = hsamples_xydxdy[3,j]
        intruder.belief_horiz[j].dy_rel = hsamples_xydxdy[4,j]
        intruder.belief_horiz[j].weight = hweights[j]
    end
end
```

Referenced In: AddADSBTrackToReport(p. 135), AddORNCTTrackToReport(p. 152)

SETDISPLAYDATAPASSIVE (Algorithm 142) populates the STMREPORT with fields needed to properly display each intruder. First, the algorithm populates a set of display parameters that are common among all types of intruders, including but not limited to the time of report generation

(*toa*), the source of the intruder state information (*source*), the sources available for the target (*available_sources*), the internal ID of the target (*id*), and the intruder ID directory containing all valid identifiers assigned to various track files within the target (*id_directory*). Then, the intruder bearing and component velocity display parameters are set based on the condition of the ownship heading state, indicated by *own.heading_state*. If the condition of the ownship heading estimate is nominal (i.e. not degraded), the horizontal estimates are used to set all relevant horizontal display parameters. If the ownship heading estimate is degraded, the x-component velocity is set to zero, the azimuth of the intruder is computed from the horizontal estimate and used to compute the y-component velocity, and the bearing validity flag (*bearing_valid*) is set to false. The algorithm then sets the the appropriate vertical display parameters for the intruder based on the condition of the vertical track and the ownship altitude source. If both the intruder and ownship vertical tracks are valid, the intruder vertical display parameters are computed and set relative to ownship; otherwise, they are set to *NaN* and the *alt_reporting* flag is set to false. Finally, the display arrow is updated based on the validity of the intruder and ownship vertical tracks, and the display structure is pushed to the TRM Report.

This algorithm takes as input *report*, *intruder*, *mu_vert*, *mu_hor*, *valid_vert*, *T*, *display_arrow_current*, and *adsb_validated*. This algorithm updates *intruder.stm_display* and appends that updated information to *report.display*.

Algorithm 142 SetDisplayDataPassive

```
function SetDisplayDataPassive(report::StmReport(p. 480), intruder::TRMIntruderInput(p. 499), mu_vert::Vector{R},
    mu_hor::Vector{R}, valid_vert::Bool, T::R, display_arrow_current::Z, adsb_validated::Bool)
    display::StmDisplayStruct(p. 481) = StmDisplayStruct(p. 481)()
    display.toa = T
    display.source = intruder.source
    display.available_sources = SetAvailableSources(p. 147)(intruder.id)
    display.id = intruder.id
    display.id_directory = intruder.id_directory
    display.validated_adsb = adsb_validated
    display.r_ground_ft = hypot(mu_hor[1], mu_hor[3])
    own_heading::R = HeadingAtToa(p. 90)(T)
    if (own.heading_state == OWN_HEADING_NOMINAL)
        display.bearing_rel_rad = WrapToPi(p. 583)(atan(mu_hor[1], mu_hor[3]) - own_heading)
        display.bearing_valid = true
        display.dx_rel_fps = mu_hor[2]
        display.dy_rel_fps = mu_hor[4]
    else
        display.bearing_rel_rad = 0.0
        display.bearing_valid = false
        azimuth_rad = atan(mu_hor[1], mu_hor[3])
        I_xydxdy::Vector{Z} = [1, 3, 2, 4]
        (mu_xgrgr, _) = RotateHorizontalFrame(p. 63)(mu_hor[I_xydxdy], eye(p. 582)(4), azimuth_rad)
        display.dx_rel_fps = 0.0
        display.dy_rel_fps = mu_xgrgr[4]
    end
    if valid_vert && (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        display.z_rel_ft = mu_vert[1] - report.trm_input.own.belief_vert[1].z
        display.dz_rel_fps = mu_vert[2] - report.trm_input.own.belief_vert[1].dz
        display.alt_reporting = true
    else
        display.z_rel_ft = NaN
        display.dz_rel_fps = NaN
        display.alt_reporting = false
    end
    if valid_vert && (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        display.arrow = display_arrow_current
    else
        display.arrow = DISPLAY_ARROW_LEVEL
    end
    intruder.stm_display = display
    push!(report.display, display)
end
```

Referenced In: AddADSBTrackToReport(p. 135), AddPointObstacleTrackToReport(p. 157), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

SETAVAILABLESOURCES (Algorithm 143) produces a bit vector *available_sources*, which indicates the availability of each possible surveillance input source for an existing target within the target database. The algorithm takes as input the *id* of the target for which the bit vector will be produced, and checks whether a track exists for each possible surveillance sources. If a track exists, the corresponding bit for that surveillance source is set within the bit vector.

This algorithm takes as input *id*. This algorithm returns *available_sources*.

Algorithm 143 SetAvailableSources

```
function SetAvailableSources(id::UInt32)
    available_sources::UInt16 = AVAILABLE_SURVEILLANCE_NONE
    if (TrackExists(p. 19)(target_db[id].adsb_track)) && (own.wgs84_state == OWN_WGS84_VALID)
        if target_db[id].adsb_track.uat
            available_sources += AVAILABLE_SURVEILLANCE_978_UAT
        else
            available_sources += AVAILABLE_SURVEILLANCE_1090ES_ADSB
        end
    end
    if (TrackExists(p. 19)(target_db[id].ornct_track))
        available_sources += AVAILABLE_SURVEILLANCE_ORNCT
    end
    if (TrackExists(p. 19)(target_db[id].agt_tracks)) && (own.wgs84_state == OWN_WGS84_VALID)
        available_sources += AVAILABLE_SURVEILLANCE_AGT
    end
    if (TrackExists(p. 19)(target_db[id].v2v_track)) && (own.wgs84_state == OWN_WGS84_VALID)
        available_sources += AVAILABLE_SURVEILLANCE_V2V
    end
    if (TrackExists(p. 19)(target_db[id].point_obstacle_track)) && (own.wgs84_state == OWN_WGS84_VALID)
        available_sources += AVAILABLE_SURVEILLANCE_POINT_OBSTACLE
    end
    return available_sources::UInt16
end
```

Referenced In: SetDisplayDataORNCT(p. 153), SetDisplayDataPassive(p. 146)

VERTICALRATEARROWUPDATE (Algorithm 144) updates the vertical rate arrow shown on the pilot's display. It first reads in the following parameters:

- *arrow_N* - the number of past track estimates used in the count, set as *N*.
- *arrow_M* - the number of past track estimates out of N with the same vertical direction needed to set the arrow on the display, set as *M*.
- *level_arrow_threshold* - the vertical speed over which the intruder is no longer considered level.

If the number of past elements exceeds the total number needed for the comparison (*N*), the oldest stored value is removed from the vertical arrow history data structure, *trk.vert_arrow_history*. The current track's vertical speed (*mu_dz*) is then generated based on the input variable *is_relative_alt*, along with the validity of ownship pressure altitude and HAE altitude vertical rates. The history is then updated with the latest vertical direction by comparing the current track's vertical speed to *level_arrow_threshold*. Next, the number of climb, descend, and level arrows are tallied from the history. If any of the tallies are greater than or equal to *M*, the current vertical arrow (*trk.display_arrow_current*) is set to that direction. If none of the tallies satisfy this condition, the vertical rate arrow is not updated.

This algorithm takes as input *trk* and *is_relative_alt*. This algorithm updates *trk.vert_arrow_history* and *trk.display_arrow_current*.

Algorithm 144 VerticalRateArrowUpdate

```
function VerticalRateArrowUpdate(trk::TrackFile(p. 483), is_relative_alt::Bool)
    level_arrow_threshold::R = params().display.level_arrow_threshold
    M::Z = params().display.arrow_M
    N::Z = params().display.arrow_N
    if (length(trk.vert_arrow_history) >= N)
        pop!(trk.vert_arrow_history)
    end
    mu_dz::R = trk.mu_vert[2]
    if is_relative_alt
        own_valid_alt_pres::Bool = !isnan(own.toa_vert)
        own_valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
        if own_valid_alt_pres
            mu_dz = mu_dz + own.mu_vert[2]
        elseif own_valid_alt_hae
            mu_dz = mu_dz + own.wgs84_mu_vert[2]
        end
    end
    if (mu_dz > level_arrow_threshold)
        pushfirst!(trk.vert_arrow_history, DISPLAY_ARROW_CLIMB)
    elseif (mu_dz < -level_arrow_threshold)
        pushfirst!(trk.vert_arrow_history, DISPLAY_ARROW_DESCEND)
    else
        pushfirst!(trk.vert_arrow_history, DISPLAY_ARROW_LEVEL)
    end
    numClimb::Z = 0
    numDes::Z = 0
    numLevel::Z = 0
    for i in 1:length(trk.vert_arrow_history)
        if (trk.vert_arrow_history[i] == DISPLAY_ARROW_CLIMB)
            numClimb += 1
        elseif (trk.vert_arrow_history[i] == DISPLAY_ARROW_DESCEND)
            numDes += 1
        else
            numLevel += 1
        end
    end
    if (numClimb >= M)
        trk.display_arrow_current = DISPLAY_ARROW_CLIMB
    elseif (numDes >= M)
        trk.display_arrow_current = DISPLAY_ARROW_DESCEND
    elseif (numLevel >= M)
        trk.display_arrow_current = DISPLAY_ARROW_LEVEL
    end
end
```

Referenced In: UpdateORNCTTrack(p. 65), UpdateAGTTrack(p. 76), AdvanceVerticalTrack(p. 53)

2.8.4.2 V2V Track File

V2VTRACKFILEs are added to an STMREPORT through ADDV2VTRACKTOREPORT (Algorithm 145). First, the corresponding Track File is retrieved from memory and a new TRMINTRUDERINPUT (i.e., the data structure in the STMREPORT which represents an intruding aircraft) is initialized. Several of the data elements in the TRMINTRUDERINPUT are then populated with the associated data in the V2VTRACKFILE. This includes the track ID (*id*), the source of the intruder state information (*source*), and the validity of the track data (*valid*) for RA processing by the TRM. The addresses assigned by an external surveillance system, the V2V-UID and potentially a Mode S address in the case of an V2VTRACKFILE, are set through a call to SETINTRUDERIDDIRECTORY (Algorithm 134). Intruder classification is set with a call to SETCLASSIFICATION (Algorithm 133).

SETCOORDINATION (Algorithm 104) is called to set equipage information as well as several coordination related fields. Targets are marked as invalid for RA processing when ownship is in standby mode, as indicated by *own.opmode*. Degraded surveillance flags are set in a call to SETINTRUDERDEGRADEDFLAGS.

If necessary, the two tracks contained in the V2VTRACKFILE are extrapolated to the time of the report by calling the corresponding prediction algorithms. If *trm_altmode* is not equal to *TRM_ALTMODE_NONE*, the altitude of the intruder undergoes a conversion to ensure it is defined with respect to the same vertical datum as ownship. This is done by first generating the relative altitude by a call to GETRELATIVEALTITUDE, and then converting back into an absolute altitude by adding the reported altitude of ownship to the relative altitude. The horizontal extrapolation is not performed if the time difference between the last track update and the TOA of the report is less than a defined minimum threshold (*min_extrap_toa_step*). After predicting the absolute horizontal tracker, the mean position is re-centered about the new geodetic LLA position by a call to RECENTERHORIZONTALTRACKLOCATION. The reported horizontal uncertainty is then inflated using ADDHORBIAS. Finally, the absolute geodetic V2V track is converted to ownship relative Cartesian states with a call to ABSOLUTEGEODETICTOOWNSHIPRELATIVE.

The representation of the target aircraft's vertical state is provided as a set of weighted samples given by *vsamples* and *vweights*. These samples are produced using the mean (*mu_vert*) and covariance (*Sigma_vert*) of the vertical track through a call to ADDALTBIASANDSAMPLE. The representation of the target aircraft's relative horizontal state is also provided as a set of weighted samples given by *hsamples* and *hweights*. These samples are produced using the mean (*mu_hor*) and covariance (*Sigma_hor*) of the horizontal track that has been converted from metric to English units. In the last steps the STMDISPLAYSTRUCT vector gets filled by SETDISPLAYDATAPASSIVE and the TRMINTRUDERINPUT is appended to STMREPORT.

This algorithm takes as input *report*, *id*, *T*, and *passive_only*. This algorithm updates *report.trm_input.intruder*.

Algorithm 145 AddV2VTrackToReport

```
function AddV2VTrackToReport(report::StmReport(p. 480), id::UInt32, T::R, passive_only::Bool)
    min_extrap_toa_step::R = params().surveillance.min_extrap_toa_step
    m2ft::R = geoutils.meters_to_feet
    I_lat::Vector{Z} = [1, 2, 3, 4]
    tgt = target_db[id]
    trk = tgt.v2v_track
    intruder = TRMIntruderInput(p. 499)()
    intruder.id = id
    intruder.source = SOURCE_V2V
    SetClassification(p. 136)(intruder)
    SetIntruderIDDirectory(p. 137)(tgt, intruder.id_directory)
    SetCoordination(p. 104)(intruder)
    SetIntruderDegradedFlags(p. 138)(intruder, tgt, trk, passive_only)
    valid_vert = (0 == (intruder.degraded_surveillance & DEGRADED_SURVEILLANCE_NAR))
    h_own = report.trm_input.own.belief_vert[1].z
    intruder.valid = true
    if (own.opmode == OPMODE_STANDBY)
        intruder.valid = false
    end
    dt_vert = T - trk.toa_vert
    dt_hor = T - trk.toa_hor
    is_large::Bool = (trk.classification != CLASSIFICATION_SMALL_UNMANNED)
    (mu_vert, Sigma_vert) = PredictVerticalTracker(p. 54)(trk.mu_vert, trk.Sigma_vert, dt_vert, trk.alt_src_hae,
        false, is_large)
    if (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        z_rel = GetRelativeAltitude(p. 139)(mu_vert[1], trk.alt_src_hae)
        mu_vert[1] = z_rel + h_own
    end
    trk_temp = deepcopy(trk)
    if (dt_hor >= min_extrap_toa_step)
        (trk_temp.mu_hor, trk_temp.Sigma_hor) = PredictPassiveTracker(p. 46)(trk_temp.mu_hor, trk_temp.Sigma_hor,
            dt_hor, true)
        ReCenterHorizontalTrackLocation(p. 42)(trk_temp, T)
    end
    AddHorBias(p. 140)(trk_temp)
    (mu_hor, Sigma_hor) = AbsoluteGeodeticToOwnshipRelative(p. 141)(trk_temp)
    (vsamples,vweights) = AddAltBiasAndSample(p. 143)(trk, mu_vert, Sigma_vert, valid_vert, trk.alt_src_hae,
        is_large, T)
    resize!(intruder.belief_vert,length(vweights))
    for j in 1:length(vweights)
        intruder.belief_vert[j] = IntruderVerticalBelief(p. 500)()
        intruder.belief_vert[j].z = vsamples[1,j]
        intruder.belief_vert[j].dz = vsamples[2,j]
        intruder.belief_vert[j].weight = vweights[j]
    end
    mu_hor = mu_hor[I_lat] * m2ft
    Sigma_hor = Sigma_hor[I_lat,I_lat] * (m2ft^2)
    (hsamples,hweights) = SigmaPointSample(p. 144)(mu_hor, Sigma_hor)
    resize!(intruder.belief_horiz, length(hweights))
    for j in 1:length(hweights)
        intruder.belief_horiz[j] = IntruderHorizontalBelief(p. 500)()
        intruder.belief_horiz[j].x_rel = hsamples[1,j]
        intruder.belief_horiz[j].dx_rel = hsamples[2,j]
        intruder.belief_horiz[j].y_rel = hsamples[3,j]
        intruder.belief_horiz[j].dy_rel = hsamples[4,j]
        intruder.belief_horiz[j].weight = hweights[j]
    end
    SetDisplayDataPassive(p. 146)(report, intruder, mu_vert, mu_hor, valid_vert, T, trk.display_arrow_current, !
        passive_only)
    intruder.adsb_cccb = deepcopy(tgt.adsb_osm.adsb_cccb)
    intruder.v2v_cccb = deepcopy(tgt.v2v_osm.v2v_cccb)
    push!(report.trm_input.intruder,intruder)
end
```

Referenced In: AddTracksToReport(p. 119)

2.8.4.3 ORNC Track File

ADDORNCTTRACKTOREPORT (Algorithm 146) defines the process through which the data in a ORNCTTRACKFILE is added to an STMREPORT. First, the corresponding Track File is retrieved from memory and a new TRMINTRUDERINPUT (i.e., the data structure in the STMREPORT which represents an intruding aircraft) is initialized. Several of the data elements in the TRMINTRUDERINPUT are then populated with the associated data in the ORNCTTRACKFILE. This includes the track ID (*id*), the source of the intruder state information (*source*), and the validity of the track data (*valid*) for RA processing by the TRM. The addresses assigned by an external surveillance system, the ORNCT-ID in the case of an ORNCTTRACKFILE, are set through a call to SETINTRUDERIDDIRECTORY (Algorithm 134). Intruder classification is set with a call to SETCLASSIFICATION (Algorithm 133). Targets are marked as invalid for RA processing when ownship is in standby mode, as indicated by *own.opmode*. Degraded surveillance flags are set in a call to SETINTRUDERDEGRADEDFLAGS.

The tracks contained in the ORNCTTRACKFILE are then extrapolated to the time of the report with a call to PREDICTORNCTTRACK (Algorithm 66). However, extrapolation is not performed if the time difference between the last track update and the TOA of the report is less than a defined minimum threshold (*min_extrap_toa_step*).

The representation of the target aircraft's vertical state is provided as a set of weighted samples given by *vsamples* and *vweights*. These samples are produced using the mean (*mu_vert*) and covariance (*Sigma_vert*) of the vertical track through a call to SIGMAPOINTSAMPLE (Algorithm 140). The representation of the target aircraft's relative horizontal state is also provided as a set of weighted samples given by *hsamples* and *hweights*. These samples are produced using the mean (*mu_cart*) and covariance (*Sigma_cart*) of the horizontal track.

The vertical and horizontal samples are provided to SETINTRUDERBELIEFSTATES (Algorithm 141) for assignment within the TRMINTRUDERINPUT. In the last steps, the STMDISPLAYSTRUCT vector gets filled by SETDISPLAYDATAORNCT (Algorithm 147) and the TRMINTRUDERINPUT is appended to STMREPORT.

This algorithm takes as input *report*, *id*, and *T*. This algorithm updates *report.trm_input.intruder*.

Algorithm 146 AddORNCTTrackToReport

```
function AddORNCTTrackToReport(report::StmReport(p. 480), id::UInt32, T::R)
    min_extrap_toa_step::R = params().surveillance.min_extrap_toa_step
    I_xydxdy::Vector{Z} = [1, 3, 2, 4]
    tgt = target_db[id]
    trk = tgt.ornct_track
    intruder = TRMIntruderInput(p. 499)()
    intruder.id = id
    intruder.source = SOURCE_ORNCT
    SetClassification(p. 136)(intruder)
    SetIntruderIDDirectory(p. 137)(tgt, intruder.id_directory)
    SetCoordination(p. 104)(intruder)
    SetIntruderDegradedFlags(p. 138)(intruder, tgt, trk, false)
    intruder.valid = true
    if (own.opmode == OPMODE_STANDBY)
        intruder.valid = false
    end
    (mu_vert::Vector{R},  Sigma_vert::Matrix{R})  = (trk.mu_vert, trk.Sigma_vert)
    (mu_rng::Vector{R}, Sigma_rng::Matrix{R}) = (trk.mu_rng,  trk.Sigma_rng)
    (mu_cart::Vector{R}, Sigma_cart::Matrix{R} ) = (trk.mu_cart, trk.Sigma_cart)
    dt::R =  T - trk.toa
    if (dt >= min_extrap_toa_step)
        (mu_vert, Sigma_vert, mu_rng, Sigma_rng, mu_cart, Sigma_cart) = PredictORNCTTrack(p. 64)(trk, dt)
    end
    if (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        zdz_rel = mu_vert
        hdh_own = [report.trm_input.own.belief_vert[1].z, report.trm_input.own.belief_vert[1].dz]
        mu_vert = zdz_rel + hdh_own
    end
    (vsamples,vweights) = SigmaPointSample(p. 144)(mu_vert, Sigma_vert)
    (hsamples,hweights) = SigmaPointSample(p. 144)(mu_cart[I_xydxdy], Sigma_cart[I_xydxdy, I_xydxdy])
    SetIntruderBeliefStates(p. 144)(intruder, vsamples, vweights, hsamples, hweights)
    SetDisplayDataORNCT(p. 153)(report, intruder, mu_vert, mu_cart, mu_rng, trk.ornct_id, T, trk.
        display_arrow_current)
    intruder.adsb_cccb = deepcopy(tgt.adsb_osm.adsb_cccb)
    intruder.v2v_cccb = deepcopy(tgt.v2v_osm.v2v_cccb)
    push!(report.trm_input.intruder,intruder)
end
```

Referenced In: AddTracksToReport(p. 119)

SETDISPLAYDATAORNCT (Algorithm 147) populates the STMREPORT with fields needed to properly display each intruder. These fields include the time the report was generated (*toa*), source of the intruder state information (*source*), the sources available in the TARGET (*available_sources*), the internal ID of the TARGET (*id*), an INTRUDERIDDIRECTORY containing all valid identifiers assigned to various TRACKFILEs inside the TARGET (*id_directory*), whether or not the ADS-B has been validated (*validated_adsb*), the ground range to the intruder (*r_ground_ft*), the bearing relative to ownship (*bearing_rel_rad*), the validity of the bearing data (*bearing_valid*), the east/west and north/south velocity relative to own (*dx_rel_fps* and dy_rel_fps), the relative altitude of the intruder (*z_rel_ft*), the vertical state validity (*alt_reporting*), the vertical rate relative to own (*dz_rel_fps*), and the direction of the intruder vertical velocity (*arrow*).

This algorithm takes as input *report*, *intruder*, *mu_vert*, *mu_cart*, *mu_rng*, *ornct_id*, *T*, and *display_arrow_current*. This algorithm updates *intruder.stm_display* and appends that updated information to *report.display*.

Algorithm 147 SetDisplayDataORNCT

```
function SetDisplayDataORNCT(report::StmReport(p. 480), intruder::TRMIntruderInput(p. 499), mu_vert::Vector{R},
     mu_cart::Vector{R}, mu_rng::Vector{R}, ornct_id::UInt32, T::R, display_arrow_current::Z)
    display::StmDisplayStruct(p. 481) = StmDisplayStruct(p. 481)()
    display.toa = T
    display.source = intruder.source
    display.available_sources = SetAvailableSources(p. 147)(intruder.id)
    display.id = intruder.id
    display.id_directory = intruder.id_directory
    display.validated_adsb = false
    display.r_ground_ft = mu_rng[1]
    own_heading::R = HeadingAtToa(p. 90)(T)
    if (own.heading_state == OWN_HEADING_NOMINAL)
        display.bearing_rel_rad = WrapToPi(p. 583)(atan(mu_cart[1], mu_cart[3]) - own_heading)
        display.bearing_valid = true
        display.dx_rel_fps = mu_cart[2]
        display.dy_rel_fps = mu_cart[4]
    else
        display.bearing_rel_rad = 0.0
        display.bearing_valid = false
        display.dx_rel_fps = 0.0
        display.dy_rel_fps = mu_rng[2]
    end
    if (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        display.z_rel_ft = mu_vert[1] - report.trm_input.own.belief_vert[1].z
        display.dz_rel_fps = mu_vert[2] - report.trm_input.own.belief_vert[1].dz
    else
        display.z_rel_ft = mu_vert[1]
        display.dz_rel_fps = mu_vert[2]
    end
    display.alt_reporting = true
    if (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        display.arrow = display_arrow_current
    else
        display.arrow = DISPLAY_ARROW_LEVEL
    end
    intruder.stm_display = display
    push!(report.display, display)
end
```

Referenced In: AddORNCTTrackToReport(p. 152)

2.8.4.4 AGT Track File

ADDAGTTRACKTOREPORT (Algorithm 148) defines the process through which the data in a AGTTRACKFILE is added to an STMREPORT. First, a new TRMINTRUDERINPUT (i.e., the data structure in the STMREPORT which represents an intruding aircraft) is initialized. Several of the data elements in the TRMINTRUDERINPUT are then populated with the associated data in the AGTTRACKFILE. This includes the track ID (*id*), the source of the intruder state information (*source*), and the validity of the track data (*valid*) for RA processing by the TRM. The addresses assigned by an external surveillance system, the AGT-ID and potentially a Mode S address and a V2V-UID in the case of an AGTTRACKFILE, are set through a call to SETINTRUDERIDDIRECTORY (Algorithm 134). Intruder classification is set with a call to SETCLASSIFICATION (Algorithm 133). SETCOORDINATION (Algorithm 104) is called to set equipage information as well as several coordination related fields. Targets are marked as invalid for RA processing when ownship is in standby mode, as indicated by *own.opmode*. Degraded surveillance flags are set in a call to SETINTRUDERDEGRADEDFLAGS.

If necessary, the tracks contained in the AGTTRACKFILE are extrapolated to the time of the report by calling PREDICTPASSIVETRACKER and PREDICTVERTICALTRACKER. If *trm_altmode* is not equal to *TRM_ALTMODE_NONE*, the altitude of the intruder undergoes a conversion to ensure it is defined with respect to the same vertical datum as ownship. This is done by first generating the relative altitude by a call to GETRELATIVEALTITUDE, and then converting back into an absolute altitude by adding the reported altitude of ownship to the relative altitude. The horizontal extrapolation is not performed if the time difference between the last track update and the TOA of the report is less than a defined minimum threshold (*min_extrap_toa_step*). After predicting the absolute horizontal tracker, the mean position is re-centered about the new geodetic LLA position by a call to RECENTERHORIZONTALTRACKLOCATION. Finally, the absolute geodetic AGT track is converted to ownship relative Cartesian states with a call to ABSOLUTEGEODETICTOOWNSHIPRELATIVE.

The representation of the target aircraft's vertical state is provided as a set of weighted samples given by *vsamples* and *vweights*. These samples are produced using the mean (*mu_vert*) and covariance (*Sigma_vert*) of the vertical track through a call to ADDALTBIASANDSAMPLE. The representation of the target aircraft's relative horizontal state is also provided as a set of weighted samples given by *hsamples* and *hweights*. These samples are produced using the mean (*mu_hor*) and covariance (*Sigma_hor*) of the horizontal track.

In the last steps, available surveillance sources are set by SETAVAILABLESOURCES, the STMDISPLAYSTRUCT vector gets filled by SETDISPLAYDATAPASSIVE, and the TRMINTRUDERINPUT is appended to STMREPORT.

This algorithm takes as input *report*, *id*, *agt_idx*, and *T*. This algorithm updates *report.trm_input.intruder*.

Algorithm 148 AddAGTTrackToReport

```
function AddAGTTrackToReport(report::StmReport(p. 480), id::UInt32, trk::AGTTrackFile(p. 486), T::R, passive_only::Bool)
    min_extrap_toa_step::R = params().surveillance.min_extrap_toa_step
    m2ft::R = geoutils.meters_to_feet
    tgt = target_db[id]
    intruder = TRMIntruderInput(p. 499)()
    intruder.id = id
    intruder.source = SOURCE_AGT
    SetClassification(p. 136)(intruder)
    SetIntruderIDDirectory(p. 137)(tgt, intruder.id_directory)
    SetCoordination(p. 104)(intruder)
    SetIntruderDegradedFlags(p. 138)(intruder, tgt, trk, passive_only)
    valid_vert = (0 == (intruder.degraded_surveillance & DEGRADED_SURVEILLANCE_NAR))
    h_own = report.trm_input.own.belief_vert[1].z
    intruder.valid = true
    if (own.opmode == OPMODE_STANDBY)
        intruder.valid = false
    end
    is_large::Bool = (trk.classification != CLASSIFICATION_SMALL_UNMANNED)
    trk_temp = deepcopy(trk)
    dt::R = T - trk.toa
    if (dt >= min_extrap_toa_step)
        (trk_temp.mu_hor, trk_temp.Sigma_hor) = PredictPassiveTracker(p. 46)(trk_temp.mu_hor, trk_temp.Sigma_hor, dt, false)
        ReCenterHorizontalTrackLocation(p. 42)(trk_temp, T)
        (trk_temp.mu_vert, trk_temp.Sigma_vert) = PredictVerticalTracker(p. 54)(trk_temp.mu_vert, trk_temp.Sigma_vert, dt, trk_temp.alt_src_hae, false, is_large)
    end
    (mu_vert, Sigma_vert) = (trk_temp.mu_vert, trk_temp.Sigma_vert)
    if (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        z_rel = GetRelativeAltitude(p. 139)(mu_vert[1], trk.alt_src_hae)
        mu_vert[1] = z_rel + h_own
    end
    (mu_hor, Sigma_hor) = AbsoluteGeodeticToOwnshipRelative(p. 141)(trk_temp)
    (vsamples,vweights) = AddAltBiasAndSample(p. 143)(trk, mu_vert, Sigma_vert, valid_vert, trk.alt_src_hae, is_large, T)
    resize!(intruder.belief_vert, length(vweights))
    for k in 1:length(vweights)
        intruder.belief_vert[k]        = IntruderVerticalBelief(p. 500)()
        intruder.belief_vert[k].z      = vsamples[1,k]
        intruder.belief_vert[k].dz     = vsamples[2,k]
        intruder.belief_vert[k].weight = vweights[k]
    end
    mu_hor *= m2ft
    Sigma_hor *= (m2ft^2)
    (hsamples,hweights) = SigmaPointSample(p. 144)(mu_hor, Sigma_hor)
    resize!(intruder.belief_horiz, length(hweights))
    for k in 1:length(hweights)
        intruder.belief_horiz[k]        = IntruderHorizontalBelief(p. 500)()
        intruder.belief_horiz[k].x_rel  = hsamples[1,k]
        intruder.belief_horiz[k].dx_rel = hsamples[2,k]
        intruder.belief_horiz[k].y_rel  = hsamples[3,k]
        intruder.belief_horiz[k].dy_rel = hsamples[4,k]
        intruder.belief_horiz[k].weight = hweights[k]
    end
    SetDisplayDataPassive(p. 146)(report, intruder, mu_vert, mu_hor, valid_vert, T, trk.display_arrow_current, !passive_only)
    intruder.adsb_cccb = deepcopy(tgt.adsb_osm.adsb_cccb)
    intruder.v2v_cccb = deepcopy(tgt.v2v_osm.v2v_cccb)
    push!(report.trm_input.intruder,intruder)
end
```

Referenced In: AddTracksToReport(p. 119)

2.8.4.5 Point Obstacle Track File

ADDPOINTOBSTACLETRACKTOREPORT (Algorithm 149) defines the process through which the data in a POINTOBSTACLETRACKFILE is added to an STMREPORT. First, the corresponding Track File is retrieved from memory, and a new TRMINTRUDERINPUT (i.e., the data structure in the STMREPORT that represents an intruding aircraft) is initialized. Several of the data elements in the TRMINTRUDERINPUT are then populated with the associated data in the POINTOBSTACLE-TRACKFILE. This includes the track id (*id*), the source of the intruder state information (*source*), a point-obstacle *classification*, the degraded surveillance state (*degraded_surveillance*), and the validity of the track data (*valid*) for RA processing by the TRM. The addresses assigned by an external surveillance system, the reported point obstacle identifier in the case of an POINTOB-STACLETRACKFILE, (*po_id*) are set through a call to SETINTRUDERIDDIRECTORY (Algorithm 134). SETCOORDINATION (Algorithm 104) is called to set show the lack of equipage and coordination capability by a point obstacle intruder.

If *trm_altmode* is not equal to *TRM_ALTMODE_NONE*, the elevation of the point obstacle undergoes a conversion to ensure it is defined with respect to the same vertical datum as ownship. This is done by first generating the relative altitude by a call to GETRELATIVEALTITUDE, and then converting back into an absolute altitude by adding the reported altitude of ownship to the relative altitude. Finally, the absolute geodetic point-obstalce track is converted to ownship relative Cartesian states.

The representation of the point-obstacle's horizontal and vertical states are provided as a single sample with a weight of one. In the last steps, the STMDISPLAYSTRUCT vector gets filled by SETDISPLAYDATAPASSIVE, and the TRMINTRUDERINPUT is appended to STMREPORT.

This algorithm takes as input *report*, *id*, and *T*. This algorithm updates *report.trm_input.intruder*.

Algorithm 149 AddPointObstacleTrackToReport

```
function AddPointObstacleTrackToReport(report::StmReport(p. 480), id::UInt32, T::R)
    m2ft::R = geoutils.meters_to_feet
    tgt = target_db[id]
    trk = tgt.point_obstacle_track
    intruder = TRMIntruderInput(p. 499)()
    intruder.id = id
    intruder.source = SOURCE_POINT_OBSTACLE
    intruder.classification = CLASSIFICATION_POINT_OBSTACLE
    SetIntruderIDDirectory(p. 137)(tgt, intruder.id_directory)
    SetCoordination(p. 104)(intruder)
    intruder.valid = true
    if (own.opmode == OPMODE_STANDBY)
        intruder.valid = false
    end
    mu_vert = [trk.lla_hae_rad_ft[3], 0.0]
    if (report.trm_input.own.trm_altmode != TRM_ALTMODE_NONE)
        z_rel = GetRelativeAltitude(p. 139)(mu_vert[1], true)
        h_own = report.trm_input.own.belief_vert[1].z
        mu_vert[1] = z_rel + h_own
    end
    (_, ecef, own_lla, own_ecef) = AlignAltitudeTypesUsedInGeodeticPositions(p. 109)(trk.lla_hae_rad_ft, trk.
        ecef_hae_m, true, true)
    delta_ecef = ecef - own_ecef
    delta_enu = RotateECEFToENU(p. 23)(delta_ecef, own_lla)
    dx_rel = 0.0 - own.geo_states_hae_alt.vel_ew_mps
    dy_rel = 0.0 - own.geo_states_hae_alt.vel_ns_mps
    mu_hor = [delta_enu[1], dx_rel, delta_enu[2], dy_rel] * m2ft
    vweights = 1.0
    resize!(intruder.belief_vert,length(vweights))
    intruder.belief_vert[1] = IntruderVerticalBelief(p. 500)()
    intruder.belief_vert[1].z = mu_vert[1]
    intruder.belief_vert[1].dz = mu_vert[2]
    intruder.belief_vert[1].weight = vweights
    hweights = 1.0
    resize!(intruder.belief_horiz,length(hweights))
    intruder.belief_horiz[1] = IntruderHorizontalBelief(p. 500)()
    intruder.belief_horiz[1].x_rel = mu_hor[1]
    intruder.belief_horiz[1].dx_rel = mu_hor[2]
    intruder.belief_horiz[1].y_rel = mu_hor[3]
    intruder.belief_horiz[1].dy_rel = mu_hor[4]
    intruder.belief_horiz[1].weight = hweights
    SetDisplayDataPassive(p. 146)(report, intruder, mu_vert, mu_hor, true, T, DISPLAY_ARROW_LEVEL, false)
    intruder.adsb_cccb = deepcopy(tgt.adsb_osm.adsb_cccb)
    intruder.v2v_cccb = deepcopy(tgt.v2v_osm.v2v_cccb)
    push!(report.trm_input.intruder,intruder)
end
```

Referenced In: AddTracksToReport(p. 119)

2.8.5 Track Deletion

A Track File is deleted from memory if it has not been updated for a period of time that depends on the surveillance source and the expected update rate. The function that handles the deletion of deprecated tracks is REMOVESTALETRACKS (Algorithm 150). This algorithm takes as input the ID, *id*, of the TARGET to be examined and a time value, *T*, needed for comparison to determine if a Track File should be deleted. The coast limits for each surveillance region are read in from the parameters file. REMOVESTALETRACKS begins with determining if the TARGET contains a ADSB-TRACKFILE with a call to TRACKEXISTS. A call is made to REMOVESTALEADSBTRACK (Algorithm 151) to complete the process of removing stale ADSBTRACKFILE or component tracks.

Next, the algorithm will delete a ORNCTTRACKFILE if it has not been updated with a position update within the maximum coast limit by a call to REMOVESTALEORNCTTRACK (Algorithm 152).

The algorithm then deletes any AGTTRACKFILE which were not updated within the maximum coast limit. If the AGTTRACKFILE to be deleted was the previously selected track for the target, *tgt.previously_selected_track_uid* is set to nothing. Additionally, the Optimal Track History is reset.

Finally, the algorithm will delete a V2VTRACKFILE if it has not been updated with a position update within the maximum coast limit by a call to REMOVESTALEV2VTRACK (Algorithm 153).

Next, the algorithm will delete a POINTOBSTACLETRACKFILE if the *to_be_deleted* flag has been set.

Besides tracks, stale coordination data is deleted in this algorithm; COORDINATIONTIMEOUTCHECK provides that functionality.

At the end of this process, if no Track Files remain for a TARGET and the TARGET is not designated, the TARGET is deleted from memory and the algorithm returns as FALSE. If Track Files do remain for the TARGET or the TARGET is designated, the algorithm returns as TRUE.

This algorithm takes as input *id*, and *T*. This algorithm returns TRUE or FALSE.

Algorithm 150 RemoveStaleTracks

```
function RemoveStaleTracks(id::UInt32, T::R)
    max_coasts_agt::Z = params().surveillance.agt.max_coasts
    hybrid_override_timeout::R = params().surveillance.hybrid_override_timeout
    tgt = target_db[id]
    if (TrackExists(p. 19)(tgt.adsb_track))
        RemoveStaleADSBTrack(p. 160)(tgt, T)
    end
    if (tgt.adsb_qual_override) && ( (T-tgt.adsb_qual_override_toa) > hybrid_override_timeout )
        tgt.adsb_qual_override = false
    end
    if (TrackExists(p. 19)(tgt.ornct_track))
        RemoveStaleORNCTTrack(p. 160)(tgt, T)
    end
    for i in reverse(1:length(tgt.agt_tracks))
        dt = T - tgt.agt_tracks[i].toa_update
        if (dt > max_coasts_agt)
            if (tgt.agt_tracks[i].track_uid == tgt.previously_selected_trk_uid)
                tgt.previously_selected_trk_uid = nothing
            end
            tgt.optimal_trk_history = String[]
            deleteat!(tgt.agt_tracks,i)
        end
    end
    if (TrackExists(p. 19)(tgt.v2v_track))
        RemoveStaleV2VTrack(p. 161)(tgt, T)
    end
    if (TrackExists(p. 19)(tgt.point_obstacle_track))
        if (tgt.point_obstacle_track.to_be_deleted)
            tgt.point_obstacle_track = nothing
        end
    end
    CoordinationTimeoutCheck(p. 102)(tgt, T)
    if (TargetIsEmpty(p. 161)(tgt))
        delete!(target_db, id)
        return false
    else
        return true
    end
end
```

Referenced In: AddTracksToReport(p. 119)

REMOVESTALEADSBTRACK (Algorithm 151) deletes an ADSBTRACKFILE if the horizontal track has not been updated with a position update within the specified maximum coast limit for the given surveillance mode. If the track to be deleted was the previously selected track for the target, *tgt.previously_selected_trk_uid* is set to nothing. Additionally, the Optimal Track History is reset. The vertical track of an ADSBTRACKFILE may be invalidated independently if it has reached the coast limit without receiving a valid update.

This algorithm takes as input *tgt*, *T*, and *max_coast*. This algorithm updates *tgt.adsb_track*.

Algorithm 151 RemoveStaleADSBTrack

```
function RemoveStaleADSBTrack(tgt::Target(p. 493), T::R)
    max_coasts::Z = params().surveillance.horizontal_adsb.max_coasts
    dt = T - tgt.adsb_track.toa
    dt_vert = T - tgt.adsb_track.toa_vert
    dt_pos = T - tgt.adsb_track.toa_pos_update
    if (dt >= max_coasts) || (dt_pos >= max_coasts)
        if (tgt.adsb_track.track_uid == tgt.previously_selected_trk_uid)
            tgt.previously_selected_trk_uid = nothing
        end
        tgt.optimal_trk_history = String[]
        tgt.adsb_track = nothing
    else
        if (dt_vert >= max_coasts)
            tgt.adsb_track.valid_vert = false
            tgt.adsb_track.updates_vert = 0
        end
    end
end
```

Referenced In: RemoveStaleTracks(p. 159)

REMOVESTALEORNCTTRACK (Algorithm 152) will determine if the ORNCTTRACKFILE has not been updated within *max_coasts* seconds. Intruders identified for deletion that meet the criteria for extended coasting (*is_FOV_coast*) will instead be deleted only once they have not been updated within *max_coasts_ornct_high_priority* seconds. These criteria are that the ORNCT track file had at least *min_FOV_coast_age* updates, has been flagged as high priority or is within *close_rng_ornct_ft* feet of ownship, and had an advisory sometime in the past. If the track to be deleted was the previously selected track for the target, *tgt.previously_selected_trk_uid* is set to nothing. Additionally, the Optimal Track History is reset.

This algorithm takes as input *tgt*, *T*, and *max_coasts*. This algorithm updates *tgt.ornct_track*.

Algorithm 152 RemoveStaleORNCTTrack

```
function RemoveStaleORNCTTrack(tgt::Target(p. 493), T::R)
    max_coasts::Z = params().surveillance.ornct.max_coasts_normal
    max_coasts_high_priority::Z = params().surveillance.ornct.max_coasts_high_priority
    close_rng_ft::R = params().surveillance.ornct.close_rng_ft
    min_FOV_coast_age::Z = params().surveillance.ornct.min_FOV_coast_age
    dt = T - tgt.ornct_track.toa_update
    if (dt > max_coasts)
        is_established_trk::Bool = tgt.ornct_track.update_count >= min_FOV_coast_age
        FOV_spatially_eligible::Bool = (tgt.ornct_track.high_priority) || (tgt.ornct_track.mu_rng[1] <
            close_rng_ft)
        FOV_threat::Bool = tgt.priority_codes.had_code
        tgt.ornct_track.is_FOV_coast = (is_established_trk) && (FOV_spatially_eligible) && (FOV_threat)
        if ((!tgt.ornct_track.is_FOV_coast) || (dt > max_coasts_high_priority))
            if (tgt.ornct_track.track_uid == tgt.previously_selected_trk_uid)
                tgt.previously_selected_trk_uid = nothing
            end
            tgt.optimal_trk_history = String[]
            tgt.ornct_track = nothing
        end
    end
end
```

Referenced In: RemoveStaleTracks(p. 159)

REMOVESTALEV2VTRACK (Algorithm 153) deletes a V2VTRACKFILE if the horizontal track has not been updated within the specified maximum coast limit for the given surveillance mode. If the track to be deleted was the previously selected track for the target, *tgt.previously_selected_trk_uid* is set to nothing. Additionally, the Optimal Track History is reset. The vertical track of a V2VTRACKFILE may be invalidated independently if it has reached the coast limit without receiving a valid update.

This algorithm takes as input *tgt* and *T*. This algorithm updates *tgt.v2v_track*.

Algorithm 153 RemoveStaleV2VTrack

```
function RemoveStaleV2VTrack(tgt::Target(p. 493), T::R)
    max_coasts::Z = params().surveillance.horizontal_v2v.max_coasts
    dt = T - tgt.v2v_track.toa
    dt_vert = T - tgt.v2v_track.toa_vert
    dt_hor = T - tgt.v2v_track.toa_hor
    if (dt > max_coasts) || (dt_hor > max_coasts)
        if (tgt.v2v_track.track_uid == tgt.previously_selected_trk_uid)
            tgt.previously_selected_trk_uid = nothing
        end
        tgt.optimal_trk_history = String[]
        tgt.v2v_track = nothing
    else
        if (dt_vert > max_coasts)
            tgt.v2v_track.valid_vert = false
            tgt.v2v_track.updates_vert = 0
        end
    end
end
```

Referenced In: RemoveStaleTracks(p. 159)

TARGETISEMPTY (Algorithm 154) returns TRUE if the Target, *tgt*, does not contain any ADS-B, AGT, V2V, point-obstacle, and ORNCT track files. Otherwise, FALSE is returned.

This algorithm takes as input *tgt* and returns a boolean.

Algorithm 154 TargetIsEmpty

```
function TargetIsEmpty( tgt::Target(p. 493) )
    return (!TrackExists(p. 19)(tgt.adsb_track))   &&
           (!TrackExists(p. 19)(tgt.agt_tracks)) &&
           (!TrackExists(p. 19)(tgt.v2v_track)) &&
           (!TrackExists(p. 19)(tgt.point_obstacle_track)) &&
           (!TrackExists(p. 19)(tgt.ornct_track))::Bool
end
```

Referenced In: CorrelateIndividualTracksToOwnship(p. 454), DeleteAGTTrack(p. 74), DetermineORNCTDeletion(p. 62), RemoveStaleTracks(p. 159)

2.8.6 Validation

VALIDATIONCHECK (Algorithm 155) is used to check if a Target passes validation. A target is validated if an external system passes a flag indicating this is the case.

This algorithm takes as input *trk*. This algorithm returns the *validated* boolean.

Algorithm 155 ValidationCheck

```
function ValidationCheck(trk::TrackFile(p. 483))
    validated::Bool = trk.externally_validated
    return validated::Bool
end
```

Referenced In: TrackSourceSelection(p. 121), AddTracksToReport(p. 119)

3 Threat Resolution Module Description

3.1 sXu TRM Overview

The threat resolution module for ACAS sXu consists of several components, each processing component is described in a separate section of the document and is invoked from sXuTRMUpdate (Algorithm 156).

The ACAS sXu TRM calculates horizontal and vertical maneuver recommendations to avoid nearby aircraft using the outputs from the STM. Processing through the sXu TRM is linear.

The VerticalTRMUpdate component (Section 3.3) is the Vertical Threat Resolution Module (Vertical TRM or VTRM). It assesses collision risk and issues vertical resolution advisories when necessary.

The HorizontalTRMUpdate component (Section 3.2) is the Horizontal Threat Resolution Module (Horizontal TRM or HTRM). It assesses collision risk and issues horizontal resolution advisories when necessary.

Finally, the GenerateBlendedsXuTRMOutput component (Section 3.4) prepares the advisory information for output based on the blended selection.

Each component is discussed in detail in the following sections.

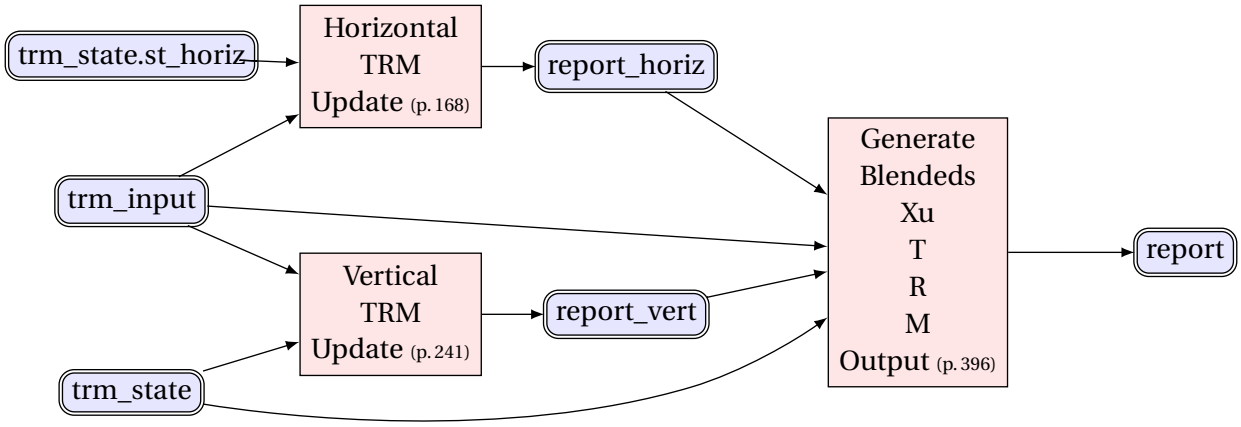

Figure 2. sXuTRMUpdate overview.

Table 30. sXuTRMUpdate Variables shown in Figure 2

Variable	Units	Type	Description
report	N/A	sXuTRMReport(p.501)	Report containing resolution guidance for horizontal and vertical
report_horiz	N/A	HTRMReport(p.519)	Report containing horizontal resolution guidance
report_vert	N/A	VTRMReport(p.524)	Report containing vertical resolution guidance
trm_input	N/A	TRMInput(p.497)	TRM input data prepared by the STM
st_horiz	N/A	sXuTRMState(p.510)	sXu TRM state information; contains vertical state (*st_vert*) and horizontal state (*st_horiz*)

Algorithm 156 sXuTRMUpdate

```
1 function sXuTRMUpdate( trm_state::sXuTRMState(p.510), trm_input::TRMInput(p.497) )
2     report_horiz::HTRMReport(p.519) =
3         HorizontalTRMUpdate(p.168)( trm_state.st_horiz, trm_input )
4     report_vert::VTRMReport(p.524) =
5         VerticalTRMUpdate(p.241)( trm_input, trm_state )
6     report::sXuTRMReport(p.501) =
7         GenerateBlendedsXuTRMOutput(p.396)( trm_state, trm_input,
8                                     report_vert, report_horiz )
9     return report::sXuTRMReport(p.501)
10 end
```

3.1.1 Inputs and Outputs to the sXu TRM

One of the primary ways in which ACAS X improves on previous collision avoidance efforts is in explicitly considering uncertainty about the conditions of the local airspace. Instead of single-sample measurements of various physical quantities, the system works with collections of weighted samples (called beliefs), where every sample of inputs is assigned a weight representing the probability that the actual values are close to the measurements in the sample.

Among the inputs to the sXu TRM, there are three such beliefs: INTRUDERHORIZONTALBELIEF (p.500), INTRUDERVERTICALBELIEF (p.500), and OWNVERTICALBELIEF (p.501). INTRUDERHORIZONTALBELIEF contains relative horizontal distance and rate beliefs from own aircraft to an intruder aircraft. INTRUDERVERTICALBELIEF and OWNVERTICALBELIEF contain altitude and vertical rate beliefs; they are used for intruder aircraft and own aircraft, respectively.

The entire input to the sXu TRM, which includes the own and intruder beliefs, is defined by a nested type TRMINPUT (p.497). TRMINPUT contains own aircraft data, TRMOWNINPUT (p.498), and an array of intruder aircraft data of type TRMINTRUDERINPUT (p.499), one per known intruder. This structure is used throughout the sXu TRM, by each of its components.

Most TRM input is produced by the STM as output from GENERATESTMREPORT (Algorithm 112).

The output from the sXu TRM is defined by another nested type SXUTRMREPORT (p.501). SXUTRMREPORT contains flags to indicate whether vertical and/horizontal guidance should be used; vertical and horizontal guidance information for the operator, (VTRMDISPLAYDATA (p.502) and HTRMDISPLAYDATA (p.503)).

In addition, the sXu TRM inputs and outputs state information, sXuTRMState, details of which are provided in Section 3.1.2.

3.1.2 sXu TRM System Architecture

The sXu TRM has limited state, or memory, which is updated each time the main algorithm loop is run.

This memory is captured by sXuTRMState, which is passed as an argument into the main sXuTRMUpdate method. sXuTRMState holds state information for the Vertical TRM (TRMState) and state information for the Horizontal TRM (HTRMState). The vertical and horizontal TRM states, in turn, contain state information on previous advisories for own aircraft (TRMOwnState and HTRMOwnState), state information on previous advisories and online cost settings for each intruder (TRMIntruderState and HTRMIntruderState), and configuration parameters (paramsfile_type). In addition to using the state information to retain information about previous advisories, the Horizontal TRM also stores and uses information about the processing state on the current cycle.

The dynamics, and consequently the online cost settings and best individual action with respect to own aircraft, are different for each intruder aircraft. Thus, updates to the state variables for one intruder should not affect those of another intruder. To that end, the Vertical TRM maintains a vector of TRMIntruderState, one for each individual intruder and the Horizontal TRM maintains a vector of HTRMIntruderState, also one for each individual intruder.

The intruder state variable *id* is the target identifier supplied by the STM. It is used to associate the TRMIntruderInput structure for an individual intruder with the appropriate intruder state. It can also be used to associate the state and inputs with the outputs.

The parameters contained in *params*, of type paramsfile_type are referenced by name in the algorithms. Their values are stored in a parameter file written in JavaScript Object Notation (JSON). The parameters in that JSON file are described in Appendix E.

3.1.3 Intruder Designation and Protection Modes

ACAS sXu supports only two alerting operational modes that pertain across intruders: Resolution Advisory (RA), and Standby. ACAS sXu includes a Surveillance Only Mode that does not provide alerts to the RPIC but will display the surrounding traffic. Surveillance Only Mode does not have a defined constant and is instead controlled by setting *surv_only_disp_on* in ReceiveDiscretes (Algorithm 80) and altitude thresholds *AGL_SURVONLYMODE_L* and *AGL_SURVONLYMODE_U* defined in constants.

The codes for identifying the currently supported protection modes are given in constants (Appendix C):

1 = Xa (not used for ACAS sXu)

2 = Xo CSPO3k

3 = Xu / sXu

Other protection modes may be defined in the future.

3.2 Horizontal Threat Resolution Module

3.2.1 Horizontal TRM Overview

The Horizontal Threat Resolution Module (Horizontal TRM or HTRM) calculates horizontal maneuver recommendations to avoid nearby aircraft using the outputs from the STM. Processing through the Horizontal TRM is linear.

HORIZONTALTRMUPDATE (Algorithm 157) is the entry point for Horizontal TRM processing. The inputs are: *st_horiz* (HTRMSTATE) containing horizontal TRM state information, and *trm_input* (TRMINPUT), containing data generated by the STM. The state information is updated by the horizontal TRM each ACAS sXu processing cycle and must be maintained by the monitor function. In addition to state information, HORIZONTALTRMUPDATE outputs a horizontal TRM report (HTRMREPORT).

The horizontal TRM requires valid ownship WGS84 information, and ownship ground speed or airspeed. Without such information the horizontal TRM cannot issue a resolution advisory. Invalid ownship WGS84 data is indicated by the bit mask *DEGRADED_OWN_WGS84_INVALID* set by the STM. Similarly, if the operational mode is set to *TRM_OPMODE_SURV_ONLY*, the horizontal TRM will not produce resolution advisories.

HORIZONTALTRMUPDATEPREP (Algorithm 477) and HORIZONTALSTATEESTIMATION (Algorithm 163) pre-process the STM report in preparation for subsequent horizontal TRM processing.

UPDATEINTRUDERHRC (Algorithm 101) will query the incoming horizontal coordination data mid-cycle to ensure the most recent coordination information is being used.

If there are no valid intruders, RESETHTRMSTATE (Algorithm 162) will clear ownship state information, otherwise horizontal TRM processing will continue.

PRIORITIZEANDFILTERINTRUDERS (Algorithm 159) determines which intruders are highest priority for subsequent processing for horizontal RAs.

SELECTHORIZONTALADVISORY (Algorithm 206) determines when a maneuver is necessary to maintain safe separation.

UPDATEPOLICYSPEEDBINS (Algorithm 180) saves intruder speed information for subsequent horizontal TRM calls to ensure that maneuver recommendations are robust to surveillance noise.

Once the horizontal advisory information is produced, the output is prepared.

HORIZONTALDISPLAYLOGICDETERMINATION (Algorithm 223) produces the horizontal advisory display output structure.

Finally, GENERATEHTRMOUTPUT (Algorithm 228) prepares the horizontal TRM output report.

Table 31. HorizontalTRMUpdate Input and Output Variables

Variable	Units	Type	Description
st_horiz	N/A	TRMInput(p. 497)	Input: TRM input data from STM
trm_state	N/A	HTRMState(p. 513)	Input: Horizontal TRM state information
report	N/A	HTRMReport(p. 519)	Output: Report with data for horizontal coordination and the RPIC display
st_int_valid	N/A	VectorHTRMIntruderState	Output: Updated intruder state information needed

Algorithm 157 HorizontalTRMUpdate

```
function HorizontalTRMUpdate( st_horiz::HTRMState(p. 513), trm_input::TRMInput(p. 497) )
    N_actions::Z = params().turn_actions.num_actions
    st_horiz.st_own.is_turn_recommended_prev = st_horiz.st_own.is_turn_recommended
    no_advisory::HorizontalAdvisory(p. 518) = HorizontalAdvisory(p. 518)( )
    no_advisory.costs = fill( NaN, N_actions )
    received_hrcs::Vector{Bool} = fill(false, 2)
    is_wind_relative::Bool = false
    (num_intruders::Z, input_int_valid::Vector{TRMIntruderInput(p. 499)},
        input_int_invalid::Vector{TRMIntruderInput(p. 499)},
        st_int_valid::Vector{HTRMIntruderState(p. 514)},
        st_int_invalid::Vector{HTRMIntruderState(p. 514)},
        st_int_dropped::Vector{HTRMIntruderState(p. 514)}) =
            HorizontalTRMUpdatePrep(p. 589)( trm_input, st_horiz )
    selected_advisory::HorizontalAdvisory(p. 518) = no_advisory
    if (trm_input.own.h_opmode == OPMODE_SURV_ONLY)
        received_hrcs = SetHorizontalTRMSurvOnlyMode(p. 169)(st_horiz.st_own, trm_input.own, input_int_valid,
            st_int_valid )
    elseif (0 < num_intruders)
        HorizontalStateEstimation(p. 175)( trm_input.own, input_int_valid, st_horiz.st_own, st_int_valid )
        if (trm_input.own.h_opmode == OPMODE_RA) && (0 == (trm_input.own.degraded_own_surveillance &
            DEGRADED_OWN_WGS84_INVALID))
            UpdateIntruderHRC(p. 102)( input_int_valid )
            prioritized_intruders::Vector{HTRMIntruderState(p. 514)} =
                    PrioritizeAndFilterIntruders(p. 171)( st_horiz.st_own, st_int_valid, input_int_valid )
            received_hrcs = UpdateHTRMIntruderInputs(p. 173)( input_int_valid, st_int_valid, trm_input.own )
            if (trm_input.own.trm_velmode == TRM_VELMODE_GROUNDSPEED_TRACKANGLE)
                selected_advisory =
                    SelectHorizontalAdvisory(p. 210)( trm_input.own.track_angle, trm_input.own.turn_rate_limit,
                        trm_input.own.vert_rate_limit, st_horiz.st_own, prioritized_intruders )
            else
                selected_advisory =
                    SelectHorizontalAdvisoryWindRelative(p. 211)( trm_input.own.psi, trm_input.own.turn_rate_limit,
                        trm_input.own.vert_rate_limit, st_horiz.st_own, prioritized_intruders )
                is_wind_relative = true
            end
            UpdateHTRMIntruderStates(p. 436)( st_int_valid, false, false )
        else
            selected_advisory = no_advisory
            st_horiz.st_own.is_advisory_prev = false
            received_hrcs = UpdateHTRMIntruderInputs(p. 173)( input_int_valid, st_int_valid, trm_input.own )
            UpdateHTRMIntruderStates(p. 436)( st_int_valid, true, false )
        end
        UpdatePolicySpeedBins(p. 191)( st_horiz.st_own, st_int_valid )
    else
        received_hrcs = UpdateHTRMIntruderInputs(p. 173)( input_int_valid, st_int_valid, trm_input.own )
        ResetHTRMState(p. 174)( st_horiz.st_own, trm_input.own, no_advisory, false, true )
    end
    displaylogic::HorizontalDisplayLogic(p. 518) = HorizontalDisplayLogicDetermination(p. 231)(
        selected_advisory.track_angle, selected_advisory.turn_rate, st_horiz.st_own, trm_input.own.
            turn_rate_limit, trm_input.own.vert_rate_limit )
    displaylogic.wind_relative = is_wind_relative
    displaylogic.h_opmode = trm_input.own.h_opmode
    displaylogic.trm_velmode = trm_input.own.trm_velmode
    report::HTRMReport(p. 519) = GenerateHTRMOutput(p. 234)(trm_input.own, displaylogic, selected_advisory,
        input_int_valid, input_int_invalid, st_int_valid, st_int_invalid, st_int_dropped, st_horiz,
            received_hrcs)
    return (report::HTRMReport(p. 519))
end
```

Referenced In: sXuTRMUpdate(p. 164)

SETHORIZONTALTRMSURVONLYMODE (Algorithm 158) is used when own aircraft is in Surveil-

lance Only Operational Mode to keep the internal state of the TRM updated, so when the own aircraft can produce RAs it will be ready.

SETHORIZONTALTRMSURVONLYMODE takes as input *st_own*, *trm_own*, *input_int_valid*, and *st_int_valid*.

This algorithm returns *received_hrcs*, and modifies *st_int_valid* and *st_own*.

Algorithm 158 SetHorizontalTRMSurvOnlyMode

```
function SetHorizontalTRMSurvOnlyMode(st_own::HTRMOwnState(p. 513), trm_own::TRMOwnInput(p. 498),
                                      input_int_valid::Vector{TRMIntruderInput(p. 499)},
                                      st_int_valid::Vector{HTRMIntruderState(p. 514)})
    N_actions::Z = params().turn_actions.num_actions
    no_advisory::HorizontalAdvisory(p. 518) = HorizontalAdvisory(p. 518)()
    no_advisory.costs = fill( NaN, N_actions )
    HorizontalStateEstimation(p. 175)( trm_own, input_int_valid, st_own, st_int_valid )
    received_hrcs::Vector{Bool} = UpdateHTRMIntruderInputs(p. 173)( input_int_valid, st_int_valid, trm_own )
    st_own.is_advisory_prev = false
    UpdateHTRMIntruderStates(p. 436)( st_int_valid, true, false )
    if (length( st_int_valid ) > 0)
        UpdatePolicySpeedBins(p. 191)( st_own, st_int_valid )
        ResetHTRMState(p. 174)( st_own, trm_own, no_advisory, true, false )
        for i in 1:length( st_int_valid )
            if (0 != (input_int_valid[i].degraded_surveillance & DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED
                )) ||
               (0 != (input_int_valid[i].degraded_surveillance & DEGRADED_SURVEILLANCE_NO_BEARING))
                st_int_valid[i].processing = RA_PROCESSING_DEGRADED_SURVEILLANCE
            else
                st_int_valid[i].processing = RA_PROCESSING_GLOBAL_SURV_ONLY
            end
        end
    else
        ResetHTRMState(p. 174)( st_own, trm_own, no_advisory, true, true )
    end
    return received_hrcs::Vector{Bool}
end
```

Referenced In: HorizontalTRMUpdate(p. 168)

PRIORITIZEANDFILTERINTRUDERS (Algorithm 159) is used to determine the *N_max_prioritized* intruders for subsequent Horizontal TRM processing. The highest priority intruders are determined by performing an offline cost look-up for every intruder within scope, determined by ISINTRUDERINSCOPE (Algorithm 160), and finding those with the highest cost, determined by GETHORIZONTALPOLICYCOSTS (Algorithm 171), for the no maneuver action. When calculating costs to determine intruder priority using GETHORIZONTALPOLICYCOSTS (Algorithm 171), *apply_online_costs* is set to TRUE such that no online costs are applied during cost calculation. A high cost for the no maneuver action indicates that there is a high risk of collision if no maneuvering is performed.

The degraded surveillance flag of each intruder is first considered as processing of the intruder is dependent on surveillance state.

A non-zero value for the *DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED* bit indicates passive-only surveillance (ADS-B, V2V, or, in some cases, AGT) that has not been externally validated.

A non-zero value for the *DEGRADED_SURVEILLANCE_NO_BEARING* bit indicates surveillance with no valid bearing estimate. If an intruder demonstrates either of these degraded surveillance states, then *st_int.processing* is set to *RA_PROCESSING_DEGRADED_SURVEILLANCE*. If an intruder transitioned from *RA_PROCESSING_DEGRADED_SURVEILLANCE* at the previous cycle to non-degraded, then UPDATEHTRMINTRUDERSTATE is called to reset the intruder data structures. Otherwise, *st_int.processing* is set to *RA_PROCESSING_GLOBAL_RA*.

Information about each intruder within scope, along with its minimum cost, is stored in the *priority_list* vector. Intruders marked as bearingless will be omitted from the *priority_list* vector. The *priority_list* is sorted from highest to lowest cost using costs provided by the tabulated cost table. Separate prioritized intruder lists are kept for RA processing, *prioritized_intruders*. Intruders marked as *DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED* will be omitted from the *prioritized_intruders* list for RAs. The *N_max_prioritized* intruders with the highest costs are the highest priority intruders.

This algorithm takes as inputs *st_own*, *st_int_valid*, and *input_int_valid*. This algorithm returns *prioritized_intruders*.

Algorithm 159 PrioritizeAndFilterIntruders

```
function PrioritizeAndFilterIntruders( st_own::HTRMOwnState(p.513), st_int_valid::Vector{HTRMIntruderState(p.514)},
     input_int_valid::Vector{TRMIntruderInput(p.499)} )
    N_max_prioritized::Z = params().horizontal_trm.htrm_prioritization.N_max_prioritized
    priority_list::Vector{Tuple{R,Z,HTRMIntruderState(p.514)}}   = Tuple{R,Z,HTRMIntruderState(p.514)}[]
    prioritized_intruders::Vector{HTRMIntruderState(p.514)} = HTRMIntruderState(p.514)[]
    action_prev::Z = st_own.advisory_prev.action
    enu_beliefs_own::EnuBeliefSet(p.517) = st_own.enu_beliefs
    for i in 1:length(st_int_valid)
        st_int::HTRMIntruderState(p.514)  = st_int_valid[i]
        st_int.in_advisory         = false
        st_int.coc_cost            = NaN
        input_int::TRMIntruderInput(p.499) = input_int_valid[i]
        if (0 != (input_int.degraded_surveillance & DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED)) ||
           (0 != (input_int.degraded_surveillance & DEGRADED_SURVEILLANCE_NO_BEARING))
           st_int.processing = RA_PROCESSING_DEGRADED_SURVEILLANCE
        else
            if (st_int.processing == RA_PROCESSING_DEGRADED_SURVEILLANCE)
                UpdateHTRMIntruderState(p.435)(st_int, true, true)
            end
            st_int.processing = RA_PROCESSING_GLOBAL_RA
        end
        if (0 == (input_int.degraded_surveillance & DEGRADED_SURVEILLANCE_NO_BEARING)) &&
              IsIntruderInScope(p.172)( enu_beliefs_own.enu_ave, st_int.enu_beliefs.enu_ave, st_int.t_init )
            costs::Vector{R} = GetHorizontalPolicyCosts(p.184)( enu_beliefs_own, st_int.enu_beliefs,
                                                   action_prev, true, st_own, st_int )
            cost_min::R      = minimum( costs )
            push!( priority_list, (cost_min, st_int.id, st_int) )
        end
    end
    num_prioritized::Z = length( priority_list )
    if (0 < num_prioritized)
        sort!( priority_list, rev=true )
        num_prioritized = min( num_prioritized, N_max_prioritized )
        for i in 1:num_prioritized
            intruder::HTRMIntruderState(p.514) = priority_list[i][end]
            if (intruder.processing != RA_PROCESSING_DEGRADED_SURVEILLANCE)
                push!( prioritized_intruders, intruder )
                prioritized_intruders[end].in_priority_list = true
            end
        end
    end
    return (prioritized_intruders::Vector{HTRMIntruderState(p.514)})
end
```

Referenced In: HorizontalTRMUpdate(p.168)

IsIntruderInScope (Algorithm 160) is used to avoid processing intruder aircraft that are so far away as to pose no collision risk. This can occur when there can be surveillance information for intruders out to very large ranges. IsIntruderInScope also verifies the relative altitude is within scope and that the track duration for the intruder aircraft is at or beyond the initialization time threshold (*t_int*).

This algorithm takes as inputs *enu_own_ave*, *enu_int_ave*, and *t_int*. This algorithm returns *in_-scope*.

Algorithm 160 IsIntruderInScope

```
function IsIntruderInScope( enu_own_ave::EnuPositionVelocity(p.517), enu_int_ave::EnuPositionVelocity(p.517), t_init::R
    )
    H_alt_diff_activation_threshold::R =
        params().horizontal_trm.htrm_prioritization.H_alt_diff_activation_threshold
    D_rng_diff_activation_threshold::R =
        params().horizontal_trm.htrm_prioritization.D_rng_diff_activation_threshold
    T_min_track_duration::R =
        params().horizontal_trm.htrm_prioritization.T_min_track_duration
    in_scope::Bool = false
    alt_diff::R   = abs( enu_int_ave.pos_enu[ENU_UP_IDX] - enu_own_ave.pos_enu[ENU_UP_IDX] )
    range_diff::R = norm( enu_int_ave.pos_enu - enu_own_ave.pos_enu )
    if (T_min_track_duration <= t_init) &&
       (alt_diff <= H_alt_diff_activation_threshold) &&
       (range_diff <= D_rng_diff_activation_threshold)
        in_scope = true
    end
    return in_scope::Bool
end
```

Referenced In: PrioritizeAndFilterIntruders(p.171)

UPDATEHTRMINTRUDERINPUTS (Algorithm 161) is used to retrieve updated information from the STM in the middle of a processing cycle. It retrieves the most recent HRC values from the STM only for those intruders already being processed by the TRM. This retrieval happens at the latest time that HRC inputs can be ingested into the TRM for processing. Subsequent threat processing requires information from received resolution advisory coordination messages.

The call back to the STM, UPDATEINTRUDERHRC, updates the *hrc* field for each intruder represented in the *input_int_valid* vector. UPDATEINTRUDERHRC also returns the *received_hrcs* vector.

Once the TRM receives updated HRC information from the STM, UPDATEHTRMINTRUDERINPUTS loops over all HTRM state intruders *st_int_valid* to set the received HRCs.

ISINTRUDEREQUIPPED and ISINTRUDERMASTER are called to potentially update *st_int.is_equipped* and *st_int.is_master* in the case where an intruder with *EQUIPAGE_LARGE_CAS* sends a coordination message in the middle of a processing cycle and is therefore a ACAS Xr intruder. This intruder will be updated to be equipped and master relative to ownship.

This algorithm takes as input *input_int_valid*, *st_int_valid*, and *input_own*. This algorithm returns *received_hrcs* and modifies *input_int_valid* and *st_int_valid*.

Algorithm 161 UpdateHTRMIntruderInputs

```
function UpdateHTRMIntruderInputs(input_int_valid::Vector{TRMIntruderInput(p. 499)}, st_int_valid::Vector{
    HTRMIntruderState(p. 514)}, input_own::TRMOwnInput(p. 498))
    received_hrcs::Vector{Bool} = UpdateIntruderHRC(p. 102)(input_int_valid)
    for i in 1:length( input_int_valid )
        st_int_valid[i].received_hrc = input_int_valid[i].hrc
        st_int_valid[i].is_equipped = IsIntruderEquipped(p. 423)(input_own.h_opmode, input_int_valid[i].equipage)
        st_int_valid[i].is_master = IsIntruderMaster(p. 424)(input_own, input_int_valid[i])
    end
    return received_hrcs::Vector{Bool}
end
```

Referenced In: SetHorizontalTRMSurvOnlyMode(p. 169), HorizontalTRMUpdate(p. 168)

RESETHTRMSTATE (Algorithm 162) resets the horizontal TRM outputs and calculates relevant ownship state information (CONVERTOWNINPUTSTOENUBELIEFS (Algorithm 164), CONVERTOWNINPUTSTOENUBELIEFSGROUNDANDCOMPASSMIX (Algorithm 165), CONVERTOWNINPUTSTOENUBELIEFSWINDRELATIVE (Algorithm 166)) when there are no valid intruders. The algorithm used is dependent on the STM velocity mode determined by UPDATEHORIZONTALVELOCITYMODE (Algorithm 117).

This algorithm takes as inputs *st_own*, *trm_own*, *no_advisory*, *surv_only_reset*, and *speed_bin_reset*. This algorithm updates *st_own*.

Algorithm 162 ResetHTRMState

```
function ResetHTRMState( st_own::HTRMOwnState(p. 513), trm_own::TRMOwnInput(p. 498), no_advisory::
     HorizontalAdvisory(p. 518), surv_only_reset::Bool, speed_bin_reset::Bool)
    N_horizontal_scaling_options::Z = size(params().horizontal_trm.horizontal_offline.horizontal_-
         scaling.values, 1)
    if (trm_own.trm_velmode == TRM_VELMODE_GROUNDSPEED_TRACKANGLE) || (trm_own.trm_velmode ==
         TRM_VELMODE_NONE)
        st_own.enu_beliefs =
            ConvertOwnInputsToEnuBeliefs(p. 176)( trm_own.ground_speed, trm_own.track_angle,
                                          trm_own.belief_vert )
    elseif (trm_own.trm_velmode == TRM_VELMODE_GROUNDSPEED_HEADING)
        st_own.enu_beliefs =
            ConvertOwnInputsToEnuBeliefsGroundAndCompassMix(p. 177)( trm_own.ground_speed, trm_own.psi,
                                                     trm_own.belief_vert )
    elseif (trm_own.trm_velmode == TRM_VELMODE_AIRSPEED_HEADING)
        st_own.enu_beliefs =
            ConvertOwnInputsToEnuBeliefsWindRelative(p. 178)( trm_own.airspeed, trm_own.psi,
                                                     trm_own.belief_vert )
    end
    st_own.is_advisory_prev = false
    st_own.advisory_prev = no_advisory
    st_own.prev_word = 0
    st_own.num_reversals = 0
    if (speed_bin_reset == true)
        st_own.speed_bin_prev_policy    = fill(0, N_horizontal_scaling_options)
        st_own.is_initialized_speed_bin = false
    end
    if (surv_only_reset == true)
      st_own.is_turn_recommended          = false
      st_own.highest_threat_prev          = UInt32[]
    end
    return
end
```

Referenced In: SetHorizontalTRMSurvOnlyMode(p. 169), HorizontalTRMUpdate(p. 168)

3.2.2 Horizontal Logic State Estimation

HORIZONTALSTATEESTIMATION (Algorithm 163) preprocesses STM reports for horizontal TRM usage.

First, CONVERTOWNINPUTSTOENUBELIEFS is used to incorporate ownship ground speed information into the ENU belief states and precompute mean states.

CONVERTINTRUDERINPUTSTOENUBELIEFS is used to preprocess belief states and calculate mean states for a single intruder. The STM provides separate sets of beliefs for both horizontal and vertical states. These separate sets of beliefs are combined into a single set of ENU beliefs that includes all the horizontal and vertical belief information.

Next, for each intruder aircraft, ENU belief information is created and the coordination information in the intruder state is updated.

The Horizontal TRM state for the individual intruder (*st_int_valid*) is updated by calling ISINTRUDEREQUIPPED and ISINTRUDERMASTER and setting *st_int_valid.own_coordination_intruder*.

This algorithm takes as inputs *input_own*, *input_int_valid*, *st_own* and *st_int_valid*. This algorithm updates (*st_int_valid*).

Algorithm 163 HorizontalStateEstimation

```
function HorizontalStateEstimation( input_own::TRMOwnInput(p. 498),
                            input_int_valid::Vector{TRMIntruderInput(p. 499)},
                            st_own::HTRMOwnState(p. 513),
                            st_int_valid::Vector{HTRMIntruderState(p. 514)} )
    if (input_own.trm_velmode == TRM_VELMODE_GROUNDSPEED_TRACKANGLE) || (input_own.trm_velmode ==
         TRM_VELMODE_NONE)
        st_own.enu_beliefs =
            ConvertOwnInputsToEnuBeliefs(p. 176)( input_own.ground_speed, input_own.track_angle,
                                        input_own.belief_vert )
    elseif (input_own.trm_velmode == TRM_VELMODE_GROUNDSPEED_HEADING)
        st_own.enu_beliefs =
            ConvertOwnInputsToEnuBeliefsGroundAndCompassMix(p. 177)( input_own.ground_speed, input_own.psi,
                                                    input_own.belief_vert )
    elseif (input_own.trm_velmode == TRM_VELMODE_AIRSPEED_HEADING)
        st_own.enu_beliefs =
            ConvertOwnInputsToEnuBeliefsWindRelative(p. 178)( input_own.airspeed, input_own.psi,
                                                    input_own.belief_vert )
    end
    enu_vel_own_ave::Vector{R} = st_own.enu_beliefs.enu_ave.vel_enu
    for i in 1:length( input_int_valid )
        intruder::TRMIntruderInput(p. 499) = input_int_valid[i]
        st_int_valid[i].enu_beliefs =
            ConvertIntruderInputsToEnuBeliefs(p. 179)( enu_vel_own_ave, input_own.belief_vert, intruder.belief_vert
                 , intruder.belief_horiz, intruder.degraded_surveillance, intruder.classification )
       st_int_valid[i].is_equipped = IsIntruderEquipped(p. 423)(input_own.h_opmode, input_int_valid[i].equipage)
       st_int_valid[i].is_master = IsIntruderMaster(p. 424)(input_own, input_int_valid[i])
       st_int_valid[i].own_coordination_policy = input_int_valid[i].own_coordination_policy
    end
end
```

Referenced In: SetHorizontalTRMSurvOnlyMode(p. 169), HorizontalTRMUpdate(p. 168)

CONVERTOWNINPUTSTOENUBELIEFS (Algorithm 164) converts ownship aircraft belief states of ground speed, track angle, and altitude to ENU before their usage within the horizontal TRM. During this preprocessing step, the mean ownship position and mean velocity are also computed for use throughout the horizontal TRM.

This algorithm takes as inputs *ground_speed*, *track_angle*, and *belief_vert*. This algorithm returns *enu_beliefs*.

Algorithm 164 ConvertOwnInputsToEnuBeliefs

```
function ConvertOwnInputsToEnuBeliefs( ground_speed::R, track_angle::R,
                                       belief_vert::Vector{OwnVerticalBelief(p. 501)} )
    m2ft::R = geoutils.meters_to_feet
    b_enu::Vector{EnuBelief(p. 516)} = EnuBelief(p. 516)[]
    enu_ave::EnuPositionVelocity(p. 517) = EnuPositionVelocity(p. 517)()
    for b_vert in belief_vert
        enu_belief::EnuBelief(p. 516) = EnuBelief(p. 516)()
        enu_belief.weight = b_vert.weight
        enu_belief.enu.pos_enu = [ 0.0, 0.0, b_vert.z / m2ft ]
        horiz_speed_squared::R = ground_speed^2
        horiz_speed_mps::R = sqrt( nanmax(p. 584)( 0.0, horiz_speed_squared ) ) / m2ft
        enu_belief.enu.vel_enu = [ horiz_speed_mps * sin(track_angle),
                                   horiz_speed_mps * cos(track_angle),
                                   b_vert.dz / m2ft ]
        enu_ave.pos_enu = enu_ave.pos_enu + (enu_belief.enu.pos_enu * enu_belief.weight)
        enu_ave.vel_enu = enu_ave.vel_enu + (enu_belief.enu.vel_enu * enu_belief.weight)
        push!(b_enu, enu_belief)
    end
    enu_beliefs::EnuBeliefSet(p. 517) = EnuBeliefSet(p. 517)()
    enu_beliefs.b_enu = b_enu
    enu_beliefs.enu_ave = enu_ave
    return enu_beliefs::EnuBeliefSet(p. 517)
end
```

Referenced In: ResetHTRMState(p. 174), HorizontalStateEstimation(p. 175)

CONVERTOWNINPUTSTOENUBELIEFSGROUNDANDCOMPASSMIX (Algorithm 165) is an implementation which uses ground speed and heading when the STM has determined the ownship WGS84 velocity measurements can not provide adequate track angle measurements, such as when ownship is hovering. The determination of velocity mode is performed by the STM algorithm UPDATEHORIZONTALVELOCITYMODE (Algorithm 117). This implementation use ownship ground speed and heading to calculate ownship velocity estimates, which are used to calculate the relative velocity of the intruder (instead of ground speed and track angle to determine the velocities).

The algorithm CONVERTOWNINPUTSTOENUBELIEFSGROUNDANDCOMPASSMIX (Algorithm 165) takes as inputs *ground_speed*, *heading*, and *belief_vert*. This algorithm returns *enu_beliefs*.

Algorithm 165 ConvertOwnInputsToEnuBeliefsGroundAndCompassMix

```
function ConvertOwnInputsToEnuBeliefsGroundAndCompassMix( ground_speed::R, heading::R,
                                belief_vert::Vector{OwnVerticalBelief(p. 501)} )
    m2ft::R = geoutils.meters_to_feet
    b_enu::Vector{EnuBelief(p. 516)} = EnuBelief(p. 516)[]
    enu_ave::EnuPositionVelocity(p. 517) = EnuPositionVelocity(p. 517)()
    for b_vert in belief_vert
        enu_belief::EnuBelief(p. 516) = EnuBelief(p. 516)()
        enu_belief.weight = b_vert.weight
        enu_belief.enu.pos_enu = [ 0.0, 0.0, b_vert.z / m2ft ]
        horiz_speed_squared::R = ground_speed^2
        horiz_speed_mps::R = sqrt( nanmax(p. 584)( 0.0, horiz_speed_squared ) ) / m2ft
        enu_belief.enu.vel_enu = [ horiz_speed_mps * sin(heading),
                                   horiz_speed_mps * cos(heading),
                                   b_vert.dz / m2ft ]
        enu_ave.pos_enu = enu_ave.pos_enu + (enu_belief.enu.pos_enu * enu_belief.weight)
        enu_ave.vel_enu = enu_ave.vel_enu + (enu_belief.enu.vel_enu * enu_belief.weight)
        push!(b_enu, enu_belief)
    end
    enu_beliefs::EnuBeliefSet(p. 517) = EnuBeliefSet(p. 517)()
    enu_beliefs.b_enu = b_enu
    enu_beliefs.enu_ave = enu_ave
    return enu_beliefs::EnuBeliefSet(p. 517)
end
```

Referenced In: ResetHTRMState(p. 174), HorizontalStateEstimation(p. 175)

CONVERTOWNINPUTSTOENUBELIEFSWINDRELATIVE (Algorithm 166) is an implementation using wind relative coordinates of CONVERTOWNINPUTSTOENUBELIEFS (Algorithm 164), respectively. The wind relative implementations use ownship airspeed and track angle to calculate ownship velocity estimates, which are used to calculate the relative velocity of the intruder (instead of ground speed and track angle to determine the velocities).

The algorithm CONVERTOWNINPUTSTOENUBELIEFSWINDRELATIVE (Algorithm 166) takes as inputs *airspeed*, *heading*, and *belief_vert*. This algorithm returns *enu_beliefs*.

Algorithm 166 ConvertOwnInputsToEnuBeliefsWindRelative

```
function ConvertOwnInputsToEnuBeliefsWindRelative( airspeed::R, heading::R,
                                                   belief_vert::Vector{OwnVerticalBelief(p. 501)} )
    m2ft::R = geoutils.meters_to_feet
    kts2mps::R = geoutils.kts_to_mps
    b_enu::Vector{EnuBelief(p. 516)} = EnuBelief(p. 516)[]
    enu_ave::EnuPositionVelocity(p. 517) = EnuPositionVelocity(p. 517)()
    for b_vert in belief_vert
        enu_belief::EnuBelief(p. 516) = EnuBelief(p. 516)()
        enu_belief.weight = b_vert.weight
        enu_belief.enu.pos_enu = [ 0.0, 0.0, b_vert.z / m2ft ]
        airspeed_squared::R = airspeed^2
        airspeed_mps::R = sqrt( max( 0.0, airspeed_squared ) ) * kts2mps
        enu_belief.enu.vel_enu = [ airspeed_mps * sin(heading),
                                   airspeed_mps * cos(heading),
                                   b_vert.dz / m2ft ]
        enu_ave.pos_enu = enu_ave.pos_enu + (enu_belief.enu.pos_enu * enu_belief.weight)
        enu_ave.vel_enu = enu_ave.vel_enu + (enu_belief.enu.vel_enu * enu_belief.weight)
        push!(b_enu, enu_belief)
    end
    enu_beliefs::EnuBeliefSet(p. 517) = EnuBeliefSet(p. 517)()
    enu_beliefs.b_enu = b_enu
    enu_beliefs.enu_ave = enu_ave
    return enu_beliefs::EnuBeliefSet(p. 517)
end
```

Referenced In: ResetHTRMState(p. 174), HorizontalStateEstimation(p. 175)

CONVERTINTRUDERINPUTSTOENUBELIEFS (Algorithm 167) handles the conversion of intruder aircraft belief states from relative Cartesian to ENU through a call to POINTOBSTACLEENUBELIEFESTIMATION or ENUBELIEFESTIMATION. If an intruder reports *DEGRADED_SURVEILLANCE_NAR*, the vertical beliefs for own aircraft are used to populate the intruder vertical beliefs from the input.

This algorithm takes as inputs *enu_vel_own_ave*, *belief_vert_own*, *belief_vert_int*, *belief_horiz_int*, and *degraded_surveillance*. This algorithm returns *enu_beliefs*.

Algorithm 167 ConvertIntruderInputsToEnuBeliefs

```
function ConvertIntruderInputsToEnuBeliefs( enu_vel_own_ave::Vector{R},
                                            belief_vert_own::Vector{OwnVerticalBelief(p. 501)},
                                            belief_vert_int::Vector{IntruderVerticalBelief(p. 500)},
                                            belief_horiz_int::Vector{IntruderHorizontalBelief(p. 500)},
                                            degraded_surveillance::UInt16, classification::UInt8 )
    enu_beliefs::EnuBeliefSet(p. 517) = EnuBeliefSet(p. 517)()
    if (0 != (degraded_surveillance & DEGRADED_SURVEILLANCE_NAR))
        belief_vert_int = Vector{IntruderVerticalBelief(p. 500)}(undef, length(belief_vert_own))
        for i in 1:length(belief_vert_own)
            belief_vert_int[i] = IntruderVerticalBelief(p. 500)(belief_vert_own[i].z, belief_vert_own[i].dz,
                 belief_vert_own[i].weight)
        end
    end
    if ( classification == CLASSIFICATION_POINT_OBSTACLE )
        enu_beliefs.b_enu = PointObstacleEnuBeliefEstimation(p. 180)( enu_vel_own_ave, belief_vert_int,
             belief_horiz_int )
    else
        enu_beliefs.b_enu = EnuBeliefEstimation(p. 181)( enu_vel_own_ave, belief_vert_int, belief_horiz_int )
    end
    enu_ave::EnuPositionVelocity(p. 517) = EnuPositionVelocity(p. 517)()
    for b::EnuBelief(p. 516) in enu_beliefs.b_enu
        enu_ave.pos_enu = enu_ave.pos_enu + (b.enu.pos_enu * b.weight)
        enu_ave.vel_enu = enu_ave.vel_enu + (b.enu.vel_enu * b.weight)
    end
    enu_beliefs.enu_ave = enu_ave
    return enu_beliefs::EnuBeliefSet(p. 517)
end
```

Referenced In: HorizontalStateEstimation(p. 175)

POINTOBSTACLEENUBELIEFESTIMATION (Algorithm 168) combines the beliefs provided by the STM into ENU beliefs to be used for Horizontal TRM processing. The STM provides separate sets of belief states for both horizontal and vertical states in relative Cartesian coordinates. POINTOBSTACLEENUBELIEFESTIMATION (Algorithm 168) combines these separate sets of belief states into a single set of ENU belief states that includes all horizontal and vertical belief state information.

Since the horizontal and vertical belief state sets provided by the STM have equal numbers of states for point obstacles, when recombining into a single set of belief states, there is no need for the belief state weights to be reweighted for point obstacles.

This algorithm takes as inputs *enu_vel_own_ave*, *belief_vert*, and *belief_horiz*. This algorithm returns *b_enu*.

Algorithm 168 PointObstacleEnuBeliefEstimation

```
function PointObstacleEnuBeliefEstimation( enu_vel_own_ave::Vector{R},
                                belief_vert::Vector{IntruderVerticalBelief(p. 500)},
                                belief_horiz::Vector{IntruderHorizontalBelief(p. 500)} )
    m2ft::R = geoutils.meters_to_feet
    b_enu::Vector{EnuBelief(p. 516)} = EnuBelief(p. 516)[]
    enu_belief::EnuBelief(p. 516) = EnuBelief(p. 516)()
    enu_belief.weight = 1.0
    enu_belief.enu.pos_enu = [ belief_horiz[1].x_rel / m2ft,
                               belief_horiz[1].y_rel / m2ft,
                               belief_vert[1].z / m2ft ]
    enu_belief.enu.vel_enu = [ (belief_horiz[1].dx_rel / m2ft) + enu_vel_own_ave[1],
                               (belief_horiz[1].dy_rel / m2ft) + enu_vel_own_ave[2],
                               belief_vert[1].dz / m2ft ]
    push!(b_enu, enu_belief)
    return b_enu::Vector{EnuBelief(p. 516)}
end
```

Referenced In: ConvertIntruderInputsToEnuBeliefs(p. 179)

ENUBELIEFESTIMATION (Algorithm 169) combines the beliefs provided by the STM into ENU beliefs to be used for Horizontal TRM processing. The STM provides separate sets of belief states for both horizontal and vertical states in relative Cartesian coordinates. ENUBELIEFESTIMATION (Algorithm 169) combines these separate sets of belief states into a single set of ENU belief states that includes all horizontal and vertical belief state information.

Since the horizontal and vertical belief state sets provided by the STM have unequal numbers of states, when recombining into a single set of belief states, the belief state weights must be reweighted and adjusted such that the combined set represents the original information. Belief states are adjusted by scaling the difference of each belief state from the mean by a scale factor, *gamma_vert* for vertical belief states and *gamma_horiz* for horizontal belief states and assigning *w_sample* to the all combined beliefs. If the set of horizontal belief states has N entries and the set of vertical belief states has M entries, then using the approach implemented in ENUBELIEFESTIMATION (Algorithm 169) results in N+M-1 belief states in the combined set. If total independence were assumed, there would be a total of N*M states, but that is computationally expensive and provides minimal additional information beyond the combined states.

This algorithm takes as inputs *enu_vel_own_ave*, *belief_vert*, and *belief_horiz*. This algorithm returns *b_enu*.

Algorithm 169 EnuBeliefEstimation

```
function EnuBeliefEstimation( enu_vel_own_ave::Vector{R},
                              belief_vert::Vector{IntruderVerticalBelief(p. 500)},
                              belief_horiz::Vector{IntruderHorizontalBelief(p. 500)} )
    W_mean::R = 1/3
    m2ft::R = geoutils.meters_to_feet
    (w_max_horiz, mean_horiz_index) = FindMaximumWeightAndIndex(p. 182)(belief_horiz)
    (w_max_vert,  mean_vert_index)  = FindMaximumWeightAndIndex(p. 182)(belief_vert)
    w_sample::R = (1 - W_mean) / ( (length(belief_horiz) - 1 + length(belief_vert) - 1) )
    gamma_vert::R = sqrt( ((1 - w_max_vert) / (length(belief_vert) - 1)) / w_sample)
    gamma_horiz::R = sqrt( ((1 - w_max_horiz) / (length(belief_horiz) - 1)) / w_sample)
    b_enu::Vector{EnuBelief(p. 516)} = EnuBelief(p. 516)[]
    for i in 1:length(belief_horiz)
        b_horiz_mean::IntruderHorizontalBelief(p. 500) = belief_horiz[mean_horiz_index]
        b_vert_mean::IntruderVerticalBelief(p. 500)    = belief_vert[mean_vert_index]
        b_horiz::IntruderHorizontalBelief(p. 500) = belief_horiz[i]
        enu_belief::EnuBelief(p. 516) = EnuBelief(p. 516)()
        if (i == mean_horiz_index)
            enu_belief.weight = W_mean
        else
            enu_belief.weight = w_sample
        end
        enu_belief.enu.pos_enu = [ (b_horiz_mean.x_rel +
                                        (gamma_horiz * (b_horiz.x_rel - b_horiz_mean.x_rel))) / m2ft,
                                   (b_horiz_mean.y_rel +
                                        (gamma_horiz * (b_horiz.y_rel - b_horiz_mean.y_rel))) / m2ft,
                                   b_vert_mean.z / m2ft ]
        enu_belief.enu.vel_enu = [ ((b_horiz_mean.dx_rel +
                                         (gamma_horiz * (b_horiz.dx_rel - b_horiz_mean.dx_rel))) / m2ft) +
                                             enu_vel_own_ave[1],
                                   ((b_horiz_mean.dy_rel +
                                         (gamma_horiz * (b_horiz.dy_rel - b_horiz_mean.dy_rel))) / m2ft) +
                                             enu_vel_own_ave[2],
                                   b_vert_mean.dz / m2ft ]
        push!(b_enu, enu_belief)
    end
    for i in 1:length(belief_vert)
        if (i != mean_vert_index)
            b_horiz_mean::IntruderHorizontalBelief(p. 500) = belief_horiz[mean_horiz_index]
            b_vert_mean::IntruderVerticalBelief(p. 500)    = belief_vert[mean_vert_index]
            b_vert::IntruderVerticalBelief(p. 500) = belief_vert[i]
            enu_belief::EnuBelief(p. 516) = EnuBelief(p. 516)()
            enu_belief.weight = w_sample
            enu_belief.enu.pos_enu = [ b_horiz_mean.x_rel / m2ft,
                                       b_horiz_mean.y_rel / m2ft,
                                       (b_vert_mean.z + (gamma_vert * (b_vert.z - b_vert_mean.z))) / m2ft ]
            enu_belief.enu.vel_enu = [ (b_horiz_mean.dx_rel / m2ft) + enu_vel_own_ave[1],
                                       (b_horiz_mean.dy_rel / m2ft) + enu_vel_own_ave[2],
                                       (b_vert_mean.dz + (gamma_vert * (b_vert.dz - b_vert_mean.dz))) / m2ft ]
            push!(b_enu, enu_belief)
        end
    end
    return b_enu::Vector{EnuBelief(p. 516)}
end
```

Referenced In: ConvertIntruderInputsToEnuBeliefs(p. 179)

FINDMAXIMUMWEIGHTANDINDEX (Algorithm 170) searches through each belief state and returns the weight and index of the sample with the maximum weight, which corresponds to the mean sample.

This algorithm takes as input *beliefs*. This algorithm returns *max_weight* and *index*.

Algorithm 170 FindMaximumWeightAndIndex

```
function FindMaximumWeightAndIndex( beliefs::Union{Vector{IntruderVerticalBelief(p. 500)}, Vector{
    IntruderHorizontalBelief(p. 500)}} )
    index::Z = 1
    max_weight::R = 0.0
    for i in 1:length(beliefs)
        belief = beliefs[i]
        weight::R = belief.weight
        if ( weight > max_weight )
            max_weight = weight
            index = i
        end
    end
    return (max_weight::R, index::Z)
end
```

Referenced In: EnuBeliefEstimation(p. 181)

3.2.3 Offline Horizontal Cost Estimation

An overview of the GETHORIZONTALPOLICYCOSTS component of the Horizontal TRM is shown in Fig. 3.

For ACAS sXu, the offline policy is key to achieving the balance of safety and operational suitability for the system. During system tuning and optimization, costs are tuned to balance the tradeoff between system safety and operational suitability in the system. The pre-computed costs are tabulated and stored in look-up tables, known as policies, for availability during system execution. These costs represent the relative urgency of selecting a particular maneuver or of doing nothing and are precomputed to balance the system performance trade-offs and the dynamic uncertainty of both ownship and intruder aircraft.

Appendix G describes the functionality that enables scaling of the offline policy to make it more or less sensitive. Selection of scaling parameters throughout the algorithms in this section is controlled by the variable *idx_scale*, which specifies the row of scaling values to utilize in the scaling matrix. In typical implementations, this variable will always take the value of 1, which refers to standard setting of all scaling factors at 1.0. Implementations including the scaling provision permit other values of *idx_scale*.

In ACAS sXu, the costs provided by the tabulated cost tables are used to identify the highest threat intruders (PRIORITIZEANDFILTERINTRUDERS), determine the best maneuver recommendation to address the highest threat intruders (SELECTHORIZONTALADVISORY), and to help prioritize intruders for the display (HORIZONTALDISPLAYLOGICDETERMINATION).

GETHORIZONTALPOLICYCOSTS (Algorithm 171) uses the processed STM belief states to calculate the costs for each intruder aircraft.

It calls the following algorithms in sequence:

GETMAXAPPLICABILITYRANGE (Algorithm 172) to retrieve the maximum range from the specified cut set using *idx_scale*. For information on cut sets and cut points, see the description of Auxiliary Data in Appendix F.1.

CREATEPOLICYSTATEBELIEFS (Algorithm 175) to combine intruder ENU belief state information with ownship ENU belief information and convert the result to the policy coordinate system

(POLICYSTATEBELIEF).

SELECTANDMODIFYPOLICYBINS (Algorithm 177) to incorporate information about the ownship speed and intruder speed from the previous update cycle to provide robust maneuver guidance.

GETPOLICYTABLECOSTS (Algorithm 179) to obtain the vector of costs for each action based on the policy state beliefs and the previous action.

APPLYSTATEDEPENDENTONLINECOSTS (Algorithm 183) to apply online costs that are dependent on the geometry between two aircraft. Online costs are used to adjust offline costs at run-time to adjust system performance and improve overall behavior. State dependent online costs are only applied when the previous action was clear of conflict.

This algorithm takes as inputs *own_beliefs*, *int_beliefs*, *last_action*, *apply_online_costs*, *st_own*, and *st_int*.

This algorithm returns *costs*.

If an intruder is outside of the maximum table range of the cost table, then the vector of costs is populated with default values that ensure clear of conflict is the selected action.

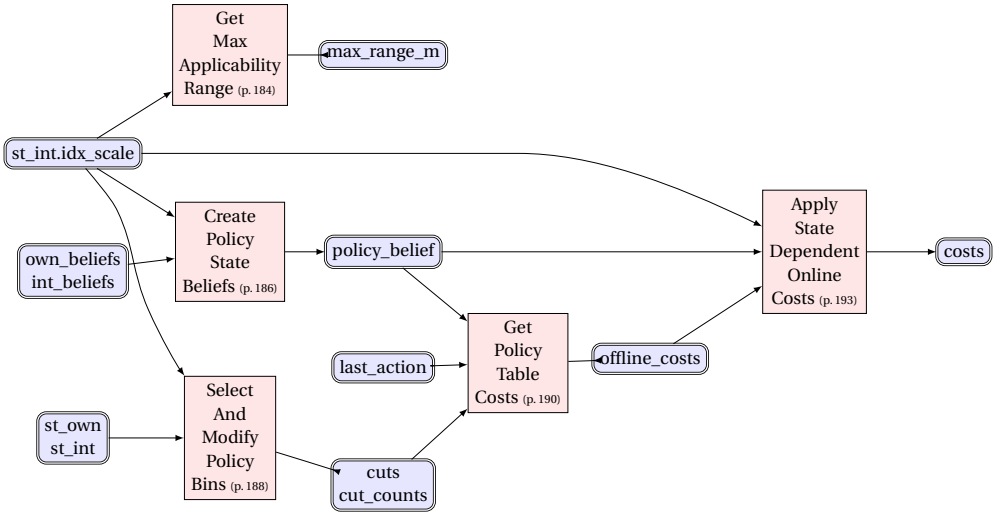

Figure 3. GetHorizontalPolicyCosts overview.

Table 32. GetHorizontalPolicyCosts Output Variables

Variable	Units	Type	Description
costs	N/A	real	vector of costs for an individual intruder, one cost for each turn action

Algorithm 171 GetHorizontalPolicyCosts

```
function GetHorizontalPolicyCosts( own_beliefs::EnuBeliefSet(p. 517), int_beliefs::EnuBeliefSet(p. 517),
                                   last_action::Z, apply_online_costs::Bool,
                                   st_own::HTRMOwnState(p. 513), st_int::HTRMIntruderState(p. 514) )
    C_nominal::R     = params().horizontal_trm.horizontal_offline.C_nominal
    N_actions::Z     = params().turn_actions.num_actions
    idx_en::Vector{Z} = [ENU_EAST_IDX, ENU_NORTH_IDX]
    costs::Vector{R} = zeros( R, N_actions )
    max_range_m::R = GetMaxApplicabilityRange(p. 184)(st_int.idx_scale)
    if (norm(own_beliefs.enu_ave.pos_enu[idx_en] - int_beliefs.enu_ave.pos_enu[idx_en]) < max_range_m)
        policy_belief::Vector{PolicyStateBelief(p. 521)} =
            CreatePolicyStateBeliefs(p. 186)( own_beliefs, int_beliefs, st_int.idx_scale )
        (cuts::Vector{R}, cut_counts::Vector{Z}) =
            SelectAndModifyPolicyBins(p. 188)( st_own, st_int, st_int.idx_scale )
        offline_costs::Vector{R} =
            GetPolicyTableCosts(p. 190)( policy_belief, last_action, cuts, cut_counts )
        if (last_action == COC) && apply_online_costs
            costs = ApplyStateDependentOnlineCosts(p. 193)( policy_belief, offline_costs, st_int.idx_scale )
        else
            costs = offline_costs
        end
     else
        costs      = fill( C_nominal, N_actions )
        costs[COC] = 0.0
     end
    return costs::Vector{R}
end
```

Referenced In: HorizontalCostFusion(p. 217), PrioritizeAndFilterIntruders(p. 171)

GETMAXAPPLICABILITYRANGE (Algorithm 172) returns the maximum range from the specified cut set using *idx_scale* by calling SELECTSCALEDPOLICYBINS and GETCUTPOINTINDEXVECTOR.

This algorithm takes as inputs *idx_scale*. This algorithm returns *max_range_m*.

Algorithm 172 GetMaxApplicabilityRange

```
function GetMaxApplicabilityRange( idx_scale::Z )
    (cuts::Vector{R}, cut_counts::Vector{Z}) = SelectScaledPolicyBins(p. 185)(idx_scale)
    range_indices::Vector{Z} = GetCutpointIndexVector(p. 185)( POLICY_RANGE, cut_counts )
    max_range_m::R = cuts[range_indices[end]]
    return max_range_m::R
end
```

Referenced In: GetHorizontalPolicyCosts(p. 184)

SELECTSCALEDPOLICYBINS (Algorithm 173) returns the cut point vector and cut count vector for the specified cut set as defined by *idx_scale*. The cut point vectors are defined by the *scaled_-policy_cuts_set*.

This algorithm takes as inputs *idx_scale*. This algorithm returns *cuts* and *cut_counts*.

Algorithm 173 SelectScaledPolicyBins

```
function SelectScaledPolicyBins( idx_scale::Z )
    ec_table1_content::RDataTableScaled(p. 522) =
        params().horizontal_trm.horizontal_offline.hcost_policy.origami.equiv_class_table_content[1]
    cut_counts::Vector{Z} = ec_table1_content.cut_counts
    cuts::Vector{R} = ec_table1_content.scaled_policy_cuts_set[idx_scale]
    return (cuts::Vector{R}, cut_counts::Vector{Z})
end
```

Referenced In: ApplyStateDependentOnlineCosts(p. 193), SelectAndModifyPolicyBins(p. 188), GetMaxApplicabilityRange(p. 184)

GETCUTPOINTINDEXVECTOR (Algorithm 174) returns the index vector for the specified cut type and uses *cut_counts* to define an accumulated offset to use.

This algorithm takes as inputs *index* and *cut_counts*. This algorithm returns *index_vector*.

Algorithm 174 GetCutpointIndexVector

```
function GetCutpointIndexVector( index::Z, cut_counts::Vector{Z} )
    index_offset::Z = 0
    if (index != 1)
        index_offset = sum(cut_counts[1:(index-1)])
    end
    index_vector::Vector{Z} = (1:cut_counts[index]) .+ index_offset
    return index_vector::Vector{Z}
end
```

Referenced In: GetMaxApplicabilityRange(p. 184), ScaleHorizontalCuts(p. 577), ScaleVerticalCuts(p. 579)

CREATEPOLICYSTATEBELIEFS (Algorithm 175) combines intruder belief state information with ownship belief information and converts the result to the policy coordinate system.

This algorithm takes as inputs *own_beliefs*, *intr_beliefs*, and *idx_scale*. This algorithm returns *policy_beliefs*.

Algorithm 175 CreatePolicyStateBeliefs

```
function CreatePolicyStateBeliefs( own_beliefs::EnuBeliefSet(p. 517), intr_beliefs::EnuBeliefSet(p. 517), idx_scale::Z )
    N_beliefs::Z  = length( intr_beliefs.b_enu )
    policy_beliefs::Vector{PolicyStateBelief(p. 521)} = Vector{PolicyStateBelief(p. 521)}( undef, N_beliefs )
    z_rel::Vector{R}  = zeros( R, N_beliefs )
    b_enu_own::EnuBelief(p. 516) = EnuBelief(p. 516)( own_beliefs.enu_ave, 1.0 )
    for i::Z in 1:N_beliefs
        b_enu_int::EnuBelief(p. 516) = intr_beliefs.b_enu[i]
        policy_beliefs[i] = PolicyStateBelief(p. 521)()
        (policy_beliefs[i].states, z_rel[i]) =
           CalculatePolicyState(p. 187)( b_enu_own.enu, b_enu_int.enu, idx_scale )
        policy_beliefs[i].weight = b_enu_own.weight * b_enu_int.weight
     end
    z_rel_min::R = nanminimum(p. 584)( z_rel )
    z_rel_max::R = nanmaximum(p. 585)( z_rel )
    if (z_rel_min < 0) && (0 < z_rel_max)
        for b_policy in policy_beliefs
            b_policy.states[POLICY_VERTICAL_TAU] = 0.0
        end
    end
    return policy_beliefs::Vector{PolicyStateBelief(p. 521)}
end
```

Referenced In: GetHorizontalPolicyCosts(p. 184), UpdatePolicySpeedBinIndividual(p. 192), GetHorizontalCoordinationPolicy(p. 223)

CALCULATEPOLICYSTATE (Algorithm 176) calculates the relative geometry for intruder and ownship used for value look-ups in the offline horizontal policy tables. Multi-dimensional inputs are used to look-up a value in the horizontal cost policy table and horizontal coordination policy table. The inputs utilize a relative coordinate system for the intruder. Given ownship and intruder dynamic state, CALCULATEPOLICYSTATE calculates range, relative bearing, relative track angle, ownship speed, intruder speed, and time to loss of vertical separation (also known as vertical tau). It returns a vector containing those values. The vector indices are defined as constants. The vertical tau value is used for the horizontal cost policy table only; it is not used for the horizontal coordination policy table.

This algorithm takes as inputs *enu_own*, *enu_int*, and *idx_scale*. This algorithm returns *policy_state* and *z_rel*.

Algorithm 176 CalculatePolicyState

```
function CalculatePolicyState( enu_own::EnuPositionVelocity(p. 517), enu_int::EnuPositionVelocity(p. 517), idx_scale::Z )
    T_vertical_tau_max::R = params().horizontal_trm.horizontal_offline.T_vertical_tau_max
    H_separation_min_m    = params().horizontal_trm.horizontal_offline.H_separation_min_ras_m
    scales::Matrix{R} = params().horizontal_trm.horizontal_offline.horizontal_scaling.values
    H_point_obstacle_offset_m::R = params().horizontal_trm.horizontal_offline.horizontal_scaling.H_point_-
        obstacle_offset_m
    Classifications::Array{Z} = params().threat_resolution.vertical_scaling.classification
    scale_factor::R = scales[idx_scale,SCALING_HTRM_POLICY_TARGET_SEPARATION]
    policy_state::Vector{R} = zeros( R, POLICY_LAST )
    own_track_angle::R = atan( enu_own.vel_enu[ENU_EAST_IDX], enu_own.vel_enu[ENU_NORTH_IDX] )
    intr_pos_rel::Vector{R} = enu_int.pos_enu - enu_own.pos_enu
    int_range::R = hypot( intr_pos_rel[ENU_EAST_IDX], intr_pos_rel[ENU_NORTH_IDX] )
    policy_state[POLICY_RANGE] = int_range
    theta::R = atan( intr_pos_rel[ENU_EAST_IDX], intr_pos_rel[ENU_NORTH_IDX] ) - own_track_angle
    policy_state[POLICY_THETA] = WrapToPi(p. 583)( theta )
    psi::R = atan( enu_int.vel_enu[ENU_EAST_IDX], enu_int.vel_enu[ENU_NORTH_IDX] ) - own_track_angle
    policy_state[POLICY_PSI]   = WrapToPi(p. 583)( psi )
    own_speed::R = hypot( enu_own.vel_enu[ENU_EAST_IDX], enu_own.vel_enu[ENU_NORTH_IDX] )
    int_speed::R = hypot( enu_int.vel_enu[ENU_EAST_IDX], enu_int.vel_enu[ENU_NORTH_IDX] )
    closure_rate::R = int_speed-own_speed
    policy_state[POLICY_OWN_SPEED]  = own_speed
    policy_state[POLICY_INTR_SPEED] = int_speed
    dz_rel::R = enu_int.vel_enu[ENU_UP_IDX] - enu_own.vel_enu[ENU_UP_IDX]
    z_rel::R  = intr_pos_rel[ENU_UP_IDX]
    H_separation_min_m = H_separation_min_m*scale_factor
    if ( Classifications[idx_scale] == CLASSIFICATION_POINT_OBSTACLE )
        z_rel = -1 * enu_own.pos_enu[ENU_UP_IDX]
        H_separation_min_m = enu_int.pos_enu[ENU_UP_IDX] + H_point_obstacle_offset_m
    end
    if (abs(z_rel) <= H_separation_min_m)
        policy_state[POLICY_VERTICAL_TAU] = 0.0
    else
        if ((0 <= dz_rel) && (0 <= z_rel)) ||
           ((dz_rel <= 0) && (z_rel <= 0))
            policy_state[POLICY_VERTICAL_TAU] = T_vertical_tau_max
        else
            policy_state[POLICY_VERTICAL_TAU] = (abs(z_rel) - H_separation_min_m) / abs(dz_rel)
        end
    end
    policy_state[POLICY_VERTICAL_TAU] = nanmin(p. 584)( policy_state[POLICY_VERTICAL_TAU], T_vertical_tau_max
        )
    return (policy_state::Vector{R}, z_rel::R)
end
```

Referenced In: CreatePolicyStateBeliefs(p. 186)

SELECTANDMODIFYPOLICYBINS (Algorithm 177) incorporates information about the ownship and the individual intruder speed from the previous update cycle to provide robust maneuver guidance. It calls MODIFYPOLICYSPEEDBINSINDIVIDUAL twice: the first time to modify ownship speed bins and the second time to modify the individual intruder speed bins.

This algorithm takes as inputs *st_own*, *st_int*, and *idx_scale*. The modified cut values (*modified_-cuts* and *cut_counts*) are returned.

Algorithm 177 SelectAndModifyPolicyBins

```
function SelectAndModifyPolicyBins( st_own::HTRMOwnState(p. 513), st_int::HTRMIntruderState(p. 514), idx_scale )
    (cuts::Vector{R}, cut_counts::Vector{Z}) = SelectScaledPolicyBins(p. 185)(idx_scale)
    modified_cuts::Vector{R} = deepcopy( cuts )
    own_speed_bin::Z = st_own.speed_bin_prev_policy[idx_scale]
    int_speed_bin::Z = st_int.speed_bin_prev_policy
    modified_cuts =
        ModifyPolicySpeedBinsIndividual(p. 188)( POLICY_OWN_SPEED, st_own.is_initialized_speed_bin,
                                        own_speed_bin, modified_cuts, cut_counts )
    modified_cuts =
        ModifyPolicySpeedBinsIndividual(p. 188)( POLICY_INTR_SPEED, st_int.is_initialized_speed_bin,
                                        int_speed_bin, modified_cuts, cut_counts )
    return (modified_cuts::Vector{R}, cut_counts::Vector{Z})
end
```

Referenced In: GetHorizontalPolicyCosts(p. 184), UpdatePolicySpeedBinIndividual(p. 192)

MODIFYPOLICYSPEEDBINSINDIVIDUAL (Algorithm 178) returns the speed bins with hysteresis applied. Aircraft speed information is used to slightly adjust the policy boundaries for the indicated speed bin type such that surveillance noise will not cause unwarranted maneuver reversals. Without this modification, the policy look-ups can be particularly sensitive to aircraft speed estimation noise when the aircraft speed is near the boundary between neighboring policy look-up bins. The speed bin type is given by *idx_speed*. It is the policy state index for either ownship speed or intruder speed.

This algorithm takes as inputs *idx_speed*, *is_initialized_speed_bin*, *speed_bin_prev*, *modified_cuts*, and *cut_counts*. This algorithm modifies the values input in *modified_cuts*.

Algorithm 178 ModifyPolicySpeedBinsIndividual

```
function ModifyPolicySpeedBinsIndividual( idx_speed::Z, is_initialized_speed_bin::Bool,
                                  speed_bin_prev::Z, modified_cuts::Vector{R},
                                  cut_counts::Vector{Z} )
    X_speed_tolerance_factor::R =
        params().horizontal_trm.horizontal_offline.X_speed_tolerance_factor
    idx::Z        = 0
    hysteresis::R = 0.0
    if is_initialized_speed_bin &&
       (1 <= speed_bin_prev) && (speed_bin_prev < (cut_counts[idx_speed] + 1))
        speed_offset::Z = sum( cut_counts[1:(idx_speed - 1)] )
        if (speed_bin_prev != 1)
            idx = speed_offset + speed_bin_prev - 1
            hysteresis = (modified_cuts[idx+1] - modified_cuts[idx]) * X_speed_tolerance_factor
            modified_cuts[idx] = modified_cuts[idx] - hysteresis
        end
        if (speed_bin_prev != cut_counts[idx_speed])
            idx = speed_offset + speed_bin_prev + 1
            hysteresis = (modified_cuts[idx] - modified_cuts[idx-1]) * X_speed_tolerance_factor
            modified_cuts[idx] = modified_cuts[idx] + hysteresis
        end
    end
    return modified_cuts::Vector{R}
end
```

Referenced In: SelectAndModifyPolicyBins(p. 188)

GETPOLICYTABLECOSTS (Algorithm 179) returns the vector of costs for each action based on the policy state beliefs and the previous action. In this algorithm, the output is called *policy_costs*.

INTERPOLATEONEDIMENSION (Algorithm 386) and TAUBELIEF are used to find the vertical tau index of the table nearest to each belief state. The weighted average of the costs for all belief states are returned as long as its optimal action is at least one of the beliefs' preferred action. If the weighted average does not have one of these actions as its desired action, the costs of the mean belief are returned.

In GETHORIZONTALPOLICYCOSTS, the output is called *offline_costs*. These are the same.

This algorithm takes as inputs *policy_beliefs*, *action_prev*, *modified_cuts*, and *cut_counts*. This algorithm returns *policy_costs*.

Algorithm 179 GetPolicyTableCosts

```
function GetPolicyTableCosts( policy_beliefs::Vector{PolicyStateBelief(p. 521)}, action_prev::Z,
                              modified_cuts::Vector{R}, cut_counts::Vector{Z} )
    N_actions::Z = params().turn_actions.num_actions
    action_pattern_number::Vector{Z} = params().horizontal_trm.horizontal_offline.hcost_-
        policy.origami.action_pattern_number
    N_allowable_actions::Vector{Z}   = params().horizontal_trm.horizontal_offline.hcost_policy.origami.N_-
        allowable_actions
    allowable_actions::Matrix{Z}     = params().horizontal_trm.horizontal_offline.hcost_-
        policy.origami.allowable_actions
    equiv_class_number::Matrix{Z}    = params().horizontal_trm.horizontal_offline.hcost_-
        policy.origami.equiv_class_number
    minblocks_index::Matrix{Z}       = params().horizontal_trm.horizontal_offline.hcost_-
        policy.origami.minblocks_index
    minblocks_table_content::RDataTable(p. 521) = params().horizontal_trm.horizontal_offline.hcost_-
        policy.origami.minblocks_table_content
    equiv_class_table_content::Vector{RDataTableScaled(p. 522)} = params().horizontal_trm.horizontal_-
        offline.hcost_policy.origami.equiv_class_table_content
    N_tau_values::Z          = cut_counts[POLICY_VERTICAL_TAU]
    T_tau_values::Vector{R}  = modified_cuts[1:N_tau_values]
    N_indices::Vector{Z}     = Z[1]
    W_weights::Vector{R}     = R[1.0]
    idx_cut_start::Z         = 2
    N_cut_idx::Z             = length( cut_counts ) - idx_cut_start + 1
    B_use_nearest_neighbor::Bool = true
    policy_costs::Vector{R} = zeros( R, N_actions )
    b_tau_ec_int::Vector{TauBelief(p. 522)} = Vector{TauBelief(p. 522)}( undef, N_tau_values )
    beliefs_best_action::Vector{Z} = zeros(Z,length(policy_beliefs))
    W_belief_weights::Vector{R} = zeros(R,length(policy_beliefs))
    beliefs_costs::Matrix{R} = zeros( length(policy_beliefs), N_actions )
    max_weight_index::Z = 0
    action_contained::Bool = false
    for i::Z in 1:length( policy_beliefs )
        block_size::Z = 1
        (indices::Vector{Z}, _, block_size) =
            InterpolateOneDimension(p. 418)( N_indices, W_weights,
                                    policy_beliefs[i].states[POLICY_VERTICAL_TAU],
                                    T_tau_values, block_size,
                                    B_use_nearest_neighbor )
        idx_closest_tau::Z = indices[1]
        for j::Z in 1:N_tau_values
            b_tau_ec_int[j] = TauBelief(p. 522)( T_tau_values[j], 0.0 )
        end
        b_tau_ec_int[idx_closest_tau] = TauBelief(p. 522)( T_tau_values[idx_closest_tau], 1.0 )
        point::Vector{R} = [ NaN,
                             policy_beliefs[i].states[POLICY_RANGE],
                             WrapToPi(p. 583)( policy_beliefs[i].states[POLICY_THETA] ),
                             WrapToPi(p. 583)( policy_beliefs[i].states[POLICY_PSI] ),
                             policy_beliefs[i].states[POLICY_OWN_SPEED],
                             policy_beliefs[i].states[POLICY_INTR_SPEED],
                             action_prev ]
        offline_table::TRMOfflineTable(p. 525) = TRMOfflineTable(p. 525)( N_allowable_actions,
                                                  minblocks_index, minblocks_table_content,
                                                  equiv_class_table_content, action_pattern_number,
                                                  equiv_class_number, allowable_actions )
        belief_cost::Vector{R} =
            LookupOfflineCost(p. 281)( point, N_actions, b_tau_ec_int, offline_table,
                                 modified_cuts, cut_counts, idx_cut_start,
                                 N_cut_idx, B_use_nearest_neighbor )
        beliefs_best_action[i] = argmin(belief_cost)
        W_belief_weights[i] = policy_beliefs[i].weight
        beliefs_costs[i,:] = belief_cost
        policy_costs = policy_costs + (policy_beliefs[i].weight * belief_cost)
    end
    mean_belief_index::Z = argmax(W_belief_weights)
    offline_action::Z = argmin(policy_costs)
```

```
    for i in 1:length(beliefs_best_action)
        if offline_action == beliefs_best_action[i]
            action_contained = true
        end
    end
    if !action_contained
        policy_costs = beliefs_costs[mean_belief_index,:]
    end
    return policy_costs::Vector{R}
end
```

Referenced In: GetHorizontalPolicyCosts(p. 184)

UPDATEPOLICYSPEEDBINS (Algorithm 180) saves ownship and intruder speed information for subsequent horizontal TRM calls to ensure that maneuver recommendations are robust to surveillance noise for both the maneuver selection policy parameters by calling UPDATEPOLICYSPEEDBININDIVIDUAL (Algorithm 181) for each.

This algorithm takes as inputs *st_own* and *st_int*. This algorithm updates *st_own.is_initialized_speed_bin* and *st_int[i].is_initialized_speed_bin*.

Algorithm 180 UpdatePolicySpeedBins

```
function UpdatePolicySpeedBins( st_own::HTRMOwnState(p. 513), st_int::Vector{HTRMIntruderState(p. 514)} )
    N_horizontal_scaling_options::Z = size(params().horizontal_trm.horizontal_offline.horizontal_-
        scaling.values, 1)
    for idx_scale::Z in 1:N_horizontal_scaling_options
        st_own.speed_bin_prev_policy[idx_scale] =
            UpdatePolicySpeedBinIndividual(p. 192)(st_own, st_int[1], idx_scale, POLICY_OWN_SPEED )
    end
    st_own.is_initialized_speed_bin = true
    for i::Z in 1:length( st_int )
        st_int[i].speed_bin_prev_policy =
            UpdatePolicySpeedBinIndividual(p. 192)(st_own, st_int[i], st_int[i].idx_scale, POLICY_INTR_SPEED )
        st_int[i].is_initialized_speed_bin = true
    end
end
```

Referenced In: SetHorizontalTRMSurvOnlyMode(p. 169), HorizontalTRMUpdate(p. 168)

UPDATEPOLICYSPEEDBININDIVIDUAL (Algorithm 181) takes in the specified intruder scaling index (*idx_scale*) and updates the specified intruder's speed bins using the selected scaled set. CREATEPOLICYSTATEBELIEFS (Algorithm 175) is used to calculate the set of belief states in the policy coordinate system and SELECTANDMODIFYPOLICYBINS (Algorithm 177) is used to get the edges of the look-up table. UPDATEPOLICYSPEEDBININDIVIDUAL returns nearest index to the mean index of all belief states.

This algorithm takes as inputs *st_own*, *st_int*, *idx_scale*, *idx_speed*, and *use_bands*. This algorithm returns *speed_bin_integer*.

Algorithm 181 UpdatePolicySpeedBinIndividual

```
function UpdatePolicySpeedBinIndividual( st_own::HTRMOwnState(p. 513), st_int::HTRMIntruderState(p. 514),
                                         idx_scale::Z, idx_speed::Z )
    policy_beliefs::Vector{PolicyStateBelief(p. 521)} =
            CreatePolicyStateBeliefs(p. 186)( st_own.enu_beliefs, st_int.enu_beliefs, idx_scale )
    (cuts::Vector{R}, cut_counts::Vector{Z}) =
        SelectAndModifyPolicyBins(p. 188)( st_own, st_int, idx_scale )
    speed_bin::R  = 0.0
    for b_policy::PolicyStateBelief(p. 521) in policy_beliefs
        (indices::Vector{Z}, weights::Vector{R}) =
            DiscretizePolicyState(p. 192)( b_policy, cuts, cut_counts )
        speed_bin  = speed_bin  + (indices[idx_speed]  * weights[idx_speed])
    end
     if isnan(speed_bin)
         speed_bin = 1
     end
    speed_bin_integer::Z = nanmax(p. 584)( 1, round( speed_bin ) )
    return speed_bin_integer::Z
end
```

Referenced In: UpdatePolicySpeedBins(p. 191)

DISCRETIZEPOLICYSTATE (Algorithm 182) returns the policy indices and policy weights obtained from a policy table look-up based on the policy state beliefs.

This algorithm takes as inputs *b_policy*, *cuts*, and *cut_counts*. This algorithm returns *policy_-indices* and *policy_weights*.

Algorithm 182 DiscretizePolicyState

```
function DiscretizePolicyState( b_policy::PolicyStateBelief(p. 521), cuts::Vector{R}, cut_counts::Vector{Z} )
    N_policy_states = length( cut_counts )
    N_indices::Vector{Z}         = Z[1]
    W_weights::Vector{R}         = R[1.0]
    N_offset::Z                  = 1
    B_use_nearest_neighbor::Bool = true
    policy_indices::Vector{Z} = Vector{Z}( undef, N_policy_states )
    policy_weights::Vector{R} = Vector{R}( undef, N_policy_states )
    idx_cut_start::Z = 1
    for idx_state::Z in 1:N_policy_states
        idx_cut_end::Z = idx_cut_start + cut_counts[idx_state] - 1
        (indices::Vector{Z}, weights::Vector{R}) =
            InterpolateOneDimension(p. 418)( N_indices, W_weights, b_policy.states[idx_state],
                                     cuts[idx_cut_start:idx_cut_end], N_offset,
                                     B_use_nearest_neighbor )
        policy_indices[idx_state] = indices[1]
        policy_weights[idx_state] = b_policy.weight
        idx_cut_start  = idx_cut_end + 1
    end
    return (policy_indices::Vector{Z}, policy_weights::Vector{R})
end
```

Referenced In: ApplyStateDependentOnlineCosts(p. 193), UpdatePolicySpeedBinIndividual(p. 192)

3.2.4 Online Horizontal Cost Estimation

Online costs can be categorized generally into two types, online costs that depend on the geometry between the two aircraft, termed state dependent, and online costs that do not depend on the geometry.

The only state-dependent online cost is calculated in STATEDEPENDENTCROSSINGCOSTFACTORS (Algorithm 184). The online crossing cost factor is designed to reduce some operational effects of policy discretization by slightly encouraging ownship to turn behind an intruder aircraft under certain circumstances.

3.2.4.1 State Dependent Online Costs

APPLYSTATEDEPENDENTONLINECOSTS (Algorithm 183) applies online costs that are dependent on the geometry between two aircraft.

After determining the mean states, STATEDEPENDENTCROSSINGCOSTFACTORS is called to calculate the online cost factors which are then applied to the offline costs.

This algorithm takes as inputs *policy_beliefs*, *offline_costs*, and *idx_scale*. This algorithm returns *costs*.

Algorithm 183 ApplyStateDependentOnlineCosts

```
function ApplyStateDependentOnlineCosts( policy_beliefs::Vector{PolicyStateBelief(p. 521)},
                                         offline_costs::Vector{R}, idx_scale::Z )
    N_actions::Z  = params().turn_actions.num_actions
    (cuts::Vector{R}, cut_counts::Vector{Z}) = SelectScaledPolicyBins(p. 185)( idx_scale )
    N_policy_states::Z = length( cut_counts )
    mean_idx::Z   = 0
    max_weight::R = 0.0
    mean_state::PolicyStateBelief(p. 521) = PolicyStateBelief(p. 521)()
    mean_state.states = zeros( R, N_policy_states )
    mean_state.weight = 1.0
    for idx::Z in 1:length( policy_beliefs )
        if (max_weight < policy_beliefs[idx].weight)
            mean_idx   = idx
            max_weight = policy_beliefs[idx].weight
            mean_state.states = policy_beliefs[idx].states
        end
    end
    (mean_indices::Vector{Z}, _) =
        DiscretizePolicyState(p. 192)( mean_state, cuts, cut_counts )
    crossing_cost_factors::Vector{R} =
        StateDependentCrossingCostFactors(p. 195)( mean_state.states, mean_indices, cuts, cut_counts )
    costs::Vector{R} = zeros( R, N_actions )
    for i::Z in 1:N_actions
        costs[i] = offline_costs[i] * crossing_cost_factors[i]
    end
    return costs::Vector{R}
end
```

Referenced In: GetHorizontalPolicyCosts(p. 184)

STATEDEPENDENTCROSSINGCOSTFACTORS (Algorithm 184) returns the state dependent crossing cost factor (*cost_factor*) for each of the horizontal advisory actions. In certain circumstances, this cost factor increase the likelihood of a "turn behind" horizontal advisory. The crossing cost fac-

tor consists of components that relate to the aircraft track angles (*psi_factor*), speeds(*speed_factor*), vertical separation (*vertical_tau_factor*), and intruder bearing (*theta_factor*) each of which are calculated through calls to specific subroutines. These subroutines, CALCULATECROSSINGCOSTPSIFACTOR, CALCULATECROSSINGCOSTSPEEDFACTOR, and CALCULATECROSSINGCOSTVERTICALTAUFACTOR all return a single cost factor and CALCULATECROSSINGCOSTTHETAFACTOR returns the *theta_factor* and flags encouraging left or right turns.

The aggregate of these cost factor components *scale_factor*, returns a value greater than 1 which is applied to the COC action and actions which do not turn behind the intruder (indicated by *encourage_right* and *encourage_left*).

When the state dependent crossing factor is applied to the offline cost from the policy table, it reduces some operational effects of policy discretization which made the policy table biased towards having ownship turn parallel to an intruder when encountering an intruder which is crossing ownships trajectory.

STATEDEPENDENTCROSSINGCOSTFACTORS takes as inputs *mean_state*, *mean_indices*, *cuts*, and *cut_counts*. STATEDEPENDENTCROSSINGCOSTFACTORS returns *cost_factor*.

Algorithm 184 StateDependentCrossingCostFactors

```
function StateDependentCrossingCostFactors( mean_state::Vector{R}, mean_indices::Vector{Z}, cuts::Vector{R},
    cut_counts::Vector{Z} )
    X_scale_factor::R     = params().horizontal_trm.horizontal_online.h_crossing.X_scale_factor
    X_exponential_scale_factor::R = params().horizontal_trm.horizontal_online.h_crossing.X_exponential_-
        scale_factor
    N_actions::Z            = params().turn_actions.num_actions
    R_turn_rates::Vector{R} = params().turn_actions.turn_rates
    cost_factor::Vector{R} = ones( R, N_actions )
    psi::R          = mean_state[POLICY_PSI]
    own_speed::R    = mean_state[POLICY_OWN_SPEED]
    intr_speed::R   = mean_state[POLICY_INTR_SPEED]
    vertical_tau::R = mean_state[POLICY_VERTICAL_TAU]
    psi_factor::R = CalculateCrossingCostPsiFactor(p. 196)(psi)
    speed_factor::R =
        CalculateCrossingCostSpeedFactor(p. 196)(own_speed, intr_speed, mean_indices, cut_counts, cuts)
    (theta_factor::R, encourage_left::Bool, encourage_right::Bool) =
        CalculateCrossingCostThetaFactor(p. 197)(own_speed, intr_speed, mean_indices, cut_counts, cuts, psi)
    vertical_tau_factor::R = CalculateCrossingCostVerticalTauFactor(p. 198)(cut_counts, cuts, vertical_tau)
    scale_factor::R = 1.0 + X_scale_factor * (theta_factor^X_exponential_scale_factor) * speed_factor *
        psi_factor * vertical_tau_factor
    if encourage_right
        for i in 1:N_actions
            if (COC == i)
                cost_factor[i] = scale_factor
            elseif (R_turn_rates[i] < 0)
                cost_factor[i] = scale_factor
            end
        end
    end
    if encourage_left
        for i in 1:N_actions
            if (COC == i)
                cost_factor[i] = scale_factor
            elseif (0 < R_turn_rates[i])
                cost_factor[i] = scale_factor
            end
        end
    end
    return cost_factor::Vector{R}
end
```

Referenced In: ApplyStateDependentOnlineCosts(p. 193)

CALCULATECROSSINGCOSTPSIFACTOR (Algorithm 185) calculates a scale factor based on the difference between ownship and intruder track angles. When ownship and intruder track angles are close to the same or opposite (0 or 180°) the scale factor is zero and a the scale factor reaches a maximum of 1.0 when the track angle difference is 90°.

CALCULATECROSSINGCOSTPSIFACTOR takes as input *psi*. CALCULATECROSSINGCOSTPSIFACTOR returns *psi_factor*.

Algorithm 185 CalculateCrossingCostPsiFactor

```
function CalculateCrossingCostPsiFactor(psi::R)
    X_crossing_psi_min::R = params().horizontal_trm.horizontal_online.h_crossing.X_crossing_psi_min
    psi_factor::R = 0.0
    psi_f_zero::R = abs(AngleDifference(p. 580)(       0.0, abs(psi) ))
    psi_f_pi::R   = abs(AngleDifference(p. 580)( float(pi), abs(psi) ))
    if (psi_f_pi < X_crossing_psi_min)
        psi_f_pi = 0.0
    end
    if (psi_f_zero < X_crossing_psi_min)
        psi_f_zero = 0.0
    end
    if (psi_f_pi < psi_f_zero)
        psi_factor = psi_f_pi / (pi/2.0)
    else
        psi_factor = psi_f_zero / (pi/2.0)
    end
    if (1.0 < psi_factor)
        psi_factor = 1.0
    elseif (psi_factor < 0.0)
        psi_factor = 0.0
    end
    return psi_factor::R
end
```

Referenced In: StateDependentCrossingCostFactors(p. 195)

CALCULATECROSSINGCOSTSPEEDFACTOR (Algorithm 186) returns the a scale factor based on the difference between the current ownship and intruder speed and the speed entries in the cost table (DETERMINESTATEDEPENDENTSPEEDFACTOR (Algorithm 189).

CALCULATECROSSINGCOSTSPEEDFACTOR takes as inputs *own_speed*, *intr_speed*, *mean_indices*, *cut_counts*, and *cuts*. CALCULATECROSSINGCOSTSPEEDFACTOR returns *speed_factor*.

Algorithm 186 CalculateCrossingCostSpeedFactor

```
function CalculateCrossingCostSpeedFactor(own_speed::R, intr_speed::R, mean_indices::Vector{Z}, cut_counts::
    Vector{Z}, cuts::Vector{R})
    own_speed_factor::R =
        DetermineStateDependentSpeedFactor(p. 198)( own_speed, POLICY_OWN_SPEED, mean_indices, cut_counts, cuts )
    intr_speed_factor::R =
        DetermineStateDependentSpeedFactor(p. 198)( intr_speed, POLICY_INTR_SPEED, mean_indices, cut_counts, cuts
            )
    speed_factor::R = max( own_speed_factor, intr_speed_factor )
    if (1.0 < speed_factor)
        speed_factor = 1.0
    end
    return speed_factor::R
end
```

Referenced In: StateDependentCrossingCostFactors(p. 195)

CALCULATECROSSINGCOSTTHETAFACTOR (Algorithm 187) calculates a scale factor which is based on the difference between current encounter geometry and the bearing entries in the cost table. If the closure angle (theta) is not represented well by the discretized table, a factor is returned so that the airraft is encourage to turn behind the other aircraft. The function also uses the en-

counter geometry to determine if it is appropriate to encourage right of left turns based on the constant closure angle for a zero horizontal miss distance encounter with the same speeds and track angles (*constant_theta*). The maximum speed factor that is returned by the algorithm is 1.0.

CALCULATECROSSINGCOSTTHETAFACTOR takes as inputs *own_speed*, *intr_speed*, *mean_indices*, *cut_counts*, *cuts*, and *psi*. CALCULATECROSSINGCOSTTHETAFACTOR returns *theta_factor*, *encourage_left*, and *encourage_right*.

Algorithm 187 CalculateCrossingCostThetaFactor

```
function CalculateCrossingCostThetaFactor(own_speed::R, intr_speed::R, mean_indices::Vector{Z},
                                          cut_counts::Vector{Z}, cuts::Vector{R}, psi::R)
    X_theta_tolerance::R  = params().horizontal_trm.horizontal_online.h_crossing.X_theta_tolerance
    X_theta_diff_max::R   = params().horizontal_trm.horizontal_online.h_crossing.X_theta_diff_max
    constant_theta::R = atan( -intr_speed * sin(psi), own_speed - (intr_speed * cos(psi)) )
    bin_idx::Z        = mean_indices[POLICY_THETA]
    offset::Z         = sum( cut_counts[1:(POLICY_THETA - 1)] )
    theta_diff::R  = AngleDifference(p.580)( constant_theta, cuts[offset+bin_idx] )
    encourage_right::Bool = false
    if (theta_diff < X_theta_tolerance) && (abs(theta_diff) < X_theta_diff_max) && (0 <= constant_theta)
        encourage_right = true
    end
    encourage_left::Bool = false
    if (-X_theta_tolerance <= theta_diff) && (abs(theta_diff) < X_theta_diff_max) && (constant_theta < 0)
        encourage_left = true
    end
    theta_factor::R = abs(AngleDifference(p.580)( X_theta_diff_max, abs(theta_diff) )) / X_theta_diff_max
    if !encourage_right && !encourage_left
        theta_factor = 0.0
    elseif (1.0 < theta_factor)
        theta_factor = 1.0
    elseif (theta_factor < 0.0)
        theta_factor = 0.0
    end
    return (theta_factor::R, encourage_left::Bool, encourage_right::Bool)
end
```

Referenced In: StateDependentCrossingCostFactors(p.195)

CALCULATECROSSINGCOSTVERTICALTAUFACTOR (Algorithm 188) calculates a scale factor based on the mean vertical tau. When vertical tau is large (aircraft are vertically separated or converging very slowly), the *vertical_tau_factor* is 0, otherwise *vertical_tau_factor* increases to a maximum of 1.0 (when the aircraft are not separated vertically).

CALCULATECROSSINGCOSTVERTICALTAUFACTOR takes as inputs *cut_counts*, *cuts*, and *vertical_tau*. CALCULATECROSSINGCOSTVERTICALTAUFACTOR returns *vertical_tau_factor*.

Algorithm 188 CalculateCrossingCostVerticalTauFactor

```
function CalculateCrossingCostVerticalTauFactor(cut_counts::Vector{Z}, cuts::Vector{R}, vertical_tau::R)
    offset = sum( cut_counts[1:(POLICY_VERTICAL_TAU - 1)] )
    max_vertical_tau::R = cuts[offset+cut_counts[POLICY_VERTICAL_TAU]]
    vertical_tau_factor::R = 1.0;
    if (max_vertical_tau > 0)
        vertical_tau_factor = (max_vertical_tau - vertical_tau) / max_vertical_tau
        if (vertical_tau_factor < 0.0)
            vertical_tau_factor = 0.0
        end
    end
    return vertical_tau_factor::R
end
```

Referenced In: StateDependentCrossingCostFactors(p. 195)

DETERMINESTATEDEPENDENTSPEEDFACTOR (Algorithm 189) returns the speed factor based on the input speed and table discretization information. The speed factor is a maximum of 1.0 when the current speed is far from the nearest speed entry in the cost table, and reduces to 0 as the current speed nears the closest speed entry in the cost table.

This algorithm takes as inputs *speed*, *speed_idx*, *mean_indices*, *cut_counts*, and *cuts*. This algorithm returns *speed_factor*.

Algorithm 189 DetermineStateDependentSpeedFactor

```
function DetermineStateDependentSpeedFactor( speed::R, speed_idx::Z, mean_indices::Vector{Z},
                                             cut_counts::Vector{Z}, cuts::Vector{R} )
    N_cuts::Z = cut_counts[speed_idx]
    speed_factor::R = 0.0
    offset::Z   = sum( cut_counts[1:(speed_idx - 1)] )
    idx_bin1::Z = mean_indices[speed_idx]
    idx_bin2::Z = idx_bin1
    if (idx_bin1 == 1) && (speed <= cuts[offset+idx_bin1])
        idx_bin2 = 2
    elseif (idx_bin1 == N_cuts) && (cuts[offset+idx_bin1] < speed)
        idx_bin2 = N_cuts - 1
    elseif (cuts[offset+idx_bin1] < speed)
        idx_bin2 = idx_bin1 + 1
    else
        idx_bin2 = idx_bin1 - 1
    end
    speed_factor = abs(speed - cuts[offset+idx_bin1]) /
                   (abs(cuts[offset+idx_bin1] - cuts[offset+idx_bin2]) * 0.5)
    return speed_factor::R
end
```

Referenced In: CalculateCrossingCostSpeedFactor(p. 196)

3.2.4.2 State Independent Online Costs

APPLYHORIZONTALONLINECOSTS (Algorithm 190) returns a vector of costs for each of the horizontal advisory actions. It contains the result of applying online costs to the input vector of fused cost (*fused_cost*). The input vector *highest_threat* contains the intruder id of the greatest threat associated with each of the horizontal advisory actions.

ApplyHorizontalOnlineCosts applies a sequence of online costs for each of the horizontal advisory actions. The horizontal advisory actions are passed to each algorithm as a turn rate. GetPerformanceBasedParams uses the ownship maneuverability reported through /namerefalgReceiveDiscretes to extract the set costs for HorizontalDesensitivityCost and HorizontalTurnRateSensitivityCostFactor. Note that while most of the costs are added, six costs are multipliers: HorizontalWeakeningCostFactor, HorizontalTurnRateSensitivityCostFactor, HorizontalMaintainHeadingIncentive, HorizontalTurnFromMaintainIncentive, HorizontalMultiThreatMaintainHeadingCost, and HorizontalMultiThreatTurnFromMaintainCost. Because of that, the order in which the costs are applied is important. The order of appearance is defined by ApplyHorizontalOnlineCosts. Thus the order, specifically of multiplicative costs in relation to additive costs, cannot be switched.

Non-state dependent online costs include:

- HorizontalReversalCost (Algorithm 192)
  - Discourages horizontal reversals.
- HorizontalWeakeningCostFactor (Algorithm 193)
  - Reduces horizontal RA changes and splits.
- HorizontalInitialCACost (Algorithm 194)
  - Discourages horizontal RA when current action is COC.
- HorizontalDesensitivityCost (Algorithm 195)
  - Discourages the selection of an RA.
- HorizontalTurnRateSensitivityCostFactor (Algorithm 196)
  - Prevents certain advisories when turn rate is limited.
- HorizontalCoordinationCost (Algorithm 197)
  - Penalizes turn RAs that are not compliant with an HRC received by the master aircraft.
- HorizontalCoordinatedReversalCost (Algorithm 199)
  - Penalizes RAs that are not compliant with received HRC messages from intruders that are currently threats.
- HorizontalMultiThreatReversalCost (Algorithm 198)
  - Reduces the reversal penalty when ownship is a multithreat scenario.
- HorizontalInitialMaintainCost (Algorithm 201)
  - Discourages the initial horizontal maintain.
- HorizontalMaintainHeadingIncentive (Algorithm 202)
  - Incentivizes horizontal maintains from turns.
- HorizontalTurnFromMaintainIncentive (Algorithm 203)
  - Incentivizes turns from horizontal maintains.
- HorizontalMultiThreatMaintainHeadingCost (Algorithm 204)
  - Discourages horizontal maintains from turns in multi-threat scenarios.
- HorizontalMultiThreatTurnFromMaintainCost (Algorithm 205)
  - Incentivizes turns from horizontal maintains in multi-threat scenarios.

This algorithm takes as inputs *fused_cost*, *turn_rate_prev*, *turn_rate_limit*, *vert_rate_limit*, *highest_threat*, *highest_threat_unconditioned*, *highest_threat_prev*, *prioritized_intruders*, *is_turn_recommended_prev*, *num_reversals*, and *multithreat_turn_from_maintain_hold*. This algorithm returns *cost*.

Algorithm 190 ApplyHorizontalOnlineCosts

```
function ApplyHorizontalOnlineCosts( fused_cost::Vector{R}, turn_rate_prev::R, turn_rate_limit::R,
     vert_rate_limit::R, highest_threat::Vector{UInt32}, highest_threat_unconditioned::Vector{UInt32},
     highest_threat_prev::Vector{UInt32}, prioritized_intruders::Vector{HTRMIntruderState(p. 514)},
     is_turn_recommended_prev::Bool, num_reversals::UInt32, multihreat_turn_from_maintain_hold::Bool )
    N_actions::Z = params().turn_actions.num_actions
    cost::Vector{R} = copy( fused_cost )
    inc_cost::R = 0.0
    (C_direct_sensitivity_factor::R, C_desensitivity::R) = GetPerformanceBasedParams(p. 201)( vert_rate_limit,
        turn_rate_limit )
    for act::Z in 1:N_actions
        turn_rate::R = HorizontalActionToRate(p. 413)( act )
        inc_cost = HorizontalReversalCost(p. 202)( turn_rate, turn_rate_prev )
        cost[act] = cost[act] + inc_cost
        inc_cost = HorizontalWeakeningCostFactor(p. 202)( turn_rate, turn_rate_prev )
        cost[act] = cost[act] * inc_cost
        inc_cost = HorizontalInitialCACost(p. 202)( turn_rate, turn_rate_prev )
        cost[act] = cost[act] + inc_cost
        inc_cost = HorizontalDesensitivityCost(p. 203)( turn_rate, turn_rate_prev, C_desensitivity )
        cost[act] = cost[act] + inc_cost
        inc_cost = HorizontalTurnRateSensitivityCostFactor(p. 203)( turn_rate, turn_rate_prev,
            C_direct_sensitivity_factor, C_desensitivity )
        cost[act] = cost[act] * inc_cost
        inc_cost = HorizontalCoordinationCost(p. 204)( turn_rate, highest_threat[act], prioritized_intruders,
            is_turn_recommended_prev )
        cost[act] = cost[act] + inc_cost
        inc_cost = HorizontalMultiThreatReversalCost(p. 205)( turn_rate, turn_rate_prev, highest_threat,
            highest_threat_prev )
        cost[act] = cost[act] + inc_cost
        inc_cost = HorizontalCoordinatedReversalCost(p. 206)( turn_rate, turn_rate_prev, act,
            highest_threat_unconditioned, prioritized_intruders, num_reversals )
        cost[act] = cost[act] + inc_cost
        inc_cost = HorizontalInitialMaintainCost(p. 207)( turn_rate, turn_rate_prev )
        cost[act] = cost[act] + inc_cost
        inc_cost = HorizontalMaintainHeadingIncentive(p. 207)( turn_rate, turn_rate_prev )
        cost[act] = cost[act] * inc_cost
        inc_cost = HorizontalTurnFromMaintainIncentive(p. 207)( turn_rate, turn_rate_prev )
        cost[act] = cost[act] * inc_cost
        inc_cost = HorizontalMultiThreatMaintainHeadingCost(p. 208)( turn_rate, turn_rate_prev, highest_threat,
            highest_threat_prev )
        cost[act] = cost[act] * inc_cost
        inc_cost = HorizontalMultiThreatTurnFromMaintainCost(p. 209)( turn_rate, turn_rate_prev, highest_threat,
            highest_threat_prev, multihreat_turn_from_maintain_hold )
        cost[act] = cost[act] * inc_cost
    end
    return cost::Vector{R}
end
```

Referenced In: SelectHorizontalAlertLookaheadAction(p. 216)

GETPERFORMANCEBASEDPARAMS (Algorithm 191) selects the set of costs for use based on ownship maneuverability, as indicated by the vertical rate limit (*vert_rate_limit_own*) and turn rate limit (*turn_rate_limit_own*) inputs. The *performance_params* matrix is the table of performance parameters, found in the Appendix E.3. Each row in the *performance_params* matrix is an entry that contains the maximum vertical rate capability (column 1), the maximum turn rate capabil-

ity (column 2), the direct sensitivity factor (which sensitizes the logic, and is located in column 3), and the desensitivity factor (which desensitizes the logic, and is located in column 4), respectively. The *best_idx* is the index of the entry chosen from the *performance_params* matrix. The value of *best_idx* is initialized to 1. The variables *idx_max_vert_rate* and *idx_max_turn_rate* are the indices of the columns corresponding to the maximum vertical rate capability and the maximum turn rate capability, respectively. The algorithm sequentially goes through the table of performance parameters to identify which entry contains the combination of vertical and turn rates that most closely match the limits provided through the ownship discretes interface. The set of costs are chosen for the resulting entry.

GETPERFORMANCEBASEDPARAMS takes as input *vert_rate_limit_own* and *turn_rate_limit_own*. This algorithm returns *C_direct_sensitivity_factor* and *C_desensitivity*.

Algorithm 191 GetPerformanceBasedParams

```
function GetPerformanceBasedParams( vert_rate_limit_own::R, turn_rate_limit_own::R )
    performance_params::Matrix{R} = params().trm_performance.surv_params
    best_idx::Z = 1
    idx_max_vert_rate::Z = 1
    idx_max_turn_rate::Z = 2
    param_entry::Vector{R} = collect(performance_params[best_idx, :])
    min_dz_diff::R         = abs(vert_rate_limit_own) - abs(param_entry[idx_max_vert_rate])
    min_turn_rate_diff::R  = abs(turn_rate_limit_own) - abs(param_entry[idx_max_turn_rate])
    for idx in 2:size( performance_params, 1 )
        param_entry       = collect(performance_params[idx, :])
        dz_diff::R        = abs(vert_rate_limit_own) - abs(param_entry[idx_max_vert_rate])
        turn_rate_diff::R = abs(turn_rate_limit_own) - abs(param_entry[idx_max_turn_rate])
        if ( (abs(dz_diff) <= abs(min_dz_diff)) && (abs(turn_rate_diff) < abs(min_turn_rate_diff)) ) ||
           ( (abs(dz_diff) < abs(min_dz_diff)) && (abs(turn_rate_diff) <= abs(min_turn_rate_diff)) )
            min_dz_diff        = dz_diff
            min_turn_rate_diff = turn_rate_diff
            best_idx           = idx
        end
    end
    selected_params::Vector{R} = collect(performance_params[best_idx, :])
    C_direct_sensitivity_factor::R = selected_params[3]
    C_desensitivity::R = selected_params[4]
    return (C_direct_sensitivity_factor::R, C_desensitivity::R)
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200), DetermineHorizontalDesensitivity(p. 233)

HORIZONTALREVERSALCOST (Algorithm 192) compares the current horizontal sense *sense_curr* to the previous horizontal sense *sense_prev* and when they are different returns an online cost to discourage horizontal reversals.

This algorithm takes as inputs *turn_rate* and *turn_rate_prev*. This algorithm returns *cost*.

Algorithm 192 HorizontalReversalCost

```
function HorizontalReversalCost( turn_rate::R, turn_rate_prev::R  )
    C_reversal::R = params().horizontal_trm.horizontal_online.h_reversal.C_reversal
    cost::R    = 0.0
    sense_prev::Symbol = HorizontalRateToSense(p. 414)(turn_rate_prev)
    sense_curr::Symbol = HorizontalRateToSense(p. 414)(turn_rate)
    if IsHorizontalReversal(p. 422)(sense_prev, sense_curr)
        cost = C_reversal
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALWEAKENINGCOSTFACTOR (Algorithm 193) returns an online cost factor *X_weakening_factor* for actions with lower turn rate magnitude than the previously selected action. Specifically, a horizontal weakening will occur when the selected action is COC and previous action is a maintain or turn. This effectively reduces splits and reversals by adding a to the COC action.

This algorithm takes as inputs *turn_rate* and *turn_rate_prev*. This algorithm returns *factor*.

Algorithm 193 HorizontalWeakeningCostFactor

```
function HorizontalWeakeningCostFactor( turn_rate::R, turn_rate_prev::R )
    X_weakening_factor::R = params().horizontal_trm.horizontal_online.h_weakening.X_weakening_factor
    factor::R = 1.0
    if IsHorizontalWeakening(p. 423)( turn_rate, turn_rate_prev )
        factor = X_weakening_factor
    end
    return (factor::R)
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALINITIALCACOST (Algorithm 194) returns an online cost (*C_initial_ca*) to penalize issuance of a collision avoidance advisory (turn rate greater than *R_ca_threshold*) when the previous action is COC.

This algorithm takes as inputs *turn_rate* and *turn_rate_prev*. This algorithm returns *cost*.

Algorithm 194 HorizontalInitialCACost

```
function HorizontalInitialCACost( turn_rate::R, turn_rate_prev::R )
    C_initial_ca::R   = params().horizontal_trm.horizontal_online.h_initial_ca.C_initial_ca
    R_ca_threshold::R = params().horizontal_trm.R_ca_threshold
    cost::R                 = 0.0
    if IsHorizontalCOC(p. 422)(turn_rate_prev) && (R_ca_threshold < abs(turn_rate))
        cost = C_initial_ca
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALDESENSITIVITYCOST (Algorithm 195) discourages the selection of an RA based on ownship maneuverability. If the previous and current actions are COC, the horizontal advisories are desensitized by adding *C_desensitivity* (which is a value less than or equal to zero) to the cost of COC. If the previous action was a maintain and the turn rate has changed to indicate a collision avoidance advisory by exceeding *R_ca_threshold*, the horizontal advisories are desensitized by subtracting *C_desensitivity*, making the selection of a horizontal RA less probable.

This algorithm takes as inputs *turn_rate*, *turn_rate_prev*, and *C_desensitivity*. This algorithm returns *cost*.

Algorithm 195 HorizontalDesensitivityCost

```
function HorizontalDesensitivityCost( turn_rate::R, turn_rate_prev::R, C_desensitivity::R )
    R_ca_threshold::R  = params().horizontal_trm.R_ca_threshold
    cost::R = 0.0
    if IsHorizontalCOC(p. 422)(turn_rate_prev) && IsHorizontalCOC(p. 422)(turn_rate)
        cost = C_desensitivity
    elseif !IsHorizontalCOC(p. 422)(turn_rate_prev) && (abs(turn_rate_prev) < R_ca_threshold) && (abs(turn_rate)
          > R_ca_threshold)
        cost = -C_desensitivity
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALTURNRATESENSITIVITYCOSTFACTOR (Algorithm 196) returns a scale factor to penalize certain advisories when the turn rate is limited. The sensitivity factor is only applied when the desensitivity cost *C_desensitivity* is 0 and the next action being evaluated is horizontally COC. When used, the sensitivity factor is set directly based on platform performance (*C_direct_sensitivity_factor*).

Horizontal TRM sensitivity is increased by increasing the cost of horizontal COC. If the next action being evaluated is horizontal COC, the cost factor is set to the sensitivity factor.

This algorithm takes as inputs *turn_rate*, *turn_rate_prev*, *C_direct_sensitivity_factor*, and *C_desensitivity*. This algorithm returns *cost_factor*.

Algorithm 196 HorizontalTurnRateSensitivityCostFactor

```
function HorizontalTurnRateSensitivityCostFactor( turn_rate::R, turn_rate_prev::R, C_direct_sensitivity_factor::R,
      C_desensitivity::R )
    sensitivity_factor::R = 1.0
    cost_factor::R        = 1.0
    if (C_desensitivity == 0.0)
        if (C_direct_sensitivity_factor != 1.0)
            sensitivity_factor = C_direct_sensitivity_factor
        end
        if IsHorizontalCOC(p. 422)(turn_rate)
            cost_factor = sensitivity_factor
        end
    end
    return cost_factor::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALCOORDINATIONCOST (Algorithm 197) (HRC) returns a cost to heavily penalize turn actions which are not compliant with an HRC received by a master aircraft. The turn action being considered is indicated by *turn_rate*.

The HRC from the highest threat intruder (and no others) is applied for the action under consideration. This distinction is critical to resolving and addressing multiple HRCs, which may be conflicting, received from multiple intruders. To avoid such conflict, only the HRC from the most imminent risk (*highest_threat*) is used for each action. All other incoming HRCs are deferred until an intruder rises in priority. Attempting to use all HRCs simultaneously can cause coordination lock and prevent appropriate responses to imminent collision risks in multithreat encounters.

FINDHIGHESTTHREATINDEX returns the index for the prioritized intruder with an id that matches *highest_threat*. A return value of -1 indicates no match was found.

The algorithm first checks that a highest threat intruder was found and ownship's coordination policy against the intruder is not senior. If true, then it checks that the intruder is master or if a horizontal RA was not issued last cycle. If either case is true, then the coordination penalty cost is considered.

The turn rate is incompatible (and the penalty cost *C_hcoord* is applied) if it is to the right (greater than 0) and the HRC advisory is *:DontTurnRight* or if it is to the left (less than 0) and the HRC advisory is *:DontTurnLeft*.

This algorithm takes as inputs *turn_rate*, *highest_threat*, *prioritized_intruders*, and *is_turn_recommended_prev*. This algorithm returns *cost*.

Algorithm 197 HorizontalCoordinationCost

```
function HorizontalCoordinationCost( turn_rate::R, highest_threat::UInt32,
                                     prioritized_intruders::Vector{HTRMIntruderState(p. 514)},
                                     is_turn_recommended_prev::Bool )
    C_hcoord::R = params().horizontal_trm.horizontal_online.h_coordination.C_hcoord
    cost::R = 0.0
    if !IsHorizontalCOC(p. 422)( turn_rate )
        threat_index::Z = FindHighestThreatIndex(p. 206)( highest_threat, prioritized_intruders )
        if (0 < threat_index) && (prioritized_intruders[threat_index].own_coordination_policy !=
             OWN_COORDINATION_POLICY_SENIOR)
            if prioritized_intruders[threat_index].is_master || !is_turn_recommended_prev
                advisory_int::Symbol = HRCToAdvisory(p. 415)(prioritized_intruders[threat_index].received_hrc)
                if (:DontTurnRight == advisory_int) && (0.0 < turn_rate)
                    cost = C_hcoord
                end
                if (:DontTurnLeft == advisory_int) && (turn_rate < 0.0)
                    cost = C_hcoord
                end
            end
        end
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALMULTITHREATREVERSALCOST (Algorithm 198) determines whether the current threat is different from the previous cycle threat, and if there are multiple unique threats.

When there are multiple threats at the current time step or different threats between the current time step and the previous time step, then *C_multi_threat_reversal_reduction* is applied to any RAs that are a reversal from the previous sense.

This algorithm takes as inputs *turn_rate*, *turn_rate_prev*, *highest_threat*, and *highest_threat_prev*. This algorithm returns *cost*.

Algorithm 198 HorizontalMultiThreatReversalCost

```
function HorizontalMultiThreatReversalCost( turn_rate::R, turn_rate_prev::R, highest_threat::Vector{UInt32},
    highest_threat_prev::Vector{UInt32} )
    C_multi_threat_reversal_reduction::R = params().horizontal_trm.horizontal_online.h_multi_threat_-
        reversal.C_multi_threat_reversal_reduction
    N_actions::Z = params().turn_actions.num_actions
    sense_prev::Symbol = HorizontalRateToSense(p. 414)(turn_rate_prev)
    sense_curr::Symbol = HorizontalRateToSense(p. 414)(turn_rate)
    cost::R = 0.0
    diff_threat_prev::Bool = false
    if !isempty(highest_threat_prev)
        for act in 1:N_actions
            if (highest_threat[act] != highest_threat_prev[act]) && (highest_threat[act] != 0) && (
                highest_threat_prev[act] != 0)
                diff_threat_prev = true
            end
        end
    end
    unique_threat_count::Z = length(unique(highest_threat))
    if any(highest_threat .== 0)
        unique_threat_count = unique_threat_count - 1
    end
    multiple_threats::Bool = ( 1 < unique_threat_count ) || diff_threat_prev
    if !IsHorizontalCOC(p. 422)( turn_rate ) && !IsHorizontalCOC(p. 422)( turn_rate_prev) && IsHorizontalReversal(p. 422)(
        sense_prev, sense_curr) && multiple_threats
        cost = C_multi_threat_reversal_reduction
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALCOORDINATEDREVERSALCOST (Algorithm 199) (HCRC) returns a cost to penalize horizontal geometric reversals by a master aircraft if the slave intruder has sent an HRC. The penalty increases as the number of reversals issued increases.

This algorithm takes as input *turn_rate*, *turn_rate_prev*, *actionInd*, *highest_threat*, *prioritized_intruders*, and *num_reversals*. This algorithm returns *cost*.

Algorithm 199 HorizontalCoordinatedReversalCost

```
function HorizontalCoordinatedReversalCost( turn_rate::R, turn_rate_prev::R, actionInd::Z,
                                            highest_threat::Vector{UInt32},
                                            prioritized_intruders::Vector{HTRMIntruderState(p. 514)},
                                            num_reversals::UInt32 )
    C_hcoordrev::R = params().horizontal_trm.horizontal_online.h_coordination.C_hcoordrev
    cost::R = 0.0
    if !IsHorizontalCOC(p. 422)( turn_rate )
        unique_threat_count::Z = length(unique(highest_threat))
        multiple_threats::Bool = ( 1 < unique_threat_count )
        if !multiple_threats
            threat_index::Z = FindHighestThreatIndex(p. 206)( highest_threat[actionInd], prioritized_intruders )
            if (0 < threat_index) && (!prioritized_intruders[threat_index].is_master)
                advisory_int::Symbol = HRCToAdvisory(p. 415)(prioritized_intruders[threat_index].received_hrc)
                if (:None != advisory_int)
                    sense_prev::Symbol = HorizontalRateToSense(p. 414)(turn_rate_prev)
                    sense_curr::Symbol = HorizontalRateToSense(p. 414)(turn_rate)
                    if IsHorizontalReversal(p. 422)(sense_prev, sense_curr) && !IsHorizontalCOC(p. 422)(turn_rate_prev
                        )
                        cost = C_hcoordrev * ((num_reversals + 1) ^2)
                    end
                end
            end
        end
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

FINDHIGHESTTHREATINDEX (Algorithm 200) returns the index for the prioritized intruder with an id that matches *highest_threat*. A return value of -1 indicates no match was found.

This algorithm takes as inputs *highest_threat* and *prioritized_intruders*. This algorithm returns *threat_index*.

Algorithm 200 FindHighestThreatIndex

```
function FindHighestThreatIndex( highest_threat::UInt32, prioritized_intruders::Vector{HTRMIntruderState(p. 514)}
    )
    threat_index::Z = -1
    for i::Z in 1:length( prioritized_intruders )
        if (prioritized_intruders[i].id == highest_threat)
            threat_index = i
            break
        end
    end
    return threat_index::Z
end
```

Referenced In: HorizontalCoordinatedReversalCost(p. 206), HorizontalCoordinationCost(p. 204)

HORIZONTALINITIALMAINTAINCOST (Algorithm 201) returns an online cost which discourages the initial horizontal maintain alert. This algorithm takes as inputs *turn_rate* and *turn_rate_prev*. This algorithm returns *cost*.

Algorithm 201 HorizontalInitialMaintainCost

```
function HorizontalInitialMaintainCost( turn_rate::R, turn_rate_prev::R  )
    C_initial_maintain::R = params().horizontal_trm.horizontal_online.h_maintain.C_initial_maintain
    cost::R    = 0.0
    sense_curr::Symbol = HorizontalRateToSense(p. 414)(turn_rate)
    if IsHorizontalCOC(p. 422)(turn_rate_prev) && (:Straight == sense_curr)
        cost = C_initial_maintain
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALMAINTAINHEADINGINCENTIVE (Algorithm 202) returns a multiplicative online cost which encourages a horizontal maintain alert if the previous alert was a turn. This online cost was designed to prevent aircraft from being locked into spiraling maneuvers. This algorithm takes as inputs *turn_rate* and *turn_rate_prev*. This algorithm returns *cost*.

Algorithm 202 HorizontalMaintainHeadingIncentive

```
function HorizontalMaintainHeadingIncentive( turn_rate::R, turn_rate_prev::R  )
    C_maintain::R = params().horizontal_trm.horizontal_online.h_maintain.C_maintain_heading_incentive
    cost::R    = 1.0
    sense_prev::Symbol = HorizontalRateToSense(p. 414)(turn_rate_prev)
    sense_curr::Symbol = HorizontalRateToSense(p. 414)(turn_rate)
    if !IsHorizontalCOC(p. 422)(turn_rate_prev) && (:Straight == sense_curr) && (sense_prev != sense_curr)
        cost = C_maintain
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALTURNFROMMAINTAININCENTIVE (Algorithm 203) returns a multiplicative online cost which encourages a turn when the previous alert was a horizontal maintain. This online cost was designed to prevent aircraft from being locked into maintains. This algorithm takes as inputs *turn_rate* and *turn_rate_prev*. This algorithm returns *cost*.

Algorithm 203 HorizontalTurnFromMaintainIncentive

```
function HorizontalTurnFromMaintainIncentive( turn_rate::R, turn_rate_prev::R  )
    C_turn_from_maintain::R = params().horizontal_trm.horizontal_online.h_maintain.C_turn_from_maintain_-
        incentive
    cost::R    = 1.0
    sense_prev::Symbol = HorizontalRateToSense(p. 414)(turn_rate_prev)
    sense_curr::Symbol = HorizontalRateToSense(p. 414)(turn_rate)
    if !IsHorizontalCOC(p. 422)(turn_rate_prev) && (:Straight == sense_prev) && (:Left == sense_curr || :Right
        == sense_curr)
        cost = C_turn_from_maintain
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALMULTITHREATMAINTAINHEADINGCOST (Algorithm 204) returns a multiplicative online cost which discourages a horizontal maintain alert if the previous alert was a turn and there are multiple intruders. This cost was designed to prevent aircraft from selecting maintains when choosing between conflicting turn RAs in a multi-threat scenario. This algorithm takes as inputs *turn_rate* and *turn_rate_prev*, *highest_threat*, and *highest_threat_prev*. This algorithm returns *cost*.

Algorithm 204 HorizontalMultiThreatMaintainHeadingCost

```
function HorizontalMultiThreatMaintainHeadingCost( turn_rate::R, turn_rate_prev::R, highest_threat::Vector{UInt32},
    highest_threat_prev::Vector{UInt32} )
    C_maintain::R = params().horizontal_trm.horizontal_online.h_maintain.C_multithreat_maintain_heading_cost
    N_actions::Z = params().turn_actions.num_actions
    cost::R    = 1.0
    sense_prev::Symbol = HorizontalRateToSense(p. 414)(turn_rate_prev)
    sense_curr::Symbol = HorizontalRateToSense(p. 414)(turn_rate)
    diff_threat_prev::Bool = false
    if !isempty(highest_threat_prev)
        for act in 1:N_actions
            if (highest_threat[act] != highest_threat_prev[act]) && (highest_threat[act] != 0) && (highest_threat_prev[act] != 0)
                diff_threat_prev = true
            end
        end
    end
    unique_threat_count::Z = length(unique(highest_threat))
    if any(highest_threat .== 0)
        unique_threat_count = unique_threat_count - 1
    end
    multiple_threats::Bool = ( 1 < unique_threat_count ) || diff_threat_prev
    if !IsHorizontalCOC(p. 422)(turn_rate_prev) && (:Straight == sense_curr) && (sense_prev != sense_curr) && multiple_threats
        cost = C_maintain
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALMULTITHREATTURNFROMMAINTAINCOST (Algorithm 205) returns a multiplicative online cost which encourages a turn when the previous alert was a horizontal maintain when there are multiple intruders. This cost was designed to prevent aircraft from being locked into maintains in a multi-threat scenario. This algorithm takes as inputs *turn_rate*, *turn_rate_prev*, *highest_threat*, and *highest_threat_prev*, and *multihreat_turn_from_maintain_hold*. This algorithm returns *cost*.

Algorithm 205 HorizontalMultiThreatTurnFromMaintainCost

```
function HorizontalMultiThreatTurnFromMaintainCost( turn_rate::R, turn_rate_prev::R, highest_threat::Vector{
     UInt32}, highest_threat_prev::Vector{UInt32}, multihreat_turn_from_maintain_hold::Bool )
    C_turn_from_maintain::R = params().horizontal_trm.horizontal_online.h_maintain.C_multithreat_turn_from_-
         maintain_cost
    N_actions::Z = params().turn_actions.num_actions
    cost::R    = 1.0
    sense_prev::Symbol = HorizontalRateToSense(p. 414)(turn_rate_prev)
    sense_curr::Symbol = HorizontalRateToSense(p. 414)(turn_rate)
    diff_threat_prev::Bool = false
    if !isempty(highest_threat_prev)
        for act in 1:N_actions
            if (highest_threat[act] != highest_threat_prev[act]) && (highest_threat[act] != 0) && (
                 highest_threat_prev[act] != 0)
                diff_threat_prev = true
            end
        end
    end
    unique_threat_count::Z = length(unique(highest_threat))
    if any(highest_threat .== 0)
        unique_threat_count = unique_threat_count - 1
    end
    multiple_threats::Bool = ( 1 < unique_threat_count ) || diff_threat_prev
    if ( !IsHorizontalCOC(p. 422)(turn_rate_prev) && (:Straight == sense_prev) && (:Left == sense_curr || :Right
          == sense_curr) && (multiple_threats || multihreat_turn_from_maintain_hold) )
        multihreat_turn_from_maintain_hold = true;
        cost = C_turn_from_maintain
    elseif ( :Left == sense_prev || :Right == sense_prev )
        multihreat_turn_from_maintain_hold = false;
    end
    return cost::R
end
```

Referenced In: ApplyHorizontalOnlineCosts(p. 200)

3.2.5 Horizontal Action Selection

SELECTHORIZONTALADVISORY (Algorithm 206) determines when a maneuver is necessary to maintain safe separation and selects an advisory to do so.

First, *selected_advisory* is initialized to no advisory using the HORIZONTALADVISORY data type constructor.

HORIZONTALALERTLOOKAHEAD (Algorithm 208) is used next to determine if a horizontal maneuver is needed, if there is at least one prioritized intruder. It handles multiple simultaneous intruders and provides a robust maneuver recommendation for them by selecting the advisory that best addresses all prioritized threats, accounting for the ownship turn rate and vertical rate limits. The selected advisory provides an estimate of the maneuver final track angle obtained by projecting the intruder and ownship positions forward in time (looking ahead) during maneuver assessment. A list of intruders (*selected_advisory.threat_list_unconditioned*) is maintained in the case where an advisory is issued and no threat intruder is identified using the conventional method in the algorithm. The intruder with highest threat for an action of horizontal COC, the intruder that is most threatening if no maneuver is undertaken, is added to the threat list. Otherwise, if no intruders exist in the list of *prioritized_intruders*, the cost for each action is set to NaN.

Finally, ownship and intruder horizontal TRM state information (*st_own* and *st_int*) is updated

and the advisory track angle is adjusted. If there is no advisory, the track angle in *selected_advisory* is cleared. Otherwise, the track angle is adjusted to be between -π and +π and intruders that contributed to the advisory are flagged. UPDATETARGETTRACKANGLE (Algorithm 211) is used to prevent unnecessary frequent updates to the advisory track angle by applying directional rounding and a set of logic-based checks.

Before returning the selected advisory, UPDATENUMHORIZONTALREVERSALS is called to increment the ownship horizontal reversal counter.

This algorithm takes as inputs *track_angle_own*, *turn_rate_limit*, *vert_rate_limit*, *st_own*, and *prioritized_intruders*. This algorithm returns the selected horizontal action as *selected_advisory*.

Table 33. SelectHorizontalAdvisory Output Variables

Variable	Units	Type	Description
selected_advisory	N/A	HorizontalAdvisory(p. 518)	horizontal maneuver advisory information

Algorithm 206 SelectHorizontalAdvisory

```
function SelectHorizontalAdvisory( track_angle_own::R, turn_rate_limit::R,
                           vert_rate_limit::R, st_own::HTRMOwnState(p. 513),
                           prioritized_intruders::Vector{HTRMIntruderState(p. 514)} )
    N_actions::Z = params().turn_actions.num_actions
    selected_advisory::HorizontalAdvisory(p. 518) = HorizontalAdvisory(p. 518)()
    if (0 < length( prioritized_intruders ))
        selected_advisory =
            HorizontalAlertLookahead(p. 214)( st_own, prioritized_intruders, turn_rate_limit, vert_rate_limit )
        if !IsHorizontalCOC(p. 422)( selected_advisory.turn_rate ) && (length(selected_advisory.threat_list) ==
             0) && (length(selected_advisory.threat_list_unconditioned) > 0)
            push!(selected_advisory.threat_list, selected_advisory.threat_list_unconditioned[COC])
        end
    else
        selected_advisory.costs = fill( NaN, N_actions )
    end
    if !IsHorizontalCOC(p. 422)( selected_advisory.turn_rate )
        selected_advisory.track_angle = WrapToPi(p. 583)( track_angle_own + selected_advisory.track_angle )
        UpdateTargetTrackAngle(p. 219)(track_angle_own, st_own, selected_advisory)
        for st_int in prioritized_intruders
            if in( st_int.id, selected_advisory.threat_list )
                st_int.in_advisory = true
            else
                st_int.in_advisory = false
            end
        end
        st_own.is_advisory_prev = true
    else
        selected_advisory.track_angle = NaN
        st_own.is_advisory_prev   = false
    end
    UpdateNumHorizontalReversals(p. 220)(st_own, selected_advisory)
    return selected_advisory::HorizontalAdvisory(p. 518)
end
```

Referenced In: HorizontalTRMUpdate(p. 168)

SELECTHORIZONTALADVISORYWINDRELATIVE (Algorithm 207) is an implementation using wind relative coordinates of SELECTHORIZONTALADVISORY (Algorithm 206) respectively. The wind relative implementations use ownship airspeed and track angle, *track_angle_own*, to calculate ownship velocity estimates, which are used to calculate the relative velocity of the intruder (instead of ground speed and track angle to determine the velocities).

The algorithm SELECTHORIZONTALADVISORYWINDRELATIVE takes as inputs *track_angle_own*, *turn_rate_limit*, *vert_rate_limit*, *st_own*, and *prioritized_intruders*. This algorithm returns *selected_advisory*.

Algorithm 207 SelectHorizontalAdvisoryWindRelative

```
function SelectHorizontalAdvisoryWindRelative( track_angle_own::R, turn_rate_limit::R, vert_rate_limit::R,
                                               st_own::HTRMOwnState(p. 513),
                                               prioritized_intruders::Vector{HTRMIntruderState(p. 514)} )
    # The track angles used in wind relative are actually heading angles. The usage of track
    # angles is for the sake of consistency throughout the TRM.
    N_actions::Z = params().turn_actions.num_actions
    selected_advisory::HorizontalAdvisory(p. 518) = HorizontalAdvisory(p. 518)( )
    if (0 < length( prioritized_intruders ))
        selected_advisory =
            HorizontalAlertLookahead(p. 214)( st_own, prioritized_intruders, turn_rate_limit, vert_rate_limit )
        if !IsHorizontalCOC(p. 422)(selected_advisory.turn_rate) && (length(selected_advisory.threat_list) == 0)
              && (length(selected_advisory.threat_list_unconditioned) > 0)
            push!(selected_advisory.threat_list, selected_advisory.threat_list_unconditioned[
                selected_advisory.action])
        end
    else
        selected_advisory.costs = fill( NaN, N_actions )
    end
    if !IsHorizontalCOC(p. 422)( selected_advisory.turn_rate )
        selected_advisory.track_angle = WrapToPi(p. 583)( track_angle_own + selected_advisory.track_angle )
        UpdateTargetTrackAngle(p. 219)(track_angle_own, st_own, selected_advisory)
        for st_int in prioritized_intruders
            if in( st_int.id, selected_advisory.threat_list )
                st_int.in_advisory = true
            else
                st_int.in_advisory = false
            end
        end
        st_own.is_advisory_prev = true
    else
        selected_advisory.track_angle = NaN
        st_own.is_advisory_prev = false
    end
    UpdateNumHorizontalReversals(p. 220)(st_own, selected_advisory)
    return selected_advisory::HorizontalAdvisory(p. 518)
end
```

Referenced In: HorizontalTRMUpdate(p. 168)

HORIZONTALALERTLOOKAHEAD (Algorithm 208) determines whether a horizontal maneuver is needed by projecting the maneuver assessment forward in time. It evaluates only the intruders in the list of prioritized intruders.

The horizontal policy contains only information that indicates if a maneuver should be performed at the current time. To provide an estimate of the maneuver final track angle, intruder and ownship positions are projected forward in time during maneuver assessment. This projec-

tion forward in time is termed look-ahead. When an ownship maneuver is recommended during the look-ahead, then the ownship is projected to follow that maneuver. The intruder aircraft are projected forward using their unchanged trajectories.

The *t_coc* variable denotes the duration of the current COC sequence and is used to reduce segmented RAs to safely avoid an intruder aircraft. It is set to *0* whenever a maneuver is recommended. It is incremented by the look-ahead time-step (*T_dt*) when there is no advisory and ownship position is coasting. The acceptable margin of safety is given by *T_max_coc*. Any RAs that are anticipated to have brief COC segments will be treated as a single maneuver and the accumulated track angle target is provided to the operator.

When the acceptable margin of safety is achieved after a sufficient number of projections, the maneuver assessment look-ahead will terminate. The maneuver assessment look-ahead will also terminate when the maximum look-ahead time has been reached.

The number of projections that are sufficient is determined by the minimum look-ahead time (*T_min*). The maximum look-ahead time is given by *T_max*.

Temporary copies of the ownship and intruder beliefs are made so they can be projected forward in time during the look-ahead.

SELECTHORIZONTALALERTLOOKAHEADACTION is used to select a maneuver based on the projected position of the intruders and ownship.

The ownship track angle change recommendation in the advisory (*selected_advisory.track_angle*) is the final target angle anticipated during the look-ahead process. If during the look-ahead, a reversal in horizontal maneuver sense is anticipated (*found_reversal* is TRUE), the accumulation of that target track angle is stopped. Note that the forward projection does not stop if a reversal is anticipated; the selected advisory turn rate will only change at this point if advisory is horizontal COC, if the projected turn rate is smaller in magnitude than the selected advisory turn rate, or if the selected advisory turn rate is within a tolerance (*R_turn_epsilon*) of horizontal COC. This process is used to capture maintain actions. The variable *R_turn_epsilon* is turn rate value used as a threshold to determine whether or not an advisory is a collision avoidance adivsory. In this way, a multi-intruder encounter, for which the logic may anticipate a multi-segment maneuver with several different directions, can be represented to the operator as sequential track angle changes.

If there is no maneuver at this look-ahead time, COASTENUBELIEFBASEDONINDIVIDUALSAMPLES is used to obtain ownship position for the next look-ahead time step, *T_dt* seconds in the future.

Otherwise, if there is a maneuver at this look-ahead time then TURNENUBELIEFBASEDONINDIVIDUALSAMPLES is used to project ownship position forward *T_dt* seconds using the turn rate of the advisory (*selected_advisory.turn_rate*).

Finally, each intruder aircraft position is projected forward *T_dt* seconds using COASTENUBELIEFBASEDONINDIVIDUALSAMPLES.

COASTENUBELIEFBASEDONINDIVIDUALSAMPLES uses a simple straight-flight assumption of the position at the provided time offset.

It is important to clarify that the safety of the ACAS sXu system does not rely on the simple straight-flight assumption used to estimate the maneuver track angle. The offline policy table

determines what action ownship should pursue now (no maneuver necessary, maintain, turn right, or turn left) based on the probabilistic possibilities of the intruder and the ownship aircraft. Thus, the optimized policy table provides the safe and robust maneuver recommendation about what to do now not subject to a straight-flight assumption, and only the target track angle is estimated using simple straight-flight assumption. Therefore, the system can provide reliably robust maneuvers with a useful estimate of the target track angle, and the fact that the intruder or ownship may maneuver differently is captured in the offline policy table.

HORIZONTALALERTLOOKAHEAD takes as inputs *st_own*, *prioritized_intruders*, *turn_rate_limit*, and *vert_rate_limit*. This algorithm returns *selected_advisory*.

Algorithm 208 HorizontalAlertLookahead

```
function HorizontalAlertLookahead( st_own::HTRMOwnState(p. 513), prioritized_intruders::Vector{
    HTRMIntruderState(p. 514)}, turn_rate_limit::R, vert_rate_limit::R )
    T_dt::R      = params().horizontal_trm.htrm_alert_lookahead.T_dt
    T_min::R     = params().horizontal_trm.htrm_alert_lookahead.T_min
    T_max::R     = params().horizontal_trm.htrm_alert_lookahead.T_max
    T_max_coc::R = params().horizontal_trm.htrm_alert_lookahead.T_max_coc
    T_threat_timeout::R = params().horizontal_trm.htrm_alert_lookahead.T_threat_timeout
    R_turn_epsilon::R = params().horizontal_trm.R_turn_epsilon
    t_lookahead::R       = 0.0
    t_coc::R             = 0.0
    t_threat::R          = T_threat_timeout
    found_reversal::Bool = false
    action::Z            = COC
    last_action::Z       = st_own.advisory_prev.action
    selected_advisory::HorizontalAdvisory(p. 518) = HorizontalAdvisory(p. 518)()
    selected_advisory.track_angle = 0.0
    enu_beliefs_own::EnuBeliefSet(p. 517) = deepcopy( st_own.enu_beliefs )
    enu_beliefs_int::Vector{EnuBeliefSet(p. 517)} = EnuBeliefSet(p. 517)[]
    for i in 1:length( prioritized_intruders )
        push!( enu_beliefs_int, deepcopy( prioritized_intruders[i].enu_beliefs ) )
    end
    while (t_coc <= T_max_coc) && (t_lookahead <= T_max) &&
          (st_own.is_advisory_prev || (t_lookahead <= T_min) || (COC != selected_advisory.action))
        t_lookahead = t_lookahead + T_dt
        update_new_threat_list::Bool  = (0 < t_threat)
        update_active_threat_list::Bool = ((t_lookahead < T_min) || (COC != st_own.advisory_prev.action))
        action = SelectHorizontalAlertLookaheadAction(p. 216)( enu_beliefs_own, enu_beliefs_int, st_own,
            prioritized_intruders, last_action, turn_rate_limit, vert_rate_limit, update_active_threat_list,
             update_new_threat_list, selected_advisory )
        if (COC != action) && update_new_threat_list
            t_threat = t_threat - T_dt
        end
        turn_rate::R = HorizontalActionToRate(p. 413)( action )
        sense_action::Symbol = HorizontalRateToSense(p. 414)( turn_rate )
        sense_advisory::Symbol = HorizontalRateToSense(p. 414)( selected_advisory.turn_rate )
        if (COC == action)
            t_coc = t_coc + T_dt
            last_action = action
        elseif !IsHorizontalReversal(p. 422)( sense_advisory,sense_action ) || isapprox( turn_rate *
            selected_advisory.turn_rate, 0.0, atol=R_turn_epsilon )
            t_coc = 0
            if !found_reversal
                selected_advisory.track_angle = WrapToPi(p. 583)( selected_advisory.track_angle + (turn_rate *
                    T_dt) )
            end
            if IsHorizontalCOC(p. 422)( selected_advisory.turn_rate ) ||
               (abs( selected_advisory.turn_rate ) < abs( turn_rate )) && !isapprox( selected_advisory.
                   turn_rate, 0.0, atol=R_turn_epsilon )
                selected_advisory.turn_rate = turn_rate
                selected_advisory.action    = action
            end
            last_action = action
        else
            t_coc          = 0.0
            last_action    = selected_advisory.action
            found_reversal = true
        end
        if IsHorizontalCOC(p. 422)( turn_rate )
           CoastEnuBeliefBasedOnIndividualSamples(p. 411)( enu_beliefs_own, T_dt )
        else
           TurnEnuBeliefBasedOnIndividualSamples(p. 434)( enu_beliefs_own, turn_rate, T_dt )
        end
        for enu_beliefs::EnuBeliefSet(p. 517) in enu_beliefs_int
           CoastEnuBeliefBasedOnIndividualSamples(p. 411)( enu_beliefs, T_dt )
        end
    end
```

```
    return selected_advisory::HorizontalAdvisory(p. 518)
end
```

Referenced In: SelectHorizontalAdvisoryWindRelative(p. 211), SelectHorizontalAdvisory(p. 210)

SELECTHORIZONTALALERTLOOKAHEADACTION (Algorithm 209) determines the best individual action to address all intruders.

First, HORIZONTALCOSTFUSION is used to obtain the vector with the highest cost amongst the prioritized intruders for each possible action.

Next, APPLYHORIZONTALONLINECOSTS is used to apply online cost to the fused offline costs. The threat list *highest_threats_unconditined* (which contains the most threatening intruder for each potential horizontal action) is passed to APPLYHORIZONTALONLINECOSTS instead of *highest_threats* (which only includes intruders for which an active RA is necessary).

The overall action is then determined to be the action that has the minimum cost of these fused actions after online costs have been applied.

Finally, some bookkeeping is done for the look-ahead advisory data structure (*selected_advisory*). The cost values are stored if this is the first time step in the look-ahead. The threat list is updated as needed.

SELECTHORIZONTALALERTLOOKAHEADACTION takes as inputs *enu_beliefs_own*, *enu_beliefs_int*, *st_own*, *prioritized_intruders*, *last_action*, *turn_rate_limit*, *vert_rate_limit*, *update_active_threat_list*, *update_new_threat_list*, and *selected_advisory*. This algorithm returns *action*.

Algorithm 209 SelectHorizontalAlertLookaheadAction

```
function SelectHorizontalAlertLookaheadAction( enu_beliefs_own::EnuBeliefSet(p. 517),
                                               enu_beliefs_int::Vector{EnuBeliefSet(p. 517)},
                                               st_own::HTRMOwnState(p. 513),
                                               prioritized_intruders::Vector{HTRMIntruderState(p. 514)},
                                               last_action::Z, turn_rate_limit::R, vert_rate_limit::R,
                                               update_active_threat_list::Bool, update_new_threat_list::Bool,
                                               selected_advisory::HorizontalAdvisory(p. 518) )
    N_actions::Z = params().turn_actions.num_actions
    (fused_costs::Vector{R}, highest_threat::Vector{UInt32}, highest_threat_unconditioned::Vector{UInt32}) =
        HorizontalCostFusion(p. 217)( enu_beliefs_own, enu_beliefs_int, st_own, prioritized_intruders,
                                      last_action, update_active_threat_list )
    cost::Vector{R} =
        ApplyHorizontalOnlineCosts(p. 200)( fused_costs, st_own.advisory_prev.turn_rate, turn_rate_limit,
                                            vert_rate_limit, highest_threat, highest_threat_unconditioned,
                                            st_own.highest_threat_prev, prioritized_intruders,
                                            st_own.is_turn_recommended_prev, st_own.num_reversals,
                                            st_own.multihreat_turn_from_maintain_hold )
    st_own.highest_threat_prev = copy(highest_threat)
    action::Z = argmin( cost )
    if (0 == length( selected_advisory.costs ))
        selected_advisory.costs = copy( cost )
    end
    if (COC != action) && update_new_threat_list
        for i::Z in 1:N_actions
            active_threat::Bool = false
            if (highest_threat[i] != 0)
                if in( highest_threat[i], st_own.advisory_prev.threat_list )
                    active_threat = true
                end
                if !st_own.is_advisory_prev || (st_own.is_advisory_prev && active_threat)
                    if !in( highest_threat[i], selected_advisory.threat_list )
                        push!( selected_advisory.threat_list, highest_threat[i] )
                    end
                end
            end
        end
    end
    if (COC != action)
        for i::Z in 1:N_actions
            push!( selected_advisory.threat_list_unconditioned, highest_threat_unconditioned[i] )
        end
    end
    return action::Z
end
```

Referenced In: HorizontalAlertLookahead(p. 214)

HORIZONTALCOSTFUSION (Algorithm 210) returns the fused offline policy costs, a vector of intruder IDs indicating which intruder had the highest cost for each action, and a vector of intruder IDs containing the unconditioned highest threats. The fused costs are set to the highest cost amongst the intruders for each possible action.

GETHORIZONTALPOLICYCOSTS is used twice for each intruder. The first time, it is used to obtain the offline costs with no maneuver. That information is used to update the threat list for the active advisory and obtain the initial cost of no advisory. The second time, GETHORIZONTALPOLICYCOSTS is used in determining the fused cost.

To get the fused cost, HORIZONTALCOSTFUSION uses GETHORIZONTALPOLICYCOSTS with the current advisory to look-up the offline cost for each action for an individual intruder. It finds

the action associated with the minimum cost for that intruder. If the minimum cost action is a horizontal RA, the intruder is considered to be a threat. Then, it fuses costs by selecting the maximum cost for each action across all the intruders. The individual intruder is selected as the highest threat for this action if it has the maximum cost and it is a threat. If an intruder was previously in the threat list, an action is still needed, and the worst case intruder has not been identified for that action, then the previously threatening intruder is added to the threat list.

This algorithm takes as inputs *enu_beliefs_own*, *enu_beliefs_int*, *st_own*, *prioritized_intruders*, *last_action*, and *update_active_threat_list*. This algorithm returns *fused_costs*, *highest_threat*, and *highest_threat_unconditioned*.

Algorithm 210 HorizontalCostFusion

```
function HorizontalCostFusion( enu_beliefs_own::EnuBeliefSet(p. 517), enu_beliefs_int::Vector{EnuBeliefSet(p. 517)},
                      st_own::HTRMOwnState(p. 513), prioritized_intruders::Vector{HTRMIntruderState(p. 514)},
                      last_action::Z, update_active_threat_list::Bool )
    N_actions::Z = params().turn_actions.num_actions
    fused_costs::Vector{R}         = -Inf * ones( R, N_actions )
    highest_threat::Vector{UInt32} = zeros( UInt32, N_actions )
    highest_threat_unconditioned::Vector{UInt32} = zeros( UInt32, N_actions )
    for i in 1:length( prioritized_intruders )
        intruder_id::UInt32 = prioritized_intruders[i].id
        costs_indiv::Vector{R} =
            GetHorizontalPolicyCosts(p. 184)( enu_beliefs_own, enu_beliefs_int[i], COC,
                                      true, st_own, prioritized_intruders[i] )
        cost_min_idx::Z = argmin( costs_indiv )
        cost_min_idx_COC::Z = cost_min_idx
        if (COC != cost_min_idx) && update_active_threat_list &&
           !in( intruder_id, st_own.advisory_prev.threat_list )
            push!( st_own.advisory_prev.threat_list, intruder_id )
        end
        if isnan( prioritized_intruders[i].coc_cost )
            prioritized_intruders[i].coc_cost = costs_indiv[COC]
        end
        if (COC != last_action)
            costs_indiv  =
                GetHorizontalPolicyCosts(p. 184)( enu_beliefs_own, enu_beliefs_int[i],
                                          last_action, true, st_own,
                                          prioritized_intruders[i] )
            cost_min_idx = argmin( costs_indiv )
        end
        for act in 1:N_actions
            if (!IsHorizontalCOC(p. 422)(HorizontalActionToRate(p. 413)(last_action)) &&
                in( intruder_id, st_own.advisory_prev.threat_list ) &&
                (cost_min_idx != COC) && (highest_threat[act] == 0))
                highest_threat[act] = intruder_id
            end
            if (fused_costs[act] < costs_indiv[act])
                fused_costs[act] = costs_indiv[act]
                if !IsHorizontalCOC(p. 422)(HorizontalActionToRate(p. 413)(cost_min_idx_COC))
                    highest_threat[act] = intruder_id
                end
                highest_threat_unconditioned[act] = intruder_id
            end
        end
    end
    return (fused_costs::Vector{R}, highest_threat::Vector{UInt32}, highest_threat_unconditioned::Vector{
        UInt32})
end
```

Referenced In: SelectHorizontalAlertLookaheadAction(p. 216)

The UPDATETARGETTRACKANGLE (Algorithm 211) function applies a set of logic-based checks to the ownship target track angle to prevent unnecessary updates. Frequent updates or fluctuations in the target track angle output to display may be distracting to the pilot. The ownship target track angle is first rounded in the direction of the turn rate. If an advisory is continued in the same direction as the previous advisory and the target track angle was changed, the algorithm will decide whether or not to update the target track angle depending on the change in target track angle and the current ownship track angle. If an update is not necessary, it will maintain the previous target track angle until the current ownship track angle is within a threshold of the target track angle. If a reversal advisory is used, hysteresis is not applied and the target track angle is immediately updated.

This algorithm takes *track_angle_own*, *st_own*, and *selected_advisory* as inputs. This algorithm updates *selected_advisory.track_angle*.

Algorithm 211 UpdateTargetTrackAngle

```
function UpdateTargetTrackAngle(track_angle_own::R, st_own::HTRMOwnState(p. 513), selected_advisory::
    HorizontalAdvisory(p. 518))
    target_precision::R       = deg2rad(params().horizontal_trm.update_track_angle.target_precision)
    strengthen_threshold::R   = deg2rad(params().horizontal_trm.update_track_angle.strengthen_threshold)
    initial_track_angle::R    = deg2rad(params().horizontal_trm.update_track_angle.initial_track_angle)
    prev_action_COC::Bool       = (st_own.advisory_prev.action == COC)
    curr_action_COC::Bool       = (selected_advisory.action == COC)
    if !curr_action_COC
        if prev_action_COC && (abs(AngleDifference(p. 580)(selected_advisory.track_angle, track_angle_own)) <
            initial_track_angle)
            selected_advisory.track_angle = WrapToPi(p. 583)(track_angle_own + sign(selected_advisory.turn_rate
                ) * initial_track_angle)
        end
        if (selected_advisory.turn_rate < 0)
            selected_advisory.track_angle = floor(selected_advisory.track_angle / target_precision) *
                target_precision
        elseif (selected_advisory.turn_rate > 0)
            selected_advisory.track_angle = ceil(selected_advisory.track_angle / target_precision) *
                target_precision
        else
            selected_advisory.track_angle = round(selected_advisory.track_angle / target_precision) *
                target_precision
        end
        turn_rates_equal::Bool      = (sign(selected_advisory.turn_rate) == sign(st_own.advisory_prev.
            turn_rate))
        track_angles_equal::Bool    = (st_own.advisory_prev.track_angle == selected_advisory.track_angle)
        is_strengthen::Bool         = !xor((selected_advisory.turn_rate < 0), (AngleDifference(p. 580)(st_own.
            advisory_prev.track_angle, selected_advisory.track_angle) < 0))
        is_outside_threshold::Bool  = (abs(AngleDifference(p. 580)(st_own.advisory_prev.track_angle,
            track_angle_own)) > strengthen_threshold)
        if turn_rates_equal && !prev_action_COC
            if track_angles_equal
                selected_advisory.track_angle = st_own.advisory_prev.track_angle
            elseif is_strengthen
                if is_outside_threshold
                    selected_advisory.track_angle = st_own.advisory_prev.track_angle
                end
            else
                selected_advisory.track_angle = st_own.advisory_prev.track_angle
            end
        end
    end
    return
end
```

Referenced In: SelectHorizontalAdvisoryWindRelative(p. 211), SelectHorizontalAdvisory(p. 210)

UPDATENUMHORIZONTALREVERSALS (Algorithm 212) Counts horizontal RA reversals issued on ownship. The counter is reset any time ownship selects an action that is not collision avoidance strength. The counter is incremented when a reversal between CA strength actions occurs.

This algorithm takes as input *st_own* and *selected_advisory*. This algorithm updates *st_own*.

Algorithm 212 UpdateNumHorizontalReversals

```
function UpdateNumHorizontalReversals(st_own::HTRMOwnState(p. 513) , selected_advisory::HorizontalAdvisory(p. 518))
    if !IsHorizontalCOC(p. 422)(st_own.advisory_prev.turn_rate) && !IsHorizontalCOC(p. 422)(selected_advisory.
        turn_rate) &&
        IsHorizontalReversal(p. 422)(HorizontalRateToSense(p. 414)(st_own.advisory_prev.turn_rate),
            HorizontalRateToSense(p. 414)(selected_advisory.turn_rate))
        st_own.num_reversals += 1
    end
    if IsHorizontalCOC(p. 422)(selected_advisory.turn_rate)
        st_own.num_reversals = 0
    end
end
```

Referenced In: SelectHorizontalAdvisoryWindRelative(p. 211), SelectHorizontalAdvisory(p. 210)

3.2.6 Horizontal Coordination Selection

An overview of the HORIZONTALCOORDINATIONSELECTION component of the Horizontal Threat Resolution Module is shown in Figure 4.

Appendix G describes a provision that enables scaling the offline policy to make it more or less sensitive. Selection of scaling parameters throughout the algorithms in this section is controlled by the variable *idx_scale*. In typical implementations, this variable will always take the value 1. Implementations including the scaling provision permit other values for this variable.

HORIZONTALCOORDINATIONSELECTION (Algorithm 213) calculates and returns the coordination message settings (*hrc*, *chc*, *hsb*) to be sent to the intruder.

It should be noted that ACAS sXu only coordinates with other aircraft equipped with ACAS sXu via vehicle-to-vehicle (V2V) communication.

First, the intruder is evaluated for being a threat. An intruder is identified as a threat for the purposes of issuing an HRC when the following are true:

- There is a horizontal resolution advisory for ownship as indicated by the *turn_rate* strength AND
- The intruder contributed to the advisory, as indicated by *st_int.in_advisory*.

If the intruder is identified as a threat, the sense of the turn advisory to send to the intruder is determined, (*hcoord_sense*) where the sense of the turn is either the same or different direction as the ownship turn advisory. Lastly, the corresponding *hrc*, *chc* and *hsb* are determined to send to the intruder.

GETHORIZONTALCOORDINATIONPOLICY (Algorithm 214) is used to determine if the direction of the turn advisory to send to the intruder should be in the same or different direction, or sense, as the ownship turn advisory.

ENCODEHRC (Algorithm 217) returns the value of *hrc* if the intruder is identified as a threat based on the direction of the ownship turn advisory (*sense_own*), whether the turn advisory for the individual intruder should be in the same direction as ownship (*match_sense_indiv*), and the intruder's equipage (*equip_int*). Otherwise, if the intruder is not a threat, *hrc* defaults to *0*.

ENCODECHC (Algorithm 218) returns the value of *chc* based on *hrc* and the values of HRC and

CHC on the previous cycle (*st_int.hrc_prev* and *st_int.chc_prev*).

ENCODEHSB (Algorithm 219) returns the value of *hsb* based on the values of *hrc* and *chc*.

The current values of HRC and CHC are recorded in *st_int.hrc_prev* and *st_int.chc_prev*, to be used as the previous values in the next cycle. The previous coordination sense is also recorded in *st_int.hcoord_sense_prev*. This information from the previous cycle is used in coordination selection to reduce slave reversals.

This algorithm takes as inputs *prev_turn_rate*, *turn_rate*, *enu_beliefs_own*, *equipage_int*, and *st_int*. This algorithm returns the values in Table 34, HorizontalCoordinationSelection Output Variables.

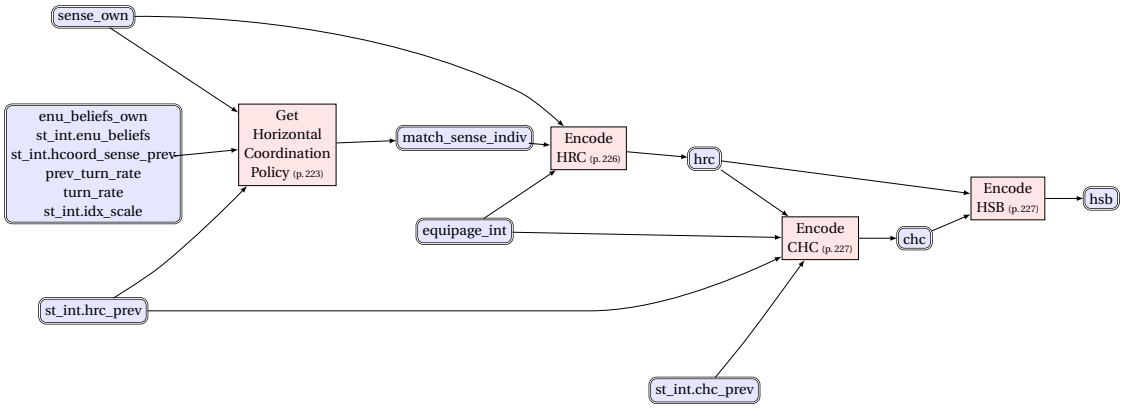

Figure 4. HorizontalCoordinationSelection overview.

Table 34. HorizontalCoordinationSelection Output Variables

Variable	Units	Type	Description
chc	N/A	uint32	Cancel horizontal complement
hrc	N/A	uint32	Horizontal resolution complement
hsb	N/A	uint32	Horizontal sense bits

Algorithm 213 HorizontalCoordinationSelection

```
function HorizontalCoordinationSelection( prev_turn_rate::R, turn_rate::R, enu_beliefs_own::EnuBeliefSet(p. 517),
                                          equipage_int::Z, st_int::HTRMIntruderState(p. 514) )
    chc::UInt32 = 0
    hrc::UInt32 = 0
    hsb::UInt32 = 0
    hcoord_sense::Z = HCOORD_SENSE_NONE
    if !IsHorizontalCOC(p. 422)( turn_rate ) && st_int.in_advisory
        sense_own::Symbol = HorizontalRateToSense(p. 414)( turn_rate )
        match_sense_indiv::Bool =
            GetHorizontalCoordinationPolicy(p. 223)( sense_own, enu_beliefs_own, st_int.enu_beliefs, st_int.
                hrc_prev, st_int.hcoord_sense_prev, prev_turn_rate, turn_rate, st_int.idx_scale )
        hrc = EncodeHRC(p. 226)( sense_own, match_sense_indiv, equipage_int )
        if match_sense_indiv
            hcoord_sense = HCOORD_SENSE_SAME
        else
            hcoord_sense = HCOORD_SENSE_DIFFERENT
        end
    end
    chc = EncodeCHC(p. 227)( hrc, st_int.hrc_prev, st_int.chc_prev, equipage_int )
    hsb = EncodeHSB(p. 227)( hrc, chc )
    st_int.hrc_prev = hrc
    st_int.chc_prev = chc
    st_int.hcoord_sense_prev = hcoord_sense
    return (chc::UInt32, hrc::UInt32, hsb::UInt32)
end
```

Referenced In: SetHorizontalIntruderOutput(p. 235), SetHorizontalDroppedIntruderOutput(p. 236)

GETHORIZONTALCOORDINATIONPOLICY (Algorithm 214) returns whether the direction of the horizontal resolution advisory for the individual intruder should be in the same direction, or sense, as the ownship resolution advisory. The decision is obtained from the horizontal coordination policy table based on the ownship and intruder position and velocity beliefs and the previously sent horizontal coordination sense. The position and velocity beliefs are converted for proper look-up into the horizontal coordination policy table, and an *offset_factor* is applied to look-up to prevent slave reversals based on the previously sent coordination sense. Since the horizontal coordination table does not use previous RA as an input variable like the horizontal cost table, only elements 2 through 6 of the belief states are used in the call to INTERPOLANTS to determine the correct horizontal coordination table indices. For horizontal coordination, it may be desirable to have the intruder turn in the same direction as ownship. In that case, *match_sense_indiv* will be TRUE. Otherwise, if the intruder should turn in the different direction as ownship, *match_sense_indiv* will be FALSE.

GETHORIZONTALCOORDINATIONOFFSETFACTOR (Algorithm 215) determines the *offset_factor* into the horizontal coordination table based on the ownship sense and the previously sent horizontal coordination sense. The *offset_factor* accounts for the previously sent horizontal coordination sense in order to prevent slave reversals.

CREATEPOLICYSTATEBELIEFS is used to convert the ownship and intruder position and velocity beliefs (*enu_beliefs_own* and *enu_beliefs_int*) into the relative coordinate system for the intruder used in the policy table.

INTERPOLANTS is called for each policy state belief to get the offset into the horizontal coordination policy table. The offset is adjusted to look-up the result based on the direction or sense of the ownship turn advisory and the previously sent horizontal coordination sense (*offset_factor*).

The result of the look-up (*match_sense*) is *1* to match the direction and *0* otherwise. The determination of whether the direction of the intruder advisory should match the direction of the ownship advisory is based on the weighted average (*match_thresh*) of those results. The look-up result of *0* is converted to a value of -1 to produce the weighted average.

APPLYHORIZONTALCOORDINATIONHYSTERESIS (Algorithm 216) adjusts the *match_thresh* by applying hysteresis to the horizontal coordination selection to account for surveillance noise.

The direction of the resolution advisory for the individual intruder is in the same direction as ownship (*send_same_sense* is TRUE) if the weighted average has a positive value. Otherwise, *send_same_sense* is FALSE.

This algorithm takes as inputs *sense_own*, *enu_beliefs_own*, *enu_beliefs_int*, *prev_sent_hrc*, *prev_hcoord_sense*, *prev_turn_rate*, *turn_rate*, and *idx_scale*. This algorithm returns *send_same_sense*.

Algorithm 214 GetHorizontalCoordinationPolicy

```
function GetHorizontalCoordinationPolicy( sense_own::Symbol, enu_beliefs_own::EnuBeliefSet(p. 517),
                                          enu_beliefs_int::EnuBeliefSet(p. 517), prev_sent_hrc::UInt32,
                                          prev_hcoord_sense::Z, prev_turn_rate::R, turn_rate::R,
                                          idx_scale::Z )
    hcoord_table_content::PolicyUInt8DataTable(p. 521) =
        params().horizontal_trm.horizontal_offline.hcoord_policy.hcoord_table_content
    cuts::Vector{R}       = hcoord_table_content.cuts
    cut_counts::Vector{Z} = hcoord_table_content.cut_counts
    data::Vector{UInt8}   = hcoord_table_content.data
    N_policy_states::Z    = length( cut_counts )
    B_use_nearest_neighbor::Bool = true
    offset_factor::Z = GetHorizontalCoordinationOffsetFactor(p. 224)(sense_own, prev_hcoord_sense)
    policy_beliefs::Vector{PolicyStateBelief(p. 521)} =
        CreatePolicyStateBeliefs(p. 186)( enu_beliefs_own, enu_beliefs_int, idx_scale )
    match_thresh::R = 0.0
    for b_policy::PolicyStateBelief(p. 521) in policy_beliefs
        (offsets::Vector{Z}, _, block_size::Z) =
            Interpolants(p. 416)( b_policy.states[2:6], cut_counts, 1, N_policy_states, cuts,
                          B_use_nearest_neighbor )
        offset::Z = offsets[1] + (block_size * offset_factor)
        match_sense::UInt8 = data[offset]
        if (1 == match_sense)
            match_thresh = match_thresh + b_policy.weight
        else
            match_thresh = match_thresh - b_policy.weight
        end
    end
    match_thresh = ApplyHorizontalCoordinationHysteresis(p. 225)( match_thresh, prev_sent_hrc, prev_turn_rate,
         turn_rate )
    send_same_sense::Bool = false
    if (match_thresh > 0.0)
        send_same_sense = true
    end
    return send_same_sense::Bool
end
```

Referenced In: HorizontalCoordinationSelection(p. 222)

GETHORIZONTALCOORDINATIONOFFSETFACTOR (Algorithm 215) determines the offset into the horizontal coordination policy table to account for the previously sent coordination sense (*prev_hcoord_sense*) and the current ownship advisory sense (*sense_own*). This offset will result in a look-up into a different index of the policy table with the goal of preventing slave reversals, which

can occur when sending an coordination sense in the opposite direction of the previously sent coordination sense. By performing the look-up with the offset, the policy will favor keeping the coordination sense consistent.

This algorithm takes as inputs *sense_own* and *prev_hcoord_sense*. This algorithm returns *offset_-factor*.

Algorithm 215 GetHorizontalCoordinationOffsetFactor

```
function GetHorizontalCoordinationOffsetFactor( sense_own::Symbol, prev_hcoord_sense::Z )
    offset_factor::Z = 0
    is_left_advisory::Bool = (sense_own == :Left)
    if (prev_hcoord_sense == HCOORD_SENSE_DIFFERENT)
        if (is_left_advisory)
            offset_factor = 0
        else
            offset_factor = 1
        end
    elseif (prev_hcoord_sense == HCOORD_SENSE_SAME)
        if (is_left_advisory)
            offset_factor = 2
        else
            offset_factor = 3
        end
    else
        if (is_left_advisory)
            offset_factor = 4
        else
            offset_factor = 5
        end
    end
    return offset_factor::Z
end
```

Referenced In: GetHorizontalCoordinationPolicy(p. 223)

APPLYHORIZONTALCOORDINATIONHYSTERESIS (Algorithm 216) adjusts the threshold used to determine if the coordination senses should match (*match_thresh*) based on the previously sent HRC (*prev_sent_hrc*), previous turn rate advisory (*prev_turn_rate*), and current turn rate advisory (*turn_rate*). This threshold adjustment will only apply if an HRC was sent in the previous cycle (i.e. non-zero) and if the previous ownship advisory and current advisory is not a reversal, as determined by ISHORIZONTALREVERSAL.

If the previous and current ownship advisory is not a reversal, then the specified coordination hysteresis value will be applied when checking if *match_thresh* should be set to force continuation of the same sense or opposite sense. The hysteresis is applied such that it will force continuation unless all the policy beliefs weights suggest changing the coordination sense, as determined in GETHORIZONTALCOORDINATIONPOLICY. If the previous coordination look-up was the same sense, *match_thresh* will be set to *1* to force continuation of the same sense. Otherwise, if the previous coordination look-up had the opposite sense, *match_thresh* will be set to -1 to force continuation of the opposite sense.

This algorithm takes as inputs *match_thresh*, *prev_sent_hrc*, *prev_turn_rate*, and *turn_rate*. This algorithm returns the adjusted *match_thresh* value.

Algorithm 216 ApplyHorizontalCoordinationHysteresis

```
function ApplyHorizontalCoordinationHysteresis( match_thresh::R, prev_sent_hrc::UInt32, prev_turn_rate::R,
     turn_rate::R)
    hcoord_hysteresis::R = params().horizontal_trm.horizontal_offline.hcoord_policy.C_hcoord_hysteresis
    if (prev_sent_hrc != 0)
        sense_prev::Symbol = HorizontalRateToSense(p. 414)( prev_turn_rate )
        sense_curr::Symbol = HorizontalRateToSense(p. 414)( turn_rate )
        if !IsHorizontalReversal(p. 422)(sense_prev, sense_curr)
            prev_sent_hcoord_msg::Symbol = HRCToAdvisory(p. 415)( prev_sent_hrc )
            curr_hcoord_msg::Symbol = HorizontalRateToAdvisory(p. 414)( turn_rate )
            if ( (curr_hcoord_msg == :DontTurnRight) && (prev_sent_hcoord_msg == :DontTurnRight) ) ||
               ( (curr_hcoord_msg == :DontTurnLeft)  && (prev_sent_hcoord_msg == :DontTurnLeft) )
                if (match_thresh > -1 * hcoord_hysteresis)
                    match_thresh = 1.0
                end
            elseif ( (curr_hcoord_msg == :DontTurnLeft)  && (prev_sent_hcoord_msg == :DontTurnRight) ) ||
                   ( (curr_hcoord_msg == :DontTurnRight) && (prev_sent_hcoord_msg == :DontTurnLeft) )
                if (match_thresh < 1 * hcoord_hysteresis)
                    match_thresh = -1.0
                end
            end
        end
    end
    return match_thresh::R
end
```

Referenced In: GetHorizontalCoordinationPolicy(p. 223)

ENCODEHRC (Algorithm 217) returns the Horizontal Resolution Complement (HRC) value for the outgoing coordination message to an individual intruder.

If the individual intruder is not equipped to receive collision avoidance coordination messages, as determined from *equipage_int*, *hrc* is set to *0*. An individual intruder is considered equipped if it is equipped with an ACAS sXu system.

If the individual intruder is equipped to receive coordination messages, the turn sense of the ownship resolution advisory (*sense_own*) and individual resolution advisory (*match_sense_indiv*) are used to determine the value of *hrc*.

- The turn sense of the ownship and individual RAs are both left: *hrc* is set to *2*.
- The turn sense of the ownship RA is left and the turn sense of the individual RA is right: *hrc* is set to *1*.
- The turn sense of the ownship and individual RAs are both right: *hrc* is set to *5*.
- The turn sense of the ownship RA is right and the turn sense of the individual RA is left: *hrc* is set to *6*.
- Otherwise there is no RA for ownship: *hrc* is *0*.

This algorithm takes as inputs *sense_own*, *match_sense_indiv*, and *equipage_int*. This algorithm returns *hrc*.

Algorithm 217 EncodeHRC

```
function EncodeHRC( sense_own::Symbol, match_sense_indiv::Bool, equipage_int::Z )
    hrc::UInt32 = 0
    if (equipage_int != EQUIPAGE_CASRA)   &&
       (equipage_int != EQUIPAGE_CASTA)   &&
       (equipage_int != EQUIPAGE_CASRESP) &&
       (equipage_int != EQUIPAGE_DAARESP)
        hrc = 0
    elseif (:Left == sense_own)
        if match_sense_indiv
            hrc = 2
        else
            hrc = 1
        end
    elseif (:Right == sense_own)
        if match_sense_indiv
            hrc = 5
        else
            hrc = 6
        end
    end
    return hrc::UInt32
end
```

Referenced In: HorizontalCoordinationSelection(p. 222)

ENCODECHC (Algorithm 218) returns the Cancel Horizontal Complement (CHC) value for the outgoing coordination message to an individual intruder. The algorithm will return the CHC canceling the previous Horizontal Resolution Complement (HRC) when the current HRC for an individual intruder changed from the HRC sent in the previous cycle, either due to clear of conflict or reversal in advisory. Otherwise, it will persist the CHC value from the previous cycle to account for possible loss of messages.

If the individual intruder is equipped to receive coordination messages, the turn advisory associated with the current individual output HRC (*advisory_indiv*) and the turn advisory associated with the individual output HRC on the previous cycle (*advisory_indiv_prev*) are examined to see if there has been a change. A value of *:None* indicates no advisory. When following a resolution advisory, a *advisory_indiv* value of *:None* indicates clear of conflict. A resolution advisory on the previous cycle is indicated by a *advisory_indiv_prev* value of *:DontTurnLeft* or *:DontTurnRight*.

For an intruder equipped to receive coordination messages, the value of *chc* is set as follows:

- There is no HRC to be cancelled: *chc* is set to *0*.
- A "Don't Turn Left" advisory cleared; the previous HRC is cancelled: *chc* is set to *1*.
- A "Don't Turn Right" advisory cleared; the previous HRC is cancelled: *chc* is set to *2*.
- There is no change in the type of advisory: the previous value of *chc* is persisted.
- There is a reversal from a "Don't Turn Left" advisory to a "Don't Turn Right" advisory; the previous HRC is cancelled: *chc* is set to *1*.
- There is a reversal from a "Don't Turn Right" advisory to a "Don't Turn Left" advisory; the previous HRC is cancelled: *chc* is set to *2*.
- This is an initial advisory; there is no HRC to be cancelled: *chc* is set to *0*.

This algorithm takes as inputs *hrc*, *hrc_prev*, *chc_prev*, and *equipage_int*. This algorithm returns *chc*.

Algorithm 218 EncodeCHC

```
function EncodeCHC( hrc::UInt32, hrc_prev::UInt32, chc_prev::UInt32, equipage_int::Z )
    advisory_indiv::Symbol        = HRCToAdvisory(p. 415)( hrc )
    advisory_indiv_prev::Symbol   = HRCToAdvisory(p. 415)( hrc_prev )
    cancel_advisory_indiv::Symbol = HRCToAdvisory(p. 415)( chc_prev )
    chc::UInt32 = 0
    if (equipage_int != EQUIPAGE_CASRA)   &&
       (equipage_int != EQUIPAGE_CASTA)   &&
       (equipage_int != EQUIPAGE_CASRESP) &&
       (equipage_int != EQUIPAGE_DAARESP)
        chc = 0
    elseif (advisory_indiv == :None) && (advisory_indiv_prev == :None)
        chc = 0
    elseif (advisory_indiv == :None) && (advisory_indiv_prev == :DontTurnLeft)
        chc = 1
    elseif (advisory_indiv == :None) && (advisory_indiv_prev == :DontTurnRight)
        chc = 2
    elseif (advisory_indiv == advisory_indiv_prev) && (cancel_advisory_indiv != advisory_indiv)
        chc = chc_prev
    elseif (advisory_indiv == :DontTurnRight) && (advisory_indiv_prev == :DontTurnLeft)
        chc = 1
    elseif (advisory_indiv == :DontTurnLeft) && (advisory_indiv_prev == :DontTurnRight)
        chc = 2
    else
        chc = 0
    end
    return chc::UInt32
end
```

Referenced In: HorizontalCoordinationSelection(p. 222)

ENCODEHSB (Algorithm 219) returns the Horizontal Sense Bits (HSB) value for the outgoing coordination message to an individual intruder.

The *hsb* field is a parity field used to protect the *hrc* and *chc* fields. The value of *hsb* is encoded directly from *hrc* and *chc* using a parity table. The parity table (*X_parity_table*) is an input parameter, the use of which is described in Section 2.7.1.

When *hrc* and *chc* are both *0*, *hsb* will be *0* as well.

This algorithm takes as inputs *hrc* and *chc*. This algorithm returns *hsb*.

Algorithm 219 EncodeHSB

```
function EncodeHSB( hrc::UInt32, chc::UInt32 )
    X_parity_table::Vector{Z} = params().coordination.parity_table_hor
    hsb::UInt32 = UInt32( X_parity_table[ (chc * 8) + hrc + 1 ] )
    return hsb::UInt32
end
```

Referenced In: HorizontalCoordinationSelection(p. 222)

3.2.7 Horizontal Track Threat Assessment

An overview of the HORIZONTALTRACKTHREATASSESSMENT component of the Horizontal TRM is shown in Fig. 5.

HORIZONTALTRACKTHREATASSESSMENT (Algorithm 220) is responsible for creating a code that indicates the status of the track in the horizontal dimension and a Track Display Score (TDS) for prioritizing that track. The two values are used to determine how the track will be displayed to the operator. The list of codes can be found in Appendix C, where a higher value code indicates higher priority.

DETERMINEHORIZONTALCODE (Algorithm 221) determines the track status code for the individual intruder.

DETERMINEHORIZONTALSCORE (Algorithm 222) determines the track display score for the individual intruder. The TDS is a real-valued quantity that allows the intruder tracks to be prioritized in case of limited display resources.

HORIZONTALTRACKTHREATASSESSMENT takes as inputs *selected_advisory*, *st_int*, and *degraded_surveillance*. This algorithm returns the values in Table 35, Output Variables.

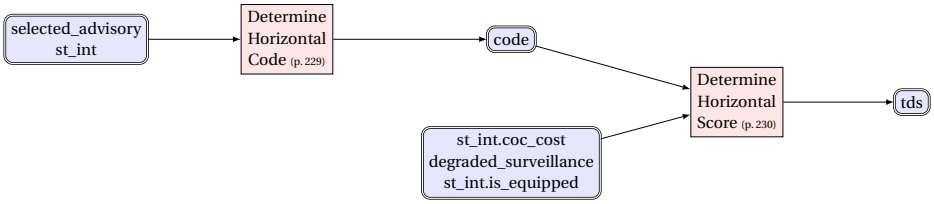

Figure 5. HorizontalTrackThreatAssessment overview.

Table 35. HorizontalTrackThreatAssessment Output Variables

Variable	Units	Type	Description
code	N/A	uint8	horizontal TRM track status code
tds	N/A	real	horizontal TRM track display score

Algorithm 220 HorizontalTrackThreatAssessment

```
function HorizontalTrackThreatAssessment( selected_advisory::HorizontalAdvisory(p. 518),
                                          st_int::HTRMIntruderState(p. 514), degraded_surveillance::UInt16 )
    code::UInt8 = DetermineHorizontalCode(p. 229)( selected_advisory, st_int )
    tds::R = DetermineHorizontalScore(p. 230)( code, st_int.coc_cost, degraded_surveillance, st_int.is_equipped )
    return (code::UInt8, tds::R)
end
```

Referenced In: SetHorizontalIntruderOutput(p. 235), SetHorizontalDroppedIntruderOutput(p. 236)

3.2.7.1 Determine Horizontal Code

DETERMINEHORIZONTALCODE (Algorithm 221) determines the Horizontal TRM track status code for an individual intruder. The track status code is assigned based on the most critical advisory for the individual intruder.

The algorithm considers resolution advisories as indicated by the turn rate. If none of the criteria are met, the *code* remains set to *SXUCODE_CLEAR*.

This algorithm takes as inputs *selected_advisory*, *st_int*, and *corrective_alert_global*. This algorithm returns *code*.

Algorithm 221 DetermineHorizontalCode

```
function DetermineHorizontalCode( selected_advisory::HorizontalAdvisory(p.518), st_int::HTRMIntruderState(p.514) )
        code::UInt8 = SXUCODE_CLEAR
        if !IsHorizontalCOC(p.422)( selected_advisory.turn_rate ) && st_int.in_advisory
            code = SXUCODE_RA
        end
        return code::UInt8
    end
```

Referenced In: HorizontalTrackThreatAssessment(p.228)

3.2.7.2 Determine Horizontal Score

DETERMINEHORIZONTALSCORE (Algorithm 222) determines the Horizontal TRM track display score (*tds*) for an individual intruder based on the *code* value. The *tds* value is based on the track status code (*code*). The clear of conflict cost (*coc_cost*) is used to prioritize intruders with the same code. A higher value of *coc_cost* indicates a greater threat. An intruder with *coc_cost* set to NaN did not contribute to a resolution advisory or corrective alert. In this case, *coc_cost* is set to -10.0, which is a large negative magnitude and indicates a non-threat.

The degraded surveillance status of the intruder (*degraded_surveillance*) is used to determine which score factor (*scorefactor*) to weight the scores by to prioritize intruders with non-degraded surveillance.

If the intruder code value indicates a resolution advisory, the equipage (*equip_int*) of the intruder is considered and the algorithm will increase the score if coordination with the intruder is possible.

This algorithm takes as inputs *code*, *coc_cost*, *degraded_surveillance*, and *equip_int*. This algorithm returns *tds*.

Algorithm 222 DetermineHorizontalScore

```
function DetermineHorizontalScore( code::UInt8, coc_cost::R, degraded_surveillance::UInt16, equip_int::Bool)
    X_scorfactr_good::R     = params().display.scorfactr_good
    X_scorfactr_degraded::R = params().display.scorfactr_degraded
    X_hiscore::R            = params().display.hiscore
    X_medhiscore::R         = params().display.medhiscore
    X_loscore::R            = params().display.loscore
    X_tinyscore::R          = params().display.tinyscore
    tds::R         = 0.0
    scorefactor::R = X_scorfactr_good
    if isnan( coc_cost )
        coc_cost = -10.0
    end
    if ((degraded_surveillance & DEGRADED_SURVEILLANCE_NO_BEARING) != 0) ||
       ((degraded_surveillance & DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED) != 0)
        scorefactor = X_scorfactr_degraded
    end
    if (code == SXUCODE_RA)
        if equip_int
            tds = X_hiscore + X_medhiscore + coc_cost
        else
            tds = X_hiscore + coc_cost
        end
    elseif (code == SXUCODE_CLEAR)
        tds = (X_tinyscore * scorefactor) + coc_cost
    end
    return tds::R
end
```

Referenced In: HorizontalTrackThreatAssessment(p. 228)

3.2.8 Horizontal Display Logic

An overview of the HORIZONTALDISPLAYLOGICDETERMINATION component of the Horizontal TRM is shown in Fig. 6.

HORIZONTALDISPLAYLOGICDETERMINATION (Algorithm 223) converts the horizontal advisories provided by the collision avoidance logic to a format that can be displayed for the operator. The display logic (*display*) is populated by a series of functions.

DETERMINEDISPLAYTRACKANGLEANDTURNRATE (Algorithm 224) determines the target track angle (*display.target_track_angle*) in degrees and turn rate (*display.turn_rate*) in degrees per second from the track angle (*track_angle*), turn rate (*turn_rate*), and ownship TRM state (*st_own*).

DETERMINELABEL271 (Algorithm 225) produces the combined control (*cc*) value associated with the resolution advisory using the target turn rate (*display.turn_rate*) and ownship TRM state (*st_own*).

DETERMINEHORIZONTALDESENSITIVITY (Algorithm 227) determines the horizontal desensitivity mode value (*horizontal_desensitivity_mode*) using the turn rate limit (*turn_rate_limit*) and vertical rate limit (*vert_rate_limit*).

HORIZONTALDISPLAYLOGICDETERMINATION takes as inputs *track_angle*, *turn_rate*, *expected_rwc*, *st_own*, (*turn_rate_limit*), and (*vert_rate_limit*). This algorithm returns the values in Table 36, HorizontalDisplayLogicDetermination Output Variables.

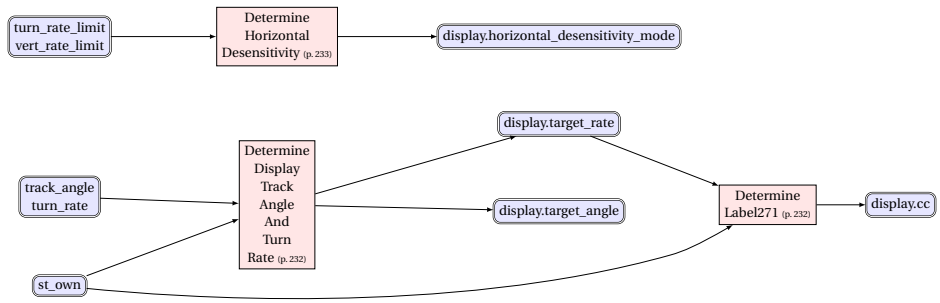

Figure 6. HorizontalDisplayLogicDetermination overview.

Table 36. HorizontalDisplayLogicDetermination Output Variables

Variable	Units	Type	Description
display.cc	N/A	uint8	combined control providing the turn advisory
display.target_rate	deg/s	real	turn rate for ownship to target
display.target_track_angle	deg	real	track angle for ownship to target
display.horizontal_desensitivity_mode	N/A	uint8	desensitivity regarding horizontal RA selection (through online costs)

Algorithm 223 HorizontalDisplayLogicDetermination

```
function HorizontalDisplayLogicDetermination( track_angle::R, turn_rate::R,
                                    st_own::HTRMOwnState(p. 513), turn_rate_limit::R, vert_rate_limit::R )
    display::HorizontalDisplayLogic(p. 518) = HorizontalDisplayLogic(p. 518)()
    (display.target_angle, display.target_rate) =
        DetermineDisplayTrackAngleAndTurnRate(p. 232)( track_angle, turn_rate, st_own )
    display.cc =
        DetermineLabel271(p. 232)( display.target_rate, st_own )
    display.horizontal_desensitivity_mode =
        DetermineHorizontalDesensitivity(p. 233)(turn_rate_limit, vert_rate_limit)
    return display::HorizontalDisplayLogic(p. 518)
end
```

Referenced In: HorizontalTRMUpdate(p. 168)

3.2.8.1 Determine Target Track Angle and Turn Rate

DETERMINEDISPLAYTRACKANGLEANDTURNRATE (Algorithm 224) is used to convert the track angle and turn rate from the collision avoidance logic in radians to a target track angle and target turn rate in degrees to be used by the display logic. The target track angle and target turn rate are only converted if the *turn_rate* is not clear of conflict in which case a turn is also recommended and *st_own* is updated.

This algorithm takes as inputs *track_angle*, *turn_rate*, and *st_own*. This algorithm returns *tar-*

get_track_angle_deg and *target_turn_rate_deg*. This algorithm updates *st_own*.

Algorithm 224 DetermineDisplayTrackAngleAndTurnRate

```
function DetermineDisplayTrackAngleAndTurnRate( track_angle::R, turn_rate::R, st_own::HTRMOwnState(p. 513) )
    target_track_angle_deg::R       = NaN
    target_turn_rate_deg::R         = NaN
    st_own.is_turn_recommended      = false
    if !IsHorizontalCOC(p. 422)( turn_rate )
        target_track_angle_deg      = round( rad2deg( track_angle ) )
        target_turn_rate_deg        = rad2deg( turn_rate )
        st_own.is_turn_recommended  = true
    end
    return (target_track_angle_deg::R, target_turn_rate_deg::R)
end
```

Referenced In: HorizontalDisplayLogicDetermination(p. 231)

3.2.8.2 Determine Label 271 Value

DETERMINELABEL271 (Algorithm 225) returns the combined control (*cc*) value associated with the resolution advisory.

The global action for the current cycle (*action*) and the global action on the previous cycle (*action_prev*) are compared with the logic rules using CONSISTENT271. If a consistent logic rule is found, the corresponding *cc* is returned, otherwise a value of 0 is returned for *cc*.

This algorithm takes as inputs *target_rate* and *st_own*. This algorithm returns *cc*.

Algorithm 225 DetermineLabel271

```
function DetermineLabel271( target_rate::R, st_own::HTRMOwnState(p. 513) )
    L_act_in::Vector{Z}        = params().display.label271rules.action
    L_act_in_prev::Vector{R}   = params().display.label271rules.prevaction
    L_cc_out::Vector{Z}        = params().display.label271rules.cc
    N_rules::Z = length( L_act_in )
    cc::UInt8 = UInt8( CC_H_NO_ADVISORY )
    action_prev::Z =  st_own.advisory_prev.action
    action::Z      = HorizontalRateToAction(p. 413)( deg2rad( target_rate ) )
    for i in 1:N_rules
        if Consistent271(p. 233)( action, action_prev, L_act_in[i], L_act_in_prev[i] )
            cc = UInt8( L_cc_out[i] )
            break
        end
    end
    return cc::UInt8
end
```

Referenced In: HorizontalDisplayLogicDetermination(p. 231)

CONSISTENT271 (Algorithm 226) determines whether the current advisory and dynamic state is consistent with the specified logic rule.

The logic rule being tested is specified using a global action for the current cycle (*L_act_in*) and a global action for the previous cycle (*L_act_in_prev*).

A value of *NaN* (Not a Number) for a logic rule variable functions as a wildcard, it will match any input value.

The advisory and dynamic state for comparison are specified using the global action for the current cycle (*action*) and the global action on the previous cycle (*action_prev*).

If any of the inputs fail to match a rule, *is_consistent* is set to FALSE. If all the inputs match all of the rules, *is_consistent* is TRUE.

This algorithm takes as inputs *action*, *action_prev*, *L_act_in*, *L_act_in_prev*. This algorithm returns *is_consistent*.

Algorithm 226 Consistent271

```
function Consistent271( action::Z, action_prev::Z, L_act_in::Z, L_act_in_prev::R )
    is_consistent::Bool = true
    if  (action != L_act_in) || (!isnan( L_act_in_prev )  && (action_prev != Int( L_act_in_prev )))
        is_consistent = false
    end
    return is_consistent::Bool
end
```

Referenced In: DetermineLabel271 (p. 232)

3.2.8.3 Determine Horizontal Desensitivity Mode

DETERMINEHORIZONTALDESENSITIVITY (Algorithm 227) is used to determine the horizontal desensitivity mode with regard to horizontal RA selection through online costs.

DETERMINEHORIZONTALDESENSITIVITY takes as inputs *turn_rate_limit* and *vert_rate_limit*. This algorithm returns as output *horizontal_desensitivity_mode*.

Algorithm 227 DetermineHorizontalDesensitivity

```
function DetermineHorizontalDesensitivity( turn_rate_limit::R, vert_rate_limit::R )
    (C_direct_sensitivity_factor::R, C_desensitivity::R) = GetPerformanceBasedParams(p. 201)( vert_rate_limit,
         turn_rate_limit )
    horizontal_desensitivity_mode::UInt8 = HORIZONTAL_DESENSITIVITY_NONE
    if (C_direct_sensitivity_factor > 1.0) && (C_desensitivity == 0.0)
        horizontal_desensitivity_mode = HORIZONTAL_DESENSITIVITY_SENSITIZE
    elseif (C_direct_sensitivity_factor == 1.0) && (C_desensitivity < 0.0)
        horizontal_desensitivity_mode = HORIZONTAL_DESENSITIVITY_DESENSITIZE
    end
    return horizontal_desensitivity_mode::UInt8
end
```

Referenced In: HorizontalDisplayLogicDetermination (p. 231)

3.2.9 Generate Horizontal TRM Output

GENERATEHTRMOUTPUT (Algorithm 228) creates the HTRMREPORT for the horizontal TRM.

The algorithm first initializes fields necessary to generate the horizontal TRM reports. The active HRA compliments are set in the RA ground and broadcast reports.

Then valid intruders, invalid intruders, and dropped intruders are iterated through using SETHORIZONTALINTRUDEROUTPUT and SETHORIZONTALDROPPEDINTRUDEROUTPUT to populate the reports and the horizontal TRM state.

Horizontal RA termination (RAT) is then determined through HORIZONTALDISPLAYLOGIC combined control bits.

This algorithm takes as inputs *input_own*, *display*, *selected_advisory*, *input_int_valid*, *input_int_invalid*, *st_int_valid*, *st_int_invalid*, *st_int_dropped*, and *trm_state*. This algorithm returns *report* of type HTRMREPORT.

Algorithm 228 GenerateHTRMOutput

```
function GenerateHTRMOutput( input_own::TRMOwnInput(p. 498), display::HorizontalDisplayLogic(p. 518),
                             selected_advisory::HorizontalAdvisory(p. 518),
                             input_int_valid::Vector{TRMIntruderInput(p. 499)},
                             input_int_invalid::Vector{TRMIntruderInput(p. 499)},
                             st_int_valid::Vector{HTRMIntruderState(p. 514)},
                             st_int_invalid::Vector{HTRMIntruderState(p. 514)},
                             st_int_dropped::Vector{HTRMIntruderState(p. 514)},
                             trm_state::HTRMState(p. 513), received_hrcs::Vector{Bool})
    N_actions::Z = params().turn_actions.num_actions
    report::HTRMReport(p. 519) = HTRMReport(p. 519)()
    report.display = HTRMDisplayData(p. 503)( display, HTRMIntruderDisplayData(p. 504)[] )
    no_advisory::HorizontalAdvisory(p. 518) = HorizontalAdvisory(p. 518)( )
    no_advisory.costs = fill( NaN, N_actions )
    multithreat::Bool = (length( selected_advisory.threat_list ) > 1)
    report.broadcast_msg.hmte = multithreat
    if in(true, received_hrcs)
        report.broadcast_msg.rac[3] = received_hrcs[1]
        report.broadcast_msg.rac[4] = received_hrcs[2]
    end
    st_intruder::Vector{HTRMIntruderState(p. 514)} = HTRMIntruderState(p. 514)[]
    for i in 1:length( st_int_valid )
        SetHorizontalIntruderOutput(p. 235)( report, selected_advisory, multithreat,
            input_own, input_int_valid[i], trm_state.st_own, st_int_valid[i], false )
        push!( st_intruder, st_int_valid[i] )
    end
    for i in 1:length( st_int_invalid )
        SetHorizontalIntruderOutput(p. 235)( report, no_advisory, multithreat,
            input_own, input_int_invalid[i], trm_state.st_own, st_int_invalid[i], true)
        push!( st_intruder, st_int_invalid[i] )
    end
    for i in 1:length( st_int_dropped )
        SetHorizontalDroppedIntruderOutput(p. 236)( report, no_advisory, multithreat,
            input_own, trm_state.st_own, st_int_dropped[i] )
    end
    trm_state.st_own.advisory_prev = selected_advisory
    trm_state.st_intruder = deepcopy( st_intruder )
    SetAHRAReportData(p. 237)(report, display, st_int_valid)
    if (display.cc > CC_H_COC)
        report.broadcast_msg.rat = false
    else
        report.broadcast_msg.rat = true
    end
    report.broadcast_msg.hri = (input_own.trm_velmode == TRM_VELMODE_NONE) || (input_own.h_opmode ==
        OPMODE_SURV_ONLY) || (input_own.h_opmode == OPMODE_STANDBY)
    return report::HTRMReport(p. 519)
end
```

Referenced In: HorizontalTRMUpdate(p. 168)

SETHORIZONTALINTRUDEROUTPUT (Algorithm 229) produces ACAS sXu output for the individual intruder.

First, the input flag *is_state_update_needed* determines if UPDATEHTRMINTRUDERSTATE is called to clear the information int the intruder state data structure (*st_int*). This is only applied to the invalid intruders such that unnecessary information does not contribute to horizontal TRM report.

HORIZONTALCOORDINATIONSELECTION produces the horizontal coordination information to be sent to the intruder. That information is then added to the *coordination* vector in the TRM report output structure.

HORIZONTALTRACKTHREATASSESSMENT produces the track display code and track display score to be output for the intruder. The information is added to the *display.intruder* vector in the TRM report output structure.

Finally, *equipage* and *coordination_msg* are stored in the intruder state. The state values are used to prepare a CANCEL coordination message when the intruder has been dropped by the STM. The algorithm checks the intruder's valid flag and if invalid, the status field for the intruder is set to :NotValid.

This algorithm takes as inputs *report*, *selected_advisory*, *multithreat*, *input_own*, *input_int*, *st_own*, *st_int*, and *is_state_update_needed*. This algorithm updates *report*, *selected_advisory*, and *st_int*.

Algorithm 229 SetHorizontalIntruderOutput

```
function SetHorizontalIntruderOutput( report::HTRMReport(p. 519), selected_advisory::HorizontalAdvisory(p. 518),
                                      multithreat::Bool, input_own::TRMOwnInput(p. 498),
                                      input_int::TRMIntruderInput(p. 499), st_own::HTRMOwnState(p. 513),
                                      st_int::HTRMIntruderState(p. 514), is_state_update_needed::Bool )
    if is_state_update_needed
        UpdateHTRMIntruderState(p. 435)( st_int, true, true )
    end
    (chc::UInt32, hrc::UInt32, hsb::UInt32) =
        HorizontalCoordinationSelection(p. 222)( st_own.advisory_prev.turn_rate, selected_advisory.turn_rate,
             st_own.enu_beliefs, input_int.equipage, st_int )
    push!( report.coordination,
           TRMCoordinationInterrogationData(p. 504)( input_int.id, UInt32(0), UInt32(0), UInt32(0), chc, hrc, hsb,
                multithreat, input_own.v2v_uid, input_int.id_directory, input_int.coordination_msg ) )
    (code::UInt8, tds::R) =
        HorizontalTrackThreatAssessment(p. 228)( selected_advisory, st_int, input_int.degraded_surveillance )
    push!( report.display.intruder,
           HTRMIntruderDisplayData(p. 504)( input_int.id, input_int.id_directory, tds, code,
                                            input_int.classification, input_int.source ) )
    st_int.equipage_prev         = input_int.equipage
    st_int.coordination_msg_prev = input_int.coordination_msg
    if !input_int.valid
        st_int.status = :NotValid
    end
    return
end
```

Referenced In: GenerateHTRMOutput(p. 234)

SETHORIZONTALDROPPEDINTRUDEROUTPUT (Algorithm 230) produces ACAS sXu output for the individual intruder when that intruder was not present in the input for the current cycle.

TRMCOORDINATIONINTERROGATIONDATA produces the horizontal coordination information to be sent to the intruder. That information is then added to the *coordination* vector in the TRM report output structure. Instead of using information from TRMINTRUDERINPUT, as in SETHORIZONTALINTRUDEROUTPUT, *equipage*, *coordination_msg*, *classification*, and *source* are obtained from HTRMINTRUDERSTATE instead.

HORIZONTALTRACKTHREATASSESSMENT produces the track display code and track display score to be output for the intruder. The information is added to the *display.intruder* vector in the TRM report output structure.

For an intruder being marked for deletion (dropped), there is never a collision avoidance advisory.

Finally, *st_int.status* is set to indicate the state is *:MarkedForDeletion*.

This algorithm takes as inputs *report*, *no_advisory*, *multithreat*, *input_own*, *st_own*, and *st_int*. This algorithm updates *report* and *st_own*.

Algorithm 230 SetHorizontalDroppedIntruderOutput

```
function SetHorizontalDroppedIntruderOutput( report::HTRMReport(p.519), no_advisory::HorizontalAdvisory(p.518),
                                             multithreat::Bool, input_own::TRMOwnInput(p.498),
                                             st_own::HTRMOwnState(p.513), st_int::HTRMIntruderState(p.514) )
    UpdateHTRMIntruderState(p.435)( st_int, true, true )
    (chc::UInt32, hrc::UInt32, hsb::UInt32) =
        HorizontalCoordinationSelection(p.222)( st_own.advisory_prev.turn_rate, no_advisory.turn_rate,
                                                st_own.enu_beliefs, st_int.equipage_prev, st_int )
    push!( report.coordination,
           TRMCoordinationInterrogationData(p.504)( st_int.id, UInt32(0), UInt32(0), UInt32(0),
                                                    chc, hrc, hsb, multithreat,
                                                    input_own.v2v_uid, st_int.id_directory,
                                                    st_int.coordination_msg_prev ) )
    (code::UInt8, tds::R) =
        HorizontalTrackThreatAssessment(p.228)( no_advisory, st_int, DEGRADED_SURVEILLANCE_NONE )
    push!( report.display.intruder,
           HTRMIntruderDisplayData(p.504)( st_int.id, st_int.id_directory,
                                           tds, code, st_int.classification, st_int.source ) )
    st_int.status = :MarkedForDeletion
end
```

Referenced In: GenerateHTRMOutput(p.234)

SETAHRAREPORTDATA (Algorithm 231) sets the ARA data for the horizontal dimension. First, SETAHRAREPORTDATA checks the *st_int_valid* for valid intruders. If the list is empty (no intruders), the *broadcast.ahra_match* is set to false. Otherwise a list of sense values are compiled for all the intruders. Notice that the *st_int.hcoord_sense_prev* is current for the cycle at this point.

If the list of sense values is empty it is indicative of no RAs for the current intruder set and *broadcast.ahra_match* is set to false. If there are intruders contributing to the active RA, the first valid sense is compared to the *sense_values* list. If all the list items match, then the *broadcast.ahra_match* is set to true. Oherwise, they are set false indicating senses don't match across all RA inducing intruders.

Next, the *display* values are checked and correlated into the *ahra_turn* variable for SXUTRMBROADCASTDATA ARA data.

Finally, the *display.target_track_angle* is checked for NaN. If *display.target_track_angle* is NaN, the *ahra_track_angle* field is set to the MOPS coded representation of NaN, defined by the parameter *AHRA_TRACK_ANGLE_NAN*. If *display.target_track_angle* is not NaN, the *ahra_track_angle* field of the sXuTRMBroadcastData structure are mapped to the integer-value bins by means of a floored division of the mod of *display.target_track_angle* by the bin size, defined by the parameter *AHRA_TRACK_ANGLE_BIN*.

This algorithm takes as inputs type HTRMReport (*report*), HorizontalDisplayLogic (*display*), and list of HTRMIntruderState (*st_int_valid*). This algorithm updates the sXuTRMBroadcastData AHRA data fields within the HTRMReport.

Algorithm 231 SetAHRAReportData

```
function SetAHRAReportData( report::HTRMReport(p.519), display::HorizontalDisplayLogic(p.518), st_int_valid::
    Vector{HTRMIntruderState(p.514)} )
    if (isempty(st_int_valid))
        report.broadcast_msg.ahra_match = false
    else
        sense_values = Z[]
        for st_int in st_int_valid
            if (st_int.hcoord_sense_prev != HCOORD_SENSE_NONE) && st_int.in_advisory
                append!(sense_values, st_int.hcoord_sense_prev)
            end
        end
        if(isempty(sense_values))
            report.broadcast_msg.ahra_match = false
        else
            first_sense = sense_values[1]
            same_sense::Bool = all(sense_values .== first_sense)
            if (same_sense)
                report.broadcast_msg.ahra_match = true
            else
                report.broadcast_msg.ahra_match = false
            end
        end
    end
    if (display.cc == CC_H_TURN_RIGHT)
        report.broadcast_msg.ahra_turn  = AHRA_RIGHT_TURN
    elseif (display.cc == CC_H_TURN_LEFT)
        report.broadcast_msg.ahra_turn  = AHRA_LEFT_TURN
    elseif (display.cc == CC_H_STRAIGHT)
        report.broadcast_msg.ahra_turn  = AHRA_STRAIGHT
    else
        report.broadcast_msg.ahra_turn  = AHRA_NO_TURN
    end
    if (isnan(display.target_angle))
        report.broadcast_msg.ahra_track_angle  = AHRA_TRACK_ANGLE_NAN
    else
        report.broadcast_msg.ahra_track_angle  = fld(mod(display.target_angle,360),AHRA_TRACK_ANGLE_BIN)
    end
end
```

Referenced In: GenerateHTRMOutput(p.234)

3.3 Vertical Threat Resolution Module

3.3.1 Vertical TRM Overview

The main data and processing components of the Vertical Threat Resolution Module (Vertical TRM or VTRM) are described in a separate section of the document and is invoked from VERTICALTRMUPDATE (Algorithm 232). The variables in the figure, and their minimal types, are described in Table 37.

First an intruder representing the ground directly below ownship is created with CREATEGROUNDINTRUDER. This intruder exists only in the vertical logic and will be considered a valid intruder if terrain avoidance is on and ownship has valid height above-ground-level and altitude measurements.

If the own aircraft is in Surveillance Only operational mode, the SETVERTICALTRMSURVONLYMODE component is utilized to maintain TRM processing information between operation cycles. Otherwise, the Vertical TRM calculates vertical maneuver recommendations to avoid nearby aircraft using the outputs from the STM.

The STATEANDCOSTESTIMATION component (Section 3.3.2) initializes intruder state to account for special processing modes and intruder coordination capabilities. This component evaluates the probabilistic threat represented by each intruder in isolation.

The resulting state and cost information for all intruders are then supplied to the ACTIONSELECTION component (Section 3.3.3), which selects a single global vertical action for own aircraft and individual vertical actions against each intruder aircraft.

The resulting global vertical action, as well as the individual vertical actions against each intruder, are supplied to the COORDINATIONSELECTION component (Section 3.3.4) which produces data for each individual intruder to be used for coordination and the operator display.

The DISPLAYLOGICDETERMINATION component (Section 3.3.5) uses these results, along with state information, to produce data used for visual and aural alerts to be displayed to the operator.

The SETINTRUDERVERTICALSCORES component determines if intruders are capable of coordination and sets the track display score.

The GENERATEVTRMOUTPUT component (Section 3.3.6) adjusts the data used for coordination and the operator display to account for degraded surveillance, Surveillance Only operational mode and Standby Operational Mode. GENERATEVTRMOUTPUT also does housekeeping on dropped and invalid intruders, updates the Vertical TRM state variables, and packages the data for output from the Vertical TRM in the VTRMREPORT.

VERTICALTRMUPDATE takes as input *input* and *sxu_trm_state*. The variable *input* contains information about the ownship (heading, height-AGL, identifying information needed for TRM outputs, operational mode, surveillance quality, and availability of Xo modes) as well as information about the tracked state of nearby intruding aircraft. More information about the contents of *input* can be found in TRMINPUT. The variable *sxu_trm_state* contains information about the internal TRM state for both vertical and horizontal. This includes own aircraft previous and current states as well as information about processing states for each intruding aircraft. More information about the contents of *sxu_trm_state* can be found in SXUTRMSTATE. The algorithm returns *report_vert*.

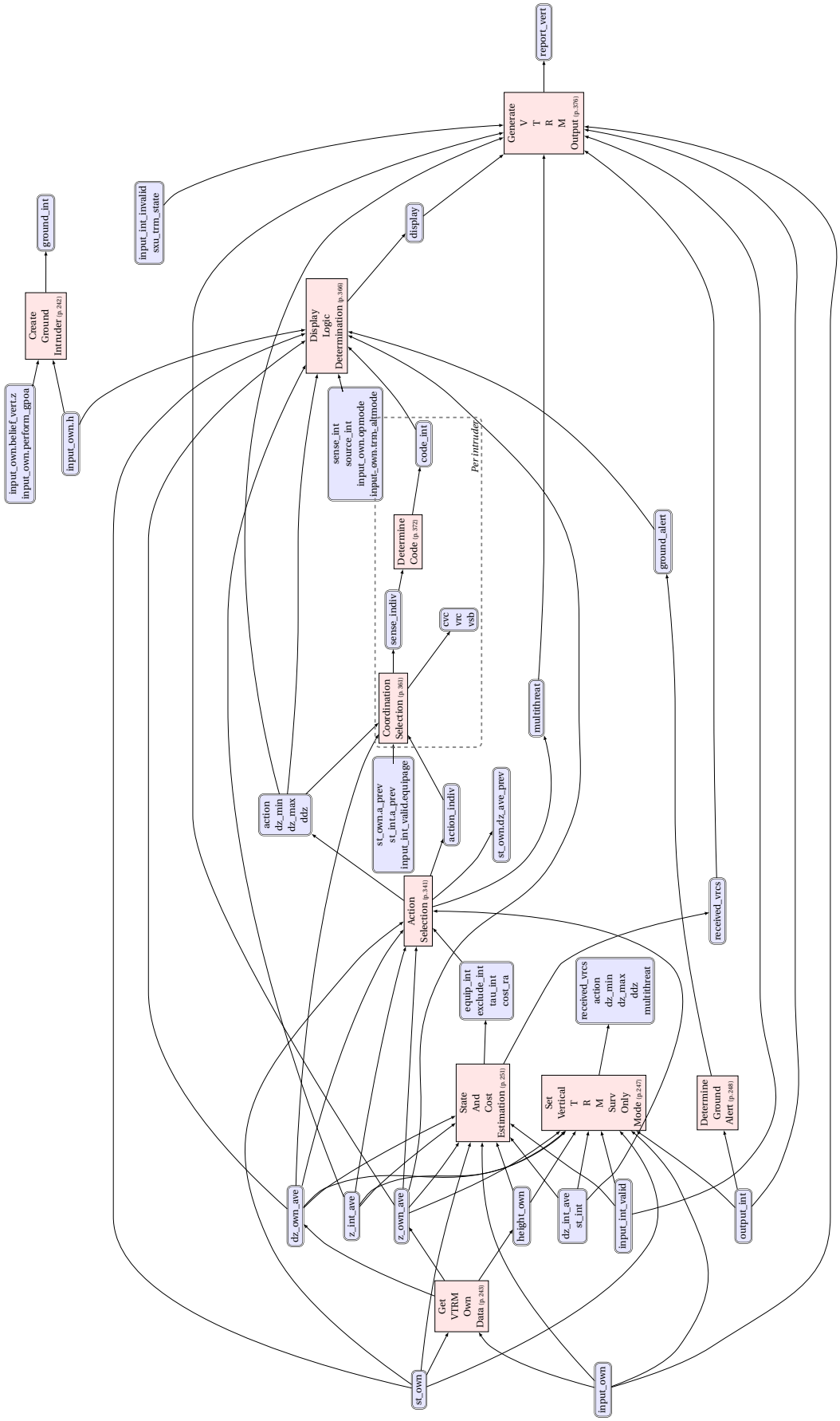

Figure 7. VerticalTRMUpdate overview.

Table 37. Vertical TRM Internal Variables - VerticalTRMUpdate (Figure 7)

Variable	Units	Type	Description
action	N/A	integer	Selected global action (Resolution Advisory (RA)) for own ship
action_indiv	N/A	integer	Array of current advisory action (RA) for each individual intruder
code_int	N/A	uint8	Pilot Display value: Code indicating type of advisory for each intruder
cost_ra	N/A	real	Matrix of combined RA costs of each action and each intruder
cvc	N/A	uint32	Coordination value: Cancel Vertical Complement
ddz	ft/s^2	real	Vertical acceleration associated with the selected global action
display	N/A	DisplayLogic(p. 516)	Information for the RPIC display
dz_int_ave	ft/s	real	Vector of the weighted average of vertical rate samples for each intruder
dz_max	ft/s	real	Maximum vertical rate associated with the selected global action
dz_min	ft/s	real	Minimum vertical rate associated with the selected global action
dz_own_ave	ft/s	real	Weighted average of vertical rate samples for own aircraft
equip_int	N/A	bool	Array of whether each intruder is capable of coordination
exclude_int	N/A	bool	Array of whether each intruder should be excluded from action selection
ground_alert	N/A	uint8	Integer indicating whether the ground is not causing an RA, is the only intruder causing a
ground_int	N/A	TRMIntruderInput(p. 499)	Vertical only intruder represnting the ground located directly below ownship
height_own	ft	real	Height of own aircraft; either height AGL, pressure altitude, or HAE altitude
input_int_invalid	N/A	TRMIntruderInput(p. 499)	Array of input data for each intruder not valid for advisory processing
input_int_valid	N/A	TRMIntruderInput(p. 499)	Array of input data for each intruder valid for advisory processing
input_own	N/A	TRMOwnInput(p. 498)	Input data for own aircraft
mode_int	N/A	integer	Array of protection mode index for each intruder (not shown)
multithreat	N/A	bool	Flag indicating whether or not there are multiple threats (RAs)
num_intruders	N/A	integer	Number of valid intruders from STM
output_int	N/A	TRMIntruderData(p. 523)	Array of coordination and display data for each intruder
received_vrcs	N/A	bool	Coordination intent vector from STM
report_vert	N/A	VTRMReport(p. 524)	Report containing advisories for coordination and the RPIC display
sense_indiv	N/A	symbol	Up or down RA sense selected for an individual intruder
sense_int	N/A	symbol	Array of up or down RA sense selected for each intruder
source_int	N/A	integer	Array of surveillance source for each intruder
st_int	N/A	TRMIntruderState(p. 512)	Array of state information for each individual intruder
st_own	N/A	TRMOwnState(p. 511)	Vertical State information for own aircraft
st_vert	N/A	TRMState(p. 510)	Vertical State information for own aircraft and all intruders processed by the TRM
tau_int	s	real	Vector of expected tau (time of closest point of approach) for each intruder
vrc	N/A	uint32	Coordination value: Vertical Resolution Advisory Complement
vsb	N/A	uint32	Coordination value: Vertical Sense Bits
xu_trm_state	N/A	sXuTRMState(p. 510)	Internal ownship TRM information from previous and current cycles
z_int_ave	ft	real	Vector of the weighted average of vertical samples for each intruder
z_own_ave	ft	real	Weighted average of vertical samples for own aircraft

Algorithm 232 VerticalTRMUpdate

```
function VerticalTRMUpdate( input::TRMInput(p.497), sxu_trm_state::sXuTRMState(p.510) )
    input_own::TRMOwnInput(p.498) = input.own
    st_own::TRMOwnState(p.511)    = sxu_trm_state.st_vert.st_own
    action::Z = 0
    dz_min::R = 0.0
    dz_max::R = 0.0
    ddz::R    = 0.0
    multithreat::Bool = false
    received_vrcs::Vector{Bool} = zeros( Bool, 4 )
    ground_int::TRMIntruderInput(p.499) = CreateGroundIntruder(p.242)(input_own.belief_vert[1].z, input_own.h,
        input_own.perform_gpoa)
    (st_int::Vector{TRMIntruderState(p.512)}, input_int_invalid::Vector{TRMIntruderInput(p.499)},
      input_int_valid::Vector{TRMIntruderInput(p.499)}, num_intruders::Z, z_int_ave::Vector{R},
      dz_int_ave::Vector{R}, mode_int::Vector{Z}, code_int::Vector{UInt8}, sense_int::Vector{Symbol},
      output_int::Vector{TRMIntruderData(p.523)}, source_int::Vector{Z}) =
         VerticalTRMUpdatePrep(p.587)( sxu_trm_state.st_vert, input, ground_int )
    (z_own_ave::R, dz_own_ave::R, height_own::R) = GetVTRMOwnData(p.243)( input_own::TRMOwnInput(p.498),
        st_own::TRMOwnState(p.511) )
    if (input_own.opmode == OPMODE_SURV_ONLY) && (input_own.trm_altmode != TRM_ALTMODE_NONE)
        (received_vrcs, action, dz_min, dz_max, ddz, multithreat) =
            SetVerticalTRMSurvOnlyMode(p.247)(height_own, z_own_ave, dz_own_ave, input_own, input_int_valid,
                output_int, mode_int, z_int_ave, dz_int_ave, st_own, st_int)
    elseif (0 < num_intruders)
        (equip_int::Vector{Bool}, exclude_int::Vector{Bool}, tau_int::Vector{R}, cost_ra::Matrix{R},
            received_vrcs) =
           StateAndCostEstimation(p.251)( height_own, z_own_ave, dz_own_ave, input_own,
                input_int_valid, mode_int, z_int_ave, dz_int_ave, st_own, st_int )
        (action, dz_min, dz_max, ddz, action_indiv::Vector{Z}, st_own.dz_ave_prev, multithreat) =
           ActionSelection(p.341)( mode_int, cost_ra, z_int_ave, dz_int_ave, tau_int, z_own_ave, dz_own_ave,
                equip_int, exclude_int, st_own, st_int )
        for j in 1:num_intruders
            (cvc::UInt32, vrc::UInt32, vsb::UInt32, sense_indiv::Symbol) =
               CoordinationSelection(p.361)( action_indiv[j], action, dz_min, dz_max, ddz, dz_own_ave,
                   st_own.a_prev, st_int[j].a_prev, input_int_valid[j].equipage )
            code_int[j]::UInt8 = DetermineCode(p.372)( sense_indiv )
            output_int[j] = TRMIntruderData(p.523)( input_int_valid[j].id, sense_indiv, input_own.v2v_uid,
                                input_int_valid[j].id_directory, cvc, vrc, vsb, multithreat,
                                input_int_valid[j].coordination_msg, 0.0, code_int[j] )
                                output_int[j].display.classification = input_int_valid[j].classification
                                output_int[j].display.source = input_int_valid[j].source
            sense_int[j] = sense_indiv
            source_int[j] = input_int_valid[j].source
        end
    else
        received_vrcs = UpdateIntruderVRC(p.101)( input_int_valid )
        (action, dz_min, dz_max, ddz, multithreat) = NoIntrudersAction(p.342)( dz_own_ave, st_own )
    end
    ground_alert::UInt8 = DetermineGroundAlert(p.248)( output_int )
    display::DisplayLogic(p.516) =
         DisplayLogicDetermination(p.366)( input_own.h, z_own_ave, dz_own_ave, z_int_ave, action, dz_min, dz_max,
                                   ddz, sense_int, code_int, st_own, source_int, ground_alert,
                                   input_own.opmode, input_own.trm_altmode )
    SetIntruderVerticalScores(p.372)( st_int, input_int_valid, input_own.opmode, output_int )
    report_vert::VTRMReport(p.524) =
         GenerateVTRMOutput(p.376)( input_own, display, multithreat, action, dz_min, dz_max, ddz, z_own_ave,
                   input_int_invalid, input_int_valid, output_int, received_vrcs, sxu_trm_state )
    return (report_vert::VTRMReport(p.524))
end
```

Referenced In: sXuTRMUpdate(p.164)

CreateGroundIntruder (Algorithm 233) returns the ground intruder as a TRMIntruderIn-

PUT. The algorithm first checks if the ground intruder is valid based on if the ownship's height above-ground-level and altitude are valid and the terrain avoidance is turned on by the user.

It then fills in the fields of TRMINTRUDERINPUT so that the ground intruder is always located directly below the ownship by *h_agl* and traveling at the same rate as the ownship. Identical to point obstacles, the vertical and horizontal weights are set to 1 because there is only one belief for the point.

This algorithm takes as input *z*, *h_agl*, and *perform_gpoa*. The algorithm returns *intruder*.

Algorithm 233 CreateGroundIntruder

```
function CreateGroundIntruder( z::R, h_agl::R, perform_gpoa::Bool)
    intruder = TRMIntruderInput(p. 499)()
    intruder.valid = !isnan(h_agl) && !isnan(z) && perform_gpoa
    intruder.id = ID_GROUND
    intruder.classification = CLASSIFICATION_GROUND
    intruder.degraded_surveillance = DEGRADED_SURVEILLANCE_NONE
    mu_vert = [z - h_agl, 0.0]
    mu_hor = [0.0, 0.0, 0.0, 0.0]
    vweights = 1.0
    resize!(intruder.belief_vert,length(vweights))
    intruder.belief_vert[1] = IntruderVerticalBelief(p. 500)()
    intruder.belief_vert[1].z = mu_vert[1]
    intruder.belief_vert[1].dz = mu_vert[2]
    intruder.belief_vert[1].weight = vweights
    hweights = 1.0
    resize!(intruder.belief_horiz,length(hweights))
    intruder.belief_horiz[1] = IntruderHorizontalBelief(p. 500)()
    intruder.belief_horiz[1].x_rel = mu_hor[1]
    intruder.belief_horiz[1].dx_rel = mu_hor[2]
    intruder.belief_horiz[1].y_rel = mu_hor[3]
    intruder.belief_horiz[1].dy_rel = mu_hor[4]
    intruder.belief_horiz[1].weight = hweights
    return (intruder::TRMIntruderInput(p. 499))
end
```

Referenced In: VerticalTRMUpdate(p. 241)

GETVTRMOWNDATA (Algorithm 234) returns the ownship pressure/HAE altitude estimate in feet, the ownship vertical rate estimate in feet/second, and the value to use for ownship height above ground level in feet. It also updates the state variables associated with low altitude resolution advisory inhibits.

The algorithm first computes the weighted mean for ownship pressure/HAE altitude (*z_own_ave*) and ownship vertical rate (*dz_own_ave*) from the ownship vertical beliefs by calling GETOWNWEIGHTEDAVERAGES.

It then determines the value to be used for ownship height AGL (*height_own*).

The algorithm finishes by calling UPDATEALTITUDEINHIBITCSTATE where it updates the setting of *inhibited* in ALTITUDEINHIBITCSTATE (*st_own.st_altitude_inhibit*). UPDATEALTITUDEINHIBITCSTATE also sets the low level desent inhibits states *s_c.ldi* with a call to DETERMINELDI. Note that altitude inhibits are only applicable if the deviation with the altitude inhibit receive function is implemented.

This algorithm takes as input vertical *input_own* and vertical *st_own*. The algorithm returns *z_-*

own_ave, *dz_own_ave*, and *height_own*. GETVTRMOWNDATA modifies *st_own.st_alt_inhibit*.

Algorithm 234 GetVTRMOwnData

```
function GetVTRMOwnData( input_own::TRMOwnInput(p. 498), st_own::TRMOwnState(p. 511) )
    (z_own_ave::R, dz_own_ave::R ) = GetOwnWeightedAverages(p. 243)(input_own.belief_vert)
    height_own::R = GetOwnHeight(p. 412)( input_own.h, z_own_ave )
    for mode_idx::Z in 1:length( st_own.st_alt_inhibit )
        UpdateAltitudeInhibitCState(p. 244)( mode_idx, input_own.h, input_own.h_lo_ft,
                                             input_own.h_hi_ft, st_own.st_alt_inhibit[mode_idx] )
    end
    return (z_own_ave::R, dz_own_ave::R, height_own::R)
end
```

Referenced In: VerticalTRMUpdate(p. 241)

GETOWNWEIGHTEDAVERAGES (Algorithm 235) computes the ownship pressure/HAE altitude (*z_own_ave*) and ownship vertical rate (*dz_own_ave*) from a weighted mean of the ownship vertical beliefs. The weight for each belief state represents how much importance is placed on a specific vertical sample. The weights for each sample come from the ownship input, which contains a vector of ownship vertical beliefs. Weights for the vertical belief are set in the STM and passed along in the report structure that is outputted by the STM.

This algorithm takes as input *belief_vert_own*. The algorithm returns *z_own_ave* and *dz_own_ave*.

Algorithm 235 GetOwnWeightedAverages

```
function GetOwnWeightedAverages( belief_vert_own::Vector{OwnVerticalBelief(p. 501)} )
    z_own_ave::R  = 0.0
    dz_own_ave::R = 0.0
    for b::OwnVerticalBelief(p. 501) in belief_vert_own
        z_own_ave  =  z_own_ave + (b.z  * b.weight)
        dz_own_ave = dz_own_ave + (b.dz * b.weight)
    end
    return (z_own_ave::R, dz_own_ave::R)
end
```

Referenced In: GetVTRMOwnData(p. 243), SetThreatIdentityData(p. 401)

UPDATEALTITUDEINHIBITCSTATE (Algorithm 236) is used to update the state variables used in ALTITUDEINHIBITCOST once per cycle. On the first processing cycle after start-up, the values in inhibit vector (*s_c.inhibited*) are initialized using the parameter file.

First, the validity of the ownship height above-ground-level (AGL) is considered. If valid (not NaN), then the counter for consecutive cycles of NaN AGL (*s_c.invalid_agl_cycles*) is reset to 0. If invalid (NaN), then the counter is incremented.

The inhibited vector is implicitly initialized to the precious cycle's state because *s_c.inhibited* is not reset between cycles. The algorithm then iterates over each inhibit and compares the altitude of the own aircraft (*h_own*) to parameter defined thresholds. If ownship height above-ground-level is valid and below the low threshold for a given inhibit (*h_lo_ft[i]*), the inhibit is enabled. If

ownship height above-ground-level is valid and at or above the high threshold for a given inhibit (*h_hi_ft[i]*), the inhibit is disabled. If the ownship height above-ground-level is invalid for more than *AGL_LOST* cycles consecutively, then the previous cycle inhibit state is used. If ownship height above-ground-level is invalid and (*s_c.invalid_agl_cycles*) is greater than *AGL_LOST*, the inhibit is disabled.

DETERMINELDI is used to obtain the low-level descend inhibit value (*s_c.ldi*) based on the values in *s_c.inhibited*.

This algorithm takes as input *mode_idx*, *h_own*, and *s_c*. It updates *s_c::AltitudeInhibitCState*, which is passed by reference.

Algorithm 236 UpdateAltitudeInhibitCState

```
function UpdateAltitudeInhibitCState( mode_idx::Z, h_own::R, h_lo_ft::Vector{R}, h_hi_ft::Vector{R}, s_c::
    AltitudeInhibitCState(p. 527) )
    B_init::Vector{Bool} = params().modes[mode_idx].cost_estimation.online.altitude_inhibit.B_init
    if !s_c.initialized
        for i = 1:length( B_init )
            s_c.inhibited[i] = B_init[i]
        end
        s_c.initialized = true
    end
    if (!isnan(h_own))
        s_c.invalid_agl_cycles = 0
    else
        s_c.invalid_agl_cycles += 1
    end
    for i = 1:length( B_init )
        if (!isnan(h_own))
            if (h_own < h_lo_ft[i])
                s_c.inhibited[i] = true
            elseif (h_hi_ft[i] <= h_own)
                s_c.inhibited[i] = false
            end
        elseif (s_c.invalid_agl_cycles > AGL_LOST)
            s_c.inhibited[i] = false
        end
    end
    s_c.ldi = DetermineLDI(p. 245)( mode_idx, s_c.inhibited )
    return
end
```

Referenced In: GetVTRMOwnData(p. 243)

DETERMINELDI (Algorithm 237) iterates over each entry in *inhibited*. There is an entry in *inhibited* to correspond with each altitude inhibit rule. In order for an altitude inhibit to be active, the value of *inhibited* for the current entry must be TRUE and a penalty is to be applied. The altitude inhibit rules use minimum and maximum vertical rates to represent the RAs pertaining to each rule.

The algorithm first obtains the altitude inhibit rules from the *cost_estimation.online.altitude_-inhibit* parameter structure. It also obtains the vertical rate for climb and descend RAs from the parameter *actions.corrective_rate*

If the maximum vertical rate is negative, indicating a descend RA, a decision is made as to whether the inhibit is for all positive descend RAs or whether it is for increased rate descend RAs only. That

determination is made by comparing the maximum vertical rate in the altitude inhibit rule (*R_dz_hi[i]*) against the corrective rate (*R_corrective*). If the vertical rate is greater than the corrective rate, the altitude inhibit is for an increased rate descend RA.

Otherwise, if the maximum vertical rate is +Infinity and the minimum vertical rate is -Infinity, all RAs will be inhibited.

ldi is then set based on which active altitude inhibit encompasses the most RAs.

The parameter sets used in the DETERMINELDI have the following effects on alerting:

- Descend Inhibit - Inhibits initiation of Descend, Increase Descend, or Maintain (Down RA). If any of these are already issued, converts it to a Do Not Climb RA.

This algorithm takes as input *mode_idx* and *inhibited*. The algorithm returns *ldi*.

Algorithm 237 DetermineLDI

```
function DetermineLDI( mode_idx::Z, inhibited::Vector{Bool} )
    C_inhibit::Vector{R} =
        params().modes[mode_idx].cost_estimation.online.altitude_inhibit.C_inhibit
    R_dz_hi::Vector{R} =
        params().modes[mode_idx].cost_estimation.online.altitude_inhibit.R_dz_hi
    R_dz_lo::Vector{R} =
        params().modes[mode_idx].cost_estimation.online.altitude_inhibit.R_dz_lo
    R_corrective::R = params().actions.corrective_rate
    ldi::UInt8 = LDI_NONE
    inhibited_inc_descend::Bool = false
    inhibited_descends::Bool    = false
    inhibited_all::Bool         = false
    for i = 1:length( inhibited )
        if inhibited[i] && (0 < C_inhibit[i])
            if (R_dz_hi[i] < 0)
                if (abs( R_dz_hi[i] ) <= R_corrective)
                    inhibited_descends = true
                else
                    inhibited_inc_descend = true
                end
            elseif (Inf == R_dz_hi[i]) && (-Inf == R_dz_lo[i])
                inhibited_all = true
            end
        end
    end
    if inhibited_all
        ldi = LDI_ALL
    elseif inhibited_descends
        ldi = LDI_DESCENDS
    elseif inhibited_inc_descend
        ldi = LDI_INCREASE_DESCEND
    end
    return ldi::UInt8
end
```

Referenced In: UpdateAltitudeInhibitCState(p. 244)

SETVERTICALTRMSURVONLYMODE (Algorithm 238) is used when own aircraft is in Surveillance Only Operational Mode to keep the internal state of the TRM and information on intruders updated, such that the own aircraft will be able to issue RAs against intruders when it returns to RA Mode.

All valid intruders are looped over to update, if needed, the online cost state, *st_int[j].st_cost_on*, the exclude boolean, *exclude_int*, and the state's proccessing field, *st_int[j].processing*. If the intruder is a point obstacle or the ground point obstacle, exclude is set to true and the online cost state is set back to the default values. The state's processing field is set to indicate degraded RA processing, the exclude boolean is set to true, and the online cost state is set to the default values when the intruder is non-altitute reporting or the passive intruder is not validated. Otherwise, the state's processing field is set to the surveillance only RA processing, the state's COC cost is set to zero and UPDATEINITIALIZATIONCSTATE (Algorithm 247) is called to increment the initialization count. As part of the initialization, *mode_int* is set to *NO_Xo_MODE_INDEX* and does not require a mode look up as sXu does not support multiple modes.

After looping through the valid intruders, the intruders' VRCs are updated, NOINTRUDERSACTION (Algorithm 316) is called to update ownship information to reflect no advisory, and then SETVERTICALTRMSURVONLYMODE loops through the valid intruders again. If the intruder in not to be excluded, UPDATECROSSINGNOALERTCSTATE (Algorithm 301) is called to update the crossing state. Then for each intruder, *output_int* is created by calling COORDINATIONSELECTION (Algorithm 330) and the constructor for TRMINTRUDERDATA to be used for the TRM Report.

This algorithm modifies *output_int*, *z_int_ave*, *dz_int_ave* and *st_int*.

SETVERTICALTRMSURVONLYMODE takes as input *height_own*, *z_own_ave*, *dz_own_ave*, *input_own*, *input_int_valid*, *output_int*, *mode_int*, *z_int_ave*, *dz_int_ave*, *st_own*, and *st_int*. The outputs for the algorithm are *received_vrcs*, *action*, *dz_min*, *dz_max*, *ddz*, and *multithreat*.

Algorithm 238 SetVerticalTRMSurvOnlyMode

```
function SetVerticalTRMSurvOnlyMode(height_own::R, z_own_ave::R, dz_own_ave::R, input_own::TRMOwnInput(p. 498),
                        input_int_valid::Vector{TRMIntruderInput(p. 499)},
                        output_int::Vector{TRMIntruderData(p. 523)},
                        mode_int::Vector{Z}, z_int_ave::Vector{R}, dz_int_ave::Vector{R},
                        st_own::TRMOwnState(p. 511), st_int::Vector{TRMIntruderState(p. 512)} )
    N_intruders::Z          = length( input_int_valid )
    vrc_int::Vector{UInt32} = zeros( UInt32, N_intruders )
    equip_int::Vector{Bool} = zeros( Bool,   N_intruders )
    b_tau_int_all::Vector{Vector{TauBelief(p. 522)}} = []
    exclude_int::Vector{Bool} = zeros( Bool,   N_intruders )
    for j in 1:N_intruders
        if ( input_int_valid[j].classification == CLASSIFICATION_POINT_OBSTACLE ) ||
           ( input_int_valid[j].classification == CLASSIFICATION_GROUND )
            exclude_int[j] = true
            b_tau_int_none::Vector{TauBelief(p. 522)} = []
            push!( b_tau_int_all, b_tau_int_none )
            st_int[j].st_cost_on = OnlineCostState(p. 520)()
        elseif  ( 0 != (input_int_valid[j].degraded_surveillance & DEGRADED_SURVEILLANCE_NAR) ) ||
           ( 0 != (input_int_valid[j].degraded_surveillance & DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED) )
            st_int[j].processing = RA_PROCESSING_DEGRADED_SURVEILLANCE
            exclude_int[j] = true
            b_tau_int_none = []
            push!( b_tau_int_all, b_tau_int_none )
            st_int[j].st_cost_on = OnlineCostState(p. 520)()
        else
            st_int[j].processing = RA_PROCESSING_GLOBAL_SURV_ONLY
            z_int_ave[j]  = 0.0
            dz_int_ave[j] = 0.0
            for b::IntruderVerticalBelief(p. 500) in input_int_valid[j].belief_vert
                z_int_ave[j]  = z_int_ave[j]  + (b.z  * b.weight)
                dz_int_ave[j] = dz_int_ave[j] + (b.dz * b.weight)
            end
            mode_int[j] = NO_Xo_MODE_INDEX
            (_, b_tau_int::Vector{TauBelief(p. 522)}) =
               StateEstimation(p. 257)( mode_int[j], st_int[j].b_prev, input_own.belief_vert,
                                input_int_valid[j].belief_vert, input_int_valid[j].belief_horiz,
                                height_own, z_own_ave, dz_own_ave, z_int_ave[j], dz_int_ave[j],
                                st_int[j].st_cost_on, st_int[j].idx_scale, false, false,
                                st_int[j].idx_online_cost )
            push!( b_tau_int_all, b_tau_int )
            st_int[j].coc_cost = 0.0
            UpdateInitializationCState(p. 261)(mode_int[j], st_int[j].st_cost_on.initialization)
        end
    end
    received_vrcs::Vector{Bool} = UpdateIntruderVRC(p. 101)( input_int_valid )
    (action::Z, dz_min::R, dz_max::R, ddz::R, multithreat::Bool) =
        NoIntrudersAction(p. 342)( dz_own_ave, st_own )
    for j in 1:N_intruders
        if (exclude_int[j] == false)
            UpdateCrossingNoAlertCState(p. 325)( z_own_ave, z_int_ave[j], vrc_int[j],
                                        st_int[j].st_cost_on.crossing_no_alert )
        end
        (cvc::UInt32, vrc::UInt32, vsb::UInt32, sense_indiv::Symbol) =
           CoordinationSelection(p. 361)( COC, action, dz_min, dz_max, ddz, dz_own_ave,
                st_own.a_prev, st_int[j].a_prev, input_int_valid[j].equipage )
        (output_int[j]) =
            TRMIntruderData(p. 523)( input_int_valid[j].id, sense_indiv, input_own.v2v_uid,
                input_int_valid[j].id_directory, cvc, vrc, vsb, multithreat,
                input_int_valid[j].coordination_msg, 0.0, UInt8(SXUCODE_CLEAR) )
    end
    return (received_vrcs::Vector{Bool}, action::Z, dz_min::R, dz_max::R, ddz::R, multithreat::Bool)
end
```

Referenced In: VerticalTRMUpdate(p. 241)

DETERMINEGROUNDALERT (Algorithm 239) returns an integer indicating if the ground did not contribute to an RA, if the only intruder to cause an RA is ground, or if ground contributes to a mixed RA.

This algorithm takes as input *output_int*, a vector containing TRM intruder data. The algorithm loops through all the intruders stored in *output_int* and increments *num_int* for every intruder that causes an RA. If an intruder that caused an RA is also the ground intruder, *is_ground_ra* is set to true. Then DETERMINEGROUNDALERT sets *ground_alert* by checking both *is_ground_ra* and *num_int*.

The algorithm returns *ground_alert*.

Algorithm 239 DetermineGroundAlert

```
function DetermineGroundAlert( output_int::Vector{TRMIntruderData(p. 523)} )
    num_int::Z = 0
    is_ground_ra::Bool = false
    ground_alert::UInt8 = GROUND_ALERT_NONE
    for intruder::TRMIntruderData(p. 523) in output_int
        if intruder.display.code == SXUCODE_RA
            num_int += 1
            if intruder.id == ID_GROUND
                is_ground_ra = true
            end
        end
    end
    if is_ground_ra
        if (num_int == 1)
            ground_alert = GROUND_ALERT_ONLY
        else
            ground_alert = GROUND_ALERT_MIXED
        end
    end
    return (ground_alert::UInt8)
end
```

Referenced In: VerticalTRMUpdate(p. 241)

3.3.2 Vertical Logic State and Cost Estimation

An overview of the STATEANDCOSTESTIMATION algorithm of the Threat Resolution Module is shown in Figure 8. This algorithm evaluates the probabilistic threat represented by each individual intruder.

STATEANDCOSTESTIMATION (Algorithm 240) has four general processing elements.

- Initialize the variables.
- Loop over the valid intruders to compute values not dependent on resolution advisory coordination inputs. This first loop calls INTRUDERPREP (Section 3.3.2.1), STATEESTIMATION (Section 3.3.2.2), OFFLINECOSTESTIMATION (Section 3.3.2.3), and ONLINEUNCOORDINATEDCOSTESTIMATION (Section 3.3.2.4).
  - INTRUDERPREP preprocesses the inputs for each intruder to account for ACAS X operational modes (Standby, Surveillance Only, and RA).
  - STATEESTIMATION produces vertical samples and tau beliefs for estimating offline and online costs.

  - OFFLINECOSTESTIMATION uses the weighted samples to index into the precomputed policy table to determine alerting.
  - ONLINEUNCOORDINATEDCOSTESTIMATION estimates online coordination-independent costs for each action based on the current and previous dynamic and advisory states of own aircraft and the individual intruder aircraft.
- Acquire the most recent VRC values for the valid intruders using UPDATEINTRUDERINPUTS (Section 3.3.2.5).
- Loop over intruders to compute cost values dependent on resolution advisory coordination inputs and then compute the cost totals. This final loop calls ONLINECOSTESTIMATION (Section 3.3.2.6) when the intruder is not a point obstacle. Point obstacles do not have an online coordinated cost therefore the total online cost is set equal to the uncoordinated cost for this intruder type. Then INDIVIDUALCOSTESTIMATION (Section 3.3.2.7) is called in the loop for all intruder types.
  - ONLINECOSTESTIMATION estimates online coordination-dependent costs for each action based on the current and previous dynamic and advisory states of own aircraft and the individual intruder aircraft.
  - INDIVIDUALCOSTESTIMATION adjusts and sums the offline and online costs for the individual intruder aircraft.

Each of these algorithms is described in detail below.

STATEANDCOSTESTIMATION takes as inputs *height_own*, *z_own_ave*, *dz_own_ave*, *input_own*, *input_int_valid*, *mode_int*, *z_int_ave*, *dz_int_ave*, *st_own* and *st_int*. Descriptions of the STATEANDCOSTESTIMATION internal modicications and output variables are found in Table 38.

Appendix G describes the functionality that enables scaling of the offline policy to make it more or less sensitive. Selection of scaling parameters throughout the algorithms in this section is controlled by the variable *idx_scale*, which specifies the row of scaling values to utilize in the scaling matrix.

Table 38. Vertical TRM Algorithm Modifications and Output Variables - StateAndCostEstimation

Variable	Units	Type	Description
cost_ra	N/A	real	Matrix of intruder-specific costs to be used for issuing RAs
dz_int_ave	ft/s	real	Vector of averaged vertical rate for each intruder
equip_int	N/A	bool	Vector of whether each intruder can produce RA coordination messages
exclude_int	N/A	bool	Vector of whether each intruder should be excluded from RA processing
mode_int	N/A	integer	Vector of protection mode index for each intruder
received_vrcs	N/A	bool	Union of all VRCs received by STM; used as RAC
st_int	N/A	TRMIntruderState(p. 512)	Vector of state information for each intruder
tau_int	s	real	Vector of expected time of closest point of approach for each intruder
z_int_ave	ft	real	Vector of averaged altitude for each intruder

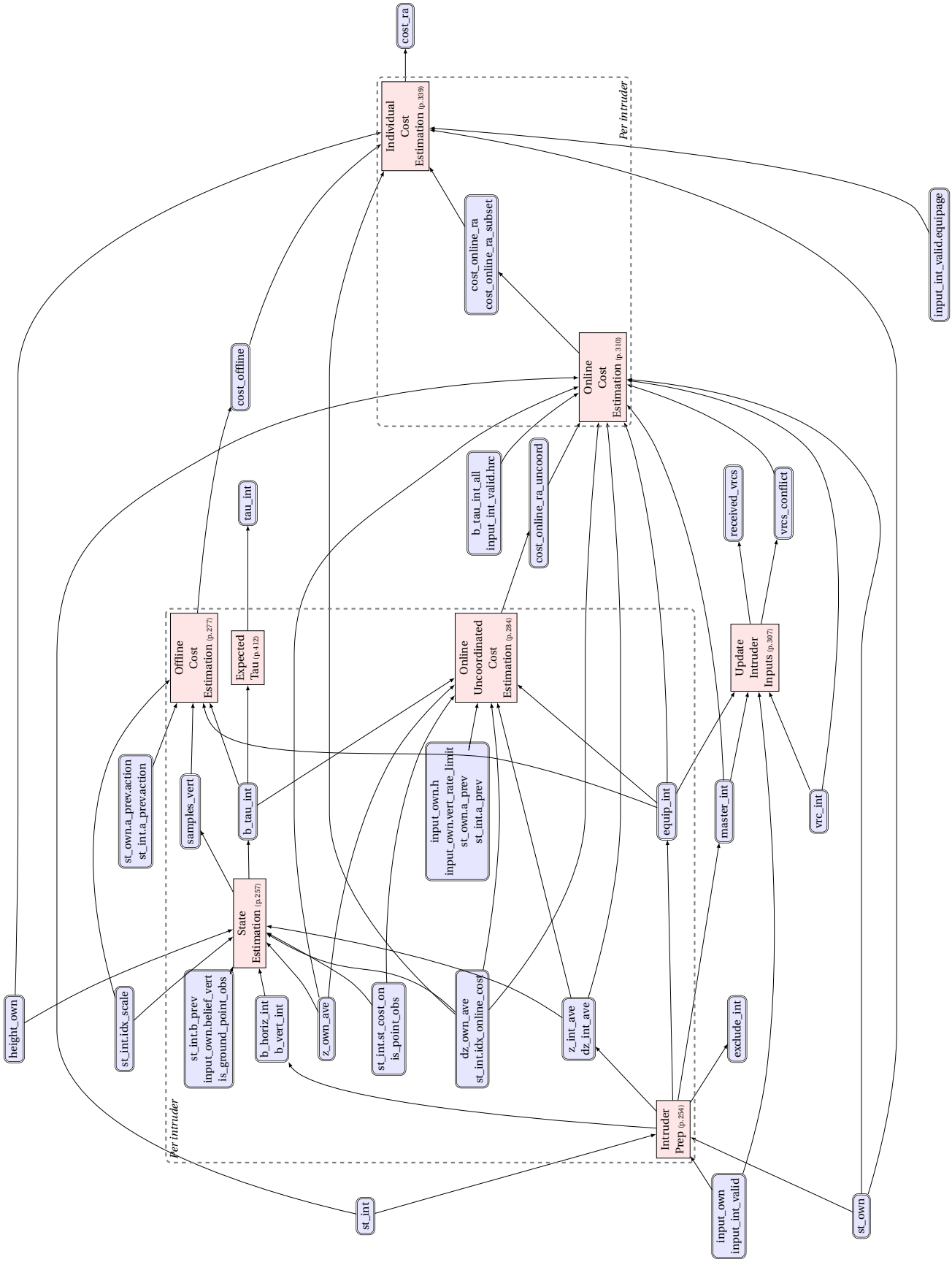

Figure 8. StateAndCostEstimation overview.

Algorithm 240 StateAndCostEstimation

```
function StateAndCostEstimation( height_own::R, z_own_ave::R, dz_own_ave::R, input_own::TRMOwnInput(p.498),
            input_int_valid::Vector{TRMIntruderInput(p.499)}, mode_int::Vector{Z}, z_int_ave::Vector{R},
            dz_int_ave::Vector{R}, st_own::TRMOwnState(p.511), st_int::Vector{TRMIntruderState(p.512)} )
    N_actions::Z   = params().actions.num_actions
    N_intruders::Z = length( input_int_valid )
    cost_offline::Matrix{R}           = zeros( R, N_intruders, N_actions )
    cost_online_ra_uncoord::Matrix{R} = zeros( R, N_intruders, N_actions )
    cost_online_ra::Vector{R}         = zeros( R, N_actions )
    cost_online_ra_subset::Vector{R}  = zeros( R, N_actions )
    cost_ra::Matrix{R}                = zeros( R, N_intruders, N_actions )
    equip_int::Vector{Bool}           = zeros( Bool,   N_intruders )
    master_int::Vector{Bool}          = zeros( Bool,   N_intruders )
    exclude_int::Vector{Bool}         = zeros( Bool,   N_intruders )
    vrc_int::Vector{UInt32}           = zeros( UInt32, N_intruders )
    tau_int::Vector{R}                = zeros( R,      N_intruders )
    is_point_obs::Bool                = false;
    is_ground_point_obs::Bool         = false;
    b_tau_int_all::Vector{Vector{TauBelief(p.522)}} = []
#
    for j in 1:N_intruders
        (mode_int[j], equip_int[j], master_int[j], exclude_int[j],
         b_horiz_int::Vector{IntruderHorizontalBelief(p.500)}, b_vert_int::Vector{IntruderVerticalBelief(p.500)},
         z_int_ave[j], dz_int_ave[j]) = IntruderPrep(p.254)(input_own, input_int_valid[j], st_own, st_int[j])
        if ( input_int_valid[j].classification == CLASSIFICATION_POINT_OBSTACLE ) || ( input_int_valid[j].
             classification == CLASSIFICATION_GROUND )
            #Point obstacles are expected to have one belief
            is_point_obs = true
            if input_int_valid[j].classification == CLASSIFICATION_GROUND
                is_ground_point_obs = true
            else
                is_ground_point_obs = false
            end
        else
            is_point_obs = false
            is_ground_point_obs = false
        end
        (samples_vert::Vector{CombinedVerticalBelief(p.515)}, b_tau_int::Vector{TauBelief(p.522)}) =
           StateEstimation(p.257)( mode_int[j], st_int[j].b_prev, input_own.belief_vert, b_vert_int,
            b_horiz_int, height_own, z_own_ave, dz_own_ave, z_int_ave[j], dz_int_ave[j],
            st_int[j].st_cost_on, st_int[j].idx_scale, is_point_obs, is_ground_point_obs,
            st_int[j].idx_online_cost )
        cost_offline[j,:] = OfflineCostEstimation(p.277)( mode_int[j], samples_vert, b_tau_int,
            equip_int[j], st_own.a_prev.action, st_int[j].a_prev.action, st_int[j].idx_scale )
        st_int[j].coc_cost = cost_offline[j,COC]
        if (exclude_int[j] == false)
            cost_online_ra_uncoord[j,:] =
                OnlineUncoordinatedCostEstimation(p.284)( mode_int[j], input_own.h, input_own.vert_rate_limit,
                    z_own_ave, dz_own_ave, z_int_ave[j], dz_int_ave[j], b_tau_int, st_own.a_prev,
                    st_int[j].a_prev, equip_int[j], st_int[j].st_cost_on, is_point_obs,
                    st_int[j].idx_online_cost )
            tau_int[j] = ExpectedTau(p.412)( b_tau_int, false )
        else
            UpdateInitializationCState(p.261)(mode_int[j], st_int[j].st_cost_on.initialization)
        end
        push!( b_tau_int_all, b_tau_int )
    end
    # IMPORTANT: Input any new V2VCoordination messages via ReceiveV2VCoordination() by this point
    (received_vrcs::Vector{Bool}, vrcs_conflict::Bool) =
       UpdateIntruderInputs(p.307)( input_int_valid, equip_int, master_int, vrc_int, input_own )
#
    for j in 1:N_intruders
        if ( input_int_valid[j].classification == CLASSIFICATION_POINT_OBSTACLE ) || ( input_int_valid[j].
             classification == CLASSIFICATION_GROUND )
            #Point obstacles are expected to have one belief
            is_point_obs = true
        else
```

```
            is_point_obs = false
        end
        if (exclude_int[j] == false)
            if (!is_point_obs)
                (cost_online_ra, cost_online_ra_subset) =
                    OnlineCostEstimation(p. 310)( mode_int[j], z_own_ave, dz_own_ave, z_int_ave[j],
                        dz_int_ave[j], b_tau_int_all[j], st_own, st_int[j], master_int[j], vrc_int[j],
                        input_int_valid[j].hrc, equip_int[j], vrcs_conflict, cost_online_ra_uncoord[j,:],
                        st_int[j].idx_online_cost )
            else
                #Point obstacles are expected to have one belief
                cost_online_ra = cost_online_ra_uncoord[j,:]
                cost_online_ra_subset = cost_online_ra_uncoord[j,:]
            end
            (cost_ra[j,:]) = IndividualCostEstimation(p. 339)( mode_int[j], height_own, dz_own_ave,
                        input_int_valid[j].equipage, st_own, cost_offline[j,:], cost_online_ra,
                        cost_online_ra_subset, st_int[j].idx_online_cost )
        else
            UpdateCrossingNoAlertCState(p. 325)( z_own_ave, z_int_ave[j], vrc_int[j],
                                        st_int[j].st_cost_on.crossing_no_alert )
        end
    end
    return (equip_int::Vector{Bool}, exclude_int::Vector{Bool}, tau_int::Vector{R},
        cost_ra::Matrix{R}, received_vrcs::Vector{Bool})
end
```

Referenced In: VerticalTRMUpdate(p. 241)

3.3.2.1 Intruder Prep

INTRUDERPREP (Algorithm 241) is used to prepare the intruder inputs for processing. It sets fields in the intruder state, *st_int*, as well as returning values to be stored in processing arrays. Note that in ACAS sXu the operational mode Designated No Alerts (DNA) Xo mode is not available.

Next the value of *st_int.is_threat* is cleared to indicate that this intruder is not recognized as a threat for this cycle. The output variable *belief_vert_int* is initialized with the contents of *input_-int.belief_vert*.

The first conditional block sets the processing mode based on the quality of the surveillance for this intruder and the ACAS X operational mode. The TRM processing is adjusted if the quality of the surveillance is inadequate for producing resolution advisories. The quality of the surveillance from the STM is indicated by *input_int.degraded_surveillance*, which is a bit vector. Each bit represents a specific type of degraded surveillance. If no bits are set, the value of *input_-int.degraded_surveillance* is 0 (*DEGRADED_SURVEILLANCE_NONE*) indicating no degradation. TRM processing for this intruder is based on the setting of the *degraded_surveillance* bits as follows:

- A non-zero value for the *DEGRADED_SURVEILLANCE_NAR* bit indicates Non-Altitude Reporting (NAR) surveillance. This type of surveillance is inadequate for producing vertical resolution advisories (RAs). The intruder is flagged by INTRUDERPREP for degraded surveillance processing: (*st_int.processing* is set to *RA_PROCESSING_DEGRADED_SURVEILLANCE*). The vertical beliefs for own aircraft are used to populate *belief_vert_int*, replacing the intruder vertical beliefs missing in the input. An intruder with this type of surveillance degradation will be excluded from determination of the global RA.

- A non-zero value for the *DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED* bit indicates passive only surveillance (ADS-B/V2V/AGT), with no external validation. This type of surveillance is inadequate for producing resolution advisories (RAs). The intruder is flagged by INTRUDERPREP for degraded surveillance processing: (*st_int.processing* is set to *RA_PROCESSING_DEGRADED_SURVEILLANCE*). An intruder with this type of surveillance degradation will be excluded from determination of the global RA.
- A non-zero value for the *DEGRADED_SURVEILLANCE_NO_BEARING* bit indicates surveillance with no valid bearing estimate. This type of surveillance is inadequate for producing horizontal resolution advisories (RAs) and horizontal Remain Well Clear guidance. There is no special processing for this case in INTRUDERPREP. Processing of this degraded surveillance occurs in the Horizontal TRM in PRIORITIZEANDFILTERINTRUDERS. Because no special processing is required, this case is not called out in the conditional in this algorithm.

When the surveillance quality is adequate for producing resolution advisories (RAs), the ownship ACAS X operational mode setting (*input_own.opmode*) is used to determine the type of TRM processing to be used for the intruder as follows:

- When *input_own.opmode* is *TRM_OPMODE_RA*, own aircraft is in ACAS X RA operational mode, and non-degraded surveillance intruders are flagged with *st_int.processing* set to *RA_PROCESSING_GLOBAL_RA*. If the previous cycle used degraded surveillance processing for this intruder, the full cost state is reset. A full reset forces a reset of the initialization counter in INITIALIZATIONCSTATE to enforce the initial delay for track stabilization. It fully clears the online cost state for selecting resolution advisories. Otherwise, there is no reset of the online cost state in this algorithm. In the case of an intruder that was invalid on the previous cycle, the entire intruder online cost state would have been reset on the previous cycle.
- When *input_own.opmode* is *TRM_OPMODE_SURV_ONLY*, own aircraft is in ACAS X Surveillance Only operational mode. *st_int.processing* is set to *RA_PROCESSING_GLOBAL_-SURV_ONLY*. The advisory state was partially reset on the previous cycle.
- When *input_own.opmode* is *TRM_OPMODE_STANDBY*, own aircraft is in Standby ACAS X operational mode. In Standby mode no resolution advisories (RAs) are produced. When ownship is in Standby ACAS X operational mode, all intruders are invalid for vertical TRM processing, and therefore all intruders **must** be omitted from inputs to INTRUDERPREP. For that reason, this setting is not called out in the conditional.

Next, the values to be used for the intruder altitude (*z_int_ave*) and vertical rate (*dz_int_ave*) are determined by taking the weighted mean of the vertical beliefs for each of those values.

mode_int is an array index used throughout the TRM to retrieve sXu parameters for use. It is set to *NO_Xo_MODE_INDEX*, as support for Xo does not exist in Xu or sXu The value of *protect_mode_-int* must be validated prior to running the TRM. *exclude_int* is set to TRUE when the intruder is marked as having certain types of degraded surveillance.

Next, *st_int.processing* is checked for whether the intruder is excluded from determination of the global RA (*exclude_int*), which is based on the results of the processing above.

The altitude inhibit cost state (ALTITUDEINHIBITCSTATE) for the intruder is assigned based on the protection mode selected for this cycle. Finally, if the ownship operational mode transitioned from *TRM_OPMODE_SURV_ONLY* on the last cycle to RA mode on this cycle, COORDINA-

TIONDELAYCOST is activated by resetting COORDINATIONDELAYCSTATE with *is_count_enabled* initialized to TRUE. Otherwise, COORDINATIONDELAYCSTATE settings from the previous cycle are retained.

If the ownship's coordination policy against the intruder is not a peer or junior, then COORDINATIONDELAYCOST is deactivated.

This algorithm takes as input *input_own*, *input_int*, *st_own*, and *st_int*. This algorithm returns *mode_int*, *equip_int*, *master_int*, *exclude_int*, *input_int.belief_horiz*, *belief_vert_int*, *z_int_ave*, and *dz_int_ave*. It also sets fields in the intruder state *st_int*.

Algorithm 241 IntruderPrep

```
function IntruderPrep( input_own::TRMOwnInput(p.498), input_int::TRMIntruderInput(p.499),
                       st_own::TRMOwnState(p.511), st_int::TRMIntruderState(p.512) )
    st_int.is_threat = false
    belief_vert_int::Vector{IntruderVerticalBelief(p.500)} = deepcopy( input_int.belief_vert )
    if (0 != (input_int.degraded_surveillance & DEGRADED_SURVEILLANCE_NAR))
        st_int.processing = RA_PROCESSING_DEGRADED_SURVEILLANCE
        belief_vert_int = Vector{IntruderVerticalBelief(p.500)}( undef, length( input_own.belief_vert ) )
        for i in 1:length( input_own.belief_vert )
            belief_vert_int[i] = IntruderVerticalBelief(p.500)( input_own.belief_vert[i].z,
                                                                input_own.belief_vert[i].dz,
                                                                input_own.belief_vert[i].weight )
        end
    elseif (0 != (input_int.degraded_surveillance & DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED))
         st_int.processing = RA_PROCESSING_DEGRADED_SURVEILLANCE
    elseif (OPMODE_RA == input_own.opmode)
        if (RA_PROCESSING_DEGRADED_SURVEILLANCE == st_int.processing)
            st_int.st_cost_on = OnlineCostState(p.520)()
        end
        st_int.processing = RA_PROCESSING_GLOBAL_RA
    end
    z_int_ave::R  = 0.0
    dz_int_ave::R = 0.0
    for b::IntruderVerticalBelief(p.500) in belief_vert_int
        z_int_ave  = z_int_ave  + (b.z  * b.weight)
        dz_int_ave = dz_int_ave + (b.dz * b.weight)
    end
    mode_int::Z       = NO_Xo_MODE_INDEX
    exclude_int::Bool = (RA_PROCESSING_DEGRADED_SURVEILLANCE == st_int.processing)
    equip_int::Bool = IsIntruderEquipped(p.423)(input_own.opmode, input_int.equipage)
    master_int::Bool  = IsIntruderMaster(p.424)( input_own, input_int )
    st_int.st_cost_on.alt_inhibit = st_own.st_alt_inhibit[mode_int]
    if (OPMODE_SURV_ONLY == st_own.opmode_prev) && (OPMODE_RA == input_own.opmode)
        st_int.st_cost_on.coord_delay = CoordinationDelayCState(p.528)( true )
    end
    if (OWN_COORDINATION_POLICY_PEER != input_int.own_coordination_policy) && (
         OWN_COORDINATION_POLICY_JUNIOR != input_int.own_coordination_policy)
        st_int.st_cost_on.coord_delay = CoordinationDelayCState(p.528)( false )
    end
    return (mode_int::Z, equip_int::Bool, master_int::Bool, exclude_int::Bool, input_int.belief_horiz,
            belief_vert_int::Vector{IntruderVerticalBelief(p.500)}, z_int_ave::R, dz_int_ave::R)
end
```

Referenced In: StateAndCostEstimation(p.251)

3.3.2.2 Vertical Logic State Estimation

STATEESTIMATION (Algorithm 242) produces own aircraft and intruder state variables for use in

estimating offline and online costs. It inputs horizontal and vertical beliefs for a single intruder aircraft (*b_int_prev*, *b_vert_int*, *b_horiz_int*, *z_int_ave*, and *source_int*) and vertical beliefs for own aircraft (*b_vert_own* and *height_own*). These beliefs are received directly as generated by the STM.

The STATEESTIMATION algorithm always performs its evaluation with regard to a single intruder, independent of any other intruders.

CONVERTHORIZONTAL (Algorithm 243) is used to convert the intruder horizontal beliefs from *b_horiz_int* into beliefs for relative range in feet (*r_ground*), relative speed in feet/second (*s_ground*), and relative bearing in radians (*phi_rel*) with corresponding weights (*w_int_horiz*).

HORIZONTALWORSTCASE (Algorithm 244) is used next to compute an additional belief for the worst case condition in the horizontal dimension. If a non-zero weight belief is produced by HORIZONTALWORSTCASE, it is added to the horizontal beliefs produced by CONVERTHORIZONTAL. That will result in the worst case condition being included in the belief structure produced by TAUESTIMATION. NORMALIZE (Algorithm 468) is used to ensure the resulting weights add to 1.0. A horizontal worst case belief is added whenever the possible relative position and velocity beliefs for the intruding aircraft could include a zero horizontal miss distance trajectory.

WRAPTOPI (Algorithm 471) ensures the magnitude of the relative bearing angle is no less than zero nor greater than π radians.

TAUESTIMATION (Algorithm 248) estimates the time until horizontal conflict. It produces a set of tau beliefs (*b_tau_int*).

POINTOBSTACLETAUESTIMATION (Algorithm 254) estimated the time until horizontal conflict for a point obstacle. It produces a single tau belief (*b_tau_int*) and the projected intruder point obstacle altitude (*int_h_shifted*).

COMBINEVERTICALBELIEFS (Algorithm 255) produces a set of vertical estimates for own aircraft and the intruder, assuming no pilot response. The estimates include the vertical separation and the estimated vertical rate of each aircraft. These beliefs are stored in *samples_vert*.

SETBELIEFBASEDONLINECOSTSTATE (Algorithm 256) is used last to set online cost state variables that are dependent on horizontal and vertical state information. Those state variables are stored in the ONLINECOSTSTATE (Type 67) data structure (*s_c*).

This algorithm takes as input *mode_int*, *b_int_prev*, *b_vert_own*, *b_vert_int*, *b_horiz_int*, *height_own*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *s_c*, *isx_scale*, and *is_point_obs*. Descriptions of the STATEESTIMATION output variables are found in Table 39.

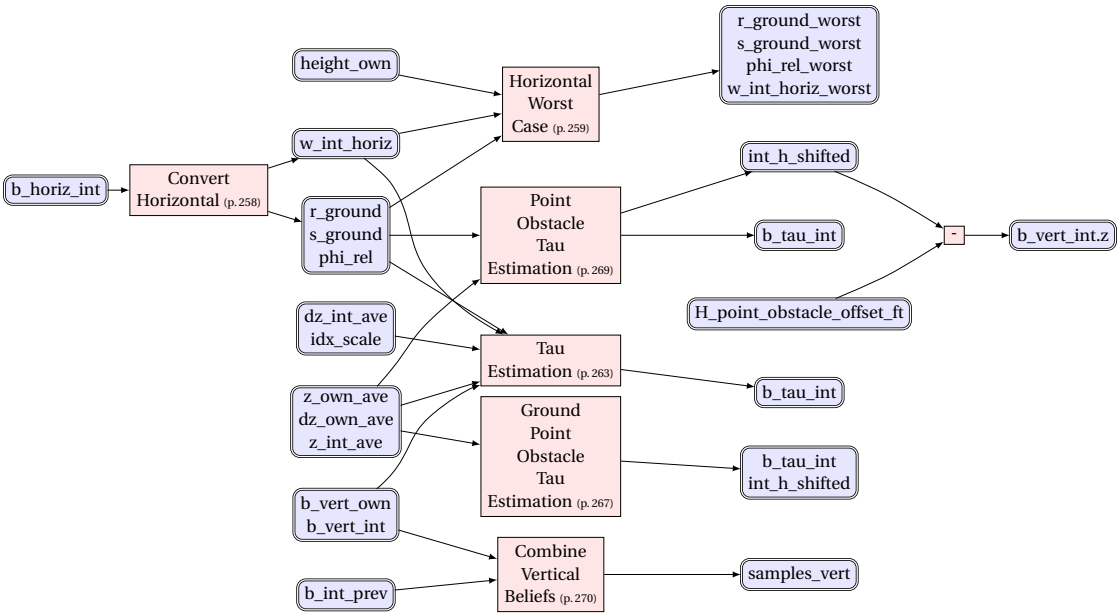

Figure 9. StateEstimation overview.

Table 39. Vertical TRM Algorithm Output Variables - StateEstimation

Variable	Units	Type	Description
b_tau_int	(s,N/A)	(real,real)	Vector of times until horizontal conflict and their weights
samples_vert	(ft,ft/s,ft/s,N/A)	(real,real,real,real)	Vector of vertical beliefs for each time until horizontal conflict

Algorithm 242 StateEstimation

```
function StateEstimation( mode_int::Z, b_int_prev::AdvisoryBeliefState(p.515),
                    b_vert_own::Vector{OwnVerticalBelief(p.501)}, b_vert_int::Vector{IntruderVerticalBelief(p.500)},
                    b_horiz_int::Vector{IntruderHorizontalBelief(p.500)}, height_own::R,
                    z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R, s_c::OnlineCostState(p.520),
                    idx_scale::Z, is_point_obs::Bool, is_ground_point_obs::Bool, idx_online_cost::Z )
     H_point_obstacle_offset_ft::R = params().threat_resolution.vertical_scaling.H_point_obstacle_offset_ft
    (r_ground::Vector{R}, s_ground::Vector{R}, phi_rel::Vector{R}, w_int_horiz::Vector{R}) =
          ConvertHorizontal(p.258)( b_horiz_int )
    (r_ground_worst::R, s_ground_worst::R, phi_rel_worst::R, w_int_horiz_worst::R) =
          HorizontalWorstCase(p.259)( mode_int, r_ground, s_ground, phi_rel, w_int_horiz, height_own )
    if (0.0 < w_int_horiz_worst)
        push!( r_ground,    r_ground_worst )
        push!( s_ground,    s_ground_worst )
        push!( phi_rel,     phi_rel_worst )
        push!( w_int_horiz, w_int_horiz_worst )
        w_int_horiz = Normalize(p.582)( w_int_horiz )
    end
    for i = 1:length(phi_rel)
        phi_rel[i] = abs( WrapToPi(p.583)( phi_rel[i] ) )
    end
    if ( !is_point_obs )
        b_tau_int::Vector{TauBelief(p.522)} =
            TauEstimation(p.263)( mode_int, z_own_ave, dz_own_ave, z_int_ave, dz_int_ave,
                  r_ground, s_ground, phi_rel, w_int_horiz, b_vert_own, b_vert_int, idx_scale )
        SetBeliefBasedOnlineCostState(p.271)( mode_int, height_own, z_own_ave, dz_own_ave,
            r_ground, s_ground, phi_rel, w_int_horiz, b_vert_int, b_tau_int, s_c, idx_online_cost )
    else
        if is_ground_point_obs
            ( b_tau_int, int_h_shifted ) = GroundPointObstacleTauEstimation(p.267)( z_own_ave, dz_own_ave,
                                                                                    z_int_ave )
        else
            ( b_tau_int, int_h_shifted ) = PointObstacleTauEstimation(p.269)( r_ground[1], s_ground[1],
                                                   phi_rel[1], z_own_ave, dz_own_ave, z_int_ave )
        end
        b_vert_int[1].z = int_h_shifted - H_point_obstacle_offset_ft
    end
    samples_vert::Vector{CombinedVerticalBelief(p.515)} =
          CombineVerticalBeliefs(p.270)( b_vert_own, b_vert_int, b_int_prev )
    return (samples_vert::Vector{CombinedVerticalBelief(p.515)}, b_tau_int::Vector{TauBelief(p.522)})
end
```

Referenced In: StateAndCostEstimation(p.251), SetVerticalTRMSurvOnlyMode(p.247)

CONVERTHORIZONTAL (Algorithm 243) converts the relative horizontal position and relative horizontal velocity samples of the intruder into a three-dimensional coordinate system, which is described in Section 5.3 of [2]. An important step not introduced in [2] is taking the absolute value of angle phi, which exploits symmetry of the distribution of tau to reduce file sizes by about half.

This algorithm takes as input *b_horiz_int*. This algorithm returns *r_ground*, *s_ground*, *phi_rel*, and *w_int_horiz*.

Algorithm 243 ConvertHorizontal

```
function ConvertHorizontal( b_horiz_int::Vector{IntruderHorizontalBelief(p.500)} )
    N_beliefs::Z     = length( b_horiz_int )
    r_ground::Vector{R}    = zeros( R, N_beliefs )
    s_ground::Vector{R}    = zeros( R, N_beliefs )
    phi_rel::Vector{R}     = zeros( R, N_beliefs )
    w_int_horiz::Vector{R} = zeros( R, N_beliefs )
    for i = 1:N_beliefs
        b::IntruderHorizontalBelief(p.500) = b_horiz_int[i]
        r_ground[i]    = hypot( b.x_rel,  b.y_rel )
        s_ground[i]    = hypot( b.dx_rel, b.dy_rel )
        phi_rel[i]     = atan( b.dy_rel, b.dx_rel ) - atan( b.y_rel, b.x_rel )
        w_int_horiz[i] = b.weight
    end
    return (r_ground::Vector{R}, s_ground::Vector{R}, phi_rel::Vector{R}, w_int_horiz::Vector{R})
end
```

Referenced In: StateEstimation(p.257)

HORIZONTALWORSTCASE (Algorithm 244) computes an additional horizontal belief with a zero horizontal miss distance trajectory for an individual intruder. This additional belief ensures the cost computations will account for the worst case condition in the horizontal dimension.

All the relative bearing estimates are checked to identify the maximum relative bearing (*phi_max*), the minimum relative bearing (*phi_min*), and the the mean relative bearing (*phi_mean*). All relative bearing estimates are wrapped between zero and 2π to properly deal with angle measures beyond a full circle. The max and min relative bearing estimates are used to determine if the zero horizontal miss distance trajectory falls within the span of the relative bearing estimates. Additionally, the mean relative bearing must indicate convergence for the worst case belief to be added. The relevance of the head-on trajectory is indicated by the returned weight (*w_int_horiz_worst*). It is calculated by interpolating the weights of the mean estimate and either the min or max estimate, whichever is closer.

If the worst case belief is determined to be necessary, a complete horizontal belief estimate is constructed. The mean range and speed are computed to be used in the worst case belief state and worst case relative bearing is set as π. Two additional algorithms, DETERMINEHORIZONTALWORSTCASEPHISPREADFACTOR and DETERMINEHORIZONTALWORSTCASENEARMEANFACTOR, are used to reduce or increase the weight of the worst case belief in special conditions. The range, speed, relative bearing, and weight of the worst case belief are returned to STATEESTIMATION where, if a new belief is added, the weights are renormalized.

This algorithm takes as input *mode_int*, *r_ground*, *s_ground*, *phi_rel*, *w_int_horiz*, and *height_own*. This algorithm returns *r_ground_worst*, *s_ground_worst*, *phi_rel_worst*, and *w_int_horiz_worst*.

Algorithm 244 HorizontalWorstCase

```
function HorizontalWorstCase( mode_int::Z, r_ground::Vector{R}, s_ground::Vector{R},
                    phi_rel::Vector{R}, w_int_horiz::Vector{R}, height_own::R )
    r_ground_worst::R    = 0.0
    s_ground_worst::R    = 0.0
    phi_rel_worst::R     = 0.0
    w_int_horiz_worst::R = 0.0
    phi_max::R           = -Inf
    phi_min::R           = Inf
    phi_mean::R          = Inf
    w_phi_max::R         = 0.0
    w_phi_min::R         = 0.0
    w_phi_mean::R        = 0.0
    for i = 1:length( w_int_horiz )
        phi::R = WrapTo2Pi(p. 583)( phi_rel[i] )
        if (phi_max < phi)
            phi_max = phi
            w_phi_max = w_int_horiz[i]
        end
        if (phi < phi_min)
            phi_min = phi
            w_phi_min = w_int_horiz[i]
        end
        if (w_phi_mean < w_int_horiz[i])
            phi_mean = phi
            w_phi_mean = w_int_horiz[i]
        end
    end
    if (pi/2 < phi_mean < 3pi/2)
        w_worst_case::R = 0.0
        if (phi_mean <= pi) && (pi < phi_max)
            w_worst_case = w_phi_mean + ((w_phi_max - w_phi_mean) * ((pi - phi_mean) / (phi_max - phi_mean))
                )
        elseif (phi_min < pi) && (pi <= phi_mean)
            w_worst_case = w_phi_min + ((w_phi_mean - w_phi_min) * ((pi - phi_min) / (phi_mean - phi_min)))
        end
        if (0.0 < w_worst_case)
            for i = 1:length( w_int_horiz )
                r_ground_worst = r_ground_worst + (r_ground[i] * w_int_horiz[i])
                s_ground_worst = s_ground_worst + (s_ground[i] * w_int_horiz[i])
            end
            phi_rel_worst = pi
            phi_mean_wc_delta::R = abs( phi_mean - pi )
            near_mean_scale_factor::R =
                DetermineHorizontalWorstCaseNearMeanFactor(p. 260)( mode_int, phi_mean_wc_delta, r_ground_worst )
            phi_spread::R = phi_max - phi_min
            phi_spread_scale_factor::R =
                DetermineHorizontalWorstCasePhiSpreadFactor(p. 261)( mode_int, phi_spread, height_own )
            w_int_horiz_worst = w_worst_case * near_mean_scale_factor * phi_spread_scale_factor
        end
    end
    return (r_ground_worst::R, s_ground_worst::R, phi_rel_worst::R, w_int_horiz_worst::R)
end
```

Referenced In: StateEstimation(p. 257)

DETERMINEHORIZONTALWORSTCASENEARMEANFACTOR (Algorithm 245) is used to increase the weight of the worst case belief state according the proximity of the mean relative bearing estimate to the worst case. Although in most instances the uncertainty associated with belief states improves the performance, it can also lead to delayed action in some unique geometries and surveillances. The most prevalent of these is fast-closing, zero horizontal miss distance geometries with range dependent bearing uncertainty in the surveillance.

Two sub-factors control the overall amount of increase (*near_mean_factor*) to apply to the worst case belief state. The angle factor (*angle_factor*) is computed using the angle difference between the worst case relative bearing and the mean relative bearing estimate. As the difference between them increases, the value of the factor decreases. The range factor (*range_factor*) is computed using the worst case range. As the range decreases, the value of the factor decreases. The overall factor uses the minimum of the two sub-factors so that either factor can prevent the worst case weight from being increased.

This algorithm takes as input *mode_int*, *phi_mean_wc_delta*, and *r_ground_worst*. This algorithm returns *near_mean_factor*.

Algorithm 245 DetermineHorizontalWorstCaseNearMeanFactor

```
function DetermineHorizontalWorstCaseNearMeanFactor( mode_int::Z, phi_mean_wc_delta::R, r_ground_worst::R )
    X_near_mean_factor_max::R         =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.X_near_mean_factor_max
    X_near_mean_angle_threshold_lo::R =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.X_near_mean_angle_threshold_lo
    X_near_mean_angle_threshold_hi::R =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.X_near_mean_angle_threshold_hi
    D_near_mean_range_threshold_lo::R =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.D_near_mean_range_threshold_lo
    D_near_mean_range_threshold_hi::R =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.D_near_mean_range_threshold_hi
    angle_factor::R = 0.0
    range_factor::R = 0.0
    rolloff::R      = 0.0
    rolloff   = X_near_mean_angle_threshold_hi - X_near_mean_angle_threshold_lo
    angle_factor = CalculateThresholdRampDownFactor(p.410)( phi_mean_wc_delta, X_near_mean_angle_threshold_lo,
        rolloff )
    rolloff   = D_near_mean_range_threshold_hi - D_near_mean_range_threshold_lo
    range_factor = CalculateThresholdRampUpFactor(p.411)( r_ground_worst, D_near_mean_range_threshold_lo,
        rolloff )
    near_mean_factor::R = ( X_near_mean_factor_max - 1.0 ) * min( angle_factor, range_factor ) + 1.0
    return (near_mean_factor::R)
end
```

Referenced In: HorizontalWorstCase(p.259)

DETERMINEHORIZONTALWORSTCASEPHISPREADFACTOR (Algorithm 246) is used to reduce or increase the weight of the worst case belief state according the spread of the relative bearing estimates. In slow closure cases, such as parallel geometries, the spread of the relative bearing estimates can increase dramatically, leading to an improper application of the worst case belief.

A number of conditions are checked to determine the value of the scaling factor to be applied (*spread_scale_factor*). Nominally the weight is changed by the high factor (*X_phi_spread_factor_hi*). However, if the own aircraft height is below a threshold (*H_phi_spread_threshold*) and the spread of the relative bearing estimates is larger than the minimum threshold (*X_phi_spread_threshold_lo*), the scaling factor is changed. If the spread is greater than the high threshold (*X_phi_spread_threshold_hi*), the scale factor is reduced to the lowest possible threshold (*X_phi_spread_factor_lo*). Otherwise, the scale factor is calculated by interpolating the high and low factors based on the spread of the relative bearing estimates.

This algorithm takes as input *mode_int*, *phi_spread*, and *height_own*. This algorithm returns *spread_scale_factor*.

Algorithm 246 DetermineHorizontalWorstCasePhiSpreadFactor

```
function DetermineHorizontalWorstCasePhiSpreadFactor( mode_int::Z, phi_spread::R, height_own::R )
    X_phi_spread_factor_lo::R    =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.X_phi_spread_factor_lo
    X_phi_spread_factor_hi::R    =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.X_phi_spread_factor_hi
    H_phi_spread_threshold::R    =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.H_phi_spread_threshold
    X_phi_spread_threshold_lo::R =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.X_phi_spread_threshold_lo
    X_phi_spread_threshold_hi::R =
        params().modes[mode_int].state_estimation.tau.horiz_worst_case.X_phi_spread_threshold_hi
    spread_scale_factor::R = X_phi_spread_factor_hi
    if (height_own <= H_phi_spread_threshold) && (X_phi_spread_threshold_lo <= phi_spread)
        if (X_phi_spread_threshold_hi <= phi_spread)
            spread_scale_factor = X_phi_spread_factor_lo
        else
            rolloff::R   = X_phi_spread_threshold_hi - X_phi_spread_threshold_lo
            ramp_factor = CalculateThresholdRampDownFactor(p. 410)( phi_spread, X_phi_spread_threshold_lo,
                rolloff )
            spread_scale_factor = X_phi_spread_factor_lo + ((X_phi_spread_factor_hi - X_phi_spread_factor_lo
                ) * ramp_factor)
        end
    end
    return (spread_scale_factor::R)
end
```

Referenced In: HorizontalWorstCase(p. 259)

UPDATEINITIALIZATIONCSTATE (Algorithm 247) is used to update the state variables of INITIALIZATIONCSTATE (Type 90) *t_count*.

This algorithm's primary goal is to ensure the Initialization counter is updated for degraded intruders and when own ship is in Surveillance only mode.

This algorithm takes as input *mode_int*, and *s_c*. The algorithm updates *s_c.t_count*.

Algorithm 247 UpdateInitializationCState

```
function UpdateInitializationCState( mode_int::Z, s_c::InitializationCState(p. 529) )
    T_init::Z = params().modes[mode_int].cost_estimation.online.initialization.T_init
    s_c.t_count = min( T_init, s_c.t_count + 1 )
end
```

Referenced In: InitializationCost(p. 299), StateAndCostEstimation(p. 251), SetVerticalTRMSurvOnlyMode(p. 247)

3.3.2.2.1 Tau Estimation

TAUESTIMATION (Algorithm 248) takes three-dimensional samples for the observed state of the intruder. It produces a probability distribution over the values of tau (in seconds) given by *T_tau_values*.

Tau is the time until the intruder comes within a lateral conflict distance of own aircraft. Since tau is a quantity that is defined in the future and there is stochasticity in the horizontal dynamics, it is not possible to know this value with absolute certainty. It is important to distinguish the

tau in this logic from the tau that is used by the legacy TCAS logic. In the legacy logic, tau is interpreted to mean the time of closest approach and the logic uses simple models to provide a point estimate of tau.

The estimation process involves averaging the probability distributions associated with the interpolants of all the three-dimensional samples. Determining the probability distribution of an interpolant involves looking up distributions organized in the horizontal entry distribution file (*T_horizontal_table*). The same horizontal entry distribution file is used for ADS-R and for ADS-B surveillance. For sXu, the same horizontal entry distribution file is used for all surveillance sources. The state samples are derived from intruder aircraft horizontal belief states.

TAUESTIMATION takes as input *mode_int*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *r_ground*, *s_ground*, *phi_rel*, *w_horiz*, *b_vert_own*, *b_vert_int*, and *idx_scale*. This algorithm returns *b_tau_int*.

GETENTRYDISTRIBUTIONTABLES determines which horizontal and vertical entry distribution files should be used based on the protection mode index (*mode_int*).

To address vertically converging, slow closure encounters, the probability distribution associated with the interpolants is based on vertical state and distribution information, as opposed to horizontal. A slow closure encounter is defined to be one where the sampled relative range and relative ground speed for the intruder are below parameterized thresholds, *D_range_thres* and *R_speed_thres*. Additionally, the vertical entry table must have a large enough vertical separation (*H_alt_rel_thres*) and enough weight above the weight threshold (*W_thres_vert*), to be considered for use. Conditions must then be satisfied based on the horizontal convergence or divergence of the intruder, based on *phi_rel*. In the case that the intruder is horizontally converging, the horizontal distribution must have sufficient weight below a tau threshold (*T_thresh_horiz*), and more weight than the vertical distribution below the same tau threshold. In the case that the intruder is horizontally diverging, the intruder must be within *D_range_diverge_thres* horizontally. In these instances, a vertical entry distribution file (*T_vertical_table*), is used. The state samples, in this case, are derived from own aircraft and intruder aircraft vertical belief states.

DETERMINETAUVERTICALDISTRIBUTION is used to calculate the probability distribution associated with the interpolants based on the vertical entry distribution file. It is called only in the case of a vertically converging slow closure encounter. It is called at most once per intruder per cycle; it is independent of the horizontal beliefs.

Algorithm 248 TauEstimation

```
function TauEstimation( mode_int::Z, z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R,
                  r_ground::Vector{R}, s_ground::Vector{R}, phi_rel::Vector{R}, w_horiz::Vector{R},
                  b_vert_own::Vector{OwnVerticalBelief(p.501)}, b_vert_int::Vector{IntruderVerticalBelief(p.500)},
                  idx_scale::Z )
    T_thres_horiz::R        = params().modes[mode_int].state_estimation.tau.T_thres_horiz
    W_thres_horiz::R        = params().modes[mode_int].state_estimation.tau.W_thres_horiz
    W_thres_vert::R         = params().modes[mode_int].state_estimation.tau.W_thres_vert
    D_range_diverge_thres::R= params().modes[mode_int].state_estimation.tau.D_range_diverge_thres
    X_phi_converge_thres::R = params().modes[mode_int].state_estimation.tau.X_phi_converge_thres
    (T_horizontal_table::RDataTableScaled(p.522), T_vertical_table::RDataTableScaled(p.522),
        T_tau_values::Vector{R}) = GetEntryDistributionTables(p.264)( mode_int )
    t::Z              = length( T_tau_values )
    num_tau_blocks::Z = t - 1
    w_out::Vector{R}  = zeros( R, t )
    w_vert::Vector{R} = zeros( R, t )
    w_below_thres_vert::R     = -Inf
    is_vert_initialized::Bool = false
    is_vertically_converging::Bool =
        IsVerticallyConverging(p.428)( mode_int, z_own_ave, dz_own_ave, z_int_ave, dz_int_ave )
    for i = 1:length( w_horiz )
        w_temp::Vector{R} = zeros( R, t )
        w_below_thres_horiz::R = 0.0
        datum  = R[ r_ground[i], s_ground[i], phi_rel[i] ]
        w_temp = LookupEntryDistribution(p.266)( mode_int, datum, T_horizontal_table, num_tau_blocks, idx_scale
            )
        for j = 1:t
            if (T_tau_values[j] <= T_thres_horiz)
                w_below_thres_horiz = w_below_thres_horiz + w_temp[j]
            end
        end
        if is_vertically_converging &&
           IsSlowClosure(p.427)( mode_int, r_ground[i], s_ground[i], (z_int_ave - z_own_ave) )
            if !is_vert_initialized
                (w_vert, w_below_thres_vert) =
                    DetermineTauVerticalDistribution(p.265)( mode_int, b_vert_own, b_vert_int,
                                                    T_vertical_table, T_tau_values, idx_scale )
                is_vert_initialized = true
            end
            if (0.0 <= w_below_thres_vert < W_thres_vert)
                if (r_ground[i] < D_range_diverge_thres) ||
                   ((phi_rel[i] < X_phi_converge_thres) &&
                    (W_thres_horiz < w_below_thres_horiz) &&
                    (w_below_thres_vert < w_below_thres_horiz))
                    w_temp = w_vert
                end
            end
        end
        w_out = w_out + (w_horiz[i] * w_temp)
    end
    w_out[t] = nanmax(p.584)( 0.0, 1.0 - sum( w_out[1:num_tau_blocks] ) )
    b_tau_int::Vector{TauBelief(p.522)} = Vector{TauBelief(p.522)}( )
    for i = 1:t
        if (0.0 < w_out[i]) || (i == t)
            push!( b_tau_int, TauBelief(p.522)( T_tau_values[i], w_out[i] ) )
        end
    end
    return b_tau_int::Vector{TauBelief(p.522)}
end
```

Referenced In: StateEstimation(p.257)

GETENTRYDISTRIBUTIONTABLES (Algorithm 249) determines which horizontal and vertical entry

distribution files should be used based on the protection mode index, *mode_int*, and surveillance source, *source_int*.

GETENTRYDISTRIBUTIONTABLES takes as input *mode_int*. This algorithm returns *t_horizontal_table*, *t_vertical_table*, and *t_tau_values*.

Algorithm 249 GetEntryDistributionTables

```
function GetEntryDistributionTables( mode_int::Z )
    T_tau_values::Vector{R} =
        params().modes[mode_int].state_estimation.tau.entry_dist.T_tau_values
    T_vertical_table::RDataTableScaled(p. 522) =
        params().modes[mode_int].state_estimation.tau.entry_dist.vertical_table_content
    T_horizontal_table::RDataTableScaled(p. 522) =
        params().modes[mode_int].state_estimation.tau.entry_dist.horizontal_table_content
    local t_horizontal_table::RDataTableScaled(p. 522)
    t_horizontal_table = T_horizontal_table
    return (t_horizontal_table::RDataTableScaled(p. 522), T_vertical_table::RDataTableScaled(p. 522), T_tau_values::
        Vector{R})
end
```

Referenced In: TauEstimation(p. 263)

DETERMINETAUVERTICALDISTRIBUTION (Algorithm 250) returns the probability distribution associated with the interpolants based on the vertical entry distribution file. The vertical entry distribution file is used only under certain conditions as described in TAUESTIMATION. The probability distribution based on the vertical entry distribution file is calculated at most once per intruder each cycle, if needed.

DETERMINETAUVERTICALDISTRIBUTION takes as input *mode_int*, *b_vert_own*, *b_vert_int*, *T_vertical_table*, *T_tau_values*, and *idx_scale*. This algorithm returns *w_vert*, and *w_below_thres_vert*.

Algorithm 250 DetermineTauVerticalDistribution

```
function DetermineTauVerticalDistribution( mode_int::Z, b_vert_own::Vector{OwnVerticalBelief(p. 501)},
                                b_vert_int::Vector{IntruderVerticalBelief(p. 500)},
                                T_vertical_table::RDataTableScaled(p. 522), T_tau_values::Vector{R},
                                idx_scale::Z )
    T_thres_horiz::R = params().modes[mode_int].state_estimation.tau.T_thres_horiz
    n::Z = length( b_vert_own )
    m::Z = length( b_vert_int )
    t::Z = length( T_tau_values )
    num_tau_blocks::Z = t - 1
    w_vert::Vector{R} = zeros( R, t )
    w_below_thres_vert::R = 0.0
    for j in 1:n
        for k in 1:m
            datum::Vector{R}  = R[ (b_vert_int[k].z - b_vert_own[j].z), b_vert_own[j].dz, b_vert_int[k].dz ]
            w_samp::Vector{R} =
                LookupEntryDistribution(p. 266)( mode_int, datum, T_vertical_table, num_tau_blocks, idx_scale )
            w_vert = w_vert + (b_vert_own[j].weight * b_vert_int[k].weight * w_samp)
        end
    end
    for j in 1:t
        if (T_tau_values[j] <= T_thres_horiz)
            w_below_thres_vert = w_below_thres_vert + w_vert[j]
        end
    end
    return (w_vert::Vector{R}, w_below_thres_vert::R)
end
```

Referenced In: TauEstimation(p. 263)

LOOKUPENTRYDISTRIBUTION (Algorithm 251) determines the probability distribution of an interpolant by looking up distributions organized in the horizontal entry distribution file. The state samples are derived from intruder horizontal belief states. In the slow closure case, LOOKUPENTRYDISTRIBUTION uses a vertical entry distribution file. The state samples are derived from own aircraft and intruder aircraft vertical belief states.

The algorithm uses INTERPOLATEBLOCKS to generate the distributions that are to be returned. The horizontal and vertical entry distribution files are both fixed block dictionaries. The value of *w_samp* returned by LOOKUPENTRYDISTRIBUTION is the average of all the probability distributions associated with the interpolants.

This algorithm takes as input *mode_int*, *datum*, *table*, and *num_blocks*. It returns *w_samp*.

Algorithm 251 LookupEntryDistribution

```
function LookupEntryDistribution( mode_int::Z, datum::Vector{R}, table::RDataTableScaled(p. 522), num_blocks::Z,
     idx_scale::Z )
    T_threshold_factor::R = params().modes[mode_int].state_estimation.tau.threshold_factor
    w_samp::Vector{R}   = zeros( R, num_blocks + 1 )
    w_interp::Vector{R} = InterpolateBlocks(p. 266)( datum, table, num_blocks, idx_scale )
    threshold::R = T_threshold_factor * nanmax(p. 584)( nanmaximum(p. 585)( w_interp ), 1.0 - sum( w_interp ) )
    for j = 1:num_blocks
        if (threshold <= w_interp[j])
            w_samp[j] = w_interp[j]
        end
    end
    w_samp[end] = 1.0 - sum( w_samp )
    return w_samp::Vector{R}
end
```

Referenced In: DetermineTauVerticalDistribution(p. 265), TauEstimation(p. 263)

INTERPOLATEBLOCKS (Algorithm 252) interpolates the input values to produce the weighted entries that can be looked up in the table format. This is done through a call to INTERPOLANTS which returns the table indices to use and the weight associated with each index. These indices are used to create the blocks containing the weighted values that represent the distributions.

This algorithm takes as input *x*, *table*, *block_size*, and *idx_scale*. This algorithm returns *block*.

Algorithm 252 InterpolateBlocks

```
function InterpolateBlocks( x::Vector{R}, table::RDataTableScaled(p. 522), block_size::Z, idx_scale::Z )
    D::Z = length( x )
    block::Vector{R} = zeros( R, block_size )
    (indices::Vector{Z}, weights::Vector{R}) =
        Interpolants(p. 416)( x, table.cut_counts, 1, D, table.scaled_policy_cuts_set[idx_scale] )
    for j = 1:length( indices )
        offset::Z = table.index[ indices[j] ]
        for k = 1:block_size
            block[k] = block[k] + (weights[j] * table.data[offset+k])
        end
    end
    return block::Vector{R}
end
```

Referenced In: LookupEntryDistribution(p. 266)

3.3.2.2.2 Ground Point Obstacle Tau Estimation

GROUNDPOINTOBSTACLETAUESTIMATION (Algorithm 253) takes ownship's altitude (*z_own*), ownship's vertical rate (*dz_own*), and the ground's altitude (*z_int*). It produces a point estimate of tau (in seconds) given by *vert_tau_estimate* as well as the projected altitude shift of the point obstacle *int_h_shifted*.

Tau is the time until the ownship comes within a vertical conflict distance of the ground. Since the point obstacle does not have dynamic stochasticity, a probability distribution is not used and instead a point estimate of tau is calculated. Since the ground point obstacles is represented as a point directly below ownship with an altitude offset derived from ownship AGL altitude, a

vertical distance is used to calculate tau. A vertical distance is used because a horizontal distance would create the illusion of always having min tau, since the horizontal distance would always be below a target separation. Tau is bound by *T_min_tau* and *T_max_tau* to properly interact with and fit into the policy table. The max tau value corresponds to the maximum tau value used in the training of the offline vertical cost table and correspond to a low risk intruder. This value is appropriate to use when ownship is level or vertically diverging because the dynamic uncertainty of the encounter is purely based on ownship (i.e. point obstacle will not move). The minimum tau value was chosen as it forces ownship to continue to maneuver against the ground point obstacle even though the loss of separation has been achieved and the point obstacle does not have dynamic uncertainty.

The algorithm first checks if ownship has less than the targeted seperation, *D_vert_target_separation*, above the ground and assigns tau value of *T_min_tau* if this seperation is not maintained. If ownship is above the desired altitude and has a vertical rate of approximately zero, *vert_tau_estimate* will be set to the maximum possible value of *T_max_tau*. Otherwise, the *vert_tau_estimate* variable is calculated with respect to a target vertical separation from the point obstacle, *D_vert_target_separation*. The *vert_tau_estimate* is reassigned to *T_max_tau* if ownship moving up, away from the ground, i.e. *vert_tau_estimate* is negative. If *vert_tau_estimate* is outside the bounds of *T_min_tau* and *T_max_tau*, it will be set to the appropriate bound. The resulting tau estimate is stored as a TAUBELIEF, *b_tau_int*, with a weight of one to be returned and used in offline cost estimations.

The ground's shifted altitude, *int_h_shifted*, is set to be the minumum of the projected ownship altitude at *vert_tau_estimate* and the altitude of the ground to assure that only up advisories can be given on the ground point obstacle.

GROUNDPOINTOBSTACLETAUESTIMATION returns *b_tau_int* and *int_h_shifted*.

Algorithm 253 GroundPointObstacleTauEstimation

```
function GroundPointObstacleTauEstimation( z_own::R, dz_own::R, z_int::R )
    D_vert_target_separation::R = params().threat_resolution.vertical_point_obstacle_awareness.D_vert_-
        target_separation
    T_min_tau::R                = params().threat_resolution.vertical_point_obstacle_awareness.T_min_tau
    T_max_tau::R                = params().threat_resolution.vertical_point_obstacle_awareness.T_max_tau
    approx_zero::R              = params().surveillance.approx_zero
    if z_own < (z_int + D_vert_target_separation)
        vert_tau_estimate = T_min_tau
    elseif abs(dz_own) < approx_zero
        vert_tau_estimate = T_max_tau
    else
        vert_tau_estimate = ((z_int - z_own) + D_vert_target_separation) / dz_own
        if vert_tau_estimate < 0
            vert_tau_estimate = T_max_tau
        elseif vert_tau_estimate < T_min_tau
            vert_tau_estimate = T_min_tau
        elseif vert_tau_estimate > T_max_tau
            vert_tau_estimate = T_max_tau
        end
    end
    b_tau_int::Vector{TauBelief(p.522)} = Vector{TauBelief(p.522)}( )
    push!( b_tau_int, TauBelief(p.522)( vert_tau_estimate, 1.0 ) )
    int_h_shifted::R = min(z_int, vert_tau_estimate * dz_own + z_own)
    return ( b_tau_int::Vector{TauBelief(p.522)}, int_h_shifted::R )
end
```

Referenced In: StateEstimation(p.257)

3.3.2.2.3 Point Obstacle Tau Estimation

POINTOBSTACLETAUESTIMATION (Algorithm 254) takes three-dimensional samples for the observed state of the obstacle or terrain object. It produces a point estimate of tau (in seconds) given by *mod_tau_estimate* as well as the projected altitude shift of the point obstacle *int_h_-shifted*.

Tau is the time until the intruder comes within a lateral conflict distance of own aircraft. The use of a lateral conflict distance allows the logic to avoid future collisions without having unnecessary alerts. Since the point obstacle does not have dynamic stochasticity, a probability distribution is not used and instead a point estimate of tau is calculated. Tau is bound by *T_min_tau* and *T_max_tau* to properly interact with and fit into the policy table. The max tau value corresponds to the maximum tau value used in the training of the offline vertical cost table and correspond to a low risk intruder. This value is appropriate to use when ownship is level or vertically diverging because the dynamic uncertainty of the encounter is purely based on ownship (i.e. point obstacle will not move). The minimum tau value was chosen as it forces ownship to continue to maneuver against the ground point obstacle even though the loss of separation has been achieved and the point obstacle does not have dynamic uncertainty.

The projected ground range, *r_project*, is calculated using the intruder's relative horizontal distance, speed, and angle from ownship. The *mod_tau_estimate* variable is calculated with respect to a target horizontal separation from the point obstacle, *D_horiz_target_separation*. After calculating *mod_tau_estimate*, any point obstacle within *D_horiz_target_separation* is assigned a tau value of *T_min_tau*. The algorithm then checks if the point obstacle is diverging from the point obstacle by comparing current ground range, *r_ground*, with the projected ground range *r_project*. The estimated tau value, *mod_tau_estimate*, is set to *T_max_tau* if ownship is diverging due to the obstacle no longer being a threat. *mod_tau_estimate* is bound by *T_min_tau* and *T_max_tau*. The resulting tau estimate is stored as a TAUBELIEF, *b_tau_int*, with a weight of one to be returned and used in offline cost estimations.

The point obstacle is assumed to be the highest point of a structure for which protection is desired. Therefore, the intruder shifted altitude, *int_h_shifted*, is based on the projected ownship altitude at *mod_tau_estimate* or the maximum altitude of the point obstacle, which ever is lower.

POINTOBSTACLETAUESTIMATION takes as input the point obstacle's relative distance (*r_ground*), relative velocity (*s_ground*), relative angle (*phi_rel*), ownship's alitude (*z_own*), ownship's vertical velocity (*dz_own*), the point obstacle's altitude (*z_int*). It returns the tau belief, *b_tau_int*, and the point obstacle's shifted altitude, *int_h_shifted*.

Algorithm 254 PointObstacleTauEstimation

```
function PointObstacleTauEstimation( r_ground::R, s_ground::R, phi_rel::R, z_own::R, dz_own::R, z_int::R )
    D_horiz_target_separation = params().threat_resolution.vertical_point_obstacle_awareness.D_horiz_-
        target_separation
    T_min_tau           = params().threat_resolution.vertical_point_obstacle_awareness.T_min_tau
    T_max_tau           = params().threat_resolution.vertical_point_obstacle_awareness.T_max_tau
    r_project::R        = r_ground + s_ground * cos(phi_rel)
    mod_tau_estimate::R = (r_ground - D_horiz_target_separation) / abs( s_ground * cos(phi_rel) )
    if (r_ground < D_horiz_target_separation)
        mod_tau_estimate = T_min_tau
    elseif ( r_project > r_ground )
        mod_tau_estimate = T_max_tau
    elseif (mod_tau_estimate < T_min_tau)
        mod_tau_estimate = T_min_tau
    elseif ( mod_tau_estimate > T_max_tau )
        mod_tau_estimate = T_max_tau
    end
    b_tau_int::Vector{TauBelief(p. 522)} = Vector{TauBelief(p. 522)}( )
    push!( b_tau_int, TauBelief(p. 522)( mod_tau_estimate, 1.0 ) )
    int_h_shifted::R = min(z_int, mod_tau_estimate * dz_own + z_own)
    return ( b_tau_int::Vector{TauBelief(p. 522)}, int_h_shifted::R )
end
```

Referenced In: StateEstimation(p. 257)

3.3.2.2.4 Combine Vertical Beliefs

COMBINEVERTICALBELIEFS (Algorithm 255) combines the intruder and own aircraft vertical beliefs to produce a set of vertical samples. The vertical samples are used as inputs to OFFLINE-COSTESTIMATION (Algorithm 260).

This algorithm takes as input *b_vert_own*, *b_vert_int*, and *b_int*. It returns *samples_vert*.

Algorithm 255 CombineVerticalBeliefs

```
function CombineVerticalBeliefs( b_vert_own::Vector{OwnVerticalBelief(p. 501)},
                                 b_vert_int::Vector{IntruderVerticalBelief(p. 500)},
                                 b_int::AdvisoryBeliefState(p. 515) )
    w_vert::R = 0.0
    if !b_int.need_init
        w_vert_prev_sum::R = sum( b_int.w_vert_prev )
    end
    samples_vert::Vector{CombinedVerticalBelief(p. 515)} = CombinedVerticalBelief(p. 515)[]
    b_int.w_vert_prev   = R[]
    for i in 1:length( b_vert_own )
        for j in 1:length( b_vert_int )
            if !b_int.need_init
                w_vert = w_vert_prev_sum * b_vert_own[i].weight * b_vert_int[j].weight
            else
                w_vert = b_vert_own[i].weight * b_vert_int[j].weight
            end
            push!( samples_vert,
                   CombinedVerticalBelief(p. 515)( b_vert_int[j].z - b_vert_own[i].z,
                                                   b_vert_own[i].dz, b_vert_int[j].dz, w_vert )
                 )
            push!( b_int.w_vert_prev, w_vert )
        end
    end
    b_int.w_vert_prev = Normalize(p. 582)( b_int.w_vert_prev )
    b_int.need_init   = (length( b_int.w_vert_prev ) == 0)
    for i in 1:length( samples_vert )
        samples_vert[i].weight = b_int.w_vert_prev[i]
    end
    return samples_vert::Vector{CombinedVerticalBelief(p. 515)}
end
```

Referenced In: StateEstimation(p. 257)

3.3.2.2.5 Set Belief-Based Online Cost

SETBELIEFBASEDONLINECOSTSTATE (Algorithm 256) is used to set state variables for online costs that are dependent on details of the intruder aircraft and own aircraft dynamic state.

Several online cost algorithms have state variables dependent on vertical and/or horizontal beliefs:

- ALTITUDEDEPENDENTCOCCOST (Algorithm 271)
- CRITICALINTERVALPROTECTIONCOST (Algorithm 276)
- PREVENTEARLYCOCCOST (Algorithm 281)
- SA01HEURISTIC (Algorithm 307)

The state variable values are set here to confine the dynamic state information they use for inputs to STATEESTIMATION and to closely associate the calculated values with the appropriate online cost algorithms. Setting the state variables within STATEESTIMATION also provides more options for adjusting the method of calculating the values.

The calculations of values for the CRITICALINTERVALPROTECTIONCSTATE (Type 88), PREVENTEARLYCOCCSTATE (Type 93), and SA01HEURISTICCSTATE (Type 96) variables are performed within this algorithm. The variable *s_c.low_alt_parallel_ra_deferral* contains state settings used for selecting resolution advisories. The calculations for ALTITUDEDEPENDENTCOCCSTATE (Type

82) are performed in DETERMINEALTITUDEDEPENDENTCOCFACTOR (Algorithm 257). DETERMINEALTITUDEDEPENDENTCOCFACTOR is called once to calculate the factor used to set *altitude_dependent_coc.scaled_cost_coc_ra*.

This algorithm takes as input *mode_int*, *height_own*, *z_own_ave*, *dz_own_ave*, *r_ground*, *s_ground*, *phi_rel*, *w_int_horiz*, *b_vert_int*, *b_tau_int*, and *s_c*. This algorithm updates *s_c::OnlineCostState* which contains state variables for online costs.

Algorithm 256 SetBeliefBasedOnlineCostState

```
function SetBeliefBasedOnlineCostState( mode_int::Z, height_own::R, z_own_ave::R, dz_own_ave::R,
                r_ground::Vector{R}, s_ground::Vector{R}, phi_rel::Vector{R}, w_int_horiz::Vector{R},
                b_vert_int::Vector{IntruderVerticalBelief(p.500)}, b_tau_int::Vector{TauBelief(p.522)},
                s_c::OnlineCostState(p.520), idx_online_cost::Z )
    r_ground_int_ave::R = 0.0
    s_ground_int_ave::R = 0.0
    phi_rel_ave::R      = 0.0
    for i in 1:length( phi_rel )
        r_ground_int_ave = r_ground_int_ave + (w_int_horiz[i] * r_ground[i])
        s_ground_int_ave = s_ground_int_ave + (w_int_horiz[i] * s_ground[i])
        phi_rel_ave      = phi_rel_ave + (w_int_horiz[i] * phi_rel[i])
    end
    s_c.critical_interval_protection.angle = phi_rel_ave
    s_c.critical_interval_protection.range = r_ground_int_ave
    s_c.critical_interval_protection.speed = s_ground_int_ave
    s_c.sa01_heuristic.range    = r_ground_int_ave
    s_c.prevent_early_coc.range = r_ground_int_ave
    h_rel_threshold_proximate::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_rel_threshold_-
            proximate[idx_online_cost]
    t_threshold::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.T_threshold[idx_online_cost]
    c_coc::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.C_coc[idx_online_cost]
    factor::R =
        DetermineAltitudeDependentCOCFactor(p.273)( mode_int, height_own, z_own_ave, dz_own_ave,
                                                    b_vert_int, b_tau_int, h_rel_threshold_proximate,
                                                    t_threshold, idx_online_cost )
    s_c.altitude_dependent_coc.scaled_cost_coc_ra = factor * c_coc
end
```

Referenced In: StateEstimation(p.257)

DETERMINEALTITUDEDEPENDENTCOCFACTOR (Algorithm 257) is used to set state variables for ALTITUDEDEPENDENTCOCCOST.

This algorithm determines how much of an additional cost will be applied to clear of conflict at the current state (*factor_coc*). This factor varies with altitude so that as altitude increases the likelihood of having an alert increases proportionally. In this way, the *factor_coc* can be thought of as the amount of additional logic sensitivity to be applied at a given altitude for a given intruder.

At various points in this algorithm the CALCULATETHRESHOLDRAMPUPFACTOR and CALCULATETHRESHOLDRAMPDOWNFACTOR algorithms are used. These methods take an input value, a threshold parameter, and a roll-off parameter and return a value between 0 and 1. The different factors combine to determine the additional sensitivity added to the logic.

The algorithm loops over each vertical sample in turn to determine the necessity of the additional sensitivity. Due to this iterative approach, the intermediate calculations are weighted and

summed during the loop. Two factors (*factor_reduction* and *factor_proximate*) are calculated using sub-functions and components of the vertical sample. The reduction factor reduces the amount of sensitivity increase in circumstances where the own aircraft vertical rate is low. The proximate factor is the main discriminator of when to increase the sensitivity based on the vertical sample and a forward projection in time.

Nested within the vertical sample loop is another loop which iterates over the horizontal tau samples. Since many tau samples have zero weight, a check is performed to determine if further calculations are required. Furthermore, the tau value itself must be within the maximum tau applicability defined by the parameters and less than the largest possible tau value, to prevent improper increases in sensitivity. For each valid tau sample a vertical projection of the value of tau forward in time is performed. This projected separation (*z_rel_actual*) represents the expected vertical separation at the time of loss of horizontal separation. It is used to calculate a factor (*factor_actual*) which reduces the applicability of the additional sensitivity in cases where there will be a large vertical separation when horizontal separation is lost. This condition occurs most frequently in geometries where the aircraft are expected to cross altitude very early in an encounter. To allow these well-separated crossings to occur, the additional sensitivity should not be applied. The intermediate calculation of *factor_coc* combines the weight, proximate factor, actual factor, and a final factor computed using the tau sample value. As the value of tau increases, this factor reduces the additional sensitivity to be applied.

Once the algorithm finishes looping over the vertical samples, the *factor_coc* is scaled by the altitude of the own aircraft. This causes the additional sensitivity to be applied in full above the high altitude threshold (*H_thresold_hi*), not at all below the low threshold (*H_thresold_lo*), and proportionally between. Finally, the reduction factor is applied here as well to appropriately reduce the additional sensitivity in the desired cases. The sensitivity due to this factor can be reduced up to a maximum of *X_vrrf*.

This algorithm takes as input *mode_int*, *height_own*, *z_own_ave*, *dz_own_ave*, *b_vert_int*, *b_tau_int*, *H_rel_threshold_proximate*, and *T_threshold*. This algorithm returns *factor_coc*.

Algorithm 257 DetermineAltitudeDependentCOCFactor

```
function DetermineAltitudeDependentCOCFactor( mode_int::Z, height_own::R, z_own_ave::R, dz_own_ave::R,
                        b_vert_int::Vector{IntruderVerticalBelief(p. 500)}, b_tau_int::Vector{TauBelief(p. 522)},
                        H_rel_threshold_proximate::R, T_threshold::R, idx_online_cost::Z )
    H_rel_rolloff_proximate::R   =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_rel_rolloff_proximate[idx_-
             online_cost]
    X_proximate_to_actual_factor::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.X_proximate_to_actual_-
             factor[idx_online_cost]
    T_rolloff::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.T_rolloff[idx_online_cost]
    X_vrrf::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.X_vrrf[idx_online_cost]
    H_threshold_lo::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_threshold_lo
    H_threshold_hi::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_threshold_hi
    T_tau_values::Vector{R} =
        params().modes[mode_int].state_estimation.tau.entry_dist.T_tau_values
    factor_coc::R       = 0.0
    factor_reduction::R = 0.0
    if (H_threshold_lo <= height_own)
        for bv_int in b_vert_int
            z_rel::R = bv_int.z - z_own_ave
            factor_reduction = factor_reduction + (bv_int.weight *
                DetermineVerticalRateReductionFactor(p. 274)( mode_int, height_own, z_rel, dz_own_ave,
                     idx_online_cost ))
            factor_proximate::R = bv_int.weight *
                DetermineVerticalProximateFactor(p. 275)( mode_int, height_own, z_rel, dz_own_ave,
                                                    bv_int.dz, H_rel_threshold_proximate,
                                                    idx_online_cost )
            for bt_int in b_tau_int
                if (0.0 < bt_int.weight) && ((bt_int.tau <= (T_threshold + T_rolloff))) &&
                   (bt_int.tau < T_tau_values[end])
                    z_rel_actual::R = abs( z_rel + ((bv_int.dz - dz_own_ave) * bt_int.tau) )
                    threshold::R = H_rel_threshold_proximate * X_proximate_to_actual_factor
                    rolloff::R   = H_rel_rolloff_proximate * X_proximate_to_actual_factor
                    factor_actual::R =
                        CalculateThresholdRampDownFactor(p. 410)( z_rel_actual, threshold, rolloff )
                    factor_coc = factor_coc + (bt_int.weight * factor_proximate * factor_actual *
                        CalculateThresholdRampDownFactor(p. 410)( bt_int.tau, T_threshold, T_rolloff ))
                end
            end
        end
        factor_coc = factor_coc * CalculateThresholdRampUpFactor(p. 411)( height_own, H_threshold_lo,
                                                                  (H_threshold_hi - H_threshold_lo) )
        factor_coc = factor_coc * (X_vrrf + ((1.0 - X_vrrf) * (1.0 - factor_reduction)))
    end
    return factor_coc::R
end
```

Referenced In: SetBeliefBasedOnlineCostState(p. 271)

DETERMINEVERTICALRATEREDUCTIONFACTOR (Algorithm 258) uses two factors to determine when to reduce the additional sensitivity of the logic on near-level aircraft with sufficient separation. The first involves comparing the own aircraft vertical rate to a set of thresholds to determine how close it is to level. When the own aircraft is perfectly level, the reduction will be at a maximum and as the vertical rate increases, the reduction will decrease. The second component is based on the current vertical separation. As the current relative altitude approaches co-altitude, the reduction is decreased. This provides a safe-guard on the vertical rate-reduction factor so that

even if an aircraft is level, the additional sensitivity will be applied if separation is lost.

This algorithm takes as input *mode_int*, *height_own*, *z_rel*, and *dz_own_ave*. This algorithm returns *factor_vrrf*.

Algorithm 258 DetermineVerticalRateReductionFactor

```
function DetermineVerticalRateReductionFactor( mode_int::Z, height_own::R, z_rel::R, dz_own_ave::R,
    idx_online_cost::Z )
    R_own_threshold_vrrf::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.R_own_threshold_vrrf
    R_own_rolloff_vrrf::R   =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.R_own_rolloff_vrrf
    H_threshold_hi::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_threshold_hi
    H_rel_threshold_vrrf_lo::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_rel_threshold_vrrf_lo[idx_-
            online_cost]
    H_rel_threshold_vrrf_hi::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_rel_threshold_vrrf_hi[idx_-
            online_cost]
    H_rel_rolloff_vrrf::R   =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_rel_rolloff_vrrf[idx_-
            online_cost]
    factor_dz_own::R =
        CalculateThresholdRampDownFactor(p. 410)( abs( dz_own_ave ), R_own_threshold_vrrf, R_own_rolloff_vrrf )
    z_rel_threshold_vrrf::R = H_rel_threshold_vrrf_lo
    if (H_threshold_hi < height_own)
        z_rel_threshold_vrrf = H_rel_threshold_vrrf_hi
    end
    factor_z_rel::R =
        CalculateThresholdRampUpFactor(p. 411)( abs( z_rel ), z_rel_threshold_vrrf, H_rel_rolloff_vrrf )
    factor_vrrf::R = min( factor_z_rel, factor_dz_own )
    return factor_vrrf::R
end
```

Referenced In: DetermineAltitudeDependentCOCFactor(p. 273)

DETERMINEVERTICALPROXIMATEFACTOR (Algorithm 259) uses the vertical state of the own aircraft and intruder to determine how much additional sensitivity is required. The vertical samples are projected forward in time by *T_proximate* seconds to obtain the expected vertical separation in the near future. If the vertical samples indicate the own aircraft and intruder will cross altitude in the next *T_proximate* seconds, the anticipated vertical separation is set to zero. Furthermore, if the projected vertical separation is greater than the current separation, the current separation is used instead. The vertical separation samples compared to their associated threshold parameters to determine the proximate factor (*factor_proximate*). As the expected vertical separation decreases, more sensitivity will be applied.

At higher altitudes there is a need to provide greater protection against slow vertical closure encounters due to altimetry error biases. To increase sensitivity in these geometries, a special factor (*factor_z_rel_high_alt*) is calculated using the current altitude separation. As the separation decreases, the sensitivity is increased. If at any time this factor would yield a greater level of sensitivity than the proximate factor, it replaces the proximate factor.

This algorithm takes as input *mode_int*, *height_own*, *z_rel*, *dz_own_ave*, *dz_int*, and *H_rel_threshold_proximate*. This algorithm returns *factor_proximate*.

Algorithm 259 DetermineVerticalProximateFactor

```
function DetermineVerticalProximateFactor( mode_int::Z, height_own::R, z_rel::R, dz_own_ave::R,
                                   dz_int::R, H_rel_threshold_proximate::R, idx_online_cost::Z )
    T_proximate::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.T_proximate[idx_online_cost]
    H_threshold_hi::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_threshold_hi
    H_rel_rolloff_proximate::R   =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_rel_rolloff_proximate[idx_-
            online_cost]
    H_rel_highalt_threshold::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_rel_highalt_threshold[idx_-
            online_cost]
    H_rel_highalt_rolloff::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.H_rel_highalt_rolloff[idx_-
            online_cost]
    z_rel_proximate::R = z_rel + ((dz_int - dz_own_ave) * T_proximate)
    if IsProjectedCrossing(p. 426)( z_rel, z_rel_proximate )
        z_rel_proximate = 0.0
    elseif (abs( z_rel ) < abs(  z_rel_proximate ))
        z_rel_proximate = abs( z_rel )
    else
        z_rel_proximate = abs( z_rel_proximate )
    end
    factor_proximate::R = CalculateThresholdRampDownFactor(p. 410)( z_rel_proximate, H_rel_threshold_proximate,
        H_rel_rolloff_proximate )
    factor_z_rel_high_alt::R = 0
    if (H_threshold_hi < height_own)
        factor_z_rel_high_alt =
            CalculateThresholdRampDownFactor(p. 410)( abs(z_rel), H_rel_highalt_threshold, H_rel_highalt_rolloff
                )
    end
    if (factor_proximate < factor_z_rel_high_alt)
        factor_proximate = factor_z_rel_high_alt
    end
    return factor_proximate::R
end
```

Referenced In: DetermineAltitudeDependentCOCFactor(p. 273)

3.3.2.3 Offline Vertical Cost Estimation

OFFLINECOSTESTIMATION (Algorithm 260) outputs a vector containing offline costs associated with each action for an individual intruder based on the dynamic state and the resolution advisory (RA) state. The dynamic state is received as input through the *samples_vert* and *b_tau_int* variables while *action_own_prev* and *action_indiv_prev* contain the RA state. The costs are computed by looking up pre-calculated values stored in tables that were generated and compressed offline. The principles of offline cost table generation can be found in [2]. The compressed offline cost tables are provided as part of the ACAS sXu system.

The input variable *equip_int* is used to choose either the previous global action or the previous individual action for this intruder to use as the current action when looking up the offline cost values for all possible subsequent actions. If the intruder is equipped, the individual advisory is used to maintain consistency with the last coordination message transmitted. If the intruder is unequipped, the global advisory is used. In this context, the value of *equip_int* is TRUE only when the intruder is able to send resolution advisory coordination messages.

Pre-processing is required to prepare the dynamic states for use by the tables. The tau values

in the offline cost table may not align perfectly with the tau values in *b_tau_int* produced by TAUESTIMATION. GETEQUIVCLASSTABLEMAXCUTVALUES (Algorithm 263) returns a COMBINED-VERTICALBELIEF structure (*vtable_max*), with the values of *z_rel*, *dz_own*, and *dz_int* set to the edges of the offline cost table. The *weight* value in that data structure is always set to *1.0*.

The algorithm loops over the set of vertical samples to produce an offline cost vector of all the vertical actions. Vertical belief samples are extracted one-by-one from *samples_vert* for individual processing. OFFLINESTATESSCALEFACTOR (Algorithm 262) is used to gracefully handle values of *z_rel*, *dz_own*, and *dz_int* that go beyond the edges of the offline cost table. If one or more of the dynamic state values is beyond the edge of the table, as captured in *vtable_max*, then *scale_-factor* is used to proportionally adjust all three values to the appropriate table values. In most cases, the value of *scale_factor* will be *1.0* and have no effect on the belief state values.

The dynamic state point and offline tables produce an offline cost vector of all the actions, including those not currently available, through a call to LOOKUPOFFLINECOST (Algorithm 265). OFFLINECOSTESTIMATION sums the weighted, negative of the results from each call to LOOKUPOFFLINECOST to convert from "values" (negative costs) to costs.

Appendix G describes the functionality that enables scaling of the offline policy to make it more or less sensitive. Selection of scaling parameters throughout the algorithms in this section is controlled by the variable *idx_scale*, which specifies the row of scaling values to utilize in the scaling matrix. Implementations including the scaling provision permit other values of *idx_-scale*.

This algorithm takes as input *mode_int*, *samples_vert*, *b_tau_int*, *equip_int*, *action_own_prev*, *action_indiv_prev*, and *idx_scale*. This algorithm returns *cost*.

Algorithm 260 OfflineCostEstimation

```
function OfflineCostEstimation( mode_int::Z, samples_vert::Vector{CombinedVerticalBelief(p. 515)},
                        b_tau_int::Vector{TauBelief(p. 522)}, equip_int::Bool,
                        action_own_prev::Z, action_indiv_prev::Z , idx_scale::Z )
    N_actions::Z             = params().actions.num_actions
    action_pattern_number::Vector{Z} = params().modes[mode_int].cost_estimation.offline.origami.action_-
        pattern_number
    N_allowable_actions::Vector{Z}   = params().modes[mode_int].cost_estimation.offline.origami.N_-
        allowable_actions
    allowable_actions::Matrix{Z}     = params().modes[mode_int].cost_estimation.offline.origami.allowable_-
        actions
    equiv_class_number::Matrix{Z}    = params().modes[mode_int].cost_estimation.offline.origami.equiv_-
        class_number
    minblocks_index::Matrix{Z}       = params().modes[mode_int].cost_estimation.offline.origami.minblocks_-
        index
    minblocks_table_content::RDataTable(p. 521) = params().modes[mode_int].cost_-
        estimation.offline.origami.minblocks_table_content
    equiv_class_table_content::Vector{RDataTableScaled(p. 522)} = params().modes[mode_int].cost_-
        estimation.offline.origami.equiv_class_table_content
    cut_points::Vector{R}    = params().modes[mode_int].cost_estimation.offline.origami.equiv_class_table_-
        content[1].scaled_policy_cuts_set[idx_scale]
    cut_counts::Vector{Z}    = params().modes[mode_int].cost_estimation.offline.origami.equiv_class_table_-
        content[1].cut_counts
    idx_cut_start::Z         = 2
    N_cut_idx::Z             = length( cut_counts ) - idx_cut_start + 1
    action_prev_real::R = convert( R, COC )
    if equip_int
       action_prev_real = convert( R, action_indiv_prev )
    else
       action_prev_real = convert( R, action_own_prev )
    end
    b_tau_ec_int::Vector{TauBelief(p. 522)} = GetOfflineCostTauBeliefs(p. 278)( mode_int, b_tau_int, idx_scale )
    vtable_max::CombinedVerticalBelief(p. 515) = GetEquivClassTableMaxCutValues(p. 279)( mode_int, idx_scale )
    cost::Vector{R} = zeros( R, N_actions )
    for j in 1:length( samples_vert )
        scale_factor::R  = OfflineStatesScaleFactor(p. 279)( samples_vert[j], vtable_max )
        z_rel_scaled::R  = scale_factor * samples_vert[j].z_rel
        dz_own_scaled::R = scale_factor * samples_vert[j].dz_own
        dz_int_scaled::R = scale_factor * samples_vert[j].dz_int
        point::Vector{R} = [ NaN,
                             z_rel_scaled,
                             dz_own_scaled,
                             dz_int_scaled,
                             action_prev_real
                           ]
        offline_table::TRMOfflineTable(p. 525) = TRMOfflineTable(p. 525)(N_allowable_actions,
                                                minblocks_index, minblocks_table_content,
                                                equiv_class_table_content, action_pattern_number,
                                                equiv_class_number, allowable_actions)
        point_cost::Vector{R} =
            LookupOfflineCost(p. 281)( point, N_actions, b_tau_ec_int, offline_table,
                                 cut_points, cut_counts, idx_cut_start, N_cut_idx )
        for k = 1:N_actions
            cost[k] = cost[k] - (samples_vert[j].weight * point_cost[k])
        end
    end
    return cost::Vector{R}
end
```

Referenced In: StateAndCostEstimation(p. 251)

GETOFFLINECOSTTAUBELIEFS (Algorithm 261) maps the tau beliefs from TAUESTIMATION into the tau values in the equivalence class table to create an array of tau beliefs that can be used in

LOOKUPOFFLINECOST. The tau values used in the equivalence class table are read directly from that table into *T_tau_values*. The tau values in the output tau belief vector (*b_tau_ec_int*) are set to the tau values in *T_tau_values*. The weight associated with each tau value in *b_tau_ec_int* is the sum of the weights for the associated range of tau values in *b_tau_int*, the tau beliefs from TAUESTIMATION. The weight for the last tau value in *b_tau_ec_int* is the sum of all the remaining weights in *b_tau_int*.

This algorithm takes as input *mode_int*, *b_tau_int*, and *idx_scale*. This algorithm returns *b_tau_-ec_int*.

Algorithm 261 GetOfflineCostTauBeliefs

```
function GetOfflineCostTauBeliefs( mode_int::Z, b_tau_int::Vector{TauBelief(p. 522)}, idx_scale::Z )
    equiv_class_table_content::Vector{RDataTableScaled(p. 522)} =
          params().modes[mode_int].cost_estimation.offline.origami.equiv_class_table_content
    N_tau_values::Z         = equiv_class_table_content[1].cut_counts[1]
    T_tau_values::Vector{R} = equiv_class_table_content[1].scaled_policy_cuts_set[idx_scale][1:N_tau_values]
    t_in::Z = length( b_tau_int )
    i_in::Z = 1
    b_tau_ec_int::Vector{TauBelief(p. 522)} = Vector{TauBelief(p. 522)}( undef, N_tau_values )
    for i_out = 1:N_tau_values
        b_tau_ec_int[i_out] = TauBelief(p. 522)( T_tau_values[i_out], 0.0 )
        while (i_in <= t_in) && (b_tau_int[i_in].tau <= b_tau_ec_int[i_out].tau)
            b_tau_ec_int[i_out].weight = b_tau_ec_int[i_out].weight + b_tau_int[i_in].weight
            i_in = i_in + 1
        end
    end
    while (i_in <= t_in)
        b_tau_ec_int[N_tau_values].weight = b_tau_ec_int[N_tau_values].weight + b_tau_int[i_in].weight
        i_in = i_in + 1
    end
    return b_tau_ec_int::Vector{TauBelief(p. 522)}
end
```

Referenced In: OfflineCostEstimation(p. 277)

OFFLINESTATESSCALEFACTOR (Algorithm 262) scales values that are beyond the edges of the offline cost table. Two of the table states, the current advisory and horizontal tau, cannot have values outside the prescribed limits. All valid RA states are represented and the final tau state is understood to be infinite.

The remaining three states, relative vertical separation, own aircraft vertical rate, and intruder vertical rate, can all fall outside the bounds of the offline cost table. The case of large relative vertical separation is generally not of interest to the logic and the final entry can be safely used for all values that fall beyond it. The case of large vertical rates, on the other hand, does have performance implications if not handled properly. While not generally expected, high performance aircraft can exceed the vertical rates used in the table.

In order to account for this case, the estimated vertical separation and both vertical rates can be scaled while preserving the time until loss of vertical separation. This is accomplished by computing a *scale_factor*, which is simply the largest magnitude vertical rate outside the table bounds divided by the largest vertical rate in the table. In the likely event that neither vertical rate is outside the table edge, the *scale_factor* is set to *1.0* so as to have no effect. The *scale_factor* can then be used in OFFLINECOSTESTIMATION to alter the vertical belief samples immediately

before they are used to look-up costs in the table. The scaling does not persist beyond the table look-ups; all future uses of the vertical states use the original, unscaled values.

This algorithm takes as input *sample_vert* and *vtable_max*. This algorithm returns *scale_factor*.

Algorithm 262 OfflineStatesScaleFactor

```
function OfflineStatesScaleFactor( sample_vert::CombinedVerticalBelief(p. 515), vtable_max::CombinedVerticalBelief(p. 515) )
    scale_factor::R = 1.0
    if (abs( vtable_max.dz_own ) < abs( sample_vert.dz_own ))
        scale_factor = abs( vtable_max.dz_own / sample_vert.dz_own )
    end
    if (abs( vtable_max.dz_int ) < abs( sample_vert.dz_int ))
        scale_factor = min( scale_factor, abs( vtable_max.dz_int / sample_vert.dz_int ) )
    end
    return scale_factor::R
end
```

Referenced In: OfflineCostEstimation(p. 277)

GETEQUIVCLASSTABLEMAXCUTVALUES (Algorithm 263) returns the maximum cut value for each dimension in the offline cost table. The values are returned in a COMBINEDVERTICALBELIEF data structure in which the weight value is set to *1.0*.

It uses the helper algorithm GETTABLEMAXCUTVALUE to return the maximum value for a specific dimension. The dimensions are referenced by name.

The format of the offline cost table is described in Appendix F, DATA TABLE FORMAT SPECIFICATION.

This algorithm takes as input *mode_int* and *idx_scale*. This algorithm returns *table_max*.

Algorithm 263 GetEquivClassTableMaxCutValues

```
function GetEquivClassTableMaxCutValues( mode_int::Z, idx_scale::Z )
    equiv_class_table_content::Vector{RDataTableScaled(p. 522)}  =
         params().modes[mode_int].cost_estimation.offline.origami.equiv_class_table_content
    table_max::CombinedVerticalBelief(p. 515) = CombinedVerticalBelief(p. 515)()
    table_max.z_rel  = GetTableMaxCutValue(p. 280)( equiv_class_table_content[1], "relh", idx_scale )
    table_max.dz_own = GetTableMaxCutValue(p. 280)( equiv_class_table_content[1], "dh.0", idx_scale )
    table_max.dz_int = GetTableMaxCutValue(p. 280)( equiv_class_table_content[1], "dh.1", idx_scale )
    table_max.weight = 1.0
    return table_max::CombinedVerticalBelief(p. 515)
end
```

Referenced In: OfflineCostEstimation(p. 277)

GETTABLEMAXCUTVALUE (Algorithm 264) returns the maximum value for a specific dimension in the table. This is done by first finding the index in the *table_data.cut_names* array corresponding to the specified dimension. It then determines the indices for the start and end of the cuts associated with that dimension. Finally, it goes through that list of cuts to find the maximum value, which it returns.

This algorithm takes as input *table_data*, *cut_name*, and *idx_scale*. This algorithm returns *max_-*

cut_value.

Algorithm 264 GetTableMaxCutValue

```
function GetTableMaxCutValue( table_data::RDataTableScaled(p. 522), cut_name::String , idx_scale::Z )
    max_cut_value::R = 0.0
    cut_start::Z = 1
    cut_index::Z = 1
    for i in 1:length(table_data.cut_names)
         if (table_data.cut_names[i] == cut_name)
             cut_index = i
             break
         end
    end
    for i in 1:cut_index-1
         cut_start = cut_start + table_data.cut_counts[i]
    end
    cut_end::Z = cut_start + table_data.cut_counts[cut_index] - 1
    for i in cut_start:cut_end
        max_cut_value = max( max_cut_value, table_data.scaled_policy_cuts_set[idx_scale][i] )
    end
    return max_cut_value::R
end
```

Referenced In: GetEquivClassTableMaxCutValues(p. 279)

LOOKUPOFFLINECOST (Algorithm 265) performs the cost table lookups for a given state point across all the tau beliefs. The generated tables are compressed using a technique called Data Origami [14]. Data Origami reformats the original offline table into a collection of smaller tables referred to as splits, where each split corresponds to one of the allowable state transitions from the current action (resolution advisory state). Each of these split tables is normalized by subtracting out a set of MinBlock costs, which are the minimum costs across the tau dimension. Lastly, the normalized split tables are compared against one another and similar splits are identified and grouped into equivalence classes. The resulting MinBlocks and equivalence class tables are fixed sized block dictionaries as specified in Appendix F.

Due to the discrete nature of the data tables, most input states cannot be looked up directly in the table. Instead, input states must be represented using a collection of valid table indices that have been weighted appropriately. This mapping from input states to table indices is performed using multi-linear interpolation through the call to INTERPOLANTS (Algorithm 385). After this mapping, LOOKUPOFFLINECOST loops over the allowable actions, the mapped table indices, and the tau beliefs to extract the appropriate values in the cost table, weighting and summing as it goes along. Unavailable actions are given infinite cost as a result of the initialization of *cost*. Upon completion, the cost vector for all the actions is returned.

This algorithm takes as input *point*, *num_actions*, *b_tau_int*, *origami*, *cut_points*, *cut_counts*, *idx_cut_start*, *N_cut_idx*, and *B_use_nearest_neighbor*. This algorithm returns *cost*.

Algorithm 265 LookupOfflineCost

```
function LookupOfflineCost( point::Vector{R}, num_actions::Z, b_tau_int::Vector{TauBelief(p. 522)},
                    offline_table::TRMOfflineTable(p. 525), cut_points::Vector{R}, cut_counts::Vector{Z},
                    idx_cut_start::Z, N_cut_idx::Z,
                    B_use_nearest_neighbor::Bool = false )
    action_pattern_number::Vector{Z} = offline_table.action_pattern_number
    N_allowable_actions::Vector{Z}   = offline_table.N_allowable_actions
    allowable_actions::Matrix{Z}     = offline_table.allowable_actions
    equiv_class_number::Matrix{Z}    = offline_table.equiv_class_number
    minblocks_index::Matrix{Z}       = offline_table.minblocks_index
    minblocks_table_content::RDataTable(p. 521) = offline_table.minblocks_table_content
    equiv_class_table_content::Vector{RDataTableScaled(p. 522)} = offline_table.equiv_class_table_content
    cost::Vector{R} = fill( convert( R, -Inf ), num_actions )
    st_trans::Z     = convert( Z, point[end] )
    action_pattern::Z           = action_pattern_number[st_trans]
    N_allowable_action_count::Z = N_allowable_actions[action_pattern]
    (indices::Vector{Z}, weights::Vector{R}, block_size::Z) =
           Interpolants(p. 416)( point, cut_counts, idx_cut_start, N_cut_idx,
                         cut_points, B_use_nearest_neighbor )
    tau_value_count::Z = equiv_class_table_content[1].cut_counts[1]
    split_num_count::Z = minblocks_table_content.cut_counts[1]
    index_ec_max::Z    = (block_size * tau_value_count) + 1
    index_ec_pos::Vector{Z} = Vector{Z}( undef, length( indices ) )
    index_ec_neg::Vector{Z} = Vector{Z}( undef, length( indices ) )
    index_mb::Vector{Z}     = Vector{Z}( undef, length( indices ) )
    for j = 1:length( indices )
        index_ec_pos[j] = ((indices[j] - 1) * tau_value_count) + 1
    index_ec_neg[j] = index_ec_max - index_ec_pos[j] - tau_value_count + 1
        index_mb[j] = ((indices[j] - 1) * split_num_count) + 1
    end
    for k = 1:N_allowable_action_count
        action::Z = allowable_actions[ action_pattern, k ]
        equiv_class::Z = equiv_class_number[ action, st_trans ]
        split_num::Z   = minblocks_index[ action, st_trans ]
        local index_ec::Vector{Z}
        index_ec = index_ec_pos
    if (0 <= equiv_class)
       index_ec = index_ec_pos
    else
       equiv_class = -equiv_class
       index_ec = index_ec_neg
    end
        data_ec::Vector{Float16} = equiv_class_table_content[equiv_class].data
        data_mb::Vector{Float16} = minblocks_table_content.data
        cost_act::R = -Inf
        for j = 1:length( indices )
            cost_point::R = 0.0
            offset_ec::UInt32 = index_ec[j] - 1
            for i = 1:length( b_tau_int )
                cost_point = cost_point + (b_tau_int[i].weight * convert( R, data_ec[offset_ec + i] ))
            end
            offset_mb::UInt32 = index_mb[j] + split_num - 1
            if (cost_act == -Inf)
                cost_act = weights[j] * (cost_point + convert( R, data_mb[offset_mb] ))
            else
                cost_act = cost_act + (weights[j] * (cost_point + convert( R, data_mb[offset_mb] )))
            end
        end
        cost[action] = cost_act
    end
    return (cost::Vector{R})
end
```

Referenced In: GetPolicyTableCosts(p. 190), OfflineCostEstimation(p. 277)

3.3.2.4 Online Vertical Cost Estimation Independent of Coordination

An overview of the ONLINEUNCOORDINATEDCOSTESTIMATION component in STATEANDCOSTESTIMATION is shown in Figure 10.

ONLINEUNCOORDINATEDCOSTESTIMATION (Algorithm 266) determines the values of the online costs that are not dependent on coordination with intruding aircraft. Online costs capture characteristics of the current state, such as altitudes, that cannot be suitably reflected in the offline cost tables. The online costs determined in this algorithm are independent of the resolution advisory coordination state of own aircraft and the intruder aircraft.

ONLINEUNCOORDINATEDCOSTESTIMATION always performs its evaluation with regard to a single intruder, independent of any other intruders.

It first computes values needed by the various online cost algorithms.

GETMODIFIEDGLOBALRATES (Algorithm 267) determines the vertical rate limits for the previous cycle (*dz_min_prev* and *dz_max_prev*) to be used for computing online costs.

EXPECTEDTAU (Algorithm 378) is called twice to provide a point estimate for time until loss of separation under different assumptions. The weighted average value for tau including the horizon (*tau_expected*) and excluding the horizon (*tau_expected_no_horizon*) are used for computing online costs. See section 3.3.2.2.1 for a discussion of tau.

The cost for each action in the output cost vector, *cost_ra*, is initialized to 0.0.

The *update* variable is initialized to TRUE. The *update* variable is used to flag the first iteration so each online cost algorithm will know to update the state variables in *s_c*. That set of updates will occur only once for each intruder on each cycle.

ONLINEUNCOORDINATEDCOSTESTIMATION then loops over all the possible actions to compute the online costs for each action. Each action (*act*) is represented as a minimum and maximum vertical rate (*dz_min* and *dz_max*).

ACTIONTORATES (Algorithm 373) is used to obtain the minimum and maximum vertical rates for the action under evaluation. ACTIONTORATES is a frequently used helper algorithm that is intentionally omitted from the flow diagrams.

For each action under evaluation, ONLINEUNCOORDINATEDACTIONCOSTESTIMATION (Algorithm 268) computes the sum of the online costs to be used for selecting resolution advisories (*cost_ra[act]*).

Later, in ONLINECOSTESTIMATION (Section 3.3.2.6), the coordination-independent online costs for each action, in *cost_ra*, will be summed with the associated coordination-dependent online costs.

ONLINEUNCOORDINATEDCOSTESTIMATION requires knowledge of numerous variables to calculate the various online costs. The required inputs include the previous global resolution advisory issued by the system (*a_global_prev*), the previous resolution advisory for the intruder considered in isolation (*a_indiv_prev*), and the online cost state (*s_c*). The weighted average of the own aircraft and intruder's vertical states (*h_own*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, and *dz_int_ave*) are also needed along with the samples and corresponding weights for tau (*b_tau_int*) and whether the intruder is able to send resolution advisory coordination messages (*equip_int*).

Descriptions of the ONLINEUNCOORDINATEDCOSTESTIMATION output variables are found in Table 40.

This algorithm takes as input *mode_int*, *h_own*, *vert_rate_limit*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *b_tau_int*, *a_global_prev*, *a_indiv_prev*, *equip_int*, and *s_c*. This algorithm outputs *cost_ra*, and updates *s_c*.

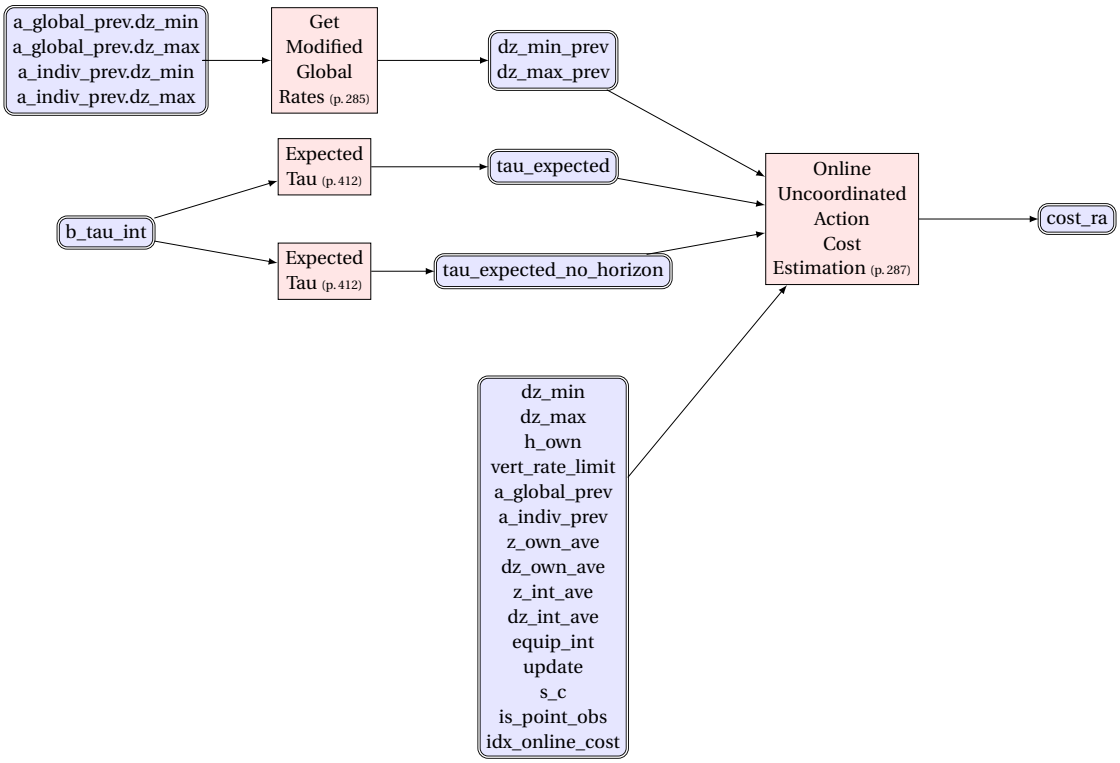

Figure 10. OnlineUncoordinatedCostEstimation overview.

Table 40. Vertical TRM Algorithm Output Variables - OnlineUncoordinatedCostEstimation

Variable	Units	Type	Description
cost_ra	N/A	real	Vector with online coordination-independent cost of each action; used to select RAs for single intruder

Algorithm 266 OnlineUncoordinatedCostEstimation

```
function OnlineUncoordinatedCostEstimation( mode_int::Z, h_own::R, vert_rate_limit::R,
                                  z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R,
                                  b_tau_int::Vector{TauBelief(p.522)}, a_global_prev::GlobalAdvisory(p.517),
                                  a_indiv_prev::IndividualAdvisory(p.519), equip_int::Bool,
                                  s_c::OnlineCostState(p.520), is_point_obs::Bool, idx_online_cost::Z )
    N_actions::Z = params().actions.num_actions
    (dz_min_prev::R, dz_max_prev::R)  =
          GetModifiedGlobalRates(p.285)( a_global_prev.dz_min, a_global_prev.dz_max,
                                  a_indiv_prev.dz_min, a_indiv_prev.dz_max )
    tau_expected::R            = ExpectedTau(p.412)( b_tau_int, false )
    tau_expected_no_horizon::R = ExpectedTau(p.412)( b_tau_int, true )
    cost_ra::Vector{R} = zeros( R, N_actions )
    update::Bool       = true
    for act::Z in 1:N_actions
        (dz_min::R, dz_max::R) = ActionToRates(p.410)( act, dz_own_ave, a_global_prev.action,
                                                a_global_prev.dz_min, a_global_prev.dz_max,
                                                a_global_prev.ddz )
        cost_ra[act] =
            OnlineUncoordinatedActionCostEstimation(p.287)( mode_int, dz_min, dz_max, h_own, vert_rate_limit,
                                a_global_prev, a_indiv_prev, dz_min_prev, dz_max_prev, z_own_ave,
                                dz_own_ave, z_int_ave, dz_int_ave, tau_expected,tau_expected_no_horizon,
                                equip_int, update, s_c, is_point_obs, idx_online_cost )
        update = false
    end
    return (cost_ra::Vector{R})
end
```

Referenced In: StateAndCostEstimation(p.251)

GETMODIFIEDGLOBALRATES (Algorithm 267) determines the values of the minimum and maximum vertical rates representing the advisory for the previous cycle. These vertical rate limits are used for computing online costs for a single intruder.

The vertical rate limits are adjusted in multi-intruder scenarios to properly account for differences between the resolution advisory (RA) for the individual intruder (or individual RA) and the global RA. When the individual RA is clear of conflict (COC) or the global RA is Multi-Threat Level-Off (MTLO), the vertical rate limits associated with the individual RA are used.

When there is a single intruder, the global RA and the individual RA will be the same, and so will their minimum and maximum vertical rates.

The individual RA is represented as the minimum and maximum vertical rates (*dz_min_int_prev* and *dz_max_int_prev*). The global RA is represented as the minimum and maximum vertical rates (*dz_min_prev* and *dz_max_prev*). Both RAs were produced on the previous TRM cycle.

GETMODIFIEDGLOBALRATES sets the vertical rate limits (*dz_min* and *dz_max*) as follows:

- When there was no RA on the previous cycle for the individual intruder, the vertical rate limits for clear of conflict (COC) are used.
- When the previous global RA was a Multi-Threat Level-Off (MTLO), the sense of the individual RA (*sense_indiv*) is used to determine the vertical rate limits. In this case, the vertical rate limits are set to 'Do Not Descend' if the sense is up and to 'Do Not Climb' if the sense is down. If the sense is neither up nor down, the vertical rate limits associated with the individual RA are used.
- Otherwise, the vertical rate limits for the global RA are used.

This algorithm takes as input *dz_min_prev*, *dz_max_prev*, *dz_min_int_prev*, and *dz_max_int_prev*. This algorithm returns *dz_min* and *dz_max*.

Algorithm 267 GetModifiedGlobalRates

```
function GetModifiedGlobalRates( dz_min_prev::R, dz_max_prev::R, dz_min_int_prev::R, dz_max_int_prev::R )
    local dz_min::R
    local dz_max::R
    if IsCOC(p. 419)( dz_min_int_prev, dz_max_int_prev )
        dz_min = -Inf
        dz_max = Inf
    elseif IsMTLO(p. 425)( dz_min_prev, dz_max_prev )
        sense_indiv::Symbol = RatesToSense(p. 432)( dz_min_int_prev, dz_max_int_prev )
        if (sense_indiv == :Up)
            dz_min = 0
            dz_max = Inf
        elseif (sense_indiv == :Down)
            dz_min = -Inf
            dz_max = 0
        else
            dz_min = dz_min_int_prev
            dz_max = dz_max_int_prev
        end
    else
        dz_min = dz_min_prev
        dz_max = dz_max_prev
    end
    return (dz_min::R, dz_max::R)
end
```

Referenced In: OnlineUncoordinatedCostEstimation(p. 284), OnlineCostEstimation(p. 310)

ONLINEUNCOORDINATEDACTIONCOSTESTIMATION (Algorithm 268) returns an online cost sum for the given action and the individual intruder. The online costs determined in this algorithm are independent of the coordination state of own aircraft and the intruding aircraft. The associated algorithm, ONLINECOORDINATEDACTIONCOSTESTIMATION, determines online costs that are dependent on the resolution advisory coordination state of own aircraft and the intruding aircraft.

The value $cost_ra$ contains the sum of all the individual online costs for the given action. These costs are used when determining whether a resolution advisory should be issued, and if so, the type of resolution advisory to issue.

Each online cost algorithm called in ONLINEUNCOORDINATEDACTIONCOSTESTIMATION corresponds to a different event to be penalized, such as reversing or restarting the resolution advisory. After initialization, the algorithms update their own state variables only once during each call to ONLINEUNCOORDINATEDACTIONCOSTESTIMATION, even though ONLINEUNCOORDINATEDACTIONCOSTESTIMATION, and consequently each online cost algorithm, is called once for each action. The $update$ value is used to indicate when the individual online cost algorithms should update their state variables, contained in the ONLINECOSTSTATE data structure. The value is set to TRUE when computing the online cost for the first action and FALSE subsequently.

In the individual online cost algorithms, an equipped intruder is an intruding aircraft that is able to send resolution advisory coordination messages. The equipped intruder is indicated by

a value of TRUE for *equip_int*.

Common inputs to the individual online cost algorithms include the index to be used for acquiring the appropriate values of the configuration parameters (*mode_int*), the minimum vertical rate associated with the action (*dz_min*), the maximum vertical rate associated with the action (*dz_max*), *update*, and the relevant set of online cost state variables (*s_c*). The variable *mode_int* is the protection mode index.

This algorithm takes as input *mode_int, dz_min, dz_max, h_own, vert_rate_limit, a_global_prev, a_indiv_prev, dz_min_prev, dz_max_prev, z_own_ave, dz_own_ave, z_int_ave, dz_int_ave, tau_-expected, tau_expected_no_horizon, equip_int, update* and *s_c*. This algorithm returns *cost_ra*.

Algorithm 268 OnlineUncoordinatedActionCostEstimation

```
function OnlineUncoordinatedActionCostEstimation( mode_int::Z, dz_min::R, dz_max::R, h_own::R,
                            vert_rate_limit::R, a_global_prev::GlobalAdvisory(p.517),
                            a_indiv_prev::IndividualAdvisory(p.519), dz_min_prev::R, dz_max_prev::R,
                            z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R,
                            tau_expected::R, tau_expected_no_horizon::R, equip_int::Bool,
                            update::Bool, s_c::OnlineCostState(p.520), is_point_obs::Bool,
                            idx_online_cost::Z )
    inc_cost::R = 0.0
    cost_ra::R  = inc_cost
    inc_cost =  BadTransitionCost(p.291)( mode_int, dz_min, dz_max, dz_own_ave, a_global_prev.dz_min,
                                  a_global_prev.dz_max, a_indiv_prev.dz_min, a_indiv_prev.dz_max,
                                  equip_int, update, s_c.bad_transition )
    cost_ra = inc_cost
    if ( !is_point_obs )
        inc_cost = AltitudeInhibitCost(p.290)( mode_int, dz_min, dz_max, dz_min_prev, dz_max_prev,
                                       s_c.alt_inhibit )
        cost_ra = cost_ra + inc_cost
        inc_cost = AdvisoryRestartCost(p.288)( mode_int, dz_min, dz_max, dz_min_prev, dz_max_prev,
                                       update, s_c.advisory_restart )
        cost_ra = cost_ra + inc_cost
        inc_cost =  InitializationCost(p.299)( mode_int, dz_min, dz_max, update, s_c.initialization )
        cost_ra = cost_ra + inc_cost
        inc_cost = PreventEarlyCOCCost(p.299)( mode_int, dz_min, dz_max, dz_min_prev, dz_max_prev,
                                       z_own_ave, dz_own_ave, z_int_ave, dz_int_ave,
                                       tau_expected, update, s_c.prevent_early_coc,
                                       idx_online_cost )
        cost_ra = cost_ra + inc_cost
        inc_cost = AltitudeDependentCOCCost(p.289)( mode_int, dz_min, dz_max, dz_min_prev, dz_max_prev,
                                            dz_own_ave, s_c.altitude_dependent_coc.scaled_cost_coc_ra,
                                            idx_online_cost )
        cost_ra = cost_ra + inc_cost
        inc_cost = CriticalIntervalProtectionCost(p.294)( mode_int, dz_min, dz_max, dz_min_prev, dz_max_prev,
                                                  z_own_ave, z_int_ave, dz_own_ave, dz_int_ave,
                                                  update, s_c.critical_interval_protection,
                                                  idx_online_cost )
        cost_ra = cost_ra + inc_cost
        inc_cost = SafeCrossingRADeferralCost(p.302)( mode_int, dz_min, dz_max, equip_int, z_own_ave, z_int_ave,
                                              dz_own_ave, dz_int_ave, dz_min_prev, dz_max_prev,
                                              tau_expected_no_horizon, update,
                                              s_c.safe_crossing_ra_deferral, idx_online_cost )
        cost_ra = cost_ra + inc_cost
        inc_cost = VerticalRateLimitDependentStrengthenCost(p.304)( mode_int, dz_min, dz_max, dz_min_prev,
                                                                    dz_max_prev, vert_rate_limit )
        cost_ra = cost_ra + inc_cost
    else
        inc_cost = PointObsDescendCost(p.305)( mode_int, dz_min, dz_max )
        cost_ra = cost_ra + inc_cost
        inc_cost = PointObsPreventEarlyCOCCost(p.306)( mode_int, dz_min_prev, dz_max_prev, z_own_ave,
                                                z_int_ave, dz_own_ave, dz_int_ave, tau_expected )
         cost_ra = cost_ra + inc_cost
    end
    return (cost_ra::R)
end
```

Referenced In: OnlineUncoordinatedCostEstimation(p.284)

3.3.2.4.1 Advisory Restart Cost

ADVISORYRESTARTCOST (Algorithm 269) is used to penalize the issuance of a new advisory if a previous advisory was issued recently. The ADVISORYRESTARTCOST algorithm penalizes resolution advisories by *C_advisory* whenever a prior resolution advisory has been terminated for

fewer than *T_limit* processing cycles. To do this, ADVISORYRESTARTCOST maintains memory of whether an advisory has been issued (*s_c.alerted*), whether an advisory has terminated (*s_c.term*), and the amount of time for which an advisory has terminated (*s_c.t_term*).

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATEADVISORYRESTARTCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 269 AdvisoryRestartCost

```
function AdvisoryRestartCost( mode_int::Z, dz_min::R, dz_max::R, dz_min_prev::R, dz_max_prev::R,
                              update::Bool, s_c::AdvisoryRestartCState(p. 526) )
    T_limit::Z    = params().modes[mode_int].cost_estimation.online.advisory_restart.T_limit
    C_advisory::R = params().modes[mode_int].cost_estimation.online.advisory_restart.C_advisory
    if update
        UpdateAdvisoryRestartCState(p. 288)( dz_min_prev, dz_max_prev, s_c )
    end
    cost::R = 0.0
    if s_c.term && (s_c.t_term < T_limit) && !IsCOC(p. 419)( dz_min, dz_max )
        cost = C_advisory
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

UPDATEADVISORYRESTARTCSTATE (Algorithm 270) is used to update the state variables of ADVISORYRESTARTCOST once per cycle. First, the algorithm checks to see if there was no alert on the previous cycle and either there was an alert two cycles prior or that an alert has been terminated any time previously. If true, the advisory termination flag (*s_c.term*) is set to TRUE and the alert termination counter (*s_c.t_term*) is incremented. If not, the advisory termination flag is set to FALSE and the alert termination counter is reset. Finally, the previously alerted flag (*s_c.alerted*) is updated using the vertical rate limits of the previous cycle for use in the next cycle.

This algorithm takes as input *dz_min_prev*, *dz_max_prev*, and *s_c*. The algorithm outputs updates to *s_c::AdvisoryRestartCState*.

Algorithm 270 UpdateAdvisoryRestartCState

```
function UpdateAdvisoryRestartCState( dz_min_prev::R, dz_max_prev::R, s_c::AdvisoryRestartCState(p. 526) )
    if (s_c.alerted || s_c.term) && IsCOC(p. 419)( dz_min_prev, dz_max_prev )
        s_c.term   = true
        s_c.t_term = s_c.t_term + 1
    else
        s_c.term   = false
        s_c.t_term = 0
    end
    s_c.alerted = !IsCOC(p. 419)( dz_min_prev, dz_max_prev )
end
```

Referenced In: AdvisoryRestartCost(p. 288)

3.3.2.4.2 Altitude Dependent Clear of Conflict Cost

ALTITUDEDEPENDENTCOCCOST (Algorithm 271) is used to penalize transitions to clear of conflict as a function of altitude. The effect is scaled so that at higher altitudes the transitions to clear of conflict are more heavily penalized. Initial transitions to maintain RAs are also penalized but at a rate reduced by *X_maintain_factor*.

The altitude scaling and other geometric scaling takes place before this function when the *scaled_cost_coc* value is computed using DETERMINEALTITUDEDEPENDENTCOCFACTOR (Algorithm 257). The *scaled_cost_coc* input is calculated once per intruder per cycle.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, *dz_own_ave*, and *scaled_cost_coc*. This algorithm returns *cost*.

Algorithm 271 AltitudeDependentCOCCost

```
function AltitudeDependentCOCCost( mode_int::Z, dz_min::R, dz_max::R, dz_min_prev::R, dz_max_prev::R,
                                   dz_own_ave::R, scaled_cost_coc::R, idx_online_cost::Z )
    R_threshold_lolo::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.R_threshold_lolo
    R_rolloff_lolo::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.R_rolloff_lolo
    X_maintain_factor::R =
        params().modes[mode_int].cost_estimation.online.altitude_dependent_coc.X_maintain_factor[idx_-
            online_cost]
    cost::R  = 0.0
    if IsCOC(p. 419)( dz_min, dz_max )
        cost = scaled_cost_coc
    elseif IsPreventive(p. 426)( dz_min, dz_max ) && !IsPreventive(p. 426)( dz_min_prev, dz_max_prev )
        if (dz_min < dz_own_ave < dz_max)
            cost = scaled_cost_coc
        else
            cost = scaled_cost_coc * CalculateThresholdRampDownFactor(p. 410)( abs( dz_own_ave ),
                R_threshold_lolo, R_rolloff_lolo )
        end
    elseif IsMaintain(p. 424)( dz_min, dz_max ) && IsCOC(p. 419)( dz_min_prev, dz_max_prev )
        cost = scaled_cost_coc * X_maintain_factor
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

3.3.2.4.3 Altitude Inhibit Cost

ALTITUDEINHIBITCOST (Algorithm 272) controls the alerting behavior against intruder aircraft as the ownship approaches the ground. This is done by penalizing sets of resolution advisories below specified altitudes with the *C_inhibit* cost. The actions for which the cost applies are specified using *R_dz_lo*, *R_dz_hi*, *R_dz_lo_prev*, and *R_dz_hi_prev*. A negative value in *C_inhibit* is used to limit the applicability of certain altitude inhibits that are too broadly applicable.

The inhibit costs are applied with a simple hysteresis check. To do this, the algorithm keeps knowledge of current altitude inhibited state (*s_c.inhibited*). Initialization of the *s_c.inhibited* is defined by the parameter file. The cost is never applied if the action corresponds to clear of conflict (COC). The algorithm ISCOC (Algorithm 387) is used to indicate whether the action

corresponds to a clear of conflict.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, and *s_c*. This algorithm returns *cost*.

Algorithm 272 AltitudeInhibitCost

```
function AltitudeInhibitCost( mode_int::Z, dz_min::R, dz_max::R, dz_min_prev::R, dz_max_prev::R, s_c::
    AltitudeInhibitCState(p. 527) )
    C_inhibit::Vector{R} =
        params().modes[mode_int].cost_estimation.online.altitude_inhibit.C_inhibit
    R_dz_lo::Vector{R} =
        params().modes[mode_int].cost_estimation.online.altitude_inhibit.R_dz_lo
    R_dz_hi::Vector{R} =
        params().modes[mode_int].cost_estimation.online.altitude_inhibit.R_dz_hi
    R_dz_lo_prev::Vector{R} =
        params().modes[mode_int].cost_estimation.online.altitude_inhibit.R_dz_lo_prev
    R_dz_hi_prev::Vector{R} =
        params().modes[mode_int].cost_estimation.online.altitude_inhibit.R_dz_hi_prev
    cost::R = 0.0
    if !IsCOC(p. 419)( dz_min, dz_max )
        for i = 1:length( C_inhibit )
            if s_c.inhibited[i] &&
               ((R_dz_lo[i] <= dz_min) && (dz_max <= R_dz_hi[i])) &&
               ((R_dz_lo_prev[i] <= dz_min_prev) && (dz_max_prev <= R_dz_hi_prev[i]))
                cost = cost + C_inhibit[i]
            end
        end
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

3.3.2.4.4 Bad Transition Cost

BADTRANSITIONCOST (Algorithm 273) penalizes resolution advisories by *C_bad_transition* if the transition to that action from the current state is illegal. Penalties for some illegal transitions are encoded directly into the offline cost table. However, due to the limitations inherent to the offline cost table construction and represented states, some transitions must be dealt with online.

Illegal transitions are those matching one or more of the following conditions. Resolution advisories matching more than one of the conditions incur the penalty for each condition matched.

- Maintain Climb to Climb or Increase Descend
- Maintain Descend to Descend or Increase Climb
- Maintain Climb to Increase Climb (if not a strengthening)
- Maintain Descend to Increase Descend (if not a strengthening)
- Climb to Maintain Climb or Descend to Maintain Descend
- Increase Climb to Maintain Climb
- Increase Descend to Maintain Descend
- Transitions to Climbs, Descends, Increase Climbs, and Increase Descends if the aircraft has a vertical rate higher than that specified by the advisory

- Transitions to Maintain Climb or Maintain Descend if the aircraft has a vertical rate lower than *R_strengthen*

The helper algorithm BADMAINTAINTRANSITIONCOST is used to calculate the costs associated with bad transitions to and from Maintain advisories.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATEBADTRANSITIONCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_own_ave*, *dz_min_global_prev*, *dz_max_global_prev*, *dz_min_indiv_prev*, *dz_max_indiv_prev*, *equip_int*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 273 BadTransitionCost

```
function BadTransitionCost( mode_int::Z, dz_min::R, dz_max::R, dz_own_ave::R,
                   dz_min_global_prev::R, dz_max_global_prev::R,
                   dz_min_indiv_prev::R, dz_max_indiv_prev::R,
                   equip_int::Bool, update::Bool, s_c::BadTransitionCState(p. 527) )
    C_bad_transition::R =
        params().modes[mode_int].cost_estimation.online.bad_transition.C_bad_transition
    R_corrective::R     = params().actions.corrective_rate
    R_strengthen::R     = params().actions.strengthen_rate
    if update
        UpdateBadTransitionCState(p. 292)( dz_min_global_prev, dz_max_global_prev,
                                   dz_min_indiv_prev, dz_max_indiv_prev, equip_int, s_c )
    end
    cost::R = 0.0
    sense_own::Symbol         = RatesToSense(p. 432)( dz_min, dz_max )
    ra_is_maintain::Bool      = IsMaintain(p. 424)( dz_min, dz_max )
    ra_is_strengthening::Bool = IsStrengthening(p. 428)( s_c.dz_min_prev, s_c.dz_max_prev, dz_min, dz_max )
    cost = BadMaintainTransitionCost(p. 293)( mode_int, dz_min, dz_max, dz_own_ave, C_bad_transition,
                                      sense_own, ra_is_maintain, ra_is_strengthening, s_c )
    if IsPreventive(p. 426)( dz_min, dz_max ) && IsReversal(p. 427)( s_c.sense_own_prev, sense_own )
        cost = cost + C_bad_transition
    end
    if (R_corrective < dz_own_ave) &&
       (dz_min == R_corrective) && (dz_max == Inf) &&
       !((s_c.dz_min_prev == R_corrective) && (s_c.dz_max_prev == Inf))
        cost = cost + C_bad_transition
    elseif (R_strengthen < dz_own_ave) &&
           (dz_min == R_strengthen) && (dz_max == Inf) &&
           !((s_c.dz_min_prev == R_strengthen) && (s_c.dz_max_prev == Inf))
        cost = cost + C_bad_transition
    elseif (dz_own_ave < -R_corrective) &&
           (dz_min == -Inf) && (dz_max == -R_corrective) &&
           !((s_c.dz_min_prev == -Inf) && (s_c.dz_max_prev == -R_corrective))
        cost = cost + C_bad_transition
    elseif (dz_own_ave < -R_strengthen) &&
           (dz_min == -Inf) && (dz_max == -R_strengthen) &&
           !((s_c.dz_min_prev == -Inf) && (s_c.dz_max_prev == -R_strengthen))
        cost = cost + C_bad_transition
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

UPDATEBADTRANSITIONCSTATE (Algorithm 274) is used to update the state variables of BADTRANSITIONCOST once per cycle. Due to pairwise coordination between equipped aircraft, bad

transitions for equipped intruders are calculated using the previous individual RA while bad transitions for unequipped intruders use the previous global RA. If either the maximum or minimum vertical rate limit changed on the previous cycle, additional processing is required. If the transition on the previous cycle was to clear of conflict, the previous own aircraft RA sense (*s_c.sense_own_prev*) and the previous RA was a Maintain (*s_c.ra_is_maintain_prev*) variables are reset. Otherwise, the previous RA limits are used to determine the sense and whether or not it was a Maintain RA. Finally, the state variables for the previous RA vertical rate limits (*s_c.dz_min_prev* and *s_c.dz_max_prev*) are updated.

This algorithm takes as input *dz_min_global_prev*, *dz_max_global_prev*, *dz_min_indiv_prev*, *dz_max_indiv_prev*, *equip_int*, and *s_c*. This algorithm updates *s_c::BadTransitionCState*.

Algorithm 274 UpdateBadTransitionCState

```
function UpdateBadTransitionCState( dz_min_global_prev::R, dz_max_global_prev::R,
                            dz_min_indiv_prev::R, dz_max_indiv_prev::R,
                            equip_int::Bool, s_c::BadTransitionCState(p. 527) )
    if equip_int
        dz_min_prev::R = dz_min_indiv_prev
        dz_max_prev::R = dz_max_indiv_prev
    else
        dz_min_prev = dz_min_global_prev
        dz_max_prev = dz_max_global_prev
    end
    if (s_c.dz_min_prev != dz_min_prev) || (s_c.dz_max_prev != dz_max_prev)
        if IsCOC(p. 419)( dz_min_prev, dz_max_prev )
            s_c.ra_is_maintain_prev = false
            s_c.sense_own_prev = :None
        else
            sense_own_prev = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
            s_c.ra_is_maintain_prev = IsMaintain(p. 424)( dz_min_prev, dz_max_prev )
            s_c.sense_own_prev = sense_own_prev
        end
        s_c.dz_min_prev = dz_min_prev
        s_c.dz_max_prev = dz_max_prev
    end
end
```

Referenced In: BadTransitionCost(p. 291)

BADMAINTAINTRANSITIONCOST (Algorithm 275) is used to calculate the costs associated with bad transitions to and from Maintain advisories. First, the algorithm determines if a transition is being attempted from a Maintain RA to a non-Maintain RA. If so the transition is from Maintain Climb to either Climb or Increase Descend the cost is applied. Additionally, transitions from Maintain Descend to either Descend or Increase Climb are penalized. Finally, any transition from Maintain Climb to Increase Climb and from Maintain Descend to Increase Descend that is not also a strengthening also has the cost applied.

Similarly, the algorithm checks to see if the transition being attempted is from a non-Maintain RA to a Maintain RA. Transitions from Climb or Increase Climb to Maintain Climb and transitions from Descend or Increase Descend to Maintain Descend are penalized. Moreover, the initiation of a Maintain RA while the own aircraft vertical rate is below *R_corrective* is likewise penalized.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_own_ave*, *C_bad_transition*, *sense_own*, *ra_is_maintain*, *ra_is_strengthening*, and *s_c*. This algorithm returns *cost*.

Algorithm 275 BadMaintainTransitionCost

```
function BadMaintainTransitionCost( mode_int::Z, dz_min::R, dz_max::R, dz_own_ave::R, C_bad_transition::R,
                            sense_own::Symbol, ra_is_maintain::Bool, ra_is_strengthening::Bool,
                            s_c::BadTransitionCState(p. 527) )
    R_corrective::R = params().actions.corrective_rate
    R_strengthen::R = params().actions.strengthen_rate
    C_bad_maintain_initiation::R =
       params().modes[mode_int].cost_estimation.online.bad_transition.C_bad_maintain_initiation
    cost::R = 0.0
    if s_c.ra_is_maintain_prev && !ra_is_maintain
        if (s_c.sense_own_prev == :Up) &&
           ( ((dz_min == R_corrective) && (dz_max == Inf)) ||
             ((dz_min == -Inf) && (dz_max == -R_strengthen)) )
            cost = C_bad_transition
        elseif (s_c.sense_own_prev == :Down) &&
               ( ((dz_min == -Inf) && (dz_max == -R_corrective)) ||
                 ((dz_min == R_strengthen) && (dz_max == Inf)) )
            cost = C_bad_transition
        elseif (s_c.sense_own_prev == :Up) && !ra_is_strengthening &&
               ((dz_min == R_strengthen) && (dz_max == Inf))
            cost = C_bad_transition
        elseif (s_c.sense_own_prev == :Down) && !ra_is_strengthening &&
               ((dz_min == -Inf) && (dz_max == -R_strengthen))
            cost = C_bad_transition
        end
    elseif !s_c.ra_is_maintain_prev && ra_is_maintain
        if (sense_own == :Up) &&
           ( ((s_c.dz_min_prev == R_corrective) && (s_c.dz_max_prev == Inf)) ||
             ((s_c.dz_min_prev == R_strengthen) && (s_c.dz_max_prev == Inf)) )
            cost = C_bad_transition
        elseif (sense_own == :Down) &&
               ( ((s_c.dz_min_prev == -Inf) && (s_c.dz_max_prev == -R_corrective)) ||
                 ((s_c.dz_min_prev == -Inf) && (s_c.dz_max_prev == -R_strengthen)) )
            cost = C_bad_transition
        end
        if (abs( dz_own_ave ) < R_corrective)
            cost = cost + C_bad_maintain_initiation
        end
    end
    return cost::R
end
```

Referenced In: BadTransitionCost(p. 291)

3.3.2.4.5 Critical Interval Protection Cost

CRITICALINTERVALPROTECTIONCOST (Algorithm 276) is used to force alerts and eventually corrective actions in the particularly dangerous situation of extremely slow closure encounters. Slow closures are notoriously difficult as the estimation of tau becomes highly sensitive to the estimated closure rate. Additionally, they suffer from the unique characteristic of potentially being in horizontal conflict for extended periods of time. In most cases, other methods in the logic can provide sufficient robustness, but a few unique instances are covered by this method. To ensure safety, this cost uses the raw horizontal belief states, as opposed to tau, to identify cases where the horizontal separation and horizontal closure rate are sufficiently low to require action be taken.

Using the boolean flags in the cost state (*s_c*), alerts are forced at the prescribed levels. If *force_alert* is TRUE, transitions to Clear of Conflict are penalized by *C_force_alert*. If *force_climbdescend*

is TRUE, transitions to RAs with a target rate below *R_corrective* are penalized by *C_force_alert*. If *force_inc_climbdescend* is TRUE, transitions to RAs with a target rate below *R_strengthen* are penalized by *C_force_alert*. Furthermore, if either *force_climbdescend* or *force_inc_climbdescend* is TRUE, all convergent RAs will be penalized so long as the relative altitude between intruder and ownship is greater than *CROSSING_ALTITUDE_BUFFER*.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATECRITICALINTERVALPROTECTIONCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 276 CriticalIntervalProtectionCost

```
function CriticalIntervalProtectionCost( mode_int::Z, dz_min::R, dz_max::R,
                                dz_min_prev::R, dz_max_prev::R,
                                z_own_ave::R, z_int_ave::R,
                                dz_own_ave::R, dz_int_ave::R,
                                update::Bool, s_c::CriticalIntervalProtectionCState(p. 529),
                                idx_online_cost::Z )
    global CROSSING_ALTITUDE_BUFFER
    R_corrective::R  = params().actions.corrective_rate
    R_strengthen::R  = params().actions.strengthen_rate
    C_force_alert::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.C_force_alert[idx_-
          online_cost]
    if update
        UpdateCriticalIntervalProtectionCState(p. 296)( mode_int, dz_min_prev, dz_max_prev,
                                                z_own_ave, z_int_ave, dz_own_ave, dz_int_ave,
                                                s_c, idx_online_cost )
    end
    cost::R  = 0.0
    z_rel::R = z_int_ave - z_own_ave
    if s_c.force_inc_climbdescend
        if (CROSSING_ALTITUDE_BUFFER < z_rel) && (-R_strengthen < dz_max)
            cost = C_force_alert
        elseif (z_rel < -CROSSING_ALTITUDE_BUFFER) && (dz_min < R_strengthen)
            cost = C_force_alert
        end
    elseif s_c.force_climbdescend
        if (CROSSING_ALTITUDE_BUFFER < z_rel) && (-R_corrective < dz_max)
            cost = C_force_alert
        elseif (z_rel < -CROSSING_ALTITUDE_BUFFER) && (dz_min < R_corrective)
            cost = C_force_alert
        end
    elseif s_c.force_alert && IsCOC(p. 419)( dz_min, dz_max )
        cost = C_force_alert
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

UPDATECRITICALINTERVALPROTECTIONCSTATE (Algorithm 277) is used to update the state variables of CRITICALINTERVALPROTECTIONCOST once per cycle.

The three boolean flags, *force_alert*, *force_climbdescend*, and *force_inc_climbdescend*, are set to TRUE when an alert, a climb or descend alert, or an increase rate alert is needed, respectively. These flags are initialized to false, meaning no alerts will be forced. These conditions limit the

effect of this cost only to those cases where it is needed to provide safety.

First, applicability requirements are checked by calling IsOutsideCriticalInterval. If the cost is applicable, checks are performed to determine the strength of alert that is required. These involve projecting the vertical states of the own aircraft and intruder forward in time to determine the expected separation. A different time horizon is used for each alert strength. General alerts have a time horizon of *T_proximate*, climb and descend alerts have a time horizon of *T_proximate_climbdescend*, and increase rate alerts have a time horizon of *T_proximate_inc_climbdescend*. If the expected separation of a given alert strength falls below the required level, that type of alert will be forced. For the *force_climbdescend* and *force_inc_climbdescend* the RAs are required to be non-crossing. An exception to this general procedure is the case where the current vertical separation is below the crossing altitude buffer. In this case, a crossing alert is undefined so only *force_alert* is set to TRUE while *force_climbdescend* and *force_inc_climbdescend* are set to FALSE.

A final check using CriticalIntervalRequiresVerticalDivergence ensures only divergent RAs are issued in specific circumstances.

This algorithm takes as input *mode_int*, *dz_min_prev*, *dz_max_prev*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, and *s_c*. This algorithm updates *s_c*.

Algorithm 277 UpdateCriticalIntervalProtectionCState

```
function UpdateCriticalIntervalProtectionCState( mode_int::Z, dz_min_prev::R, dz_max_prev::R,
                                                 z_own_ave::R, z_int_ave::R,
                                                 dz_own_ave::R, dz_int_ave::R,
                                                 s_c::CriticalIntervalProtectionCState(p. 529),
                                                 idx_online_cost::Z )
    global CROSSING_ALTITUDE_BUFFER
    R_corrective::R  = params().actions.corrective_rate
    R_strengthen::R  = params().actions.strengthen_rate
    T_proximate::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.T_proximate[idx_online_-
           cost]
    T_proximate_climbdescend::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.T_proximate_-
           climbdescend[idx_online_cost]
    T_proximate_inc_climbdescend::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.T_proximate_inc_-
           climbdescend[idx_online_cost]
    H_rel_required::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.H_rel_required[idx_-
           online_cost]
    s_c.force_inc_climbdescend = false
    s_c.force_climbdescend     = false
    s_c.force_alert            = false
    z_rel::R = z_int_ave - z_own_ave
    coc_prev::Bool = IsCOC(p. 419)( dz_min_prev, dz_max_prev );
    sense_own_prev::Symbol = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
    if IsOutsideCriticalInterval(p. 297)( mode_int, coc_prev, dz_own_ave, dz_int_ave, s_c, idx_online_cost )
        s_c.force_alert_prev = false
    else
        dz_rel::R = dz_int_ave - dz_own_ave
        z_rel_proximate::R                  = z_rel + (dz_rel * T_proximate)
        z_rel_proximate_climbdescend::R     = z_rel + (dz_rel * T_proximate_climbdescend)
        z_rel_proximate_inc_climbdescend::R = z_rel + (dz_rel * T_proximate_inc_climbdescend)
        if s_c.force_alert_prev ||
           (abs( z_rel ) < H_rel_required) ||
           (abs( z_rel_proximate ) < H_rel_required) ||
           IsProjectedCrossing(p. 426)( z_rel, z_rel_proximate )
            s_c.force_alert = true
        end
        s_c.force_alert_prev = s_c.force_alert
        if s_c.force_alert && (R_corrective < abs( dz_int_ave )) &&
           ((abs( z_rel_proximate_climbdescend ) < H_rel_required) ||
            IsProjectedCrossing(p. 426)( z_rel, z_rel_proximate_climbdescend ))
            s_c.force_climbdescend = true
        end
        if s_c.force_alert && (R_strengthen < abs( dz_int_ave )) &&
           ((abs( z_rel_proximate_inc_climbdescend ) < H_rel_required) ||
            IsProjectedCrossing(p. 426)( z_rel, z_rel_proximate_inc_climbdescend )) &&
           ( ((dz_min_prev < R_strengthen) && (R_corrective <= dz_min_prev) &&
              (z_rel < -CROSSING_ALTITUDE_BUFFER)) ||
             ((-R_strengthen < dz_max_prev) && (dz_max_prev <= -R_corrective) &&
              (CROSSING_ALTITUDE_BUFFER < z_rel)) )
            s_c.force_inc_climbdescend = true
        end
    end
    if CriticalIntervalRequiresVerticalDivergence(p. 298)( mode_int, coc_prev, sense_own_prev, z_rel, s_c,
         idx_online_cost )
        s_c.force_alert_prev = true
        s_c.force_alert = true
        s_c.force_climbdescend = true
    end
end
```

Referenced In: CriticalIntervalProtectionCost(p. 294)

IsOutsideCriticalInterval (Algorithm 278) uses geometric and RA information to determine if the own aircraft is outside the critical interval. The own aircraft is outside the critical interval when any of the following conditions are true:

- The horizontal closure speed (magnitude) is above a threshold (*R_speed_ground_threshold*)
- The horizontal range is at or above a larger threshold (*D_range_ground_expanded*)
- The horizontal range is at or above a smaller threshold (*D_range_ground_required*) and the aircraft are diverging horizontally
- The intruder vertical rate is below a threshold (*R_int_vert_required*)
- The ownship vertical rate is below a threshold (*R_own_vert_required*)
- The own advisory is currently clear of conflict and the horizontal range is at or above the smaller threshold (*D_range_ground_required*)

If none of these conditions are true, the own aircraft is considered to be inside the critical interval.

This algorithm takes as input *mode_int*, *coc_prev*, *dz_own_ave*, *dz_int_ave*, and *s_c*. This algorithm returns *is_outside_ci*.

Algorithm 278 IsOutsideCriticalInterval

```
function IsOutsideCriticalInterval( mode_int::Z, coc_prev::Bool, dz_own_ave::R, dz_int_ave::R,
                                    s_c::CriticalIntervalProtectionCState(p. 529), idx_online_cost::Z )
    D_range_ground_required::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.D_range_ground_-
           required[idx_online_cost]
    D_range_ground_expanded::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.D_range_ground_-
           expanded[idx_online_cost]
    R_speed_ground_threshold::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.R_speed_ground_-
           threshold[idx_online_cost]
    R_int_vert_threshold::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.R_int_vert_-
           threshold[idx_online_cost]
    R_own_vert_threshold::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.R_own_vert_-
           threshold[idx_online_cost]
    is_outside_ci::Bool = false
    if (R_speed_ground_threshold <  s_c.speed) ||
       (D_range_ground_expanded  <= s_c.range) ||
       ((D_range_ground_required <= s_c.range) && (s_c.angle < pi/2)) ||
       (coc_prev && (D_range_ground_required <= s_c.range)) ||
       (abs( dz_int_ave ) < R_int_vert_threshold) ||
       (abs( dz_own_ave ) < R_own_vert_threshold)
        is_outside_ci = true
    end
    return is_outside_ci::Bool
end
```

Referenced In: UpdateCriticalIntervalProtectionCState(p. 296)

CriticalIntervalRequiresVerticalDivergence (Algorithm 279) uses geometric and RA information to determine if vertical divergence is required in the critical interval. The own aircraft is required to issue divergent alerts if all of the following conditions are true:

- The own aircraft has previously alerted or will be forced to alert on this cycle
- The horizontal range is below or equal to a threshold (*D_range_ground_required*)
- The horizontal closure speed is below a threshold (*R_speed_ground_force_diverge_threshold*)
- Either
  - The intruder is above the own aircraft and the previous RA sense is up
  - The intruder is below the own aircraft and the previous RA sense is down

If these conditions are met, the own aircraft is required to issue a divergent alert.

This algorithm takes as input *mode_int*, *coc_prev*, *sense_own_prev*, *z_rel*, and *s_c*. This algorithm returns *requires_diverge*.

Algorithm 279 CriticalIntervalRequiresVerticalDivergence

```
function CriticalIntervalRequiresVerticalDivergence( mode_int::Z, coc_prev::Bool, sense_own_prev::Symbol,
                                                     z_rel::R, s_c::CriticalIntervalProtectionCState(p. 529),
                                                     idx_online_cost::Z )
    D_range_ground_required::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.D_range_ground_-
           required[idx_online_cost]
    R_speed_ground_force_diverge_threshold::R =
      params().modes[mode_int].cost_estimation.online.critical_interval_protection.R_speed_ground_force_-
           diverge_threshold[idx_online_cost]
    requires_diverge::Bool = false
    if (!coc_prev || s_c.force_alert ) &&
       (s_c.range <= D_range_ground_required) &&
       (s_c.speed < R_speed_ground_force_diverge_threshold) &&
       (((0 < z_rel) && (sense_own_prev == :Up)) ||
        ((z_rel < 0) && (sense_own_prev == :Down)))
        requires_diverge = true
    end
    return requires_diverge::Bool
end
```

Referenced In: UpdateCriticalIntervalProtectionCState(p. 296)

3.3.2.4.6 Initialization Cost

INITIALIZATIONCOST (Algorithm 280) prohibits resolution advisories from being issued for the first *T_init* processing cycles. This cost allows the tracker estimates to stabilize. Initialized to *0*, the variable *s_c.t_count* counts up until reaching *T_init*, after which resolution advisories are no longer penalized.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 280 InitializationCost

```
function InitializationCost( mode_int::Z, dz_min::R, dz_max::R, update::Bool, s_c::InitializationCState(p. 529) )
    T_init::Z = params().modes[mode_int].cost_estimation.online.initialization.T_init
    if update
        UpdateInitializationCState(p. 261)(mode_int, s_c)
    end
    cost::R   = 0.0
    if (s_c.t_count < T_init) && !IsCOC(p. 419)( dz_min, dz_max )
        cost = Inf
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

3.3.2.4.7 Prevent Early COC Cost

PREVENTEARLYCOCCOST (Algorithm 281) penalizes issuance of a clear of conflict by *C_coc* when there is an active resolution advisory and the geometry of the encounter necessitates the continuation of the alert. If the early clear of conflict flag is TRUE (*s_c.is_early_coc*) and the proposed action is clear of conflict, the cost is applied.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATEPREVENTEARLYCOCCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *tau_expected*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 281 PreventEarlyCOCCost

```
function PreventEarlyCOCCost( mode_int::Z, dz_min::R, dz_max::R,
                    dz_min_prev::R, dz_max_prev::R, z_own_ave::R, dz_own_ave::R,
                    z_int_ave::R, dz_int_ave::R, tau_expected::R, update::Bool,
                    s_c::PreventEarlyCOCCState(p. 530), idx_online_cost::Z )
    C_coc::R = params().modes[mode_int].cost_estimation.online.prevent_early_coc.C_coc[idx_online_cost]
    if update
        UpdatePreventEarlyCOCCState(p. 301)( mode_int, dz_min_prev, dz_max_prev, z_own_ave, dz_own_ave,
                                    z_int_ave, dz_int_ave, tau_expected, s_c, idx_online_cost )
    end
    cost::R = 0.0
    if s_c.is_early_coc && IsCOC(p. 419)( dz_min, dz_max )
        cost = C_coc
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

UPDATEPREVENTEARLYCOCCSTATE (Algorithm 282) is used to update the state variables of PREVENTEARLYCOCCOST once per cycle. The ultimate purpose of this algorithm is to set the early clear of conflict flag (*s_c.is_early_coc*) based on geometric considerations. This flag is initialized to FALSE.

If the action selected on the previous cycle was clear of conflict, the state variables are reset. If the range on the previous time step is undefined or greater than the current range there is no horizontal divergence and the early clear of conflict flag is TRUE. Otherwise, the range divergence time count (*s_c.t_consec_range_divergence*) is incremented, and can be checked against a threshold (*T_min_horizontal_divergence*). If the aircraft have diverged in horizontal range for less than the threshold, the early clear of conflict flag is TRUE. Likewise, if the horizontal range is not greater than a threshold (*D_range_min*), the early clear of conflict flag is TRUE.

While sufficient for most cases, the horizontal checks can lead to continued RAs even if the vertical separation has become excessive so additional checks are required. If the horizontal checks determine the early clear of conflict flag should be TRUE, a series of vertical tests are used to determine if clear of conflict can be issued to prevent unnecessary long duration RAs. This creates three layers that allow clear of conflict to be issued due to vertical geometry considerations even if the standard horizontal checks are not passed. These vertical tests rely on relative vertical separation (*z_rel*) and relative vertical convergence/divergence (*vertical_tau*). The resulting effect is that either greater vertical separation or greater vertical divergence is required if the horizontal checks are not passed. Eventually if sufficient vertical separation is achieved and the aircraft are not converging vertically, RAs will be allowed to clear regardless of horizontal geometry.

This algorithm takes as input *mode_int*, *dz_min_prev*, *dz_max_prev*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *tau_expected*, and *s_c*. This algorithm updates *s_c*.

Algorithm 282 UpdatePreventEarlyCOCCState

```
function UpdatePreventEarlyCOCCState( mode_int::Z, dz_min_prev::R, dz_max_prev::R,
                                      z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R,
                                      tau_expected::R, s_c::PreventEarlyCOCCState(p. 530), idx_online_cost::Z )
    T_threshold::R =
        params().modes[mode_int].cost_estimation.online.prevent_early_coc.T_threshold[idx_online_cost]
    T_min_horizontal_divergence::Z =
        params().modes[mode_int].cost_estimation.online.prevent_early_coc.T_min_horizontal_divergence[idx_-
            online_cost]
    D_range_min::R =
        params().modes[mode_int].cost_estimation.online.prevent_early_coc.D_range_min[idx_online_cost]
    R_rel_vert_threshold::R =
        params().modes[mode_int].cost_estimation.online.prevent_early_coc.R_rel_vert_threshold[idx_online_-
            cost]
    H_rel_hi_threshold::R =
        params().modes[mode_int].cost_estimation.online.prevent_early_coc.H_rel_hi_threshold[idx_online_-
            cost]
    H_rel_mid_threshold::R =
        params().modes[mode_int].cost_estimation.online.prevent_early_coc.H_rel_mid_threshold[idx_online_-
            cost]
    H_rel_lo_threshold::R =
        params().modes[mode_int].cost_estimation.online.prevent_early_coc.H_rel_lo_threshold[idx_online_-
            cost]
    s_c.is_early_coc = false
    if IsCOC(p. 419)( dz_min_prev, dz_max_prev )
        s_c.range_prev = NaN
        s_c.t_consec_range_divergence = 0
    else
        if isnan(s_c.range_prev) || (s_c.range <= s_c.range_prev)
            s_c.t_consec_range_divergence = 0
            s_c.is_early_coc = true
        else
            s_c.t_consec_range_divergence = s_c.t_consec_range_divergence + 1
            if (s_c.t_consec_range_divergence <= T_min_horizontal_divergence)
                s_c.is_early_coc = true
            elseif (s_c.range <= D_range_min)
                s_c.is_early_coc = true
            end
        end
        s_c.range_prev = s_c.range
    end
    if (s_c.is_early_coc == true)
        z_rel::R  = z_int_ave - z_own_ave
        dz_rel::R = dz_int_ave - dz_own_ave
        vertical_tau::R = 0.0
        if (R_rel_vert_threshold < abs(dz_rel))
            vertical_tau = z_rel / dz_rel
        end
        is_horizontal_sufficient::Bool = (T_threshold <= tau_expected) ||
                                         (T_min_horizontal_divergence < s_c.t_consec_range_divergence)
        if ( (H_rel_hi_threshold  <= abs(z_rel)) && (0.0 <= vertical_tau) ) ||
           ( (H_rel_mid_threshold <= abs(z_rel)) && (0.0 <= vertical_tau) && is_horizontal_sufficient ) ||
           ( (H_rel_lo_threshold  <= abs(z_rel)) && (0.0 <  vertical_tau) && is_horizontal_sufficient &&
             (D_range_min < s_c.range) )
            s_c.is_early_coc = false
        end
    end
end
```

Referenced In: PreventEarlyCOCCost(p. 299)

3.3.2.4.8 Safe Crossing Resolution Advisory Deferral Cost

SAFECROSSINGRADEFERRALCOST (Algorithm 283) penalizes the issuance of any resolution advisory when an altitude crossing is expected to be safe until the crossing advisory can be issued. The cost applied to issuing advisories is stored as a state variable (*s_c.c_deferral*). If the current action is clear of conflict and the proposed action is not clear of conflict, the cost is applied.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATESAFECROSSINGRADEFERRALCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *equip_int*, *z_own_ave*, *z_int_ave*, *dz_own_ave*, *dz_int_ave*, *dz_min_prev*, *dz_max_prev*, *tau_expected_no_horizon*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 283 SafeCrossingRADeferralCost

```
1 function SafeCrossingRADeferralCost( mode_int::Z, dz_min::R, dz_max::R, equip_int::Bool, z_own_ave::R,
2                                      z_int_ave::R, dz_own_ave::R, dz_int_ave::R, dz_min_prev::R,
3                                      dz_max_prev::R, tau_expected_no_horizon::R, update::Bool,
4                                      s_c::SafeCrossingRADeferralCState(p.532), idx_online_cost::Z )
5     if update
6        UpdateSafeCrossingRADeferralCState(p.303)( mode_int, equip_int, z_own_ave, z_int_ave,
7                                                   dz_own_ave, dz_int_ave, tau_expected_no_horizon, s_c,
8                                                   idx_online_cost )
9     end
10    cost::R = 0.0
11    if !IsCOC(p.419)( dz_min, dz_max ) && IsCOC(p.419)(dz_min_prev, dz_max_prev)
12        cost = s_c.c_deferral
13    end
14    return (cost::R)
15 end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p.287)

UPDATESAFECROSSINGRADEFERRALCSTATE (Algorithm 284) is used to update the state variables of SAFECROSSINGRADEFERRALCOST once per cycle. The ultimate purpose of this algorithm is to set deferral cost state variable (*s_c.c_deferral*) based on geometric and equipage considerations.

If the intruder is equipped no deferral cost is applied.

A linear projection of the vertical states of own aircraft and intruder is performed to determine the vertical separation expected at the closest point of approach (*z_cpa_expected*). Additionally, thresholds for the current vertical separation (*s_deferral_threshold*) and the separation at the closest point of approach (*cpa_separation*) are calculated using the ownship vertical rate. As the own aircraft vertical rate increases, the separation thresholds increase. Using these calculated values the following conditions are checked:

- If the vertical separation indicates an altitude crossing will occur
- If the vertical separation at closest point of approach is projected to be larger than the *cpa_separation* threshold
- If the current vertical separation is currently larger than the *s_deferral_threshold* value
- If the vertical rate of the intruder aircraft is above the *R_int_vertical_threshold* value

If all of these conditions are true, the deferral cost state variable is set to *C_crossing_deferral* for use in SAFECROSSINGRADEFERRALCOST.

This algorithm takes as input *mode_int*, *equip_int*, *z_own_ave*, *z_int_ave*, *dz_own_ave*, *dz_int_ave*, *tau_expected_no_horizon*, and *s_c*. This algorithm updates *s_c*.

Algorithm 284 UpdateSafeCrossingRADeferralCState

```
function UpdateSafeCrossingRADeferralCState( mode_int::Z, equip_int::Bool, z_own_ave::R, z_int_ave::R,
                                             dz_own_ave::R, dz_int_ave::R, tau_expected_no_horizon::R,
                                             s_c::SafeCrossingRADeferralCState(p. 532), idx_online_cost::Z )
    C_crossing_deferral::R =
        params().modes[mode_int].cost_estimation.online.safe_crossing_ra_deferral.C_crossing_deferral[idx_online_cost]
    H_min_threshold::R =
        params().modes[mode_int].cost_estimation.online.safe_crossing_ra_deferral.H_min_threshold[idx_online_cost]
    H_cpa_separation_threshold::R =
        params().modes[mode_int].cost_estimation.online.safe_crossing_ra_deferral.H_cpa_separation_threshold[idx_online_cost]
    H_own_dz_vertical_threshold::R =
        params().modes[mode_int].cost_estimation.online.safe_crossing_ra_deferral.H_own_dz_vertical_threshold[idx_online_cost]
    H_own_dz_cpa_vertical_threshold::R =
        params().modes[mode_int].cost_estimation.online.safe_crossing_ra_deferral.H_own_dz_cpa_vertical_threshold[idx_online_cost]
    R_int_vertical_threshold::R =
        params().modes[mode_int].cost_estimation.online.safe_crossing_ra_deferral.R_int_vertical_threshold[idx_online_cost]
    R_own_threshold::R =
        params().modes[mode_int].cost_estimation.online.safe_crossing_ra_deferral.R_own_threshold[idx_online_cost]
    R_own_rolloff::R =
        params().modes[mode_int].cost_estimation.online.safe_crossing_ra_deferral.R_own_rolloff[idx_online_cost]
    if equip_int
       s_c.c_deferral = 0.0
    else
        z_rel::R                   = z_int_ave - z_own_ave
        dz_rel::R                  = dz_int_ave - dz_own_ave
        z_cpa_expected::R          = z_rel + (dz_rel * tau_expected_no_horizon)
        diff_sign_rates::Bool      = ((dz_own_ave * dz_int_ave) < 0)
        s_deferral_threshold::R    = H_min_threshold
        cpa_separation::R          = H_cpa_separation_threshold
        if diff_sign_rates
            dz_own_scale_factor::R = CalculateThresholdRampUpFactor(p. 411)( abs( dz_own_ave ), R_own_threshold, R_own_rolloff )
            s_deferral_threshold = s_deferral_threshold + (H_own_dz_vertical_threshold * dz_own_scale_factor )
            cpa_separation       = cpa_separation + (H_own_dz_cpa_vertical_threshold * dz_own_scale_factor)
        end
        if IsProjectedCrossing(p. 426)( z_rel, z_cpa_expected ) && (s_deferral_threshold < abs( z_rel )) &&
                (cpa_separation < abs( z_cpa_expected )) && (R_int_vertical_threshold < abs( dz_int_ave ))
            s_c.c_deferral = C_crossing_deferral
        else
            s_c.c_deferral = 0.0
        end
    end
end
```

Referenced In: SafeCrossingRADeferralCost(p. 302)

3.3.2.4.9 Vertical Rate Limit Dependent Strengthen Cost

VERTICALRATELIMITDEPENDENTSTRENGTHENCOST (Algorithm 285) penalizes increase climb and descend advisories by *C_strengthen_inhibit* if the ownship's vertical rate limits do not support the vertical rate of the increase climb or descend advisories. This online cost is intended to suppress these advisories unless the ownship has the vertical rate capability to achieve the actions.

The algorithm checks if the current action rates is a strengthening relative to the previous action rates. It also checks that the current action rates correspond to the assumed strengthen rate for the increase climb and descend advisories. If both are true, and the ownship vertical rate limit is lower than the strengthen rate, then the online cost is applied.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, and *vert_rate_limit*. This algorithm returns *cost*.

Algorithm 285 VerticalRateLimitDependentStrengthenCost

```
function VerticalRateLimitDependentStrengthenCost( mode_int::Z, dz_min::R, dz_max::R,
                                        dz_min_prev::R, dz_max_prev::R, vert_rate_limit::R )
    C_strengthen_inhibit::R =
        params().modes[mode_int].cost_estimation.online.vertical_rate_limit_dependent_strengthen.C_-
            strengthen_inhibit
    R_strengthen::R = params().actions.strengthen_rate
    cost::R = 0.0
    ra_is_strengthening::Bool = IsStrengthening(p. 428)( dz_min_prev, dz_max_prev, dz_min, dz_max )
    ra_is_strengthen_rate::Bool = ( ( dz_min == R_strengthen ) && ( dz_max == Inf ) ) ||
                                  ( ( dz_min == -Inf) && ( dz_max == -R_strengthen ) )
    if ( vert_rate_limit < R_strengthen ) &&
       ( ra_is_strengthening ) && ( ra_is_strengthen_rate )
       cost = cost + C_strengthen_inhibit
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

3.3.2.4.10 Point Obstacle Descend Cost

POINTOBSDESCENDCOST (Algorithm 286) penalizes down sense advisories by *C_descend*.

The algorithm only applies to the point obstacle intruders to encourage up sense advisories against them.

This algorithm takes as input the protection mode index (*mode_int*), an action's minmum vertical velocity (*dz_min*), an action's maximum vertical velocity (*dz_max*). This algorithm returns the updated action cost, *cost*.

Algorithm 286 PointObsDescendCost

```
function PointObsDescendCost(  mode_int::Z, dz_min::R, dz_max::R )
    C_descend::R =
        params().modes[mode_int].cost_estimation.online.point_obs_descend_cost.C_descend
    cost::R = 0.0
    if ( RatesToSense(p. 432)(dz_min,dz_max) == :Down)
        cost = cost + C_descend
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

3.3.2.4.11 Point Obstacle Early COC Cost

POINTOBSPREVENTEARLYCOCCOST (Algorithm 287) penalizes issuance of a clear of conflict by *C_point_obs_early_coc* when there is an active resolution advisory and the geometry of the encounter necessitates the continuation of the alert.

Unlike its airborne intruder counterpart PREVENTEARLYCOCCOST (Algorithm 281), no state variables are set. This is due to the dynamic prioritization of point obstacles by the STM which does not assure that the same point obstacle will be passed each cycle.

First POINTOBSPREVENTEARLYCOCCOST, determines if the expected time until collision is less than the threshold *T_point_obs_early_coc_tau_threshold* and stores theresulting Boolean into *tau_check*. If the previous action was a clear of conflict, the relative vertical distance, velocity, projected relative distance in *T_point_obs_early_coc_vertical_tau_threshold* seconds, and a vertical tau estimate is determined. *Seperation_check* is then set to false if the vertical tau estimate is greater than zero and the relative distance is greater than *H_point_obs_early_coc_altitude_threshold* or if the projected relative distance is greater than *H_point_obs_early_coc_altitude_threshold*.

If both *tau_check* and *seperation_check* are true, *C_point_obs_early_coc* is returned. Otherwise zero is returned.

This algorithm takes as input *mode_int*, *dz_min_prev*, *dz_max_prev*, *z_own_ave*, *z_int_ave*, *dz_own_ave*, *dz_int_ave*, and *tau_expected*. This algorithm returns *cost*.

Algorithm 287 PointObsPreventEarlyCOCCost

```
function PointObsPreventEarlyCOCCost( mode_int::Z, dz_min_prev::R, dz_max_prev::R, z_own_ave::R, z_int_ave::R,
                                      dz_own_ave::R, dz_int_ave::R, tau_expected::R )
    C_point_obs_early_coc::R =
        params().modes[mode_int].cost_estimation.online.point_obs_early_coc_cost.C_point_obs_early_coc
    H_point_obs_early_coc_altitude_threshold::R =
        params().modes[mode_int].cost_estimation.online.point_obs_early_coc_cost.H_point_obs_early_coc_-
            altitude_threshold
    T_point_obs_early_coc_tau_threshold::R =
        params().modes[mode_int].cost_estimation.online.point_obs_early_coc_cost.T_point_obs_early_coc_tau_-
            threshold
    T_point_obs_early_coc_vertical_tau_threshold::R =
        params().modes[mode_int].cost_estimation.online.point_obs_early_coc_cost.T_point_obs_early_coc_-
            vertical_tau_threshold
    cost::R = 0.0
    separation_check::Bool = true
    tau_check::Bool = tau_expected < T_point_obs_early_coc_tau_threshold
    if ( IsCOC(p. 419)( dz_min_prev, dz_max_prev ) )
        z_rel::R  = z_int_ave  - z_own_ave
        dz_rel::R = dz_int_ave - dz_own_ave
        z_rel_projection = z_rel + T_point_obs_early_coc_vertical_tau_threshold * dz_rel
        if ( abs(dz_rel) > 0.1 )
            t_vertical_tau_estimate::R = z_rel/dz_rel
        else
            t_vertical_tau_estimate = 80
        end
        if ( ( t_vertical_tau_estimate > 0 ) && ( abs(z_rel) > H_point_obs_early_coc_altitude_threshold ) )
            separation_check = false
        elseif ( abs(z_rel_projection) > H_point_obs_early_coc_altitude_threshold )
            separation_check = false
        end
        if ( tau_check && separation_check )
            cost = cost + C_point_obs_early_coc
        end
    end
    return cost::R
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287)

3.3.2.5 Update Intruder Inputs Mid-Cycle

UPDATEINTRUDERINPUTS (Algorithm 288) is used to retrieve updated information from the STM in the middle of a processing cycle. It retrieves the most recent VRC values from the STM only for those intruders already being processed by the TRM. This retrieval happens at the latest time that VRC inputs can be ingested into the TRM for processing. Subsequent threat processing requires information from received resolution advisory coordination messages.

The call back to the STM, UPDATEINTRUDERVRC, updates the *vrc* field for each intruder represented in the *input_int* vector. UPDATEINTRUDERVRC also returns the *received_vrcs* vector.

Once the TRM receives updated VRC information from the STM, UPDATEINTRUDERINPUTS loops over all intruders to set variables for later use.

ISINTRUDEREQUIPPED and ISINTRUDERMASTER are called to potentially update *equip_int* and *master_int* in the case where an intruder with *EQUIPAGE_LARGE_CAS* sends a coordination message in the middle of a processing cycle and is therefore a ACAS Xr intruder. This intruder will be updated to be equipped and master relative to ownship.

In the loop, the algorithm first updates the *vrc_int* value when both the intruder and ownship are capable of producing resolution advisories and coordinating with each other, as indicated by *equip_int* having a value of TRUE. Otherwise, the value of *vrc_int* is always *0*.

Next, it sets *vrcs_conflict*. If the intruder aircraft has sent a VRC, then the two aircraft are coordinating their advisories. In that case, the VRC from the intruder (*intruder_input.vrc*) is examined to determine if it conflicts with VRCs received from other intruders. The *vrcs_conflict* value is set to indicate the result. Note that *received_vrcs* is used for RAC output only and is not used to determine whether intruder VRCs conflict. That is because *received_vrcs* may reflect VRCs received from intruders that became known to the STM after the start of the TRM cycle; such intruders aren't reflected in the TRM input. *vrcs_conflict* is used in ONLINECOSTESTIMATION for COMPATIBILITYCOST.

This algorithm takes as input *input_int*, *equip_int*, *master_int*, *vrc_int*, and *input_own*. This algorithm returns *received_vrcs* and *vrcs_conflict*, and modifies *equip_int*, *master_int*, and *vrc_int*.

Algorithm 288 UpdateIntruderInputs

```
function UpdateIntruderInputs( input_int::Vector{TRMIntruderInput(p. 499)}, equip_int::Vector{Bool}, master_int::
     Vector{Bool}, vrc_int::Vector{UInt32}, input_own::TRMOwnInput(p. 498) )
    received_vrcs::Vector{Bool} = UpdateIntruderVRC(p. 101)( input_int )
    vrcs_conflict::Bool = false
    sense::Symbol       = :None
    for j in 1:length(input_int)
        equip_int[j] = IsIntruderEquipped(p. 423)(input_own.opmode, input_int[j].equipage)
        master_int[j] = IsIntruderMaster(p. 424)(input_own, input_int[j])
        if equip_int[j]
            vrc_int[j]        = input_int[j].vrc
            sense_vrc::Symbol = VRCToSense(p. 436)( input_int[j].vrc )
            if (:None != sense_vrc)
                if (:None == sense)
                    sense = sense_vrc
                elseif (sense_vrc != sense)
                    vrcs_conflict = true
                end
            end
        else
            vrc_int[j] = EncodeVRC(p. 364)( :None, EQUIPAGE_NONE )
        end
    end
    return (received_vrcs::Vector{Bool}, vrcs_conflict::Bool)
end
```

Referenced In: StateAndCostEstimation(p. 251)

3.3.2.6 Online Vertical Cost Estimation Dependent on Coordination

An overview of the ONLINECOSTESTIMATION component in STATEANDCOSTESTIMATION is shown in Figure 11.

ONLINECOSTESTIMATION (Algorithm 289) determines the values of the online costs that are dependent on coordination with intruding aircraft. Online costs capture characteristics of the current state, such as ACAS X coordination, that cannot be suitably reflected in the offline cost tables. The online costs determined in this algorithm are dependent on the resolution advisory

coordination state of own aircraft and the intruder aircraft.

ONLINECOSTESTIMATION always performs its evaluation with regard to a single intruder, independent of any other intruders.

It first computes values needed by the various online cost algorithms.

GETMODIFIEDGLOBALRATES (Algorithm 267) determines the vertical rate limits for the previous cycle (*dz_min_prev* and *dz_max_prev*) to be used for computing online costs.

OWNRESPONSEESTIMATION (Algorithm 292) estimates the responsiveness of the own aircraft to its own resolution advisories (*resp_own*).

INTRUDERRESPONSEESTIMATION (Algorithm 291) estimates the responsiveness of the intruder to its own advisories, as communicated to own aircraft in resolution advisory coordination messages (*resp_int*). If the intruder is not equipped to send coordination messages, it is treated as being responsive.

EXPECTEDTAU (Algorithm 378) determines the weighted average value for tau to be used for computing online costs. It is called twice with a slight difference in input arguments. The first time it is called to compute the weighted average value for tau including the horizon (*tau_expected*). This variation is used to determine the applicability of a given functionality in an online cost. The second time it is called to compute the weighted average value for tau excluding the horizon (*tau_expected_no_horizon*). This variation is used when a projection to loss of horizontal separation is required.

See section 3.3.2.2.1 for a discussion of tau.

The cost is initialized to 0.0 for each action in the resolution advisory cost vectors (*cost_ra*, *cost_ra_subset*, *cost_ra_coord*, and *cost_ra_coord_subset*). These costs are used when determining whether a resolution advisory should be issued, and if so, the type of resolution advisory to issue.

The *update* variable is initialized to TRUE. The *update* variable is used to flag the first iteration so each online cost algorithm will know to update the state variables in *s_c*. That set of updates will occur only once for each intruder on each cycle.

ONLINECOSTESTIMATION then loops over all the possible actions to compute the online costs for each action. Each action (*act*) is represented as a minimum and maximum vertical rate (*dz_min* and *dz_max*).

ACTIONTORATES (Algorithm 373) is used to obtain the minimum and maximum vertical rates for the action under evaluation. ACTIONTORATES is a frequently used helper algorithm that is intentionally omitted from the flow diagrams.

For each action, ONLINECOORDINATEDACTIONCOSTESTIMATION (Algorithm 290) computes a sum of the online costs that are dependent on resolution advisory coordination between own aircraft and the intruding aircraft for a specific action (*cost_ra_coord[act]* and *cost_ra_coord_subset[act]*). *cost_ra_coord[act]* and *cost_ra_coord_subset[act]* are each then summed with *cost_ra_uncoord[act]*, computed by ONLINEUNCOORDINATEDACTIONCOSTESTIMATION, to fill the *cost_ra* and *cost_ra_subset* vectors. The *cost_ra* and *cost_ra_subset* vectors are inputs to INDIVIDUALCOSTESTIMATION.

The algorithm requires knowledge of numerous variables to calculate the various online costs. The required inputs include the previous global resolution advisory issued by the system (*a_-*

global_prev), the previous resolution advisory for the intruder considered in isolation (*a_indiv_-prev*), and the intruder inputs from the STM (*input_int*). The weighted average of the own aircraft and intruder's vertical states (*z_own_ave*, *dz_own_ave*, *z_int_ave*, and *dz_int_ave*) are also needed along with the samples and corresponding weights for tau (*b_tau_int*) and whether the intruder is able to send resolution advisory coordination messages (*equip_int*). Input variables related to exchange of resolution advisory coordination messages are whether the intruder is master (*master_int*), the VRC received from the intruder (*vrc_int*), and whether conficting VRCs were received from different intruders (*vrcs_conflict*).

ONLINECOSTESTIMATION takes as input *mode_int*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_-ave*, *b_tau_int*, *st_own*, *st_int*, *master_int*, *vrc_int*, *hrc_int*, *equip_int*, *vrcs_conflict*, and *cost_ra_-uncoord*. Descriptions of the ONLINECOSTESTIMATION output variables are found in Table 41.

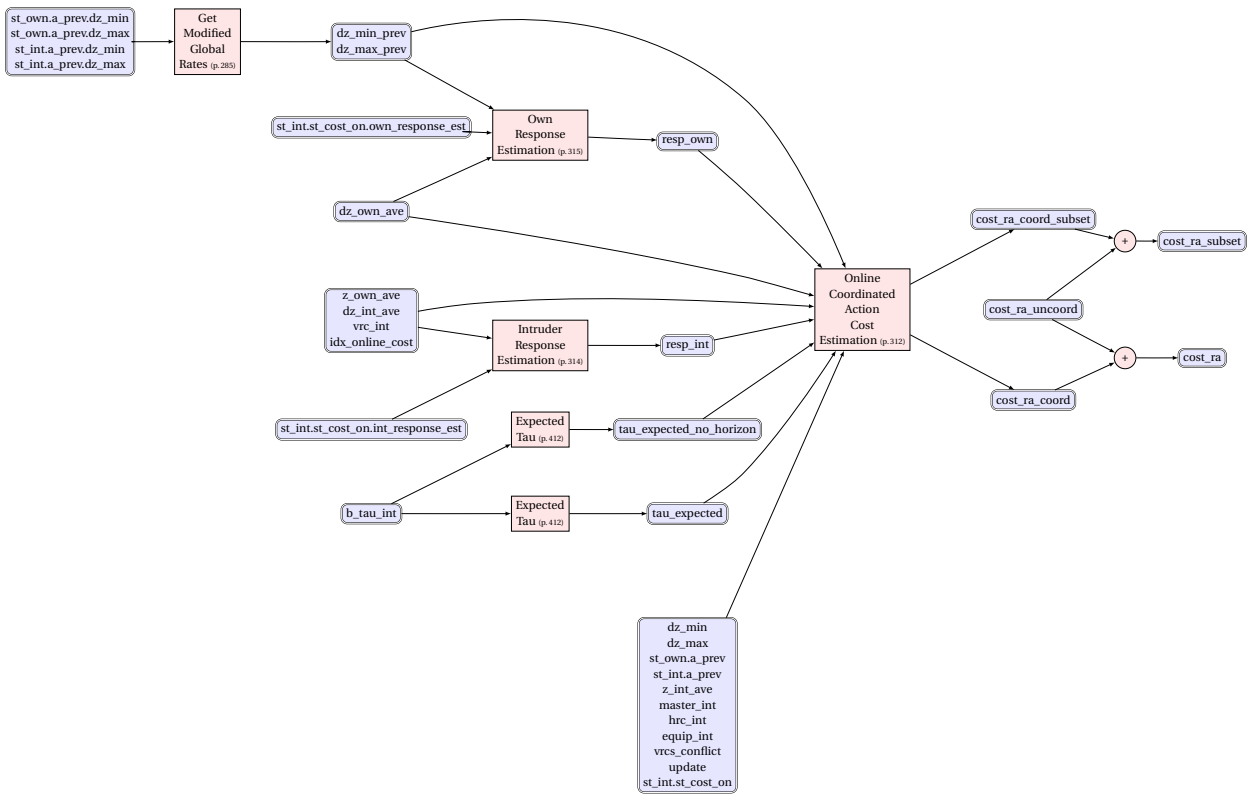

Figure 11. OnlineCostEstimation overview.

Table 41. Vertical TRM Algorithm Output Variables - OnlineCostEstimation

Variable	Units	Type	Description
cost_ra	N/A	real	Vector of combined offline and online cost of each action for single intruder; used for RAs
cost_ra_subset	N/A	real	Vector of combined offline cost and online cost subset of each action for single intruder; used for RAs
st_int.st_cost_on	N/A	OnlineCostState(p. 520)	State variables for estimating online costs

Algorithm 289 OnlineCostEstimation

```
function OnlineCostEstimation( mode_int::Z,
                      z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R,
                      b_tau_int::Vector{TauBelief(p. 522)},
                      st_own::TRMOwnState(p. 511), st_int::TRMIntruderState(p. 512),
                      master_int::Bool, vrc_int::UInt32, hrc_int::UInt32,
                      equip_int::Bool, vrcs_conflict::Bool, cost_ra_uncoord::Vector{R},
                      idx_online_cost::Z )
    N_actions::Z = params().actions.num_actions
    (dz_min_prev::R, dz_max_prev::R)  =
        GetModifiedGlobalRates(p. 285)( st_own.a_prev.dz_min, st_own.a_prev.dz_max,
                                st_int.a_prev.dz_min, st_int.a_prev.dz_max )
    resp_own::Bool = OwnResponseEstimation(p. 315)( mode_int, dz_min_prev, dz_max_prev, dz_own_ave,
                                            st_int.st_cost_on.own_response_est )
    resp_int::Bool = IntruderResponseEstimation(p. 314)( mode_int, z_own_ave, dz_int_ave, vrc_int,
                                                  st_int.st_cost_on.int_response_est,
                                                  idx_online_cost )
    tau_expected::R = ExpectedTau(p. 412)( b_tau_int, false )
    tau_expected_no_horizon::R = ExpectedTau(p. 412)( b_tau_int, true )
    cost_ra::Vector{R}              = zeros( R, N_actions )
    cost_ra_subset::Vector{R}       = zeros( R, N_actions )
    cost_ra_coord::Vector{R}        = zeros( R, N_actions )
    cost_ra_coord_subset::Vector{R} = zeros( R, N_actions )
    update::Bool                    = true
    for act::Z in 1:N_actions
        (dz_min::R, dz_max::R) =
            ActionToRates(p. 410)( act, dz_own_ave, st_own.a_prev.action, st_own.a_prev.dz_min,
                           st_own.a_prev.dz_max, st_own.a_prev.ddz )
        (cost_ra_coord[act], cost_ra_coord_subset[act]) =
            OnlineCoordinatedActionCostEstimation(p. 312)( mode_int, dz_min, dz_max,
                                st_own.a_prev, st_int.a_prev, dz_min_prev, dz_max_prev,
                                z_own_ave, dz_own_ave, z_int_ave, dz_int_ave,
                                resp_own, resp_int, tau_expected, tau_expected_no_horizon,
                                master_int, vrc_int, hrc_int, equip_int, vrcs_conflict,
                                update, st_int.st_cost_on, idx_online_cost )
        update = false
        cost_ra[act]        = cost_ra_uncoord[act] + cost_ra_coord[act]
        cost_ra_subset[act] = cost_ra_uncoord[act] + cost_ra_coord_subset[act]
    end
    return (cost_ra::Vector{R}, cost_ra_subset::Vector{R})
end
```

Referenced In: StateAndCostEstimation(p. 251)

ONLINECOORDINATEDACTIONCOSTESTIMATION (Algorithm 290) returns two online cost sums for the given action and the individual intruder. The online costs determined in this algorithm

are dependent on the coordination state of own aircraft and the intruding aircraft. The associated algorithm, ONLINEUNCOORDINATEDACTIONCOSTESTIMATION, determines online costs that are independent of the resolution advisory coordination state of own aircraft and the intruding aircraft.

The two cost sums, *cost_ra* and *cost_ra_subset*, are used when determining whether or not to issue a resolution advisory. The value *cost_ra* contains the sum of all the individual coordination-dependent online costs for the given action. The value *cost_ra_subset* contains the sum of the subset of individual coordination-dependent online costs for the given action. An online cost is excluded from *cost_ra_subset* when that cost is designed to force a specific resolution advisory. The costs excluded from the subset are the SA01HEURISTIC (Algorithm 307), the COMPATIBILITYCOST (Algorithm 294), and the CROSSINGNOALERTCOST (Algorithm 300). Each of these costs will force specific advisories to be given and are not included in the *cost_ra_subset*.

Each online cost algorithm called in ONLINECOORDINATEDACTIONCOSTESTIMATION corresponds to a different event to be penalized, such as reversing the previous resolution advisory. After initialization, the algorithms update their own state variables only once during each call to ONLINECOORDINATEDACTIONCOSTESTIMATION, even though ONLINECOORDINATEDACTIONCOSTESTIMATION, and consequently each online cost algorithm, is called once for each action. The *update* value is used to indicate when the individual online cost algorithms should update their state variables, contained in the ONLINECOSTSTATE data structure. The value is set to TRUE when computing the online cost for the first action and FALSE subsequently.

In the individual online cost algorithms, an equipped intruder is an intruding aircraft that is able to send resolution advisory coordination messages. The equipped intruder is indicated by a value of TRUE for *equip_int*.

Common inputs to the individual online cost algorithms include the index to be used for acquiring the appropriate values of the configuration parameters (*mode_int*), whether the intruder aircraft is a master or not (*master_int*), the minimum vertical rate associated with the action (dz_min), the maximum vertical rate associated with the action (*dz_max*), *update*, and the relevant set of online cost state variables (*s_c*). *mode_int* is the protection mode index. ISINTRUDERMASTER (Algorithm 397) is used in INTRUDERPREP to set *master_int*.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *a_global_prev*, *a_indiv_prev*, *dz_min_prev*,*dz_max_prev*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *resp_own*, *resp_int*, *tau_expected*, *tau_expected_no_horizon*, *master_int*, *vrc_int*, *hrc_int*, *equip_int*, *vrcs_conflict*, *update* and *s_c*. This algorithm returns *cost_ra* and *cost_ra_subset*.

Algorithm 290 OnlineCoordinatedActionCostEstimation

```
function OnlineCoordinatedActionCostEstimation( mode_int::Z, dz_min::R, dz_max::R,
                            a_global_prev::GlobalAdvisory(p.517), a_indiv_prev::IndividualAdvisory(p.519),
                            dz_min_prev::R, dz_max_prev::R,
                            z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R,
                            resp_own::Bool, resp_int::Bool,
                            tau_expected::R, tau_expected_no_horizon::R,
                            master_int::Bool, vrc_int::UInt32, hrc_int::UInt32,
                            equip_int::Bool, vrcs_conflict::Bool, update::Bool,
                            s_c::OnlineCostState(p.520), idx_online_cost::Z )
    inc_cost::R = 0.0
    cost_ra::R  = 0.0
    inc_cost = CoordinationDelayCost(p.323)( mode_int, dz_min, dz_max, equip_int, vrc_int,
                                     hrc_int, update, s_c.coord_delay, idx_online_cost )
    cost_ra = cost_ra + inc_cost
    inc_cost =  RestrictCOCDueToReversal(p.329)( mode_int, dz_min, dz_max, vrc_int, master_int,
                                         dz_min_prev, dz_max_prev, update, s_c.restrict_coc,
                                         idx_online_cost )
    cost_ra = cost_ra + inc_cost
    inc_cost =  MaxReversalCost(p.326)( mode_int, dz_min, dz_max, vrc_int, master_int,
                                a_indiv_prev.dz_min, a_indiv_prev.dz_max, equip_int,
                                a_global_prev.multithreat, update, s_c.max_reversal,
                                idx_online_cost )
    cost_ra = cost_ra + inc_cost
    inc_cost = PreventEarlyWeakeningCost(p.328)( mode_int, dz_min, dz_max, dz_min_prev, dz_max_prev, z_own_ave,
                                         dz_own_ave, z_int_ave, dz_int_ave, vrc_int, tau_expected,
                                         idx_online_cost )
    cost_ra = cost_ra + inc_cost
    inc_cost = CoordinatedRADeferralCost(p.321)( mode_int, dz_min, dz_max, dz_min_prev, dz_max_prev,
                                         z_own_ave, dz_own_ave, z_int_ave, dz_int_ave,
                                         equip_int, vrc_int,
                                         update, s_c.coord_ra_deferral, idx_online_cost )
    cost_ra = cost_ra + inc_cost
    inc_cost = TimeBasedNonComplianceCost(p.333)( mode_int, dz_min, dz_max, dz_min_prev, dz_max_prev,
                                          z_own_ave, dz_own_ave, z_int_ave, dz_int_ave,
                                          tau_expected_no_horizon, equip_int, resp_int,
                                          update, s_c.time_based_non_compliance, idx_online_cost )
    cost_ra = cost_ra + inc_cost
    cost_ra_subset::R = cost_ra
    inc_cost =  SA01Heuristic(p.330)( mode_int, dz_min, dz_max, vrc_int, equip_int, resp_own, master_int,
                              (z_int_ave - z_own_ave), dz_own_ave, dz_int_ave, dz_min_prev, dz_max_prev,
                              tau_expected_no_horizon, update, s_c.sa01_heuristic, idx_online_cost )
    cost_ra = cost_ra + inc_cost
    inc_cost =  CompatibilityCost(p.319)( mode_int, dz_min, dz_max, vrc_int, resp_own, master_int,
                                  z_own_ave, z_int_ave, a_indiv_prev.dz_min, a_indiv_prev.dz_max,
                                  a_global_prev.multithreat, resp_int, vrcs_conflict, update,
                                  s_c.compatibility_cost, idx_online_cost )
    cost_ra = cost_ra + inc_cost
    inc_cost = CrossingNoAlertCost(p.324)( mode_int, dz_min, dz_max, z_own_ave, z_int_ave, vrc_int,
                                   tau_expected, update, s_c.crossing_no_alert, idx_online_cost )
    cost_ra = cost_ra + inc_cost
    return (cost_ra::R, cost_ra_subset::R)
end
```

Referenced In: OnlineCostEstimation(p.310)

3.3.2.6.1 Response Estimation

Determination of responsiveness to active resolution advisories is implemented by separate algorithms for the intruder aircraft and for own aircraft. Both algorithms output a value (*is_responding*) indicating whether it appears the aircraft is responding to the active resolution advisory or not. The two algorithms differ only in the inputs to DETERMINERESPONSE. After setting

the state variables in the response estimation cost state data structure (*s_c*), each algorithm calls DETERMINERESPONSE to get the value of *is_responding*.

INTRUDERRESPONSEESTIMATION (Algorithm 291) is used to estimate the responsiveness of the intruder aircraft to resolution advisories. The received VRC (*vrc_int*) is used to establish the sense of the active resolution advisory. The sense is set to *:None* for an unequipped intruder. If the own aircraft is below a given altitude threshold (*H_own_wait_int_threshold*), the time delay in determining if the intruder is responding to advisories (*t_wait_int*) is shortened. At lower altitudes the own aircraft advisories can be issued much later due to altitude sensitivity and the determination of intruder response must be performed in a more timely manner. The vertical rate sensitivity for the intruder aircraft is given by *R_buffer_int*.

This algorithm takes as input *mode_int*, *z_own_ave*, *dz_int_ave*, *vrc_int*, and *s_c*. This algorithm returns *is_responding*, and updates *s_c*.

Algorithm 291 IntruderResponseEstimation

```
function IntruderResponseEstimation( mode_int::Z, z_own_ave::R, dz_int_ave::R, vrc_int::UInt32,
                                     s_c::ResponseEstimationCState(p. 531), idx_online_cost::Z )
    R_buffer_int::R =
        params().modes[mode_int].cost_estimation.online.response_estimation.R_buffer_int[idx_online_cost]
    H_own_wait_int_threshold::R =
        params().modes[mode_int].cost_estimation.online.response_estimation.H_own_wait_int_threshold[idx_-
            online_cost]
    t_wait_int::Z  = 0
    if (z_own_ave <= H_own_wait_int_threshold)
        t_wait_int = params().modes[mode_int].cost_estimation.online.response_estimation.T_wait_int_low_-
            alt[idx_online_cost]
    else
        t_wait_int = params().modes[mode_int].cost_estimation.online.response_estimation.T_wait_int[idx_-
            online_cost]
    end
    sense_int::Symbol = VRCToSense(p. 436)( vrc_int )
    if (sense_int == :None)
        s_c.t_same_sense        = 0
        s_c.sense_prev          = :None
        s_c.dz_after_wait       = NaN
        s_c.is_responding_prev  = true
        s_c.dz_response_initial = dz_int_ave
        s_c.dz_response_max     = dz_int_ave
        s_c.t_dz_response_max   = 0
    elseif (s_c.sense_prev == sense_int)
        s_c.t_same_sense = s_c.t_same_sense + 1
        if (abs( s_c.dz_response_max ) < abs( dz_int_ave ))
            s_c.dz_response_max   = dz_int_ave
            s_c.t_dz_response_max = s_c.t_same_sense
        end
    else
        s_c.t_same_sense        = 0
        s_c.dz_response_initial = dz_int_ave
        s_c.dz_response_max     = dz_int_ave
        s_c.t_dz_response_max   = 0
    end
    if (s_c.t_same_sense < t_wait_int)
        s_c.dz_after_wait = NaN
    elseif (s_c.t_same_sense == t_wait_int)
        s_c.dz_after_wait = dz_int_ave
    end
    is_responding::Bool = DetermineResponse(p. 316)( mode_int, sense_int, dz_int_ave,
                                                     R_buffer_int, t_wait_int, s_c )
    s_c.is_responding_prev = is_responding
    s_c.sense_prev         = sense_int
    return is_responding::Bool
end
```

Referenced In: OnlineCostEstimation(p. 310)

OwnResponseEstimation (Algorithm 292) is used to estimate the responsiveness of own aircraft to resolution advisories. The previous global vertical rate limits, as provided by GetModifiedGlobalRates, are used to establish the sense of the active resolution advisory. The own aircraft maximum response time and vertical rate sensitivity are given by *T_wait_own* and *R_buffer_own*.

This algorithm takes as input *mode_int*, *dz_min_prev*, *dz_max_prev*, *dz_own_ave*, and *s_c*. This algorithm returns *is_responding*. This algorithm updates *s_c*.

Algorithm 292 OwnResponseEstimation

```
function OwnResponseEstimation( mode_int::Z, dz_min_prev::R, dz_max_prev::R, dz_own_ave::R,
                        s_c::ResponseEstimationCState(p. 531) )
    T_wait_own::Z   = params().modes[mode_int].cost_estimation.online.response_estimation.T_wait_own
    R_buffer_own::R = params().modes[mode_int].cost_estimation.online.response_estimation.R_buffer_own
    sense_own::Symbol = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
    if (sense_own == :None)
        s_c.t_same_sense        = 0
        s_c.sense_prev          = :None
        s_c.dz_after_wait       = NaN
        s_c.is_responding_prev  = true
        s_c.dz_response_initial = dz_own_ave
        s_c.dz_response_max     = dz_own_ave
        s_c.t_dz_response_max   = 0
    elseif (s_c.sense_prev == sense_own)
        s_c.t_same_sense = s_c.t_same_sense + 1
        if (abs( s_c.dz_response_max ) < abs( dz_own_ave ))
            s_c.dz_response_max   = dz_own_ave
            s_c.t_dz_response_max = s_c.t_same_sense
        end
    else
        s_c.t_same_sense        = 0
        s_c.dz_response_initial = dz_own_ave
        s_c.dz_response_max     = dz_own_ave
        s_c.t_dz_response_max   = 0
    end
    if (s_c.t_same_sense < T_wait_own)
        s_c.dz_after_wait = NaN
    elseif (s_c.t_same_sense == T_wait_own)
        s_c.dz_after_wait = dz_own_ave
    end
    is_responding::Bool = DetermineResponse(p. 316)( mode_int, sense_own, dz_own_ave,
                                                R_buffer_own, T_wait_own, s_c )
    s_c.is_responding_prev = is_responding
    s_c.sense_prev         = sense_own
    return is_responding::Bool
end
```

Referenced In: OnlineCostEstimation(p. 310)

DETERMINERESPONSE (Algorithm 293) determines whether the aircraft appears to be responding to its active resolution advisory based on the sense of the active resolution advisory, the vertical rate of the aircraft, the vertical rate sensitivity, the maximum response time, and the response estimation state. It always outputs the aircraft is responding when there is no active resolution advisory, as indicated by a sense of *:None*. When the active resolution advisory has an up sense or down sense, responsiveness is determined by estimating the vertical acceleration. The aircraft is considered to be non-responsive if the acceleration is not compliant with the active resolution advisory or the aircraft appears to be leveling off in a direction opposite to the sense of the active resolution advisory.

This algorithm takes as input *mode_int*, *sense*, *dz_curr*, *dz_buffer*, *t_wait*, and *s_c*. This algorithm returns *is_responding*.

Algorithm 293 DetermineResponse

```
function DetermineResponse( mode_int::Z, sense::Symbol, dz_curr::R,
                  dz_buffer::R, t_wait::Z, s_c::ResponseEstimationCState(p. 531) )
    R_acceleration::R =
          params().modes[mode_int].cost_estimation.online.response_estimation.R_acceleration
    R_threshold_acceleration_lo::R =
          params().modes[mode_int].cost_estimation.online.response_estimation.R_threshold_acceleration_lo
    R_threshold_acceleration_hi::R =
          params().modes[mode_int].cost_estimation.online.response_estimation.R_threshold_acceleration_hi
    is_responding::Bool = true
    sense_sign::Z = 1
    if (sense == :Down)
       sense_sign = -1
    end
    if (sense == :None) || isnan( s_c.dz_after_wait ) || (0 == s_c.t_same_sense)
        is_responding = true
    else
        ddz_response::R     = (s_c.dz_response_initial - dz_curr) /  s_c.t_same_sense
        ddz_response_max::R = 0.0
        if (s_c.t_same_sense != s_c.t_dz_response_max)
            ddz_response_max = (s_c.dz_response_max - dz_curr) / (s_c.t_same_sense - s_c.t_dz_response_max)
        end
        if (R_threshold_acceleration_lo < abs( ddz_response )) && (0 < (ddz_response * sense_sign)) &&
            ((abs( ddz_response_max ) < R_threshold_acceleration_hi) ||
             (0 < (ddz_response_max * sense_sign)))
            is_responding = false
        else
            dz_target::R = s_c.dz_after_wait - (sense_sign * dz_buffer) +
                           (sense_sign * (s_c.t_same_sense - t_wait) * R_acceleration)
            if (0 < ((dz_target + (sense_sign * dz_buffer)) * sense_sign))
                dz_target = -1 * sense_sign * dz_buffer
            end
            dz_target_wob::R = s_c.dz_after_wait +
                               (sense_sign * (s_c.t_same_sense - t_wait) * R_acceleration)
            if (0 < (dz_target_wob * sense_sign))
                dz_target_wob = -1 * sense_sign
            end
            if (0 < (sense_sign * (dz_curr + (sense_sign * dz_buffer))))
                is_responding = true
            elseif s_c.is_responding_prev
                is_responding = (0 < (sense_sign * (dz_curr - dz_target)))
            else
                is_responding = (0 < (sense_sign * (dz_curr - dz_target_wob)))
            end
        end
    end
    return is_responding::Bool
end
```

Referenced In: OwnResponseEstimation(p. 315), IntruderResponseEstimation(p. 314)

3.3.2.6.2 Compatibility Cost

COMPATIBILITYCOST (Algorithm 294) adds a cost each time the resolution advisory has a sense that conflicts with the VRC message of an intruder. It is used, in combination with other online costs, to enforce coordination rules. Small costs allow for flexibility in selecting a resolution advisory but still reduce reversal frequency in equipped encounters. Large costs enforce compatibility. The sense of the resolution advisory under consideration for own aircraft is *sense_own*. The sense of the intruder, as inferred from the VRC message, is *sense_int*. These two senses conflict if they are the same, that is, if own aircraft and the intruder are both climbing or both are

descending. If the ownship is a master, several conditions are checked to determine the value of the cost to apply. Additionally, these costs only apply to the master if the intruder aircraft is determined to be responding, based on *resp_int*, which is calculated in INTRUDERRESPONSEESTIMATION (Algorithm 291). This check is necessary to ensure the master is unconstrained in the event of a non-responsive intruder and can maintain an appropriate level of safety. Similar to the SA01HEURISTIC (Algorithm 307), which protects the intruder if the ownship pilot chooses not to respond, this check has a significant impact on the safety of vertical chase scenarios where one pilot does not respond. Furthermore if the master ownship is determined to be non responsive when the slave intruder is responsive the cost is also removed to allow unconstrained reversals. If the intruder is a master, the ownship is required to pick a compliant action. Thus, the cost for a conflicting action in this case, *C_master* is large.

There are seven different cost values:

- *C_master* is a large cost, used to prevent a reversal when the intruder is master;
- *C_slave_init* is a large cost, used to enforce coordination rules that prevent the master from reversing the slave intruder when the previous action of the own aircraft was clear of conflict (COC) and there is no potential crossing;
- *C_slave_init_crossing* is a very small cost, used to allow the master to reverse the slave intruder when there is no previous resolution advisory, there is a potential crossing, and the slave VRC was received within 3 seconds, under the coordination rules;
- *C_slave_sub* is a moderate cost, used to constrain the master reversing the slave intruder when the previous action of the own aircraft was neither COC nor a crossing advisory and there was no prior multithreat advisory;
- *C_slave_sub_noncrossing* is a small cost, used to constrain the master reversing the slave intruder when the previous action of the own aircraft was a crossing advisory and there was no prior multithreat advisory;
- *C_slave_sub_no_response* is a null cost, used to remove the constraints on the master reversing the slave intruder when the master is not responding;
- *C_slave_sub_multithreat* is a small cost, used to constrain the master reversing the slave intruder when the previous action was not COC and there was a previous multithreat situation.

When own aircraft is master and there are multiple intruders from which own aircraft is receiving conflicting VRCs, no cost is applied. Eliminating the cost in that situation allows own aircraft more flexibility in choosing the best resolution advisory.

The *master_int* variable is a boolean representing if the intruder aircraft is the master. It is set via ISINTRUDERMASTER. The *vrcs_conflict* variable is a boolean indicating whether multiple intruders are sending conflicting VRCs to own aircraft. It is set in UPDATEINTRUDERINPUTS.

The state variable *s_c.t_since_first_vrc* is used with *T_reversal_thres* when there is a potential crossing. It is used to determine when to enforce a coordination rule for a slave reversal and apply *C_slave_init*, a large cost, instead of *C_slave_init_crossing*, a small cost. In a similar manner, the *H_noncrossing_thres* and *T_noncrossing_thres* are used to determine if a subsequent advisory will force a reversal to a non-crossing RA early in the encounter. If this is true, the smaller *C_slave_sub_noncrossing* will be applied as opposed to the larger *C_slave_sub*.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATECOMPATIBILITYCSTATE.

The COMPATIBILITYCOST is excluded from *cost_ra_subset*, which is used in INDIVIDUALCOSTESTIMATION to determine whether to penalize an action to force a resolution advisory.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *vrc_int*, *resp_own*, *master_int*, *z_own_ave*, *z_int_ave*, *dz_min_prev*, *dz_max_prev*, *multithreat_prev*, *resp_int*, *vrcs_conflict*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 294 CompatibilityCost

```
function CompatibilityCost( mode_int::Z, dz_min::R, dz_max::R, vrc_int::UInt32,
                    resp_own::Bool, master_int::Bool,  z_own_ave::R,
                    z_int_ave::R, dz_min_prev::R, dz_max_prev::R,
                    multithreat_prev::Bool, resp_int::Bool, vrcs_conflict::Bool,
                    update::Bool, s_c::CompatibilityCState(p. 528), idx_online_cost::Z )
    C_master::R =
            params().modes[mode_int].cost_estimation.online.compatibility.C_master[idx_online_cost]
    C_slave_sub_multithreat::R =
            params().modes[mode_int].cost_estimation.online.compatibility.C_slave_sub_multithreat[idx_-
                online_cost]
    T_reversal_thres::Z =
            params().modes[mode_int].cost_estimation.online.compatibility.T_reversal_thres[idx_online_cost]
    H_noncrossing_thres::R =
            params().modes[mode_int].cost_estimation.online.compatibility.H_noncrossing_thres[idx_online_-
                cost]
    T_noncrossing_thres::Z =
            params().modes[mode_int].cost_estimation.online.compatibility.T_noncrossing_thres[idx_online_-
                cost]
    if update
        UpdateCompatibilityCState(p. 320)( mode_int, vrc_int, s_c, idx_online_cost )
    end
    cost::R = 0.0
    sense_int::Symbol      = VRCToSense(p. 436)( vrc_int )
    sense_own::Symbol      = RatesToSense(p. 432)( dz_min, dz_max )
    sense_own_prev::Symbol = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
    if (:None != sense_own) && (sense_own == sense_int)
        if master_int
            cost = C_master
        elseif IsCOC(p. 419)( dz_min_prev, dz_max_prev )
            if vrcs_conflict
                cost = 0.0
            elseif (s_c.t_since_first_vrc <= T_reversal_thres) &&
                   IsCrossing(p. 420)( z_own_ave, z_int_ave, sense_int )
                cost = s_c.c_slave_init_crossing
            else
                cost = s_c.c_slave_init
            end
        elseif resp_int && (sense_own != sense_own_prev)
            if multithreat_prev
                cost = C_slave_sub_multithreat
            elseif !resp_own
                cost = s_c.c_slave_sub_no_response
            elseif IsCrossing(p. 420)( z_int_ave, z_own_ave, sense_own_prev ) &&
                   !IsCrossing(p. 420)( z_int_ave, z_own_ave, sense_own ) &&
                   (H_noncrossing_thres < abs( z_own_ave - z_int_ave )) &&
                   (s_c.t_since_first_vrc < T_noncrossing_thres)
                cost = s_c.c_slave_sub_noncrossing
            else
                cost = s_c.c_slave_sub
            end
        end
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

UPDATECOMPATIBILITYCSTATE (Algorithm 295) is used to update the state variables of COMPATIBILITYCOST once per cycle. This algorithm has two primary goals. The first is to keep a record of how long the intruder has been sending a VRC (*s_c.t_since_first_vrc*). The second is to set the correct parameters in the state variable.

This algorithm takes as input *mode_int*, *vrc_int*, and *s_c*. The algorithm updates *s_c*.

Algorithm 295 UpdateCompatibilityCState

```
function UpdateCompatibilityCState( mode_int::Z, vrc_int::UInt32, s_c::CompatibilityCState(p.528), idx_online_cost::
    Z )
    s_c.c_slave_init =
        params().modes[mode_int].cost_estimation.online.compatibility.C_slave_init[idx_online_cost]
    s_c.c_slave_init_crossing =
        params().modes[mode_int].cost_estimation.online.compatibility.C_slave_init_crossing[idx_online_cost]
    s_c.c_slave_sub =
        params().modes[mode_int].cost_estimation.online.compatibility.C_slave_sub[idx_online_cost]
    s_c.c_slave_sub_noncrossing =
        params().modes[mode_int].cost_estimation.online.compatibility.C_slave_sub_noncrossing[idx_online_-
            cost]
    s_c.c_slave_sub_no_response =
        params().modes[mode_int].cost_estimation.online.compatibility.C_slave_sub_no_response[idx_online_-
            cost]
    sense_int::Symbol = VRCToSense(p.436)( vrc_int )
    if (:None == sense_int)
        s_c.t_since_first_vrc = 0
    else
        s_c.t_since_first_vrc = s_c.t_since_first_vrc + 1
    end
end
```

Referenced In: CompatibilityCost(p.319)

3.3.2.6.3 Coordinated Resolution Advisory Deferral Cost

COORDINATEDRADEFERRALCOST (Algorithm 296) is used to reduce the frequency of both aircraft issuing corrective alerts in a coordinated encounter. As many geometries can be resolved with just one aircraft issuing a corrective alert, this cost suppresses non-preventive alerts on the own aircraft if the intruder has already alerted. If the deferral cost state variable (*s_c.deferral_cost*) is set, the current action is clear of conflict, and the proposed action is not clear of conflict or a preventive advisory, the cost is applied.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATECOORDINATEDRADEFERRALCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *equip_int*, *vrc_int*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 296 CoordinatedRADeferralCost

```
function CoordinatedRADeferralCost( mode_int::Z, dz_min::R, dz_max::R,
                            dz_min_prev::R, dz_max_prev::R,
                            z_own_ave::R, dz_own_ave::R,
                            z_int_ave::R, dz_int_ave::R,
                            equip_int::Bool, vrc_int::UInt32, update::Bool,
                            s_c::CoordinatedRADeferralCState(p. 528),
                            idx_online_cost::Z )
    if update
        UpdateCoordinatedRADeferralCState(p. 322)( mode_int, z_own_ave, dz_own_ave, z_int_ave, dz_int_ave,
                                            equip_int, vrc_int, s_c, idx_online_cost )
    end
    cost::R = 0.0
    if IsCOC(p. 419)( dz_min_prev, dz_max_prev ) &&
       !IsCOC(p. 419)( dz_min, dz_max ) &&
       !IsPreventive(p. 426)( dz_min, dz_max )
        cost = s_c.deferral_cost
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

UPDATECOORDINATEDRADEFERRALCSTATE (Algorithm 297) is used to update the state variables of COORDINATEDRADEFERRALCOST once per cycle.

The RA deferral is subject to several requirements to ensure safety can be maintained by the aircraft during the encounter. The first requirement is that the intruder aircraft must be equipped and have an active RA against own aircraft. Furthermore, the RA on the intruder aircraft must be a non-crossing RA, as most crossings should be secured by advisories on both aircraft. If these conditions are met, the vertical states of own aircraft and intruder are projected forward in time *T_proximate* seconds to determine the expected level of separation in the near future. If sufficient separation is expected, then corrective RAs will be suppressed by a moderate cost. If loss of separation or a crossing is expected, then the cost applied to corrective RAs will be reduced to zero, with interpolation when the separation is between.

This algorithm takes as input *mode_int*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *equip_-int*, *vrc_int*, and *s_c*. The algorithm outputs updates to *s_c*.

Algorithm 297 UpdateCoordinatedRADeferralCState

```
function UpdateCoordinatedRADeferralCState( mode_int::Z, z_own_ave::R, dz_own_ave::R,
                                            z_int_ave::R, dz_int_ave::R,
                                            equip_int::Bool, vrc_int::UInt32,
                                            s_c::CoordinatedRADeferralCState(p. 528),
                                            idx_online_cost::Z )
    T_proximate::R =
        params().modes[mode_int].cost_estimation.online.coord_ra_deferral.T_proximate[idx_online_cost]
    H_threshold_proximate::R =
        params().modes[mode_int].cost_estimation.online.coord_ra_deferral.H_threshold_proximate[idx_online_-
            cost]
    H_rolloff_proximate::R =
        params().modes[mode_int].cost_estimation.online.coord_ra_deferral.H_rolloff_proximate[idx_online_-
            cost]
    C_deferral::R =
        params().modes[mode_int].cost_estimation.online.coord_ra_deferral.C_deferral[idx_online_cost]
    s_c.deferral_cost = 0.0
    sense_int::Symbol = VRCToSense(p. 436)( vrc_int )
    if equip_int && (:None != sense_int)
        z_rel::R = z_int_ave - z_own_ave
        z_rel_proximate::R = z_rel + ((dz_int_ave - dz_own_ave) * T_proximate)
        if IsDiverging(p. 420)( z_own_ave, z_int_ave, sense_int ) &&
           !IsProjectedCrossing(p. 426)( z_rel, z_rel_proximate )
            z_rel_proximate = min( abs( z_rel ), abs( z_rel_proximate ) )
            scale_factor::R   = CalculateThresholdRampUpFactor(p. 411)( abs( z_rel_proximate ),
                H_threshold_proximate, H_rolloff_proximate )
            s_c.deferral_cost = C_deferral * scale_factor
        end
    end
end
```

Referenced In: CoordinatedRADeferralCost(p. 321)

3.3.2.6.4 Coordination Delay Cost

COORDINATIONDELAYCOST (Algorithm 298) prohibits resolution advisories (RAs) from being issued immediately for a CAS-equipped intruder that can issue RAs. The delay associated with this cost (*s_c.t_count*) allows time for a coordination message to be received from a CAS-equipped intruder following transition from Surveillance Only operational mode to Alerting operational mode on ownship. *s_c.t_count* is initialized to 0 when the intruder becomes eligible for RA processing. That occurs when the intruder is first received for TRM processing, when it no longer has degraded surveillance, or when ownship transitions from Surveillance Only operational mode to Alerting operational mode. If ownship is in Surveillance Only operational mode, the intruder is not equipped with a CAS, or the intruder is equipped with a CAS but cannot issue RAs, *s_c.t_count* is set to *T_init*. Otherwise, *s_c.t_count* is updated based on receipt of a VRC from the intruder. Once *s_c.t_count* is greater than or equal to *T_init*, resolution advisories are not penalized.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATECOORDINATIONDELAYCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *equip_int*, *vrc_int*, *hrc_int*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 298 CoordinationDelayCost

```
function CoordinationDelayCost( mode_int::Z, dz_min::R, dz_max::R, equip_int::Bool, vrc_int::UInt32,
                        hrc_int::UInt32, update::Bool, s_c::CoordinationDelayCState(p. 528), idx_online_cost::Z )
    T_init::Z = params().modes[mode_int].cost_estimation.online.coord_delay.T_init[idx_online_cost]
    if update
        UpdateCoordinationDelayCState(p. 323)( mode_int, equip_int, vrc_int, hrc_int, s_c, idx_online_cost )
    end
    cost::R   = 0.0
    if (s_c.t_count < T_init) && !IsCOC(p. 419)( dz_min, dz_max )
        cost = Inf
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

UPDATECOORDINATIONDELAYCSTATE (Algorithm 299) is used to update the state variables of COORDINATIONDELAYCOST once per cycle.

Variable *s_c.t_count* is initialized to *0*.

When ownship and the intruder can coordinate resolution advisories as indicated by *equip_int*, and *s_c.is_count_enabled* is TRUE, *s_c.t_count* counts up until a VRC or HRC is received from the intruder or *T_init* is reached. When a VRC or HRC is received from the intruder and the sense of the intruder (*sense_int*) has a value other than *:NONE*, *s_c.t_count* is set to *T_init*.

If either ownship or the intruder cannot issue resolution advisories, or *s_c.is_count_enabled* is FALSE, *s_c.t_count* is set to *T_init*.

This algorithm takes as input *mode_int*, *equip_int*, *vrc_int*, *hrc_int*, and *s_c*. The algorithm updates *s_c*.

Algorithm 299 UpdateCoordinationDelayCState

```
function UpdateCoordinationDelayCState( mode_int::Z, equip_int::Bool, vrc_int::UInt32,
                                hrc_int::UInt32, s_c::CoordinationDelayCState(p. 528), idx_online_cost::Z )
    T_init::Z = params().modes[mode_int].cost_estimation.online.coord_delay.T_init[idx_online_cost]
    if equip_int && s_c.is_count_enabled
        v_sense_int::Symbol = VRCToSense(p. 436)( vrc_int )
        h_sense_int::Symbol = HRCToAdvisory(p. 415)( hrc_int )
        if (:None == v_sense_int) && (:None == h_sense_int)
            s_c.t_count = min( T_init, s_c.t_count + 1 )
        else
            s_c.t_count = T_init
        end
    else
        s_c.t_count = T_init
    end
end
```

Referenced In: CoordinationDelayCost(p. 323)

3.3.2.6.5 Crossing No Alert Cost

CROSSINGNOALERTCOST (Algorithm 300) forces the aircraft to issue a resolution advisory (RA)

when the intruder intends to cross altitudes. This algorithm forces the aircraft to issue an RA by penalizing the clear of conflict (COC) action by *C_coc*. This ensures that when one intruder initiates a crossing, the ownship will issue an RA, possibly a reversal, subject to coordination rules in COMPATIBILITYCOST. If the crossing flag state variable (*s_c.is_crossing*) is set, the proposed action is clear of conflict, the geometric crossing flag (*s_c.is_crossing_caused_by_geometry*) is not set, and tau is less than a threshold (*T_threshold*), the cost is applied. The value of tau (*tau_expected*) must be within the threshold to prevent inappropriate application of the cost during slow closure encounters. Tau is the time until the intruder comes within a lateral conflict distance of own aircraft. EXPECTEDTAU (Algorithm 378) is used to estimate a value for tau from the distribution. See section 3.3.2.2.1 for a discussion of tau.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATECROSSINGNOALERTCSTATE.

The CROSSINGNOALERTCOST is excluded from *cost_ra_subset*, which is used in INDIVIDUALCOSTESTIMATION to determine whether to penalize an action to force a resolution advisory.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *z_own_ave*, *z_int_ave*, *vrc_int*, *tau_expected*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 300 CrossingNoAlertCost

```
function CrossingNoAlertCost( mode_int::Z, dz_min::R, dz_max::R,
                    z_own_ave::R, z_int_ave::R, vrc_int::UInt32, tau_expected::R,
                    update::Bool, s_c::CrossingNoAlertCState(p. 529), idx_online_cost::Z )
    C_coc::R =
        params().modes[mode_int].cost_estimation.online.crossing_no_alert.C_coc[idx_online_cost]
    T_threshold::R =
        params().modes[mode_int].cost_estimation.online.crossing_no_alert.T_threshold[idx_online_cost]
    if update
        UpdateCrossingNoAlertCState(p. 325)( z_own_ave, z_int_ave, vrc_int, s_c )
    end
    cost::R = 0.0
    if s_c.is_crossing &&
       IsCOC(p. 419)( dz_min, dz_max ) &&
       !s_c.is_crossing_caused_by_geometry &&
       (tau_expected < T_threshold)
        cost = C_coc
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

UPDATECROSSINGNOALERTCSTATE (Algorithm 301) is used to update the state variables of CROSSINGNOALERTCOST once per cycle. The intruder's intent is inferred from the received VRC message and the altitude of each aircraft using ISCROSSING. If the intruder's intent indicates crossing on this cycle, but did not indicate a crossing on the previous cycle, and the VRC did not change, the crossing indication is due to aircraft geometry changing. For example, an intruder above the ownship may issue a climb resolution advisory while descending and, if the response is too slow, it may cross altitudes with the ownship. In this case, the cost is not applied to prevent it from forcing an alert that could potentially reverse the intruder.

This algorithm takes as input *z_own_ave*, *z_int_ave*, *vrc_int*, and *s_c*. The algorithm updates *s_c*.

Algorithm 301 UpdateCrossingNoAlertCState

```
function UpdateCrossingNoAlertCState( z_own_ave::R, z_int_ave::R, vrc_int::UInt32,
                                      s_c::CrossingNoAlertCState(p. 529) )
    sense_int::Symbol = VRCToSense(p. 436)( vrc_int )
    s_c.is_crossing = IsCrossing(p. 420)( z_own_ave, z_int_ave, sense_int )
    if s_c.is_crossing && !s_c.is_crossing_prev && (vrc_int == s_c.vrc_int_prev)
        s_c.is_crossing_caused_by_geometry = true
    else
        s_c.is_crossing_caused_by_geometry = false
        s_c.is_crossing_prev = s_c.is_crossing
    end
    s_c.vrc_int_prev = vrc_int
end
```

Referenced In: CrossingNoAlertCost(p. 324), StateAndCostEstimation(p. 251), SetVerticalTRMSurvOnlyMode(p. 247)

3.3.2.6.6 Max Reversal Cost

MAXREVERSALCOST (Algorithm 302) penalizes certain actions if the number of reversals exceeds a threshold (*N_limit*). To prevent premature advisory termination, clear of conflict is restricted by the cost *C_coc* for *T_coc_threshold* TRM cycles after the reversal limit is reached. Reversals are restricted by the cost *C_reversal* if the reversal limit has been reached, unless the master intruder aircraft is forcing the reversal, as determined by ISMASTERFORCINGREVERSAL. The reversal count and time since reaching the reversal limit are contained within the internal state information (*s_c*).

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATEMAXREVERSALCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *vrc_int*, *master_int*, *dz_min_prev*, *dz_max_prev*, *equip_int*, *multithreat_prev*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 302 MaxReversalCost

```
function MaxReversalCost( mode_int::Z, dz_min::R, dz_max::R, vrc_int::UInt32, master_int::Bool,
                dz_min_prev::R, dz_max_prev::R, equip_int::Bool,
                multithreat_prev::Bool, update::Bool, s_c::MaxReversalCState(p. 530),
                idx_online_cost::Z )
    T_coc_threshold::Z = params().modes[mode_int].cost_estimation.online.max_reversal.T_coc_threshold[idx_-
        online_cost]
    N_limit::Z         = params().modes[mode_int].cost_estimation.online.max_reversal.N_limit[idx_online_-
        cost]
    C_reversal::R      = params().modes[mode_int].cost_estimation.online.max_reversal.C_reversal[idx_-
        online_cost]
    C_coc::R           = params().modes[mode_int].cost_estimation.online.max_reversal.C_coc[idx_online_cost]
    if update
        UpdateMaxReversalCState(p. 327)( mode_int, vrc_int, master_int, dz_min_prev, dz_max_prev,
                                equip_int, multithreat_prev, s_c, idx_online_cost )
    end
    cost::R = 0.0
    if IsCOC(p. 419)( dz_min, dz_max )
        if (s_c.crossed_thres_time < T_coc_threshold) && (N_limit <= s_c.num_reversals)
            cost = C_coc
        end
    elseif (N_limit <= s_c.num_reversals) &&
           !IsMasterForcingReversal(p. 425)( master_int, s_c.sense_own_prev, s_c.sense_int )
        sense_own = RatesToSense(p. 432)( dz_min, dz_max )
        if IsReversal(p. 427)( s_c.sense_own_prev, sense_own )
            cost = C_reversal
        end
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

UPDATEMAXREVERSALCSTATE (Algorithm 303) is used to update the state variables of MAXREVERSALCOST once per cycle. The reversal count is incremented if there was a reversal in the previous TRM cycle, with two additional constraints. The first constraint is that the reversal was not forced by an intruder master aircraft, as determined by ISMASTERFORCINGREVERSAL. The second constraint is that ownship was not in a multithreat encounter (*multithreat_prev*) with an unequipped intruder (*equip_int*). Reversals for unequipped intruders in multithreat encounters are not counted in order to maintain performance with equipped intruders. When the previous action was clear of conflict, the reversal counter and time since last reversal are reset.

This algorithm takes as input *mode_int*, *vrc_int*, *master_int*, *dz_min_prev*, *dz_max_prev*, *equip_-int*, *multithreat_prev*, and *s_c*. The algorithm updates *s_c*.

Algorithm 303 UpdateMaxReversalCState

```
function UpdateMaxReversalCState( mode_int::Z, vrc_int::UInt32, master_int::Bool,
                        dz_min_prev::R, dz_max_prev::R, equip_int::Bool,
                        multithreat_prev::Bool, s_c::MaxReversalCState(p. 530),
                        idx_online_cost::Z )
    N_limit::Z = params().modes[mode_int].cost_estimation.online.max_reversal.N_limit[idx_online_cost]
    sense_own_prev::Symbol = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
    if IsReversal(p. 427)( s_c.sense_own_prev, sense_own_prev )
        if !IsMasterForcingReversal(p. 425)( master_int, s_c.sense_own_prev, s_c.sense_int ) &&
           (!multithreat_prev || (multithreat_prev && equip_int))
            s_c.num_reversals = s_c.num_reversals + 1
        end
    elseif IsCOC(p. 419)( dz_min_prev, dz_max_prev )
        s_c.num_reversals = 0
    end
    if (N_limit <= s_c.num_reversals)
        s_c.crossed_thres_time = s_c.crossed_thres_time + 1
    else
        s_c.crossed_thres_time = 0
    end
    s_c.sense_int = VRCToSense(p. 436)( vrc_int )
    s_c.sense_own_prev = sense_own_prev
end
```

Referenced In: MaxReversalCost(p. 326)

3.3.2.6.7 Prevent Early Weakening Cost

PREVENTEARLYWEAKENINGCOST (Algorithm 304) penalizes issuance of a weakening resolution advisory by *C_weakening* in equipped-equipped encounters to ensure the coordinating aircraft continue to issue corrective resolution advisories until they are projected to diverge. Divergence is determined by verifying that the expected vertical separation at minimum tau is positive if intruder VRC sense is positive or that the expected vertical separation at minimum tau is negative if intruder VRC sense is negative. Tau is the time until the intruder comes within a lateral conflict distance of own aircraft and is provided as an input (*tau_expected*).

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, *z_own_-ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *vrc_int*, and *tau_expected*. This algorithm returns *cost*.

Algorithm 304 PreventEarlyWeakeningCost

```
function PreventEarlyWeakeningCost( mode_int::Z, dz_min::R, dz_max::R,
                            dz_min_prev::R, dz_max_prev::R,
                            z_own_ave::R, dz_own_ave::R,
                            z_int_ave::R, dz_int_ave::R,
                            vrc_int::UInt32, tau_expected::R,
                            idx_online_cost::Z )
    C_weakening::R =
        params().modes[mode_int].cost_estimation.online.prevent_early_weakening.C_weakening[idx_online_cost]
    cost::R = 0.0
    sense_int::Symbol  = VRCToSense(p. 436)( vrc_int )
    if (:None != sense_int) && (IsDND(p. 421)( dz_min, dz_max ) || IsDNC(p. 421)( dz_min, dz_max ))
        z_rel_projected::R = (z_own_ave - z_int_ave) + ((dz_own_ave - dz_int_ave) * tau_expected)
        if IsCorrective(p. 419)( dz_min_prev, dz_max_prev ) &&
           (((0 <= z_rel_projected) && (:Up == sense_int)) ||
            ((z_rel_projected < 0) && (:Down == sense_int)))
            cost = C_weakening
        end
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

3.3.2.6.8 Restrict Clear of Conflict Due To Reversal

RESTRICTCOCDUETOREVERSAL (Algorithm 305) penalizes the transition to clear of conflict (COC) by *C_restrict_coc* when own aircraft is a slave and is forced to reverse due to coordination. The *master_int* variable is a boolean representing if the intruder aircraft is the master. ISMASTERFORCINGREVERSAL (Algorithm 399) ensures that the restriction of COC only occurs for the one time step that a VRC message is received from an intruder aircraft that is the master. It forces an opposite sense advisory than was already issued. Whether an intruder is the master is determined in VERTICALTRMUPDATE using ISINTRUDERMASTER.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATERESTRICTCOCDUETOREVERSALCSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *vrc_int*, *master_int*, *dz_min_prev*, *dz_max_prev*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 305 RestrictCOCDueToReversal

```
function RestrictCOCDueToReversal( mode_int::Z, dz_min::R, dz_max::R, vrc_int::UInt32,
                           master_int::Bool, dz_min_prev::R, dz_max_prev::R,
                           update::Bool, s_c::RestrictCOCCState(p. 531), idx_online_cost::Z )
    C_restrict_coc::R =
      params().modes[mode_int].cost_estimation.online.restrict_coc_due_to_reversal.C_restrict_coc[idx_-
           online_cost]
    if update
        UpdateRestrictCOCDueToReversalCState(p. 329)( dz_min_prev, dz_max_prev, s_c )
    end
    cost::R = 0.0
    if IsCOC(p. 419)( dz_min, dz_max )
        sense_int::Symbol = VRCToSense(p. 436)( vrc_int )
        if IsMasterForcingReversal(p. 425)( master_int, s_c.sense_own_prev, sense_int ) && !s_c.coc_prev
            cost = C_restrict_coc
        end
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

UPDATERESTRICTCOCDUETOREVERSALCSTATE (Algorithm 306) is used to update the state variables of RESTRICTCOCDUETOREVERSAL once per cycle. A flag indicating clear of conflict on the previous cycle and memory of the own aircraft sense on the previous cycle are updated.

This algorithm takes as input *dz_min*, *dz_max*, and *s_c*. The algorithm updates *s_c*.

Algorithm 306 UpdateRestrictCOCDueToReversalCState

```
function UpdateRestrictCOCDueToReversalCState( dz_min_prev::R, dz_max_prev::R, s_c::RestrictCOCCState(p. 531) )
    s_c.coc_prev = IsCOC(p. 419)( dz_min_prev, dz_max_prev )
    s_c.sense_own_prev = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
end
```

Referenced In: RestrictCOCDueToReversal(p. 329)

3.3.2.6.9 SA01 Heuristic

SA01HEURISTIC (Algorithm 307) penalizes resolution advisories to force a reversal. If the force reversal flag (*s_c.force_reversal*) is set, clear of conflict and all non-reverse advisories are penalized. This cost uses the sense of the own aircraft advisory from the previous cycle to determine which proposed actions are not reversals.

Once per cycle, the state variables in *s_c* are set using UPDATESA01HEURISTICCSTATE (Algorithm 308), a helper algorithm.

The SA01HEURISTIC is excluded from *cost_ra_subset*, which is used in INDIVIDUALCOSTESTIMATION to determine whether to penalize an action to force a resolution advisory.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *vrc_int*, *equip_int*, *resp_own*, *master_int*, *z_rel*, *dz_own_ave*, *dz_int_ave*, *dz_min_prev*, *dz_max_prev*, *tau_expected_no_horizon*, *update* and *s_c*. This algorithm returns *cost*.

Algorithm 307 SA01Heuristic

```
function SA01Heuristic( mode_int::Z, dz_min::R, dz_max::R, vrc_int::UInt32, equip_int::Bool,
               resp_own::Bool, master_int::Bool, z_rel::R, dz_own_ave::R, dz_int_ave::R,
               dz_min_prev::R, dz_max_prev::R, tau_expected_no_horizon::R,
               update::Bool, s_c::SA01HeuristicCState(p. 531), idx_online_cost::Z )
    C_non_reversal::R = params().modes[mode_int].cost_estimation.online.SA01.C_non_reversal[idx_online_cost]
    C_coc::R          = params().modes[mode_int].cost_estimation.online.SA01.C_coc[idx_online_cost]
    if update
        UpdateSA01HeuristicCState(p. 331)( mode_int, vrc_int, equip_int, resp_own, master_int, z_rel,
                                   dz_own_ave, dz_int_ave, dz_min_prev, dz_max_prev,
                                   tau_expected_no_horizon, s_c, idx_online_cost )
    end
    sense_own_prev::Symbol = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
    sense_own::Symbol      = RatesToSense(p. 432)( dz_min, dz_max )
    cost::R = 0.0
    if s_c.force_reversal
        if IsCOC(p. 419)( dz_min, dz_max )
            cost = C_coc
        elseif (sense_own_prev == sense_own) && (sense_own_prev != :None)
            cost = C_non_reversal
        end
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

UPDATESA01HEURISTICCSTATE (Algorithm 308) is used once per cycle to update the variable-force_reversal field of SA01HEURISTIC. If the own aircraft is not responding to advisories (*resp_own*), the own aircraft and intruder have vertical rates in the same direction and of sufficient magnitude (*R_min*), and the intruder is equipped but not master, the SHOULDREVERSE algorithm is invoked. These criteria help identify an SA01 (vertical chase) encounter when the master is not responding and there is an equipped intruder. During these situations, a reversal is often the only way to resolve an encounter. The state variable *s_c.force_reversal* stores the result of SHOULDREVERSE, which is used to determine if a reversal should be issued.

This algorithm takes as input *mode_int*, *vrc_int*, *equip_int*, *resp_own*, *master_int*, *z_rel*, *dz_own_ave*, *dz_int_ave*, *dz_min_prev*, *dz_max_prev*, *tau_expected_no_horizon*, and *s_c*. The algorithm updates *s_c*.

Algorithm 308 UpdateSA01HeuristicCState

```
function UpdateSA01HeuristicCState( mode_int::Z, vrc_int::UInt32, equip_int::Bool,
                            resp_own::Bool, master_int::Bool,
                            z_rel::R, dz_own_ave::R, dz_int_ave::R,
                            dz_min_prev::R, dz_max_prev::R,
                            tau_expected_no_horizon::R, s_c::SA01HeuristicCState(p. 531),
                            idx_online_cost::Z )
    R_min::R = params().modes[mode_int].cost_estimation.online.SA01.R_min[idx_online_cost]
    sense_own_prev::Symbol = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
    same_sign_rates::Bool = (0 < (dz_own_ave * dz_int_ave)) &&
                             (R_min < abs( dz_own_ave )) &&
                             (R_min < abs( dz_int_ave ))
    s_c.force_reversal = false
    if !resp_own && same_sign_rates && equip_int && !master_int
        s_c.force_reversal = ShouldReverse(p. 332)( mode_int, z_rel, dz_own_ave, dz_int_ave,
                                                    tau_expected_no_horizon, vrc_int, sense_own_prev,
                                                    s_c.range, idx_online_cost )
    end
end
```

Referenced In: SA01Heuristic(p. 330)

SHOULDREVERSE (Algorithm 309) uses a linear approximation, with constant acceleration to the vertical rate for the current resolution advisory, to estimate the vertical separation with the current resolution advisory (*z_rel_proj*), and after issuing a reversal (*z_rel_proj_rev*). The algorithm then uses the estimates to determine whether a reversal should be issued. If more separation is expected with the reversal or an altitude crossing is expected without a reversal, a reversal will be forced. SHOULDREVERSE returns FALSE if the estimated value of tau is too short to allow time for the intruder to reverse (*T_min_tau*) or if the estimated value of tau is long enough that a reversal may not be necessary (*T_max_tau*). It also returns FALSE if a reversal would cause an altitude crossing and the vertical separation between the two aircraft is greater than *H_rel_threshold* or the horizontal separation is less than *D_range_threshold*.

This algorithm takes as input *mode_int*, *z_rel*, *dz_own*, *dz_int*, *tau*, *vrc_int*, *sense_own_prev*, and *range*. This algorithm returns *force_reversal*.

Algorithm 309 ShouldReverse

```
function ShouldReverse( mode_int::Z, z_rel::R, dz_own::R, dz_int::R, tau::R,
               vrc_int::UInt32, sense_own_prev::Symbol, range::R, idx_online_cost::Z )
    R_acceleration::R = params().modes[mode_int].cost_estimation.online.SA01.R_acceleration[idx_online_cost]
    R_strengthen::R   = params().modes[mode_int].cost_estimation.online.SA01.R_strengthen[idx_online_cost]
    R_standard::R     = params().modes[mode_int].cost_estimation.online.SA01.R_standard[idx_online_cost]
    T_min::R          = params().modes[mode_int].cost_estimation.online.SA01.T_min[idx_online_cost]
    T_max::R          = params().modes[mode_int].cost_estimation.online.SA01.T_max[idx_online_cost]
    T_delay::R        = params().modes[mode_int].cost_estimation.online.SA01.T_delay[idx_online_cost]
    H_rel_threshold::R = params().modes[mode_int].cost_estimation.online.SA01.H_rel_threshold[idx_online_-
        cost]
    D_range_threshold::R = params().modes[mode_int].cost_estimation.online.SA01.D_range_threshold[idx_-
        online_cost]
    sense_int::Symbol      = VRCToSense(p. 436)( vrc_int )
    force_reversal::Bool   = false
    sense_own_prev_sign::Z = 1
    if (sense_own_prev == :Down)
        sense_own_prev_sign = -1
    end
    reversal_is_cross::Bool = 0 < z_rel*dz_own
    if (sense_int != :None) && ((sense_own_prev_sign * dz_own) <= 0) &&
       (T_min <= tau) && (tau <= T_max) &&
       (!reversal_is_cross || ((abs(z_rel) <= H_rel_threshold) && (D_range_threshold <= range)))
        t_max::Z          = floor( tau - T_delay - 1 )
        sense_int_sign::Z = 1
        if (sense_int == :Down)
            sense_int_sign = -1
        end
        z_rel_proj::R = z_rel + ((dz_int - dz_own) * T_delay)
        dz_int_new::R = dz_int
        for t = 1:t_max
            if (0 < (dz_int * sense_int_sign)) && (R_strengthen <= abs( dz_int ))
                dz_int_new = dz_int
            elseif ((dz_int_new * sense_int_sign) < 0) || (abs( dz_int_new ) < R_strengthen)
                dz_int_new = dz_int_new + (sense_int_sign * R_acceleration)
            elseif (R_strengthen < abs( dz_int_new ))
                dz_int_new = sense_int_sign * R_strengthen
            end
            z_rel_proj = z_rel_proj + (dz_int_new - dz_own)
        end
        z_rel_proj_rev = z_rel + ((dz_int - dz_own) * T_delay)
        dz_int_new = dz_int
        for t = 1:t_max
            if ((dz_int_new * sense_own_prev_sign) < 0) || (abs( dz_int_new ) < R_standard)
                dz_int_new = dz_int_new + (sense_own_prev_sign * R_acceleration)
            elseif (R_standard < abs( dz_int_new ))
                dz_int_new = sense_own_prev_sign * R_standard
            end
            z_rel_proj_rev = z_rel_proj_rev + (dz_int_new - dz_own)
        end
        if (abs( z_rel_proj ) < abs( z_rel_proj_rev ))
            force_reversal = true
        elseif (0 < sense_own_prev_sign) && (0 < z_rel_proj)
            force_reversal = true
        elseif (sense_own_prev_sign < 0) && (z_rel_proj < 0)
            force_reversal = true
        else
            force_reversal = false
        end
    end
    return force_reversal::Bool
end
```

Referenced In: UpdateSA01HeuristicCState(p. 331)

3.3.2.6.10 Time-Based Non-Compliance Cost

TIMEBASEDNONCOMPLIANCECOST (Algorithm 310) penalizes persisting the current resolution advisory when either aircraft is not responding to RAs and the situation has continued to deteriorate. The necessity of these penalties is determined by the current RA cost state variable (*s_c.current_ra_cost*). A cost (*C_coc*) is also applied to clear of conflict whenever the current RA is penalized to prevent a transition to no alert.

Once per cycle, the state variables in *s_c* are set using the helper algorithm UPDATETIMEBASED-NONCOMPLIANCECSTATE.

This algorithm takes as input *mode_int*, *dz_min*, *dz_max*, *dz_min_prev*, *dz_max_prev*, *z_own_-ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *tau_expected_no_horizon*, *equip_int*, *resp_int*, *update*, and *s_c*. This algorithm returns *cost*.

Algorithm 310 TimeBasedNonComplianceCost

```
function TimeBasedNonComplianceCost( mode_int::Z, dz_min::R, dz_max::R, dz_min_prev::R, dz_max_prev::R,
                                     z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R,
                                     tau_expected_no_horizon::R, equip_int::Bool, resp_int::Bool,
                                     update::Bool, s_c::TimeBasedNonComplianceCState(p. 532),
                                     idx_online_cost::Z )
    C_coc::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.C_coc[idx_online_cost]
    C_ra_epsilon::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.C_ra_epsilon[idx_online_-
            cost]
    if update
        UpdateTimeBasedNonComplianceCState(p. 335)( mode_int, dz_min_prev, dz_max_prev, z_own_ave,
                                                    dz_own_ave, z_int_ave, dz_int_ave, tau_expected_no_horizon,
                                                    equip_int, resp_int, s_c, idx_online_cost )
    end
    cost::R = 0.0
    if (C_ra_epsilon < s_c.current_ra_cost)
        if (dz_min_prev == dz_min) && (dz_max_prev == dz_max)
            cost = s_c.current_ra_cost
        elseif IsCOC(p. 419)( dz_min, dz_max )
            cost = C_coc
        end
    end
    return cost::R
end
```

Referenced In: OnlineCoordinatedActionCostEstimation(p. 312)

UPDATETIMEBASEDNONCOMPLIANCECSTATE (Algorithm 311) is used to update the state variables of TIMEBASEDNONCOMPLIANCECOST once per cycle. The algorithm first calculates a factor (*factor_separation*) based on the expected separation at closest point of approach (*z_rel_actual*) and a set of thresholds to determine cost applicability. If an altitude crossing is expected, larger thresholds are used. Three values are then calculated to determine factors that reduce or increase the value of the cost applied:

- *current_ra_max_tau* - the largest tau value observed during the current RA
- *current_ra_min_tau_time* - the number of cycles that the tau value has been below *T_-threshold_min_tau* during the current RA

- *current_ra_dz_initial_delta* - the difference between the ownship vertical rate when the RA was issued and the target vertical rate of the current RA
  - this value is calculated using the helper algorithm FINDCOMPLIANTRATEDELTA

All of these values are reset each time the RA is clear of conflict or whenever the RA changes. Additionally the *current_ra_dz_initial_delta* value is bounded by the *R_vert_rel_compare_min* parameter to prevent the possibility of division by zero.

The own aircraft contribution to the non-compliance cost is calculated using DETERMINEOWNNONCOMPLIANCEFACTOR. It is combined with the separation factor (*factor_separation*) to determine the total cost to apply. The separation factor is also used in conjunction with the standard intruder response model calculated with INTRUDERRESPONSEESTIMATION. If the intruder is believed to be non-responsive and the separation appears to be insufficient, the intruder factor (*factor_int*) will encourage transitions to stronger RAs, especially if reversals are prohibited by coordination.

The greater of the two aircraft factors (*factor_own* and *factor_int*) is applied to the nominal cost *C_current_ra*. This ensures that the cost will be applied if either of the two aircraft is not responding and the separation appears to be insufficient. However, in the event that the current RA is clear of conflict or has a target rate of magnitude 2,000 fpm or higher, the cost is removed. This prevents initial alerts and undesirable transitions from the strongest possible RAs due to this cost.

This algorithm takes as input *mode_int*, *dz_min_prev*, *dz_max_prev*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *tau_expected_no_horizon*, *equip_int*, *resp_int*, and *s_c*. The algorithm updates *s_c*.

Algorithm 311 UpdateTimeBasedNonComplianceCState

```
function UpdateTimeBasedNonComplianceCState( mode_int::Z, dz_min_prev::R, dz_max_prev::R, z_own_ave::R,
                                    dz_own_ave::R, z_int_ave::R, dz_int_ave::R, tau_expected_no_horizon::R,
                                    equip_int::Bool, resp_int::Bool, s_c::TimeBasedNonComplianceCState(p. 532),
                                    idx_online_cost::Z )
    R_strengthen::R  = params().actions.strengthen_rate
    T_threshold_min_tau::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.T_threshold_min_tau[idx_-
            online_cost]
    R_vert_rel_compare_min::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.R_vert_rel_compare_-
            min[idx_online_cost]
    H_rel_threshold::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.H_rel_threshold[idx_-
            online_cost]
    H_rel_rolloff::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.H_rel_rolloff[idx_online_-
            cost]
    H_rel_threshold_cross::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.H_rel_threshold_-
            cross[idx_online_cost]
    H_rel_rolloff_cross::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.H_rel_rolloff_cross[idx_-
            online_cost]
    C_current_ra::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.C_current_ra[idx_online_-
            cost]
    dz_delta::R = FindCompliantRateDelta(p. 336)( dz_own_ave, dz_min_prev, dz_max_prev )
    z_rel_actual::R = (z_int_ave - z_own_ave) + ((dz_int_ave - dz_own_ave) * tau_expected_no_horizon)
    factor_separation::R = 0.0
    if IsProjectedCrossing(p. 426)( (z_int_ave - z_own_ave), z_rel_actual )
        factor_separation =
            CalculateThresholdRampDownFactor(p. 410)( abs(z_rel_actual), H_rel_threshold_cross,
                                                H_rel_rolloff_cross )
    else
        factor_separation =
            CalculateThresholdRampDownFactor(p. 410)( abs(z_rel_actual), H_rel_threshold, H_rel_rolloff )
    end
    if IsCOC(p. 419)( dz_min_prev, dz_max_prev ) ||
       (dz_min_prev != s_c.dz_min_prev2) ||
       (dz_max_prev != s_c.dz_max_prev2)
        s_c.current_ra_max_tau = tau_expected_no_horizon
        s_c.current_ra_min_tau_time = 0
        s_c.current_ra_dz_initial_delta = dz_delta
    else
        if (s_c.current_ra_max_tau < tau_expected_no_horizon)
            s_c.current_ra_max_tau = tau_expected_no_horizon
        end
        if (tau_expected_no_horizon < T_threshold_min_tau)
            s_c.current_ra_min_tau_time = s_c.current_ra_min_tau_time + 1
        end
    end
    if (s_c.current_ra_dz_initial_delta < R_vert_rel_compare_min)
        s_c.current_ra_dz_initial_delta = R_vert_rel_compare_min
    end
    factor_own::R = factor_separation *
        DetermineOwnNonComplianceFactor(p. 337)( mode_int, tau_expected_no_horizon, dz_delta, s_c,
            idx_online_cost )
    factor_int::R = 0.0
    if equip_int && !resp_int
        factor_int = factor_separation
    end
    s_c.current_ra_cost = C_current_ra * max( factor_own, factor_int )
    if (R_strengthen <= dz_min_prev) || (dz_max_prev <= -R_strengthen) || IsCOC(p. 419)( dz_min_prev,
         dz_max_prev )
        s_c.current_ra_cost = 0.0
    end
    s_c.dz_min_prev2 = dz_min_prev
    s_c.dz_max_prev2 = dz_max_prev
end
```

Referenced In: TimeBasedNonComplianceCost(p. 333)

FINDCOMPLIANTRATEDELTA (Algorithm 312) calculates the difference between the current own aircraft vertical rate and the vertical rate limits of the advisory. The advisory vertical rate limits are compared to the own aircraft vertical rate to determine which one should be used in the calculation. The calculated difference (*dz_delta*) is returned.

This algorithm takes as input *dz_own*, *dz_min*, and *dz_max*. This algorithm returns *dz_delta*.

Algorithm 312 FindCompliantRateDelta

```
function FindCompliantRateDelta( dz_own::R, dz_min::R, dz_max::R )
    dz_delta::R = 0.0
    if (dz_own < dz_min)
        dz_delta = dz_min - dz_own
    elseif (dz_max < dz_own)
        dz_delta = dz_own - dz_max
    end
    return dz_delta::R
end
```

Referenced In: UpdateTimeBasedNonComplianceCState(p. 335)

DETERMINEOWNNONCOMPLIANCEFACTOR (Algorithm 313) uses tau and own aircraft vertical rate to determine whether to apply the non-compliance cost. Three factors are calculated to determine the overall factor for own aircraft behavior *factor_own*. The *factor_tau_apply* ensures that the cost is only applied if *tau_expected_no_horizon* is low enough, i.e., when the aircraft are close to CPA (closest point of approach). The *factor_tau_diff* ensures that the cost is only applied if tau has decreased sufficiently and/or been below the minimum for enough time. The *factor_rate* ensures that the cost is only applied if the own aircraft vertical rate is not compliant with the target rate of the current RA. All three factors are combined and returned to UPDATETIMEBASEDNONCOMPLIANCECSTATE.

This algorithm takes as input *mode_int*, *tau_expected_no_horizon*, *dz_delta*, and *s_c*. This algorithm returns *factor_own*.

Algorithm 313 DetermineOwnNonComplianceFactor

```
function DetermineOwnNonComplianceFactor( mode_int::Z, tau_expected_no_horizon::R, dz_delta::R,
                                s_c::TimeBasedNonComplianceCState(p. 532), idx_online_cost::Z )
    T_threshold_apply::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.T_threshold_apply[idx_-
            online_cost]
    T_rolloff_apply::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.T_rolloff_apply[idx_-
            online_cost]
    T_threshold_tau_delta::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.T_threshold_tau_-
            delta[idx_online_cost]
    T_rolloff_tau_delta::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.T_rolloff_tau_delta[idx_-
            online_cost]
    X_vert_rel_factor_max::R =
        params().modes[mode_int].cost_estimation.online.time_based_non_compliance.X_vert_rel_factor_-
            max[idx_online_cost]
    factor_tau_apply::R =
        CalculateThresholdRampDownFactor(p. 410)( tau_expected_no_horizon, T_threshold_apply, T_rolloff_apply )
    tau_delta = s_c.current_ra_max_tau - tau_expected_no_horizon + s_c.current_ra_min_tau_time
    factor_tau_diff::R =
        CalculateThresholdRampUpFactor(p. 411)( tau_delta, T_threshold_tau_delta, T_rolloff_tau_delta )
    factor_rate::R = dz_delta / s_c.current_ra_dz_initial_delta
    if (X_vert_rel_factor_max < factor_rate)
        factor_rate = X_vert_rel_factor_max
    end
    factor_own::R = factor_tau_apply * factor_tau_diff * factor_rate
    return factor_own::R
end
```

Referenced In: UpdateTimeBasedNonComplianceCState(p. 335)

3.3.2.7 Individual Cost Estimation

INDIVIDUALCOSTESTIMATION (Algorithm 314) processes and combines the offline and online costs for an individual intruder aircraft. It takes as input the estimated offline and online costs, as well as additional inputs, such as own altitude above ground, shown collectively in Figure 12.

The estimated offline and online costs are combined using vector addition represented by "+" in Figure 12. The offline costs are combined with two distinct online cost vectors: *cost_online_ra* and *cost_online_ra_subset*. The index of the best action in *cost_online_ra_subset* is determined through a call to MINCOSTINDEX (Algorithm 409) and stored in *act_subset*. It is used to access the value of the best action, which is stored in *act_subset_value*.

The subset of costs (*cost_ra_subset*) is used to determine whether to apply an additional penalty (*C_force_alert*) on Clear of Conflict and preventive advisories to force a corrective advisory. This penalty is only applied if the current height of ownship (*height_own*) is above a threshold (*H_threshold_lower*) to prevent adverse interactions with ALTITUDEINHIBITCOST and COMPATIBILITYCOST. Additionally, different thresholds are used in the processing of *cost_ra_subset* as the height of ownship increases.

If the current RA is clear of conflict and the offline cost for clear of conflict is above a threshold (*c_coc_threshold*), the *ra_imminent* flag is set to TRUE. This allows INDIVIDUALCOSTESTIMATION to bypass the remaining checks that involve *cost_ra_subset* and ensures both Clear of Conflict and preventive RAs are always penalized by *C_force_alert*.

Even if *ra_imminent* is FALSE, the values from *cost_ra_subset* can still result in the penalty being applied. If the lowest cost action from the subset, *act_subset*, is not Clear of Conflict, *C_force_-alert* will be applied to Clear of Conflict. Furthermore, if the lowest cost action from the subset is not a preventive RA, both Clear of Conflict and all preventive RAs will be penalized by *C_-force_alert*. Finally, if the current RA is not Clear of Conflict and the value of the lowest cost from the subset (*act_subset_value*) is above *c_coc_threshold*, Clear of Conflict will be penalized by *C_-force_alert*.

To prevent the multi-threat level-off (MTLO) action from being selected for an individual threat, *C_restrict* is always applied to the MTLO action.

The vector of combined online and offline costs, *cost_ra*, is sent to the ACTIONSELECTION component.

This algorithm takes as input *mode_int*, *height_own*, *dz_own_ave*, *equipage_int*, *st_own*, *cost_-offline*, *cost_online_ra*, and *cost_online_ra_subset*. Descriptions of the INDIVIDUALCOSTESTIMATION output variables are found in Table 42.

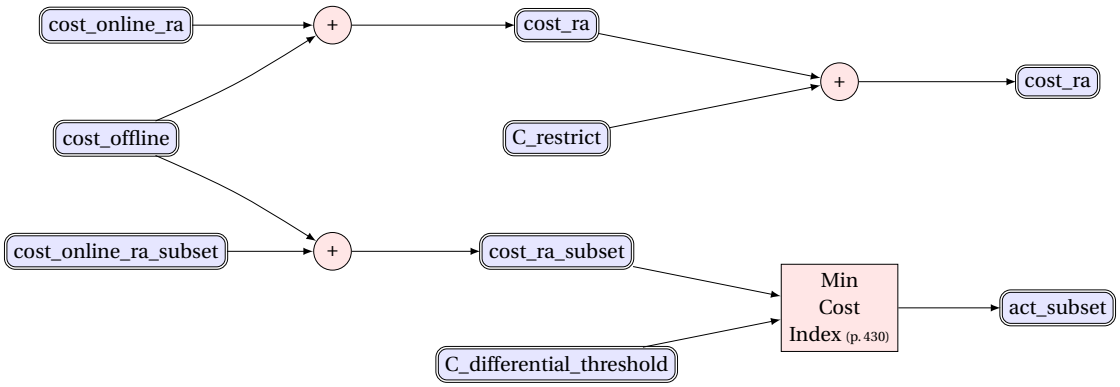

Figure 12. IndividualCostEstimation overview.

Table 42. Vertical TRM Algorithm Output Variables - IndividualCostEstimation

Variable	Units	Type	Description
cost_ra	N/A	real	Vector of total online cost of each action for single intruder; used to select RAs

Algorithm 314 IndividualCostEstimation

```
function IndividualCostEstimation( mode_int::Z, height_own::R, dz_own_ave::R, equipage_int::Z,
                        st_own::TRMOwnState(p. 511), cost_offline::Vector{R}, cost_online_ra::Vector{R},
                        cost_online_ra_subset::Vector{R}, idx_online_cost::Z )
    N_actions::Z                = params().actions.num_actions
    C_force_alert::R            = params().modes[mode_int].cost_estimation.online.force_alert.C_force_-
        alert[idx_online_cost]
    H_threshold_lower_equip::R  = params().modes[mode_int].cost_estimation.online.force_alert.fa_equip.H_-
        threshold_lower[idx_online_cost]
    H_threshold_lower_unequip::R= params().modes[mode_int].cost_estimation.online.force_alert.fa_unequip.H_-
        threshold_lower[idx_online_cost]
    H_threshold_upper::R        = params().modes[mode_int].cost_estimation.online.force_alert.H_threshold_-
        upper[idx_online_cost]
    C_restrict::R               = params().threat_resolution.C_restrict
    C_differential_threshold::R = params().threat_resolution.C_differential_threshold
    C_coc_threshold_upper::R    = params().modes[mode_int].cost_estimation.online.force_alert.C_coc_-
        threshold_upper[idx_online_cost]
    C_coc_threshold_lower::R    = params().modes[mode_int].cost_estimation.online.force_alert.C_coc_-
        threshold_lower[idx_online_cost]
    h_threshold_lower::R = 0.0
    if ((EQUIPAGE_NONE != equipage_int) && (EQUIPAGE_TCAS == equipage_int) && (EQUIPAGE_LARGE_CAS ==
        equipage_int))
        h_threshold_lower = H_threshold_lower_equip
    else
        h_threshold_lower = H_threshold_lower_unequip
    end
    cost_ra::Vector{R}        = cost_offline + cost_online_ra
    cost_ra_subset::Vector{R} = cost_offline + cost_online_ra_subset
    act_subset::Z       = MinCostIndex(p. 430)( cost_ra_subset, C_differential_threshold )
    act_subset_value::R = cost_ra_subset[act_subset]
    c_coc_threshold::R  = 0
    if (h_threshold_lower < height_own)
        if (H_threshold_upper < height_own)
            c_coc_threshold  = C_coc_threshold_upper
        else
            c_coc_threshold  = C_coc_threshold_lower
        end
        ra_imminent::Bool = false
        if (st_own.a_prev.action == COC) && (c_coc_threshold < cost_offline[COC])
            ra_imminent = true
        end
        if (act_subset != COC) || ra_imminent
            (dz_act_min::R, dz_act_max::R) =
                ActionToRates(p. 410)( act_subset, dz_own_ave, st_own.a_prev.action,
                                st_own.a_prev.dz_min, st_own.a_prev.dz_max, st_own.a_prev.ddz )
            if !IsPreventive(p. 426)( dz_act_min, dz_act_max ) || ra_imminent
                for act::Z in 1:N_actions
                    (dz_min::R, dz_max::R) =
                        ActionToRates(p. 410)( act, dz_own_ave, st_own.a_prev.action,
                                st_own.a_prev.dz_min, st_own.a_prev.dz_max, st_own.a_prev.ddz )
                    if IsWithinRateBounds(p. 429)( act, dz_own_ave ) && IsPreventive(p. 426)( dz_min, dz_max )
                        cost_ra[act] = cost_ra[act] + C_force_alert
                    end
                end
            end
            cost_ra[COC] = cost_ra[COC] + C_force_alert
        elseif (st_own.a_prev.action != COC) && (c_coc_threshold < act_subset_value)
            cost_ra[COC] = cost_ra[COC] + C_force_alert
        end
    end
    mtlo::Z = MTLOAction(p. 430)()
    cost_ra[mtlo] = cost_ra[mtlo] + C_restrict
    return (cost_ra::Vector{R})
end
```

Referenced In: StateAndCostEstimation(p. 251)

3.3.3 Vertical Action Selection

An overview of the ACTIONSELECTION algorithm of the Threat Resolution Module is shown in Figure 13.

ACTIONSELECTION (Algorithm 315) determines the global action for own aircraft along with individual actions for each intruder considered in isolation. It accepts a set of action costs from each of the intruders as well as additional inputs, such as the intruder altitude samples and associated weights.

Action selection is handled differently for the cases of a single intruder and multiple intruders.

DETERMINEMINIMUMCOSTACTION (Algorithm 317) determines the global action and individual action when there is a single intruder.

DETERMINEMULTIINTRUDERACTION (Algorithm 318) determines the global and individual actions when there are multiple intruders.

One or more intruders may have degraded surveillance that precludes them from being used to produce resolution advisories. In that case, the individual action for such an intruder is determined but is not considered when determining the global action. The *exclude_int* variable is used to indicate when an intruder should be excluded from consideration for the global action.

If no valid intruders are input into the TRM, NOINTRUDERSACTION is used to produce the appropriate action and vertical rate values in lieu of ACTIONSELECTION.

This algorithm takes as input *mode_int*, *cost_int*, *z_int_ave*, *dz_int_ave*, *z_own_ave*, *dz_own_ave*, *tau_int*, *equip_int*, *exclude_int*, *st_own*, and *st_int*. A description of the ACTIONSELECTION output variables appears in Table 43.

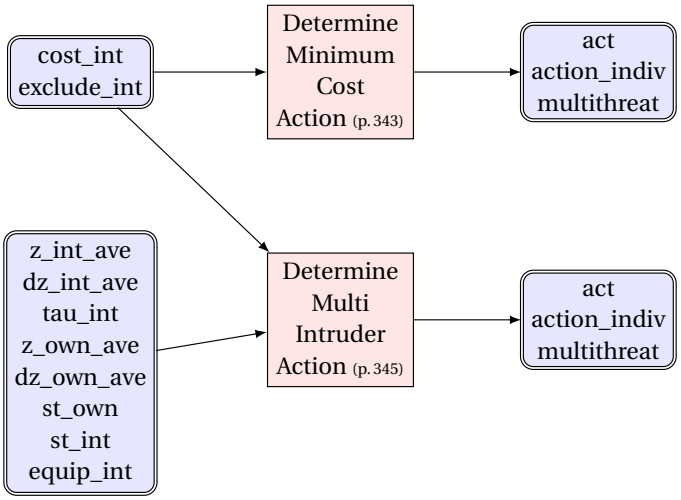

Figure 13. ActionSelection overview.

Table 43. Vertical TRM Algorithm Output Variables - ActionSelection

Variable	Units	Type	Description
act	N/A	integer	Single global resolution advisory for own aircraft
action_indiv	N/A	integer	Array of individual resolution advisories, one for each intruder
ddz	ft/s^2	real	Vertical acceleration compatible with global advisory
dz_max	ft/s	real	Maximum vertical rate compatible with global advisory
dz_min	ft/s	real	Minimum vertical rate compatible with global advisory
dz_own_ave_prev	ft/s	real	Estimated own vertical rate to be saved for next cycle
multithreat	N/A	bool	Whether or not there are multiple threats

Algorithm 315 ActionSelection

```
function ActionSelection( mode_int::Vector{Z}, cost_int::Matrix{R},
                z_int_ave::Vector{R}, dz_int_ave::Vector{R}, tau_int::Vector{R},
                z_own_ave::R, dz_own_ave::R, equip_int::Vector{Bool},
                exclude_int::Vector{Bool}, st_own::TRMOwnState(p.511),
                st_int::Vector{TRMIntruderState(p.512)} )
    N_intruders::Z = length( z_int_ave )
    dz_min::R = 0.0
    dz_max::R = 0.0
    ddz::R    = 0.0
    act::Z    = COC
    action_indiv::Vector{Z} = Z[]
    multithreat::Bool = false
    num_considered::Z = 0
    for i = 1:N_intruders
        if !exclude_int[i]
            num_considered = num_considered + 1
        end
    end
    if (num_considered <= 1)
        (act, action_indiv, multithreat) =
            DetermineMinimumCostAction(p.343)( cost_int, exclude_int )
        st_own.st_multithreat_cost_balancing = MultithreatCostBalancingCState(p.530)()
        st_own.st_arbitrate = ActionArbitrationGlobalCState(p.526)()
    else
        (act, action_indiv, multithreat) =
            DetermineMultiIntruderAction(p.345)( mode_int, cost_int, z_int_ave, dz_int_ave, tau_int,
                                        z_own_ave, dz_own_ave, st_own, st_int, equip_int,
                                        exclude_int )
    end
    (dz_min, dz_max, ddz) = ActionToRates(p.410)( act, dz_own_ave, st_own.a_prev.action,
                                         st_own.a_prev.dz_min, st_own.a_prev.dz_max, st_own.a_prev.ddz )
    dz_own_ave_prev::R = dz_own_ave
    return (act::Z, dz_min::R, dz_max::R, ddz::R, action_indiv::Vector{Z},
            dz_own_ave_prev::R, multithreat::Bool)
end
```

Referenced In: VerticalTRMUpdate(p.241)

NoIntrudersAction (Algorithm 316) is used when there are no valid intruders for TRM processing. It sets all necessary outputs and state variables to appropriate values for the condition when no intruders are valid.

This algorithm takes as input *dz_own_ave* and *st_own*. This algorithm returns *act*, *dz_min*, *dz_max*, *ddz*, and *multithreat*. The algorithm updates *st_own*.

Algorithm 316 NoIntrudersAction

```
function NoIntrudersAction( dz_own_ave::R, st_own::TRMOwnState(p. 511) )
    dz_min::R = 0.0
    dz_max::R = 0.0
    ddz::R    = 0.0
    act::Z    = COC
    multithreat::Bool = false
    (dz_min, dz_max, ddz) = ActionToRates(p. 410)( act, dz_own_ave, st_own.a_prev.action,
                                    st_own.a_prev.dz_min, st_own.a_prev.dz_max, st_own.a_prev.ddz )
    st_own.dz_ave_prev = dz_own_ave
    st_own.action_prev   = COC
    st_own.word_prev     = 0
    st_own.crossing_prev = false
    st_own.strength_prev = 0
    st_own.st_multithreat_cost_balancing = MultithreatCostBalancingCState(p. 530)()
    st_own.st_arbitrate = ActionArbitrationGlobalCState(p. 526)()
    return (act::Z, dz_min::R, dz_max::R, ddz::R, multithreat::Bool)
end
```

Referenced In: VerticalTRMUpdate(p. 241), SetVerticalTRMSurvOnlyMode(p. 247)

3.3.3.1 Single Intruder Rate Selection

For the single intruder case, determining the best action is performed by DETERMINEMINIMUMCOSTACTION (Algorithm 317), which determines the action with minimum cost for the single intruder that is valid as indicated by *exclude_int*. To ensure that the multi-threat level-off (MTLO) table entry is never used during single threat processing the MTLO action cost receives the penalty *C_restrict* prior to determining the minimum cost for each action. The minimum cost action is found through a call to MINCOSTINDEX (Algorithm 409). The global action corresponds to the individual action for the single valid intruder; the individual actions for invalid intruders are not used.

The *exclude_int* variable is used to indicate when an intruder, such as one with degraded surveillance, should be excluded as a threat and from consideration for the global action. There are no special logic considerations for designated intruders.

This algorithm takes as input *cost_int* and *exclude_int*. This algorithm returns *action*, *action_indiv*, and *multithreat*.

Algorithm 317 DetermineMinimumCostAction

```
function DetermineMinimumCostAction( cost_int::Matrix{R}, exclude_int::Vector{Bool} )
    C_restrict::R               = params().threat_resolution.C_restrict
    C_differential_threshold::R = params().threat_resolution.C_differential_threshold
    N_intruders::Z = length( exclude_int )
    mtlo_action::Z = MTLOAction(p. 430)()
    action_indiv::Vector{Z} = zeros( Z, N_intruders )
    action::Z = COC
    for i = 1:N_intruders
        cost_int[i, mtlo_action] = cost_int[i, mtlo_action] + C_restrict
        action_indiv[i] = MinCostIndex(p. 430)( cost_int[i,:], C_differential_threshold )
        if !exclude_int[i]
            action = action_indiv[i]
        end
    end
    multithreat::Bool = false
    return (action::Z, action_indiv::Vector{Z}, multithreat::Bool)
end
```

Referenced In: ActionSelection(p. 341)

3.3.3.2 Multiple Intruder Rate Selection

When there are multiple valid intruders, DETERMINEMULTIINTRUDERACTION (Algorithm 318) is used to determine the action to address all intruders. Because the effects of coordination between aircraft have a major role in multithreat encounters, action selection for multiple intruders treats intruders that are unequipped differently from those that are equipped. Since unequipped aircraft do not need to account for coordination effects, the unequipped intruders are treated together, independently from equipped intruders and the global action is then determined. In addition to selecting an action based on the individual actions, the DETERMINEMULTIINTRUDERACTION logic applies some online costs that are specific to multithreat encounters and dependent on multiple aircraft.

In INDIVIDUALSELECTIONCOSTESTIMATION (Algorithm 319), the individual actions for each intruder are determined individually after applying additional online costs.

Once the individual intruder actions and the number of unequipped threat intruders have been determined, the global action is determined. First the best resolution advisory (RA) that addresses all unequipped threats is determined in UNEQUIPPEDCOSTFUSION (Algorithm 321). In MTLODETERMINATION (Algorithm 322), individual intruder actions are evaluated to determine if a 'Multi-Threat Level-Off' (MTLO) advisory is necessary due to conflicting up and down actions for different intruders. If an MTLO action is not selected, the global RA is determined by arbitration in ACTIONARBITRATION (Algorithm 327) to be the strongest up or down sense advisory. PERSISTMTLO (Algorithm 411) is then used to determine whether to persist an MTLO advisory rather than initiate a 'Do Not Climb' or 'Do Not Descend' advisory.

Finally, if a global MTLO advisory is continuing, the previous individual intruder advisories are persisted to maintain consistency. When the global RA is a continuing MTLO advisory and a previous individual intruder advisory is a maintain, the individual advisory is converted to a 'Do Not Climb' or 'Do Not Descend' advisory. That conversion properly preserves the individual advisory sense through a subsequent level-off. All other individual intruder advisories are persisted without change.

One or more intruders may have degraded surveillance that requires processing for Surveillance only mode. In that case, the individual action for each such intruder is determined but is not considered when determining the global action. In a situation with multiple intruders, even if the individual action is something other than clear of conflict for the intruder undergoing degraded surveillance processing, that intruder will not be counted as a threat. In each of the algorithms mentioned above, the *exclude_int* variable is used to indicate when an intruder should be excluded as a threat and from consideration for the global action.

This algorithm takes as input *mode_int*, *cost_min*, *z_int_ave*, *dz_int_ave*, *tau_int*, *z_own_ave*, *dz_own_ave*, *st_own*, *st_int*, *equip_int*, and *exclude_int*. This algorithm returns *act*, *act_indiv*, and *multithreat*.

Algorithm 318 DetermineMultiIntruderAction

```
function DetermineMultiIntruderAction( mode_int::Vector{Z}, cost_int::Matrix{R},
                                z_int_ave::Vector{R}, dz_int_ave::Vector{R}, tau_int::Vector{R},
                                z_own_ave::R, dz_own_ave::R,
                                st_own::TRMOwnState(p.511), st_int::Vector{TRMIntruderState(p.512)},
                                equip_int::Vector{Bool}, exclude_int::Vector{Bool} )
    R_initial_ddz::R = params().actions.initial_acceleration
    N_intruders::Z   = length( st_int )
    num_unequipped::Z = 0
    for i = 1:N_intruders
        if !equip_int[i] && !exclude_int[i]
           num_unequipped = num_unequipped + 1
        end
    end
    (act_indiv::Vector{Z}, cost_indiv::Vector{R}, num_threats::Z, num_unequipped_threats::Z) =
          IndividualSelectionCostEstimation(p.346)( mode_int, cost_int, num_unequipped,
                                                    z_own_ave, dz_own_ave,
                                                    z_int_ave, dz_int_ave, tau_int,
                                                    equip_int, exclude_int, st_int )
    multithreat::Bool = (1 < num_threats)
    act_unequipped::Z = COC
    if (0 < num_unequipped)
        act_unequipped =
          UnequippedCostFusion(p.349)( act_indiv, cost_int, num_unequipped_threats,
                                       equip_int, exclude_int )
    end
    (issue_mtlo::Bool, arbitrate_conflicting_senses::Bool) =
          MTLODetermination(p.350)( mode_int, act_indiv, cost_indiv, act_unequipped,
                                    multithreat, z_own_ave, dz_own_ave, st_own, st_int,
                                    z_int_ave, dz_int_ave, equip_int, exclude_int )
    act::Z  = COC
    MTLO::Z = MTLOAction(p.430)()
    if !issue_mtlo
        act = ActionArbitration(p.355)( act_unequipped, act_indiv, cost_indiv,
                                        dz_own_ave, arbitrate_conflicting_senses,
                                        num_unequipped_threats, st_int, equip_int, exclude_int,
                                        st_own.st_arbitrate )
        if PersistMTLO(p.431)( act, multithreat, dz_own_ave, st_own.dz_ave_prev, st_own.a_prev )
            act = MTLO
            st_own.st_arbitrate = ActionArbitrationGlobalCState(p.526)()
        end
    else
        act = MTLO
        st_own.st_arbitrate = ActionArbitrationGlobalCState(p.526)()
    end
    if (MTLO == act) && (MTLO == st_own.a_prev.action)
        maintain_action::Z = RatesToAction(p.432)( NaN, NaN, NaN )
        for i = 1:N_intruders
            if (maintain_action == st_int[i].a_prev.action)
                if (0 <= dz_own_ave)
                    act_indiv[i] = RatesToAction(p.432)( 0.0, Inf, R_initial_ddz )
                else
                    act_indiv[i] = RatesToAction(p.432)( -Inf, 0.0, R_initial_ddz )
                end
            else
                act_indiv[i] = st_int[i].a_prev.action
            end
        end
    end
    return (act::Z, act_indiv::Vector{Z}, multithreat::Bool)
end
```

Referenced In: ActionSelection(p.341)

INDIVIDUALSELECTIONCOSTESTIMATION (Algorithm 319) determines the individual actions for each intruder individually after applying two additional online costs.

The first is a penalty to the MTLO action if a MTLO is not forced due to coordination.

The second is a penalty to prevent resolution advisories that may lead to an encounter where own aircraft is sandwiched between intruder aircraft. That is, one intruder aircraft is above own aircraft at the same time another intruder aircraft is below. At least one of the intruders must be unequipped for this penalty to be applied. SANDWICHPREVENTIONCOST (Algorithm 320) is used to determine to what actions the sandwich penalty should apply.

This algorithm takes as input *mode_int*, *cost_min*, *num_unequipped*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *tau_int*, *equip_int*, and *exclude_int*. This algorithm returns *act_indiv*, *cost_indiv*, *num_threats*, and *num_unequipped_threats*.

Algorithm 319 IndividualSelectionCostEstimation

```
function IndividualSelectionCostEstimation( mode_int::Vector{Z}, cost_int::Matrix{R}, num_unequipped::Z,
                                    z_own_ave::R, dz_own_ave::R,
                                    z_int_ave::Vector{R}, dz_int_ave::Vector{R}, tau_int::Vector{R},
                                    equip_int::Vector{Bool}, exclude_int::Vector{Bool},
                                    st_int::Vector{TRMIntruderState(p. 512)} )
    C_restrict::R               = params().threat_resolution.C_restrict
    C_differential_threshold::R = params().threat_resolution.C_differential_threshold
    N_intruders::Z  = length( z_int_ave )
    act_indiv::Vector{Z}  = zeros( Z, N_intruders )
    cost_indiv::Vector{R} = zeros( R, N_intruders )
    mtlo_action::Z = MTLOAction(p. 430)()
    num_threats::Z = 0
    num_unequipped_threats::Z = 0
    cost_coc::Vector{R} = cost_int[:,COC]
    for i = 1:N_intruders
        cost_int[i, mtlo_action] = cost_int[i, mtlo_action] + C_restrict
        has_unequipped_secondary::Bool = false
        if !equip_int[i]
            has_unequipped_secondary = (1 < num_unequipped)
        else
            has_unequipped_secondary = (0 < num_unequipped)
        end
        if has_unequipped_secondary
            inc_cost::Vector{R} =
                SandwichPreventionCost(p. 348)( i, mode_int[i], cost_coc, z_own_ave, dz_own_ave,
                                        z_int_ave, dz_int_ave, tau_int,
                                        equip_int, exclude_int, st_int[i].idx_online_cost )
            cost_int[i,:] = cost_int[i,:] + inc_cost[:]
        end
        act_indiv[i]  = MinCostIndex(p. 430)( cost_int[i,:], C_differential_threshold )
        cost_indiv[i] = cost_int[i,act_indiv[i]]
        if (act_indiv[i] != COC) && !exclude_int[i]
            num_threats = num_threats + 1
            if !equip_int[i]
                num_unequipped_threats = num_unequipped_threats + 1
            end
        end
    end
    return (act_indiv::Vector{Z}, cost_indiv::Vector{R}, num_threats::Z, num_unequipped_threats::Z)
end
```

Referenced In: DetermineMultiIntruderAction(p. 345)

3.3.3.2.1 Sandwich Prevention Cost for Unequipped Intruders

SANDWICHPREVENTIONCOST (Algorithm 320) helps to discourage actions towards a primary intruder when a secondary intruder will be in the same direction at the closest point of approach (CPA). To prevent unnecessary application, the cost is only applied when the risk that the secondary intruder poses, indicated by the clear of conflict (COC) cost of the secondary intruder (*cost_coc*), is above a threshold (*C_sandwich_prevention_coc_threshold*). The projected altitude of the primary intruder at CPA (*z_int_idx_at_tau*) and the projected altitude of each secondary intruder at CPA (*z_int_j_at_tau*) are used over current altitudes to account for the possibility that the primary and secondary intruder cross altitudes before CPA. Separate checks are made depending on the current location of the primary intruder. If the primary intruder is below the ownship and the secondary intruder will be below the primary intruder at CPA, the *penalize_downsense* variable is set to TRUE. If the primary intruder is above the ownship and the secondary intruder will be above the primary intruder at CPA, the *penalize_upsense* variable is set to TRUE.

If only one of the penalize flags (*penalize_downsense*, *penalize_upsense*) is TRUE, the corresponding sense RAs will be penalized by *C_sandwich*. Equipped and unequipped aircraft are penalized by different amounts due to the effects of coordination costs and other equipage dependent costs. Both preventive and maintain advisories are excluded from the penalty. If both penalize flags are TRUE (or FALSE) no penalty will be applied. A vector containing the costs to be applied to the actions that should be penalized is updated and returned.

The *exclude_int* variable is used to omit intruders with degraded surveillance from the checks.

This algorithm takes as input *intruder_idx*, *mode_int*, *cost_coc*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *tau_int*, *equip_int*, and *exclude_int*. This algorithm returns *cost*.

Algorithm 320 SandwichPreventionCost

```
function SandwichPreventionCost( intruder_idx::Z, mode_int::Z, cost_coc::Vector{R},
                        z_own_ave::R, dz_own_ave::R, z_int_ave::Vector{R}, dz_int_ave::Vector{R},
                        tau_int::Vector{R}, equip_int::Vector{Bool}, exclude_int::Vector{Bool},
                        idx_online_cost::Z )
    N_actions::Z       = params().actions.num_actions
    C_coc_threshold::R = params().modes[mode_int].cost_estimation.online.sandwich_prevention.C_coc_-
         threshold[idx_online_cost]
    c_sandwich::R  = params().modes[mode_int].cost_estimation.online.sandwich_prevention.C_sandwich_-
         unequip[idx_online_cost]
    if equip_int[intruder_idx]
        c_sandwich = params().modes[mode_int].cost_estimation.online.sandwich_prevention.C_sandwich_-
             equip[idx_online_cost]
    end
    cost::Vector{R} = zeros( R, N_actions )
    z_int_idx_at_tau::R = z_int_ave[intruder_idx] + (dz_int_ave[intruder_idx] * tau_int[intruder_idx])
    penalize_upsense::Bool   = false
    penalize_downsense::Bool = false
    for j = 1:length( cost_coc )
        if (j != intruder_idx) && !equip_int[j] && (C_coc_threshold < cost_coc[j]) && !exclude_int[j]
            z_int_j_at_tau::R = z_int_ave[j] + (dz_int_ave[j] * tau_int[intruder_idx])
            if (z_int_j_at_tau < z_int_idx_at_tau) && (z_int_ave[j] <= z_own_ave)
                penalize_downsense = true
            elseif (z_int_idx_at_tau <= z_int_j_at_tau) && (z_own_ave < z_int_ave[j])
                penalize_upsense = true
            end
        end
    end
    if (penalize_upsense && !penalize_downsense) || (!penalize_upsense && penalize_downsense)
        for act = 1:N_actions
            (dz_min::R, dz_max::R) = ActionToRates(p. 410)( act, dz_own_ave, -1, NaN, NaN, NaN )
            sense_own::Symbol = RatesToSense(p. 432)( dz_min, dz_max )
            if !IsPreventive(p. 426)( dz_min, dz_max) && !IsMaintain(p. 424)( dz_min, dz_max ) &&
               ((penalize_upsense && (:Up == sense_own)) || (penalize_downsense && (:Down == sense_own)))
                cost[act] = c_sandwich
            end
        end
    end
    return cost::Vector{R}
end
```

Referenced In: IndividualSelectionCostEstimation(p. 346)

3.3.3.2.2 Cost Fusion for Unequipped Intruders

UNEQUIPPEDCOSTFUSION (Algorithm 321) determines the optimal action that attempts to address all unequipped intruders. The individual actions are first found by taking the lowest cost action for each unequipped intruder. An intruder is a threat if the optimal action toward that intruder in isolation is not clear of conflict (COC).

The costs from the individual unequipped intruders are fused by taking the worst cost (maximum) over all unequipped intruders for every action (*q_star*) to give each element of *fused_cost*. If there is more than one unequipped threat, Clear of Conflict is penalized by *C_restrict* to ensure an alert is always issued. To prevent the selection of a multi-threat level-off (MTLO) advisory during cost fusion, a penalty (*C_restrict*) is applied to the MTLO for all intruders. Although this prevents the MTLO advisory from being selected during the fusion process, MTLODETERMINATION (Algorithm 322) can override this effect. The single action for all unequipped intruders is determined by finding the best action over the new *fused_cost* vector through a call to MIN-

COSTINDEX (Algorithm 409).

The *exclude_int* variable is used to omit intruders with degraded surveillance from the checks.

This algorithm takes as input *act_indiv*, *cost_int*, *num_unequipped_threats*, *equip_int*, and *exclude_int*. This algorithm returns *action*.

Algorithm 321 UnequippedCostFusion

```
function UnequippedCostFusion( act_indiv::Vector{Z}, cost_int::Matrix{R}, num_unequipped_threats::Z,
                      equip_int::Vector{Bool}, exclude_int::Vector{Bool} )
    C_restrict::R                   = params().threat_resolution.C_restrict
    C_differential_threshold::R     = params().threat_resolution.C_differential_threshold
    N_actions::Z                    = params().actions.num_actions
    mtlo_action::Z = MTLOAction(p.430)()
    fused_cost::Vector{R} = zeros( R, N_actions )
    for act = 1:N_actions
        q_star::R = -Inf
        for j = 1:length( act_indiv )
            q::R = cost_int[j,act]
            if !equip_int[j] && ((0 == num_unequipped_threats) || (act_indiv[j] != COC)) && !exclude_int[j]
                q_star = max( q_star, q )
            end
        end
        fused_cost[act] = q_star
    end
    if (1 < num_unequipped_threats)
        fused_cost[COC] = fused_cost[COC] + C_restrict
    end
    fused_cost[mtlo_action] = fused_cost[mtlo_action] + C_restrict
    action::Z = MinCostIndex(p.430)( fused_cost, C_differential_threshold )
    return (action::Z)
end
```

Referenced In: DetermineMultiIntruderAction(p.345)

3.3.3.2.3 Multi Threat Level Off (MTLO) Determination

MTLODETERMINATION (Algorithm 322) determines if a multi-threat level-off (MTLO) can or should be issued under the current conditions. The issuance of the MTLO advisory is limited in a number of situations where the issuance can lead to undesirable behaviors.

If PERSISTMTLO (Algorithm 411) determines that a MTLO action should continue from the previous cycle, the *issue_mtlo* flag is set to TRUE.

DETERMINESENSES (Algorithm 323) provides the sense of the action the logic has selected for each intruder independently for use in subsequent algorithms. MULTITHREATCOSTBALANCING (Algorithm 326) sets *costs_allows_mtlo* to TRUE if it determines the individual costs for each intruder would support the selection of MTLO. While the conclusion reached in PERSISTMTLO can make the output of MULTITHREATCOSTBALANCING irrelevant, it must be called on each cycle to keep the values in *st_own.st_multithreat_cost_balancing* up to date.

Two other checks are performed to determine if MTLO should be issued. DETERMINECONFLICTINGSENSES (Algorithm 324) sets *conflicting_senses* to TRUE if the senses selected for individual intruders conflict in sense. ALLOWUNEQUIPPEDMTLO (Algorithm 325) sets *geometry_allows_mtlo* to TRUE if the geometry of unequipped intruders and the presence of equipped intruders

would support the selection of MTLO. The three flags (*costs_allows_mtlo*, *conflicting_senses*, and *geometry_allows_mtlo*) must all be TRUE for *issue_mtlo* to be TRUE. Additionally, *arbitrate_conflicting_senses* is set as a direct copy of *conflicting_senses* for use in ACTIONARBITRATION (Algorithm 327).

The *exclude_int* variable is used to omit intruders with degraded surveillance from the checks.

This algorithm takes as input *mode_int*, *act_indiv*, *cost_indiv*, *act_unequipped*, *multithreat*, *z_own_ave*, *dz_own_ave*, *st_own*, *z_int_ave*, *dz_int_ave*, *equip_int*, and *exclude_int*. This algorithm returns *issue_mtlo* and *arbitrate_conflicting_senses*.

Algorithm 322 MTLODetermination

```
function MTLODetermination( mode_int::Vector{Z}, act_indiv::Vector{Z}, cost_indiv::Vector{R},
                   act_unequipped::Z, multithreat::Bool,
                   z_own_ave::R, dz_own_ave::R, st_own::TRMOwnState(p. 511),
                   st_int::Vector{TRMIntruderState(p. 512)},
                   z_int_ave::Vector{R}, dz_int_ave::Vector{R},
                   equip_int::Vector{Bool}, exclude_int::Vector{Bool} )
    issue_mtlo::Bool = false
    arbitrate_conflicting_senses::Bool = false
    costs_allow_mtlo::Bool = true
    if PersistMTLO(p. 431)( act_unequipped, multithreat, dz_own_ave, dz_own_ave, st_own.a_prev )
        issue_mtlo = true
    end
    senses_own_indiv::Vector{Symbol} = DetermineSenses(p. 351)( act_indiv, dz_own_ave, st_own.a_prev )
    costs_allow_mtlo = MultithreatCostBalancing(p. 354)( act_indiv, cost_indiv, senses_own_indiv, exclude_int,
                                                st_own.st_multithreat_cost_balancing, st_int )
    if !issue_mtlo
        conflicting_senses::Bool   = DetermineConflictingSenses(p. 351)( senses_own_indiv, exclude_int )
        geometry_allows_mtlo::Bool = AllowUnequippedMTLO(p. 352)( mode_int, z_own_ave, dz_own_ave,
                                                          z_int_ave, dz_int_ave, senses_own_indiv,
                                                          equip_int, exclude_int, st_int )
        arbitrate_conflicting_senses = conflicting_senses
        issue_mtlo = (conflicting_senses && geometry_allows_mtlo && costs_allow_mtlo)
    end
    return (issue_mtlo::Bool, arbitrate_conflicting_senses::Bool)
end
```

Referenced In: DetermineMultiIntruderAction(p. 345)

DETERMINESENSES (Algorithm 323) provides the sense of each action in the input array. Each element of *action_indiv* contains an action index that can be used to look up the vertical rate limits (*dz_min*, *dz_max*) of each action. An explicit check for *NaN* is required to identify Maintain RAs. If the Maintain RA is was also selected on the previous cycle, the vertical rate limits from the previous cycle are used. Otherwise, a call to MAINTAINRATES (Algorithm 408) sets the vertical rate limits appropriately.

The sense of each action is determined by invoking RATESTOSENSE (Algorithm 413) with the vertical rate limits (*dz_min* and *dz_max*).

This algorithm takes as input *action_indiv*, *dz_own_ave*, and *a_global*. This algorithm returns *senses_own_indiv*.

Algorithm 323 DetermineSenses

```
function DetermineSenses( action_indiv::Vector{Z}, dz_own_ave::R, a_global::GlobalAdvisory(p. 517) )
    R_dz_min_adv::Vector{R} = params().actions.min_rates
    R_dz_max_adv::Vector{R} = params().actions.max_rates
    N_intruders::Z = length( action_indiv )
    senses_own_indiv::Vector{Symbol} = Vector{Symbol}(undef, N_intruders)
    for i = 1:N_intruders
        dz_min::R = 0.0
        dz_max::R = 0.0
        act::Z = action_indiv[i]
        if isnan( R_dz_min_adv[act] ) || isnan( R_dz_max_adv[act] )
            if (a_global.action == act)
                dz_min = a_global.dz_min
                dz_max = a_global.dz_max
            else
                (dz_min, dz_max) = MaintainRates(p. 429)( dz_own_ave )
            end
        else
            dz_min = R_dz_min_adv[act]
            dz_max = R_dz_max_adv[act]
        end
        senses_own_indiv[i] = RatesToSense(p. 432)( dz_min, dz_max )
    end
    return senses_own_indiv::Vector{Symbol}
end
```

Referenced In: MTLODetermination(p. 350)

DETERMINECONFLICTINGSENSES (Algorithm 324) is used to determine if the actions selected for individual intruders conflict in sense. This is done by looping over the sense of the action selected for each intruder by ownship and setting the appropriate flag if the sense is up (*up_sense_issued*) or down (*down_sense_issued*). If both up and down sense RAs have been selected across the set of threats, *conflicting_senses* is set to TRUE.

The *exclude_int* variable is used to omit intruders with degraded surveillance from the checks.

This algorithm takes as input *senses_own_indiv* and *exclude_int*. This algorithm returns *conflicting_senses*.

Algorithm 324 DetermineConflictingSenses

```
function DetermineConflictingSenses( senses_own_indiv::Vector{Symbol}, exclude_int::Vector{Bool} )
    up_sense_issued::Bool   = false
    down_sense_issued::Bool = false
    for i = 1:length( senses_own_indiv )
        if !exclude_int[i]
            if (senses_own_indiv[i] == :Up)
                up_sense_issued = true
            elseif (senses_own_indiv[i] == :Down)
                down_sense_issued = true
            end
        end
    end
    conflicting_senses::Bool = up_sense_issued && down_sense_issued
    return conflicting_senses::Bool
end
```

Referenced In: MTLODetermination(p. 350)

ALLOWUNEQUIPPEDMTLO (Algorithm 325) determines if the geometry of unequipped intruders or the presence of an equipped intruder would allow the selection of MTLO. The thresholds used in the geometric considerations (*H_threshold* and *R_threshold*) are mode dependent and can be different for each intruder. Any intruder that did not have an action selected during single-threat processing is omitted from the checks. If any intruder is equipped, ALLOWUNEQUIPPEDMTLO will always consider MTLO to be an allowable action. The remaining intruders are subjected to the geometric tests. In order for MTLO to be permitted, an intruder being checked must have more than *H_threshold* vertical separation or be diverging vertical at a rate greater than *R_threshold*. If any intruder does not meet these requirements, *allow_mtlo* is set to FALSE.

The *exclude_int* variable is used to omit intruders with degraded surveillance from the checks.

This algorithm takes as input *mode_int*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *dz_int_ave*, *senses_own_indiv*, *equip_int*, and *exclude_int*. This algorithm returns *allow_mtlo*.

Algorithm 325 AllowUnequippedMTLO

```
function AllowUnequippedMTLO( mode_int::Vector{Z}, z_own_ave::R, dz_own_ave::R,
                      z_int_ave::Vector{R}, dz_int_ave::Vector{R},
                      senses_own_indiv::Vector{Symbol}, equip_int::Vector{Bool},
                      exclude_int::Vector{Bool},
                      st_int::Vector{TRMIntruderState(p. 512)} )
    N_intruders::Z   = length( mode_int )
    H_threshold::Vector{R} = zeros( R, N_intruders )
    R_threshold::Vector{R} = zeros( R, N_intruders )
    for i = 1:N_intruders
        H_threshold[i] = params().modes[mode_int[i]].cost_estimation.online.unequipped_mtlo.H_threshold[st_-
             int[i].idx_online_cost]
        R_threshold[i] = params().modes[mode_int[i]].cost_estimation.online.unequipped_mtlo.R_threshold[st_-
             int[i].idx_online_cost]
    end
    allow_mtlo::Bool = true
    equipped_intruder::Bool = false
    for i = 1:N_intruders
        if  (:None != senses_own_indiv[i]) && !exclude_int[i]
            z_rel::R  = z_own_ave  - z_int_ave[i]
            dz_rel::R = dz_own_ave - dz_int_ave[i]
            if equip_int[i]
                equipped_intruder = true
            elseif (abs( z_rel ) < H_threshold[i])
                if (z_rel < 0) && (-R_threshold[i] < dz_rel)
                    allow_mtlo = false
                elseif (0 < z_rel) && (dz_rel < R_threshold[i])
                    allow_mtlo = false
                end
            end
        end
    end
    if equipped_intruder
        allow_mtlo = true
    end
    return allow_mtlo::Bool
end
```

Referenced In: MTLODetermination(p. 350)

3.3.3.2.4 Balancing Intruder Costs

MULTITHREATCOSTBALANCING (Algorithm 326) compares costs between intruders and allows multi-threat level-off (MTLO) as a global action if the threat level of each intruder is within a tolerance.

Because the cost values for each action can be interpreted to represent the relative threat of an intruder, comparing the costs for the actions selected for each individual intruder allows a rough determination of the relative threat for each intruder. When there are multiple actions selected that have large differences in costs and the selected actions have conflicting senses, it is an indicator that the MTLO that would be issued may not adequately reduce risk for one of the threats. In this undesirable situation, the MULTITHREATCOSTBALANCING algorithm compares costs between intruders and allows the prohibition of MTLO as a global action until the threat level of each intruder, as indicated by the individual action cost, is within a tolerance. The tolerance is initially given by *C_differential_threshold_lo* but when the costs were not balanced previously, *C_differential_threshold_hi* adds hysteresis to the tolerance. When the costs of both intruders are larger than *C_absolute_threshold_hi*, the up and down sense actions have become critical. If both up and down sense actions are becoming critical, MTLO is not allowed even if the costs are balanced. Once this condition arises, *C_absolute_threshold_lo* is used for hysteresis.

The *exclude_int* variable is used to indicate when an intruder should be excluded as a threat and from consideration for the global action.

This algorithm takes as input *act_indiv*, *cost_indiv*, *senses_own_indiv*, *exclude_int*, and *s_c*. This algorithm returns *cost_allow_mtlo*. This algorithm updates *s_c*.

Algorithm 326 MultithreatCostBalancing

```
function MultithreatCostBalancing( act_indiv::Vector{Z}, cost_indiv::Vector{R},
                           senses_own_indiv::Vector{Symbol}, exclude_int::Vector{Bool},
                           s_c::MultithreatCostBalancingCState(p. 530), st_int::Vector{TRMIntruderState(p. 512)} )
    C_differential_threshold_hi::R = params().threat_resolution.balance_costs.C_differential_threshold_hi
    C_differential_threshold_lo::R = params().threat_resolution.balance_costs.C_differential_threshold_lo
    C_absolute_threshold_hi::Vector{R} = params().threat_resolution.balance_costs.C_absolute_threshold_hi
    C_absolute_threshold_lo::Vector{R} = params().threat_resolution.balance_costs.C_absolute_threshold_lo
    N_intruders::Z   = length( act_indiv )
    costs_allow_mtlo::Bool = true
    diff_max::R            = 0.0
    diff_set::Bool         = false
    above_abs_threshold_hi::Bool = false
    above_abs_threshold_lo::Bool = false
    for i = 1:(N_intruders-1)
        idx_online_cost_int1::Z = st_int[i].idx_online_cost
        C_absolute_threshold_hi_int1::R = C_absolute_threshold_hi[idx_online_cost_int1]
        C_absolute_threshold_lo_int1::R = C_absolute_threshold_lo[idx_online_cost_int1]
        if (COC != act_indiv[i]) && !exclude_int[i]
            for j = (i+1):N_intruders
                idx_online_cost_int2::Z = st_int[j].idx_online_cost
                C_absolute_threshold_hi_int2::R = C_absolute_threshold_hi[idx_online_cost_int2]
                C_absolute_threshold_lo_int2::R = C_absolute_threshold_lo[idx_online_cost_int2]
                if (COC != act_indiv[j]) && (senses_own_indiv[j] != senses_own_indiv[i]) && !exclude_int[j]
                    diff_max = max( diff_max, abs( cost_indiv[i] - cost_indiv[j] ) )
                    diff_set = true
                    if (C_absolute_threshold_hi_int1 < abs( cost_indiv[i] )) &&
                       (C_absolute_threshold_hi_int2 < abs( cost_indiv[j] ))
                        above_abs_threshold_hi = true
                    end
                    if (C_absolute_threshold_lo_int1 < abs( cost_indiv[i] )) &&
                       (C_absolute_threshold_lo_int2 < abs( cost_indiv[j] ))
                        above_abs_threshold_lo = true
                    end
                end
            end
        end
    end
    if above_abs_threshold_hi && !s_c.above_abs_threshold_prev
        costs_allow_mtlo             = false
        s_c.costs_allow_mtlo_prev    = true
        s_c.above_abs_threshold_prev = true
    elseif above_abs_threshold_lo && s_c.above_abs_threshold_prev
        costs_allow_mtlo             = false
        s_c.costs_allow_mtlo_prev    = true
        s_c.above_abs_threshold_prev = true
    elseif diff_set
        if s_c.costs_allow_mtlo_prev && (C_differential_threshold_hi <= diff_max)
            costs_allow_mtlo = false
        elseif !s_c.costs_allow_mtlo_prev && (C_differential_threshold_lo <= diff_max)
            costs_allow_mtlo = false
        else
            costs_allow_mtlo = true
        end
        s_c.costs_allow_mtlo_prev    = costs_allow_mtlo
        s_c.above_abs_threshold_prev = false
    else
        costs_allow_mtlo             = true
        s_c.costs_allow_mtlo_prev    = costs_allow_mtlo
        s_c.above_abs_threshold_prev = false
    end
    return costs_allow_mtlo::Bool
end
```

Referenced In: MTLODetermination(p. 350)

3.3.3.2.5 Action Arbitration

ACTIONARBITRATION (Algorithm 327) is utilized to select the global action when a multi-threat level-off (MTLO) advisory is not issued and there are multiple threats.

ARBITRATECONFLICTINGSENSES (Algorithm 328) is used for the special cases where MTLO was prohibited despite the presence of multiple individual actions with conflicting senses. The input flag (*arbitrate_conflicting_senses*) that enables this processing is determined by MTLODETERMINATION (Algorithm 322) through a call to DETERMINECONFLICTINGSENSES (Algorithm 324).

ARBITRATEMATCHINGSENSES (Algorithm 329) is used when there are multiple actions that have a common sense. In this case, the online cost state variables associated with ARBITRATECONFLICTINGSENSES in ACTIONARBITRATIONGLOBALCSTATE must be reset.

The *exclude_int* variable is used to indicate when an intruder should be excluded as a threat and from consideration for the global action.

This algorithm takes as input *act_unequipped*, *act_indiv*, *cost_indiv*, *dz_own_ave*, *arbitrate_conflicting_senses*, *num_unequipped_threats*, *st_int*, *equip_int*, *exclude_int*, and *s_c*. This algorithm returns *act*. This algorithm updates *s_c*.

Algorithm 327 ActionArbitration

```
function ActionArbitration( act_unequipped::Z, act_indiv::Vector{Z}, cost_indiv::Vector{R}, dz_own_ave::R,
                   arbitrate_conflicting_senses::Bool, num_unequipped_threats::Z,
                   st_int::Vector{TRMIntruderState(p. 512)}, equip_int::Vector{Bool}, exclude_int::Vector{Bool},
                   s_c::ActionArbitrationGlobalCState(p. 526) )
    act::Z = COC
    if arbitrate_conflicting_senses
        act = ArbitrateConflictingSenses(p. 357)( act_indiv, cost_indiv, st_int, exclude_int, s_c )
    else
        s_c.mtlo_prohibited_prev   = false
        s_c.worst_case_cost_prev   = -Inf
        s_c.worst_case_action_prev = COC
        act = ArbitrateMatchingSenses(p. 358)( act_unequipped, act_indiv, dz_own_ave, num_unequipped_threats,
                                       equip_int, st_int, exclude_int )
    end
    return (act::Z)
end
```

Referenced In: DetermineMultiIntruderAction(p. 345)

ARBITRATECONFLICTINGSENSES (Algorithm 328) is utilized to select the global action when a multi-threat level-off (MTLO) advisory is not issued and there are multiple threats. In this case the actions selected for each of the multiple threats have conflicting senses.

For the special cases where MTLO was prohibited despite the presence of multiple individual actions with conflicting senses, action arbitration amounts to selecting the action that has the highest cost (indicating that the corresponding intruder is the highest threat).

In order to keep the action selection from changing every cycle, there must be a compelling reason to change from the previous action. A new worst case is selected if the new worst case cost is above *C_differential_worst_case_threshold*. The possibility of a change in action is allowed only if conflicting senses were arbitrated on the last cycle and a different intruder has the worst

case cost this cycle.

There are two online cost state data structures associated with this cost. ACTIONARBITRATION-GLOBALCSTATE stores information about the previous worst case action. ACTIONARBITRATIONC-STATE stores state information for the individual intruder in *st_int*. Because the list of intruders can change from one cycle to the next, the intruder with the worst case action for this cycle is flagged using *st_int.st_arbitrate.was_worst_case*.

The *exclude_int* variable is used to indicate when an intruder should be excluded as a threat and from consideration for the global action.

This algorithm takes as input *act_indiv*, *cost_indiv*, *st_int*, *exclude_int*, and *s_c*. This algorithm returns *act*. This algorithm updates *s_c*.

Algorithm 328 ArbitrateConflictingSenses

```
function ArbitrateConflictingSenses( act_indiv::Vector{Z}, cost_indiv::Vector{R},
                                     st_int::Vector{TRMIntruderState(p. 512)}, exclude_int::Vector{Bool},
                                     s_c::ActionArbitrationGlobalCState(p. 526) )
    C_differential_threshold::R =
      params().threat_resolution.C_differential_threshold
    C_differential_worst_case_threshold::R =
      params().threat_resolution.action_arbitration.C_differential_worst_case_threshold
    act::Z = COC
    worst_case_cost::R     = -Inf
    worst_case_idx::Z      = 0
    worst_case_idx_prev::Z = 0
    for i = 1:length( cost_indiv )
        if !exclude_int[i]
            if (worst_case_cost < (cost_indiv[i] - C_differential_threshold))
                act             = act_indiv[i]
                worst_case_cost = cost_indiv[i]
                worst_case_idx  = i
            end
            if s_c.mtlo_prohibited_prev && st_int[i].st_arbitrate.was_worst_case
               worst_case_idx_prev = i
            end
        end
        st_int[i].st_arbitrate.was_worst_case = false
    end
    if (0 == worst_case_idx_prev)
        s_c.mtlo_prohibited_prev = false
    end
    if s_c.mtlo_prohibited_prev && (worst_case_idx != worst_case_idx_prev)
        if (worst_case_cost <= (s_c.worst_case_cost_prev + C_differential_worst_case_threshold)) ||
           (worst_case_cost <= (cost_indiv[worst_case_idx_prev] + C_differential_worst_case_threshold))
            worst_case_idx = worst_case_idx_prev
            act = s_c.worst_case_action_prev
        end
    end
    if (0 < worst_case_idx)
        st_int[worst_case_idx].st_arbitrate.was_worst_case = true
        s_c.mtlo_prohibited_prev   = true
        s_c.worst_case_cost_prev   = cost_indiv[worst_case_idx]
        s_c.worst_case_action_prev = act
    else
        s_c.mtlo_prohibited_prev   = false
        s_c.worst_case_cost_prev   = -Inf
        s_c.worst_case_action_prev = COC
    end
    return (act::Z)
end
```

Referenced In: ActionArbitration(p. 355)

ARBITRATEMATCHINGSENSES (Algorithm 329) is utilized to select the global action when a multi-threat level-off (MTLO) advisory is not issued and there are multiple threats. In this case the actions selected for each of the multiple threats have a common sense. This algorithm selects the strongest individual action being considered as the global action. As an example, if the best actions for two intruders are Climb at the *R_corrective* rate and Climb at the *R_strengthen* rate, respectively, the global action will be selected to be Climb at the *R_strengthen* rate. In the case where two different actions have the same vertical rate limits (*dz_min* and *dz_max*), the acceleration (*ddz*) is used as a tie-breaker. For example, initial Climb at the *R_corrective* rate and subsequent Climb at the *R_corrective* rate have the same vertical rate limits, but different accelerations. The subsequent *R_corrective* Climb RAs have a higher acceleration and are selected for

the global action.

The initial best action (*act*) is set as Clear of Conflict. If any unequipped threats exists, the unequipped action is used as the initial *act*.

The vertical rate limits associated with the action for each valid, equipped intruder are determined with a call to ACTIONTORATES (Algorithm 373). The vertical rate limits of the new action are compared to the current best action. If the vertical rate limits of the new action are more restrictive (i.e., permit a smaller range of vertical rates) than the current best action, the best action is replaced with the new action.

The ACTIONARBITRATIONCSTATE online cost state structure (*st_int.st_arbitrate*) stores advisory information for the individual intruder. It is updated after each comparison to ensure consistent vertical rate limits are used for Maintain actions.

The *exclude_int* variable is used to indicate when an intruder should be excluded as a threat and from consideration for the global action.

This algorithm takes as input *act_unequipped*, *act_indiv*, *dz_own_ave*, *num_unequipped_threats*, *equip_int*, *st_int*, and *exclude_int*. This algorithm returns *act*. The algorithm outputs updates to *st_int*.

Algorithm 329 ArbitrateMatchingSenses

```
function ArbitrateMatchingSenses( act_unequipped::Z, act_indiv::Vector{Z}, dz_own_ave::R,
                        num_unequipped_threats::Z, equip_int::Vector{Bool},
                        st_int::Vector{TRMIntruderState(p.512)}, exclude_int::Vector{Bool} )
    act::Z = COC
    if (0 < num_unequipped_threats)
        act = act_unequipped
    end
    (dz_min_best::R, dz_max_best::R, ddz_best::R) = ActionToRates(p.410)( act, dz_own_ave, -1, NaN, NaN, NaN )
    for i = 1:length( st_int )
        if equip_int[i] && !exclude_int[i]
            (dz_min::R, dz_max::R, ddz::R) = ActionToRates(p.410)( act_indiv[i], dz_own_ave,
                                                        st_int[i].st_arbitrate.action,
                                                        st_int[i].st_arbitrate.dz_min,
                                                        st_int[i].st_arbitrate.dz_max,
                                                        st_int[i].st_arbitrate.ddz )
            if (dz_min_best < dz_min) || (dz_max < dz_max_best) ||
               ((dz_min_best == dz_min) && (dz_max == dz_max_best) && (ddz_best < ddz))
               dz_min_best = dz_min
               dz_max_best = dz_max
               ddz_best    = ddz
               act = act_indiv[i]
            end
            if (act_indiv[i] != st_int[i].st_arbitrate.action)
                st_int[i].st_arbitrate.action = act_indiv[i]
                (st_int[i].st_arbitrate.dz_min, st_int[i].st_arbitrate.dz_max,
                 st_int[i].st_arbitrate.ddz) =
                      ActionToRates(p.410)( act_indiv[i], dz_own_ave, -1, NaN, NaN, NaN )
            end
        end
    end
    return (act::Z)
end
```

Referenced In: ActionArbitration(p.355)

3.3.4 Vertical Coordination Selection

COORDINATIONSELECTION (Algorithm 330) returns the coordination message settings (*vrc*, *cvc*, *vsb*) to be sent to the intruder as well as the vertical sense of the advisory (*sense_indiv*). The coordination message settings are generated using the individual and global advisory outputs from ACTIONSELECTION, together with ownship and intruder state information. When there is a global resolution advisory (RA), COORDINATIONSELECTION uses the optimal action that would be issued if the intruder were considered in isolation.

The global RA is given by *act_global*, *dz_min_global*, *dz_max_global*, and *ddz_global*. The optimal individual action is given by *act_indiv*. The previous global and individual advisories are given by *a_global_prev* and *a_indiv_prev*, respectively.

The algorithm first determines what individual vertical rates to use for generating the coordination message. ACTIONTORATES is used to obtain the vertical rates associated with *act_indiv*, as described in the following bulleted list. Adjustments to the individual vertical rates are needed when the global advisory is clear of conflict (COC) or a Maintain advisory.

- There is no global RA, (*act_global* is *COC*). Any advisory for the individual intruder is cleared; the vertical rates associated with COC are used.
- The global RA on the previous cycle was a Maintain advisory. When the individual advisory is also a Maintain advisory, the same vertical rates as the global Maintain advisory are used. Otherwise, the vertical rates associated with the individual advisory are used.
- The global RA on the current cycle is a Maintain advisory. When the individual advisory is also a Maintain advisory, the same vertical rates as the global Maintain advisory are used. Otherwise, the vertical rates associated with the individual advisory are used.
- The global advisory is something other than COC or a Maintain advisory. The vertical rates associated with the individual advisory are used.

Next, CROSSLINK is called to get the values of *vrc*, *cvc*, *vsb*, and *sense_indiv* based on the individual vertical rates, previous coordination message sent to this intruder (*a_indiv_prev.vrc* and *a_indiv_prev.cvc*), and the equipage of this intruder (*equipage_int*).

Finally, after CROSSLINK is run, the current values are recorded to be used as the previous values when CROSSLINK is run in the next cycle. The individual action (*a_indiv_prev.action*) on the previous cycle is used to record whether an RA was issued on the previous cycle (*a_indiv_prev.ra_prev*).

This algorithm takes as input *act_indiv*, *act_global*, *dz_min_global*, *dz_max_global*, *ddz_global*, *dz_own_ave*, *a_global_prev*, *a_indiv_prev*, and *equipage_int*.

A description of the output variables is found in Table 44.

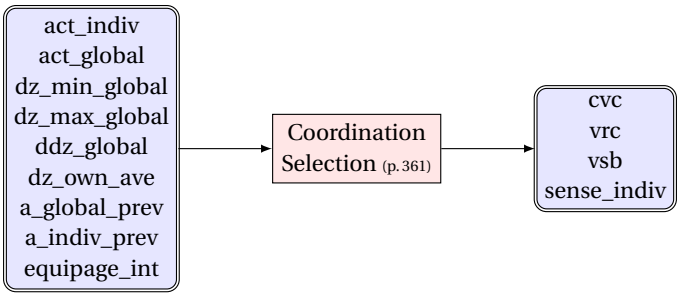

Figure 14. CoordinationSelection overview.

Table 44. Vertical TRM Algorithm Output Variables - CoordinationSelection

Variable	Units	Type	Description
cvc	N/A	uint32	Cancel Vertical Resolution Advisory Complement
vrc	N/A	uint32	Vertical Resolution Advisory Complement
vsb	N/A	uint32	Vertical Sense Bits
sense_indiv	N/A	Symbol	Resolution sense (up, down, none) for this intruder

Algorithm 330 CoordinationSelection

```
function CoordinationSelection( act_indiv::Z, act_global::Z, dz_min_global::R, dz_max_global::R,
                        ddz_global::R, dz_own_ave::R, a_global_prev::GlobalAdvisory(p. 517),
                        a_indiv_prev::IndividualAdvisory(p. 519), equipage_int::Z )
    dz_min_indiv::R = -Inf
    dz_max_indiv::R = Inf
    ddz_indiv::R    = 0.0
    if (COC == act_global)
        (dz_min_indiv, dz_max_indiv, ddz_indiv) = (dz_min_global, dz_max_global, 0.0)
    elseif IsMaintain(p. 424)( a_global_prev.dz_min, a_global_prev.dz_max )
        (dz_min_indiv, dz_max_indiv, ddz_indiv) =
           ActionToRates(p. 410)( act_indiv, dz_own_ave, a_indiv_prev.action,
                           a_global_prev.dz_min, a_global_prev.dz_max, ddz_global )
    elseif IsMaintain(p. 424)( dz_min_global, dz_max_global )
        (dz_min_indiv, dz_max_indiv, ddz_indiv) =
           ActionToRates(p. 410)( act_indiv, dz_own_ave, a_indiv_prev.action,
                           dz_min_global, dz_max_global, ddz_global )
    else
        (dz_min_indiv, dz_max_indiv, ddz_indiv) =
           ActionToRates(p. 410)( act_indiv, dz_own_ave, -1, NaN, NaN, NaN )
    end
    (cvc::UInt32, vrc::UInt32, vsb::UInt32, sense_indiv::Symbol) =
        Crosslink(p. 362)( dz_min_indiv, dz_max_indiv, a_indiv_prev.vrc, a_indiv_prev.cvc, equipage_int )
    if (COC == a_indiv_prev.action)
        a_indiv_prev.ra_prev = false
    else
        a_indiv_prev.ra_prev = true
    end
    a_indiv_prev.action = act_indiv
    a_indiv_prev.dz_min = dz_min_indiv
    a_indiv_prev.dz_max = dz_max_indiv
    a_indiv_prev.ddz    = ddz_indiv
    a_indiv_prev.sense  = sense_indiv
    a_indiv_prev.vrc    = vrc
    a_indiv_prev.cvc    = cvc
    return (cvc::UInt32, vrc::UInt32, vsb::UInt32, sense_indiv::Symbol)
end
```

Referenced In: VerticalTRMUpdate(p. 241), SetVerticalTRMSurvOnlyMode(p. 247)

CROSSLINK (Algorithm 331) selects the appropriate values for *vrc* (Vertical Resolution Advisory Complement), *cvc* (Cancel Vertical Resolution Advisory Complement), *vsb* (Vertical Sense Bits), and the individual sense *sense_indiv*. CROSSLINK makes no determination about whether VRCs need to be sent, but includes the information needed to make the decision. The responsibility rests with the V2V capability data transmission device. CROSSLINK accepts the current individual advisory, expressed as vertical rates (*dz_min_indiv* and *dz_max_indiv*), the VRC and CVC of the previous coordination message issued (*vrc_prev* and *cvc_prev*), and the equipage of the intruder (*equipage_int*). CROSSLINK calls RATESTOSENSE with the vertical rates to obtain the sense of the current individual resolution advisory (*sense_indiv*). Next, *sense_indiv* is used to determine *vrc* and *cvc*. The VRC transmitted to the intruder on the previous cycle is used by VRCTOSENSE to determine the sense of the RA for the previous cycle. That sense is used for determining *cvc*.

CROSSLINK calls ENCODEVRC (Algorithm 333) to determine *vrc*, then ENCODECVC (Algorithm 332) to determine *cvc*. It calls ENCODEVSB (Algorithm 334) with those results to determine *vsb*. Those three values are used to encode the coordination message.

Downstream processing will ensure that no coordination message is sent to the intruder if *cvc* = *0* and *vrc* = *0*.

If the intruder is not capable of receiving resolution advisory coordination messages, the values of *vrc*, *cvc*, and *vsb* will be 0. Intruders that are equipped with ACAS sXu are all considered capable of receiving coordination messages.

This algorithm takes as input *dz_min_indiv*, *dz_max_indiv*, *vrc_prev*, *cvc_prev*, and *equipage_-int*. This algorithm returns *cvc*, *vrc*, *vsb* and *sense_indiv*.

Algorithm 331 Crosslink

```
function Crosslink( dz_min_indiv::R, dz_max_indiv::R, vrc_prev::UInt32, cvc_prev::UInt32, equipage_int::Z )
    sense_indiv::Symbol = RatesToSense(p. 432)( dz_min_indiv, dz_max_indiv )
    vrc::UInt32 = EncodeVRC(p. 364)( sense_indiv, equipage_int )
    cvc::UInt32 = EncodeCVC(p. 363)( sense_indiv, VRCToSense(p. 436)( vrc_prev ), cvc_prev, equipage_int )
    vsb::UInt32 = EncodeVSB(p. 364)( vrc, cvc )
    return (cvc::UInt32, vrc::UInt32, vsb::UInt32, sense_indiv::Symbol)
end
```

Referenced In: SetInvalidIntruder(p. 395), CoordinationSelection(p. 361), SetDroppedIntruder(p. 394)

ENCODECVC (Algorithm 332) outputs the setting for the Cancel Vertical Resolution Advisory Complement (CVC) value in the outgoing coordination message to an individual intruder. It persists the CVC from one cycle to the next, until there is no longer an advisory, to account for possible loss of messages. When there is a clear of conflict, the CVC cancelling the previous Vertical Resolution Advisory Complement (VRC) will be sent only once.

If the individual intruder is not equipped to receive collision avoidance coordination messages, as determined from *equipage_int*, then *cvc* is set to *0*. An individual intruder is considered equipped if it is equipped with ACAS sXu.

If the individual intruder is equipped to receive coordination messages, the sense of the current individual resolution advisory (RA) (*sense_indiv*) and the sense of the individual RA on the previous cycle (*sense_indiv_prev*) are examined to see if there has been a change. The value of *sense_indiv_prev* is derived from the VRC sent to this intruder on the previous cycle. A sense of *:None* indicates no advisory. A *sense_indiv* value of *:None* indicates clear of conflict when following an RA, indicated by a *sense_indiv_prev* value of *:Up* or *:Down*.

For an intruder equipped to receive coordination messages, the value of *cvc* is set as follows:

- The sense is *:None* for both; there is no RA to be cancelled. Therefore, *cvc* is set to *0*.
- A down sense RA cleared; the previous VRC is cancelled. Therefore, *cvc* is set to *1*.
- An up sense RA cleared; the previous VRC is cancelled. Therefore, *cvc* is set to *2*.
- There is no change in the sense; the previous value of Therefore, *cvc* is persisted.
- There is a reversal from a down sense RA to an up sense RA; the previous VRC is cancelled. Therefore, *cvc* is set to *1*.
- There is a reversal from an up sense RA to a down sense RA; the previous VRC is cancelled. Therefore, *cvc* is set to *2*.
- This is an initial RA, either up or down sense; there is no RA to be cancelled. Therefore, *cvc* is set to *0*.

This algorithm takes as input *sense_indiv*, *sense_indiv_prev*, *cvc_prev*, and *equipage_int*. This algorithm returns *cvc*.

Algorithm 332 EncodeCVC

```
function EncodeCVC( sense_indiv::Symbol, sense_indiv_prev::Symbol, cvc_prev::UInt32, equipage_int::Z )
    cvc::UInt32 = 0
    if (equipage_int != EQUIPAGE_CASRA)   &&
       (equipage_int != EQUIPAGE_CASTA)   &&
       (equipage_int != EQUIPAGE_CASRESP) &&
       (equipage_int != EQUIPAGE_DAARESP)
        cvc = 0
    elseif (sense_indiv == :None) && (sense_indiv_prev == :None)
        cvc = 0
    elseif (sense_indiv == :None) && (sense_indiv_prev == :Down)
        cvc = 1
    elseif (sense_indiv == :None) && (sense_indiv_prev == :Up)
        cvc = 2
    elseif (sense_indiv == sense_indiv_prev)
        cvc = cvc_prev
    elseif (sense_indiv == :Up) && (sense_indiv_prev == :Down)
        cvc = 1
    elseif (sense_indiv == :Down) && (sense_indiv_prev == :Up)
        cvc = 2
    else
        cvc = 0
    end
    return cvc::UInt32
end
```

Referenced In: Crosslink(p. 362)

ENCODEVRC (Algorithm 333) outputs the setting for the Vertical Resolution Advisory Complement (VRC) value in the outgoing coordination message to an individual intruder.

If the individual intruder is not equipped to receive collision avoidance coordination messages, as determined from *equipage_int*, then *vrc* is set to *0*. An individual intruder is considered equipped if it is equipped with a TCAS system, ACAS X system, Collision Avoidance System (CAS) with Responsive capability, or Detect And Avoid (DAA) Responsive system.

If the individual intruder is equipped to receive coordination messages, the sense of the current individual resolution advisory (RA) (*sense_indiv*) is used to determine what value to give *vrc*. When the sense of the individual RA is down, *vrc* is set to *1* indicating 'Do not pass below.' When the sense of the individual RA is up, *vrc* is set to *2* indicating 'Do not pass above.' A sense of *:None* indicates no advisory and *vrc* is set to *0* indicating no intent.

This algorithm takes as input *sense_indiv* and *equipage_int*. This algorithm returns *vrc*.

Algorithm 333 EncodeVRC

```
function EncodeVRC( sense_indiv::Symbol, equipage_int::Z )
    vrc::UInt32 = 0
    if (equipage_int != EQUIPAGE_CASRA)   &&
       (equipage_int != EQUIPAGE_CASTA)   &&
       (equipage_int != EQUIPAGE_CASRESP) &&
       (equipage_int != EQUIPAGE_DAARESP)
        vrc = 0
    elseif (sense_indiv == :Up)
        vrc = 2
    elseif (sense_indiv == :Down)
        vrc = 1
    else
        vrc = 0
    end
    return vrc::UInt32
end
```

Referenced In: UpdateIntruderInputs(p. 307), Crosslink(p. 362)

ENCODEVSB (Algorithm 334) outputs the setting for the Vertical Sense Bits (VSB) value in the outgoing coordination message to an individual intruder.

The *vsb* field is a parity field used to protect the *vrc* and *cvc* fields. The value of *vsb* is encoded directly from *vrc* and *cvc* using a parity table parameter (*X_parity_table*).

When *vrc* and *cvc* are both *0*, *vsb* will be *0* as well.

This algorithm takes as input *vrc* and *cvc*. This algorithm returns *vsb*.

Algorithm 334 EncodeVSB

```
function EncodeVSB( vrc::UInt32, cvc::UInt32 )
    X_parity_table::Vector{Z} = params().coordination.parity_table_vert
    vsb::UInt32 = UInt32( X_parity_table[ (cvc * 4) + vrc + 1 ] )
    return vsb::UInt32
end
```

Referenced In: Crosslink(p. 362)

3.3.5 Vertical Display Logic

An overview of the DISPLAYLOGICDETERMINATION algorithm of the Threat Resolution Module is shown in Figure 15.

DISPLAYLOGICDETERMINATION (Algorithm 335) converts the action and rates for the current advisory into parameters that drive the pilot display, including the visual indicators and aural alarms.

DETERMINECROSSING (Algorithm 336) is used to determine if the global advisory is a crossing advisory and to set the crossing flag appropriately.

DETERMINEDISPLAYDATA (Algorithm 337) is used to determine the Label 270 code and target rate for the global advisory given own aircraft altitude and vertical rate, the advisory state, and

the crossing flag.

DISPLAYLOGICDETERMINATION takes as input *h_own*, *z_own_ave*, *dz_own_ave*, *z_int_ave*, *action*, *dz_min*, *dz_max*, *ddz_min*, *sense_int*, *code_int*, *st_own*, *source_int*, *ground_alert*, and *opmode*, and *trm_altmode*.

The outputs are summarized in Table 45.

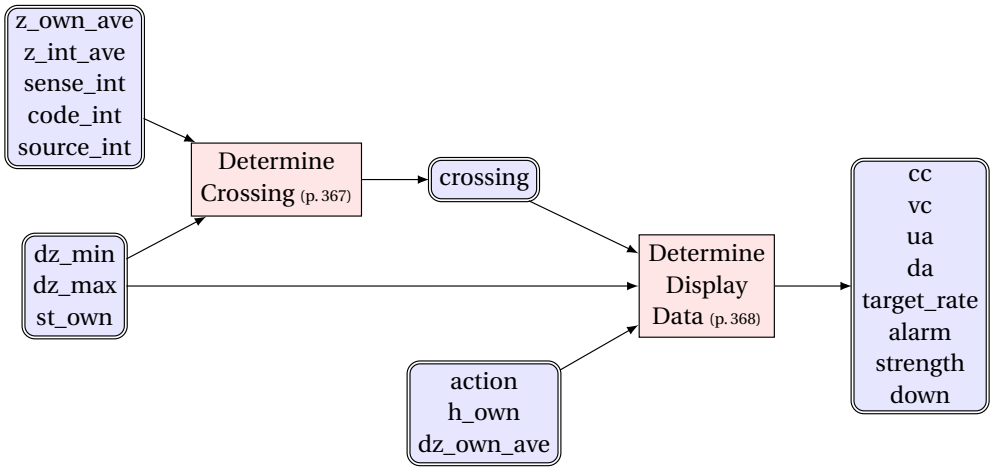

Figure 15. DisplayLogicDetermination overview.

Table 45. Vertical TRM Algorithm Output Content - DisplayLogicDetermination

Variable	Units	Type	Description
alarm	N/A	bool	Resolution Advisory changed
cc	N/A	uint8	Label270 Combined Control
crossing	N/A	bool	Resolution Advisory will result in a crossing
da	N/A	uint8	Label270 Down Advisory
ddz_min	ft/s^2	real	Compliant acceleration
down	N/A	bool	Vertical ARA Down flag
dz_max	ft/min	real	Maximum vertical rate
dz_min	ft/min	real	Minimum vertical rate
ground_alert	N/A	bool	Ground alert type
opmode	N/A	uint8	Ownship vertical operational mode
strength	N/A	uint8	Vertical ARA strength code
target_rate	N/A	real	Target rate
trm_altmode	N/A	uint8	TRM altitude type operational mode
ua	N/A	uint8	Label270 Up Advisory
vc	N/A	uint8	Label270 Vertical Control

Algorithm 335 DisplayLogicDetermination

```
function DisplayLogicDetermination( h_own::R, z_own_ave::R, dz_own_ave::R, z_int_ave::Vector{R},
                            action::Z, dz_min::R, dz_max::R, ddz::R, sense_int::Vector{Symbol},
                            code_int::Vector{UInt8}, st_own::TRMOwnState(p.511),
                            source_int::Vector{Z}, ground_alert::UInt8, opmode::UInt8, trm_altmode::UInt8 )
    crossing::Bool =
          DetermineCrossing(p.367)( dz_min, dz_max, z_own_ave, z_int_ave, sense_int, code_int, st_own,
               source_int )
    (cc::UInt8, vc::UInt8, ua::UInt8, da::UInt8, target_rate::R, alarm::Bool,
     strength::UInt8, down::Bool) =
          DetermineDisplayData(p.368)( action, crossing, dz_min, dz_max, h_own, dz_own_ave, st_own )
    fps_to_fpm::R = 60.0
    display_logic::DisplayLogic(p.516) =
          DisplayLogic(p.516)( crossing, cc, vc, ua, da, target_rate, alarm, ground_alert, strength, down,
               dz_min * fps_to_fpm, dz_max * fps_to_fpm, ddz, opmode, trm_altmode )
    return display_logic::DisplayLogic(p.516)
end
```

Referenced In: VerticalTRMUpdate(p.241)

3.3.5.1 Determine Crossing

DETERMINECROSSING (Algorithm 336) predicts whether there will be an altitude crossing with an intruder against which there is an active resolution advisory. The resulting crossing flag is used in certain situations to adjust the aural advisory annunciation. The crossing flag is also output in messages to the ground station.

DETERMINECROSSING sets the crossing flag (*is_crossing*) when it determines the global resolution advisory (RA) is a crossing advisory. For this check, an intruder is only considered if there is an active resolution advisory against it, as indicated by *code_int* having a value of *SXUCODE_RA* for that intruder. Furthermore, only those intruders with an individual RA sense (*sense_int*) in the same direction as the global sense (*sense_own*) can lead to the crossing flag being set. When there is a single threat, the sense of the RA for the individual threat will always be the same as the sense of the global RA. If the the global RA is a Multi-Threat Level-Off (MTLO), this check is considered passed as MTLO RAs are neither up sense nor down sense.

DETERMINECROSSING then checks if previousely a crossing flag was issued to change *thresh_-high* to FALSE. If previously the crossing flag was not issued, then the high threashold values are extracted from the parameter file. ORNCT surveillance is specifically given a higher threshold value as defined by the parameter file. In addition, if the threat is above and the individual RA sense is up, or if the threat is below and the individual RA sense is down. If the individual RA directs ownship towards the threat and the vertical separation is greater than *h_cross_thresh* feet, then a crossing is indicated. Whenever the separation is less than or equal to *h_cross_thresh* feet, no crossing is indicated, regardless of the individual RA sense. *h_cross_thresh* is a vertical threshold with hysteresis. If there was a crossing on the previous cycle, the minimum vertical separation for a crossing will be lower (*H_cross_thresh_lo* instead of *H_cross_thresh_hi*). This hysteresis allows the crossing flag to remain set until after the altitudes have crossed.

The crossing flag is used to determine the appropriate aural advisory and may be encoded in the "vertical control" field of Label 270. It is used, without modification, to set the crossing bit in the output to the ground station.

This algorithm takes as input *dz_min*, *dz_max*, *z_own_ave*, *z_int_ave*, *sense_int*, *code_int*, *st_own*

and *source_int*. This algorithm returns *is_crossing*. This algorithm updates *st_own*.

Algorithm 336 DetermineCrossing

```
function DetermineCrossing( dz_min::R, dz_max::R, z_own_ave::R, z_int_ave::Vector{R},
                            sense_int::Vector{Symbol}, code_int::Vector{UInt8}, st_own::TRMOwnState(p. 511),
                            source_int::Vector{Z} )
    H_ORNCTcrossthrhi    = params().display.ORNCTcrossthrhi
    H_crossthrhi        = params().display.crossthrhi
    H_ORNCTcrossthrlo    = params().display.ORNCTcrossthrlo
    H_crossthrlo        = params().display.crossthrlo
    is_crossing::Bool = false
    sense_own::Symbol = RatesToSense(p. 432)( dz_min, dz_max )
    h_cross_thresh::R = 0
    thresh_high::Bool = true
    if st_own.crossing_prev
        thresh_high = false
    end
    for i = 1:length( code_int )
        if (code_int[i] == SXUCODE_RA) &&
           ((sense_int[i] == sense_own) || IsMTLO(p. 425)( dz_min, dz_max ))
            if thresh_high
                if (source_int[i] == SOURCE_ORNCT)
                    h_cross_thresh = H_ORNCTcrossthrhi
                else
                    h_cross_thresh = H_crossthrhi
                end
            else
                if (source_int[i] == SOURCE_ORNCT)
                    h_cross_thresh = H_ORNCTcrossthrlo
                else
                    h_cross_thresh = H_crossthrlo
                end
            end
            if (sense_int[i] == :Up) && (h_cross_thresh < (z_int_ave[i] - z_own_ave))
                is_crossing = true
            elseif (sense_int[i] == :Down) && (h_cross_thresh < (z_own_ave - z_int_ave[i]))
                is_crossing = true
            end
        end
    end
    st_own.crossing_prev = is_crossing
    return is_crossing::Bool
end
```

Referenced In: DisplayLogicDetermination(p. 366)

3.3.5.2 Determine Display Data

DETERMINEDISPLAYDATA (Algorithm 337) produces the values needed to display the global resolution advisory to the pilot. In addition, it produces the strength and up values needed to convey the global resolution advisory to the ground station.

DETERMINEDISPLAYDATA first determines if an altitude inhibit should be in effect. When own aircraft is at or below *H_descend_thr*, it may be necessary to modify the annunciation, and therefore the Label 270 word. For example, when own aircraft is below 60' above ground level and transitioning from an increase descend or descend resolution advisory (RA) to a Do Not Climb RA, the annunciation should be 'Monitor Vertical Speed' (Label 270 word 6002) instead of 'Level-Off, Level-Off' (Label 270 word 5002), as would be customary. In this case, a Label 270 rule can

be constructed to produce the desired behavior based on the value of *alt_inhibit* being TRUE.

GETMATCHINGLABEL270RULE (Algorithm 338) is then called to obtain the Label 270 values (*cc*, *vc*, *ua*, and *da*), the *force_alarm* flag, and the *strength* and *up* values associated with the inputs. A Label 270 word is formed by combining *cc*, *vc*, *ua*, and *da*. For example, if *cc* is *5*, *vc* is *0*, *ua* is *0*, and *da* is *2*, the Label 270 word is *5002*.

The *alarm* flag indicates whether or not there should be an alert annunciation. Whenever the RA changes, as denoted by a change in action, there will be an annunciation.

In some cases, a change in the RA should not be annunciated, such as the transition between an initial Climb at the *R_corrective* rate and a subsequent Climb at the *R_corrective* rate. Because they are two different actions, the logic will set the *alarm* flag to TRUE. However, since these two Climb actions produce the same Label 270 word value (4010 or 4110) and the same aural message, there should be no re-annunciation. In other cases, a change in the Label 270 word within an action, that would not otherwise be annunciated, should be annunciated because the aural message has changed. An example of the latter is the transition between 6002 (Monitor Vertical Speed) and 5002 (Level-Off, Level-Off). The *force_alarm* variable is used to provide this finer tuning to the setting of *alarm*.

The target rate (ft/s) (*target_rate*) is derived from the vertical rate range for the global RA. Whenever the vertical rates for the global RA indicate a level-off or a clear of conflict, the target rate will be 0.0 ft/s. The target rate, when converted to ft/min, may appear on the pilot display as an arrow depicting the vertical advisory rate to maintain.

This algorithm takes as input *action*, *crossing*, *dz_min*, *dz_max*, *h_own*, *dz_own_ave*, and *st_own*. This algorithm returns *cc*, *vc*, *ua*, *da*, *target_rate*, *alarm*, *strength*, and *down*.

Algorithm 337 DetermineDisplayData

```
function DetermineDisplayData( action::Z, crossing::Bool, dz_min::R, dz_max::R,
                      h_own::R, dz_own_ave::R, st_own::TRMOwnState(p. 511) )
    H_descend_thr::R = params().display.hdescendthr
    action_prev::Z    = st_own.action_prev
    alt_inhibit::Bool = false
    if !isnan( h_own ) && (h_own <= H_descend_thr)
        alt_inhibit = true
    end
    (cc::UInt8, vc::UInt8, ua::UInt8, da::UInt8, force_alarm::Z, strength::UInt8, down::Bool) =
        GetMatchingLabel270Rule(p. 370)( action, crossing, alt_inhibit, dz_own_ave,
                                  action_prev, st_own.word_prev, st_own.strength_prev )
    alarm::Bool = false
    if !IsCOC(p. 419)( dz_min, dz_max ) && (FORCE_ALARM_OFF != force_alarm) &&
       ((action_prev != action) || (FORCE_ALARM_ON == force_alarm))
        alarm = true
    end
    target_rate::R = min( abs( dz_min ), abs( dz_max ) )
    if ( abs( dz_min ) == abs( dz_max ) )
        target_rate = 0.0
    elseif (RatesToSense(p. 432)( dz_min, dz_max ) == :Down)
        target_rate = -target_rate
    end
    return (cc::UInt8, vc::UInt8, ua::UInt8, da::UInt8, target_rate::R, alarm::Bool,
            strength::UInt8, down::Bool)
end
```

Referenced In: DisplayLogicDetermination(p. 366)

GETMATCHINGLABEL270RULE (Algorithm 338) returns the cc, vc, ua, da, strength, and up values associated with the global resolution advisory. It also returns a flag indicating if an annunciation should be forced or suppressed.

The current advisory and dynamic state for comparison are specified using the following inputs: the global action for the current cycle (*action*), whether there is a crossing (*crossing*), whether altitude inhibit is activated (*alt_inhibit*), the current vertical rate of ownship (ft/s) (*dz_own_ave*), the global action on the previous cycle (*action_prev*), the Label 270 code on the previous cycle (*word_prev*), and the RA strength code on the previous cycle (*strength_prev*).

The parameters file encodes a set of rules for mapping the current action, previous action, previous Label 270 word, ownship vertical rate, crossing, and altitude inhibit status into the various display outputs. The rules are represented as arrays for the various input and output variables.

The logic rules are specified using the following parameters: a global action for the current cycle (*L_act_in*), a global action for the previous cycle (*L_act_in_prev*), a Label 270 code for the previous cycle (*L_word_in_prev*), a lower bound on ownship vertical rate (ft/s) (*R_dz_lo_in*), an upper bound on ownship vertical rate (ft/s) (*R_dz_hi_in*), a crossing dependency indication (*L_crossing_in*), an altitude inhibit dependency indication (*L_alt_inhibit_in*), and an RA strength code for the previous cycle (*L_strength_in_prev*).

Each logic rule also contains the results from a match specified by the following parameters: a Combined Control (*L_cc_out*), a Vertical Control (*L_vc_out*), an Up Advisory (*L_ua_out*), a Down Advisory (*L_da_out*), whether to force an annunciation on or off (*L_force_alarm_out*), and an RA strength code (*L_strength_out*).

The loop iterates over all of the Label 270 rules until it finds a rule that is consistent with the current action, previous action, previous Label 270 word, ownship vertical rate, crossing, and altitude inhibit status. CONSISTENT (Algorithm 339) is used to determine if the input values are consistent with the given rule. GETMATCHINGLABEL270RULE returns output variables set to values associated with that rule. It also uses *cc*, *ua*, and *da* to determine whether the advisory has an up sense or not (*up*). If there is no match to any of the logic rules, the outputs all retain their initial values.

This algorithm takes as input *action*, *crossing*, *alt_inhibit*, *dz_own_ave*, *action_prev*, *word_prev*, and *strength_prev*. This algorithm returns *cc*, *vc*, *ua*, *da*, *force_alarm*, *strength*, and *down*.

Algorithm 338 GetMatchingLabel270Rule

```
function GetMatchingLabel270Rule( action::Z, crossing::Bool, alt_inhibit::Bool, dz_own_ave::R,
                          action_prev::Z, word_prev::Z, strength_prev::UInt8 )
    L_act_in::Vector{Z}           = params().display.label270rules.action
    L_act_in_prev::Vector{R}      = params().display.label270rules.prevaction
    L_word_in_prev::Vector{R}     = params().display.label270rules.prevword
    R_dz_lo_in::Vector{R}         = params().display.label270rules.lodz
    R_dz_hi_in::Vector{R}         = params().display.label270rules.hidz
    L_crossing_in::Vector{R}      = params().display.label270rules.crossing
    L_alt_inhibit_in::Vector{R}   = params().display.label270rules.altinhibit
    L_strength_in_prev::Vector{R} = params().display.label270rules.prevstrength
#
    L_cc_out::Vector{Z}           = params().display.label270rules.cc
    L_vc_out::Vector{Z}           = params().display.label270rules.vc
    L_ua_out::Vector{Z}           = params().display.label270rules.ua
    L_da_out::Vector{Z}           = params().display.label270rules.da
    L_force_alarm_out::Vector{Z}  = params().display.label270rules.forcealarm
    L_strength_out::Vector{Z}     = params().display.label270rules.strength
#
    num_rules::Z = length( L_act_in )
    cc::UInt8 = 0
    vc::UInt8 = 0
    ua::UInt8 = 0
    da::UInt8 = 0
    force_alarm::Z   = FORCE_ALARM_NONE
    strength::UInt8  = 0
    down::Bool       = false
    for i in 1:num_rules
        if Consistent(p. 371)( dz_own_ave, action, action_prev, word_prev, crossing,
                        alt_inhibit, strength_prev,
                        R_dz_lo_in[i], R_dz_hi_in[i], L_act_in[i], L_act_in_prev[i],
                        L_word_in_prev[i], L_crossing_in[i], L_alt_inhibit_in[i], L_strength_in_prev[i] )
            cc = UInt8( L_cc_out[i] )
            vc = UInt8( L_vc_out[i] )
            ua = UInt8( L_ua_out[i] )
            da = UInt8( L_da_out[i] )
            if (4 == cc)
                down = false
            elseif (5 == cc)
                down = true
            elseif (6 == cc) && (0 == ua) && (0 != da)
                down = true
            else
                down = false
            end
            strength    = UInt8( L_strength_out[i] )
            force_alarm = L_force_alarm_out[i]
            break
        end
    end
    return (cc::UInt8, vc::UInt8, ua::UInt8, da::UInt8, force_alarm::Z, strength::UInt8, down::Bool)
end
```

Referenced In: DetermineDisplayData(p. 368)

CONSISTENT (Algorithm 339) determines whether the current advisory and dynamic state is consistent with the specified logic rule.

The logic rule being tested is specified using a lower bound on ownship vertical rate (ft/s) (*R_dz_lo*), an upper bound on ownship vertical rate (ft/s) (*R_dz_hi*), a global action for the current cycle (*L_act*), a global action for the previous cycle (*L_act_prev*), a Label 270 code for the previous cycle (*L_word_prev*), a crossing dependency indication (*L_crossing*), an altitude inhibit dependency

indication (*L_alt_inhibit*), and a strength code for the previous cycle (*L_strength_prev*).

A value of *NaN* (Not a Number) for a logic rule variable functions as a wildcard. It will match any input value.

The advisory and dynamic state for comparison is specified using the current vertical rate of ownship (ft/s) (*dz_own*), the global action for the current cycle (*action*), the global action on the previous cycle (*action_prev*), the Label 270 code on the previous cycle (*word_prev*), whether there is a crossing (*crossing*), whether altitude inhibit is activated (*alt_inhibit*), and the strength code on the previous cycle (*strength_prev*).

If any of the inputs fail to match a rule, *is_consistent* is set to FALSE. If all the inputs match all of the rules, *is_consistent* is TRUE.

This algorithm takes as input *dz_own*, *action*, *action_prev*, *word_prev*, *crossing*, *alt_inhibit*, *strength_prev*, *R_dz_lo*, *R_dz_hi*, *L_act*, *L_act_prev*, *L_word_prev*, *L_crossing*, *L_alt_inhibit*, and *L_strength_prev*.
This algorithm returns *is_consistent*.

Algorithm 339 Consistent

```
function Consistent( dz_own::R, action::Z, action_prev::Z, word_prev::Z, crossing::Bool,
            alt_inhibit::Bool, strength_prev::UInt8,
            R_dz_lo::R, R_dz_hi::R, L_act::Z, L_act_prev::R,
            L_word_prev::R, L_crossing::R, L_alt_inhibit::R, L_strength_prev::R )
    is_consistent::Bool = true
    if (   (action != L_act)
        || (!isnan( R_dz_hi ) && (R_dz_hi < dz_own))
        || (!isnan( R_dz_lo ) && (dz_own < R_dz_lo))
        || (!isnan( L_act_prev ) && (action_prev != Int( L_act_prev )))
        || (!isnan( L_word_prev ) && (word_prev != Int( L_word_prev )))
        || (!isnan( L_crossing ) && (crossing != Bool( L_crossing )))
        || (!isnan( L_alt_inhibit ) && (alt_inhibit != Bool( L_alt_inhibit )))
        || (!isnan( L_strength_prev ) && (strength_prev != UInt8( L_strength_prev ))))
        is_consistent = false
    end
    return is_consistent::Bool
end
```

Referenced In: GetMatchingLabel270Rule(p. 370)

DETERMINECODE (Algorithm 340), used to determine the track status code for an individual intruder, is derived from TCAS II. The track status code is assigned based on the most critical advisory for the individual intruder.

First, the sense of the advisory for the individual intruder is examined to determine if there is a Resolution Advisory (RA). If so, the value is set to *SXUCODE_RA* (*4*) to indicate that an RA is present.

If the first conditions is not met, the code is set to *SXUCODE_CLEAR* (*0*) to indicate no advisory is in effect.

This algorithm takes as input *sense_indiv*. This algorithm returns *code*.

Algorithm 340 DetermineCode

```
function DetermineCode( sense_indiv::Symbol )
    code::UInt8 = SXUCODE_CLEAR
    if (sense_indiv != :None)
        code = SXUCODE_RA
    else
        code = SXUCODE_CLEAR
    end
    return code::UInt8
end
```

Referenced In: SetInvalidIntruder(p. 395), VerticalTRMUpdate(p. 241)

SETINTRUDERVERTICALSCORES (Algorithm 341) sets track display scores for each intruder. It calls DETERMINEVERTICALSCORE for each intruder.

This algorithm takes as an input *st_int*, *input_int_valid*, *own_opmode*, and *output_int*. This algorithm updates *output_int*.

Algorithm 341 SetIntruderVerticalScores

```
function SetIntruderVerticalScores( st_int::Vector{TRMIntruderState(p. 512)},
                                    input_int_valid::Vector{TRMIntruderInput(p. 499)}, own_opmode::UInt8,
                                    output_int::Vector{TRMIntruderData(p. 523)} )
    for i in 1:length(output_int)
        equip_int::Bool = (EQUIPAGE_NONE != input_int_valid[i].equipage) &&
                          (EQUIPAGE_TCAS != input_int_valid[i].equipage) &&
                          (EQUIPAGE_LARGE_CAS != input_int_valid[i].equipage) &&
                          (EQUIPAGE_CASTA != input_int_valid[i].equipage) && (OPMODE_RA == own_opmode)
        output_int[i].display.tds = DetermineVerticalScore(p. 373)( st_int[i].coc_cost,
                                                    output_int[i].display.code,
                                                    input_int_valid[i].degraded_surveillance, equip_int )
    end
end
```

Referenced In: VerticalTRMUpdate(p. 241)

Algorithm 342 DetermineVerticalScore

```
function DetermineVerticalScore( cost_ra_coc::R, code::UInt8, degraded_surveillance::UInt16,
                equip_int::Bool )
    X_scorfactr_good::R     = params().display.scorfactr_good
    X_scorfactr_degraded::R = params().display.scorfactr_degraded
    X_hiscore::R            = params().display.hiscore
    X_medhiscore::R         = params().display.medhiscore
    X_medscore::R           = params().display.medscore
    X_loscore::R            = params().display.loscore
    X_tinyscore::R          = params().display.tinyscore
    tds::R         = 0.0
    scorefactor::R = X_scorfactr_good
    if isnan( cost_ra_coc )
        cost_ra_coc = -10.0
    end
    if ((degraded_surveillance & DEGRADED_SURVEILLANCE_NAR) != 0) ||
       ((degraded_surveillance & DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED) != 0)
        scorefactor = X_scorfactr_degraded
    end
    if (code == SXUCODE_RA)
        if equip_int
            tds = X_hiscore + X_medhiscore + cost_ra_coc
        else
            tds = X_hiscore + cost_ra_coc
        end
    elseif (code == SXUCODE_CLEAR)
        tds = (X_tinyscore * scorefactor) + cost_ra_coc
    end
    return tds::R
end
```

Referenced In: SetInvalidIntruder(p. 395), SetIntruderVerticalScores(p. 372)

3.3.6 Generate Vertical TRM Output

The GENERATEVTRMOUTPUT algorithm of the Threat Resolution Module is shown in Figure 16.

The type of output processing to be performed during generation of the vertical TRM Report is determined by a combination of the own aircraft Operational Mode and intruder-specific processing flags set in TRMINTRUDERSTATE, variable *st_int*. One of the important fields in *st_int* for TRM output processing is *st_int.processing*, which indicates processing for resolution advisories, degraded surveillance or none. These fields in *st_int* are set by INTRUDERPREP (Algorithm 241), during preparation for TRM processing.

GENERATEVTRMOUTPUT (Algorithm 343) prepares the vertical TRM results for output and generates a vertical TRM Report (VTRMREPORT) with display, coordination, designation and the vertical representation of the RA.

GENERATEVTRMOUTPUT uses four algorithms to prepare the results for output in the vertical TRM Report and to update the sXu TRM state data structures.

Within each algorithm, intruder-specific advisory processing flags are used to further adjust the outputs. These flags include *st_int.processing*, which indicates processing for none (default), resolution advisories, or degraded surveillance.

GENERATESURVONLYMODEOUTPUT (Algorithm 348) is used when own aircraft is in Surveillance Only Operational Mode as indicated by a value of *TRM_OPMODE_SURV_ONLY* from *input_-*

own.opmode. In this mode the ACAS X system is actively keeping track of threats, but there are no resolution advisories output to the ownship, to other aircraft, or to the ground station.

GENERATERAMODEOUTPUT (Algorithm 344) is used when own aircraft is in RA Operational Mode as indicated by a value of *TRM_OPMODE_RA* from *input_own.opmode*. In this mode, resolution advisories can be produced by the ACAS X system. One exception to that is when an intruder has degraded surveillance that precludes generation of resolution advisories for that intruder.

GENERATESTANDBYMODEOUTPUT (Algorithm 349) is used when own aircraft is in Standby Operational Mode as indicated by a value of *TRM_OPMODE_STANDBY* from *input_own.opmode*. In this mode, no advisories are produced by the ACAS sXu system. All intruders are invalid for TRM processing and are handled outside this algorithm. The global advisory state is completely reset. The initial state for the next processing cycle will be no advisory.

At this point, the VTRMREPORT contains information about the global advisory and intruders that were evaluated for advisories.

DROPPEDINTRUDERSADJUSTMENT (Algorithm 357) generates report information for intruders that were not valid for TRM RA processing on this cycle. Such intruders include intruders that were omitted by the STM from input to the TRM and intruders that were marked as invalid for TRM processing. If there was a resolution advisory on the previous cycle, VRCs for invalid intruders are cancelled. The individual intruder advisory state for each invalid and dropped intruder is reset. When an intruder is dropped, its state information is deleted.

After DROPPEDINTRUDERSADJUSTMENT is called, each dropped intruder output in the list that is not the ground intruder is added to the vertical TRM report.

Finally, the ownship operational mode (*input_own.opmode*) is stored for use in the next cycle to determine coordination delay.

GENERATEVTRMOUTPUT takes as input *input_own*, *display*, *multithreat*, *action*, *dz_min*, *dz_max*, *ddz*, *z_own_ave*, *input_int_invalid*, *input_int_valid*, *output_int*, *received_vrcs*, and *sxu_trm_state*. Descriptions of the GENERATEVTRMOUTPUT output variables are found in Table 46.

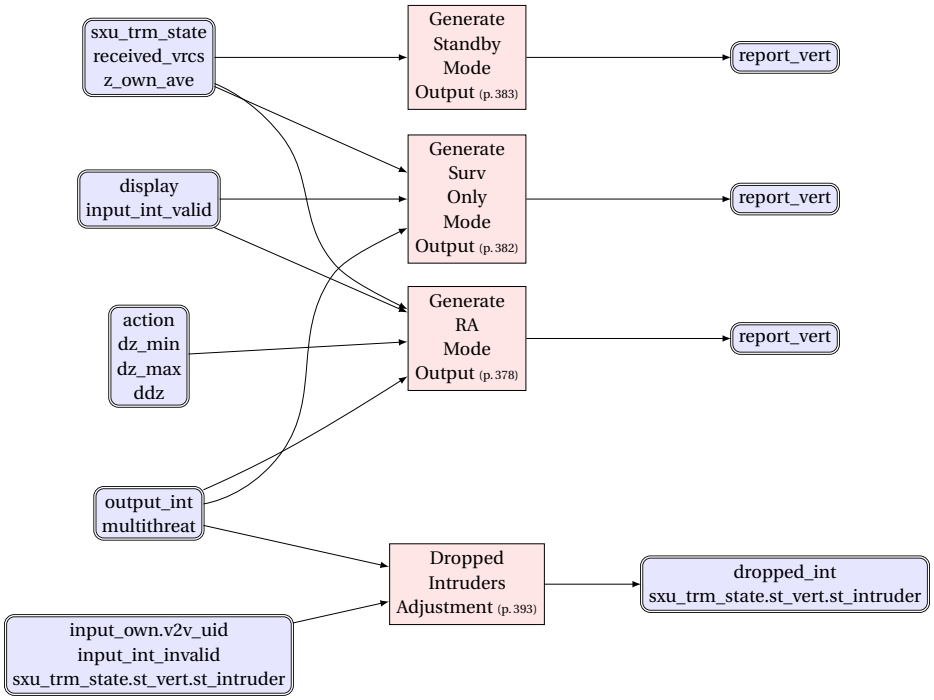

Figure 16. GenerateVTRMOutput overview.

Table 46. Vertical TRM Algorithm Output Variables - GenerateVTRMOutput

Variable	Units	Type	Description
report_vert	N/A	VTRMReport(p.524)	The vertical dimension representation of data for coordination, onboard pilot display and reports

Algorithm 343 GenerateVTRMOutput

```
function GenerateVTRMOutput( input_own::TRMOwnInput(p.498), display::DisplayLogic(p.516),
                    multithreat::Bool, action::Z, dz_min::R, dz_max::R, ddz::R,
                    z_own_ave::R, input_int_invalid::Vector{TRMIntruderInput(p.499)},
                    input_int_valid::Vector{TRMIntruderInput(p.499)},
                    output_int::Vector{TRMIntruderData(p.523)}, received_vrcs::Vector{Bool},
                    sxu_trm_state::sXuTRMState(p.510) )
    report_vert::VTRMReport(p.524) = VTRMReport(p.524)()
    if (OPMODE_SURV_ONLY == input_own.opmode)
        report_vert =
            GenerateSurvOnlyModeOutput(p.382)( display, input_int_valid, output_int, sxu_trm_state,
                                        received_vrcs, multithreat, z_own_ave )
    elseif (OPMODE_RA == input_own.opmode)
        report_vert =
            GenerateRAModeOutput(p.378)( display, action, dz_min, dz_max, ddz,
                                  input_int_valid, output_int,
                                  multithreat, sxu_trm_state, received_vrcs, z_own_ave )
    else
        report_vert =
            GenerateStandbyModeOutput(p.383)( sxu_trm_state, received_vrcs, z_own_ave)
    end
    (dropped_int::Vector{TRMIntruderData(p.523)}, sxu_trm_state.st_vert.st_intruder) =
         DroppedIntrudersAdjustment(p.393)( input_own.v2v_uid, multithreat, input_int_invalid,
                                     output_int, sxu_trm_state.st_vert.st_intruder )
    for i in 1:length( dropped_int )
        if (dropped_int[i].id != ID_GROUND)
            push!( report_vert.display.intruder, dropped_int[i].display )
            push!( report_vert.coordination,     dropped_int[i].coordination )
        end
    end
    sxu_trm_state.st_vert.st_own.opmode_prev = input_own.opmode
    return (report_vert::VTRMReport(p.524))
end
```

Referenced In: VerticalTRMUpdate(p.241)

3.3.6.1 RA Mode Processing

GENERATERAMODEOUTPUT (Algorithm 344) is used for the vertical only dimension when own aircraft is in RA Operational Mode, and after the individual and global advisories are produced. GENERATERAMODEOUTPUT packages the vertical perspective report results for use in GENERATEBLENDEDSXUTRMOUTPUT where the horizontal and vertical reports are combined to create SXUTRMREPORT. It takes into account surveillance quality for each intruder.

The ACAS X RA Operational Mode produces resolution advisories (RAs). Within this mode of operation, individual intruders can be processed differently, depending on their surveillance quality. When an individual intruder has degraded surveillance insufficient for producing RAs, RA's for said intruder are surpressed.

First, ADJUSTRAMODEINTRUDEROUTPUT is called to update the display, coordination, and designation settings for each intruder.

Once all the intruders have been updated individually, the global outputs are updated.

The threat index (*idx_tid*) is assigned based on the threat information in *st_adj* in the following order:

1. A threat that wasn't previously reported; *st_adj.idx_threat_new* is greater than *0*.

2. The threat that was last reported; *st_adj.idx_threat_last* is greater than *0*.
3. No threat; in this case, *st_adj.idx_threat_upd* has the value -1.

Next, the outputs for the pilot display are adjusted if the global advisory is Clear of Conflict and either the previous RA was suppressed and all the intruders with individual RAs on the previous cycle were excluded from determination of this global Clear of Conflict (*excluded_all_cocs* is TRUE).

In all cases, the stored state variables for the previous action and word are updated to reflect the actual outputs.

Next, the VTRMREPORT output (*report_vert*) is created. The updated global display settings (*display*) is used to initialize *report_vert*. Then, for each intruder that is not ground, the adjusted intruder entries for display and coordination are added to *report_vert*.

Next, SETVERTICALRAMESSAGEOUTPUT is used to set the values in *report_vert.broadcast*.

Lastly, the previous vertical global advisory values are set (TRMOWNSTATE, *a_prev*) as they will be used to accurately determine costs for the next cycle.

This algorithm takes as input *display*, *action*, *dz_min*, *dz_max*, *ddz*, *input_int_valid*, *output_int*, *received_vrcs*, *multithreat*, *z_own_ave* and *sxu_trm_state*. The algorithm outputs the Vertical TRM Report (*report_vert*), and modifies the output settings for each intruder (*output_int*), state settings for each intruder (*st_int*), and own state (*st_own*).

Algorithm 344 GenerateRAModeOutput

```
function GenerateRAModeOutput( display::DisplayLogic(p.516), action::Z, dz_min::R, dz_max::R, ddz::R,
                               input_int_valid::Vector{TRMIntruderInput(p.499)},
                               output_int::Vector{TRMIntruderData(p.523)}, multithreat::Bool,
                               sxu_trm_state::sXuTRMState(p.510), received_vrcs::Vector{Bool}, z_own_ave::R )
    idx_tid::Z = -1
    st_adj::TRMIndivAdjustState(p.523) = AdjustRAModeIntruderOutput(p.379)( input_int_valid, output_int,
                                         sxu_trm_state.st_vert.st_intruder, sxu_trm_state.st_vert.st_own )
    if (0 < st_adj.idx_threat_new)
        idx_tid = st_adj.idx_threat_new
    elseif (0 < st_adj.idx_threat_last)
        idx_tid = st_adj.idx_threat_last
    else
        idx_tid = st_adj.idx_threat_upd
    end
    excluded_all_cocs::Bool = (0 < st_adj.force_silent_count) &&
                              (st_adj.force_silent_count == st_adj.ra_prev_count)
    if (COC == action) && excluded_all_cocs
        display = DisplayLogic(p.516)( false, UInt8(0), UInt8(0), UInt8(0), UInt8(0), 0.0, false,
                                 GROUND_ALERT_NONE, display.strength, display.down, -Inf, Inf, 0.0,
                                 display.opmode, display.trm_altmode )
    end
    sxu_trm_state.st_vert.st_own.action_prev   = action
    sxu_trm_state.st_vert.st_own.word_prev     = (display.cc * 1000) + (display.vc * 100) + (display.ua *
         10) + display.da
    sxu_trm_state.st_vert.st_own.strength_prev = display.strength
    report_vert::VTRMReport(p.524)  = VTRMReport(p.524)( display )
    for i in 1:length( output_int )
        if (output_int[i].id != ID_GROUND)
            push!( report_vert.display.intruder, output_int[i].display )
            push!( report_vert.coordination,     output_int[i].coordination )
        end
    end
    conflicting_senses::Bool = (st_adj.upsense && st_adj.downsense)
    report_vert.broadcast_msg::sXuTRMBroadcastData(p.507) =
        SetVerticalRAMessageOutput(p.386)( idx_tid, display, multithreat, !conflicting_senses, received_vrcs,
                                           input_int_valid, z_own_ave, sxu_trm_state )
    sxu_trm_state.st_vert.st_own.a_prev.action      = action
    sxu_trm_state.st_vert.st_own.a_prev.dz_min      = dz_min
    sxu_trm_state.st_vert.st_own.a_prev.dz_max      = dz_max
    sxu_trm_state.st_vert.st_own.a_prev.ddz         = ddz
    sxu_trm_state.st_vert.st_own.a_prev.multithreat = multithreat
    return report_vert::VTRMReport(p.524)
end
```

Referenced In: GenerateVTRMOutput(p.376)

ADJUSTRAMODEINTRUDEROUTPUT (Algorithm 345) is used to adjust the intruder outputs when own aircraft is in RA Operational Mode. It is used, after the individual and global advisories are produced, to update the intruder results in preparation for output in the TRM Report. It takes surveillance quality for each intruder.

Global RA processing mode is the nominal processing mode for the TRM.

The algorithm loops over all intruders in the output (*output_int*) and treats each intruder individually. The number of intruders in *output_int* is given by *N_intruders*.

For each individual intruder, UPDATEINDIVADJUSTTHREATINFO is first called to determine if the intruder has a current resolution advisory (RA) state.

UPDATEINDIVADJUSTTHREATINFO is used to set the indices and flags in *st_adj* used within GEN-

ERATERAMODEOUTPUT.

UPDATEINDIVADJUSTCOUNTS is called next for the individual intruder to update the counts used to determine if a Clear of Conflict annunciation should be suppressed due to a previously suppressed RA or degraded surveillance. UPDATEINDIVADJUSTCOUNTS returns whether the individual intruder advisory state should be reset at the end of the intruder processing loop (*advisory_reset*).

The intruder outputs for the individual intruder are next updated as appropriate. If the intruder was excluded from the global RA, it cannot have an active RA and the logic mode processing type (*st_int.processing*) original setting (*RA_PROCESSING_DEGRADED_SURVEILLANCE*) remains in effect.

For the final step in the loop over the intruders, TRMINTRUDERSTATEUPDATE is used to update TRMINTRUDERSTATE for each intruder. Whenever *advisory_reset* is TRUE, the previous resolution advisory and the online cost state are cleared for this intruder. However, the initialization delay counter (*T_count*) in INITIALIZATIONCSTATE is preserved to allow advisories to be generated on the next processing cycle. The initialization delay counter will be reset in INTRUDERPREP when the surveillance is no longer degraded; there will be an enforced delay before any advisories are produced following that transition.

This algorithm takes as input *input_int_valid*, *output_int*, *st_int*, and *st_own*. The outputs are state variables for TRMINDIVADJUSTSTATE (*st_adj*), modified output settings for each intruder (*output_int*), and modified state settings for each intruder (*st_int*).

Algorithm 345 AdjustRAModeIntruderOutput

```
function AdjustRAModeIntruderOutput( input_int_valid::Vector{TRMIntruderInput(p. 499)},
                      output_int::Vector{TRMIntruderData(p. 523)},
                      st_int::Vector{TRMIntruderState(p. 512)}, st_own::TRMOwnState(p. 511) )
    N_intruders::Z = length( output_int )
    st_adj::TRMIndivAdjustState(p. 523) = TRMIndivAdjustState(p. 523)()
    TRM_st_int::Dict{UInt32, TRMIntruderState(p. 512)} = Dict{UInt32, TRMIntruderState(p. 512)}()
    for st_int_iter::TRMIntruderState(p. 512) in st_int
        TRM_st_int[st_int_iter.id] = st_int_iter
    end
    for i in 1:N_intruders
        st_int_indiv = TRM_st_int[output_int[i].id]
        UpdateIndivAdjustThreatInfo(p. 380)( st_own, i, output_int[i], st_int_indiv, st_adj )
        is_advisory_reset::Bool =
            UpdateIndivAdjustCounts(p. 381)( st_int_indiv, st_adj )
        TRMIntruderStateUpdate(p. 434)( st_int_indiv, output_int[i].coordination.vrc,
                                output_int[i].coordination.cvc, input_int_valid[i].equipage,
                                output_int[i].coordination.coordination_msg,
                                is_advisory_reset, false )
    end
    return st_adj::TRMIndivAdjustState(p. 523)
end
```

Referenced In: GenerateRAModeOutput(p. 378)

UPDATEINDIVADJUSTTHREATINFO (Algorithm 346) sets the indices and flags needed for preparing TRM Report content in GENERATERAMODEOUTPUT.

The algorithm gathers information to be used when preparing the resolution advisory output to

the ground station.

If there is an individual resolution advisory for this intruder, the intruder is flagged as a threat and the sense of the advisory is recorded. Next, indexing information is recorded for use determining which threat will be reported in the Threat Identifier (TID) in the output to the ground station. The index into the intruder array is registered as being for a previously unidentified threat (*st_adj.idx_threat_new*), the threat reported in the threat identifier (TID) on the last cycle (*st_adj.idx_threat_last*), or a threat that was previously reported (*st_adj.idx_threat_upd*).

Whenever an intruder is identified as a new threat, it is flagged as an identified threat. Only one intruder can be flagged as a new threat each TRM cycle.

If there is no individual resolution advisory for this intruder, the intruder is flagged as not being a threat and the identified threat flag is reset.

UPDATEINDIVADJUSTTHREATINFO takes as input *st_own*, *idx*, *output_int*, *st_int*, and *st_adj*. This algorithm modifies individual adjust state (*st_adj*) and *st_int*.

Algorithm 346 UpdateIndivAdjustThreatInfo

```
function UpdateIndivAdjustThreatInfo( st_own::TRMOwnState(p.511), idx::Z,
                                      output_int::TRMIntruderData(p.523), st_int::TRMIntruderState(p.512),
                                      st_adj::TRMIndivAdjustState(p.523) )
    if (:None != output_int.sense)
        st_int.is_threat = true
        if (:Up == output_int.sense)
            st_adj.upsense = true
        else
            st_adj.downsense = true
        end
        if (st_adj.idx_threat_new <= 0) && !st_int.is_identified_threat
            st_adj.idx_threat_new       = idx
            st_int.is_identified_threat = true
        elseif (output_int.id == st_own.ra_output_prev.tgtid_tid)
            st_adj.idx_threat_last = idx
        else
            st_adj.idx_threat_upd = idx
        end
    else
        st_int.is_threat            = false
        st_int.is_identified_threat = false
    end
end
```

Referenced In: AdjustRAModeIntruderOutput(p.379)

UPDATEINDIVADJUSTCOUNTS (Algorithm 347) updates counters used when updating DISPLAYLOGIC values in GENERATERAMODEOUTPUT.

The counters set in this algorithm are stored in a local data structure, *st_adj* of type TRMINDIVADJUSTSTATE.

At the end of the algorithm, the fields *ra_prev_count* and *force_silent_count* are set. *ra_prev_count* is used in conjunction with *force_silent_count* to determine when annunciation of Clear of Conflict should be disabled. The field *ra_prev_count* is incremented whenever there was an RA for this intruder on the previous cycle. When there was an RA for this intruder on the pre-

vious cycle and this intruder was excluded from determination of the global RA on this cycle, *force_silent_count* is also incremented. When *ra_prev_count* and *force_silent_count* have the same value, all previous RAs were excluded from determination of the current global Clear of Conflict.

UPDATEINDIVADJUSTCOUNTS takes as input *st_int*, and *st_adj*. This algorithm returns whether an advisory reset is needed (*advisory_reset*) due to output processing for degraded surveillance and modifies state data (*st_adj*).

Algorithm 347 UpdateIndivAdjustCounts

```
function UpdateIndivAdjustCounts( st_int::TRMIntruderState(p. 512), st_adj::TRMIndivAdjustState(p. 523) )
    was_excluded::Bool = (RA_PROCESSING_DEGRADED_SURVEILLANCE == st_int.processing)
    is_advisory_reset::Bool = false
    if was_excluded
        is_advisory_reset = true
    end
    if IsCOC(p. 419)( st_int.a_prev.dz_min, st_int.a_prev.dz_max ) || was_excluded
        if st_int.a_prev.ra_prev
            st_adj.ra_prev_count = st_adj.ra_prev_count + 1
            if was_excluded
               st_adj.force_silent_count = st_adj.force_silent_count + 1
            end
        end
    end
    return (is_advisory_reset::Bool)
end
```

Referenced In: AdjustRAModeIntruderOutput(p. 379)

3.3.6.2 Surveillance Only Mode Processing

GENERATESURVONLYMODEOUTPUT (Algorithm 348) is used in the vertical call stack when own aircraft is in Surveillance Only Operational Mode. It updates and packages the results for output in the Vertical TRM Report, in such a manner that it doesn't produce alarms.

First, TRMINTRUDERSTATEUPDATE (Algorithm 414) is called to save the final coordination settings, reset the advisory states and fully reset the online cost state for each intruder, except for the initialization cost. The threat state for each intruder, on the next processing cycle, will indicate no threat by setting *is_identified_threat* to *false*.

Once the outputs for each individual intruder have been adjusted, the outputs for the pilot display (*display.cc* and *display.alarm*) are adjusted so display alarms are not produced.

Next, the VTRMREPORT output (*report_vert*) is created.

The updated global display settings (*display*) is used to initialize *report_vert*.

Then, for each non-ground intruder, the adjusted intruder entries for display, coordination, and designation are added to *report_vert*.

Next, the vertical state for own aircraft (*xu_trm_state.st_vert.st_own*) is reset to remove RA history for this cycle.

Then the SETVERTICALRAMESSAGEOUTPUT algorithm is used to set the values in *report_vert.broadcast_-*

msg so RA report will not be produced during blended report generation.

Finally, the global advisory state is reset. The SETVERTICALRAMESSAGEOUTPUT algorithm takes as input for *idx_tid* = -1, which signals SETRABROADCASTDATA to set No RA, reset values for TTI, TID and the target id associated with the TID value in *st_own.ra_output_prev.tgtid_tid*, which are represented in the SXUTRMBROADCASTDATA message data structure. Additionally the inputs *display*, *multithreat*, *single_sense* (set false), *received_vrcs*, *input_int_valid*, *z_own_ave*, and *sxu_trm_state* are passed to SETVERTICALRAMESSAGEOUTPUT.

GENERATESURVONLYMODEOUTPUT takes as input *display*, *ddz*, *input_int_valid*, *output_int*, *received_vrcs*, *multithreat*, *z_own_ave*, and *sxu_trm_state*.

The outputs for GENERATESURVONLYMODEOUTPUT (Algorithm 348) are the Vertical TRM Report (*report_vert*), modified display logic data (*display*), individual intruder state (*is_identified_threat*) and the vertical TRM ownship state *st_own*.

Algorithm 348 GenerateSurvOnlyModeOutput

```
function GenerateSurvOnlyModeOutput( display::DisplayLogic(p.516),
                                     input_int_valid::Vector{TRMIntruderInput(p.499)},
                                     output_int::Vector{TRMIntruderData(p.523)},
                                     sxu_trm_state::sXuTRMState(p.510), received_vrcs::Vector{Bool},
                                     multithreat::Bool, z_own_ave::R )
    N_intruders::Z = length( input_int_valid )
    TRM_st_int::Dict{UInt32, TRMIntruderState(p.512)} = Dict{UInt32, TRMIntruderState(p.512)}()
    for st_int_iter::TRMIntruderState(p.512) = sxu_trm_state.st_vert.st_intruder
        TRM_st_int[st_int_iter.id] = st_int_iter
    end
    for i in 1:N_intruders
        st_int_indiv = TRM_st_int[output_int[i].id]
        TRMIntruderStateUpdate(p.434)( st_int_indiv, output_int[i].coordination.vrc,
                                 output_int[i].coordination.cvc, input_int_valid[i].equipage,
                                 output_int[i].coordination.coordination_msg,
                                 true, false )
        st_int_indiv.is_identified_threat = false
    end
    display.cc = 0
    display.alarm = false
    report_vert::VTRMReport(p.524) = VTRMReport(p.524)( display )
    for i in 1:length( output_int )
        if (output_int[i].id != ID_GROUND)
            push!( report_vert.display.intruder, output_int[i].display )
            push!( report_vert.coordination,     output_int[i].coordination )
        end
    end
    sxu_trm_state.st_vert.st_own.action_prev   = COC
    sxu_trm_state.st_vert.st_own.word_prev     = 0
    sxu_trm_state.st_vert.st_own.crossing_prev = false
    sxu_trm_state.st_vert.st_own.strength_prev = display.strength
    sxu_trm_state.st_vert.st_own.st_multithreat_cost_balancing = MultithreatCostBalancingCState(p.530)()
    sxu_trm_state.st_vert.st_own.st_arbitrate = ActionArbitrationGlobalCState(p.526)()
    report_vert.broadcast_msg::sXuTRMBroadcastData(p.507) =
        SetVerticalRAMessageOutput(p.386)( -1, display, multithreat, false, received_vrcs,
                                   input_int_valid, z_own_ave, sxu_trm_state )
    sxu_trm_state.st_vert.st_own.a_prev = GlobalAdvisory(p.517)()
    return (report_vert::VTRMReport(p.524))
end
```

Referenced In: GenerateVTRMOutput(p.376)

3.3.6.3 Standby Mode Processing

GENERATESTANDBYMODEOUTPUT (Algorithm 349) is used when own aircraft is in Standby Operational Mode to package results of the Vertical representation for the TRM Report.

In Standby Operational Mode, there are no advisories produced by the ACAS X system. There is no resolution advisory input to this algorithm. There are no intruder inputs to this algorithm, all intruders are invalid for TRM processing and are handled within DROPPEDINTRUDERSADJUSTMENT.

SETVERTICALRAMESSAGEOUTPUT is used to set the values in *report_vert.broadcast_msg*. The inputs to GENERATESTANDBYMODEOUTPUT indicate there is no resolution advisory for this cycle.

The vertical global advisory state is completely reset. The initial state for the next processing cycle will be no advisory.

For every intruder state, the previous resolution advisory (*sxu_trm_state.st_vert.st_intruder.sense_prev*) is cleared.

This algorithm takes as input *received_vrcs*, *z_own_ave*, and *sxu_trm_state* (type SXUTRMSTATE). The outputs are the Vertical TRM Report (*report_vert*), modified state settings for each intruder (*sxu_trm_state.st_vert.st_intruder*), and modified own state (*sxu_trm_state.st_vert.st_own*).

Algorithm 349 GenerateStandbyModeOutput

```
function GenerateStandbyModeOutput( sxu_trm_state::sXuTRMState(p.510), received_vrcs::Vector{Bool}, z_own_ave::
    R )
    report_vert::VTRMReport(p.524)                    = VTRMReport(p.524)()
    display::DisplayLogic(p.516)                     = DisplayLogic(p.516)()
    input_int_valid::Vector{TRMIntruderInput(p.499)} = Vector{TRMIntruderInput(p.499)}( )
    st_int::Vector{TRMIntruderState(p.512)}          = Vector{TRMIntruderState(p.512)}( )
    report_vert.broadcast_msg::sXuTRMBroadcastData(p.507) =
        SetVerticalRAMessageOutput(p.386)( -1, display, false, false, received_vrcs,
                                input_int_valid, z_own_ave, sxu_trm_state )
    sxu_trm_state.st_vert.st_own.a_prev = GlobalAdvisory(p.517)()
    for i in 1:length( sxu_trm_state.st_vert.st_intruder )
        sxu_trm_state.st_vert.st_intruder[i].sense_prev    = :None
    end
    return report_vert::VTRMReport(p.524)
end
```

Referenced In: GenerateVTRMOutput(p.376)

3.3.6.4 Resolution Advisory Output Messages

Collision Avoidance Systems employ three types of messages to communicate resolution advisories:

- TCAS Resolution Advisory Messages used for air-to-air coordination. The data needed for these messages is contained in TRMCOORDINATIONINTERROGATIONDATA. The data are transmitted when *vrc* or *cvc* contain a value other than *0*. The fields in this data structure are populated using CROSSLINK (Algorithm 331). The CROSSLINK algorithm is used in several different sections in this document.
- Broadcast Interrogation Messages used to indicate the presence and identity of own aircraft

and to allow monitoring of resolution advisory information. The data needed for these messages is contained in sXuTRMBroadcastData. The data are transmitted when *avra_strength* contains a value other than *0* or the *vrat* bit is set. The fields in this data structure are populated using SetRABroadcastData (Algorithm 356).

- Coordination Reply Messages used as a handshake and to allow monitoring of resolution advisory information. The data needed for these messages is contained in sXuTRMBroadcastData. The data are transmitted when *avra_strength* contains a value other than *0* or the *rat* or bit is set. The fields in this data structure are populated using SetRABroadcastData (Algorithm 356).

VTRMRAData (Type 44) encapsulates the Active Vertical Resolution Advisory (AVRA), Low-level descent inhibits (LDI), Resolution Advisory Complements (RAC), Vertical Resolution Advisory Terminated indicator (VRAT), and Vertical Multiple Threat Encounter (VMTE) values used to populate sXuTRMBroadcastData.

This section describes how the field in VTRMRAData, sXuTRMBroadcastData, is populated. DetermineVerticalRAData (Algorithm 350) populates the VTRMRAData data structure with information about the active vertical resolution advisory. This data structure is used in sXuTRMBroadcastData in the TRM Report.

The low-level descend inhibit value (*ra_data.ldi*) is taken from the ownship altitude inhibit settings (*st_own.st_alt_inhibit*).

The remaining fields in the data structure are set based on the resolution advisory state as follows:

- There is an active resolution advisory. The fields are set directly from the inputs. The new field settings are recorded in *st_own.ra_output_prev.vert_ra_data* for use when the RA is terminated.
- There was an active resolution advisory on the previous TRM cycle. The fields settings from the previous cycle are used except RA termination is indicated by setting *ra_term* to TRUE. Cleared field settings are recorded in *st_own.ra_output_prev.vert_ra_data*.
- There is no resolution advisory present or past. The field settings are cleared except the *rac* field is set from *received_vrcs*. The new field settings are recorded in *st_own.ra_output_prev.vert_ra_data*.

The inputs are

- TRMOwnState (*st_own*) - own aircraft state
- DisplayLogic (*display*) - the resolution advisory information for display
- *multithreat* - whether or not the advisory is a multi-threat advisory
- *single_sense* - whether or not the advisory has a single sense or conflicting senses
- *received_vrcs* - a list of the received VRCs to be used for RAC

The outputs are returned in the VTRMRAData data structure (*vert_ra_data*).

Algorithm 350 DetermineVerticalRAData

```
function DetermineVerticalRAData( st_own::TRMOwnState(p.511), display::DisplayLogic(p.516), multithreat::Bool,
                                  single_sense::Bool, received_vrcs::Vector{Bool} )
    # Deviation potential: Refer to ACAS Xu MOPS Vol. I section 1
    vert_ra_data::VTRMRAData(p.508) = VTRMRAData(p.508)()
    mode_idx::Z = NO_Xo_MODE_INDEX
    vert_ra_data.ldi = st_own.st_alt_inhibit[mode_idx].ldi
    if (0 < display.strength)
        vert_ra_data.avra_single_intent    = single_sense
        vert_ra_data.avra_down             = display.down
        vert_ra_data.avra_strength         = display.strength
        vert_ra_data.avra_crossing         = display.crossing
        vert_ra_data.ra_term               = false
        vert_ra_data.vmte                  = multithreat
        vert_ra_data.rac                   = copy( received_vrcs )
        st_own.ra_output_prev.vert_ra_data = deepcopy( vert_ra_data )
    elseif (0 < st_own.ra_output_prev.vert_ra_data.avra_strength)
        vert_ra_data.ra_term               = true
        st_own.ra_output_prev.vert_ra_data = VTRMRAData(p.508)()
    else
        vert_ra_data.avra_single_intent    = false
        vert_ra_data.avra_down             = false
        vert_ra_data.avra_strength         = 0
        vert_ra_data.avra_crossing         = false
        vert_ra_data.ra_term               = false
        vert_ra_data.vmte                  = false
        vert_ra_data.rac                   = copy( received_vrcs )
        st_own.ra_output_prev.vert_ra_data = deepcopy( vert_ra_data )
    end
    return vert_ra_data::VTRMRAData(p.508)
end
```

Referenced In: SetVerticalRAMessageOutput(p.386)

SETVERTICALRAMESSAGEOUTPUT (Algorithm 351) populates the SXUTRMBROADCASTDATA data structure with the vertical beliefs.

This algorithm calls three algorithms to populate the SXUTRMBROADCASTDATA data structure for the TRM Reports.

- DETERMINEVERTICALRADATA (Algorithm 350) populates the VTRMRADATA data structure (*ra_data*) that is used in SXUTRMBROADCASTDATA.
- SETRABROADCASTDATA (Algorithm 356) populates the SXUTRMBROADCASTDATA data structure, *broadcast*.

This algorithm takes as input *idx_tid*, *display*, *multithreat*, *single_sense*, *received_vrcs*, *input_int_valid*, *z_own_ave*, and *sxu_trm_state*. The outputs from SETVERTICALRAMESSAGEOUTPUT are populated data structures for SXUTRMBROADCASTDATA (*broadcast*).

Algorithm 351 SetVerticalRAMessageOutput

```
function SetVerticalRAMessageOutput( idx_tid::Z, display::DisplayLogic(p. 516), multithreat::Bool,
                                     single_sense::Bool, received_vrcs::Vector{Bool},
                                     input_int_valid::Vector{TRMIntruderInput(p. 499)},
                                     z_own_ave::R, sxu_trm_state::sXuTRMState(p. 510) )
    # Deviation potential: Refer to ACAS Xu MOPS Vol. I section 1
    ra_data::VTRMRAData(p. 508) =
        DetermineVerticalRAData(p. 385)( sxu_trm_state.st_vert.st_own, display, multithreat,
                                         single_sense, received_vrcs )
    broadcast::sXuTRMBroadcastData(p. 507) =
        SetRABroadcastData(p. 391)( ra_data, idx_tid, input_int_valid, z_own_ave, sxu_trm_state )
    return (broadcast::sXuTRMBroadcastData(p. 507))
end
```

Referenced In: GenerateRAModeOutput(p. 378), GenerateSurvOnlyModeOutput(p. 382), GenerateStandbyModeOutput(p. 383)

3.3.6.4.1 Vertical RA Report

DETERMINETTIANDTID (Algorithm 352) sets the Threat Type Indicator (TTI), Threat ID (TID) and a boolean value *is_icao*, which is set true if the address of the threat detected is an ICAO address of the most recent threat. The TTI and TID values are fields in the SXUTRMBROADCASTDATA data structure in TRM Report.

DETERMINETTIANDTID takes the vector of intruder inputs (*input_int_valid*), the index for the most recent threat (*idx_tid*), and the barometric altitude of own aircraft (*z_own_ave*) as inputs.

When the index of the most recent threat is a valid index (greater than *0*) and is less than or equal to the length of the input array (*input_int_valid*), the TID is represented as either an address or a coded altitude-range-bearing value.

If the threat is determined to be an address type, the TID is set to that address and the TTI is set to *TTI_ADDRESS*, then the ICAO address indicator *is_icao* is set to the predetermined valuse from the selected threat.

Otherwise, the TID is a coded altitude-range-bearing. The TTI is set to *TTI_ALT_RNG_BRG*. The altitude, range, and bearing information for the threat is obtained from the STMDISPLAYSTRUCT output from the STM. The coded altitude is obtained from ENCODETIDALTITUDE, using whether or not the intruder aircraft is reporting altitude (*alt_reporting*) and the barometric altitude in feet (*altitude*). The barometric altitude of the threat is obtained by adding the vertical separation distance (*z_rel*) to the own barometric altitude (*z_own_ave*). The coded range is obtained from ENCODETIDRANGE, using the ground range in feet (*r_ground*). The coded relative bearing is obtained from ENCODETIDBEARING, using the relative bearing, in radians (*Chi_rel*). The coded altitude is 13 bits, the coded range is 7 bits, and the coded relative bearing is 6 bits. DETERMINETTIANDTID protects against overflow of each of these three bit fields.

If there is no threat available, the TID and TTI information is cleared to the default setting. The default setting is a TID address of *0* with TTI set to *TTI_ADDRESS* and *is_icao* set to *false*.

This algorithm takes as input *input_int_valid*, *idx_tid*, and *z_own_ave*.

The outputs from DETERMINETTIANDTID are the TTI (*tti*), the TID (*tid* and *is_icao*).

Algorithm 352 DetermineTTIAndTID

```
function DetermineTTIAndTID( idx_tid::Z, input_int_valid::Vector{TRMIntruderInput(p.499)}, z_own_ave::R )
    tti::UInt8 = TTI_ADDRESS
    tid::TID(p.508)    = TIDAddress(p.508)()
    is_icao::Bool = false
    if (0 < idx_tid) && (idx_tid <= length( input_int_valid ))
        if input_int_valid[idx_tid].id_directory.icao24.valid && (input_int_valid[idx_tid].id_directory.
            icao24.value > 0)
            tti = TTI_ADDRESS
            tid = TIDAddress(p.508)( input_int_valid[idx_tid].id_directory.icao24.value )
            is_icao = true
        elseif input_int_valid[idx_tid].id_directory.anon24.valid && (input_int_valid[idx_tid].id_directory.
            anon24.value > 0)
            tti = TTI_ADDRESS
            tid = TIDAddress(p.508)( input_int_valid[idx_tid].id_directory.anon24.value )
            is_icao = false
        else
            tti = TTI_ALT_RNG_BRG
            stm_display_data::StmDisplayStruct(p.481) = input_int_valid[idx_tid].stm_display
            altitude::R = z_own_ave + input_int_valid[idx_tid].stm_display.z_rel_ft
            tida::UInt16 = min( 0x07ff, EncodeTIDAltitude(p.388)( stm_display_data.alt_reporting, altitude ) )
            tidr::UInt8  = min( 0x0007f, EncodeTIDRange(p.389)( stm_display_data.r_ground_ft ) )
            tidb::UInt8  = min( 0x0003f, EncodeTIDBearing(p.390)( stm_display_data.bearing_rel_rad ) )
            tid = TIDAltRngBrg(p.509)( tida, tidr, tidb )
        end
    end
    return (tti::UInt8, tid::TID(p.508), is_icao::Bool)
end
```

Referenced In: SetThreatIdentityData(p.401)

ENCODETIDALTITUDE (Algorithm 353) returns an 11-bit encoding of altitude to be used for the Threat ID. The altitude is input in feet. The coded altitude is also in feet.

First, the maximum altitude that can be encoded is determined using *TIDA_MAX_CODE* and *TIDA_OFFSET*. Then the altitude is encoded.

When the intruder aircraft is not altitude reporting (*alt_reporting* is FALSE) or the altitude is not set (it is *NaN*), the coded altitude returned by ENCODETIDALTITUDE is *0*.

Otherwise, ENCODETIDALTITUDE checks the value of the altitude to make sure it stays within the range limits for TID altitude encoding. When the value of the altitude is less than *TIDA_MIN_-ALTITUDE*, the coded altitude is *1*. When the value of the altitude is greater than *max_altitude*, the coded altitude is *TIDA_MAX_CODE*.

When the altitude is within the range limits, the altitude is coded in *100.0* foot increments offset from *TIDA_OFFSET*.

The coded altitude is returned as an unsigned integer.

This algorithm takes as input *alt_reporting*, and *altitude*. This algorithm returns *coded_altitude*.

Algorithm 353 EncodeTIDAltitude

```
function EncodeTIDAltitude( alt_reporting::Bool, altitude::R )
    coded_altitude::UInt32 = 0
    max_altitude::R = ((TIDA_MAX_CODE - 1) * 100.0) + TIDA_OFFSET
    if !alt_reporting || isnan( altitude )
        coded_altitude = 0
    elseif (altitude < TIDA_MIN_ALTITUDE)
        coded_altitude = 1
    elseif (max_altitude < altitude)
        coded_altitude = TIDA_MAX_CODE
    else
        coded_altitude = UInt32( 1 + round( (altitude - TIDA_OFFSET) * 0.01 ) )
        if (coded_altitude < 1)
            coded_altitude = 1
        elseif (TIDA_MAX_CODE < coded_altitude)
            coded_altitude = TIDA_MAX_CODE
        end
    end
    return coded_altitude::UInt32
end
```

Referenced In: DetermineTTIAndTID(p. 387)

ENCODETIDRANGE (Algorithm 354) returns the coded range. The range is input in feet. The coded range is in nautical miles.

The range is first converted from feet to nautical miles. If the range value is not set (it is *NaN*), the coded range is *0*.

Since the coded range is represented as 7 bits, the coded values must be between *1* and *127*, inclusive. If the intruder is within *0.05* nautical miles in the horizontal dimension, the coded range is *1*. If the intruder is outside *12.55* nautical miles in the horizontal dimension, the coded range is *TIDR_MAX_CODE*.

Otherwise, the range is coded in *0.1* nautical mile increments, so a range of *0.1* is coded as *2* and a range of *11.2* is coded as *113*.

The coded range is returned as an unsigned integer.

This algorithm takes as input *range*. This algorithm returns *coded_range*.

Algorithm 354 EncodeTIDRange

```
function EncodeTIDRange( range::R )
   coded_range::UInt32 = 0
   ft2nmi::R = geoutils.feet_to_nautical_miles
   range = abs( range ) * ft2nmi
   if isnan( range )
       coded_range = 0
   elseif (range < 0.05)
       coded_range = 1
   elseif (12.55 < range)
       coded_range = TIDR_MAX_CODE
   else
       coded_range = UInt32( 1 + round( range * 10.0 ) )
       if (coded_range < 2)
           coded_range = 2
       elseif (126 < coded_range)
           coded_range = 126
       end
   end
   return coded_range::UInt32
end
```

Referenced In: DetermineTTIAndTID(p. 387)

ENCODETIDBEARING (Algorithm 355) returns the coded relative bearing. The relative bearing is input in radians. The coded relative bearing is in degrees.

If the relative bearing value is not set (it is *NaN*), the coded bearing is *0*.

Otherwise, the relative bearing is first converted from radians to degrees. WRAPTOPI is used to ensure the radian value used for the conversion is between $-\pi$ and $+\pi$. After adjustment, the resulting relative bearing will be between 0 and 360 degrees.

The coded relative bearing is 6 bits, supporting values from *1* to *63*, inclusive. However, the values *61*, *62*, and *63* are not used. A relative bearing of *5.0* degrees is coded as *1*. A relative bearing of *359.0* degrees is coded as *60*.

The coded bearing is returned as an unsigned integer.

This algorithm takes as input *bearing*. This algorithm returns *coded_bearing*.

Algorithm 355 EncodeTIDBearing

```
function EncodeTIDBearing( bearing::R )
    coded_bearing::UInt32 = 0
    if !isnan( bearing )
        bearing = rad2deg( WrapToPi(p. 583)( bearing ) )
        if (bearing < 0.0)
            bearing = bearing + 360.0
        end
        coded_bearing = UInt32( 1 + floor( bearing / 6.0 ) )
        if (coded_bearing > TIDB_MAX_CODE)
            coded_bearing = TIDB_MAX_CODE
        end
    end
    return coded_bearing::UInt32
end
```

Referenced In: DetermineTTIAndTID(p. 387)

3.3.6.4.2 Broadcast Interrogation Message

SETRABROADCASTDATA (Algorithm 356) populates the SXUTRMBROADCASTDATA data structure for the Vertical TRM Report.

The inputs are VTRMRADATA (*vertical_ra_data*), which contains all the current RA data for the vertical dimension, the threat index (*idx_tid*) for the most recent threat, the list of intruder inputs (*input_int_valid*) from the STM for all valid intruders, and barometric altitude in feet (*z_own_-ave*) for own aircraft.

The *idx_tid* is the ID of the threat used to set all of the threat identity data. If the *idx_tid* is a negative number, it signals there is no current RA for this cycle and the threat identity data is omitted.

First, the VTRMRADATA AVRA data is copied and populated in SXUTRMBROADCASTDATA data structure (*broadcast_msg*. Next, the RA termination bit is set and it will only be utilized in the final compilation of the RA broadcast report during blending if all RAs in both dimensions have terminated. This is accomplished in GENERATEBLENDEDSXUTRMOUTPUT (Algorithm 360). Next the low descent inhibit (LDI) bit is set as well as the RA Complements (RAC), of which the first two bits are for the vertical dimension.

If it's determined there is an active RA, DETERMINETTIANDTID (Algorithm 352) is used to retrieve the current Threat Type Indicator (TTI), Threat Identifier Data (TID), and the Boolean value indicating if the TID is an ICAO 24-bit address values (*is_icao*). Finally, the Collision Avoidance Coordination Capability Bits (CCCB) are set for the current threat.

The output from SETRABROADCASTDATA is a populated SXUTRMBROADCASTDATA data structure (*broadcast_msg*).

Algorithm 356 SetRABroadcastData

```
function SetRABroadcastData( vertical_ra_data::VTRMRAData(p.508), idx_tid::Z,
                        input_int_valid::Vector{TRMIntruderInput(p.499)},
                        z_own_ave::R, sxu_trm_state::sXuTRMState(p.510) )
    # Deviation potential: Refer to ACAS Xu MOPS Vol. I section 1
    broadcast_msg = sXuTRMBroadcastData(p.507)()
    broadcast_msg.avra_match    = vertical_ra_data.avra_single_intent
    broadcast_msg.avra_crossing = vertical_ra_data.avra_crossing
    broadcast_msg.avra_sense    = vertical_ra_data.avra_down
    broadcast_msg.avra_strength = vertical_ra_data.avra_strength
    broadcast_msg.rmf           = RMF_ACAS_sXu
    broadcast_msg.rat           = vertical_ra_data.ra_term # Indicates the vertical RA has terminated and
        will be used in blending
    broadcast_msg.ldi           = vertical_ra_data.ldi
    broadcast_msg.rac[1]        = vertical_ra_data.rac[1]
    broadcast_msg.rac[2]        = vertical_ra_data.rac[2]
    if (0 < idx_tid) && (idx_tid <= length( input_int_valid ))
        broadcast_msg.vmte = vertical_ra_data.vmte
        current_tgtid_tid::Z = input_int_valid[idx_tid].id
        if (current_tgtid_tid != sxu_trm_state.st_vert.st_own.ra_output_prev.tgtid_tid)
            sxu_trm_state.st_vert.st_own.ra_output_prev.tgtid_tid = current_tgtid_tid
        end
    else
        sxu_trm_state.st_vert.st_own.ra_output_prev.tgtid_tid = 0
    end
    for i in 1:length( sxu_trm_state.st_vert.st_intruder )
        if !sxu_trm_state.st_vert.st_intruder[i].is_threat
            sxu_trm_state.st_vert.st_intruder[i].is_identified_threat = false
        end
    end
    return (broadcast_msg::sXuTRMBroadcastData(p.507))
end
```

Referenced In: SetVerticalRAMessageOutput(p.386)

3.3.6.5 Intruder Management

The intruder inputs to the TRM (*input.intruder*) correspond to the set of active tracks produced by the ACAS X Surveillance and Tracking Module (STM) within the threat volume around own aircraft. The TRM must handle cases when no tracks are available, as might happen when ACAS X is first starting up or the aircraft is enroute. It must also handle cases when the STM drops a track or a received intruder is not valid, as might happen with degraded surveillance. When that happens, the TRM must ensure the appropriate output is produced so the display and coordination messages will reflect the correct advisory state.

DROPPEDINTRUDERSADJUSTMENT (Algorithm 357) processes all intruder states and produces an array of intruder outputs for dropped and invalid intruders. It also produces an updated array containing intruder states for all intruders that were not dropped.

After initializing output arrays, this algorithm creates a dictionary of intruder outputs (*output_-int_dict*). The dictionary is composed of intruders that were processed for advisories by the vertical TRM (*output_int*). The key used to find intruders in this dictionary is the target id (*output_-int[i].id*).

DROPPEDINTRUDERSADJUSTMENT next creates a dictionary of invalid intruders input to the TRM (*input_invalid_int_dict*). The dictionary is composed of intruders that were skipped over for pro-

cessing by the vertical TRM (*input_invalid_int*). Only intruder inputs to the TRM flagged as valid are processed by the TRM. Intruder inputs to the TRM that are flagged as invalid are processed only by DROPPEDINTRUDERSADJUSTMENT. The key used to find intruders in this dictionary also is the target id (*input_invalid_int[i].id*).

The remainder of the algorithm is a loop over the list of intruder states. For each state, the first check is whether the intruder associated with this state was in the output, in which case the *status* is marked as *InUse* and the intruder state (*st_int_next*) is stored for use in the next TRM processing cycle in *st_int_out*.

If there is no intruder output and the intruder wasn't found in the list of invalid intruders, the intruder was dropped from the STM outputs. This situation is handled by SETDROPPEDINTRUDER. The state information for this intruder is not retained.

Otherwise, if there is no intruder output and the intruder is found in the list of invalid intruders, the intruder could be an invalid input for one of two reasons:

1. The intruder has degraded surveillance that is inadequate for use in producing advisories.
2. The ACAS X system is in Standby Operational Mode.

In either case, the invalid intruder is handled by SETINVALIDINTRUDER and the state information is always retained.

Whenever the intruder was not found in the output intruder dictionary, an output is created for that intruder (*dropped_int*) and added to the list (*dropped_int_out*).

This algorithm takes as input *mode_s_own*, *multithreat*, *input_int_invalid*, *output_int*, and *st_int*. This algorithm returns the list of intruder states (*st_int_out*) and the list of dropped intruder outputs (*dropped_int_out*).

Algorithm 357 DroppedIntrudersAdjustment

```
function DroppedIntrudersAdjustment( v2v_uid_own::UInt32, multithreat::Bool,
                            input_int_invalid::Vector{TRMIntruderInput(p. 499)},
                            output_int::Vector{TRMIntruderData(p. 523)},
                            st_int::Vector{TRMIntruderState(p. 512)} )
    dropped_int_out::Vector{TRMIntruderData(p. 523)} = Vector{TRMIntruderData(p. 523)}( )
    st_int_out::Vector{TRMIntruderState(p. 512)}     = Vector{TRMIntruderState(p. 512)}( )
    output_int_dict::Dict{Z,TRMIntruderData(p. 523)} = Dict{Z, TRMIntruderData(p. 523)}()
    input_invalid_int_dict::Dict{Z,TRMIntruderInput(p. 499)} = Dict{Z, TRMIntruderInput(p. 499)}()
    for i in 1:length( output_int )
        output_int_dict[output_int[i].id] = output_int[i]
    end
    for i in 1:length( input_int_invalid )
        input_invalid_int_dict[input_int_invalid[i].id] = input_int_invalid[i]
    end
    for st_int_next in st_int
        id::UInt32 = st_int_next.id
        if !haskey( output_int_dict, id )
            dropped_int::TRMIntruderData(p. 523) = TRMIntruderData(p. 523)()
            if !haskey( input_invalid_int_dict, id )
                dropped_int = SetDroppedIntruder(p. 394)( id, v2v_uid_own, multithreat,
                                                st_int_next )
            else
                dropped_int = SetInvalidIntruder(p. 395)( id, v2v_uid_own, multithreat,
                                                input_invalid_int_dict[id], st_int_next )
                push!( st_int_out, st_int_next )
            end
            push!( dropped_int_out, dropped_int )
        else
            st_int_next.status = :InUse
            push!( st_int_out, st_int_next )
        end
    end
    return (dropped_int_out::Vector{TRMIntruderData(p. 523)}, st_int_out::Vector{TRMIntruderState(p. 512)})
end
```

Referenced In: GenerateVTRMOutput(p.376)

SETDROPPEDINTRUDER (Algorithm 358) handles clean up functions for a dropped intruder. As no intruder inputs are available for a dropped intruder, the intruder state information (*st_int*) is used for preparing the outputs.

This algorithm first adjusts the display and coordination settings. For a dropped intruder, any advisories are cleared from the pilot display (*code* is set to *SXUCODE_CLEAR*) and this intruder is given the lowest display priority (*tds* is set to *0.0*). Next, CROSSLINK is used to produce a CANCEL coordination message for this intruder whenever a coordination message was sent to the intruder on the previous cycle. The coordination settings for the previous cycle are stored in *st_int.vrc_prev* and *st_int.cvc_prev*.

The processing mode is set to *RA_PROCESSING_DROPPED_TRACK* and the resolution advisory state variables are reset.

Next, the TRM state for that intruder is set appropriately using TRMINTRUDERSTATEUPDATE.The intruder outputs are returned in TRMINTRUDERDATA (*dropped_int*).

This algorithm takes as input *id_int*, *mode_s_own*, *multithreat*, and *st_int*. This algorithm returns *dropped_int*.

Algorithm 358 SetDroppedIntruder

```
function SetDroppedIntruder( id_int::UInt32, v2v_uid_own::UInt32, multithreat::Bool,
                     st_int::TRMIntruderState(p. 512) )
    code::UInt8 = SXUCODE_CLEAR
    tds::R      = 0.0
    (cvc::UInt32, vrc::UInt32, vsb::UInt32, sense_indiv::Symbol) =
        Crosslink(p. 362)( 0.0, 0.0, st_int.vrc_prev, st_int.cvc_prev, st_int.equipage_prev )
    st_int.processing    = RA_PROCESSING_DROPPED_TRACK
    st_int.a_prev.sense  = sense_indiv
    st_int.status = :MarkedForDeletion
    TRMIntruderStateUpdate(p. 434)( st_int, vrc, cvc, st_int.equipage_prev,
                            st_int.coordination_msg_prev, true, true )
    st_int.is_identified_threat = false
    dropped_int::TRMIntruderData(p. 523) =
        TRMIntruderData(p. 523)( id_int, :None, v2v_uid_own, st_int.id_directory,
                        cvc, vrc, vsb, multithreat, st_int.coordination_msg_prev,
                        tds, code )
    return dropped_int::TRMIntruderData(p. 523)
end
```

Referenced In: DroppedIntrudersAdjustment(p. 393)

SETINVALIDINTRUDER (Algorithm 359) handles clean up functions for an intruder flagged as invalid. Intruder inputs (*intruder*) are available when an intruder is flagged as invalid. These inputs are used for updating the state information and providing more informative outputs.

SETINVALIDINTRUDER first uses CROSSLINK to produce a CANCEL coordination message for this intruder when a coordination message was sent to the intruder previously.

It next determines the type of advisory that should be sent to the onboard pilot display depending on the current advisory. DETERMINECODE sets the advisory code (*code*).

DETERMINEVERTICALSCORE sets the track display score (*tds*) based on the code, surveillance quality and whether the intruder is equiped (*equip_int*) as *false*.

The processing state is set to *RA_PROCESSING_NONE*. Finally, the TRM state for that intruder is set appropriately, using TRMINTRUDERSTATEUPDATE. The intruder outputs are stored and returned in TRMINTRUDERDATA (*invalid_int*).

This algorithm takes as input *id_int*, *mode_s_own*, *multithreat*, *intruder*, and *st_int*. This algorithm returns *invalid_int*.

Algorithm 359 SetInvalidIntruder

```
function SetInvalidIntruder( id_int::UInt32, v2v_uid_own::UInt32, multithreat::Bool,
                    intruder::TRMIntruderInput(p. 499), st_int::TRMIntruderState(p. 512) )
    (cvc::UInt32, vrc::UInt32, vsb::UInt32, sense_indiv::Symbol) =
        Crosslink(p. 362)( 0.0, 0.0, st_int.vrc_prev, st_int.cvc_prev, intruder.equipage )
    code::UInt8 = DetermineCode(p. 372)( sense_indiv )
    tds::R =
        DetermineVerticalScore(p. 373)( 0.0, code, intruder.degraded_surveillance, false )
    st_int.status               = :NotValid
    st_int.processing           = RA_PROCESSING_NONE
    st_int.a_prev.sense         = sense_indiv
    st_int.is_identified_threat = false
    TRMIntruderStateUpdate(p. 434)( st_int, vrc, cvc, intruder.equipage,
                            intruder.coordination_msg, true, true )
    invalid_int::TRMIntruderData(p. 523) =
        TRMIntruderData(p. 523)( id_int, :None, v2v_uid_own, intruder.id_directory,
                         cvc, vrc, vsb, multithreat, intruder.coordination_msg,
                         tds, code )
    return invalid_int::TRMIntruderData(p. 523)
end
```

Referenced In: DroppedIntrudersAdjustment(p. 393)

3.4 Generate Blended sXu TRM Output

GENERATEBLENDEDSXUTRMOUTPUT (Algorithm 360) blends the results from the vertical TRM and horizontal TRM for output. This algorithm prepares the final SXUTRMREPORT of which parts are sent to the display and periodically broadcast.

SXUTRMBLENDEDREPORTPREP creates the initial SXUTRMREPORT and populates the reports with default values that are unique to the vertical and horizontal dimensions. Also updated, is the INTRUDERPUBLISHEDRASTATE dictionary contained in the SXUTRMSTATE structure with any intruders that are currently causing RAs.

SETBLENDEDINTRUDERTHREATDATA then sets the Collision Avoidance Coordination Capability Bits (CCCB), the Threat Type Indicator (TTI), Threat identifier Data (TID) and the Boolean value indicating if the TID is an ICAO 24-bit address.

SETREPORTDATAONRATERM sets the RA termination data as well as coordinating persistence of data during termination of vertical and horizontal RAs during multi-dimension threat conditions.

DETERMINEMULTIDIMENSIONALMULTITHREAT will then determine if there is a multi-threat encounter involving single unique intruders, each of which are unique to the vertical and horizontal dimensions. If the condition is met, vertical and horizontal multi-threat indicators (*vmte* and *hmte*) are set in the reports.

SETBLENDEDCOORDINATIONDATA is then called to combine the vertical and horizontal coordination data (TRMCOORDINATIONINTERROGATIONDATA).

Next, the indicator that RAs are inhibited based on the operational mode will be set in the *report.alerting_data.ras_inhibited* field.

This algorithm takes as input *trm_state*, *trm_input*, *report_vert*, and *report_horiz*. This algorithm returns the *report* for the current cycle of type SXUTRMREPORT.

Algorithm 360 GenerateBlendedsXuTRMOutput

```
function GenerateBlendedsXuTRMOutput( trm_state::sXuTRMState(p.510), trm_input::TRMInput(p.497),
                                      report_vert::VTRMReport(p.524), report_horiz::HTRMReport(p.519) )
    report::sXuTRMReport(p.501) = sXuTRMBlendedReportPrep(p.397)(trm_state, trm_input, report_vert, report_horiz
        )
    SetBlendedIntruderThreatData(p.400)(report, trm_input, trm_state)
    SetReportDataOnRATerm(p.403)(report, trm_state.broadcast_report_prev, report_vert, report_horiz)
    multidim_multithreat::Bool = DetermineMultidimensionalMultithreat(p.408)(report)
    SetBlendedCoordinationData(p.409)(report, report_horiz, multidim_multithreat)
    report.alerting_data.warning_alert = ((report.display_horiz.cc != CC_H_COC) && (report.display_horiz.cc
        != CC_H_NO_ADVISORY)) || ((report.display_vert.cc != CC_V_COC) && (report.display_vert.cc !=
        CC_V_NO_ADVISORY))
    opmode::UInt8 = trm_input.own.opmode
    report.alerting_data.ras_inhibited = (opmode != OPMODE_RA)
    if (report.broadcast_msg.rat)
        trm_state.broadcast_report_prev = sXuTRMBroadcastData(p.507)()
    else
        trm_state.broadcast_report_prev = deepcopy(report.broadcast_msg)
    end
    return report::sXuTRMReport(p.501)
end
```

Referenced In: sXuTRMUpdate(p.164)

sXuTRMBlendedReportPrep (Algorithm 361) initializes parts of the final sXuTRMReport report that are unique to the vertical and horizontal TRM.

This algorithm takes as input:

- sXuTRMState (*trm_state*) - updated to include new intruders with active RAs in this cycle
- TRMInput (*trm_input*) - contains the ownship address
- VTRMReport (*report_vert*) - used to set specific vertical data and is the seed for the *display*, *coordination* and *designation* report items
- HTRMReport (*report_horiz*) - used to set specific horizontal data

This algorithm returns sXuTRMReport (*report*) which contains the reports for the current cycle. It will be modified by additional blending logic prior to sending.

Algorithm 361 sXuTRMBlendedReportPrep

```
function sXuTRMBlendedReportPrep(trm_state::sXuTRMState(p.510), trm_input::TRMInput(p.497), report_vert::
    VTRMReport(p.524), report_horiz::HTRMReport(p.519))
    report::sXuTRMReport(p.501)     = sXuTRMReport(p.501)()
    report.display_vert      = deepcopy( report_vert.display )
    report.display_horiz     = deepcopy( report_horiz.display )
    report.coordination      = deepcopy( report_vert.coordination )
    report.designation       = deepcopy( report_vert.designation )
    UpdateRaPubState(p.398)(trm_state, report)
    report.broadcast_msg.hri = report_horiz.broadcast_msg.hri
    report.broadcast_msg.ldi = report_vert.broadcast_msg.ldi
    report.broadcast_msg.rac[1] = report_vert.broadcast_msg.rac[1]
    report.broadcast_msg.rac[2] = report_vert.broadcast_msg.rac[2]
    report.broadcast_msg.rac[3] = report_horiz.broadcast_msg.rac[3]
    report.broadcast_msg.rac[4] = report_horiz.broadcast_msg.rac[4]
    report.broadcast_msg.ahra_match       = report_horiz.broadcast_msg.ahra_match
    report.broadcast_msg.ahra_turn        = report_horiz.broadcast_msg.ahra_turn
    report.broadcast_msg.ahra_track_angle = report_horiz.broadcast_msg.ahra_track_angle
    report.broadcast_msg.avra_match       = report_vert.broadcast_msg.avra_match
    report.broadcast_msg.avra_crossing    = report_vert.broadcast_msg.avra_crossing
    report.broadcast_msg.avra_sense       = report_vert.broadcast_msg.avra_sense
    report.broadcast_msg.avra_strength    = report_vert.broadcast_msg.avra_strength
    report.broadcast_msg.vmte = report_vert.broadcast_msg.vmte
    report.broadcast_msg.hmte = report_horiz.broadcast_msg.hmte
    report.broadcast_msg.mid     = trm_input.own.v2v_uid
    report.broadcast_msg.agt_ids = trm_input.own.agt_ids
    return report::sXuTRMReport(p.501)
end
```

Referenced In: GenerateBlendedsXuTRMOutput(p.396)

UPDATERAPUBSTATE (Algorithm 362) adds any new intruders contributing to an RA and removes any intruders that have terminated RAs in the INTRUDERPUBLISHEDRASTATE dictionary, which is contained in the SXUTRMSTATE structure.

First, the vertical and horizontal intruder display lists are parsed to find all the intruders contributing to RAs. All the IDs of intruders contributing to RAs are added to a temporary list (*active_ra_ids*) and if the intruder is not in the INTRUDERPUBLISHEDRASTATE dictionary, it is added.

Lastly the temporary RA list (*active_ra_ids*) is used to compare all the intruders in the INTRUDERPUBLISHEDRASTATE dictionary and if any of the ID's in the INTRUDERPUBLISHEDRASTATE dictionary are not in the active list, they are removed from the INTRUDERPUBLISHEDRASTATE dictionary.

This algorithm takes as input:

- SXUTRMSTATE (*trm_state*) - updated to include only intruders with active RAs
- SXUTRMREPORT (*report*) - used to get vertical and horizontal intruder lists for the current cycle

This algorithm modifies the SXUTRMSTATE (*trm_state*) structure.

Algorithm 362 UpdateRaPubState

```
function UpdateRaPubState(trm_state::sXuTRMState(p. 510), report::sXuTRMReport(p. 501))
    active_ra_ids::Vector{UInt32} = UInt32[]
    for v_intruder::TRMIntruderDisplayData(p. 503) in report.display_vert.intruder
        if (v_intruder.code == SXUCODE_RA)
            push!(active_ra_ids, v_intruder.id)
            if !haskey(trm_state.int_published_ra_state, v_intruder.id)
                newInt::IntruderPublishedRaState(p. 525) = IntruderPublishedRaState(p. 525)(v_intruder.id)
                trm_state.int_published_ra_state[v_intruder.id] = newInt
            end
        end
    end
    for h_intruder::HTRMIntruderDisplayData(p. 504) in report.display_horiz.intruder
        if (h_intruder.code == SXUCODE_RA)
            push!(active_ra_ids, h_intruder.id)
            if !haskey(trm_state.int_published_ra_state, h_intruder.id)
                newInt::IntruderPublishedRaState(p. 525) = IntruderPublishedRaState(p. 525)(h_intruder.id)
                trm_state.int_published_ra_state[h_intruder.id] = newInt
            end
        end
    end
    for int_id::UInt32 in keys(trm_state.int_published_ra_state)
        if !in(int_id, active_ra_ids)
            delete!(trm_state.int_published_ra_state, int_id)
        end
    end
    return
end
```

Referenced In: sXuTRMBlendedReportPrep(p. 397)

SETBLENDEDINTRUDERTHREATDATA (Algorithm 363) sets the Threat Identity Data (TID) and the Collision Avoidance Coordination Capability Bits (CCCB) from the vertical and horizontal TRM. The results are combined in the ground and broadcast RA reports SXUTRMREPORT.

SETBLENDEDINTRUDERTHREATDATA depends on SXUTRMBLENDEDREPORTPREP and UPDATERAPUBSTATE to present the current list of active threats causing RAs in the INTRUDERPUBLISHEDRASTATE dictionary contained in the SXUTRMSTATE structure. The INTRUDERPUBLISHEDRASTATE dictionary contains the threat ID and indication of RA report publish state for all the intruders with active RAs. This RA publish state is utilized throughout the intruder's RA lifecycle to determine when each threat will be presented in the current cycle's RA ground and broadcast reports threat identity fields.

First all the ID's for threats with RAs are gathered and sorted into a list (*active_ra_ids*) from the INTRUDERPUBLISHEDRASTATE dictionary keys contained in the SXUTRMSTATE structure.

Next, the sorted ID list is utilized to determine if any of the intruders with active RAs had not been published yet. If any unpublished ID's are found, the threats ID is recorded in *found_unpublish_id* and the intruder's identity data is set in the ground and broadcast RA report messages by calling SETTHREATIDENTITYDATA. Specifically, the data is set in the first TID container for the ground RA message and the single TID container for the broadcast RA message. Then the new intruder's publish state is updated to reflect its publish state for the following cycles by calling SETRAPUBLISHEDSTATE.

Next, the condition that indicates a new threat was found and there is a multi-threat encounter taking place is checked. If this condition is met the first threat that is not the current new threat

is set in the second TID fields in the ground RA message and this threat's publish data is updated to indicate it is currently being published in the second TID fields.

If the previous condition is not met and it is determined there is only a single threat causing an RA, the threats identity data is set for publishing in the first TID fields for the ground and broadcast RA messages. Then this threat's publish data is updated to reflect its publish state.

If the previous two conditions are not met and there is more than one threat with active RAs, the current threats are rotated through the TID one and TID two fields of the ground and broadcast RA messages until the RAs terminate. The *found_prev_tid1* is checked in this state because of the chance that the intruder, which was published in the first TID fields from the previous cycle's RA reports, has gone clear of conflict and it's RA has terminated. In this case the rotation is restarted by using the first threat in the sorted active RA list (*active_ra_ids*).

This algorithm takes as input:

- sXuTRMReport (*report*) - modified here to reflect the threat identity data for the current cycle
- TRMInput (*trm_input*) - the intruder data needed for the RA reports
- sXuTRMState (*trm_state*) - the threat identity publish state utilized and modified

This algorithm modifies sXuTRMReport (*report*) which contains the reports for the current cycle.

Algorithm 363 SetBlendedIntruderThreatData

```
function SetBlendedIntruderThreatData( report::sXuTRMReport(p.501), trm_input::TRMInput(p.497), trm_state::
    sXuTRMState(p.510))
    FIRST_SET::Bool = true
    SECOND_SET::Bool = false
    id_idx::Z = 0
    pub_state::IntruderPublishedRaState(p.525) = IntruderPublishedRaState(p.525)(UInt32(0))
    found_unpublish_id::UInt32 = 0
    active_ra_ids = sort!(collect(keys(trm_state.int_published_ra_state)))
    for index = 1:length(active_ra_ids)
        pub_state = trm_state.int_published_ra_state[active_ra_ids[index]]
        if !pub_state.has_been_published
            found_unpublish_id = pub_state.id
            id_idx = findfirst(x -> x.id == active_ra_ids[index], trm_input.intruder)
            SetThreatIdentityData(p.401)(report, trm_input, id_idx, FIRST_SET)
            SetRaPublishedState(p.402)(trm_state, found_unpublish_id, FIRST_SET)
            break
        end
    end
    if (found_unpublish_id > 0) && (length(active_ra_ids) > 1)
        for index = 1:length(active_ra_ids)
            if (active_ra_ids[index] != found_unpublish_id)
                id_idx = findfirst(x -> x.id == active_ra_ids[index], trm_input.intruder)
                SetThreatIdentityData(p.401)(report, trm_input, id_idx, SECOND_SET)
                SetRaPublishedState(p.402)(trm_state, active_ra_ids[index], SECOND_SET)
                break
            end
        end
    elseif (found_unpublish_id == 0) && (length(active_ra_ids) == 1)
        id_idx = findfirst(x -> x.id == UInt32(active_ra_ids[1]), trm_input.intruder)
        SetThreatIdentityData(p.401)(report, trm_input, id_idx, FIRST_SET)
        SetRaPublishedState(p.402)(trm_state, trm_input.intruder[id_idx].id, FIRST_SET)
    elseif (found_unpublish_id == 0) && (length(active_ra_ids) > 1)
        found_prev_tid1::Bool = false
        for index = 1:length(active_ra_ids)
            pub_state = trm_state.int_published_ra_state[active_ra_ids[index]]
            if pub_state.included_in_prev_tid1
                found_prev_tid1 = true
                id_idx = findfirst(x -> x.id == UInt32(active_ra_ids[index]), trm_input.intruder)
                SetThreatIdentityData(p.401)(report, trm_input, id_idx, SECOND_SET)
                SetRaPublishedState(p.402)(trm_state, active_ra_ids[index], SECOND_SET)
                if (index == length(active_ra_ids))
                    id_idx = findfirst(x -> x.id == UInt32(active_ra_ids[1]), trm_input.intruder)
                    SetThreatIdentityData(p.401)(report, trm_input, id_idx, FIRST_SET)
                    SetRaPublishedState(p.402)(trm_state, active_ra_ids[1], FIRST_SET)
                    break
                else
                    id_idx = findfirst(x -> x.id == UInt32(active_ra_ids[index+1]), trm_input.intruder)
                    SetThreatIdentityData(p.401)(report, trm_input, id_idx, FIRST_SET)
                    SetRaPublishedState(p.402)(trm_state, active_ra_ids[index+1], FIRST_SET)
                    break
                end
            end
        end
        if !found_prev_tid1
            id_idx = findfirst(x -> x.id == UInt32(active_ra_ids[1]), trm_input.intruder)
            SetThreatIdentityData(p.401)(report, trm_input, id_idx, FIRST_SET)
            SetRaPublishedState(p.402)(trm_state, trm_input.intruder[id_idx].id, FIRST_SET)
            id_idx = findfirst(x -> x.id == UInt32(active_ra_ids[2]), trm_input.intruder)
            SetThreatIdentityData(p.401)(report, trm_input, id_idx, SECOND_SET)
            SetRaPublishedState(p.402)(trm_state, trm_input.intruder[id_idx].id, SECOND_SET)
        end
    end
    return
end
```

Referenced In: GenerateBlendedsXuTRMOutput(p.396)

SETTHREATIDENTITYDATA (Algorithm 364) sets the Threat Identity Data in the RA reports for the intruder designated by the ID index (*id_idx*).

The intruder list in TRMINPUT (*trm_input.intruder*) is accessed by ID index (*id_idx*), which are passed to DETERMINETTIANDTID to get the TID, TTI and the Boolean flag indicating if the intruder has a valid ICO address. Also passed to DETERMINETTIANDTID is the weighted average altitude of the ownship, *z_own_ave*.

The last argument of this function is a flag that indicates if the first or second threat identity fields should be set.

This algorithm takes as input:

- SXUTRMREPORT (*report*) - modified here to reflect the threat identity data for the current cycle
- TRMINPUT (*trm_input*) - the intruder data needed for the RA reports
- *id_idx* - the array index of the threat being published
- *first* - flag that indicates if the first or second TID and CCCB fields should be set

This algorithm modifies SXUTRMREPORT (*report*) which contains the RA reports for the current cycle.

Algorithm 364 SetThreatIdentityData

```
function SetThreatIdentityData(report::sXuTRMReport(p. 501), trm_input::TRMInput(p. 497), id_idx::Z, first::Bool)
    ( z_own_ave::R, _ ) = GetOwnWeightedAverages(p. 243)( trm_input.own.belief_vert )
    (tti, tid, is_icao) = DetermineTTIAndTID(p. 387)( id_idx, trm_input.intruder, z_own_ave )
    if first
        report.broadcast_msg.tid = tid
        report.broadcast_msg.tti = tti
        report.broadcast_msg.ico = is_icao
        report.broadcast_msg.intruder_ids = deepcopy(trm_input.intruder[id_idx].id_directory)
        report.broadcast_msg.adsb_ccb     = deepcopy(trm_input.intruder[id_idx].adsb_cccb)
        report.broadcast_msg.v2v_ccb      = deepcopy(trm_input.intruder[id_idx].v2v_cccb)
    else
        report.broadcast_msg.tid2 = tid
        report.broadcast_msg.tti2 = tti
        report.broadcast_msg.ico2 = is_icao
        report.broadcast_msg.intruder_ids2 = deepcopy(trm_input.intruder[id_idx].id_directory)
        report.broadcast_msg.adsb_ccb2     = deepcopy(trm_input.intruder[id_idx].adsb_cccb)
        report.broadcast_msg.v2v_ccb2      = deepcopy(trm_input.intruder[id_idx].v2v_cccb)
    end
    return
end
```

Referenced In: SetBlendedIntruderThreatData(p. 400)

SETRAPUBLISHEDSTATE (Algorithm 365) manages the INTRUDERPUBLISHEDRASTATE dictionary contained in the SXUTRMSTATE structure. The INTRUDERPUBLISHEDRASTATE dictionary is used to determine which intruders need to be published in the RA reports.

This algorithm takes as input:

- SXUTRMSTATE (*trm_state*) - the threat identity publish state utilized and modified

- *int_id* - the ID of intruder being updated
- *first* - flag that indicates if the first or second TID and CCCB fields have been published for the intruder

This algorithm modifies sXuTRMState (*trm_state*).

Algorithm 365 SetRaPublishedState

```
function SetRaPublishedState(trm_state::sXuTRMState(p.510), int_id::UInt32, first::Bool)
    active_ra_ids = keys(trm_state.int_published_ra_state)
    for id in active_ra_ids
        pub_state::IntruderPublishedRaState(p.525) = trm_state.int_published_ra_state[id]
        if (id == int_id) && first
            pub_state.has_been_published = true
            pub_state.included_in_prev_tid1 = true
            pub_state.included_in_prev_tid2 = false
        elseif (id == int_id) && !first
            pub_state.included_in_prev_tid1 = false
            pub_state.included_in_prev_tid2 = true
        elseif (id != int_id) && first
            pub_state.included_in_prev_tid1 = false
        else
            pub_state.included_in_prev_tid2 = false
        end
    end
    return
end
```

Referenced In: SetBlendedIntruderThreatData(p.400)

SetReportDataOnRATerm (Algorithm 366) sets RA data from previous cycles during RA termination (RAT) states.

SetReportRATerminationBits determines the current RAT state and sets the *rat* bit in both the ground and broadcast RA reports, as well as setting the *vrat* and *hrat* bits in the ground RA report.

The *rat*, *vrat* and *hrat* are then used to determine which data from the previous cycle should be set in the ground and broadcast RA report.

This algorithm takes as input:

- sXuTRMReport (*report*) - modified here to reflect the RAT state for the current cycle
- sXuTRMBroadcastData (*broadcast_report_prev*) - the RA broadcast report, used to set the data in the event of RA termination
- VTRMReport (*report_vert*) - used to determine the vertical dimension RAT state
- HTRMReport (*report_horiz*) - used to determine the horizontal dimension RAT state

This algorithm modifies sXuTRMBroadcastData (*report.broadcast_msg*) which contains the reports for the current cycle.

Algorithm 366 SetReportDataOnRATerm

```
function SetReportDataOnRATerm( report::sXuTRMReport(p.501), broadcast_report_prev::sXuTRMBroadcastData(p.507),
                                report_vert::VTRMReport(p.524), report_horiz::HTRMReport(p.519) )
    SetReportRATerminationBits(p.404)(report, broadcast_report_prev::sXuTRMBroadcastData(p.507), report_vert,
        report_horiz)
    if(report.broadcast_msg.vrat == true)
        SetPrevVerticalTerminationData(p.405)(report, broadcast_report_prev)
    elseif(report.broadcast_msg.hrat == true)
        SetPrevHorizontalTerminationData(p.406)(report, broadcast_report_prev)
    elseif(report.broadcast_msg.rat == true) && (report.broadcast_msg.vrat == false) && (report.
        broadcast_msg.hrat == false)
        report.broadcast_msg.avra_match       = broadcast_report_prev.avra_match
        report.broadcast_msg.avra_crossing    = broadcast_report_prev.avra_crossing
        report.broadcast_msg.avra_sense       = broadcast_report_prev.avra_sense
        report.broadcast_msg.avra_strength    = broadcast_report_prev.avra_strength
        report.broadcast_msg.ahra_match       = broadcast_report_prev.ahra_match
        report.broadcast_msg.ahra_turn        = broadcast_report_prev.ahra_turn
        report.broadcast_msg.ahra_track_angle = broadcast_report_prev.ahra_track_angle
        report.broadcast_msg.vmte             = broadcast_report_prev.vmte
        report.broadcast_msg.hmte             = broadcast_report_prev.hmte
        report.broadcast_msg.tti              = broadcast_report_prev.tti
        report.broadcast_msg.tid              = broadcast_report_prev.tid
        report.broadcast_msg.ldi              = broadcast_report_prev.ldi
        report.broadcast_msg.hri              = broadcast_report_prev.hri
        report.broadcast_msg.rac              = broadcast_report_prev.rac
        report.broadcast_msg.adsb_ccb         = broadcast_report_prev.adsb_ccb
        report.broadcast_msg.v2v_ccb          = broadcast_report_prev.v2v_ccb
        report.broadcast_msg.ico              = broadcast_report_prev.ico
        report.broadcast_msg.adsb_ccb2        = broadcast_report_prev.adsb_ccb2
        report.broadcast_msg.v2v_ccb2         = broadcast_report_prev.v2v_ccb2
        report.broadcast_msg.ico2             = broadcast_report_prev.ico2
        report.broadcast_msg.tti2             = broadcast_report_prev.tti2
        report.broadcast_msg.tid2             = broadcast_report_prev.tid2
    end
    return
end
```

Referenced In: GenerateBlendedsXuTRMOutput(p.396)

SETREPORTRATERMINATIONBITS (Algorithm 367) determines the current RA Termination (RAT) state for both the vertical and horizontal dimensions. SETREPORTRATERMINATIONBITS will set the *rat* bit in both the ground and broadcast RA reports, as well as setting the *vrat* and *hrat* bits in the ground RA report.

DETERMINEDIMENSIONALRASTATE determines if RAT has been indicated in both the vertical and horizontal TRM from the current and previous cycles.

This algorithm takes as input:

- SXUTRMREPORT (*report*) - modified here to reflect the RA state for the current cycle
- SXUTRMBROADCASTDATA (*previous_broadcast_msg*) - used to determine RAT for both dimensions on the previous cycle
- VTRMREPORT (*report_vert*) - used to determine vertical dimension RAT
- HTRMREPORT (*report_horiz*) - used to determine horizontal dimension RAT

This algorithm modifies SXUTRMBROADCASTDATA (*report.broadcast_msg*).

Algorithm 367 SetReportRATerminationBits

```
function SetReportRATerminationBits( report::sXuTRMReport(p. 501),
                                     previous_broadcast_msg::sXuTRMBroadcastData(p. 507),
                                     report_vert::VTRMReport(p. 524), report_horiz::HTRMReport(p. 519) )
    (prev_vertical_threat::Bool, prev_horizontal_threat::Bool, current_vertical_threat::Bool,
         current_horizontal_threat::Bool) =
            DetermineDimensionalRAState(p. 407)( previous_broadcast_msg, report_vert, report_horiz )
    if (prev_vertical_threat == true) && (prev_horizontal_threat == true) &&
       (current_vertical_threat == false) && (current_horizontal_threat == false)
       report.broadcast_msg.rat = true
       report.broadcast_msg.vrat   = false
       report.broadcast_msg.hrat   = false
    elseif (prev_vertical_threat == true) && (prev_horizontal_threat == true) &&
           (current_vertical_threat == false) && (current_horizontal_threat == true)
       report.broadcast_msg.rat = false
       report.broadcast_msg.vrat   = true
       report.broadcast_msg.hrat   = false
    elseif (prev_vertical_threat == true) && (prev_horizontal_threat == true) &&
           (current_vertical_threat == true) && (current_horizontal_threat == false)
       report.broadcast_msg.rat = false
       report.broadcast_msg.vrat   = false
       report.broadcast_msg.hrat   = true
    end
    if (prev_vertical_threat == true) && (prev_horizontal_threat == false) &&
       (current_vertical_threat == false) && (current_horizontal_threat == false)
       report.broadcast_msg.rat = true
       report.broadcast_msg.vrat   = false
       report.broadcast_msg.hrat = previous_broadcast_msg.hrat
    elseif (prev_vertical_threat == true) && (prev_horizontal_threat == false) &&
           (current_vertical_threat == false) && (current_horizontal_threat == true)
       report.broadcast_msg.rat = false
       report.broadcast_msg.vrat   = true
       report.broadcast_msg.hrat   = false
    elseif (prev_vertical_threat == true) && (prev_horizontal_threat == false) &&
           (current_vertical_threat == true) && (current_horizontal_threat == false)
       report.broadcast_msg.hrat   = previous_broadcast_msg.hrat
       report.broadcast_msg.rat = false
       report.broadcast_msg.vrat   = false
    end
    if (prev_vertical_threat == false) && (prev_horizontal_threat == true) &&
       (current_vertical_threat == false) && (current_horizontal_threat == false)
       report.broadcast_msg.rat = true
       report.broadcast_msg.vrat = previous_broadcast_msg.vrat
       report.broadcast_msg.hrat = false
    elseif (prev_vertical_threat == false) && (prev_horizontal_threat == true) &&
           (current_vertical_threat == true) && (current_horizontal_threat == false)
       report.broadcast_msg.rat = false
       report.broadcast_msg.vrat   = false
       report.broadcast_msg.hrat   = true
    elseif (prev_vertical_threat == false) && (prev_horizontal_threat == true) &&
           (current_vertical_threat == false) && (current_horizontal_threat == true)
       report.broadcast_msg.vrat = previous_broadcast_msg.vrat
       report.broadcast_msg.rat = false
       report.broadcast_msg.hrat   = false
    end
    return
end
```

Referenced In: SetReportDataOnRATerm(p. 403)

SETPREVVERTICALTERMINATIONDATA (Algorithm 368) sets RA data when the vertical TRM has reported an RA termination, though an RA is still active or was active on the previous cycle for the horizontal TRM. In this case the vertical data is preserved until the RA has terminated in

both dimensions. Once the RA has terminated in both dimensions the previous RA data for both dimensions is set in the ground and broadcast report for a final cycle.

This algorithm takes as input:

- sXuTRMReport (*report*) - modified here to reflect the vertical RA state for the previous cycles termination
- sXuTRMBroadcastData (*broadcast_report_prev*) - the previous cycles RA broadcast report

This algorithm modifies sXuTRMReport (*report*) which contains the reports for the current cycle.

Algorithm 368 SetPrevVerticalTerminationData

```
function SetPrevVerticalTerminationData( report::sXuTRMReport(p. 501),
                                         broadcast_report_prev::sXuTRMBroadcastData(p. 507) )
    report.broadcast_msg.avra_match    = broadcast_report_prev.avra_match
    report.broadcast_msg.avra_crossing = broadcast_report_prev.avra_crossing
    report.broadcast_msg.avra_sense    = broadcast_report_prev.avra_sense
    report.broadcast_msg.avra_strength = broadcast_report_prev.avra_strength
    report.broadcast_msg.vmte          = broadcast_report_prev.vmte
    if (report.broadcast_msg.rat == true)
        report.broadcast_msg.adsb_ccb2        = broadcast_report_prev.adsb_ccb2
        report.broadcast_msg.v2v_ccb2         = broadcast_report_prev.v2v_ccb2
        report.broadcast_msg.ico2             = broadcast_report_prev.ico2
        report.broadcast_msg.tti2             = broadcast_report_prev.tti2
        report.broadcast_msg.tid2             = broadcast_report_prev.tid2
        report.broadcast_msg.ahra_match       = broadcast_report_prev.ahra_match
        report.broadcast_msg.ahra_turn        = broadcast_report_prev.ahra_turn
        report.broadcast_msg.ahra_track_angle = broadcast_report_prev.ahra_track_angle
        report.broadcast_msg.hmte             = broadcast_report_prev.hmte
        report.broadcast_msg.rac              = broadcast_report_prev.rac
        report.broadcast_msg.mst              = broadcast_report_prev.mst
        report.broadcast_msg.ldi              = broadcast_report_prev.ldi
        report.broadcast_msg.hri              = broadcast_report_prev.hri
        report.broadcast_msg.adsb_ccb         = broadcast_report_prev.adsb_ccb
        report.broadcast_msg.v2v_ccb          = broadcast_report_prev.v2v_ccb
        report.broadcast_msg.ico              = broadcast_report_prev.ico
        report.broadcast_msg.tti              = broadcast_report_prev.tti
        report.broadcast_msg.tid              = broadcast_report_prev.tid
    end
    return
end
```

Referenced In: SetReportDataOnRATerm(p. 403)

SetPrevHorizontalTerminationData (Algorithm 369) sets RA data when the horizontal TRM has reported an RA termination, though an RA is still active or was active on the previous cycle for the vertical TRM. In this case the horizontal data is preserved until the RA has terminated in both dimensions. Once the RA has terminated in both dimensions the previous RA data for both dimensions is set in the ground and broadcast report for a final cycle.

This algorithm takes as input:

- sXuTRMReport (*report*) - modified here to reflect the horizontal RA state for the previous cycles termination

- sXuTRMBroadcastData (*broadcast_report_prev*) - the previous cycles RA broadcast report

This algorithm modifies sXuTRMBroadcastData (*eport.broadcast_msg*) which contains the reports for the current cycle.

Algorithm 369 SetPrevHorizontalTerminationData

```
function SetPrevHorizontalTerminationData( report::sXuTRMReport(p. 501),
                                           broadcast_report_prev::sXuTRMBroadcastData(p. 507) )
    report.broadcast_msg.ahra_match       = broadcast_report_prev.ahra_match
    report.broadcast_msg.ahra_turn        = broadcast_report_prev.ahra_turn
    report.broadcast_msg.ahra_track_angle = broadcast_report_prev.ahra_track_angle
    report.broadcast_msg.hmte             = broadcast_report_prev.hmte
    if (report.broadcast_msg.rat == true)
        report.broadcast_msg.adsb_ccb2        = broadcast_report_prev.adsb_ccb2
        report.broadcast_msg.v2v_ccb2         = broadcast_report_prev.v2v_ccb2
        report.broadcast_msg.ico2             = broadcast_report_prev.ico2
        report.broadcast_msg.tti2             = broadcast_report_prev.tti2
        report.broadcast_msg.tid2             = broadcast_report_prev.tid2
        report.broadcast_msg.avra_match       = broadcast_report_prev.avra_match
        report.broadcast_msg.avra_crossing    = broadcast_report_prev.avra_crossing
        report.broadcast_msg.avra_sense       = broadcast_report_prev.avra_sense
        report.broadcast_msg.avra_strength    = broadcast_report_prev.avra_strength
        report.broadcast_msg.vmte             = broadcast_report_prev.vmte
        report.broadcast_msg.rac              = broadcast_report_prev.rac
        report.broadcast_msg.mst              = broadcast_report_prev.mst
        report.broadcast_msg.ldi              = broadcast_report_prev.ldi
        report.broadcast_msg.hri              = broadcast_report_prev.hri
        report.broadcast_msg.adsb_ccb         = broadcast_report_prev.adsb_ccb
        report.broadcast_msg.v2v_ccb          = broadcast_report_prev.v2v_ccb
        report.broadcast_msg.ico              = broadcast_report_prev.ico
        report.broadcast_msg.tti              = broadcast_report_prev.tti
        report.broadcast_msg.tid              = broadcast_report_prev.tid
    end
    return
end
```

Referenced In: SetReportDataOnRATerm(p. 403)

DetermineDimensionalRAState (Algorithm 370) determines if RAs have terminated on the current and previous cycle for each dimension

This algorithm takes as input:

- sXuTRMBroadcastData (*previous_broadcast_msg*) - the RA broadcast report, from the previous cycle
- VTRMReport (*report_vert*) - the vertical report data
- HTRMReport (*report_horiz*) - the horizontal report data

This algorithm returns:

- *prev_vertical_threat* - True if there was an RA in the vertical dimension on the previous cycle, false otherwise
- *prev_horizontal_threat* - True if there was an RA in the horizontal dimension on the previous cycle, false otherwise

- *current_vertical_threat* - True if there was an RA in the vertical dimension on the current cycle, false otherwise
- *current_horizontal_threat* - True if there was an RA in the horizontal dimension on the current cycle, false otherwise

Algorithm 370 DetermineDimensionalRAState

```
function DetermineDimensionalRAState( previous_broadcast_msg::sXuTRMBroadcastData(p. 507),
                                      report_vert::VTRMReport(p. 524), report_horiz::HTRMReport(p. 519) )
    prev_vertical_threat::Bool = false
    prev_horizontal_threat::Bool = false
    current_vertical_threat::Bool = false
    current_horizontal_threat::Bool = false
    if (previous_broadcast_msg.avra_strength != AVRA_STRENGTH_NO_ADVISORY) && (previous_broadcast_msg.vrat
        == false)
        prev_vertical_threat = true
    end
    if (previous_broadcast_msg.ahra_turn != AHRA_NO_TURN) && (previous_broadcast_msg.hrat == false)
        prev_horizontal_threat = true
    end
    if (report_vert.broadcast_msg.avra_strength != AVRA_STRENGTH_NO_ADVISORY) && (report_vert.broadcast_msg.
        vrat == false)
        current_vertical_threat = true
    end
    if (report_horiz.broadcast_msg.ahra_turn != AHRA_NO_TURN) && (report_horiz.broadcast_msg.hrat == false)
        current_horizontal_threat = true
    end
    return (prev_vertical_threat::Bool, prev_horizontal_threat::Bool,
            current_vertical_threat::Bool, current_horizontal_threat::Bool)
end
```

Referenced In: SetReportRATerminationBits(p. 404)

DETERMINEMULTIDIMENSIONALMULTITHREAT (Algorithm 371) determines if there is a multidimensional multi-threat encounter where two unique intruders cause RAs in separate dimensions. Note, this is distinct from the multidimensional case where a single intruder causes vertical and horizontal RAs. That case is indicated by the multi-threat bit in the vertical and horizontal reports, respectively whereas the multi-threat case in this algorithm would not be detected.

First, the list of intruder IDs causing vertical RAs is collected. Next, if a single intruder in the vertical dimension is causing an RA and if there is a single different intruder causing horizontal RA, then it is determined to be a multidimensional multi-threat.

This algorithm takes as input *report* and returns *multidim_multithreat*.

This algorithm modifies SXUTRMREPORT (*report*).

Algorithm 371 DetermineMultidimensionalMultithreat

```
function DetermineMultidimensionalMultithreat(report::sXuTRMReport(p. 501))
    multidim_multithreat::Bool = false
    vra_ids::Vector{UInt32} = UInt32[]
    hra_count::Int = 0
    for v_intruder::TRMIntruderDisplayData(p. 503) in report.display_vert.intruder
        if (v_intruder.code == SXUCODE_RA)
            push!(vra_ids, v_intruder.id)
        end
    end
    if (length(vra_ids) == 1)
        for h_intruder::HTRMIntruderDisplayData(p. 504) in report.display_horiz.intruder
            if (h_intruder.code == SXUCODE_RA)
                hra_count += 1
            end
            if (h_intruder.code == SXUCODE_RA) && !in(h_intruder.id, vra_ids)
                multidim_multithreat = true
            end
        end
        if (hra_count > 1)
            multidim_multithreat = false
        end
    end
    if (multidim_multithreat == true)
        report.broadcast_msg.vmte = true
        report.broadcast_msg.hmte = true
        report.broadcast_msg.mst  = true
    end
    return (multidim_multithreat::Bool)
end
```

Referenced In: GenerateBlendedsXuTRMOutput(p. 396)

SETBLENDEDCOORDINATIONDATA (Algorithm 372) combines the vertical and horizontal coordination data.

The horizontal coordination information is first stored in the TRMCOORDINATIONINTERROGATIONDATA structure from *report_horiz.coordination*. It is then used to update the *report.coordination* entry, which holds the vertical coordination information associated with a particular intruder. The vertical coordination data was previously set in SXUTRMBLENDEDREPORTPREP.

The multi-threat bit, *mtb*, is set based on three independent criteria:

- more than one vertical RA among different intruders
- more than one horizontal RA among different intruders
- there is a multi-threat encounter involving single unique intruders in each of the vertical and horizontal dimensions

This algorithm takes as input SXUTRMREPORT (*report*), which was seeded with the vertical coordination data, the HTRMREPORT (*report_horiz*), which contains the horizontal coordination beliefs, and a Boolean value indicating the existence, or lack of, a current multi-threat encounter (*multidim_multithreat*). This algorithm modifies TRMCOORDINATIONINTERROGATIONDATA (*report.coordination*) for each intruder.

Algorithm 372 SetBlendedCoordinationData

```
function SetBlendedCoordinationData(report::sXuTRMReport(p.501), report_horiz::HTRMReport(p.519),
    multidim_multithreat::Bool)
    coord_dict::Dict{UInt32,TRMCoordinationInterrogationData(p.504)} = Dict{UInt32,
        TRMCoordinationInterrogationData(p.504)}()
    for intruder in report_horiz.coordination
        coord_dict[intruder.id] = intruder
    end
    for intruder in report.coordination
        if haskey( coord_dict, intruder.id )
            intruder.chc = coord_dict[intruder.id].chc
            intruder.hrc = coord_dict[intruder.id].hrc
            intruder.hsb = coord_dict[intruder.id].hsb
            intruder.mtb = (intruder.mtb || coord_dict[intruder.id].mtb || multidim_multithreat)
        else
            intruder.chc = 0
            intruder.hrc = 0
            intruder.hsb = 0
        end
    end
    return
end
```

Referenced In: GenerateBlendedsXuTRMOutput(p.396)

3.5 Standard States and Conversions

These algorithms provide definitions for standard ACAS X conversions and behavior. They are used by other algorithms throughout this document.

ACTIONTORATES (Algorithm 373) maps actions (RAs) to their corresponding vertical rate limits and vertical acceleration. Three tables from the parameters file are used to perform the mapping. The first table contains the minimum vertical rate for each possible action and is stored in the variable *R_dz_min_adv*. The second table contains the maximum vertical rate for each possible action and is stored in the variable *R_dz_max_adv*. The third table contains the vertical acceleration for each possible action and is stored in the variable *R_ddz_adv*.

The proper vertical rate limits and vertical acceleration for a given action are extracted from the parameter file tables using *act*. Vertical rate limits that are defined as *NaN* in the tables correspond to the Maintain RA and require special processing. If the Maintain RA is continuing from the previous cycle, the vertical rate limits and vertical accerlation are set to the values used in the previous cycle: *dz_min_prev*, *dz_max_prev*, and *ddz_prev*. Otherwise, new Maintain RAs use MAINTAINRATES (Algorithm 408) to determine the vertical rate limits while ddz is retrieved from the parameter file tables.

The algorithm takes as inputs the action from the current cycle (*act*), the current vertical rate of the ownship (*dz_own_ave*), the action index from the previous cycle (*act_prev*), the minimum vertical rate of the action from the previous cycle (*dz_min_prev*), the maximum vertical rate of the action from the previous cycle (*dz_max_prev*), and the vertical acceleration of the action from the previous cycle (*ddz_prev*). The algorithm outputs the minimum vertical rate of the input action (*dz_min*), the maximum vertical rate of the input action (*dz_max*), and the vertical acceleration of the input action (*ddz*).

Algorithm 373 ActionToRates

```
function ActionToRates( act::Z, dz_own_ave::R, act_prev::Z, dz_min_prev::R, dz_max_prev::R, ddz_prev::R )
    R_dz_min_adv::Vector{R} = params().actions.min_rates
    R_dz_max_adv::Vector{R} = params().actions.max_rates
    R_ddz_adv::Vector{R}    = params().actions.accelerations
    dz_min::R = 0.0
    dz_max::R = 0.0
    ddz::R    = 0.0
    if isnan( R_dz_min_adv[act] ) || isnan( R_dz_max_adv[act] )
        if (act_prev == act)
            (dz_min, dz_max, ddz) = (dz_min_prev, dz_max_prev, ddz_prev)
        else
            (dz_min, dz_max) = MaintainRates(p. 429)( dz_own_ave )
            ddz = R_ddz_adv[act]
        end
    else
        (dz_min, dz_max, ddz) = (R_dz_min_adv[act], R_dz_max_adv[act], R_ddz_adv[act])
    end
    return (dz_min::R, dz_max::R, ddz::R)
end
```

Referenced In: CoordinationSelection(p. 361), SandwichPreventionCost(p. 348), NoIntrudersAction(p. 342), OnlineUncoordinatedCostEstimation(p. 284), ArbitrateMatchingSenses(p. 358), PersistMTLO(p. 431), ActionSelection(p. 341), IndividualCostEstimation(p. 339), OnlineCostEstimation(p. 310)

CALCULATETHRESHOLDRAMPDOWNFACTOR (Algorithm 374) computes a multiplicative factor by mapping an input value onto a threshold/roll-off set.

The algorithm takes as inputs the current value to test against the threshold/roll-off set (*value*), the value below which the factor will be 1.0 (*threshold*), and the range over which the ramp goes from 1.0 to 0.0 (*rolloff*). The algorithm outputs a value between 0.0 and 1.0 (*factor*).

Algorithm 374 CalculateThresholdRampDownFactor

```
function CalculateThresholdRampDownFactor( value::R, threshold::R, rolloff::R )
    factor::R = 0.0
    if (value <= threshold)
        factor = 1.0
    elseif (value < (threshold + rolloff))
        factor = 1.0 - ((value - threshold) / rolloff)
    end
    return factor::R
end
```

Referenced In: DetermineHorizontalWorstCaseNearMeanFactor(p. 260), DetermineAltitudeDependentCOCFactor(p. 273), DetermineOwnNonComplianceFactor(p. 337), DetermineVerticalProximateFactor(p. 275), DetermineHorizontalWorstCasePhiSpreadFactor(p. 261), DetermineVerticalRateReductionFactor(p. 274), AltitudeDependentCOCCost(p. 289), UpdateTimeBasedNonComplianceCState(p. 335)

CALCULATETHRESHOLDRAMPUPFACTOR (Algorithm 375) computes a multiplicative factor by mapping an input value onto a threshold/roll-off set.

The algorithm takes as inputs the current value to test against the threshold/roll-off set (*value*), the value below which the factor will be 0.0 (*threshold*), and the range over which the ramp goes from 0.0 to 1.0 (*rolloff*). The algorithm outputs a value between 0.0 and 1.0 (*factor*).

Algorithm 375 CalculateThresholdRampUpFactor

```
function CalculateThresholdRampUpFactor( value::R, threshold::R, rolloff::R )
    factor::R = 1.0
    if (value <= threshold)
        factor = 0.0
    elseif (value < (threshold + rolloff))
        factor = (value - threshold) / rolloff
    end
    return factor::R
end
```

Referenced In: DetermineHorizontalWorstCaseNearMeanFactor(p. 260), DetermineAltitudeDependentCOCFactor(p. 273), DetermineOwnNonComplianceFactor(p. 337), UpdateSafeCrossingRADeferralCState(p. 303), DetermineVerticalRateReductionFactor(p. 274), UpdateCoordinatedRADeferralCState(p. 322)

COASTENUBELIEFBASEDONINDIVIDUALSAMPLES (Algorithm 376) takes an input set of belief states and coasts each belief state forward in time by *dt* using a straight line assumption from the current velocity estimate for that belief state. The belief state positions are updated and the belief state velocities remain unchanged.

COASTENUBELIEFBASEDONINDIVIDUALSAMPLES modifies its inputs; the updated set of belief states is returned in *b_enu_set*.

Algorithm 376 CoastEnuBeliefBasedOnIndividualSamples

```
function CoastEnuBeliefBasedOnIndividualSamples( b_enu_set::EnuBeliefSet(p. 517), dt::R )
    b_enu_set.enu_ave = EnuPositionVelocity(p. 517)()
    for b::EnuBelief(p. 516) in b_enu_set.b_enu
        CoastEnuPositionVelocity(p. 411)( b.enu, dt )
        b_enu_set.enu_ave.pos_enu = b_enu_set.enu_ave.pos_enu + (b.enu.pos_enu * b.weight)
        b_enu_set.enu_ave.vel_enu = b_enu_set.enu_ave.vel_enu + (b.enu.vel_enu * b.weight)
    end
end
```

Referenced In: HorizontalAlertLookahead(p. 214)

COASTENUPOSITIONVELOCITY (Algorithm 377) updates the position estimate assuming that the current velocity does not change over the interval *dt*. Only the position is updated; the velocity remains unchanged.

COASTENUPOSITIONVELOCITY modifies its inputs; the updated position is returned in *position_velocity*.

Algorithm 377 CoastEnuPositionVelocity

```
function CoastEnuPositionVelocity( position_velocity::EnuPositionVelocity(p. 517), dt::R )
    position_velocity.pos_enu = position_velocity.pos_enu + (dt * position_velocity.vel_enu)
end
```

Referenced In: CoastEnuBeliefBasedOnIndividualSamples(p. 411)

EXPECTEDTAU (Algorithm 378) calculates the weighted average value for tau. To do this the algorithm loops through each belief in *b_tau_int* and adds the weight to *w_cumulative* and the tau value multiplied by the weight to *tau_cumulative*. If a tau value has no weight, it has no impact on the calculation of tau. The computation of *tau_expected* is done by taking the cumulative weighted time, *tau_cumulative* and dividing it by the cumulative weight, *w_cumulative*. If *w_cumulative* is 0.0 then *tau_expected* is set to the horizon value, which is located in the final bin.

The algorithm takes as inputs the current tau beliefs (*b_tau_int*), and a flag on whether to exclude the horizon tau (the last tau bin) or not (*exclude_tau_horizon*). The algorithm outputs the weighted average value for tau (*tau_expected*).

Algorithm 378 ExpectedTau

```
function ExpectedTau( b_tau_int::Vector{TauBelief(p. 522)}, exclude_tau_horizon::Bool )
    w_cumulative::R   = 0.0
    tau_cumulative::R = 0.0
    t::Z = length( b_tau_int )
    if exclude_tau_horizon
        t = t - 1
    end
    for i = 1:t
        w_cumulative   = w_cumulative + b_tau_int[i].weight
        tau_cumulative = tau_cumulative + (b_tau_int[i].tau * b_tau_int[i].weight)
    end
    tau_expected::R = b_tau_int[end].tau
    if (0.0 < w_cumulative)
        tau_expected = tau_cumulative / w_cumulative
    end
    return tau_expected::R
end
```

Referenced In: StateAndCostEstimation(p. 251), OnlineUncoordinatedCostEstimation(p. 284), OnlineCostEstimation(p. 310)

GETOWNHEIGHT (Algorithm 379) determines the current height of ownship. If the height above-ground-level is defined (i.e., not *NaN*) and is less than the maximum reporting height for AGL sensors, then the height above-ground-level is returned. Otherwise, the tracked pressure/HAE altitude for ownship is used.

The algorithm takes as inputs ownship height above-ground-level (*h_own*), and ownship pressure/HAE altitude (*z_own_ave*). The algorithm outputs current ownship height (*height*).

Algorithm 379 GetOwnHeight

```
function GetOwnHeight( h_own::R, z_own_ave::R )
    H_aglalt_limit::R = params().threat_resolution.H_aglalt_limit
    height::R = z_own_ave
    if !isnan( h_own ) && (h_own <= H_aglalt_limit)
        height = h_own
    end
    return height::R
end
```

Referenced In: GetVTRMOwnData(p. 243)

HORIZONTALACTIONTORATE (Algorithm 380) takes as input the action number (*act*). The algorithm assumes the action number is within the vector bounds. This algorithm returns the turn rate (*turn_rate_rad*) associated with the action number. The returned value of the turn rate is in radians.

Algorithm 380 HorizontalActionToRate

```
function HorizontalActionToRate( act::Z )
    R_turn_rates::Vector{R} = params().turn_actions.turn_rates
    turn_rate_rad::R = deg2rad( R_turn_rates[act] )
    return turn_rate_rad::R
end
```

Referenced In: HorizontalCostFusion(p. 217), HorizontalAlertLookahead(p. 214), ApplyHorizontalOnlineCosts(p. 200)

HORIZONTALRATETOACTION (Algorithm 381) takes as input *turn_rate*. The value of *turn_rate* is expected to be in radians. This algorithm returns the action number (*action*) associated with the input turn rate. If no match is found, a value of *COC* is returned.

Algorithm 381 HorizontalRateToAction

```
function HorizontalRateToAction( turn_rate::R )
    R_turn_epsilon::R       = params().horizontal_trm.R_turn_epsilon
    R_turn_rates::Vector{R} = deg2rad.(params().turn_actions.turn_rates)
    N_rates::Z              = length( R_turn_rates )
    action::Z                     = COC
    for i in 1:N_rates
        if isapprox( R_turn_rates[i], turn_rate, atol=R_turn_epsilon ) ||
           (isnan( R_turn_rates[i] ) && isnan( turn_rate ))
            action = i
            break
        end
    end
    return action::Z
end
```

Referenced In: DetermineLabel271(p. 232)

HORIZONTALRATETOSENSE (Algorithm 382) returns the turn direction, or sense, associated with the input turn rate (*turn_rate*). Only the sign of the turn rate is used to determine the sense.

This algorithm takes as input *turn_rate*. One of the values: *:Right*, *:Left*, or *:None* will be returned.

Algorithm 382 HorizontalRateToSense

```
function HorizontalRateToSense( turn_rate::R )
    R_turn_epsilon::R = params().horizontal_trm.R_turn_epsilon
    local sense::Symbol
    if (0 < turn_rate)
        sense = :Right
    elseif (turn_rate < 0)
        sense = :Left
    elseif (isapprox( turn_rate, 0.0, atol=R_turn_epsilon ))
        sense = :Straight
    else
        sense = :None
    end
    return sense::Symbol
end
```

Referenced In: HorizontalCoordinatedReversalCost(p. 206), HorizontalRateToAdvisory(p. 414), HorizontalInitialMaintainCost(p. 207), HorizontalMaintainHeadingIncentive(p. 207), HorizontalReversalCost(p. 202), IsHorizontalWeakening(p. 423), HorizontalTurnFromMaintainIncentive(p. 207), ApplyHorizontalCoordinationHysteresis(p. 225), HorizontalCoordinationSelection(p. 222), HorizontalMultiThreatMaintainHeadingCost(p. 208), HorizontalAlertLookahead(p. 214), HorizontalMultiThreatReversalCost(p. 205), HorizontalMultiThreatTurnFromMaintainCost(p. 209), UpdateNumHorizontalReversals(p. 220)

HORIZONTALRATETOADVISORY (Algorithm 383) returns the horizontal advisory to be sent to the intruder based on the recommended turn rate (*turn_rate*). Only the sense of the turn rate is used to determine the advisory through a call to HORIZONTALRATETOSENSE.

This algorithm takes as input *turn_rate*. One of the values: *:DontTurnRight*, *:DontTurnLeft*, or *:None* will be returned.

Algorithm 383 HorizontalRateToAdvisory

```
function HorizontalRateToAdvisory( turn_rate::R )
    sense_own::Symbol = HorizontalRateToSense(p. 414)( turn_rate )
    advisory::Symbol = :None
    if (sense_own == :Left)
        advisory = :DontTurnRight
    elseif (sense_own == :Right)
        advisory = :DontTurnLeft
    end
    return advisory::Symbol
end
```

Referenced In: ApplyHorizontalCoordinationHysteresis(p. 225)

HRCTOADVISORY (Algorithm 384) takes as input the Horizontal Resolution Advisory Complement (*hrc*). This algorithm returns the advisory associated with the received Horizontal Resolution Advisory Complement (*hrc*). The returned advisory is one of: *:None*, *:DontTurnLeft*, or *:DontTurnRight*.

Algorithm 384 HRCToAdvisory

```
function HRCToAdvisory( hrc::UInt32 )
    advisory::Symbol = :None
    if (0 == hrc)
        advisory = :None
    elseif (1 == hrc)
        advisory = :DontTurnLeft
    elseif (2 == hrc)
        advisory = :DontTurnRight
    elseif (5 == hrc)
        advisory = :DontTurnLeft
    elseif (6 == hrc)
        advisory = :DontTurnRight
    end
    return advisory::Symbol
end
```

Referenced In: HorizontalCoordinatedReversalCost(p. 206), UpdateCoordinationDelayCState(p. 323), EncodeCHC(p. 227), HorizontalCoordinationCost(p. 204), ApplyHorizontalCoordinationHysteresis(p. 225)

INTERPOLANTS (Algorithm 385) converts input points into a weighted distribution of points that are represented in a table. This process, known as multi-linear interpolation, is done by weighting the indices in the table that are closest to the input point in such a way that their weighted sum is equivalent to the input point. Each dimension of the interpolation can be done independently using traditional linear interpolation and then combined to produce the final weighting.

Initial processing is done to determine the start and end indices of one dimension of the table to be interpolated. These indices are used to extract a vector of the table on which traditional linear interpolation can be done with INTERPOLATEONEDIMENSION (Algorithm 386). The process repeats along each dimension until the final weights across all dimensions are obtained.

The algorithm takes as inputs the multi-dimensional state to be represented with table indices (*point*), a table format parameter that indicates the number of indices along each dimension of the table (*cut_counts*), the table dimension on which interpolation should begin (*start_dim*), the total number of dimensions over which interpolation is done (*num_dimensions*), the indices of the data table (*cuts*), and the boolean flag *use_nearest_neighbor*.

The algorithm outputs a vector of indices of the data table that can be used to represent the input point (*indices*), a vector of weights that corresponds to the indices of the data table that can be used to represent the input point (*weights*), and the number of elements in the portion of the table over which interpolation was done (*block_size*).

Algorithm 385 Interpolants

```
function Interpolants( point::Vector{R}, cut_counts::Vector{Z}, start_dim::Z, num_dimensions::Z, cuts::Vector{
    R}, use_nearest_neighbor::Bool = false )
    indices::Vector{Z} = Z[1]
    weights::Vector{R} = R[1.0]
    block_size::Z      = 1
    idx_cut_start::Z = 1
    for j = 1:(start_dim-1)
        idx_cut_start = idx_cut_start + cut_counts[j]
    end
    for j = start_dim:(start_dim-1+num_dimensions)
        idx_cut_end::Z = idx_cut_start + cut_counts[j] - 1
        cuts_subset::Vector{R} = cuts[ idx_cut_start:idx_cut_end ]
        (indices, weights, block_size) =
            InterpolateOneDimension(p. 418)( indices, weights, point[j], cuts_subset, block_size,
                use_nearest_neighbor )
        idx_cut_start = idx_cut_start + cut_counts[j]
    end
    return (indices::Vector{Z}, weights::Vector{R}, block_size::Z)
end
```

Referenced In: InterpolateBlocks(p. 266), GetHorizontalCoordinationPolicy(p. 223), LookupOfflineCost(p. 281)

INTERPOLATEONEDIMENSION (Algorithm 386) performs linear interpolation across a single dimension and combines it with the results from previous dimensions through multi-linear interpolation. Initial processing is performed to determine the indices of the table that are directly below (*idx_lo*) and above (*idx_hi*) the input value. Values that are less than the first index value or greater than the last index value are capped at the end value by setting both the upper and lower indices to be the first or last index, respectively. If a value falls between the table end points values, it is compared to the value at each index in turn until an index value is not less than the input value. While generally unlikely, if the input point is exactly equal to an index value, both the upper and lower indices will be set to that index. Otherwise, the upper index is set to the current index and the lower index is set to the preceding index.

The upper and lower indices are used to perform the linear interpolation along the single dimension and the results are then combined with the indices from previous dimensions for the multi-linear interpolation. If the upper and lower indices are the same or the input flag *use_nearest_neighbor* (which is false for vertical TRM functions and true for horizontal TRM functions), the total number of indices from the input to the output does not change. However, the location of each input index must be offset by the product of the single dimension index (decremented by 1) and the block size to incorporate the information from the new dimension. The weights of the indices are not affected, which indicates that no interpolation was done. The more likely case, that the upper and lower indices are distinct, is slightly more complex. The total number of indices received as input is doubled for the output to accommodate the new interpolation. The indices of the first half are offset by the product of the lower index (decremented by 1) and the block size, while the indices of the second half are offset by the product of the upper index (decremented by 1) and the block size. The number of weights is similarly increased to match the number of indices. A weighting factor for the new dimension is calculated with standard linear interpolation methods using the input value and the values at the upper and lower indices. The first half of the new, larger weights vector is multiplied by the weighting factor while the second half is multiplied by the 1.0 minus the weighting factor. This operation applies the results of the one dimensional linear interpolation to the set of previous indices completing the multi-linear

interpolation.

Before completion, the new, larger block size is calculated for use in subsequent calls to INTERPOLATEONEDIMENSION.

The algorithm takes as inputs an array of indices from other dimensions used in multi-linear interpolation (*indices*), a vector of weights that corresponds to the indices from other dimensions used in multi-linear interpolation (*w*), the one dimensional value to be interpolated (*point*), the indices of the table along the dimension being interpolated (*cuts*), the offset needed to index into the table along this dimension (*subblock_size*), and *use_nearest_neighbor*, which is set to FALSE.

The algorithm outputs an array of indices, including this dimension, that can be used to represent a data point (*indices_out*), a vector of weights that corresponds to the indices, including this dimension, that can be used to represent data point (*w_out*), and the offset needed to index into the table along the next dimension (*subblock_size_out*).

Algorithm 386 InterpolateOneDimension

```
function InterpolateOneDimension( indices::Vector{Z}, w::Vector{R}, point::R, cuts::Vector{R}, subblock_size::
    Z, use_nearest_neighbor::Bool = false )
    L::Z = length( cuts )
    D::Z = length( w )
    idx_lo::Z = 0
    idx_hi::Z = 0
    if isnan( point ) || (point <= cuts[1])
        idx_lo = 1
        idx_hi = 1
    elseif (cuts[L] <= point)
        idx_lo = L
        idx_hi = L
    else
        idx_lo = 1
        idx_hi = L
        while (idx_hi - idx_lo > 1)
            iMiddle::Z = floor((idx_hi + idx_lo)/2)
            if (point >= cuts[iMiddle])
                idx_lo = iMiddle
            else
                idx_hi = iMiddle
            end
        end
    end
    indices_out::Vector{Z} = Z[]
    w_out::Vector{R}       = R[]
    D_out::Z               = 0
    if (idx_lo == idx_hi) || use_nearest_neighbor
        idx_nearest::Z = idx_lo
        if use_nearest_neighbor
            if (abs( point - cuts[idx_lo] ) < abs( point - cuts[idx_hi] ))
                idx_nearest = idx_lo
            else
                idx_nearest = idx_hi
            end
        end
        indices_out = Vector{Z}( undef, D )
        for i = 1:D
            indices_out[i] = indices[i] + ((idx_nearest - 1) * subblock_size)
        end
        if !isnan( point )
            w_out = copy( w )
        else
            w_out = fill( NaN, length( w ) )
        end
        D_out = D
    else
        D_out = D + D
        indices_out = Vector{Z}( undef, D_out )
        for i = 1:D
            indices_out[i  ] = indices[i] + ((idx_lo - 1) * subblock_size)
            indices_out[i+D] = indices[i] + ((idx_hi - 1) * subblock_size)
        end
        w_out = Vector{R}( undef, D_out )
        w_lo::R = 1.0 - ((point - cuts[idx_lo]) / (cuts[idx_hi] - cuts[idx_lo]))
        for i = 1:D
            w_out[i  ] = w[i] * w_lo
            w_out[i+D] = w[i] * (1.0 - w_lo)
        end
    end
    subblock_size_out::Z = subblock_size * L
    return (indices_out::Vector{Z}, w_out::Vector{R}, subblock_size_out::Z)
end
```

Referenced In: DiscretizePolicyState(p. 192), Interpolants(p. 416), GetPolicyTableCosts(p. 190)

IsCOC (Algorithm 387) determines if the input action corresponds to the clear of conflict advisory.

The algorithm takes as inputs the maximum vertical rate of the action (*dz_max*), and the minimum vertical rate of the action (*dz_min*). The algorithm outputs a boolean flag that is TRUE if the input action is clear of conflict and FALSE otherwise (*is_coc*).

Algorithm 387 IsCOC

```
function IsCOC( dz_min::R, dz_max::R )
    is_coc::Bool = false
    if (dz_min == -Inf) && (dz_max == Inf)
        is_coc = true
    end
    return is_coc::Bool
end
```

Referenced In: DetermineDisplayData(p. 368), UpdateAdvisoryRestartCState(p. 288), IsPreventive(p. 426), UpdateMaxReversalCState(p. 327), AdvisoryRestartCost(p. 288), RestrictCOCDueToReversal(p. 329), UpdateRestrictCOCDueToReversalCState(p. 329), CrossingNoAlertCost(p. 324), CriticalIntervalProtectionCost(p. 294), GetModifiedGlobalRates(p. 285), InitializationCost(p. 299), UpdatePreventEarlyCOCCState(p. 301), SafeCrossingRADeferralCost(p. 302), MaxReversalCost(p. 326), UpdateBadTransitionCState(p. 292), PreventEarlyCOCCost(p. 299), AltitudeInhibitCost(p. 290), UpdateCriticalIntervalProtectionCState(p. 296), PointObsPreventEarlyCOCCost(p. 306), SA01Heuristic(p. 330), AltitudeDependentCOCCost(p. 289), CompatibilityCost(p. 319), TimeBasedNonComplianceCost(p. 333), UpdateTimeBasedNonComplianceCState(p. 335), UpdateIndivAdjustCounts(p. 381), CoordinatedRADeferralCost(p. 321), CoordinationDelayCost(p. 323)

IsCorrective (Algorithm 388) examines the vertical rate limits of an input action (*dz_min*, *dz_max*) to determine whether that action is corrective. This algorithm considers only the vertical rate limits of the input action when determining if an action is corrective. Consequently, Level-off RAs are not considered correctives by this algorithm because they have the same vertical rate limits as Monitor Vertical Speed RAs. Additionally, Maintain RAs are not considered to be correctives by this algorithm.

The algorithm takes as inputs the minimum vertical rate of the action (*dz_min*) and the maximum vertical rate of the action (*dz_max*). The algorithm outputs a boolean flag that is TRUE if the input action is corrective and FALSE otherwise (*is_corrective*).

Algorithm 388 IsCorrective

```
function IsCorrective( dz_min::R, dz_max::R )
    is_corrective::Bool = false
    if (((-Inf == dz_min) && (dz_max < 0.0)) || ((0.0 < dz_min) && (Inf == dz_max))) &&
       !IsMaintain(p. 424)( dz_min, dz_max )
        is_corrective = true
    end
    return is_corrective::Bool
end
```

Referenced In: PreventEarlyWeakeningCost(p. 328)

IsCrossing (Algorithm 389) determines if the sense of an intruder action will result in an altitude crossing between ownship and intruder. For example, if the intruder is below the ownship and

the intruder action sense is *:Up*, the algorithm will return TRUE. If the aircraft are closer than the altitude buffer, (*CROSSING_ALTITUDE_BUFFER*), the *is_crossing* flag is always set to FALSE.

The algorithm takes as inputs current pressure/HAE altitude of ownship (*z_ac1_ave*), current pressure/HAE altitude of the intruder (*z_ac2_ave*), and current sense of the intruder RA (*sense_-ac2*). The algorithm outputs a boolean flag that is TRUE if following the intruder ra sense indicates a desire to cross altitude (*is_crossing*).

Algorithm 389 IsCrossing

```
function IsCrossing( z_ac1_ave::R, z_ac2_ave::R, sense_ac2::Symbol )
    global CROSSING_ALTITUDE_BUFFER
    is_crossing::Bool = false
    if (CROSSING_ALTITUDE_BUFFER <= abs( z_ac2_ave - z_ac1_ave ))
        if ((z_ac2_ave < z_ac1_ave) && (sense_ac2 == :Up)) ||
           ((z_ac1_ave < z_ac2_ave) && (sense_ac2 == :Down))
            is_crossing = true
        end
    end
    return is_crossing::Bool
end
```

Referenced In: CompatibilityCost(p. 319), UpdateCrossingNoAlertCState(p. 325)

IsDiverging (Algorithm 390) determines whether the sense of an intruder action is directed away from the ownship altitude. For example, if the intruder is below the ownship and the intruder action sense is *:Down*, the algorithm will return TRUE.

The algorithm takes as inputs current pressure/HAE altitude of ownship (*z_own_ave*), current pressure/HAE altitude of the intruder (*z_int_ave*), and current sense of the intruders RA (*sense_-int*). The algorithm outputs a boolean flag that is TRUE if the intruder RA sense is directed away from the ownship altitude (*is_diverging*).

Algorithm 390 IsDiverging

```
function IsDiverging( z_own_ave::R, z_int_ave::R, sense_int::Symbol )
    is_diverging::Bool = false
    if ((z_int_ave < z_own_ave) && (sense_int == :Down)) ||
       ((z_own_ave < z_int_ave) && (sense_int == :Up))
        is_diverging = true
    end
    return is_diverging::Bool
end
```

Referenced In: UpdateCoordinatedRADeferralCState(p. 322)

IsDNC (Algorithm 391) uses the vertical rate limits of a given action to determine whether the action is Do Not Climb. This is done by comparing the input vertical rate limits to the pre-defined vertical limits of Do Not Climb.

The algorithm takes as inputs the maximum vertical rate of the action (*dz_max*) and the minimum vertical rate of the action (*dz_min*). The algorithm outputs a boolean flag that is TRUE if the input action is Do Not Climb (*is_dnc*).

Algorithm 391 IsDNC

```
function IsDNC( dz_min::R, dz_max::R )
    is_dnc::Bool = false
    if (dz_min == -Inf) && (dz_max == 0)
        is_dnc = true
    end
    return is_dnc::Bool
end
```

Referenced In: PersistMTLO(p. 431), PreventEarlyWeakeningCost(p. 328)

IsDND (Algorithm 392) uses the vertical rate limits of a given action to determine whether the action is Do Not Descend. This is done by comparing the input vertical rate limits to the predefined vertical limits of Do Not Descend.

The algorithm takes as inputs the minimum vertical rate of the action (*dz_min*) and the maximum vertical rate of the action (*dz_max*). The algorithm outputs a boolean flag that is TRUE if the input action is Do Not Descend (*is_dnd*).

Algorithm 392 IsDND

```
function IsDND( dz_min::R, dz_max::R )
    is_dnd::Bool = false
    if (dz_min == 0) && (dz_max == Inf)
        is_dnd = true
    end
    return is_dnd::Bool
end
```

Referenced In: PersistMTLO(p. 431), PreventEarlyWeakeningCost(p. 328)

IsHorizontalCOC (Algorithm 393) returns TRUE if the turn rate is *NaN* representing clear of conflict.

This algorithm takes as input *turn_rate*. This algorithm returns a value of TRUE or FALSE for *is_coc*.

Algorithm 393 IsHorizontalCOC

```
function IsHorizontalCOC( turn_rate::R )
    is_coc::Bool = false
    if isnan(turn_rate)
        is_coc = true
    end
    return is_coc::Bool
end
```

Referenced In: HorizontalCoordinatedReversalCost(p. 206), DetermineDisplayTrackAngleAndTurnRate(p. 232), DetermineHorizontalCode(p. 229), HorizontalInitialMaintainCost(p. 207), HorizontalMaintainHeadingIncentive(p. 207), SelectHorizontalAdvisoryWindRelative(p. 211), HorizontalInitialCACost(p. 202), HorizontalCostFusion(p. 217), HorizontalCoordinationCost(p. 204), HorizontalTurnFromMaintainIncentive(p. 207), HorizontalCoordinationSelection(p. 222), HorizontalMultiThreatMaintainHeadingCost(p. 208), HorizontalAlertLookahead(p. 214), HorizontalDesensitivityCost(p. 203), HorizontalMultiThreatReversalCost(p. 205), HorizontalMultiThreatTurnFromMaintainCost(p. 209), UpdateNumHorizontalReversals(p. 220), HorizontalTurnRateSensitivityCostFactor(p. 203), SelectHorizontalAdvisory(p. 210)

IsHorizontalReversal (Algorithm 394) compares the sense of two horizontal advisories and returns TRUE when they are the opposite sense. For online costs, transitions to and from horizontal maintain RAs are not considered reversals.

This algorithm takes as input *sense_prev* and *sense_curr*. This algorithm returns a value of TRUE or FALSE for *is_reversal*.

Algorithm 394 IsHorizontalReversal

```
function IsHorizontalReversal( sense_prev::Symbol, sense_curr::Symbol )
    is_reversal::Bool = false
    if ((:Left == sense_prev) && (:Right == sense_curr)) ||
       ((:Right == sense_prev) && (:Left == sense_curr))
        is_reversal = true
    end
    return is_reversal::Bool
end
```

Referenced In: HorizontalCoordinatedReversalCost(p. 206), HorizontalReversalCost(p. 202), ApplyHorizontalCoordinationHysteresis(p. 225), HorizontalAlertLookahead(p. 214), HorizontalMultiThreatReversalCost(p. 205), UpdateNumHorizontalReversals(p. 220)

IsHorizontalWeakening (Algorithm 395) evaluates the transition from an advisory with *turn_rate_prev* to an advisory with *turn_rate* and returns TRUE when that transition represents a weaker RA in the same sense.

This algorithm takes as input *turn_rate* and *turn_rate_prev*. This algorithm returns a value of TRUE or FALSE for *is_weakening*.

Algorithm 395 IsHorizontalWeakening

```
function IsHorizontalWeakening( turn_rate::R, turn_rate_prev::R )
    R_turn_epsilon::R = params().horizontal_trm.R_turn_epsilon
    is_weakening::Bool = false
    if !isnan( turn_rate_prev ) && !isapprox( turn_rate, turn_rate_prev, atol=R_turn_epsilon )
        if isnan( turn_rate )
            is_weakening = true
        else
            sense_own_prev::Symbol = HorizontalRateToSense(p. 414)( turn_rate_prev )
            sense_own_curr::Symbol = HorizontalRateToSense(p. 414)( turn_rate )
            if (sense_own_prev == sense_own_curr) && (:Straight != sense_own_curr)
                is_weakening = (abs( turn_rate ) < abs( turn_rate_prev ))
            end
        end
    end
    return is_weakening::Bool
end
```

Referenced In: HorizontalWeakeningCostFactor(p. 202)

IsIntruderEquipped (Algorithm 396) determines whether or not the intruder is equipped with a collision avoidance capability and can coordinate with ACAS sXu.

The algorithm takes as inputs the intruder operational mode and (*opmode*) and equipage (*equipage*). The algorithm outputs a boolean flag that is TRUE if the intruder is equipped (*is_equipped*).

Algorithm 396 IsIntruderEquipped

```
function IsIntruderEquipped( opmode::UInt8, equipage::Z )
    is_equipped::Bool = false
    if (EQUIPAGE_NONE != equipage) && (EQUIPAGE_TCAS != equipage) && (EQUIPAGE_LARGE_CAS != equipage) && (
         EQUIPAGE_CASTA != equipage) && (OPMODE_RA == opmode)
        is_equipped = true
    end
    return is_equipped::Bool
end
```

Referenced In: UpdateIntruderInputs(p. 307), UpdateHTRMIntruderInputs(p. 173), HorizontalStateEstimation(p. 175), IntruderPrep(p. 254)

IsIntruderMaster (Algorithm 397) determines whether or not the intruder is the master aircraft in an equipped-equipped encounter. In order for intruder to be the master aircraft, the intruder must be equipped and both aircraft must be in RA mode. The ownship's coordination policy against the intruder must be a junior or none (therefore the intruder is senior in responsive coordination), or the ownship's coordination policy is peer and the intruder V2V unique ID is valid and lower than the V2V unique ID of the ownship.

The algorithm takes as inputs input data for ownship (*input_own*) and input data for the intruder (*input_int*). The algorithm outputs a boolean flag that is TRUE if the intruder is master (*is_master_int*).

Algorithm 397 IsIntruderMaster

```
function IsIntruderMaster( input_own::TRMOwnInput(p. 498), input_int::TRMIntruderInput(p. 499) )
    equipage::Z = input_int.equipage
    is_master_int::Bool = false
    is_own_policy_peer::Bool = input_int.own_coordination_policy == OWN_COORDINATION_POLICY_PEER
    is_own_policy_junior_or_none::Bool =
           (input_int.own_coordination_policy == OWN_COORDINATION_POLICY_JUNIOR) ||
           (input_int.own_coordination_policy == OWN_COORDINATION_POLICY_NONE)
    if (EQUIPAGE_NONE != equipage) && (EQUIPAGE_TCAS != equipage) && (EQUIPAGE_LARGE_CAS != equipage) &&
       (EQUIPAGE_CASTA != equipage) && (OPMODE_RA == input_own.opmode) &&
       ((is_own_policy_peer && input_int.id_directory.v2v_uid.valid &&
        (input_int.id_directory.v2v_uid.value < input_own.v2v_uid)) || is_own_policy_junior_or_none)
        is_master_int = true
    end
    return is_master_int::Bool
end
```

Referenced In: UpdateIntruderInputs(p. 307), UpdateHTRMIntruderInputs(p. 173), HorizontalStateEstimation(p. 175), IntruderPrep(p. 254)

IsMaintain (Algorithm 398) determines whether an input action is a Maintain advisory. This is done by comparing the vertical rates of the input action against all non-maintain vertical rates in the parameter file. If there is no match, the action is a Maintain RA.

The algorithm takes as inputs the minimum vertical rate of the action (*dz_min*) and the maximum vertical rate of the action (*dz_max*). The algorithm outputs a boolean flag that is TRUE if the input action is a Maintain RA (*is_maintain*).

Algorithm 398 IsMaintain

```
function IsMaintain( dz_min::R, dz_max::R )
    R_dz_min_adv::Vector{R} = params().actions.min_rates
    R_dz_max_adv::Vector{R} = params().actions.max_rates
    is_maintain::Bool = true
    for act = 1:length( R_dz_min_adv )
        if (R_dz_min_adv[act] == dz_min) && (R_dz_max_adv[act] == dz_max)
            is_maintain = false
        end
    end
    return is_maintain::Bool
end
```

Referenced In: CoordinationSelection(p. 361), IsCorrective(p. 419), SandwichPreventionCost(p. 348), UpdateBadTransitionCState(p. 292), BadTransitionCost(p. 291), MaintainRates(p. 429), AltitudeDependentCOCCost(p. 289)

IsMasterForcingReversal (Algorithm 399) determines if the intruder is forcing ownship to issue a reversal. A prerequisite for the intruder to force a reversal is that it must be master. The determination of whether the reversal is forced is done by examining whether the sense of the coordination message received from intruder is the same as the sense selected by ownship on the previous cycle.

The algorithm takes as inputs a flag for if the current intruder is the master (*master_int*), the sense of the action selected by ownship on the previous cycle (*sense_own_prev*), and the sense of the current intruder RA (*sense_int*). The algorithm outputs a boolean flag that is TRUE if the

intruder is forcing ownship to reverse (*is_forced_reversal*).

Algorithm 399 IsMasterForcingReversal

```
function IsMasterForcingReversal( master_int::Bool, sense_own_prev::Symbol, sense_int::Symbol )
    is_forced_reversal::Bool = false
    if master_int && (sense_own_prev == sense_int)
        is_forced_reversal = true
    end
    return( is_forced_reversal::Bool )
end
```

Referenced In: UpdateMaxReversalCState(p. 327), RestrictCOCDueToReversal(p. 329), MaxReversalCost(p. 326)

IsMTLO (Algorithm 400) determines whether an action is a Multi-Threat Level-Off (MTLO). This is done by comparing the vertical rates of the input action against the standard MTLO rates in the parameter file. If the vertical rates match, the action is MTLO.

The algorithm takes as inputs the minimum vertical rate of the action (*dz_min*) and maximum vertical rate of the action (*dz_max*). The algorithm outputs a boolean flag that is TRUE if the input action is an MTLO RA (*is_mtlo*).

Algorithm 400 IsMTLO

```
function IsMTLO( dz_min::R, dz_max::R )
    act_mtlo::Z = MTLOAction(p. 430)()
    R_dz_min_LO::R = params().actions.min_rates[act_mtlo]
    R_dz_max_LO::R = params().actions.max_rates[act_mtlo]
    is_mtlo::Bool = false
    if (dz_min == R_dz_min_LO) && (dz_max == R_dz_max_LO)
        is_mtlo = true
    end
    return is_mtlo::Bool
end
```

Referenced In: GetModifiedGlobalRates(p. 285), DetermineCrossing(p. 367)

IsPreventive (Algorithm 401) determines if the current action is a preventive RA. This is done by inspecting the vertical rate limits of an input action. If the action is not clear of conflict and either *dz_min* is equal to negative infinity and *dz_max* is greater than or equal to zero or *dz_max* is equal to positive infinity and *dz_min* is less than or equal to zero, the action is considered preventive. It should be noted that this algorithm does not return *is_preventive* as TRUE for Maintain RAs.

The algorithm takes as inputs the minimum vertical rate of the action (*dz_min*) and the maximum vertical rate of the action (*dz_max*). The algorithm outputs a boolean flag that is TRUE if the input action is a preventive RA (*is_preventive*).

Algorithm 401 IsPreventive

```
function IsPreventive( dz_min::R, dz_max::R )
    is_preventive::Bool = false
    if !IsCOC(p. 419)( dz_min, dz_max ) &&
       (((-Inf == dz_min) && (0.0 <= dz_max)) || ((dz_min <= 0.0) && (Inf == dz_max)))
        is_preventive = true
    end
    return is_preventive::Bool
end
```

Referenced In: SandwichPreventionCost(p. 348), BadTransitionCost(p. 291), AltitudeDependentCOCCost(p. 289), IndividualCostEstimation(p. 339), CoordinatedRADeferralCost(p. 321)

IsProjectedCrossing (Algorithm 402) uses the current and projected relative altitude between ownship and intruder to determine if an altitude crossing will occur. This is done by comparing the sign of the two relative altitude inputs, with explicit checks for zero relative altitude.

The algorithm takes as inputs the current altitude of intruder relative to ownship (*z_rel*) and the projected altitude of intruder relative to ownship (*z_rel_projected*). The algorithm outputs a boolean flag that is TRUE if ownship is projected to cross altitudes with the intruder (*is_projected_crossing*).

Algorithm 402 IsProjectedCrossing

```
function IsProjectedCrossing( z_rel::R, z_rel_projected::R )
    is_projected_crossing::Bool = false
    if (sign( z_rel ) != sign( z_rel_projected )) ||
       (0 == sign( z_rel )) || (0 == sign( z_rel_projected ))
        is_projected_crossing = true
    end
    return is_projected_crossing::Bool
end
```

Referenced In: UpdateSafeCrossingRADeferralCState(p. 303), DetermineVerticalProximateFactor(p. 275), UpdateCriticalIntervalProtectionCState(p. 296), UpdateCoordinatedRADeferralCState(p. 322), UpdateTimeBasedNonComplianceCState(p. 335)

IsReversal (Algorithm 403) determines if the current RA sense is a reversal of the RA sense from the previous cycle.

The algorithm takes as inputs the RA sense from the previous cycle (*sense_prev*) and the sense of the current RA (*sense_curr*). The algorithm outputs a boolean flag that is TRUE when the current RA is a reversal (*is_reversal*).

Algorithm 403 IsReversal

```
function IsReversal( sense_prev::Symbol, sense_curr::Symbol )
    is_reversal::Bool = false
    if ((:Up == sense_prev) && (:Down == sense_curr)) || ((:Down == sense_prev) && (:Up == sense_curr))
        is_reversal = true
    end
    return( is_reversal::Bool )
end
```

Referenced In: UpdateMaxReversalCState(p. 327), MaxReversalCost(p. 326), BadTransitionCost(p. 291)

IsSlowClosure (Algorithm 404) determines if the ownship and intruder are in a slow, horizontal convergence geometry where vertical separation still exists. This is done by comparing the relative ground range and horizontal closure rate to protection-mode-dependent thresholds obtained from the parameters file. While not directly tied to horizontal geometry, the altitude of the intruder relative to ownship is also compared to a threshold to see if sufficient vertical separation still exists.

The algorithm takes as inputs the intruder protection mode (*mode_int*), the ground range between intruder and ownship (*r_ground*), the magnitude of the relative velocity between intruder and ownship (*s_ground*), and the altitude of intruder relative to ownship (*z_rel*). The algorithm outputs a boolean flag that is TRUE if the current intruder is in a horizontal slow closure with ownship (*is_slow_closure*).

Algorithm 404 IsSlowClosure

```
function IsSlowClosure( mode_int::Z, r_ground::R, s_ground::R, z_rel::R )
    D_range_thres::R   = params().modes[mode_int].state_estimation.tau.D_range_thres
    R_speed_thres::R   = params().modes[mode_int].state_estimation.tau.R_speed_thres
    H_alt_rel_thres::R = params().modes[mode_int].state_estimation.tau.H_alt_rel_thres
    is_slow_closure::Bool = false
    if (r_ground < D_range_thres) && (s_ground < R_speed_thres) &&
       (H_alt_rel_thres < abs( z_rel ))
        is_slow_closure = true
    end
    return is_slow_closure::Bool
end
```

Referenced In: TauEstimation(p. 263)

IsStrengthening (Algorithm 405) determines if the current action is a strengthening of the previous action. This is done by testing whether the sense of the current action is the same sense as the previous action and the target rate of the current action has a greater magnitude than the previous action. The sense of the current and previous action are determined with a call to RatesToSense (Algorithm 413).

The algorithm takes as inputs the minimum vertical rate of the previous action (*dz_min_prev*), maximum vertical rate of the previous action (*dz_max_prev*), the minimum vertical rate of the current action (*dz_min*), and the maximum vertical rate of the current action (*dz_max*). The algorithm outputs a boolean flag that is TRUE if the current action is a strengthening (*is_strengthening*).

Algorithm 405 IsStrengthening

```
function IsStrengthening( dz_min_prev::R, dz_max_prev::R, dz_min::R, dz_max::R )
    is_strengthening::Bool = false
    sense_own_prev::Symbol = RatesToSense(p. 432)( dz_min_prev, dz_max_prev )
    sense_own_curr::Symbol = RatesToSense(p. 432)( dz_min, dz_max )
    if (sense_own_prev == sense_own_curr)
        if (sense_own_curr == :Down)
            is_strengthening = (dz_max < dz_max_prev)
        elseif (sense_own_curr == :Up)
            is_strengthening = (dz_min > dz_min_prev)
        end
    end
    return is_strengthening::Bool
end
```

Referenced In: VerticalRateLimitDependentStrengthenCost(p. 304), BadTransitionCost(p. 291)

IsVerticallyConverging (Algorithm 406) determines if ownship and an intruder are converging vertically. This is done by determining if the altitude and vertical rate of the intruder relative to ownship indicate the aircraft are closing in altitude. Explicit checks are performed to ensure that the magnitude of the vertical rate of the intruder relative to ownship is above a threshold. This prevents divide by zero failures and ensures sufficient relative vertical rate exists to determine convergence.

The algorithm takes as inputs the protection mode of the intruder (*mode_int*), current pressure/HAE altitude of ownship (*z_own_ave*), current vertical rate of the ownship (*dz_own_ave*), average of the belief states of intruder pressure/HAE altitude (*z_int_ave*), and average of the belief states of intruder vertical rate (*dz_int_ave*). The algorithm outputs a boolean flag that is TRUE if ownship and intruder are converging vertically (*is_vertically_converging*).

Algorithm 406 IsVerticallyConverging

```
function IsVerticallyConverging( mode_int::Z, z_own_ave::R, dz_own_ave::R, z_int_ave::R, dz_int_ave::R )
    R_converge_thres_vert::R =
        params().modes[mode_int].state_estimation.tau.R_converge_thres_vert
    is_vertically_converging::Bool = false
    if (R_converge_thres_vert < abs( dz_own_ave - dz_int_ave )) &&
       (0.0 < ((z_int_ave - z_own_ave) / (dz_own_ave - dz_int_ave)))
       is_vertically_converging = true
    end
    return is_vertically_converging::Bool
end
```

Referenced In: TauEstimation(p. 263)

IsWithinRateBounds (Algorithm 407) returns whether or not the current ownship vertical rate (*dz_own_ave*) is within the minimum and maximum vertical rate bounds of the indicated action (*act*). The ownship vertical rate is within the rate bounds whenever the action is Maintain, as indicated by NaN for both rate bounds.

This algorithm takes as input *act* and *dz_own_ave*. The algorithm returns *is_within_bounds*.

Algorithm 407 IsWithinRateBounds

```
function IsWithinRateBounds( act::Z, dz_own_ave::R )
    R_dz_min::Vector{R} = params().actions.min_rates
    R_dz_max::Vector{R} = params().actions.max_rates
    is_within_bounds::Bool = false
    if isnan( R_dz_min[act] ) || isnan( R_dz_max[act] )
        is_within_bounds = true
    elseif (R_dz_min[act] <= dz_own_ave) && (dz_own_ave <= R_dz_max[act])
        is_within_bounds = true
    end
    return is_within_bounds::Bool
end
```

Referenced In: IndividualCostEstimation(p. 339)

MAINTAINRATES (Algorithm 408) uses the current vertical rate of the ownship *dz_own_ave* to determine the minimum and maximum vertical rate limits for a Maintain RA. If *dz_own_ave* is zero or greater, *dz_min* is set to *dz_own_ave* and *dz_max* is set to infinity. Otherwise, if *dz_own_ave* is less than zero, *dz_max* is set to *dz_own_ave* and *dz_min* is set to negative infinity. ISMAINTAIN is called with the calculated rates to protect against the possibility that the ownship vertical rate is exactly equal to a standard RA target rate. If this occurs, *epsilon*, a small value, is either added to *dz_max* or subtracted from *dz_min*, as appropriate, to prevent the Maintain RA from being indistinguishable with a standard RA. Additionally, if the ownship vertical rate is exactly zero, *dz_max* is set to positive infinity and *dz_min* is set to *epsilon*.

The algorithm takes as input the current ownship vertical rate (*dz_own_ave*). The algorithm outputs the minimum vertical rate of the Maintain RA (*dz_min*) and the maximum vertical rate of the Maintain RA (*dz_max*).

Algorithm 408 MaintainRates

```
function MaintainRates( dz_own_ave::R )
    dz_min::R  = NaN
    dz_max::R  = NaN
    epsilon::R = 2e-4
    if (0 <= dz_own_ave)
        (dz_min, dz_max) = (dz_own_ave, Inf)
        if !IsMaintain(p. 424)( dz_min, dz_max )
            if (0 < dz_min)
                dz_min = dz_min - epsilon
            else
                dz_min = epsilon
            end
        end
    else
        (dz_min, dz_max) = (-Inf, dz_own_ave)
        if !IsMaintain(p. 424)( dz_min, dz_max )
            dz_max = dz_max + epsilon
        end
    end
    return (dz_min::R, dz_max::R)
end
```

Referenced In: DetermineSenses(p. 351), ActionToRates(p. 410)

MINCOSTINDEX (Algorithm 409) returns the index of the lowest cost action in *costs*. The lowest cost index is initialized to the first action in the *costs* array before beginning a comparison through the remaining elements. If an element is found to have a cost less than the current lowest cost by more than *epsilon*, a small value, the lowest cost index is updated to the current element. Action costs that are within *epsilon* are considered equivalent, with the lower indexed action being preferred.

The algorithm takes as inputs a vector containing the weighted cost of each potential action (*costs*) and the minimum difference between two actions required to consider them distinct (*epsilon*). The algorithm outputs the index of the action that has the lowest cost (*cost_min_idx*).

Algorithm 409 MinCostIndex

```
function MinCostIndex( costs::Vector{R}, epsilon::R )
    cost_min_idx::Z = 1
    cost_min::R     = costs[1]
    for i = 2:length( costs )
        if ((costs[i] + epsilon) < cost_min)
            cost_min_idx = i
            cost_min = costs[i]
        end
    end
    return (cost_min_idx::Z)
end
```

Referenced In: DetermineMinimumCostAction(p. 343), IndividualSelectionCostEstimation(p. 346), UnequippedCostFusion(p. 349), IndividualCostEstimation(p. 339)

MTLOACTION (Algorithm 410) returns the index for MTLO in the action vector. The algorithm takes no inputs. The algorithm outputs the index of MTLO in the action vector (*action_mtlo*).

Algorithm 410 MTLOAction

```
function MTLOAction()
    N_actions::Z = params().actions.num_actions
    action_mtlo::Z = N_actions
    return action_mtlo::Z
end
```

Referenced In: IsMTLO(p. 425), DetermineMinimumCostAction(p. 343), IndividualSelectionCostEstimation(p. 346), UnequippedCostFusion(p. 349), DetermineMultiIntruderAction(p. 345), PersistMTLO(p. 431), IndividualCostEstimation(p. 339)

PERSISTMTLO (Algorithm 411) determines if a multi-threat level-off (MTLO) action issued last cycle should continue this cycle. An MTLO should persist if the advisory in the previous cycle was an MTLO, if multiple threats still exist, and if the current action would be considered a corrective alert in an encounter with a single intruder. This last condition is evaluated by checking the compatibility between the ownship's vertical rate in the previous cycle and the sense of the current advisory, which in a multithreat encounter can only be DND or DNC; an advisory that restricts the vertical rate of ownship to be in the opposite sense of that of the previous cycle would be considered a corrective alert in an encounter with a single intruder. If one of these two alerts

was selected by the logic through single-threat processing and that action would be a corrective based on *dz_own_ave_prev*, the *persist_mtlo* flag is set to TRUE.

The algorithm takes as inputs the action selected for the current cycle (*action*), a boolean flag that is TRUE if there are multiple threats (*multithreat*), current vertical rate of the ownship (*dz_own_ave*), vertical rate of the ownship from the previous cycle (*dz_own_ave_prev*), and the action selected during the previous cycle (*a_global_prev*). The algorithm outputs a boolean flag that is TRUE if the MTLO should be persisted (*persist_mtlo*).

Algorithm 411 PersistMTLO

```
function PersistMTLO( action::Z, multithreat::Bool,
            dz_own_ave::R, dz_own_ave_prev::R,
            a_global_prev::GlobalAdvisory(p. 517) )
    persist_mtlo::Bool = false
    if multithreat && (a_global_prev.action == MTLOAction(p. 430)())
        (dz_min::R, dz_max::R) =
            ActionToRates(p. 410)( action, dz_own_ave, a_global_prev.action,
                        a_global_prev.dz_min, a_global_prev.dz_max, a_global_prev.ddz )
        if IsDNC(p. 421)( dz_min, dz_max ) && (0 <= dz_own_ave_prev)
            persist_mtlo = true
        elseif IsDND(p. 421)( dz_min, dz_max ) && (dz_own_ave_prev < 0)
            persist_mtlo = true
        end
    end
    return persist_mtlo::Bool
end
```

Referenced In: MTLODetermination(p. 350), DetermineMultiIntruderAction(p. 345)

RATESTOACTION (Algorithm 412) maps the vertical rate limits and vertical acceleration of an action to the corresponding action number. Three tables from the parameters file are used to perform the mapping. The first table contains the minimum vertical rate for each possible action and is stored in the variable *R_dz_min_adv*. The second table contains the maximum vertical rate for each possible action and is stored in the variable *R_dz_max_adv*. The third table contains the vertical acceleration for each possible action and is stored in the variable *R_ddz_adv*.

The input vertical rates and vertical acceleration are compared to those corresponding to the standard RAs. If a match is found, the *action* variable is set to the action number for that RA. Since Maintain RAs are represented in the vertical rate parameter file tables with *NaN*, comparison for Maintain will always return FALSE. Explicit checks for *NaN* are performed to determine the action number of Maintain during the standard RA comparison step. If no action was matched using the standard values, additional processing is done to check for Maintain RAs. This consists of checking for the potential representations of Maintain: *dz_min* as negative infinity with a negative *dz_max*, *dz_max* as positive infinity with a positive *dz_min*, or either *dz_min* or *dz_max* being defined as *NaN*.

The algorithm takes as inputs the minimum vertical rate of the action (*dz_min*), the maximum vertical rate of the action (*dz_max*), and the vertical acceleration of the action (*ddz*). The algorithm outputs the action that corresponds to the input vertical rate limits and vertical acceleration (*action*).

Algorithm 412 RatesToAction

```
function RatesToAction( dz_min::R, dz_max::R, ddz::R )
    R_dz_min_adv::Vector{R} = params().actions.min_rates
    R_dz_max_adv::Vector{R} = params().actions.max_rates
    R_ddz_adv::Vector{R}    = params().actions.accelerations
    action::Z   = 0
    maintain::Z = 0
    for i = 1:length( R_dz_min_adv )
        if (dz_min == R_dz_min_adv[i]) && (dz_max == R_dz_max_adv[i]) && (ddz == R_ddz_adv[i])
            action = i
            break
        elseif isnan( R_dz_min_adv[i] ) || isnan( R_dz_max_adv[i] )
            maintain = i
        end
    end
    if (action == 0) &&
       (((dz_min == -Inf) && (dz_max < 0))    ||
        ((0 <= dz_min)    && (dz_max == Inf)) ||
        isnan( dz_min )                       ||
        isnan( dz_max ))
        action = maintain
    end
    return action::Z
end
```

Referenced In: DetermineMultiIntruderAction(p. 345)

RATESTOSENSE (Algorithm 413) determines the sense of an RA from the vertical rate limits of that RA. This is done by comparing the magnitudes of *dz_min* and *dz_max*. If the magnitude of *dz_min* is greater than the magnitude of *dz_max* the sense is *:Down*. If the magnitude of *dz_max* is greater than the magnitude of *dz_min* the sense is *:Up*. The only remaining option, that the two vertical rate limits are of equal magnitude, does not have a defined sense and set as *:None*.

The algorithm takes as inputs the minimum vertical rate of the action (*dz_min*) and the maximum vertical rate of the action (*dz_max*). The algorithm outputs (*sense*), which indicates if the inputs correspond to an up sense, down sense, or no sense RA.

Algorithm 413 RatesToSense

```
function RatesToSense( dz_min::R, dz_max::R )
    local sense::Symbol
    if (abs( dz_min ) > abs( dz_max ))
        sense = :Down
    elseif (abs( dz_min ) < abs( dz_max ))
        sense = :Up
    else
        sense = :None
    end
    return sense::Symbol
end
```

Referenced In: DetermineDisplayData(p. 368), OwnResponseEstimation(p. 315), UpdateMaxReversalCState(p. 327), SandwichPreventionCost(p. 348), UpdateSA01HeuristicCState(p. 331), UpdateRestrictCOCDueToReversalCState(p. 329), DetermineSenses(p. 351), GetModifiedGlobalRates(p. 285), Crosslink(p. 362), MaxReversalCost(p. 326), UpdateBadTransitionCState(p. 292), DetermineCrossing(p. 367), BadTransitionCost(p. 291), IsStrengthening(p. 428), PointObsDescendCost(p. 305), UpdateCriticalIntervalProtectionCState(p. 296), SA01Heuristic(p. 330), CompatibilityCost(p. 319)

TRMIntruderStateUpdate (Algorithm 414) updates an internal data structure of the TRM related to intruder state, TRMIntruderState (Type 52). Using the inputs provided, the algorithm updates the values in the data structure appropriately.

First, information about the resolution advisory coordination messages sent on this cycle is preserved. Then the advisory reset and initialization reset flags are used to determine how much of the current advisory state should be preserved for the next cycle.

The advisory reset flag (*is_advisory_reset*) indicates whether or not any state information about the current advisory for this intruder should be preserved for the next cycle. The argument *is_advisory_reset* is set to TRUE under any of these conditions: ownship is in surveillance-only operational mode, there is degraded surveillance for this intruder, the intruder is invalid for TRM processing, or the intruder was dropped. The initialization reset flag (*is_initialization_reset*) indicates whether or not the online cost state should be fully reset when the advisory state is reset. The argument *is_initialization_reset* is set to TRUE whenever the intruder is dropped or invalid. The initialization state is also reset in IntruderPrep when entering RA operational mode.

When resetting for advisories but preserving the initialization state, the InitializationCState *t_count*, CrossingNoAlertCState *vrc_int_prev* and *range_prev* are preserved so traffic and resolution advisories will not be prohibited on the next cycle. When fully resetting for advisories and initialization, the algorithm resets all data values relevant to determining and issuing advisories.

The algorithm takes as inputs current state of the intruder (*st_int*), vrc coordination message from the intruder (*vrc*), cvc coordination message from the intruder (*cvc*), equipage code from the intruder (*equipage*), coordination message type from the intruder (*coordination_msg*), a boolean flag that is TRUE if an advisory reset should occur (*is_advisory_reset*), and a boolean flag that is true if an initialization reset should occur (*is_initialization_reset*).

The algorithm outputs no explicit return values; the algorithm makes modifications to the values stored in *st_int*.

Algorithm 414 TRMIntruderStateUpdate

```
function TRMIntruderStateUpdate( st_int::TRMIntruderState(p. 512), vrc::UInt32, cvc::UInt32,
                        equipage::Z, coordination_msg::UInt32,
                        is_advisory_reset::Bool, is_initialization_reset::Bool )
    st_int.sense_prev    = st_int.a_prev.sense
    st_int.vrc_prev      = vrc
    st_int.cvc_prev      = cvc
    st_int.equipage_prev = equipage
    st_int.coordination_msg_prev = coordination_msg
    st_int.a_prev.vrc = vrc
    st_int.a_prev.cvc = cvc
    if is_advisory_reset || is_initialization_reset
        st_int.a_prev       = IndividualAdvisory(p. 519)()
        st_int.b_prev       = AdvisoryBeliefState(p. 515)()
        st_int.st_arbitrate = ActionArbitrationCState(p. 526)()
        st_int.is_threat    = false
        if !is_initialization_reset
            t_count_init::Z = st_int.st_cost_on.initialization.t_count
            vrc_int_prev::UInt32 = st_int.st_cost_on.crossing_no_alert.vrc_int_prev
            st_int.st_cost_on = OnlineCostState(p. 520)()
            st_int.st_cost_on.initialization.t_count = t_count_init
            st_int.st_cost_on.crossing_no_alert.vrc_int_prev = vrc_int_prev
        else
            st_int.st_cost_on = OnlineCostState(p. 520)()
        end
    end
end
```

Referenced In: SetInvalidIntruder(p. 395), SetDroppedIntruder(p. 394), AdjustRAModeIntruderOutput(p. 379), GenerateSurvOnlyModeOutput(p. 382)

TURNENUBELIEFBASEDONINDIVIDUALSAMPLES (Algorithm 415) updates the belief states assuming a turn rate of *rate* radians/second for a duration of *dt* seconds.

TURNENUBELIEFBASEDONINDIVIDUALSAMPLES takes as inputs *b_enu_set*, *rate*, and *dt*. This algorithm modifies its inputs; the updated set of belief states is returned in *b_enu_set*.

Algorithm 415 TurnEnuBeliefBasedOnIndividualSamples

```
function TurnEnuBeliefBasedOnIndividualSamples( b_enu_set::EnuBeliefSet(p. 517), rate::R, dt::R )
    b_enu_set.enu_ave = EnuPositionVelocity(p. 517)()
    for b::EnuBelief(p. 516) in b_enu_set.b_enu
        TurnEnuPositionVelocity(p. 435)( b.enu, rate, dt )
        b_enu_set.enu_ave.pos_enu = b_enu_set.enu_ave.pos_enu + (b.enu.pos_enu * b.weight)
        b_enu_set.enu_ave.vel_enu = b_enu_set.enu_ave.vel_enu + (b.enu.vel_enu * b.weight)
    end
end
```

Referenced In: HorizontalAlertLookahead(p. 214)

TURNENUPOSITIONVELOCITY (Algorithm 416) updates the position and velocity of an estimate assuming a turn rate of *rate* radians/second is undertaken for *dt* seconds.

TURNENUPOSITIONVELOCITY takes as inputs *position_velocity*, *rate*, and *dt*. This algorithm modifies its inputs;the updated position and velocity is returned in *position_velocity*.

Algorithm 416 TurnEnuPositionVelocity

```
function TurnEnuPositionVelocity( position_velocity::EnuPositionVelocity(p. 517), rate::R, dt::R )
    normal_to_earth_surface::Vector{R} = [0.0, 0.0, 1.0]
    current_velocity::Vector{R} = position_velocity.vel_enu
    acceleration_direction::Vector{R} = UnitVectorNormalize(p. 583)( cross( current_velocity,
        normal_to_earth_surface ) )
    acceleration::Vector{R} = rate * norm( current_velocity ) * acceleration_direction
    position_velocity.vel_enu = norm( current_velocity ) * UnitVectorNormalize(p. 583)(current_velocity + (dt *
        acceleration))
    position_velocity.pos_enu = position_velocity.pos_enu + (dt * current_velocity) + (dt * dt * 0.5 *
        acceleration)
end
```

Referenced In: TurnEnuBeliefBasedOnIndividualSamples(p. 434)

UPDATEHTRMINTRUDERSTATE (Algorithm 417) updates the individual intruder state data structure (*st_int*) in preparation for the next processing cycle.

This algorithm takes as inputs *st_int*, *reset_priority* and *reset_state*. The fields in the state (*st_int*) are set selectively based on the values of the *reset_state* and *reset_priority* inputs.

Algorithm 417 UpdateHTRMIntruderState

```
function UpdateHTRMIntruderState( st_int::HTRMIntruderState(p. 514), reset_priority::Bool, reset_state::Bool )
    if reset_state
        st_int.enu_beliefs              = EnuBeliefSet(p. 517)()
        st_int.t_init                   = 0.0
        st_int.speed_bin_prev_policy    = 0
        st_int.is_master                = false
        st_int.is_initialized_speed_bin = false
    end
    if reset_state || reset_priority
        st_int.in_advisory         = false
        st_int.in_priority_list    = false
        st_int.coc_cost            = NaN
    end
end
```

Referenced In: PrioritizeAndFilterIntruders(p. 171), UpdateHTRMIntruderStates(p. 436), SetHorizontalIntruderOutput(p. 235), SetHorizontalDroppedIntruderOutput(p. 236)

UPDATEHTRMINTRUDERSTATES (Algorithm 418) updates a vector of intruder state data structures (*st_int*) in preparation for the next processing cycle. It calls UPDATEHTRMINTRUDERSTATE for each intruder state.

This algorithm takes as inputs *st_int*, *reset_priority* and *reset_state*. This algorithm updates *st_int*, *reset_state*, and *reset_priority*.

Algorithm 418 UpdateHTRMIntruderStates

```
function UpdateHTRMIntruderStates( st_int::Vector{HTRMIntruderState(p. 514)}, reset_priority::Bool, reset_state::
    Bool )
    for i in 1:length( st_int )
        UpdateHTRMIntruderState(p. 435)( st_int[i], reset_priority, reset_state )
    end
end
```

Referenced In: SetHorizontalTRMSurvOnlyMode(p. 169), HorizontalTRMUpdate(p. 168)

VRCTOSENSE (Algorithm 419) maps the VRC message from an intruder to the appropriate sense symbol.

The algorithm takes as input the VRC coordination message from the intruder (*vrc*). The algorithm outputs the RA sense based on the intruder VRC message (*sense*).

Algorithm 419 VRCToSense

```
function VRCToSense( vrc::UInt32 )
    local sense::Symbol
    if (vrc == 0)
        sense = :None
    elseif (vrc == 1)
        sense = :Down
    else
        sense = :Up
    end
    return sense::Symbol
end
```

Referenced In: UpdateIntruderInputs(p. 307), UpdateCoordinationDelayCState(p. 323), UpdateMaxReversalCState(p. 327), RestrictCOCDueToReversal(p. 329), IntruderResponseEstimation(p. 314), ShouldReverse(p. 332), Crosslink(p. 362), UpdateCoordinatedRADeferralCState(p. 322), PreventEarlyWeakeningCost(p. 328), UpdateCompatibilityCState(p. 320), CompatibilityCost(p. 319), UpdateCrossingNoAlertCState(p. 325)

A STM Housekeeping

At the end of each processing cycle, the STM performs housekeeping tasks through a call to STM-HOUSEKEEPING (Algorithm 420) that are required to happen after the TRM has updated.

Algorithm 420 StmHousekeeping

```
function StmHousekeeping( report_trm::sXuTRMReport(p.501) )
    # Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1
    StmHousekeepingPrioritization(p.438)( report_trm )
end
```

STMHOUSEKEEPINGPRIORITIZATION (Algorithm 421) updates the TARGETs vertical and horizontal PRIORITYCODES using information in the SXUTRMREPORT input. The intruder codes taken from the SXUTRMREPORT are set by the TRM to rank the severity of the threat of the intruder (higher values mean a higher threat).

Next, the vertical and horizontal intruder codes (stored in PRIORITYCODES) for each entry in the target database (*target_db*) are updated. The target associated with each id in the target database is retrieved using RETRIEVEWITHID. If there is a TRM intruder code output associated with that id, the PRIORITYCODES are updated from the intruder. Otherwise, the PRIORITYCODES settings that indicate an elevated threat from the intruder are reset. The TARGET maintains basic alerting history information for each intruder which is updated to reflect whether the current priority code is the highest code received thus far (*highest_code*). Whether the intruder has recieved any priority code is also updated and stored in *had_code*.

This algorithm takes as inputs *report*. This algorithm updates *tgt.priority_codes*.

Algorithm 421 StmHousekeepingPrioritization

```
function StmHousekeepingPrioritization( report::sXuTRMReport(p. 501) )
    xucode_dict_vert::Dict{UInt32,UInt8} = Dict{UInt32,UInt8}()
    for intruder::TRMIntruderDisplayData(p. 503) in report.display_vert.intruder
        xucode_dict_vert[intruder.id] = intruder.code
    end
    xucode_dict_horiz::Dict{UInt32,UInt8} = Dict{UInt32,UInt8}()
    for intruder::HTRMIntruderDisplayData(p. 504) in report.display_horiz.intruder
        xucode_dict_horiz[intruder.id] = intruder.code
    end
    for id::UInt32 in keys( target_db )
        tgt::Target(p. 493) = RetrieveWithID(p. 19)( target_db, id )
        if haskey( xucode_dict_vert, id )
            tgt.priority_codes.vert = xucode_dict_vert[id]
        else
            tgt.priority_codes.vert = SXUCODE_CLEAR
        end
        if haskey( xucode_dict_horiz, id )
            tgt.priority_codes.horiz = xucode_dict_horiz[id]
        else
            tgt.priority_codes.horiz = SXUCODE_CLEAR
        end
        tgt.priority_codes.highest_code = max(tgt.priority_codes.vert, tgt.priority_codes.horiz, tgt.
            priority_codes.highest_code)
        tgt.priority_codes.had_code = tgt.priority_codes.highest_code > SXUCODE_CLEAR
    end
    return
end
```

Referenced In: StmHousekeeping(p. 437)

B Correlation Processing Implementation

B.1 Track Extrapolation

PREDICTTRACKSUMMARY (Algorithm 422) iterates through every TARGET in memory and obtains each ADSBTRACKFILE, V2VTRACKFILE, ORNCTTRACKFILE, and AGTTRACKFILEs contained therein through a call to GETTRACKSTOEXTRAPOLATE (Algorithm 423). The TRACKSUMMARY for each track is then extrapolated to time *T* using EXTRAPOLATETRACK (Algorithm 425), which calls the appropriate algorithm depending on the track type. The AGTTRACKFILEs stored in the OWNSHIPDATA instance *own* are similarly extrapolated to time *T* through a call to EXTRAPOLATETRACK.

PREDICTTRACKSUMMARY takes as input *T*, calls EXTRAPOLATETRACK to modifies existing tracks, and generates no return value.

GETTRACKSTOEXTRAPOLATE takes as input *target*, and returns *tracks_to_extrap*, a vector of TRACKFILEs created by calling GETAIRCRAFTTRACKS.

GETAIRCRAFTTRACKS (Algorithm 424) takes as input *target* and returns a vector of all valid airborne TRACKFILEs within the TARGET. A TRACKFILE is airborne if it is used to track an intruder aircraft i.e. not a POINTOBSTACLETRACKFILE.

EXTRAPOLATETRACK takes as input *trk* and *T*. EXTRAPOLATETRACK calls either EXTRAPOLATEADSBTRACK, EXTRAPOLATEV2VTRACK, EXTRAPOLATEORNCTTRACK, or EXTRAPOLATEAGTTRACK. Then CORRELATIONTRACKTYPE (Algorithm 434) is called to populate *trk.trk_summary.type*. This track type designation is used exclusively in correlation processing to determine the suitable correlation mechanism given a pair of tracks.

Algorithm 422 PredictTrackSummary

```
function PredictTrackSummary(T::R)
    ExtrapolateOwnshipTrack(p. 442)(T)
    for id in keys(target_db)
        target = target_db[id]
        tracks = GetTracksToExtrapolate(p. 440)(target)
        for trk in tracks
            ExtrapolateTrack(p. 440)(trk, T)
        end
    end
    for trk in own.agt_tracks
        ExtrapolateTrack(p. 440)(trk, T)
    end
end
```

Referenced In: CorrelationProcessing(p. 111)

Algorithm 423 GetTracksToExtrapolate

```
function GetTracksToExtrapolate(target::Target(p. 493))
    tracks_to_extrap::Vector{TrackFile(p. 483)} = GetAircraftTracks(p. 440)(target)
    return tracks_to_extrap::Vector{TrackFile(p. 483)}
end
```

Referenced In: PredictTrackSummary(p. 439)

Algorithm 424 GetAircraftTracks

```
function GetAircraftTracks(target::Target(p. 493))
    tgt_tracks = TrackFile(p. 483)[]
    if TrackExists(p. 19)(target.adsb_track)
        push!(tgt_tracks, target.adsb_track)
    end
    if TrackExists(p. 19)(target.v2v_track)
        push!(tgt_tracks, target.v2v_track)
    end
    if TrackExists(p. 19)(target.ornct_track)
        push!(tgt_tracks, target.ornct_track)
    end
    append!(tgt_tracks, target.agt_tracks)
    return tgt_tracks::Vector{TrackFile(p. 483)}
end
```

Referenced In: MergeTargets(p. 464), GetTracksToDecorrelate(p. 458), GetTracksToExtrapolate(p. 440), CorrelateID(p. 468)

Algorithm 425 ExtrapolateTrack

```
function ExtrapolateTrack(trk::TrackFile(p. 483), T::R)
    track_type = typeof(trk)
    if (track_type == ADSBTrackFile(p. 485))
        ExtrapolateADSBTrack(p. 443)(trk, T)
    elseif (track_type == V2VTrackFile(p. 488))
        ExtrapolateV2VTrack(p. 444)(trk, T)
    elseif (track_type == ORNCTTrackFile(p. 484))
        ExtrapolateORNCTTrack(p. 445)(trk, T)
    elseif (track_type == AGTTrackFile(p. 486))
        ExtrapolateAGTTrack(p. 447)(trk, T)
    end
    trk.trk_summary.toa = T
    trk.trk_summary.type = CorrelationTrackType(p. 449)(trk)
end
```

Referenced In: PredictTrackSummary(p. 439)

To update the ownship TRACKSUMMARY to the current time, EXTRAPOLATEOWNSHIPTRACK (Algorithm 426) first retrieves ownship horizontal states, namely the means *mu_xy_meters* and the covariances *Sigma_xy_meters*. It is worth noting that, as the states are defined in an East-North Cartesian frame centered on ownship, the means are guaranteed to be zeroes (i.e., the origin), and the variable *mu_xy_meters* is hence hard-coded to that effect.

The retrieval of the vertical states first undergoes a source selection process. If the pressure al-

titude (PA) measurements have been valid, as indicated by *own.toa_vert* not being a *NaN*, the means and the covariances of the vertical states in the TRACKSUMMARY are to be populated with the PA measurements, time-propagated to the current time step *T* using PREDICTVERTICALTRACKER. The *alt_src_hae* Boolean flag within the TRACKSUMMARY is set to FALSE to reflect the fact that the PA measurements are used to populate ownship vertical states.

If the PA measurements have been invalid, yet the geodetic height above ellipsoid (HAE) measurements have been valid, as indicated by *own.wgs84_toa_vert* not being a *NaN*, the HAE means and covariances are time-propagated with PREDICTVERTICALTRACKER and are used to populate ownship vertical states. *TS.alt_src_hae* is set to TRUE accordingly.

If neither set of altitude measurements has been valid, the means and the covariances of the vertical states are set to zeroes.

Lastly, CONVERTCARTESIANTOPOLAR2D is called to convert the horizontal Cartesian states to the polar coordinate frame, again centered on ownship, so as to populate the corresponding fields in the TRACKSUMMARY.

The algorithm receives *T* as input. The algorithm modifies the TRACKSUMMARY instance, *own.trk_summary* within the data structure OWNSHIPDATA. The algorithm returns no value.

Algorithm 426 ExtrapolateOwnshipTrack

```
function ExtrapolateOwnshipTrack(T::R)
    m2ft::R = geoutils.meters_to_feet
    I_rng_az::Vector{Z} = [1, 3]
    TS::TrackSummary(p. 495)           = own.trk_summary
    mu_xy_meters::Vector{R}    = zeros(4)
    Sigma_xy_meters::Matrix{R} = copy(own.wgs84_Sigma_hor)
    mu_ra::Vector{R}           = zeros(4)
    Sigma_ra::Matrix{R}        = zeros(4,4)
    valid_alt_pres::Bool = !isnan(own.toa_vert)
    valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
    if valid_alt_pres
        dt_vert = T - own.toa_vert
        (TS.mu_vert, TS.Sigma_vert) = PredictVerticalTracker(p. 54)(own.mu_vert, own.Sigma_vert, dt_vert, false,
            true, false)
        TS.alt_src_hae = false
        TS.valid_vert  = true
    elseif valid_alt_hae
        dt_vert = T - own.wgs84_toa_vert
        (TS.mu_vert, TS.Sigma_vert) = PredictVerticalTracker(p. 54)(own.wgs84_mu_vert, own.wgs84_Sigma_vert,
            dt_vert, true, true, false)
        TS.alt_src_hae = true
        TS.valid_vert  = true
    else
        TS.mu_vert = zeros(2)
        TS.Sigma_vert = zeros(2, 2)
        TS.valid_vert = false
    end
    mu_xy             = mu_xy_meters * m2ft
    Sigma_xy          = Sigma_xy_meters * (m2ft^2)
    (mu_ra, Sigma_ra) = ConvertCartesianToPolar2D(p. 448)(mu_xy, Sigma_xy)
    TS.mu_rng_az      = mu_ra[I_rng_az]
    TS.Sigma_rng_az   = Sigma_ra[I_rng_az,I_rng_az]
    TS.mu_range       = mu_ra[1:2]
    TS.Sigma_range    = Sigma_ra[1:2,1:2]
    TS.valid_rng      = true
    TS.toa = T
    TS.type = CORRELATION_TRKTYPE_INVALID
end
```

Referenced In: PredictTrackSummary(p. 439)

To update the TRACKSUMMARY of the ADSBTRACKFILE to the current time T, EXTRAPOLATEADSBTRACK (Algorithm 427) retrieves the means and covariances of the vertical estimates and performs time propgation thereon through a call to PREDICTVERTICALTRACKER.

The time propagation of the horizontal estimates consists of four steps:

1. An identical copy of *trk*, *trk_temp*, is createdz.
2. A call to PREDICTPASSIVETRACKER time-propagates to the current time step T the means and the covariances of the horizontal states, i.e., *trk_temp.mu_hor* and *trk_temp.Sigma_hor*. These states are defined in an East-North Cartesian frame with respect to the position recorded in ADSBTRACKFILE, namely *trk_temp.lla_at_hor_toa* and its ECEF counterpart *trk_temp.ecef_at_hor_toa*. These two global coordinates were recorded at the previous time instance, *trk_temp.toa_hor*, and are not altered by PREDICTPASSIVETRACKER.
3. RECENTERHORIZONTALTRACKLOCATION is called to update *trk_temp.lla_at_hor_toa* and *trk_temp.ecef_at_hor_toa* so that they will colocate with the new values of *trk_temp.mu_hor* obtained from the previous step. This shift also results in the x and the y components

of *trk_temp.mu_hor* being reduced to *0*, as the origin of the coordinate frame changes.

4. The updated global coordinates are used to extract relative Cartesian states of the intruder with respect to ownship, *mu_xy_meters* and *Sigma_xy_meters*, through a call to ABSOLUTEGEODETICTOOWNSHIPRELATIVE.

In order to complete the *mu_range* field of the TRACKSUMMARY, the updated Cartesian estimate (*mu_xy* and *Sigma_xy*) is transformed to a ground range estimate using CONVERTCARTESIANTOPOLAR2D (Algorithm 432).

This algorithm takes as input *trk* and *T*. This algorithm updates the input *trk*.

Algorithm 427 ExtrapolateADSBTrack

```
function ExtrapolateADSBTrack(trk::ADSBTrackFile(p. 485), T::R)
    m2ft::R = geoutils.meters_to_feet
    I_rng_az::Vector{Z} = [1, 3]
    TS::TrackSummary(p. 495)           = trk.trk_summary
    mu_vert::Vector{R}         = copy(trk.mu_vert)
    Sigma_vert::Matrix{R}      = copy(trk.Sigma_vert)
    mu_ra::Vector{R}           = zeros(4)
    Sigma_ra::Matrix{R}        = zeros(4,4)
    dt_vert = T - trk.toa_vert
    dt_hor = T - trk.toa_hor
    (TS.mu_vert, TS.Sigma_vert) = PredictVerticalTracker(p. 54)(mu_vert, Sigma_vert, dt_vert, trk.alt_src_hae,
        false, true)
    TS.alt_src_hae = trk.alt_src_hae
    trk_temp = deepcopy(trk)
    (trk_temp.mu_hor, trk_temp.Sigma_hor) = PredictPassiveTracker(p. 46)(trk_temp.mu_hor, trk_temp.Sigma_hor,
        dt_hor, false)
    ReCenterHorizontalTrackLocation(p. 42)(trk_temp, T)
    (mu_xy_meters, Sigma_xy_meters) = AbsoluteGeodeticToOwnshipRelative(p. 141)(trk_temp)
    mu_xy = mu_xy_meters * m2ft
    Sigma_xy = Sigma_xy_meters * (m2ft^2)
    (mu_ra, Sigma_ra) = ConvertCartesianToPolar2D(p. 448)(mu_xy, Sigma_xy)
    TS.mu_rng_az = mu_ra[I_rng_az]
    TS.Sigma_rng_az = Sigma_ra[I_rng_az,I_rng_az]
    TS.mu_range    = mu_ra[1:2]
    TS.Sigma_range = Sigma_ra[1:2,1:2]
    TS.valid_rng = true
    TS.valid_vert = trk.valid_vert
end
```

Referenced In: ExtrapolateTrack(p. 440)

To update the TRACKSUMMARY of the V2VTRACKFILE to the current time *T*, EXTRAPOLATEV2VTRACK (Algorithm 428) retrieves the means and covariances of the vertical estimates and performs time propgation thereon through a call to PREDICTVERTICALTRACKER.

The time propagation of the horizontal estimates consists of four steps:

1. An identical copy of *trk*, *trk_temp*, is createdz.
2. A call to PREDICTPASSIVETRACKER time-propagates to the current time step *T* the means and the covariances of the horizontal states, i.e., *trk_temp.mu_hor* and *trk_temp.Sigma_hor*. These states are defined in an East-North Cartesian frame with respect to the position recorded in V2VTRACKFILE, namely *trk_temp.lla_at_hor_toa* and its ECEF counterpart *trk_temp.ecef_at_hor_toa*. These two global coordinates were recorded at the previous time

instance, *trk_temp.toa_hor*, and are not altered by PREDICTPASSIVETRACKER.

3. RECENTERHORIZONTALTRACKLOCATION is called to update *trk_temp.lla_at_hor_toa* and *trk_temp.ecef_at_hor_toa* so that they will colocate with the new values of *trk_temp.mu_-hor* obtained from the previous step. This shift also results in the x and the y components of *trk_temp.mu_hor* being reduced to *0*, as the origin of the coordinate frame changes.
4. The updated global coordinates are used to extract relative Cartesian states of the intruder with respect to ownship, *mu_xy_meters* and *Sigma_xy_meters*, through a call to ABSOLUTE-GEODETICTOOWNSHIPRELATIVE.

In order to complete the *mu_range* field of the TRACKSUMMARY, the updated Cartesian estimate (*mu_xy* and *Sigma_xy*) is transformed to a ground range estimate using CONVERTCARTESIANTOPOLAR2D (Algorithm 432).

This algorithm takes as input *trk* and *T*. This algorithm updates the input *trk*.

Algorithm 428 ExtrapolateV2VTrack

```
function ExtrapolateV2VTrack(trk::V2VTrackFile(p. 488), T::R)
    m2ft::R = geoutils.meters_to_feet
    I_rng_az::Vector{Z} = [1, 3]
    TS::TrackSummary(p. 495)           = trk.trk_summary
    mu_vert::Vector{R}         = copy(trk.mu_vert)
    Sigma_vert::Matrix{R}      = copy(trk.Sigma_vert)
    mu_ra::Vector{R}           = zeros(4)
    Sigma_ra::Matrix{R}        = zeros(4,4)
    dt_vert = T - trk.toa_vert
    dt_hor = T - trk.toa_hor
    is_large::Bool = (trk.classification != CLASSIFICATION_SMALL_UNMANNED)
    (TS.mu_vert, TS.Sigma_vert) = PredictVerticalTracker(p. 54)(mu_vert, Sigma_vert, dt_vert, trk.alt_src_hae,
        false, is_large)
    TS.alt_src_hae = trk.alt_src_hae
    trk_temp = deepcopy(trk)
    (trk_temp.mu_hor, trk_temp.Sigma_hor) = PredictPassiveTracker(p. 46)(trk_temp.mu_hor, trk_temp.Sigma_hor,
        dt_hor, false)
    ReCenterHorizontalTrackLocation(p. 42)(trk_temp, T)
    (mu_xy_meters, Sigma_xy_meters) = AbsoluteGeodeticToOwnshipRelative(p. 141)(trk_temp)
    mu_xy = mu_xy_meters * m2ft
    Sigma_xy = Sigma_xy_meters * (m2ft^2)
    (mu_ra, Sigma_ra) = ConvertCartesianToPolar2D(p. 448)(mu_xy, Sigma_xy)
    TS.mu_rng_az = mu_ra[I_rng_az]
    TS.Sigma_rng_az = Sigma_ra[I_rng_az,I_rng_az]
    TS.mu_range    = mu_ra[1:2]
    TS.Sigma_range = Sigma_ra[1:2,1:2]
    TS.valid_rng = true
    TS.valid_vert = trk.valid_vert
end
```

Referenced In: ExtrapolateTrack(p. 440)

To propagate the TRACKSUMMARY of the ORNCTTRACKFILE to the current time, EXTRAPOLATE-ORNCTTRACK (Algorithm 429) calls PREDICTORNCTTRACK (Algorithm 66) to obtain updated means and covaraince of the vertical, range, and Cartesian state estimates. The means of the ownship vertical states, *own.trk_summary.mu_vert*, are added to the time-propagated vertical state of the intruder *TS.mu_vert* so the latter is not defined with respect to the ownship but to a global datum, let it be pressure altitude or geodetic height above ellipsoid (HAE). The actual datum is indicated by the flag *TS.alt_src_hae*, which is also populated.

The updated Cartesian estimate (*mu_xy* and *Sigma_xy*) is transformed to a polar estimate using LINEARTRANSFORM (Algorithm 430). In addition, the range and azimuth-angle dimensions of the covariances may be replaced with a minimum uncertainty parameter if they are not large enough to account for lag bias errors and maneuver uncertainty when compared against other sensors. Lastly, the azimuth-angle dimension of the covariance (*Sigma_rng_az*) is further inflated to account for the range dependent angular uncertainty at close ground range.

This algorithm takes as input *trk* and *T*. This algorithm updates *trk*.

Algorithm 429 ExtrapolateORNCTTrack

```
function ExtrapolateORNCTTrack(trk::ORNCTTrackFile(p. 484), T::R)
    lag_uncertainty::Vector{R} = params().surveillance.ornct.lag_uncertainty_minimum
    I_xy::Vector{Z} = [1,3]
    dt = T - trk.toa
    TS::TrackSummary(p. 495) = trk.trk_summary
    (TS.mu_vert, TS.Sigma_vert, TS.mu_range, TS.Sigma_range, mu_xy, Sigma_xy) = PredictORNCTTrack(p. 64)(trk,
        dt)
    zdz_rel = TS.mu_vert
    hdh_own = own.trk_summary.mu_vert
    TS.mu_vert = zdz_rel + hdh_own
    TS.alt_src_hae = own.trk_summary.alt_src_hae
    (TS.mu_rng_az, TS.Sigma_rng_az) = LinearTransform(p. 446)(mu_xy[I_xy], Sigma_xy[I_xy,I_xy])
    TS.Sigma_range[1,1] = max(TS.Sigma_range[1,1], lag_uncertainty[1]^2)
    TS.Sigma_rng_az[2,2] = max(TS.Sigma_rng_az[2,2], lag_uncertainty[2]^2)
    TS.Sigma_rng_az[2,2] = max(TS.Sigma_rng_az[2,2], atan(sqrt(TS.Sigma_rng_az[1,1]), TS.mu_rng_az[1,1])^2)
    TS.valid_rng = true
    TS.valid_vert = own.trk_summary.valid_vert
end
```

Referenced In: ExtrapolateTrack(p. 440)

LINEARTRANSFORM (Algorithm 430) performs the transformation of a Cartesian state estimate to a polar (range and azimuth) state estimate. The inputs are the mean (*mu_xy*) and covariance matrix (*Sigma_xy*) of the Cartesian state estimate. The outputs are the mean (*mu_ra*) and covariance matrix (*Sigma_ra*) of the polar state estimate. The transformation between Cartesian and polar states is a non-linear process, which is applied directly to the input Cartesian means, resulting in the mean values of *range* and *azimuth* of the polar state. The non-linear transformation cannot be directly applied to the covariance matrix; instead, the transformation is linearized about the mean value in the form of the Jacobian matrix *H*, which is applied as a transformation to the input covariance matrix and results in the output *Sigma_ra*.

This algorithm inputs *mu_xy* and *Sigma_xy*. This algorithm returns *mu_ra* and *Sigma_ra*.

Algorithm 430 LinearTransform

```
function LinearTransform(mu_xy::Vector{R}, Sigma_xy::Matrix{R})
    range = sqrt(mu_xy[1]^2 + mu_xy[2]^2)
    azimuth = atan(mu_xy[1], mu_xy[2])
    mu_ra = [range, azimuth]
    H = [mu_xy[1]/mu_ra[1] mu_xy[2]/mu_ra[1]; mu_xy[2]/mu_ra[1]^2 -mu_xy[1]/mu_ra[1]^2]
    Sigma_ra = H*Sigma_xy*H';
    return (mu_ra::Vector{R}, Sigma_ra::Matrix{R})
end
```

Referenced In: ExtrapolateORNCTTrack(p. 445)

To update the TRACKSUMMARY of the AGTTRACKFILE to the current time *T*, EXTRAPOLATEAGTTRACK (Algorithm 431) retrieves the means and covariances of the vertical estimates and performs time propgation thereon through a call to PREDICTVERTICALTRACKER.

The time propagation of the horizontal estimates consists of four steps:

1. An identical copy of *trk*, *trk_temp*, is createdz.
2. A call to PREDICTPASSIVETRACKER time-propagates to the current time step *T* the means and the covariances of the horizontal states, i.e., *trk_temp.mu_hor* and *trk_temp.Sigma_hor*. These states are defined in an East-North Cartesian frame with respect to the position recorded in AGTTRACKFILE, namely *trk_temp.lla_at_hor_toa* and its ECEF counterpart *trk_temp.ecef_at_hor_toa*. These two global coordinates were recorded at the previous time instance, *trk_temp.toa_hor*, and are not altered by PREDICTPASSIVETRACKER.
3. RECENTERHORIZONTALTRACKLOCATION is called to update *trk_temp.lla_at_hor_toa* and *trk_temp.ecef_at_hor_toa* so that they will colocate with the new values of *trk_temp.mu_hor* obtained from the previous step. This shift also results in the *x* and the *y* components of *trk_temp.mu_hor* being reduced to *0*, as the origin of the coordinate frame changes.
4. The updated global coordinates are used to extract relative Cartesian states of the intruder with respect to ownship, *mu_xy_meters* and *Sigma_xy_meters*, through a call to ABSOLUTEGEODETICTOOWNSHIPRELATIVE.

In order to complete the *mu_range* field of the TRACKSUMMARY, the updated Cartesian estimate (*mu_xy* and *Sigma_xy*) is transformed to a ground range estimate using CONVERTCARTESIANTOPOLAR2D (Algorithm 432).

Lastly, the algorithm updates the *max_range_vec* vector within the AGTTRACKFILE, which records a predetermined number of largest ranges observed for the track. This is to support a mechanism that prevents ownship from correlating with an absolute geodetic track that has previously been observed to be at a long distance from ownship, rendering it highly unlikely to be an ownship AGT track.

This algorithm takes as input *trk* and *T*. This algorithm updates the input *trk*.

Algorithm 431 ExtrapolateAGTTrack

```
function ExtrapolateAGTTrack(trk::AGTTrackFile(p. 486), T::R)
    m2ft::R = geoutils.meters_to_feet
    max_size_max_range_vec::R = params().surveillance.correlation.ownship_correlation.max_size_max_range_vec
    I_rng_az::Vector{Z} = [1, 3]
    TS::TrackSummary(p. 495)           = trk.trk_summary
    mu_vert::Vector{R}         = copy(trk.mu_vert)
    Sigma_vert::Matrix{R}      = copy(trk.Sigma_vert)
    mu_ra::Vector{R}           = zeros(4)
    Sigma_ra::Matrix{R}        = zeros(4,4)
    dt::R = T - trk.toa
    is_large::Bool = (trk.classification != CLASSIFICATION_SMALL_UNMANNED)
    (TS.mu_vert, TS.Sigma_vert) = PredictVerticalTracker(p. 54)(mu_vert, Sigma_vert, dt, trk.alt_src_hae, false,
        is_large)
    TS.alt_src_hae = trk.alt_src_hae
    trk_temp = deepcopy(trk)
    (trk_temp.mu_hor, trk_temp.Sigma_hor) = PredictPassiveTracker(p. 46)(trk_temp.mu_hor, trk_temp.Sigma_hor, dt,
        false)
    ReCenterHorizontalTrackLocation(p. 42)(trk_temp, T)
    (mu_xy_meters, Sigma_xy_meters) = AbsoluteGeodeticToOwnshipRelative(p. 141)(trk_temp)
    mu_xy = mu_xy_meters * m2ft
    Sigma_xy = Sigma_xy_meters * (m2ft^2)
    (mu_ra, Sigma_ra) = ConvertCartesianToPolar2D(p. 448)(mu_xy, Sigma_xy)
    TS.mu_rng_az = mu_ra[I_rng_az]
    TS.Sigma_rng_az = Sigma_ra[I_rng_az,I_rng_az]
    TS.mu_range    = mu_ra[1:2]
    TS.Sigma_range = Sigma_ra[1:2,1:2]
    TS.valid_rng = true
    TS.valid_vert = trk.valid_vert
    z_rel = GetRelativeAltitude(p. 139)(TS.mu_vert[1], TS.alt_src_hae)
    max_range = SlantRange(p. 133)(mu_ra[1], z_rel)
    push!(trk.max_range_vec, max_range)
    if (length(trk.max_range_vec) > max_size_max_range_vec)
        idx_min = argmin(trk.max_range_vec)
        deleteat!(trk.max_range_vec, idx_min)
    end
end
```

Referenced In: ExtrapolateTrack(p. 440)

CONVERTCARTESIANTOPOLAR2D (Algorithm 432) takes the two dimensional mean and covariance of a Cartesian state estimate and performs a coordinate transformation into a polar state estimate. The coordinate transformation is performed by sigma-point sampling the Cartesian distribution using SIGMAPOINTSAMPLE and calculating the positions of the samples in polar coordinates. The samples are combined to form a polar mean and covariance using WEIGHTEDMEANANDCOVARIANCE, which are returned to the calling function.

This algorithm takes as input *mu_xy* and *Sigma_xy*. This algorithm returns *mu_ra* and *Sigma_ra*.

Algorithm 432 ConvertCartesianToPolar2D

```
function ConvertCartesianToPolar2D(mu_xy::Vector{R}, Sigma_xy::Matrix{R})
    (S_xy::Matrix{R}, weights_xy::Vector{R}) = SigmaPointSample(p. 144)(mu_xy, Sigma_xy)
    S_ra::Matrix{R} = zeros(size(S_xy))
    for i in 1:length(weights_xy)
        S_ra[1,i] = sqrt(S_xy[1,i]^2 + S_xy[3,i]^2)
        S_ra[3,i] = atan(S_xy[1,i], S_xy[3,i])
        S_ra[2,i] = (S_xy[2,i] * sin(S_ra[3,i])) + (S_xy[4,i] * cos(S_ra[3,i]))
        S_ra[4,i] = (S_xy[2,i] * cos(S_ra[3,i])) - (S_xy[4,i] * sin(S_ra[3,i]))
    end
    (mu_ra, Sigma_ra) = WeightedMeanAndCovariance(p. 448)(S_ra, weights_xy)
    return (mu_ra::Vector{R}, Sigma_ra::Matrix{R})
end
```

Referenced In: ExtrapolateADSBTrack(p. 443), ExtrapolateAGTTrack(p. 447), OptimalQualityTrack(p. 125), ExtrapolateV2VTrack(p. 444), ExtrapolateOwnshipTrack(p. 442)

WEIGHTEDMEANANDCOVARIANCE (Algorithm 433) combines sigma point samples and a set of associated weights into a weighted covariance matrix, *Sigma*, and the weighted average, *mu*. The algorithm takes as input a set of samples (*z*) and a set of weights for those samples (*w*).

This algorithm returns the mean of the distribution (*mu*) and the covariance matrix (*Sigma*).

Algorithm 433 WeightedMeanAndCovariance

```
function WeightedMeanAndCovariance(z::Array{R}, w::Vector{R})
    n = size(z,1)
    p = size(z,2)
    mu = zeros(n)
    for i in 1:p
        mu += w[i]*z[:,i]
    end
    Sigma = zeros(n,n)
    for i in 1:p
        Sigma += w[i]*(z[:,i]-mu)*(z[:,i]-mu)'
    end
    return (mu::Vector{R}, Sigma::Matrix{R})
end
```

Referenced In: ConvertCartesianToPolar2D(p. 448)

CORRELATIONTRACKTYPE (Algorithm 434) takes as input a TRACKFILE object, *trk*. The algorithm then generates a track type designation, *corr_trk_type*, based on the type of the track and the validity of various identifiers assigned to the track by external surveillance systems (e.g. Mode S address, V2V-UID). For instance, an AGTTRACKFILE with a valid ICAO Mode S address but without a valid V2V-UID will receive a track type designation of *CORRELATION_TRKTYPE_AGT_AGTID_-ICAO24*. In the case where the track is ineligible for processing by the correlation algorithm or where the track is an ownship track, it will receive a designation of *CORRELATION_TRKTYPE_-INVALID*.

The algorithm returns *corr_trk_type*, which will be one of the *CORRELATION_TRKTYPE_** constants.

Algorithm 434 CorrelationTrackType

```
function CorrelationTrackType(trk::TrackFile(p. 483))
    trk_type = typeof(trk)
    corr_trk_type::UInt8 = CORRELATION_TRKTYPE_INVALID
    if (trk_type == ADSBTrackFile(p. 485))
        if !trk.non_icao
            corr_trk_type = CORRELATION_TRKTYPE_ADSB_ICAO24
        else
            corr_trk_type = CORRELATION_TRKTYPE_ADSB_ANON24
        end
    elseif (trk_type == V2VTrackFile(p. 488))
        if !trk.mode_s_valid
            corr_trk_type = CORRELATION_TRKTYPE_V2V_V2VUID
        elseif !trk.mode_s_non_icao
            corr_trk_type = CORRELATION_TRKTYPE_V2V_V2VUID_ICAO24
        else
            corr_trk_type = CORRELATION_TRKTYPE_V2V_V2VUID_ANON24
        end
    elseif (trk_type == ORNCTTrackFile(p. 484))
        corr_trk_type = CORRELATION_TRKTYPE_ORNCT_ORNCTID
    elseif (trk_type == AGTTrackFile(p. 486))
        if !trk.v2v_uid_valid
            if !trk.mode_s_valid
                corr_trk_type = CORRELATION_TRKTYPE_AGT_AGTID
            elseif !trk.mode_s_non_icao
                corr_trk_type = CORRELATION_TRKTYPE_AGT_AGTID_ICAO24
            else
                corr_trk_type = CORRELATION_TRKTYPE_AGT_AGTID_ANON24
            end
        else
            if !trk.mode_s_valid
                corr_trk_type = CORRELATION_TRKTYPE_AGT_AGTID_V2VUID
            elseif !trk.mode_s_non_icao
                corr_trk_type = CORRELATION_TRKTYPE_AGT_AGTID_V2VUID_ICAO24
            else
                corr_trk_type = CORRELATION_TRKTYPE_AGT_AGTID_V2VUID_ANON24
            end
        end
    end
    return corr_trk_type::UInt8
end
```

Referenced In: ExtrapolateTrack(p. 440)

B.2 Ownship Track Decorrelation

DECORRELATETARGETSFROMOWNSHIP (Algorithm 435) iterates through all AGTTRACKFILEs associated to ownship (*own.agt_tracks*) and creates new TARGETs based on tracks that have diverged from ownship. If the difference between the time T and the initialization time of the track is greater than the parameter *unique_time*, then a unique scale factor will be set to *unique_-multiplier*. During evaluation, the algorithm will skip correlation for the tracks that have been coasting for more than *max_coast_time*, or those that have been correlated because their V2V unique identifiers match that of ownship.

If CORRELATETOOWNSHIPBYTYPE (Algorithm 440) returns *CORRELATION_NEGATIVE*, the track has decorrelated on the basis of mismatched unique identifiers, and the AGTTRACKFILE should immediately be made a separate TARGET. Alternatively, if CORRELATETOOWNSHIPBYTYPE returns *CORRELATION_SPATIAL_NEGATIVE*, the track has spatially decorrelated, and an M-of-

N check will ensue to determine if the AGTTRACKFILE should be separated during this cycle. Specifically, CORRELATIONHISTORYUPDATE (Algorithm 443) is called to log the spatial decorrelation, and CORRELATIONHISTORYCHECK (Algorithm 444) then determines if the M-of-N check has been fulfilled, with M and N defined in the parameter file. If such is the case, the *status* is changed to *CORRELATION_NEGATIVE*, which will allow the splitting of the AGTTRACKFILE to proceed.

In the event that decorrelation is deemed necessary, the track is added to a new TARGET using ADDDECORRELATEDTRACKTOTARGET (Algorithm 445) and subsequently removed from the ownship using REMOVEDECORRELATEDTRACKFROMOWNSHIP (Algorithm 436). The new TARGETs are then saved to memory using ADDTODB.

DECORRELATETARGETSFROMOWNSHIP receives as input *T* and generates no return value; it may, however, alter *target_db* and the OWNSHIPDATA instance *own*.

REMOVEDECORRELATEDTRACKFROMOWNSHIP receives as input *trk* and modifies OWNSHIPDATA instance *own*. It does not generate a return value.

Algorithm 435 DecorrelateTargetsFromOwnship

```
function DecorrelateTargetsFromOwnship(T::R)
    max_coast_time::Z = params().surveillance.decorrelation.ownship_decorrelation.max_coast_time
    M::Z = params().surveillance.decorrelation.ownship_decorrelation.M
    N::Z = params().surveillance.decorrelation.ownship_decorrelation.N
    unique_time::Z = params().surveillance.decorrelation.ownship_decorrelation.unique_time
    unique_multiplier::R = params().surveillance.decorrelation.ownship_decorrelation.unique_multiplier
    ownship_correlation_parameters = params().surveillance.correlation.type_selection.ownship_position
    history::CorrelationHistory(p. 489) = own.own_decorr_history
    (history.M, history.N) = (M, N)
    decorrelated_targets::Vector{Target(p. 493)} = Target(p. 493)[]
    removed_tracks = TrackFile(p. 483)[]
    for i in 1:length(own.agt_tracks)
        trk = own.agt_tracks[i]
        if ( T - trk.init_time > unique_time)
            unique_scale = unique_multiplier
        else
            unique_scale = 1.0
        end
        dt::R = T - trk.toa_update
        if (dt > max_coast_time)
            continue
        end
        status::UInt8 = CorrelateToOwnshipByType(p. 455)(trk, unique_scale)
        if (status == CORRELATION_SPATIAL_NEGATIVE)
            own_track_id::String = "OwnWGS84"
            track_id::String = trk.track_uid
            CorrelationHistoryUpdate(p. 458)(own_track_id, track_id, history, T)
            if CorrelationHistoryCheck(p. 458)(own_track_id, track_id, history)
                status = CORRELATION_NEGATIVE
            end
        end
        if (status == CORRELATION_NEGATIVE)
            tgt = Target(p. 493)()
            tgt.init_time = trk.toa - unique_time
            AddDecorrelatedTrackToTarget(p. 459)(tgt, trk)
            push!(decorrelated_targets, tgt)
            push!(removed_tracks, trk)
        end
    end
    for rm_trk in removed_tracks
        RemoveDecorrelatedTrackFromOwnship(p. 451)(rm_trk)
    end
    for dtgt in decorrelated_targets
        AddToDB(p. 19)(target_db,dtgt)
    end
end
```

Referenced In: CorrelationProcessing(p. 111)

Algorithm 436 RemoveDecorrelatedTrackFromOwnship

```
function RemoveDecorrelatedTrackFromOwnship(trk::TrackFile(p. 483))
    for i in 1:length(own.agt_tracks)
        if (own.agt_tracks[i] == trk)
            deleteat!(own.agt_tracks, i)
            return
        end
    end
end
```

Referenced In: DecorrelateTargetsFromOwnship(p. 451)

B.3 Ownship Track Correlation

CORRELATETARGETSTOOWNSHIP (Algorithm 437) iterates through all TARGETs in memory. A given intruder TARGET is omitted from processing if it contains any type of tracks other than AGTTRACKFILEs. This is determined by passing the ID of the TARGET to AGTONLYTARGET (Algorithm 438). Otherwise, the TARGET ID is paired with each AGTTRACKFILE therein, and the pair is added to *track_list*.

The algorithm then iterates through the *track_list* and examines each track for ownship correlation. Before proceeding, an efficiency check is performed to skip correlation examination should the difference in range between the two tracks exceeds *range_gate*. If the gate is not exceeded, the process continues by passing the TRACKFILE and the TARGET ID to CORRELATEINDIVIDUALTRACKSTOOWNSHIP (Algorithm 439).

CORRELATETARGETSTOOWNSHIP receives as input *T* and modifies the data structure *target_db*.

AGTONLYTARGET receives as input *target_id* and returns a Boolean flag indicating if absolute geodetic tracks are the only type of tracks present in the specified intruder target (i.e., no ADS-B track, no V2V track, and no ORNCT track). Note that *trk.point_obstacle_track* are not examined since it is presumed that a TARGET representing an intruder aircraft would have never contained a POINTOBSTACLETRACKFILE.

Algorithm 437 CorrelateTargetsToOwnship

```
function CorrelateTargetsToOwnship(T::R)
    range_gate::Z = params().surveillance.correlation.ownship_correlation.range_gate
    track_list::Vector{TrackMap(p. 494)} = TrackMap(p. 494)[]
    for id in keys(target_db)
        if AGTOnlyTarget(p. 453)(id)
            for trk in target_db[id].agt_tracks
                track_map::TrackMap(p. 494) = TrackMap(p. 494)(trk, id)
                push!(track_list, track_map)
            end
        end
    end
    for i in 1:length(track_list)
        if !haskey(target_db, track_list[i].db_id)
            continue
        end
        trk = track_list[i].track
        if (abs(trk.trk_summary.mu_range[1] - own.trk_summary.mu_range[1]) <= range_gate)
            targetB_id = track_list[i].db_id
            CorrelateIndividualTracksToOwnship(p. 454)(trk, targetB_id, T)
        end
    end
    return
end
```

Referenced In: CorrelationProcessing(p. 111)

Algorithm 438 AGTOnlyTarget

```
function AGTOnlyTarget(target_id::UInt32)
    tgt::Target(p. 493) = target_db[target_id]
    return TrackExists(p. 19)(tgt.agt_tracks) && !TrackExists(p. 19)(tgt.adsb_track) && !TrackExists(p. 19)(tgt.v2v_track
        ) && !TrackExists(p. 19)(tgt.ornct_track)
end
```

Referenced In: CorrelateTargetsToOwnship(p. 452)

CORRELATEINDIVIDUALTRACKSTOOWNSHIP (Algorithm 439) takes as input a single track and the ID of the associated TARGET and merges the specified track into ownship if they spatially correlate.

The track is evaluated for correlation using CORRELATETOOWNSHIPBYTYPE (Algorithm 440). If CORRELATETOOWNSHIPBYTYPE returns *CORRELATION_POSITIVE*, the track and ownship correlate on the basis of matching unique identifiers, and the AGTTRACKFILE should be immediately matched. Alternatively, if CORRELATEBYTYPE returns *CORRELATION_SPATIAL_POSITIVE*, the track spatially correlate with ownship, and an M-of-N check will ensue to determine if the AGTTRACKFILE should be merged during this cycle. Specifically, CORRELATIONHISTORYUPDATE (Algorithm 443) is called to log the successful spatial correlation, and CORRELATIONHISTORYCHECK (Algorithm 444) then determines if the M-of-N check has been fulfilled, with M and N defined in the parameter file. If such is the case, the *status* is upgraded to *CORRELATION_POSITIVE*, which will allow the merging of the AGTTRACKFILE to proceed.

Prior to merging, an maximum-range gate is checked to prevent ownship from merging with tracks that have previously been observed to be at a significant distance from ownship. This is achieved by maintaining a vector that stores a predetermined number of largest range observations for a given track within each AGTTRACKFILE. If the minimum of these observations exceeds the threshold *max_range_threshold*, the track is disqualified from correlation with ownship for the duration of its existence in STM.

If the maximum range gating passes, the AGTTRACKFILE is merged into ownship. Should the TARGET become empty following the merge, as indicated by TARGETISEMPTY returning TRUE, the TARGET is removed from the *target_db*.

The algorithm may modify *target_db* but generates no return values.

Algorithm 439 CorrelateIndividualTracksToOwnship

```
function CorrelateIndividualTracksToOwnship(track::AGTTrackFile(p. 486), targetB_id::UInt32, T::R)
    M::Z = params().surveillance.correlation.ownship_correlation.M
    N::Z = params().surveillance.correlation.ownship_correlation.N
    unique_time::Z = params().surveillance.correlation.ownship_correlation.unique_time
    unique_multiplier::R = params().surveillance.correlation.ownship_correlation.unique_multiplier
    max_range_thresh::R = params().surveillance.correlation.ownship_correlation.max_range
    history::CorrelationHistory(p. 489) = own.own_corr_history
    (history.M, history.N) = (M, N)
    if ( T - target_db[targetB_id].init_time > unique_time )
        unique_scale = unique_multiplier
    else
        unique_scale = 1.0
    end
    status::UInt8 = CorrelateToOwnshipByType(p. 455)(track, unique_scale)
    if (status == CORRELATION_SPATIAL_POSITIVE)
        own_track_id::String = "OwnWGS84"
        track_id::String = track.track_uid
        CorrelationHistoryUpdate(p. 458)(own_track_id, track_id, history, T)
        if CorrelationHistoryCheck(p. 458)(own_track_id, track_id, history)
            status = CORRELATION_POSITIVE
        end
    end
    if (status == CORRELATION_POSITIVE) && (minimum(track.max_range_vec) < max_range_thresh)
        target = target_db[targetB_id]
        push!(own.agt_tracks, track)
        for idx in 1:length(target.agt_tracks)
            if (target.agt_tracks[idx].agt_id == track.agt_id)
                deleteat!(target.agt_tracks, idx)
                break
            end
        end
        if TargetIsEmpty(p. 161)(target)
            delete!(target_db, targetB_id)
        end
    end
    return
end
```

Referenced In: CorrelateTargetsToOwnship(p. 452)

CORRELATETOOWNSHIPBYTYPE (Algorithm 440) evaluates the correlation between ownship and an AGTTRACKFILE, *trk*.

If *trk* possesses a valid V2V-UID but not a valid Mode S address, an ID correlation is performed to determine if the V2V-UID matches that of ownship. If it does, a conclusive positive correlation status, *CORRELATION_POSITIVE*, is returned.

If *trk* possesses neither valid V2V-UID nor Mode S address, a spatial correlation evaluation is performed between OWNSHIPDATA and the AGTTRACKFILE. The correlation status designation, *status*, is set according to the outcome of that evaluation.

In all other cases, a conclusive negative correlation status, *CORRELATION_NEGATIVE*, is returned to forbid the merging of *trk* into OWNSHIPDATA.

The algorithm takes as input *trk* and *unique_scale*. The algorithm returns a correlation status designation, *status*, which will be one of the *CORRELATION_** constants.

Algorithm 440 CorrelateToOwnshipByType

```
function CorrelateToOwnshipByType(trk::AGTTrackFile(p. 486), unique_scale::R)
    ownship_correlation_parameters = params().surveillance.correlation.type_selection.ownship_position
    if (trk.trk_summary.type == CORRELATION_TRKTYPE_AGT_AGTID_V2VUID) && (trk.v2v_uid == own.v2v_uid)
        status::UInt8 = CORRELATION_POSITIVE
    elseif (trk.trk_summary.type == CORRELATION_TRKTYPE_AGT_AGTID)
        if CorrelatePosition(p. 471)(own, trk, unique_scale, ownship_correlation_parameters)
            status = CORRELATION_SPATIAL_POSITIVE
        else
            status = CORRELATION_SPATIAL_NEGATIVE
        end
    else
        status = CORRELATION_NEGATIVE
    end
    return status::UInt8
end
```

Referenced In: CorrelateIndividualTracksToOwnship(p. 454), DecorrelateTargetsFromOwnship(p. 451)

B.4 Track Decorrelation

DECORRELATETARGETS (Algorithm 441) iterates through all *lead_track*s in the list returned by GENERATELEADTRACKLIST. If the difference between the time T and the initialization time of the TARGET is greater than the parameter *unique_time*, then a unique scale factor will be set to the parameter *unique_multiplier*. The *unique_multiplier* is used to adjust the correlation windows for tracks that have existed longer than a specified amount of time, reaching a stage known as track maturity. During track maturity, it is harder for correlated tracks to decorrelate from one another, and harder for decorrelated tracks to correlate to one another.

A list of tracks eligible for decorrelation within a given TARGET is obtained through a call to GETTRACKSTODECORRELATE (Algorithm 442), and they are individually evaluated for correlation with the *lead_track* using CORRELATEBYTYPE.

During evaluation, the algorithm skips correlation for the tracks whose *track_id*s match *lead_track_id*. Furthermore, if either *dt* or *dt_lead* is greater than *max_coast_time*, decorrelation is also skipped to ensure that two tracks do not decorrelate if either track is coasting for longer than the specified length of time. Note that, for ORNCT tracks, *dt* and *dt_lead* are computed from *toa_update* to facilitate decorrelation during an FOV coast.

If CORRELATEBYTYPE returns *CORRELATION_NEGATIVE*, the two tracks have decorrelated on the basis of mismatched unique identifiers, and the TRACKFILE should immediately be made a separate TARGET. Alternatively, if CORRELATEBYTYPE returns *CORRELATION_SPATIAL_NEGATIVE*, the two tracks have spatially decorrelated, and an M-of-N check will ensue to determine if the TRACKFILE should be separated during this cycle, with M and N defined in the parameter file. Specifically, CORRELATIONHISTORYUPDATE (Algorithm 443) is called to log the spatial decorrelation, and CORRELATIONHISTORYCHECK (Algorithm 444) then determines if the M-of-N check has been fulfilled. If such is the case, the *status* is changed to *CORRELATION_NEGATIVE*, which will allow the splitting of the TRACKFILE to proceed.

When a track is being decorrelated, it is added to a new TARGET using ADDDECORRELATEDTRACKTOTARGET (Algorithm 445) and removed from its current TARGET using REMOVEDECORRELATEDTRACKFROMTARGET (Algorithm 446). The new TARGET is added to the Target database

via a call to ADDTODB. Through this process, new TARGETS are formed for all TRACKFILES within a TARGET that have diverged from the *lead_track* thereof.

It is worth noting that a newly-formed TARGET will record its initialization time *init_time* as the *toa* of the track subtracted by *unique_time*. This guarantees that the tracks within the new TARGET instantly reach track maturity, regardless of the actual length of their existence.

DECORRELATETARGETS takes as input *T*. This algorithm calls ADDTODB that modifies the Target database.

GETTRACKSTODECORRELATE takes as input *target_id*. This algorithm returns *tracks*.

CORRELATIONHISTORYUPDATE takes as input *lead_track_id*, *track_id*, *history* and *T*. This algorithm updates the correlation history by removing the *history.id_pairs* that are older than N and committing new ID pairs of correlating tracks to memory, along with the current time *T*.

CORRELATIONHISTORYCHECK takes as input *lead_track_id*, *track_id*, and *history*. This algorithm determines *count*, the total number of records of correlation in history, by iterating through *history* and counting the number of times *lead_track_id* and *track_id* have been correlated. This algorithm returns a Boolean flag indicating whether the M-of-N check is satisfied by having *count* surpassing *M*.

ADDDECORRELATEDTRACKTOTARGET takes as input *tgt* and *trk*. This algorithm adds the track to the TARGET.

REMOVEDECORRELATEDTRACKFROMTARGET takes as input *tgt* and *trk*. This algorithm removes the track from the TARGET.

Algorithm 441 DecorrelateTargets

```
function DecorrelateTargets(T::R)
    max_coast_time::Z = params().surveillance.decorrelation.max_coast_time
    M::Z = params().surveillance.decorrelation.M
    N::Z = params().surveillance.decorrelation.N
    unique_time::Z = params().surveillance.decorrelation.unique_time
    unique_multiplier::R = params().surveillance.decorrelation.unique_multiplier
    history::CorrelationHistory(p. 489) = own.decorr_history
    (history.M, history.N) = (M, N)
    decorrelated_targets::Vector{Target(p. 493)} = Target(p. 493)[]
    lead_track_list::Vector{TrackMap(p. 494)} = GenerateLeadTrackList(p. 460)()
    for lead_track_mapping in lead_track_list
        lead_track = lead_track_mapping.track
        tgt_id = lead_track_mapping.db_id
        lead_track_id::String = lead_track.track_uid
        if ( T - target_db[tgt_id].init_time > unique_time)
            unique_scale = unique_multiplier
        else
            unique_scale = 1.0
        end
        tracks = GetTracksToDecorrelate(p. 458)(target_db[tgt_id])
        removed_tracks = TrackFile(p. 483)[]
        for trk in tracks
            track_id::String = trk.track_uid
            dt::R = T - trk.toa
            dt_lead::R = T - lead_track.toa
            if (typeof(trk) == ORNCTTrackFile(p. 484))
                dt = T - trk.toa_update
            end
            if (typeof(lead_track) == ORNCTTrackFile(p. 484))
                dt_lead = T - lead_track.toa_update
            end
            if (lead_track_id == track_id) || (dt > max_coast_time) || (dt_lead > max_coast_time)
                continue
            end
            status::UInt8 = CorrelateByType(p. 466)(tgt_id, lead_track, tgt_id, trk, unique_scale)
            if (status == CORRELATION_SPATIAL_NEGATIVE)
                CorrelationHistoryUpdate(p. 458)(lead_track_id, track_id, history, T)
                if (CorrelationHistoryCheck(p. 458)(lead_track_id, track_id, history))
                    status = CORRELATION_NEGATIVE
                end
            end
            if (status == CORRELATION_NEGATIVE)
                tgt = Target(p. 493)()
                tgt.init_time = trk.toa - unique_time
                AddDecorrelatedTrackToTarget(p. 459)(tgt, trk)
                push!(decorrelated_targets, tgt)
                push!(removed_tracks, trk)
            end
        end
        for rm_trk in removed_tracks
            RemoveDecorrelatedTrackFromTarget(p. 459)(target_db[tgt_id], rm_trk)
        end
    end
    for dtgt in decorrelated_targets
        AddToDB(p. 19)(target_db,dtgt)
    end
end
```

Referenced In: CorrelationProcessing(p. 111)

Algorithm 442 GetTracksToDecorrelate

```
function GetTracksToDecorrelate(target::Target(p. 493))
    tracks::Vector{TrackFile(p. 483)} = GetAircraftTracks(p. 440)(target)
    return tracks::Vector{TrackFile(p. 483)}
end
```

Referenced In: DecorrelateTargets(p. 457)

Algorithm 443 CorrelationHistoryUpdate

```
function CorrelationHistoryUpdate(lead_track_id::Union{UInt32, String}, track_id::Union{UInt32, String},
    history::CorrelationHistory(p. 489), T::R)
    h1 = history.id_pairs
    h2 = history.timing
    v  = [lead_track_id, track_id]
    if (!isempty(h2))
        while T - h2[end] >= history.N
            pop!(h2)
            pop!(h1)
            if (isempty(h2))
                break
            end
        end
    end
    pushfirst!(h1, v)
    pushfirst!(h2, T)
end
```

Referenced In: CorrelateIndividualTracksToOwnship(p. 454), CorrelateIndividualTracks(p. 462), DecorrelateTargets-FromOwnship(p. 451), DecorrelateTargets(p. 457)

Algorithm 444 CorrelationHistoryCheck

```
function CorrelationHistoryCheck(lead_track_id::Union{UInt32, String}, track_id::Union{UInt32, String}, history
    ::CorrelationHistory(p. 489))
    count::Z = 0
    h1 = history.id_pairs
    v = [lead_track_id, track_id]
    for k in 1:length(h1)
        if (v == h1[k])
            count += 1
        end
    end
    return (count >= history.M)::Bool
end
```

Referenced In: CorrelateIndividualTracksToOwnship(p. 454), CorrelateIndividualTracks(p. 462), DecorrelateTargets-FromOwnship(p. 451), DecorrelateTargets(p. 457)

Algorithm 445 AddDecorrelatedTrackToTarget

```
function AddDecorrelatedTrackToTarget(tgt::Target(p. 493), trk::TrackFile(p. 483))
    if (typeof(trk) == V2VTrackFile(p. 488))
        tgt.v2v_track = trk
    elseif (typeof(trk) == ORNCTTrackFile(p. 484))
        tgt.ornct_track = trk
    elseif (typeof(trk) == AGTTrackFile(p. 486))
        push!(tgt.agt_tracks, trk)
    end
end
```

Referenced In: DecorrelateTargetsFromOwnship(p. 451), DecorrelateTargets(p. 457)

Algorithm 446 RemoveDecorrelatedTrackFromTarget

```
function RemoveDecorrelatedTrackFromTarget(tgt::Target(p. 493), trk::TrackFile(p. 483))
    if (trk.track_uid == tgt.previously_selected_trk_uid)
        tgt.previously_selected_trk_uid = nothing
    end
    tgt.optimal_trk_history = String[]
    if (TrackExists(p. 19)(tgt.v2v_track)) && (tgt.v2v_track == trk)
        tgt.v2v_track = nothing
        return
    end
    if (TrackExists(p. 19)(tgt.ornct_track)) && (tgt.ornct_track == trk)
        tgt.ornct_track = nothing
        return
    end
    for i in 1:length(tgt.agt_tracks)
        if (tgt.agt_tracks[i] == trk)
            deleteat!(tgt.agt_tracks, i)
            return
        end
    end
end
```

Referenced In: DecorrelateTargets(p. 457)

GENERATELEADTRACKLIST (Algorithm 447) returns a list of preferred tracks, one for each TARGET in the Target Database. A lead track will never be decorrelated from its current TARGET, and is the track used for correlation evaluation during CORRELATETARGETS (Algorithm 448). ADS-B tracks are the preferred correlation track type, followed by V2V, ORNCT and AGT tracks.

This algorithm takes no input. This algorithm returns *lead_track_list*.

Algorithm 447 GenerateLeadTrackList

```
function GenerateLeadTrackList()
    lead_track_list::Vector{TrackMap(p. 494)} = TrackMap(p. 494)[]
    for id in keys(target_db)
        lead_track::Union{TrackFile(p. 483), Nothing} = nothing
        target = target_db[id]
        if (TrackExists(p. 19)(target.adsb_track))
            lead_track = target.adsb_track
        elseif (TrackExists(p. 19)(target.v2v_track))
            lead_track = target.v2v_track
        elseif (TrackExists(p. 19)(target.ornct_track))
            lead_track = target.ornct_track
        elseif (TrackExists(p. 19)(target.agt_tracks))
            lead_track = target.agt_tracks[1]
        end
        if (TrackExists(p. 19)(lead_track))
            track_map::TrackMap(p. 494) = TrackMap(p. 494)(lead_track, id)
            push!(lead_track_list, track_map)
        end
    end
    return lead_track_list::Vector{TrackMap(p. 494)}
end
```

Referenced In: DecorrelateTargets(p. 457), CorrelateTargets(p. 461)

B.5 Track Correlation

CORRELATETARGETS (Algorithm 448) iterates through all TARGETs in memory and merges those with correlating lead tracks into a single TARGET. To accomplish that, a list of lead tracks, *lead_track_list*, is first generated using GENERATELEADTRACKLIST. The list of lead tracks is then sorted by range through a call to SORTBYRANGE (Algorithm 449), which retrieves the ranges from the TRACKSUMMARY instance within each TARGET.

The algorithm iterates through the *lead_track_list* and evaluates every combination of Target pairs for correlation, providing that

1. The lead track of neither TARGETs has been flagged as deleted (i.e., the corresponding entry in the Boolean vector *pending_deletion* is not TRUE; and
2. The difference in range between the two tracks is no larger than a specified gating value, *range_gate*. The gating is implemented to reduce the number of evaluations and improve efficiency.

The evaluation itself begins with passing the TRACKFILEs and Target IDs to CORRELATEINDIVIDUALTRACKS (Algorithm 450). If TRUE is returned to *pending_deletion[indexA]* or *pending_deletion[indexB]*, the corresponding lead track has been merged during the call to CORRELATEINDIVIDUALTRACKS and is hence flagged as deleted. Flagging tracks in *pending_deletion* prevents TARGETs already merged from being further evaluated for correlation.

CORRELATETARGETS takes as input *T*. The algorithm modifies the Target database.

SORTBYRANGE takes as input *list*. This algorithm returns *list* sorted by range.

Algorithm 448 CorrelateTargets

```
function CorrelateTargets(T::R)
    range_gate::Z = params().surveillance.correlation.range_gate
    lead_track_list::Vector{TrackMap(p. 494)} = GenerateLeadTrackList(p. 460)()
    lead_track_list = SortByRange(p. 461)(lead_track_list)
    pending_deletion::Vector{Bool} = fill(false, length(lead_track_list))
    lowIndexB::Z = 1
    for indexA in 1:length(lead_track_list)
        if (!pending_deletion[indexA])
            trackA = lead_track_list[indexA].track
            for indexB in lowIndexB:length(lead_track_list)
                if (!pending_deletion[indexB]) && (indexA != indexB)
                    trackB = lead_track_list[indexB].track
                    (targetA_id, targetB_id) = (lead_track_list[indexA].db_id, lead_track_list[indexB].db_id
                        )
                    if (trackA.trk_summary.mu_range[1] - trackB.trk_summary.mu_range[1] > range_gate)
                        lowIndexB = indexB
                        continue
                    elseif (trackB.trk_summary.mu_range[1] - trackA.trk_summary.mu_range[1] <= range_gate)
                        (pending_deletion[indexA], pending_deletion[indexB]) = CorrelateIndividualTracks(p. 462)(
                            trackA, targetA_id, trackB, targetB_id, T)
                        if (pending_deletion[indexA])
                            break
                        end
                    end
                end
            end
        end
    end
end
```

Referenced In: CorrelationProcessing(p. 111)

Algorithm 449 SortByRange

```
function SortByRange(list::Vector{TrackMap(p. 494)})
    CompareRange(a,b) = a.track.trk_summary.mu_range[1] < b.track.trk_summary.mu_range[1]
    return sort(list, lt = CompareRange)
end
```

Referenced In: CorrelateTargets(p. 461)

CORRELATEINDIVIDUALTRACKS (Algorithm 450) takes as input two tracks, their associated Target IDs, and the current time, and merges the two TARGETs together if they correlate.

If the two tracks have both reached maturity (i.e., have been maintained for a prolonged period of time, the length of which is specified by the parameter *unique_time*), the correlation criterion will be made more restrictive, thus rendering the merger of the two tracks less likely.

The tracks are evaluated for correlation using CORRELATEBYTYPE (Algorithm 452). If CORRELATEBYTYPE returns *CORRELATION_POSITIVE*, the two tracks correlate on the basis of matching unique identifiers, and the two TARGETs should be immediately matched. Alternatively, if CORRELATEBYTYPE returns *CORRELATION_SPATIAL_POSITIVE*, the two tracks spatially correlate, and an M-of-N check will ensue to determine if the two TARGETs should be merged during this cycle, with M and N defined in the parameter file. Specifically, CORRELATIONHISTORYUP-

DATE (Algorithm 443) is called to log the successful spatial correlation, and CORRELATIONHISTORYCHECK (Algorithm 444) then determines if the M-of-N check has been fulfilled. If such is the case, the *status* is upgraded to *CORRELATION_POSITIVE*, which will allow the merging of the TARGETs to proceed.

When two TARGETs are undergoing merging, the TRACKFILEs of the more recently initialized target are merged into the older target through a call to MERGETARGETS (Algorithm 451). Two Boolean flags respectively indicating whether *trackA* and *trackB* have been deleted during the call to MERGETARGETS are returned to the calling function.

This algorithm takes as input *trackA*, *targetA_id*, *trackB*, *targetB_id*, and *T*. This algorithm returns *delete_trackA* and *delete_trackB*.

Algorithm 450 CorrelateIndividualTracks

```
function CorrelateIndividualTracks(trackA::TrackFile(p. 483), targetA_id::UInt32, trackB::TrackFile(p. 483), targetB_id::
    UInt32, T::R)
    M::Z = params().surveillance.correlation.M
    N::Z = params().surveillance.correlation.N
    unique_time::Z = params().surveillance.correlation.unique_time
    unique_multiplier::R = params().surveillance.correlation.unique_multiplier
    (target_id::UInt32, unique_scale::R) = (0, 1.0)
    (targetA, targetB) = (target_db[targetA_id], target_db[targetB_id])
    if ( T - max( targetA.init_time, targetB.init_time) > unique_time )
        unique_scale = unique_multiplier
    end
    history::CorrelationHistory(p. 489) = own.corr_history
    (history.M, history.N) = (M, N)
    delete_trackA = delete_trackB = false
    status::UInt8 = CorrelateByType(p. 466)(targetA_id, trackA, targetB_id, trackB, unique_scale)
    if (status == CORRELATION_SPATIAL_POSITIVE)
        CorrelationHistoryUpdate(p. 458)(targetA_id, targetB_id, history, T)
        if (CorrelationHistoryCheck(p. 458)(targetA_id, targetB_id, history))
            status = CORRELATION_POSITIVE
        end
    end
    if (status == CORRELATION_POSITIVE)
        if (targetB.init_time < targetA.init_time)
            (target_id, deleted_target_id) = (targetB_id, targetA_id)
            delete_trackA = true
        else
            (target_id, deleted_target_id) = (targetA_id, targetB_id)
            delete_trackB = true
        end
        MergeTargets(p. 464)(target_id, deleted_target_id)
    end
    return (delete_trackA::Bool, delete_trackB::Bool)
end
```

Referenced In: CorrelateTargets(p. 461)

MERGETARGETS (Algorithm 451) merges two TARGETs together during correlation evaluation.

If the TARGET to be deleted contains an ADSBTRACKFILE, and the TARGET to be merged does not, the ADSBTRACKFILE and the ADS-B quality override flag are copied over to the merging TARGET. The ADSBTRACKFILE of the TARGET to be deleted is subsequently removed.

If the TARGET to be deleted contains a V2VTRACKFILE, and the TARGET to be merged does not, the V2VTRACKFILE is copied over to the merging TARGET. The V2VTRACKFILE of the TARGET to

be deleted is subsequently removed.

If the TARGET to be deleted contains an ORNCTTRACKFILE and the TARGET to be merged does not, the ORNCTTRACKFILE is copied over to the merging TARGET, and the ORNCTTRACKFILE of the TARGET to be deleted is subsequently removed.

If the TARGET to be deleted contains one or more AGTTRACKFILEs and that the TARGET to be merged contains none, the vector storing the AGTTRACKFILEs in the TARGET to be deleted is copied to the merging TARGET. Alternatively, if the TARGET to be merged already contains one or more AGTTRACKFILEs, the AGTTRACKFILEs from the TARGET to be deleted is appended to the end of the vector holding the AGTTRACKFILEs in the TARGET to be merged.

Next, the most up-to-date vertical and horizontal coordination data between the two TARGETs will be saved.

Then, two flags *target_to_merge_has_code* and *target_to_delete_has_code* are set to indicate if each of the TARGETs has an active RA. The *is_deleted_prev_sel_trk_in_merged* flag is initialized to false. Next, the list of Track Files for the *target_to_merge* is iterated through and the *is_deleted_-prev_sel_trk_in_merged* flag is set to true if the previously selected track is found. The conditional statement then assesses various conditions to handle the previously selected track field for the TARGET. If the TARGET to be deleted has an active RA but the TARGET to be merged does not and the *is_deleted_prev_sel_trk_in_merged*, the previously selected track is copied over to the merging TARGET. Otherwise, if the TARGET to be merged does not have an active RA, the previously selected track is reset to nothing. The Optimal Track History of the TARGET to be merged is cleared every time TARGETs are merged.

Finally, the TARGET to be deleted is removed from *target_db*.

This algorithm takes as input *id_to_merge* and *id_to_delete*. This algorithm modifies the Target database, *target_db*.

Algorithm 451 MergeTargets

```
function MergeTargets(id_to_merge::UInt32, id_to_delete::UInt32)
    global target_db
    target_to_merge = target_db[id_to_merge]
    target_to_delete = target_db[id_to_delete]
    if (TrackExists(p. 19)(target_to_delete.ornct_track)) && ( (!TrackExists(p. 19)(target_to_merge.ornct_track)) ||
         (target_to_merge.ornct_track.is_FOV_coast) )
        target_to_merge.ornct_track = target_to_delete.ornct_track
    end
    target_to_delete.ornct_track = nothing
    if (TrackExists(p. 19)(target_to_delete.adsb_track)) && (!TrackExists(p. 19)(target_to_merge.adsb_track))
        target_to_merge.adsb_track = target_to_delete.adsb_track
        target_to_merge.adsb_track.passive_qual_history = target_to_delete.adsb_track.passive_qual_history
        target_to_merge.adsb_qual_override = target_to_delete.adsb_qual_override
    end
    target_to_delete.adsb_track = nothing
    if (TrackExists(p. 19)(target_to_delete.v2v_track)) && (!TrackExists(p. 19)(target_to_merge.v2v_track))
        target_to_merge.v2v_track = target_to_delete.v2v_track
        target_to_merge.v2v_track.passive_qual_history = target_to_delete.v2v_track.passive_qual_history
    end
    target_to_delete.v2v_track = nothing
    if (TrackExists(p. 19)(target_to_delete.agt_tracks)) && (!TrackExists(p. 19)(target_to_merge.agt_tracks))
        target_to_merge.agt_tracks = target_to_delete.agt_tracks
    elseif (TrackExists(p. 19)(target_to_delete.agt_tracks)) && (TrackExists(p. 19)(target_to_merge.agt_tracks))
        append!(target_to_merge.agt_tracks,target_to_delete.agt_tracks)
    end
    target_to_delete.agt_tracks = Vector{AGTTrackFile(p. 486)}(undef, 0)
    if (target_to_delete.coord_data.toa_vert > target_to_merge.coord_data.toa_vert)
        target_to_merge.coord_data.toa_vert = target_to_delete.coord_data.toa_vert
        target_to_merge.coord_data.vrc = target_to_delete.coord_data.vrc
        target_to_merge.bad_v2vcoordination_vert = target_to_delete.bad_v2vcoordination_vert
    end
    if (target_to_delete.coord_data.toa_hor > target_to_merge.coord_data.toa_hor)
        target_to_merge.coord_data.toa_hor = target_to_delete.coord_data.toa_hor
        target_to_merge.coord_data.hrc = target_to_delete.coord_data.hrc
        target_to_merge.bad_v2vcoordination_horz = target_to_delete.bad_v2vcoordination_horz
    end
    target_to_merge_has_code::Bool = (target_to_merge.priority_codes.vert > SXUCODE_CLEAR) || (
         target_to_merge.priority_codes.horiz > SXUCODE_CLEAR)
    target_to_delete_has_code::Bool = (target_to_delete.priority_codes.vert > SXUCODE_CLEAR) || (
         target_to_delete.priority_codes.horiz > SXUCODE_CLEAR)
    is_deleted_prev_sel_trk_in_merged::Bool = false
    for trk in GetAircraftTracks(p. 440)(target_to_merge)
        if (target_to_delete.previously_selected_trk_uid == trk.track_uid)
            is_deleted_prev_sel_trk_in_merged = true
            break
        end
    end
    if target_to_delete_has_code && !target_to_merge_has_code && is_deleted_prev_sel_trk_in_merged
        target_to_merge.previously_selected_trk_uid = target_to_delete.previously_selected_trk_uid
    elseif !target_to_merge_has_code
        target_to_merge.previously_selected_trk_uid = nothing
    end
    target_to_merge.optimal_trk_history = String[]
    delete!(target_db, id_to_delete)
    return
end
```

Referenced In: CorrelateIndividualTracks(p. 462)

CORRELATEBYTYPE (Algorithm 452) takes as input two TARGET IDs, *lead_target_id* and *target_id*, one TRACKFILE apiece from the two TARGETs identified by those IDs, *lead_trk* and *trk*, and a multiplicative factor *unique_scale* used to adjust the threshold for spatial correlation.

It is worth pointing out that the subject of evaluation in this algorithm depends on whether CORRELATEBYTYPE has been called as part of the correlation or the decorrelation cycle. Namely, the goal of the algorithm may be either:

- to determine whether the two TARGETS should be merged (i.e. when called by CORRELATEINDIVIDUALTRACKS); or
- to determine whether *trk* should be made its own TARGET (when called by DECORRELATETARGETS), in which case *lead_target_id* and *target_id* will be the same, both being the hosting TARGET of *trk*.

With that understanding, CORRELATEID (Algorithm 453) is first called with three possible outcomes:

- *CORRELATION_POSITIVE*, indicating that correlation has been conclusively established between the two TARGETS (correlation cycle) or between *trk* and the rest of the TRACKFILES in the hosting TARGET (decorrelation cycle);
- *CORRELATION_NEGATIVE*, indicating that correlation has been conclusively disproved between the two TARGETS or between *trk* and the rest of the TRACKFILES in the hosting TARGET; or
- *CORRELATION_PENDING*, indicating that no conclusion is drawn in CORRELATEID and spatial correlation should be performed between *lead_trk* and *trk*.

Spatial correlation evaluation, if necessitated by a *CORRELATION_PENDING* status code, begins with a call to GETCORRELATIONMECHANISM (Algorithm 454), which returns either *CORRELATION_MECH_S_SPATIAL* or *CORRELATION_MECH_S_STAR*. Note that CORRELATEID is such constructed that it precludes the possibility of other correlation mechanisms being authorized between the two TRACKFILES.

Per the definition in [3], an S* correlation (*CORRELATION_MECH_S_STAR*) is a special case of spatial correlation that is authorized to cases where one of the two tracks is an ADSBTRACKFILE identified with an ICAO Mode S address. If an S* correlation is authorized, an additional check is performed to ensure that the ADS-B track in question has originated from a 1090ES transponder. If it is determined that the ADS-B track has originated from an UAT transmitter, the *mechanism* is changed to *CORRELATION_MECH_NOT_AUTHORIZED*, which will result in the *status* being changed to *CORRELATION_NEGATIVE*.

If it is deemed that spatial correlation may proceed, a call is made to CORRELATEPOSITION (Algorithm 455), which returns a Boolean flag indicating whether *lead_trk* and *trk* are spatially correlated. *status* is then set accordingly to either *CORRELATION_SPATIAL_POSITIVE* or *CORRELATION_SPATIAL_NEGATIVE*.

The algorithm returns a correlation status designation, *status*, which will be one of the *CORRELATION_** constants.

Algorithm 452 CorrelateByType

```
function CorrelateByType(lead_target_id::UInt32, lead_trk::TrackFile(p. 483), target_id::UInt32, trk::
    TrackFile(p. 483), unique_scale::R)
    correlation_parameters = params().surveillance.correlation.type_selection.position
    status::UInt8 = CorrelateID(p. 468)(lead_target_id, target_id, trk)
    if (status == CORRELATION_PENDING)
        mechanism::UInt8 = GetCorrelationMechanism(p. 469)(lead_trk, trk)
        if (mechanism == CORRELATION_MECH_S_STAR) &&
            ( ((lead_trk.trk_summary.type == CORRELATION_TRKTYPE_ADSB_ICAO24) && lead_trk.uat) ||
              ((trk.trk_summary.type == CORRELATION_TRKTYPE_ADSB_ICAO24) && trk.uat) )
            mechanism = CORRELATION_MECH_NOT_AUTHORIZED
        end
        if (mechanism in [CORRELATION_MECH_SPATIAL, CORRELATION_MECH_S_STAR])
            if CorrelatePosition(p. 471)(lead_trk, trk, unique_scale, correlation_parameters)
                status = CORRELATION_SPATIAL_POSITIVE
            else
                status = CORRELATION_SPATIAL_NEGATIVE
            end
        else
            status = CORRELATION_NEGATIVE
        end
    end
    return status::UInt8
end
```

Referenced In: CorrelateIndividualTracks(p. 462), DecorrelateTargets(p. 457)

CORRELATEID (Algorithm 453) takes in two TARGET IDs, *lead_target_id* and *target_id*, and a single TRACKFILE, *trk*.

The algorithm first constructs a vector of TRACKFILE tuples, *trk_pairs*, which consists of track pairs that are to be evaluated in this algorithm.

If *lead_target_id* and *target_id* are not identical, indicating that the correlation algorithm is in the correlation cycle, *trk_pairs* will consist of all possible TRACKFILE pairs between the two distinctive TARGETs. The TRACKFILEs are extracted by calling GETAIRCRAFTTRACKS. On the other hand, if *lead_target_id* and variabletarget_id match, implying that the correlation algorithm is in the decorrelation cycle, the *trk_pairs* will instead consist of the pairs formed by *trk* and each airborne TRACKFILEs in the TARGET that is not *trk*. A TRACKFILE is airborne if it is used to track an intruder aircraft i.e. not a POINTOBSTACLETRACKFILE.

CORRELATEID then iterates through each tuple in *trk_pairs* and obtains the correlation mechanism prescribed for the pair by calling GETCORRELATIONMECHANISM (Algorithm 454). The mechanism will fall into one of the following three categories:

- Correlation Not Authorized (*CORRELATION_MECH_NOT_AUTHORIZED*) indicates that the two TRACKFILEs are forbidden to exist in the same TARGET. The Boolean flag *id_correlation_attempted* is set to TRUE and *is_correlated* is set to FALSE.
- Spatial Correlation (*CORRELATION_MECH_SPATIAL*, *CORRELATION_MECH_S_STAR*) indicates that the two TRACKFILEs require evaluation on their spatial proximity to establish correlation or the lack thereof. Hence, no processing is done in CORRELATEID.
- ID-based correlation (all other *CORRELATION_MECH_** constants) indicates that ID correlation is to be performed based on the specified mechanism. *id_correlation_attempted* is set to TRUE and *is_correlated* is set based on the outcome of the ID correlation.

At the end of each iteration, if ID correlation has been attempted during that iteration or the previous ones, the correlation status designation, *status*, is updated based on the value of *is_correlated*. Furthermore, if *status* is *CORRELATION_NEGATIVE* at the end of an iteration, the iteration is discontinued and CORRELATEID immediately returns the negative evaluation. This is due to the fact that if an ID correlation between any pair of TRACKFILES indicates that they are uncorrelated, the ID spaces of the two targets are incompatible and should never be merged (correlation cycle), or that *trk* is in an ID space that is no longer compatible with the rest of the TRACKFILES in the hosting TARGET. Hence, the algorithm returns a conclusive *CORRELATION_NEGATIVE* to convey the same and to prevent further spatial correlation from occurring in CORRELATEBYTYPE.

The algorithm returns a correlation status designation, *status*, which will be one of the *CORRELATION_** constants.

Algorithm 453 CorrelateID

```
function CorrelateID(lead_target_id::UInt32, target_id::UInt32, trk::TrackFile(p. 483))
    trk_pairs = Tuple{TrackFile(p. 483),TrackFile(p. 483)}[]
    lead_tgt_tracks::Vector{TrackFile(p. 483)} = GetAircraftTracks(p. 440)(target_db[lead_target_id])
    if (lead_target_id != target_id)
        tgt_tracks::Vector{TrackFile(p. 483)} = GetAircraftTracks(p. 440)(target_db[target_id])
        for lead_tgt_trk in lead_tgt_tracks
            for tgt_trk in tgt_tracks
                push!(trk_pairs, (lead_tgt_trk, tgt_trk))
            end
        end
    else
        for lead_tgt_trk in lead_tgt_tracks
            if (lead_tgt_trk.track_uid != trk.track_uid)
                push!(trk_pairs, (lead_tgt_trk, trk))
            end
        end
    end
    status::UInt8 = CORRELATION_PENDING
    id_correlation_attempted::Bool = false
    is_correlated::Bool = false
    for (lhs_trk,rhs_trk) in trk_pairs
        mechanism::UInt8 = GetCorrelationMechanism(p. 469)(lhs_trk, rhs_trk)
        if (mechanism == CORRELATION_MECH_ICAO24) || (mechanism == CORRELATION_MECH_ANON24)
            id_correlation_attempted = true
            is_correlated = (lhs_trk.mode_s == rhs_trk.mode_s)
        elseif (mechanism == CORRELATION_MECH_V2VUID)
            id_correlation_attempted = true
            is_correlated = (lhs_trk.v2v_uid == rhs_trk.v2v_uid)
        elseif (mechanism == CORRELATION_MECH_ICAO24_V2VUID) || (mechanism == CORRELATION_MECH_ANON24_V2VUID
            )
            id_correlation_attempted = true
            is_correlated = ((lhs_trk.mode_s == rhs_trk.mode_s) && (lhs_trk.v2v_uid == rhs_trk.v2v_uid))
        elseif (mechanism == CORRELATION_MECH_NOT_AUTHORIZED)
            id_correlation_attempted = true
            is_correlated = false
        end
        if id_correlation_attempted
            if is_correlated
                status = CORRELATION_POSITIVE
            else
                status = CORRELATION_NEGATIVE
            end
        end
        if (status == CORRELATION_NEGATIVE)
            break
        end
    end
    return status::UInt8
end
```

Referenced In: CorrelateByType(p. 466)

GETCORRELATIONMECHANISM (Algorithm 454) takes as input two TRACKFILES, *lead_trk* and *trk*, and determines the suitable mechanism to be used to examine if the two tracks correlate.

GETCORRELATIONMECHANISM first extracts the correlation track type designation, previously generated by CORRELATIONTRACKTYPE and stored in *trk_summary.type*. If neither track received a *CORRELATION_TRKTYPE_INVALID* designation, the mechnaism is obtained by looking up the correlation mechnaism table stored in the parameter file. Otherwise, the *mechanism* is set to *CORRELATION_MECH_NOT_AUTHORIZED*.

Note how the rows and columns of the mechanism table directly correspond to the numerical values of the correlation track type designations (i.e. the *CORRELATION_TRKTYPE_** constants).

The algorithm takes *lead_trk* and *trk* as input. The algorithm returns a designation for the correlation mechanism, which will be one of the *CORRELATION_MECH_** constants.

Algorithm 454 GetCorrelationMechanism

```
function GetCorrelationMechanism(lead_trk::TrackFile(p. 483), trk::TrackFile(p. 483))
    correlation_mechanism = params().surveillance.correlation.mechanism
    mechanism::UInt8 = CORRELATION_MECH_NOT_AUTHORIZED
    lead_trk_type::UInt8 = lead_trk.trk_summary.type
    trk_type::UInt8 = trk.trk_summary.type
    if (lead_trk_type != CORRELATION_TRKTYPE_INVALID) && (trk_type != CORRELATION_TRKTYPE_INVALID)
        mechanism = correlation_mechanism[lead_trk_type,trk_type]
    end
    return mechanism::UInt8
end
```

Referenced In: CorrelateByType(p. 466), CorrelateID(p. 468)

CORRELATEPOSITION (Algorithm 455) compares two TRACKFILES, or, in the event of ownship correlation evaluation, the OWNSHIPDATA and a TRACKFILE, and determines whether the two correlate with each other using spatial techniques. For ease of reference, the two entities under evaluation are hereafter referred to as "tracks", regardless of their being the OWNSHIPDATA or a TRACKFILE.

Before any evaluation may proceed, the algorithm first attempts to align the altitude datums of the two tracks. If the altitude datums of the two tracks do not match (e.g., *trackA* is reporting pressure altitude, PA, and *trackB* is reporting geodetic height above ellipsoid, HAE), and if both PA and HAE reports of ownship are being received and valid (i.e., not *NaN*s), the algorithm will apply a correction to the reported altitude of *trackA* so that it uses the same datum as *trackB*. The amount of correction is determined by the difference between reported ownship PA and reported ownship HAE.

To improve efficiency, the algorithm then performs gating on the range and altitude differences between the two tracks. If either value exceeds the specified thresholds, namely *range_gate* and *altitude_gate*, the algorithm immediately evaluates to FALSE (i.e., the tracks are not correlated), and no further computation is necessary.

Should the tracks pass the gatings, the algorithm calculates four probabilistic scores based on how well the two tracks correlate in four dimensions: range (*range_score*), altitude (*altitude_score*), range rate (*rangerate_score*) and bearing (*bearing_score*). Each score is a Mahalanobis distance between the two tracks in the specified dimension, derived through a call to MAHAL. As explored below, the total score, *score*, is obtain by summing some combination of the four, with the combination depending on the validity of measurements in each dimension. The correlation of the tracks is then determined by comparing *score* against a predetermined upper bound, *threshold*, scaled by the incoming *unique_scale*. If *score* does not exceed the scaled *threshold*, the two tracks are declared correlated (i.e., the algorithm returns TRUE), and vice versa.

The **range score** is the Mahalanobis distance of the ground ranges of the two tracks. The range score is always included in the total score.

The **altitude score** is the Mahalanobis distance of the altitudes of the two tracks and is included in the total score, except when one of the following criteria is met:

- Both tracks are NAR, as indicated by the Boolean flag *altitude_comparison*. In this case *altitude_score* is set to *0* and *threshold* is lowered by *correlation_alt_threshold*, effectively removing the altitude component from the total scoring.
- Only one of the two tracks is NAR, as indicated by the Boolean flag *altitude_mismatch*, and at least one of them is
  - An ORNCT track, as indicated by the Boolean flag *is_either_ORNCT*; or
  - An ADS-B track that is using geodetic height above ellipsoid (HAE) as its source of altitude reporting, as indicated by the Boolean flag *is_either_geoalt*.

  In this case *altitude_score* is remains at *0* and *threshold* is lowered by *correlation_alt_threshold*, again removing the altitude component from the total scoring. If none of the three Boolean flags is set to TRUE, however, the altitude score is set to infinity, barring the two tracks from ever being declared correlated.

The **range rate score** is the Mahalanobis distance of the ground range rates of the two tracks. The range rate score is included unless it is replaced by the bearing score.

The **bearing score** is the Mahalanobis distance of the bearings of the two tracks with respect to ownship. The bearing score is computed, as indicated by the Boolean flag *bearing_comparison*, if ownship is receiving nominal heading reports (i.e., *ownship.heading_state* is valid and not degraded). The bearing score replaces the range rate score if it is larger than the range rate score. In addition, if the bearing score does replace the range rate score, and that the correlation threshold, *threshold*, has previously been lowered due to NAR tracks, *threshold* is raised by *correlation_az_threshold* to account for the additional uncertainty introduced in bearing measurements.

This algorithm takes as input *trackA*, *trackB*, and *unique_scale*. This algorithm returns a Boolean flag indicating whether the spatial correlation has been successful.

Algorithm 455 CorrelatePosition

```
function CorrelatePosition(trackA::Union{TrackFile(p. 483),OwnShipData(p. 491)}, trackB::TrackFile(p. 483), unique_scale::R, correlation_params)
    altitude_gate::Z         = correlation_params.altitude_gate
    range_gate::Z            = correlation_params.range_gate
    altitude_inflation::R    = correlation_params.altitude_inflation
    correlation_threshold::R = correlation_params.correlation_threshold
    correlation_alt_threshold::R = correlation_params.correlation_alt_threshold
    correlation_az_threshold::R = correlation_params.correlation_az_threshold
    TS1 = deepcopy(trackA.trk_summary)
    TS2 = deepcopy(trackB.trk_summary)
    own_valid_alt_pres::Bool = !isnan(own.toa_vert)
    own_valid_alt_hae::Bool = !isnan(own.wgs84_toa_vert)
    if (TS1.alt_src_hae != TS2.alt_src_hae)
        if own_valid_alt_pres && own_valid_alt_hae
            own_pres_alt = own.geo_states_pres_alt.lla_rad_ft[3]
            own_hae_alt = own.geo_states_hae_alt.lla_rad_ft[3]
            if TS1.alt_src_hae
                z_rel = TS1.mu_vert[1] - own_hae_alt
                TS1.mu_vert[1] = z_rel + own_pres_alt
            else
                z_rel = TS1.mu_vert[1] - own_pres_alt
                TS1.mu_vert[1] = z_rel + own_hae_alt
            end
        end
    end
    if (abs(TS1.mu_range[1] - TS2.mu_range[1]) > range_gate)
        return false
    elseif (TS1.valid_vert) && (TS2.valid_vert) && (abs(TS1.mu_vert[1] - TS2.mu_vert[1]) > altitude_gate)
        return false
    elseif ((typeof(trackA) == ORNCTTrackFile(p. 484)) || (typeof(trackB) == ORNCTTrackFile(p. 484))) && !own_valid_alt_pres && !own_valid_alt_hae
        return false
    end
    altitude_score::R = score::R = bearing_score::R = 0.0
    (threshold, reject_correlation_score) = (correlation_threshold, Inf)
    altitude_comparison::Bool = (!TS1.valid_vert) && (!TS2.valid_vert)
    altitude_mismatch::Bool =  (!TS1.valid_vert) || (!TS2.valid_vert)
    bearing_comparison::Bool = (own.heading_state == OWN_HEADING_NOMINAL)
    range_score::R     = mahal(p. 581)(TS1.mu_range[1], TS1.Sigma_range[1,1], TS2.mu_range[1], TS2.Sigma_range[1,1])
    rangerate_score::R = mahal(p. 581)(TS1.mu_range[2], TS1.Sigma_range[2,2], TS2.mu_range[2], TS2.Sigma_range[2,2])
    if (altitude_comparison)
            threshold = threshold - correlation_alt_threshold
    elseif (altitude_mismatch)
        is_either_ORNCT::Bool = (typeof(trackA) == ORNCTTrackFile(p. 484)) || (typeof(trackB) == ORNCTTrackFile(p. 484))
        is_either_geoalt::Bool = ((typeof(trackA) == ADSBTrackFile(p. 485)) && trackA.alt_src_hae) || ((typeof(trackB) == ADSBTrackFile(p. 485)) && trackB.alt_src_hae)
        if is_either_ORNCT || is_either_geoalt
            threshold = threshold - correlation_alt_threshold
        else
            altitude_score = reject_correlation_score
        end
    else
        altitude_score = mahal(p. 581)(TS1.mu_vert[1], TS1.Sigma_vert[1,1], TS2.mu_vert[1], TS2.Sigma_vert[1,1])
    end
    if (bearing_comparison)
        bearing_score = mahal(p. 581)(WrapAngleDiffToPi(p. 580)(TS1.mu_rng_az[2],TS2.mu_rng_az[2]),TS1.Sigma_rng_az[2,2], 0, TS2.Sigma_rng_az[2,2])
        if ( (rangerate_score < bearing_score) && (threshold < correlation_threshold) )
            threshold = threshold + correlation_az_threshold
        end
        score = altitude_score + range_score + max( bearing_score, rangerate_score)
    else
        score = altitude_score + range_score + rangerate_score
    end
```

```
    if (score < threshold*unique_scale)
        return true
    else
        return false
    end
end
```

Referenced In: CorrelateByType(p. 466), CorrelateToOwnshipByType(p. 455)

C Constant Variables

C.1 Geometric Constants

Geometric parameters used to convert data in WGS84 coordinates to ENU coordinates.

Type 1 | GeoUtils This data structure holds geometric parameters used to convert data in WGS84 coordinates to ENU coordinates.

```
mutable struct  GeoUtils
    earth_semimajor_axis::R         # Semi-major axis of the Earth
    earth_semiminor_axis::R         # Semi-minor axis of the Earth
    earth_flattening::R             # Flattening of the Earth
    gravity_ratio::R                # Gravity ratio
    earth_first_eccentricity_sq::R  # First eccentricity of the Earth
    earth_second_eccentricity_sq::R # Second eccentricity of the Earth
    earth_equatorial_gravity::R     # Gravity of the Earth at the equator
    earth_gravity_lat45::R          # Gravity of the Earth at 45 degrees latitude
    somiglianas_constant::R         # Somigliana's constant
    meters_to_feet::R               # Conversion factor for meters to feet
    kts_to_mps::R                   # Conversion factor for knots to meters per second
    feet_to_nautical_miles::R       # Conversion factor for feet to nautical miles
    kts_to_fps::R                   # Conversion factor for knots to feet per second
    GeoUtils() = new(
        6.3781370e6,
        6.3567523142e6,
        0.0033528106718309896,
        0.003449787,
        6.69437999014e-3,
        6.73949674228e-3,
        9.7803253359,
        9.8061977710758619,
        1.931853e-3,
        3.28083989501,
        0.514444444444,
        1.6457883369e-4,
        1.6878098571
        )
end
```

C.2 Global Constants

A set of system constants are used throughout the logic. Listed below are the global variables and their values (speeds are in ft/s and accelerations are in ft/s^2).

Algorithm 456 constants

```
# Protection Mode Options for TRM Look-Up Tables
global const PROTECTION_MODE_Xa        = UInt8( 1 )
global const PROTECTION_MODE_Xo_CSPO3k = UInt8( 2 )
global const PROTECTION_MODE_sXu       = UInt8( 3 )
# Equipage Codes
global const EQUIPAGE_NONE       = Int( 0 )
global const EQUIPAGE_CASTA      = Int( 2 )
global const EQUIPAGE_CASRA      = Int( 3 )
global const EQUIPAGE_CASRESP    = Int( 4 )
global const EQUIPAGE_DAARESP    = Int( 5 )
global const EQUIPAGE_TCAS       = Int( 6 )
global const EQUIPAGE_LARGE_CAS  = Int( 7 )
global const EQUIPAGE_SXU_SENIOR = Int( 8 )
global const EQUIPAGE_XR_V2V     = Int( 9 )
# Aircraft Equipment Specification (bit vector)
global const EQUIPMENT_NONE            = UInt16( 0x0000 )
global const EQUIPMENT_V2V_IN          = UInt16( 0x0001 )
global const EQUIPMENT_V2V_OUT         = UInt16( 0x0002 )
global const EQUIPMENT_ADSB_UAT_IN     = UInt16( 0x0004 )
global const EQUIPMENT_ADSB_1090ES_IN  = UInt16( 0x0008 )
global const EQUIPMENT_ADSB_UAT_OUT    = UInt16( 0x0010 )
global const EQUIPMENT_ADSB_1090ES_OUT = UInt16( 0x0020 )
global const EQUIPMENT_TRANSPONDER     = UInt16( 0x0040 )
global const EQUIPMENT_1030_TRANSMIT   = UInt16( 0x0080 )
# Intruder Classification Types
global const CLASSIFICATION_NONE            = UInt8( 0 )
global const CLASSIFICATION_MANNED          = UInt8( 1 )
global const CLASSIFICATION_UNMANNED        = UInt8( 2 )
global const CLASSIFICATION_SMALL_UNMANNED  = UInt8( 3 )
global const CLASSIFICATION_POINT_OBSTACLE  = UInt8( 100 )
global const CLASSIFICATION_GROUND          = UInt8( 101 )
# Coordination Message Types
global const COORDINATION_NONE      = UInt32( 0 )
global const COORDINATION_V2V_OCM   = UInt32( 3 )
# Ownship Coordination Policy
global const OWN_COORDINATION_POLICY_NONE   = UInt8( 0 )
global const OWN_COORDINATION_POLICY_JUNIOR = UInt8( 1 )
global const OWN_COORDINATION_POLICY_SENIOR = UInt8( 2 )
global const OWN_COORDINATION_POLICY_PEER   = UInt8( 3 )
# Surveillance Sources
global const SOURCE_MODES       = Int( 0 )
global const SOURCE_MODEC       = Int( 1 )
global const SOURCE_1090ES_ADSB = Int( 2 )
global const SOURCE_ORNCT       = Int( 3 )
global const SOURCE_978_UAT     = Int( 5 )
global const SOURCE_AGT         = Int( 7 )
global const SOURCE_V2V         = Int( 8 )
global const SOURCE_POINT_OBSTACLE = Int( 9 )
# Avalilable Surveillance Sources on Intruder (bit vector)
global const AVAILABLE_SURVEILLANCE_NONE        = UInt16( 0x0000 )
global const AVAILABLE_SURVEILLANCE_MODES       = UInt16( 0x0001 )
global const AVAILABLE_SURVEILLANCE_MODEC       = UInt16( 0x0002 )
global const AVAILABLE_SURVEILLANCE_1090ES_ADSB = UInt16( 0x0004 )
global const AVAILABLE_SURVEILLANCE_ORNCT       = UInt16( 0x0008 )
global const AVAILABLE_SURVEILLANCE_978_UAT     = UInt16( 0x0020 )
global const AVAILABLE_SURVEILLANCE_AGT         = UInt16( 0x0080 )
global const AVAILABLE_SURVEILLANCE_V2V         = UInt16( 0x0100 )
global const AVAILABLE_SURVEILLANCE_POINT_OBSTACLE  = UInt16( 0x0200 )
# Gillham Encode/Decode constants
global const GILLHAM_NAR_CODE         = Int( 0 )
global const GILLHAM_INVALID_VALUE    = -9999.0
global const GILLHAM_NAR_VALUE        = NaN
global const MAX_ENCODED_ALT          = Int( 2046 )
global const GILLHAM_MAX_BINARY_VALUE = Int( 8 )
```

```
65    global const GILLHAM_TO_AC            = Int[8 10 12 2 4 6 7 9 11 1 3 5]
66    global const AC_TO_GILLHAM            = Int[10 4 11 5 12 6 7 1 8 2 9 3]
67    # RA Message Format (RMF) settings
68    global const RMF_TCAS           = UInt8( 0 )
69    global const RMF_ACAS_Xa        = UInt8( 1 )
70    global const RMF_ACAS_Xu        = UInt8( 2 )
71    global const RMF_ACAS_sXu       = UInt8( 3 )
72    global const RMF_RESERVED       = UInt8( 4 )
73    # Low-level Descend Inhibits
74    global const LDI_NONE             = UInt8( 0 )
75    global const LDI_INCREASE_DESCEND = UInt8( 1 )
76    global const LDI_DESCENDS         = UInt8( 2 )
77    global const LDI_ALL              = UInt8( 3 )
78    # Threat Type Indicator
79    global const TTI_ADDRESS        = UInt8( 1 )
80    global const TTI_ALT_RNG_BRG    = UInt8( 0 )
81    # TIDA constants
82    global const TIDA_MIN_ALTITUDE = -1000.0
83    global const TIDA_OFFSET       = -1000.0
84    global const TIDA_MAX_CODE     = UInt32( 2047 )
85    # TIDB constants
86    global const TIDB_MAX_CODE     = UInt32( 60 )
87    # TIDR constants
88    global const TIDR_MAX_CODE     = UInt32( 127 )
89    # Active Validation State
90    global const V_STATE_NOT_VALIDATED = Int( 0 )
91    global const V_STATE_VALID         = Int( 1 )
92    global const V_STATE_INVALID       = Int( 2 )
93    global const V_STATE_PROVISIONAL   = Int( 3 )
94    # ADS-B Quality State
95    global const QUAL_STATE_NOT_VALIDATED = Int( 0 )
96    global const QUAL_STATE_VALID         = Int( 1 )
97    global const QUAL_STATE_INVALID       = Int( 2 )
98    # sXu Advisory Codes - used for horizontal and vertical display outputs
99    global const SXUCODE_CLEAR          = UInt8( 0 )
100   global const SXUCODE_RA             = UInt8( 4 )
101   # COC action
102   global const COC = Int( 1 )
103   # Display combined control values
104   global const CC_H_NO_ADVISORY     = Int( 0 )
105   global const CC_H_COC             = Int( 1 )
106   global const CC_H_TURN_RIGHT      = Int( 2 )
107   global const CC_H_TURN_LEFT       = Int( 3 )
108   global const CC_H_STRAIGHT        = Int( 4 )
109   global const CC_V_NO_ADVISORY     = Int( 0 )
110   global const CC_V_COC             = Int( 1 )
111   global const CC_V_CORRECTIVE_UP   = Int( 4 )
112   global const CC_V_CORRECTIVE_DOWN = Int( 5 )
113   global const CC_V_PREVENTIVE      = Int( 6 )
114   global const AHRA_NO_TURN    = UInt8(0)
115   global const AHRA_RIGHT_TURN = UInt8(1)
116   global const AHRA_LEFT_TURN  = UInt8(2)
117   global const AHRA_STRAIGHT   = UInt8(3)
118   global const AVRA_STRENGTH_NO_ADVISORY = UInt8(0)
119   # AHRA Track Angle Indicators
120   global const AHRA_TRACK_ANGLE_NAN = Int( 63 )
121   global const AHRA_TRACK_ANGLE_BIN = Int( 10 )
122   # ENU indices
123   global const ENU_EAST_IDX     = Int( 1 )
124   global const ENU_NORTH_IDX    = Int( 2 )
125   global const ENU_UP_IDX       = Int( 3 )
126   # Vertical Policy Scaling Indices
127   global const SCALING_VTRM_POLICY_FT         = Int( 1 )
128   global const SCALING_VTRM_ENTRY_FT          = Int( 2 )
129   global const SCALING_VTRM_ENTRY_FPS         = Int( 6 )
```

```
# Vertical Policy Scaling Options
global const VERTICAL_SCALING_OPTION_DEFAULT   = Int( 1 )
# Horizontal Policy State Indices
global const POLICY_VERTICAL_TAU = Int( 1 )
global const POLICY_RANGE        = Int( 2 )
global const POLICY_THETA        = Int( 3 )
global const POLICY_PSI          = Int( 4 )
global const POLICY_OWN_SPEED    = Int( 5 )
global const POLICY_INTR_SPEED   = Int( 6 )
global const POLICY_LAST         = Int( 6 )
# Horizontal Policy Scaling Indices
global const SCALING_HTRM_POLICY_RANGE               = Int( 1 )
global const SCALING_HTRM_POLICY_TARGET_SEPARATION   = Int( 2 )
global const SCALING_HTRM_POLICY_SPEED               = Int( 3 )
global const SCALING_HTRM_POLICY_VERTICAL_TAU        = Int( 4 )
# Horizontal Policy Scaling Options
global const HORIZONTAL_SCALING_OPTION_DEFAULT   = Int( 1 )
# Horizontal RA Altitude Cutoff Options
# Horizontal Coordination
global const HCOORD_SENSE_NONE = Int( 0 )
global const HCOORD_SENSE_SAME = Int( 1 )
global const HCOORD_SENSE_DIFFERENT = Int( 2 )
# Force_alarm Settings for Display Logic
global const FORCE_ALARM_NONE = Int( 0 )
global const FORCE_ALARM_ON   = Int( 1 )
global const FORCE_ALARM_OFF  = Int( 2 )
# Target Designations for Intruders
# Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1
global const DESIGNATION_NONE                        = UInt8( 0 )
global const DESIGNATION_DESIGNATED_NO_ALERTS        = UInt8( 4 )
global const DESIGNATION_Xo_CSPO3k                   = UInt8( 5 )
global const DESIGNATION_UNDESIGNATE_NO_ALERTS       = UInt8( 7 )
global const DESIGNATION_UNDESIGNATE_PROTECTION_MODE  = UInt8( 8 )
# RA Processing of Intruders
global const RA_PROCESSING_NONE                      = UInt8( 0 )
global const RA_PROCESSING_DROPPED_TRACK             = UInt8( 1 )
global const RA_PROCESSING_GLOBAL_RA                 = UInt8( 2 )
global const RA_PROCESSING_GLOBAL_SURV_ONLY          = UInt8( 4 )
global const RA_PROCESSING_DEGRADED_SURVEILLANCE     = UInt8( 6 )
# Target Designations for Indices
# Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1
global const Xo_IDX_DNA                      = UInt8( 1 )
global const Xo_IDX_CSPO3k                   = UInt8( 2 )
# Target Designations for Status
# Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1
global const Xo_STATUS_NONE                    = UInt8( 0x00 )
global const Xo_STATUS_DELAYED_ACTIVE_RA       = UInt8( 0x11 )
global const Xo_STATUS_SUSPENDED_MULTITHREAT   = UInt8( 0x21 )
global const Xo_STATUS_SUSPENDED_INVALID       = UInt8( 0x22 )
global const Xo_STATUS_SUSPENDED_DROPPED       = UInt8( 0x23 )
global const Xo_STATUS_SUSPENDED_TIMER         = UInt8( 0x24 )
global const Xo_STATUS_UNDESIGNATED_UNAVAILABLE = UInt8( 0x31 )
global const Xo_STATUS_UNDESIGNATED_GEOMETRIC   = UInt8( 0x32 )
global const Xo_STATUS_UNDESIGNATED_DROPPED     = UInt8( 0x33 )
global const Xo_STATUS_UNDESIGNATED_TIMEOUT     = UInt8( 0x34 )
global const Xo_STATUS_UNDESIGNATED_NO_BEARING  = UInt8( 0x35 )
# Target Designation Xo mode Availability for any intruders
# Constant values not used in Xu; inherited from Xa/Xo, included for reference
global const Xo_AVAILABILITY_NOT_CONFIGURED     = UInt8( 0 )
global const Xo_AVAILABILITY_UNAVAILABLE_TO_RUN = UInt8( 1 )
global const Xo_AVAILABILITY_AVAILABLE_TO_RUN   = UInt8( 2 )
global const Xo_AVAILABILITY_ON                 = UInt8( 3 )
# Degraded Surveillance Types Intruder (bit vector)
global const DEGRADED_SURVEILLANCE_NONE                  = UInt16( 0x0000 )
global const DEGRADED_SURVEILLANCE_NAR                   = UInt16( 0x0001 )
```

```
195    global const DEGRADED_SURVEILLANCE_PASSIVE_NOT_VALIDATED  = UInt16( 0x0002 )
196    global const DEGRADED_SURVEILLANCE_NO_BEARING             = UInt16( 0x0004 )
197    global const DEGRADED_SURVEILLANCE_VERT_COAST             = UInt16( 0x0010 )
198    global const DEGRADED_SURVEILLANCE_HORIZ_COAST            = UInt16( 0x0020 )
199    global const DEGRADED_SURVEILLANCE_ORNCT_COAST            = UInt16( 0x0800 )
200    global const DEGRADED_SURVEILLANCE_INVALID_V2VCOORDINATION_VERT = UInt16( 0x1000 )
201    global const DEGRADED_SURVEILLANCE_INVALID_V2VCOORDINATION_HORZ = UInt16( 0x2000 )
202    global const DEGRADED_SURVEILLANCE_ORNCT_FOV_COAST        = UInt16( 0x4000 )
203    global const DEGRADED_SURVEILLANCE_ALL                    = UInt16( 0xffff )
204    # Degraded Surveillance Types Ownship (bit vector)
205    global const DEGRADED_OWN_NONE                 = UInt16( 0x0000 )
206    global const DEGRADED_OWN_ALTPRES_COAST        = UInt16( 0x0001 )
207    global const DEGRADED_OWN_WGS84_COAST          = UInt16( 0x0002 )
208    global const DEGRADED_OWN_HDG_COAST            = UInt16( 0x0004 )
209    global const DEGRADED_OWN_HDG_INVALID          = UInt16( 0x0008 )
210    global const DEGRADED_OWN_WGS84_INVALID        = UInt16( 0x0010 )
211    global const DEGRADED_OWN_ALTPRES_INVALID      = UInt16( 0x0020 )
212    # Ownship Heading States
213    global const OWN_HEADING_INVALID               = 0
214    global const OWN_HEADING_DEGRADED              = 1
215    global const OWN_HEADING_NOMINAL               = 2
216    # Ownship Wgs84 States
217    global const OWN_WGS84_INVALID                 = 0
218    global const OWN_WGS84_VALID                   = 1
219    # CAS Operational Mode Settings
220    global const OPMODE_STANDBY   = UInt8( 1 )
221    global const OPMODE_SURV_ONLY = UInt8( 2 )
222    global const OPMODE_RA        = UInt8( 3 )
223    # Sensitivity Level (SL) Settings
224    global const SENSITIVITY_LEVEL_AUTOMATIC = 0
225    # CAS Operational Status Settings
226    global const CA_STATUS_NONE         = UInt8( 0 )
227    global const CA_STATUS_RA_INHIBITED = UInt8( 1 )
228    global const CA_STATUS_RA           = UInt8( 2 )
229    # Horizontal Velocity Operational Mode Settings
230    global const TRM_VELMODE_NONE                   = UInt8(0)
231    global const TRM_VELMODE_GROUNDSPEED_TRACKANGLE = UInt8(1)
232    global const TRM_VELMODE_AIRSPEED_HEADING       = UInt8(2)
233    global const TRM_VELMODE_GROUNDSPEED_HEADING    = UInt8(3)
234    # Altitude Operation Mode Settings
235    global const TRM_ALTMODE_NONE     = UInt8(0)
236    global const TRM_ALTMODE_PRESSURE = UInt8(1)
237    global const TRM_ALTMODE_HAE      = UInt8(2)
238    global const AGL_LOST = 10
239    # Mode C
240    global const MODE_C_FAILED_ASSOCIATION = Int( -1 )
241    global const MODE_C_QUANT              = UInt32( 100 )
242    # Correlation
243    global const CORRELATION_MECH_NOT_AUTHORIZED = UInt8( 0 )
244    global const CORRELATION_MECH_ICAO24         = UInt8( 1 )
245    global const CORRELATION_MECH_ANON24         = UInt8( 2 )
246    global const CORRELATION_MECH_V2VUID         = UInt8( 3 )
247    global const CORRELATION_MECH_ICAO24_V2VUID  = UInt8( 4 )
248    global const CORRELATION_MECH_ANON24_V2VUID  = UInt8( 5 )
249    global const CORRELATION_MECH_SPATIAL        = UInt8( 6 )
250    global const CORRELATION_MECH_S_STAR         = UInt8( 7 )
251    global const CORRELATION_TRKTYPE_INVALID                = UInt8( 0x00 )
252    global const CORRELATION_TRKTYPE_ADSB_ICAO24            = UInt8( 0x01 )
253    global const CORRELATION_TRKTYPE_ADSB_ANON24            = UInt8( 0x02 )
254    global const CORRELATION_TRKTYPE_V2V_V2VUID             = UInt8( 0x03 )
255    global const CORRELATION_TRKTYPE_V2V_V2VUID_ICAO24      = UInt8( 0x04 )
256    global const CORRELATION_TRKTYPE_V2V_V2VUID_ANON24      = UInt8( 0x05 )
257    global const CORRELATION_TRKTYPE_ORNCT_ORNCTID          = UInt8( 0x06 )
258    global const CORRELATION_TRKTYPE_AGT_AGTID              = UInt8( 0x07 )
259    global const CORRELATION_TRKTYPE_AGT_AGTID_ICAO24       = UInt8( 0x08 )
```

```
global const CORRELATION_TRKTYPE_AGT_AGTID_ANON24        = UInt8( 0x09 )
global const CORRELATION_TRKTYPE_AGT_AGTID_V2VUID        = UInt8( 0x0A )
global const CORRELATION_TRKTYPE_AGT_AGTID_V2VUID_ICAO24 = UInt8( 0x0B )
global const CORRELATION_TRKTYPE_AGT_AGTID_V2VUID_ANON24 = UInt8( 0x0C )
global const CORRELATION_PENDING            = UInt8( 0 )
global const CORRELATION_POSITIVE           = UInt8( 1 )
global const CORRELATION_NEGATIVE           = UInt8( 2 )
global const CORRELATION_SPATIAL_POSITIVE   = UInt8( 3 )
global const CORRELATION_SPATIAL_NEGATIVE   = UInt8( 4 )
# Altitude boundaries for aircraft that have Non-Altitude Reporting Surveillance
global const NARS_THRESHOLD_MIN = ( -1200.0 )
global const NARS_THRESHOLD_MAX = ( 126750.0 )
# Surveillance Region Mode
global const SURVEILLANCE_REGION_NORMAL           = Int( 0 )
global const SURVEILLANCE_REGION_REDUCED          = Int( 1 )
global const SURVEILLANCE_REGION_HYBRID           = Int( 2 )
global const SURVEILLANCE_REGION_EXT_OR_DROPPED   = Int( 3 )
# Altitude Buffer for TRM Crossing Determination
global const CROSSING_ALTITUDE_BUFFER = 100.0
# Display Arrow Indicators
global const DISPLAY_ARROW_LEVEL   = Int(  0 )
global const DISPLAY_ARROW_CLIMB   = Int(  1 )
global const DISPLAY_ARROW_DESCEND = Int( -1 )
# CAS Coordination Capability/Type over ADS-B OSM
global const CAS_ACTIVE_TCAS             = UInt8( 0 )
global const CAS_ACTIVE_NON_TCAS         = UInt8( 1 )
global const CAS_ACTIVE_NON_TCAS_OCM     = UInt8( 2 )
global const CASRESP_ACTIVE              = UInt8( 3 )
global const CASRESP_PASSV_MODES         = UInt8( 4 )
global const CASRESP_PASSV               = UInt8( 5 )
# CAS Coordination Capability/Type over V2V OSM
global const CAS_V2V_NONE                = UInt8( 0 )
global const CAS_V2V_SXU                 = UInt8( 1 )
global const CAS_V2V_XR                  = UInt8( 2 )
# CAS Coordination Sense in CCCB
global const CAS_SENSE_VERTICAL_ONLY     = UInt8( 0 )
global const CAS_SENSE_HORIZONTAL_ONLY   = UInt8( 1 )
global const CAS_SENSE_BLENDED           = UInt8( 2 )
global const CAS_SENSE_PER_INTRUDER      = UInt8( 3 )
# V2V priority in CCCB
global const SXU_V2V_CCCB_PRIORITY_MIDDLE = UInt8( 7 )
# DAA Coordination Capability
global const DAA_RCV_NONE   = UInt8( 0 )
global const DAA_RCV_ACTIVE = UInt8( 1 )
global const DAA_RCV_PASSV  = UInt8( 2 )
# RWC Broadcast Bits
# Ownship-Relative-Non-Cooperative-Track Status Indicators
global const ORNCT_STATUS_HIGH_PRIORITY_TRACK     = UInt8( 0x1 )
global const ORNCT_STATUS_MISSED_IN_LATEST_UPDATE = UInt8( 0x2 )
global const ORNCT_STATUS_FINAL_UPDATE            = UInt8( 0x4 )
global const ORNCT_STATUS_TRACK_NEGATION          = UInt8( 0x8 )
# Absolute Geodetic Track Status Indicators
global const AGT_STATUS_HIGH_PRIORITY_TRACK     = UInt16( 0x0001 )
global const AGT_STATUS_MISSED_IN_LATEST_UPDATE = UInt16( 0x0002 )
global const AGT_STATUS_TO_BE_DELETED           = UInt16( 0x0004 )
global const AGT_STATUS_MODE_S_VALID            = UInt16( 0x0008 )
global const AGT_STATUS_MODE_S_NON_ICAO         = UInt16( 0x0010 )
global const AGT_STATUS_V2V_UID_VALID           = UInt16( 0x0020 )
# Representation of a invalid target id/track idx (both start at 1).
global const NO_TARGET_FOUND = UInt32( 0 )
global const NO_TRACK_FOUND  = Int64( 0 )
# Protection Mode Index when no Xo is implemented
global const NO_Xo_MODE_INDEX = UInt8( 1 )
# Ground Avoidance Constants
global const ID_GROUND           = UInt32( 4294967295 )
```

```
global const GROUND_ALERT_NONE   = UInt8( 0 )
global const GROUND_ALERT_ONLY   = UInt8( 1 )
global const GROUND_ALERT_MIXED  = UInt8( 2 )
# Horizontal Desensitivity Mode
global const HORIZONTAL_DESENSITIVITY_NONE        = UInt8( 0 )
global const HORIZONTAL_DESENSITIVITY_SENSITIZE   = UInt8( 1 )
global const HORIZONTAL_DESENSITIVITY_DESENSITIZE = UInt8( 2 )
```

D Data Structure Definitions

D.1 STM Output Data Structures

The algorithms contained within this document rely on a number of data structures to store and pass data within the STM. This appendix provides the definition, description and initial values for data structures used to output data from the STM. The TRMINPUT data structure is used as input by the TRM and, as such, is defined with the other TRM data structures used by the TRM.

Type 2 | StmReport The STM Report is the data structure produced by the STM to be partly consumed by the TRM to perform threat evaluation. The fields of this data structure include an exact input to the TRM, the data to be processed by the V2V capability data transmission device, and the data to be sent to the pilot display.

```
mutable struct  StmReport
    trm_input::TRMInput(p. 497)                # Input to the TRM
    own_v2v_osm::V2VOperationalStatusMessage(p. 496) # Ownship V2V operational status message
    display::Vector{StmDisplayStruct(p. 481)} # Intruder data for the display
    StmReport( trm_input::TRMInput(p. 497), own_v2v_osm::V2VOperationalStatusMessage(p. 496), display::Vector{
        StmDisplayStruct(p. 481)} ) = new( trm_input, v2v_osm, display)
    StmReport() = new( TRMInput(p. 497)(), V2VOperationalStatusMessage(p. 496)(), StmDisplayStruct(p. 481)[] )
end
```

Referenced In: AddADSBTrackToReport(p. 135), SetDisplayDataORNCT(p. 153), SetDisplayDataPassive(p. 146), AddTracksToReport(p. 119), AddPointObstacleTrackToReport(p. 157), GenerateStmReport(p. 112), AddORNCTTrackToReport(p. 152), AddAGTTrackToReport(p. 155), SetOwnshipVelocity(p. 117), SetOwnshipData(p. 116), FilterTracksForTRM(p. 132), AddV2VTrackToReport(p. 150)

Type 3 | StmDisplayStruct This data structure contains the intruder state data that is provided to the display and is part of the STM Report. It includes the vertical rate arrow for the display, indication that the intruder is reporting altitude, the intruder ID, relative altitude, tracked ground range, bearing relative to own airframe, and the ID directory recording various identifiers assigned to the intruder target by external surveillance systems.

```
mutable struct  StmDisplayStruct
    toa::R                      # Time of Applicability
    source::UInt32              # What type of surveillance did this track come from
    available_sources::UInt16   # Bit vector indicating available surveillance source tracks
    id::UInt32                  # Intruder ID inside the Target database
    id_directory::IntruderIDDirectory(p. 482)
                                # All external IDs assigned to the intruder
    validated_adsb::Bool        # If the source is ADS-B, was it validated
    r_ground_ft::R              # Tracked ground-range (feet)
    bearing_rel_rad::R          # Bearing relative to own airframe (radians)
    bearing_valid::Bool         # Indicates that a bearing is available for the intruder
    dx_rel_fps::R               # Relative horizontal velocity east/west (feet per sec)
    dy_rel_fps::R               # Relative horizontal velocity north/south (feet per sec)
    z_rel_ft::R                 # Relative altitude (feet)
    alt_reporting::Bool         # Indicates that the intruder is altitude reporting
    dz_rel_fps::R               # Relative altitude rate (feet/sec)
    arrow::Int32                # Display vertical rate arrow
    StmDisplayStruct( toa::R, source::UInt32, available_sources::UInt16, id::UInt32, id_directory::
         IntruderIDDirectory(p. 482), validated_adsb::Bool, r_ground_ft::R, bearing_rel_rad::R, bearing_valid::
         Bool, dx_rel_fps::R, dy_rel_fps::R, z_rel_ft::R, alt_reporting::Bool, dz_rel_fps::R, arrow::Int32 )
         =
        new( toa, source, available_sources, id, id_directory, validated_adsb, r_ground_ft, bearing_rel_rad,
             bearing_valid, dx_rel_fps, dy_rel_fps, z_rel_ft, alt_reporting, dz_rel_fps, arrow )
    StmDisplayStruct() =
        new(0.0, 0, 0, 0, IntruderIDDirectory(p. 482)(), false, 0.0, 0.0, false, 0.0, 0.0, 0.0, false, 0.0, 0 )
end
```

Referenced In: TRMIntruderInput(p. 499), SetDisplayDataORNCT(p. 153), SetDisplayDataPassive(p. 146), StmReport(p. 480), FilterTracksForTRM(p. 132), CompareIntruderRange(p. 133)

Type 4 | IntruderIDDirectory This data structure contains all the valid identifiers associated with a single intruder. Possible identifiers are ICAO 24 bit address, anonymous 24 bit address, V2V unique identifier, Own-Relative-Non-Cooperative Track identifier, Absolute-Geodetic Track identifier, or Point Obstacle identifier. Only one valid address or identifier of each type will be stored for the intruder except for AGT IDs. All valid AGT IDs will be stored. This structure is included in substructures of both STM and TRM reports.

```
mutable struct  IntruderIDDirectory
    icao24::IntruderIDEntry(p. 482)
    anon24::IntruderIDEntry(p. 482)
    v2v_uid::IntruderIDEntry(p. 482)
    ornct_id::IntruderIDEntry(p. 482)
    agt_ids::Vector{UInt32}
    po_id::IntruderIDEntry(p. 482)
    IntruderIDDirectory() = new(
         IntruderIDEntry(p. 482)(),
         IntruderIDEntry(p. 482)(),
         IntruderIDEntry(p. 482)(),
         IntruderIDEntry(p. 482)(),
         UInt32[],
         IntruderIDEntry(p. 482)()
        )
    IntruderIDDirectory(
        icao24::IntruderIDEntry(p. 482),
        anon24::IntruderIDEntry(p. 482),
        v2v_uid::IntruderIDEntry(p. 482),
        ornct_id::IntruderIDEntry(p. 482),
        agt_ids::Vector{UInt32},
        po_id::IntruderIDEntry(p. 482)
       ) = new(icao24,anon24,v2v_uid,ornct_id,agt_ids,po_id)
    IntruderIDDirectory( mode_s::UInt32, is_icao::Bool ) = IntruderIDDirectory(
         is_icao ? IntruderIDEntry(p. 482)(mode_s) : IntruderIDEntry(p. 482)(),
         is_icao ? IntruderIDEntry(p. 482)() : IntruderIDEntry(p. 482)(mode_s),
         IntruderIDEntry(p. 482)(),
         IntruderIDEntry(p. 482)(),
         UInt32[],
         IntruderIDEntry(p. 482)()
        )
end
```

Referenced In: TRMIntruderInput(p. 499), HTRMIntruderDisplayData(p. 504), HTRMIntruderState(p. 514), TRMCoordinationInterrogationData(p. 504), SetIntruderIDDirectory(p. 137), StmDisplayStruct(p. 481), TRMIntruderDesignationData(p. 505), TRMIntruderDisplayData(p. 503)

Type 5 | IntruderIDEntry This data structure containes the address or unique identifier of an intruder along with the validity of that value.

```
mutable struct  IntruderIDEntry
    value::UInt32
    valid::Bool
    IntruderIDEntry() = new(0x0, false)
    IntruderIDEntry(value::UInt32) = new(value, true)
end
```

Referenced In: IntruderIDDirectory(p. 482), TRMCoordinationInterrogationData(p. 504), SetIntruderIDDirectory(p. 137)

D.2 STM Internal Data Structures

The algorithms contained within this document rely on a number of data structures to store and pass data within the STM. This appendix provides the definition, description and initial values for data structures that are internal to the STM.

Type 6 | TrackFile The abstract type, Track File, represents the supertype of all other Track Files.

```
abstract type  TrackFile end
```

Referenced In: RemoveDecorrelatedTrackFromOwnship(p. 451), RemoveDecorrelatedTrackFromTarget(p. 459), AGTTrackFile(p. 486), UpdateAltInHorGeoPos(p. 20), CorrelateByType(p. 466), TrackSourceSelection(p. 121), ValidationCheck(p. 162), OptimalQualityTrack(p. 125), ADSBTrackFile(p. 485), AbsoluteGeodeticToOwnshipRelative(p. 141), CorrelateIndividualTracks(p. 462), DecorrelateTargetsFromOwnship(p. 451), GetAircraftTracks(p. 440), PassiveQualityCheck(p. 127), ExtrapolateTrack(p. 440), ReCenterHorizontalTrackLocation(p. 42), AddDecorrelatedTrackToTarget(p. 459), GetTracksToDecorrelate(p. 458), VerticalRateArrowUpdate(p. 148), TrackExists(p. 19), DecorrelateTargets(p. 457), PointObstacleTrackFile(p. 487), CorrelationTrackType(p. 449), SetIntruderDegradedFlags(p. 138), GetTracksToExtrapolate(p. 440), UpdatePassiveQualityHistory(p. 29), GetPreviouslySelectedTrack(p. 130), V2VTrackFile(p. 488), ORNCTTrackFile(p. 484), CorrelatePosition(p. 471), GetCorrelationMechanism(p. 469), CorrelateID(p. 468), SelectFinalTrack(p. 129), OptimalPreTrackedTracks(p. 122), OptimalTrackHistoryUpdate(p. 128), TrackMap(p. 494), GenerateLeadTrackList(p. 460)

Type 7 | ORNCTTrackFile The ORNCT Track File contains the data elements required to maintain the state of a target aircraft from tracked Ownship Relative Non-Cooperative Track reports. The data structure includes three tracks, one for the vertical dimension and two for the horizontal dimension. Each track share a time of applicability. A Track Summary is included for surveillance source correlation. Additional data fields determined through received ORNCT reports, such as extended coasting due to field of view track drop, DAA track priority, outlier detection count, and report updates count are included.

```
mutable struct  ORNCTTrackFile <: TrackFile(p. 483)
    ornct_id::UInt32                # Track number assigned by external surveillance system
    classification::UInt8           # intruder classification
    toa::R                          # Time of applicability (seconds)
    toa_update::R                   # Time of applicability - not updated with coasted inputs (seconds)
    mu_cart::Vector{R}              # Cartesian track in relative E-N ([ft, ft, ft/s, ft/s])
    Sigma_cart::Matrix{R}           # Cartesian track covariance matrix
    mu_rng::Vector{R}               # Ground range track ([ft, ft/s])
    Sigma_rng::Matrix{R}            # Ground range track covariance matrix
    mu_vert::Vector{R}              # Vertical track ([ft, ft/s])
    Sigma_vert::Matrix{R}           # Vertical track covariance matrix
    trk_summary::TrackSummary(p. 495)      # Track summary data structure used in correlation
    display_arrow_current::Z        # Current vertical rate arrow on display
    vert_arrow_history::Vector{Z}   # Previous vertical rate arrows sent to display
    is_FOV_coast::Bool              # Extended coasting due to field of view track drop
    high_priority::Bool             # DAA track priority
    odc::Z                          # Outlier detection count
    update_count::Z                 # ORNCT report update count
    track_uid::String               # Unique track identifier for internal processing
    ORNCTTrackFile(ornct_id::UInt32) = new(
        ornct_id,
        CLASSIFICATION_NONE,
        NaN,
        NaN,
        fill(NaN,4),
        fill(NaN,4,4),
        fill(NaN,2),
        fill(NaN,2,2),
        fill(NaN,2),
        fill(NaN,2,2),
        TrackSummary(p. 495)(),
        DISPLAY_ARROW_LEVEL,
        Z[],
        false,
        false,
        0,
        0,
        string("ORNCT", ornct_id)
    )
end
```

Referenced In: Target(p. 493), AddORNCTTrackToDB(p. 64), ReceiveOwnRelNonCoopTrack(p. 59), OptimalQualityTrack(p. 125), PredictPreTrackedTrack(p. 126), ExtrapolateTrack(p. 440), PredictORNCTTrack(p. 64), AddDecorrelatedTrackToTarget(p. 459), AddTracksToReport(p. 119), UpdateORNCTTrack(p. 65), DecorrelateTargets(p. 457), ExtrapolateORNCTTrack(p. 445), CorrelationTrackType(p. 449), SetIntruderDegradedFlags(p. 138), CorrelatePosition(p. 471), OffsetTracksByOwnshipTurn(p. 93)

Type 8 | ADSBTrackFile The ADS-B Track File contains the data elements required to maintain the state of a target aircraft using ADS-B Reports from an ADS-B report generator. The data structure includes two tracks, one for the vertical dimension and one for the horizontal dimension. Each track has their own time of applicability. A Track Summary is included for surveillance source correlation. Additional data fields determined through ADS-B Reports, such as TCAS operational status, reported ADS-B quality metrics, and an indication of a non-standard ICAO address are also stored in the data structure.

```
mutable struct  ADSBTrackFile <: TrackFile(p. 483)
    mode_s::UInt32                  # intruder Mode S address
    non_icao::Bool                  # intruder Mode S address is non-ICAO
    uat::Bool                       # track file is UAT
    updates_pos::Z                  # number of position updates
    mu_hor::Vector{R}               # horizontal track in relative East-North ([meters, meters / second,
        meters, meters/second])
    Sigma_hor::Matrix{R}            # horizontal track covariance matrix
    odc_hor::Z                      # horizontal outlier detection count
    init_velocity::Bool             # has horizontal velocity been initialized
    mu_vert::Vector{R}              # vertical track ([feet, feet / second])
    Sigma_vert::Matrix{R}           # vertical track covariance matrix
    odc_vert::Z                     # vertical outlier detection count
    updates_vert::Z                 # number of vertical updates
    valid_vert::Bool                # vertical track validity
    quant::UInt32                   # intruder vertical quantization (feet)
    adsb_version::UInt32            # reported ads-b version number
    nic::UInt32                     # reported NIC value
    nacp::UInt32                    # reported NACp value
    nacv::UInt32                    # reported NACv value
    gva::UInt32                     # reported GVA value
    sil::UInt32                     # reported SIL value
    sda::UInt32                     # reported SDA value
    trk_summary::TrackSummary(p. 495)       # Track summary data structure used in correlation
    toa_hor::R                      # ToA of horizontal track (seconds)
    toa_vert::R                     # ToA of vertical track (seconds)
    toa_pos_update::R               # ToA of last ADS-B position update (seconds)
    toa::R                          # ToA of track file (seconds)
    display_arrow_current::Z        # Current vertical rate arrow on display
    vert_arrow_history::Vector{Z}   # Previous vertical rate arrows sent to display
    lla_at_hor_toa::Vector{R}       # latitude, longitude, altitude at time of horizontal toa (radians)
    ecef_at_hor_toa::Vector{R}      # ECEF position at time of horizontal toa (meters)
    alt_src_hae::Bool               # Flag to indicate whether altitude source is HAE
    passive_qual_history::Vector{Bool} # History of track passing ADS-B reported quality requirements
    qual_state::Z                   # State of track quality check
    externally_validated::Bool      # Passive intruder has been validated by an external source
    track_uid::String               # Unique track identifier for internal processing
    ADSBTrackFile(mode_s::UInt32, non_icao::Bool, uat::Bool) = new(
        mode_s, non_icao, uat, 0,
        zeros(4), zeros(4,4), 0, false,
        zeros(2), zeros(2,2), 0, 0, false,
        0, 0, 0, 0, 0, 0, 0, 0,
        TrackSummary(p. 495)(),
        0.0, 0.0, 0.0, 0.0,
        DISPLAY_ARROW_LEVEL, Z[],
        zeros(3), zeros(3), false,
        Bool[], QUAL_STATE_NOT_VALIDATED, false,
        string("ADSB", mode_s, non_icao ? "ANON" : "ICAO")
    )
end
```

Referenced In: ReceiveStateVectorPositionReport(p. 12), ExtrapolateADSBTrack(p. 443), AdvancePassiveTrackState(p. 45), Target(p. 493), InitializeVerticalTracker(p. 51), PassiveQualityCheck(p. 127), ExtrapolateTrack(p. 440), AddAltBiasAndSample(p. 143), AddTracksToReport(p. 119), AddADSBTrackToDB(p. 18), ConvertGVAToSigmaVEPU(p. 124), AdvancePassiveTrackPosition(p. 39), AdvanceVerticalTrack(p. 53), CorrelationTrackType(p. 449), SetIntruderDegradedFlags(p. 138), InitializePassiveTrackFile(p. 17), UpdatePassiveQualityHistory(p. 29), AssessVerticalValidity(p. 123), CorrelatePosition(p. 471), GetAltForHorTrkObservation(p. 20), ReceiveStateVectorUatReport(p. 27), AdvancePassiveTrackVelocity(p. 43)

Type 9 | AGTTrackFile The AGT Track File contains the data elements required to maintain the state of a target aircraft using Absolute Geodetic Track Reports. The data structure includes two tracks, one for the vertical dimension and one for the horizontal dimension. Both tracks share a time of applicability. A Track Summary is included for surveillance source correlation. Additional data fields determined through AGT Reports, such as an indication of whether the report has been validated by an external source, whether the inputs come from a non-cooperative surveillance source or a passive surveillance source, and whether the track is a high priority for DAA are also stored in the data structure.

```
mutable struct  AGTTrackFile <: TrackFile(p. 483)
    agt_id::UInt32                  # Track ID assigned by external surveillance system
    mode_s::UInt32                  # Mode S address
    mode_s_non_icao::Bool           # Intruder Mode S address is non-ICAO
    mode_s_valid::Bool              # Intruder Mode S validity
    v2v_uid::UInt32                 # V2V unique identifier
    v2v_uid_valid::Bool             # Validity of the V2V unique identifier
    classification::UInt8           # intruder classification
    toa::R                          # Time of applicability (seconds)
    toa_update::R                   # Time of applicability - not updated with coasted inputs (seconds)
    mu_hor::Vector{R}               # Horizontal track in relative East-North ([m, m/s, m, m/s])
    Sigma_hor::Matrix{R}            # Horizontal track covariance matrix
    mu_vert::Vector{R}              # Vertical track ([ft, ft/s])
    Sigma_vert::Matrix{R}           # Vertical track covariance matrix
    valid_vert::Bool                # Validity of the vertical track
    trk_summary::TrackSummary(p. 495)      # Track summary data structure used in correlation
    toa_hor::R                      # ToA of horizontal track (seconds)
    toa_vert::R                     # ToA of vertical track (seconds)
    display_arrow_current::Z        # Current vertical rate arrow on display
    vert_arrow_history::Vector{Z}   # Previous vertical rate arrows sent to display
    high_priority::Bool             # DAA track priority
    odc::Z                          # Outlier detection count
    init_time::R                    # Time of track initialization (seconds)
    max_range_vec::Vector{R}        # Maximum range vector (ft)
    lla_at_hor_toa::Vector{R}       # latitude, longitude, altitude at time of horizontal toa (radians)
    ecef_at_hor_toa::Vector{R}      # ECEF position at time of horizontal toa (meters)
    alt_src_hae::Bool               # Flag to indicate whether altitude source is HAE
    externally_validated::Bool      # Passive intruder has been validated by an external source
    track_uid::String               # Unique track identifier for internal processing
    AGTTrackFile(agt_id::UInt32) = new(
        agt_id, 0,
        false, false,
        0, false,
        CLASSIFICATION_NONE, NaN,
        NaN, fill(NaN,4),
        fill(NaN,4,4), fill(NaN,2),
        fill(NaN,2,2), false,
        TrackSummary(p. 495)(), 0,
        0, DISPLAY_ARROW_LEVEL,
        Z[], false,
        0, -1.0,
        R[], zeros(3),
        zeros(3), false,
        false, string("AGT",agt_id)
    )
end
```

Referenced In: OwnShipData(p. 491), MergeTargets(p. 464), AssociateAGTToTarget(p. 69), Target(p. 493), ExtrapolateAGTTrack(p. 447), OptimalQualityTrack(p. 125), AssociateAGTToOwnship(p. 69), CorrelateIndividualTracksToOwnship(p. 454), PredictPreTrackedTrack(p. 126), ExtrapolateTrack(p. 440), AssociateV2VToV2VAndAGTTargets(p. 100), AddAltBiasAndSample(p. 143), AddAGTTrackToDB(p. 72), AddDecorrelatedTrackToTarget(p. 459), AddTracksToReport(p. 119), UpdateAGTTrack(p. 76), AddAGTTrackToReport(p. 155), CorrelationTrackType(p. 449), InitializeAGTTrack(p. 73), AssessVerticalValidity(p. 123), GetAltForHorTrkObservation(p. 20), CorrelateToOwnshipByType(p. 455)

Type 10 | PointObstacleTrackFile The Point Obstacle Track File contains the data elements required to maintain the state of a target aircraft using point-obstacle surveillance reports from an external surveillance system. The data structure includes an externally assigned track id, global position information, and a flag that indicates if the Track File should be deleted.

```
mutable struct  PointObstacleTrackFile <: TrackFile(p. 483)
    po_id::UInt32                 # point obstacle external identification
    lla_hae_rad_ft::Vector{R}     # point obstacle latitude, longitude, geodetic HAE altitude (rad, rad, ft)
    ecef_hae_m::Vector{R}         # point obstacle ecef location (m)
    to_be_deleted::Bool           # to be deleted boolean
    PointObstacleTrackFile() = new(
        0,
        fill(NaN,3),
        fill(NaN,3),
        false
    )
end
```

Referenced In: AddPointObstacleTrackToDB(p. 79), Target(p. 493), AssociatePointObstacleReportToTarget(p. 78), ReceivePointObstacleReport(p. 78)

Type 11 | V2VTrackFile The V2V Track File contains the data elements required to maintain the state of a target aircraft using Vehicle to Vehicle Reports. The data structure includes two tracks, one for the vertical dimension and one for the horizontal dimension. Each track has their own time of applicability. A Track Summary is included for surveillance source correlation. Additional data fields determined through V2V Reports, such as coordination capability, reported quality metrics, Quality History information, and an indication of whether the Report has been validated by an external source are also stored in the data structure.

```
mutable struct  V2VTrackFile <: TrackFile(p. 483)
    v2v_uid::UInt32                # intruder V2V unique identifier
    mode_s::UInt32                 # intruder Mode S address (UAM)
    mode_s_non_icao::Bool          # intruder Mode S address is non-ICAO
    mode_s_valid::Bool             # intruder Mode S address validity (true -> is UAM)
    ri::UInt32
    classification::UInt8          # intruder classification
    updates_pos::Z                 # number of position updates
    mu_hor::Vector{R}              # horizontal track in relative East-North ([meters, meters / second,
        meters, meters/second])
    Sigma_hor::Matrix{R}           # horizontal track covariance matrix
    odc_hor::Z                     # horizontal outlier detection count
    init_velocity::Bool            # has horizontal velocity been initialized
    mu_vert::Vector{R}             # vertical track ([feet, feet / second])
    Sigma_vert::Matrix{R}          # vertical track covariance matrix
    sigma_vepu::R                  # vertical estimated position uncertainty sigma (ft)
    odc_vert::Z                    # vertical outlier detection count
    updates_vert::Z                # number of vertical updates
    valid_vert::Bool               # vertical track validity
    quant::UInt32                  # intruder vertical quantization (feet)
    alt_src_hae::Bool              # Flag to indicate whether altitude source is HAE
    nic::UInt32                    # reported NIC value
    nacp::UInt32                   # reported NACp value
    nacv::UInt32                   # reported NACv value
    gva::UInt32                    # reported NACv value
    sil::UInt32                    # reported SIL value
    sda::UInt32                    # reported SDA value
    trk_summary::TrackSummary(p. 495)     # Track summary data structure used in correlation
    toa_hor::R                     # ToA of horizontal track (seconds)
    toa_vert::R                    # ToA of vertical track (seconds)
    toa::R                         # ToA of track file (seconds)
    toa_pos_update::R              # ToA of last position update (seconds)
    display_arrow_current::Z       # Current vertical rate arrow on display
    vert_arrow_history::Vector{Z} # Previous vertical rate arrows sent to display
    lla_at_hor_toa::Vector{R}      # latitude, longitude, altitude at time of horizontal toa (radians)
    ecef_at_hor_toa::Vector{R}     # ECEF position at time of horizontal toa (meters)
    passive_qual_history::Vector{Bool} # History of track passing ADS-B reported quality requirements
    qual_state::Z                  # State of track quality check
    externally_validated::Bool     # Passive intruder has been validated by an external source
    track_uid::String              # Unique track identifier for internal processing
    V2VTrackFile(v2v_uid::UInt32, mode_s::UInt32, mode_s_non_icao::Bool, mode_s_valid::Bool, classification::
        UInt8) = new(
        v2v_uid, mode_s, mode_s_non_icao, mode_s_valid, 0, classification,
        0, zeros(4), zeros(4,4), 0, false,
        zeros(2), zeros(2,2), NaN, 0, 0, false, 0, true,
        0, 0, 0, 0, 0, 0,
        TrackSummary(p. 495)(),
        0.0, 0.0, 0.0, 0.0,
        DISPLAY_ARROW_LEVEL, Vector{Z}(undef, 0),
        zeros(3), zeros(3),
        Bool[], QUAL_STATE_NOT_VALIDATED, false,
        string("V2V",v2v_uid)
    )
end
```

Referenced In: AdvancePassiveTrackState(p. 45), Target(p. 493), InitializeVerticalTracker(p. 51), AssociateV2VToTarget(p. 35), AddHorBias(p. 140), PassiveQualityCheck(p. 127), ExtrapolateTrack(p. 440), AssociateV2VToV2VAndAGTTargets(p. 100), AddAltBiasAndSample(p. 143), ReceiveStateVectorV2VReport(p. 33), AddV2VTrackToDB(p. 35), AddDecorrelatedTrackToTarget(p. 459), AddTracksToReport(p. 119), AdvanceVerticalTrack(p. 53), CorrelationTrackType(p. 449), InitializePassiveTrackFile(p. 17), UpdatePassiveQualityHistory(p. 29), AssessVerticalValidity(p. 123), ExtrapolateV2VTrack(p. 444), GetAltForHorTrkObservation(p. 20)

Type 12 | CorrelationHistory This data structure contains history of correlated/decorrelated tracks to be able to preform an M of N test. It saves the timing interval when the tracks correlated/decorrelated, and saves the IDs of the two tracks as an array of pairs.

```
mutable struct  CorrelationHistory
    timing::Vector{R}
    id_pairs::Vector{Vector{Any}}
    M::Z
    N::Z
    CorrelationHistory(id_type) = new(R[], id_type[], 1, 1)
end
```

Referenced In: OwnShipData(p. 491), CorrelateIndividualTracksToOwnship(p. 454), CorrelateIndividualTracks(p. 462), DecorrelateTargetsFromOwnship(p. 451), CorrelationHistoryCheck(p. 458), CorrelationHistoryUpdate(p. 458), DecorrelateTargets(p. 457)

Type 13 | DesignationState This data structure contains state information associated with target designation. There is one Designation State structure for each Target. Along with specifics about the target designation state, it contains a flag indicating whether the target is currently designated. That flag is used for managing the associated Target. The timer values are in seconds and count up from *0* to the parameterized timer limit for a target to remain designated while designation is invalid due to geometric or data quality constraints. Validity for designation is indicated by the valid[Xo_IDX_DNA] and valid[Xo_IDX_-CSPO3k] flags. The validity values output to the ASA processor are based, in part, on these validity flags. Status information intended for output to the ASA processor via the TRM is indicated in the status field.

```
mutable struct  DesignationState
    # Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1 (insufficient for sXu)
    # mode_s::UInt32                # Mode S address
    # valid::Vector{Bool}           # Valid or invalid for each Xo designation (see Xo_IDX_ constants)
    # active_dna::Bool              # Current Xo Designated No Alerts state (set by TRM)
    # active_protection_mode::UInt8  # Current protection mode (Xu, Xo CSPO-3000, ...)(set by TRM)
    # pending_dna::Bool             # Pending Xo Designated No Alerts state (set by STM)
    # pending_protection_mode::UInt8 # Pending protection mode (Xu, Xo CSPO-3000, ...)(set by STM)
    # timer_dna::Z                  # Timer for invalid timeout for Xo Designated No Alerts
    # timer_protection_mode::Z      # Timer for invalid timeout for Xo protection mode
    # is_designated::Bool           # In use, don't delete target if all tracks are lost
    # was_dropped::Bool             # No output to TRM for processing and no STM display output
    # designated_mode::UInt8        # Designation setting for output
    # status::UInt8                 # Status to be published to ASA processor (see Xo_STATUS_ constants)
    # active_ra::Bool               # This target has an RA active
    # multithreat_ra::Bool          # This target is part of multithreat RA
    # is_airborne::Bool             # DNA: Has gone above upper no aurals threshold
    # is_landing::Bool              # DNA: Has gone below lower no aurals threshold when airborne
    # was_proximate::Bool           # DNA: Whether target has been within 6nmi while designated
    # r_ground_history::Vector{R}   # DNA: Recent history of distances to target while designated (nmi)
    DesignationState() = new()
end
```

Referenced In: Target(p. 493)

Type 14 | OwnDiscreteData This data structure contains all data elements related to certain ownship parameters that change infrequently or only upon initialization. The discrete values are used to set the sensitivity level of ownship. They include the own Mode S address, the Mode A code, a flag to indicate the system is operational, a manually selected sensitivity level, a flag to indicate if the display is allowed on the ground, a flag to inhibit ownship from issuing climb advisories, a flag to inhibit ownship from issuing increase climb advisories, and a flag to indicate if ownship altitude is coarsely quantized at 100 feet.

```
mutable struct  OwnDiscreteData
    v2v_uid::UInt32            # V2V unique identifier of ownship
    opflag::Bool               # system operational
    requested_opmode::UInt8    # Requested operating mode
    turn_rate_limit::R         # Own ship turn rate limit (radians/sec)
    vert_rate_limit::R         # Own ship vertical rate limit (ft/sec)
    prefer_wind_relative::Bool # flag indicating the desire to prefer wind relative horizontal RA's, even
        when WGS84 velocities are available
    perform_poa::Bool          # Perform point-obstacle-avoidance and ground-point-obstacle-avoidance
    disable_gpoa::Bool         # Disable ground-point-obstacle-avoidance
    OwnDiscreteData() = new( 0, true, OPMODE_STANDBY, 0.0, 0.0, false, false, false)
    OwnDiscreteData(v2v_uid::UInt32, opflag::Bool, requested_opmode::UInt8, turn_rate_limit::R,
        vert_rate_limit::R, prefer_wind_relative::Bool, perform_poa::Bool, disable_gpoa::Bool) =
                new(v2v_uid, opflag, requested_opmode, turn_rate_limit, vert_rate_limit,
                    prefer_wind_relative, perform_poa, disable_gpoa)
end
```

Referenced In: OwnShipData(p. 491)

Type 15 | OwnShipData This data structure contains all data elements related to ownship state as derived from several sensor sources. This includes a track of ownship vertical state from barometric pressure altitude measurements, a track of ownship heading, ownship height-AGL, ownship WGS84 position, information related to the advisory modes (RA mode), interrogation mode, current ownship sensitivity level, previous ownship sensitivity level, externally determined sensitivity level, indication if ownship is on the ground, number of invalid height-AGL measurements, the time the report was generated, the threat resolution advisory complement array, the number of display structures, target independent correlation history, target independent decorrelation history, ownship history of heading and pressure altitude and HAE altitude, ownship discrete data, and ownship capability data.

```
mutable struct  OwnShipData
    v2v_uid::UInt32             # Ownship V2V unique identifier
    h_agl::R                    # ownship height above-ground-level (AGL) (feet)
    mu_heading::Vector{R}       # ownship heading track ([radians, radians/second])
    Sigma_heading::Matrix{R}    # ownship heading track covariance
    toa_heading::R              # ownship heading track time of applicability (seconds)
    heading_initialized::Bool   # ownship heading initialized
    heading_state::Z            # ownship heading track state (see constants)
    heading_odc::Z              # ownship heading track outlier detection count
    mu_vert::Array{R}           # ownship pressure altitude track ([feet, feet / second])
    Sigma_vert::Matrix{R}       # ownship pressure altitude track covariance
    odc_vert::Z                 # vertical outlier detection count
    updates_vert::Z             # number of vertical updates
    toa_vert::R                 # ownship pressure altitude track time of applicability (seconds)
    wgs84_lat_deg::R            # ownship WGS84 latitude (degrees)
    wgs84_lon_deg::R            # ownship WGS84 longitude (degrees)
    wgs84_vel_ew_kts::R         # ownship WGS84 east-west velocity (knots)
    wgs84_vel_ns_kts::R         # ownship WGS84 north-south velocity (knots)
    wgs84_alt_hae_ft::R         # ownship WGS84 geometric height above ellipsoid (HAE) altitude (ft)
    wgs84_alt_rate_hae_fps::R   # ownship WGS84 geometric height above ellipsoid (HAE) altitude rate (ft/s)
    wgs84_Sigma_hor::Matrix{R}  # ownship WGS84 horizontal covariance matrix ([ew, dew, ns, dns] as [m, m/s
        ])
    wgs84_toa::R                # ownship WGS84 time of applicability (seconds)
    wgs84_state::Z              # ownship WGS84 state (see constants)
    wgs84_mu_vert::Vector{R}    # vertical track ([feet, feet / second])
    wgs84_Sigma_vert::Matrix{R} # ownship WGS84 vertical covariance matrix ([z, dz] as [m, m/s])
    wgs84_sigma_vepu::R         # ownship WGS84 vertical estimated position uncertainty sigma (ft)
    wgs84_odc_vert::Z           # vertical outlier detection count
    wgs84_updates_vert::Z       # number of vertical updates
    wgs84_toa_vert::R           # ToA of vertical track (seconds)
    prev_rpt_toa::R             # ownship WGS84 previous report time of applicability (seconds)
    trm_velmode::UInt8          # TRM horizontal velocity operational mode setting
    airspeed::R                 # ownship true airspeed used for wind relative coordinate system (knots)
    opmode::UInt8               # ownship operating mode
    on_ground::Bool             # ownship on ground indicator
    invalid_h_agl_cycles::Z     # number of cycles without valid h_agl alt. data
    received_vrcs::Vector{Bool} # threat resolution advisory compl. array
    received_hrcs::Vector{Bool} # threat resolution advisory compl. array
    corr_history::CorrelationHistory(p. 489)       # history of correlated tracks
    decorr_history::CorrelationHistory(p. 489)     # history of decorrelated tracks
    own_corr_history::CorrelationHistory(p. 489)   # history of ownship correlation
    own_decorr_history::CorrelationHistory(p. 489) # history of ownship decorrelation
    trk_summary::TrackSummary(p. 495)               # track summary used in ownship correlation
    geo_states_hae_alt::GeodeticStates(p. 492)     # Ownship geodetic states with HAE altitude
    geo_states_pres_alt::GeodeticStates(p. 492)    # Ownship geodetic states with pressure altitude
    agt_tracks::Vector{AGTTrackFile(p. 486)}       # list of AGTTrackFiles that correlated to ownship
    history::OwnHistory(p. 492)                     # history of heading, pres_alt, and geo_alt observations
    discrete::OwnDiscreteData(p. 490)               # Data structure of discrete inputs
    v2v_osm::V2VOperationalStatusMessage(p. 496)   # Data structure of ownship V2V operational status message
    h_lo_ft::Vector{R}                     # Deviation potential: Descent Inhibit lower threshold (ft)
    h_hi_ft::Vector{R}                     # Deviation potential: Descent Inhibit upper threshold (ft)
    OwnShipData() =
        new(0, NaN, zeros(2), zeros(2,2), NaN, false, OWN_HEADING_INVALID, 0, zeros(2), zeros(2,2), 0, 0,
            NaN, NaN, NaN, NaN, NaN, NaN, NaN, zeros(4,4), NaN, OWN_WGS84_INVALID, zeros(2), zeros(2,2),
            NaN, 0, 0, NaN, NaN, TRM_VELMODE_NONE, NaN, OPMODE_STANDBY, true, 0, fill(false,4), fill(false
                ,2),
            CorrelationHistory(p. 489)(Z), CorrelationHistory(p. 489)(Z), CorrelationHistory(p. 489)(String),
                CorrelationHistory(p. 489)(String),
            TrackSummary(p. 495)(), GeodeticStates(p. 492)(), GeodeticStates(p. 492)(), AGTTrackFile(p. 486)[],
                OwnHistory(p. 492)(), OwnDiscreteData(p. 490)(),
                V2VOperationalStatusMessage(p. 496)(), [-Inf], [-Inf]
        )
```

Type 16 | OwnHistory This data structure contains a history of bearing and pressure/HAE ownship altitude measurements. The values are used for interpolating past ownship information when receiving DF0 messages from the past.

```
mutable struct  OwnHistory
    heading::ValueTime(p. 495)  # own heading observation    ([radians], [seconds])
    pres_alt::ValueTime(p. 495) # own pressure altitude observation ([ft], [ft/seconds])
    hae_alt::ValueTime(p. 495)  # own geodetic HAE observation ([ft], [ft/seconds])
    OwnHistory() = new(
        ValueTime(p. 495)(),
        ValueTime(p. 495)(),
        ValueTime(p. 495)()
    )
end
```

Referenced In: OwnShipData(p. 491)

Type 17 | GeodeticStates This data structure specifies the three dimensional position and velocity of an aircraft. For the position information, the frame used is the World-Geodeic-System 1984 (WGS84). The geodetic position is specified in both Cartesian ECEF or polar LLA states. The velocity information is specified in local ENU-coordinates. Finally, there is a flag to indicated if the altitude is represented as a geodetic height-above-ellipsoid or pressure altitude.

```
mutable struct  GeodeticStates
    lla_rad_ft::Vector{R}
    ecef_m::Vector{R}
    vel_ew_mps::R
    vel_ns_mps::R
    alt_rate_fps::R
    is_alt_geo_hae::Bool
    GeodeticStates() = new(
        fill(NaN,3), fill(NaN,3), NaN, NaN, NaN, true
    )
end
```

Referenced In: OwnShipData(p. 491), PropagateGeodeticToToa(p. 23), PropagateOwnshipToToa(p. 114)

Type 18 | ReceivedCoordinationData This data structure contains coordination data that has been received from a target aircraft. It includes the Cancel Vertical Resolution Advisory Complement (CVC), Vertical Resolution Advisory Complement (VRC), Vertical Sense Bits (VSB), and the time of applicability (TOA).

```
mutable struct  ReceivedCoordinationData
    vrc::UInt32 # Vertical RA complement
    hrc::UInt32 # Horizontal RA complement
    toa_vert::R # Time of Applicability for vertical (seconds)
    toa_hor::R  # Time of Applicability for horizontal (seconds)
    ReceivedCoordinationData() = new( 0, 0, 0.0, 0.0 )
end
```

Referenced In: Target(p. 493)

Type 19 | Target A Target represents all data related to a single physical aircraft that is under surveillance by ACAS X. The Target data structure contains a single Mode S Track File, ADS-B Track File, and ADS-R Track File as well as multiple Mode C Track Files. It also contains the Received Coordination Data from the associated aircraft, Active Validation History information used to validate an ADS-B Track before it is presented to the TRM, the state of the Active Validation, ADS-B Quality History information, a flag to override the ADS-B Quality Check used for Hybrid Surveillance targets, the target designation state, and the time of initialization.

```
mutable struct  Target
    adsb_track::Union{ADSBTrackFile(p. 485), Nothing}
    v2v_track::Union{V2VTrackFile(p. 488), Nothing}
    agt_tracks::Vector{AGTTrackFile(p. 486)}
    ornct_track::Union{ORNCTTrackFile(p. 484), Nothing}
    point_obstacle_track::Union{PointObstacleTrackFile(p. 487), Nothing}
    coord_data::ReceivedCoordinationData(p. 492)
    bad_v2vcoordination_vert::Bool
    bad_v2vcoordination_horz::Bool
    adsb_qual_override::Bool
    adsb_qual_override_toa::R
    designation_state::DesignationState(p. 489)
    init_time::R # seconds
    priority_codes::PriorityCodes(p. 494)
    adsb_osm::ADSBOperationalStatusMessage(p. 496)
    v2v_osm::V2VOperationalStatusMessage(p. 496)
    optimal_trk_history::Vector{String}
    previously_selected_trk_uid::Union{String, Nothing}
    Target() = new(
        nothing, nothing,
        AGTTrackFile(p. 486)[], nothing,
        nothing, ReceivedCoordinationData(p. 492)(),
        false, false,
        false, 0.0,
        DesignationState(p. 489)(), -1.0,
        PriorityCodes(p. 494)(), ADSBOperationalStatusMessage(p. 496)(),
        V2VOperationalStatusMessage(p. 496)(), String[],
        nothing
    )
    Target( mode_s::UInt32 ) = new(
        nothing, nothing,
        AGTTrackFile(p. 486)[], nothing,
        nothing, ReceivedCoordinationData(p. 492)(),
        false, false,
        false, 0.0,
        DesignationState(p. 489)(), -1.0,
        PriorityCodes(p. 494)(), ADSBOperationalStatusMessage(p. 496)(),
        V2VOperationalStatusMessage(p. 496)(), String[],
        nothing
    )
end
```

Referenced In: RemoveDecorrelatedTrackFromTarget(p. 459), AddPointObstacleTrackToDB(p. 79), AddORNCTTrackToDB(p. 64), RemoveStaleV2VTrack(p. 161), RemoveStaleADSBTrack(p. 160), DecorrelateTargetsFromOwnship(p. 451), GetAircraftTracks(p. 440), AddV2VTrackToDB(p. 35), AddAGTTrackToDB(p. 72), StmHousekeepingPrioritization(p. 438), AddDecorrelatedTrackToTarget(p. 459), GetTracksToDecorrelate(p. 458), RemoveStaleORNCTTrack(p. 160), AddADSBTrackToDB(p. 18), DecorrelateTargets(p. 457), SetIntruderDegradedFlags(p. 138), GetTracksToExtrapolate(p. 440), SetIntruderIDDirectory(p. 137), GetPreviouslySelectedTrack(p. 130), DetermineORNCTDeletion(p. 62), TargetIsEmpty(p. 161), CoordinationTimeoutCheck(p. 102), AGTOnlyTarget(p. 453)

Type 20 | Database A Database acts as a wrapper around the Julia Dictionary type (Dict) with the addition of an increment field. The increment is used as a deterministic internal ID for new entries into the dictionary.

```
mutable struct  Database{K,V}
    increment::K
    dictionary::Dict{K,V}
    Database{K,V}(i::K) where {K,V} = new(i, Dict{K,V}())
end
Base.delete!(db::Database, k)      = Base.delete!(db.dictionary, k)
Base.get(db::Database, k, default) = Base.get(db.dictionary, k, default)
Base.getindex(db::Database, k)     = Base.getindex(db.dictionary, k)
Base.haskey(db::Database, k)       = Base.haskey(db.dictionary, k)
Base.keys(db::Database)            = Base.sort(Base.collect(Base.keys(db.dictionary)))
Base.setindex!(db::Database, v, k) = Base.setindex!(db.dictionary, v, k)
```

Referenced In: AddToDB(p. 19), RetrieveWithID(p. 19)

Type 21 | PriorityCodes The Priority Codes store the intruder code values for the vertical and horizontal dimension as well as an indication of whether the intruder ever had a code in the past, and the highest code that the intruder had.

```
mutable struct  PriorityCodes
    vert::UInt8
    horiz::UInt8
    had_code::Bool
    highest_code::UInt8
    PriorityCodes() = new(
        SXUCODE_CLEAR,
        SXUCODE_CLEAR,
        true,
        SXUCODE_CLEAR,
    )
end
```

Referenced In: Target(p. 493)

Type 22 | TrackMap A Track Map associates a Track File with an ID in the Target Database. The purpose is to organize which Target the specified Track belongs to during the correlation process.

```
mutable struct  TrackMap
    track::Union{TrackFile(p. 483), Nothing}
    db_id::UInt32
end
```

Referenced In: SortByRange(p. 461), DecorrelateTargets(p. 457), CorrelateTargetsToOwnship(p. 452), CorrelateTargets(p. 461), GenerateLeadTrackList(p. 460)

Type 23 | TrackSummary This data structure is used by various surveillance sources as a common container for information pertaining to correlation processing. It hosts three tracks, one for the vertical dimension and two for the horizontal dimension. Each track has a vector to represent its state and a covariance matrix to represent its uncertainty. The ground range track and the vertical track each have a validity flag. There is a field storing the type designation of the track for correlation purposes.

```
mutable struct  TrackSummary
    toa::R                  # ToA (seconds)
    mu_vert::Vector{R}      # Vertical track ([feet, feet/second])
    mu_rng_az::Vector{R}    # Polar track ([feet, radians])
    mu_range::Vector{R}     # Ground range track ([feet, feet/second])
    Sigma_vert::Matrix{R}   # Vertical track covariance matrix
    Sigma_rng_az::Matrix{R} # Polar track covariance matrix
    Sigma_range::Matrix{R}  # Ground range track covariance matrix
    valid_rng::Bool         # Ground range track validity
    valid_vert::Bool        # Vertical track validity
    alt_src_hae::Bool       # Flag to indicate whether altitude source is HAE
    type::UInt8             # Value indicating the track type for the purpose of correlation
    TrackSummary() = new(
        0,
        zeros(2),
        zeros(2),
        zeros(2),
        zeros(2,2),
        zeros(2,2),
        zeros(2,2),
        false,
        false,
        false,
        CORRELATION_TRKTYPE_INVALID
    )
end
```

Referenced In: AGTTrackFile(p. 486), ExtrapolateADSBTrack(p. 443), OwnShipData(p. 491), ExtrapolateAGTTrack(p. 447), ADSBTrackFile(p. 485), ExtrapolateORNCTTrack(p. 445), V2VTrackFile(p. 488), ExtrapolateV2VTrack(p. 444), ORNCTTrackFile(p. 484), ExtrapolateOwnshipTrack(p. 442)

Type 24 | ValueTime A ValueTime pair stores arrays of specified values with associated times. It is used as a means of storing ownship history.

```
mutable struct  ValueTime
    value::Vector{R}
    time::Vector{R}
    ValueTime() = new(
        R[],
        R[]
    )
end
```

Referenced In: ReceivePresAltObservation(p. 82), OwnHistory(p. 492), UpdateHistory(p. 83), OwnWgs84Timeout(p. 112)

Type 25 | ADSBOperationalStatusMessage This data structure contains the data elements that comprise a ADS-B Operational Status Message. It includes the Collision avoidance operational bit field, the Collision Avoidance Coordination Capability Bits, and the Detect-and-avoid bit field. This information is used to store information about intruders that will determine how or if ownship is capable of coordinating maneuvers. This structure stores information about intruders used to determine whether or how the ownship may coordinate maneuvers with an intruder.

```
mutable struct  ADSBOperationalStatusMessage
    ca_operational::UInt8 # Collision avoidance operational bit field
    adsb_cccb::CCCB(p. 509)  # Collision Avoidance Coordination Capability Bits transmitted through ADS-B
    daa::UInt8            # Detect-and-avoid bit field
    ADSBOperationalStatusMessage() = new( CAS_ACTIVE_TCAS, CCCB(p. 509)(CAS_SENSE_BLENDED, CAS_ACTIVE_TCAS,
        UInt8(0)), DAA_RCV_NONE )
end
```

Referenced In: Target(p. 493)

Type 26 | V2VOperationalStatusMessage This data structure contains the data elements that comprise a V2V Operational Status Message. It includes CA status indicator, sense, CAS type capability, priority, and aircraft equipment and classification. This structure stores information about intruders, information which is used, first, to determine whether or how the ownship may coordinate maneuvers with an intruder, and second, to compose the ownship's outgoing V2V operational status message.

```
mutable struct  V2VOperationalStatusMessage
    ca_status::UInt8            # Collision avoidance status
    v2v_cccb::CCCB(p. 509)         # Collision Avoidance Coordination Capability Bits transmitted through V2V
    equipment::UInt16           # Aircraft equipment
    V2VOperationalStatusMessage() = new( CA_STATUS_NONE, CCCB(p. 509)(CAS_SENSE_BLENDED, CAS_V2V_NONE, UInt8(0)),
          EQUIPMENT_NONE )
end
```

Referenced In: OwnShipData(p. 491), Target(p. 493), StmReport(p. 480)

D.3 STM Output-TRM Input Data Structures

These data structures contain information generated by the STM and consumed by the TRM. The TRM input data structure is contained within the STM Report data structure, produced by the STM.

Type 27 | TRMInput This data structure is generated as part of the STM Report and gets directly processed by the TRM for threat evaluation. It includes information about ownship, including an estimate of ownship heading, an estimate of ownship height-AGL, and a distribution of ownship vertical state (derived from pressure/HAE altitude). It also includes a collection of data structures that represent the tracked state of nearby intruding aircraft.

```
mutable struct  TRMInput
    own::TRMOwnInput(p. 498)                  # Input data for own aircraft
    intruder::Vector{TRMIntruderInput(p. 499)} # Input data for each intruder
#
    TRMInput() = new( TRMOwnInput(p. 498)(), [] )
    TRMInput( own::TRMOwnInput(p. 498), intruder::Vector{TRMIntruderInput(p. 499)} ) = new( own, intruder )
end
```

Referenced In: GenerateBlendedsXuTRMOutput(p. 396), HorizontalTRMUpdate(p. 168), VerticalTRMUpdate(p. 241), VerticalTRMUpdatePrep(p. 587), HorizontalTRMUpdatePrep(p. 589), SetThreatIdentityData(p. 401), SetBlendedIntruderThreatData(p. 400), sXuTRMBlendedReportPrep(p. 397), sXuTRMUpdate(p. 164), StmReport(p. 480)

Type 28 | TRMOwnInput This data structure is generated as part of the STM Report and represents the state of ownship. It includes height-AGL and heading, identifying information needed for the TRM outputs, operational mode (as sensitivity level), surveillance quality, and availability of Xo modes. The estimated vertical state of ownship is represented by a collection of own vertical beliefs.

```
mutable struct  TRMOwnInput
    h::R                              # Height above ground level (ft)
    psi::R                            # Heading (radians)
    track_angle::R                    # Track angle (radians)
    v2v_uid::UInt32                   # Ownship V2V unique identifier
    agt_ids::Vector{UInt32}           # Ownship-associated AGT IDs
    opmode::UInt8                     # Ownship vertical operational mode (See TRM_OPMODE constants)
    h_opmode::UInt8                   # Ownship horizontal operational mode (See TRM_OPMODE constants)
    trm_velmode::UInt8                # TRM horizontal velocity operational mode setting
    trm_altmode::UInt8                # TRM altitude type operational mode setting
    perform_gpoa::Bool                # Perform ground-point-obstacle-avoidance
    degraded_own_surveillance::UInt16 # Bit vector indicating quality of surveillance
                                      #  (See DEGRADED_OWN_ constants)
    xo_availability::Vector{UInt8}    # Availability of Xo modes (see Xo_AVAILABILITY constants)
    belief_vert::Vector{OwnVerticalBelief(p. 501)}  # Own ship vertical beliefs (z, dz)
    ground_speed::R                   # Own ship ground speed (ft/s)
    airspeed::R                       # Own ship true airspeed (knots)
    turn_rate_limit::R                # Own ship turn rate limit (radians/sec)
    vert_rate_limit::R                # Own ship vertical rate limit (ft/sec)
    h_lo_ft::Vector{R}                # Deviation potential: Descent Inhibit lower threshold (ft)
    h_hi_ft::Vector{R}                # Deviation potential: Descent Inhibit upper threshold (ft)
#
    TRMOwnInput( h::R, psi::R, track_angle::R, v2v_uid::UInt32, agt_ids::Vector{UInt32}, opmode::UInt8,
                 h_opmode::UInt8, trm_velmode::UInt8, trm_altmode::UInt8, perform_gpoa::Bool,
                 degraded_own_surveillance::UInt16, xo_availability::Vector{UInt8},
                 belief_vert::Vector{OwnVerticalBelief(p. 501)}, ground_speed::R, airspeed::R,
                 turn_rate_limit::R, vert_rate_limit::R ) =
        new( h, psi, track_angle, v2v_uid, agt_ids, opmode, h_opmode, trm_velmode, trm_altmode,
             perform_gpoa, degraded_own_surveillance, xo_availability, belief_vert,
             ground_speed, airspeed, turn_rate_limit, vert_rate_limit, [-Inf], [-Inf] )
    TRMOwnInput( h::R, psi::R, track_angle::R, v2v_uid::UInt32, agt_ids::Vector{UInt32}, opmode::UInt8,
                 belief_vert::Vector{OwnVerticalBelief(p. 501)},
                 ground_speed::R, airspeed::R, turn_rate_limit::R, vert_rate_limit::R ) =
        new( h, psi, track_angle, v2v_uid, opmode, opmode, TRM_VELMODE_NONE, TRM_ALTMODE_NONE, false,
             DEGRADED_OWN_NONE, fill(Xo_AVAILABILITY_NOT_CONFIGURED,2), belief_vert,
             ground_speed, airspeed, turn_rate_limit, vert_rate_limit, [-Inf], [-Inf] )
    TRMOwnInput() =
        new( NaN, NaN, NaN, 0, UInt32[], UInt8(OPMODE_RA), UInt8(OPMODE_RA), TRM_VELMODE_NONE,
             TRM_ALTMODE_NONE, false, DEGRADED_OWN_NONE, fill(Xo_AVAILABILITY_NOT_CONFIGURED,2),
             OwnVerticalBelief(p. 501)[], NaN, NaN, NaN, NaN, [-Inf], [-Inf] )
end
```

Referenced In: ResetHTRMState(p. 174), SetHorizontalTRMSurvOnlyMode(p. 169), UpdateIntruderInputs(p. 307), VerticalTRMUpdate(p. 241), TRMInput(p. 497), UpdateHTRMIntruderInputs(p. 173), HorizontalStateEstimation(p. 175), StateAndCostEstimation(p. 251), GetVTRMOwnData(p. 243), IntruderPrep(p. 254), GenerateHTRMOutput(p. 234), SetVerticalTRMSurvOnlyMode(p. 247), GenerateVTRMOutput(p. 376), IsIntruderMaster(p. 424), SetHorizontalIntruderOutput(p. 235), SetHorizontalDroppedIntruderOutput(p. 236)

Type 29 | TRMIntruderInput This data structure is generated as part of the STM Report and represents the tracked state of a nearby intruding aircraft. It includes information for identifying the intruder, coordination information, surveillance type and quality, Xo mode information, and the intruder's vertical and horizontal position and velocity as belief states. The STM display output is included for use in forming resolution advisory messages for the ground station.

```
mutable struct  TRMIntruderInput
    valid::Bool                     # Data is valid for processing
    id::UInt32                      # Unique target identifier
    id_directory::IntruderIDDirectory(p. 482) # All external IDs assigned to the intruder
    vrc::UInt32                     # VRC received from this intruder
    hrc::UInt32                     # HRC received from this intruder
    equipage::Z                     # Intruder equipage (See EQUIPAGE_ constants)
    coordination_msg::UInt32        # Coordination message type (see COORDINATION_ constants)
    source::Z                       # Surveillance source (See SOURCE_ constants)
    own_coordination_policy::UInt8 # Ownship coordination policy for this intruder (See
        OWN_COORDINATION_POLICY_ constants)
    degraded_surveillance::UInt16  # Bit vector indicating quality of surveillance
    designated_mode::UInt8          # Selected designation, may not be active (See DESIGNATION_ constants)
    protection_mode::UInt8          # Protection mode to be applied (See PROTECTION_MODE_ constants)
    dna::Bool                       # Designated for Xo Designated No Alerts (DNA) processing, or not
    xo_valid::Vector{Bool}          # Validity for Xo designations (initial settings for output)
    xo_status::UInt8                # Xo designation status information
                                    #    (See Xo_STATUS_ constants)
    stm_display::StmDisplayStruct(p. 481)  # Target display information output by the STM
    classification::UInt8              # Intruder classification (See CLASSIFICATION_ constants)
    belief_vert::Vector{IntruderVerticalBelief(p. 500)}     # Vertical beliefs (z, dz)
    belief_horiz::Vector{IntruderHorizontalBelief(p. 500)} # Horizontal beliefs (relative x, y, dx, dy)
    adsb_cccb::CCCB(p. 509)            # Collision Avoidance Coordination Capability Bits over ADS-B
    v2v_cccb::CCCB(p. 509)             # Collision Avoidance Coordination Capability Bits over V2V
#
    TRMIntruderInput( valid::Bool, id::UInt32, id_directory::IntruderIDDirectory(p. 482), vrc::UInt32,
        hrc::UInt32, equipage::Z, coordination_msg::UInt32, source::Z, own_coordination_policy::UInt8,
        degraded_surveillance::UInt16, protection_mode::UInt8, stm_display_struct::StmDisplayStruct(p. 481),
        classification::UInt8, belief_vert::Vector{IntruderVerticalBelief(p. 500)},
        belief_horiz::Vector{IntruderHorizontalBelief(p. 500)}, adsb_cccb::CCCB(p. 509), v2v_cccb::CCCB(p. 509) ) =
        new( valid, id, id_directory, vrc, hrc, equipage, coordination_msg, source, own_coordination_policy,
             degraded_surveillance, DESIGNATION_NONE, PROTECTION_MODE_sXu, false, fill( false, 2 ),
             Xo_STATUS_NONE, stm_display_struct, classification, belief_vert, belief_horiz, adsb_cccb,
             v2v_cccb )
    TRMIntruderInput() = new( false, 0, IntruderIDDirectory(p. 482)(), 0, 0, EQUIPAGE_NONE, COORDINATION_NONE,
        SOURCE_MODES, OWN_COORDINATION_POLICY_NONE, DEGRADED_SURVEILLANCE_NONE, DESIGNATION_NONE,
        PROTECTION_MODE_sXu, false, fill( false, 2 ), Xo_STATUS_NONE, StmDisplayStruct(p. 481)(),
        CLASSIFICATION_NONE, IntruderVerticalBelief(p. 500)[], IntruderHorizontalBelief(p. 500)[],
        CCCB(p. 509)(CAS_SENSE_BLENDED, CAS_ACTIVE_TCAS, UInt8(0)),
        CCCB(p. 509)(CAS_SENSE_BLENDED, CAS_V2V_NONE, UInt8(0)) )
    TRMIntruderInput( id::UInt32, id_directory::IntruderIDDirectory(p. 482), degraded_surveillance::UInt16 ) =
    new( false, id, id_directory, 0, 0, EQUIPAGE_NONE, COORDINATION_NONE, SOURCE_MODES,
         OWN_COORDINATION_POLICY_NONE, degraded_surveillance, DESIGNATION_NONE, PROTECTION_MODE_sXu,
         false, fill( false, 2 ), Xo_STATUS_NONE, StmDisplayStruct(p. 481)(), CLASSIFICATION_NONE,
         IntruderVerticalBelief(p. 500)[], IntruderHorizontalBelief(p. 500)[],
         CCCB(p. 509)(CAS_SENSE_BLENDED, CAS_ACTIVE_TCAS, UInt8(0)),
         CCCB(p. 509)(CAS_SENSE_BLENDED, CAS_V2V_NONE, UInt8(0)) )
    TRMIntruderInput( id::UInt32, mode_s::UInt32, is_icao::Bool, degraded_surveillance::UInt16 ) =
    new( false, id, IntruderIDDirectory(p. 482)(mode_s, is_icao), 0, 0, EQUIPAGE_ATCRBS, ACTIVE_CAS_NONE,
         COORDINATION_NONE, SOURCE_MODES, degraded_surveillance, DESIGNATION_NONE, PROTECTION_MODE_sXu,
         false, fill( false, 2 ), Xo_STATUS_NONE, StmDisplayStruct(p. 481)(), CLASSIFICATION_NONE,
         IntruderVerticalBelief(p. 500)[], IntruderHorizontalBelief(p. 500)[], CCCB(p. 509)() )
    TRMIntruderInput( id::UInt32, mode_s::UInt32, degraded_surveillance::UInt16 ) =
        TRMIntruderInput( id, mode_s, false, degraded_surveillance )
end
```

Referenced In: SetInvalidIntruder(p. 395), SetHorizontalTRMSurvOnlyMode(p. 169), UpdateIntruderInputs(p. 307), HorizontalTRMUpdate(p. 168), VerticalTRMUpdate(p. 241), TRMInput(p. 497), UpdateHTRMIntruderInputs(p. 173), HorizontalStateEstimation(p. 175), DetermineTTIAndTID(p. 387), VerticalTRMUpdatePrep(p. 587), StateAndCostEstimation(p. 251), GenerateRAModeOutput(p. 378), SetIntruderVerticalScores(p. 372), HorizontalTRMUpdatePrep(p. 589), AdjustRAModeIntruderOutput(p. 379), UpdateIntruderVRC(p. 101), SetVerticalRAMessageOutput(p. 386), SetRABroadcastData(p. 391), PrioritizeAndFilterIntruders(p. 171), IntruderPrep(p. 254), GenerateHTRMOutput(p. 234), CreateGroundIntruder(p. 242), SetVerticalTRMSurvOnlyMode(p. 247), DroppedIntrudersAdjustment(p. 393), GenerateVTRMOutput(p. 376), IsIntruderMaster(p. 424), GenerateSurvOnlyModeOutput(p. 382), GenerateStandbyModeOutput(p. 383), SetHorizontalIntruderOutput(p. 235), UpdateIntruderHRC(p. 102), SetIntruderBeliefStates(p. 144), SetClassification(p. 136), AddADSBTrackToReport(p. 135), SetDisplayDataORNCT(p. 153), SetCoordination(p. 104), SetDisplayDataPassive(p. 146), AddPointOb-

Type 30 | IntruderHorizontalBelief This data structure contains the data elements that represent a weighted sample from a four state (x and y position and velocity) distribution in the horizontal dimension.

```
mutable struct  IntruderHorizontalBelief
    x_rel::R    # E/W component of position relative to own ship (ft)
    y_rel::R    # N/S component of position relative to own ship (ft)
    dx_rel::R   # E/W component of velocity relative to own ship (ft/s)
    dy_rel::R   # N/S component of velocity relative to own ship (ft/s)
    weight::R   # weight of this sample [0-1]
#
    IntruderHorizontalBelief( x_rel::R, y_rel::R, dx_rel::R, dy_rel::R, w::R ) =
        new(  x_rel, y_rel, dx_rel, dy_rel, w )
    IntruderHorizontalBelief() = new( 0.0, 0.0, 0.0, 0.0, 0.0 )
end
```

Referenced In: TRMIntruderInput(p. 499), PointObstacleEnuBeliefEstimation(p. 180), ConvertHorizontal(p. 258), FindMaximumWeightAndIndex(p. 182), StateAndCostEstimation(p. 251), ConvertIntruderInputsToEnuBeliefs(p. 179), CreateGroundIntruder(p. 242), EnuBeliefEstimation(p. 181), StateEstimation(p. 257), SetIntruderBeliefStates(p. 144), AddPointObstacleTrackToReport(p. 157), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

Type 31 | IntruderVerticalBelief This data structure contains the data elements of the intruder that represent a weighted sample from a two state (position and velocity) distribution in the vertical dimension. A collection of vertical samples are used to represent the entire distribution.

```
mutable struct  IntruderVerticalBelief
    z::R         # Pressure/HAE altitude (ft)
    dz::R        # Vertical rate (ft/s)
    weight::R    # Weight for this sample [0-1]
#
    IntruderVerticalBelief( z::R, dz::R, weight::R ) = new( z, dz, weight )
    IntruderVerticalBelief() = new( 0.0, 0.0, 0.0 )
end
```

Referenced In: TRMIntruderInput(p. 499), PointObstacleEnuBeliefEstimation(p. 180), DetermineAltitudeDependentCOCFactor(p. 273), DetermineTauVerticalDistribution(p. 265), CombineVerticalBeliefs(p. 270), FindMaximumWeightAndIndex(p. 182), StateAndCostEstimation(p. 251), ConvertIntruderInputsToEnuBeliefs(p. 179), IntruderPrep(p. 254), SetBeliefBasedOnlineCostState(p. 271), CreateGroundIntruder(p. 242), SetVerticalTRMSurvOnlyMode(p. 247), EnuBeliefEstimation(p. 181), StateEstimation(p. 257), TauEstimation(p. 263), SetIntruderBeliefStates(p. 144), AddPointObstacleTrackToReport(p. 157), AddAGTTrackToReport(p. 155), AddV2VTrackToReport(p. 150)

Type 32 | OwnVerticalBelief This data structure contains the data elements of the ownship that represent a weighted sample from a two state (position and velocity) distribution in the vertical dimension. A collection of vertical samples are used to represent the entire distribution.

```
mutable struct  OwnVerticalBelief
    z::R         # Pressure/HAE altitude (ft)
    dz::R        # Vertical rate (ft/s)
    weight::R    # Weight for this sample [0-1]
#
    OwnVerticalBelief( z::R, dz::R, weight::R ) = new( z, dz, weight )
    OwnVerticalBelief() = new( 0.0, 0.0, 0.0 )
end
```

Referenced In: DetermineTauVerticalDistribution(p. 265), CombineVerticalBeliefs(p. 270), TRMOwnInput(p. 498), ConvertOwnInputsToEnuBeliefsWindRelative(p. 178), ConvertIntruderInputsToEnuBeliefs(p. 179), ConvertOwnInputsToEnuBeliefsGroundAndCompassMix(p. 177), ConvertOwnInputsToEnuBeliefs(p. 176), StateEstimation(p. 257), GetOwnWeightedAverages(p. 243), TauEstimation(p. 263), SetOwnshipData(p. 116)

D.4 TRM Output-STM Input Data Structures

These data structures contain information produced by the TRM for the onboard pilot display and coordination. The TRM Report is also provided to the STM housekeeping function. STM housekeeping for target designation uses the relevant fields in the TRM intruder display data structure.

Type 33 | sXuTRMReport This data structure is produced by the TRM at the end of each processing cycle. It contains information for the horizontal and vertical onboard pilot display, coordination, designation, and alerts.

```
mutable struct  sXuTRMReport
    display_vert::VTRMDisplayData(p. 502)          # Data for the onboard pilot display
    display_horiz::HTRMDisplayData(p. 503)         # Data for the onboard pilot display
    coordination::Vector{TRMCoordinationInterrogationData(p. 504)}
                                                  # Per intruder data for outgoing coordination messages
    designation::TRMDesignationData(p. 505)       # Target designation data for ASA system
    broadcast_msg::sXuTRMBroadcastData(p. 507)    # Data for sXu broadcast
    alerting_data::TRMAlertingData(p. 506)        # Aggregated alerting indicators
#
    sXuTRMReport() = new( VTRMDisplayData(p. 502)(),
                          HTRMDisplayData(p. 503)(),
                          TRMCoordinationInterrogationData(p. 504)[],
                          TRMDesignationData(p. 505)(),
                          sXuTRMBroadcastData(p. 507)(),
                          TRMAlertingData(p. 506)())
end
```

Referenced In: SetReportRATerminationBits(p. 404), SetPrevHorizontalTerminationData(p. 406), GenerateBlendedsXuTRMOutput(p. 396), UpdateRaPubState(p. 398), SetPrevVerticalTerminationData(p. 405), SetThreatIdentityData(p. 401), SetBlendedCoordinationData(p. 409), DetermineMultidimensionalMultithreat(p. 408), SetBlendedIntruderThreatData(p. 400), StmHousekeepingTargetDesignation(p. 609), SetReportDataOnRATerm(p. 403), sXuTRMBlendedReportPrep(p. 397), sXuTRMUpdate(p. 164), StmHousekeeping(p. 437), StmHousekeepingPrioritization(p. 438)

Type 34 | VTRMDisplayData This data structure is generated as part of the vertical TRM Report. It contains information for the onboard pilot display evaluated by the TRM.

```
mutable struct  VTRMDisplayData
    cc::UInt8                   # combined control (label270)
    vc::UInt8                   # vertical control (label270)
    ua::UInt8                   # up advisory (label270)
    da::UInt8                   # down advisory (label270)
    target_rate::R              # vertical rate to target (ft/s)
    crossing::Bool              # crossing alert flag
    alarm::Bool                 # annunciate RA flag
    ground_alert::UInt8         # ground alert type
    dz_min::R                   # Minimum vertical rate associated with selected global action (ft/min)
    dz_max::R                   # Maximum vertical rate associated with selected global action (ft/min)
    ddz_min::R                  # Compliant acceleration associated with selected global action (ft/s^2)
    strength::UInt8             # ARA strength bits setting representing display output
    down::Bool                  # ARA down/up bit setting representing display output
    opmode::UInt8               # Ownship vertical operational mode (See TRM_OPMODE constants)
    trm_altmode::UInt8          # TRM altitude type operational mode setting
    intruder::Vector{TRMIntruderDisplayData(p. 503)}
                                # Per intruder information for the display
#
    VTRMDisplayData() =
        new( 0, 0, 0, 0, 0.0, false, false, GROUND_ALERT_NONE, -Inf, Inf, 0.0, UInt8(0), false,
             OPMODE_STANDBY, TRM_ALTMODE_NONE, TRMIntruderDisplayData(p. 503)[] )
    VTRMDisplayData( display::DisplayLogic(p. 516) ) = new(
                         display.cc, display.vc, display.ua, display.da,
                         display.target_rate, display.crossing,
                         display.alarm, display.ground_alert, display.dz_min, display.dz_max,
                         display.ddz_min, display.strength, display.down, display.opmode,
                         display.trm_altmode, TRMIntruderDisplayData(p. 503)[] )
    VTRMDisplayData( display::DisplayLogic(p. 516), intruders::Vector{TRMIntruderDisplayData(p. 503)} ) =
        new( display.cc, display.vc, display.ua, display.da,
             display.target_rate, display.crossing,
             display.alarm, display.ground_alert, display.dz_min, display.dz_max, display.ddz_min,
             display.strength, display.down, display.opmode, display.trm_altmode,
             deepcopy( intruders ))
end
```

Referenced In: VTRMReport(p. 524), sXuTRMReport(p. 501)

Type 35 | TRMIntruderDisplayData This data structure is generated as part of the TRM Report. It contains information for the onboard pilot display about an individual intruding aircraft evaluated by the TRM. The track display score is used for prioritizing intruding aircraft for filtering and display. The track display advisory code is used for determining the color and fill of the intruding aircraft icon.

```
mutable struct  TRMIntruderDisplayData
    id::UInt32                # Unique identifier
    id_directory::IntruderIDDirectory(p. 482) # All external IDs assigned to the intruder
    tds::R                    # Track display score; used for display prioritization
    code::UInt8               # Track display advisory code, corresponds to SXUCODE_ inside VTRM,
                              # SXUCODE_ after VerticalDAAUpdate
    classification::UInt8     # Intruder classification (See CLASSIFICATION_ constants)
    source::Z                 # Surveillance source (See SOURCE_ constants)
#
    TRMIntruderDisplayData() =
        new( 0, IntruderIDDirectory(p. 482)(), 0.0, SXUCODE_CLEAR, CLASSIFICATION_NONE, SOURCE_MODES )
    TRMIntruderDisplayData( id::UInt32, id_directory::IntruderIDDirectory(p. 482), tds::R, code::UInt8,
                            classification::UInt8, source::Z ) =
        new( id, id_directory, tds, code, classification, source )
end
```

Referenced In: VTRMDisplayData(p. 502), UpdateRaPubState(p. 398), TRMIntruderData(p. 523), DetermineMultidimensionalMultithreat(p. 408), StmHousekeepingPrioritization(p. 438)

Type 36 | HTRMDisplayData This data structure is generated as part of the horizontal TRM Report. It contains information for the onboard pilot display evaluated by the TRM.

```
mutable struct  HTRMDisplayData
    cc::UInt8                              # combined control (RWC/CA and strength) (label271)
    target_angle::R                        # heading_angle or track_angle to target advisory (deg from North)
    wind_relative::Bool                    # operating in wind relative mode
                                           # True = target_anlge based on heading,
                                           # False = target_angle based on track angle
    h_opmode::UInt8                        # Ownship horizontal operational mode (See TRM_OPMODE constants)
    trm_velmode::UInt8                     # TRM horizontal velocity operational mode setting
    horizontal_desensitivity_mode::UInt8   # Mode variable indicating sensitivity or
                                           # desensitivity of RA selection regarding horiz TRM's online costs
    intruder::Vector{HTRMIntruderDisplayData(p. 504)}
                                           # Per intruder information for the display
#
    HTRMDisplayData() =
        new( 0, NaN,  false, OPMODE_STANDBY, TRM_VELMODE_NONE, HORIZONTAL_DESENSITIVITY_NONE,
             HTRMIntruderDisplayData(p. 504)[] )
    HTRMDisplayData(cc::UInt8, target_track_angle::R, wind_relative::Bool, h_opmode::UInt8,
                    trm_velmode::UInt8, horizontal_desensitivity_mode::UInt8 ) =
        new( cc, target_track_angle, wind_relative, h_opmode, HTRMIntruderDisplayData(p. 504)[] )
    HTRMDisplayData( display::HorizontalDisplayLogic(p. 518),
                     intruders::Vector{HTRMIntruderDisplayData(p. 504)} ) =
        new( display.cc, display.target_angle, display.wind_relative, display.h_opmode, display.trm_velmode,
             display.horizontal_desensitivity_mode, intruders )
end
```

Referenced In: HTRMReport(p. 519), GenerateHTRMOutput(p. 234), sXuTRMReport(p. 501)

Type 37 | HTRMIntruderDisplayData This data structure is generated as part of the horizontal TRM Report. It contains information for the onboard pilot display about an individual intruding aircraft evaluated by the TRM. The track display score is used for prioritizing intruding aircraft for filtering and display. The track display advisory code is used for determining the color and fill of the intruding aircraft icon.

```
mutable struct  HTRMIntruderDisplayData
    id::UInt32                              # Unique identifier
    id_directory::IntruderIDDirectory(p. 482)  # All external IDs assigned to the intruder
    tds::R                                  # Track display score; used for display prioritization
    code::UInt8                             # Horizontal advisory code (see SXUCODE_* constants)
    classification::UInt8                   # Intruder classification (See CLASSIFICATION_ constants)
    source::Z                               # Surveillance source (See SOURCE_ constants)
#
    HTRMIntruderDisplayData() =
        new( 0, IntruderIDDirectory(p. 482)(),  0.0, SXUCODE_CLEAR, CLASSIFICATION_NONE, SOURCE_MODES )
    HTRMIntruderDisplayData( id::UInt32, id_directory::IntruderIDDirectory(p. 482), tds::R, code::UInt8,
                             classification::UInt8, source::Z ) =
        new( id, id_directory, tds, code, classification, source )
end
```

Referenced In: HTRMDisplayData(p. 503), HTRMReport(p. 519), UpdateRaPubState(p. 398), DetermineMultidimensionalMultithreat(p. 408), GenerateHTRMOutput(p. 234), SetHorizontalIntruderOutput(p. 235), SetHorizontalDroppedIntruderOutput(p. 236), StmHousekeepingPrioritization(p. 438)

Type 38 | TRMCoordinationInterrogationData This data structure is generated as part of the TRM Report. It contains coordination information for output to individual aircraft equipped with TCAS, CAS Responsive, or Detect And Avoid (DAA) Responsive systems.

```
mutable struct  TRMCoordinationInterrogationData
    id::UInt32                    # Unique identifier for intruder
    cvc::UInt32                   # Cancel Vertical RA Complement
    vrc::UInt32                   # Vertical RA complement
    vsb::UInt32                   # Vertical Sense Bits (parity check)
    chc::UInt32                   # Cancel Horizontal RA complement
    hrc::UInt32                   # Horizontal RA complement
    hsb::UInt32                   # Horizontal Sense Bits (parity check)
    mtb::Bool                     # multiple threats for this RA
    mid::UInt32                   # Ownship V2V unique identifier (sender)
    icao24::IntruderIDEntry(p. 482)  # Intruder 24-bit ICAO address (intended receiver)
    anon24::IntruderIDEntry(p. 482)  # Intruder 24-bit anonymous address (intended receiver)
    v2v_uid::IntruderIDEntry(p. 482) # Intruder V2V unique ID (intended receiver)
    coordination_msg::UInt32      # Coordination mechanism (see COORDINATION_ constants)
#
    TRMCoordinationInterrogationData() =
            new( 0, 0, 0, 0, 0, 0, 0, false, 0, IntruderIDEntry(p. 482)(), IntruderIDEntry(p. 482)(),
                 IntruderIDEntry(p. 482)(), COORDINATION_NONE )
    TRMCoordinationInterrogationData( id::UInt32, cvc::UInt32, vrc::UInt32, vsb::UInt32, mtb::Bool,
        mid::UInt32, id_directory::IntruderIDDirectory(p. 482), coordination_msg::UInt32 ) =
            new( id, cvc, vrc, vsb, 0, 0, 0, mtb, mid, id_directory.icao24, id_directory.anon24,
                 id_directory.v2v_uid, coordination_msg )
    TRMCoordinationInterrogationData( id::UInt32, cvc::UInt32, vrc::UInt32, vsb::UInt32, chc::UInt32,
        hrc::UInt32, hsb::UInt32, mtb::Bool, mid::UInt32, id_directory::IntruderIDDirectory(p. 482),
        coordination_msg::UInt32 ) =
            new( id, cvc, vrc, vsb, chc, hrc, hsb, mtb, mid, id_directory.icao24, id_directory.anon24,
                 id_directory.v2v_uid, coordination_msg )
end
```

Referenced In: HTRMReport(p. 519), TRMIntruderData(p. 523), VTRMReport(p. 524), SetBlendedCoordinationData(p. 409), sXuTRMReport(p. 501), SetHorizontalIntruderOutput(p. 235), SetHorizontalDroppedIntruderOutput(p. 236)

Type 39 | TRMDesignationData This data structure is generated as part of the TRM Report. It contains information on the Xo mode availabilty. A collection of designation information for each individual intruding aircraft is also included.

```
mutable struct  TRMDesignationData
    # Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1
    availability::Vector{UInt8}  # Xo mode availability (See Xo_AVAILABILITY constants)
    intruder::Vector{TRMIntruderDesignationData(p. 505)}
                                 # Per intruder designation information
#
    TRMDesignationData() =
        new( fill(Xo_AVAILABILITY_NOT_CONFIGURED,2), TRMIntruderDesignationData(p. 505)[] )
end
```

Referenced In: VTRMReport(p. 524), sXuTRMReport(p. 501)

Type 40 | TRMIntruderDesignationData This data structure is generated as part of the TRM Report. It contains information describing the Xo designation state for the intruding aircraft. The selected designation mode and the logic processing mode are given, as is the current advisory state. The valid field indicates whether the intruder can be designated to a particular Xo mode. The status field provides additional information about transitions and differences between the designated mode and the logic mode.

```
mutable struct  TRMIntruderDesignationData
    # Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1
    id::UInt32               # Target id of intruder
    id_directory::IntruderIDDirectory(p. 482) # All external IDs assigned to the intruder
    designated_mode::UInt8   # Xo designation, from input (See DESIGNATION_ constants)
    logic_mode::TRMLogicModeData(p. 506)
                             # Actual logic mode used for TRM processing
    active_ra::Bool          # This intruder has an active RA (may be suppressed)
    suppressed_ra::Bool      # This intruder has an active RA for which display is suppressed
    multithreat::Bool        # There are multiple threats. If active_ra is true, includes this intruder
    valid::Vector{Bool}      # Valid Xo processing modes (See Xo_IDX_ constants for indices)
    status::UInt8            # Designation status (See Xo_STATUS_ constants)
#
    TRMIntruderDesignationData() =
         new( 0, IntruderIDDirectory(p. 482)(),
              DESIGNATION_NONE,
              TRMLogicModeData(p. 506)(),
              false, false, false,
              fill(false,2), Xo_STATUS_NONE )
end
```

Referenced In: TRMDesignationData(p. 505), TRMIntruderData(p. 523)

Type 41 | TRMLogicModeData This data structure encapsulates information describing the TRM logic processing mode associated with designation. This information reflects the designation and processing settings actually used by the TRM.

```
mutable struct  TRMLogicModeData
    processing::UInt8        # TRM logic processing mode (See RA_PROCESSING_)
    dna::Bool                # Whether Designated No Alerts (DNA) is active
    protection_mode::UInt8   # Active protection mode (See PROTECTION_MODE_)
#
    TRMLogicModeData() =
         new( RA_PROCESSING_NONE, false, PROTECTION_MODE_sXu )
end
```

Referenced In: TRMIntruderDesignationData(p. 505)

Type 42 | TRMAlertingData This data structure is generated as part of the TRM Report. It provides information about the TRM's alerting behavior.

```
mutable struct  TRMAlertingData
    warning_alert::Bool             # Flag denoting a warning alert is active
    ras_inhibited::Bool             # Flag denoting when RAs are inhibited based on TRM_OPMODE constants
#
    TRMAlertingData() =
        new( false, false )
    TRMAlertingData( warning_alert::Bool, ras_inhibited::Bool ) =
        new( warning_alert, ras_inhibited )
end
```

Referenced In: sXuTRMReport(p. 501)

Type 43 | sXuTRMBroadcastData This data structure is generated as part of the TRM Report. It represents resolution advisory information communicated via air-to-air broadcast.

```
mutable struct  sXuTRMBroadcastData
    avra_match::Bool        # AVRA Single intent / conflicting intents (1 bit)
    avra_crossing::Bool     # AVRA Crossing / not crossing (1 bit)
    avra_sense::Bool        # AVRA up/down sense
                            #   Down sense = 1 (true) - ownship intent is to pass below the threat
                            #   Up sense = 0 (false) - ownship intent is to pass above the threat
    avra_strength::UInt8    # AVRA Subfield Vertical Strength Bit (4 bits)
    ahra_match::Bool        # AHRA Single intent / conflicting intents (1 bit)
    ahra_turn::UInt8        # AHRA Turn Left / Turn Right / Remain Straight bits (2 bit)
                            #   Indicate the direction of turn required to comply with the
                            #   horizontal guidance and target track angle.
    ahra_track_angle::UInt8 # AHRA Target Track Angle
    rmf::UInt8              # RA Message Format as integer value (2 bits)
    vmte::Bool              # There are multiple threats for this RA - vertical
    hmte::Bool              # There are multiple threats for this RA - horizontal
    vrat::Bool              # Used to indicate that an RA terminated in vertical dimension,
                            #   but is still active in horizontal
    hrat::Bool              # Used to indicate that an RA terminated in horizontal dimension,
                            #   but is still active in vertical
    rat::Bool               # RA terminated (vertical and horizontal)
                            #   0 = The RA, as indicated in one or both of the AVRA and AHRA subfields,
                            #       is currently active or there is no RA
                            #   1 = The RA indicated in the AVRA and/or AHRA subfields terminated on this
                                 cycle
    rac::Vector{Bool}       # RA Complements - first two are vertical second two are horizontal (4 bits)
    mst::UInt8              # Multiple Single-dimensional Threats - single bit used to indicate that the
                            #   overall multiple threat situation, as indicated by VMTE=1 and HTME=1,
                            #   actually consists of a single threat in the vertical dimension and a
                            #   different single threat in the horizontal dimension.
                            #     0 = There is a single threat overall (VMTE=0 and HMTE=0) or there are
                            #         two or more threats in at least one dimension
                            #     1 = There are two distinct threats overall and VMTE=1 and HTME=1,
                            #         but there is only a single threat in each dimension
    mid::UInt32             # 24-bit subfield contains the Unique V2V ID of ownship
    agt_ids::Vector{UInt32} # Ownship-associated AGT IDs
    ldi::UInt8              # Low-level Descend Inhibits (2 bits)
    hri::Bool               # Horizontal Resolution Advisory Cutoff Indicator (1 bit)
                            #   0 (false) = No horizontal RAs are inhibited.
                            #   1 (true) = All horizontal RAs are inhibited.
    intruder_ids::IntruderIDDirectory(p.482) # All external IDs assigned to the intruder
    adsb_ccb::CCCB(p.509)    # ADSB Collision Avoidance Coordination Capability Bits - first threat
    v2v_ccb::CCCB(p.509)     # V2V Collision Avoidance Coordination Capability Bits - first threat
    ico::Bool               # Is ICAO 24-bit address - indicates whether the 24-bit address in the
                            #   TID subfield is an ICAO address
    tti::UInt8              # Threat type indicator for most recent threat (1 bit) - (see TTI_ in constants)
    tid::TID(p.508)          # Threat identifier for most recent threat (24 bits)
                            #   Set to TIDAddress   when tti = TTI_ADDRESS
                            #   Set to TIDAltRngBrg when tti = TTI_ALT_RNG_BRG
    intruder_ids2::IntruderIDDirectory(p.482) # All external IDs assigned to the intruder
    adsb_ccb2::CCCB(p.509)   # ADSB Collision Avoidance Coordination Capability Bits - second threat
    v2v_ccb2::CCCB(p.509)    # V2V Collision Avoidance Coordination Capability Bits - second threat
    ico2::Bool              # Is ICAO 24-bit address - indicates whether the 24-bit address in the
                            #   TID2 subfield is an ICAO address
    tti2::UInt8             # Threat Type Indicator 2 defines the type of identity data contained in the
                            #   TID2 subfield. If there is only one threat, this subfield is set to zero
    tid2::TID(p.508)         # Threat identifier Data 2 contains the identity or position data for a
                            #   threat other than the threat reported in the TID subfield on the
                            #   same processing cycle
#
    sXuTRMBroadcastData() =
        new( false, false, false, 0, false, 0, 0, RMF_ACAS_sXu, false, false, false, false, false,
             fill(false,4), 0, 0, UInt32[], LDI_NONE, false, IntruderIDDirectory(p.482)(),
             CCCB(p.509)(CAS_SENSE_BLENDED, CAS_ACTIVE_TCAS, UInt8(0)),
             CCCB(p.509)(CAS_SENSE_BLENDED, CAS_V2V_NONE, UInt8(0)), false, TTI_ADDRESS, TIDAddress(p.508)(),
             IntruderIDDirectory(p.482)(), CCCB(p.509)(CAS_SENSE_BLENDED, CAS_ACTIVE_TCAS, UInt8(0)),
             CCCB(p.509)(CAS_SENSE_BLENDED, CAS_V2V_NONE, UInt8(0)), false, TTI_ADDRESS, TIDAddress(p.508)() )
end
```

Referenced In: SetReportRATerminationBits(p.404), SetPrevHorizontalTerminationData(p.406), GenerateBlendedsXuTRMOutput(p.396), TTRtvMReport(p.519), GenerateRAModeOutput(p.378), SetPrevVerticalTerminationData(p.405), [illegible] Output(p.386), SetRABroadcastData(p.391), sXuTRMState(p.510), SetReport[illegible] DataOnRATerm(p.403), DetermineDimensionalRAState(p.407), sXuTRMReport(p.501), GenerateSurvOnlyModeOut-

Type 44 | VTRMRAData This data structure is used within the TRM to store vertical resolution advisory information that is used to populate ground message data and broadcast message data in the TRM report.

```
mutable struct  VTRMRAData
                            # Active Vertical RA (AVRA) (7 bits)
    avra_single_intent::Bool #    Single intent / conflicting intents (1 bit)
    avra_crossing::Bool      #    Crossing / not crossing (1 bit)
    avra_down::Bool          #    Down / Up (1 bit)
    avra_strength::UInt8     #    Strength bits as integer value (4 bits)
    ldi::UInt8               # Low-level Descend Inhibits as integer value (2 bits)
    rac::Vector{Bool}        # RA Complements (4 bits)
    ra_term::Bool            # Vertical RA terminated
    vmte::Bool               # There are multiple vertical threats for this RA
#
    VTRMRAData() =
        new( false, false, false, 0, LDI_NONE, fill(false,4), false, false )
end
```

Referenced In: VerticalRAOutputState(p. 524), DetermineVerticalRAData(p. 385), SetVerticalRAMessageOutput(p. 386), SetRABroadcastData(p. 391)

Type 45 | TID This is an abstract data structure used in sXuTRMBroadcastData to reference TIDAddress and TIDAltRngBrg. TID stands for Threat ID.

```
abstract type  TID end
```

Referenced In: DetermineTTIAndTID(p. 387), sXuTRMBroadcastData(p. 507), TIDAltRngBrg(p. 509), TIDAddress(p. 508)

Type 46 | TIDAddress This data structure is used in sXuTRMBroadcastData when the threat is identified as an address type. It is derived from the abstract type TID. TID stands for Threat ID.

```
mutable struct  TIDAddress <: TID(p. 508)
    tid::UInt32        # Threat ID as an address (24 bits)
#
    TIDAddress() =
        new( UInt32(0) )
    TIDAddress( tid::UInt32 ) =
        new( tid )
end
```

Referenced In: DetermineTTIAndTID(p. 387), sXuTRMBroadcastData(p. 507)

Type 47 | TIDAltRngBrg This data structure is used in sXuTRMBroadcastData when the threat is identified by encoded altitude, range, and bearing. It is derived from the abstract type TID. TID stands for Threat ID.

```
mutable struct  TIDAltRngBrg <: TID(p. 508)
    tida::UInt16  # Threat ID encoded altitude (ft) (11 bits)
    tidr::UInt8   # Threat ID encoded range (ft)    ( 7 bits)
    tidb::UInt8   # Threat ID encoded bearing (deg) ( 6 bits)
#
    TIDAltRngBrg() =
        new( UInt16(0), UInt8(0), UInt8(0) )
    TIDAltRngBrg( tida::UInt16, tidr::UInt8, tidb::UInt8 ) =
        new( tida, tidr, tidb )
end
```

Referenced In: DetermineTTIAndTID(p. 387)

Type 48 | CCCB Definition for the Collision Avoidance Coordination Capability Bits (CCCB) for the ADS-B and V2V Operational Status Messages.

```
mutable struct  CCCB
    sense::UInt8                # Sense - two bits field for Vertical and Horizontal
                                #      0 = Vertical only
                                #      1 = Horizontal only
                                #      2 = Blended
                                #      3 = Vertical-only or Horizontal-only per intruder
    type_capability::UInt8      # Aircraft CAS Type/Capability
                                # Note: different enumerations of type capability are
                                # used in the ADS-B CCCB and V2V CCCB
    priority::UInt8             # Priority - two bits field envisioned for UAS use, to serve as a priority
                                #     field to distinguish among users with different levels of capability
                                #     or as directed by regulatory authorities
                                # Note: diffrent enumerations of priority are
                                # used in the ADS-B CCCB and V2V CCCB
    CCCB( sense::UInt8, type_capability::UInt8, priority::UInt8 ) = new(sense, type_capability, priority)
end
```

Referenced In: TRMIntruderInput(p. 499), constants(p. 474), sXuTRMBroadcastData(p. 507), ADSBOperationalStatusMessage(p. 496), V2VOperationalStatusMessage(p. 496), SetOwnV2VOperationalStatusMessage(p. 110)

D.5 TRM Internal Data Structures

The algorithms contained within this document rely on a number of data structures to store and pass data within the TRM. This appendix provides the definition, description and initial values for data structures that are internal to the TRM. These data structures are used to encapsulate history, state, and variables. The fields in the RDataTableScaled data structure are described in the Data Table Format Specification in Appendix F.

Type 49 | sXuTRMState This data structure provides information about internal TRM state for both veritcal and horizontal. It contains information about own aircraft previous and current states and a collection with information about the processing states for each individual intruding aircraft.

```
mutable struct  sXuTRMState
    st_vert::TRMState(p. 510)                          # Vertical TRM state
    st_horiz::HTRMState(p. 513)                         # Horizontal TRM state
    broadcast_report_prev::sXuTRMBroadcastData(p. 507)  # The previous cycle broadcast report data
    int_published_ra_state::Dict{UInt32, IntruderPublishedRaState(p. 525)}
                                                  # The dictionary of intruder ID's and their
                                                  #    indication of RA report publish state
#
    sXuTRMState() =
        new( TRMState(p. 510)(), HTRMState(p. 513)(), sXuTRMBroadcastData(p. 507)(), Dict{UInt32,
            IntruderPublishedRaState(p. 525)}() )
end
```

Referenced In: GenerateBlendedsXuTRMOutput(p. 396), VerticalTRMUpdate(p. 241), UpdateRaPubState(p. 398), GenerateRAModeOutput(p. 378), SetRaPublishedState(p. 402), SetVerticalRAMessageOutput(p. 386), SetBlendedIntruderThreatData(p. 400), SetRABroadcastData(p. 391), GenerateVTRMOutput(p. 376), sXuTRMBlendedReportPrep(p. 397), sXuTRMUpdate(p. 164), GenerateSurvOnlyModeOutput(p. 382), GenerateStandbyModeOutput(p. 383)

Type 50 | TRMState This data structure provides information about internal Vertical TRM state. It contains information about own aircraft previous and current states and a collection with information about the processing states for each individual intruding aircraft.

```
mutable struct  TRMState
    st_own::TRMOwnState(p. 511)           # Ownship TRM state information
    st_intruder::Vector{TRMIntruderState(p. 512)}
                                  # Intruder TRM state information
    params::paramsfile_type
#
    TRMState() = new( TRMOwnState(p. 511)(), TRMIntruderState(p. 512)[] )
end
```

Referenced In: VerticalTRMUpdatePrep(p. 587), sXuTRMState(p. 510)

Type 51 | TRMOwnState This data structure is used by the Vertical TRM to maintain state information directly associated with previous and current own aircraft state and global advisories. It includes context-specific state structures and information used to produce resolution advisory messages for the Ground Station.

```
mutable struct  TRMOwnState
    a_prev::GlobalAdvisory(p. 517)          # Previous global advisory
    dz_ave_prev::R                  # Previous own vertical rate from vertical beliefs (ft/sec)
    opmode_prev::UInt8              # Ownship operational mode on last cycle
    action_prev::Z                  # Previous global advisory action for display
    word_prev::Z                    # Previous Label270 word
    strength_prev::UInt8            # Previous ARA strength value
    crossing_prev::Bool             # Previous advisory was for a crossing
    ra_output_prev::VerticalRAOutputState(p. 524)
                                    # Previous VTRMRAData
    st_multithreat_cost_balancing::MultithreatCostBalancingCState(p. 530)
                                    # State variables for MultithreatCostBalancing
    st_arbitrate::ActionArbitrationGlobalCState(p. 526)
                                    # State variables for ActionArbitration
    st_alt_inhibit::Vector{AltitudeInhibitCState(p. 527)}
                                    # State variables for AltitudeInhibitCost
#
    TRMOwnState() =
      new( GlobalAdvisory(p. 517)(),
           0.0, OPMODE_STANDBY, COC, 0, UInt8(0), false,
           VerticalRAOutputState(p. 524)(),
           MultithreatCostBalancingCState(p. 530)(), ActionArbitrationGlobalCState(p. 526)(),
           AltitudeInhibitCState(p. 527)[] )
end
```

Referenced In: DetermineDisplayData(p. 368), MTLODetermination(p. 350), VerticalTRMUpdate(p. 241), DisplayLogicDetermination(p. 366), NoIntrudersAction(p. 342), StateAndCostEstimation(p. 251), GetVTRMOwnData(p. 243), AdjustRAModeIntruderOutput(p. 379), DetermineVerticalRAData(p. 385), DetermineCrossing(p. 367), IntruderPrep(p. 254), DetermineMultiIntruderAction(p. 345), SetVerticalTRMSurvOnlyMode(p. 247), ActionSelection(p. 341), UpdateIndivAdjustThreatInfo(p. 380), IndividualCostEstimation(p. 339), OnlineCostEstimation(p. 310), TRMState(p. 510)

Type 52 | TRMIntruderState This data structure is used by the TRM to maintain state information directly associated with previous and current intruding aircraft state and individual advisories.

```
mutable struct  TRMIntruderState
    id::UInt32                    # Unique identifier
    id_directory::IntruderIDDirectory(p. 482) # All external IDs assigned to the intruder
    a_prev::IndividualAdvisory(p. 519)   # Previous individual advisory for this intruder
    b_prev::AdvisoryBeliefState(p. 515)  # Previous individual advisory beliefs for CombineVerticalBeliefs
    sense_prev::Symbol            # Up or down sense of last individual advisory
    vrc_prev::UInt32              # VRC for transmission to this intruder in the last cycle
    cvc_prev::UInt32              # CVC for transmission to this intruder in the last cycle
    equipage_prev::Z              # Previous equipage (See EQUIPAGE_ constants)
    coordination_msg_prev::UInt32# Previous coordination message type (See COORDINATION_ constants)
    st_cost_on::OnlineCostState(p. 520)  # State variables for online cost algorithms
    st_arbitrate::ActionArbitrationCState(p. 526)
                                  # State variables for ActionArbitration
    is_threat::Bool               # Has an RA
    is_identified_threat::Bool    # Has been identified as a threat for setting TTI and TID
    processing::UInt8             # Processing type (See RA_PROCESSING_ constants)
    status::Symbol                # Processing status (:New, :InUse, :Dropped, :NotValid,
                                  #                :DesignationStateTimeout, :MarkedForDeletion)
    range_prev::R                 # Previous range, used by Vertical DAA (ft)
    idx_scale::Z                  # Index of scaling to use for this intruder
    idx_online_cost::Z            # Index of online cost parameters to use for this intruder
    coc_cost::R                   # Cost used for prioritizing intruders
#
    TRMIntruderState( id::UInt32, id_directory::IntruderIDDirectory(p. 482) ) =
          new( id, id_directory, IndividualAdvisory(p. 519)(), AdvisoryBeliefState(p. 515)(),
               :None, UInt32(0), UInt32(0), EQUIPAGE_NONE, COORDINATION_NONE,
               OnlineCostState(p. 520)(), ActionArbitrationCState(p. 526)(),
               false, false, RA_PROCESSING_NONE, :New,
               NaN, VERTICAL_SCALING_OPTION_DEFAULT,
               VERTICAL_SCALING_OPTION_DEFAULT,
               0.0 )
end
```

Referenced In: SetInvalidIntruder(p. 395), MTLODetermination(p. 350), VerticalTRMUpdate(p. 241), ActionArbitration(p. 355), IndividualSelectionCostEstimation(p. 346), VerticalTRMUpdatePrep(p. 587), MultithreatCostBalancing(p. 354), SetDroppedIntruder(p. 394), StateAndCostEstimation(p. 251), SetIntruderVerticalScores(p. 372), AdjustRAModeIntruderOutput(p. 379), TRMIntruderStateUpdate(p. 434), ArbitrateMatchingSenses(p. 358), IntruderPrep(p. 254), DetermineMultiIntruderAction(p. 345), SetVerticalTRMSurvOnlyMode(p. 247), DroppedIntrudersAdjustment(p. 393), ArbitrateConflictingSenses(p. 357), AllowUnequippedMTLO(p. 352), ActionSelection(p. 341), UpdateIndivAdjustThreatInfo(p. 380), GenerateSurvOnlyModeOutput(p. 382), OnlineCostEstimation(p. 310), GenerateStandbyModeOutput(p. 383), UpdateIndivAdjustCounts(p. 381), TRMState(p. 510)

Type 53 | HTRMState This data structure provides information about internal horizontal TRM state. It contains information about own aircraft previous and current states and a collection with information about the processing states for each individual intruding aircraft.

```
mutable struct  HTRMState
    st_own::HTRMOwnState(p. 513)                    # Global state for Horizontal TRM
    st_intruder::Vector{HTRMIntruderState(p. 514)}  # Individual intruder state for Horizontal TRM
#
    HTRMState() =
        new( HTRMOwnState(p. 513)(), HTRMIntruderState(p. 514)[] )
    HTRMState( own::HTRMOwnState(p. 513), intruder::Vector{HTRMIntruderState(p. 514)} ) =
        new( own, intruder )
end
```

Referenced In: HorizontalTRMUpdate(p. 168), HorizontalTRMUpdatePrep(p. 589), sXuTRMState(p. 510), GenerateHTRMOutput(p. 234)

Type 54 | HTRMOwnState This data structure is used by the TRM to maintain state information directly associated with previous and current own aircraft state and global advisories in the horizontal dimension. It includes context-specific state structures and information used to produce resolution advisory messages for the Ground Station.

```
mutable struct  HTRMOwnState
    enu_beliefs::EnuBeliefSet(p. 517)             # ENU Position and Velocity beliefs for current position
    advisory_prev::HorizontalAdvisory(p. 518)     # Horizontal advisory selected for internal alerting
    is_advisory_prev::Bool                # Whether there was an advisory on the previous cycle (for
        internal use)
    is_turn_recommended::Bool             # Whether a turn is recommended on this cycle
    is_turn_recommended_prev::Bool        # Whether a turn was recommended on the previous cycle
    prev_word::Z                          # Previous label271 word
    is_initialized_speed_bin::Bool        # Flag indicating speed bin was initialized for ownship
    speed_bin_prev_policy::Vector{Z}      # Value of speed bin on the previous cycle for policy per scaling
    highest_threat_prev::Vector{UInt32}   # Previous list of highest threats from online cost fusion
    num_reversals::UInt32                 # Counter for number of horizontal RA reversals.
    invalid_agl_cycles::Z                 # number of cycles without valid height-AGL data
    multihreat_turn_from_maintain_hold::Bool # Flag to apply multithreat turn from maintain cost until turn
        is issued
#
    HTRMOwnState() =
        new( EnuBeliefSet(p. 517)(),
             HorizontalAdvisory(p. 518)(),
             false, false, false,
             0,
             false,
             Z[],
             UInt32[],
             0,
             0,
             false )
end
```

Referenced In: ResetHTRMState(p. 174), UpdateTargetTrackAngle(p. 219), DetermineDisplayTrackAngleAndTurnRate(p. 232), SetHorizontalTRMSurvOnlyMode(p. 169), HorizontalStateEstimation(p. 175), HorizontalDisplayLogicDetermination(p. 231), GetHorizontalPolicyCosts(p. 184), ResetHorizontalTRMOwnshipState(p. 590), SelectHorizontalAdvisoryWindRelative(p. 211), SelectHorizontalAlertLookaheadAction(p. 216), SelectAndModifyPolicyBins(p. 188), HorizontalCostFusion(p. 217), HTRMState(p. 513), UpdatePolicySpeedBins(p. 191), PrioritizeAndFilterIntruders(p. 171), HorizontalAlertLookahead(p. 214), UpdatePolicySpeedBinIndividual(p. 192), UpdateNumHorizontalReversals(p. 220), DetermineLabel271(p. 232), SetHorizontalIntruderOutput(p. 235), SelectHorizontalAdvisory(p. 210), SetHorizontalDroppedIntruderOutput(p. 236)

Type 55 | HTRMIntruderState This data structure is used by the HTRM to maintain state and advisory information directly associated with previous and current intruder aircraft. This data structure also maintains the index of the policy scaling factors *idx_scale* to be used with this intruder.

```
mutable struct  HTRMIntruderState
    id::UInt32                      # Unique identifier
    id_directory::IntruderIDDirectory(p. 482) # All external IDs assigned to the intruder
    enu_beliefs::EnuBeliefSet(p. 517)      # ENU Position and Velocity beliefs for current position
    t_init::R                       # Time since track was initialized (sec)
    in_advisory::Bool               # Contributed to the advisory on this cycle
    in_priority_list::Bool          # Used when determining the advisory on this cycle
    is_initialized_speed_bin::Bool # Flag indicating speed bin was initialized
    speed_bin_prev_policy::Z        # Value of speed bin on the previous cycle for policy
    coc_cost::R                     # Cost used for prioritizing intruders
    processing::UInt8               # Processing type (See RA_PROCESSING_ constants)
    status::Symbol                  # Processing status (:New, :InUse, :Dropped, :NotValid,
                                    #                 :DesignationStateTimeout, :MarkedForDeletion)
    is_equipped::Bool               # Whether this intruder is capable of horizontal coordination
    received_hrc::UInt32            # Horizontal Resolution Complement received from this intruder
    hrc_prev::UInt32                # Horizontal Resolution Complement sent on previous cycle
    chc_prev::UInt32                # Cancel Horizontal Resolution Complement sent on previous cycle
    hcoord_sense_prev::Z            # Indication of matching / differing horizontal coordination sense sent
        on previous cycle
    equipage_prev::Z                # Previous equipage (See EQUIPAGE_ constants)
    coordination_msg_prev::UInt32   # Previous coordination message type (See COORDINATION_ constants)
    is_master::Bool                 # Whether this intruder is master
    own_coordination_policy::UInt8 # Ownship coordination policy for this intruder (See
        OWN_COORDINATION_POLICY_ constants)
    idx_scale::Z                    # Index of scaling to use for this intruder
    classification::UInt8           # Intruder classification (See CLASSIFICATION_ constants)
    source::Z                       # Surveillance source (See SOURCE_ constants)
#
    HTRMIntruderState() =
        new( 0, IntruderIDDirectory(p. 482)(), EnuBeliefSet(p. 517)(),
             0.0, false, false, false, 0, NaN, RA_PROCESSING_NONE, :New, false,
             UInt32(0), UInt32(0), UInt32(0), HCOORD_SENSE_NONE, EQUIPAGE_NONE,
             COORDINATION_NONE, false, OWN_COORDINATION_POLICY_NONE, HORIZONTAL_SCALING_OPTION_DEFAULT,
                 CLASSIFICATION_NONE, SOURCE_MODES)
    HTRMIntruderState( id::UInt32, id_directory::IntruderIDDirectory(p. 482) ) =
        new( id, id_directory, EnuBeliefSet(p. 517)(),
             0.0, false, false, false, 0, NaN, RA_PROCESSING_NONE, :New, false,
             UInt32(0), UInt32(0), UInt32(0), HCOORD_SENSE_NONE, EQUIPAGE_NONE,
             COORDINATION_NONE, false, OWN_COORDINATION_POLICY_NONE, HORIZONTAL_SCALING_OPTION_DEFAULT,
                 CLASSIFICATION_NONE, SOURCE_MODES)
end
```

Referenced In: HorizontalCoordinatedReversalCost(p. 206), SetHorizontalTRMSurvOnlyMode(p. 169), DetermineHorizontalCode(p. 229), HorizontalTRMUpdate(p. 168), UpdateHTRMIntruderInputs(p. 173), HorizontalStateEstimation(p. 175), HorizontalTrackThreatAssessment(p. 228), SetAHRAReportData(p. 237), GetHorizontalPolicyCosts(p. 184), SelectHorizontalAdvisoryWindRelative(p. 211), SelectHorizontalAlertLookaheadAction(p. 216), UpdateHTRMIntruderState(p. 435), SelectAndModifyPolicyBins(p. 188), HorizontalCostFusion(p. 217), HTRMState(p. 513), HorizontalTRMUpdatePrep(p. 589), HorizontalCoordinationCost(p. 204), HorizontalCoordinationSelection(p. 222), UpdatePolicySpeedBins(p. 191), PrioritizeAndFilterIntruders(p. 171), HorizontalAlertLookahead(p. 214), UpdatePolicySpeedBinIndividual(p. 192), GenerateHTRMOutput(p. 234), ApplyHorizontalOnlineCosts(p. 200), UpdateHTRMIntruderStates(p. 436), FindHighestThreatIndex(p. 206), SetHorizontalIntruderOutput(p. 235), SelectHorizontalAdvisory(p. 210), SetHorizontalDroppedIntruderOutput(p. 236)

Type 56 | AdvisoryBeliefState This data structure is used to store and pass information about the vertical beliefs and the previous advisory state for a single intruding aircraft. The vertical belief values are expressed as weights and are generated in COMBINEVERTICALBELIEFS.

```
mutable struct  AdvisoryBeliefState
    need_init::Bool           # Uninitialized advisory state
    w_vert_prev::Vector{R}    # Previous vertical sample weights
#
    AdvisoryBeliefState() = new( true, R[1.0] )
end
```

Referenced In: CombineVerticalBeliefs(p. 270), TRMIntruderStateUpdate(p. 434), TRMIntruderState(p. 512), StateEstimation(p. 257)

Type 57 | CombinedVerticalBelief This data structure is used for storing and passing vertical state samples used for estimating the offline cost in OFFLINECOSTESTIMATION. The samples are produced by COMBINEVERTICALBELIEFS.

```
mutable struct  CombinedVerticalBelief
    z_rel::R     # Vertical separation (ft)
    dz_own::R    # Vertical rate of ownship (ft/s)
    dz_int::R    # Vertical rate of intruder (ft/s)
    weight::R    # Sample weight (0.0-1.0)
#
    CombinedVerticalBelief() = new( 0.0, 0.0, 0.0, 0.0 )
    CombinedVerticalBelief( z_rel::R, dz_own::R, dz_int::R, weight::R ) =
        new( z_rel, dz_own, dz_int, weight )
end
```

Referenced In: CombineVerticalBeliefs(p. 270), StateAndCostEstimation(p. 251), OfflineStatesScaleFactor(p. 279), OfflineCostEstimation(p. 277), StateEstimation(p. 257), GetEquivClassTableMaxCutValues(p. 279)

Type 58 | DisplayLogic This data structure is used within the TRM to store and pass display data associated with the global resolution advisory.

```
mutable struct  DisplayLogic
    crossing::Bool        # RA will result in a crossing
    cc::UInt8             # combined control (label270)
    vc::UInt8             # vertical control (label270)
    ua::UInt8             # up advisory (label270)
    da::UInt8             # down advisory (label270)
    target_rate::R        # vertical rate to target (ft/s)
    alarm::Bool           # Resolution Advisory changed flag
    ground_alert::UInt8   # Type of ground alert (See GROUND_ALERT_ constants)
    strength::UInt8       # ARA strength bits setting representing display output
    down::Bool            # ARA down/up bit setting representing display output
    dz_min::R             # Minimum vertical rate associated with selected global action (ft/min)
    dz_max::R             # Maximum vertical rate associated with selected global action (ft/min)
    ddz_min::R            # Compliant acceleration associated with selected global action (ft/s^2)
    opmode::UInt8         # Ownship vertical operational mode (See TRM_OPMODE constants)
    trm_altmode::UInt8    # TRM altitude type operational mode setting (See TRM_ALTMODE_ constants)
#
    DisplayLogic( crossing::Bool, cc::UInt8, vc::UInt8, ua::UInt8, da::UInt8, target_rate::R, alarm::Bool,
                  ground_alert::UInt8, strength::UInt8, down::Bool, dz_min::R, dz_max::R, ddz_min::R,
                  opmode::UInt8, trm_altmode::UInt8 ) =
        new( crossing, cc, vc, ua, da, target_rate, alarm, ground_alert, strength, down, dz_min, dz_max,
             ddz_min, opmode, trm_altmode )
    DisplayLogic() = new( false, UInt8(0), UInt8(0), UInt8(0), UInt8(0), 0.0, false, GROUND_ALERT_NONE, UInt8
         (0), false, -Inf, Inf, 0.0, OPMODE_STANDBY, TRM_ALTMODE_NONE )
end
```

Referenced In: VTRMDisplayData(p. 502), VerticalTRMUpdate(p. 241), DisplayLogicDetermination(p. 366), GenerateRAModeOutput(p. 378), VTRMReport(p. 524), DetermineVerticalRAData(p. 385), SetVerticalRAMessageOutput(p. 386), GenerateVTRMOutput(p. 376), GenerateSurvOnlyModeOutput(p. 382), GenerateStandbyModeOutput(p. 383)

Type 59 | EnuBelief This data structure is used within the TRM to pair the East, North, Up position and velocity states to its weight to describe the belief.

```
mutable struct  EnuBelief
    enu::EnuPositionVelocity(p. 517)    # a set of ENU PositionVelocity
    weight::R                   # weights for each set
#
    EnuBelief() = new( EnuPositionVelocity(p. 517)(), 0.0 )
    EnuBelief(enu::EnuPositionVelocity(p. 517), weight::R) = new( enu, weight )
end
```

Referenced In: PointObstacleEnuBeliefEstimation(p. 180), CreatePolicyStateBeliefs(p. 186), CoastEnuBeliefBasedOnIndividualSamples(p. 411), TurnEnuBeliefBasedOnIndividualSamples(p. 434), ConvertOwnInputsToEnuBeliefsWindRelative(p. 178), EnuBeliefSet(p. 517), ConvertIntruderInputsToEnuBeliefs(p. 179), ConvertOwnInputsToEnuBeliefsGroundAndCompassMix(p. 177), ConvertOwnInputsToEnuBeliefs(p. 176), EnuBeliefEstimation(p. 181)

Type 60 | EnuBeliefSet This data structure is used within the TRM to store the vector of ENU beliefs and the weighted average of the position and velocity beliefs.

```
mutable struct  EnuBeliefSet
    b_enu::Vector{EnuBelief(p. 516)}       # ENU beliefs
    enu_ave::EnuPositionVelocity(p. 517)  # Weighted average of position and velocity
#
    EnuBeliefSet() =
        new( EnuBelief(p. 516)[], EnuPositionVelocity(p. 517)() )
    EnuBeliefSet( b_enu::Vector{EnuBelief(p. 516)}, enu_ave::EnuPositionVelocity(p. 517) ) =
        new( b_enu, enu_ave )
end
```

Referenced In: HTRMIntruderState(p. 514), CreatePolicyStateBeliefs(p. 186), CoastEnuBeliefBasedOnIndividualSamples(p. 411), GetHorizontalPolicyCosts(p. 184), SelectHorizontalAlertLookaheadAction(p. 216), UpdateHTRMIntruderState(p. 435), TurnEnuBeliefBasedOnIndividualSamples(p. 434), ConvertOwnInputsToEnuBeliefsWindRelative(p. 178), HorizontalCostFusion(p. 217), HTRMOwnState(p. 513), ConvertIntruderInputsToEnuBeliefs(p. 179), ConvertOwnInputsToEnuBeliefsGroundAndCompassMix(p. 177), HorizontalCoordinationSelection(p. 222), PrioritizeAndFilterIntruders(p. 171), HorizontalAlertLookahead(p. 214), ConvertOwnInputsToEnuBeliefs(p. 176), GetHorizontalCoordinationPolicy(p. 223)

Type 61 | EnuPositionVelocity This data structure is used within the TRM to describe the East, North, Up position and velocity vectors.

```
mutable struct  EnuPositionVelocity
    pos_enu::Vector{R}   # position, EastNorthUp, meters
    vel_enu::Vector{R}   # velocity, EastNorthUp, m/s
#
    EnuPositionVelocity() = new( fill( 0.0, 3) , fill( 0.0, 3)  )
    EnuPositionVelocity(pos_enu::Vector{R}, vel_enu::Vector{R}) = new( pos_enu, vel_enu )
end
```

Referenced In: IsIntruderInScope(p. 172), CoastEnuBeliefBasedOnIndividualSamples(p. 411), TurnEnuBeliefBasedOnIndividualSamples(p. 434), ConvertOwnInputsToEnuBeliefsWindRelative(p. 178), EnuBelief(p. 516), EnuBeliefSet(p. 517), CalculatePolicyState(p. 187), ConvertIntruderInputsToEnuBeliefs(p. 179), ConvertOwnInputsToEnuBeliefsGroundAndCompassMix(p. 177), ConvertOwnInputsToEnuBeliefs(p. 176), CoastEnuPositionVelocity(p. 411), TurnEnuPositionVelocity(p. 435)

Type 62 | GlobalAdvisory This data structure is used within the TRM to store information that defines the global resolution advisory.

```
mutable struct  GlobalAdvisory
    action::Z           # global advisory action
    dz_min::R           # global advisory minimum vertical rate (ft/s)
    dz_max::R           # global advisory maximum vertical rate (ft/s)
    ddz::R              # global advisory acceleration (ft/s/s)
    multithreat::Bool   # multiple threats for global advisory
#
    GlobalAdvisory() = new( COC, -Inf, Inf, 0.0, false )
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287), CoordinationSelection(p. 361), DetermineSenses(p. 351), TRMOwnState(p. 511), OnlineUncoordinatedCostEstimation(p. 284), PersistMTLO(p. 431), OnlineCoordinatedActionCostEstimation(p. 312), GenerateSurvOnlyModeOutput(p. 382), GenerateStandbyModeOutput(p. 383)

Type 63 | HorizontalAdvisory This data structure is used within the TRM to store horizontal resolution advisory information.

```
mutable struct  HorizontalAdvisory
    action::Z                   # Selected action
    track_angle::R              # Associated track angle (rad)
    turn_rate::R                # Associated turn rate (rad/s)
    costs::Vector{R}            # Minimum cost for each action
    threat_list::Vector{UInt32} # IDs of threats contributing to advisory
    threat_list_unconditioned::Vector{UInt32} # IDs of threats contributing to advisory (without
        conditioning)
#
    HorizontalAdvisory() =
        new( COC, NaN, NaN,
             R[],
             UInt32[],
             UInt32[] )
end
```

Referenced In: ResetHTRMState(p. 174), UpdateTargetTrackAngle(p. 219), SetHorizontalTRMSurvOnlyMode(p. 169), DetermineHorizontalCode(p. 229), HorizontalTRMUpdate(p. 168), HorizontalTrackThreatAssessment(p. 228), SelectHorizontalAdvisoryWindRelative(p. 211), SelectHorizontalAlertLookaheadAction(p. 216), HTRMOwnState(p. 513), HorizontalAlertLookahead(p. 214), GenerateHTRMOutput(p. 234), UpdateNumHorizontalReversals(p. 220), SetHorizontalIntruderOutput(p. 235), SelectHorizontalAdvisory(p. 210), SetHorizontalDroppedIntruderOutput(p. 236)

Type 64 | HorizontalDisplayLogic This data structure is generated as part of the TRM Report. It is used to store data for the onboard pilot display.

```
mutable struct  HorizontalDisplayLogic
    cc::UInt8                           # combined control (RWC/CA and strength) (label271)
    target_rate::R                      # turn rate to target for advisory (deg/s)
    target_angle::R                     # heading_angle or track_angle to target advisory (deg from North
        )
    wind_relative::Bool                 # operating in wind relative mode True = target_anlge based on
        heading, False = target_angle based on track angle
    h_opmode::UInt8                     # Ownship horizontal operational mode (See TRM_OPMODE constants)
    trm_velmode::UInt8                  # TRM horizontal velocity operational mode setting
    horizontal_desensitivity_mode::UInt8   # Mode variable indicating sensitivity or desensitivity of RA
        selection regarding horizontal TRM's online costs
#
    HorizontalDisplayLogic( cc::UInt8, target_rate::R, target_angle::R, wind_relative::Bool, h_opmode::UInt8,
        trm_velmode::UInt8, horizontal_desensitivity_mode::UInt8 ) =
        new( cc, target_rate, target_angle, wind_relative, h_opmode, trm_velmode,
            horizontal_desensitivity_mode)
    HorizontalDisplayLogic() =
        new( UInt8(0), NaN, NaN, false, OPMODE_STANDBY, TRM_VELMODE_NONE, HORIZONTAL_DESENSITIVITY_NONE )
end
```

Referenced In: HTRMDisplayData(p. 503), HorizontalTRMUpdate(p. 168), HorizontalDisplayLogicDetermination(p. 231), SetAHRAReportData(p. 237), HTRMReport(p. 519), GenerateHTRMOutput(p. 234)

Type 65 | HTRMReport This data structure is produced by the TRM at the end of each processing cycle. It contains information for the horizontal onboard pilot display, coordination, RA broadcast, and RA transmission to the Ground Station.

```
mutable struct  HTRMReport
    display::HTRMDisplayData(p. 503) # Data for the onboard pilot display
    coordination::Vector{TRMCoordinationInterrogationData(p. 504)}
                                # Per intruder data for outgoing coordination messages
    broadcast_msg::sXuTRMBroadcastData(p. 507)
                                # Data for sXu broadcast
#
    HTRMReport() = new( HTRMDisplayData(p. 503)(),
                        TRMCoordinationInterrogationData(p. 504)[],
                        sXuTRMBroadcastData(p. 507)() )
    HTRMReport( display_logic::HorizontalDisplayLogic(p. 518) ) =
         new( HTRMDisplayData(p. 503)( display_logic, HTRMIntruderDisplayData(p. 504)[] ),
              TRMCoordinationInterrogationData(p. 504)[],
              sXuTRMBroadcastData(p. 507)() )
end
```

Referenced In: SetReportRATerminationBits(p. 404), GenerateBlendedsXuTRMOutput(p. 396), HorizontalTRMUpdate(p. 168), SetAHRAReportData(p. 237), SetBlendedCoordinationData(p. 409), SetReportDataOnRATerm(p. 403), GenerateHTRMOutput(p. 234), DetermineDimensionalRAState(p. 407), sXuTRMBlendedReportPrep(p. 397), sXuTRMUpdate(p. 164), SetHorizontalIntruderOutput(p. 235), SetHorizontalDroppedIntruderOutput(p. 236)

Type 66 | IndividualAdvisory This data structure is used within the TRM to store information that defines a resolution advisory for an individual intruding aircraft.

```
mutable struct  IndividualAdvisory
    action::Z       # individual advisory action
    dz_min::R       # individual advisory minimum vertical rate (ft/s)
    dz_max::R       # individual advisory maximum vertical rate (ft/s)
    ddz::R          # individual advisory acceleration (ft/s/s)
    ra_prev::Bool   # previous individual advisory was an RA
    sense::Symbol   # up or down sense of the advisory for this intruder
    vrc::UInt32     # Vertical RA Complement for transmission
    cvc::UInt32     # Cancel Vertical RA Complement for transmission
#
    IndividualAdvisory() = new( COC, -Inf, Inf, 0.0, false, :None, 0, 0 )
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287), CoordinationSelection(p. 361), TRMIntruderStateUpdate(p. 434), OnlineUncoordinatedCostEstimation(p. 284), TRMIntruderState(p. 512), OnlineCoordinatedActionCostEstimation(p. 312)

Type 67 | OnlineCostState This data structure is used within the TRM to store and pass context-specific state information. It contains all of the context-specific state information for the algorithms that apply different online costs in ONLINEUNCOORDINATEDCOSTESTIMATION and ONLINECOSTESTIMATION. See the TRM Context-Specific State Data Structures in Appendix D.6 for descriptions of each context-specific state data structure.

```
mutable struct  OnlineCostState
    alt_inhibit::Union{AltitudeInhibitCState(p. 527),Nothing}  # Read-only
    advisory_restart::AdvisoryRestartCState(p. 526)
    initialization::InitializationCState(p. 529)
    restrict_coc::RestrictCOCCState(p. 531)
    max_reversal::MaxReversalCState(p. 530)
    own_response_est::ResponseEstimationCState(p. 531)
    int_response_est::ResponseEstimationCState(p. 531)
    compatibility_cost::CompatibilityCState(p. 528)
    sa01_heuristic::SA01HeuristicCState(p. 531)
    bad_transition::BadTransitionCState(p. 527)
    crossing_no_alert::CrossingNoAlertCState(p. 529)
    prevent_early_coc::PreventEarlyCOCCState(p. 530)
    coord_ra_deferral::CoordinatedRADeferralCState(p. 528)
    altitude_dependent_coc::AltitudeDependentCOCCState(p. 527)
    safe_crossing_ra_deferral::SafeCrossingRADeferralCState(p. 532)
    critical_interval_protection::CriticalIntervalProtectionCState(p. 529)
    time_based_non_compliance::TimeBasedNonComplianceCState(p. 532)
    coord_delay::CoordinationDelayCState(p. 528)
#
    OnlineCostState() = new(
                                nothing,
                                AdvisoryRestartCState(p. 526)(),
                                InitializationCState(p. 529)(),
                                RestrictCOCCState(p. 531)(),
                                MaxReversalCState(p. 530)(),
                                ResponseEstimationCState(p. 531)(),
                                ResponseEstimationCState(p. 531)(),
                                CompatibilityCState(p. 528)(),
                                SA01HeuristicCState(p. 531)(),
                                BadTransitionCState(p. 527)(),
                                CrossingNoAlertCState(p. 529)(),
                                PreventEarlyCOCCState(p. 530)(),
                                CoordinatedRADeferralCState(p. 528)(),
                                AltitudeDependentCOCCState(p. 527)(),
                                SafeCrossingRADeferralCState(p. 532)(),
                                CriticalIntervalProtectionCState(p. 529)(),
                                TimeBasedNonComplianceCState(p. 532)(),
                                CoordinationDelayCState(p. 528)( false )
                            )
end
```

Referenced In: OnlineUncoordinatedActionCostEstimation(p. 287), TRMIntruderStateUpdate(p. 434), OnlineUncoordinatedCostEstimation(p. 284), TRMIntruderState(p. 512), IntruderPrep(p. 254), SetBeliefBasedOnlineCostState(p. 271), SetVerticalTRMSurvOnlyMode(p. 247), OnlineCoordinatedActionCostEstimation(p. 312), StateEstimation(p. 257)

Type 68 | PolicyStateBelief This data structure is used within the TRM to pair a policy state vector to its weight.

```
mutable struct  PolicyStateBelief
    states::Vector{R}   # Policy state vector
    weight::R           # Weight of this sample
#
    PolicyStateBelief() =
        new( zeros( POLICY_LAST ), 0.0 )
    PolicyStateBelief( states::Vector{R}, weight::R ) =
        new( states, weight )
end
```

Referenced In: DiscretizePolicyState(p. 192), CreatePolicyStateBeliefs(p. 186), ApplyStateDependentOnlineCosts(p. 193), GetHorizontalPolicyCosts(p. 184), GetPolicyTableCosts(p. 190), UpdatePolicySpeedBinIndividual(p. 192), GetHorizontalCoordinationPolicy(p. 223)

Type 69 | PolicyUInt8DataTable This data structure is used within the TRM to store the horizontal coordination policy table used to determine the intruder RA sense. See the Data Table Format Specification in Appendix F.

```
mutable struct  PolicyUInt8DataTable
    cut_names::Vector{String} # The names associated with the cut dimensions
    cut_counts::Vector{Z}     # The number of cut points for each dimension
    cuts::Vector{R}           # All the cut points for all the dimensions
    data::Vector{UInt8}       # Contents of the block dictionary data field
#
    PolicyUInt8DataTable() =
        new( String[], Z[], R[], UInt8[] )
    PolicyUInt8DataTable( cut_names::Vector{String}, cut_counts::Vector{Z}, cuts::Vector{R}, data::Vector{UInt8
        } ) =
        new( cut_names, cut_counts, cuts, data )
end
```

Referenced In: GetHorizontalCoordinationPolicy(p. 223)

Type 70 | RDataTable This data structure is used within the TRM to store the entry distribution tables used for tau estimation and the offline cost tables used for offline cost estimation. See the Data Table Format Specification in Appendix F.

```
mutable struct  RDataTable
    cut_names::Vector{String} # The names associated with the cut dimensions
    cut_counts::Vector{Z}     # The number of cut points for each dimension
    cuts::Vector{R}           # All the cut points for all the dimensions
    index::Vector{UInt32}     # Offsets into data, may contain all 0s
    data::Vector{Float16}     # Contents of the block dictionary data field
#
    RDataTable() =
        new( String[], Z[], R[], UInt32[], Float16[] )
    RDataTable( cut_names::Vector{String}, cut_counts::Vector{Z}, cuts::Vector{R}, index::Vector{UInt32},
        data::Vector{Float16} ) =
        new( cut_names, cut_counts, cuts, index, data )
end
```

Referenced In: RDataTableScaled(p. 522), GetPolicyTableCosts(p. 190), OfflineCostEstimation(p. 277), TRMOfflineTable(p. 525), LookupOfflineCost(p. 281)

Type 71 | RDataTableScaled This data structure is used within the TRM to store the entry distribution tables used for tau estimation and the offline cost tables used for offline cost estimation. See the Data Table Format Specification in Appendix F.

```
mutable struct  RDataTableScaled
    cut_names::Vector{String}                   # The names associated with the cut dimensions
    cut_counts::Vector{Z}                       # The number of cut points for each dimension
    scaled_policy_cuts_set::Vector{Vector{R}}   # All the cut points for all the dimensions for all the
         scales for RAs
    index::Vector{UInt32}                       # Offsets into data, may contain all 0s
    data::Vector{Float16}                       # Contents of the block dictionary data field
#
    RDataTableScaled() =
        new( String[], Z[], Vector{Vector{R}}(undef, 0), UInt32[], Float16[] )
    function RDataTableScaled( data_table::RDataTable(p. 521), scales::Matrix{R}, vertical_scaling::Bool )
        vertical_scaling_option_count::Z = size(params().threat_resolution.vertical_scaling.values, 1)
        if vertical_scaling
            (scaled_policy_cuts_set::Vector{Vector{R}}) = CreateVerticalScaledCutsSets(p. 578)(data_table.cut_names
                 , data_table.cut_counts, data_table.cuts, scales )
        else
            scaled_policy_cuts_set = CreateHorizontalScaledCutsSets(p. 577)( data_table.cut_counts, data_table.cuts
                 , scales )
        end
        new( data_table.cut_names, data_table.cut_counts, scaled_policy_cuts_set, data_table.index,
             data_table.data )
    end
end
```

Referenced In: GetTableMaxCutValue(p. 280), DetermineTauVerticalDistribution(p. 265), SelectScaledPolicyBins(p. 185), InterpolateBlocks(p. 266), GetEntryDistributionTables(p. 264), GetPolicyTableCosts(p. 190), LookupEntryDistribution(p. 266), OfflineCostEstimation(p. 277), TauEstimation(p. 263), GetEquivClassTableMaxCutValues(p. 279), TRMOfflineTable(p. 525), GetOfflineCostTauBeliefs(p. 278), LookupOfflineCost(p. 281)

Type 72 | TauBelief This data structure is used within the TRM to store and pass tau belief information. It contains the data elements that represent a weighted sample from a distribution of tau values.

```
mutable struct  TauBelief
    tau::R         # Time to closest point of approach (sec)
    weight::R      # Weight for this sample [0-1]
#
    TauBelief( tau::R, weight::R ) = new( tau, weight )
    TauBelief() = new( 0.0, 0.0 )
end
```

Referenced In: DetermineAltitudeDependentCOCFactor(p. 273), GroundPointObstacleTauEstimation(p. 267), StateAndCostEstimation(p. 251), OnlineUncoordinatedCostEstimation(p. 284), PointObstacleTauEstimation(p. 269), GetPolicyTableCosts(p. 190), SetBeliefBasedOnlineCostState(p. 271), SetVerticalTRMSurvOnlyMode(p. 247), ExpectedTau(p. 412), OfflineCostEstimation(p. 277), StateEstimation(p. 257), TauEstimation(p. 263), OnlineCostEstimation(p. 310), GetOfflineCostTauBeliefs(p. 278), LookupOfflineCost(p. 281)

Type 73 | TRMIndivAdjustState This data structure is used within GENERATERAMODEOUTPUT for storing local state and passing information to helper algorithms for update.

```
mutable struct  TRMIndivAdjustState
    ra_prev_count::Z          # Individual RAs that transitioned to COC
    force_silent_count::Z     # RAs in ra_prev_count excluded from global RA determination
    upsense::Bool             # Presence of an individual up-sense RA
    downsense::Bool           # Presence of an individual down-sense RA
    idx_threat_new::Z         # Index of previously unreported threat (used for TID)
    idx_threat_upd::Z         # Index of an unprioritized threat (used for TID)
    idx_threat_last::Z        # Index of threat reported using TID on the previous cycle
#
    TRMIndivAdjustState() =
       new( 0, 0, false, false, -1, -1, -1 )
end
```

Referenced In: GenerateRAModeOutput(p. 378), AdjustRAModeIntruderOutput(p. 379), UpdateIndivAdjustThreatInfo(p. 380), UpdateIndivAdjustCounts(p. 381)

Type 74 | TRMIntruderData This data structure is used within the TRM, during output processing, for storing and passing information about the individual intruder.

```
mutable struct  TRMIntruderData
    id::UInt32    # Unique id for this intruder
    sense::Symbol # Up or down sense of individual RA
    coordination::TRMCoordinationInterrogationData(p. 504)
                  # Coordination data for this intruder
    display::TRMIntruderDisplayData(p. 503)
                  # Display data for this intruder
    designation::TRMIntruderDesignationData(p. 505)
                  # Xo designation data for this intruder
#
    TRMIntruderData() = new( 0, :None,
                             TRMCoordinationInterrogationData(p. 504)(),
                             TRMIntruderDisplayData(p. 503)(),
                             TRMIntruderDesignationData(p. 505)() )
    TRMIntruderData( id::UInt32, sense::Symbol, v2v_uid_own::UInt32, id_directory::IntruderIDDirectory(p. 482),
                     cvc::UInt32, vrc::UInt32, vsb::UInt32, multithreat::Bool,
                     coordination_msg::UInt32, tds::R, code::UInt8) =
        new( id, sense,
             TRMCoordinationInterrogationData(p. 504)( id, cvc, vrc, vsb, multithreat, v2v_uid_own, id_directory,
                 coordination_msg ),
             TRMIntruderDisplayData(p. 503)( id, id_directory, tds, code, CLASSIFICATION_NONE, SOURCE_MODES ),
             TRMIntruderDesignationData(p. 505)() )
end
```

Referenced In: SetInvalidIntruder(p. 395), VerticalTRMUpdate(p. 241), DetermineGroundAlert(p. 248), VerticalTRMUpdatePrep(p. 587), SetDroppedIntruder(p. 394), GenerateRAModeOutput(p. 378), SetIntruderVerticalScores(p. 372), AdjustRAModeIntruderOutput(p. 379), SetVerticalTRMSurvOnlyMode(p. 247), DroppedIntrudersAdjustment(p. 393), GenerateVTRMOutput(p. 376), UpdateIndivAdjustThreatInfo(p. 380), GenerateSurvOnlyModeOutput(p. 382)

Type 75 | VTRMReport This data structure is produced by the TRM at the end of each processing cycle. It contains information for the vertical onboard pilot display, coordination, RA broadcast, and RA transmission to the Ground Station.

```
mutable struct  VTRMReport
    display::VTRMDisplayData(p.502) # Data for the onboard pilot display
    coordination::Vector{TRMCoordinationInterrogationData(p.504)}
                                # Per intruder data for outgoing coordination messages
    designation::TRMDesignationData(p.505)
                                # Target designation data for ASA system
    broadcast_msg::sXuTRMBroadcastData(p.507)
                                # Data for sXu broadcast
#
    VTRMReport() =
         new( VTRMDisplayData(p.502)(),
              TRMCoordinationInterrogationData(p.504)[],
              TRMDesignationData(p.505)(),
              sXuTRMBroadcastData(p.507)() )
    VTRMReport( display_logic::DisplayLogic(p.516) ) =
         new( VTRMDisplayData(p.502)( display_logic ),
              TRMCoordinationInterrogationData(p.504)[],
              TRMDesignationData(p.505)(),
              sXuTRMBroadcastData(p.507)() )
end
```

Referenced In: SetReportRATerminationBits(p.404), GenerateBlendedsXuTRMOutput(p.396), VerticalTRMUpdate(p.241), GenerateRAModeOutput(p.378), SetReportDataOnRATerm(p.403), DetermineDimensionalRAState(p.407), GenerateVTRMOutput(p.376), sXuTRMBlendedReportPrep(p.397), sXuTRMUpdate(p.164), GenerateSurvOnlyModeOutput(p.382), GenerateStandbyModeOutput(p.383)

Type 76 | VerticalRAOutputState This data structure is used within the TRM to store information associated with generation of messages.

```
mutable struct  VerticalRAOutputState
    vert_ra_data::VTRMRAData(p.508)        # Settings of Vertical RA data
    tgtid_tid::Z                     # Target id associated with the TID value
#
    VerticalRAOutputState() = new( VTRMRAData(p.508)(), Z(0) )
end
```

Referenced In: TRMOwnState(p.511)

Type 77 | TRMOfflineTable This data structure stores the parameters for Offline Data Origami tables. It contains information about the actions, equivalance classes, and miniblock tables.

```
mutable struct  TRMOfflineTable
  N_allowable_actions::Vector{Z}        # number of allowable actions for each row
                                        #     in the allowable_actions matrix
  minblocks_index::Matrix{Z}            # indexes into the minblocks table, 1-based
  minblocks_table_content::RDataTable(p. 521)                 # contents of the minblocks table file
  equiv_class_table_content::Vector{RDataTableScaled(p. 522)}  # contents of equivalence class tables file
  action_pattern_number::Vector{Z}      # action pattern numbers, 1-based
  equiv_class_number::Matrix{Z}         # equivalence class for each transition, 1-based
  allowable_actions::Matrix{Z}          # allowable action transitions for each action, 1-based
  TRMOfflineTable(N_allowable_actions::Vector{Z}, minblocks_index::Matrix{Z},
                  minblocks_table_content::RDataTable(p. 521),
                  equiv_class_table_content::Vector{RDataTableScaled(p. 522)}, action_pattern_number::Vector{Z},
                  equiv_class_number::Matrix{Z}, allowable_actions::Matrix{Z}) =
      new(N_allowable_actions, minblocks_index, minblocks_table_content, equiv_class_table_content,
          action_pattern_number, equiv_class_number, allowable_actions)
end
```

Referenced In: GetPolicyTableCosts(p. 190), OfflineCostEstimation(p. 277), LookupOfflineCost(p. 281)

Type 78 | IntruderPublishedRaState Definition for the RA report state tracking. Allows individual threats that have active RAs to be published in the order of the RA initialization and is utilized during report blending.

```
mutable struct  IntruderPublishedRaState
    id::UInt32                  # Unique target identifier
    has_been_published::Bool    # Indication of RA report publish state
    included_in_prev_tid1::Bool # Used to signal that this intruder was included in
                                #     RA TID field One in the prevvious cycles report
    included_in_prev_tid2::Bool # Used to signal that this intruder was included in
                                #     RA TID field two in the prevvious cycles report
#
    IntruderPublishedRaState(id::UInt32) = new( id, false, false, false)
end
```

Referenced In: UpdateRaPubState(p. 398), SetRaPublishedState(p. 402), SetBlendedIntruderThreatData(p. 400), sXuTRMState(p. 510)

D.6 TRM Context-Specific State Data Structures

While some state variables are used throughout the TRM, others are specific to the algorithms in which they are used. To maintain separation of the algorithms, separate context-specific state data structures are defined for each algorithm. This appendix defines these context-specific data structures. The name of each data structure reflects the algorithm that uses it. 'CState' in the data structure name stands for Cost State. For example, ACTIONARBITRATIONCSTATE (Type 79) is the cost state data structure used within ACTIONARBITRATION (Algorithm 327). ADVISORYRESTARTCSTATE (Type 81) is the cost state data structure used within ADVISORYRESTARTCOST (Algorithm 269).

Most of the context-specific data structures are associated with an individual intruder and are used when determining online cost. They are stored in the ONLINECOSTSTATE data structure.

Other context-specific data structures are used to maintain state for specific algorithms associated with the multi-threat logic. They are stored in TRMOWNSTATE or TRMINTRUDERSTATE.

Type 79 | ActionArbitrationCState This data structure stores the state information for the individual intruder. It contains information about the advisory action, vertical rate limits, and acceleration. For multiple intruders, the intruder with the worst case action is flagged with *was_worst_case*. This cost state is used by TRMINTRUDERSTATEUPDATE.

```
mutable struct  ActionArbitrationCState
    action::Z            # advisory action
    dz_min::R            # advisory minimum vertical rate (ft/s)
    dz_max::R            # advisory maximum vertical rate (ft/s)
    ddz::R               # advisory acceleration (ft/s/s)
    was_worst_case::Bool # had worst case action on previous cycle
#
    ActionArbitrationCState() = new( COC, -Inf, Inf, 0.0, false )
end
```

Referenced In: TRMIntruderStateUpdate(p. 434), TRMIntruderState(p. 512)

Type 80 | ActionArbitrationGlobalCState This data structure stores information about the previous worst case action; this includes a flag to check if MTLO was prohibited, the cost, and the action at the previous cycle. This cost state is used by ACTIONARBITRATION.

```
mutable struct  ActionArbitrationGlobalCState
    mtlo_prohibited_prev::Bool   # MTLO was prohibited on the previous cycle
    worst_case_cost_prev::R      # Value of worst case cost on previous cycle
    worst_case_action_prev::Z    # Worst case action selected on previous cycle
#
    ActionArbitrationGlobalCState() = new( false, -Inf, COC )
end
```

Referenced In: ActionArbitration(p. 355), NoIntrudersAction(p. 342), TRMOwnState(p. 511), DetermineMultiIntruderAction(p. 345), ArbitrateConflictingSenses(p. 357), ActionSelection(p. 341), GenerateSurvOnlyModeOutput(p. 382)

Type 81 | AdvisoryRestartCState This data structure stores information about the resolution advisory on the previous cycle relevant to ADVISORYRESTARTCOST to determine RA duration and generation.

```
mutable struct  AdvisoryRestartCState
    alerted::Bool   # RA generated on previous cycle
    term::Bool      # RA ended on a previous cycle; timer is active
    t_term::R       # Time since the RA ended (s)
#
    AdvisoryRestartCState() = new( false, false, 0.0 )
end
```

Referenced In: UpdateAdvisoryRestartCState(p. 288), AdvisoryRestartCost(p. 288), OnlineCostState(p. 520)

Type 82 | AltitudeDependentCOCCState This data structure stores the cost (to force resolution advisories) associated with ALTITUDEDEPENDENTCOCCOST.

```
mutable struct  AltitudeDependentCOCCState
    scaled_cost_coc_ra::R      # Cost to be applied to force RAs
#
    AltitudeDependentCOCCState() = new( 0.0 )
end
```

Referenced In: OnlineCostState(p. 520)

Type 83 | AltitudeInhibitCState This data structure stores information about ownship altitude inhibits. This includes a vector of flags to check whether or not an altitude inhibit was in effect, the initialization of the altitude inhibit cost state, and descend inhibits. This cost state is used by ALTITUDEINHIBITCOST.

```
mutable struct  AltitudeInhibitCState
    inhibited::Vector{Bool}  # Whether each altitude inhibit is in effect
    initialized::Bool        # Whether the AltitudeInhibitCost has been initialized
    ldi::UInt8               # Low-level Descend Inhibits
    invalid_agl_cycles::Z    # number of cycles without valid height-AGL data
#
    AltitudeInhibitCState( params::paramsfile_type, mode_idx::Z ) = new( ones( Bool, length( params.modes[
        mode_idx].cost_estimation.online.altitude_inhibit.B_init ) ), false, LDI_ALL, AGL_LOST )
end
```

Referenced In: VerticalTRMUpdatePrep(p. 587), TRMOwnState(p. 511), UpdateAltitudeInhibitCState(p. 244), AltitudeInhibitCost(p. 290), OnlineCostState(p. 520)

Type 84 | BadTransitionCState This data structure stores the cost state associated with BADTRANSITIONCOST. This includes the vertical rate limits, sense, maintain information on the previous RA cycle.

```
mutable struct  BadTransitionCState
    dz_min_prev::R               # Min vertical rate for RA on previous cycle (ft/s)
    dz_max_prev::R               # Max vertical rate for RA on previous cycle (ft/s)
    sense_own_prev::Symbol       # Up/down RA sense on previous cycle
    ra_is_maintain_prev::Bool    # Maintain RA on previous cycle
#
    BadTransitionCState() = new( -Inf, Inf, :None, false )
end
```

Referenced In: UpdateBadTransitionCState(p. 292), BadTransitionCost(p. 291), OnlineCostState(p. 520), BadMaintainTransitionCost(p. 293)

Type 85 | CompatibilityCState This data structure is the cost state associated with the COMPATIBILITYCOST. The state includes the costs of initiating resolution advisories and issuing subsequent advisories. The state also includes a counter since the first VRC was received.

```
mutable struct  CompatibilityCState
    t_since_first_vrc::Z         # Time since the first VRC was received (s)
    c_slave_init::R              # Cost of initiating an RA
    c_slave_init_crossing::R     # Cost of initiating an RA when there is a crossing
    c_slave_sub::R               # Cost for a subsequent RA
    c_slave_sub_noncrossing::R   # Cost for a subsequent RA when there is no crossing
    c_slave_sub_no_response::R   # Cost for any RA against a non-responsive intruder
#
    CompatibilityCState() = new( 0, 0.0, 0.0, 0.0, 0.0, 0.0 )
end
```

Referenced In: OnlineCostState(p. 520), UpdateCompatibilityCState(p. 320), CompatibilityCost(p. 319)

Type 86 | CoordinationDelayCState This data structure is the cost state associated with COORDINATIONDELAYCOST. It stores the delay counter for prohibiting resolution advisories from being issued immediately, as well as a flag to use the counter.

```
mutable struct  CoordinationDelayCState
    t_count::Z             # CoordinationDelay counter (s)
    is_count_enabled::Bool # Whether the CoordinationDelay counter should be used
#
    CoordinationDelayCState( is_count_enabled::Bool ) = new( 0, is_count_enabled )
end
```

Referenced In: UpdateCoordinationDelayCState(p. 323), OnlineCostState(p. 520), IntruderPrep(p. 254), CoordinationDelayCost(p. 323)

Type 87 | CoordinatedRADeferralCState This data structure is the cost state associated with COORDINATEDRADEFERRALCOST, and contains the cost for a resolution advisory deferral.

```
mutable struct  CoordinatedRADeferralCState
    deferral_cost::R          # Cost to defer an RA
#
    CoordinatedRADeferralCState() = new( 0.0 )
end
```

Referenced In: OnlineCostState(p. 520), UpdateCoordinatedRADeferralCState(p. 322), CoordinatedRADeferralCost(p. 321)

Type 88 | CriticalIntervalProtectionCState This data structure is the cost state associated with CRITICALINTERVALPROTECTIONCOST. The structure stores the estimates of the intruder's position, angle, and speed away from the ownship, as well as flags to determine whether to force an alert, climb or descend, or increase climb or descend.

```
mutable struct  CriticalIntervalProtectionCState
    angle::R                    # Estimated angle to intruder (0-pi) (|rad|)
    range::R                    # Estimated ground range to intruder (|ft|)
    speed::R                    # Estimated relative ground speed of intruder (|ft/s|)
    force_alert_prev::Bool      # Value of force_alert for last cycle
    force_alert::Bool           # Force an alert
    force_climbdescend::Bool    # Force a Climb or Descend
    force_inc_climbdescend::Bool # Force an Increase Climb or Descend
#
    CriticalIntervalProtectionCState() = new( NaN, NaN, NaN, false, false, false, false )
end
```

Referenced In: CriticalIntervalRequiresVerticalDivergence(p. 298), CriticalIntervalProtectionCost(p. 294), OnlineCostState(p. 520), UpdateCriticalIntervalProtectionCState(p. 296), IsOutsideCriticalInterval(p. 297)

Type 89 | CrossingNoAlertCState This data structure is the cost state associated with CROSSINGNOALERTCOST; it contains flags for determining crossing based on ownship belief states and geometry.

```
mutable struct  CrossingNoAlertCState
    is_crossing::Bool                       # Crossing is anticipated
    is_crossing_prev::Bool                  # Value of is_crossing on last cycle
    vrc_int_prev::UInt32                    # Received VRC on the previous cycle
    is_crossing_caused_by_geometry::Bool    # Crossing is due to geometry
#
    CrossingNoAlertCState() = new( false, false, 0, false )
end
```

Referenced In: CrossingNoAlertCost(p. 324), OnlineCostState(p. 520), UpdateCrossingNoAlertCState(p. 325)

Type 90 | InitializationCState This data structure is a cost state associated with INITIALIZATIONCOST; it stores the processing cycle threshold that prohibits resolution advisories from being issued.

```
mutable struct  InitializationCState
    t_count::Z          # Initialization counter (s)
#
    InitializationCState() = new( 0 )
end
```

Referenced In: UpdateInitializationCState(p. 261), InitializationCost(p. 299), OnlineCostState(p. 520)

Type 91 | MaxReversalCState This data structure is the cost state associated with MAXREVERSALCOST. The structure holds information about the reversal history as well as vertical resolution advisory.

```
mutable struct  MaxReversalCState
    num_reversals::Z           # Count of reversals
    crossed_thres_time::R      # Time since the number of reversals reached the limit (s)
    sense_own_prev::Symbol     # Vertical RA up/down sense for ownship
    sense_int::Symbol          # Vertical RA up/down sense based on intruder VRC
#
    MaxReversalCState() = new( 0, 0, :None, :None )
end
```

Referenced In: UpdateMaxReversalCState(p. 327), MaxReversalCost(p. 326), OnlineCostState(p. 520)

Type 92 | MultithreatCostBalancingCState This data structure is the cost state associated with MULTITHREATCOSTBALANCING. It contains flags for applying hysteresis to the MTLO cost threshold and whether or not an MTLO is allowed at the current cycle.

```
mutable struct  MultithreatCostBalancingCState
    costs_allow_mtlo_prev::Bool       # MTLO allowed based on costs on previous cycle
    above_abs_threshold_prev::Bool    # Whether to apply hysteresis for cost threshold
#
    MultithreatCostBalancingCState() = new( true, false )
end
```

Referenced In: MultithreatCostBalancing(p. 354), NoIntrudersAction(p. 342), TRMOwnState(p. 511), ActionSelection(p. 341), GenerateSurvOnlyModeOutput(p. 382)

Type 93 | PreventEarlyCOCCState This data structure is the cost state associated with PREVENTEARLYCOCCOST; this structure contains information about the ground range to the intruder and a flag for determining if an early COC is issued.

```
mutable struct  PreventEarlyCOCCState
    range::R                        # Estimated ground range to intruder (|ft|)
    range_prev::R                   # Estimated ground range to intruder from last cycle (|ft|)
    t_consec_range_divergence::Z    # Time since ground range to intruder began diverging (s)
    is_early_coc::Bool              # Issuing Clear of Conflict would be considered early
#
    PreventEarlyCOCCState() = new( NaN, NaN, 0, false )
end
```

Referenced In: UpdatePreventEarlyCOCCState(p. 301), PreventEarlyCOCCCost(p. 299), OnlineCostState(p. 520)

Type 94 | ResponseEstimationCState This data structure is the cost state associated with OWNRESPONSEESTIMATION and INTRUDERRESPONSEESTIMATION, and is used to store information about the responsiveness of the aircraft.

```
mutable struct  ResponseEstimationCState
    t_same_sense::Z              # Amount of time at the same up/down sense (s)
    sense_prev::Symbol           # The previous RA sense
    dz_after_wait::R             # Observed vertical rate (ft/s)
    is_responding_prev::Bool     # Deemed to be responsive on last cycle
    dz_response_initial::R       # Expected initial vertical rate when responsive (ft/s)
    dz_response_max::R           # Maximum observed vertical rate when responsive (ft/s)
    t_dz_response_max::Z         # Value of t_same_sense when dz_response_max was observed (s)
#
    ResponseEstimationCState() = new( 0, :None, NaN, true, NaN, NaN, 0 )
end
```

Referenced In: OwnResponseEstimation(p. 315), IntruderResponseEstimation(p. 314), DetermineResponse(p. 316), OnlineCostState(p. 520)

Type 95 | RestrictCOCCState This data structure is a cost state associated with RESTRICTCOCDUETOREVERSAL; it checks if the aircraft was COC (as well as the sense of the RA) at the previous cycle.

```
mutable struct  RestrictCOCCState
    coc_prev::Bool             # Clear of conflict on previous cycle
    sense_own_prev::Symbol     # Up/down sense of previous global RA
#
    RestrictCOCCState() = new( true, :None )
end
```

Referenced In: RestrictCOCDueToReversal(p. 329), UpdateRestrictCOCDueToReversalCState(p. 329), OnlineCostState(p. 520)

Type 96 | SA01HeuristicCState This data structure is the cost state associated with SA01HEURISTIC. It contains information about the ground range to the intruder and whether or not a reversal is forced.

```
mutable struct  SA01HeuristicCState
    range::R                  # Estimated ground range to intruder (|ft|)
    force_reversal::Bool      # Force a reversal
#
    SA01HeuristicCState() = new( NaN, false )
end
```

Referenced In: UpdateSA01HeuristicCState(p. 331), OnlineCostState(p. 520), SA01Heuristic(p. 330)

Type 97 | SafeCrossingRADeferralCState This data structure is the cost state associated with SAFECROSSINGRADEFERRALCOST; it contains the cost for deferring an RA.

```
mutable struct  SafeCrossingRADeferralCState
    c_deferral::R                # Cost to be applied to defer an RA
#
    SafeCrossingRADeferralCState() = new( 0.0 )
end
```

Referenced In: UpdateSafeCrossingRADeferralCState(p. 303), SafeCrossingRADeferralCost(p. 302), OnlineCostState(p. 520)

Type 98 | TimeBasedNonComplianceCState This data structure is the cost state for TIMEBASEDNONCOMPLIANCECOST. It contains information about the current resolution advisory, *tau*, the desired advisory vertical rate range from two cycles previous, and a cost for issuing the current RA.

```
mutable struct  TimeBasedNonComplianceCState
    current_ra_max_tau::R           # Maximum tau observed during the current advisory (s)
    current_ra_min_tau_time::Z      # Amount of time spent at or below the minimum tau (s)
    current_ra_dz_initial_delta::R  # Vertical rate difference required to be compliant when
                                    #   the advisory is issued (ft/s)
    dz_min_prev2::R                 # Minimum desired rate from advisory two cycles previous (ft/s)
    dz_max_prev2::R                 # Maximum desired rate from advisory two cycles previous (ft/s)
    current_ra_cost::R              # Cost applied to the current advisory
#
    TimeBasedNonComplianceCState() = new( 0.0, 0, 0.0, -Inf, Inf, 0.0)
end
```

Referenced In: DetermineOwnNonComplianceFactor(p. 337), OnlineCostState(p. 520), TimeBasedNonComplianceCost(p. 333), UpdateTimeBasedNonComplianceCState(p. 335)

E Parameter File Specification

E.1 ACAS X Parameter Data Item File (PDIF) Checksums

The ACAS X design is highly dependent on the Parameter Data Item Files (PDIFs), which are used in the ACAS X equipment, to resolve conflicts in real time. The values of all the data elements for sXu are found in the "Parameter file" PDIF. Those data element values include the names of the other PDIFs used in the ACAS X system. The data elements in the "Parameter file" PDIF and their values are described in the other two sections in this Appendix. In the "Parameter file" PDIF, data elements that are 1-D arrays are represented in brackets separated by commas or spaces. 2-D arrays are represented in tables. The table below lists the PDIFs used in the ACAS sXu equipment along with the size in bytes, MD5 checksum, and SHA256 checksum.

Unlike ACAS Xa/Xo (RTCA/DO-385), there is no separate PDIF "Horizontal entry table for active surveillance only". In ACAS Xa/Xo, that separate table is used to support Xo CSPO-3000 only. In ACAS sXu, target designation for special operations (ACAS Xo) is a provision for potential modification, not a standard functionality (see ACAS sXu MOPS Vol I, section 1).

Table 47. Parameter Data Item File (PDIF) List

PDIF	File Name	Bytes	Checksum
VTRM minimum blocks index table	q_sxuvtrm_v4r0_73ec_minBlocks.dat	3216534	*md5:*c0d6b5c04edfe2dc7a0bd4af5d5e7db5 *sha256:*28e8ecb1b7a78b955e5f9082866b05e2 8c85bf7859e3fedc648b24f04cb9ebde
VTRM equivalence class cost table	q_sxuvtrm_v4r0_73ec_allSplits.dat	93318236	*md5:*994266134abeb9808c8739bdd0826617 *sha256:*34f3e2ebe6f416db511234c899797d3d dde59e11aa42dc9a95627a1c1c964704
Horizontal entry table	entry_sxuvtrm_v4r0_compressed.dat	26128095	*md5:*001a1d0478143a34355245f7f72bab14 *sha256:*5bca98f762ec9b43ea57c407efe65ae3 5195f37f9c7724213a0433d45b6d8df7
Vertical entry table	entry_vert_sxuvtrm_v4r0_compressed.dat	2122250	*md5:*91ddd6d8b09f15c3e7c2a6545b70aae6 *sha256:*ad028c52311ff83347570cd1a7e6c199 62030939691fe90beed9c2c85749ee0c
HTRM minimum blocks index table	q_sxuhtrm_v4r0_16ec_minBlocks_NoMB.dat	76832072	*md5:*e1ff3b8ee0848c2645cd810c3db62c7d *sha256:*da936ef9d0e9978a61228df118294f74 9da16ef4a127d307140a32861e6b29c3
HTRM equivalence class cost table	q_sxuhtrm_v4r0_16ec_allSplits_NoMB.dat	768328176	*md5:*31a4d9c2fb176ff59ac37ce33413b874 *sha256:*0e04187b26db2459c7d630696187f0ed 9d70b24b2c85dfecabe09bcc67d1ffcf
Horizontal coordination table	xuhtrm_coord_sxu_v4r0.bin	271971216	*md5:*8acbc1956f963c0921fa7aecbb77419c *sha256:*39b09bb9c86bc0d9008d6bdd47f240cc 39a53eb0b405045dbbcd456e6a0c5310
Parameter file	paramsfile_sxu_origami.txt	35563	*md5:*6c964b4653b6d20b9ee3c5b15857e53a *sha256:*ef4c1051b88700d82d1aa39b23e336bf 08af5aed54b073525c16246a132e4315

E.2 STM Parameters

A full specification of all the parameters used in the STM can be found in the "Parameter File" PDIF, *paramsfile_sxu_origami.txt*. The parameter file contains a hierarchy of structures encoded in the JavaScript Object Notation (JSON) [15]. The JSON parameter hierarchy is organized as follows:

- **coordination**: Information for coordination processing
  - **hor_intent**: map of codes in the coordination message into the horizontal intent
    value: *[1, 2, 0, 0, 1, 2, 0]*
  - **parity_table_hor**: table used to encode the horizontal sense bits of the coordination message
    value: *[0, 11, 19, 24, 28, 23, 15, 4, 13, 6, 30, 21, 17, 26, 2, 9, 21, 30, 6, 13, 9, 2, 26, 17, 24, 19, 11, 0, 4, 15, 23, 28, 25, 18, 10, 1, 5, 14, 22, 29, 20, 31, 7, 12, 8, 3, 27, 16, 12, 7, 31, 20, 16, 27, 3, 8, 1, 10, 18, 25, 29, 22, 14, 5]*
  - **parity_table_vert**: table used to encode the vertical sense bits of the coordination message
    value: *[0, 14, 7, 9, 11, 5, 12, 2, 13, 3, 10, 4, 6, 8, 1, 15]*
  - **timeout**: maximum time allowed between coordination messages before timing out (s)
    value: *6.0*
  - **vert_intent**: map of codes in the coordination message into the vertical intent
    value: *[1, 2, 0]*
- **display**: Information for the display logic
  - **arrow_M**: number of successful vertical rate comparisons necessary to transition the displayed rate
    value: *3*
  - **arrow_N**: number of vertical rate comparisons
    value: *4*
  - **level_arrow_threshold**: vertical rate threshold to indicate intruder is level (ft/s)
    value: *10.0*
- **surveillance**: Information about the parameters used in the STM
  - **agt**: Absolute-Geodetic-Track (AGT) parameters
    - **max_coasts**: Maximum number of coasts before a track file is deleted
      value: *6*
    - **max_outlier_detections**: Number of outlier detections before track reinitialization
      value: *4*
    - **outlier_threshold**: Threshold for outlier detection
      value: *6*
  - **approx_zero**: Approximately zero
    value: *0.001*
  - **correlation**: Correlation processing parameters
    - **M**: Number of successful comparisons to consider correlating tracks
      value: *1*

* **N**: Total number of comparisons
  value: *1*
* **mechanism**: Correlation mechanism look-up table
  value: $\begin{pmatrix} 0.0 & 7.0 & 0.0 & 1.0 & 6.0 & 6.0 & 6.0 & 2.0 & 6.0 & 0.0 & 1.0 & 6.0 \\ 7.0 & 0.0 & 0.0 & 6.0 & 2.0 & 6.0 & 6.0 & 6.0 & 2.0 & 0.0 & 6.0 & 2.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 6.0 & 6.0 & 0.0 & 0.0 & 3.0 & 0.0 & 0.0 \\ 1.0 & 6.0 & 0.0 & 0.0 & 0.0 & 6.0 & 6.0 & 1.0 & 6.0 & 0.0 & 4.0 & 3.0 \\ 6.0 & 2.0 & 0.0 & 0.0 & 0.0 & 6.0 & 6.0 & 6.0 & 2.0 & 0.0 & 3.0 & 5.0 \\ 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 0.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 \\ 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 & 6.0 \\ 2.0 & 6.0 & 0.0 & 1.0 & 6.0 & 6.0 & 6.0 & 1.0 & 6.0 & 0.0 & 1.0 & 6.0 \\ 6.0 & 2.0 & 0.0 & 6.0 & 2.0 & 6.0 & 6.0 & 6.0 & 2.0 & 0.0 & 6.0 & 2.0 \\ 0.0 & 0.0 & 3.0 & 0.0 & 0.0 & 6.0 & 6.0 & 0.0 & 0.0 & 3.0 & 0.0 & 0.0 \\ 1.0 & 6.0 & 0.0 & 4.0 & 3.0 & 6.0 & 6.0 & 1.0 & 6.0 & 0.0 & 4.0 & 3.0 \\ 6.0 & 2.0 & 0.0 & 3.0 & 5.0 & 6.0 & 6.0 & 6.0 & 2.0 & 0.0 & 3.0 & 5.0 \end{pmatrix}$
* **ownship_correlation**: Ownship correlation processing parameters
  - **M**: Number of successful comparisons to consider correlating tracks
    value: *1*
  - **N**: Total number of comparisons
    value: *1*
  - **max_range**: Maximum slant-range threshold used to prevent AGT tracks from correlation to ownship (ft)
    value: *3038.06*
  - **max_size_max_range_vec**: Maximum size of the maximum-slant-range vector
    value: *3*
  - **range_gate**: Maximum range window for efficient iteration (ft)
    value: *5001*
  - **unique_multiplier**: Scaling applied to correlation thresholds based on track uniqueness
    value: *0.5*
  - **unique_time**: Time threshold used to mark Target as unique (s)
    value: *10*
* **range_gate**: Maximum range window for efficient iteration (ft)
  value: *5001*
* **type_selection**: Parameters for different methods of correlation
  - **ownship_position**: Parameters for correlating tracks to ownship by position

    altitude_gate: Maximum altitude window for efficient iteration (ft)
    value: *5001*

    altitude_inflation: Altitude inflation to account for observation noise (ft)
    value: *28.9*

    correlation_alt_threshold: Score threshold reduction value for NAR tracks
    value: *4*

    correlation_az_threshold: Score threshold reduction value for bearingless tracks
    value: *1*

    correlation_threshold: Score threshold for position correlation based
    value: *6*

range_gate: Maximum range window for efficient iteration (ft)
value: *10001*

- **position**: Parameters for correlating by position

altitude_gate: Maximum altitude window for efficient iteration (ft)
value: *5001*

altitude_inflation: Altitude inflation to account for observation noise (ft)
value: *28.9*

correlation_alt_threshold: Score threshold reduction value for NAR tracks
value: *4*

correlation_az_threshold: Score threshold reduction value for bearingless tracks
value: *1*

correlation_threshold: Score threshold for position correlation based
value: *6*

range_gate: Maximum range window for efficient iteration (ft)
value: *10001*

* **unique_multiplier**: Scaling applied to correlation thresholds based on track uniqueness
value: *0.5*
* **unique_time**: Time threshold used to mark Target as unique (s)
value: *10*

– **decorrelation**: Decorrelation parameters
* **M**: Number of successful comparisons to consider decorrelating tracks
value: *1*
* **N**: Total number of comparisons
value: *1*
* **max_coast_time**: Maximum coast time before a track is decorrelated (s)
value: *1*
* **ownship_decorrelation**: Ownship decorrelation parameters
  - **M**: Number of successful comparisons to consider decorrelating tracks
  value: *2*
  - **N**: Total number of comparisons
  value: *4*
  - **max_coast_time**: Maximum coast time before a track is decorrelated (s)
  value: *1*
  - **unique_multiplier**: Scaling applied to decorrelation thresholds based on track uniqueness
  value: *2*
  - **unique_time**: Time threshold used to mark Target as unique (s)
  value: *10*
* **unique_multiplier**: Scaling applied to decorrelation thresholds based on track uniqueness
value: *2*
* **unique_time**: Time threshold used to mark Target as unique (s)
value: *10*

- **horizontal_adsb**: Horizontal tracking parameters for passive ADS-B surveillance
  - **Q**: Process noise variance (m^2)
    value: *48.0*
  - **cov_inflation_factor**: Posterior state estimate covariance inflation factor
    value: *1.5*
  - **init_nacv**: Initial NACv value assigned to track until receipt of Mode Status Report
    value: *1*
  - **init_sigma_pos**: Position standard deviation used during initialization (m)
    value: *378.26797385620915*
  - **kappa**: Stability factor added during initialization to ensure matrix is positive semidefinite
    value: *1.0e-8*
  - **max_coasts**: Maximum coast time before a track is decorrelated (s)
    value: *6*
  - **max_outlier_detections**: Number of outlier detections before track reinitialization
    value: *2*
  - **outlier_threshold**: Threshold for outlier detection
    value: *16.0*
  - **uat_outlier_threshold**: Threshold for outlier detection for UAT surveillance
    value: *21.1*
  - **update_sigma_pos**: Position standard deviation used during update (m)
    value: *75.65359477124183*
  - **vel_init_lat_thresh**: Limit of absolute latitude value below which a track can be initialized on a single position and velocity report (deg)
    value: *85.0*
- **horizontal_v2v**: Horizontal tracking parameters for passive V2V surveillance
  - **Q**: Process noise variance (m^2)
    value: *16.0*
  - **cov_inflation_factor**: Posterior state estimate covariance inflation factor
    value: *1.0*
  - **covariance_inflation**: Parameters used to inflate the uncertainty used in outlier detection STM Report and outlier detection. They account for error characteristics of passive data
    - **init_NACp_bias**: NACp value used in uncertainty inflation before NACp is initialized for the intruder
      value: *9*
    - **init_NACv_bias**: NACv value used in uncertainty inflation before NACv is initialized for the intruder
      value: *2*
    - **nacp_factor**: factor applied to bias added to uncertainty from NACp
      value: *0.7*
    - **nacv_factor**: factor applied to bias added to uncertainty from NACv
      value: *0.6*
    - **uncompensated_latency**: Parameters used to inflate the uncertainty to account for the effects of uncompensated latency

Q_outlier_ul: Process noise used when determining uncertainty from uncompensated latency during outlier detection (m^2)
value: *121.0*

Q_ul: Process noise used when determining uncertainty from uncompensated latency (m^2)
value: *121.0*

mean_delay_ul: mean time delay from uncompensated latency (s)
value: *0.3*

vel_min_ul: constant velocity added when determining uncertainty from uncompensated latency (mps)
value: *12.0*

* **init_nacv**: Initial NACv value assigned to track until receipt of Mode Status Report
  value: *2*
* **init_sigma_pos**: Position standard deviation used during initialization (m)
  value: *37.826797385620914*
* **kappa**: Stability factor added during initialization to ensure matrix is positive semidefinite
  value: *1.0e-8*
* **max_coasts**: Maximum coast time before a track is decorrelated (s)
  value: *6*
* **max_outlier_detections**: Number of outlier detections before track reinitialization
  value: *2*
* **outlier_threshold**: Threshold for outlier detection
  value: *25.0*
* **update_sigma_pos**: Position standard deviation used during update (m)
  value: *5.0*

– **hybrid_override_timeout**: Time duration after which front end hybrid surveillance indication is no longer valid (s)
  value: *65*
– **max_duplicate_coasts**: Maximum number of coasts before a Mode C track file is deleted when there are multiple Mode C track files
  value: *4*
– **max_heading_coasts**: Maximum number of heading coasts before a track file is deleted
  value: *5*
– **max_intruders**: Maximum number of intruders sent to the TRM and display (excluding point-obstacles)
  value: *30*
– **max_point_obstacles**: Maximum number of point-obstacles sent to the TRM and display
  value: *10*
– **min_extrap_toa_step**: Minimum duration of time since previous track update required to extrapolate track to a TOA (s)
  value: *0.01*
– **min_obs_toa_step**: Minimum duration of time since previous track update required to process an observation (s)

value: *0.01*

- **ornct**: Ownship-Relatative-Non-Cooperative-Track (ORNCT) parameters
  - **close_rng_ft**: Range threshold within which low priority ORNCT track is eligible FOV coast (ft)
    value: *3000*
  - **lag_uncertainty_minimum**: Minimum range and azimuth uncertainty parameters used to account for lag bias errors
    value: *[100.0, 0.0218]*
  - **max_coasts_high_priority**: Maximum number of coasts for ORNCT surveillance intruder existing FOV before a track file is deleted
    value: *12*
  - **max_coasts_normal**: Maximum number of coasts under Normal surveillance before a track file is deleted
    value: *6*
  - **max_outlier_detections**: Number of outlier detections before track reinitialization
    value: *4*
  - **min_FOV_coast_age**: Number of ORNCT reports required for a track to be eligible for extended coasting outside the field of view
    value: *3*
  - **outlier_threshold**: Threshold for outlier detection
    value: *6*
  - **variance_bound_range_dependent**: Multipliers used to calculate the range dependent components of the ORNCT variance threshold
    value: *[0.0175, 0.000306, 0.00131, 0.00131, 0.0175, 0.00131]*
  - **variance_bound_static**: Range below which all components of the ORNCT variance threshold are calculated using static values
    value: *[1.0e15, 1.0e15, 1.0e15, 1.0e15, 1.0e15, 1.0e15]*
  - **variance_check_range_floor**: Multipliers used to calculate the static components of the ORNCT variance threshold
    value: *4000.0*
  - **variance_check_scalar**: Scaling multiplier applied to the threshold used in the ORNCT variance limit check
    value: *4.0*
- **own_history_window**: Time window in seconds to indicate how long to save ownship history (s)
  value: *10*
- **ownship_heading**: Ownship heading tracking parameters
  - **Q**: Process noise variance (rad^2/s^4)
    value: *7.615435494667714e-5*
  - **R**: Observation noise variance (rad^2)
    value: *0.0027415567780803775*
  - **max_outlier_detections**: Number of outlier detections before track reinitialization
    value: *4*

    - **outlier_threshold**: Mahalanobis distance threshold for outlier detection
      value: *1000.0*
    - **var_dhint**: Initial heading rate variance (rad^2/s^2)
      value: *0.0010153913992890287*
  - **ownship_wgs84**: Ownship WGS84 parameters
    - **min_nacp**: Minimum NACp value
      value: *7*
    - **min_nacv**: Minimum NACv value
      value: *1*
    - **timeout**: Time duration after which the ownship WGS84 information is no longer considered valid (s)
      value: *2.7*
  - **psd_stability_factor**: Stability factor to ensure matrices are positive semidefinite during Upper Cholesky Decomposition
    value: *1.0e-8*
  - **report_generation**: Report generation parameters
    - **hor_trm_velmode**: Parameters for determining the horizontal velocity-mode (ground-relative vs wind-relative)
      - **v_hi**: Higher velocity threshold for leaving wind-relative mode - entering ground-relative mode (m/s)
        value: *30.0*
      - **v_lo**: Lower velocity threshold for entering wind-relative mode - leaving ground-relative mode (m/s)
        value: *10.0*
    - **min_adsb_quality**: Minimum ADS-B Data Quality Parameters
      - **M**: Number of successul ADS-B data quality check comparisions needed to consider an ADS-B track validated
        value: *2*
      - **M_revalidate**: the number of times the quality tests must pass out of N in order to validate ADS-B (from the invalid to the valid state)
        value: *3*
      - **N**: Total number of comparisons
        value: *3*
      - **adsb_version**: Minimum allowable ADS-B version
        value: *2*
      - **gva**: Minimum GVA value
        value: *2*
      - **init_M**: Initial M value to validate ADS-B quality during initialization
        value: *1*
      - **init_N**: Initial N value to validate ADS-B quality during initialization
        value: *1*
      - **nacp**: Minimum NACp value
        value: *7*
      - **nacv**: Minimum NACv value
        value: *1*

        - **nic**: Minimum NIC value
          value: *6*
        - **sda**: Minimum SDA value
          value: *1*
        - **sil**: Minimum SIL value
          value: *3*
    - **min_v2v_quality**: Minimum V2V Data Quality Parameters
        - **M**: Number of successul V2V data quality check comparisions needed to consider an V2V track validated
          value: *2*
        - **M_revalidate**: the number of times the quality tests must pass out of N in order to validate V2V (from the invalid to the valid state)
          value: *3*
        - **N**: Total number of comparisons
          value: *3*
        - **init_M**: Initial M value to validate V2V quality during initialization
          value: *1*
        - **init_N**: Initial N value to validate V2V quality during initialization
          value: *1*
        - **nacp**: Minimum NACp value
          value: *7*
        - **nacv**: Minimum NACv value
          value: *1*
        - **nic**: Minimum NIC value
          value: *6*
        - **sda**: Minimum SDA value
          value: *1*
        - **sil**: Minimum SIL value
          value: *3*
    - **optimal_quality_track**: Parameters for the track quality metric computation for selecting optimal track
        - **horizontal_weight**: Weight for horizontal velocity uncertainty component
          value: *0.1*
        - **vertical_weight**: Weight for vertical velocity uncertainty component
          value: *0.9*
    - **optimal_track_history**: Parameters for the track source selection lock hysteresis
        - **M**: Number of successful comparisons to consider a track optimal
          value: *3*
        - **N**: Total number of comparisons
          value: *4*
- **vertical**: Vertical tracking parameters
    - **L**: Number of partitions
      value: *20*
    - **aem_max_pres_alt**: Altitude bands used to apply pressure altitude bias (ft)
      value: *[2300, 5000, 10000, 20000, 41000, 99999]*

* **detections_to_recover**: Number of detections needed to recover from an invalid state
  value: *3*
* **dt_too_small_for_velocity**: Minimum time threshold for seeding range filter velocity by differencing two measurements (s)
  value: *0.9*
* **hae_large_intruder**: Geodetic HAE altitude large intruder tracking parameters
  - **Q**: Process noise variance (ft^2/s^4)
    value: *5.0*
  - **R**: Observation noise variance (ft^2)
    value: *1.69*
  - **max_outlier_detections**: Number of outlier detections before track reinitialization
    value: *2*
  - **max_vfom**: Maximum acceptable Vertical Figure of Merit (m)
    value: *45.0*
  - **max_vpl**: Maximum acceptable Vertical Protection Level (m)
    value: *50.0*
  - **outlier_threshold**: Mahalanobis distance threshold for outlier detection
    value: *49.0*
  - **var_dzint**: Initial altitude rate variance (ft^2/s^2)
    value: *125.0*
  - **var_zint**: Initial altitude variance (ft^2)
    value: *73.0*
* **hae_small_intruder**: Geodetic HAE altitude small intruder tracking parameters
  - **Q**: Process noise variance (ft^2/s^4)
    value: *1.44*
  - **R**: Observation noise variance (ft^2)
    value: *1.69*
  - **max_outlier_detections**: Number of outlier detections before track reinitialization
    value: *2*
  - **max_vfom**: Maximum acceptable Vertical Figure of Merit (m)
    value: *8.0*
  - **max_vpl**: Maximum acceptable Vertical Protection Level (m)
    value: *35.0*
  - **outlier_threshold**: Mahalanobis distance threshold for outlier detection
    value: *121.0*
  - **var_dzint**: Initial altitude rate variance (ft^2/s^2)
    value: *18.0*
  - **var_zint**: Initial altitude variance (ft^2)
    value: *1.69*
* **hae_small_ownship**: Geodetic HAE altitude small ownship tracking parameters
  - **Q**: Process noise variance (ft^2/s^4)
    value: *1.44*

- **R**: 2×2 Observation noise variance (diagonal terms: ft^2, ft^2/s^2)
  value: $\begin{pmatrix} 1.69 & 0.0 \\ 0.0 & 1.0 \end{pmatrix}$
- **max_outlier_detections**: Number of outlier detections before track reinitialization
  value: *2*
- **max_vfom**: Maximum acceptable Vertical Figure of Merit (m)
  value: *8.0*
- **outlier_threshold**: Mahalanobis distance threshold for outlier detection
  value: *121.0*
- **var_dzint**: Initial altitude rate variance (ft^2/s^2)
  value: *18.0*
- **var_zint**: Initial altitude variance (ft^2)
  value: *1.69*

* **outlier_window**: Static outlier window applied during second received update (ft/s)
  value: *356*
* **partition_estimator_threshold**: Quantization threshold for using the partition estimator (ft)
  value: *1*
* **pres_large_intruder**: Pressure altitude large intruder tracking parameters
  - **Q**: Process noise variance (ft^2/s^4)
    value: *5.0*
  - **R**: Observation noise variance (ft^2)
    value: *22.09*
  - **aem_sigma**: Uncertainty due to altitude bias (ft)
    value: *[49.5, 53.7, 60.8, 65.2, 65.2, 65.2]*
  - **max_outlier_detections**: Number of outlier detections before track reinitialization
    value: *2*
  - **outlier_threshold**: Mahalanobis distance threshold for outlier detection
    value: *49.0*
  - **var_dzint**: Initial altitude rate variance (ft^2/s^2)
    value: *125.0*
  - **var_zint**: Initial altitude variance (ft^2)
    value: *73.0*
* **pres_small_intruder**: Pressure altitude small intruder tracking parameters
  - **Q**: Process noise variance (ft^2/s^4)
    value: *1.44*
  - **R**: Observation noise variance (ft^2)
    value: *2.25*
  - **aem_sigma**: Uncertainty due to altitude bias (ft)
    value: *[8.33333, 8.33333, 11.6667, 16.6667, 34.0, 46.0]*
  - **max_outlier_detections**: Number of outlier detections before track reinitialization
    value: *2*

- **outlier_threshold**: Mahalanobis distance threshold for outlier detection
  value: *121.0*
- **var_dzint**: Initial altitude rate variance (ft^2/s^2)
  value: *18.0*
- **var_zint**: Initial altitude variance (ft^2)
  value: *2.25*

* **pres_small_ownship**: Pressure altitude small ownship tracking parameters
  - **Q**: Process noise variance (ft^2/s^4)
    value: *1.44*
  - **R**: Observation noise variance (ft^2)
    value: *2.25*
  - **aem_sigma**: Uncertainty due to altitude bias (ft)
    value: *[8.33333, 8.33333, 11.6667, 16.6667, 34.0, 46.0]*
  - **max_outlier_detections**: Number of outlier detections before track reinitialization
    value: *2*
  - **outlier_threshold**: Mahalanobis distance threshold for outlier detection
    value: *121.0*
  - **var_dzint**: Initial altitude rate variance (ft^2/s^2)
    value: *18.0*
  - **var_zint**: Initial altitude variance (ft^2)
    value: *2.25*
* **stability_exponent**: Exponent used in stability factor
  value: *-38*

E.3 TRM Parameters

A full specification of all the parameters used in the threat resolution module can be found in the "Parameter File" PDIF, *paramsfile_sxu_origami.txt*. The parameter file contains a hierarchy of structures encoded in the JavaScript Object Notation (JSON) [15]. Several special symbols mentioned in the Julia code, like Inf (positive infinity), $-Inf$ (negative infinity), and NaN (not a number), are encoded in JSON using strings. In particular, Inf is represented by the string "_Inf_", $-Inf$ by "-_Inf_", and NaN by "_NaN_".

The JSON parameter hierarchy is organized as follows. The abbreviation "COC" is used in the definitions for "clear of conflict"; which is also treated as a synonym for "no advisory". The abbreviation "MTLO" is used in the definitions for "multi-threat level-off".

- **actions**: Information about the vertical resolution advisories represented in the vertical cost table. When a subfield description does not provide a value, the value is provided in table(s) following the last subfield.
  - **accelerations**: vector of vertical accelerations that are compliant with each resolution advisory. Actual acceleration is dependent upon equipment capabilities and pilot response to the advisory. (ft/s^2). Values are listed in table(s) below.
  - **corrective_rate**: magnitude of target vertical rate for corrective advisories (ft/s)
    value: *16.666*

- **initial_acceleration**: vertical acceleration associated with the first advisory against a given intruder (ft/s^2)
  value: *8.0435*
- **leveloff_acceleration**: acceleration that defines a level-off (ft/s^2)
  value: *8.0435*
- **max_leveloff_rate**: maximum vertical rate that defines a level-off (ft/s)
  value: *4.1667*
- **max_rates**: vector of maximum vertical rates that are compliant with each resolution advisory. The maximum vertical rate for a maintain rate advisory is represented as NaN. (ft/s). Values are listed in table(s) below.
- **min_leveloff_rate**: minimum vertical rate that defines a level-off (ft/s)
  value: *-4.1667*
- **min_rates**: vector of minimum vertical rates that are compliant with each resolution advisory. The minimum vertical rate for a maintain rate advisory is represented as NaN. (ft/s). Values are listed in table(s) below.
- **names**: vector of the possible vertical resolution advisories. The first advisory is always COC. Values are listed in table(s) below.
- **num_actions**: total number of vertical advisory actions
  value: *11*
- **strengthen_rate**: magnitude of target vertical rate for strengthening advisories (ft/s)
  value: *33.333*
- **subsequent_acceleration**: vertical acceleration associated with all corrective advisories following the first advisory against a given intruder (ft/s^2)
  value: *10.7247*

Table 48. actions

no.	names	min_rates	max_rates	accelerations
1	COC	-Inf	Inf	0
2	DNC	-Inf	0	8.0435
3	DND	0	Inf	8.0435
4	MAINTAIN	NaN	NaN	8.0435
5	DES1000	-Inf	-16.666	8.0435
6	CL1000	16.666	Inf	8.0435
7	SDES1000	-Inf	-16.666	10.7247
8	SCL1000	16.666	Inf	10.7247
9	SDES2000	-Inf	-33.333	10.7247
10	SCL2000	33.333	Inf	10.7247
11	MTLO	-4.1667	4.1667	8.0435

- **coordination**: Information for the coordination selection
  - **parity_table_hor**: table used to encode the horizontal sense bits of the coordination message
    value: *[0, 11, 19, 24, 28, 23, 15, 4, 13, 6, 30, 21, 17, 26, 2, 9, 21, 30, 6, 13, 9, 2, 26, 17, 24, 19, 11, 0, 4, 15, 23, 28, 25, 18, 10, 1, 5, 14, 22, 29, 20, 31, 7, 12, 8, 3, 27, 16, 12, 7, 31, 20, 16,*

27, 3, 8, 1, 10, 18, 25, 29, 22, 14, 5]

- **parity_table_vert**: table used to encode the vertical sense bits of the coordination message
  value: *[0, 14, 7, 9, 11, 5, 12, 2, 13, 3, 10, 4, 6, 8, 1, 15]*

- **display**: Information for the display logic
  - **ORNCTcrossthrhi**: upper threshold for vertical separation during crossing for ORNCT, at which crossing bit is first set (ft)
    value: *300.0*
  - **ORNCTcrossthrlo**: lower threshold for vertical separation during crossing for ORNCT, above which crossing bit is maintained once set (ft)
    value: *-300.0*
  - **crossthrhi**: upper threshold for vertical separation during crossing, at which crossing bit is first set (ft)
    value: *100.0*
  - **crossthrlo**: lower threshold for vertical separation during crossing, above which crossing bit is maintained once set (ft)
    value: *-100.0*
  - **hdescendthr**: label270rules.altinhibit: altitude at and below which default annunciation is overridden (ft)
    value: *1000.0*
  - **hiscore**: track display score: score for intruder-specific resolution advisory
    value: *12000.0*
  - **label270rules**: Rules for setting the Label 270 values. A single rule is obtained using the same index into all vectors. A value of NaN means any value will match. The rules are evaluated in order, with the first matching index giving the values of cc, vc, ua, and da that will be used. When a subfield description does not provide a value, the value is provided in table(s) following the last subfield.
    * **action**: vector of current advisory action values. Names associated with the values are given in actions.names. Values are listed in table(s) below.
    * **altinhibit**: vector indicating whether hdescendthr is taken into account. Values are listed in table(s) below.
    * **cc**: vector of combined control values (output). Values are listed in table(s) below.
    * **crossing**: vector of crossing flags. Values are listed in table(s) below.
    * **da**: vector of down advisory values (output). Values are listed in table(s) below.
    * **forcealarm**: vector of force alarm values for explict control of alarm flag, and therefore annunciation, status (e.g., force Level-Off Level-Off annunciation after Monitor Vertical Speed for Do Not Climb advisory). Values are listed in table(s) below.
    * **hidz**: vector of upper vertical rate limit values (ft/s). Values are listed in table(s) below.
    * **lodz**: vector of lower vertical rate limit values (ft/s). Values are listed in table(s) below.
    * **prevaction**: vector of previous advisory action values. Names associated with the values are given in actions.names. Values are listed in table(s) below.
    * **prevstrength**: vector of previous Vertical Active RA strength values. Values are listed in table(s) below.

* **prevword**: vector of previous label270 codes.
  value: *[NaN, NaN, NaN, NaN, NaN, NaN, 5401.0, 5201.0, 5002.0, 5002.0, 5002.0, 6002.0, 6002.0, 6002.0, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, 4410.0, 4210.0, 4020.0, 4020.0, 4020.0, 6020.0, 6020.0, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, NaN, 4210.0, 4410.0, 5201.0, 5401.0, NaN, NaN, NaN, NaN, NaN, NaN, 4410.0, 4210.0, 5101.0, 5201.0, 5001.0, NaN, NaN, NaN, NaN, NaN, NaN, NaN, 5401.0, 5201.0, 4110.0, 4210.0, 4010.0, NaN, NaN, NaN, NaN, NaN, NaN, NaN, 4410.0, 4210.0, 5101.0, 5201.0, 5001.0, NaN, NaN, NaN, NaN, NaN, NaN, NaN, 5401.0, 5201.0, 4110.0, 4210.0, 4010.0, NaN, NaN, NaN, NaN, NaN, 4022.0, 5022.0, 6422.0, NaN, NaN, NaN]*
* **strength**: vector of Vertical Active RA strength values for output in RA ground messages (output). Values are listed in table(s) below.
* **ua**: vector of up advisory values (output). Values are listed in table(s) below.
* **vc**: vector of vertical control values (output). Values are listed in table(s) below.

Table 49. display.label270rules (1 of 4)

no.	action	prevaction	lodz	hidz	crossing	altinhibit	cc	vc	ua	da	forcealarm	prevstrength	strength
1	1	1	NaN	NaN	NaN	NaN	0	0	0	0	0	NaN	0
2	1	NaN	NaN	NaN	NaN	NaN	1	0	0	0	0	NaN	0
3	2	5	NaN	NaN	NaN	1	6	0	0	2	0	NaN	10
4	2	7	NaN	NaN	NaN	1	6	0	0	2	0	NaN	10
5	2	9	NaN	NaN	NaN	1	6	0	0	2	0	NaN	10
6	2	1	-Inf	8.333333333	NaN	NaN	6	0	0	2	0	NaN	1
7	2	4	NaN	NaN	NaN	NaN	6	0	0	2	0	NaN	1
8	2	4	NaN	NaN	NaN	NaN	6	0	0	2	0	NaN	1
9	2	NaN	NaN	NaN	NaN	NaN	5	0	0	2	0	2	2
10	2	NaN	NaN	NaN	NaN	NaN	5	0	0	2	0	3	3
11	2	NaN	NaN	NaN	NaN	NaN	5	0	0	2	0	9	3
12	2	NaN	-Inf	12.5	NaN	NaN	6	0	0	2	0	10	10
13	2	NaN	-Inf	12.5	NaN	NaN	6	0	0	2	0	NaN	1
14	2	NaN	NaN	NaN	NaN	NaN	5	0	0	2	1	NaN	3
15	2	3	-Inf	8.333333333	NaN	NaN	6	0	0	2	0	NaN	11
16	2	4	-Inf	8.333333333	NaN	NaN	6	0	0	2	0	NaN	11
17	2	6	-Inf	8.333333333	NaN	NaN	6	0	0	2	0	NaN	11
18	2	8	-Inf	8.333333333	NaN	NaN	6	0	0	2	0	NaN	11
19	2	10	-Inf	8.333333333	NaN	NaN	6	0	0	2	0	NaN	11
20	2	NaN	-8.333333333	8.333333333	NaN	NaN	6	0	0	2	0	NaN	1
21	2	1	NaN	NaN	NaN	NaN	5	0	0	2	0	NaN	3
22	2	5	NaN	NaN	NaN	NaN	5	0	0	2	0	NaN	2
23	2	7	NaN	NaN	NaN	NaN	5	0	0	2	0	NaN	2
24	2	9	NaN	NaN	NaN	NaN	5	0	0	2	0	NaN	2
25	2	11	-Inf	8.333333333	NaN	NaN	6	0	0	2	0	NaN	1
26	2	11	NaN	NaN	NaN	NaN	5	0	0	2	0	NaN	3
27	2	NaN	NaN	NaN	NaN	NaN	5	0	0	2	0	NaN	9
28	3	1	-8.333333333	Inf	NaN	NaN	6	0	2	0	0	NaN	1
29	3	4	NaN	NaN	NaN	NaN	6	0	2	0	0	NaN	1
30	3	4	NaN	NaN	NaN	NaN	6	0	2	0	0	NaN	1
31	3	NaN	NaN	NaN	NaN	NaN	4	0	2	0	0	2	2
32	3	NaN	NaN	NaN	NaN	NaN	4	0	2	0	0	3	3

Table 50. display.label270rules (2 of 4)

no.	action	prevaction	lodz	hidz	crossing	altinhibit	cc	vc	ua	da	forcealarm	prevstrength	strength
33	3	NaN	NaN	NaN	NaN	NaN	4	0	2	0	0	9	3
34	3	NaN	-12.5	Inf	NaN	NaN	6	0	2	0	0	NaN	1
35	3	NaN	NaN	NaN	NaN	NaN	4	0	2	0	1	NaN	3
36	3	2	-8.333333333	Inf	NaN	NaN	6	0	2	0	0	NaN	11
37	3	4	-8.333333333	Inf	NaN	NaN	6	0	2	0	0	NaN	11
38	3	5	-8.333333333	Inf	NaN	NaN	6	0	2	0	0	NaN	11
39	3	7	-8.333333333	Inf	NaN	NaN	6	0	2	0	0	NaN	11
40	3	9	-8.333333333	Inf	NaN	NaN	6	0	2	0	0	NaN	11
41	3	NaN	-8.333333333	8.333333333	NaN	NaN	6	0	2	0	0	NaN	1
42	3	1	NaN	NaN	NaN	NaN	4	0	2	0	0	NaN	3
43	3	6	NaN	NaN	NaN	NaN	4	0	2	0	0	NaN	2
44	3	8	NaN	NaN	NaN	NaN	4	0	2	0	0	NaN	2
45	3	10	NaN	NaN	NaN	NaN	4	0	2	0	0	NaN	2
46	3	11	-8.333333333	Inf	NaN	NaN	6	0	2	0	0	NaN	1
47	3	11	NaN	NaN	NaN	NaN	4	0	2	0	0	NaN	3
48	3	NaN	NaN	NaN	NaN	NaN	4	0	2	0	0	NaN	9
49	4	2	0	Inf	NaN	NaN	4	2	1	0	0	NaN	8
50	4	5	0	Inf	NaN	NaN	4	2	1	0	0	NaN	8
51	4	7	0	Inf	NaN	NaN	4	2	1	0	0	NaN	8
52	4	9	0	Inf	NaN	NaN	4	2	1	0	0	NaN	8
53	4	3	-Inf	0	NaN	NaN	5	2	0	1	0	NaN	8
54	4	6	-Inf	0	NaN	NaN	5	2	0	1	0	NaN	8
55	4	8	-Inf	0	NaN	NaN	5	2	0	1	0	NaN	8
56	4	10	-Inf	0	NaN	NaN	5	2	0	1	0	NaN	8
57	4	4	NaN	NaN	NaN	NaN	4	4	1	0	0	NaN	7
58	4	NaN	NaN	NaN	NaN	NaN	4	4	1	0	0	NaN	7
59	4	4	NaN	NaN	NaN	NaN	5	4	0	1	0	NaN	7
60	4	NaN	NaN	NaN	NaN	NaN	5	4	0	1	0	NaN	7
61	4	NaN	0	Inf	NaN	NaN	4	4	1	0	0	NaN	7
62	4	NaN	NaN	NaN	NaN	NaN	5	4	0	1	0	NaN	7
63	5	3	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
64	5	6	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5

Table 51. display.label270rules (3 of 4)

no.	action	prevaction	lodz	hidz	crossing	altinhibit	cc	vc	ua	da	forcealarm	prevstrength	strength
65	5	8	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
66	5	10	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
67	5	NaN	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
68	5	4	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
69	5	NaN	NaN	NaN	1	NaN	5	1	0	1	2	NaN	4
70	5	NaN	NaN	NaN	1	NaN	5	1	0	1	2	NaN	4
71	5	NaN	NaN	NaN	1	NaN	5	1	0	1	2	NaN	4
72	5	NaN	NaN	NaN	1	NaN	5	1	0	1	1	NaN	4
73	5	7	NaN	NaN	NaN	NaN	5	0	0	1	2	NaN	4
74	5	NaN	NaN	NaN	NaN	NaN	5	0	0	1	0	NaN	4
75	6	2	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
76	6	5	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
77	6	7	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
78	6	9	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
79	6	NaN	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
80	6	4	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
81	6	NaN	NaN	NaN	1	NaN	4	1	1	0	2	NaN	4
82	6	NaN	NaN	NaN	1	NaN	4	1	1	0	2	NaN	4
83	6	NaN	NaN	NaN	1	NaN	4	1	1	0	2	NaN	4
84	6	NaN	NaN	NaN	1	NaN	4	1	1	0	1	NaN	4
85	6	8	NaN	NaN	NaN	NaN	4	0	1	0	2	NaN	4
86	6	NaN	NaN	NaN	NaN	NaN	4	0	1	0	0	NaN	4
87	7	3	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
88	7	6	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
89	7	8	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
90	7	10	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
91	7	NaN	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
92	7	4	NaN	NaN	NaN	NaN	5	2	0	1	0	NaN	5
93	7	NaN	NaN	NaN	1	NaN	5	1	0	1	2	NaN	4
94	7	NaN	NaN	NaN	1	NaN	5	1	0	1	2	NaN	4
95	7	NaN	NaN	NaN	1	NaN	5	1	0	1	2	NaN	4
96	7	NaN	NaN	NaN	1	NaN	5	1	0	1	1	NaN	4

Table 52. display.label270rules (4 of 4)

no.	action	prevaction	lodz	hidz	crossing	altinhibit	cc	vc	ua	da	forcealarm	prevstrength	strength
97	7	5	NaN	NaN	NaN	NaN	5	0	0	1	2	NaN	4
98	7	NaN	NaN	NaN	NaN	NaN	5	0	0	1	0	NaN	4
99	8	2	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
100	8	5	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
101	8	7	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
102	8	9	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
103	8	NaN	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
104	8	4	NaN	NaN	NaN	NaN	4	2	1	0	0	NaN	5
105	8	NaN	NaN	NaN	1	NaN	4	1	1	0	2	NaN	4
106	8	NaN	NaN	NaN	1	NaN	4	1	1	0	2	NaN	4
107	8	NaN	NaN	NaN	1	NaN	4	1	1	0	2	NaN	4
108	8	NaN	NaN	NaN	1	NaN	4	1	1	0	1	NaN	4
109	8	6	NaN	NaN	NaN	NaN	4	0	1	0	2	NaN	4
110	8	NaN	NaN	NaN	NaN	NaN	4	0	1	0	0	NaN	4
111	9	NaN	NaN	NaN	NaN	NaN	5	3	0	1	0	NaN	6
112	10	NaN	NaN	NaN	NaN	NaN	4	3	1	0	0	NaN	6
113	11	NaN	NaN	NaN	NaN	NaN	4	0	2	2	0	NaN	15
114	11	NaN	NaN	NaN	NaN	NaN	5	0	2	2	0	NaN	15
115	11	NaN	NaN	NaN	NaN	NaN	6	4	2	2	0	NaN	14
116	11	NaN	-Inf	-8.333333333	NaN	NaN	4	0	2	2	0	NaN	15
117	11	NaN	8.333333333	Inf	NaN	NaN	5	0	2	2	0	NaN	15
118	11	NaN	NaN	NaN	NaN	NaN	6	4	2	2	0	NaN	14

- **label271rules**: Rules for setting the Label 271 value for horizontal resolution advisories. A single rule is obtained using the same index into all arrays. A value of NaN means any value will match. The rules are evaluated in order, with the first matching index giving the value of cc that will be used. When a subfield description does not provide a value, the value is provided in table(s) following the last subfield.
  * **action**: array of current horizontal advisory action values. Names associated with the values are given in turn_actions.names. Values are listed in table(s) below.
  * **cc**: array of combined control values (output). Values are listed in table(s) below.
  * **prevaction**: array of previous horizontal advisory action values. Names associated with the values are given in turn_actions.names. Values are listed in table(s) below.

Table 53. display.label271rules

no.	action	prevaction	cc
1	1	1	0
2	1	NaN	1
3	2	NaN	2
4	3	NaN	3
5	4	NaN	4

- **loscore**: track display score: base score for intruder-specific remain well clear bands
  value: *700.0*
- **medhiscore**: track display score: base score for intruder-specific remain well clear corrective
  value: *5000.0*
- **medscore**: track display score: base score for intruder-specific remain well clear preventive
  value: *2000.0*
- **scorfactr_degraded**: track display score: base score multiplier used for non-altitude reporting and ADS-B only targets
  value: *1.0*
- **scorfactr_good**: track diplay score: base score multiplier used for targets with good surveillance quality
  value: *2.0*
- **tinyscore**: track display score: base score for intruder-specific no advisory or clear of conflict
  value: *40.0*

• **horizontal_trm**: parameters for the horizontal TRM
- **R_ca_threshold**: turn rate at or above which the advisory is a collision avoidance advisory (rad)
  value: *0.04*
- **R_turn_epsilon**: angular difference within which two turn rates are considered equal (rad)
  value: *0.0001*
- **horizontal_offline**: parameters for the offline tables used by the horizontal TRM

* **C_nominal**: horizontal cost when dynamic state values are beyond the edge of the horizontal cost table
  value: *0.001*
* **H_separation_min_ras_m**: minimum vertical separation used for horizontal RAs (m)
  value: *38.1*
* **T_vertical_tau_max**: maximum tau (s)
  value: *200.0*
* **X_speed_tolerance_factor**: factor for speed
  value: *0.33*
* **hcoord_policy**: parameters describing horizontal coordination table
  - **C_hcoord_hysteresis**: cost value for horizontal coordination hysteresis
    value: *0.9*
  - **hcoord_cuts**: specification of edges in the horizontal coordination table
    intrspeed: vector of intruder speed table edges (ft/s)
    value: *[0.0, 88.366, 176.732, 265.098, 353.464, 441.83, 530.196, 618.562, 706.928]*
    order: vector of edge names in the same order as in the table
    value: *["range", "theta", "psi", "ownspeed", "intrspeed"]*
    ownspeed: vector of ownship speed table edges (ft/s)
    value: *[58.9106, 151.485, 244.058, 336.632, 429.206, 521.78, 614.354, 706.928]*
    psi: vector of psi table edges (rad)
    value: *[-3.1416, -3.0892, -3.0369, -2.9845, -2.9322, -2.8798, -2.8274, -2.7751, -2.7227, -2.6704, -2.618, -2.5656, -2.5133, -2.4609, -2.4086, -2.3562, -2.3038, -2.2515, -2.1991, -2.1468, -2.0944, -2.042, -1.9897, -1.9373, -1.885, -1.8326, -1.7802, -1.7279, -1.6755, -1.6232, -1.5708, -1.5184, -1.4661, -1.4137, -1.3614, -1.309, -1.2566, -1.2043, -1.1519, -1.0996, -1.0472, -0.99484, -0.94248, -0.89012, -0.83776, -0.7854, -0.73304, -0.68068, -0.62832, -0.57596, -0.5236, -0.47124, -0.41888, -0.36652, -0.31416, -0.2618, -0.20944, -0.15708, -0.10472, -0.05236, 0.0, 0.05236, 0.10472, 0.15708, 0.20944, 0.2618, 0.31416, 0.36652, 0.41888, 0.47124, 0.5236, 0.57596, 0.62832, 0.68068, 0.73304, 0.7854, 0.83776, 0.89012, 0.94248, 0.99484, 1.0472, 1.0996, 1.1519, 1.2043, 1.2566, 1.309, 1.3614, 1.4137, 1.4661, 1.5184, 1.5708, 1.6232, 1.6755, 1.7279, 1.7802, 1.8326, 1.885, 1.9373, 1.9897, 2.042, 2.0944, 2.1468, 2.1991, 2.2515, 2.3038, 2.3562, 2.4086, 2.4609, 2.5133, 2.5656, 2.618, 2.6704, 2.7227, 2.7751, 2.8274, 2.8798, 2.9322, 2.9845, 3.0369, 3.0892, 3.1416]*
    range: vector of range table edges (ft)
    value: *[99.8, 102.0, 160.0, 300.0, 400.0, 500.0, 607.6, 911.4, 1063.2, 1215.2, 1290.0, 1367.0, 1440.0, 1519.0, 1590.0, 1670.8, 1745.0, 1822.8, 2126.6, 2430.4, 2734.2, 3038.0, 3341.8, 3645.6, 3949.4, 4253.2, 4557.0, 4860.8, 5164.6, 5468.6, 5772.4, 6076.2, 6380.0, 6683.8, 6987.6, 7291.4, 7595.2, 7899.0, 8202.8, 8506.6, 9721.6, 10936.8, 12152.0]*
    theta: vector of theta table edges (rad)
    value: *[-3.1416, -3.0892, -3.0369, -2.9845, -2.9322, -2.8798, -2.8274, -2.7751, -2.7227, -2.6704, -2.618, -2.5656, -2.5133, -2.4609, -2.4086, -2.3562, -2.3038, -2.2515, -2.1991, -2.1468, -2.0944, -2.042, -1.9897, -1.9373, -1.885, -1.8326, -1.7802, -1.7279, -1.6755, -1.6232, -1.5708, -1.5184, -1.4661, -1.4137, -1.3614, -1.309, -1.2566, -1.2043, -1.1519, -1.0996, -1.0472, -0.99484, -0.94248, -0.89012,*

-0.83776, -0.7854, -0.73304, -0.68068, -0.62832, -0.57596, -0.5236, -0.47124, -0.41888, -0.36652, -0.31416, -0.2618, -0.20944, -0.15708, -0.10472, -0.05236, 0.0, 0.05236, 0.10472, 0.15708, 0.20944, 0.2618, 0.31416, 0.36652, 0.41888, 0.47124, 0.5236, 0.57596, 0.62832, 0.68068, 0.73304, 0.7854, 0.83776, 0.89012, 0.94248, 0.99484, 1.0472, 1.0996, 1.1519, 1.2043, 1.2566, 1.309, 1.3614, 1.4137, 1.4661, 1.5184, 1.5708, 1.6232, 1.6755, 1.7279, 1.7802, 1.8326, 1.885, 1.9373, 1.9897, 2.042, 2.0944, 2.1468, 2.1991, 2.2515, 2.3038, 2.3562, 2.4086, 2.4609, 2.5133, 2.5656, 2.618, 2.6704, 2.7227, 2.7751, 2.8274, 2.8798, 2.9322, 2.9845, 3.0369, 3.0892, 3.1416]

- **hcoord_table**: name of the horizontal coordination table file
  value: *xuhtrm_coord_sxu_v4r0.bin*

* **hcost_policy**: parameters describing horizontal cost table
  - **origami**: Parameters for Data Origami tables

    N_allowable_actions: number of allowable actions for each row in the allowable_actions matrix
    value: *[4]*

    action_pattern_number: action pattern numbers, 1-based
    value: *[1, 1, 1, 1]*

    allowable_actions: allowable action transitions for each action, 1-based. 0 indicates no allowable transition
    value: $(1.0\ 2.0\ 3.0\ 4.0)$

    equiv_class_number: equivalence class for each transition, 1-based. A minus sign is used to indicate a negative equivalence class. 0 indicates no valid transition
    value: $\begin{pmatrix} 1.0 & 5.0 & 9.0 & 13.0 \\ 2.0 & 6.0 & 10.0 & 14.0 \\ 3.0 & 7.0 & 11.0 & 15.0 \\ 4.0 & 8.0 & 12.0 & 16.0 \end{pmatrix}$

    equiv_class_table: name of the equivalence class tables file
    value: *q_sxuhtrm_v4r0_16ec_allSplits_NoMB.dat*

    minblocks_index: indexes into the minblocks table, 1-based. 0 indicates no mapping
    value: $\begin{pmatrix} 1.0 & 5.0 & 9.0 & 13.0 \\ 2.0 & 6.0 & 10.0 & 14.0 \\ 3.0 & 7.0 & 11.0 & 15.0 \\ 4.0 & 8.0 & 12.0 & 16.0 \end{pmatrix}$

    minblocks_table: name of the minblocks table file
    value: *q_sxuhtrm_v4r0_16ec_minBlocks_NoMB.dat*

* **horizontal_scaling**: parameters for horizontal policy and band scaling
  - **H_point_obstacle_offset_m**: additional point obstacle target separation (m)
    value: *5*
  - **classification**: classification associated with each scale factor vector
    value: *[1, 3, 100]*
  - **values**: a 6-column matrix of horizontal scales; where the columns indicate the policy range scale, policy target vertical separation scale, policy speed scale, DAA band range scale, DAA band target vertical separation, DAA band speed scale, and DAA band lookahead time scale, respectively. Note the band scales are affected by the policy scales, but are not used in ACAS sXu.

$$value: \begin{pmatrix} 0.530578465188752 & 0.805868569103711 & 1.288937014021658 & 1.0 & 1.0 & 1.0 & 1.0 & 1.0 \\ 0.070716177888095 & 0.190085547435064 & 0.535120782402347 & 0.145165682390867 & 1.0 & 1.0 & 1.0 & 1.0 \\ 0.25 & 0.358862419999103 & 1.0 & 0.1 & 1.0 & 1.0 & 1.0 & 1.0 \end{pmatrix}$$

- **horizontal_online**: parameters for ApplyHorizontalOnlineCosts
  - **h_coordination**: parameters for HorizontalCoordinationCost
    - **C_hcoord**: online cost to penalize advisories based on horizontal coordination
      value: *100000.0*
    - **C_hcoordrev**: online cost to penalize horizontal geometric reversals based on horizontal coordination
      value: *1.5*
  - **h_crossing**: parameters for StateDependentCrossingCostFactors
    - **X_crossing_psi_min**: psi threshold below which crossing cost doesn't apply (rad)
      value: *0.3491*
    - **X_exponential_scale_factor**: exponential value to apply to theta scale factor
      value: *1.0*
    - **X_scale_factor**: crossing cost scale factor
      value: *1.0*
    - **X_theta_diff_max**: threshold for angular difference in bearing angle, above which crossing cost doesn't apply (rad)
      value: *0.4189*
    - **X_theta_tolerance**: threshold used to determine which turn advisories to penalize (rad)
      value: *0.0524*
  - **h_initial_ca**: parameters for HorizontalInitialCACost
    - **C_initial_ca**: online cost to penalize an initial horizontal advisory
      value: *0.0442*
  - **h_maintain**: parameters for all online costs associated with horizontal maintains
    - **C_initial_maintain**: online cost for HorizontalInitialMaintainCost
      value: *20.0*
    - **C_maintain_heading_incentive**: online incentive for HorizontalMaintainHeadingIncentive
      value: *0.93*
    - **C_multithreat_maintain_heading_cost**: online incentive for HorizontalMultiThreatMaintainHeadingCost
      value: *1.08*
    - **C_multithreat_turn_from_maintain_cost**: online incentive for HorizontalMultiThreatTurnFromMaintainCost
      value: *0.74*
    - **C_turn_from_maintain_incentive**: online incentive for HorizontalTurnFromMaintainIncentive
      value: *0.97*
  - **h_multi_threat_reversal**: parameters for HorizontalMultiThreatReversalCost
    - **C_multi_threat_reversal_reduction**: online reward for HorizontalMultiThreatReversalCost
      value: *-6.0*

    - **h_reversal**: parameters for HorizontalReversalCost
      - **C_reversal**: online cost to penalize horizontal reversals
        value: *0.2*
    - **h_weakening**: parameters for HorizontalWeakeningCostFactor
      - **X_weakening_factor**: amount to scale cost when penalizing weakenings
        value: *1.4*
  - **htrm_alert_lookahead**: parameters for HorizontalAlertLookahead
    - **T_dt**: look-ahead time step (s)
      value: *5.0*
    - **T_max**: maximum time to look-ahead (s)
      value: *20.0*
    - **T_max_coc**: maximum time to look-ahead with with no advisory (s)
      value: *10.0*
    - **T_min**: minimum time to look-ahead (s)
      value: *5.0*
    - **T_threat_timeout**: maximum time to keep adding threats (s)
      value: *10.0*
  - **htrm_prioritization**: parameters for PrioritizeAndFilterIntruders
    - **D_rng_diff_activation_threshold**: minimum horizontal separation for intruder to be considered a priority (m)
      value: *129640.0*
    - **H_alt_diff_activation_threshold**: minimum vertical separation for intruder to be considered a priority (m)
      value: *1524.0*
    - **N_max_prioritized**: maximum number of intruders to be evaluated for horizontal maneuver advisories and DAA bands
      value: *5*
    - **T_min_track_duration**: minimum time the intruder has been available to the horizontal TRM to be considered a priority (s)
      value: *2.5*
  - **update_track_angle**: parameters for UpdateTargetTrackAngle
    - **initial_track_angle**: minimum initial target track angle relative to ownship track angle (in degrees) for horizontal RA
      value: *30.0*
    - **strengthen_threshold**: degrees from ownship track angle to target track angle to update target track angle
      value: *15.0*
    - **target_precision**: precision (in degrees) that is used to round target track angle in horizontal logic
      value: *10.0*
- **modes**: Parameters associated with specific protection modes
  - **cost_estimation**: Information about the parameters used in the cost estimation subsystem
    - **offline**: Offline cost estimation parameters
      - **origami**: Parameters for Data Origami tables

N_allowable_actions: number of allowable actions for each row in the allowable_actions matrix
sXu value: *[6, 6, 8, 7, 7, 7, 7, 6, 6, 7]*

action_pattern_number: action pattern numbers, 1-based
sXu value: *[1, 2, 2, 3, 4, 5, 6, 7, 8, 9, 10]*

allowable_actions: allowable action transitions for each action, 1-based. 0 indicates no allowable transition

sXu value:
$$\begin{pmatrix} 1.0 & 2.0 & 3.0 & 4.0 & 5.0 & 6.0 & 0.0 & 0.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 7.0 & 8.0 & 0.0 & 0.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 7.0 & 8.0 & 9.0 & 10.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 5.0 & 8.0 & 9.0 & 0.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 6.0 & 7.0 & 10.0 & 0.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 7.0 & 8.0 & 9.0 & 0.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 7.0 & 8.0 & 10.0 & 0.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 8.0 & 9.0 & 0.0 & 0.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 7.0 & 10.0 & 0.0 & 0.0 \\ 1.0 & 2.0 & 3.0 & 4.0 & 7.0 & 8.0 & 11.0 & 0.0 \end{pmatrix}$$

equiv_class_number: equivalence class for each transition, 1-based. A minus sign is used to indicate a negative equivalence class. 0 indicates no valid transition

sXu value:
$$\begin{pmatrix} 1.0 & 7.0 & 13.0 & 19.0 & 27.0 & 34.0 & 41.0 & 48.0 & 55.0 & 61.0 & 67.0 \\ 2.0 & 8.0 & 14.0 & 20.0 & 28.0 & 35.0 & 42.0 & 49.0 & 56.0 & 62.0 & 68.0 \\ 3.0 & 9.0 & 15.0 & 21.0 & 29.0 & 36.0 & 43.0 & 50.0 & 57.0 & 63.0 & 69.0 \\ 4.0 & 10.0 & 16.0 & 22.0 & 30.0 & 37.0 & 44.0 & 51.0 & 58.0 & 64.0 & 70.0 \\ 5.0 & 0.0 & 0.0 & 0.0 & 31.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 6.0 & 0.0 & 0.0 & 0.0 & 0.0 & 38.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 11.0 & 17.0 & 23.0 & 0.0 & 39.0 & 45.0 & 52.0 & 0.0 & 65.0 & 71.0 \\ 0.0 & 12.0 & 18.0 & 24.0 & 32.0 & 0.0 & 46.0 & 53.0 & 59.0 & 0.0 & 72.0 \\ 0.0 & 0.0 & 0.0 & 25.0 & 33.0 & 0.0 & 47.0 & 0.0 & 60.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 26.0 & 0.0 & 40.0 & 0.0 & 54.0 & 0.0 & 66.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 73.0 \end{pmatrix}$$

equiv_class_table: name of the equivalence class tables file
sXu value: *q_sxuvtrm_v4r0_73ec_allSplits.dat*

minblocks_index: indexes into the minblocks table, 1-based. 0 indicates no mapping

sXu value:
$$\begin{pmatrix} 1.0 & 7.0 & 13.0 & 19.0 & 27.0 & 34.0 & 41.0 & 48.0 & 55.0 & 61.0 & 67.0 \\ 2.0 & 8.0 & 14.0 & 20.0 & 28.0 & 35.0 & 42.0 & 49.0 & 56.0 & 62.0 & 68.0 \\ 3.0 & 9.0 & 15.0 & 21.0 & 29.0 & 36.0 & 43.0 & 50.0 & 57.0 & 63.0 & 69.0 \\ 4.0 & 10.0 & 16.0 & 22.0 & 30.0 & 37.0 & 44.0 & 51.0 & 58.0 & 64.0 & 70.0 \\ 5.0 & 0.0 & 0.0 & 0.0 & 31.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 6.0 & 0.0 & 0.0 & 0.0 & 0.0 & 38.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 11.0 & 17.0 & 23.0 & 0.0 & 39.0 & 45.0 & 52.0 & 0.0 & 65.0 & 71.0 \\ 0.0 & 12.0 & 18.0 & 24.0 & 32.0 & 0.0 & 46.0 & 53.0 & 59.0 & 0.0 & 72.0 \\ 0.0 & 0.0 & 0.0 & 25.0 & 33.0 & 0.0 & 47.0 & 0.0 & 60.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 26.0 & 0.0 & 40.0 & 0.0 & 54.0 & 0.0 & 66.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 73.0 \end{pmatrix}$$

minblocks_table: name of the minblocks table file
sXu value: *q_sxuvtrm_v4r0_73ec_minBlocks.dat*

* **online**: Online cost estimation parameters
  - **SA01**: SA01Heuristic cost parameters

    C_coc: cost for COC when the SA01 heuristic determines a reversal is needed
    sXu value: *[201.0, 201.0]*

    C_non_reversal: cost for non-reversal advisories when the SA01 heuristic determines a reversal is needed

sXu value: *[201.0, 201.0]*

D_range_threshold: horizontal separation below which a reversal cannot be considered if it would induce an altitude crossing (ft)
sXu value: *[3000.0, 3000.0]*

H_rel_threshold: relative vertical separation above which a reversal cannot be considered if it would induce an altitude crossing (ft)
sXu value: *[400.0, 400.0]*

R_acceleration: assumed acceleration of aircraft used in the linear approximation (ft/s^2)
sXu value: *[10.7247, 10.7247]*

R_min: the minimum rate that both aircraft need to have for the SA01 heuristic to be activated (ft/s)
sXu value: *[11.0, 11.0]*

R_standard: vertical speed of aircraft allowed when reversing the advisory in the linear approximation (ft/s)
sXu value: *[25.0, 25.0]*

R_strengthen: vertical speed of aircraft allowed when maintaining the original advisory in the linear approximation (ft/s)
sXu value: *[41.6667, 41.6667]*

T_delay: time delay in response used for the linear approximation (s)
sXu value: *[3.0, 3.0]*

T_max: maximum time until closest approach that the SA01 heuristic should consider a reversal (s)
sXu value: *[35.0, 35.0]*

T_min: minimum time until closest approach that the SA01 heuristic should consider a reversal (s)
sXu value: *[5.0, 5.0]*

- **advisory_restart**: AdvisoryRestartCost parameters

  C_advisory: cost for restarting alert after having terminated for fewer than N_restart
  sXu value: *0.05*

  T_limit: minimum number of cycles for which an alert must be terminated (s)
  sXu value: *3*

- **altitude_dependent_coc**: AltitudeDependentCOCCost parameters

  C_coc: cost for issuing a clear of conflict or preventive advisory
  sXu value: *[1.0, 1.0]*

  H_rel_highalt_rolloff: ramp down interval for H_rel_highalt_threshold (ft)
  sXu value: *[75.0, 75.0]*

  H_rel_highalt_threshold: minimum proximate separation for applying cost at high altitudes (ft)
  sXu value: *[900.0, 900.0]*

  H_rel_rolloff_proximate: ramp down interval for H_rel_threshold_proximate (ft)
  sXu value: *[250.0, 250.0]*

  H_rel_rolloff_vrrf: ramp up interval for H_rel_threshold_vrrf (ft)
  sXu value: *[150.0, 150.0]*

H_rel_threshold_proximate: minimum proximate separation for applying cost (ft)
sXu value: *[600.0, 600.0]*

H_rel_threshold_vrrf_hi: separation for altitudes above H_threshold_hi at which to begin applying vertical rate reduction factor (ft)
sXu value: *[750.0, 750.0]*

H_rel_threshold_vrrf_lo: separation for altitudes at or below H_threshold_hi at which to begin applying vertical rate reduction factor (ft)
sXu value: *[300.0, 300.0]*

H_threshold_hi: altitude of own aircraft at or above which the full cost is applied, below this altitude the cost is adjusted based on the closeness of own aircraft to this altitude relative to H_threshold_lo (ft)
sXu value: *20000.0*

H_threshold_lo: altitude of own aircraft above which cost can be applied (ft)
sXu value: *5000.0*

R_own_rolloff_vrrf: ramp down interval for R_own_threshold_vrrf (ft/s)
sXu value: *8.3333333333*

R_own_threshold_vrrf: own aircraft altitude at which to apply vertical rate reduction factor (ft/s)
sXu value: *8.3333333333*

R_rolloff_lolo: ramp down interval for R_threshold_lolo (ft/s)
sXu value: *8.3333333333*

R_threshold_lolo: vertical rate threshold for further scaling the cost when a preventive would cause a level-off (ft/s)
sXu value: *12.5*

T_proximate: time value defining 'proximate' for determining proximate loss of separation (s)
sXu value: *[7.5, 7.5]*

T_rolloff: ramp down interval for T_threshold (s)
sXu value: *[7.0, 7.0]*

T_threshold: tau value below which the full cost can be applied (s)
sXu value: *[24.7, 24.7]*

X_maintain_factor: factor applied to the clear of conflict penalty to apply to maintains
sXu value: *[0.4, 0.4]*

X_proximate_to_actual_factor: factor applied to the proximate separation thresholds for use in the actual separation checks
sXu value: *[1.98, 1.98]*

X_vrrf: vertical rate reduction scale factor
sXu value: *[0.25, 0.25]*

- **altitude_inhibit**: Online cost parameters for the altitude inhibit rules; a negative cost for one rule is used as an exclusion from another rule. When a subfield description does not provide a value, the value is provided in table(s) following the last subfield.

  B_init: initial setting of the altitude inhibit flag for each altitude inhibit rule. Values are listed in table(s) below.

C_inhibit: vector of costs for each altitude inhibit rule. Values are listed in table(s) below.

R_dz_hi: vector of upper vertical rates for each altitude inhibit rule; with dz_-lo, specifies resolution advisories to be inhibited (ft/s). Values are listed in table(s) below.

R_dz_hi_prev: vector of previous upper vertical rates for each altitude inhibit rule; with prev_dz_lo, specifies previously issued resolution advisories (ft/s). Values are listed in table(s) below.

R_dz_lo: vector of lower vertical rates for each altitude inhibit rule; with dz_-hi, specifies resolution advisories to be inhibited (ft/s). Values are listed in table(s) below.

R_dz_lo_prev: vector of previous lower vertical rates for each altitude inhibit rule; with prev_dz_hi, specifies previously issued resolution advisories (ft/s). Values are listed in table(s) below.

Table 54. modes[n].cost_estimation.online.altitude_inhibit - sXu

no.	R_dz_lo	R_dz_hi	R_dz_lo_prev	R_dz_hi_prev	B_init	C_inhibit
1	-Inf	-1.6667	-Inf	Inf	0	100

- **bad_transition**: BadTransitionCost parameters

  C_bad_maintain_initiation: huge cost to prevent initiation of maintain when ownship vertical rate is too low
  sXu value: *1.0e10*

  C_bad_transition: cost for advisories that would result in illegal state transitions
  sXu value: *10.0*

- **compatibility**: CompatibilityCost parameters

  C_master: cost received for issuing action incompatible with master aircraft
  sXu value: *[1.0e10, 1.0e10]*

  C_slave_init: cost received for issuing action incompatible with slave aircraft for first advisory
  sXu value: *[1.0e10, 1.0e10]*

  C_slave_init_crossing: cost received for issuing action incompatible with slave aircraft
  sXu value: *[0.1, 0.1]*

  C_slave_sub: cost received for issuing action incompatible with slave aircraft after an initial advisory
  sXu value: *[0.6, 0.6]*

  C_slave_sub_multithreat: cost received for issuing action incompatible with slave aircraft after an initial advisory in a multi-threat scenario
  sXu value: *[1.5, 1.5]*

  C_slave_sub_no_response: cost received for issuing action incompatible with non-responsive slave aircraft
  sXu value: *[0.0, 0.0]*

C_slave_sub_noncrossing: cost received for issuing action incompatible with slave aircraft after an initial advisory with no crossing
sXu value: *[0.05, 0.05]*

H_noncrossing_thres: vertical separation distance above which the C_slave_-sub_noncrossing cost can be applied (ft)
sXu value: *[100.0, 100.0]*

T_noncrossing_thres: maximum time that VRC can be active and have C_-slave_sub_noncrossing cost applied (s)
sXu value: *[5.0, 5.0]*

T_reversal_thres: time allowed for a master aircraft to reverse an advisory of a slave if the slave issued the advisory first (s)
sXu value: *[2.0, 2.0]*

- **coord_delay**: CoordinationDelayCost parameters

  T_init: minimum number of time steps to wait for a VRC from an intruder equipped with TCAS (s)
  sXu value: *[4.0, 4.0, 4.0]*

- **coord_ra_deferral**: CoordinatedRADeferralCost parameters

  C_deferral: cost applied to resolution advisories to force deferral based on coordination information
  sXu value: *[0.15, 0.15]*

  H_rolloff_proximate: ramp up interval for H_threshold_proximate (ft)
  sXu value: *[0.0, 0.0]*

  H_threshold_proximate: threshold for loss of separation (ft)
  sXu value: *[500.0, 500.0]*

  T_proximate: time value defining 'proximate' for determining proximate loss of separation (s)
  sXu value: *[10.0, 10.0]*

- **critical_interval_protection**: CriticalIntervalProtectionCost parameters

  C_force_alert: cost applied to force a resolution advisory
  sXu value: *[0.5, 0.5]*

  D_range_ground_expanded: ground range for non-diverging aircraft at or above which cost is not applicable (ft)
  sXu value: *[3000.0, 3000.0]*

  D_range_ground_required: ground range at or above which cost is not applicable (ft)
  sXu value: *[1500.0, 1500.0]*

  H_rel_required: required vertical separation (ft)
  sXu value: *[800.0, 800.0]*

  R_int_vert_threshold: intruder aircraft vertical rate below which cost is not applicable (ft/s)
  sXu value: *[15.0, 15.0]*

  R_own_vert_threshold: own aircraft vertical rate below which cost is not applicable (ft/s)
  sXu value: *[12.5, 12.5]*

  R_speed_ground_force_diverge_threshold: ground speed above which the force diverge is not applicable (ft/s)

sXu value: *[300.0, 300.0]*

R_speed_ground_threshold: ground speed above which cost is not applicable (ft/s)
sXu value: *[150.0, 150.0]*

T_proximate: time value defining 'proximate' for determining proximate loss of separation (s)
sXu value: *[30.0, 30.0]*

T_proximate_climbdescend: time value defining 'proximate' for determining proximate loss of separation for corrective climb and descend advisories (s)
sXu value: *[20.0, 20.0]*

T_proximate_inc_climbdescend: time value defining 'proximate' for determining proximate loss of separation for strengthening climb and descend advisories (s)
sXu value: *[10.0, 10.0]*

- **crossing_no_alert**: CrossingNoAlertCost parameters

  C_coc: cost applied to force an advisory when ownship would cross altitudes with another equipped aircraft
  sXu value: *[5.0, 5.0]*

  T_threshold: time threshold for tau below which COC will be penalized when aircraft altitudes would cross (s)
  sXu value: *[30.0, 30.0]*

- **force_alert**: Online cost parameters applied to penalize a clear of conflict

  C_coc_threshold_lower: low altitude cost threshold which COC needs to attain before an advisory is terminated
  sXu value: *[1.75, 1.75, 1.75, 1.75]*

  C_coc_threshold_upper: cost threshold which COC needs to attain before an advisory is terminated
  sXu value: *[1.25, 1.25, 1.25, 1.25]*

  C_force_alert: cost applied to COC or corrective advisories when forcing an alert
  sXu value: *[1.0, 1.0, 1.0, 1.0]*

  H_threshold_lower: low altitude threshold for using C_coc_threshold_lower (ft)
  sXu value: *[-1.0, -1.0, -1.0, -1.0]*

  H_threshold_upper: altitude threshold for using C_coc_threshold_upper (ft)
  sXu value: *[2350.0, 2350.0, 2350.0, 2350.0]*

- **initialization**: InitializationCost parameters

  T_init: minimum number of time steps before a resolution advisory can be issued (s)
  sXu value: *3.0*

- **max_reversal**: MaxReversalCost parameters

  C_coc: cost for issuing COC after the maximum reversal limit has been reached
  sXu value: *[400.0, 400.0]*

  C_reversal: cost for issuing a reversal after the maximum limit threshold has been reached

sXu value: *[401.0, 401.0]*

N_limit: maximum number of reversals threshold
sXu value: *[100, 100]*

T_coc_threshold: minimum number of time steps after the maximum reversal limit has been reached to penalize COC (s)
sXu value: *[10.0, 10.0]*

- **point_obs_descend_cost**: PointObsDescendCost parameters

  C_descend: cost applied to discourage desecend resolution advisory against point obstacle
  sXu value: *0.9*

- **point_obs_early_coc_cost**: PointObsPreventEarlyCOCCost parameters

  C_point_obs_early_coc: cost for issuing a premature clear of conflict advisory
  sXu value: *0.125*

  H_point_obs_early_coc_altitude_threshold: altitude separation above which cost is not applied
  sXu value: *100.0*

  T_point_obs_early_coc_tau_threshold: tau threshold below which cost is not applied
  sXu value: *60.0*

  T_point_obs_early_coc_vertical_tau_threshold: vertical tau used for projection of relative altitude
  sXu value: *20.0*

- **prevent_early_coc**: PreventEarlyCOCCost parameters

  C_coc: cost for issuing a premature clear of conflict advisory
  sXu value: *[0.52, 0.1]*

  D_range_min: horizontal range below which clear of conflict is penalized unless rel_mid vertical separation has been achieved (ft)
  sXu value: *[800.0, 150.0]*

  H_rel_hi_threshold: relative vertical separation above which clear of conflict is not penalized so long as the aircraft are not converging vertically (ft)
  sXu value: *[2400.0, 360.0]*

  H_rel_lo_threshold: relative vertical separation above which clear of conflict is not penalized so long as the aircraft are diverging vertically, one horizontal divergence test is passed, and the minimum horizontal range has been achieved (ft)
  sXu value: *[1200.0, 180.0]*

  H_rel_mid_threshold: relative vertical separation above which clear of conflict is not penalized so long as the aircraft are not converging vertically and one horizontal divergence test is passed (ft)
  sXu value: *[1800.0, 270.0]*

  R_rel_vert_threshold: relative vertical rate above which a determination of converging/diverging can be made; below is considered neither (ft/s)
  sXu value: *[19.0, 19.0]*

  T_min_horizontal_divergence: time that divergence in range must persist to allow clear of conflict to be issued unless rel_lo vertical separation has been achieved (s)

sXu value: *[3, 3]*

T_threshold: time threshold for tau above which clear of conflict will not be penalized if the rel_lo vertical separation has been achieved (s)
sXu value: *[37.0, 37.0]*

- **prevent_early_weakening**: PreventEarlyWeakeningCost parameters

  C_weakening: cost for issuing a weakening advisory
  sXu value: *[0.25, 0.25]*

- **response_estimation**: Parameters for estimating whether an aircraft is responding to an advisory

  H_own_wait_int_threshold: altitude threshold above which T_wait_int is used for the intruder wait time (ft)
  sXu value: *[5000.0, 5000.0]*

  R_acceleration: acceleration assumed (ft/s^2)
  sXu value: *6.4348*

  R_buffer_int: vertical rate buffer for intruder aircraft (ft/s)
  sXu value: *[8.33333, 8.33333]*

  R_buffer_own: vertical rate buffer for own aircraft (ft/s)
  sXu value: *8.3333333333*

  R_threshold_acceleration_hi: magnitude of acceleration below which the aircraft is judged to be leveling off, and thus non-responding (ft/s^2)
  sXu value: *2.0*

  R_threshold_acceleration_lo: magnitude of acceleration below which the aircraft is judged to be non-responding (ft/s^2)
  sXu value: *1.75*

  T_wait_int: time to wait before the intruder's rate is stored (s)
  sXu value: *[9, 9]*

  T_wait_int_low_alt: time to wait before the intruder's rate is stored when own altitude is at or below H_own_wait_int_threshold (s)
  sXu value: *[8, 8]*

  T_wait_own: time to wait before the own aircraft's rate is stored (s)
  sXu value: *8*

- **restrict_coc_due_to_reversal**: RestrictCOCDueToReversal parameters

  C_restrict_coc: cost for issuing an inappropriate clear of conflict advisory when the intruder aircraft is master and is forcing a reversal
  sXu value: *[500.0, 500.0]*

- **safe_crossing_ra_deferral**: SafeCrossingRADeferralCost parameters

  C_crossing_deferral: cost applied to any alert when the safe crossing deferral heuristic determines that a deferral is required
  sXu value: *[10.0, 10.0]*

  H_cpa_separation_threshold: projected relative altitude threshold at closest point of approach between two aircraft above which deferral can be applied (ft)
  sXu value: *[350.0, 350.0]*

  H_min_threshold: relative altitude threshold between two aircraft above which deferral can be applied (ft)
  sXu value: *[300.0, 300.0]*

H_own_dz_cpa_vertical_threshold: increase in the relative altitude threshold at closest point of approach due to ownship vertical rate (ft)
sXu value: *[250.0, 250.0]*

H_own_dz_vertical_threshold: increase in the relative altitude threshold due to ownship vertical rate (ft)
sXu value: *[300.0, 300.0]*

R_int_vertical_threshold: intruder vertical rate above which the deferral can be applied (ft/s)
sXu value: *[30.0, 30.0]*

R_own_rolloff: ramp up interval for the R_own_threshold after which the full increased thresholds are used (ft/s)
sXu value: *[15.0, 15.0]*

R_own_threshold: ownship vertical rate above which the vertical thresholds begin to increase (ft/s)
sXu value: *[15.0, 15.0]*

- **sandwich_prevention**: SandwichPreventionCost parameters

  C_coc_threshold: cost of COC above which RAs are considered likely and the sandwich cost may be applied
  sXu value: *[0.05, 0.05, 0.05, 0.05]*

  C_sandwich_equip: cost applied to actions associated with equipped intruder to prevent own aircraft from being sandwiched in a multithreat situation
  sXu value: *[0.1939, 0.1939, 0.1939, 0.1939]*

  C_sandwich_unequip: cost applied to actions associated with unequipped intruder to prevent own aircraft from being sandwiched in a multithreat situation
  sXu value: *[0.6, 0.6, 0.6, 0.6]*

- **time_based_non_compliance**: TimeBasedNonComplianceCost parameters

  C_coc: cost applied to clear of conflict to ensure that force a resolution advisory transition does not lead to premature clear of conflict
  sXu value: *[10.0, 10.0]*

  C_current_ra: cost applied to current action force a resolution advisory transition
  sXu value: *[0.25, 0.25]*

  C_ra_epsilon: value of current_ra_cost which is effectively equal to 0.0
  sXu value: *[1.0e-13, 1.0e-13]*

  H_rel_rolloff: ramp down interval for H_rel_threshold (ft)
  sXu value: *[125.0, 125.0]*

  H_rel_rolloff_cross: ramp down interval for H_rel_threshold_cross (ft)
  sXu value: *[250.0, 250.0]*

  H_rel_threshold: projected vertical separation at or below which the cost can be fully applied (ft)
  sXu value: *[75.0, 75.0]*

  H_rel_threshold_cross: projected vertical separation at or below which the cost can be fully applied if crossing is expected (ft)
  sXu value: *[150.0, 150.0]*

R_vert_rel_compare_min: minimum vertical rate difference that can be used when computing vertical rate compliance factor (ft/s)
sXu value: *[15.0, 15.0]*

T_rolloff_apply: ramp down interval for T_threshold_apply (s)
sXu value: *[4.0, 4.0]*

T_rolloff_tau_delta: ramp up interval for T_threshold_tau_delta (s)
sXu value: *[5.0, 5.0]*

T_threshold_apply: tau value at or below which the cost can be fully applied (s)
sXu value: *[7.0, 7.0]*

T_threshold_min_tau: tau threshold below which the minimum tau counter is incremented (s)
sXu value: *[7.5, 7.5]*

T_threshold_tau_delta: tau difference at which the cost begins to apply (s)
sXu value: *[5.0, 5.0]*

X_vert_rel_factor_max: maximum factor that can be applied based on non-compliance with target vertical rate
sXu value: *[3.0, 3.0]*

· **unequipped_mtlo**: AllowUnequippedMTLO parameters

H_threshold: altitude threshold for a multithreat level-off check (ft)
sXu value: *[125.0, 50.0, 50.0, 50.0]*

R_threshold: vertical rate threshold for a multithreat level-off check (ft/s)
sXu value: *[4.1667, 4.1667, 4.1667, 4.1667]*

· **vertical_rate_limit_dependent_strengthen**: VerticalRateLimitDependentStrengthenCost parameters

C_strengthen_inhibit: cost applied to strengthen actions when ownship's vertical rate limit do not support the vertical rate of increase climb or descend advisories
sXu value: *100.0*

– **protection_mode_code**: Protection mode for the associated collection of parameters in the modes array. There is only one protection mode supported as part of the standard implemention of sXu. Additional protection modes for Xo processing as deviations: Refer to ACAS sXu MOPS Vol. I section 1
sXu value: *3*

– **state_estimation**: Information about the parameters used in the state estimation subsystem

∗ **tau**: Tau estimation parameters

· **D_range_diverge_thres**: ground range below which the vertical entry table can be used (ft)
sXu value: *3000.0*

· **D_range_thres**: horizontal range below which an encounter is considered a slow closure encounter (ft)
sXu value: *4500.0*

· **H_alt_rel_thres**: vertical separation above which an encounter is considered a slow closure encounter (ft)
sXu value: *1000.0*

- **R_converge_thres_vert**: relative vertical rate below which the encounter is not converging (ft/s)
  sXu value: *4.0*
- **R_speed_thres**: relative horizontal speed below which an encounter is considered a slow closure encounter (ft/s)
  sXu value: *300.0*
- **T_thres_horiz**: tau threshold for determining if encounter is slow closure (s)
  sXu value: *9.0*
- **W_thres_horiz**: sum of horizontal weights for taus before T_thres_horiz
  sXu value: *0.5*
- **W_thres_vert**: sum of vertical weights for taus before T_thres_horiz
  sXu value: *0.75*
- **X_phi_converge_thres**: relative bearing below which the vertical entry table can be used (rad)
  sXu value: *6.2831853072*
- **entry_dist**: Entry distribution parameters

  T_tau_values: vector of time steps in the entry tables (s)
  sXu value: *[0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0, 17.0, 18.0, 19.0, 20.0, 21.0, 22.0, 23.0, 24.0, 25.0, 26.0, 27.0, 28.0, 29.0, 30.0, 31.0, 32.0, 33.0, 34.0, 35.0, 36.0, 37.0, 38.0, 39.0, 40.0, 41.0, 42.0, 43.0, 44.0, 45.0, 46.0, 47.0, 48.0, 49.0, 50.0, 51.0, 52.0, 53.0, 54.0, 55.0, 56.0, 57.0, 58.0, 59.0, 60.0]*

  horizontal_active_table: name of the horizontal entry table for active surveillance only
  sXu value: *entry_sxuvtrm_v4r0_compressed.dat*

  horizontal_table: name of the horizontal entry table
  sXu value: *entry_sxuvtrm_v4r0_compressed.dat*

  vertical_table: name of the vertical entry table
  sXu value: *entry_vert_sxuvtrm_v4r0_compressed.dat*
- **horiz_worst_case**: HorizontalWorstCase parameters

  D_near_mean_range_threshold_hi: range between the two aircraft above which the maximum worst case near mean factor can be applied (ft)
  sXu value: *30000.0*

  D_near_mean_range_threshold_lo: range between the two aircraft below which no factor related to worst case near mean can be applied (ft)
  sXu value: *6000.0*

  H_phi_spread_threshold: altitude threshold at or below which the worst case scale factor may be reduced due to angle spread of belief states (ft)
  sXu value: *12500.0*

  X_near_mean_angle_threshold_hi: angle between the mean belief state and the worst case above which no factor related to worst case near mean can be applied (rad)
  sXu value: *0.436332312998582*

  X_near_mean_angle_threshold_lo: angle between the mean belief state and the worst case below which the maximum worst case near mean factor can be applied (rad)

sXu value: *0.261799387799149*

X_near_mean_factor_max: maximum factor than can be applied to worst case weight due to the angle between the mean and the worst case
sXu value: *10.0*

X_phi_spread_factor_hi: largest scale factor applied to worst case weight due to angle spread of belief states
sXu value: *4.0*

X_phi_spread_factor_lo: smallest scale factor applied to worst case weight due to angle spread of belief states
sXu value: *0.5*

X_phi_spread_threshold_hi: relative bearing spread above which the scale factor will be X_factor_lo due to angle spread of belief states (rad)
sXu value: *1.5707963267948966*

X_phi_spread_threshold_lo: relative bearing spread below which the scale factor will not be reduced due to angle spread of belief states (rad)
sXu value: *0.7853981633974483*

· **threshold_factor**: threshold factor for determining whether to include a weight in the tau distribution
sXu value: *1.0e-6*

- **threat_resolution**: threat resolution parameters independent of protection mode
  - **C_differential_threshold**: differential by which one cost is considered better than another
    value: *1.0e-6*
  - **C_restrict**: very high cost used to prevent selection of an action; applied to COC to force an alert and to MTLO prevent an individual MTLO
    value: *500.0*
  - **H_aglalt_limit**: altitude at or below which AGL altitude value can be used for setting height (ft)
    value: *2350.0*
  - **action_arbitration**: ActionArbitration parameters
    * **C_differential_worst_case_threshold**: cost difference hysteresis used by ArbitrateConflictingSenses
      value: *2.0*
  - **balance_costs**: MultithreatCostBalancing parameters
    * **C_absolute_threshold_hi**: cost above which MTLO is not allowed, regardless of whether costs balance
      value: *[1.546, 1.546, 1.546, 1.546]*
    * **C_absolute_threshold_lo**: hysteresis on cost above which MTLO is not allowed, regardless of whether costs balance
      value: *[0.09, 0.09, 0.09, 0.09]*
    * **C_differential_threshold_hi**: difference in costs above which the costs for any two intruders are considered too unbalanced to allow an MTLO
      value: *2.0*
    * **C_differential_threshold_lo**: difference in costs below which the costs for any two intruders are no longer considered too unbalanced to allow an MTLO
      value: *0.15*

- **online_cost_classification**: classification associated with each intruder type, to be used for online costs
  value: *[1, 3, 100, 101]*
- **vertical_point_obstacle_awareness**: Vertical point obstacle parameters
  - **D_horiz_target_separation**: target horizontal separation (ft)
    value: *100.0*
  - **D_vert_target_separation**: target vertical separation (ft)
    value: *15.0*
  - **T_max_tau**: maximum tau value assigned to point obstacles (s)
    value: *60.0*
  - **T_min_tau**: minimum tau value assigned to point obstacles (s)
    value: *6.0*
- **vertical_scaling**: Vertical RA and band scaling parameters
  - **H_point_obstacle_offset_ft**: offset altitude for point obstacles (ft)
    value: *10*
  - **classification**: classification associated with each scale factor vector
    value: *[1, 3, 100, 101]*
  - **values**: a 5-column matrix of vertical scales; where the columns indicate the policy range scale, entry table range scale, DAA band range scale, DAA band horizontal dimension range scale, and DAA band lookahead time scale, respectively. Note the band range scale is affected by the policy range scale. Note the band scales are affected by the policy scales, but are not used in ACAS sXu.
    value: $\begin{pmatrix} 0.830974069443027 & 0.800963521913608 & 1.0 & 1.0 & 1.0 & 1.0 \\ 0.062237109339776 & 0.14 & 1.0 & 1.0 & 1.0 & 0.18 \\ 0.02 & 1.0 & 1.0 & 1.0 & 1.0 & 1.0 \\ 0.02 & 1.0 & 1.0 & 1.0 & 1.0 & 1.0 \end{pmatrix}$

- **trm_performance**: performance-based parameters for the TRM blended decision logic
  - **parameter_order**: vector of names for values in each 1-dimensional parameter vector
    value: *["C_direct_sensitivity_factor", "C_desensitivity"]*
  - **surv_params**: matrix of parameters to use when validated surveillance source is available

$$
\begin{array}{rc}
 & \begin{array}{cccc} max\ vert.\ rate & max\ turn\ rate & parameter[1] & parameter[2] \end{array} \\
value: & \left(\begin{array}{cccc}
33.33333333 & 0.46332312999 & 1.0 & -7.0 \\
33.33333333 & 0.34906585039 & 1.0 & -7.0 \\
33.33333333 & 0.2617993878 & 1.0 & -7.0 \\
33.33333333 & 0.17453292199 & 1.0 & -5.0 \\
33.33333333 & 0.0872664626 & 1.0 & -2.0 \\
33.33333333 & 0.05235987756 & 1.0 & -2.0 \\
33.33333333 & 0.03490658504 & 1.0 & -1.0 \\
33.33333333 & 0.01745329252 & 1.0 & 0.0 \\
33.33333333 & 0.00872664626 & 1.0 & 0.0 \\
16.66666667 & 0.46332312999 & 1.0 & -7.0 \\
16.66666667 & 0.34906585039 & 1.0 & -7.0 \\
16.66666667 & 0.2617993878 & 1.0 & -7.0 \\
16.66666667 & 0.17453292199 & 1.0 & -5.0 \\
16.66666667 & 0.0872664626 & 1.0 & -2.0 \\
16.66666667 & 0.05235987756 & 1.0 & -1.0 \\
16.66666667 & 0.03490658504 & 1.0 & 0.0 \\
16.66666667 & 0.01745329252 & 1.01 & 0.0 \\
16.66666667 & 0.00872664626 & 1.025 & 0.0 \\
8.333333333 & 0.46332312999 & 1.0 & -5.0 \\
8.333333333 & 0.34906585039 & 1.0 & -5.0 \\
8.333333333 & 0.2617993878 & 1.0 & -5.0 \\
8.333333333 & 0.17453292199 & 1.0 & -5.0 \\
8.333333333 & 0.0872664626 & 1.0 & -2.0 \\
8.333333333 & 0.05235987756 & 1.0 & 0.0 \\
8.333333333 & 0.03490658504 & 1.01 & 0.0 \\
8.333333333 & 0.01745329252 & 2.1 & 0.0 \\
8.333333333 & 0.00872664626 & 2.1 & 0.0 \\
4.166666667 & 0.46332312999 & 1.0 & -2.0 \\
4.166666667 & 0.34906585039 & 1.0 & -2.0 \\
4.166666667 & 0.2617993878 & 1.0 & -2.0 \\
4.166666667 & 0.17453292199 & 1.0 & -2.0 \\
4.166666667 & 0.0872664626 & 1.0 & 0.0 \\
4.166666667 & 0.05235987756 & 1.05 & 0.0 \\
4.166666667 & 0.03490658504 & 1.5 & 0.0 \\
4.166666667 & 0.01745329252 & 2.1 & 0.0 \\
4.166666667 & 0.00872664626 & 3.0 & 0.0 \\
0.0 & 0.46332312999 & 1.0 & -1.0 \\
0.0 & 0.34906585039 & 1.0 & -1.0 \\
0.0 & 0.2617993878 & 1.0 & -1.0 \\
0.0 & 0.17453292199 & 1.0 & 0.0 \\
0.0 & 0.0872664626 & 1.0 & 0.0 \\
0.0 & 0.05235987756 & 1.5 & 0.0 \\
0.0 & 0.03490658504 & 1.5 & 0.0 \\
0.0 & 0.01745329252 & 2.1 & 0.0 \\
0.0 & 0.00872664626 & 3.0 & 0.0
\end{array}\right)
\end{array}
$$

- **turn_actions**: Information about the horizontal advisories represented in the horizontal cost table. When a subfield description does not provide a value, the value is provided in table(s) following the last subfield.
  - **names**: vector of the possible horizontal advisories. The first advisory is always COC. Values are listed in table(s) below.
  - **num_actions**: total number of horizontal advisory actions
    value: *4*
  - **turn_rates**: vector of turn rates that are compliant with each horizontal advisory (deg/s). Values are listed in table(s) below.

Table 55. turn_actions

no.	names	turn_rates
1	COC	NaN
2	RIGHT_CA	3
3	LEFT_CA	-3
4	STRAIGHT	0

All references made in the text to data contained in this file use dot notation to access the data. The names of the fields are preceded by params() and followed by an appropriate 1-based index, if needed. For example,

params().modes[i].protection_mode_code

refers to the $protection_mode_code$ for the ith $modes$ data structure in the file.

Similarly,

$params().modes[mode_int].cost_estimation.online.crossing_no_alert.C_coc$

refers to the value of the CROSSINGNOALERTCOST online cost applied to the clear of conflict, or no advisory, action. The $mode_int$ index value is derived from the value of the $protection_mode$ field in the TRMINTRUDERINPUT data structure.

F Data Table Format Specification

This appendix describes the formats used for storing data in the tables used by the Threat Resolution Module (TRM). Adjustments will be made to the formats in future versions of this document.

F.1 Block Dictionary Table Format

The entry distribution tables, horizontal offline cost tables, and vertical offline cost table use a file format designed for storing block dictionaries. Given a particular state, these block dictionaries allow efficient lookup of the entry distributions, and the offline costs for vertical actions. This appendix provides the format specification.

The term "entry distribution tables" refers to the horizontal entry distribution table and vertical entry distribution table. There is a single entry distribution table per file.

The term "offline cost tables" refers to the action and value class tables.

Each individual "offline cost table" follows the format specification provided below.

There are six basic types used in the binary representation of the block dictionary:

- **uint8**: unsigned 8 bit integer.
- **int16**: signed 16 bit integer.
- **uint32**: unsigned 32 bit integer.
- **half**: IEEE 754-2008 binary16 format half-precision floating point number.
- **double**: IEEE 754-2008 binary64 format double-precision floating point number.
- **varchar**: an array of (8 bit) bytes, beginning with a uint8 that specifies the number of single-byte ASCII characters that follow.

The table contains the following fields:

- magic_number (uint32). This field serves as validation and contains the number 1634 for files processed as described in this document.
- file_type (varchar). Depending on whether the block size is fixed or variable, this field should contain the string `fixedblockdictionary` or `varblockdictionary`, respectively. The entry distribution tables and offline cost tables use fixed block dictionaries.
- auxiliary_data_size (uint32). This field specifies the size of the auxiliary data in bytes. The auxiliary data contains information about the grid edges, as described later.
- auxiliary_data. This field contains the auxiliary data, described later.
- index_type (varchar). Data type for index field. The entry distribution tables and offline cost tables are `uint32`.
- data_type (varchar). Data type for contents of the data field. The entry distribution tables and offline cost tables are `half`.
- count_included (uint8). Indicates whether the count is included. The value in this field is undefined for fixed block dictionaries.

- maximum_block_elements (uint8). Specifies the maximum number of block elements. For fixed block dictionaries, this is the number of elements in each block.
- index_element_count (uint32). This field contains the number of index elements.
- data_element_count (uint32). This field contains the number of data elements.
- index. This field is omitted if index_element_count is 0. Otherwise, it contains the index. The index provides data element (not necessarily byte) offsets, which are 0-based, into the data segment. The entry distribution tables have index elements, the offline cost tables do not.
- data. This field contains the data. If the count is included, then the first element of each block contains the count of the number of elements in the block.

The auxiliary data specifies the grid names and has the following fields:

- dimension_count (uint32). This specifies the number of dimensions. The vertical equivalence class tables described in this document have four dimensions (tau, relative vertical separation, own vertical rate, and intruder vertical rate). The vertical MinBlocks table described in this document has four dimensions (split, relative vertical separation, own vertical rate, and intruder vertical rate). The horizontal equivalence class tables described in this document have six dimensions (range, theta, psi, own horizontal speed, intruder horizontal speed, and vertical tau). The horizontal MinBlocks table described in this document has six dimensions (split, theta, psi, own horizontal speed, and intruder horizontal speed). The vertical entry distribution table described in this document has three dimensions (relative vertical separation, own vertical rate, and intruder vertical rate). The horizontal entry distribution tables described in this document have three dimensions (ground range, ground speed, and relative bearing).
- cut_counts (uint32 array). This field contains the number of cuts for each dimension.
- names (varchar array). This field contains the names for each dimension and can be used for validation purposes.
- cuts (double array). This field contains all the cuts for all the dimensions. The cuts for each dimension are specified in increasing order.

The entry distribution tables and MinBlocks table are loaded into RDATATABLE (p.521) or RDATATABLESCALED (p.522) data structures. The equivalence class tables are loaded into an array of RDATATABLESCALED data structures, one for each table. The RDATATABLESCALED data structure enables scaling of the offline policy to make it more or less sensitive. Additional detail about offline policy scaling is found in Appendix Section G.1 and Section G.2. If the returned structure is s, the cut counts can be accessed using dot notation, which would be *s.cut_counts*.

F.2 Horizontal Policy Table Format

The horizontal TRM uses two data tables, one for the horizontal cost table (which uses the Block Dictionary Table Format) and one for the horizontal coordination table. This appendix provides the format specification for the horizontal coordination table.

The horizontal coordination table is structured as follows:

- table_data (int8). Raw coordination table data stored as int8. The data values are boolean, whether or not to match ownship turn sense.

The table_data is ordered by: range, theta, psi, ownship speed, intruder speed, and ownship turn sense.

For the horizontal policy tables, that information is stored in the parameter file.

The horizontal coordination table parameters are in the *hcoord_cuts* parameter structure which contains the following information.

- order (varchar array). This field contains the names for each dimension and the order they have in the file.
- range (double array). This field contains all the cuts for the range dimension, in increasing order.
- theta (double array). This field contains all the cuts for the theta dimension, in increasing order.
- psi (double array). This field contains all the cuts for the psi dimension, in increasing order.
- ownspeed (double array). This field contains all the cuts for the ownship speed dimension, in increasing order.
- intrspeed (double array). This field contains all the cuts for the intruder speed dimension, in increasing order.

The number of cuts for each of dimension listed in the parameter file is obtained from the length of the array.

If the returned structure is s, the cut counts can be accessed using dot notation, which would be *s.cut_counts*.

G Offline-Table Scaling and Online Cost Index Selection

This appendix describes how table scaling is implemented by the vertical and horizontal TRM.

Offline-table scaling provides a way of conceptually stretching or shrinking aspects of an encounter geometry with respect to a particular intruder to enable increased or decreased levels of alert sensitivity with respect to that intruder.

GETINDEXFROMCLASSIFICATION (Algorithm 457) adjusts the intruder *int_scale_factor_idx* to use the appropriate scaled protection volume, and adjusts the intruder *int_online_cost_idx* to use the appropriate online cost parameter index.

GETINDEXFROMCLASSIFICATION (Algorithm 457) searches for the classification index of the intruder (*int_classification*) in the array of classifications (*classification_array*). The variable *idx_class* represents the index (in the *classification_array* vector) of the intruder, while *int_classification* is a constant that represents the classification of the intruder. If the desired classification is not found, the algorithm checks to see if the default *CLASSIFICATION_MANNED* index was found. In the case that the manned classification was found, the default is set to this index. In the case that the manned classification was not found, the default is undetermined and a warning is thrown. The algorithm outputs *idx_class*.

Algorithm 457 GetIndexFromClassification

```
function GetIndexFromClassification( int_classification::UInt8, classification_array::Array{Z} )
    idx_class::Z = 0
    int_class_manned_idx::Z = 0
    counter::Z = 0
    found_idx::Bool = false
    # Search for the desired intruder classification index
    for class::UInt8 in classification_array
        counter += 1
        if int_classification==class
            found_idx = true
            idx_class = counter
        elseif class == CLASSIFICATION_MANNED
            int_class_manned_idx = counter
        end
    end
    if !found_idx
        if (int_class_manned_idx != 0)
            idx_class = int_class_manned_idx
        end
    end
    return idx_class::Z
end
```

Referenced In: VerticalTRMUpdatePrep(p. 587), HorizontalTRMUpdatePrep(p. 589)

G.1 Horizontal Offline-Table Scaling

CREATEHORIZONTALSCALEDCUTSSETS (Algorithm 458) returns vectors of cut points for each scale factor set defined in the parameter file. This algorithm is called in the constructor of the *RDataTableScaled*, which is the data structure used by the TRM to store entry distribution tables used for cost estimation for RA. The algorithm uses the constants *SCALING_HTRM_POLICY_RANGE*, *SCALING_HTRM_POLICY_SPEED*, and *SCALING_HTRM_POLICY_VERTICAL_TAU* to access the

correct entry of the scale factor set. Sets of scaled cut points are returned to assess the cost of the offline policy.

This algorithm takes as input *cut_counts*, *cuts*, and *scale_factors*. This algorithm returns *scaled_policy_cuts_set*.

Algorithm 458 CreateHorizontalScaledCutsSets

```
function CreateHorizontalScaledCutsSets( cut_counts::Vector{Z}, cuts::Vector{R}, scale_factors::Matrix{R} )
    N_horizontal_scaling_options::Z = size(params().horizontal_trm.horizontal_offline.horizontal_-
        scaling.values, 1)
    scaled_policy_cuts_set::Vector{Vector{R}} = Vector{Vector{R}}( undef, N_horizontal_scaling_options )
    for i::Z in 1:N_horizontal_scaling_options
        scaled_policy_cuts_set[i] =
            ScaleHorizontalCuts(p. 577)( cut_counts, cuts, scale_factors[i,SCALING_HTRM_POLICY_RANGE],
                scale_factors[i,SCALING_HTRM_POLICY_SPEED], scale_factors[i,SCALING_HTRM_POLICY_VERTICAL_TAU
                ])
    end
    return scaled_policy_cuts_set::Vector{Vector{R}}
end
```

Referenced In: RDataTableScaled(p. 522)

SCALEHORIZONTALCUTS (Algorithm 459) returns a vector of scaled cut points (*scaled_cuts*) given the original *cut_counts* and *cuts* and the range, speed, and vertical tau scale factors. The algorithm extracts the indices of range (*range_indices*) and scales the corresponding cutpoints by the supplied *range_scale*. The algorithm extracts the indices of ownship and intruder speeds (*speed_indices*) and applies the supplied *speed_scale*. The algorithm extracts the indices of vertical tau (*vertical_tau_indices*) and applies the supplied *vertical_tau_scale*.

This algorithm takes as input *cut_counts*, *cuts*, *range_scale*, *speed_scale*, *vertical_tau_scale*. This algorithm returns *scaled_cuts*.

Algorithm 459 ScaleHorizontalCuts

```
function ScaleHorizontalCuts( cut_counts::Vector{Z}, cuts::Vector{R}, range_scale::R, speed_scale::R,
    vertical_tau_scale::R)
    range_indices::Vector{Z} = GetCutpointIndexVector(p. 185)( POLICY_RANGE, cut_counts )
    speed_indices::Vector{Z} = vcat( GetCutpointIndexVector(p. 185)( POLICY_OWN_SPEED, cut_counts ),
                                     GetCutpointIndexVector(p. 185)( POLICY_INTR_SPEED, cut_counts ) )
    vertical_tau_indices::Vector{Z} = GetCutpointIndexVector(p. 185)( POLICY_VERTICAL_TAU, cut_counts )
    scaled_cuts::Vector{R} = deepcopy(cuts)
    for j::Z in range_indices
        scaled_cuts[j] = scaled_cuts[j]*range_scale
    end
    for j::Z in speed_indices
        scaled_cuts[j] = scaled_cuts[j]*speed_scale
    end
    for j::Z in vertical_tau_indices
        scaled_cuts[j] = scaled_cuts[j]*vertical_tau_scale
    end
    return scaled_cuts::Vector{R}
end
```

Referenced In: CreateHorizontalScaledCutsSets(p. 577)

G.2 Vertical Offline-Table Scaling

CREATEVERTICALSCALEDCUTSSETS (Algorithm 460) returns vectors of cut points for each scale factor set defined in the parameter file. This algorithm is called in the constructor of the *RDataTableScaled* and *Z32DataTable*, which are the data structures used by the TRM to store entry distribution tables used for cost estimation for RA calculation. *vertical_scaling_option_count* is the number of pre-defined vertical scaling configurations, the values for which are contained in the scales matrix.

This algorithm takes as input *cut_names*, *cut_counts*, *cuts*, and *scale_factors*. This algorithm returns *scaled_policy_cuts_set*.

Algorithm 460 CreateVerticalScaledCutsSets

```
function CreateVerticalScaledCutsSets( cut_names::Vector{String}, cut_counts::Vector{Z}, cuts::Vector{R},
    scale_factors::Matrix{R} )
    vertical_scaling_option_count::Z = size(params().threat_resolution.vertical_scaling.values, 1)
    scaled_policy_cuts_set::Vector{Vector{R}} = Vector{Vector{R}}( undef, vertical_scaling_option_count
        )
    for i::Z in 1:vertical_scaling_option_count
        scaled_policy_cuts_set[i] = ScaleVerticalCuts(p. 579)( cut_names, cut_counts, cuts, scale_factors[i,:]
            )
    end
    return ( scaled_policy_cuts_set::Vector{Vector{R}} )
end
```

Referenced In: RDataTableScaled(p. 522)

SCALEVERTICALCUTS (Algorithm 461) returns a vector of cut points scaled by a specified *scale_factor*. The algorithm uses the constant *SCALING_VTRM_POLICY_FT* to access the correct entry of the scale factor set. If *cut_names* contains relative altitude (i.e. "relh"), it finds the indices of *relh*, *dh0*, and *dh1* to scale those cut points by *scale_factor*. If *cut_names* does not contain "relh", the algorithm finds the indices of *rho* with the constant *SCALING_VTRM_ENTRY_FT* to scale those cut points by *rho_scale_factor* and finds the indices of *drho* with the constant *SCALING_VTRM_ENTRY_FPS* to scale those cut points by *drho_scale_factor*.

This algorithm takes as input *cut_names*, *cut_counts*, *cuts*, and *scale_factor*. This algorithm returns *scaled_cuts*.

Algorithm 461 ScaleVerticalCuts

```
function ScaleVerticalCuts( cut_names::Vector{String} ,cut_counts::Vector{Z}, cuts::Vector{R}, scale_factor::
    Array{R} )
    scaled_cuts::Vector{R} = deepcopy(cuts)
    idx_relh::Z  = something(findfirst(isequal("relh"), cut_names), 0)
    idx_dh0::Z   = something(findfirst(isequal("dh.0"), cut_names), 0)
    idx_dh1::Z   = something(findfirst(isequal("dh.1"), cut_names), 0)
    idx_rho::Z   = something(findfirst(isequal("rho"), cut_names), 0)
    idx_drho::Z  = something(findfirst(isequal("drho"), cut_names), 0)
    if ( 0 < idx_relh )
        scale_factor = scale_factor[SCALING_VTRM_POLICY_FT]
        relh_indices::Vector{Z}  = GetCutpointIndexVector(p. 185)( idx_relh, cut_counts )
        dh_indices::Vector{Z}  = vcat( GetCutpointIndexVector(p. 185)( idx_dh0, cut_counts ),
                                       GetCutpointIndexVector(p. 185)( idx_dh1, cut_counts ) )
        for j::Z in relh_indices
            scaled_cuts[j] = scaled_cuts[j]*scale_factor
        end
        for j::Z in dh_indices
            scaled_cuts[j] = scaled_cuts[j]*scale_factor
        end
    else
        rho_scale_factor = scale_factor[SCALING_VTRM_ENTRY_FT]
        drho_scale_factor = scale_factor[SCALING_VTRM_ENTRY_FPS]
        rho_indices::Vector{Z}  = GetCutpointIndexVector(p. 185)( idx_rho, cut_counts )
        drho_indices::Vector{Z} = GetCutpointIndexVector(p. 185)( idx_drho, cut_counts )
        for j::Z in rho_indices
            scaled_cuts[j] = scaled_cuts[j]*rho_scale_factor
        end
        for j::Z in drho_indices
            scaled_cuts[j] = scaled_cuts[j]*drho_scale_factor
        end
    end
    return scaled_cuts::Vector{R}
end
```

Referenced In: CreateVerticalScaledCutsSets(p. 578)

H Math Utilities

This appendix lists the utility functions used in the ACAS X algorithms. ANGLEDIFFERENCE (Algorithm 462) takes as input two angles (A and B). This algorithm returns the angle subtending the inputs.

Algorithm 462 AngleDifference

```
function AngleDifference(A::R, B::R)
    return mod(B-A + pi,2pi) - pi
end
```

Referenced In: UpdateTargetTrackAngle(p. 219), CalculateCrossingCostPsiFactor(p. 196), CalculateCrossingCostThetaFactor(p. 197), ReceiveHeadingObservation(p. 88), HeadingAtToa(p. 90), WrapAngleDiffToPi(p. 580)

WRAPANGLEDIFFTOPI (Algorithm 463) takes as input two angles (A and B). This algorithm finds the angle subtending the inputs through a call to ANGLEDIFFERENCE and maps an angle into the interval -π to +π through a call to WRAPTOPI. This algorithm returns *phi_out*.

The input and output angles are both in radians.

Algorithm 463 WrapAngleDiffToPi

```
function WrapAngleDiffToPi(A::R, B::R)
    phi_out::R = WrapToPi(p. 583)(AngleDifference(p. 580)(A, B))
    return phi_out::R
end
```

Referenced In: UpdateHeadingTracker(p. 89), CorrelatePosition(p. 471)

BLOCK_DIAG (Algorithm 464) combines two input matrices (m x n matrix A and p x q matrix B) diagonally into an output matrix of size $(m+p)$ x $(n+q)$. All other elements of the output matrix are zeros.

Algorithm 464 block_diag

```
function block_diag(A::Union{Vector,Matrix}, B::Union{Vector,Matrix})
    return [A zeros(size(A, 1), size(B, 2)); zeros(size(B, 1), size(A, 2)) B]
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59), UpdateAGTTrack(p. 76)

The MAHAL (Algorithm 465) algorithm computes the Mahalanobis distance. This calculation uses the covariance of one or two distributions to provide a normalized measure of distance. It is used extensively in association and correlation algorithms of the STM where distributions are related to one another.

This algorithm takes as input two sets of means and covariances, namely *muA*, *SigmaA*, *muB*, and *SigmaB*. This algorithm returns the Mahalanobis distance.

Algorithm 465 mahal

```
function mahal(muA::Union{Vector{R},R,Z}, SigmaA::Union{Matrix{R},R,Z}, muB::Union{Vector{R},R,Z}, SigmaB::
    Union{Matrix{R},R,Z})
    psd_stability_factor::R = params().surveillance.psd_stability_factor
    Sigma = copy(SigmaA)
    if (SigmaB != 0)
        Sigma = SigmaA .+ SigmaB
    end
    dmu = muA
    if (muB != 0)
        dmu = muA .- muB
    end
    if (ndims(Sigma) == 2)
        if any( diag(Sigma) .< psd_stability_factor )
            Sigma = Sigma + psd_stability_factor*eye(p. 582)(size(Sigma,1))
        end
        D = dmu' * inv(Sigma) * dmu
        D = D[1]
    else
        D = (dmu * dmu) / Sigma
    end
    return sqrt(D)::R
end
```

Referenced In: ReceiveOwnRelNonCoopTrack(p. 59), UpdateAGTTrack(p. 76), CorrelatePosition(p. 471), IsOutlier(p. 41)

The UCHOL (Algorithm 466) algorithm computes the Upper Cholesky Decomposition which is an efficient and stable method for calculating a matrix square root. It is used within SIGMA-POINTSAMPLE to draw discrete samples from a continuous distribution. Sometimes numerical errors will cause the diagonal terms of the input matrix to evaluate to very small negative values (defined by the *psd_stability_factor* threshold). This function includes a check that will make such values very small positive numbers, thus guaranteeing the input matrix is positive semidefinite.

This algorithm takes as input a matrix A and returns the Upper Cholesky Decomposition.

Algorithm 466 uchol

```
function uchol(A::Matrix{R})
    psd_stability_factor::R = params().surveillance.psd_stability_factor
    n = size(A,1)
    AA = copy(A)
    U = zeros(size(A))
    for j in n:-1:1
        if AA[j,j] < psd_stability_factor
            AA[j,j] = psd_stability_factor
        end
        U[j,j] = sqrt(AA[j,j])
        d = 1 / U[j,j]
        if (j > 1)
            for k in 1:j
                U[j,k] = d * AA[j,k]
            end
            for k in 1:j
                for i in 1:k
                    AA[k,i] -= U[j,k] * U[j,i]
                end
            end
        end
    end
    return Matrix{R}(U')
end
```

Referenced In: SigmaPointSample(p. 144)

The EYE (Algorithm 467) algorithm computes the N by N identity metrics.

This algorithm takes as input N. The algorithm returns *identity*.

Algorithm 467 eye

```
function eye(N::Number)
    identity::Matrix{R} = diagm(0=>ones(N))
    return identity::Matrix{R}
end
```

Referenced In: InitializePassiveTrackerPosition(p. 37), AddHorBias(p. 140), UpdateKalmanFilter(p. 49), ReceiveWgs84Observation(p. 92), SetDisplayDataPassive(p. 146), UpdatePassiveTrackerState(p. 50), mahal(p. 581)

NORMALIZE (Algorithm 468) generates a vector of values that sum to 1.0 given a vector of positive values. This algorithm is used to produce weights for belief structures.

This algorithm takes as input x. This algorithm returns *x_norm*.

Algorithm 468 Normalize

```
function Normalize( x::Vector{R} )
    x_norm::Vector{R} = x / sum( x )
    return (x_norm::Vector{R})
end
```

Referenced In: CombineVerticalBeliefs(p. 270), StateEstimation(p. 257)

WRAPTO2PI (Algorithm 469) maps an angle into the interval 0 to 2π. The input and output angles are both in radians.

This algorithm takes as input *phi*. This algorithm returns *phi_out*.

Algorithm 469 WrapTo2Pi

```
function WrapTo2Pi( phi::R )
    phi_out::R = WrapToPi(p. 583)( phi )
    if ( phi_out < 0.0)
        phi_out = phi_out + 2pi
    end
    phi_out = max( 0.0, phi_out )
    phi_out = min( 2pi, phi_out )
    return phi_out::R
end
```

Referenced In: HorizontalWorstCase(p. 259)

UNITVECTORNORMALIZE (Algorithm 470) returns a vector of norm one.

Algorithm 470 UnitVectorNormalize

```
function UnitVectorNormalize( x::Array{R} )
    return x / norm( x )
end
```

Referenced In: TurnEnuPositionVelocity(p. 435)

WRAPTOPI (Algorithm 471) maps an angle into the half-open interval $(-\pi, \pi]$. The input and output angles are both in radians.

This algorithm takes as input *phi*. This algorithm returns *phi_out*.

Algorithm 471 WrapToPi

```
function WrapToPi(phi::R)
    phi_out = phi
    while ( phi_out > pi )
        phi_out = phi_out - 2 * pi
    end
    while ( phi_out <= -pi )
        phi_out = phi_out + 2 * pi
    end
    return phi_out::R
end
```

Referenced In: UpdateTargetTrackAngle(p. 219), SelectHorizontalAdvisoryWindRelative(p. 211), WrapTo2Pi(p. 583), CalculatePolicyState(p. 187), EncodeTIDBearing(p. 390), GetPolicyTableCosts(p. 190), HorizontalAlertLookahead(p. 214), StateEstimation(p. 257), SelectHorizontalAdvisory(p. 210), ReceiveHeadingObservation(p. 88), HeadingAtToa(p. 90), SetDisplayDataORNCT(p. 153), SetDisplayDataPassive(p. 146), UpdateHeadingTracker(p. 89), PolarAlignmentHorizontalRotation(p. 25), WrapAngleDiffToPi(p. 580)

NANMIN (Algorithm 472) returns the minimum of two numbers that is not NaN.

This algorithm takes as input two numbers, x and y. This algorithm returns a single value.

Algorithm 472 nanmin

```
function nanmin(x::Number, y::Number)
    if isnan(x) && !isnan(y)
        return y
    elseif !isnan(x) && isnan(y)
        return x
    else
        return min(x, y)
    end
end
```

Referenced In: CalculatePolicyState(p. 187)

NANMAX (Algorithm 473) returns the maximum of two numbers that is not NaN.

This algorithm takes as input two numbers, x and y. This algorithm returns a single value.

Algorithm 473 nanmax

```
function nanmax(x::Number, y::Number)
    if isnan(x) && !isnan(y)
        return y
    elseif !isnan(x) && isnan(y)
        return x
    else
        return max(x, y)
    end
end
```

Referenced In: ConvertOwnInputsToEnuBeliefsGroundAndCompassMix(p. 177), UpdatePolicySpeedBinIndividual(p. 192), ConvertOwnInputsToEnuBeliefs(p. 176), LookupEntryDistribution(p. 266), TauEstimation(p. 263)

NANMINIMUM (Algorithm 474) returns the minimum value in a vector that is not NaN.

This algorithm takes as input a vector, a. This algorithm returns a single value.

Algorithm 474 nanminimum

```
function nanminimum(a::Vector{R})
    if (all(isnan,a))
        return NaN
    else
        return minimum(filter(!isnan, a))
    end
end
```

Referenced In: CreatePolicyStateBeliefs(p. 186)

NANMAXIMUM (Algorithm 475) returns the maximum value in a vector that is not NaN.

This algorithm takes as input a vector, a. This algorithm returns a single value.

Algorithm 475 nanmaximum

```
function nanmaximum(a::Vector{R})
    if (all(isnan,a))
        return NaN
    else
        return maximum(filter(!isnan, a))
    end
end
```

Referenced In: CreatePolicyStateBeliefs(p. 186), LookupEntryDistribution(p. 266)

I Data Management Utilities

The algorithms in this section are included for informational purposes only. They perform minimal formatting and preparation of the data for use in the ACAS X system.

The formatting and preparation required by any particular ACAS X implementation is dependent on the details of that system. The algorithms shown here perform a minimal amount of preparation, other than formatting, and assume high data quality. They also make use of Julia language capabilities for conciseness. To have the algorithms fit on a single page in this document, code that checks data quality and issues warning messages have been hidden. For example, code that checks data quality and issues warning messages has been hidden. Subsequent processing in the TRM assumes that the data inputs are appropriate.

The manner in which data quality requirements are handled for the ACAS X system are determined by the implementer. Many potential issues with data quality are overlooked in this document.

VERTICALTRMUPDATEPREP (Algorithm 476) is called from VERTICALTRMUPDATE. It is used to format and prepare the TRM inputs for Vertical TRM processing.

The algorithm first sorts the intruders, including the ground point obstacle, into vectors of valid and invalid intruders (*input_int_invalid* or *input_int_valid*) based on the STM's assigned values (set in SETOWNSHIPDATA and the Add Track To Report algorithms) for the ownship's operational mode, the ownship's altitude mode, and the intruder's valid flag. When *trm_altmode* is set to *OPMODE_STANDBY*, each intruder is pushed into *input_int_invalid*. This vector is later handled in DROPPEDINTRUDERSADJUSTMENT and all intruders are handled as dropped intruders. The status of all the intruders in the TRMINTRUDERSTATE, *st_trm*, is updated to indicate whether the intruder was dropped or is in use based on if the intruder was passed to the TRM by the STM this cycle in *input*. *st_int* is updated with any new intruders.

VERTICALTRMUPDATEPREP also allocates space per valid intruder for the function's outputted vectors of *z_int_ave*, *dz_int_ave*, *source_int*, *mode_int*, *code_int*, *sense_int*, *st_int*, and *output_int*. Then, an altitude inhibit cost state is created for each protection mode.

This algorithm takes as input *st_trm*, *input*, and *ground_int*. This algorithm returns *st_int*, *input_int_invalid*, *input_int_valid*, *num_intruders*, *z_int_ave*, *dz_int_ave*, *mode_int*, *code_int*, *sense_int*, *output_int*, and *source_int*.

Algorithm 476 VerticalTRMUpdatePrep

```
function VerticalTRMUpdatePrep( st_trm::TRMState(p.510), input::TRMInput(p.497), ground_int::
    TRMIntruderInput(p.499) )
    N_actions::Z    = params().actions.num_actions
    N_modes::Z      = length( params().modes )
    scale_factor_classifications::Array{Z} = params().threat_resolution.vertical_scaling.classification
    online_cost_classifications::Array{Z} = params().threat_resolution.online_cost_classification
    input_int_invalid::Vector{TRMIntruderInput(p.499)} = []
    input_int_valid::Vector{TRMIntruderInput(p.499)}   = []
    input_dict_int::Dict{Z,TRMIntruderInput(p.499)} = Dict{Z,TRMIntruderInput(p.499)}()
    for intruder::TRMIntruderInput(p.499) in input.intruder
        input_dict_int[intruder.id] = intruder
        if (OPMODE_RA != input.own.opmode) && (OPMODE_SURV_ONLY != input.own.opmode)
            push!( input_int_invalid, intruder )
        elseif (input.own.trm_altmode==TRM_ALTMODE_NONE)
            push!( input_int_invalid, intruder )
        elseif !intruder.valid
            push!( input_int_invalid, intruder )
        else
            push!( input_int_valid, intruder )
        end
    end
    input_dict_int[ground_int.id] = ground_int
    if (input.own.trm_altmode==TRM_ALTMODE_NONE)
        push!( input_int_invalid, ground_int )
    elseif !ground_int.valid
        push!( input_int_invalid, ground_int )
    else
        push!( input_int_valid, ground_int )
    end
    num_intruders::Z = length( input_int_valid )
    TRM_st_int::Dict{UInt32,TRMIntruderState(p.512)} = Dict{UInt32,TRMIntruderState(p.512)}()
    for st_int_iter::TRMIntruderState(p.512) in st_trm.st_intruder
        if !haskey( input_dict_int, st_int_iter.id )
            st_int_iter.status = :Dropped
        else
            TRM_st_int[st_int_iter.id] = st_int_iter
            st_int_iter.status = :InUse
        end
    end
    for intruder in input_int_invalid
        if !haskey( TRM_st_int, intruder.id )
            st_int_iter = TRMIntruderState(p.512)( intruder.id, intruder.id_directory )
            st_int_iter.idx_scale = GetIndexFromClassification(p.576)( intruder.classification,
                scale_factor_classifications )
            st_int_iter.idx_online_cost = GetIndexFromClassification(p.576)( intruder.classification,
                online_cost_classifications )
            push!( st_trm.st_intruder, st_int_iter )
            TRM_st_int[st_int_iter.id] = st_int_iter
        end
    end
    (z_int_ave::Vector{R}, dz_int_ave::Vector{R}) = (zeros(R, num_intruders), zeros(R, num_intruders))
    (source_int::Vector{Z}, mode_int::Vector{Z}) = (zeros(Z, num_intruders), zeros(Z, num_intruders))
    (code_int::Vector{UInt8}, sense_int::Vector{Symbol}) = (zeros(UInt8, num_intruders), fill(:None,
        num_intruders))
    st_int::Vector{TRMIntruderState(p.512)}    = Vector{TRMIntruderState(p.512)}( undef, num_intruders )
    output_int::Vector{TRMIntruderData(p.523)} = Vector{TRMIntruderData(p.523)}( undef, num_intruders )
    if (0 == length( st_trm.st_own.st_alt_inhibit ))
        st_trm.st_own.st_alt_inhibit = Vector{AltitudeInhibitCState(p.527)}( undef, N_modes )
        for mode_idx in 1:N_modes
            st_trm.st_own.st_alt_inhibit[mode_idx] = AltitudeInhibitCState(p.527)( params(), mode_idx )
        end
    end
    for j in 1:length( input_int_valid )
        intruder   = input_int_valid[j]
        id::UInt32 = intruder.id
        if haskey( TRM_st_int, id )
```

```
            st_int[j] = TRM_st_int[id]
        else
            st_int[j] = TRMIntruderState(p.512)( id, intruder.id_directory )
            st_int[j].idx_scale = GetIndexFromClassification(p.576)( intruder.classification,
                scale_factor_classifications )
            st_int[j].idx_online_cost = GetIndexFromClassification(p.576)( intruder.classification,
                online_cost_classifications )
            push!( st_trm.st_intruder, st_int[j] )
            TRM_st_int[id] = st_int[j]
        end
    end
    return (st_int::Vector{TRMIntruderState(p.512)}, input_int_invalid::Vector{TRMIntruderInput(p.499)},
            input_int_valid::Vector{TRMIntruderInput(p.499)}, num_intruders::Z, z_int_ave::Vector{R},
            dz_int_ave::Vector{R}, mode_int::Vector{Z}, code_int::Vector{UInt8},
            sense_int::Vector{Symbol}, output_int::Vector{TRMIntruderData(p.523)}, source_int::Vector{Z})
end
```

Referenced In: VerticalTRMUpdate(p.241)

HORIZONTALTRMUPDATEPREP (Algorithm 477) is called from HORIZONTALTRMUPDATE. It is used to format and prepare the TRM inputs for Horizontal TRM processing.

Intruders from *input* are sorted into the appropriate vectors (*input_int_invalid* or *input_int_valid*) based on the STM's assigned values (set in SETOWNSHIPDATA and the Add Track To Report algorithms) for the ownship's operational mode, the ownship's velocity mode, and the intruder's valid flag. When *trm_velmode* is set to *TRM_VELMODE_NONE*, each intruder is pushed into *input_int_invalid*. This vector is later handled in GENERATEHTRMOUTPUT in which horizontal alerting will be disabled for these intruders.

For each intruder that exists in *st_int*, the status of that intruder's state is updated to indicate whether the intruder was dropped or is in use this cycle. Next, *st_int* is updated with any new intruders. Finally for every intruder in the global state dictionary, the time since track was initialized, *t_init*, is increased by one second.

This algorithm takes as input *input* and *st_trm*. This algorithm returns *num_intruders*, *input_int_valid*, *input_int_invalid*, *st_int_valid*, *st_int_invalid*, and *st_int_dropped*.

Algorithm 477 HorizontalTRMUpdatePrep

```
function HorizontalTRMUpdatePrep( input::TRMInput(p. 497), st_trm::HTRMState(p. 513) )
    scale_factor_classifications::Array{Z} = params().threat_resolution.vertical_scaling.classification
    input_int_invalid::Vector{TRMIntruderInput(p. 499)} = []
    input_int_valid::Vector{TRMIntruderInput(p. 499)}   = []
    input_dict_int::Dict{UInt32,TRMIntruderInput(p. 499)} = Dict{UInt32,TRMIntruderInput(p. 499)}()
    for intruder::TRMIntruderInput(p. 499) in input.intruder
        input_dict_int[intruder.id] = intruder
        if (OPMODE_RA != input.own.h_opmode) && (OPMODE_SURV_ONLY != input.own.h_opmode)
            push!( input_int_invalid, intruder )
        elseif  (input.own.trm_velmode == TRM_VELMODE_NONE)
            push!( input_int_invalid, intruder )
        elseif !intruder.valid
            push!( input_int_invalid, intruder )
        else
            push!( input_int_valid, intruder )
        end
    end
    num_intruders::Z = length( input_int_valid )
    HTRM_st_int::Dict{UInt32,HTRMIntruderState(p. 514)} = Dict{UInt32,HTRMIntruderState(p. 514)}()
    st_int_dropped::Vector{HTRMIntruderState(p. 514)} = HTRMIntruderState(p. 514)[]
    for st_int_iter::HTRMIntruderState(p. 514) in st_trm.st_intruder
        if !haskey( input_dict_int, st_int_iter.id )
            st_int_iter.status = :Dropped
            push!( st_int_dropped, st_int_iter )
        else
            HTRM_st_int[st_int_iter.id] = st_int_iter
            st_int_iter.status = :InUse
        end
    end
    st_int_invalid = Vector{HTRMIntruderState(p. 514)}( undef, length( input_int_invalid ) )
    for j in 1:length( input_int_invalid )
        intruder = input_int_invalid[j]
        if !haskey( HTRM_st_int, intruder.id )
            st_int_iter = HTRMIntruderState(p. 514)( intruder.id, intruder.id_directory )
            st_int_iter.idx_scale = GetIndexFromClassification(p. 576)( intruder.classification,
                 scale_factor_classifications )
            st_int_iter.classification        = intruder.classification
            st_int_iter.source                = intruder.source
            push!( st_trm.st_intruder, st_int_iter )
            HTRM_st_int[st_int_iter.id] = st_int_iter
        end
        st_int_invalid[j] = HTRM_st_int[intruder.id]
    end
    st_int_valid = Vector{HTRMIntruderState(p. 514)}( undef, num_intruders )
    for j in 1:num_intruders
        intruder = input_int_valid[j]
        if !haskey( HTRM_st_int, intruder.id )
            st_int_iter = HTRMIntruderState(p. 514)( intruder.id, intruder.id_directory )
            st_int_iter.idx_scale = GetIndexFromClassification(p. 576)( intruder.classification,
                 scale_factor_classifications )
            st_int_iter.classification        = intruder.classification
            st_int_iter.source                = intruder.source
            push!( st_trm.st_intruder, st_int_iter )
            HTRM_st_int[intruder.id] = st_int_iter
        end
        st_int_valid[j] = HTRM_st_int[intruder.id]
        st_int_valid[j].idx_scale = GetIndexFromClassification(p. 576)( intruder.classification,
             scale_factor_classifications )
        st_int_valid[j].classification        = intruder.classification
        st_int_valid[j].source                = intruder.source
    end
    for id in keys( HTRM_st_int )
        HTRM_st_int[id].t_init = HTRM_st_int[id].t_init + 1.0
```

```
    end
    ResetHorizontalTRMOwnshipState(p. 590)(st_trm.st_own)
    return (num_intruders::Z, input_int_valid::Vector{TRMIntruderInput(p. 499)},
            input_int_invalid::Vector{TRMIntruderInput(p. 499)}, st_int_valid::Vector{HTRMIntruderState(p. 514)},
            st_int_invalid::Vector{HTRMIntruderState(p. 514)}, st_int_dropped::Vector{HTRMIntruderState(p. 514)})
end
```

Referenced In: HorizontalTRMUpdate(p. 168)

RESETHORIZONTALTRMOWNSHIPSTATE (Algorithm 478) resets all ownship speed bins for all scales if they are not initialized.

This algorithm takes as an input *st_own*. This algorithm updates *st_own*.

Algorithm 478 ResetHorizontalTRMOwnshipState

```
function ResetHorizontalTRMOwnshipState(st_own::HTRMOwnState(p. 513))
    N_horizontal_scaling_options::Z = size( params().horizontal_trm.horizontal_offline.horizontal_-
        scaling.values, 1 )
    if (0 == length( st_own.speed_bin_prev_policy ))
        st_own.speed_bin_prev_policy = fill(0, N_horizontal_scaling_options)
    end
end
```

Referenced In: HorizontalTRMUpdatePrep(p. 589)

J Julia Standard Library Functions

Several built-in functions in Julia are used in the ACAS sXu algorithms. Those functions are listed in the tables below. Definitions contain text taken verbatim from the Julia help function.

Julia data types are listed in Table 2. Julia keywords are listed in Table 3. The version of Julia used in this document is 1.0.3.

Note: To create a shallow copy of an array, a call to copy() is required due to Julia's intended behavior to assign by reference. To make a deep copy of an object, a call to deepcopy() is required.

By convention, "!" at the end of a built-in Julia function name indicates that function modifies its arguments. For example, the "delete!" function, modifies the collection passed to it as the first argument ("A") to delete the key passed to it as the second argument ("k"). If the Julia built-in function name has no "!" at the end of its name, its arguments are unmodified by the call.

Table 56. Julia Functions Used in the ACAS sXu Algorithms

Function	Interface	Description
abs	abs(x)	Returns the absolute value of the argument
all	all(A,dims)	Test whether all elements of a boolean collection are TRUE
any	any(A,dims)	Test whether any elements of a boolean collection are TRUE
append!	append!(A,x)	Add the elements of "x" to the end of a collection "A"
argmax	argmax(A)	Returns the index of the maximum element in collection "A"
argmin	argmin(A)	Returns the index of the minimum element in collection "A"
Array	ArrayT(undef, dims)	Construct an uninitialized dense array of size "dims" and of type "T"
atan	atan(y,x)	Compute the inverse tangent of "y/x", using the signs of both "x" and "y" to determine the quadrant of the return value
Bool	Bool(x)	Convert a number to boolean If the value is 0, bool returns FALSE. Otherwise, it returns TRUE
collect	collect(A)	Return an array of all items in collection "A"
convert	convert(T,x)	Try to convert "x" to the given type
copy	copy(x)	Create a shallow copy of "x": the outer structure is copied, but not all internal values
cos	cos(x)	Compute cosine of "x", where "x" is in radians
cross	cross(x,y)	Compute the cross product of two 3-vectors
deepcopy	deepcopy(x)	Create a deep copy of "x": everything is copied recursively, resulting in a fully independent object
deg2rad	deg2rad(x)	Convert "x" from degrees to radians
delete!	delete!(A,k)	Delete the mapping for the given key "k" in collection "A"
deleteat!	deleteat!(A,i)	Remove the item(s) at the given index (indices), and return the modified collection "A" Subsequent items are shifted to fill the resulting gap
diag	diag(A[, k])	Extract the "k"-th diagonal of a matrix as a vector
diagm	diagm(k => v)	Construct a matrix by placing vector "v" on the "k"-th diagonal
eigvals	eigvals(A)	Return the eigenvalues of "A"
error	error(v1,v2,...)	Output error message "v1v2..." and exit
exp	exp(x)	Compute the exponential of x (e^x)
falses	falses(dims)	Create a "BitArray" with the specified dimensions and all values set to FALSE
fill	fill(v,dims)	Create an array with the specified dimensions and all values set to "v" The array type is determined by the type of "v"
findfirst	findfirst(isequal(v),A)	Return the index of the first element equal to "v" in "A"
float	float(x)	Convert a number or array to a floating point data type
floor	floor(x)	Returns the nearest integral value of the same type as "x" not greater than "x"
get	get(A,k,default)	Return the value stored in collection "A" for the given key "k" Return the given default value if no mapping for the key is present
getindex	getindex(A, key...)	Retrieve the value(s) stored at the given key or index within collection "A"
haskey	haskey(A,key)	Determine whether collection "A" has a mapping for a given key
hypot	hypot(x,y)	Compute the $\sqrt{x^2+y^2}$ avoiding overflow and underflow
in	in(k,A)	Determine whether item "k" is in collection "A"
Int	Int(x)	Convert a number to the platform-dependent integer type
inv	inv(M)	Matrix inverse of "M"
is	is(x, y)	Returns "true" if "x" and "y" are identical
isapprox	isapprox(x, y, atol)	Returns "true" if abs(x-y) <= atol
@isdefined	@isdefined(var)	Test whether global variable "var" is defined
isdefined	isdefined(object, :field)	Test whether a field of an object is defined
isempty	isempty(A)	Determine whether collection "A" is empty (has no elements)
isinf	isinf(x)	Test whether a number is infinite (Inf)
isnan	isnan(x)	Test whether a floating point number is not a number (NaN), returns a Bool
keys	keys(A)	Return an iterator over all keys in collection "A"

Table 57. Julia Functions Used in the ACAS sXu Algorithms (cont.)

Function	Interface	Description
length	length(A)	For ordered, indexable collections, the maximum index "i" for which "getindex(A,i)" is valid For unordered collections, the number of elements in "A"
max	max(x,y,...)	Return the maximum of the arguments. Operates elementwise over arrays
maximum	maximum(A)	Returns the largest element in collection "A"
	maximum(A,dims)	Compute the maximum value of array "A" over the given dimensions
min	min(x,y,...)	Return the minimum of the arguments. Operates elementwise over arrays
minimum	minimum(A)	Returns the smallest element in collection "A"
	minimum(A,dims)	Compute the minimum value of array "A" over the given dimensions
mod	mod(x, m)	Modulus of "x" after division by "m", returning in the range [0,m)
ndims	ndims(A)	Returns the number of dimensions of "A"
norm	norm(A)	Compute the 2-norm of a vector "A"
oftype	oftype(x,y)	Convert "y" to the type of "x"
ones	ones(T,dims)	Create an array of all ones of specified type "T"
pop!	pop!(A)	Remove the last item in collection "A"
popfirst!	popfirst!(A)	Remove the first item in collection "A"
push!	push!(A,x)	Insert items at the end of collection "A"
pushfirst!	pushfirst!(A,x)	Insert item "x" at the beginning of collection "A"
rad2deg	rad2deg(x)	Convert "x" from radians to degrees
reshape	reshape(A,dims)	Create an array with the same data as array "A", but with dimensions given by "dims"
resize!	resize!(A,n)	Resize A to contain n elements. If n is smaller than the current length of A, the first n elements will be retained. If n is larger, the new elements are not guaranteed to be initialized.
reverse	reverse(v)	Return a copy of "v" reversed from 1 to length(v)
round	round(x)	Returns the nearest integral value of the same type as "x" to "x"
setindex!	setindex!(A,value,key...)	Store the given value at the given key or index within collection "A"
sign	sign(x)	Return "+1" if "x" is positive, "0" if "x == 0", and "-1" if "x" is negative
sin	sin(x)	Compute sine of "x", where "x" is in radians
size	size(A)	Returns a tuple containing the dimensions of "A"
size	size(A,n)	Returns the size of "A" in dimension "n"
something	something(x,y,...)	Returns the first value in the arguments which is not equal to nothing
sort	sort(v)	Returns a sorted copy of "v" leaving "v" itself unmodified
sort!	sort!(v)	Sort the vector "v" in place
sortperm	sortperm(v,lt=comparison)	Return a permutation vector of indices of "v" that puts it in sorted order The "lt=" keyword specifies a custom "less than" function "comparison" The resulting permutation will be the lexicographically first one that puts the input array into sorted order i.e. indices of equal elements appear in ascending order
splice!	splice!(A, range)	Remove items in the specified index range from collection "A" Subsequent items are shifted down to fill the resulting gap
sqrt	sqrt(x)	Returns the square root of "x"
string	string(xs...)	Create a string from any values
sum	sum(A)	Returns the sum of all elements in collection "A"
tan	tan(x)	Compute the tangent of "x", where "x" is in radians
typemin	typemin(T)	The lowest value representable by the given (real) numeric type "T"
typeof	typeof(x)	Get the concrete type of "x"
UInt8	UInt8(x)	Convert a number to "UInt8" (8-bit unsigned integer) data type
UInt16	UInt16(x)	Convert a number to "UInt16" (16-bit unsigned integer) data type
UInt32	UInt32(x)	Convert a number to "UInt32" (32-bit unsigned integer) data type
vcat	vcat(A...)	Concatenate the list of collections "A" along dimension 1
zeros	zeros(T,dims)	Create an array of type "T" with the specified dimensions and all values set to zero

The $params()$ notation used for accessing parameter values is not a function. It is short-hand for ($params_file :: paramsfile_type$), where $params_file$ is the variable storing the parameters and $paramsfile_type$ is its data type.

K Design Notes

K.1 Overview

This appendix provides guidance on several low-level design considerations which are out of scope of the high-level algorithmic specifications found elsewhere in this document. The recommendations found here are aimed at ACAS X software developers and will point to design concerns and choices which are best addressed early in the development process.

K.2 Interfacing with ACAS X

K.2.1 General

The ACAS X algorithms are designed to run in a continuous cycle consisting of a period of input processing followed by end-of-cycle report generation. In practice, it is recommended that the implementation cycle consist of two processes:

- A process dedicated to ongoing buffering of sensor readings and V2V capability data transmission device events (collecting input data)
- The ACAS X process, consisting of:
  - Pre-report processing of buffered input messages, i.e. the "Receive" STM entry points. *Note, immediate asynchronous processing (i.e. without buffering) of coordination messages with the* RECEIVEV2VCOORDINATION *entry point is recommended for best alerting performance, further discussed below.*
  - STM and TRM report generation on a fixed timestep, nominally on a 1 Hz schedule

There are several constraints placed on the relative ordering of entry point calls, described in Section 2.2.3.12 ("Interfaces With Other Systems") in ACAS sXu MOPS Volume I [3]. As a summary:

- Coordination messages of a tracked intruder have highest priority. ACAS X must process coordination messages before messages of any other type. If these messages are not being processed asynchronously, it is recommended to use a priority queue mechanism to prioritize these messages over others.
- All ownship message processing must be called in a timely order, i.e. with monotonically increasing Time of Applicability (ToA) values
- Similarly, for any given intruder, all input intruder message processing must be called in a timely order with monotonically increasing ToA values for those input message types that include a "Mode S ID" uniquely identifying the intruder.

The ACAS X TRM algorithms are based on a 1 Hz processing cycle. An appropriate time-overrun scheme should be considered to handle the case when ACAS X cycles are taking longer than 1 Hz, such as in cases of extremely high traffic.

K.2.2 Input Message Entry Points

The following section lays out the entry points to the ACAS X system and considerations in interacting with them. Each call handles a specific sensor reading or V2V capability data transmission

device message, allowing the STM to combine those individual data points into a coherent, per-aircraft estimation. Entry points are categorized below by the type of traffic they support and their expected rate of use, which can be useful in system design and while debugging test cases.

Mandatory Ownship State Messages:

- RECEIVEHEADINGOBSERVATION: Nominally 1 Hz
- RECEIVEHEIGHTAGLOBSERVATION: Nominally 1 Hz
- RECEIVEPRESALTOBSERVATION: Nominally 1 Hz
- RECEIVEDISCRETES: Nominally 1 Hz or asynchronously when values change

Entry points for ADS-B inputs:

- RECEIVESTATEVECTORPOSITIONREPORT and RECEIVESTATEVECTORVELOCITYREPORT: Expected message reception rate of 2 Hz per target per report type. For any given ADS-B target, reports are expected to have at least 10 ms of timing separation. If no such timing delay is present, input any position report prior to any velocity report with the same time of applicability.
- RECEIVEMODESTATUSREPORT: Expected message reception rate of 1 Hz per target
- RECEIVEWGS84OBSERVATION: Nominally 1Hz. A minimum of 1/3 Hz is required to enable ADS-B functionality. An ownship WGS84 message must be received before intruder ADS-B messages will be processed.

Entry points for coordination inputs:

- RECEIVEV2VCOORDINATION: Expected message reception rate of 1 Hz per target while coordinating. To be processed with the highest priority if a buffer-and-process methodology is chosen. The preferred alternate method is immediate asynchronous processing. This entry point is unique in that it may be asynchronously updated until the time of the UPDATEINTRUDERINPUTS call in STATEANDCOSTESTIMATION and still inform the final TRM report. This should be taken into account when designing the entry point processing architecture.
- RECEIVEADSBOPERATIONALSTATUSMESSAGE: Asynchronous. Future ADS-B operationla status message report type which describes an intruder's CAS equipage and coordination capabilities. Required to support coordination with passive CAS equipages. If no Capability Report is input, active CAS equipage defaults to TCAS.
- RECEIVEV2VOPERATIONALSTATUSMESSAGE: Asynchronous. Future V2V operational status message report type which describes an intruder's CAS equipage and coordination capabilities. Required to support coordination with CAS equipped intruders through V2V link. If no Capability Report is input, active CAS equipage defaults to unequipped.

Entry points for target designation inputs, a provision for potential modification (see ACAS sXu MOPS Vol I Appendix L and Vol II section 1):

- RECEIVETARGETDESIGNATION: Received asynchronously from the flight deck when the values change

K.2.3 Report Generation

Once per cycle, after the period of input message processing, the implementation generates STM and TRM Reports. This process is performed in the following sequence:

1. Call GENERATESTMREPORT to generate an output STMREPORT. The STMREPORT primarily summarizes the tracked state and display information for the ownship and all intruders
2. Input the TRMINPUT substructure of the STMREPORT, along with stored TRM state information, into a call to SXUTRMUPDATE. This function will return the SXUTRMREPORT structure which contains important advisory, coordination, and display information
3. Input the SXUTRMREPORT into a call to STMHOUSEKEEPING to update the STM state with new designation information.

K.2.3.1 Module Memory Management

The STM uses global data structures to maintain internal state memory between cycles. If implementing the ADD as specified, the implementation is expected to maintain instances of the following at a level globally visible to the STM:

- target_db::DATABASE{Uint32,TARGET}
- own::OWNSHIPDATA

The TRM differs from the STM in memory management. The TRM's state information is not stored globally but is instead kept in a data structure that is passed as an argument to SXUTRMUPDATE, provided as the *st_trm*::TRMSTATE parameter. This data structure is actively updated within the SXUTRMUPDATE function, and so a pass-by-reference for this parameter is expected if implementing the ADD as specified. The implementation is responsible for persisting the state of this data structure between cycles.

A data structure to hold static parameters, of type *paramsfile_type*, must be prepared and accessible to the implementation. This data structure is to be made globally visible to both the STM and TRM through a *params()* function handle. It is recommended to populate this data structure once at startup by parsing the provided parameters file or to compile the parameters directly into the system's binary.

The implementation will need a reset function for reinitializing the previously mentioned state variables to their initial state. The reset function should be called at system startup, system restart, and between test encounters when testing. The reset method should reinitialize the state objects as defined by their default constructors in their respective data structure definitions. The values in the parameter data structure are constant and do not need to be reset during a system restart or between test encounters.

K.3 Specific Implementation Details

K.3.1 Julia Pseudocode

All of the algorithms in this ADD are executable software written in the Julia programming language. These algorithms completely specify the operation of the ACAS X STM and TRM and were used to produce the ACAS X test suite supplement to the ACAS sXu MOPS.

This section highlights aspects of the Julia programming language requiring attention during implementation in other programming languages. Julia data types are listed in Table 2. Julia keywords are listed in Table 3. Standard Julia functions, including math functions, are described in Appendix J.

K.3.1.1 Julia Math and Array Notation

Julia has built-in math and array operators which are used in the algorithms throughout this document. Several of the math operators are used for manipulating vectors and matrices. In other programming languages, it will be necessary to implement some or all of these operations as functions. The following table describes Julia operators used in this ADD.

Table 58. Julia Operators

Operator	Description
A'	Matrix transpose of A
A .+ B	Elementwise addition of arrays A and B; B may be the same size as A in all dimensions or only one dimension
A .- B	Elementwise subtraction of matrices A and B; B may be the same size as A in all dimensions or only one dimension
A + B	Elementwise addition of arrays A and B; A and B must have the same dimensions
A - B	Elementwise subtraction of arrays A and B; A and B must have the same dimensions
A * B	Elementwise multiplication of arrays A and B; A and B must have the same dimensions
A / B	Elementwise division of arrays A and B; A and B must have the same dimensions
x ^ y	x^y
&	Bitwise AND
|	Bitwise OR
$	Exclusive OR
»	Right bit shift operator
«	Left bit shift operator
x += y	Equivalent to x = x + y
x -= y	Equivalent to x = x - y
x *= y	Equivalent to x = x * y
x /= y	Equivalent to x = x / y
[ ]	Construct an uninitialized array of type "Any"
T[ ]	Construct an uninitialized array of type T
[x y]	Construct a one-dimensional array (row vector) containing x and y
[x, y]	Construct a one-dimensional array (column vector) containing x and y
[A, B]	Construct a one-dimensional array (column vector) by concatenating the contents of column vector B to the contents of column vector A
[x1 x2 ; y1 y2]	Construct a two-dimensional array (matrix) with x1 and x2 in row 1 and y1 and y2 in row 2
A[1,:]	Returns all of the elements in row 1 of array A
A[:,1]	Returns all of the elements in column 1 of array A
A[1,2]	Returns the array element $A_{1,2}$
A[end]	Returns the last element in array A

K.3.1.2 Automatic Data Type Conversions

Algorithms in this ADD define the data types of their input arguments. However, the data types of local variables are not always defined. Julia assigns data types to variables at the time that a line of code is executed. The symbol "::" following a variable name is a data type assertion that enforces the variable having the indicated type at that point. If a variable is not forced to have a particular type, Julia will choose a type for the variable when a value is assigned to it. In that

case, the data type of the variable can change within an algorithm. When possible, Julia performs automatic type conversions when assigning a value of one type to a variable with a different type. An example of an automatic type conversion that is not possible is a conversion from NaN to an integer value.

Implementers should be aware of the potential for implicit data type conversions in the ADD and be careful to declare variable types appropriately.

K.3.1.3 Handling Symbols

This ADD makes use of a Julia language primitive called a "symbol", seen as an identifier preceded by a single colon. For example, see ":Up" in the VRCTOSENSE algorithm. This type is similar to an enumeration with the exception that the possible values do not need to be predefined. If the implementation language does not explicitly support symbols, it is recommended to instead use enumerations or constants.

K.3.2 Single versus Multi-Threaded Execution

The ACAS X algorithms were developed and tested as a single-threaded application, and the algorithms in this ADD do not describe any concurrency protection. However the ACAS X algorithms design is intended to support adaptation to run asynchronously and concurrently, especially for coordination processing. If pursuing this design, make sure to introduce protective code to prevent concurrent access to all globally shared state data and intruder coordination data.

K.3.3 32-Bit versus 64-Bit Applications

The ACAS X implementation will not require more than 16 gigabytes of RAM (excepting any exceptionally large memory allocations introduced by the implementer) and so both 32-bit and 64-bit applications are suitable. Note that testing and evaluation of the algorithms in this ADD used 64-bit integer and floating point values. See Appendix K.3.4 ("Floating Point Precision") below for information regarding 32-bit vs 64-bit data types.

K.3.4 Floating Point Precision

This ADD represents all real values with the "R" symbol and uses a 64-bit floating primitive to store these values. If 32-bit floats are preferred over 64-bit floats in an effort to improve processing efficiency or reduce memory usage, it is recommended to closely monitor the function-level results as some algorithms are particularly sensitive to precision loss.

The ACAS X algorithms have not been tested for robustness over the full range of possible floating point inputs. The implementer is responsible for assuring that all algorithms operate as intended for all such inputs. The following is recommended:

- Adjust algorithms to maximize numerical stability. Some algorithms may improperly introduce a loss of precision through repeated mathematical operations/transformations on the input data, especially with extremely small and large inputs. To best preserve the dynamic range of output, an implementation might reorder an algorithm's order of operations or use intermediate variables with higher bit precision than is specified in the ADD
- Introduce checks to handle otherwise unexpected exceptional floating point values, such as NaN and Infinity

Where floating point equivalence ("==") is tested in the ADD algorithms, the comparison should

not be changed. For example, in INTERPOLATEONEDIMENSION the comparison *(cuts[i] == point)* is intended to apply only when the two values are exactly equal.

K.3.5 Subsystem Modularity

This ADD maintains the STM and TRM as two separately encapsulated modules, with the following notable exceptions:

- The UPDATEINTRUDERVRC and UPDATEINTRUDERHRC algorithms allows the TRM to directly access the STM target database, creating a dependency between the two modules
- A single parameters structure is shared between the two modules

These inter-dependencies should be taken into account in early system design.

K.3.6 Running Julia as a Development Tool

The Julia code included in this document completely describes the behavior of the prescribed and suggested ACAS X algorithms. All necessary parameters, including data structure sizes, are either directly specified or can be derived from specified information through a static examination of the provided code.

In place of static analysis, use of the Julia interpreter to examine running instances of the ACAS X algorithms may be a significant aid in quickly determining the types and sizes of various data structures, especially those passed as function arguments or stored as internal state. This dynamic analysis accelerates and simplifies the task of developing low-level implementations of these algorithms in the language of choice.

For example, the Julia algorithms often utilize dynamically sized containers with no pre-defined dimensions. While a static analysis will reveal the ultimate size of these containers, running the algorithms in the Julia interpreter and inspecting the data states during execution (utilizing triggered breakpoints, targeted print statements, or other forms of inspection) provides an alternate and possibly more efficient means to establish these dimensions.

One specific example is the *Sigma_vert* field within the ADSBTRACKFILE. It is defined as matrix with real-valued elements with no additional information on its dimensions. Running the Julia algorithms with nominal input and printing out the state of *Sigma_vert* periodically demonstrates the matrix to be of 2x2 size. The ACAS sXu Test Suite provides a near-comprehensive exercise of all ACAS X branches and may be utilized in demonstrating array sizing with a high confidence of covering all dynamic structure states.

The following Julia commands are useful to inspect runtime behavior:

Table 59. Example Julia Commands

Command	Output	Description
println("Sigma_vert", trk.Sigma_vert)	Sigma_vert[73.0 0.0 0.0 125.0]	Print matrix
println("Size = ", size(trk.Sigma_vert))	Size = (2,2)	Print matrix dimensions
dump(trk)	ADSBTrackFile modes: Uint32 999 non_icao: Bool false rebroadcast: Bool false updates_pos: Int64 2 ...	Prints the contents of a structure (and any substructures) with field labels
@bp	*breakpoint at this line*	Provides an interactive terminal at this location (requires the external Julia "Debug" package)
@bp size(trk.Sigma_vert) != (2,2)	*breakpoint at this line if a condition is met*	Triggers a breakpoint at this location if ever the Sigma_vert matrix is not 2x2 at this line (requires external Julia "Debug" package)

K.3.7 Static Allocation

This ADD uses dynamically sized containers for the data storage and the passing of arguments. For a given implementation, dynamic memory allocation may be impossible or may be undesirable for a safety-critical embedded target, with tightly controlled memory management being preferred. In such cases, it is recommended to statically preallocate a container with a maximum storage size and to mark the number of populated indices with an accompanying counter variable.

The table below points out several prominent dynamically sized containers and suggests a recommended minimum capacity for a statically sized replacement. In practice, the system should scale these capacities to take effective advantage of all memory that is made available to the ACAS X process.

Note for all such statically sized replacements, protective code must be added to handle cases where storage demands exceed what is available, usually necessitating the system to discard data or to return from the function without further action. It is recommended to intelligently shed data as to minimally affect system performance in this regard. For example:

- Prefer to maintain and report intruders which are closer to ownship and especially those involved in active advisories, coordination, or target designation
- Prefer to persist track data elements which are closest or most recent

Table 60. (Minimum) Recommended Container Allocations

ADD Container	Suggested Size	Description	Sizing Rationale
target_db	120	Number of intruders independently tracked in the STM	An arbitrary limit chosen based on observed flight recordings
StmReport.display	30	Number of intruder records passed to the hardware display by the STM	An arbitrary limit chosen based on observed flight recordings
TRMInput.intruder	30	Number of intruder records passed to and processed by the TRM	An arbitrary limit chosen based on observed flight recordings
VTRMReport.coordination TRMDesignationData.intruder VTRMDisplayData.intruder	61	Number of intruder records included in the VTRMReport	A precise limit of twice the TRMInput.intruder array length, doubled to support the maximum number of dropped intruders independent of designation, plus 1 for the limit on the number of designated intruders
Target.adsb_qual_history	3	Size of ADS-B Quality History	A precise limit tied to the min_adsb_quality.n parameter

Table 61. (Minimum) Recommended Container Allocations (Continued)

ADD Container	Suggested Size	Description	Sizing Rationale
TRMOwnInput.belief_vert	1	Number of vertical belief states maintained for Ownship	An arbitrary limit chosen by convention
TRMIntruderInput.belief_vert	5	Number of vertical belief states maintained per intruder	An arbitrary limit chosen by convention
TRMIntruderInput.belief_-horiz	9	Number of horizontal belief states maintained per intruder	An arbitrary limit chosen by convention
OfflineCostEstimation *b_tau_ec_int* variable	29	Maximum number of tau estimation bins	A precise limit derived from table properties

K.3.8 Target IDs

The STM generates and assigns a numeric identifier to each Target. This Target identifier is propagated through the STM and TRM output data structures as the *id* field. The *id* can be used to associate the outputs from the STM and TRM with each other. It is recommended that implementers ensure identifiers are generated such that any given value of *id* is not reused until the new Target can reliably be disambiguated from any prior Target with the same id.

K.4 Error Checking

This ADD aims to provide an abstract, high-level representation of the code and leaves the issue of protective code and error handling to the implementer. Depending on the implementation language and architecture, unprotected operations may lead to immediate runtime errors, undefined belief states leading to later errors, or generally degraded performance.

It is advised that the implementers independently apply best coding practices and achieve any necessary DO-178C objectives [16] in this regard to ensure reliable and robust runtime behavior.

The lists provided below are included as examples and are not comprehensive lists; implementers are responsible for ensuring the robust and reliable operation of their implementation.

K.4.1 Divide-by-Zero Errors

The section points out areas of the ADD which are susceptible to divide-by-zero errors. To address these susceptibilities, protective checks may be added. A protective check may consist of replacing zero denominators with a small epsilon number, returning from the enclosing function with a nominal value, or avoiding a block of code which relies on that denominator.

Critical areas to review and to consider for protection include the following:

ADVANCEPASSIVETRACKPOSITION:

When implemented correctly, the denominator *dt* values enumerated below are prevented from approaching zero by a check against the *min_obs_toa_step* parameter value in the RECEIVESTATEVECTORPOSITIONREPORT calling algorithm. If an additional error check is desired in this context, the implementer may choose to bypass the velocity seeding and track prediction step.

- trk.mu_hor[2] = (x_rel-trk.mu_hor[1])/dt
- trk.mu_hor[4] = (y_rel-trk.mu_hor[3])/dt

PRESALTATTOA:

When implemented correctly, the denominator value below is prevented from approaching zero by a check against the *min_obs_toa_step* parameter value within the RECEIVEPRESALTOBSERVATION algorithm (which adds entries to the *own.history.pres_alt* structure). If an additional error check is desired in this context, the implementer may choose to bypass this altitude interpolation step, returning the value of a single altitude measurement retrieved from the *own.history.pres_alt* structure.

- rate::R = (b2 - b1) / (time2 - time1)

CONVERTECEFTOWGS84:

Only in geographically improbable locations can the denominators below approach zero. These cases are partly protected against by outlier detection within the RECEIVESTATEVECTORPOSITIONREPORT and RECEIVESTATEVECTORVELOCITYREPORT algorithms. If an additional error check is desired in this context, the implementer may choose to abandon the calling receive algorithm in the case of a zero-valued denominator.

- C = (e1_sq*e1_sq)*F*(r_ecef*r_ecef)/(G*G*G)
- P = F/(3*((S+(1/S)+1)^2)*(G*G))
- inner_square_root = ((a*a)/2)*(1+(1/Q)) - (P*(1-e1_sq)*(Za*Za))/(Q*(1+Q)) - P*(r_ecef*r_ecef)/2
- Z_0 = ((b*b)*Za)/(a*V)
- h = U*(1 - (b*b)/(a*V)) _ecef)/(G*G*G)

CONVERTORTHOMETRICTOGEODETICHEIGHT:

Only in geographically improbable locations can the denominators below approach zero. These cases are partly protected against by outlier detection within the RECEIVESTATEVECTORPOSITIONREPORT and RECEIVESTATEVECTORVELOCITYREPORT algorithms. If an additional error check is desired in this context, the implementer may choose to abandon the calling receive algorithm in the case of a zero-valued denominator.

- geo_h::R = R_phi * h_o / ((R_phi * gamma_s_phi / gamma_s_45) - h_o)

HEADINGATTOA:

Under most circumstances the denominator value below is prevented from approaching zero by a check against the *min_obs_toa_step* parameter value within the RECEIVEHEADINGOBSERVATION algorithm (which adds entries to the *own.history.heading* structure). If an additional error check is desired in this context, the implementer may choose to bypass the heading interpolation step, returning the value of a single altitude measurement retrieved from the *own.history.heading* structure when two exist with similar time values.

- rate::R = AngleDifference(h1, h2) / (time2 - time1)

LINEARTRANSFORM:

The linear transform between Cartesian and polar coordinates is undefined when range is zero. In this circumstance the implementation can assume an epsilon range value or apply other numerical techniques to complete the transformation.

- H = [mu_xy[1]/mu_ra[1] mu_xy[2]/mu_ra[1]; mu_xy[2]/mu_ra[1]2 -mu_xy[1]/mu_ra[1]2]

MAHAL:

Under correct usage the Mahalanobis distance utility function should not be called with two zero-valued variance or covariance parameters. In such a circumstance, the implementer may choose to return a normalized distance measure between the two mean inputs.

- D = (dmu*dmu)/SigmaA

NORMALIZE:

This algorithm is used only in the TRM and only for normalizing weights. If the STM is operating correctly, there will always be at least one weight value in the vector and the sum of the weight values will never equal 0. If an error check is desired in this context, the values in *x* should be modified only when there are at least two values in *x* AND the sum of the values in *x* is greater than 0. If the algorithm is to be used more generally, it is recommended the implementer assess the need for additional error checking in that context.

- x_norm::VectorR = x / sum(x)

K.4.2 Invalid Input

Certain functions have undefined behavior for a subset of possible inputs. It is recommended that the implementation include checks to prevent against invalid inputs to such functions.

The first such function is *atan2*. Julia returns 0.0 for *atan2(0,0)*. It is recommended implementers use a version of *atan2* that produces the same result.

The second such function is *sqrt*. Cases are pointed out below where the *sqrt* function may have a negative input. The following steps can be taken to protect these areas of the code:

- Swap in a custom version of the function which explicitly returns a nominal value for invalid input cases

- Check the inputs before the expression is called, returning from the function with a nominal value
- Check the inputs before the expression is called, skipping the block of code in question
- In the case of *sqrt*, take the absolute value of the inputs

Critical areas to review and to consider for protection include the following:

CONVERTECEFTOWGS84:

- S = (1 + C + sqrt(C*C+2*C))(1.0/3.0)
- Q = sqrt(1+2*(e1_sq*e1_sq)*P)
- V = sqrt((r_ecef-e1_sq*r_0)2 + (1-e1_sq)*(Za*Za))

MAHAL:

- return sqrt(D[1])

K.4.3 Matrix Inversion

The STM extensively uses matrices to represent state estimates and performs calculations requiring these matrices to be invertible (see UCHOL and MAHAL). In order to be invertible, matrices processed in the STM must be positive definite (PD). Floating point issues could potentially produce non-PD matrices due to numerical errors in the terms of the matrix which have very small values. STM-produced matrices should be PD (invertible), include protections (stability factors) in calculations where non-PD errors are possible, such as when tracks are initializing (matrices containing small initial values close to zero). The Julia Implementation of the ADD has been extensively simulated to ensure non-PD matrices have been mitigated.

Received matrices (ORNCT and AGT covariances) will be PD as required by ACAS sXu MOPS Vol. I [3] sections 2.2.3.5.10 and 2.2.3.5.17.

L Provisions for Potential Modifications

The algorithms and algorithm modifications included in this appendix are explicitly not elements of the ACAS sXu algorithms described in this document. The code provided below are examples of possible provision for potential modification to the ACAS sXu algorithms. The following code is to be treated as unvalidated and untested by the accompanying Test Suite. Any certification effort of an ACAS sXu implementation which includes these code examples or other modifications to the prescriptive algorithms must include additional testing and validation related to these modifications and cannot rely upon the Test Suite for this purpose.

L.1 Descent Inhibit Thresholds

RECEIVEDESCENTINHIBITTHRESHOLDS (Algorithm 479) is used in conjunction with the parameters in *modes.cost_estimation.online.altitude_inhibit* to inhibit down-sensed vertical RA selection. To inhibit descents, pass an upper and lower bound through this interface. These thresholds are used in UPDATEALTITUDEINHIBITCSTATE to determine if the ownship is in the inhibited region.

This algorithm takes as input *h_lo_ft* and *h_hi_ft*. This algorithm updates the corresponding *own* fields using this input.

Algorithm 479 ReceiveDescentInhibitThresholds

```
function ReceiveDescentInhibitThresholds( h_lo_ft::R, h_hi_ft::R )
    # Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1
    own.h_lo_ft[1] = h_lo_ft
    own.h_hi_ft[1] = h_hi_ft
end
```

L.2 Target Designation

ACAS X allows the flight crew to assign special processing to a target aircraft by designating that Target (e.g. through the display). Such a command enters ACAS X through the STM via RECEIVETARGETDESIGNATION (Algorithm 480). ACAS X target designation support is provided via the Aircraft Surveillance Applications (ASA) System described in DO-317B [6]. The Aircraft Surveillance and Separation Assurance Processing (ASSAP) function in the ASA System provides the interface to the ACAS X system.

This algorithm takes as input *mode_s*, *mode_s_non_icao*, *mode_s_valid*, *v2v_uid*, *v2v_uid_valid*, and *designation*. This algorithm produces no outputs.

Algorithm 480 ReceiveTargetDesignation

```
function ReceiveTargetDesignation( mode_s::UInt32, mode_s_non_icao::Bool, mode_s_valid::Bool, v2v_uid::UInt32,
     v2v_uid_valid::Bool, designation::UInt32 )
    # Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1 (insufficient for sXu)
end
```

STMHOUSEKEEPINGTARGETDESIGNATION (Algorithm 481) updates the Target DESIGNATIONSTATE using information in the SXUTRMREPORT input.

This algorithm is included as a stub for Xo functionality: Refer to the latest DO-385 for Xo target designation reference implementation (insufficient for Xu)

This algorithm takes as input *report*.

Algorithm 481 StmHousekeepingTargetDesignation

```
function StmHousekeepingTargetDesignation( report::sXuTRMReport(p.501) )
    # Deviation potential: Refer to ACAS sXu MOPS Vol. I section 1 (insufficient for sXu)
end
```

REFERENCES

[1] W. Bender, "Concept of Use for the Airborne Collision Avoidance System Xu for Small UAS (sXu)," Traffic Alert & Collision Avoidance System Program Office, Federal Aviation Administration, v0r2 (2018).

[2] M.J. Kochenderfer and J.P. Chryssanthacopoulos, "Robust Airborne Collision Avoidance through Dynamic Programming," Massachusetts Institute of Technology, Lincoln Laboratory, Project Report ATC-371 (2011).

[3] SC-147 Traffic Alert and Collision Avoidance System (TCAS), "Minimum Operational Performance Standards for Airborne Collision Avoidance System sXu (ACAS sXu) Volume I," RTCA, DO-?? (??).

[4] J. Bezanson, S. Karpinski, V.B. Shah, and A. Edelman, "Julia: A Fast Dynamic Language for Technical Computing," *CoRR* abs/1209.5145 (2012).

[5] SC-186 Automatic Dependent Surveillance-Broadcast (ADS-B), "Minimum Operational Performance Standards for 1090 MHz Extended Squitter Automatic Dependent Surveillance - Broadcast (ADS-B) and Traffic Information Services - Broadcast (TIS-B)," RTCA, DO-260C (2020).

[6] SC-186 Automatic Dependent Surveillance-Broadcast (ADS-B), "Minimum Operational Performance Standards (MOPS) for Aircraft Surveillance Applications (ASA) Systems," RTCA, DO-317B (2014).

[7] International Civil Aviation Organization, "Annex 10 to the Convention on International Civil Aviation, Volume IV," ICAO (2007).

[8] H. Hutchinson, "Report on work package 3: Update of altimetry-error assumptions," Eurocontrol ACAS Safety Analysis post-RVSM (ASARP) Project, techreport (2006).

[9] SC-186 Automatic Dependent Surveillance-Broadcast (ADS-B), "Minimum Aviation System Performance Standards (MASPS) for Automatic Dependent Surveillance-Broadcast (ADS-B)," RTCA, DO-242A (2002).

[10] D.M. Asmar, M.J. Kochenderfer, and J.P. Chryssanthacopoulos, "Vertical State Estimation for Aircraft Collision Avoidance with Quantized Measurements," *Journal of Guidance, Control, and Dynamics* 36(6), 1797–1802 (2013).

[11] Z. Duan, V.P. Jilkov, and X.R. Li, "State estimation with quantized measurements: Approximate MMSE approach," in *11th International Conference on Information Fusion* (2008).

[12] E. Lorenzo, "ACAS X STM Ownship Tracker Alternatives Analysis, Version 2 Revision 1," Johns Hopkins University Applied Physics Laboratory (2013).

[13] A-4 Air Data Subcommittee, "Air Data Computer – Minimum Performance Standard," SAE International, AS-8002B (2020).

[14] J. Brush and J. Lopez, "ACAS X: Cost File Compression," Traffic Alert & Collision Avoidance System Program Office, Federal Aviation Administration, AOS-14-0226 (2014).

[15] D. Crockford, "The application/json Media Type for JavaScript Object Notation (JSON)," IETF, Internet informational RFC 4627 (2006).

[16] SC-205 Software Assurance', "Software Considerations in Airborne Systems and Equipment Certification," RTCA, DO-178C (2011).